ویژه‌ی آمادگی کوییز

کتاب جامع درسنامه و تست برنامه‌نویسی وب

JavaScript، کار با DOM در صفحه‌ی وب، Node.js و TypeScript — از صفر تا تسلط، با درسنامه‌ی ساده، تست‌های چهارگزینه‌ای سطح‌بندی‌شده و پاسخنامه‌ی تشریحی

JavaScript · DOM · Node.js · npm · TypeScript

```javascript
console.log("5" + 1);       // ?
console.log("5" - 1);       // ?
console.log([] == false);   // ?
setTimeout(() => console.log("timeout"), 0);
Promise.resolve().then(() => console.log("promise"));
```

۳	۵۵	۱۲۹۵	۱۲۹۵
فصل	درسنامه	تست چهارگزینه‌ای	پاسخ تشریحی

مطابق سه مجموعه اسلاید درس: «JS in HTML»، «Introduction to JS» و «NodeJS & TS». پاسخ همه‌ی تست‌های کدنویسی با اجرای واقعی کد در Node.js، jsdom و کامپایلر TypeScript (tsc) بررسی شده است.

فهرست مطالب

راهنمای استفاده از کتاب و برنامه‌ی مطالعه ۴

فصل ۱ مقدمه‌ای بر JavaScript ۶

۱-۱ JavaScript چیست؟ سه لایه‌ی وب، تاریخچه و ECMAScript ۷
۱-۲ Transpile و ویژگی‌های زبان JavaScript ۱۵
۱-۳ JavaScript چطور اجرا می‌شود؟ موتورها و APIها ۲۹
۱-۴ Console API و محل نوشتن کد JavaScript ۳۸
۱-۵ متغیرها: let، const و var ۴۹
۱-۶ انواع داده: Primitiveها و Objectها ۶۰
۱-۷ عملگرهای حسابی و انتساب ۷۳
۱-۸ رشته‌ها و Template literals ۸۳
۱-۹ تبدیل نوع صریح (Casting) ۹۵
۱-۱۰ تبدیل نوع ضمنی (Type coercion) و دام‌هایش ۱۰۶
۱-۱۱ عملگرهای مقایسه و منطقی ۱۱۹
۱-۱۲ کنترل جریان و دستورهای شرطی (if و switch) ۱۳۱
۱-۱۳ حلقه‌ها (Loops) ۱۵۰
۱-۱۴ Scope، Block و Global scope ۱۶۶
۱-۱۵ توابع: پارامتر، مقدار بازگشتی و توابع درجه‌ی یک ۱۸۲
۱-۱۶ Function declaration در برابر Arrow function ۱۹۷
۱-۱۷ مشکلات var ۲۱۲
۱-۱۸ Hoisting و Temporal Dead Zone (TDZ) ۲۲۵
۱-۱۹ Closure (بستار) ۲۴۴
۱-۲۰ اشیاء (Objects) و Object literal ۲۶۵
۱-۲۱ آرایه‌ها و متدهای shift، pop، push و unshift ۲۸۲
۱-۲۲ متدهای findIndex، find، filter، map، forEach و reduce ۲۹۷
۱-۲۳ Destructuring (تجزیه‌ی ساختار) و Spread ۳۲۰
۱-۲۴ کلاس‌ها (Classes) در JavaScript ۳۳۴
۱-۲۵ کلمه‌ی کلیدی this ۳۴۸
۱-۲۶ مدیریت خطا: try، catch، finally و throw ۳۶۸
۱-۲۷ Promise ۳۸۷

فصل ۲ JavaScript در HTML (کار با DOM) ۴۰۸

۲-۱ مرور پیش‌نیازها: سه لایه‌ی وب، شیء، this و async ۴۰۹
۲-۲ زبان‌های نشانه‌گذاری و درخت DOM ۴۲۴
۲-۳ سلسله‌مراتب DOM در JavaScript و شیء document ۴۳۹
۲-۴ پیدا کردن عناصر (Querying the DOM) ۴۵۲
۲-۵ رویدادها (Events) و addEventListener ۴۶۷
۲-۶ شیء event، انتشار رویداد (Bubbling) و Delegation ۴۸۰

۲-۷ تغییر متن، HTML و استایل عناصر ۴۹۶
۲-۸ ساختن و افزودن عناصر جدید ۵۱۰
۲-۹ حذف و جایگزینی عناصر ۵۲۵
۲-۱۰ ویژگی‌ها (Attributes) و کلاس‌ها: setAttribute و classList ۵۳۸
۲-۱۱ کار با فرم‌ها (Forms) ۵۵۰
۲-۱۲ شیء window: پنجره‌ی مرورگر ۵۶۵
۲-۱۳ ذخیره‌سازی در مرورگر: localStorage و sessionStorage ۵۷۷
آزمون جامع فصل دوم ۵۸۹

فصل ۳ Node.js و TypeScript ۶۰۲
۳-۱ محدودیت‌های JavaScript و تولد Node.js ۶۰۳
۳-۲ ویژگی‌های Node.js ۶۱۵
۳-۳ پکیج‌های خارجی و npm ۶۲۹
۳-۴ دستورات npm و تنظیم mirror ۶۳۸
۳-۵ اسکریپت‌های npm و فایل package.json ۶۴۷
۳-۶ Bundling برای مرورگر ۶۵۸
۳-۷ پکیج‌های محبوب ۶۶۶
۳-۸ مشکلات تایپ در JavaScript و معرفی TypeScript ۶۷۵
۳-۹ متغیرها و انواع داده در TypeScript ۶۸۵
۳-۱۰ Interface در TypeScript ۶۹۷
۳-۱۱ توابع در TypeScript ۷۱۲
۳-۱۲ کلاس‌ها در TypeScript ۷۲۸
۳-۱۳ Generics در TypeScript ۷۴۸
۳-۱۴ ماژول‌ها (Modules) ۷۶۲
۳-۱۵ بهترین شیوه‌ها (Best Practices) در TypeScript ۷۷۹

پاسخنامه‌ی تشریحی ۷۹۰
فصل ۱ پاسخ‌های مقدمه‌ای بر JavaScript ۷۹۱
فصل ۲ پاسخ‌های JavaScript در HTML (کار با DOM) ۸۷۵
فصل ۳ پاسخ‌های Node.js و TypeScript ۹۱۳

راهنمای استفاده از کتاب

این کتاب برای دانشجویی نوشته شده که **هیچ پیش‌زمینه‌ای** از JavaScript ندارد و می‌خواهد در چند روز برای کوییز درس برنامه‌نویسی وب (CE 40419) آماده شود. محدوده‌ی کتاب دقیقاً همان سه مجموعه اسلاید درس است: «Introduction to JS»، «JS in HTML» و «NodeJS & TS». همه‌ی مفاهیم اسلایدها پوشش داده شده‌اند؛ چه مفاهیمی که به‌صورت متن روی اسلایدند و چه کدها، نمودارها، میم‌ها و «Class Activity»هایی که فقط به‌صورت تصویر آمده‌اند. هر جا برای فهم واقعی یک اسلاید به پیش‌زمینه‌ی نزدیکی نیاز بوده (مثل TDZ، event loop یا bubbling رویدادها)، آن هم آموزش داده شده است.

این کتاب ۵۵ درسنامه و ۱۲۹۵ تست چهارگزینه‌ای با پاسخ تشریحی دارد.

ساختار کتاب

- کتاب سه فصل دارد: **فصل ۱** مقدمه‌ای بر JavaScript، **فصل ۲** JavaScript در HTML (کار با DOM) و **فصل ۳** Node.js و TypeScript.
- هر فصل به درس‌های کوچک تقسیم شده است. هر درس دو بخش دارد:
  1. **درسنامه:** توضیح ساده و قدم‌به‌قدم با مثال، تشبیه، نکته و دام تستی.
  2. **تست‌های تمرینی:** بلافاصله بعد از درسنامه، برای تثبیت همان درس.
- درس‌ها به ترتیب **یادگیری** چیده شده‌اند (نه لزوماً ترتیب اسلایدها). زیر عنوان هر درس نوشته شده که از کدام اسلایدها آمده است.
- آخر هر فصل یک **آزمون جامع فصل** با تست‌های ترکیبی و چالشی آمده است.
- **پاسخنامه‌ی تشریحی** همه‌ی تست‌ها در انتهای کتاب است؛ به ترتیب فصل و درس و با همان شماره‌ها. اول هر فصلِ پاسخنامه یک **کلید سریع** هم هست.
- شماره‌ی تست‌ها در کل کتاب پشت سر هم است. در فایل PDF روی شماره‌ی هر تست بزنید تا به پاسخش بروید و در پاسخنامه روی شماره بزنید تا به خود تست برگردید. فهرست مطالب هم قابل کلیک است.

سطح تست‌ها

برچسب	یعنی
آسان	سؤال مستقیم از خود مفهوم
متوسط	به‌کار بستن مفهوم در یک مثال
سخت	سؤال ترکیبی، دام‌دار یا پیش‌بینی خروجی کد

جعبه‌های درسنامه

> **نکته**
>
> نکته‌های مهمی که باید حفظ باشید.

> **دام تستی**
>
> اشتباه‌های رایجی که طراحان سؤال روی آن‌ها تست می‌سازند. قبل از کوییز همه‌ی این جعبه‌ها را دوباره بخوانید.

در درسنامه‌ها جعبه‌های «یک تشبیه ساده»، «مثال»، «بیشتر بدانیم» و «جمع‌بندی» هم می‌بینید.

خروجی کدها را چطور بخوانیم؟

در تست‌های «خروجی کد زیر چیست؟» هر خانه‌ی خاکستری **یک خط** از خروجی است و خانه‌ها از چپ به راست به همان ترتیبی که چاپ می‌شوند آمده‌اند. مثلاً گزینه‌ی `123` `33` `123` یعنی اول `33` چاپ می‌شود و در خط بعد `123` . خانه‌ی قرمز `Uncaught TypeError` یعنی برنامه در آن نقطه با این خطا متوقف می‌شود.

- خروجی‌ها مطابق `console.log` در **Node.js** نوشته شده‌اند: رشته‌ها در سطح اول بدون کوتیشن چاپ می‌شوند و رشته‌های داخل آرایه با کوتیشن تکی، مثل `['a', 'b']` .
- هر جا نتیجه در مرورگر و Node.js فرق دارد، در صورت سؤال گفته شده که منظور کدام محیط است.
- در فصل TypeScript فرض بر این است که تنظیم `strict` روشن است (پیش‌فرض پروژه‌هایی که با `tsc --init` ساخته می‌شوند).

> **درستی پاسخ‌ها**
>
> پاسخ همه‌ی تست‌هایی که به اجرای کد وابسته‌اند **با اجرای واقعی کد** بررسی شده است: کدهای JavaScript با Node.js، کدهای DOM با jsdom (شبیه‌ساز مرورگر) و کدهای TypeScript با کامپایلر `tsc` . یک اسکریپت خودکار تأیید کرده که در هر کدام از این تست‌ها فقط گزینه‌ی کلید با نتیجه‌ی واقعی یکی است.

روش پیشنهادی مطالعه‌ی هر درس

1. درسنامه را با دقت بخوانید و کدهای مثال را در ذهن (یا در Console مرورگر) اجرا کنید.
2. تست‌های همان درس را **بدون نگاه کردن به پاسخ** بزنید و کنار هر تست علامت بزنید: مطمئن، شک‌دار یا نمی‌دانم.
3. پاسخ تشریحی همه‌ی تست‌ها را بخوانید، حتی آن‌هایی که درست زده‌اید.
4. تست‌های غلط و شک‌دار را علامت بزنید تا روز آخر دوباره حل کنید.

برنامه‌ی مطالعه‌ی ۴ روزه

روز	چه بخوانیم	تمرکز ویژه
روز اول	فصل ۱، درس‌های ۱ تا ۱۷	انواع داده، Type coercion و `==` ، `let` / `const` / `var` ، scope، توابع
روز دوم	فصل ۱، درس‌های ۱۸ تا ۳۱ و آزمون جامع فصل ۱	hoisting و closure، TDZ، آرایه‌ها، `this` ، Promise، `async/await` و event loop
روز سوم	کل فصل ۲ و درس‌های ۱ تا ۷ فصل ۳	DOM، رویدادها، `preventDefault` ، `localStorage` ، npm و `package.json`
روز چهارم	درس‌های ۸ تا ۱۵ فصل ۳، آزمون‌های جامع فصل ۲ و ۳، مرور تست‌های غلط	TypeScript (generic، interface، کلاس، ماژول) و همه‌ی جعبه‌های «دام تستی»

> **نکته**
>
> در تست‌های خروجی، کد را خط به خط روی کاغذ دنبال کنید و مقدار متغیرها را بنویسید. بیشترِ اشتباه‌ها از عجله در همین مرحله است، نه از ندانستن مفهوم.

فصل اول

۱
مقدمه‌ای بر JavaScript

بر اساس اسلایدهای «3- Introduction to JS.pdf»

در این فصل از صفر شروع می‌کنیم: JavaScript چیست، از کجا آمده و چطور اجرا می‌شود. بعد سراغ داده‌ها، تبدیل نوع‌ها و دام‌های معروفش، متغیرها و scope، توابع و closure، hoisting، اشیاء و `this`، آرایه‌ها و متدهایشان، مدیریت خطا و JSON می‌رویم و فصل را با برنامه‌نویسی ناهمگام (callback، Promise، `async/await` و event loop) تمام می‌کنیم. بیشترِ تست‌های «خروجی کد» کوییز از همین فصل طرح می‌شوند.

۲۷ درس · ۶۹۰ تست · آزمون جامع پایان فصل

۱-۱ JavaScript چیست؟ سه لایه‌ی وب، تاریخچه و ECMAScript
۱-۲ Transpile و ویژگی‌های زبان JavaScript
۱-۳ JavaScript چطور اجرا می‌شود؟ موتورها و APIها
۱-۴ Console API و محل نوشتن کد JavaScript
۱-۵ متغیرها: var، let و const
۱-۶ انواع داده: Primitiveها و Objectها
۱-۷ عملگرهای حسابی و انتساب
۱-۸ رشته‌ها و Template literals
۱-۹ تبدیل نوع صریح (Casting)
۱-۱۰ تبدیل نوع ضمنی (Type coercion) و دام‌هایش
۱-۱۱ عملگرهای مقایسه و منطقی
۱-۱۲ کنترل جریان و دستورهای شرطی (if و switch)
۱-۱۳ حلقه‌ها (Loops)
۱-۱۴ Scope، Block و Global scope
۱-۱۵ توابع: پارامتر، مقدار بازگشتی و توابع درجه‌ی یک
۱-۱۶ Function declaration در برابر Arrow function
۱-۱۷ مشکلات var
۱-۱۸ Hoisting و Temporal Dead Zone (TDZ)
۱-۱۹ Closure (بستار)
۱-۲۰ اشیاء (Objects) و Object literal
۱-۲۱ آرایه‌ها و متدهای shift، pop، push و unshift
۱-۲۲ متدهای findIndex، find، filter، map، forEach و reduce
۱-۲۳ Destructuring (تجزیه‌ی ساختار) و Spread
۱-۲۴ کلاس‌ها (Classes) در JavaScript
۱-۲۵ کلمه‌ی کلیدی this
۱-۲۶ مدیریت خطا: try، catch، finally و throw
۱-۲۷ Promise

۱-۱ JavaScript چیست؟ سه لایه‌ی وب، تاریخچه و ECMAScript

منبع: Introduction to JS، اسلایدهای ۱ تا ۷

درسنامه

عنوان این جلسه از درس **«Make 'em Interactive!»** است؛ یعنی «تعاملی‌شان کن!». تا اینجا با HTML و CSS صفحه‌هایی ساختیم که فقط «نمایش» می‌دهند. از این فصل یاد می‌گیریم با **JavaScript** کاری کنیم که صفحه‌ها به کارهای کاربر **واکنش** نشان دهند. خیالت راحت باشد: از صفر شروع می‌کنیم و هیچ پیش‌نیازی لازم نیست.

اهداف این جلسه (اسلاید ۲)

این جلسه (و این فصل) سه هدف دارد:

- **(الف)** یاد بگیریم **JavaScript چیست**.
- **(ب) ویژگی‌ها (features) و محدودیت‌های (limitations)** آن را بفهمیم.
- **(ج)** یاد بگیریم **برنامه‌نویسی ناهمگام** (asynchronous یا async؛ یعنی کارهایی که نتیجه‌شان «بعداً» می‌رسد، مثل گرفتن داده از اینترنت) در JavaScript چطور کار می‌کند.

یادآوری: سه لایه‌ی یک وب‌سایت (اسلاید ۳)

یک تشبیه ساده

یک خانه را تصور کن. **اسکلت و دیوارها** مشخص می‌کنند خانه چند اتاق دارد و در و پنجره‌ها کجا هستند. **رنگ و دکوراسیون** ظاهر خانه را زیبا می‌کند. اما **سیم‌کشی برق** است که وقتی کلید را می‌زنی، لامپ روشن می‌شود؛ یعنی خانه به کار تو **واکنش** نشان می‌دهد. یک وب‌سایت هم دقیقاً همین سه لایه را دارد.

لایه	به چه سؤالی جواب می‌دهد؟	زبان	در مثال خانه
Structure (ساختار)	محتوا چیست؟ (What the **content** is)	HTML	دیوارها، اتاق‌ها، در و پنجره
Style (ظاهر)	چه شکلی است؟ (How it **looks**)	CSS	رنگ دیوار و دکوراسیون
Behaviour (رفتار)	چطور واکنش نشان می‌دهد؟ (How it **reacts**)	JavaScript	سیم‌کشی برق: کلید را بزنی، لامپ روشن می‌شود

خود اسلاید اسم زبان‌ها را نیاورده، ولی در این درس همیشه Structure یعنی HTML، Style یعنی CSS و Behaviour یعنی JavaScript.

بیا هر سه لایه را در یک صفحه‌ی کوچک ببینیم. HTML یک تیتر و یک دکمه می‌سازد، CSS دکمه را طلایی می‌کند و JavaScript تعداد کلیک‌ها را می‌شمارد:

index.html

```
<!DOCTYPE html>
<html>
  <head>
    <style>
      /* Style layer (CSS): how it looks */
      button { background: gold; padding: 8px 16px; }
    </style>
  </head>
  <body>
    <!-- Structure layer (HTML): what the content is -->
    <h1 id="title">Hello!</h1>
    <button id="btn">Click me</button>

    <script>
      // Behaviour layer (JavaScript): how it reacts
      const button = document.getElementById("btn");
      const title = document.getElementById("title");
      let clicks = 0;
      button.addEventListener("click", () => {
        clicks = clicks + 1;
        title.textContent = "Clicked " + clicks + " times";
      });
    </script>
  </body>
</html>
```

بعد از دو بار کلیک، متن تیتر می‌شود `Clicked 2 times` . اگر بخش `<script>` را حذف کنی، دکمه هنوز دیده می‌شود و طلایی است، ولی کلیک روی آن **هیچ اتفاقی** نمی‌اندازد و تیتر همان `Hello!` می‌ماند. لازم نیست الان تک‌تک خط‌های این کد را بفهمی؛ کار با صفحه (DOM) را در فصل ۲ کامل یاد می‌گیریم.

تا اینجا چه کارهایی بلدیم؟ (اسلاید ۴)

- یاد گرفتیم **ساختار** (structure) صفحه‌های وب را مشخص کنیم.
- یاد گرفتیم **محتوا** را به **بخش‌های معنادار** تقسیم کنیم (مثلاً با تگ‌های `<header>` ، `<nav>` ، `<main>` ، `<section>` و `<footer>`).
- یاد گرفتیم صفحه‌ها را **استایل** بدهیم (رنگ، فونت، چیدمان).
- **توانایی‌ها و محدودیت‌های CSS** را شناختیم.
- اما هنوز **نمی‌توانیم** صفحه‌ها را **interactive** (تعاملی؛ یعنی واکنش‌دهنده به کارهای کاربر) کنیم...

CSS فقط می‌تواند ظاهر را در چند حالتِ از پیش تعیین‌شده عوض کند (مثلاً با `:hover` رنگ دکمه را وقتی ماوس رویش می‌رود تغییر دهد)، اما **منطق** ندارد: نمی‌تواند تعداد کلیک‌ها را بشمارد، تصمیم بگیرد (مثلاً «اگر رمز کوتاه بود، پیام خطا بده»)، داده‌ای از سرور بگیرد یا متن صفحه را بر اساس ورودی کاربر بسازد. برای این کارها به یک **زبان برنامه‌نویسی** نیاز داریم.

JavaScript چیست؟ (اسلاید ۵)

- **JavaScript** (به اختصار **JS**) یک **زبان برنامه‌نویسی واقعی** (an actual programming language) است: متغیر، شرط، حلقه و تابع دارد. در مقابل، HTML یک زبان نشانه‌گذاری (markup) و CSS یک زبان استایل‌دهی است و هیچ‌کدام زبان برنامه‌نویسی نیستند.

- JavaScript هم **در مرورگر** اجرا می‌شود و هم **روی کامپیوتر خودت** (بیرون از مرورگر) به کمک **Node.js** (یک محیط اجرای JavaScript که در فصل ۳ مفصل می‌بینیم).
- **Brendan Eich** (برندن آیک) اولین **نمونه‌ی اولیه** (prototype) JavaScript را ساخت.
- او این کار را فقط در **۱۰ روز**، در ماه **مه (May) سال ۱۹۹۵**، وقتی در شرکت **Netscape** (سازنده‌ی مرورگر معروف آن زمان، Netscape Navigator) کار می‌کرد، انجام داد.

اولین برنامه‌ی JavaScript ما فقط یک پیام چاپ می‌کند (`console.log`) را در درس «Console API و محل نوشتن کد JavaScript» کامل می‌بینیم):

```JavaScript
console.log("Hello, JavaScript!"); // Hello, JavaScript!
```

تاریخچه‌ی JavaScript (اسلاید ۶)

- **۱۹۹۵:** Brendan Eich اولین نسخه‌ی JavaScript را فقط در ۱۰ روز ساخت. اسم آن نسخه **Mocha** بود.
- **۱۹۹۶:** اسم Mocha اول به **LiveScript** و بعد به **JavaScript** تغییر کرد. چرا؟ برای اینکه **برنامه‌نویس‌های Java جذب شوند** (Java آن روزها خیلی محبوب بود و این اسم یک تصمیم تبلیغاتی بود). اما اسلاید با تأکید می‌گوید: **JavaScript تقریباً هیچ ربطی به Java ندارد** (JavaScript has almost nothing to do with Java).
- **۱۹۹۶:** شرکت **Microsoft** در مرورگر **Internet Explorer**، JavaScript را از Netscape **کپی** کرد و اسمش را **JScript** گذاشت.
- **۱۹۹۷:** حالا هر شرکت نسخه‌ی خودش را داشت و زبان باید **استاندارد** می‌شد. سازمان **ECMA** استاندارد **ECMAScript 1** (به اختصار **ES1**) را منتشر کرد: **اولین استاندارد رسمی JavaScript**.

> **یک تشبیه ساده**
>
> Java و JavaScript مثل «کار» و «کارتن» هستند: فقط اسمشان شبیه است! شباهت اسم یک تصمیم تبلیغاتی بود، نه نشانه‌ی خویشاوندی دو زبان.

ECMAScript در برابر JavaScript

- **ECMAScript** «استاندارد» است: سندی که قواعد زبان را دقیق روی کاغذ می‌نویسد.
- **JavaScript** «زبان در عمل» است: همان زبانی که در مرورگرها و Node.js با آن کد می‌نویسیم و اجرا می‌کنیم و بر اساس این استاندارد ساخته شده است.
- **ES** مخفف ECMAScript است؛ پس ES1 یعنی ECMAScript نسخه‌ی ۱.

> **یک تشبیه ساده**
>
> ECMAScript مثل **کتاب قوانین شطرنج** است و JavaScript خودِ **بازی شطرنجی** که روی صفحه انجام می‌دهی. سازنده‌های مرورگرها کتاب قانون را می‌خوانند و زبان را طبق آن در مرورگرشان پیاده می‌کنند.

JavaScript مدرن (اسلاید ۷)

- **۲۰۰۹:** نسخه‌ی (ECMAScript 5) **ES5** با کلی امکان جدید منتشر شد؛ مثلاً حالت strict، شیء `JSON` و متدهای آرایه مثل `forEach`، `map` و `filter`.
- **۲۰۱۵:** نسخه‌ی **ES6** که اسم دیگرش (ECMAScript 2015) **ES2015** است منتشر شد: **بزرگ‌ترین به‌روزرسانی تاریخ زبان** (the biggest update to the language ever). بیشتر چیزهایی که در این فصل می‌خوانی از همین نسخه آمده‌اند: `let` و

`const`، Promise، destructuring، template literal، `class`، arrow function و ماژول‌ها (`import` / `export`).

- **۲۰۱۵:** ECMAScript به **چرخه‌ی انتشار سالانه** (annual release cycle) تغییر کرد تا در هر به‌روزرسانی **ویژگی‌های کمتری** منتشر شود؛ یعنی به جای یک بسته‌ی خیلی بزرگ هر چند سال یک بار، هر سال یک بسته‌ی کوچک.
- **۲۰۱۶ تا امروز:** هر سال یک نسخه: ES2021، ES2020، ES2019، ES2018، ES2017، ES2016 و ...

تصویر اسلاید ۷ (با عنوان «JAVASCRIPT EVOLUTION») شوخی معروف «تکامل انسان» است: از یک میمون خمیده تا مردی کت‌وشلواری با کیف و گوشی هوشمند. پیامش ساده است: JavaScript از ۱۹۹۵ تا امروز **خیلی** تکامل پیدا کرده است.

اسم نسخه‌ها: عددی یا سالی؟

از ES6 به بعد هر نسخه دو اسم دارد: یک اسم عددی و یک اسم سالی. قاعده‌اش ساده است: **سال = عدد نسخه + ۲۰۰۹**.

اسم عددی	اسم سالی	یک نمونه از ویژگی‌های جدید
ES6	ES2015	`let`، `const`، Promise، `class`، arrow function
ES7	ES2016	عملگر توان `**` و متد `includes` آرایه
ES8	ES2017	`async` / `await`
ES11	ES2020	`BigInt`، عملگرهای `?.` و `??`

کل تاریخچه در یک جدول

سال	چه اتفاقی افتاد؟
۱۹۹۵	Brendan Eich در ۱۰ روز (مه ۱۹۹۵، در Netscape) نسخه‌ی اول را ساخت؛ اسمش Mocha بود
۱۹۹۶	Mocha به LiveScript و بعد به JavaScript تغییر نام داد (برای جذب برنامه‌نویس‌های Java)
۱۹۹۶	Microsoft در Internet Explorer از JavaScript کپی کرد و اسمش را JScript گذاشت
۱۹۹۷	ECMA اولین استاندارد رسمی را منتشر کرد: ECMAScript 1 (ES1)
۲۰۰۹	ES5 با کلی امکان جدید
۲۰۱۵	ES6/ES2015: بزرگ‌ترین به‌روزرسانی تاریخ زبان
۲۰۱۵	شروع چرخه‌ی انتشار سالانه (ویژگی‌های کمتر در هر نسخه)
۲۰۱۶ تا امروز	ES2016، ES2017، ES2018، ...

> **بیشتر بدانیم**
>
> خارج از اسلاید، برای اینکه جای خالی‌ها گیجت نکند: بعد از ES1، نسخه‌های ES2 (۱۹۹۸) و ES3 (۱۹۹۹) آمدند. ES4 هیچ‌وقت منتشر نشد و کنار گذاشته شد؛ برای همین بعد از ES3 مستقیم به ES5 (۲۰۰۹) می‌رسیم. اگر دقیق‌تر بخواهی، تغییر نام به LiveScript و JavaScript در اواخر ۱۹۹۵ انجام شد و JavaScript در سال ۱۹۹۶ همراه با Netscape Navigator 2 به دست کاربران رسید؛ JScript هم در سال ۱۹۹۶ در Internet Explorer 3 عرضه شد. در تست‌های کوییز، همان سال‌های اسلاید را ملاک بگذار.

نکته

ES6 و ES2015 **دو اسم برای یک نسخه‌اند**. برای تبدیل اسم عددی به سالی، عدد را با ۲۰۰۹ جمع کن: ES6 یعنی ES2015، ES7 یعنی ES2016 و ES8 یعنی ES2017.

دام تستی

- **ES7 یعنی ES2016، نه ES2017!** این تله در تست‌ها زیاد دیده می‌شود.
- **ECMAScript** زبان جداگانه یا «نسخه‌ی جدید» JavaScript نیست؛ **استاندارد** JavaScript است. JScript هم نسخه‌ی کپی‌شده‌ی Microsoft بود، نه استاندارد.
- Java و JavaScript **دو زبان متفاوت‌اند**؛ شباهت اسم فقط برای جذب برنامه‌نویس‌های Java بود.
- ترتیب اسم‌ها: اول **Mocha**، بعد **LiveScript** و در آخر **JavaScript**.

جمع‌بندی

- وب‌سایت سه لایه دارد: Structure (HTML، محتوا)، Style (CSS، ظاهر) و Behaviour (JavaScript، واکنش).
- با HTML و CSS تنها، صفحه interactive نمی‌شود؛ برای منطق و واکنش به JavaScript نیاز داریم.
- JavaScript یک زبان برنامه‌نویسی واقعی است که هم در مرورگر و هم با Node.js روی کامپیوتر اجرا می‌شود.
- سازنده: Brendan Eich، در ۱۰ روز، مه ۱۹۹۵، در Netscape.
- اسم‌ها به ترتیب: Mocha، LiveScript، JavaScript. نسخه‌ی کپی‌شده‌ی JScript :Microsoft.
- ۱۹۹۷: ES1، اولین استاندارد (ECMAScript) استاندارد است و JavaScript زبان در عمل). ۲۰۰۹: ES5. ۲۰۱۵: ES6/ES2015، بزرگ‌ترین به‌روزرسانی و شروع انتشار سالانه.

تست‌های تمرینی

۱۵ تست · شماره‌ی ۱ تا ۱۵

۱ آسان

کدام مورد جزو اهداف این جلسه (اسلاید ۲) **نیست**؟

۱) یاد گرفتن اینکه JavaScript چیست

۲) شناخت ویژگی‌ها و محدودیت‌های JavaScript

۳) یاد گرفتن نحوه‌ی استایل‌دهی به صفحه با CSS

۴) یاد گرفتن نحوه‌ی کار برنامه‌نویسی ناهمگام (async) در JavaScript

۲ آسان

در مدل «سه لایه‌ی یک وب‌سایت»، لایه‌ی **Behaviour** به کدام سؤال جواب می‌دهد و با کدام زبان ساخته می‌شود؟

۱) «محتوا چیست؟» با HTML

۲) «چه شکلی است؟» با CSS

۳) «چطور واکنش نشان می‌دهد؟» با JavaScript

۴) «چطور واکنش نشان می‌دهد؟» با CSS

۳ متوسط

فرض کن فقط HTML و CSS بلدی. کدام کار را **بدون JavaScript** نمی‌توانی انجام دهی؟

۱) تقسیم محتوای صفحه به بخش‌های معنادار مثل header و footer

۲) نمایش عددی که با هر کلیک روی یک دکمه، یکی زیاد شود

۳) تعیین رنگ، فونت و چیدمان عناصر صفحه

۴) مشخص کردن تیترها، پاراگراف‌ها و تصاویر صفحه

۴ آسان

طبق اسلاید ۵، اولین نمونه‌ی اولیه‌ی JavaScript را چه کسی، کجا و در چه مدتی ساخت؟

۱) Brendan Eich، در شرکت Netscape، در ۱۰ روز

۲) Brendan Eich، در شرکت Microsoft، در ۱۰ ماه

۳) James Gosling، در شرکت Sun، در ۱۰ روز

۴) Tim Berners-Lee، در CERN، در یک سال

۵ متوسط

ترتیب درست نام‌های این زبان، از اولین نام تا نام امروزی، کدام است؟

۱) اول LiveScript، بعد Mocha، در آخر JavaScript

۲) اول Mocha، بعد LiveScript، در آخر JavaScript

۳) اول Mocha، بعد JavaScript، در آخر LiveScript

۴) اول JScript، بعد LiveScript، در آخر JavaScript

۶ آسان

طبق اسلاید ۶، چرا نام این زبان در نهایت JavaScript شد؟

۱) چون JavaScript نسخه‌ی ساده‌شده‌ی زبان Java است.

۲) چون کد JavaScript قبل از اجرا به Java ترجمه می‌شود.

۳) چون شرکت Sun (سازنده‌ی Java) آن را ساخته بود.

۴) برای جذب برنامه‌نویس‌های Java؛ در حالی که دو زبان تقریباً هیچ ربطی به هم ندارند.

۷ آسان

JScript چه بود؟

۱) اولین استاندارد رسمی JavaScript که ECMA منتشر کرد

۲) نسخه‌ای که Microsoft از روی JavaScript کپی کرد و در Internet Explorer به کار برد

۳) نام اولیه‌ی JavaScript، پیش از LiveScript

۴) ابزاری برای تبدیل کد JavaScript جدید به نسخه‌ی قدیمی‌تر

۸ متوسط

کدام جمله رابطه‌ی **ECMAScript** و **JavaScript** را درست بیان می‌کند؟

۱) ECMAScript استاندارد زبان است و JavaScript زبانی است که در عمل و بر اساس این استاندارد استفاده می‌شود.

۲) ECMAScript زبان جدیدی است که در سال ۲۰۱۵ جای JavaScript را گرفت.

۳) ECMAScript نسخه‌ی مخصوص Microsoft از JavaScript است.

۴) ECMAScript کتابخانه‌ای است که روی JavaScript نصب می‌شود تا امکانات جدید اضافه کند.

۹ متوسط

اولین استاندارد رسمی JavaScript چه نام داشت، در چه سالی و توسط کدام سازمان منتشر شد؟

۱) ES1 (ECMAScript 1)، سال ۱۹۹۵، Netscape

۲) ES5، سال ۲۰۰۹، ECMA

۳) JScript، سال ۱۹۹۶، Microsoft

۴) ES1 (ECMAScript 1)، سال ۱۹۹۷، ECMA

۱۰ متوسط

در جای خالی کدام گزینه قرار می‌گیرد؟

«در سال ۲۰۰۹ نسخه‌ی ..... منتشر شد و در سال ۲۰۱۵ نسخه‌ی ..... که بزرگ‌ترین به‌روزرسانی تاریخ زبان بود.»

۱) ES5 و ES6/ES2015

۲) ES6 و ES2015

۳) ES1 و ES5

۴) ES5 و ES2016

۱۱ متوسط

طبق اسلاید ۷، چرا ECMAScript از سال ۲۰۱۵ به چرخه‌ی انتشار سالانه (annual release cycle) تغییر کرد؟

۱) تا هر سال ویژگی‌های قدیمی زبان حذف شوند.

۲) چون ES2015 ایراد داشت و باید هر سال اصلاح می‌شد.

۳) تا در هر به‌روزرسانی ویژگی‌های کمتری منتشر شود و تغییرات کوچک‌تر و منظم‌تر باشند.

۴) تا مرورگرهای قدیمی دیگر نتوانند هیچ کد JavaScript را اجرا کنند.

۱۲ سخت

کدام جمله **نادرست** است؟

۱) ES6 و ES2015 دو نام برای یک نسخه‌اند.

۲) ES7 همان ES2017 است.

۳) از سال ۲۰۱۵ هر سال یک نسخه‌ی جدید با ویژگی‌های کمتر منتشر می‌شود.

۴) ES5 در سال ۲۰۰۹ منتشر شد.

۱۳ سخت

رویدادهای زیر را از قدیمی به جدید مرتب کن:

(الف) انتشار ES5 ــ (ب) عرضه‌ی JScript در Internet Explorer ــ (ج) انتشار ES1 توسط ECMA ــ (د) انتشار ES2015

۱) ج، ب، الف، د
۲) ب، الف، ج، د
۳) ج، الف، ب، د
۴) ب، ج، الف، د

۱۴ سخت

`async` / `await` در نسخه‌ی **ES2017** به زبان اضافه شد. نام عددی (ESn) این نسخه کدام است؟

۱) ES7
۲) ES17
۳) ES6
۴) ES8

۱۵ سخت

کدام جمله **درست** است؟

۱) JavaScript در سال ۱۹۹۵ در Microsoft ساخته شد و Netscape آن را با نام JScript کپی کرد.

۲) ES5 بزرگ‌ترین به‌روزرسانی تاریخ JavaScript بود.

۳) JavaScript یک زبان برنامه‌نویسی واقعی است که هم در مرورگر و هم به کمک Node.js روی کامپیوتر خودمان اجرا می‌شود.

۴) ECMAScript نام زبان جدیدی است که از سال ۲۰۱۵ جای JavaScript را گرفت.

۱-۲ Transpile و ویژگی‌های زبان JavaScript

منبع: Introduction to JS، اسلایدهای ۸ و ۹

درسنامه

در اسلاید ۸ با یک ابزار مهم دنیای JavaScript آشنا می‌شویم: **transpiler**. در اسلاید ۹ هم یک فعالیت کلاسی داریم: معنی نُه ویژگی اصلی JavaScript. هر دو را کامل و با مثال می‌بینیم.

مشکل: کد جدید، مرورگر قدیمی

در درس قبل دیدیم که JavaScript هر سال نسخه‌ی جدیدی می‌گیرد. اما همه‌ی کاربرها مرورگرشان را به‌روز نمی‌کنند. کد زیر از امکانات ES2015 استفاده می‌کند:

```JavaScript
const greet = (name) => `Hello, ${name}!`;
console.log(greet("Sara")); // Hello, Sara!
```

اینجا دو چیز جدید هست که بعداً مفصل یاد می‌گیری: **arrow function** (روش کوتاه نوشتن تابع با `=>`) و **template literal** (رشته‌ای بین دو backtick که با `${...}` یک مقدار را داخل متن می‌گذارد). یک مرورگر قدیمی مثل Internet Explorer 11 هیچ‌کدام از این دو را نمی‌شناسد؛ با دیدن `=>` خطای syntax (یعنی SyntaxError: «این دستور نگارش را نمی‌فهمم») می‌دهد و **کل اسکریپت اجرا نمی‌شود**.

Transpile چیست؟ (اسلاید ۸)

Transpile (ترکیب دو کلمه‌ی translate و compile؛ یعنی «ترجمه‌ی کد از یک نسخه یا زبان به نسخه یا زبانِ هم‌سطح دیگر») راه‌حل این مشکل است. اسلاید ۸ پنج نکته درباره‌اش می‌گوید:

نکته‌ی اسلاید	یعنی چه؟
Converts new JS to older syntax	syntax (دستور نگارش) جدید را به syntax قدیمی‌تر تبدیل می‌کند؛ مثلاً `=>` را به `function`
Keeps code working on **old browsers**	کد روی مرورگرهای قدیمی هم کار می‌کند
Uses tools like Babel or SWC	با ابزارهایی مثل **Babel** یا **SWC** (Speedy Web Compiler) انجام می‌شود
Lets devs use future JS features	برنامه‌نویس می‌تواند از امکانات جدید و حتی «آینده‌ی» زبان استفاده کند، پیش از آنکه همه‌ی مرورگرها از آن‌ها پشتیبانی کنند
Adds a small build step	یک **مرحله‌ی build** (ساخت) کوچک اضافه می‌کند: کدی که می‌نویسی مستقیماً به مرورگر نمی‌رود؛ اول ابزار اجرا می‌شود و نسخه‌ی تبدیل‌شده را می‌سازد

یک تشبیه ساده

meme اسلاید ۸ یک صحنه‌ی دادگاه است. مردی با برچسب **ES5+** (یعنی کد JavaScript جدید) ساکت ایستاده، چون قاضی زبانش را نمی‌فهمد. کنارش مردی با برچسب **Babel** به جای او حرف می‌زند و مثل **مترجم** حرف‌هایش را برای قاضی ترجمه می‌کند. قاضی با برچسب **Browser** (مرورگر) فقط زبان قدیمی را می‌فهمد. Transpile یعنی همین ترجمه: معنی همان می‌ماند، فقط کلمه‌ها عوض می‌شوند.

یک مثال واقعی

کد بالا را با Babel (تنظیم‌شده برای مرورگرهای قدیمی) transpile کردیم. فایل اول (`src/greet.js`) را ما نوشته‌ایم و فایل دوم (`dist/greet.js`) خروجی واقعی Babel است:

src/greet.js

```
const greet = (name) => `Hello, ${name}!`;
console.log(greet("Sara")); // Hello, Sara!
```

dist/greet.js

```
var greet = function greet(name) {
  return "Hello, ".concat(name, "!");
};
console.log(greet("Sara")); // Hello, Sara!
```

چه چیزهایی عوض شد؟

- `const` شد `var` (روش قدیمی تعریف متغیر).
- arrow function شد یک `function` معمولی، همراه با `return` صریح.
- template literal شد `"Hello, ".concat(name, "!")` که دقیقاً همان کار `"Hello, " + name + "!"` را می‌کند.
- و مهم‌تر از همه: **خروجی هر دو برنامه یکی است** (`Hello, Sara!`). Transpile رفتار برنامه را عوض نمی‌کند، فقط شکل نوشتنش را.

در عمل، تو فقط پوشه‌ی `src` را می‌نویسی و با یک دستور در ترمینال، نسخه‌ی `dist` ساخته می‌شود. مرورگر کاربر نسخه‌ی `dist` را دریافت می‌کند. این همان «مرحله‌ی build» است:

Terminal

```
npx babel src --out-dir dist   # build step: transpile every file in src/ into dist/
```

بیشتر بدانیم

Babel فقط **syntax** را ترجمه می‌کند. اگر یک **تابع آماده‌ی** جدید (مثلاً متد `includes` آرایه) در مرورگر قدیمی وجود نداشته باشد، باید کد جایگزینی به نام **polyfill** هم به برنامه اضافه شود. به همین دلیل در مثال بالا `concat` دست نخورد: این متد از قدیم در زبان بوده است.

Compile، Transpile و Interpret؛ فرقشان چیست؟

اصطلاح	چه می‌کند؟	مثال
Compile (کامپایل)	کد سطح بالا را به کد سطح پایین (مثلاً زبان ماشین) تبدیل می‌کند	تبدیل برنامه‌ی C به فایل اجرایی؛ تبدیل JavaScript به زبان ماشین داخل موتور V8
Transpile	کد را به یک زبان یا نسخه‌ی **هم‌سطح** دیگر تبدیل می‌کند	Babel؛ تبدیل ES2015+ به ES5 یا TypeScript به JavaScript (فصل ۳)
Interpret (تفسیر)	بدون ساختن فایل خروجی، کد را دستور به دستور می‌خواند و همان لحظه اجرا می‌کند	اجرای خط‌به‌خط کد در Console مرورگر

فعالیت کلاسی اسلاید ۹: نُه ویژگی JavaScript

اسلاید ۹ می‌پرسد: «These are some of the features of JavaScript. What does each one of them mean?»؛ یعنی این‌ها برخی از ویژگی‌های JavaScript هستند؛ هر کدام چه معنایی دارد؟ پاسخ کامل را یکی‌یکی می‌بینیم.

ویژگی ۱: High Level (سطح بالا)

یعنی زبان از جزئیات سخت‌افزار **فاصله** دارد: لازم نیست به آدرس‌های حافظه، ثبات‌های (register) پردازنده یا تعداد بایت‌های لازم برای یک داده فکر کنی و کد به زبان انسان نزدیک‌تر است. در زبان‌های سطح پایین (مثل Assembly و تا حدی C) این کارها با خود برنامه‌نویس است.

```JavaScript
const message = "Hello" + " " + "World";
console.log(message.length); // 11
```

نگفتیم این رشته چند بایت جا لازم دارد یا کجای حافظه ذخیره شود؛ موتور JavaScript خودش ترتیبش را می‌دهد. هزینه‌اش: کنترل کمتر روی سخت‌افزار.

ویژگی ۲: Garbage Collected (جمع‌آوری خودکار حافظه)

Garbage Collector (زباله‌جمع‌کن) بخشی از موتور است که حافظه‌ی مقدارهایی را که **دیگر هیچ راهی برای دسترسی به آن‌ها نمانده** به‌طور خودکار آزاد می‌کند. در زبانی مثل C باید خودت حافظه را بگیری و آزاد کنی و اگر فراموش کنی، حافظه هدر می‌رود (memory leak).

```JavaScript
let user = { name: "Sara" };
user = null; // the object is now unreachable, so the garbage collector can free it
```

> **یک تشبیه ساده**
>
> Garbage Collector مثل گارسونی است که میزهای خالی رستوران را جمع می‌کند: هر میزی که دیگر کسی پشتش ننشسته (مقداری که دیگر به آن دسترسی نداریم) را خودش تمیز می‌کند. **زمانش** را هم خودش تعیین می‌کند، نه تو.

ویژگی ۳: Interpreted/JIT Compiled (تفسیری / کامپایل درجا)

در JavaScript لازم نیست قبل از اجرا خودت کد را کامپایل کنی و فایل اجرایی (مثلاً exe) بسازی؛ فایل `.js` را مستقیماً به موتور (مرورگر یا Node.js) می‌دهی:

```
Terminal
node app.js   # no separate compile step: the engine reads and runs the file directly
```

موتورهای قدیمی کد را فقط **تفسیر** (interpret) می‌کردند. موتورهای امروزی مثل V8 از **JIT** (Just-In-Time، یعنی «درست در لحظه‌ی نیاز») استفاده می‌کنند: کد را parse (تجزیه) می‌کنند، سریع اجرایش را شروع می‌کنند و بخش‌هایی را که زیاد اجرا می‌شوند، **حین اجرا** به زبان ماشینِ بهینه کامپایل می‌کنند. اسلاید ۱۰ هم همین را می‌گوید: «Code is parsed, optimized, and compiled to machine code».

ویژگی ۴: Multi-paradigm (چندالگویی)

Paradigm یعنی «سبک برنامه‌نویسی». JavaScript به یک سبک محدود نیست و این سه سبک را پشتیبانی می‌کند:

- **Procedural** (رویه‌ای): نوشتن دستورها قدم‌به‌قدم و پشت سر هم.
- **Object-oriented** (شیءگرا یا OOP): گذاشتن داده و رفتار کنار هم در یک «شیء».
- **Functional** (تابعی): ساختن برنامه با پاس دادن تابع‌ها به هم.

یک مسئله (جمع قیمت‌ها) به هر سه سبک:

```
JavaScript
const prices = [10, 20, 30];

// 1) Procedural: step-by-step instructions
let total = 0;
for (let i = 0; i < prices.length; i++) {
  total = total + prices[i];
}
console.log(total); // 60

// 2) Object-oriented: data and behaviour live together in an object
const cart = {
  items: [10, 20, 30],
  getTotal() {
    let sum = 0;
    for (const p of this.items) sum = sum + p;
    return sum;
  },
};
console.log(cart.getTotal()); // 60

// 3) Functional: give a function to another function
console.log(prices.reduce((sum, p) => sum + p, 0)); // 60
```

نگران جزئیات `for`، `this` و `reduce` نباش؛ هر کدام درس خودش را دارد. فعلاً فقط ببین که **یک مسئله را می‌شود به سه سبک حل کرد**.

ویژگی ۵: Prototype-based OO (شیءگرایی مبتنی بر prototype)

در JavaScript هر شیء می‌تواند یک شیء دیگر را به‌عنوان **prototype** (الگو یا «والد») داشته باشد. اگر خاصیتی را از یک شیء بخواهی و خودش آن را نداشته باشد، JavaScript سراغ prototype آن می‌رود (و اگر آنجا هم نبود، سراغِ prototype آن prototype؛ به این زنجیره **prototype chain** می‌گویند). یعنی **ارث‌بری مستقیماً از یک شیء دیگر** انجام می‌شود.

JavaScript

```
const animal = {
  eats: true,
  walk() {
    return "walking...";
  },
};

const rabbit = Object.create(animal); // animal becomes rabbit's prototype
rabbit.jumps = true;

console.log(rabbit.jumps);  // true  (rabbit's own property)
console.log(rabbit.eats);   // true  (found on the prototype)
console.log(rabbit.walk()); // walking... (method borrowed from the prototype)
console.log(Object.keys(rabbit)); // [ 'jumps' ] (only its own keys)
```

`Object.create(animal)` یک شیء تازه می‌سازد که prototype آن `animal` است. `Object.keys` فقط کلیدهای **خودِ** شیء را برمی‌گرداند؛ برای همین `eats` در آن نیست، با اینکه `rabbit.eats` کار می‌کند. حتی `class` که در ES6 آمد، در پشت صحنه روی همین prototypeها ساخته شده است (`typeof` یک class برابر `"function"` است). متدهایی مثل `push` و `map` هم که روی همه‌ی آرایه‌ها کار می‌کنند، در prototype آرایه‌ها (`Array.prototype`) قرار دارند.

ویژگی ۶: First-class functions (تابع‌ها به‌عنوان «شهروند درجه‌یک»)

یعنی تابع در JavaScript یک **مقدار** است، درست مثل عدد و رشته. پس می‌توانی:

- تابع را در یک **متغیر** بگذاری؛
- تابع را به‌عنوان **ورودی** به تابع دیگر بدهی (به این تابعِ ورودی **callback** می‌گویند)؛
- تابع را از یک تابع دیگر **برگردانی**.

JavaScript

```
// 1) Store a function in a variable
const sayHi = function (name) {
  return "Hi, " + name;
};

// 2) Pass a function to another function as an argument
function runTwice(fn, value) {
  console.log(fn(value));
  console.log(fn(value));
}
runTwice(sayHi, "Ali"); // Hi, Ali (two times)

// 3) Return a function from a function
function makeMultiplier(factor) {
  return function (n) {
    return n * factor;
  };
}
const triple = makeMultiplier(3);
console.log(triple(5)); // 15
```

این ویژگی پایه‌ی خیلی از مباحث بعدی است: callbackها، متدهای `map` و `filter`، رویدادهای صفحه و closure.

ویژگی ۷: Dynamic (پویا؛ dynamically typed)

یعنی **نوع** (type) داده‌ها **هنگام اجرا** مشخص و بررسی می‌شود، نه قبل از آن. متغیر نوع ثابت ندارد و یک متغیر می‌تواند اول عدد، بعد رشته و بعد بولین نگه دارد. (عملگر `typeof` نوع یک مقدار را به‌صورت رشته برمی‌گرداند؛ در درس‌های بعدی کامل می‌بینیمش.)

```JavaScript
let x = 5;
console.log(typeof x); // "number"
x = "five";
console.log(typeof x); // "string"
x = true;
console.log(typeof x); // "boolean"
```

در زبان‌های **static** (ایستا) مثل C، Java یا TypeScript (فصل ۳)، نوع هر متغیر از قبل مشخص است و کامپایلر جلوی چنین کدی را می‌گیرد و خطا می‌دهد. آزادیِ dynamic بودن یک هزینه هم دارد: خطاهای مربوط به نوع فقط **هنگام اجرا** معلوم می‌شوند و تبدیل نوع خودکار (type coercion) گاهی نتیجه‌های عجیبی می‌دهد.

ویژگی ۸: Single threaded (تک‌نخی)

Thread (نخ یا رشته‌ی اجرا) یعنی یک مسیر اجرای دستورها. JavaScript فقط **یک** thread اصلی دارد؛ یعنی در هر لحظه فقط **یک کار** انجام می‌دهد. اگر یک کار طولانی شروع شود، بقیه‌ی کارها (حتی کلیک کاربر و تایمرها) باید صبر کنند و صفحه «هنگ» می‌کند. (`setTimeout` تابعی را بعد از گذشت زمانی مشخص اجرا می‌کند؛ در درس‌های بخش async کامل می‌بینیمش.)

```JavaScript
setTimeout(() => console.log("timer"), 0);

const start = Date.now();
while (Date.now() - start < 200) {
  // busy: the only thread is stuck here for 200 ms
}
console.log("loop done");
```

خروجی:

```
loop done
timer
```

با اینکه زمان تایمر صفر میلی‌ثانیه است، `timer` بعد از `loop done` چاپ می‌شود؛ چون تنها thread تا پایان حلقه مشغول است و نوبت تایمر نمی‌رسد.

ویژگی ۹: Non-blocking event loop (حلقه‌ی رویداد غیرمسدودکننده)

اگر JavaScript تک‌نخی است، چرا وقتی منتظر جواب سرور است صفحه هنگ نمی‌کند؟ به خاطر **event loop** (حلقه‌ی رویداد). کارهای زمان‌بر (تایمر، درخواست شبکه، خواندن فایل، منتظر کلیک ماندن) به **محیط اجرا** (مرورگر یا Node.js) سپرده می‌شوند و JavaScript **منتظر نمی‌ماند** (non-blocking) و سراغ خط‌های بعدی می‌رود. وقتی آن کار تمام شد، تابعِ مربوط به آن در یک **صف** (queue) قرار می‌گیرد و event loop هر وقت thread بیکار شد، آن را اجرا می‌کند.

```JavaScript
console.log("start");
setTimeout(() => console.log("timeout"), 0);
console.log("end");
```

خروجی:

```
start
end
timeout
```

> **یک تشبیه ساده**
>
> رستورانی را تصور کن که فقط **یک گارسون** دارد (single threaded). گارسون سفارش را به آشپزخانه می‌دهد (محیط اجرا) و به جای اینکه کنار آشپزخانه منتظر بماند، به میزهای دیگر می‌رسد (non-blocking). وقتی غذا آماده شد، آشپزخانه زنگ می‌زند (صف) و گارسون **هر وقت دستش خالی شد** غذا را می‌برد (event loop).

جزئیات کامل این ماجرا را در درس‌های async/await، Promise، callback و event loop می‌بینیم.

خلاصه‌ی فعالیت کلاسی

ویژگی	معنی در یک جمله	کلید تشخیص در تست
High Level	از جزئیات سخت‌افزار و مدیریت دستی حافظه دور است	«نزدیک به زبان انسان»
Garbage Collected	حافظه‌ی بلااستفاده خودکار آزاد می‌شود	«آزادسازی خودکار حافظه»
Interpreted/JIT Compiled	بدون کامپایل جداگانه اجرا می‌شود و موتور حین اجرا کد را به زبان ماشین کامپایل می‌کند	«parse، بهینه‌سازی، machine code»
Multi-paradigm	سبک‌های OOP، procedural و functional	«چند سبک برنامه‌نویسی»
Prototype-based OO	اشیاء از اشیاء دیگر (prototype) ارث می‌برند	«`Object.create`»، «prototype chain»
First-class functions	تابع مقدار است: ذخیره، پاس دادن، برگرداندن	«تابع به‌عنوان ورودی تابع دیگر»
Dynamic	نوع متغیر هنگام اجرا تعیین می‌شود و می‌تواند عوض شود	«`typeof` عوض می‌شود»
Single threaded	در هر لحظه فقط یک کار	«حلقه‌ی طولانی همه‌چیز را معطل می‌کند»
Non-blocking event loop	منتظر کارهای زمان‌بر نمی‌ماند؛ نتیجه بعداً از صف اجرا می‌شود	«`setTimeout` بعد از کدهای عادی»

> **نکته**
>
> Single threaded و Non-blocking event loop **مکمل هم‌اند**: JavaScript فقط یک thread دارد، اما چون برای کارهای زمان‌بر منتظر نمی‌ماند، روان و پاسخ‌گو می‌ماند. Transpile هم فقط شکل کد را عوض می‌کند؛ خروجی برنامه بعد از transpile همان قبلی است.

⚠ دام تستی

- **Transpile با Compile به زبان ماشین فرق دارد:** Babel کد JavaScript جدید را به JavaScript قدیمی (هم‌سطح) تبدیل می‌کند، نه به زبان ماشین. فشرده‌سازی کد (minify) هم transpile نیست.
- Transpile مرحله‌ی build را **اضافه** می‌کند، حذف نمی‌کند.
- «Dynamic» در این فهرست یعنی **dynamic typing** (نوع متغیر هنگام اجرا مشخص می‌شود)، نه «صفحه‌ی وب پویا».
- «Single threaded» یعنی در هر لحظه **یک تکه کد JavaScript** اجرا می‌شود؛ ولی تعداد زیادی تایمر و درخواست شبکه می‌توانند همزمان در جریان باشند، چون انتظارشان را محیط اجرا می‌کشد.
- «Interpreted» به این معنی نیست که کد هیچ‌وقت به زبان ماشین تبدیل نمی‌شود؛ موتورهای امروزی JIT دارند.
- وقتی arrow function را به `function` معمولی تبدیل می‌کنی، `return` را فراموش نکن؛ِ arrow function یک‌خطی مقدارش را خودکار برمی‌گرداند ولی `function` معمولی نه.

تست‌های تمرینی

۲۰ تست · شماره‌ی ۱۶ تا ۳۵

۱۶ آسان

Transpile کردن کد JavaScript یعنی چه؟

۱) تبدیل کد JavaScript به زبان ماشین برای اجرای سریع‌تر

۲) تبدیل syntax جدید JavaScript به syntax قدیمی‌تر تا مرورگرهای قدیمی هم آن را اجرا کنند

۳) فشرده‌کردن کد و حذف فاصله‌ها برای کم‌حجم‌شدن فایل

۴) تبدیل کد JavaScript به زبان Java

۱۷ آسان

طبق اسلاید ۸، کدام دو ابزار برای transpile کردن JavaScript به کار می‌روند؟

۱) V8 و SpiderMonkey

۲) CodePen و JSFiddle

۳) Node.js و npm

۴) Babel و SWC

۱۸ آسان

در meme اسلاید ۸ (صحنه‌ی دادگاه)، **Babel** چه نقشی دارد؟

۱) قاضی‌ای که درباره‌ی درست یا غلط بودن کد تصمیم می‌گیرد

۲) مترجمی که حرف‌های کد جدید (ES5+) را به زبانی که مرورگر (قاضی) می‌فهمد برمی‌گرداند

۳) متهمی که کد قدیمی نوشته است

۴) وکیلی که مرورگر را مجبور می‌کند syntax جدید را یاد بگیرد

۱۹ متوسط

کدام مورد از ویژگی‌های transpile که در اسلاید ۸ آمده **نیست**؟

۱) به برنامه‌نویس اجازه می‌دهد از امکانات آینده‌ی JavaScript استفاده کند.

۲) کد را روی مرورگرهای قدیمی قابل اجرا نگه می‌دارد.

۳) نیاز به هر نوع مرحله‌ی build را از بین می‌برد.

۴) syntax جدید را به syntax قدیمی‌تر تبدیل می‌کند.

۲۰ متوسط

منظور از جمله‌ی «Adds a small build step» در اسلاید ۸ چیست؟

۱) کاربرِ سایت باید Babel را روی کامپیوتر خودش نصب کند تا بتواند سایت را ببیند.

۲)

۳) کدی که می‌نویسیم مستقیماً به مرورگر داده نمی‌شود؛ قبل از انتشار، ابزاری مثل Babel اجرا می‌شود و نسخه‌ی تبدیل‌شده را می‌سازد.

۴) کد پس از transpile فقط در مرورگرهای جدید اجرا می‌شود.

۲۱ متوسط

کدام مورد نمونه‌ای از **transpile** است؟

۱) تبدیل کد ES2015 به ES5 با Babel

۲) تبدیل کد JavaScript به زبان ماشین توسط موتور V8 هنگام اجرا

۳) اجرای خط‌به‌خط دستورها در Console مرورگر

۴) فشرده‌سازی فایل‌های پروژه با zip

۲۲ سخت

Babel کد زیر را به ES5 تبدیل می‌کند. کدام گزینه یک معادل **درست** در ES5 است؛ یعنی دقیقاً همان خروجی `Hello, Sara!` را چاپ می‌کند؟

```JavaScript
const greet = (name) => `Hello, ${name}!`;
console.log(greet("Sara"));
```

۱

```JavaScript
var greet = function (name) {
  "Hello, " + name + "!";
};
console.log(greet("Sara"));
```

۲

```JavaScript
var greet = function (name) {
  return 'Hello, ${name}!';
};
console.log(greet("Sara"));
```

۳

```JavaScript
var greet = function (name) {
  return "Hello, " + "name" + "!";
};
console.log(greet("Sara"));
```

۴

```JavaScript
var greet = function (name) {
  return "Hello, " + name + "!";
};
console.log(greet("Sara"));
```

۲۳ آسان

منظور از **High Level** بودن JavaScript چیست؟

۱ فقط روی کامپیوترهای قدرتمند اجرا می‌شود.

۲ از همه‌ی زبان‌های برنامه‌نویسی سریع‌تر است.

۳ برای اجرا باید ابتدا به یک فایل اجرایی (مثلاً exe) کامپایل شود.

۴ برنامه‌نویس با جزئیات سخت‌افزار مثل آدرس‌های حافظه و ثبات‌های پردازنده درگیر نمی‌شود و کد به زبان انسان نزدیک‌تر است.

۲۴ آسان

کدام ویژگی JavaScript یعنی «حافظه‌ی مقدارهایی که دیگر هیچ راهی برای دسترسی به آن‌ها نمانده، به‌طور خودکار آزاد می‌شود»؟

۱) Garbage Collected
۲) First-class functions
۳) Dynamic
۴) Single threaded

۲۵ آسان

اینکه در JavaScript می‌توان یک تابع را در متغیر ذخیره کرد، آن را به تابع دیگری پاس داد یا از یک تابع برگرداند، کدام ویژگی است؟

۱) Prototype-based OO
۲) Garbage Collected
۳) First-class functions
۴) Single threaded

۲۶ متوسط

منظور از **Multi-paradigm** بودن JavaScript چیست؟

۱) می‌توان در JavaScript به سبک‌های مختلفی مثل رویه‌ای (procedural)، شیءگرا (OOP) و تابعی (functional) برنامه نوشت.
۲) JavaScript هم در مرورگر و هم در Node.js اجرا می‌شود.
۳) JavaScript می‌تواند چند تکه کد را دقیقاً همزمان روی چند هسته‌ی پردازنده اجرا کند.
۴) نوع متغیرها در JavaScript هنگام اجرا مشخص می‌شود و می‌تواند تغییر کند.

۲۷ متوسط

خروجی کد زیر چیست؟ (این کد ویژگی **Dynamic** را نشان می‌دهد.)

```javascript
let x = 5;
console.log(typeof x);
x = "five";
console.log(typeof x);
x = true;
console.log(typeof x);
```

۱) `number` `number` `number`
۲) `number` `string` `boolean`
۳) `number` `Uncaught TypeError`
۴) `5` `five` `true`

۲۸ متوسط

خروجی کد زیر چیست؟

```JavaScript
function shout(text) {
  return text.toUpperCase() + "!";
}
function whisper(text) {
  return text.toLowerCase() + "...";
}
function speak(style, text) {
  return style(text);
}
console.log(speak(shout, "Hello"));
console.log(speak(whisper, "Hello"));
```

۱) `HELLO!` `hello...`

۲) `Hello!` `Hello...`

۳) `Uncaught TypeError`

۴) `[Function: shout]` `[Function: whisper]`

۲۹ متوسط

خروجی کد زیر چیست؟

```JavaScript
console.log("start");
setTimeout(() => console.log("timeout"), 0);
console.log("end");
```

۱) `start` `timeout` `end`

۲) `start` `end` `timeout`

۳) `timeout` `start` `end`

۴) `start` `end`

۳۰ سخت

خروجی کد زیر چیست؟

```JavaScript
const animal = { eats: true };
const rabbit = Object.create(animal);
rabbit.jumps = true;

console.log(rabbit.jumps, rabbit.eats);
console.log(Object.keys(rabbit));
```

۱) `true undefined` `['jumps']`

۲) `true true` `['jumps', 'eats']`

۳) `Uncaught TypeError`

۴) `true true` `['jumps']`

۳۱ سخت

کدام جمله درباره‌ی «Interpreted/JIT Compiled» بودن JavaScript در موتورهای امروزی مثل V8 **درست** است؟

۱) کد JavaScript قبل از اجرا باید توسط برنامه‌نویس به یک فایل اجرایی کامپایل شود.

۲) کد فقط خط‌به‌خط تفسیر می‌شود و هیچ‌وقت به زبان ماشین تبدیل نمی‌شود.

۳) موتور کد را parse می‌کند و حین اجرا، بخش‌هایی را که زیاد اجرا می‌شوند به زبان ماشینِ بهینه کامپایل می‌کند.

۴) کد JavaScript ابتدا به bytecode زبان Java تبدیل و روی JVM اجرا می‌شود.

۳۲ سخت

بعد از اجرای خط دوم کد زیر، چه بر سر شیء `{ name: "Sara" }` می‌آید؟

```JavaScript
let user = { name: "Sara" };
user = null;
```

۱) چون دیگر هیچ راهی برای دسترسی به آن نمانده، Garbage Collector هر وقت خودش تشخیص دهد، حافظه‌اش را آزاد می‌کند.

۲) برنامه‌نویس باید با دستوری مثل `free(user)` حافظه‌اش را آزاد کند.

۳) خطا رخ می‌دهد، چون نمی‌توان به متغیری که یک شیء داشته `null` داد.

۴) شیء تا بسته شدن برنامه در حافظه می‌ماند و هرگز آزاد نمی‌شود.

۳۳ سخت

خروجی کد زیر چیست؟

```JavaScript
setTimeout(() => console.log("timer"), 0);

const start = Date.now();
while (Date.now() - start < 200) {
  // busy for 200 ms
}
console.log("loop done");
```

۱) `timer` `loop done`

۲) `loop done`

۳) `loop done` `timer`

۴) `timer`

۳۴ سخت

کدام جمله **نادرست** است؟

۱) First-class functions: می‌توان یک تابع را به‌عنوان ورودی به تابع دیگری داد.

۲) Dynamic: نوع یک متغیر هنگام اجرا مشخص می‌شود و می‌تواند عوض شود.

۳) Garbage Collected: حافظه‌ی بلااستفاده به‌طور خودکار آزاد می‌شود.

۴) Single threaded: JavaScript می‌تواند چند تکه کد JavaScript را دقیقاً در یک لحظه روی چند هسته‌ی پردازنده اجرا کند.

۳۵ سخت

در جای خالی کدام گزینه قرار می‌گیرد؟

«JavaScript ..... است، یعنی در هر لحظه فقط یک کار انجام می‌دهد؛ اما به لطف .....، منتظر ماندن برای یک تایمر یا جواب شبکه برنامه را متوقف نمی‌کند.»

۱) multi-threaded و Garbage Collector

۲) single threaded و non-blocking event loop

۳) single threaded و JIT compiler

۴) dynamic و prototype chain

۱-۳ JavaScript چطور اجرا می‌شود؟ موتورها و APIها

منبع: Introduction to JS، اسلایدهای ۱۰ و ۱۱

درسنامه

تا اینجا فهمیدیم JavaScript چیست. حالا سؤال این است: وقتی کد JavaScript می‌نویسیم، **چه کسی** آن را می‌خواند و اجرا می‌کند؟ و یک زبان برنامه‌نویسی چطور می‌تواند پیام نشان دهد، از اینترنت داده بگیرد یا صفحه را عوض کند؟

> **یک تشبیه ساده**
>
> کد JavaScript مثل یک **دستور پخت** است. **موتور** (engine) آشپزی است که دستور را می‌خواند و قدم‌به‌قدم اجرا می‌کند. اما آشپز بدون **آشپزخانه** کاری از پیش نمی‌برد: اجاق، یخچال و چاقو را آشپزخانه در اختیارش می‌گذارد. این آشپزخانه همان **محیط اجرا** (environment) است و ابزارهایش همان **APIها**. نکته‌ی جالب: یک آشپز (مثلاً موتور V8) می‌تواند در دو آشپزخانه‌ی مختلف کار کند: مرورگر و Node.js.

JavaScript چطور اجرا می‌شود؟ (اسلاید ۱۰)

اجرای خط‌به‌خط

کد JavaScript **خط‌به‌خط** (line-by-line) و از بالا به پایین اجرا می‌شود؛ یا **داخل مرورگر** به کمک یک **موتور** (engine: برنامه‌ای که کد JavaScript را می‌خواند و اجرا می‌کند)، یا **روی کامپیوتر خودت** به کمک **Node.js**.

```JavaScript
console.log("1: start");
const sum = 2 + 3;
console.log("2: sum is", sum);
console.log("3: end");
```

خروجی:

```
1: start
2: sum is 5
3: end
```

اگر یک خط **هنگام اجرا** خطا بدهد، خط‌های قبلی اجرا شده‌اند، ولی برنامه همان‌جا متوقف می‌شود:

```JavaScript
console.log("A");
console.log(B);   // ReferenceError: B is not defined
console.log("C"); // never runs
```

خروجی: `A` و سپس خطای `ReferenceError` (یعنی «این اسم تعریف نشده است»)؛ `C` هرگز چاپ نمی‌شود.

موتورهای JavaScript

هر مرورگر موتور مخصوص خودش را دارد:

مرورگر یا محیط	موتور JavaScript
Chrome (و مرورگرهای مبتنی بر Chromium مثل Edge)	**V8** (ساخت Google)
Firefox	**SpiderMonkey** (اولین موتور JavaScript تاریخ؛ آن را Brendan Eich در Netscape نوشت و امروز Mozilla نگهداری‌اش می‌کند)
Safari	JavaScriptCore (ساخت Apple)
Node.js	**V8** (همان موتور Chrome)

اسلاید فقط V8 و SpiderMonkey را نام برده و بقیه را با «etc.» رد شده است؛ این دو را حتماً به خاطر بسپار.

داخل موتور چه می‌گذرد؟

اسلاید ۱۰ می‌گوید: «Code is **parsed**, **optimized**, and compiled to **machine code**». یعنی:

```
source code (.js file)
        |
        v
 [ parse ]           read the whole code, check the syntax, build a tree of it
        |
        v
 [ run + optimize ]  start running quickly, watch which parts run often
        |
        v
 [ machine code ]    hot parts are compiled (JIT) to fast code the CPU understands
```

- **Parse** (تجزیه): موتور اول کل کد را می‌خواند، syntax (دستور نگارش) آن را بررسی می‌کند و یک ساختار درختی از آن می‌سازد.
- **Optimize** (بهینه‌سازی): موتور حین اجرا می‌بیند کدام قسمت‌ها زیاد اجرا می‌شوند.
- **Machine code** (زبان ماشین): آن قسمت‌ها به زبان ماشین (کدی که مستقیماً روی پردازنده اجرا می‌شود) کامپایل می‌شوند؛ همان JIT که در درس «Transpile و ویژگی‌های زبان JavaScript» دیدیم.

یک نتیجه‌ی مهم از مرحله‌ی parse: چون موتور **قبل از اجرا** کل فایل را می‌خواند، اگر جایی از فایل **خطای syntax** (SyntaxError) باشد، حتی خط‌های سالمِ قبل از آن هم اجرا نمی‌شوند:

```JavaScript
console.log("A");
console.log("B");
console.log("C"; // SyntaxError: missing ) after argument list
```

اجرای این فایل با Node.js **هیچ خروجی‌ای** ندارد و فقط `SyntaxError` می‌دهد. پس «خط‌به‌خط» درباره‌ی **اجرا** است؛ خطای syntax قبل از شروع اجرا پیدا می‌شود. (در Console مرورگر، هر بار که Enter می‌زنی همان ورودی جداگانه parse و اجرا می‌شود.)

کارهای async بدون مسدود کردن

موتور کارهای ناهمگام مثل **تایمرها** (timers)، **Promiseها** (promises: قول یک نتیجه در آینده) و **رویدادهای کاربر** (user events، مثل کلیک) را **بدون مسدود کردن** (without blocking) برنامه مدیریت می‌کند: کد منتظر نمی‌ماند و جلو می‌رود و هر وقت نتیجه آماده شد، کار مربوطه انجام می‌شود. این همان non-blocking event loop است که در درس قبل دیدیم.

APIهای محیط

مرورگر یا Node.js، **API**هایی مثل **HTTP**، **DOM** و **File system** در اختیار JavaScript می‌گذارند. اسلاید ۱۱ همین موضوع را باز می‌کند.

APIهای JavaScript (اسلاید ۱۱)

- JavaScript فقط «داده و تابع» نیست (JavaScript isn't just about data and functions).
- JavaScript همیشه داخل یک **محیط** (environment) اجرا می‌شود: مرورگر، Node.js و ...
- این محیط‌ها **API** فراهم می‌کنند (Application Programming Interfaces؛ «رابط‌های برنامه‌نویسی کاربردی»): مجموعه‌ای از اشیاء و تابع‌های آماده که کد تو با آن‌ها از امکانات محیط استفاده می‌کند.
- APIها به JavaScript اجازه می‌دهند با **دنیای بیرون** ارتباط بگیرد: پیام نشان بدهد (show messages)، داده بگیرد (fetch data) و صفحه را تغییر دهد (manipulate the page).
- **بدون این APIها، JavaScript به‌تنهایی کار زیادی جز محاسبات نمی‌تواند بکند.**

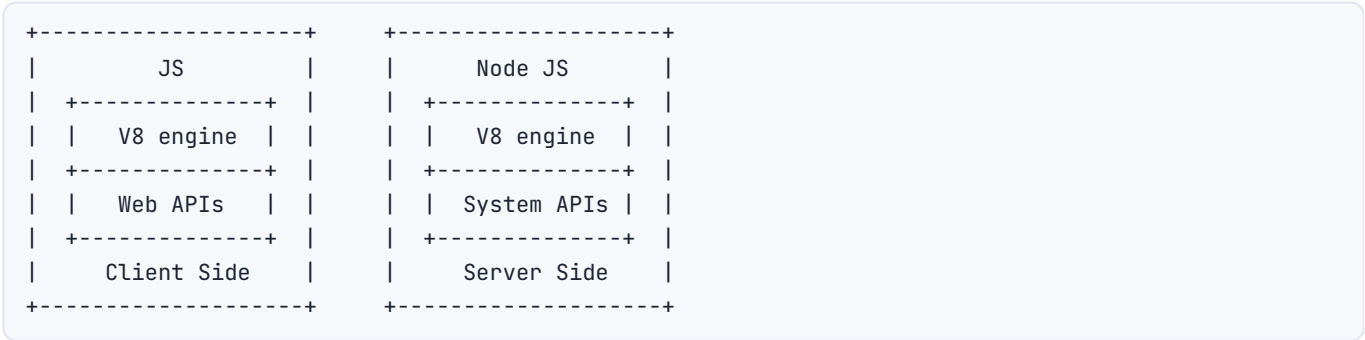
> **یک تشبیه ساده**
>
> API مثل **منوی رستوران** است: از روی منو «کباب» سفارش می‌دهی و کباب تحویل می‌گیری، بدون اینکه بدانی آشپزخانه چطور کار می‌کند. کد تو هم مثلاً `fetch(...)` را صدا می‌زند و داده می‌گیرد، بدون اینکه درگیر جزئیات شبکه شود.

JavaScript خالص (فقط خود زبان) این کار را بلد است:

```JavaScript
const prices = [120, 80, 50];
const total = prices[0] + prices[1] + prices[2];
const average = total / prices.length;
console.log(total, average); // 250 83.33333333333333
console.log(Math.round(average)); // 83
```

همه‌اش محاسبه است. البته برای **دیدن** نتیجه از `console.log` استفاده کردیم که خودش یک API محیط است!

نمودار اسلاید ۱۱: یک موتور، دو محیط



- **جعبه‌ی چپ (JS در مرورگر):** موتور V8 به‌علاوه‌ی **Web APIs**. این سمت **Client Side** (سمت کاربر) است: کد روی دستگاه کاربر و داخل مرورگرش اجرا می‌شود.
- **جعبه‌ی راست (Node JS):** همان موتور V8 به‌علاوه‌ی **System APIs**. این سمت **Server Side** (سمت سرور) است: کد روی سرور (یا کامپیوتر خودت) اجرا می‌شود و به امکانات سیستم‌عامل، مثل فایل‌ها، دسترسی دارد.

پیام نمودار: **موتور در هر دو یکی است؛ چیزی که فرق می‌کند APIهای محیط است.**

چند نمونه API

در مرورگر (Web APIs):

```JavaScript
alert("Welcome!");                                  // show a message (dialog)
document.querySelector("h1").textContent = "Hi!";  // manipulate the page (DOM)
fetch("/api/products");                             // fetch data over HTTP
```

در Node.js (System APIs)، مثلاً کار با **فایل‌ها** (File system):

```notes.js
const fs = require("fs"); // Node.js System API: the file system

fs.writeFileSync("note.txt", "Hello from Node.js!");
const text = fs.readFileSync("note.txt", "utf8");
console.log(text); // Hello from Node.js!
```

قابلیت	مرورگر	Node.js
خود زبان: `Math` ، `JSON` ، آرایه‌ها، رشته‌ها	دارد	دارد
`console` و `setTimeout`	دارد	دارد
DOM (`document`) و `alert`	دارد	**ندارد**
کار با فایل‌ها (`fs`) و شیء `process`	**ندارد**	دارد

می‌توانی این تفاوت را با `typeof` امتحان کنی (`typeof` برای اسمی که اصلاً تعریف نشده، خطا نمی‌دهد و `"undefined"` برمی‌گرداند):

```JavaScript
// In Node.js:
console.log(typeof Math, typeof JSON); // object object (part of the language)
console.log(typeof console);           // object (provided by the environment)
console.log(typeof document);          // undefined (no DOM in Node.js)
console.log(typeof process);           // object (a Node.js System API)
```

همین کد در مرورگر، برای `typeof document` مقدار `"object"` و برای `typeof process` مقدار `"undefined"` می‌دهد. و اگر در Node.js مستقیماً از `document` استفاده کنی (مثلاً `document.title = "Hi"`)، خطای `ReferenceError: document is not defined` می‌گیری.

زبان، موتور و محیط اجرا: سه چیز متفاوت

مفهوم	چیست؟	مثال
زبان (language)	قواعد و syntax؛ استانداردش ECMAScript است	JavaScript
موتور (engine)	برنامه‌ای که کد را parse، بهینه و به زبان ماشین تبدیل و اجرا می‌کند	V8، SpiderMonkey
محیط اجرا (runtime environment)	موتور + APIها (+ event loop)	مرورگر، Node.js

نکته

Node.js نه زبان برنامه‌نویسی است و نه موتور؛ یک **محیط اجرا** (runtime) است: موتور V8 به‌علاوه‌ی System APIها، برای اجرای JavaScript بیرون از مرورگر. V8 هم مرورگر نیست؛ موتوری است که داخل Chrome (و Node.js) کار می‌کند.

دام تستی

- DOM (یعنی `document` و هر چیزی که صفحه را تغییر می‌دهد) **جزو خود زبان JavaScript نیست**؛ مرورگر آن را می‌دهد. برای همین `document` در Node.js وجود ندارد و استفاده از آن `ReferenceError` می‌دهد.
- `console` و `setTimeout` هم جزو خود زبان (استاندارد ECMAScript) نیستند و محیط‌ها آن‌ها را می‌دهند؛ فقط چون هم مرورگر و هم Node.js آن‌ها را دارند، همه‌جا در دسترس‌اند.
- «اجرای خط‌به‌خط» یعنی خطای **هنگام اجرا** برنامه را از همان خط متوقف می‌کند؛ ولی خطای **syntax** قبل از اجرا (در مرحله‌ی parse) پیدا می‌شود و هیچ خطی اجرا نمی‌شود.
- Chrome و Node.js موتور **مشترک** دارند (V8)، ولی APIهایشان فرق دارد.

جمع‌بندی

- کد JavaScript خط‌به‌خط اجرا می‌شود: در مرورگر با یک موتور، یا روی کامپیوتر با Node.js.
- موتورها: Chrome از V8 و Firefox از SpiderMonkey استفاده می‌کند؛ Node.js هم V8 دارد.
- موتور کد را parse، بهینه و به machine code کامپایل می‌کند.
- کارهای async (تایمر، Promise، رویدادهای کاربر) بدون مسدود کردن برنامه انجام می‌شوند.
- محیط اجرا APIها را می‌دهد: مرورگر = V8 + Web APIs (Client Side)؛ Node.js = V8 + System APIs (Server Side).
بدون APIها، JavaScript فقط محاسبه می‌کند.

تست‌های تمرینی

۱۵ تست · شماره‌ی ۳۶ تا ۵۰

۳۶ آسان

طبق اسلاید ۱۰، کدام جفت «مرورگر و موتور JavaScript آن» درست است؟

۱) Chrome و SpiderMonkey
۲) Firefox و V8
۳) Firefox و SpiderMonkey
۴) Chrome و Babel

۳۷ آسان

Node.js برای اجرای کد JavaScript از کدام موتور استفاده می‌کند؟

۱) SpiderMonkey
۲) V8
۳) یک موتور اختصاصی که هیچ ربطی به مرورگرها ندارد
۴) JScript

۳۸ آسان

طبق اسلاید ۱۰، کدام کارها به‌صورت ناهمگام (async) و **بدون مسدود کردن** برنامه انجام می‌شوند؟

۱) تعریف متغیرها و حلقه‌ها

۲) جمع و ضرب اعداد

۳) نوشتن تگ‌های HTML

۴) تایمرها، Promiseها و رویدادهای کاربر مثل کلیک

۳۹ آسان

API مخفف چیست و در این درس به چه معناست؟

۱) Application Programming Interface: امکاناتی که محیط اجرا در اختیار کد می‌گذارد تا با دنیای بیرون ارتباط بگیرد

۲) Advanced Programming Interpreter: نوعی موتور اجرای JavaScript

۳) Application Program Installer: ابزاری برای نصب برنامه‌ها

۴) Automatic Program Integration: ابزاری برای transpile کردن کد

۴۰ متوسط

در جای خالی کدام گزینه قرار می‌گیرد؟

«طبق اسلاید ۱۰، کد JavaScript توسط موتور .....، بهینه (optimized) و به ..... کامپایل می‌شود.»

۱) transpile می‌شود ــ زبان Java

۲) فشرده می‌شود ــ HTML

۳) parse می‌شود ــ زبان ماشین (machine code)

۴) parse می‌شود ــ ECMAScript

۴۱ متوسط

کدام جمله درباره‌ی Node.js **درست** است؟

۱) یک زبان برنامه‌نویسی جدید است که از JavaScript مشتق شده.

۲) یک موتور JavaScript است که Mozilla ساخته.

۳) یک مرورگر مخصوص سرورهاست.

۴) یک محیط اجراست که با موتور V8 و System APIها، JavaScript را بیرون از مرورگر اجرا می‌کند.

۴۲ متوسط

جاهای خالی نمودار اسلاید ۱۱ را پر کن:

«مرورگر = V8 + ... (Client Side) و Node.js = V8 + ... (Server Side)»

۱) Web APIs ــ System APIs

۲) System APIs ــ Web APIs

۳) Babel ــ DOM

۴) SpiderMonkey ــ Web APIs

۴۳ متوسط

طبق اسلاید ۱۱، اگر محیط اجرا هیچ APIای در اختیار JavaScript نگذارد، JavaScript به‌تنهایی چه کاری می‌تواند بکند؟

۱) فقط می‌تواند صفحه را تغییر دهد، ولی نمی‌تواند داده بگیرد.

۲) فقط می‌تواند پیام نمایش دهد.

۳) کار زیادی جز محاسبات (کار با عدد، رشته و داده‌ها در حافظه) نمی‌تواند بکند.

۴) همه‌ی کارها را می‌تواند، چون APIها بخشی از خود زبان‌اند.

۴۴ متوسط

کدام مورد جزو **خود زبان** JavaScript نیست و محیط اجرا (مرورگر) آن را فراهم می‌کند؟

۱) `Math.round(2.6)`

۲) `JSON.stringify([1, 2])`

۳) `"abc".toUpperCase()`

۴) `document.getElementById("btn")`

۴۵ متوسط

خروجی اجرای کد زیر با Node.js چیست؟

```JavaScript
console.log("first");
console.log(total);
console.log("last");
```

۱) `first` `Uncaught ReferenceError`

۲) `Uncaught ReferenceError`

۳) `first` `undefined` `last`

۴) `first` `total` `last`

۴۶ سخت

خروجی اجرای کد زیر با Node.js چیست؟ (دقت کن که در خط ۳ پرانتز بسته جا افتاده است.)

```JavaScript
1  console.log("one");
2  console.log("two");
3  console.log("three";
```

۱) `one` `two` `Uncaught SyntaxError`

۲) `one` `two` `three`

۳) `one` `Uncaught SyntaxError`

۴) `Uncaught SyntaxError`

۴۷ سخت

خروجی اجرای کد زیر با **Node.js** چیست؟

```javascript
console.log(typeof Math, typeof console);
console.log(typeof document, typeof process);
```

۱) `object object` `object undefined`

۲) `object object` `undefined object`

۳) `object undefined` `undefined object`

۴) `object object` `Uncaught ReferenceError`

۴۸ سخت

کد زیر داخل یک تگ `<script>` در یک صفحه‌ی وب و در **مرورگر** اجرا می‌شود. خروجی Console چیست؟

```html
<h1>Hello</h1>
```

```javascript
console.log(typeof document, typeof window);
console.log(typeof process);
```

۱) `object object` `undefined`

۲) `undefined undefined` `object`

۳) `object object` `object`

۴) `object object` `Uncaught ReferenceError`

۴۹ سخت

کد زیر را با Node.js اجرا می‌کنیم (`node app.js`). کدام خط خطا می‌دهد؟

```javascript
1  const max = Math.max(3, 9, 4);
2  console.log("max:", max);
3  const data = JSON.stringify({ max: max });
4  console.log(data);
5  document.title = data;
```

۱) خط ۱

۲) خط ۳

۳) خط ۵

۴) هیچ خطایی رخ نمی‌دهد

۵۰ سخت

کدام جمله **نادرست** است؟

۱) Chrome و Node.js هر دو از موتور V8 استفاده می‌کنند.

۲) DOM بخشی از خود زبان JavaScript است؛ برای همین در Node.js هم `document` در دسترس است.

۳) مرورگر Web APIها را و Node.js، System APIها را در اختیار JavaScript می‌گذارد.

۴) موتور JavaScript کد را parse می‌کند و آن را به زبان ماشین کامپایل می‌کند.

۱-۴ Console API و محل نوشتن کد JavaScript

منبع: Introduction to JS، اسلایدهای ۱۲ تا ۱۴

درسنامه

وقتی برنامه می‌نویسی، باید بتوانی ببینی «داخلش چه خبر است»: این متغیر الان چه مقداری دارد؟ این خط اصلاً اجرا شد؟ ابزار این کار **Console** است. در این درس هم Console را می‌شناسیم و هم یاد می‌گیریم کد JavaScript را **کجا** بنویسیم و اجرا کنیم.

Console API چیست؟ (اسلاید ۱۲)

- یک ابزار **داخلی** (built-in) است که مرورگرها فراهم می‌کنند (بخشی از **DevTools**، یعنی ابزارهای توسعه‌دهنده‌ی مرورگر).
- بیشتر برای **debugging** (اشکال‌زدایی: پیدا کردن و رفع باگ‌ها) و **inspecting** values (بررسی مقدارها: دیدن اینکه داخل یک متغیر چیست) استفاده می‌شود.
- از طریق شیء **سراسری** (global؛ یعنی همه‌جا و بدون تعریف یا import در دسترس) به نام `console` به آن دسترسی داریم.
- **متد**های (method: تابعی که متعلق به یک شیء است و با نقطه صدا زده می‌شود) زیادی دارد: `log`، `info`، `warn`، `error`، `table` و ...
- **رایج‌ترین** و اولین متدی که استفاده می‌کنیم: `console.log()`.

> **یک تشبیه ساده**
>
> Console مثل **دفترچه‌ی گزارش** برنامه است. هر جا بنویسی `console.log(x)`، برنامه در آن لحظه مقدار `x` را در دفترچه یادداشت می‌کند تا تو بخوانی. پیام‌ها را می‌شود با «برچسب‌های» مختلف نوشت: یادداشت معمولی، اطلاع‌رسانی، هشدار یا خطا.

Node.js هم شیء `console` را دارد؛ آنجا پیام‌ها در **ترمینال** چاپ می‌شوند.

فعالیت کلاسی اسلاید ۱۲: هر خط چه می‌کند؟

اسلاید ۱۲ این کد را نشان می‌دهد و می‌پرسد «What does each line do?»:

console.js

```
console.log("Hello, world!");

console.info("THIS IS AN INFO MESSAGE.");

console.warn("YOU WERE WARNED WHAT DEFEAT WOULD BRING!!!");

console.error("AH DOODOO, HERE WE GO AGAIN!");
```

خط	متد	کارش	ظاهرش در Console مرورگر (تصویر اسلاید ۱۴، Firefox)
۱	`console.log`	یک پیام معمولی چاپ می‌کند	متن ساده، بدون آیکون
۳	`console.info`	یک پیام اطلاع‌رسانی (info) چاپ می‌کند	آیکون کوچک «i» کنار پیام
۵	`console.warn`	یک **هشدار** (warning) چاپ می‌کند	پس‌زمینه‌ی **زرد** و آیکون مثلث هشدار
۷	`console.error`	یک پیام **خطا** (error) چاپ می‌کند	پس‌زمینه‌ی **قرمز**، آیکون علامت تعجب و یک فلش کوچک که با کلیک روی آن **stack trace** باز می‌شود

Stack trace یعنی فهرست تابع‌هایی که پشت سر هم صدا زده شدند تا برنامه به این خط رسید؛ برای پیدا کردن محل باگ خیلی به کار می‌آید. بالای Console هم دکمه‌های فیلتر **Logs**، **Info**، **Warnings**، **Errors** و **Debug** هست تا فقط نوع خاصی از پیام‌ها را ببینی.

اگر همین فایل را با Node.js اجرا کنی (`node console.js`)، هر چهار پیام به همان ترتیب در ترمینال چاپ می‌شوند، ولی بدون آیکون و پس‌زمینه‌ی رنگی:

```
Hello, world!
THIS IS AN INFO MESSAGE.
YOU WERE WARNED WHAT DEFEAT WOULD BRING!!!
AH DOODOO, HERE WE GO AGAIN!
```

> ⚠ **هشدار**
>
> `console.error` فقط پیامی **با ظاهر خطا** نشان می‌دهد؛ خودش خطا ایجاد نمی‌کند و برنامه را **متوقف نمی‌کند**. در کد زیر هر دو پیام چاپ می‌شوند:
>
> ```JavaScript
> console.error("Something looks wrong!");
> console.log("...but the program keeps running");
> ```

> ⓘ **بیشتر بدانیم**
>
> در Node.js، `console.log` و `console.info` در «خروجی استاندارد» (stdout) و `console.warn` و `console.error` در «خروجی خطای استاندارد» (stderr) نوشته می‌شوند. در ترمینال هر دو را کنار هم می‌بینی؛ فرقشان وقتی معلوم می‌شود که خروجی برنامه را به یک فایل هدایت کنی.

`console.table` : نمایش جدولی

`console.table` یک آرایه یا شیء را به شکل **جدول** نشان می‌دهد؛ برای آرایه‌ای از اشیاء خیلی خواناتر از `console.log` است:

```JavaScript
const students = [
  { name: "Sara", score: 18 },
  { name: "Ali", score: 15 },
];
console.table(students);
```

خروجی در Node.js (در مرورگر هم یک جدول واقعی با همین ستون‌ها نشان داده می‌شود):

```
┌─────────┬────────┬───────┐
│ (index) │ name   │ score │
├─────────┼────────┼───────┤
│ 0       │ 'Sara' │ 18    │
│ 1       │ 'Ali'  │ 15    │
└─────────┴────────┴───────┘
```

ستون `(index)` شماره‌ی هر عضو آرایه است (از صفر شروع می‌شود) و بقیه‌ی ستون‌ها کلیدهای اشیاء هستند.

چاپ چند مقدار با `console.log`

JavaScript

```
console.log("Name:", "Sara", "Age:", 20); // Name: Sara Age: 20
console.log("Age:" + 20);                 // Age:20
console.log(2 + 3, "2" + "3");            // 5 23
console.log("2 + 3");                     // 2 + 3
console.log(5, "5");                      // 5 5
console.log(["a", 1]);                    // [ 'a', 1 ]
console.log();                            // (prints an empty line)
```

قاعده‌ها:

- اگر چند مقدار را با **کاما** بدهی، همه در یک خط و با **یک فاصله** بینشان چاپ می‌شوند.
- `+` بین دو رشته آن‌ها را **بدون فاصله** به هم می‌چسباند.
- اول عبارتِ داخل پرانتز **محاسبه** می‌شود و بعد **نتیجه‌اش** چاپ می‌شود (`2 + 3` می‌شود `5`)؛ اما هر چیزی داخل کوتیشن فقط **متن** است (`"2 + 3"` همان‌طور چاپ می‌شود).
- رشته‌ها بدون کوتیشن چاپ می‌شوند؛ برای همین `5` و `"5"` در خروجی **یکسان** دیده می‌شوند! فقط وقتی رشته داخل آرایه یا شیء باشد، Node.js آن را با کوتیشن تکی نشان می‌دهد: `[ 'a', 1 ]`. (ظاهر آرایه‌ها و اشیاء در Console مرورگر کمی فرق دارد؛ در این کتاب خروجی‌ها مطابق Node.js نوشته شده‌اند.)
- `console.log()` بدون ورودی، یک خط خالی چاپ می‌کند.

مقدار برگشتی متدهای console

هر تابعی بعد از اجرا یک مقدار **برمی‌گرداند** (return value). متدهای console فقط چیزی **چاپ** می‌کنند و **هیچ مقداری برنمی‌گردانند** (console methods return nothing)؛ در JavaScript «هیچ مقدار» یعنی `undefined`:

JavaScript

```
console.log(console.log("hi"));
// hi
// undefined
```

قدم‌به‌قدم: برای اجرای `console.log` بیرونی، اول باید ورودی‌اش معلوم شود؛ پس `console.log("hi")` درونی اجرا می‌شود، `hi` را چاپ می‌کند و `undefined` برمی‌گرداند. حالا `console.log` بیرونی همان `undefined` را چاپ می‌کند. بقیه‌ی متدها هم همین‌طورند:

JavaScript

```
const result = console.info("Saving...");
console.log(result); // undefined
```

کد JavaScript را کجا بنویسیم؟ (اسلاید ۱۳)

اسلاید ۱۳ («But Where Do I Code?») با تصویر آدمکی که پشت کامپیوتر نشسته و با تردید فکر می‌کند «خب، کدم را کجا بنویسم؟»، چهار راه را نام می‌برد.

راه اول: Console مرورگر (Browser Console)

- **DevTools** را باز کن (در ویندوز و لینوکس معمولاً با کلید F12، یا با راست‌کلیک روی صفحه و انتخاب Inspect).
- به تب **Console** برو.
- کد JavaScript را **مستقیماً** تایپ کن و Enter بزن.

راه دوم: داخل فایل HTML (Inside HTML)

کد را بین تگ‌های `<script>` و `</script>` بنویس:

index.html

```
<!DOCTYPE html>
<html>
  <head>
    <title>My first JS page</title>
  </head>
  <body>
    <h1>Hello!</h1>
    <script>
      console.log("Hello from an inline script!");
    </script>
  </body>
</html>
```

با باز کردن صفحه در مرورگر، پیام `Hello from an inline script!` در Console چاپ می‌شود.

راه سوم: فایل JavaScript جداگانه (External JS File)

- یک فایل با پسوند `.js` بساز (مثلاً `app.js`).
- آن را با `<script src="...">` به صفحه وصل کن؛ **درست مثل کاری که برای CSS** با تگ `<link>` انجام می‌دادی:

index.html

```
<!DOCTYPE html>
<html>
  <head>
    <title>My first JS page</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <h1>Hello!</h1>
    <script src="app.js"></script>
  </body>
</html>
```

app.js

```
console.log("Hello from app.js!");
```

مزیت فایل جداگانه: HTML (ساختار) و JavaScript (رفتار) از هم جدا می‌مانند، یک فایل را می‌شود در چند صفحه استفاده کرد و مرورگر می‌تواند آن را cache کند (برای دفعه‌های بعد نگه دارد). اینکه تگ `<script>` را دقیقاً کجای صفحه بگذاریم هم مهم است؛ در فصل ۲ می‌بینیم چرا معمولاً آن را انتهای `<body>` می‌گذارند.

راه چهارم: محیط‌های آنلاین (Online Playgrounds)

سایت‌هایی مثل **JSFiddle**، **CodePen** و **StackBlitz** به تو اجازه می‌دهند بدون نصب هیچ برنامه‌ای، در خود مرورگر HTML و CSS و JavaScript بنویسی، فوراً نتیجه را ببینی و کدت را با یک لینک به دیگران نشان دهی.

و البته Node.js

طبق اسلاید ۵، JavaScript روی کامپیوتر خودت هم با **Node.js** اجرا می‌شود (فصل ۳). فایل را در ترمینال اجرا می‌کنی و خروجی `console.log` در همان **ترمینال** چاپ می‌شود:

Terminal

```
node app.js
# Hello from app.js!
```

روش	کجا می‌نویسی؟	مناسب برای
Console مرورگر	تب Console در DevTools	امتحان سریع چند خط کد و یادگیری
داخل HTML	بین `<script>` و `</script>`	کدهای کوتاهِ مخصوص همان صفحه
فایل جداگانه	فایل `.js` و تگ `<script src="...">`	پروژه‌های واقعی
محیط آنلاین	CodePen، JSFiddle، StackBlitz	تمرین و اشتراک‌گذاری بدون نصب
Node.js	فایل `.js` و دستور `node` در ترمینال	اجرای JavaScript بیرون از مرورگر

امتحان کد در Console (اسلاید ۱۴)

- برای سادگی، **فعلاً** کدهایمان را در Console ابزار DevTools اجرا می‌کنیم.
- Console کد را **خط‌به‌خط** اجرا می‌کند؛ **مثل Python shell**. به این نوع محیط **REPL** هم می‌گویند (Read–Eval–Print Loop: یک خط را بخوان، اجرا کن، نتیجه را نشان بده و دوباره از اول).
- این روش برای **یادگیری مبانی** JavaScript عالی است.

تصویر اسلاید ۱۴، همان کد اسلاید ۱۲ را در Console مرورگر Firefox نشان می‌دهد (`>>` یعنی خطی که ما تایپ کرده‌ایم و `<-` یعنی مقداری که Console برگردانده است):

Firefox-Console

```
>> console.log("Hello, world!");
   Hello, world!
<- undefined
>> console.info("THIS IS AN INFO MESSAGE.");
   THIS IS AN INFO MESSAGE.                    (info icon)
<- undefined
>> console.warn("YOU WERE WARNED WHAT DEFEAT WOULD BRING!!!");
   YOU WERE WARNED WHAT DEFEAT WOULD BRING!!!   (yellow warning)
<- undefined
>> console.error("AH DOODOO, HERE WE GO AGAIN!");
   AH DOODOO, HERE WE GO AGAIN!                 (red error + stack trace)
<- undefined
```

دو نکته از این تصویر:

- انواع پیام‌ها **ظاهر متفاوتی** دارند (اطلاع‌رسانی، هشدار زرد، خطای قرمز).
- بعد از هر دستور یک `undefined` دیده می‌شود. این خطا نیست! Console بعد از اجرای هر خط، **مقدار برگشتی** آن را نشان می‌دهد و چون متدهای console چیزی برنمی‌گردانند، این مقدار `undefined` است.

Console مقدار هر عبارت را نشان می‌دهد

قاعده‌ی کلی: Console بعد از هر خط، **مقدار** آن خط را نشان می‌دهد، حتی اگر `console.log` ننوشته باشی. همین را در تصویر Console اسلاید ۲۳ هم می‌بینیم:

Firefox-Console

```
>> let a = 12
<- undefined
>> a = "Hello!"
<- "Hello!"
>> console.log(a)
   Hello!
<- undefined
>> 2 + 3
<- 5
```

- `let a = 12` یک **تعریف** متغیر است و مقداری ندارد؛ پس `undefined` .
- `a = "Hello!"` یک **انتساب** است و مقدارش همان مقدار جدید است؛ پس Console همان `"Hello!"` را (با کوتیشن، چون یک رشته است) نشان می‌دهد.
- `console.log(a)` متن `Hello!` را (بدون کوتیشن) **چاپ** می‌کند و بعد مقدار برگشتی‌اش، یعنی `undefined` ، نشان داده می‌شود.
- `2 + 3` یک عبارت است و Console نتیجه‌اش (`5`) را نشان می‌دهد.

REPL ِNode.js (با اجرای دستور `node` بدون نام فایل) هم دقیقاً همین رفتار را دارد؛ فقط رشته‌ها را با کوتیشن تکی نشان می‌دهد (`'Hello!'`).

> **نکته**
> نمایش خودکار مقدارها فقط مخصوص Console و REPL است. در یک **فایل** `.js` ، خطی مثل `"Hello!";` یا `2 + 3;` **هیچ چیزی چاپ نمی‌کند**؛ برای دیدن خروجیِ یک فایل حتماً باید `console.log` بنویسی.

⚠ دام تستی

- `undefined` بعد از `console.log` در Console مرورگر **خطا نیست** و **چاپ** خود `console.log` هم نیست؛ مقدار برگشتی آن است. اما `console.log(console.log("hi"))` واقعاً `undefined` را **چاپ** می‌کند: اول `hi` و بعد `undefined`.
- `console.error` و `console.warn` برنامه را **متوقف نمی‌کنند**؛ فقط پیام را با ظاهر متفاوت نشان می‌دهند.
- `console.log("2 + 3")` متن `2 + 3` را چاپ می‌کند، نه `5`.
- JavaScript به **بزرگی و کوچکی حروف حساس** است (case-sensitive): `Console.log("hi")` خطای `ReferenceError: Console is not defined` و `console.Log("hi")` خطای `TypeError: console.Log is not a function` می‌دهد. متدی به اسم `console.print` هم وجود ندارد.
- وقتی تگ `<script>` ویژگی `src` دارد، کدی که **بین** `<script src="app.js">` و `</script>` بنویسی **اجرا نمی‌شود** و فقط فایل `app.js` اجرا می‌شود. برای کد درون‌خطی یک تگ `<script>` جداگانه بگذار.
- تگ `<script>` را همیشه با `</script>` ببند. شکل `<script src="app.js" />` در HTML کار نمی‌کند: مرورگر آن را یک تگ باز حساب می‌کند و محتوای بعدی صفحه را تا اولین `</script>` جزو همین تگ می‌گیرد.

✓ جمع‌بندی

- Console یک ابزار داخلی مرورگر برای debugging و بررسی مقدارهاست و با شیء سراسری `console` در دسترس است.
- `log` پیام معمولی، `info` پیام اطلاع‌رسانی، `warn` هشدار زرد، `error` خطای قرمز با stack trace و `table` نمایش جدولی.
- متدهای console مقداری برنمی‌گردانند (`undefined`)؛ برای همین Console بعد از هر کدام `undefined` نشان می‌دهد.
- محل نوشتن کد: Console مرورگر، تگ `<script>` در HTML، فایل `.js` جداگانه با `<script src="...">`، محیط‌های آنلاین (CodePen، JSFiddle، StackBlitz) و Node.js در ترمینال.
- Console مثل Python shell خط‌به‌خط اجرا می‌کند و مقدار هر عبارت را نشان می‌دهد؛ ولی در یک فایل فقط `console.log` چیزی چاپ می‌کند.

✓ تست‌های تمرینی

۱۸ تست · شماره‌ی ۵۱ تا ۶۸

۵۱ آسان

`console` در JavaScript چیست؟

۱) یک شیء سراسری (global) که متدهایی مثل `log` و `warn` دارد

۲) یک کلمه‌ی کلیدی (keyword) زبان، مثل `let`

۳) نام فایلی که باید با `<script src>` به صفحه اضافه شود

۴) یک تگ HTML، مثل `<script>`

۵۲ آسان

طبق اسلاید ۱۲، Console مرورگر بیشتر برای چه کاری استفاده می‌شود؟

۱) طراحی ظاهر صفحه

۲) اشکال‌زدایی (debugging) و بررسی مقدارها (inspecting values)

۳) ذخیره‌ی دائمی اطلاعات کاربر

۴) ارسال صفحه به سرور

۵۳ آسان

کدام متد، داده‌ها (مثلاً آرایه‌ای از اشیاء) را به شکل **جدول** نمایش می‌دهد؟

۱) `console.log` ۲) `console.info` ۳) `console.grid` ۴) `console.table`

۵۴ متوسط

مطابق تصویر اسلاید ۱۴ (Console مرورگر Firefox)، کدام توصیف **درست** است؟

۱) `console.warn` با پس‌زمینه‌ی قرمز و `console.error` با پس‌زمینه‌ی زرد نمایش داده می‌شود.

۲) همه‌ی متدهای console دقیقاً یک‌شکل نمایش داده می‌شوند و فرقی با هم ندارند.

۳) `console.info` برنامه را متوقف می‌کند تا کاربر پیام را بخواند.

۴) `console.warn` با پس‌زمینه‌ی زرد و `console.error` با پس‌زمینه‌ی قرمز و امکان باز کردن stack trace نمایش داده می‌شود.

۵۵ متوسط

خروجی کد زیر چیست؟

```javascript
console.log("Name:", "Sara", 20);
console.log("Age:" + 20);
```

۱) `Name: Sara 20` `Age: 20`

۲) `Name:Sara20` `Age:20`

۳) `Name: Sara 20` `Age:20`

۴) `Name: Sara, 20` `Age:20`

۵۶ متوسط

خروجی کد زیر چیست؟

```javascript
console.log(2 + 3);
console.log("2" + "3");
console.log("2 + 3");
```

۱) `5` `5` `5`

۲) `5` `23` `5`

۳) `5` `"23"` `"2 + 3"`

۴) `5` `23` `2 + 3`

۵۷ سخت

خروجی کد زیر چیست؟

```javascript
console.log(console.log("hi"));
```

۱) `hi` `hi`
۲) `undefined` `hi`
۳) `hi` `undefined`
۴) `hi`

۵۸ سخت

خروجی کد زیر چیست؟

```javascript
const x = console.log("x") + 1;
console.log(x);
```

۱) `x` `x1`
۲) `x` `NaN`
۳) `x` `1`
۴) `NaN`

۵۹ متوسط

کدام دستور **بدون خطا** اجرا می‌شود؟

۱) `Console.log("hi");`
۲) `console.Log("hi");`
۳) `console.print("hi");`
۴) `console.log("hi");`

۶۰ سخت

کد زیر را با Node.js اجرا می‌کنیم. کدام خط باعث توقف برنامه با خطا می‌شود؟

```javascript
1  console.log("start");
2  console.warn("careful!");
3  console.error("something went wrong");
4  console.log(total);
```

۱) خط ۲
۲) خط ۳
۳) خط ۴
۴) هیچ خطایی رخ نمی‌دهد

۶۱ آسان

کدام مورد جزو روش‌های نوشتن و اجرای کد JavaScript که در اسلاید ۱۳ آمده **نیست**؟

۱) نوشتن کد در یک فایل `.css` و وصل کردن آن با تگ `<link>`
۲) تایپ مستقیم کد در تب Console ابزار DevTools مرورگر
۳) نوشتن کد بین تگ‌های `<script>` و `</script>` در فایل HTML
۴) استفاده از محیط‌های آنلاین مثل JSFiddle، CodePen و StackBlitz

۶۲ متوسط

برای وصل کردن فایل `app.js` به یک صفحه‌ی HTML، کدام تگ درست است؟

۱) `<link rel="script" href="app.js">`
۲) `<script href="app.js"></script>`
۳) `<script src="app.js"></script>`
۴) `<js src="app.js"></js>`

۶۳ سخت

با باز کردن `index.html` در مرورگر، چه چیزی در Console چاپ می‌شود؟

index.html

```
<!DOCTYPE html>
<html>
  <body>
    <script src="app.js">
      console.log("inline");
    </script>
  </body>
</html>
```

app.js

```
console.log("external");
```

۱) `inline` `external`
۲) `external`
۳) `inline`
۴) `external` `inline`

۶۴ آسان

JSFiddle، CodePen و StackBlitz چه هستند؟

۱) موتورهای JavaScript مرورگرهای مختلف
۲) ابزارهای transpile، مثل Babel
۳) محیط‌های آنلاین (playground) که می‌توان در خود مرورگر کد نوشت و فوراً نتیجه را دید
۴) افزونه‌هایی که JavaScript را در Node.js فعال می‌کنند

۶۵ متوسط

در Console مرورگر عبارت `console.log("Hello")` را تایپ می‌کنیم و Enter می‌زنیم. چه چیزی نمایش داده می‌شود؟

۱) فقط `Hello`
۲) فقط `undefined`
۳) `Hello` و سپس پیام خطا، چون `console.log` چیزی برنمی‌گرداند
۴) `Hello` و سپس یک خط `undefined` که مقدار برگشتی `console.log` است

۶۶ سخت

این سه خط را **یکی‌یکی** در Console مرورگر تایپ می‌کنیم (بعد از هر خط Enter). Console بعد از هر خط چه نشان می‌دهد؟

```JavaScript
let a = 12
a = "Hello!"
console.log(a)
```

۱) خط ۱: `undefined` ــ خط ۲: `"Hello!"` ــ خط ۳: چاپ `Hello!` و سپس `undefined`

۲) خط ۱: `12` ــ خط ۲: `"Hello!"` ــ خط ۳: فقط چاپ `Hello!`

۳) خط ۱: `undefined` ــ خط ۲: `undefined` ــ خط ۳: چاپ `Hello!` و سپس `undefined`

۴) خط ۱: `12` ــ خط ۲: چاپ `Hello!` ــ خط ۳: `"Hello!"`

۶۷ سخت

فایل `app.js` زیر را با دستور `node app.js` اجرا می‌کنیم. خروجی چیست؟

```JavaScript
2 + 3;
let a = 12;
a = "Hello!";
console.log(a);
```

۱) `Hello!`

۲) `5` `undefined` `Hello!` `Hello!`

۳) `5` `Hello!`

۴) `"Hello!"`

۶۸ آسان

وقتی فایلی را با دستور `node app.js` اجرا می‌کنیم، خروجی `console.log` کجا نمایش داده می‌شود؟

۱) در Console مرورگر

۲) در ترمینال؛ همان پنجره‌ای که دستور را در آن اجرا کردیم

۳) در فایلی به نام `app.log`

۴) هیچ‌جا؛ چون `console` فقط در مرورگر وجود دارد

۱-۵ متغیرها: let، const و var

منبع: Introduction to JS، اسلایدهای ۲۳ و ۲۴

درسنامه

فرض کن در آشپزخانه چند ظرف داری و روی هر کدام یک برچسب چسبانده‌ای: «شکر»، «نمک»، «برنج». هر وقت شکر لازم داشته باشی، سراغ ظرفی می‌روی که برچسب «شکر» دارد. در برنامه‌نویسی، **متغیر (variable)** دقیقاً همین ظرف برچسب‌دار است: یک **نام** دارد (برچسب) و یک **مقدار** (چیزی که داخلش ریخته‌ایم).

اسلاید ۲۳ همین را این‌طور می‌گوید:

- متغیرها **ظرف‌هایی برای نگه‌داری داده** هستند (Variables are containers for storing data).
- با یکی از سه کلمه‌ی کلیدی `let`، `const` یا کلمه‌ی قدیمی‌تر `var` تعریف می‌شوند.
- `let` برای **مقدارهای قابل‌تغییر** است و `const` برای **مقدارهای ثابت** (که نمی‌شود دوباره به آن‌ها مقدار داد).
- `var` **قدیمی (outdated)** و **function-scoped** (محدوده‌اش کل تابع است، نه یک بلوک) است و نباید در کد مدرن از آن استفاده کرد.
- متغیرها به ما اجازه می‌دهند داده‌ها را بارها **استفاده** و **دست‌کاری** کنیم.

تعریف، انتساب و مقداردهی اولیه

```JavaScript
let city;                // declaration: a box named "city" is created (value: undefined)
city = "Tehran";         // assignment: put a value into the box
let year = 2026;         // declaration + initialization in one line
console.log(city, year); // Tehran 2026
```

این سه اصطلاح را از هم جدا کن:

اصطلاح	یعنی چه؟	مثال
declaration (اعلان یا تعریف)	ساختن متغیر و انتخاب نام برای آن	`let city;`
assignment (انتساب)	ریختن یک مقدار در متغیر با عملگر `=`	`city = "Tehran";`
initialization (مقداردهی اولیه)	تعریف و اولین انتساب، با هم در یک خط	`let year = 2026;`

اگر متغیری را تعریف کنی ولی مقداری در آن نریزی، مقدارش `undefined` (یعنی «هنوز مقدار نگرفته») است:

```JavaScript
let score;
console.log(score); // undefined
score = 10;
console.log(score); // 10
```

`let` : متغیرِ قابل‌تغییر

با `let` متغیری می‌سازی که هر وقت خواستی می‌توانی مقدار تازه‌ای در آن بریزی. به این کار **reassignment** (انتساب دوباره) می‌گویند. این کد اسلاید ۲۴ است:

letconst.js

```
let age = 21;

age = 22;  // works

const pi = 3.14;

pi = 3.1415;  // error
```

خط `age = 22` بی‌دردسر اجرا می‌شود، ولی خط آخر در Node.js و Chrome این خطا را می‌دهد و برنامه همان‌جا متوقف می‌شود:

```
TypeError: Assignment to constant variable.
```

`const` : متغیرِ ثابت

`const` (مخفف constant یعنی «ثابت») متغیری می‌سازد که **بعد از مقداردهی، دیگر نمی‌شود به آن مقدار تازه داد.** هر تلاشی برای انتساب دوباره، چه با `=` و چه با `+=` یا `++`، خطای `TypeError` (خطای نوع) می‌دهد.

دو قانون مهم `const`:

1. **باید همان لحظه‌ی تعریف مقدار بگیرد.** چون بعداً نمی‌شود به آن مقدار داد، تعریفِ خالیِ آن بی‌معنی است:

JavaScript

```
const z; // SyntaxError: Missing initializer in const declaration
```

این خطا از نوع `SyntaxError` (خطای نحوی یا دستوری) است و **قبل از اجرای برنامه** پیدا می‌شود؛ یعنی وقتی در یک فایل باشد، **هیچ خطی از آن فایل اجرا نمی‌شود**، حتی خط‌های قبل از آن.

2. **فقط «اتصال» ثابت است، نه محتوای شیء.** اگر در یک `const` یک شیء (object) یا آرایه (array) بگذاری، نمی‌توانی متغیر را به شیء یا آرایه‌ی دیگری وصل کنی، ولی **می‌توانی محتوای همان شیء را تغییر دهی:**

JavaScript

```
const user = { name: "Ali" };
user.name = "Sara";   // OK: we change the content of the same object
const nums = [1, 2];
nums.push(3);         // OK: nums is now [1, 2, 3]
nums = [4, 5];        // TypeError: Assignment to constant variable.
```

یک تشبیه ساده

`const` مثل برچسبی است که با چسب قطره‌ای روی یک جعبه چسبانده شده: دیگر نمی‌توانی آن را بکَنی و روی جعبه‌ی دیگری بزنی (انتساب دوباره ممنوع است)، ولی اگر جعبه در داشته باشد (شیء یا آرایه)، هنوز می‌توانی وسایل داخلش را جابه‌جا کنی.

دموی کنسول اسلاید ۲۳

در اسلاید ۲۳ این دستورها یکی‌یکی در کنسول Firefox تایپ شده‌اند. علامت `»` یعنی چیزی که ما تایپ کردیم و `←` یعنی مقداری که کنسول به‌عنوان «نتیجه‌ی آن دستور» نشان می‌دهد:

```
» let a = 12
← undefined
» const b = 13
← undefined
» b = 12
Uncaught TypeError: invalid assignment to const 'b'
» a = "Hello!"
← "Hello!"
» console.log(a)
  Hello!
← undefined
```

بیا خط‌به‌خط بخوانیم:

- `let a = 12` و `const b = 13` : یک **تعریف** مقداری تولید نمی‌کند، پس کنسول `undefined` نشان می‌دهد. این یعنی «این دستور نتیجه‌ای نداشت»، **نه** این‌که مقدار `a` برابر `undefined` شده باشد. مقدار `a` همان `12` است.
- `b = 12` : انتساب دوباره به یک `const` ، پس `TypeError` . در Firefox (موتور SpiderMonkey) متن پیام `invalid assignment to const 'b'` است و در Chrome و Node.js (موتور V8) `Assignment to constant variable.` . **متن پیام به موتور بستگی دارد، ولی نوع خطا همیشه** `TypeError` **است.**
- `a = "Hello!"` : هر **انتساب** خودش یک عبارت (expression، یعنی تکه‌کدی که مقدار دارد) است و مقدارش همان چیزی است که منتسب شد؛ برای همین کنسول `"Hello!"` را نشان می‌دهد. این خط ضمناً نشان می‌دهد متغیر `let` که تا الان عدد بود، حالا رشته (string) شده است.
- `console.log(a)` : متن `Hello!` را **چاپ** می‌کند و چون خود `console.log` چیزی برنمی‌گرداند، بعدش `undefined` می‌آید.

> **یادداشت**
>
> کنسول مرورگر هر خط را جداگانه اجرا می‌کند؛ برای همین بعد از خطای خط سوم، دستورهای بعدی هم اجرا شدند. اما اگر همین پنج خط را در **یک فایل** بنویسی و با Node.js اجرا کنی، برنامه روی خط `b = 12` متوقف می‌شود و خط‌های بعدی اصلاً اجرا نمی‌شوند.

`var` : روش قدیمی (پیش‌نمایش)

قبل از ES6 (سال ۲۰۱۵) تنها راه تعریف متغیر `var` بود. اسلاید ۲۳ می‌گوید `var` قدیمی و **function-scoped** است؛ یعنی محدوده‌ی (scope) آن کل تابع است، نه بلوک `{ }` . فعلاً همین دو رفتار عجیبش را بدان:

JavaScript

```
var color = "red";
var color = "blue";  // var allows redeclaration: no error
console.log(color);  // blue

{
  var leaked = "I escape the block";
}
console.log(leaked); // I escape the block (var ignores { } blocks)
```

جزئیات کامل مشکلات `var` و **hoisting** (بالا بردن اعلان‌ها به ابتدای محدوده) را در درس‌های «Scope»، «مشکلات var» و «Hoisting و TDZ» می‌بینیم.

let و const هر دو block-scoped هستند (پیش‌نمایش)

اسلاید ۲۴ می‌گوید let و const هر دو **block-scoped** هستند؛ یعنی فقط داخل همان { ... } (بلوکی) که در آن تعریف شده‌اند وجود دارند. بیرون از بلوک، استفاده از آن‌ها خطای ReferenceError (خطای ارجاع به متغیری که وجود ندارد) می‌دهد:

```JavaScript
{
  let inside = "hi";
  console.log(inside); // hi
}
console.log(inside);   // ReferenceError: inside is not defined
```

تعریف دوباره (redeclaration) در برابر انتساب دوباره (reassignment)

این دو را با هم قاطی نکن. **انتساب دوباره** یعنی ریختن مقدار تازه در متغیری که از قبل هست (x = 2). **تعریف دوباره** یعنی نوشتن دوباره‌ی let x یا const x برای نامی که در همان محدوده از قبل تعریف شده است.

	let	const	var
انتساب دوباره (x = 2)	مجاز	TypeError	مجاز
تعریف دوباره در همان محدوده	SyntaxError	SyntaxError	مجاز (بی‌صدا)
مقدار دادن هنگام تعریف	اختیاری	**اجباری**	اختیاری
محدوده (scope)	بلوک { }	بلوک { }	تابع

```JavaScript
let total = 5;
total = 10;     // reassignment: fine
let total = 20; // redeclaration: SyntaxError: Identifier 'total' has already been declared
```

قوانین نام‌گذاری متغیرها

- نام می‌تواند از **حروف**، **رقم‌ها**، $ و _ ساخته شود: price ، $price ، _count ، item2 .
- نام **نمی‌تواند با رقم شروع شود**: 2ndPlace غلط است.
- فاصله و خط تیره مجاز نیست: user name و first-name غلط‌اند (خط تیره همان عملگر منها است!).
- **کلمه‌های رزروشده (reserved words)** مثل class ، const ، function ، return ، if ، new و typeof را نمی‌توان نام متغیر گذاشت.
- JavaScript **به بزرگی و کوچکی حروف حساس است (case-sensitive)**: userName و username دو متغیر کاملاً جدا هستند.
- **قرارداد (convention)**: در JavaScript نام متغیرها را به سبک **camelCase** می‌نویسیم؛ کلمه‌ی اول با حرف کوچک و هر کلمه‌ی بعدی با حرف بزرگ شروع می‌شود: firstName ، totalPrice ، isLoggedIn . سبک PascalCase (مثل TotalPrice) معمولاً برای نام کلاس‌ها است و snake_case (مثل total_price) سبک رایج Python است. این‌ها خطا نمی‌دهند؛ فقط قرارداد نیستند.

Dynamic typing (نوع‌دهی پویا): پیش‌نمایش

در زبان‌هایی مثل Java یا C باید نوع متغیر را بنویسی (`int x = 5;`) و بعداً نمی‌توانی در آن رشته بریزی. در JavaScript متغیرها **نوع ثابت ندارند**؛ نوع به **مقدار** تعلق دارد، نه به متغیر. همان متغیر `let` می‌تواند اول عدد و بعد رشته باشد:

```JavaScript
let data = 42;
console.log(typeof data); // number
data = "forty-two";
console.log(typeof data); // string
```

(عملگر `typeof` نوع یک مقدار را به‌صورت یک رشته برمی‌گرداند؛ در درس بعد کامل با آن آشنا می‌شویم.)

بهترین روش (BEST PRACTICE)

بنر اسلاید ۲۴ می‌گوید: **به‌طور پیش‌فرض از `const` استفاده کن و فقط وقتی به انتساب دوباره نیاز داری، سراغ `let` برو.** از `var` هم در کد جدید استفاده نکن. این‌طوری اگر جایی به‌اشتباه به متغیری مقدار تازه بدهی، همان لحظه خطا می‌گیری و باگ پنهان نمی‌ماند.

> ⚠ **هشدار**
>
> هیچ‌وقت متغیر را بدون `let` / `const` به کار نبر. اگر بنویسی `total = 5` و `total` قبلاً تعریف نشده باشد، در حالت عادی (sloppy) بی‌صدا یک متغیر سراسری (global) ساخته می‌شود و در حالت `"use strict"` خطای `ReferenceError` می‌گیری. خط `a = "Hello!"` در اسلاید ۲۳ مشکلی ندارد، چون `a` قبلاً با `let` تعریف شده بود.

> 💡 **نکته**
>
> - `const` یعنی «انتساب دوباره ممنوع»، نه «محتوای تغییرناپذیر». `push` روی آرایه‌ی `const` و تغییر ویژگی شیء `const` کاملاً مجاز است.
> - هر چیزی که مقدار تازه‌ای در متغیر می‌ریزد انتساب دوباره است: `x = ...` ، `x += 1` ، `x++` . همه‌ی این‌ها روی `const` خطای `TypeError` می‌دهند.
> - متغیری که فقط تعریف شده و مقدار نگرفته `undefined` است. این برای `let` و `var` ممکن است، ولی `const` بدون مقدار اصلاً مجاز نیست.

⚠ دام تستی

- «تعریف `const` بدون مقدار» و «تعریف دوباره‌ی `let` / `const` در همان محدوده» از نوع `SyntaxError` هستند و چون قبل از اجرا کشف می‌شوند، **هیچ خطی از آن فایل اجرا نمی‌شود**، حتی `console.log` های قبل از آن. ولی «انتساب دوباره به `const` » از نوع `TypeError` است و در **زمان اجرا** رخ می‌دهد؛ پس خط‌های قبل از آن اجرا شده‌اند و خروجی‌شان چاپ شده است.
- `undefined` ی که کنسول بعد از `let a = 12` نشان می‌دهد نتیجه‌ی **دستور** است، نه مقدار `a` .
- متن پیام خطا در موتورهای مختلف فرق دارد (`invalid assignment to const 'b'` در Firefox و `Assignment to constant variable.` در Chrome و Node.js)؛ در تست‌ها به **نوع خطا** یعنی `TypeError` دقت کن.
- `let x` داخل یک بلوک جدید `{ }` تعریف دوباره حساب نمی‌شود؛ آن یک متغیر **جدید و جدا** در محدوده‌ی همان بلوک است.

✓ جمع‌بندی

- متغیر یعنی ظرف داده با یک نام. تعریف با `const` (پیش‌فرض)، `let` (وقتی مقدار عوض می‌شود) و `var` (قدیمی؛ استفاده نکن).
- `let` : انتساب دوباره مجاز. `const` : انتساب دوباره `TypeError` و مقداردهی هنگام تعریف اجباری (وگرنه `SyntaxError`).
- `let` و `const` block-scoped هستند و تعریف دوباره‌شان در همان محدوده `SyntaxError` است؛ `var` تابع‌محور است و تعریف دوباره را اجازه می‌دهد.
- نام‌ها: حرف، رقم، `$` و `_` ؛ بدون رقم در ابتدا؛ حساس به حروف بزرگ و کوچک؛ بدون کلمه‌های رزروشده؛ به سبک camelCase.

✓ تست‌های تمرینی

۲۳ تست · شماره‌ی ۶۹ تا ۹۱

۶۹ آسان

کدام کلمه‌ی کلیدی متغیری می‌سازد که **نمی‌توان** به آن مقدار تازه‌ای منتسب کرد؟

۱) `let`　۲) `var`　۳) `final`　۴) `const`

۷۰ آسان

طبق اسلاید ۲۳، کدام توصیف درباره‌ی `var` درست است؟

۱) قدیمی است، function-scoped است و بهتر است در کد مدرن از آن استفاده نشود.

۲) جدیدترین روش تعریف متغیر است که در ES6 به زبان اضافه شد.

۳) مثل `const` است و نمی‌توان به آن مقدار تازه داد.

۴) block-scoped است و فقط داخل `{ ... }` وجود دارد.

۷۱ آسان

خروجی کد زیر چیست؟

```JavaScript
let age = 21;
age = 22;
age = age + 1;
console.log(age);
```

۱) `23`
۲) `21`
۳) `22`
۴) `Uncaught TypeError`

۷۲ آسان

خروجی کد زیر چیست؟

```JavaScript
let score;
console.log(score);
score = 10;
console.log(score);
```

۱) `undefined` `10`
۲) `null` `10`
۳) `0` `10`
۴) `Uncaught ReferenceError`

۷۳ آسان

کد زیر (همان کد اسلاید ۲۴) در Node.js اجرا می‌شود. کدام خط خطا می‌دهد؟

```JavaScript
1 let age = 21;
2 age = 22;
3 const pi = 3.14;
4 pi = 3.1415;
5 console.log(age, pi);
```

۱) خط ۲
۲) خط ۴
۳) خط ۵
۴) هیچ خطایی رخ نمی‌دهد

۷۴ آسان

طبق «BEST PRACTICE» اسلاید ۲۴، کدام روش برای تعریف متغیرها توصیه می‌شود؟

۱) همیشه `let`، چون انعطاف بیشتری دارد.
۲) به‌طور پیش‌فرض `const` و فقط وقتی انتساب دوباره لازم است `let`.
۳) همیشه `var`، چون در همه‌ی مرورگرها کار می‌کند.
۴) `const` فقط برای ثابت‌های ریاضی مثل عدد پی، و `let` برای بقیه‌ی متغیرها.

۷۵ متوسط

اجرای کدام قطعه‌کد **خطا** می‌دهد؟

۱) `const user = { name: "Ali" }; user.name = "Sara";`

۲) `const nums = [1, 2]; nums.push(3);`

۳) `let count = 1; count = count + 1;`

۴) `const city = "Tehran"; city = "Shiraz";`

۷۶ متوسط

خروجی کد زیر چیست؟

```JavaScript
const nums = [1, 2, 3];
nums.push(4);
nums[0] = 10;
console.log(nums);
```

۱) `[1, 2, 3]`

۲) `[1, 2, 3, 4]`

۳) `Uncaught TypeError`

۴) `[10, 2, 3, 4]`

۷۷ سخت

خروجی کد زیر در Node.js چیست؟

```JavaScript
const settings = { theme: "dark" };
settings.theme = "light";
console.log(settings.theme);
settings = { theme: "blue" };
console.log(settings.theme);
```

۱) `light` `blue`

۲) `dark` `Uncaught TypeError`

۳) `light` `Uncaught TypeError`

۴) `Uncaught TypeError`

۷۸ سخت

کدام خط کد زیر خطا می‌دهد؟

```JavaScript
1  let a = 1;
2  const b;
3  b = 2;
4  console.log(a, b);
```

۱) خط ۲

۲) خط ۳

۳) خط ۴

۴) هیچ خطایی رخ نمی‌دهد

۷۹ متوسط

در کنسول مرورگر (مثل اسلاید ۲۳) دستور `let a = 12` را وارد می‌کنیم. کنسول بلافاصله بعد از آن، به‌عنوان نتیجه‌ی این دستور چه چیزی نشان می‌دهد؟

۱) `12`　۲) `undefined`　۳) `null`　۴) `"a"`

۸۰ متوسط

در کنسول مرورگر ابتدا `let a = 12` و سپس `a = "Hello!"` را وارد می‌کنیم. کنسول بعد از خط دوم چه مقداری را به‌عنوان نتیجه نشان می‌دهد؟

۱) `"Hello!"`　۲) `undefined`　۳) `12`　۴) `true`

۸۱ سخت

در اسلاید ۲۳، Firefox برای `b = 12` (که قبلاً با `const` تعریف شده بود) پیام `invalid assignment to const 'b'` را نشان داد. اگر همین کار در Node.js انجام شود، کدام گزینه درست است؟

۱) خطایی رخ نمی‌دهد و مقدار `b` برابر `12` می‌شود.

۲) خطا از نوع `SyntaxError` است، چون به `const` اصلاً نمی‌شود چیزی منتسب کرد.

۳) نوع خطا همان `TypeError` است، ولی متن پیام فرق می‌کند: `Assignment to constant variable.`

۴) خطا از نوع `ReferenceError` است، چون متغیر `b` پیدا نمی‌شود.

۸۲ متوسط

کدام گزینه درست است؟

۱) `const` اجازه نمی‌دهد محتوای آرایه‌ای که در آن ذخیره شده تغییر کند.

۲) `let` و `const` هر دو block-scoped هستند؛ یعنی فقط داخل همان `{ ... }` که در آن تعریف شده‌اند وجود دارند.

۳) متغیر `let` فقط یک بار مقدار می‌گیرد و بعد از آن قفل می‌شود.

۴) `var` جدیدتر از `let` است و block-scoped است.

۸۳ سخت

خروجی کد زیر در Node.js چیست؟

```javascript
{
  var a = 1;
  let b = 2;
}
console.log(a);
console.log(b);
```

۱) `1` `2`　۲) `Uncaught ReferenceError`

۳) `undefined` `2`　۴) `1` `Uncaught ReferenceError`

۸۴ سخت

خروجی کد زیر چیست؟

```javascript
let level = "outer";
{
  let level = "inner";
  console.log(level);
}
console.log(level);
```

۱) `inner` `inner`
۲) `inner` `outer`
۳) `outer` `outer`
۴) `Uncaught SyntaxError`

۸۵ سخت

کدام خط کد زیر خطا می‌دهد؟

```javascript
1 var x = 1;
2 var x = 2;
3 let y = 1;
4 y = 2;
5 let y = 3;
6 console.log(x, y);
```

۱) خط ۲
۲) خط ۴
۳) خط ۵
۴) هیچ خطایی رخ نمی‌دهد

۸۶ سخت

کد زیر به‌صورت یک فایل با Node.js اجرا می‌شود. خروجی کدام است؟

```javascript
console.log("start");
let total = 5;
total = 10;
let total = 20;
console.log(total);
```

۱) `start` `20`
۲) `start` `Uncaught SyntaxError`
۳) `start` `10`
۴) `Uncaught SyntaxError`

۸۷ متوسط

کدام تعریف متغیر **بدون خطا** اجرا می‌شود؟

۱) `let first-name = "Ali";`
۲) `let 2ndPlace = "Sara";`
۳) `let class = "A1";`
۴) `let _total$ = 0;`

۸۸ متوسط

خروجی کد زیر چیست؟

```javascript
let userName = "Ali";
let username = "Reza";
userName = "Sara";
console.log(username);
```

۱) `Ali`
۲) `Reza`
۳) `Sara`
۴) `Uncaught SyntaxError`

۸۹ آسان

طبق قرارداد رایج نام‌گذاری در JavaScript (camelCase)، کدام نام برای متغیری که «تعداد کل سفارش‌ها» را نگه می‌دارد مناسب‌تر است؟

۱) `total_orders`
۲) `TotalOrders`
۳) `totalOrders`
۴) `totalorders`

۹۰ متوسط

خروجی کد زیر چیست؟

```javascript
let data = 42;
console.log(typeof data);
data = "forty-two";
console.log(typeof data);
```

۱) `number` `string`
۲) `number` `number`
۳) `number` `Uncaught TypeError`
۴) `string` `string`

۹۱ متوسط

در جای خالی کدام کلمه قرار بگیرد تا کد زیر بدون خطا اجرا شود و `3` را چاپ کند؟

```javascript
___ count = 1;
count = count + 2;
console.log(count);
```

۱) `const`
۲) `int`
۳) `let`
۴) `final`

۱-۶ انواع داده: Primitiveها و Objectها

منبع: Introduction to JS، اسلایدهای ۱۵ تا ۱۷

درسنامه

فرض کن یک جعبه‌ی لگو داری. بعضی چیزها **تک‌آجر** هستند: یک آجر قرمز، یک آجر آبی؛ ساده، تجزیه‌ناپذیر و همان‌طور که هستند. بعضی چیزها **سازه** هستند: یک ماشین که از چند آجر ساخته شده، هر قسمتش اسمی دارد (چرخ، سقف، در) و می‌توانی قسمت‌هایش را عوض کنی. داده‌ها در JavaScript هم همین‌طورند:

- **primitive** (مقدار پایه یا اولیه): یک مقدار ساده و تکی، مثل عدد `42` یا رشته‌ی `"Ali"`.
- **object** (شیء): مجموعه‌ای از مقدارها که هر کدام یک نام (کلید) دارند، مثل یک کارت دانشجویی که نام، سن و درس‌ها را با هم نگه می‌دارد.

داده‌ها در JavaScript (اسلاید ۱۵)

- داده دو گروه اصلی دارد: **هر چیزی یا primitive است یا object.**
- JavaScript ۷ **نوع داده‌ی primitive** دارد: `number`، `string`، `boolean`، `undefined`، `null`، `Symbol` و `BigInt`. (با object، روی هم ۸ نوع داده می‌شود.)
- متغیرها می‌توانند **هر نوعی** را نگه دارند و نوعشان در زمان اجرا عوض شود؛ به این **dynamic typing** (نوع‌دهی پویا) می‌گویند.
- آرایه‌ها (arrays)، توابع (functions) و حتی تاریخ‌ها (dates) در واقع **objectهای خاص** هستند.
- تبدیل نوع‌ها یا **خودکار** انجام می‌شود که به آن **coercion** (تبدیل ضمنی) می‌گویند، یا **صریح** و به دستور ما که به آن **casting** (تبدیل صریح) می‌گویند؛ مثلاً `"5" * 2` خودکار به `10` می‌رسد، ولی `Number("5")` صریحاً رشته را عدد می‌کند. جزئیات این دو را در درس‌های Casting و Type coercion می‌بینیم.

```JavaScript
let value = 7;      // number
value = "seven";    // now a string
value = true;       // now a boolean (dynamic typing)
```

هفت primitive (اسلاید ۱۶)

نوع	توضیح اسلاید ۱۶	مثال	نتیجه‌ی `typeof`
`number`	اعداد صحیح و اعشاری (floating-point)	`42` ، `3.14` ، `-7`	`"number"`
`string`	دنباله‌ای از کاراکترها داخل علامت نقل‌قول	`"Ali"` ، `'Ali'` ، `` `Ali` ``	`"string"`
`boolean`	فقط `true` یا `false`	`true`	`"boolean"`
`undefined`	متغیری که تعریف شده ولی مقدار نگرفته	`let x;`	`"undefined"`
`null`	نبودِ **عمدیِ** مقدار	`let car = null;`	`"object"` (اشتباه تاریخی)
`Symbol`	شناسه‌ی یکتا و تغییرناپذیر	`Symbol("id")`	`"symbol"`
`BigInt`	اعداد صحیح بزرگ‌تر از `Number.MAX_SAFE_INTEGER`	`10n`	`"bigint"`

`number`

JavaScript برخلاف Java یا C نوع جداگانه‌ای برای اعداد صحیح (`int`) و اعشاری (`float` / `double`) ندارد؛ **همه‌ی اعداد از یک نوع** `number` **هستند**. برای همین `5 === 5.0` برابر `true` است. سه مقدار ویژه هم از نوع `number` هستند: `Infinity` (بی‌نهایت)، `-Infinity` (منفی بی‌نهایت) و `NaN` (مخفف Not a Number، یعنی «عدد نیست»؛ نتیجه‌ی محاسبه‌های بی‌معنی مثل `0 / 0` یا `"abc" * 2`). جالب است که `typeof NaN` برابر `"number"` است!

`string`

متن داخل علامت نقل‌قول: تک‌کوتیشن `'...'` ، دابل‌کوتیشن `"..."` یا backtick (بک‌تیک) `` `...` `` . هر سه یک نوع `string` می‌سازند. JavaScript نوع جداگانه‌ای برای یک کاراکتر (مثل `char` در Java) ندارد؛ `'a'` یک رشته به طول ۱ است. رشته‌های backtick را در درس «رشته‌ها و Template literals» کامل می‌بینیم.

`boolean`

فقط دو مقدار دارد: `true` و `false` . نتیجه‌ی مقایسه‌ها از این نوع است: `5 > 3` برابر `true` است.

`undefined` و `null`

هر دو یعنی «مقداری نیست»، ولی فرقشان در **این است که چه کسی** آن را گذاشته:

	`undefined`	`null`
معنی	هنوز مقداری داده نشده	عمداً «خالی» گذاشته شده
چه کسی می‌گذارد؟	خود JavaScript (متغیر بی‌مقدار، کلید ناموجود در شیء)	برنامه‌نویس
`typeof`	`"undefined"`	`"object"`
در محاسبه (پیش‌نمایش)	`undefined + 1` برابر `NaN`	`null + 1` برابر `1`

```JavaScript
let a;              // JavaScript puts undefined here
let car = null;     // we say: "no car selected yet"
console.log(a, car); // undefined null
```

> **یک تشبیه ساده**
>
> `undefined` مثل کشوی میزی است که هیچ‌کس هنوز سراغش نرفته؛ `null` مثل کشویی است که رویش یادداشت گذاشته‌ای: «عمداً خالی است».

`Symbol`

`Symbol("id")` هر بار یک مقدار **کاملاً جدید و یکتا** می‌سازد. متن `"id"` فقط یک برچسب توضیحی برای خودمان است؛ پس دو Symbol با یک برچسب، با هم برابر نیستند. Symbolها تغییرناپذیرند (immutable) و معمولاً به‌عنوان **کلید ویژگی‌های شیء** به کار می‌روند تا با هیچ کلید دیگری تداخل نکنند:

```JavaScript
const id = Symbol("id");
const user = { name: "Ali", [id]: 123 }; // [id] means: use the symbol as the key
console.log(user[id]);     // 123
console.log(user["id"]);   // undefined (the string "id" is a different key)
console.log(Symbol("id") === Symbol("id")); // false
```

`BigInt`

نوع `number` اعداد صحیح را فقط تا `Number.MAX_SAFE_INTEGER` یعنی `9007199254740991` (برابر `2 ** 53 - 1`) **دقیق** نگه می‌دارد. بالاتر از آن، دقت از دست می‌رود و دو عدد متفاوت یکی دیده می‌شوند:

```JavaScript
console.log(Number.MAX_SAFE_INTEGER);              // 9007199254740991
console.log(Number.MAX_SAFE_INTEGER + 2);          // 9007199254740992 (wrong!)
console.log(9007199254740992 === 9007199254740993); // true (precision lost)
```

برای اعداد صحیحِ بزرگ از **BigInt** استفاده می‌کنیم: کافی است آخر عدد یک `n` بگذاریم (یا از `BigInt("...")` استفاده کنیم):

```JavaScript
const big = 9007199254740993n;
console.log(big + 2n);    // 9007199254740995n (exact)
console.log(typeof big);  // bigint
console.log(7n / 2n);     // 3n (BigInt has no decimals)
console.log(10n + 5);     // TypeError: Cannot mix BigInt and other types, use explicit conversions
console.log(10n + BigInt(5)); // 15n
console.log(Number(10n) + 5); // 15
```

پس BigInt و number را **نمی‌شود مستقیم با هم قاطی کرد**؛ باید اول یکی را صریحاً به نوع دیگر تبدیل کنی.

عملگر `typeof`

`typeof` (عملگر «نوعِ این مقدار چیست؟») نام نوع یک مقدار را به‌صورت یک **رشته‌ی حروف کوچک** برمی‌گرداند. این جدول را خوب حفظ کن؛ تست‌های زیادی از آن طرح می‌شود:

عبارت	نتیجه	نکته
`typeof 42` و `typeof 3.14`	`"number"`	صحیح و اعشاری یک نوع‌اند
`typeof NaN`	`"number"`	NaN هم عدد حساب می‌شود!
`typeof "hi"`	`"string"`	
`typeof true`	`"boolean"`	
`typeof undefined`	`"undefined"`	
`typeof notDeclaredYet`	`"undefined"`	متغیر تعریف‌نشده؛ **خطا نمی‌دهد**
`typeof null`	`"object"`	اشتباه تاریخی
`typeof Symbol("id")`	`"symbol"`	با حرف کوچک
`typeof 10n`	`"bigint"`	با حرف کوچک
`typeof {}`	`"object"`	
`typeof []`	`"object"`	آرایه هم object است
`typeof new Date()`	`"object"`	تاریخ هم object است
`typeof function () {}`	`"function"`	تابع object است، ولی typeof می‌گوید `"function"`
`typeof typeof 42`	`"string"`	خروجی typeof همیشه رشته است

- `typeof null` برابر `"object"` است؛ این یک اشتباه قدیمی از نسخه‌ی اول JavaScript است که برای این‌که کدهای قدیمی خراب نشوند، هیچ‌وقت اصلاح نشد. برای تشخیص `null` بنویس `value === null` .
- چون `typeof []` هم `"object"` است، برای تشخیص آرایه از `Array.isArray(value)` استفاده کن: `Array.isArray([1, 2])` برابر `true` و `Array.isArray({})` برابر `false` است.
- `typeof` روی متغیری که اصلاً تعریف نشده خطا نمی‌دهد و `"undefined"` برمی‌گرداند، ولی **استفاده‌ی مستقیم** از همان متغیر (مثلاً `console.log(price)`) خطای `ReferenceError` می‌دهد.

Objectها، آرایه‌ها و توابع (اسلاید ۱۷)

- **Objectها** داده‌ها و رفتارهای مرتبط را به شکل **جفت‌های کلید-مقدار (key-value pairs)** کنار هم نگه می‌دارند.
- Objectها می‌توانند objectهای دیگر، آرایه‌ها یا توابع را **درون خود** داشته باشند (تودرتو یا nested).
- **آرایه‌ها** مجموعه‌هایی **مرتب** هستند که با **اندیس عددی** (index، شماره‌ی خانه که از ۰ شروع می‌شود) به خانه‌هایشان دسترسی داریم.
- آرایه‌ها **متدهای آماده (built-in methods)** دارند، «مثل Java»؛ مثلاً `push` برای اضافه کردن به انتها. (درس‌های «آرایه‌ها» و «متدهای آرایه» را ببین.)
- **توابع first-class object هستند** (شهروند درجه‌یک): می‌شود آن‌ها را در متغیر **ذخیره** کرد، به تابع دیگر **پاس داد** و از یک تابع **برگرداند**.
- توابع می‌توانند **متد یک object** باشند یا به‌صورت **مستقل** (standalone) نوشته شوند.

JavaScript

```
const student = {
  name: "Sara",                // key: name, value: a string
  age: 20,                     // number
  courses: ["Web", "AI"],      // nested array
  address: { city: "Tehran" }, // nested object
  greet: function () {         // a function stored in an object = a method
    console.log("Hello!");
  },
};

console.log(student.name);         // Sara
console.log(student.courses[1]);   // AI (indices start at 0)
console.log(student.address.city); // Tehran
student.greet();                   // Hello!
console.log(student.phone);        // undefined (no such key)
```

و این هم سه کاری که «first-class» بودن توابع ممکن می‌کند:

JavaScript

```
function double(n) {          // a standalone function
  return n * 2;
}
const f = double;             // 1) stored in a variable
function applyTwice(fn, x) {  // 2) passed as an argument
  return fn(fn(x));
}
console.log(applyTwice(f, 3)); // 12
function makeGreeter() {      // 3) returned from a function
  return function () {
    return "Hi!";
  };
}
console.log(makeGreeter()());  // Hi!
```

primitive در برابر object: کپی و مقایسه (پیش‌نمایش)

primitiveها تغییرناپذیرند (immutable). نمی‌شود یک حرف از یک رشته را عوض کرد؛ متدهایی مثل `toUpperCase()` یک رشته‌ی **جدید** می‌سازند:

JavaScript

```
let s = "cat";
s[0] = "b";                    // silently ignored (TypeError in strict mode)
console.log(s);                // cat
console.log(s.toUpperCase(), s); // CAT cat
```

(البته خود متغیر `let` را می‌توانی به رشته‌ی دیگری وصل کنی: `s = "bat"`؛ تغییرناپذیر بودن یعنی خود مقدار عوض نمی‌شود.)

کپی: وقتی یک primitive را در متغیر دیگری می‌ریزی، **خود مقدار** کپی می‌شود؛ ولی وقتی یک object را می‌ریزی، فقط **نشانی (reference)** آن کپی می‌شود و هر دو متغیر به **همان یک شیء** اشاره می‌کنند:

```JavaScript
let a = 10;
let b = a;     // b gets a copy of the value
b = 20;
console.log(a); // 10

const obj1 = { score: 10 };
const obj2 = obj1;  // both names point to the SAME object
obj2.score = 20;
console.log(obj1.score); // 20
```

مقایسه: primitiveها با **مقدار** مقایسه می‌شوند و objectها با **نشانی**. دو آرایه یا دو شیء جداگانه، حتی با محتوای یکسان، برابر نیستند:

```JavaScript
console.log("web" === "web");   // true  (same value)
console.log([1, 2] === [1, 2]); // false (two different arrays)
console.log({} === {});         // false (two different objects)
```

نکته

- خروجی `typeof` همیشه رشته‌ای با **حروف کوچک** است: `"bigint"` (نه `"BigInt"`) و `"symbol"` (نه `"Symbol"`).
- برای آرایه از `Array.isArray` و برای `null` از `=== null` استفاده کن؛ `typeof` برای هر دو `"object"` می‌گوید.
- برای اعداد صحیح بزرگ‌تر از `9007199254740991` از BigInt (پسوند `n`) استفاده کن و BigInt را با number قاطی نکن.

دام تستی

- `typeof null` برابر `"object"`، `typeof []` برابر `"object"`، `typeof NaN` برابر `"number"` و `typeof function () {}` برابر `"function"` است.
- `typeof x` برای متغیر تعریف‌نشده `"undefined"` می‌دهد و **خطا نمی‌دهد**، ولی خواندن مستقیم `x` خطای `ReferenceError` می‌دهد.
- `Symbol("id") === Symbol("id")` برابر `false` است؛ هر Symbol یکتاست، حتی با برچسب یکسان.
- `9007199254740992 === 9007199254740993` برابر `true` است (از دست رفتن دقت) و `10n + 5` خطای `TypeError` می‌دهد.
- `[1, 2] === [1, 2]` و `{} === {}` هر دو `false` هستند؛ objectها با نشانی مقایسه می‌شوند، نه محتوا.

جمع‌بندی

- هر چیزی یا primitive است یا object. ۷ primitive: `number`، `string`، `boolean`، `undefined`، `null`، `Symbol`، `BigInt`.
- آرایه، تابع و تاریخ object‌های خاص‌اند. objectها جفت‌های کلید-مقدارند و می‌توانند تودرتو باشند. توابع first-class هستند.
- `undefined` را JavaScript می‌گذارد و `null` را برنامه‌نویس. Symbol یکتاست. BigInt برای اعداد صحیح خیلی بزرگ است.
- primitiveها تغییرناپذیرند و با مقدار کپی و مقایسه می‌شوند؛ objectها با نشانی (reference).

تست‌های تمرینی

۲۸ تست · شماره‌ی ۹۲ تا ۱۱۹

۹۲ آسان

طبق اسلاید ۱۵، JavaScript چند نوع داده‌ی **primitive** دارد؟

۱) ۵ ۲) ۶ ۳) ۷ ۴) ۸

۹۳ آسان

کدام مورد از انواع **primitive** در JavaScript **نیست**؟

۱) `Symbol` ۲) `BigInt` ۳) `null` ۴) آرایه (Array)

۹۴ آسان

به تبدیل نوعی که JavaScript **خودش و بدون درخواست ما** انجام می‌دهد (مثل `"5" * 2` که `10` می‌شود) چه می‌گویند؟

۱) casting ۲) coercion ۳) hoisting ۴) parsing

۹۵ آسان

کدام مقدار را **برنامه‌نویس عمداً** در متغیر می‌گذارد تا نشان دهد «این‌جا مقداری وجود ندارد»؟

۱) `undefined` ۲) `NaN` ۳) `0` ۴) `null`

۹۶ آسان

خروجی کد زیر چیست؟

```javascript
console.log(typeof "42");
console.log(typeof 42);
console.log(typeof true);
```

۱) `string` `number` `boolean` ۲) `number` `number` `boolean`
۳) `string` `number` `string` ۴) `string` `int` `boolean`

۹۷ آسان

حاصل `typeof 10n` چیست؟

۱) `"number"`
۲) `"BigInt"`
۳) `"bigint"`
۴) `"integer"`

۹۸ آسان

مقدار `Number.MAX_SAFE_INTEGER` (بزرگ‌ترین عدد صحیحی که نوع `number` دقیق نگه می‌دارد) کدام است؟

۱) `9007199254740991`
۲) `2147483647`
۳) `9007199254740992`
۴) `Infinity`

۹۹ متوسط

حاصل `typeof null` چیست؟

۱) `"null"`
۲) `"undefined"`
۳) `"object"`
۴) `null`

۱۰۰ متوسط

حاصل `typeof` برای کدام مقدار، **نیست**؟ `"number"`

۱) `10n`
۲) `NaN`
۳) `Infinity`
۴) `3.14`

۱۰۱ متوسط

خروجی کد زیر چیست؟ (متغیر `price` هیچ‌جا تعریف نشده است.)

```JavaScript
console.log(typeof price);
console.log(price);
```

۱) `undefined` `undefined`
۲) `Uncaught ReferenceError`
۳) `undefined` `Uncaught ReferenceError`
۴) `object` `null`

۱۰۲ متوسط

خروجی کد زیر چیست؟

```JavaScript
const list = [1, 2, 3];
console.log(typeof list);
console.log(Array.isArray(list));
console.log(Array.isArray("123"));
```

۱) `array` `true` `false`
۲) `object` `true` `false`
۳) `object` `true` `true`
۴) `array` `false` `false`

۱۰۳ متوسط

خروجی کد زیر چیست؟

```javascript
let a;
let b = null;
console.log(a, b);
console.log(typeof a, typeof b);
```

۱) `undefined null` `undefined object`

۲) `undefined null` `undefined null`

۳) `null null` `object object`

۴) `undefined undefined` `undefined undefined`

۱۰۴ متوسط

خروجی کد زیر چیست؟

```javascript
const id1 = Symbol("id");
const id2 = Symbol("id");
console.log(id1 === id2);
console.log(typeof id1);
```

۱) `true` `symbol`

۲) `false` `symbol`

۳) `false` `string`

۴) `true` `string`

۱۰۵ سخت

خروجی کد زیر چیست؟

```javascript
const id = Symbol("id");
const user = { name: "Ali", [id]: 123 };
console.log(user[id]);
console.log(user["id"]);
```

۱) `123` `123`

۲) `undefined` `123`

۳) `Uncaught TypeError`

۴) `123` `undefined`

۱۰۶ سخت

خروجی کد زیر چیست؟

```JavaScript
console.log(9007199254740992 === 9007199254740993);
console.log(9007199254740992n === 9007199254740993n);
```

۱) `false` `false`
۲) `true` `false`
۳) `false` `true`
۴) `true` `true`

۱۰۷ سخت

خروجی کد زیر چیست؟

```JavaScript
const big = 10n;
console.log(big + 5n);
console.log(big + 5);
```

۱) `15n` `15`
۲) `15n` `15n`
۳) `15n` `Uncaught TypeError`
۴) `15` `15`

۱۰۸ آسان

کدام گزینه درباره‌ی نوشتن رشته‌ها در JavaScript درست است؟

۱) فقط دابل‌کوتیشن (`"`) مجاز است و تک‌کوتیشن خطای `SyntaxError` می‌دهد.

۲) `'a'` از نوع `char` است، نه `string`.

۳) رشته‌ای که با backtick نوشته شود، نوع جداگانه‌ای به نام `template` دارد.

۴) رشته را می‌توان با تک‌کوتیشن، دابل‌کوتیشن یا backtick نوشت و نوع هر سه `string` است.

۱۰۹ متوسط

خروجی کد زیر چیست؟

```JavaScript
let value = 7;
value = "seven";
value = value + 1;
console.log(value, typeof value);
```

۱) `seven1 string`
۲) `8 number`
۳) `Uncaught TypeError`
۴) `NaN number`

۱۱۰ متوسط

طبق اسلاید ۱۷، این جمله که «توابع در JavaScript first-class object هستند» یعنی چه؟

۱) توابع همیشه قبل از بقیه‌ی کد اجرا می‌شوند.

۲) توابع فقط داخل کلاس‌ها و به‌عنوان متد تعریف می‌شوند.

۳) می‌توان توابع را در متغیر ذخیره کرد، به تابع دیگر پاس داد و از یک تابع برگرداند.

۴) توابع از نوع primitive هستند و مثل عدد کپی می‌شوند.

۱۱۱ متوسط

خروجی کد زیر چیست؟

```JavaScript
function shout(text) {
  return text.toUpperCase() + "!";
}
function apply(fn, value) {
  return fn(value);
}
const myFn = shout;
console.log(apply(myFn, "hi"));
console.log(typeof myFn);
```

۱) `hi!` `function`

۲) `HI!` `object`

۳) `Uncaught TypeError`

۴) `HI!` `function`

۱۱۲ متوسط

خروجی کد زیر چیست؟

```JavaScript
const student = {
  name: "Sara",
  courses: ["Web", "AI"],
  address: { city: "Tehran" },
};
console.log(student.courses[1]);
console.log(student.address.city);
console.log(student.phone);
```

۱) `Web` `Tehran` `undefined`

۲) `AI` `Tehran` `undefined`

۳) `AI` `Tehran` `null`

۴) `AI` `Tehran` `Uncaught ReferenceError`

۱۱۳ متوسط

کدام مقدار از نوع **primitive** است؟

۱) `new Date()`　۲) `[1, 2, 3]`　۳) `"2026-11-13"`　۴) `function () {}`

۱۱۴ متوسط

کدام گزینه **نادرست** است؟

۱) حاصل `typeof function () {}` برابر `"function"` است.

۲) حاصل `typeof new Date()` برابر `"date"` است.

۳) حاصل `typeof undefined` برابر `"undefined"` است.

۴) حاصل `typeof Symbol("id")` برابر `"symbol"` است.

۱۱۵ سخت

حاصل `typeof typeof 10n` چیست؟

۱) `"bigint"`　۲) `"number"`　۳) `"object"`　۴) `"string"`

۱۱۶ سخت

خروجی کد زیر چیست؟

```JavaScript
let a = 10;
let b = a;
b = 20;
const obj1 = { score: 10 };
const obj2 = obj1;
obj2.score = 20;
console.log(a, obj1.score);
```

۱) `10 10`　۲) `20 20`　۳) `10 20`　۴) `20 10`

۱۱۷ سخت

خروجی کد زیر چیست؟

```JavaScript
console.log("web" === "web");
console.log([1, 2] === [1, 2]);
console.log({} === {});
```

۱) `true` `false` `false`　۲) `true` `true` `true`

۳) `false` `false` `false`　۴) `true` `true` `false`

۱۱۸ سخت

خروجی کد زیر (بدون `"use strict"`) چیست؟

```javascript
let word = "cat";
word[0] = "b";
console.log(word);
word = "bat";
console.log(word);
```

۱) `cat` `bat`

۲) `bat` `bat`

۳) `Uncaught TypeError`

۴) `cat` `cat`

۱۱۹ سخت

کدام گزینه درباره‌ی نوع `number` درست است؟

۱) برای اعداد اعشاری نوع جداگانه‌ای وجود دارد و `typeof 5.5` برابر `"float"` است.

۲) `5` و `5.0` دو مقدار از دو نوع متفاوت‌اند و `5 === 5.0` برابر `false` است.

۳) نوشتن عددی بزرگ‌تر از `Number.MAX_SAFE_INTEGER` بدون پسوند `n`، خطای `RangeError` می‌دهد.

۴) اعداد صحیح و اعشاری هر دو از یک نوع `number` هستند؛ `typeof 5` و `typeof 5.5` هر دو `"number"` است.

۱-۷ عملگرهای حسابی و انتساب

منبع: Introduction to JS، اسلایدهای ۱۸ و ۱۹

درسنامه

یک ماشین‌حساب را تصور کن: عددها را وارد می‌کنی و با دکمه‌های `+`، `-`، `×` و `÷` رویشان کاری انجام می‌دهی. در برنامه‌نویسی به این دکمه‌ها **عملگر (operator)** می‌گوییم و به مقدارهایی که عملگر رویشان کار می‌کند **عملوند (operand)**. در `7 + 2`، علامت `+` عملگر است و `7` و `2` عملوند.

عملگرهای حسابی (اسلاید ۱۸)

عملگر	نام	مثال	نتیجه
`+`	جمع (addition)	`7 + 2`	`9`
`-`	تفریق (subtraction)	`7 - 2`	`5`
`*`	ضرب (multiplication)	`7 * 2`	`14`
`/`	تقسیم (division)	`7 / 2`	`3.5`
`%`	باقی‌مانده (remainder یا modulo)	`7 % 2`	`1`
`**`	توان (exponentiation)	`7 ** 2`	`49`

میم اسلاید ۱۸: «`1 + 1` چند می‌شود؟»

در میم اسلاید ۱۸ رئیس جلسه می‌پرسد «`1 + 1` چند می‌شود؟». کارمندِ Python می‌گوید `2`، کارمندِ C++ هم `2`، ولی کارمندِ JavaScript با خونسردی می‌گوید `"11"` و رئیس او را از پنجره بیرون می‌اندازد! واقعیت این است که در JavaScript هم `1 + 1` برابر `2` است. جواب `"11"` فقط وقتی پیش می‌آید که **یکی از دو طرف رشته باشد**؛ چون عملگر `+` دو شغل دارد:

- اگر هر دو طرف عدد باشند، **جمع** می‌کند.
- اگر حتی یک طرف **رشته (string)** باشد، دو طرف را به هم **می‌چسباند** که به آن **concatenation** (الحاق) می‌گویند.

بقیه‌ی عملگرهای حسابی (`-`، `*`، `/`، `%`، `**`) فقط معنای ریاضی دارند و رشته‌ها را به عدد تبدیل می‌کنند:

```JavaScript
console.log(1 + 1);       // 2
console.log("1" + 1);     // 11 (a string: concatenation)
console.log("5" - 1);     // 4  (minus only does math)
console.log(1 + 2 + "3"); // 33 (left to right: 1 + 2 = 3, then 3 + "3")
```

(قانون‌های کامل این تبدیل‌های خودکار را در درس Type coercion می‌بینیم.)

تقسیم `/` همیشه اعشاری است

در Java یا C، تقسیم دو عدد صحیح، نتیجه‌ی صحیح می‌دهد (`7 / 2` می‌شود `3`). در JavaScript فقط یک نوع عدد داریم و تقسیم **همیشه نتیجه‌ی واقعی** می‌دهد: `7 / 2` برابر `3.5` است. اگر **تقسیم صحیح** می‌خواهی، باید خودت قسمت اعشاری را حذف کنی:

```JavaScript
console.log(7 / 2);              // 3.5
console.log(Math.floor(7 / 2));  // 3  (round down)
console.log(Math.trunc(7 / 2));  // 3  (cut off the decimals)
console.log(Math.floor(-7 / 2)); // -4 (down means toward minus infinity)
console.log(Math.trunc(-7 / 2)); // -3
console.log(Math.round(7 / 2));  // 4  (nearest integer, NOT integer division)
```

تقسیم بر صفر خطا نمی‌دهد! نتیجه یکی از مقدارهای ویژه‌ی نوع `number` است:

```JavaScript
console.log(1 / 0);  // Infinity
console.log(-1 / 0); // -Infinity
console.log(0 / 0);  // NaN (Not a Number)
console.log(5 % 0);  // NaN
```

(این قانون مخصوص نوع `number` است؛ تقسیم یک BigInt بر `0n` ، مثلاً `1n / 0n` ، خطای `RangeError: Division by zero` می‌دهد.)

باقی‌مانده `%`

فرض کن ۱۷ شکلات را بین ۵ نفر تقسیم می‌کنی: به هر نفر ۳ شکلات می‌رسد و ۲ شکلات **باقی می‌ماند**؛ پس `17 % 5` برابر `2` است.

- کاربرد معروف: تشخیص زوج بودن با `n % 2 === 0` (مثلاً `10 % 2` برابر `0` است).
- **علامت نتیجه از عدد سمت چپ (مقسوم) می‌آید:** `-7 % 3` برابر `-1` است (نه `2`) و `7 % -3` برابر `1` . برای همین دقیق‌تر است بگوییم «باقی‌مانده» تا «modulo» ریاضی.
- به همین دلیل برای تشخیص **فرد** بودن بنویس `n % 2 !== 0` ؛ چون `-3 % 2` برابر `-1` است و شرط `n % 2 === 1` برای اعداد منفی جواب غلط می‌دهد.
- روی اعداد اعشاری هم کار می‌کند: `5.5 % 2` برابر `1.5` است.

توان `**`

- `2 ** 3` برابر `8` ، `2 ** 0.5` برابر جذر ۲ (`1.4142135623730951`) و `2 ** -1` برابر `0.5` است. روش قدیمی‌تر همین کار `Math.pow(2, 3)` است.
- علامت `^` در JavaScript توان **نیست** (یک عملگر بیتی به نام XOR است): `2 ^ 3` برابر `1` است، نه `8` !
- `**` **از راست به چپ** حساب می‌شود: `2 ** 3 ** 2` یعنی `2 ** (3 ** 2)` که می‌شود `2 ** 9` یعنی `512` (نه `64`).
- منفیِ یک‌طرفه (unary minus) درست قبل از `**` مبهم است و `SyntaxError` می‌دهد: `-2 ** 2` خطاست. باید با پرانتز منظورت را روشن کنی: `(-2) ** 2` برابر `4` و `-(2 ** 2)` برابر `-4` .

اعداد اعشاری و ماجرای `0.1 + 0.2`

```JavaScript
console.log(0.1 + 0.2);         // 0.30000000000000004
console.log(0.1 + 0.2 === 0.3); // false
```

کامپیوتر اعداد را در مبنای ۲ ذخیره می‌کند و عددی مثل `0.1` در مبنای ۲ دقیق نوشته نمی‌شود (مثل `1/3` که در مبنای ۱۰ می‌شود `0.333...`). این باگ JavaScript نیست؛ Python و Java هم همین رفتار را دارند (استاندارد IEEE 754). راه‌حل: برای نمایش گرد کن (`(0.1 + 0.2).toFixed(2)` که رشته‌ی `"0.30"` می‌دهد) یا با یک خطای مجاز مقایسه کن: `Math.abs((0.1 + 0.2) - 0.3) < Number.EPSILON` که `true` است.

اولویت عملگرها (precedence)

وقتی چند عملگر در یک عبارت هستند، به این ترتیب حساب می‌شوند:

1. پرانتز `( )`
2. توان `**` (از **راست به چپ**)
3. `*` ، `/` و `%` (هم‌سطح؛ از **چپ به راست**)
4. `+` و `-` (هم‌سطح؛ از **چپ به راست**)

JavaScript

```
console.log(2 + 3 * 4);      // 14  (multiplication first)
console.log((2 + 3) * 4);    // 20  (parentheses first)
console.log(2 * 3 ** 2);     // 18  (power first: 2 * 9)
console.log(20 / 4 * 2);     // 10  (left to right: 5 * 2)
console.log(10 - 4 - 3);     // 3   (left to right: 6 - 3)
console.log(2 + 3 * 4 ** 2); // 50  (2 + 3 * 16)
```

عملگرهای انتساب (اسلاید ۱۹)

عملگر `=` مقدار سمت راست را در متغیر سمت چپ می‌ریزد. عملگرهای ترکیبی (compound) یک محاسبه و یک انتساب را با هم انجام می‌دهند. در این جدول فرض کن هر بار از `x = 10` شروع می‌کنیم:

عملگر	معنی	مثال	معادلِ	مقدار جدید `x`
`=`	انتساب (assign)	`x = 5`		`5`
`+=`	جمع و انتساب	`x += 3`	`x = x + 3`	`13`
`-=`	تفریق و انتساب	`x -= 3`	`x = x - 3`	`7`
`*=`	ضرب و انتساب	`x *= 3`	`x = x * 3`	`30`
`/=`	تقسیم و انتساب	`x /= 4`	`x = x / 4`	`2.5`
`%=`	باقی‌مانده و انتساب	`x %= 3`	`x = x % 3`	`1`

(عملگر `**=` هم وجود دارد که روی اسلاید نیامده: `x **= 2` یعنی `x = x ** 2` که می‌شود `100`.)

میم اسلاید ۱۹: `x = x + 1`

در میم اسلاید ۱۹، برنامه‌نویس‌ها با لبخند `X = X+1` می‌نویسند و ریاضی‌دان‌ها با وحشت نگاه می‌کنند؛ چون در ریاضی هیچ عددی با «خودش به‌علاوه‌ی ۱» برابر نیست! نکته این است که در برنامه‌نویسی `=` یعنی **انتساب (assignment)**، نه **برابری**. `x = x + 1` یعنی: «سمت راست را حساب کن، بعد نتیجه را در سمت چپ بریز».

```JavaScript
let y = 5;
y = y + 1;      // read y (5), compute 5 + 1, store 6 back into y
console.log(y); // 6
```

همین کار را کوتاه‌تر می‌شود با `y += 1` یا `y++` نوشت. برای **مقایسه‌ی برابری** در JavaScript از `===` استفاده می‌کنیم (درس «عملگرهای مقایسه و منطقی»).

دام‌های عملگرهای ترکیبی

- **اول کل سمت راست حساب می‌شود:** `x *= 2 + 3` یعنی `x = x * (2 + 3)`، نه `x * 2 + 3`.
- `+=` **روی رشته الحاق می‌کند**، ولی `-=` همیشه عددی است:

```JavaScript
let z = 4;
z *= 2 + 3;
console.log(z);   // 20

let msg = "Hello";
msg += " World";
console.log(msg); // Hello World

let s1 = "8";
s1 += 2;          // "82" (string concatenation)
let s2 = "8";
s2 -= 2;          // 6 (minus converts "8" to a number)
console.log(s1, s2); // 82 6
```

- همه‌ی عملگرهای انتساب، **انتساب دوباره** هستند؛ پس روی `const` خطای `TypeError` می‌دهند: `const total = 10; total += 5;` خطاست.

افزایش و کاهش: `++` و `--`

`x++` یعنی «یکی به `x` اضافه کن» و `x--` یعنی «یکی کم کن». ولی **جای** علامت مهم است:

- **postfix** (پسوندی، `a++`): اول مقدار **فعلی** استفاده می‌شود، **بعد** یکی اضافه می‌شود.
- **prefix** (پیشوندی، `++a`): اول یکی اضافه می‌شود، **بعد** مقدار **جدید** استفاده می‌شود.

```JavaScript
let a = 5;
let b = a++;      // b gets the OLD value (5), then a becomes 6
console.log(a, b); // 6 5

let c = 5;
let d = ++c;      // c becomes 6 first, then d gets the NEW value (6)
console.log(c, d); // 6 6
```

یک تشبیه ساده

`a++` مثل این است که بگویی «اول عکس بگیر، بعد مدل موهایت را عوض کن»؛ عکس، مدل قبلی را نشان می‌دهد. `++a` یعنی «اول مدل موها را عوض کن، بعد عکس بگیر».

خود انتساب هم مقدار دارد

هر انتساب یک عبارت (expression) است و مقدارش، **مقدار جدید متغیر** است. برای همین کنسول اسلاید ۲۳ بعد از `a = "Hello!"` مقدار `"Hello!"` را نشان داد:

```JavaScript
let w = 1;
console.log(w += 2); // 3
console.log(w = 10); // 10
let p, q;
p = q = 5;           // right to left: q = 5, then p = 5
console.log(p, q);   // 5 5
```

نکته

- برای تقسیم صحیح از `Math.floor` (یا برای اعداد منفی، بسته به نیاز، `Math.trunc`) استفاده کن، نه `Math.round`.
- برای زوج/فرد از `n % 2 === 0` و `n % 2 !== 0` استفاده کن.
- برای توان از `**` (یا `Math.pow`) استفاده کن؛ `^` توان نیست.
- اگر در اولویت عملگرها شک داری، پرانتز بگذار؛ هم درست‌تر است و هم خواناتر.

دام تستی

- `"1" + 1` برابر `"11"` است (الحاق)، ولی `"5" - 1` برابر `4` است (تفریق عددی).
- `7 / 2` برابر `3.5` است، نه `3`. و `1 / 0` خطا نمی‌دهد: `Infinity` است؛ `0 / 0` هم `NaN` است.
- `-7 % 3` برابر `-1` است (علامت از عدد سمت چپ).
- `2 ** 3 ** 2` برابر `512` است (راست به چپ) و `-2 ** 2` خطای `SyntaxError` می‌دهد.
- `0.1 + 0.2 === 0.3` برابر `false` است.
- `let x = 4; x *= 2 + 3;` مقدار `20` می‌دهد، نه `11`.
- `let b = a++` مقدار **قدیمی** `a` را در `b` می‌ریزد.

جمع‌بندی

- عملگرهای حسابی: `+` (جمع یا الحاق)، `-`، `*`، `/` (همیشه اعشاری)، `%` (باقی‌مانده با علامت سمت چپ)، `**` (توان، راست به چپ).
- اولویت: پرانتز، بعد `**`، بعد `* / %`، بعد `+ -`؛ هم‌سطح‌ها از چپ به راست.
- عملگرهای انتساب: `=`، `+=`، `-=`، `*=`، `/=`، `%=`؛ `x op= y` یعنی `x = x op (y)`.
- `=` انتساب است، نه برابری. `a++` مقدار قدیمی و `++a` مقدار جدید را برمی‌گرداند.

تست‌های تمرینی

۲۶ تست · شماره‌ی ۱۲۰ تا ۱۴۵

۱۲۰ آسان

حاصل `7 / 2` در JavaScript چیست؟

۱) `3`　۲) `3.5`　۳) `4`　۴) `"3.5"`

۱۲۱ آسان

حاصل `17 % 5` چیست؟

۱) `3`　۲) `3.4`　۳) `2`　۴) `12`

۱۲۲ آسان

حاصل `2 ** 4` چیست؟

۱) `16`　۲) `8`　۳) `6`　۴) `32`

۱۲۳ آسان

در میم اسلاید ۱۸، کارمندِ JavaScript جواب «`1 + 1` چند می‌شود؟» را `"11"` داد. حاصل `"1" + 1` در JavaScript چیست؟

۱) `2`　۲) `11`　۳) `"2"`　۴) `"11"`

۱۲۴ آسان

در میم اسلاید ۱۹، ریاضی‌دان‌ها از دیدن `x = x + 1` تعجب می‌کنند. این دستور در JavaScript چه می‌کند؟

۱) مقدار فعلی `x` را با ۱ جمع می‌کند و حاصل را دوباره در `x` می‌ریزد.

۲) بررسی می‌کند که `x` با `x + 1` برابر است یا نه، و `false` برمی‌گرداند.

۳) خطای `SyntaxError` می‌دهد، چون چنین معادله‌ای جواب ندارد.

۴) مقدار `x` را برابر `1` می‌کند.

۱۲۵ آسان

کدام کد عدد `8` (یعنی ۲ به توان ۳) را چاپ می‌کند؟

۱) `console.log(2 ^ 3);`　۲) `console.log(2 ** 3);`

۳) `console.log(2 * 3);`　۴) `console.log(3 ** 2);`

۱۲۶ آسان

خروجی کد زیر چیست؟

```javascript
console.log(1 / 0);
console.log(-1 / 0);
console.log(0 / 0);
```

۱) `Infinity` `Infinity` `0`　۲) `Uncaught RangeError`

۳) `NaN` `NaN` `NaN`　۴) `Infinity` `-Infinity` `NaN`

۱۲۷ متوسط

خروجی کد زیر چیست؟

```javascript
let x = 10;
x += 5;
x -= 3;
x *= 2;
x /= 4;
x %= 4;
console.log(x);
```

۱) `2`　۲) `6`　۳) `1.5`　۴) `0`

۱۲۸ متوسط

حاصل `-7 % 3` در JavaScript چیست؟

۱) `2`　۲) `1`　۳) `-1`　۴) `-2`

۱۲۹ متوسط

حاصل `2 ** 3 ** 2` چیست؟

۱) `64`　۲) `512`　۳) `12`　۴) `36`

۱۳۰ سخت

اجرای کدام قطعه‌کد **خطا** می‌دهد؟

۱) `console.log((-2) ** 2);`
۲) `console.log(-(2 ** 2));`
۳) `console.log(2 ** -2);`
۴) `console.log(-2 ** 2);`

۱۳۱ متوسط

حاصل `2 + 3 * 4 ** 2` چیست؟

۱) `400`　۲) `80`　۳) `50`　۴) `146`

۱۳۲ متوسط

حاصل `20 / 4 * 2` چیست؟

۱) `10`　۲) `2.5`　۳) `40`　۴) `5`

۱۳۳ سخت

حاصل `2 * 3 ** 2 % 5` چیست؟

۱) `8`　۲) `3`　۳) `1`　۴) `18`

۱۳۴ سخت

خروجی کد زیر چیست؟

```JavaScript
let a = 5;
let b = a++;
let c = ++a;
console.log(a, b, c);
```

۱) 7 5 7
۲) 7 6 7
۳) 6 5 6
۴) 7 5 6

۱۳۵ متوسط

خروجی کد زیر چیست؟

```JavaScript
let count = 0;
count++;
count++;
count--;
console.log(count);
console.log(count++);
console.log(count);
```

۱) 1 2 2
۲) 1 1 2
۳) 2 2 3
۴) 1 1 1

۱۳۶ سخت

خروجی کد زیر چیست؟

```JavaScript
let x = 4;
x *= 2 + 3;
console.log(x);
```

۱) 11
۲) 9
۳) 24
۴) 20

۱۳۷ متوسط

خروجی کد زیر چیست؟

```JavaScript
let a = "8";
let b = "8";
a += 2;
b -= 2;
console.log(a, b);
```

۱) 10 6
۲) 82 NaN
۳) 82 6
۴) 10 10

۱۳۸ متوسط

خروجی کد زیر چیست؟

```JavaScript
console.log(0.1 + 0.2);
console.log(0.1 + 0.2 === 0.3);
```

۱) `0.3` `true`
۲) `0.3` `false`
۳) `0.30000000000000004` `false`
۴) `0.30000000000000004` `true`

۱۳۹ متوسط

کدام گزینه **نادرست** است؟

۱) حاصل `5 / 0` خطا می‌دهد، چون تقسیم بر صفر تعریف نشده است.
۲) حاصل `10 % 2` برابر `0` است، پس ۱۰ زوج است.
۳) حاصل `7 / 2` برابر `3.5` است.
۴) حاصل `Math.floor(7 / 2)` برابر `3` است.

۱۴۰ سخت

خروجی کد زیر چیست؟

```JavaScript
function isOdd(n) {
  return n % 2 === 1;
}
console.log(isOdd(3));
console.log(isOdd(-3));
console.log(-3 % 2);
```

۱) `true` `true` `1`
۲) `true` `false` `-1`
۳) `true` `false` `1`
۴) `true` `true` `-1`

۱۴۱ متوسط

می‌خواهیم **تقسیم صحیح** ۷ بر ۲ (یعنی `3`) را چاپ کنیم. در جای خالی کدام گزینه قرار بگیرد؟

```JavaScript
console.log(___); // should print 3
```

۱) `7 / 2`
۲) `7 % 2`
۳) `Math.round(7 / 2)`
۴) `Math.floor(7 / 2)`

۱۴۲ سخت

خروجی کد زیر چیست؟

```javascript
console.log(Math.floor(-7 / 2));
console.log(Math.trunc(-7 / 2));
```

۱) `-3` `-3`
۲) `-4` `-4`
۳) `-3` `-4`
۴) `-4` `-3`

۱۴۳ سخت

خروجی کد زیر چیست؟

```javascript
let score = 10;
console.log(score += 5);
console.log(score++);
console.log(score);
```

۱) `15` `16` `16`
۲) `10` `15` `16`
۳) `15` `15` `16`
۴) `Uncaught SyntaxError`

۱۴۴ سخت

کدام خط کد زیر خطا می‌دهد؟

```javascript
let points = 10;
points += 5;
const bonus = 3;
points *= bonus;
bonus++;
console.log(points);
```

۱) خط ۴
۲) خط ۵
۳) خط ۶
۴) هیچ خطایی رخ نمی‌دهد

۱۴۵ متوسط

خروجی کد زیر چیست؟

```javascript
console.log("5" + 1);
console.log("5" - 1);
console.log(1 + 2 + "3");
```

۱) `51` `4` `33`
۲) `6` `4` `33`
۳) `51` `51` `123`
۴) `51` `4` `123`

۱-۸ رشته‌ها و Template literals

منبع: Introduction to JS، اسلاید ۵۱

درسنامه

یک فرم کاغذی را تصور کن که جاهای خالی دارد: «سلام ____، شما ____ عدد ____ سفارش داده‌اید». کافی است جاهای خالی را پر کنی تا متن کامل شود. **template literal** (رشته‌ی الگو) در JavaScript دقیقاً همین فرم است: متن را می‌نویسی، جاهای خالی را با `${...}` مشخص می‌کنی و JavaScript خودش آن‌ها را با مقدار درست پر می‌کند. ولی قبل از آن، چند نکته‌ی پایه درباره‌ی رشته‌ها را مرور کنیم.

رشته‌ها: مرور پایه

رشته (string) یعنی متن؛ دنباله‌ای از کاراکترها داخل علامت نقل‌قول. سه راه برای نوشتنش داریم:

- تک‌کوتیشن: `'Hello'`
- دابل‌کوتیشن: `"Hello"`
- backtick (بک‌تیک؛ کلید بالای Tab و سمت چپ عدد ۱ در حالت انگلیسی کیبورد): `` `Hello` ``

تک‌کوتیشن و دابل‌کوتیشن هیچ فرقی با هم ندارند؛ فقط آن را انتخاب کن که با متنت تداخل نکند: `"It's fine"` یا `'He said "hi"'`. اگر همان علامتی که رشته را باز کرده داخل متن بیاید، رشته زودتر از موعد بسته می‌شود و `SyntaxError` می‌گیری؛ مثلاً `'It's broken'`.

کاراکترهای گریز (escape sequences): با بک‌اسلش `\` می‌توانی کاراکترهای خاص را داخل رشته بنویسی:

دنباله	معنی
`\n`	رفتن به خط جدید (new line)
`\t`	فاصله‌ی تب (tab)
`\"`	دابل‌کوتیشن داخل رشته‌ی دابل‌کوتیشنی
`\'`	تک‌کوتیشن داخل رشته‌ی تک‌کوتیشنی
`\\`	خود بک‌اسلش

```JavaScript
const a = 'It\'s fine';     // escaped single quote
const b = "She said \"hi\""; // escaped double quotes
const c = "Line 1\nLine 2";  // \n starts a new line
console.log(b);              // She said "hi"
console.log(c);              // Line 1
                             // Line 2
```

طول و اندیس: ویژگی `length` تعداد کاراکترها را می‌دهد و با `[i]` به کاراکتر شماره‌ی `i` (از ۰) می‌رسی. دقت کن که `\n` **یک** کاراکتر حساب می‌شود:

```JavaScript
console.log("hello".length);        // 5
console.log("".length);             // 0 (empty string)
console.log("JavaScript"[4]);       // S (indices start at 0)
console.log("Hello\nWorld".length); // 11 (5 + 1 + 5)
```

چسباندن با `+`: اگر یک طرف `+` رشته باشد، دو طرف به هم چسبانده می‌شوند (concatenation یا الحاق):
`"Hello, " + "Sara"` می‌شود `"Hello, Sara"`.

رشته‌ی معمولی چندخطی نمی‌شود: اگر وسط `"..."` یا `'...'` کلید Enter بزنی، `SyntaxError` می‌گیری؛ برای خط جدید باید `\n` بنویسی.

دردسرهای چسباندن با `+`

```JavaScript
const name = "Sara";
const age = 20;
const text = "Name: " + name + ", next year: " + (age + 1) + "\n" + "Done!";
```

این همه `+` و علامت نقل‌قول هم خواندن را سخت می‌کند و هم راه را برای اشتباه باز می‌گذارد: فاصله‌ها را فراموش می‌کنی، و اگر پرانتز `(age + 1)` را ننویسی، به‌جای جمع، چسباندن رخ می‌دهد:

```JavaScript
console.log("Sum: " + 2 + 3);   // Sum: 23 (left to right: "Sum: 2" + 3)
console.log("Sum: " + (2 + 3)); // Sum: 5
```

Template literals (اسلاید ۵۱)

- در **ES6** معرفی شدند و با **backtick** (`` ` ``) نوشته می‌شوند.
- اجازه می‌دهند رشته‌ی **چندخطی** بنویسی، **بدون** نیاز به `\n`.
- از **string interpolation** (جاسازی مقدار در رشته) با `${expression}` پشتیبانی می‌کنند؛ هر چیزی داخل `${ }` حساب می‌شود و نتیجه‌اش در متن قرار می‌گیرد.

همان مثال بالا با template literal:

```JavaScript
const text2 = `Name: ${name}, next year: ${age + 1}
Done!`;
console.log(text2); // Name: Sara, next year: 21
                    // Done!
```

کد اسلاید ۵۱

templateliterals.js

```
const product = "Laptop";
const price = 1200;
const quantity = 2;

const message = `
You ordered: ${quantity} × ${product}
Total price: $${price * quantity}
Thank you for shopping with us!
`;

console.log(message);
```

خروجی این کد **پنج خط** است:

خط خروجی	متن چاپ‌شده
۱	(خالی)
۲	`You ordered: 2 × Laptop`
۳	`Total price: $2400`
۴	`Thank you for shopping with us!`
۵	(خالی)

بیا خط‌به‌خط ببینیم چرا:

- **خط خالی اول:** بلافاصله بعد از backtick بازکننده Enter زده شده، پس رشته با یک `\n` **شروع** می‌شود.
- `${quantity}` با `2` و `${product}` با `Laptop` جایگزین می‌شود. علامت `×` بین آن‌ها فقط یک **کاراکتر معمولی متن** (علامت ضرب) است، نه عملگر؛ عیناً چاپ می‌شود.
- `$${price * quantity}` : `$` اول چون بعدش `{` نیامده، یک علامت دلار معمولی است. بعد از آن `${price * quantity}` می‌آید که حاصل `1200 * 2` یعنی `2400` را می‌گذارد؛ نتیجه: `$2400` .
- **خط خالی آخر:** قبل از backtick بسته‌کننده هم Enter زده شده، پس رشته با یک `\n` **تمام** می‌شود.

برای این‌که این خط‌های خالیِ اول و آخر را با چشم ببینی، پیام را بین دو علامت `|` چاپ کن:

JavaScript

```
console.log("|" + message + "|");
```

```
|
You ordered: 2 × Laptop
Total price: $2400
Thank you for shopping with us!
|
```

هر «عبارتی» داخل `${}` مجاز است

داخل `${}` می‌توانی هر **عبارت (expression)** بنویسی؛ یعنی هر تکه‌کدی که یک **مقدار** تولید می‌کند: محاسبه، صدا زدن تابع یا متد، مقایسه، یا عملگر سه‌تایی `شرط ? الف : ب` (اگر شرط درست بود «الف»، وگرنه «ب»):

JavaScript

```
const nm = "sara";
function double(n) {
  return n * 2;
}
console.log(`Next year: ${age + 1}`);                    // Next year: 21
console.log(`Status: ${age >= 18 ? "adult" : "minor"}`); // Status: adult
console.log(`Name: ${nm.toUpperCase()}`);                // Name: SARA
console.log(`Double: ${double(21)}`);                    // Double: 42
```

ولی **دستورها (statements)** مثل `if` ، `for` یا `let` مقدار تولید نمی‌کنند و داخل `${}` مجاز نیستند: `` `${if (x) { "yes" }}` `` خطای `SyntaxError` می‌دهد. به‌جای `if` از عملگر سه‌تایی استفاده کن.

مقدارها چطور به متن تبدیل می‌شوند؟

نتیجه‌ی هر `${}` به رشته تبدیل می‌شود و خود template literal همیشه یک **رشته‌ی معمولی** است (`typeof` آن `"string"` است):

مقدار داخل `${}`	چیزی که در متن قرار می‌گیرد
`2400`	`2400`
`true`	`true`
`null`	`null`
`undefined`	`undefined`
`[1, 2, 3]`	`1,2,3` (عضوها با کاما، بدون کروشه و فاصله)
`[]`	هیچ (رشته‌ی خالی)
`{ a: 1 }`	`[object Object]`

برای دیدن محتوای یک شیء داخل متن، از `JSON.stringify` استفاده کن (درس JSON): `` `${JSON.stringify({ a: 1 })}` `` می‌شود `{"a":1}` .

فقط backtick، جاسازی می‌کند

`${}` فقط داخل backtick معنا دارد. داخل تک‌کوتیشن یا دابل‌کوتیشن، همان حروف عیناً چاپ می‌شوند:

JavaScript

```
console.log(`Hello ${name}`); // Hello Sara
console.log("Hello ${name}"); // Hello ${name}
```

template literal تودرتو (nested)

داخل `${}` می‌شود یک template literal دیگر نوشت:

```JavaScript
const isOnline = true;
const user = "Reza";
console.log(`${user} is ${isOnline ? `online (${2 + 3} friends)` : "offline"}`);
// Reza is online (5 friends)
```

گریز (escape) داخل template literal

```JavaScript
console.log(`Use \` to write a template`); // Use ` to write a template
console.log(`Price: \${price}`);           // Price: ${price} (no interpolation)
console.log(`a\nb`);                       // a and b on two lines (\n still works)
```

فاصله‌ها و Enterها جزو رشته‌اند

در template literal چندخطی، **هر** Enter و **هر** فاصله‌ی ابتدای خط جزو رشته است:

```JavaScript
const text3 = `A
  B`;
console.log(text3.length); // 5 ("A" + new line + 2 spaces + "B")
```

مقایسه‌ی دو روش

	چسباندن با `+`	template literal
علامت	`'...'` یا `"..."`	`` `...` ``
جاسازی مقدار	`"Hi " + name`	`` `Hi ${name}` ``
چندخطی	فقط با `\n`	مستقیم با Enter
محاسبه داخل متن	نیاز به پرانتز: `"Sum: " + (a + b)`	`` `Sum: ${a + b}` ``

نکته

- هر وقت متغیر یا محاسبه‌ای داخل متن داری، template literal خواناتر و کم‌خطاتر از `+` است.
- `$` معمولی (بدون `{` بعدش) فقط یک کاراکتر است؛ برای همین `$${price}` می‌شود علامت دلار و بعد قیمت.
- برای نوشتن خود backtick یا `${` به‌صورت متن، از `\`` و `\${` استفاده کن.

دام تستی

- `"Hello ${name}"` (با دابل‌کوتیشن) جاسازی **نمی‌کند** و عیناً `Hello ${name}` چاپ می‌کند.
- `"Sum: " + 2 + 3` برابر `"Sum: 23"` است، ولی `` `Sum: ${2 + 3}` `` برابر `"Sum: 5"` .
- آرایه داخل `${}` به شکل `1,2,3` و شیء به شکل `[object Object]` درمی‌آید.
- خروجی کد اسلاید ۵۱ یک خط خالی در **ابتدا** و یک خط خالی در **انتها** دارد، چون رشته با Enter شروع و تمام شده است.
- `\n` یک کاراکتر است: `"Hello\nWorld".length` برابر `11` است، نه `12` .
- داخل `${}` فقط **عبارت** مجاز است؛ `if` و `for` خطای `SyntaxError` می‌دهند.

جمع‌بندی

- رشته‌ها با `'...'`، `"..."` یا backtick نوشته می‌شوند؛ کاراکترهای خاص با `\` (مثل `\n`، `\"`، `\``).
- template literal (ES6، با backtick): چندخطی بدون `\n` و جاسازی با `${expression}` .
- داخل `${}` هر عبارتی مجاز است (محاسبه، تابع، سه‌تایی، حتی template دیگر)، ولی دستور نه. نتیجه همیشه به رشته تبدیل می‌شود.
- در کد اسلاید ۵۱، `$${price * quantity}` می‌شود `$2400` و خروجی با یک خط خالی شروع و تمام می‌شود.

تست‌های تمرینی

۲۱ تست · شماره‌ی ۱۴۶ تا ۱۶۶

۱۴۶ آسان

template literal با کدام علامت نوشته می‌شود؟

۱) `'`　۲) `"`　۳) `` ` ``　۴) `$`

۱۴۷ آسان

template literalها در کدام نسخه‌ی ECMAScript به JavaScript اضافه شدند؟

۱) ES5 (سال ۲۰۰۹)
۲) ES6 یا همان ES2015
۳) ES1 (سال ۱۹۹۷)
۴) ES2020

۱۴۸ آسان

کدام مورد از ویژگی‌های template literal که در اسلاید ۵۱ آمده **نیست**؟

۱) نوشتن رشته‌ی چندخطی بدون نیاز به `\n`
۲) جاسازی (interpolation) مقدار عبارت‌ها با `${expression}`
۳) نوشته شدن بین دو backtick
۴) تبدیل خودکار رشته به عدد

۱۴۹ آسان

حاصل `typeof \`Hello ${1 + 1}\`` چیست؟

۱) `"template"`　۲) `"object"`　۳) `"number"`　۴) `"string"`

۱۵۰ آسان

خروجی کد زیر چیست؟

```javascript
const name = "Sara";
console.log(`Hello, ${name}!`);
console.log("Hello, ${name}!");
```

۱) `Hello, Sara!` `Hello, ${name}!`

۲) `Hello, Sara!` `Hello, Sara!`

۳) `Hello, ${name}!` `Hello, Sara!`

۴) `Hello, Sara!` `Uncaught SyntaxError`

۱۵۱ آسان

خروجی کد زیر چیست؟

```javascript
const a = 5;
const b = 10;
console.log(`${a} + ${b} = ${a + b}`);
```

۱) `5 + 10 = 15`

۲) `${a} + ${b} = ${a + b}`

۳) `5 + 10 = 510`

۴) `a + b = 15`

۱۵۲ متوسط

حاصل `"Hello\nWorld".length` چیست؟

۱) `12`

۲) `10`

۳) `13`

۴) `11`

۱۵۳ متوسط

اجرای کدام قطعه‌کد **خطا** می‌دهد؟

۱) `const c = 'It's broken';`

۲) `const a = "It's fine";`

۳) `const b = 'He said "hi"';`

۴) `const d = `It's "fine"`;`

۱۵۴ متوسط

اجرای کدام قطعه‌کد **خطا** می‌دهد؟

۱)
```JavaScript
const a = `line 1
line 2`;
```

۲)
```JavaScript
const c = "line 1
line 2";
```

۳) `const b = "line 1\nline 2";`

۴) `const d = `line 1` + "\n" + `line 2`;`

۱۵۵ متوسط

خروجی کد اسلاید ۵۱ (کد زیر) کدام است؟

```JavaScript
const product = "Laptop";
const price = 1200;
const quantity = 2;

const message = `
You ordered: ${quantity} × ${product}
Total price: $${price * quantity}
Thank you for shopping with us!
`;

console.log(message);
```

۱) `You ordered: ${quantity} × ${product}` `Total price: $${price * quantity}` `Thank you for shopping with us!`

۲) `You ordered: 2 × Laptop` `Total price: $2400` `Thank you for shopping with us!`

۳) `You ordered: 2 × Laptop` `Total price: $1200 * 2` `Thank you for shopping with us!`

۴) `You ordered: 2 × Laptop` `Total price: 2400` `Thank you for shopping with us!`

۱۵۶ سخت

کد اسلاید ۵۱ را طوری تغییر داده‌ایم که پیام بین دو علامت `|` چاپ شود. خروجی کدام است؟

```javascript
const product = "Laptop";
const price = 1200;
const quantity = 2;

const message = `
You ordered: ${quantity} × ${product}
Total price: $${price * quantity}
Thank you for shopping with us!
`;

console.log("|" + message + "|");
```

۱) `|You ordered: 2 × Laptop` `Total price: $2400` `Thank you for shopping with us!|`

۲) `|You ordered: 2 × Laptop Total price: $2400 Thank you for shopping with us!|`

۳) `|` `You ordered: 2 × Laptop` `Total price: $2400` `Thank you for shopping with us!` `|`

۴) `|` `You ordered: 2 × Laptop` `Total price: 2400` `Thank you for shopping with us!` `|`

۱۵۷ متوسط

می‌خواهیم خروجی دقیقاً `Total: $30` باشد. در جای خالی کدام گزینه قرار بگیرد؟

```javascript
const price = 10;
const qty = 3;
console.log(___); // should print: Total: $30
```

۱) `` `Total: ${price * qty}` ``

۲) `"Total: $${price * qty}"`

۳) `` `Total: $${price * qty}` ``

۴) `` `Total: $$price * qty` ``

۱۵۸ متوسط

خروجی کد زیر چیست؟

```javascript
const age = 20;
console.log(`Status: ${age >= 18 ? "adult" : "minor"}`);
console.log(`Next year: ${age + 1}`);
console.log(`Name: ${"sara".toUpperCase()}`);
```

۱) `Status: adult` `Next year: 201` `Name: SARA`

۲) `Status: adult` `Next year: 21` `Name: SARA`

۳) `Status: adult` `Next year: 21` `Name: sara.toUpperCase()`

۴) `Uncaught SyntaxError`

۱۵۹ سخت

اجرای کدام قطعه‌کد **خطا** می‌دهد؟

۱) `const s = `${1 > 0 ? "yes" : "no"}`;`

۲) `const s = `${Math.max(1, 2)}`;`

۳) `const s = `${if (true) { "yes" }}`;`

۴) `const s = `${(2 + 3) * 4}`;`

۱۶۰ متوسط

حاصل عبارت `` `${[1, 2, 3]}` `` چیست؟

۱) `"1,2,3"`

۲) `"[1, 2, 3]"`

۳) `"1 2 3"`

۴) `"[object Array]"`

۱۶۱ متوسط

خروجی کد زیر چیست؟

```JavaScript
const user = { name: "Ali" };
const tags = ["js", "web"];
console.log(`User: ${user}`);
console.log(`Tags: ${tags}`);
```

۱) `User: { name: 'Ali' }` `Tags: ['js', 'web']`

۲) `User: Ali` `Tags: js web`

۳) `User: [object Object]` `Tags: [js,web]`

۴) `User: [object Object]` `Tags: js,web`

۱۶۲ متوسط

خروجی کد زیر چیست؟

```JavaScript
let city;
const phone = null;
const active = false;
console.log(`City: ${city}, Phone: ${phone}, Active: ${active}`);
```

۱) `City: , Phone: , Active: false`

۲) `City: undefined, Phone: null, Active: false`

۳) `City: undefined, Phone: undefined, Active: false`

۴) `Uncaught ReferenceError`

۱۶۳ سخت

خروجی کد زیر چیست؟

```JavaScript
const a = 2;
const b = 3;
console.log("Sum: " + a + b);
console.log(`Sum: ${a + b}`);
console.log("Sum: " + (a + b));
```

۱) `Sum: 5` `Sum: 5` `Sum: 5`

۲) `Sum: 23` `Sum: 23` `Sum: 5`

۳) `Sum: 23` `Sum: 5` `Sum: 5`

۴) `Sum: 23` `Sum: 5` `Sum: 23`

۱۶۴ سخت

خروجی کد زیر چیست؟

```JavaScript
const isOnline = true;
const user = "Reza";
console.log(`${user} is ${isOnline ? `online (${2 + 3} friends)` : "offline"}`);
```

۱) `Reza is offline`

۲) `Reza is online (${2 + 3} friends)`

۳) `Uncaught SyntaxError`

۴) `Reza is online (5 friends)`

۱۶۵ سخت

خروجی کد زیر چیست؟ (متغیری به نام `price` تعریف نشده است.)

```JavaScript
console.log(`Use \` to write a template`);
console.log(`Price: \${price}`);
console.log("She said \"hi\"");
```

۱) `Use ` to write a template` `Price: ${price}` `She said "hi"`

۲) `Use ` to write a template` `Uncaught ReferenceError`

۳) `Uncaught SyntaxError`

۴) `Use \` to write a template` `Price: \${price}` `She said \"hi\"`

۱۶۶ سخت

خروجی کد زیر چیست؟ (خط دوم رشته با **دو فاصله** شروع شده است.)

```JavaScript
const text = `A
  B`;
console.log(text.length);
```

۱) 3
۲) 5
۳) 2
۴) 4

۱-۹ تبدیل نوع صریح (Casting)

منبع: Introduction to JS، اسلاید ۲۰

درسنامه

فرض کن از سفر برگشته‌ای و چند اسکناس دلار در جیبت داری، اما مغازه‌ی سر کوچه فقط ریال قبول می‌کند. چه می‌کنی؟ به صرافی می‌روی و **خودت صریحاً** می‌گویی: «لطفاً این را به ریال تبدیل کن». در برنامه‌نویسی هم همین اتفاق زیاد می‌افتد: مقداری داریم که از یک نوع است (مثلاً رشته‌ی `"42"` که کاربر در یک فرم تایپ کرده)، ولی برای کارمان نوع دیگری لازم داریم (عدد `42` تا بتوانیم با آن حساب کنیم).

Casting چیست؟

طبق اسلاید ۲۰، **casting** (تبدیل نوع؛ یعنی تبدیل یک مقدار از یک نوع داده به نوعی دیگر) در JavaScript دو شکل دارد:

1. **Explicit casting** (تبدیل صریح): خودمان با صدا زدن توابع `Number()`، `String()` و `Boolean()` درخواست تبدیل می‌کنیم؛ درست مثل رفتن به صرافی.
2. **Implicit casting** یا **type coercion** (تبدیل ضمنی یا خودکار): خود JavaScript وسط یک عملیات، بدون اینکه ما چیزی بخواهیم، نوع را عوض می‌کند. این موضوعِ درس بعد است و اینجا فقط پیش‌نمایشش را می‌بینیم.

کد اسلاید ۲۰:

```JavaScript
// Explicit casting: we ask for the conversion ourselves
console.log(Number("42")); // 42   (string -> number)
console.log(String(99));   // "99" (number -> string)
console.log(Boolean(1));   // true (number -> boolean)

// Implicit casting (type coercion): JavaScript converts on its own
console.log("5" + 1);      // "51"
console.log(2 * "3");      // 6
```

> **قرارداد این کتاب درباره‌ی گیومه‌ها**
>
> `console.log` رشته‌ها را **بدون** گیومه چاپ می‌کند؛ یعنی `console.log(String(99))` در کنسول فقط `99` نشان می‌دهد. ما در توضیحِ کنار کد (comment) رشته‌ها را داخل گیومه می‌نویسیم (`"99"`) تا معلوم باشد نتیجه رشته است، نه عدد. در گزینه‌های تست‌های «خروجی کد» دقیقاً همان چیزی آمده که کنسول چاپ می‌کند.

فعالیت کلاسی اسلاید ۲۰: حاصل `"5" - 1` چیست؟

جواب: عدد `4`. عملگر `-` برای رشته‌ها هیچ معنایی ندارد (چیزی به اسم «تفریق رشته» نداریم)، پس JavaScript رشته‌ی `"5"` را به عدد `5` تبدیل می‌کند و `5 - 1` می‌شود `4`. در مقابل، `+` برای رشته‌ها معنا دارد (چسباندن یا concatenation)، برای همین `"5" + 1` می‌شود `"51"`. جزئیات این رفتار را در درس بعد (Type coercion) می‌خوانیم.

```JavaScript
console.log("5" - 1);          // 4 (a number)
console.log(typeof ("5" - 1)); // "number"
console.log("5" + 1);          // "51" (a string)
```

تبدیل به عدد: `Number(x)`

`Number` تبدیل‌کننده‌ای **سخت‌گیر** است: یا **کل** رشته یک عدد معتبر است، یا نتیجه `NaN` (Not a Number؛ یعنی «عدد معتبری نیست») می‌شود. جدول زیر را با دقت بخوان؛ بیشتر تست‌ها از همین‌جا طرح می‌شوند:

مقدار ورودی	`Number(...)`	چرا؟
`"42"`	`42`	رشته‌ی عددی معتبر
`"3.14"`	`3.14`	عدد اعشاری هم قبول است
`" 12 "`	`12`	فاصله‌های ابتدا و انتها نادیده گرفته می‌شوند
`""` و `" "`	`0`	رشته‌ی خالی (یا فقط فاصله) صفر حساب می‌شود
`"12px"`	`NaN`	حتی یک کاراکتر غیرعددی کل تبدیل را خراب می‌کند
`"abc"`	`NaN`	اصلاً عدد نیست
`false` / `true`	`0` / `1`	
`null`	`0`	«هیچ» به صفر تبدیل می‌شود
`undefined`	`NaN`	«تعریف‌نشده» عدد معتبری ندارد
`[]`	`0`	آرایه‌ی خالی اول رشته‌ی خالی می‌شود: `[] -> "" -> 0`
`[5]`	`5`	`[5] -> "5" -> 5`
`[1, 2]`	`NaN`	`[1, 2] -> "1,2" -> NaN` (ویرگول جزو عدد نیست)
`{}`	`NaN`	`{} -> "[object Object]" -> NaN`

آرایه‌ها و شیءها (object) مستقیم به عدد تبدیل نمی‌شوند: **اول به رشته** تبدیل می‌شوند و بعد آن رشته به عدد. آرایه به رشته‌ای تبدیل می‌شود که عناصرش با ویرگول به هم چسبیده‌اند و هر شیء ساده به `"[object Object]"`.

JavaScript

```
console.log(Number(""));        // 0
console.log(Number("  12  "));  // 12
console.log(Number("12px"));    // NaN
console.log(Number(null));      // 0
console.log(Number(undefined)); // NaN
console.log(Number(true));      // 1
console.log(Number([]));        // 0
console.log(Number([5]));       // 5
console.log(Number([1, 2]));    // NaN
console.log(Number({}));        // NaN
```

`parseInt` و `parseFloat`: خواندن از ابتدای رشته

این دو تابع **آسان‌گیر**ند: رشته را از سمت چپ کاراکتربه‌کاراکتر می‌خوانند، به **اولین کاراکتر نامعتبر** که برسند می‌ایستند و هر چه تا آنجا خوانده‌اند را برمی‌گردانند. اگر همان کاراکتر اول نامعتبر باشد، نتیجه `NaN` است.

- `parseInt` (parse integer؛ یعنی «عدد صحیح را بخوان») فقط رقم می‌خواند و به نقطه‌ی اعشار که برسد می‌ایستد؛ پس بخش اعشاری را **دور می‌ریزد** و گرد نمی‌کند!
- `parseFloat` (یعنی «عدد اعشاری را بخوان») اولین نقطه‌ی اعشار را هم می‌پذیرد.

```JavaScript
console.log(parseInt("12px"));    // 12   (stops at "p")
console.log(parseInt("3.9"));     // 3    (stops at "."; no rounding!)
console.log(parseFloat("3.9kg")); // 3.9  (stops at "k")
console.log(parseInt("abc"));     // NaN  (the first character is already invalid)
console.log(parseInt("px12"));    // NaN  (it must START with a number)
console.log(parseInt("  42  "));  // 42   (leading spaces are skipped)
console.log(parseInt(""));        // NaN  (but Number("") is 0!)
```

مقایسه‌ی سه تابع (این جدول را حفظ کن):

ورودی	Number	parseInt	parseFloat
`"42"`	42	42	42
`"3.9"`	3.9	3	3.9
`"12px"`	NaN	12	12
`"3.9kg"`	NaN	3	3.9
`""`	0	NaN	NaN
`"abc"`	NaN	NaN	NaN

بیشتر بدانیم

`parseInt` یک ورودی دوم اختیاری هم دارد به نام **radix** (مبنای عدد): `parseInt("ff", 16)` برابر `255` و `parseInt("101", 2)` برابر `5` است. در کدهای واقعی معمولاً برای اطمینان می‌نویسند `parseInt(text, 10)`.

تبدیل به رشته: `String(x)` و `toString()`

تبدیل به رشته ساده‌ترین تبدیل است: تقریباً هر مقدار همان‌طور که «دیده می‌شود» به متن تبدیل می‌شود.

```JavaScript
console.log(String(99));        // "99"
console.log(String(true));      // "true"
console.log(String(null));      // "null"
console.log(String(undefined)); // "undefined"
console.log(String([1, 2]));    // "1,2"  (elements joined with commas)
console.log(String([]));        // ""     (an empty string)
console.log(String({}));        // "[object Object]"
console.log((123).toString());  // "123"
```

راه دوم، متد `toString()` است که روی خود مقدار صدا زده می‌شود: `(123).toString()` یا `n.toString()`. دو نکته درباره‌اش:

- روی `null` و `undefined` **کار نمی‌کند** و خطای `TypeError` می‌دهد (مثلاً `null.toString()`)؛ اما `String(null)` بی‌خطر است و `"null"` می‌دهد.
- برای عددی که مستقیم در کد نوشته شده، پرانتز لازم است: `(123).toString()`. اگر بنویسی `123.toString()`، JavaScript نقطه را نقطه‌ی اعشارِ عدد فرض می‌کند و خطای نحوی (SyntaxError) می‌دهد. روی یک متغیر مشکلی نیست: `n.toString()`.

تبدیل به boolean: `Boolean(x)` ، مقدارهای truthy و falsy

در JavaScript فقط **۸ مقدار** هنگام تبدیل به boolean، `false` می‌شوند. به این‌ها **falsy** (شبه‌نادرست) می‌گوییم:

`false` ، `0` ، `-0` ، `0n` (صفرِ BigInt)، `""` (رشته‌ی خالی)، `null` ، `undefined` ، `NaN`

هر مقدار دیگری **truthy** (شبه‌درست) است و `true` می‌شود؛ حتی این‌ها که ظاهرشان گول‌زننده است: `"0"` ، `"false"` ، `" "` (رشته‌ای که فقط یک فاصله دارد)، `[]` (آرایه‌ی خالی)، `{}` (شیء خالی)، `-1` و `Infinity` .

> **یک تشبیه ساده**
>
> falsyها را مثل یک «لیست سیاه» ۸ نفره دمِ در یک مهمانی تصور کن: نگهبان فقط همین ۸ نفر را راه نمی‌دهد و بقیه، با هر ظاهری، وارد می‌شوند. رشته‌ی `"0"` شبیه عدد `0` است، ولی اسمش در لیست سیاه نیست؛ یک رشته‌ی غیرخالی است، پس truthy است.

JavaScript

```
console.log(Boolean(1));       // true  (slide 20)
console.log(Boolean(0));       // false
console.log(Boolean(""));      // false (empty string)
console.log(Boolean(" "));     // true  (a space is a character!)
console.log(Boolean("0"));     // true  (non-empty string)
console.log(Boolean("false")); // true  (non-empty string)
console.log(Boolean([]));      // true  (every array/object is truthy)
console.log(Boolean({}));      // true
console.log(Boolean(null));    // false
console.log(Boolean(NaN));     // false
```

این همان تبدیلی است که شرط `if` پشت پرده انجام می‌دهد: `if (x)` یعنی «اگر `x` truthy است». (درباره‌ی `if` در درس «دستورهای شرطی» مفصل می‌خوانیم.)

میان‌برها: `+x` ، `!!x` و `x + ""`

برنامه‌نویس‌های JavaScript اغلب به جای نوشتن نام تابع از این میان‌برها استفاده می‌کنند و تو باید بتوانی آن‌ها را بخوانی:

میان‌بر	معادل	مثال
`+x` (unary plus؛ علامت مثبتِ تک‌عملوندی)	`Number(x)`	`+"42"` می‌شود `42` ، `+""` می‌شود `0` ، `+true` می‌شود `1` ، `+"3.5px"` می‌شود `NaN`
`!!x` (دو بار نقیض)	`Boolean(x)`	`!!"hello"` می‌شود `true` ، `!!0` می‌شود `false`
`x + ""`	`String(x)`	`5 + ""` می‌شود `"5"`

(تنها استثنای `+x` مقدارهای BigInt هستند: `Number(10n)` برابر `10` است، ولی `+10n` خطای `TypeError` می‌دهد.)

`!!x` چطور کار می‌کند؟ عملگر `!` (NOT) مقدار را به boolean تبدیل و برعکس می‌کند؛ `!` دوم دوباره برعکسش می‌کند. پس `!!"hello"` یعنی `!false` ، یعنی `true` .

JavaScript

```
console.log(+"42", +"", +true, +"3.5px"); // 42 0 1 NaN
console.log(!!"hello", !!0, !!"0");       // true false true
console.log(5 + "", typeof (5 + ""));     // "5" "string"
```

`NaN` : «عدد نیست»ی که نوعش number است!

`NaN` (کوتاه‌شده‌ی Not a Number) نتیجه‌ی یک تبدیل یا محاسبه‌ی عددیِ **شکست‌خورده** است: `Number("abc")` ، `0 / 0` ، `"abc" * 2` یا `undefined + 1` . سه ویژگی عجیب دارد:

1. `typeof NaN` برابر `"number"` است! `NaN` یک مقدار خاص از نوع number است که معنایش «عدد نامعتبر» است.
2. **مسری** است: هر محاسبه‌ی ریاضی با `NaN` نتیجه‌اش `NaN` است (`NaN + 1` برابر `NaN` است).
3. با **هیچ** چیز، حتی خودش، برابر نیست: `NaN === NaN` برابر `false` است (در درس «عملگرهای مقایسه و منطقی» بیشتر می‌بینیم). پس برای تشخیصش نمی‌توان از `===` کمک گرفت.

برای تشخیص `NaN` دو تابع داریم که فرق مهمی با هم دارند:

- `isNaN(x)` (تابع قدیمی و سراسری): **اول** `x` را با `Number` به عدد تبدیل می‌کند و **بعد** می‌پرسد «نتیجه `NaN` است؟». پس `isNaN("abc")` برابر `true` است، چون `"abc"` به `NaN` تبدیل می‌شود.
- `Number.isNaN(x)` (از ES6): **هیچ تبدیلی** انجام نمی‌دهد و فقط وقتی `true` می‌دهد که `x` واقعاً خودِ مقدار `NaN` باشد. پس `Number.isNaN("abc")` برابر `false` است؛ `"abc"` یک رشته است، نه `NaN` .

JavaScript

```
console.log(typeof NaN);          // "number"
console.log(isNaN("abc"));        // true  ("abc" -> NaN)
console.log(Number.isNaN("abc")); // false (a string is not the NaN value)
console.log(isNaN(""));           // false ("" -> 0)
console.log(isNaN(undefined));    // true  (undefined -> NaN)
console.log(Number.isNaN(NaN));   // true
console.log(Number.isNaN(0 / 0)); // true
```

نکته

هر چیزی که از ورودی کاربر می‌آید (مقدار یک فیلد فرم یا جواب `prompt`) **رشته** است. قبل از محاسبه صریحاً تبدیلش کن: اگر کل ورودی باید عدد باشد `Number(input)` ، و اگر عدد در ابتدای متن آمده (مثل `"12px"`) `parseFloat` / `parseInt` . برای تشخیص `NaN` هم همیشه `Number.isNaN` را ترجیح بده.

دام تستی

- `Number("")` برابر `0` است، ولی `parseInt("")` برابر `NaN` .
- `parseInt("3.9")` برابر `3` است، نه `4` (گرد نمی‌کند، فقط می‌بُرد).
- `Boolean("0")` و `Boolean("false")` هر دو `true` هستند (رشته‌ی غیرخالی)، ولی `Boolean("")` برابر `false` است.
- `Number(" ")` برابر `0` است، ولی `Boolean(" ")` برابر `true` .
- `typeof NaN` برابر `"number"` است، نه `"NaN"` .
- `isNaN("abc")` برابر `true` است، ولی `Number.isNaN("abc")` برابر `false` .
- `String(null)` می‌شود `"null"` ، اما `null.toString()` خطای `TypeError` می‌دهد.

بیشتر بدانیم

`Number` ، `String` و `Boolean` را برای تبدیل همیشه **بدون** `new` صدا بزن. `new Number(5)` یک شیء «پوشاننده» (wrapper object) می‌سازد که `typeof` آن `"object"` است، نه `"number"` .

جمع‌بندی

تبدیل	تابع صریح	میان‌بر	نکته‌ی کلیدی
به عدد	`Number(x)`	`+x`	سخت‌گیر: `"12px"` می‌شود `NaN` ؛ `""` ، `null` و `[]` می‌شوند `0` ؛ `undefined` می‌شود `NaN`
به عدد (از ابتدای متن)	`parseInt` / `parseFloat`	—	تا اولین کاراکتر نامعتبر می‌خواند؛ `""` می‌شود `NaN`
به رشته	`String(x)`	`x + ""`	`[1, 2]` می‌شود `"1,2"` ؛ `{}` می‌شود `"[object Object]"`
به boolean	`Boolean(x)`	`!!x`	فقط ۸ مقدار falsy داریم؛ بقیه truthy‌اند

تست‌های تمرینی

۲۵ تست · شماره‌ی ۱۶۷ تا ۱۹۱

۱۶۷ آسان

طبق اسلاید ۲۰، **casting** در JavaScript به چه معناست؟

۱) ذخیره کردن یک مقدار در یک متغیر تازه

۲) تبدیل یک مقدار از یک نوع داده به نوعی دیگر

۳) بررسی نوع یک مقدار با عملگر `typeof`

۴) چاپ یک مقدار در کنسول با `console.log`

۱۶۸ آسان

کدام عبارت یک تبدیل نوع **صریح** (explicit casting) است؟

۱) `"5" + 1` ۲) `2 * "3"` ۳) `"5" - 1` ۴) `String(99)`

۱۶۹ آسان

حاصل `Number("42")` کدام است؟

۱) `"42"` ۲) `NaN` ۳) `42` ۴) `undefined`

۱۷۰ آسان

فعالیت کلاسی اسلاید ۲۰: حاصل `"5" - 1` کدام است؟

۱) `4` ۲) `"51"` ۳) `"4"` ۴) `NaN`

۱۷۱ متوسط

خروجی کد زیر کدام است؟

```JavaScript
console.log(Number(""));
console.log(Number("  12  "));
console.log(Number("12px"));
```

۱) `0` `12` `NaN`
۲) `NaN` `12` `12`
۳) `0` `NaN` `12`
۴) `NaN` `NaN` `NaN`

۱۷۲ متوسط

کدام گزینه **نادرست** است؟

۱) `Number(null)` برابر `0` است.
۲) `Number(true)` برابر `1` است.
۳) `Number(undefined)` برابر `0` است.
۴) `Number([])` برابر `0` است.

۱۷۳ متوسط

خروجی کد زیر کدام است؟

```JavaScript
console.log(Number([]));
console.log(Number([5]));
console.log(Number([1, 2]));
console.log(Number({}));
```

۱) `NaN` `5` `3` `NaN`
۲) `0` `5` `12` `0`
۳) `NaN` `NaN` `NaN` `NaN`
۴) `0` `5` `NaN` `NaN`

۱۷۴ سخت

حاصل کدام عبارت `NaN` **نیست**؟

۱) `Number("12px")`
۲) `Number(undefined)`
۳) `Number(" ")`
۴) `Number({})`

۱۷۵ متوسط

خروجی کد زیر کدام است؟

```javascript
console.log(parseInt("12px"));
console.log(parseInt("3.9"));
console.log(parseFloat("3.9kg"));
console.log(parseInt("abc"));
```

۱) `12` `4` `3.9` `NaN`
۲) `NaN` `3` `NaN` `NaN`
۳) `12` `3` `3.9` `NaN`
۴) `12` `3` `3.9` `0`

۱۷۶ متوسط

در جای خالی کد زیر کدام تابع قرار گیرد تا خروجی `3.9` شود؟

```javascript
console.log(____("3.9kg"));
```

۱) `parseInt`
۲) `Number`
۳) `String`
۴) `parseFloat`

۱۷۷ سخت

خروجی کد زیر کدام است؟

```javascript
console.log(Number(""), parseInt(""));
console.log(Number("3.9"), parseInt("3.9"));
console.log(Number("12px"), parseInt("12px"));
```

۱) `0 0` `3.9 4` `NaN 12`
۲) `0 NaN` `3.9 3` `NaN 12`
۳) `NaN NaN` `3.9 3` `12 12`
۴) `0 NaN` `4 3` `NaN NaN`

۱۷۸ آسان

حاصل `typeof String(99)` کدام است؟

۱) `"number"`
۲) `"string"`
۳) `"object"`
۴) `"undefined"`

۱۷۹ متوسط

حاصل `String(null)` کدام است؟

۱) `""`
۲) `"null"`
۳) `null`
۴) خطای `TypeError`

۱۸۰ متوسط

خروجی کد زیر کدام است؟

```javascript
console.log(String([1, 2]));
console.log(String({}));
console.log(String([]).length);
```

۱) `1,2` `[object Object]` `0`
۲) `[1, 2]` `{}` `2`
۳) `12` `[object Object]` `0`
۴) `1,2` `object` `1`

۱۸۱ سخت

اجرای کد زیر در کدام خط با خطا متوقف می‌شود؟

```javascript
1 const a = String(null);
2 const b = (123).toString();
3 const c = String(undefined);
4 const d = null.toString();
5 console.log(a, b, c, d);
```

۱) خط ۱
۲) خط ۲
۳) خط ۴
۴) هیچ خطایی رخ نمی‌دهد

۱۸۲ آسان

کدام مقدار **falsy** است (یعنی `Boolean` آن `false` می‌شود)؟

۱) `"0"`
۲) `" "`
۳) `[]`
۴) `NaN`

۱۸۳ متوسط

کدام گزینه **درست** است؟

۱) `Boolean("false")` برابر `false` است.
۲) `Boolean(" ")` برابر `true` است.
۳) `Boolean(0n)` برابر `true` است.
۴) `Boolean({})` برابر `false` است.

۱۸۴ آسان

اگر `x` یک **رشته** باشد، کدام عبارت همیشه همان نتیجه‌ی `Number(x)` را می‌دهد؟

۱) `!!x`
۲) `+x`
۳) `x + ""`
۴) `-x`

۱۸۵ متوسط

خروجی کد زیر کدام است؟

```javascript
console.log(!!"0", !!"", !![], !!NaN);
```

۱) `true false true false`
۲) `false false false false`
۳) `false false true false`
۴) `true false false false`

۱۸۶ متوسط

خروجی کد زیر کدام است؟

```javascript
console.log(+"", +"3.5", +"3.5px", +true, +[]);
```

۱) `0 3.5 NaN 1 0`
۲) `NaN 3.5 3 1 NaN`
۳) `0 3.5 3.5 true 0`
۴) `NaN 3.5 NaN 1 0`

۱۸۷ سخت

کدام مقدار در شرط `if` است و در عین حال **truthy** `Number` آن برابر `0` می‌شود؟

۱) `""`
۲) `null`
۳) `"1"`
۴) `"0"`

۱۸۸ آسان

کدام گزینه دربارهٔ `NaN` **درست** است؟

۱) نوع آن `"NaN"` است؛ یعنی `typeof NaN` برابر `"NaN"` است.
۲) نتیجهٔ یک تبدیل یا محاسبهٔ عددیِ ناموفق است؛ مثل `Number("abc")` یا `0 / 0`.
۳) با `===` می‌توان تشخیصش داد؛ یعنی `x === NaN` برای `x = NaN` برابر `true` است.
۴) هر محاسبهٔ ریاضی با آن، مقدار `0` می‌دهد؛ مثلاً `NaN + 1` برابر `0` است.

۱۸۹ سخت

خروجی کد زیر کدام است؟

```javascript
console.log(isNaN("abc"));
console.log(Number.isNaN("abc"));
console.log(isNaN(""));
console.log(Number.isNaN(NaN));
```

۱) `true` `true` `true` `true`

۲) `true` `false` `true` `true`

۳) `false` `false` `false` `true`

۴) `true` `false` `false` `true`

۱۹۰ سخت

کدام گزینه **درست** است؟

۱) `isNaN("12")` برابر `true` است.

۲) `Number.isNaN("abc")` برابر `true` است.

۳) `typeof Number("abc")` برابر `"number"` است.

۴) `isNaN(undefined)` برابر `false` است.

۱۹۱ سخت

مقدار `age` از یک فیلد فرم خوانده شده است. خروجی کد زیر کدام است؟

```javascript
const age = "20"; // values read from form fields are strings
console.log(age + 1);
console.log(Number(age) + 1);
console.log(+age + 1);
console.log(parseInt(age) + "1");
```

۱) `201` `21` `21` `201`

۲) `21` `21` `21` `21`

۳) `201` `21` `201` `201`

۴) `201` `21` `21` `21`

۱-۱۰ تبدیل نوع ضمنی (Type coercion) و دام‌هایش

منبع: Introduction to JS، اسلایدهای ۲۰ تا ۲۲

درسنامه

از میم «1 + 1» (اسلاید ۱۸) تا type coercion

میم اسلاید ۱۸ را در درس «عملگرهای حسابی و انتساب» دیدیم: رئیس می‌پرسد «`1 + 1` چند می‌شود؟»؛ Python و C++ می‌گویند `2`، JavaScript می‌گوید `11` و از پنجره بیرون انداخته می‌شود! در واقع در JavaScript هم `1 + 1` برابر `2` است. جوابِ `"11"` فقط وقتی پیدا می‌شود که یکی از دو طرف **رشته** باشد (`"1" + 1`): JavaScript به جای خطا دادن، بی‌صدا عدد را به رشته تبدیل می‌کند و دو رشته را به هم می‌چسباند. در این درس می‌فهمیم این «تبدیل بی‌صدا»، یعنی **type coercion**، دقیقاً با چه قاعده‌هایی کار می‌کند.

```JavaScript
console.log(1 + 1);   // 2
console.log("1" + 1); // "11"  (the joke in the meme)
```

Type coercion چیست؟

طبق اسلاید ۲۲، **type coercion** (تبدیل نوع ضمنی؛ یعنی تبدیلی که خود JavaScript بدون درخواست ما انجام می‌دهد) یعنی تبدیل **خودکار** یک نوع داده به نوعی دیگر، وقتی در یک عملیات مقدارهایی از **نوع‌های مختلف** شرکت دارند. این رفتار یکی از ویژگی‌های کلیدی **dynamic typing system** (سیستم نوع‌دهی پویا) در JavaScript است: متغیرها نوع ثابتی ندارند و موتور JavaScript سعی می‌کند حتی وقتی نوع دو طرف عملیات یکی نیست، «یک معنایی» برای آن پیدا کند.

> **یک تشبیه ساده**
>
> JavaScript مثل گارسونی است که هیچ‌وقت نمی‌گوید «نداریم»: اگر چیزی سفارش بدهی که دقیقاً در منو نیست، نزدیک‌ترین چیزی را که به ذهنش می‌رسد می‌آورد. Python در همین موقعیت می‌گوید «این سفارش معنا ندارد» (برای `"1" + 1` خطای `TypeError` می‌دهد). برای همین باید قاعده‌های «حدس زدنِ» JavaScript را دقیق بلد باشی.

در درس قبل (Casting) دیدیم که خودمان می‌توانیم صریحاً با `Number()`، `String()` و `Boolean()` نوع را عوض کنیم. type coercion همان تبدیل‌هاست، با این فرق که JavaScript **خودش** و بی‌صدا انجامشان می‌دهد. شش قاعده‌ی زیر تقریباً همه‌ی تست‌ها را پوشش می‌دهند.

قاعده‌ی ۱: `+` با یک رشته یعنی چسباندن

اسلاید ۲۲: `String + Anything -> String`؛ یعنی اگر **حداقل یکی** از دو طرف `+` رشته باشد، طرف دیگر هم به رشته تبدیل می‌شود و دو رشته به هم چسبانده می‌شوند (concatenation):

```JavaScript
console.log("5" + 1);         // "51"  (slide 21: concatenation instead of math)
console.log('10' + 5);        // "105" (slide 22)
console.log("5" + true);      // "5true"
console.log("x" + null);      // "xnull"
console.log("x" + undefined); // "xundefined"
```

اگر **هیچ‌کدام** از دو طرف رشته نباشد، `+` جمع ریاضی است و boolean، `null` و `undefined` به عدد تبدیل می‌شوند (`true -> 1`، `false -> 0`، `null -> 0`، `undefined -> NaN`):

```JavaScript
console.log(true + true);   // 2   (slide 21: booleans become numbers)
console.log(true + 1);      // 2
console.log(null + 1);      // 1   (slide 21: null -> 0)
console.log(undefined + 1); // NaN (slide 21: undefined -> NaN)
```

قاعده‌ی ۲: `-` ، `*` ، `/` و `%` همیشه عددی‌اند

طبق اسلاید ۲۲، عملگرهای `-` ، `*` و `/` در **Numeric Context** (بافت عددی) کار می‌کنند. این عملگرها برای رشته‌ها معنایی ندارند، پس JavaScript **همیشه** هر دو طرف را به عدد تبدیل می‌کند (همین‌طور `%` و `**`):

```JavaScript
console.log("5" - 1);   // 4   (slide 20 question)
console.log(2 * "3");   // 6   (slide 20)
console.log('10' - 5);  // 5   (slide 22)
console.log('10' * 2);  // 20  (slide 22)
console.log("6" / "2"); // 3   (both strings become numbers)
console.log("7" % 2);   // 1
console.log("abc" - 1); // NaN ("abc" is not a valid number)
```

تبدیل به عدد با همان قاعده‌های `Number()` درس قبل انجام می‌شود: `"" -> 0`، `"12px" -> NaN`، `null -> 0`، `undefined -> NaN`، `true -> 1`.

عبارت	نتیجه	چرا؟
`"5" + 1`	`"51"`	یک طرف رشته است، پس چسباندن
`"5" - 1`	`4`	`-` همیشه عددی است
`2 * "3"`	`6`	`*` همیشه عددی است
`true + true`	`2`	رشته‌ای در کار نیست، پس `1 + 1`
`null + 1`	`1`	`null -> 0`
`undefined + 1`	`NaN`	`undefined -> NaN`
`"5" + null`	`"5null"`	یک طرف رشته است، پس چسباندن

قاعده‌ی ۳: از چپ به راست، دوبه‌دو

`+` و `-` از چپ به راست و **دوبه‌دو** حساب می‌شوند و نوعِ نتیجه‌ی هر مرحله، قاعده‌ی مرحله‌ی بعد را تعیین می‌کند. (مثل ریاضی، `*` و `/` زودتر از `+` و `-` انجام می‌شوند و پرانتز از همه زودتر.)

```JavaScript
console.log(1 + 2 + "3");   // "33"  -> 1 + 2 = 3, then 3 + "3" = "33"
console.log("1" + 2 + 3);   // "123" -> "1" + 2 = "12", then "12" + 3 = "123"
console.log("1" + (2 + 3)); // "15"  -> parentheses first: 2 + 3 = 5
console.log(1 + "2" - 1);   // 11    -> 1 + "2" = "12", then "12" - 1 = 11
console.log("3" + 4 * 5);   // "320" -> 4 * 5 = 20 first, then "3" + 20
```

قاعده‌ی ۴: آرایه و شیء اول به مقدار ساده (primitive) تبدیل می‌شوند

وقتی یک آرایه یا شیء در `+` ، `-` یا `==` شرکت می‌کند، JavaScript اول آن را به یک **primitive** (مقدار ساده مثل رشته یا عدد) تبدیل می‌کند. برای آرایه‌ها و شیءهای معمولی این یعنی **رشته‌شان** (همان `String(x)` درس قبل). بعد قاعده‌های ۱ و ۲ اعمال می‌شوند:

مقدار	تبدیل به primitive
`[]`	`""`
`[5]`	`"5"`
`[1, 2]`	`"1,2"`
`{}`	`"[object Object]"`

JavaScript

```
console.log([] + []);   // ""   ("" + "") -> an empty line is printed
console.log([] + {});   // "[object Object]" ("" + "[object Object]")
console.log(1 + [2]);   // "12" (1 + "2")
console.log([] - 1);    // -1   ("" -> 0, then 0 - 1)
console.log([2] * [3]); // 6    ("2" * "3")
```

> **بیشتر بدانیم: ماجرای `{} + []`**
>
> اگر یک **دستور** با `{` شروع شود، JavaScript آن `{}` را یک «بلوک کد» خالی می‌فهمد، نه یک شیء؛ پس دستورِ تنهای `{} + []` در واقع همان `+[]` است و `0` می‌دهد. اما وقتی همین عبارت داخل یک عبارت دیگر بیاید، مثل `console.log({} + [])` یا `const r = {} + [];` ، آن `{}` یک شیء است و نتیجه `"[object Object]"` می‌شود.
>
> در این کتاب، مگر خلافش گفته شود، هر عبارت «داخل یک عبارت» حساب می‌شود.

قاعده‌ی ۵: الگوریتم `==` (تساوی سهل‌گیر)

اسلاید ۲۲ سه مثال از coercion در مقایسه می‌آورد (خط آخر را ما اضافه کرده‌ایم):

JavaScript

```
console.log('5' == 5);          // true  (string -> number)
console.log(false == 0);        // true  (boolean -> number)
console.log(null == undefined); // true  (special rule)
console.log(null === undefined); // false (=== never converts)
```

الگوریتم ساده‌شده‌ی `a == b` (به ترتیب بررسی کن):

1. اگر نوع دو طرف **یکی** است، مثل `===` مقایسه کن (بدون تبدیل). `NaN` با هیچ چیز، حتی خودش، برابر نیست.
2. `null` **و** `undefined` فقط با هم (و با خودشان) برابرند و با هیچ چیز دیگری، حتی `0` ، `""` یا `false` ، برابر نیستند (قانون ویژه).
3. **عدد در برابر رشته:** رشته به عدد تبدیل می‌شود: `'5' == 5` یعنی `5 == 5` .
4. اگر یک طرف **boolean** است، اول آن boolean به عدد تبدیل می‌شود (`true -> 1` ، `false -> 0`) و مقایسه از نو بررسی می‌شود.
5. **آرایه/شیء در برابر primitive:** آرایه/شیء به primitive تبدیل می‌شود (قاعده‌ی ۴) و مقایسه از نو بررسی می‌شود.

6. **دو آرایه/شیء:** هیچ تبدیلی رخ نمی‌دهد؛ فقط وقتی برابرند که هر دو دقیقاً **همان یک** شیء باشند (`[] == []` برابر `false` است؛ در درس «عملگرهای مقایسه و منطقی» مفصل می‌بینیم).

حالا دام‌های اسلاید ۲۱ و چند دام مشابه را قدم‌به‌قدم حل کنیم:

عبارت	مراحل	نتیجه
`[] == false`	`false -> 0` ؛ سپس `[] -> "" -> 0` ؛ پس `0 == 0`	`true`
`[] == 0`	`[] -> "" -> 0` ؛ پس `0 == 0`	`true`
`"0" == false`	`false -> 0` ؛ سپس `"0" -> 0` ؛ پس `0 == 0`	`true`
`true == "1"`	`true -> 1` ؛ سپس `"1" -> 1` ؛ پس `1 == 1`	`true`
`true == "true"`	`true -> 1` ؛ سپس `"true" -> NaN` ؛ پس `1 == NaN`	`false`
`"2" == true`	`true -> 1` ؛ سپس `"2" -> 2` ؛ پس `2 == 1`	`false`
`null == 0`	قانون ویژه: `null` فقط با `undefined` برابر است	`false`
`NaN == NaN`	`NaN` با هیچ چیز برابر نیست	`false`

عملگر `===` (تساوی سخت‌گیر) هیچ‌کدام از این مراحل را ندارد: اگر نوع دو طرف فرق کند، **بلافاصله** `false` می‌دهد. برای همین `[] === false`، `'5' === 5` و `null === undefined` همگی `false` هستند. توصیه‌ی اسلاید ۲۱: **`==` می‌تواند تبدیل‌های غیرمنتظره ایجاد کند، پس `===` را ترجیح بده.**

قاعده‌ی ۶: `if` فقط truthy/falsy را می‌پرسد؛ `[] == false` ولی `if ([])` اجرا می‌شود!

این تناقض ظاهری محبوب طراحان تست است. `if (x)` فقط می‌پرسد «`x` truthy است یا falsy؟» (همان `Boolean(x)` درس قبل) و آرایه‌ی خالی truthy است. اما `==` اصلاً سراغ truthy/falsy نمی‌رود؛ الگوریتم بالا را اجرا می‌کند که مقدارها را به سمت **عدد** می‌برد. دو سؤال متفاوت، دو جواب متفاوت:

```javascript
const list = [];
if (list) {
  console.log("list is truthy"); // this runs!
}
console.log(list == false);  // true
console.log(list === false); // false
```

مقدار `x`	`Boolean(x)` (همان که `if` می‌پرسد)	`x == false`
`[]`	`true`	`true`
`"0"`	`true`	`true`
`" "`	`true`	`true`
`""`	`false`	`true`
`0`	`false`	`true`
`null`	`false`	`false`
`undefined`	`false`	`false`
`NaN`	`false`	`false`

نتیجه‌ی عملی: هیچ‌وقت ننویس `if (x == false)`؛ اگر منظورت falsy بودن است بنویس `if (!x)`.

میم اسلاید ۲۱: `sort()` روی عددها

در میم اسلاید ۲۱، کسی که تازه JavaScript یاد می‌گیرد می‌گوید: «از `array.sort()` استفاده می‌کنم تا این لیست عددها را مرتب کنم!» و JavaScript (با تصویر خنده‌ی لئوناردو دی‌کاپریو) جواب می‌دهد: `[1, 100000, 21, 30, 4]`. سؤال اسلاید این است: «می‌توانی بگویی وقتی این اتفاق می‌افتد چه خبر است؟»

جواب: `sort()` اگر **تابع مقایسه** نگیرد، عناصر را به **رشته** تبدیل می‌کند و مثل کلمه‌های یک لغت‌نامه، **حرف‌به‌حرف** مقایسه می‌کند (lexicographic order؛ ترتیب لغت‌نامه‌ای). `"100000"` قبل از `"21"` می‌آید، چون حرف اولش (`"1"`) از `"2"` کوچک‌تر است؛ و `"4"` آخر از همه است، چون از `"1"`، `"2"` و `"3"` بزرگ‌تر است. طول عدد اصلاً مهم نیست؛ درست مثل لغت‌نامه که «آبشار» را قبل از «ب» می‌آورد.

```JavaScript
const nums = [4, 30, 1, 100000, 21];
console.log(nums.sort());                // [ 1, 100000, 21, 30, 4 ]  (compared as strings)
console.log(nums.sort((a, b) => a - b)); // [ 1, 4, 21, 30, 100000 ]  (numeric, ascending)
console.log(nums.sort((a, b) => b - a)); // [ 100000, 30, 21, 4, 1 ]  (numeric, descending)
```

راه درست، دادن یک **تابع مقایسه** (compare function) به `sort` است. `(a, b) => a - b` یک تابع کوتاه است (arrow function؛ در درس توابع می‌خوانیم) که دو عنصر را می‌گیرد: اگر حاصلش **منفی** باشد یعنی `a` باید جلوتر بیاید، اگر **مثبت** باشد `b` جلوتر می‌آید و اگر **صفر** باشد ترتیبشان مهم نیست. چون `-` همیشه عددی است (قاعده‌ی ۲)، این بار مقایسه‌ی عددی واقعی انجام می‌شود. برای ترتیب نزولی از `(a, b) => b - a` استفاده می‌کنیم. بقیه‌ی جزئیات `sort` را در درس «متدهای آرایه» می‌خوانیم.

unary plus و معمای «baNaNa»

`+` اگر فقط **یک** عملوند داشته باشد (unary plus)، کارش تبدیل به عدد است: `+"3"` می‌شود `3` و `+"a"` می‌شود `NaN`. اولویت این عملگر از `+` دوعملوندی بالاتر است. حالا معمای معروف:

```JavaScript
console.log("b" + "a" + +"a" + "a"); // "baNaNa"
// +"a"          -> NaN      (unary plus: "a" is not a number)
// "b" + "a"     -> "ba"
// "ba" + NaN    -> "baNaN"
// "baNaN" + "a" -> "baNaNa"
```

بیشتر بدانیم: BigInt استثناست

`BigInt` (عددهای صحیح خیلی بزرگ مثل `10n`) با عددهای معمولی coercion نمی‌شود: `10n + 1` خطای `TypeError: Cannot mix BigInt and other types` می‌دهد. باید صریحاً تبدیل کنی: `10n + 1n` (حاصل `11n`) یا `Number(10n) + 1` (حاصل `11`).

قانون طلایی

- `+` با حداقل یک رشته: چسباندن (نتیجه رشته است)؛ وگرنه جمع عددی.
- `-`، `*`، `/` و `%` : همیشه عددی.
- آرایه/شیء: اول به رشته تبدیل می‌شود (`[] -> ""` و `{} -> "[object Object]"`).
- `==` : معمولاً همه‌چیز به سمت عدد می‌رود، به جز `null` / `undefined` که فقط با هم برابرند.
- `if` : فقط truthy یا falsy بودن را می‌پرسد.
- در کد واقعی همیشه `===` را ترجیح بده و قبل از محاسبه صریحاً تبدیل کن (`Number(input)`).

دام تستی

- `null + 1` برابر `1` است، ولی `undefined + 1` برابر `NaN` .
- `null` در جمع مثل `0` رفتار می‌کند، ولی `null == 0` برابر `false` است، چون `==` برای `null` قانون ویژه دارد.
- `true == "true"` برابر `false` است (`1 == NaN` و `"2" == true` هم `false` است (`2 == 1`).
- `"" == 0` و `"0" == 0` هر دو `true` هستند، اما `"0" == ""` برابر `false` است؛ دو رشته بدون هیچ تبدیلی با هم مقایسه می‌شوند.
- `[] == false` برابر `true` است، ولی آرایه‌ی خالی truthy است و `if ([])` اجرا می‌شود.
- `1 + 2 + "3"` برابر `"33"` است، نه `"123"` .
- `[1, 100000, 21, 30, 4]` حاصل `sort()` بدون تابع مقایسه است؛ برای مرتب‌سازی عددی `(a, b) => a - b` لازم است.

تست‌های تمرینی

۳۳ تست · شماره‌ی ۱۹۲ تا ۲۲۴

۱۹۲ آسان

طبق اسلاید ۲۲، **type coercion** یعنی:

۱) تبدیل خودکار نوع داده توسط خود JavaScript، وقتی در یک عملیات مقدارهایی از نوع‌های مختلف شرکت دارند

۲) تبدیل نوعی که برنامه‌نویس خودش با `Number()` یا `String()` انجام می‌دهد

۳) خطایی که JavaScript هنگام جمع کردن یک رشته با یک عدد می‌دهد

۴) اینکه نوع هر متغیر بعد از تعریف ثابت می‌ماند و دیگر عوض نمی‌شود

۱۹۳ آسان

طبق اسلاید ۲۲، در مقایسه‌ی `'5' == 5` چه تبدیلی رخ می‌دهد؟

۱) عدد `5` به رشته‌ی `"5"` تبدیل می‌شود

۲) هر دو طرف به boolean تبدیل می‌شوند

۳) رشته‌ی `'5'` به عدد `5` تبدیل می‌شود

۴) هیچ تبدیلی رخ نمی‌دهد؛ چون نوع‌ها فرق دارند، نتیجه `false` است

۱۹۴ آسان

حاصل `'10' + 5` کدام است؟

۱) `15`　۲) `"105"`　۳) `"15"`　۴) `105`

۱۹۵ آسان

حاصل `'10' - 5` کدام است؟

۱) `"105"`　۲) `"5"`　۳) `5`　۴) `NaN`

۱۹۶ آسان

حاصل `true + true` کدام است؟

۱) `"truetrue"`　۲) `true`　۳) `NaN`　۴) `2`

۱۹۷ آسان

در میم اسلاید ۱۸، کارمندِ JavaScript به سؤال «`1 + 1` چند می‌شود؟» جواب `11` می‌دهد. کدام دستور **واقعاً** `11` چاپ می‌کند؟

۱) `console.log(1 + 1);`

۲) `console.log("1" + 1);`

۳) `console.log("1" - 1);`

۴) `console.log(1 + +"1");`

۱۹۸ آسان

در جای خالی عبارت `"8" __ 2` کدام عملگر قرار گیرد تا حاصل از نوع **string** باشد؟

۱) `-`
۲) `*`
۳) `/`
۴) `+`

۱۹۹ آسان

حاصل `undefined + 1` کدام است؟

۱) `1`
۲) `"undefined1"`
۳) `NaN`
۴) `undefined`

۲۰۰ آسان

خروجی کد زیر کدام است؟

```JavaScript
console.log(null == undefined, null === undefined);
```

۱) `true true`
۲) `true false`
۳) `false false`
۴) `false true`

۲۰۱ آسان

اسلاید ۲۱ برای جلوگیری از تبدیل‌های غیرمنتظره هنگام مقایسه، استفاده از کدام عملگر را توصیه می‌کند؟

۱) `===`
۲) `==`
۳) `=`
۴) `!=`

۲۰۲ متوسط

خروجی کد زیر کدام است؟

```JavaScript
console.log("5" + 1);
console.log(null + 1);
console.log(null + "1");
console.log(undefined + "1");
```

۱) `51` `1` `null1` `undefined1`
۲) `6` `1` `1` `NaN`
۳) `51` `1` `1` `NaN`
۴) `51` `NaN` `null1` `undefined1`

۲۰۳ متوسط

خروجی کد زیر کدام است؟

```javascript
console.log(1 + 2 + "3" + 4 + 5);
console.log("1" + (2 + 3));
console.log(1 + "2" - 1);
```

۱) `12345` `15` `11`
۲) `3345` `123` `11`
۳) `3345` `15` `NaN`
۴) `3345` `15` `11`

۲۰۴ متوسط

کدام گزینه **نادرست** است؟

۱) `'5' == 5` برابر `true` است.
۲) `false == 0` برابر `true` است.
۳) `'5' === 5` برابر `true` است.
۴) `null == undefined` برابر `true` است.

۲۰۵ متوسط

خروجی کد زیر کدام است؟

```javascript
console.log([] == false);
console.log([] === false);
console.log([] == 0);
```

۱) `true` `false` `true`
۲) `false` `false` `false`
۳) `true` `true` `true`
۴) `false` `false` `true`

۲۰۶ متوسط

خروجی کد زیر کدام است؟

```javascript
console.log(2 * "3", '10' * 2, "5" + "2", "6" / "2", "7" % 2);
```

۱) `NaN NaN 52 NaN NaN`
۲) `6 20 52 3 1`
۳) `6 20 7 3 1`
۴) `33 1010 52 3 1`

۲۰۷ متوسط

کدام گزینه **درست** است؟

۱) `false + 1` برابر `"false1"` است.
۲) `null + null` برابر `"nullnull"` است.
۳) `"3" * true` برابر `"3true"` است.
۴) `true + "1"` برابر `"true1"` است.

۲۰۸ متوسط

حاصل `"b" + "a" + +"a" + "a"` کدام است؟

۱) `"baNaNa"`
۲) `"baaa"`
۳) `"ba+aa"`
۴) `NaN`

۲۰۹ متوسط

خروجی کد زیر کدام است؟

```JavaScript
const list = [];
if (list) {
  console.log("A");
}
if (list == false) {
  console.log("B");
}
if (list === false) {
  console.log("C");
}
```

۱) `B`
۲) `A`
۳) `A` `B`
۴) `A` `B` `C`

۲۱۰ متوسط

خروجی کد زیر کدام است؟ (میم اسلاید ۲۱)

```JavaScript
const nums = [4, 30, 1, 100000, 21];
console.log(nums.sort());
```

۱) `[1, 4, 21, 30, 100000]`
۲) `[1, 100000, 21, 30, 4]`
۳) `[100000, 30, 21, 4, 1]`
۴) `[4, 30, 1, 100000, 21]`

۲۱۱ متوسط

چرا حاصل `[10, 9, 1].sort()` برابر `[1, 10, 9]` است؟

۱) چون `sort()` به‌طور پیش‌فرض ترتیب نزولی می‌سازد
۲) چون `sort()` عناصر را بر اساس تعداد رقم‌هایشان مرتب می‌کند
۳) چون آرایه‌هایی که عدد دورقمی دارند در JavaScript مرتب نمی‌شوند
۴) چون `sort()` بدون تابع مقایسه، عناصر را به رشته تبدیل و مثل کلمه‌های لغت‌نامه حرف‌به‌حرف مقایسه می‌کند

۲۱۲ متوسط

حاصل عبارت زیر کدام است؟

```
[4, 30, 1, 100000, 21].sort((a, b) => b - a)
```

۱) `[100000, 30, 21, 4, 1]`
۲) `[1, 4, 21, 30, 100000]`
۳) `[4, 30, 21, 100000, 1]`
۴) `[1, 100000, 21, 30, 4]`

۲۱۳ متوسط

در کدام عبارت **هیچ** type coercion (تبدیل ضمنی) رخ نمی‌دهد؟

۱) `"5" * 1`
۲) `5 == "5"`
۳) `Number("5") + 1`
۴) `"5" + 1`

۲۱۴ متوسط

خروجی کد زیر کدام است؟

```JavaScript
const r = {} + [];
console.log(r);
```

۱) `0`
۲) `[object Object]`
۳) `NaN`
۴) `[object Object][]`

۲۱۵ سخت

خروجی کد زیر کدام است؟

```JavaScript
console.log(1 + [2]);
console.log([2] * [3]);
console.log([] - 1);
console.log([1, 2] + [3]);
```

۱) `3` `6` `-1` `1,2,3`
۲) `12` `6` `-1` `1,23`
۳) `12` `23` `NaN` `1,23`
۴) `3` `6` `NaN` `1,2,3`

۲۱۶ سخت

کدام گزینه **درست** است؟

۱) `[] + []` برابر `"[]"` است.
۲) `[] + 1` برابر عدد `1` است.
۳) `[] + {}` برابر `"[object Object]"` است.
۴) `[] - []` برابر رشته‌ی `""` است.

۲۱۷ سخت

خروجی کد زیر کدام است؟

```javascript
console.log(Boolean("0"), "0" == false);
console.log(Boolean(null), null == false);
```

۱) `true true` `false false`
۲) `false true` `false true`
۳) `true false` `false true`
۴) `false false` `false false`

۲۱۸ سخت

خروجی کد زیر کدام است؟

```javascript
console.log(true == "1");
console.log(true == "true");
console.log("2" == true);
```

۱) `true` `true` `true`
۲) `true` `false` `false`
۳) `false` `false` `false`
۴) `true` `true` `false`

۲۱۹ سخت

کدام گزینه **نادرست** است؟

۱) `null == 0` برابر `false` است.
۲) `undefined == 0` برابر `false` است.
۳) `NaN == NaN` برابر `false` است.
۴) `null == false` برابر `true` است.

۲۲۰ سخت

اجرای کد زیر در کدام خط با خطا متوقف می‌شود؟

```javascript
1  const a = "5" * 2;
2  const b = null + true;
3  const c = undefined + 1;
4  const d = 10n + 1;
5  console.log(a, b, c, d);
```

۱) خط ۲
۲) خط ۳
۳) خط ۴
۴) هیچ خطایی رخ نمی‌دهد

۲۲۱ سخت

حاصل `true + null + "1" + undefined` کدام است؟

۱) `"truenull1undefined"`
۲) `NaN`
۳) `"2undefined"`
۴) `"11undefined"`

۲۲۲ سخت

خروجی کد زیر کدام است؟

```JavaScript
console.log("3" - 1 + "1");
console.log("3" + 4 * 5);
console.log(+"4" + +"4");
```

۱) `21` `320` `8`
۲) `3` `23` `8`
۳) `21` `170` `44`
۴) `21` `320` `44`

۲۲۳ سخت

حاصل کدام مقایسه `true` است؟

۱) `"abc" == NaN`
۲) `[] == []`
۳) `undefined == ""`
۴) `[0] == false`

۲۲۴ سخت

خروجی کد زیر کدام است؟

```JavaScript
console.log("" == 0, " " == 0, "0" == "");
```

۱) `true false false`
۲) `true true false`
۳) `true true true`
۴) `false false false`

۱-۱۱ عملگرهای مقایسه و منطقی

منبع: Introduction to JS، اسلاید ۳۱

درسنامه

برنامه‌ها مدام سؤال‌های «بله یا نه» می‌پرسند: «کاربر ۱۸ سالش شده؟»، «رمز درست است؟»، «سبد خرید خالی است؟». جواب هر کدام یک مقدار **boolean** است: `true` یا `false`. **عملگرهای مقایسه** (comparison operators؛ عملگرهایی که دو مقدار را با هم می‌سنجند) این سؤال‌ها را می‌سازند و **عملگرهای منطقی** (logical operators؛ عملگرهایی که چند شرط را با هم ترکیب یا برعکس می‌کنند) چند سؤال را به هم وصل می‌کنند. همه‌ی شرط‌های `if` و حلقه‌ها روی همین‌ها سوارند.

عملگرهای تساوی: `==` ، `===` ، `!=` و `!==`

طبق اسلاید ۳۱:

عملگر	نام	کارش
`==`	Equal (تساوی)	**مقدار**ها را مقایسه می‌کند و اجازه‌ی type coercion (تبدیل نوع خودکار) می‌دهد
`===`	Strict Equal (تساوی سخت‌گیر)	**مقدار و نوع** را مقایسه می‌کند؛ هیچ تبدیلی انجام نمی‌دهد
`!=`	Not Equal (نابرابری)	نقیضِ `==` (با تبدیل نوع)
`!==`	Strict Not Equal (نابرابری سخت‌گیر)	نقیضِ `===` (بدون تبدیل نوع)

```JavaScript
console.log(5 == "5");  // true  ("5" is converted to 5)
console.log(5 === "5"); // false (number vs string)
console.log(5 != "5");  // false (they ARE loosely equal)
console.log(5 !== "5"); // true  (the types differ)
```

قاعده‌های کامل تبدیل در `==` را در درس «تبدیل نوع ضمنی» دیدیم (مثلاً `null == undefined` برابر `true` و `null == 0` برابر `false` است). قاعده‌ی عملی: همیشه `===` و `!==` را ترجیح بده، مگر اینکه دلیل خاصی داشته باشی.

مقایسه با مقدار (by value) یا با مرجع (by reference)

یادداشت اسلاید ۳۱: **primitiveها با مقدارشان مقایسه می‌شوند، نه با مرجعشان.**

یک تشبیه ساده

primitiveها (عدد، رشته، boolean و ...) مثل عددی هستند که روی کاغذ نوشته شده: دو کاغذ که رویشان `5` نوشته شده با هم برابرند. اما آرایه‌ها و شیءها مثل **خانه** هستند و متغیر فقط **آدرس** خانه را نگه می‌دارد؛ به این آدرس **reference** (مرجع) می‌گوییم. دو خانه با دکوراسیون کاملاً یکسان هنوز دو خانه‌ی جدا هستند. `==` و `===` برای آرایه‌ها و شیءها فقط می‌پرسند: «آیا هر دو آدرس به **همان یک** خانه اشاره می‌کنند؟»

```JavaScript
console.log("hi" === "hi"); // true  (primitives: compared by value)

const a = [1, 2];
const b = [1, 2];
const c = a;            // c points to the SAME array as a
console.log(a === b);   // false (two different arrays with equal content)
console.log(a == b);    // false (== does not help: both sides are objects)
console.log(a === c);   // true  (same reference)
console.log({} === {}); // false
console.log([] == []);  // false
```

> **نکته**
>
> `[1] == 1` برابر `true` است (یک طرف primitive است، پس آرایه اول رشته‌ی `"1"` و بعد عدد `1` می‌شود)، اما `[1] == [1]` برابر `false` است: وقتی **هر دو** طرف آرایه یا شیء باشند هیچ تبدیلی رخ نمی‌دهد و فقط مرجع‌ها مقایسه می‌شوند.

عملگرهای بزرگ‌تر و کوچک‌تر: `>` ، `<` ، `>=` و `<=`

برای عددها دقیقاً مثل ریاضی کار می‌کنند: `5 > 3` برابر `true` ، `5 >= 5` برابر `true` و `5 > 5` برابر `false` است. اما دو حالت خاص داریم.

حالت ۱: هر دو طرف رشته باشند؛ مقایسه‌ی لغت‌نامه‌ای (lexicographic)

رشته‌ها حرف‌به‌حرف از چپ مقایسه می‌شوند و **اولین حرفِ متفاوت** تکلیف را روشن می‌کند. هر کاراکتر یک کد عددی (Unicode) دارد و مقایسه بر اساس همین کد است. اگر یک رشته ابتدای رشته‌ی دیگر باشد (مثل `"app"` و `"apple"`)، رشته‌ی کوتاه‌تر کوچک‌تر است.

کاراکترها	کدها	ترتیب
رقم‌ها `"0"` تا `"9"`	۴۸ تا ۵۷	کوچک‌ترین
حروف بزرگ `"A"` تا `"Z"`	۶۵ تا ۹۰	وسط
حروف کوچک `"a"` تا `"z"`	۹۷ تا ۱۲۲	بزرگ‌ترین

```JavaScript
console.log("apple" < "banana"); // true  ("a" < "b")
console.log("Z" < "a");          // true  (uppercase codes are smaller: 90 < 97)
console.log("10" < "9");         // true  (compares "1" with "9" first!)
console.log("app" < "apple");    // true  (a prefix is smaller)
```

حالت ۲: نوع دو طرف فرق کند؛ هر دو به عدد تبدیل می‌شوند

اگر حداقل یکی از دو طرف رشته نباشد، هر دو طرف با قاعده‌های `Number` به **عدد** تبدیل می‌شوند:

```JavaScript
console.log("10" < 9); // false (numeric: 10 < 9)
console.log("5" > 3);  // true  (5 > 3)
console.log(true > 0); // true  (1 > 0)
console.log("a" > 1);  // false ("a" becomes NaN)
```

`null`، `undefined` و `NaN` در مقایسه

- عملگرهای `>`، `<`، `>=` و `<=` مقدار `null` را به `0` تبدیل می‌کنند، ولی `==` برای `null` قانون ویژه دارد (فقط با `undefined` برابر است). نتیجه‌ی عجیب: `null >= 0` برابر `true` است، ولی `null == 0` و `null > 0` هر دو `false` هستند!
- `undefined` به `NaN` تبدیل می‌شود، پس `undefined >= 0`، `undefined <= 0` و `undefined == 0` همگی `false` هستند.
- **هر** مقایسه‌ای با `NaN` (`==`، `===`، `<`، `>`، `<=`، `>=`) نتیجه‌اش `false` است؛ حتی `NaN === NaN`. فقط `!=` و `!==` با `NaN` جواب `true` می‌دهند. برای تشخیص `NaN` از `Number.isNaN(x)` استفاده کن.

```
JavaScript
console.log(null >= 0, null == 0, null > 0); // true false false
console.log(undefined == 0, undefined >= 0); // false false
console.log(NaN === NaN, NaN !== NaN);       // false true
console.log(Number.isNaN(NaN));              // true
```

دام مقایسه‌ی زنجیره‌ای

در ریاضی `1 < 2 < 3` درست است، اما JavaScript آن را **دوبه‌دو از چپ** حساب می‌کند و نتیجه‌ی مرحله‌ی اول (یک boolean) در مرحله‌ی دوم به عدد تبدیل می‌شود:

```
JavaScript
console.log(1 < 2 < 3); // true  -> (1 < 2) is true, then true < 3 -> 1 < 3 -> true
console.log(3 > 2 > 1); // false -> (3 > 2) is true, then true > 1 -> 1 > 1 -> false
```

اولی فقط «از شانس» درست درآمده. برای بررسی اینکه `b` بین `a` و `c` است بنویس: `a < b && b < c`.

عملگرهای منطقی: `&&`، `||` و `!`

اسلاید ۳۱: عملگرهای منطقی **درست مثل زبان C** هستند: `&&` (AND؛ «و»)، `||` (OR؛ «یا») و `!` (NOT؛ «نقیض»). با آن‌ها می‌توانی مقدارهای boolean را **ترکیب** یا **برعکس** کنی.

a	b	a && b	a || b
true	true	true	true
true	false	false	true
false	true	false	true
false	false	false	false

- `a && b` فقط وقتی `true` است که **هر دو** `true` باشند.
- `a || b` وقتی `true` است که **حداقل یکی** `true` باشد.
- `!a` مقدار را برعکس می‌کند: `!true` برابر `false` است.

```JavaScript
const age = 20;
const hasTicket = true;
console.log(age >= 18 && hasTicket); // true  (both conditions hold)
console.log(age < 12 || age > 65);   // false (neither condition holds)
console.log(!(age >= 18));           // false (inverts the result)
```

اولویت (precedence): اول `!` ، بعد `&&` و در آخر `||` ؛ درست مثل ریاضی که ضرب قبل از جمع انجام می‌شود. پس `true || false && false` یعنی `true || (false && false)` که `true` است. هر جا شک داری، پرانتز بگذار.

برعکس کردن یک ترکیب (قوانین دمورگان): `!(a && b)` همان `!a || !b` است و `!(a || b)` همان `!a && !b` . یعنی وقتی `!` را به داخل پرانتز می‌بری، `&&` و `||` جایشان را با هم عوض می‌کنند.

اتصال کوتاه (short-circuit evaluation)

JavaScript (مثل C) طرف راست را فقط وقتی حساب می‌کند که لازم باشد:

- در `a && b` اگر `a` falsy باشد، جواب از قبل معلوم است، پس `b` **اصلاً اجرا نمی‌شود**.
- در `a || b` اگر `a` truthy باشد، `b` **اصلاً اجرا نمی‌شود**.

```JavaScript
false && console.log("A"); // nothing is printed: the right side is skipped
true && console.log("B");  // prints B
true || console.log("C");  // nothing is printed
false || console.log("D"); // prints D
```

کاربرد مهمش «نگهبان» (guard) است: در `user && user.name` اگر `user` برابر `null` باشد، JavaScript اصلاً به `user.name` نمی‌رسد و خطا نمی‌دهد؛ در حالی که `null.name` خطای `TypeError` می‌دهد.

`&&` و `||` یکی از عملوندها را برمی‌گردانند، نه لزوماً boolean

اینجا JavaScript با C فرق دارد: حاصل `&&` و `||` لزوماً `true` یا `false` نیست، بلکه **یکی از خود عملوندها**ست:

- `a || b` : اگر `a` truthy باشد **خودِ** `a` را برمی‌گرداند، وگرنه **خودِ** `b` را. (اولین مقدار truthy، یا اگر نبود آخرین عملوند)
- `a && b` : اگر `a` falsy باشد **خودِ** `a` را برمی‌گرداند، وگرنه **خودِ** `b` را. (اولین مقدار falsy، یا اگر نبود آخرین عملوند)

```JavaScript
console.log("a" && "b");          // "b"  ("a" is truthy, so the result is the second operand)
console.log(0 || "x");            // "x"
console.log(null && "yes");       // null (the first falsy operand)
console.log(1 && 2 && 3);         // 3    (no falsy operand: the last one)
console.log(null || 0 || "last"); // "last"
```

اما `!` **همیشه** یک boolean واقعی برمی‌گرداند: `!"hello"` برابر `false` و `!0` برابر `true` است. برای همین `!!x` (دو بار نقیض) همان `Boolean(x)` است.

مقدار پیش‌فرض با `||` و جایگزین بهترش `??`

چون `||` اولین مقدار truthy را برمی‌گرداند، برای دادن مقدار پیش‌فرض (default value) زیاد استفاده می‌شود:

```JavaScript
const input = "";
const name = input || "Guest";
console.log(name); // "Guest" (empty input, so the default is used)
```

اما دام بزرگی دارد: `0` و `""` هم falsy هستند؛ پس اگر `0` یک مقدار **معتبر** باشد، به اشتباه جایگزین می‌شود. عملگر `??` (nullish coalescing؛ از ES2020) این مشکل را ندارد: طرف راست را **فقط** وقتی برمی‌گرداند که طرف چپ `null` یا `undefined` باشد.

```JavaScript
const volume = 0;              // 0 is a valid volume!
console.log(volume || 50);     // 50 (bug: 0 is falsy, so it was replaced)
console.log(volume ?? 50);     // 0  (?? only replaces null/undefined)
console.log(null ?? "default"); // "default"
```

یادداشت

`??` را نمی‌توان بدون پرانتز کنار `||` یا `&&` نوشت: `a || b ?? c` خطای نحوی (SyntaxError) می‌دهد و باید نوشت `(a || b) ?? c`.

نکته

- همیشه `===` و `!==` را ترجیح بده.
- برای `NaN` از `Number.isNaN` استفاده کن، نه از `x === NaN`.
- برای مقدار پیش‌فرضی که `0` یا `""` هم می‌تواند معتبر باشد، `??` را به جای `||` به کار ببر.
- در ترکیب‌های طولانی پرانتز بگذار تا مجبور نشوی اولویت‌ها را حدس بزنی.

دام تستی

- `"10" < "9"` برابر `true` است (دو رشته، حرف‌به‌حرف)، ولی `"10" < 9` برابر `false` (یک طرف عدد است، پس مقایسه‌ی عددی).
- `"Z" < "a"` برابر `true` است: حروف بزرگ کد کوچک‌تری دارند.
- `null >= 0` برابر `true` است، ولی `null == 0` برابر `false`.
- `NaN === NaN` برابر `false` است.
- `3 > 2 > 1` برابر `false` است.
- `[1, 2] === [1, 2]` برابر `false` است (دو آرایه‌ی جدا).
- `"a" && "b"` برابر `"b"` است، نه `true`؛ و `0 || 10` برابر `10` است، حتی اگر `0` را عمداً گذاشته باشی.

تست‌های تمرینی

۲۹ تست · شماره‌ی ۲۲۵ تا ۲۵۳

۲۲۵ آسان

طبق اسلاید ۳۱، عملگر `===` (Strict Equal) چه چیزی را مقایسه می‌کند؟

۱) فقط مقدار را، و در صورت نیاز نوع‌ها را به هم تبدیل می‌کند

۲) مقدار و نوع را، بدون هیچ تبدیل نوعی

۳) فقط نوع دو طرف را، بدون توجه به مقدار

۴) آدرس حافظه‌ی دو طرف را، حتی برای عددها و رشته‌ها

۲۲۶ آسان

خروجی کد زیر کدام است؟

```JavaScript
console.log(5 == "5", 5 === "5", 5 != "5", 5 !== "5");
```

۱) `true true false false`

۲) `true false true false`

۳) `true false false true`

۴) `false false true true`

۲۲۷ آسان

کدام گزینه **نادرست** است؟

۱) `"hi" === "hi"` برابر `true` است.

۲) `10 >= 10` برابر `true` است.

۳) `3 < 5` برابر `true` است.

۴) `7 !== 7` برابر `true` است.

۲۲۸ متوسط

کدام گزینه **درست** است؟

۱) `null != undefined` برابر `true` است.

۲) `"1" !== 1` برابر `true` است.

۳) `NaN != NaN` برابر `false` است.

۴) `"1" != 1` برابر `true` است.

۲۲۹ سخت

خروجی کد زیر کدام است؟

```JavaScript
const a = [1, 2];
const b = [1, 2];
const c = a;
console.log(a == b, a === c);
console.log([1] == 1, [1] == [1]);
```

۱) `false true` `true false`

۲) `true true` `true true`

۳) `false true` `false false`

۴) `true false` `true true`

۲۳۰ سخت

خروجی کد زیر کدام است؟

```javascript
const p1 = { name: "Ali" };
const p2 = { name: "Ali" };
const p3 = p1;
console.log(p1 == p2, p1 === p3, p1.name === p2.name);
```

۱) `true true true`
۲) `false false true`
۳) `false true false`
۴) `false true true`

۲۳۱ سخت

خروجی کد زیر کدام است؟

```javascript
console.log("apple" < "banana");
console.log("Z" < "a");
console.log("10" < "9");
console.log("10" < 9);
```

۱) `true` `false` `false` `false`
۲) `true` `true` `true` `false`
۳) `true` `false` `true` `false`
۴) `true` `true` `false` `false`

۲۳۲ سخت

کدام گزینه **نادرست** است؟

۱) `"apple" > "app"` برابر `true` است.
۲) `"Apple" < "apple"` برابر `true` است.
۳) `2 > "12"` برابر `true` است.
۴) `"2" > "12"` برابر `true` است.

۲۳۳ متوسط

خروجی کد زیر کدام است؟

```javascript
console.log(true > false, "5" > 3, "a" > 1);
```

۱) `true true false`
۲) `false true false`
۳) `true false false`
۴) `true true true`

۲۳۴ سخت

خروجی کد زیر کدام است؟

```javascript
console.log(null == 0, null > 0, null >= 0);
console.log(undefined == 0, undefined >= 0);
```

۱) `false false true` `false false`

۲) `true false true` `true true`

۳) `false false false` `false false`

۴) `true true true` `false false`

۲۳۵ متوسط

خروجی کد زیر کدام است؟

```javascript
const x = NaN;
console.log(x == NaN, x === x, x !== x, Number.isNaN(x));
```

۱) `true true false true`

۲) `false true false true`

۳) `false false true true`

۴) `false false false false`

۲۳۶ سخت

خروجی کد زیر کدام است؟

```javascript
console.log(1 < 2 < 3);
console.log(3 > 2 > 1);
```

۱) `true` `true`

۲) `true` `false`

۳) `false` `false`

۴) `false` `true`

۲۳۷ آسان

عملگر منطقی «یا» (OR) در JavaScript کدام است؟

۱) `||`

۲) `or`

۳) `OR`

۴) `|`

۲۳۸ آسان

خروجی کد زیر کدام است؟

```JavaScript
console.log(true && false, true || false, !true);
```

۱) `true true false`
۲) `false false true`
۳) `true false false`
۴) `false true false`

۲۳۹ آسان

می‌خواهیم شرط زیر فقط وقتی `true` باشد که کاربر **هم** حداقل ۱۸ سال داشته باشد **و هم** بلیت داشته باشد. در جای خالی کدام عملگر قرار می‌گیرد؟

```
age >= 18 ____ hasTicket
```

۱) `||`
۲) `!`
۳) `&&`
۴) `==`

۲۴۰ متوسط

خروجی کد زیر کدام است؟

```JavaScript
console.log(true || false && false);
console.log((true || false) && false);
console.log(!false && false);
```

۱) `false` `false` `false`
۲) `true` `false` `false`
۳) `true` `false` `true`
۴) `false` `false` `true`

۲۴۱ متوسط

اگر `a` و `b` دو مقدار boolean باشند، کدام عبارت **همیشه** با `!(a && b)` هم‌ارز است؟

۱) `!a && !b`
۲) `a || b`
۳) `!a && b`
۴) `!a || !b`

۲۴۲ آسان

در عبارت `a && b`، اگر `a` مقداری **falsy** باشد چه اتفاقی می‌افتد؟

۱) `b` هم ارزیابی می‌شود و حاصل همیشه `false` است.
۲) `b` اصلاً ارزیابی نمی‌شود و حاصل، خودِ `a` است.
۳) خطای `TypeError` رخ می‌دهد، چون `a` boolean نیست.
۴) `b` ارزیابی می‌شود و حاصل، خودِ `b` است.

۲۴۳ متوسط

خروجی کد زیر کدام است؟

```javascript
false && console.log("A");
true && console.log("B");
true || console.log("C");
false || console.log("D");
```

۱) A B C D
۲) B C
۳) A C
۴) B D

۲۴۴ متوسط

اجرای کد زیر در کدام خط با خطا متوقف می‌شود؟

```javascript
1 const user = null;
2 const a = user && user.name;
3 const b = user || "Guest";
4 const c = user.name || "Guest";
5 console.log(a, b, c);
```

۱) خط ۲
۲) خط ۳
۳) خط ۴
۴) هیچ خطایی رخ نمی‌دهد

۲۴۵ سخت

اجرای کد زیر در کدام خط با خطا متوقف می‌شود؟ (متغیر `y` هیچ‌جا تعریف نشده است.)

```javascript
1 const a = "x" || y;
2 const b = false && y;
3 const c = y || "x";
4 console.log(a, b, c);
```

۱) خط ۱
۲) خط ۲
۳) خط ۳
۴) هیچ خطایی رخ نمی‌دهد

۲۴۶ متوسط

حاصل `"a" && "b"` کدام است؟

۱) `true`
۲) `"a"`
۳) `"ab"`
۴) `"b"`

۲۴۷ متوسط

حاصل کدام عبارت **درست** بیان شده است؟

۱) `"" && "x"` برابر `"x"` است.
۲) `1 && 0` برابر `1` است.
۳) `null || "x"` برابر `"x"` است.
۴) `0 || null` برابر `0` است.

۲۴۸ سخت

خروجی کد زیر کدام است؟

```JavaScript
console.log(0 && "A" || "B");
console.log("A" || "B" && "C");
console.log(null || 0 || "" || "last");
```

۱) `B` `A` `last`
۲) `false` `true` `true`
۳) `A` `C` `last`
۴) `B` `C` `null`

۲۴۹ متوسط

کدام گزینه درباره‌ی عملگرهای منطقی در JavaScript **نادرست** است؟

۱) `&&` و `||` مثل زبان C اتصال کوتاه (short-circuit) دارند.
۲) `!` همیشه یک مقدار boolean برمی‌گرداند.
۳) اولویت `&&` از `||` بیشتر است.
۴) `&&` و `||` همیشه `true` یا `false` برمی‌گردانند.

۲۵۰ آسان

خروجی کد زیر کدام است؟

```JavaScript
console.log(!"hello", !!"hello", !0);
```

۱) `false true true`
۲) `true false false`
۳) `false false true`
۴) `false true false`

۲۵۱ سخت

حاصل `[] == ![]` کدام است؟

۱) `false`
۲) `true`
۳) خطای `TypeError`
۴) `undefined`

۲۵۲ متوسط

خروجی کد زیر کدام است؟

```javascript
const volume = 0;
const a = volume || 50;
const b = volume ?? 50;
console.log(a, b);
```

۱) `0 0`
۲) `50 50`
۳) `0 50`
۴) `50 0`

۲۵۳ متوسط

حاصل کدام عبارت `"x"` است؟

۱) `undefined ?? "x"`
۲) `0 ?? "x"`
۳) `"" ?? "x"`
۴) `false ?? "x"`

۱-۱۲ کنترل جریان و دستورهای شرطی (if و switch)

منبع: Introduction to JS، اسلایدهای ۲۹ و ۳۰

درسنامه

فرض کن با یک مسیریاب (GPS) رانندگی می‌کنی. بیشترِ راه را مستقیم جلو می‌روی، ولی سر هر تقاطع باید **تصمیم** بگیری: «اگر ترافیک است، از خیابان فرعی برو؛ وگرنه مستقیم برو». گاهی هم باید کاری را **تکرار** کنی: «دور میدان بچرخ تا به خروجی سوم برسی». برنامه‌ها هم دقیقاً همین‌طورند. به ترتیبی که دستورهای یک برنامه اجرا می‌شوند **control flow** (کنترل جریان؛ یعنی «کدام دستور، کِی و به چه ترتیبی اجرا شود») می‌گوییم.

کنترل جریان (Control flow): اسلاید ۲۹

اسلاید ۲۹ این پنج نکته را می‌گوید:

- control flow تعیین می‌کند **کدام کد** و **چه زمانی** اجرا شود.
- به‌طور پیش‌فرض، JavaScript کد را **از بالا به پایین** و خط‌به‌خط اجرا می‌کند.
- **دستورهای شرطی (conditionals)** به ما اجازه می‌دهند مسیر برنامه را **دوشاخه کنیم (branch)**؛ مثل `if` و `else`.
- **حلقه‌ها (loops)** به ما اجازه می‌دهند کدی را **تکرار** کنیم؛ مثل `for` و `while`.
- این دو با هم به برنامه **منطق و ساختار** می‌دهند.

کد اسلاید ۲۹ هر دو ابزار را کنار هم نشان می‌دهد:

cf.js

```
let age = 18;

if (age >= 18) {
  console.log("You can vote!");
} else {
  console.log("Too young to vote.");
}

for (let i = 1; i <= 3; i++) {
  console.log("Count:", i);
}
```

خروجی:

```
You can vote!
Count: 1
Count: 2
Count: 3
```

بیا ببینیم چه شد:

- شرط `age >= 18` یعنی «آیا `age` بزرگ‌تر **یا مساوی** ۱۸ است؟». چون `18 >= 18` درست (`true`) است، فقط بلوک `if` اجرا شد و بلوک `else` کاملاً **رد شد**. اگر خط اول `let age = 16;` بود، فقط `Too young to vote.` چاپ می‌شد.
- حلقه‌ی `for` بدنه‌اش را برای `i` برابر ۱، ۲ و ۳ اجرا کرد. وقتی `i` به ۴ رسید، شرط `i <= 3` نادرست شد و حلقه تمام شد. (`console.log` وقتی چند ورودی بگیرد، آن‌ها را با یک فاصله کنار هم چاپ می‌کند؛ برای همین `Count: 1` چاپ شد.) جزئیات

حلقه‌ها را در درس بعد، «حلقه‌ها (Loops)»، کامل می‌خوانیم.

در این درس روی **دستورهای شرطی** تمرکز می‌کنیم.

دستورهای شرطی (Conditionals): اسلاید ۳۰

طبق اسلاید ۳۰، دستورهای شرطی:

- به کد اجازه می‌دهند **تصمیم بگیرد** (make decisions)؛
- بلوک‌هایی از کد را **فقط وقتی یک شرط درست است** اجرا می‌کنند؛

و چهار شکل اصلی دارند:

شکل	کاربرد (طبق اسلاید ۳۰)
`if`	اگر شرط درست است، کدی را اجرا کن
`if / else`	بین **دو مسیر** یکی را انتخاب کن
`else if`	چند شرط را **پشت سر هم زنجیر کن**
`switch`	**چند حالت مشخص** (چند مقدار معین) را مدیریت کن

`if` : «اگر ... آن‌گاه ...»

```JavaScript
const temperature = 35;

if (temperature > 30) {
  console.log("It's hot! Drink water.");
}
console.log("Have a nice day.");
// It's hot! Drink water.
// Have a nice day.
```

- شرط همیشه داخل **پرانتز** `( )` می‌آید.
- کدی که باید در صورت درست بودن شرط اجرا شود داخل **آکولاد** `{ }` می‌آید؛ به این بخش **بلوک (block)** می‌گوییم.
- خط آخر بیرون از `if` است، پس **همیشه** اجرا می‌شود. اگر `temperature` برابر ۲۰ بود، فقط `Have a nice day.` چاپ می‌شد.

شرط چطور سنجیده می‌شود؟ truthy و falsy

داخل پرانتز `if` لازم نیست حتماً `true` یا `false` بنویسی؛ هر مقداری مجاز است. JavaScript آن مقدار را پشت پرده با همان قانون `Boolean(x)` (درس «تبدیل نوع صریح (Casting)») به boolean تبدیل می‌کند. یادآوری:

- فقط ۸ مقدار **falsy** (شبه‌نادرست؛ یعنی در شرط مثل `false` رفتار می‌کنند) داریم: `false` ، `0` ، `-0` ، `0n` ، `""` (رشته‌ی خالی)، `null` ، `undefined` و `NaN` .
- **هر** مقدار دیگری **truthy** (شبه‌درست) است؛ حتی `"0"` ، `"false"` ، `" "` ، `[]` ، `{}` و `-1` .

```JavaScript
if ("0") console.log("A");       // runs: "0" is a non-empty string
if ([]) console.log("B");        // runs: every array (even an empty one) is truthy
if ({}) console.log("C");        // runs: every object is truthy
if (0) console.log("D");         // skipped: 0 is falsy
if ("") console.log("E");        // skipped: the empty string is falsy
if (null) console.log("F");      // skipped
if (undefined) console.log("G"); // skipped
if (NaN) console.log("H");       // skipped
// Output: A, B, C (each on its own line)
```

(این‌جا بدنه‌ی هر `if` فقط یک دستور است و آکولاد نگذاشته‌ایم. چند بخش پایین‌تر می‌بینیم این کار چه دامی دارد.)

کاربرد رایج: بررسی «خالی نبودن» یک مقدار.

```JavaScript
const username = "";
if (username) {
  console.log("Hello, " + username);
} else {
  console.log("Please enter your name.");
}
// Please enter your name.
```

`if / else` : انتخاب بین دو مسیر

`else` یعنی «وگرنه». در ساختار `if / else` **دقیقاً یکی** از دو بلوک اجرا می‌شود: اگر شرط truthy باشد بلوک `if`، وگرنه بلوک `else`. هیچ‌وقت هر دو با هم اجرا نمی‌شوند و هیچ‌وقت هر دو جا نمی‌مانند.

```JavaScript
const num = 7;
if (num % 2 === 0) {
  console.log("even");
} else {
  console.log("odd");
}
// odd
```

`else if` : زنجیره‌ی چند شرط

وقتی بیش از دو حالت داریم، شرط‌ها را با `else if` پشت سر هم می‌چینیم:

```JavaScript
const score = 85;
if (score >= 90) {
  console.log("A");
} else if (score >= 75) {
  console.log("B");
} else if (score >= 50) {
  console.log("C");
} else {
  console.log("F");
}
// B
```

قانون طلایی زنجیره: شرط‌ها **به ترتیب از بالا** بررسی می‌شوند و **فقط اولین شاخه‌ای که شرطش درست باشد** اجرا می‌شود. بعد از آن، بقیه‌ی زنجیره (حتی اگر شرطشان درست باشد) **اصلاً بررسی نمی‌شود**. این‌جا `85 >= 50` هم درست است، ولی چون `85 >= 75` زودتر درست شد، فقط `B` چاپ شد.

`else` آخر اختیاری است و نقش «در هر حالت دیگر» را دارد. اگر نباشد و هیچ شرطی درست نباشد، **هیچ** شاخه‌ای اجرا نمی‌شود.

> ⚠ **ترتیب شرط‌ها مهم است**
>
> اگر شرط کلی‌تر را اول بگذاری، شرط‌های خاص‌تر هیچ‌وقت به نوبت نمی‌رسند:
>
> ```JavaScript
> const score = 95;
> if (score >= 50) {
>   console.log("C");   // 95 >= 50 is true, so this runs
> } else if (score >= 90) {
>   console.log("A");   // never reached for any score >= 50
> }
> // C
> ```
>
> قاعده: در زنجیره، شرط‌های **سخت‌گیرانه‌تر** را بالاتر بگذار.

دقت کن **چند** `if` **جدا** با **یک زنجیره‌ی** `else if` فرق دارد. `if` های جدا مستقل‌اند و هر کدام که شرطش درست باشد اجرا می‌شود:

```JavaScript
const n = 15;
if (n > 10) console.log("greater than 10");
if (n > 5) console.log("greater than 5");   // a separate if: checked as well
// greater than 10
// greater than 5
```

`if` تودرتو (Nested if)

داخل بلوک یک `if` می‌توان `if` دیگری نوشت. شرط درونی فقط وقتی بررسی می‌شود که شرط بیرونی درست باشد:

```JavaScript
const age = 20;
const hasTicket = false;

if (age >= 18) {
  if (hasTicket) {
    console.log("Enjoy the movie!");
  } else {
    console.log("Please buy a ticket.");
  }
} else {
  console.log("Too young for this movie.");
}
// Please buy a ticket.
```

`switch` : مدیریت چند حالت مشخص

وقتی یک مقدار را با **چند مقدار معین** مقایسه می‌کنی (مثل شماره‌ی روز هفته یا نام یک دستور)، `switch` از یک زنجیره‌ی بلند `else if` خواناتر است:

```JavaScript
const day = 3;

switch (day) {
  case 1:
    console.log("Saturday");
    break;
  case 2:
    console.log("Sunday");
    break;
  case 3:
    console.log("Monday");
    break;
  default:
    console.log("Another day");
}
// Monday
```

طرز کار `switch` :

1. عبارتِ داخل پرانتز (این‌جا `day`) **یک بار** حساب می‌شود.
2. مقدارش به ترتیب با مقدار هر `case` مقایسه می‌شود. این مقایسه **سخت‌گیرانه** است؛ یعنی دقیقاً مثل `===` (هم مقدار و هم نوع باید یکی باشند).
3. اجرا از **اولین** `case` **مطابق** شروع می‌شود.
4. `break` یعنی «همین الان از `switch` بیرون برو».
5. اگر هیچ `case` ی مطابق نباشد، اجرا از بخش `default` (حالت پیش‌فرض) شروع می‌شود. `default` اختیاری است؛ اگر نباشد و هیچ `case` ی مطابق نباشد، هیچ اتفاقی نمی‌افتد.

مقایسه با `===` : رشته‌ی `"1"` با عدد `1` جور نمی‌شود

```JavaScript
const choice = "1";   // e.g. a value typed by the user is always a string

switch (choice) {
  case 1:
    console.log("You chose one");
    break;
  default:
    console.log("Unknown choice");
}
// Unknown choice   ("1" === 1 is false)
```

برخلاف `==` ، این‌جا هیچ تبدیل نوعی (coercion) رخ نمی‌دهد. راه درست: یا بنویس `case "1":` یا ورودی را اول با `Number(choice)` به عدد تبدیل کن.

فراموش کردن `break` : سقوط به caseهای بعدی (Fall-through)

`case`ها فقط **نقطه‌ی شروع** اجرا هستند، نه دیوار. وقتی اجرا از یک `case` شروع شد، تا رسیدن به `break` (یا پایان `switch`) ادامه پیدا می‌کند و **کد caseهای بعدی را هم بدون بررسی مقدارشان اجرا می‌کند**. به این پدیده **fall-through** (سقوط به caseهای پایینی) می‌گویند:

```JavaScript
const fruit = "apple";

switch (fruit) {
  case "apple":
    console.log("red");      // execution starts here
  case "banana":
    console.log("yellow");   // no break above, so it falls through
    break;                   // and stops here
  case "grape":
    console.log("purple");
}
// red
// yellow
```

چند case با یک کد مشترک (گروه‌بندی)

از همین رفتار می‌شود آگاهانه استفاده کرد تا چند مقدار یک کد مشترک داشته باشند:

```JavaScript
const today = "Friday";

switch (today) {
  case "Thursday":
  case "Friday":
    console.log("Weekend!");
    break;
  default:
    console.log("Work day");
}
// Weekend!
```

`default` لازم نیست آخر باشد

`default` می‌تواند هر جای `switch` بیاید و در هر حال فقط وقتی **نقطه‌ی شروع** اجرا می‌شود که هیچ `case` ی مطابق نباشد. ولی اگر وسط باشد و `break` نداشته باشد، اجرا از آن‌جا به caseهای پایینی سقوط می‌کند:

JavaScript

```
const level = 9;

switch (level) {
  default:
    console.log("unknown level");   // chosen: no case matches 9
  case 1:
    console.log("beginner");        // falls through (no break above)
    break;
  case 2:
    console.log("expert");
}
// unknown level
// beginner
```

عملگر سه‌تایی (Ternary operator): `condition ? a : b`

گاهی فقط می‌خواهی بر اساس یک شرط، **یکی از دو مقدار** را انتخاب کنی. عملگر سه‌تایی (**ternary**؛ تنها عملگر JavaScript که سه بخش دارد) یک `if / else` کوتاه است که **مقدار برمی‌گرداند**:

JavaScript

```
const age = 15;
const status = age >= 18 ? "adult" : "minor";
console.log(status); // minor

// the same logic with if / else:
let status2;
if (age >= 18) {
  status2 = "adult";
} else {
  status2 = "minor";
}
console.log(status2); // minor
```

این‌طور بخوانش: «شرط `?` مقدارِ حالت درست `:` مقدارِ حالت نادرست». چون ternary یک **عبارت** (expression؛ تکه‌کدی که یک مقدار تولید می‌کند) است، می‌شود آن را مستقیم داخل `console.log` یا یک template literal گذاشت. `if` یک **دستور** (statement) است و چنین کاری از آن برنمی‌آید:

JavaScript

```
const items = 1;
console.log(`You have ${items} item${items === 1 ? "" : "s"}`); // You have 1 item
```

ternaryها را می‌شود زنجیر هم کرد و مثل یک زنجیره‌ی `else if` خواند («اگر ... وگرنه اگر ... وگرنه ...»)، ولی زیاده‌روی در آن خوانایی را از بین می‌برد:

```JavaScript
const n = 0;
const sign = n > 0 ? "positive" : n < 0 ? "negative" : "zero";
console.log(sign); // zero
```

نکته

- شرط `if` هر مقداری می‌تواند باشد و با قانون truthy/falsy سنجیده می‌شود: `if ("0")`، `if ("false")`، `if ([])` و `if ({})` اجرا **می‌شوند**؛ `if (0)`، `if ("")`، `if (null)`، `if (undefined)` و `if (NaN)` اجرا **نمی‌شوند**.
- در زنجیره‌ی `if / else if / ...` حداکثر **یک** شاخه اجرا می‌شود: اولین شاخه‌ای که شرطش درست است. اگر `else` پایانی داشته باشد، **دقیقاً** یک شاخه اجرا می‌شود.
- `switch` با `===` مقایسه می‌کند؛ `break` از `switch` بیرون می‌برد؛ بدون `break` اجرا به caseهای بعدی سقوط می‌کند؛ `default` (هر جای `switch` که باشد) وقتی نقطه‌ی شروع است که هیچ caseی مطابق نباشد.
- برای انتخاب یک **مقدار** از بین دو مقدار، ternary (`cond ? a : b`) کوتاه‌ترین راه است.

⚠ دام تستی

۱. بدون آکولاد، فقط دستورِ بعدی مال `if` است. تورفتگی (indentation) در JavaScript برخلاف Python هیچ معنایی ندارد:

```JavaScript
const isLoggedIn = false;
if (isLoggedIn)
  console.log("Welcome back!");
  console.log("Loading your profile..."); // NOT part of the if: always runs
// Loading your profile...
```

اگر بعد از چنین `if` بی‌آکولادی دو دستور بیاید و بعد `else`، آن `else` دیگر به `if` نمی‌چسبد و کل فایل `SyntaxError: Unexpected token 'else'` می‌دهد. **همیشه آکولاد بگذار.**

۲. `=` به جای `===` داخل شرط. `if (x = 5)` انتساب است، نه مقایسه! اول عدد ۵ را در `x` **می‌ریزد** و بعد مقدار همان انتساب (یعنی `5` که truthy است) شرط می‌شود؛ پس شرط همیشه درست است و `x` هم عوض شده است. برعکس `if (x = 0)` هیچ‌وقت اجرا نمی‌شود. اگر `x` با `const` تعریف شده باشد، همین خط `TypeError` می‌دهد.

```JavaScript
let x = 10;
if (x = 5) {              // assignment, not comparison!
  console.log("x is 5");  // runs
}
console.log(x);           // 5 (x has been changed)
```

۳. سمی‌کالن بعد از پرانتز `if`. در `if (score > 100); { ... }` آن `;` یک «دستور خالی» است که بدنه‌ی `if` حساب می‌شود، و بلوک `{ ... }` بعدی یک بلوک مستقل است که **همیشه** اجرا می‌شود.

۴. `||` مقایسه را تکرار نمی‌کند. `if (color === "red" || "blue")` یعنی «`color === "red"` یا مقدار `"blue"`»؛ و چون رشته‌ی غیرخالی `"blue"` truthy است، این شرط **همیشه** درست است. شکل درست: `if (color === "red" || color === "blue")`.

۵. ورودی کاربر رشته است. `switch ("1")` با `case 1:` جور نمی‌شود، چون مقایسه با `===` است.

✓ جمع‌بندی

- control flow تعیین می‌کند کدام کد و کِی اجرا شود؛ پیش‌فرض: از بالا به پایین. شرط‌ها مسیر را دوشاخه می‌کنند (`if`، `else`) و حلقه‌ها تکرار می‌کنند (`for`، `while`).
- چهار شکل شرط (اسلاید ۳۰): `if`، `if / else` (دو مسیر)، `else if` (زنجیره‌ی شرط‌ها) و `switch` (چند حالت مشخص).
- شرط با truthy/falsy سنجیده می‌شود؛ در زنجیره فقط اولین شاخه‌ی درست اجرا می‌شود و ترتیب شرط‌ها مهم است.
- `switch`: مقایسه با `===`، `break` برای خروج، fall-through بدون `break`، گروه‌بندی caseها و `default` در هر جای `switch`.
- ternary (`cond ? a : b`) یک عبارت است و یکی از دو مقدار را برمی‌گرداند.

تست‌های تمرینی

۲۶ تست · شماره‌ی ۲۵۴ تا ۲۷۹

۲۵۴ آسان

طبق اسلاید ۲۹، کدام جمله درباره‌ی **control flow** (کنترل جریان) درست است؟

۱) JavaScript همه‌ی خط‌های برنامه را هم‌زمان اجرا می‌کند و ترتیب نوشتن خط‌ها اهمیتی ندارد.

۲) کد به‌طور پیش‌فرض از بالا به پایین اجرا می‌شود و control flow (با شرط‌ها و حلقه‌ها) تعیین می‌کند کدام کد و چه زمانی اجرا شود.

۳) دستورهای شرطی کد را تکرار می‌کنند و حلقه‌ها بین دو مسیر یکی را انتخاب می‌کنند.

۴) کد به‌طور پیش‌فرض از پایین به بالا اجرا می‌شود، مگر این‌که از `if` استفاده کنیم.

۲۵۵ آسان

خروجی کد اسلاید ۲۹ (فایل `cf.js`) چیست؟

```JavaScript
let age = 18;

if (age >= 18) {
  console.log("You can vote!");
} else {
  console.log("Too young to vote.");
}

for (let i = 1; i <= 3; i++) {
  console.log("Count:", i);
}
```

۱) `You can vote!` `Count: 1` `Count: 2` `Count: 3`

۲) `Too young to vote.` `Count: 1` `Count: 2` `Count: 3`

۳) `You can vote!` `Count: 0` `Count: 1` `Count: 2`

۴) `You can vote!` `Too young to vote.` `Count: 1` `Count: 2` `Count: 3`

۲۵۶ آسان

طبق اسلاید ۳۰، کدام شکل از دستورهای شرطی برای «انتخاب بین **دو** مسیر» (choose between two paths) به کار می‌رود؟

۱) `if` ۲) `else if` ۳) `if / else` ۴) `switch`

۲۵۷ آسان

در جای خالی کدام مقدار قرار بگیرد تا خروجی کد زیر `no` باشد؟

```JavaScript
if (___) {
  console.log("yes");
} else {
  console.log("no");
}
```

۱) `"0"`
۲) `"false"`
۳) `[]`
۴) `NaN`

۲۵۸ سخت

خروجی کد زیر چیست؟

```JavaScript
const input = "0";
if (input === 0) {
  console.log("number zero");
} else if (input) {
  console.log("non-empty text");
} else {
  console.log("empty");
}
```

۱) `number zero`
۲) `non-empty text`
۳) `empty`
۴) `number zero` `non-empty text`

۲۵۹ متوسط

خروجی کد زیر چیست؟

```JavaScript
const temp = 28;
if (temp > 30) {
  console.log("Hot");
} else if (temp > 20) {
  console.log("Warm");
} else if (temp > 10) {
  console.log("Cool");
} else {
  console.log("Cold");
}
```

۱) `Warm`
۲) `Warm` `Cool`
۳) `Warm` `Cool` `Cold`
۴) `Hot`

۲۶۰ سخت

خروجی کد زیر چیست؟

```JavaScript
const total = 1200;
let discount;
if (total > 100) {
  discount = 5;
} else if (total > 500) {
  discount = 10;
} else if (total > 1000) {
  discount = 20;
} else {
  discount = 0;
}
console.log(discount);
```

۱) 20
۲) 10
۳) 5
۴) 0

۲۶۱ سخت

خروجی کد زیر چیست؟

```JavaScript
const n = 15;
if (n > 10) console.log("A");
if (n > 5) console.log("B");
else if (n > 0) console.log("C");
else console.log("D");
```

۱) A
۲) B
۳) A B C
۴) A B

۲۶۲ متوسط

خروجی کد زیر چیست؟

```javascript
const loggedIn = true;
const itemCount = 0;
if (loggedIn) {
  if (itemCount) {
    console.log("Checkout");
  } else {
    console.log("Cart is empty");
  }
} else {
  console.log("Please log in");
}
```

۱) `Checkout`

۲) `Cart is empty`

۳) `Please log in`

۴) `Cart is empty` `Please log in`

۲۶۳ آسان

خروجی کد زیر چیست؟

```javascript
const light = "yellow";
switch (light) {
  case "green":
    console.log("Go");
    break;
  case "yellow":
    console.log("Slow down");
    break;
  case "red":
    console.log("Stop");
    break;
  default:
    console.log("Broken light");
}
```

۱) `Broken light`

۲) `Slow down` `Stop`

۳) `Slow down`

۴) `Slow down` `Stop` `Broken light`

۲۶۴ آسان

در یک `switch` که **همه‌ی** caseهایش با `break` تمام می‌شوند، بخش `default` چه زمانی اجرا می‌شود؟

۱) فقط وقتی مقدار مورد بررسی با هیچ‌کدام از caseها برابر نباشد.

۲) همیشه؛ بعد از اجرای case مطابق، `default` هم اجرا می‌شود.

۳) فقط وقتی مقدار مورد بررسی با اولین case برابر باشد.

۴) هیچ‌وقت؛ `default` فقط توضیحی برای خوانایی کد است.

۲۶۵ متوسط

خروجی کد زیر چیست؟

```JavaScript
const input = "2";
switch (input) {
  case 1:
    console.log("one");
    break;
  case 2:
    console.log("two");
    break;
  default:
    console.log("not found");
}
```

۱) `two`

۲) `two` `not found`

۳) `Uncaught TypeError`

۴) `not found`

۲۶۶ متوسط

خروجی کد زیر چیست؟

```JavaScript
const fruit = "banana";
switch (fruit) {
  case "apple":
    console.log("red");
  case "banana":
    console.log("yellow");
  case "grape":
    console.log("purple");
    break;
  default:
    console.log("unknown");
}
```

۱) `yellow`

۲) `yellow` `purple`

۳) `yellow` `purple` `unknown`

۴) `red` `yellow` `purple`

۲۶۷ متوسط

خروجی کد زیر چیست؟

```JavaScript
const month = 2;
switch (month) {
  case 12:
  case 1:
  case 2:
    console.log("Winter");
    break;
  case 3:
  case 4:
  case 5:
    console.log("Spring");
    break;
  default:
    console.log("Other season");
}
```

۱) `Winter`
۲) `Winter` `Spring`
۳) `Other season`
۴) `Uncaught SyntaxError`

۲۶۸ متوسط

در **هر دو** جای خالی کد زیر کدام دستور قرار بگیرد تا خروجی **فقط** `Tue` باشد؟

```JavaScript
const day = 2;
switch (day) {
  case 1:
    console.log("Mon");
    ___
  case 2:
    console.log("Tue");
    ___
  case 3:
    console.log("Wed");
}
```

۱) `continue;`
۲) `default;`
۳) `break;`
۴) `stop;`

۲۶۹ سخت

خروجی کد زیر چیست؟

```javascript
const code = 7;
switch (code) {
  default:
    console.log("unknown");
  case 1:
    console.log("start");
    break;
  case 2:
    console.log("stop");
}
```

۱) `unknown`

۲) `Uncaught SyntaxError`

۳) `unknown` `start` `stop`

۴) `unknown` `start`

۲۷۰ متوسط

کدام گزینه درباره‌ی `switch` **نادرست** است؟

۱) مقایسه‌ی مقدار با هر case سخت‌گیرانه (مانند `===`) است.

۲) اگر یک case با `break` تمام نشود، اجرا وارد کد case بعدی می‌شود.

۳) `default` باید حتماً آخرین بخش `switch` باشد؛ وگرنه `SyntaxError` رخ می‌دهد.

۴) چند case می‌توانند پشت سر هم بیایند و یک کد مشترک داشته باشند.

۲۷۱ آسان

خروجی کد زیر چیست؟

```javascript
const balance = 0;
const message = balance > 0 ? "You have money" : "Your wallet is empty";
console.log(message);
```

۱) `You have money`

۲) `Your wallet is empty`

۳) `false`

۴) `Uncaught SyntaxError`

۲۷۲ متوسط

خروجی کد زیر چیست؟

```JavaScript
const n = -3;
const sign = n > 0 ? "positive" : n < 0 ? "negative" : "zero";
console.log(sign);
```

۱) `negative`
۲) `positive`
۳) `zero`
۴) `Uncaught SyntaxError`

۲۷۳ متوسط

در جای خالی کدام گزینه قرار بگیرد تا خروجی کد زیر `odd` باشد؟

```JavaScript
const n = 7;
const label = ___;
console.log(label);
```

۱) `n % 2 === 0 : "even" ? "odd"`
۲) `n % 2 ? "even" : "odd"`
۳) `if (n % 2 === 0) "even" else "odd"`
۴) `n % 2 === 0 ? "even" : "odd"`

۲۷۴ متوسط

خروجی کد زیر چیست؟

```JavaScript
const isAdmin = false;
if (isAdmin)
  console.log("Delete button visible");
  console.log("Edit button visible");
console.log("Page loaded");
```

۱) `Page loaded`
۲) `Delete button visible` `Edit button visible` `Page loaded`
۳) `Edit button visible` `Page loaded`
۴) `Uncaught SyntaxError`

۲۷۵ سخت

کدام خط کد زیر خطا می‌دهد؟

```JavaScript
1 const x = 5;
2 if (x > 0)
3   console.log("positive");
4   console.log("checked");
5 else
6   console.log("not positive");
```

۱) خط ۲
۲) خط ۵
۳) خط ۴
۴) هیچ خطایی رخ نمی‌دهد

۲۷۶ سخت

خروجی کد زیر چیست؟

```JavaScript
let count = 3;
if (count = 0) {
  console.log("empty");
} else {
  console.log("not empty");
}
console.log(count);
```

۱) `not empty` `0`
۲) `not empty` `3`
۳) `empty` `0`
۴) `Uncaught SyntaxError`

۲۷۷ متوسط

کد زیر در Node.js اجرا می‌شود. کدام خط خطا می‌دهد؟

```JavaScript
1 const limit = 5;
2 let used = 3;
3 if (used < limit) console.log("ok");
4 if (limit = 10) console.log("limit changed");
5 console.log(limit);
```

۱) خط ۱
۲) خط ۳
۳) خط ۵
۴) خط ۴

۲۷۸ سخت

خروجی کد زیر چیست؟

```javascript
const score = 40;
if (score > 100); {
  console.log("Score too high!");
}
console.log("Checked");
```

۱) `Checked`
۲) `Uncaught SyntaxError`
۳) `Score too high!` `Checked`
۴) `Score too high!`

۲۷۹ سخت

خروجی کد زیر چیست؟

```javascript
const color = "green";
if (color === "red" || "blue") {
  console.log("red or blue");
} else {
  console.log("another color");
}
```

۱) `another color`
۲) `Uncaught SyntaxError`
۳) `red or blue` `another color`
۴) `red or blue`

۱-۱۳ حلقه‌ها (Loops)

منبع: Introduction to JS، اسلاید ۳۲

درسنامه

فرض کن معلم گفته صد بار بنویسی «سر کلاس حرف نمی‌زنم». با دست باید صد خط بنویسی؛ اما یک برنامه‌نویس می‌گوید: «این جمله را بنویس و صد بار تکرارش کن». این همان **حلقه (loop)** است: ساختاری که یک تکه کد را **تکرار** می‌کند، بدون این‌که مجبور باشیم آن را چند بار بنویسیم.

اسلاید ۳۲ در یک نگاه

- حلقه‌ها به ما اجازه می‌دهند کد را **بدون نوشتنِ چندباره‌اش** تکرار کنیم.
- یک بلوک را **تا وقتی یک شرط درست است** اجرا می‌کنند.
- انواع اصلی حلقه:

حلقه	کاربرد (طبق اسلاید ۳۲)
`for`	تکرار به **تعداد مشخص**
`while`	تکرار **تا وقتی شرط درست است**
`do...while`	**حداقل یک بار** اجرا، بعد بررسی شرط
`for...of`	پیمایش **مقدارهای (items)** یک آرایه
`for...in`	پیمایش **کلیدهای (keys)** یک شیء

به هر بار اجرای بدنه‌ی حلقه یک **iteration** (دور یا تکرار) می‌گوییم.

حلقه‌ی `for` : تکرار به تعداد مشخص

```JavaScript
for (let i = 1; i <= 3; i++) {
  console.log("Count:", i);
}
// Count: 1
// Count: 2
// Count: 3
```

(این همان حلقه‌ی اسلاید ۲۹ است.) سرِ حلقه‌ی `for` سه بخش دارد که با `;` از هم جدا می‌شوند:

بخش	در مثال	کِی اجرا می‌شود؟
مقداردهی اولیه (initialization)	`let i = 1`	**فقط یک بار**، قبل از شروع حلقه
شرط (condition)	`i <= 3`	**قبل از هر دور**؛ اگر نادرست (falsy) باشد، حلقه تمام می‌شود
به‌روزرسانی (update)	`i++`	**بعد از هر دور** (بعد از اجرای بدنه)

ترتیب دقیق اجرا: init، شرط، بدنه، update، شرط، بدنه، update، ... و در آخر شرطی که نادرست است و حلقه را تمام می‌کند.

ردیابی (trace) مثال بالا:

مقدار `i`	`i <= 3` ؟	چه می‌شود؟
۱	true	چاپ `Count: 1` و بعد `i++`
۲	true	چاپ `Count: 2` و بعد `i++`
۳	true	چاپ `Count: 3` و بعد `i++`
۴	false	پایان حلقه

دقت کن: بدنه ۳ بار اجرا شد ولی شرط **۴ بار** بررسی شد و `i` در پایان به ۴ رسید.

چند بار تکرار می‌شود؟ (دام off-by-one)

اشتباهِ «یکی کم یا یکی زیاد» (**off-by-one**) رایج‌ترین خطای حلقه‌هاست. این جدول را خوب ببین:

سرِ حلقه	مقدارهای `i` در بدنه	تعداد دور
`for (let i = 0; i < 5; i++)`	۰ تا ۴	۵
`for (let i = 0; i <= 5; i++)`	۰ تا ۵	۶
`for (let i = 1; i <= 5; i++)`	۱ تا ۵	۵
`for (let i = 1; i < 5; i++)`	۱ تا ۴	۴

قاعده‌ی کلی: «از `a` شروع کن و تا وقتی `i < b` است یکی‌یکی جلو برو» یعنی `b - a` دور؛ با `<=` یک دور بیشتر.

گام حلقه لازم نیست ۱ باشد و حلقه می‌تواند رو به پایین هم بشمارد:

```JavaScript
for (let i = 10; i > 0; i -= 3) {
  console.log(i);
}
// 10
// 7
// 4
// 1
```

مقدار بعدی `-2` است که شرط `i > 0` را نادرست می‌کند، پس چاپ نمی‌شود.

متغیر حلقه بعد از حلقه کجاست؟

متغیری که با `let` در سرِ حلقه تعریف می‌شود فقط **داخل حلقه** وجود دارد (دلیلش را در درس بعد، «Block Scope و Global scope»، می‌بینیم). اما `var` به بلوک‌ها اهمیت نمی‌دهد و بعد از حلقه هم باقی می‌ماند، با همان مقداری که شرط را نادرست کرد (پیش‌نمایش درس «مشکلات var»):

```JavaScript
for (var a = 0; a < 3; a++) {}
console.log(a); // 3 (var leaks out of the loop)

for (let b = 0; b < 3; b++) {}
console.log(b); // ReferenceError: b is not defined
```

حلقه‌ی `while` : تا وقتی شرط درست است

```JavaScript
let countdown = 3;
while (countdown > 0) {
  console.log(countdown);
  countdown--;
}
console.log("Go!");
// 3
// 2
// 1
// Go!
```

`while` فقط یک شرط دارد. **قبل از هر دور** شرط بررسی می‌شود؛ اگر درست بود بدنه اجرا می‌شود و دوباره سراغ شرط می‌رود. پس:

- اگر شرط **از همان اول** نادرست باشد، بدنه **حتی یک بار** هم اجرا نمی‌شود.
- مقداردهی اولیه را باید **قبل از** حلقه و به‌روزرسانی را **داخل** بدنه بنویسی. اگر به‌روزرسانی را فراموش کنی، شرط هیچ‌وقت نادرست نمی‌شود (حلقه‌ی بی‌نهایت؛ پایین‌تر می‌بینیم).

`while` وقتی مناسب است که **تعداد دورها را از قبل نمی‌دانی**؛ مثلاً «تا وقتی عدد از ۱۰۰ کمتر است، دو برابرش کن»:

```JavaScript
let n = 1;
while (n < 100) {
  n = n * 2;
}
console.log(n); // 128
```

حلقه‌ی `do...while` : اول اجرا، بعد بررسی

```JavaScript
let tries = 10;
do {
  console.log("Attempt", tries);
  tries++;
} while (tries < 5);
// Attempt 10
```

در `do...while` بدنه **اول** اجرا می‌شود و شرط **بعد از** هر دور بررسی می‌شود. پس بدنه **حداقل یک بار** اجرا می‌شود؛ حتی اگر شرط از اول نادرست باشد (مثل همین‌جا: `10 < 5` نادرست است، ولی یک بار چاپ شد). کاربرد معمولش: «یک بار بپرس؛ اگر جواب نامعتبر بود، دوباره بپرس». به `;` در انتهای `while (...)` دقت کن.

مقایسه‌ی `while` و `do...while` با یک شرطِ نادرست:

```JavaScript
let a = 5;
while (a < 5) {
  console.log("while body");   // never runs
}

let b = 5;
do {
  console.log("do body");      // runs once
} while (b < 5);
// do body
```

حلقه‌ی `for...of` : مقدارهای یک آرایه

آرایه (array) فهرستی مرتب از مقدارهاست (درس «آرایه‌ها» آن را کامل می‌گوید). `colors.length` تعداد عضوهای آرایه است و `colors[i]` عضوِ شماره‌ی `i` را می‌دهد (شماره‌گذاری یا **index** از صفر شروع می‌شود). با `for` معمولی می‌شود آرایه را این‌طور پیمود:

```JavaScript
const colors = ["red", "green", "blue"];

for (let i = 0; i < colors.length; i++) {
  console.log(i, colors[i]);
}
// 0 red
// 1 green
// 2 blue
```

اما `for...of` خیلی ساده‌تر است: خانه‌به‌خانه **مقدارها** را به ترتیب می‌دهد و دیگر لازم نیست با شمارنده و index کار کنی:

```JavaScript
// the same colors array as above
for (const color of colors) {
  console.log(color);
}
// red
// green
// blue
```

(متغیر حلقه در هر دور از نو ساخته می‌شود؛ برای همین `const` هم مجاز است.)

`for...of` روی هر چیز **iterable** (پیمایش‌پذیر؛ یعنی چیزی که بشود عضوهایش را یکی‌یکی گرفت) کار می‌کند؛ مثلاً **رشته‌ها** که حرف‌به‌حرف پیمایش می‌شوند:

```JavaScript
for (const ch of "Hi!") {
  console.log(ch);
}
// H
// i
// !
```

اما یک **شیء معمولی (plain object)** iterable نیست و `for...of` روی آن خطا می‌دهد:

```JavaScript
const user = { name: "Ali", age: 20 };
for (const v of user) {}   // TypeError: user is not iterable
```

حلقه‌ی `for...in` : کلیدهای یک شیء

شیء (object) مجموعه‌ای از جفت‌های «کلید: مقدار» است (درس «اشیاء» آن را کامل می‌گوید). `for...in` **نام کلیدها** را به‌صورت **رشته** می‌دهد و با `obj[key]` می‌توان به مقدار هر کلید رسید:

```JavaScript
const user = { name: "Ali", age: 20 };

for (const key in user) {
  console.log(key, user[key]);
}
// name Ali
// age 20
```

اگر `for...in` را روی یک **آرایه** بزنی، کلیدهای آرایه یعنی **indexها** را می‌دهد، آن هم به‌صورت **رشته** (`"0"`، `"1"`، ...)، نه مقدارها را:

```JavaScript
const nums = [10, 20, 30];

for (const i in nums) {
  console.log(i, typeof i);
}
// 0 string
// 1 string
// 2 string

for (const i in nums) {
  console.log(i + 1);   // string concatenation, not addition!
}
// 01
// 11
// 21

for (const n of nums) {
  console.log(n);
}
// 10
// 20
// 30
```

به همین دلیل برای آرایه‌ها `for...of` (یا `for` معمولی) را ترجیح می‌دهیم: معمولاً مقدارها را لازم داریم، indexهای رشته‌ای در محاسبه دردسر می‌سازند (`"0" + 1` می‌شود `"01"`) و `for...in` هر property (ویژگی) دیگری را هم که به آرایه اضافه شده باشد پیمایش می‌کند:

```JavaScript
const list = [10, 20];
list.note = "extra";
for (const k in list) {
  console.log(k);
}
// 0
// 1
// note
```

> **یک تشبیه ساده**
>
> برای به خاطر سپردن: **in** را با **in**dex (کلید) جفت کن و **of** را با «مقدارهایِ» (values **of**) عضوها. `for...in` می‌پرسد «چه کلیدهایی **در** این شیء هست؟» و `for...of` می‌گوید «مقدارهای این مجموعه را یکی‌یکی بده».

`break` و `continue`

- `break` : حلقه را **همین الان کامل تمام می‌کند** و اجرا از خطِ بعد از حلقه ادامه پیدا می‌کند.
- `continue` : **فقط بقیه‌ی همین دور** را رد می‌کند و سراغ دور بعد می‌رود (در `for` ، بخش update اجرا می‌شود و بعد شرط بررسی می‌شود).

```JavaScript
for (let i = 1; i <= 10; i++) {
  if (i === 4) break;        // stop the whole loop at 4
  console.log(i);
}
// 1
// 2
// 3

for (let i = 1; i <= 5; i++) {
  if (i % 2 === 0) continue; // skip even numbers
  console.log(i);
}
// 1
// 3
// 5
```

حلقه‌ی بی‌نهایت (Infinite loop)

اگر شرط حلقه **هیچ‌وقت** نادرست نشود، حلقه تا ابد ادامه پیدا می‌کند. چون JavaScript **تک‌نخی (single-threaded)** است (یعنی در هر لحظه فقط یک کار انجام می‌دهد)، یک حلقه‌ی بی‌نهایت در مرورگر کل صفحه را **قفل** می‌کند: دکمه‌ها کار نمی‌کنند و مرورگر بعد از مدتی پیغام «صفحه پاسخ نمی‌دهد» نشان می‌دهد. سه علت رایج (این کدها را اجرا نکن!):

```JavaScript
// 1) forgetting the update
let i = 0;
while (i < 3) {
  console.log(i);   // i never changes, so this prints 0 forever
}

// 2) updating in the wrong direction
for (let j = 0; j < 3; j--) {}   // j: 0, -1, -2, ... is always < 3

// 3) continue before the update in a while loop
let k = 0;
while (k < 5) {
  if (k === 2) continue;   // k stays 2 forever: k++ below is never reached
  k++;
}
```

`while (true)` عمداً بی‌نهایت است و فقط وقتی امن است که داخلش با `break` از حلقه خارج شوی:

```JavaScript
let total = 0;
while (true) {
  total += 5;
  if (total >= 20) break;
}
console.log(total); // 20
```

حلقه‌های تودرتو (Nested loops)

حلقه داخل حلقه: به ازای **هر** دورِ حلقه‌ی بیرونی، حلقه‌ی درونی **از اول تا آخر** اجرا می‌شود. پس تعداد کل اجراهای بدنه‌ی درونی برابر است با (تعداد دورهای بیرونی) × (تعداد دورهای درونی):

```JavaScript
for (let row = 1; row <= 2; row++) {
  for (let col = 1; col <= 3; col++) {
    console.log(row, col);
  }
}
// 1 1
// 1 2
// 1 3
// 2 1
// 2 2
// 2 3   (2 x 3 = 6 lines)
```

`break` و `continue` فقط روی **نزدیک‌ترین** حلقه‌ای که داخلش هستند اثر دارند: `break` در حلقه‌ی درونی فقط حلقه‌ی درونی را تمام می‌کند و حلقه‌ی بیرونی دور بعدی‌اش را شروع می‌کند.

کدام حلقه برای کدام کار؟

موقعیت	حلقه‌ی مناسب
تعداد دورها مشخص است (مثلاً ۱۰ بار)	`for`
تعداد دورها از قبل معلوم نیست؛ «تا وقتی ...»	`while`
بدنه باید حداقل یک بار اجرا شود	`do...while`
مقدارهای یک آرایه یا حرف‌های یک رشته	`for...of`
کلیدهای یک شیء	`for...in`

نکته

- `for (init; condition; update)` : init یک بار، شرط قبل از هر دور و update بعد از هر دور اجرا می‌شود.
- شروع از ۰ با `i < n` یعنی **n** دور؛ با `i <= n` یعنی **n + ۱** دور.
- `while` ممکن است **صفر** بار اجرا شود؛ `do...while` **حداقل یک** بار.
- `for...of` : مقدارها (آرایه، رشته). `for...in` : کلیدها به‌صورت رشته (شیء؛ روی آرایه، indexهای رشته‌ای).
- `break` کل حلقه (فقط نزدیک‌ترین حلقه) را تمام می‌کند؛ `continue` فقط دور فعلی را رد می‌کند.

دام تستی

- `for...in` روی آرایه **indexها را به‌صورت رشته** می‌دهد، نه مقدارها را: `i + 1` می‌شود `"01"`، نه `1`.
- `for...of` روی یک شیء معمولی خطای `TypeError: ... is not iterable` می‌دهد.
- `do...while` با شرط نادرست هم **یک بار** اجرا می‌شود.
- متغیر حلقه‌ای که با `let` تعریف شده بعد از حلقه وجود ندارد (`ReferenceError`)؛ اما `var` بیرون می‌ماند و مقدار پایانی‌اش (اولین مقداری که شرط را نادرست کرد) را دارد.
- در `while`، اگر `continue` قبل از به‌روزرسانی شمارنده بیاید، حلقه بی‌نهایت می‌شود.
- اگر داخل بدنه‌ی `for` هم شمارنده را تغییر دهی (مثلاً `i++` اضافه)، هر دور دو بار جلو می‌روی.

جمع‌بندی

- حلقه یعنی تکرار یک بلوک تا وقتی شرط درست است؛ بدون نوشتنِ چندباره‌ی کد.
- `for` (تعداد مشخص)، `while` (تا وقتی شرط درست است؛ شاید صفر بار)، `do...while` (حداقل یک بار)، `for...of` (مقدارها)، `for...in` (کلیدها).
- مراقب off-by-one (`<` در برابر `<=`)، حلقه‌ی بی‌نهایت و تفاوت `for...in` و `for...of` باش.
- `break` خروج از حلقه، `continue` رفتن به دور بعد؛ هر دو فقط روی نزدیک‌ترین حلقه.

تست‌های تمرینی

۲۵ تست · شماره‌ی ۲۸۰ تا ۳۰۴

۲۸۰ آسان

کدام حلقه بدنه‌اش را **حداقل یک بار** اجرا می‌کند، حتی اگر شرطش از همان اول نادرست باشد؟

۱) `for`
۲) `while`
۳) `do...while`
۴) `for...in`

۲۸۱ آسان

طبق اسلاید ۳۲، برای پیمایش **کلیدهای** (keys) یک شیء از کدام حلقه استفاده می‌شود؟

۱) `for...of`
۲) `for...in`
۳) `do...while`
۴) `while`

۲۸۲ آسان

کدام جمله درباره‌ی حلقه‌ی `while` درست است؟

۱) شرط بعد از هر دور بررسی می‌شود، پس بدنه حداقل یک بار اجرا می‌شود.

۲) همیشه دقیقاً ۱۰ بار تکرار می‌شود، مگر این‌که از `break` استفاده کنیم.

۳) فقط برای پیمایش آرایه‌ها کاربرد دارد.

۴) شرط قبل از هر دور بررسی می‌شود؛ اگر از همان اول نادرست باشد، بدنه حتی یک بار هم اجرا نمی‌شود.

۲۸۳ آسان

خروجی کد زیر چیست؟

```JavaScript
for (let i = 0; i < 3; i++) {
  console.log("Hi", i);
}
```

۱) `Hi 0` `Hi 1` `Hi 2`
۲) `Hi 1` `Hi 2` `Hi 3`
۳) `Hi 0` `Hi 1` `Hi 2` `Hi 3`
۴) `Hi 3`

۲۸۴ آسان

خروجی کد زیر چیست؟

```JavaScript
let count = 0;
for (let i = 0; i < 10; i++) {
  count++;
}
console.log(count);
```

۱) `9`
۲) `10`
۳) `11`
۴) `Uncaught ReferenceError`

۲۸۵ متوسط

خروجی کد زیر چیست؟

```JavaScript
let sum = 0;
for (let i = 1; i <= 4; i++) {
  sum += i;
}
console.log(sum);
```

۱) 6
۲) 15
۳) 10
۴) 4

۲۸۶ متوسط

خروجی کد زیر چیست؟

```JavaScript
for (let i = 10; i > 0; i -= 3) {
  console.log(i);
}
```

۱) 10 7 4 1
۲) 10 7 4
۳) 10 7 4 1 -2
۴) 7 4 1

۲۸۷ سخت

خروجی کد زیر چیست؟

```JavaScript
for (let i = 0; i < 5; i++) {
  console.log(i);
  i++;
}
```

۱) 0 1 2 3 4
۲) 0 2 4 6
۳) 0 2
۴) 0 2 4

۲۸۸ سخت

خروجی کد زیر در Node.js چیست؟

```JavaScript
for (var i = 0; i < 3; i++) {}
console.log(i);
for (let j = 0; j < 3; j++) {}
console.log(j);
```

۱) 2 Uncaught ReferenceError
۲) 3 Uncaught ReferenceError
۳) 3 3
۴) Uncaught ReferenceError

۲۸۹ آسان

خروجی کد زیر چیست؟

```javascript
let n = 3;
while (n > 0) {
  console.log(n);
  n--;
}
console.log("Go!");
```

۱) `3 2 1 Go!`
۲) `3 2 1 0 Go!`
۳) `2 1 0 Go!`
۴) `Go!`

۲۹۰ متوسط

خروجی کد زیر چیست؟

```javascript
let x = 1;
while (x < 100) {
  x = x * 3;
}
console.log(x);
```

۱) `81`
۲) `100`
۳) `243`
۴) `729`

۲۹۱ متوسط

خروجی کد زیر چیست؟

```javascript
let a = 5;
while (a < 5) {
  console.log("while", a);
  a++;
}
let b = 5;
do {
  console.log("do", b);
  b++;
} while (b < 5);
```

۱) `do 5` `while 5`
۲) (بدون خروجی)
۳) `while 5`
۴) `do 5`

۲۹۲ آسان

خروجی کد زیر چیست؟

```JavaScript
const pets = ["cat", "dog", "fish"];
for (const pet of pets) {
  console.log(pet);
}
```

۱) `cat` `dog` `fish`

۲) `0` `1` `2`

۳) `['cat', 'dog', 'fish']`

۴) `Uncaught TypeError`

۲۹۳ متوسط

خروجی کد زیر چیست؟

```JavaScript
let count = 0;
for (const ch of "banana") {
  if (ch === "a") count++;
}
console.log(count);
```

۱) `6`

۲) `3`

۳) `1`

۴) `Uncaught TypeError`

۲۹۴ متوسط

در جای خالی کدام گزینه قرار بگیرد تا خروجی کد زیر `a`، `b` و `c` (هر کدام در یک خط) باشد؟

```JavaScript
const letters = ["a", "b", "c"];
___ {
  console.log(item);
}
```

۱) `for (const item in letters)`

۲) `for (let item = 0; item < letters.length; item++)`

۳) `for (const item of letters)`

۴) `for (const item of letters.length)`

۲۹۵ متوسط

خروجی کد زیر چیست؟

```javascript
const car = { brand: "BMW", year: 2020 };
for (const key in car) {
  console.log(key);
}
```

۱) `BMW` `2020`
۲) `0` `1`
۳) `Uncaught TypeError`
۴) `brand` `year`

۲۹۶ سخت

خروجی کد زیر چیست؟

```javascript
const scores = [10, 20, 30];
for (const i in scores) {
  console.log(i + 1);
}
```

۱) `1` `2` `3`
۲) `11` `21` `31`
۳) `01` `11` `21`
۴) `101` `201` `301`

۲۹۷ سخت

کدام خط کد زیر خطا می‌دهد؟

```javascript
const scores = [12, 18];
for (const s of scores) console.log(s);
const student = { name: "Sara", score: 19 };
for (const key in student) console.log(key);
for (const value of student) console.log(value);
```

۱) خط ۲
۲) خط ۵
۳) خط ۴
۴) هیچ خطایی رخ نمی‌دهد

۲۹۸ متوسط

خروجی کد زیر چیست؟

```JavaScript
for (let i = 1; i <= 10; i++) {
  if (i === 4) {
    break;
  }
  console.log(i);
}
console.log("done");
```

۱) `1` `2` `3` `done`

۲) `1` `2` `3` `4` `done`

۳) `1` `2` `3`

۴) `1` `2` `3` `5` `6` `7` `8` `9` `10` `done`

۲۹۹ متوسط

خروجی کد زیر چیست؟

```JavaScript
for (let i = 1; i <= 6; i++) {
  if (i % 3 === 0) continue;
  console.log(i);
}
```

۱) `1` `2`

۲) `3` `6`

۳) `1` `2` `3` `4` `5` `6`

۴) `1` `2` `4` `5`

۳۰۰ متوسط

کدام حلقه **هرگز تمام نمی‌شود** (حلقه‌ی بی‌نهایت است)؟

۱) `for (let i = 0; i < 3; i++) {}`

۲) `let i = 5; while (i > 0) { i--; }`

۳) `for (let i = 0; i < 3; i--) {}`

۴) `let i = 0; do { i += 2; } while (i < 7);`

۳۰۱ سخت

درباره‌ی رفتار کد زیر کدام گزینه درست است؟

```JavaScript
let k = 0;
while (k < 5) {
  if (k === 2) continue;
  console.log(k);
  k++;
}
console.log("end");
```

۱) عددهای `0`، `1`، `3` و `4` و سپس `end` را چاپ می‌کند.

۲) عددهای `0` و `1` را چاپ می‌کند و بعد برای همیشه در حلقه می‌ماند (حلقه‌ی بی‌نهایت)؛ `end` هرگز چاپ نمی‌شود.

۳) عددهای `0` تا `4` و سپس `end` را چاپ می‌کند.

۴) خطای `SyntaxError` می‌دهد، چون `continue` فقط داخل حلقه‌ی `for` مجاز است.

۳۰۲ متوسط

خروجی کد زیر چیست؟

```JavaScript
let count = 0;
for (let i = 0; i < 3; i++) {
  for (let j = 0; j < 4; j++) {
    count++;
  }
}
console.log(count);
```

۱) `7`　۲) `4`　۳) `16`　۴) `12`

۳۰۳ سخت

خروجی کد زیر چیست؟

```JavaScript
for (let i = 1; i <= 3; i++) {
  for (let j = 1; j <= 3; j++) {
    if (j === 2) break;
    console.log(i, j);
  }
}
```

۱) `1 1` `2 1` `3 1`

۲) `1 1`

۳) `1 1` `1 3` `2 1` `2 3` `3 1` `3 3`

۴) `1 1` `1 2` `2 1` `2 2` `3 1` `3 2`

۳۰۴ سخت

کدام گزینه **نادرست** است؟

۱) `for...of` روی یک رشته، حرف‌های آن را یکی‌یکی می‌دهد.

۲) `for...in` روی یک آرایه، indexها را به‌صورت رشته (`"0"`، `"1"`، ...) می‌دهد.

۳) در `do...while` شرط قبل از اولین اجرای بدنه بررسی می‌شود.

۴) `break` داخل یک حلقه، اجرای همان حلقه را کامل متوقف می‌کند.

۱-۱۴ Block، Scope و Global scope

منبع: Introduction to JS، اسلایدهای ۲۵ و ۲۶

درسنامه

یک ساختمان با شیشه‌های **دودی یک‌طرفه** را تصور کن: از داخل هر اتاق می‌توانی بیرون را ببینی، ولی از بیرون نمی‌شود داخل اتاق را دید. متغیرها در JavaScript دقیقاً همین‌طور «دیده می‌شوند»: کدی که داخل یک بلوک است متغیرهای بیرون را می‌بیند، ولی کد بیرونی متغیرهای داخل بلوک را نمی‌بیند.

به محدوده‌ای از کد که یک متغیر در آن **قابل دسترسی** است، **scope** (محدوده یا حوزه‌ی دیدِ متغیر) می‌گوییم.

بلوک (Block): اسلاید ۲۵

- **بلوک** یعنی کدی که داخل آکولاد `{ ... }` است.
- متغیرهایی که با `let` یا `const` داخل یک بلوک تعریف شوند **فقط داخل همان بلوک** وجود دارند.
- **بیرون از بلوک** به آن‌ها **دسترسی نداریم**.

کد اسلاید ۲۵:

scopes.js

```
{
  let x = 10;
  console.log(x); // works
}
console.log(x);   // error: x is not defined
```

خروجی در Node.js:

```
10
ReferenceError: x is not defined
```

خط سوم داخل بلوک است و `10` را چاپ می‌کند. بلوک که تمام شد، `x` هم از بین رفت؛ پس خط آخر خطای `ReferenceError` (خطای ارجاع؛ یعنی «چنین نامی پیدا نشد») می‌دهد. این خطا در **زمان اجرا** رخ می‌دهد؛ برای همین `10` قبل از آن چاپ شده است.

به این رفتار **block scope** (محدوده‌ی بلوکی) می‌گوییم و `let` و `const` را **block-scoped** می‌نامیم. بلوک‌ها همه‌جا هستند: بدنه‌ی `if` و `else`، بدنه‌ی حلقه‌ها، بدنه‌ی `switch`، بدنه‌ی توابع یا حتی یک جفت آکولادِ تنها مثل همین مثال. (آکولادهای یک شیء مثل `{ name: "Ali" }` بلوک نیستند؛ آن‌ها یک شیء می‌سازند که در درس «اشیاء» با آن آشنا می‌شویم.)

JavaScript

```
const age = 20;
if (age >= 18) {
  const message = "Adult";   // lives only inside this if-block
  console.log(message);      // Adult
}
console.log(typeof message); // undefined (message does not exist out here)

for (let i = 0; i < 2; i++) {
  const square = i * i;      // a fresh `square` in every iteration
}
console.log(typeof i, typeof square); // undefined undefined
```

(عملگر `typeof` نوع یک مقدار را به‌صورت رشته برمی‌گرداند و روی نامی که پیدا نشود، به جای خطا `"undefined"` می‌دهد؛ پایین‌تر دوباره به آن می‌رسیم.)

جمله‌ی پایانی اسلاید ۲۵ می‌گوید: «بلوک‌ها فقط یک قاعده‌ی نوشتاری (syntax) نیستند؛ ابزاری برای **سازمان‌دهی** و **محافظت** از متغیرها هستند.» یعنی:

- **سازمان‌دهی:** متغیرهای موقت (مثل `square` بالا) فقط همان‌جایی هستند که لازم‌اند و بقیه‌ی کد شلوغ نمی‌شود.
- **محافظت:** کدِ بیرون از بلوک نمی‌تواند به‌اشتباه متغیرهای داخل آن را بخواند یا خراب کند، و نام‌های داخل بلوک با نام‌های بیرون **تداخل** پیدا نمی‌کنند.

Global scope (محدوده‌ی سراسری): اسلاید ۲۶

- متغیری که **بیرون از هر بلوکی** (و بیرون از هر تابعی) تعریف شود **global** (سراسری) است.
- به متغیرهای سراسری **از هر جای کد** می‌شود دسترسی داشت.
- تا وقتی برنامه اجرا می‌شود **زنده می‌مانند** (برخلاف متغیرهای یک بلوک که با تمام شدن بلوک از بین می‌روند).
- در مرورگرها، متغیرهای سراسری به property (ویژگی)های شیء `window` تبدیل می‌شوند (جزئیات دقیقش را پایین‌تر می‌بینیم).

کد اسلاید ۲۶:

global.js

```
let message = "Hello";  // global

function greet() {
  console.log(message); // accessible here
}

greet();
console.log(message);   // accessible here too
// Hello
// Hello
```

`function greet() { ... }` یک **تابع (function)** تعریف می‌کند: یک تکه کدِ نام‌دار که هر بار `greet()` را بنویسیم اجرا می‌شود (توابع را در درس «توابع» کامل می‌خوانیم). `message` بیرون از همه‌ی بلوک‌ها تعریف شده، پس هم داخل تابع و هم بیرون از آن در دسترس است.

Function scope (محدوده‌ی تابع)

بدنه‌ی هر تابع هم یک محدوده است: هر متغیری که داخل تابع تعریف شود (با `let` ، `const` و حتی `var`) فقط داخل همان تابع وجود دارد. این محدوده با **هر بار صدا زدنِ** تابع از نو ساخته می‌شود.

JavaScript

```
function makeTea() {
  const sugar = 2;            // local to makeTea
  console.log("Sugar:", sugar);
}
makeTea();                    // Sugar: 2
console.log(typeof sugar);    // undefined (sugar does not exist out here)
```

تفاوت مهم `var` : `var` ی که داخل یک `if` یا حلقه تعریف شود، در کل تابعِ دربرگیرنده (یا اگر داخل هیچ تابعی نباشد، در کل برنامه) دیده می‌شود. جزئیات بیشتر در درس «مشکلات var».

JavaScript

```
function test() {
  if (true) {
    var a = "var";   // function-scoped: visible in the whole function
    let b = "let";   // block-scoped: visible only in this if-block
  }
  console.log(a);    // var
  console.log(b);    // ReferenceError: b is not defined
}
test();
```

	`let`	`const`	`var`
محدوده	بلوک `{ }`	بلوک `{ }`	کل تابع
دیده شدن بعد از بلوکِ `if` /حلقه	خیر	خیر	بله
دیده شدن بیرون از تابع	خیر	خیر	خیر

محدوده‌های تودرتو و زنجیره‌ی scope (Scope chain)

محدوده‌ها مثل عروسک‌های روسی داخل هم قرار می‌گیرند. وقتی کد از یک نام استفاده می‌کند، JavaScript آن را این‌طور جست‌وجو می‌کند:

1. اول در **همان محدوده‌ی فعلی**؛
2. اگر نبود، در **محدوده‌ی بیرونیِ** آن؛
3. و همین‌طور بیرون‌تر، تا **global scope**؛
4. اگر آن‌جا هم نبود: `ReferenceError` .

به این مسیر جست‌وجو **scope chain** (زنجیره‌ی محدوده‌ها) می‌گوییم. جست‌وجو **فقط رو به بیرون** است: محدوده‌ی درونی محدوده‌های بیرونی را می‌بیند، ولی محدوده‌ی بیرونی هیچ‌وقت داخل محدوده‌های درونی را نمی‌بیند.

```JavaScript
const city = "Tehran";                 // global scope

function outer() {
  const street = "Azadi";              // outer's scope
  function inner() {
    const house = 12;                  // inner's scope
    console.log(city, street, house);  // inner sees all three
  }
  inner();
  console.log(typeof house);           // outer cannot see inside inner
}

outer();
// Tehran Azadi 12
// undefined
```

Shadowing (سایه انداختن)

اگر در یک محدوده‌ی درونی متغیری با **همان نامِ** یک متغیر بیرونی تعریف کنی، متغیر درونی **روی** بیرونی سایه می‌اندازد (**shadowing**): داخل آن محدوده هر جا آن نام را بنویسی به متغیر درونی می‌رسی و متغیر بیرونی **دست‌نخورده** می‌ماند. دلیلش این است که جست‌وجو از محدوده‌ی فعلی شروع می‌شود و با **اولین** پیدا شدن متوقف می‌شود.

```JavaScript
let color = "blue";

{
  let color = "red";     // a NEW variable that shadows the outer one
  console.log(color);    // red
}

console.log(color);      // blue (the outer variable was never touched)
```

این **تعریف دوباره** حساب نمی‌شود (چون دو متغیر در دو محدوده‌ی جدا هستند) و خطا نمی‌دهد. اما مراقب باش: اگر داخل بلوک `let` را **ننویسی**، متغیر جدیدی نساخته‌ای و داری **همان متغیر بیرونی** را تغییر می‌دهی:

```JavaScript
let count = 1;
{
  count = 2;             // no let: this changes the OUTER count
}
console.log(count);      // 2
```

پارامترهای تابع (ورودی‌هایی که داخل پرانتزِ تعریف تابع می‌آیند) هم متغیرهای محلیِ همان تابع‌اند و می‌توانند روی یک متغیر بیرونیِ هم‌نام سایه بیندازند (`return` مقدار را به جایی که تابع صدا زده شده برمی‌گرداند):

```JavaScript
let x = 10;
function double(x) {     // this x is a parameter, local to double
  x = x * 2;
  return x;
}
console.log(double(3));  // 6
console.log(x);          // 10
```

(اما داخل **خودِ** بدنه‌ی تابع نمی‌توانی با `let` متغیری هم‌نامِ یکی از پارامترها تعریف کنی: `function f(x) { let x = 1; }` تعریف دوباره است و `SyntaxError` می‌دهد، چون پارامتر و آن `let` در یک محدوده‌اند.)

متغیرهای سراسری و شیء `window` (در مرورگر)

اسلاید ۲۶ می‌گوید در مرورگر، متغیرهای سراسری property شیء `window` (شیء سراسری مرورگر که پنجره‌ی فعلی را نشان می‌دهد) می‌شوند. دقیق‌ترش این است:

تعریف در سطح بالای یک اسکریپت مرورگر	سراسری است؟	property شیء `window` می‌شود؟
`var a = 1;`	بله	**بله** (`window.a` برابر `1`)
`function f() {}`	بله	**بله** (`typeof window.f` برابر `"function"`)
`let b = 2;`	بله	**خیر** (`window.b` برابر `undefined`)
`const c = 3;`	بله	**خیر** (`window.c` برابر `undefined`)

```JavaScript
// in a browser (a classic <script>)
var version = "1.0";
let appName = "Quiz";
console.log(window.version); // 1.0
console.log(window.appName); // undefined
console.log(appName);        // Quiz (still global and usable everywhere)
```

پس `let` و `const` سطح بالا هم **سراسری‌اند** (از همه‌جا در دسترس‌اند)، فقط روی `window` نمی‌نشینند. این‌که `var` روی `window` می‌نشیند یکی از مشکلات `var` است (آلوده کردن محدوده‌ی سراسری) که در درس «مشکلات var» می‌بینیم.

در **Node.js** اصلاً `window` نداریم (`typeof window` برابر `"undefined"` است) و شیء سراسری با نام `globalThis` (یا `global`) در دسترس است. سطح بالای یک فایل Node.js هم محدوده‌ی **همان فایل** (module) است، نه محدوده‌ی سراسری واقعی؛ پس آن‌جا حتی `var` سطح بالا هم property شیء سراسری نمی‌شود.

چرا متغیرهای سراسریِ زیاد بد است؟

جمله‌ی پایانی اسلاید ۲۶: «متغیرهای سراسریِ زیاد می‌توانند باعث **تداخل نام‌ها (naming conflicts)** و **باگ** شوند و **نگه‌داری و دیباگِ** کد را سخت‌تر می‌کنند.» چرا؟

- همه‌ی اسکریپت‌های معمولیِ یک صفحه **یک global scope مشترک** دارند. اگر دو فایل (مثلاً کد خودت و یک کتابخانه) هر دو `var count` یا `function init()` داشته باشند، دومی **بی‌صدا** اولی را بازنویسی می‌کند:

```HTML
<script>
  // menu.js
  var count = 0;
  function init() { console.log("menu init"); }
</script>
<script>
  // cart.js (written by someone else)
  var count = 5;
  function init() { console.log("cart init"); }
</script>
<script>
  init();             // cart init (menu's init was silently replaced!)
  console.log(count); // 5
</script>
```

- اگر همان نام‌ها در هر دو فایل با `let` تعریف شده باشند، اسکریپت دوم با `SyntaxError: Identifier 'count' has already been declared` **اصلاً اجرا نمی‌شود**.
- هر تابعی در هر جای برنامه می‌تواند یک متغیر سراسری را عوض کند؛ وقتی مقدارش غلط شد، پیدا کردن مقصر سخت است.
- متغیرهای سراسری تا آخر برنامه در حافظه می‌مانند.

بهترین روش: هر متغیر را در **کوچک‌ترین محدوده‌ای** که لازم دارد تعریف کن (داخل همان تابع یا بلوک).

دام: متغیر سراسریِ ناخواسته (Implicit global)

اگر به نامی که **هیچ‌جا تعریف نشده** مقدار بدهی (بدون `let` ، `const` یا `var`):

- در **حالت عادی (sloppy mode)** (حالت پیش‌فرض و سهل‌گیر JavaScript) بی‌صدا یک متغیر **سراسری** ساخته می‌شود (در مرورگر، `window.total`)، حتی اگر این کار داخل یک تابع باشد!
- در **حالت strict** (با نوشتن `"use strict";` در ابتدای فایل یا تابع؛ حالتی سخت‌گیرتر که جلوی خطاهای بی‌صدا را می‌گیرد) همان خط خطای `ReferenceError` می‌دهد.

```JavaScript
function init() {
  total = 100;          // no let/const/var: an accidental global!
}
init();
console.log(total);     // 100 (sloppy mode)
```

```JavaScript
"use strict";
function init() {
  total = 100;          // ReferenceError: total is not defined
}
init();
```

`typeof` روی متغیری که در دسترس نیست

خواندن مستقیمِ متغیری که در محدوده‌ی فعلی وجود ندارد `ReferenceError` می‌دهد، ولی `typeof` روی نامی که اصلاً پیدا نمی‌شود **خطا نمی‌دهد** و رشته‌ی `"undefined"` برمی‌گرداند:

```JavaScript
{
  const secret = 42;
}
console.log(typeof secret); // undefined (no error)
console.log(secret);        // ReferenceError: secret is not defined
```

(یک استثنا هست: `typeof` روی یک `const` / `let` **در همان محدوده ولی قبل از خطِ تعریفش** خطا می‌دهد. این را در درس «Hoisting و TDZ» می‌بینیم.)

نکته

- بلوک یعنی `{ ... }`. `let` و `const` block-scoped هستند و `var` function-scoped است.
- جست‌وجوی نام‌ها از داخل به بیرون است (scope chain): محدوده‌ی درونی بیرونی را می‌بیند، نه برعکس.
- shadowing: متغیر درونیِ هم‌نام، بیرونی را **می‌پوشاند** ولی **تغییر نمی‌دهد**. بدون `let` داخل بلوک، همان متغیر بیرونی (نزدیک‌ترین متغیر هم‌نام) تغییر می‌کند.
- در مرورگر فقط `var` و اعلان تابعِ سطح بالا property شیء `window` می‌شوند؛ `let` و `const` سراسری‌اند ولی روی `window` نیستند. در Node.js اصلاً `window` نداریم.
- متغیر را در کوچک‌ترین محدوده‌ی لازم تعریف کن تا تداخل نام و باگ کمتر شود.

دام تستی

- خطای «استفاده بیرون از بلوک» از نوع `ReferenceError` است و در زمان اجرا رخ می‌دهد؛ پس خروجی خط‌های قبل از آن چاپ شده است.
- `typeof` روی متغیری که بیرون از بلوکش است `"undefined"` می‌دهد، نه خطا؛ ولی خواندن مستقیمش خطا می‌دهد.
- انتساب به نامِ تعریف‌نشده در حالت sloppy یک global ناخواسته می‌سازد و در حالت strict خطای `ReferenceError` می‌دهد.
- جمله‌ی «همه‌ی متغیرهای سراسری property شیء `window` می‌شوند» دقیق نیست: `let` و `const` نمی‌شوند.
- `let x` داخل یک بلوک تازه، تعریف دوباره نیست و خطا نمی‌دهد (shadowing است). اما اگر `var x` را داخل یک بلوک بنویسی و یکی از محدوده‌های **دربرگیرنده‌ی** آن بلوک (داخل همان تابع، یا سطح بالای برنامه) `let x` داشته باشد، این تعریف دوباره است و `SyntaxError` می‌دهد؛ چون `var` از بلوک بیرون می‌زند.

جمع‌بندی

- scope یعنی محدوده‌ای که یک متغیر در آن قابل دسترسی است. بلوک = `{ ... }`؛ `let` / `const` فقط داخل بلوک خودشان وجود دارند (اسلاید ۲۵).
- global: بیرون از هر بلوک تعریف می‌شود، از همه‌جا در دسترس است و تا پایان برنامه زنده است (اسلاید ۲۶). globalهای زیاد یعنی تداخل نام، باگ و دیباگ سخت.
- توابع محدوده‌ی خودشان را دارند؛ `var` function-scoped است.
- scope chain: جست‌وجو از داخل به بیرون؛ shadowing: نام درونی، بیرونی را می‌پوشاند.
- در مرورگر فقط `var` و تابع‌های سطح بالا روی `window` می‌نشینند؛ انتساب به نام تعریف‌نشده: global ناخواسته (sloppy) یا `ReferenceError` (strict).

تست‌های تمرینی

۲۴ تست · شماره‌ی ۳۰۵ تا ۳۲۸

۳۰۵ آسان

طبق اسلاید ۲۵، **بلوک (block)** در JavaScript چیست؟

۱) هر خطی از کد که با `;` تمام شود.

۲) کدی که داخل آکولاد `{ ... }` قرار دارد.

۳) کدی که داخل پرانتز `( ... )` قرار دارد.

۴) فقط بدنه‌ی یک تابع؛ بدنه‌ی `if` و حلقه‌ها بلوک حساب نمی‌شوند.

۳۰۶ آسان

خروجی کد اسلاید ۲۵ (فایل `scopes.js`) در Node.js چیست؟

```JavaScript
{
  let x = 10;
  console.log(x); // works
}
console.log(x);   // error: x is not defined
```

۱) `10` `10`

۲) `Uncaught ReferenceError`

۳) `10` `Uncaught ReferenceError`

۴) `10` `undefined`

۳۰۷ آسان

جمله‌ی پایانی اسلاید ۲۵ درباره‌ی بلوک‌ها چه می‌گوید؟

۱) بلوک‌ها فقط برای زیبایی و خوانایی کد هستند و روی رفتار متغیرها اثری ندارند.

۲) بلوک‌ها سرعت اجرای کد را بیشتر می‌کنند، چون کدِ داخلشان زودتر اجرا می‌شود.

۳) بلوک‌ها فقط در حلقه‌ها معنا دارند و بیرون از حلقه نمی‌توان از آن‌ها استفاده کرد.

۴) بلوک‌ها فقط یک قاعده‌ی نوشتاری (syntax) نیستند؛ ابزاری برای سازمان‌دهی و محافظت از متغیرها هستند.

۳۰۸ آسان

کدام خط کد زیر خطا می‌دهد؟

```JavaScript
function calculateTotal() {
  const tax = 9;
  console.log("Tax:", tax);
}
calculateTotal();
console.log("Done");
console.log(tax);
```

۱) خط ۳

۲) خط ۵

۳) هیچ خطایی رخ نمی‌دهد

۴) خط ۷

۳۰۹ متوسط

خروجی کد زیر چیست؟

```JavaScript
{
  const token = "abc123";
}
console.log(typeof token);
console.log(token);
```

۱) `undefined` `Uncaught ReferenceError`

۲) `string` `abc123`

۳) `Uncaught ReferenceError`

۴) `undefined` `undefined`

۳۱۰ آسان

خروجی کد اسلاید ۲۶ (فایل `global.js`) چیست؟

```JavaScript
let message = "Hello";  // global

function greet() {
  console.log(message); // accessible here
}

greet();
console.log(message);   // accessible here too
```

۱) `Hello` `Hello`

۲) `undefined` `Hello`

۳) `Uncaught ReferenceError`

۴) `Hello` `Uncaught ReferenceError`

۳۱۱ آسان

طبق اسلاید ۲۶، کدام جمله درباره‌ی متغیرهای **سراسری (global)** درست است؟

۱) فقط داخل تابعی که از آن‌ها استفاده می‌کند قابل دسترسی‌اند.

۲) بیرون از هر بلوکی تعریف می‌شوند، از هر جای کد قابل دسترسی‌اند و تا وقتی برنامه اجرا می‌شود زنده می‌مانند.

۳) بعد از اولین استفاده، به‌طور خودکار از حافظه پاک می‌شوند.

۴) فقط با کلمه‌ی کلیدی `var` می‌توان متغیر سراسری ساخت.

۳۱۲ آسان

جمله‌ی پایانی اسلاید ۲۶ درباره‌ی متغیرهای سراسریِ **زیاد** چه هشداری می‌دهد؟

۱) JavaScript حداکثر ۱۰ متغیر سراسری را مجاز می‌داند و بعد از آن خطا می‌دهد.

۲) متغیرهای سراسری داخل تابع‌ها خوانده نمی‌شوند و باید دوباره تعریف شوند.

۳) متغیرهای سراسری فقط در Node.js کار می‌کنند و در مرورگر خطا می‌دهند.

۴) می‌توانند باعث تداخل نام‌ها (naming conflicts) و باگ شوند و نگه‌داری و دیباگ کد را سخت‌تر می‌کنند.

۳۱۳ متوسط

خروجی کد زیر چیست؟

```JavaScript
let score = 0;
function addPoint() {
  score++;
}
addPoint();
addPoint();
console.log(score);
```

۱) `0`　　۲) `1`

۳) `2`　　۴) `Uncaught ReferenceError`

۳۱۴ متوسط

خروجی کد زیر چیست؟

```JavaScript
const a = "A";
function outer() {
  const b = "B";
  function inner() {
    const c = "C";
    console.log(a + b + c);
  }
  inner();
}
outer();
```

۱) `ABC`　　۲) `C`

۳) `Uncaught ReferenceError`　　۴) `undefinedundefinedC`

۳۱۵ متوسط

کدام خط کد زیر خطا می‌دهد؟

JavaScript

```
1  function outer() {
2    const x = 1;
3    function inner() {
4      const y = 2;
5      console.log(x + y);
6    }
7    inner();
8    console.log(y);
9  }
10 outer();
```

۱) خط ۵
۲) خط ۸
۳) خط ۱۰
۴) هیچ خطایی رخ نمی‌دهد

۳۱۶ متوسط

خروجی کد زیر چیست؟

JavaScript

```
const level = "global";
function outer() {
  const level = "outer";
  function inner() {
    console.log(level);
  }
  inner();
}
outer();
console.log(level);
```

۱) `global` `global`
۲) `outer` `outer`
۳) `Uncaught SyntaxError`
۴) `outer` `global`

۳۱۷ متوسط

خروجی کد زیر چیست؟

```JavaScript
let total = 10;
function reset() {
  let total = 0;
}
function clear() {
  total = 0;
}
reset();
console.log(total);
clear();
console.log(total);
```

۱) 0 0
۲) 10 10
۳) 10 0
۴) 0 10

۳۱۸ سخت

خروجی کد زیر چیست؟

```JavaScript
let x = 1;
{
  let x = 2;
  {
    x = 3;
    console.log(x);
  }
  console.log(x);
}
console.log(x);
```

۱) 3 2 1
۲) 3 3 1
۳) 3 3 3
۴) 3 2 3

۳۱۹ متوسط

در جای خالی کدام گزینه قرار بگیرد تا خروجی کد زیر `inner` و سپس **دوباره** `inner` باشد؟

```JavaScript
let label = "outer";
{
  ___ label = "inner";
  console.log(label);
}
console.log(label);
```

۱) `let`
۲) `const`
۳) `var`
۴) هیچ کلمه‌ای (فقط `label = "inner";`)

۳۲۰ متوسط

خروجی کد زیر چیست؟

```JavaScript
let price = 100;
function applyDiscount(price) {
  price = price - 20;
  return price;
}
console.log(applyDiscount(50));
console.log(price);
```

۱) `30` `100`
۲) `30` `30`
۳) `80` `80`
۴) `80` `100`

۳۲۱ سخت

کدام خط کد زیر خطا می‌دهد؟

```JavaScript
function report() {
  for (var i = 0; i < 2; i++) {
    let doubled = i * 2;
  }
  console.log(i);
  console.log(doubled);
}
report();
console.log(i);
```

۱) خط ۵
۲) خط ۹
۳) خط ۶
۴) هیچ خطایی رخ نمی‌دهد

۳۲۲ سخت

کد زیر در **مرورگر** (یک اسکریپت معمولی) اجرا می‌شود. خروجی کدام است؟

```javascript
let message = "Hello";
var counter = 1;
function greet() {
  console.log(message);
}
greet();
console.log(window.message);
console.log(window.counter);
```

۱) `Hello` `Hello` `1`

۲) `Hello` `undefined` `1`

۳) `Hello` `undefined` `undefined`

۴) `Hello` `Hello` `undefined`

۳۲۳ آسان

در **Node.js** حاصل `typeof window` چیست؟

۱) `"object"`

۲) `"global"`

۳) `"function"`

۴) `"undefined"`

۳۲۴ سخت

در یک صفحه‌ی وب، دو اسکریپت (`menu.js` و `cart.js`) پشت سر هم بارگذاری شده‌اند و بعد از آن‌ها کدِ پایین اجرا می‌شود. خروجی در مرورگر کدام است؟

```html
<script>
  // menu.js
  var count = 0;
  function init() { console.log("menu init"); }
</script>
<script>
  // cart.js
  var count = 5;
  function init() { console.log("cart init"); }
</script>
```

```javascript
init();
console.log(count);
```

۱) `cart init` `5`

۲) `menu init` `0`

۳) `menu init` `cart init` `5`

۴) `cart init` `0`

۳۲۵ سخت

دو اسکریپت زیر پشت سر هم در یک صفحه اجرا می‌شوند و بعد از آن‌ها کدِ پایین (اسکریپت سوم) اجرا می‌شود. خروجی کنسول مرورگر کدام است؟ (سطر `Uncaught ...` یعنی خطایی که در کنسول نمایش داده می‌شود.)

HTML

```
<script>
  let count = 0;
  console.log("menu loaded");
</script>
<script>
  console.log("cart loading");
  let count = 5;
  console.log("cart loaded");
</script>
```

JavaScript

```
console.log(count);
```

۱) `menu loaded` `cart loading` `cart loaded` `5`

۲) `menu loaded` `cart loading` `Uncaught SyntaxError` `0`

۳) `menu loaded` `Uncaught SyntaxError` `0`

۴) `menu loaded` `cart loading` `cart loaded` `0`

۳۲۶ متوسط

خروجی کد زیر در Node.js (حالت عادی، بدون `"use strict"`) چیست؟

JavaScript

```
function setup() {
  counter = 5;
  let local = 10;
}
setup();
console.log(counter);
console.log(typeof local);
```

۱) `5` `number`

۲) `5` `undefined`

۳) `Uncaught ReferenceError`

۴) `undefined` `undefined`

۳۲۷ سخت

کدام خط کد زیر خطا می‌دهد؟

JavaScript

```
1  "use strict";
2  function init() {
3    total = 100;
4  }
5  init();
6  console.log(total);
```

۱) خط ۳
۲) خط ۶
۳) خط ۵
۴) هیچ خطایی رخ نمی‌دهد

۳۲۸ متوسط

کدام گزینه **نادرست** است؟

۱) کد داخل یک تابع می‌تواند متغیرهای محدوده‌های بیرونی (مثلاً متغیرهای سراسری) را بخواند.

۲) متغیری که با `let` داخل یک بلوک تعریف شده، بیرون از آن بلوک در دسترس نیست.

۳) کد بیرون از یک تابع می‌تواند متغیرهای `let` تعریف‌شده داخل آن تابع را بخواند، به شرط این‌که تابع قبلاً صدا زده شده باشد.

۴) اگر بیرون از یک بلوک `let x` داشته باشیم، تعریف `let x` دیگری داخل آن بلوک خطا نمی‌دهد و فقط روی متغیر بیرونی سایه می‌اندازد.

۱-۱۵ توابع: پارامتر، مقدار بازگشتی و توابع درجه‌ی یک

منبع: Introduction to JS، اسلایدهای ۱۷ و ۳۳

درسنامه

فرض کن یک دستگاه آب‌میوه‌گیری داری. میوه را داخلش می‌ریزی (ورودی)، دکمه را می‌زنی و آب‌میوه تحویل می‌گیری (خروجی). هر بار که آب‌میوه بخواهی، دستگاه تازه‌ای نمی‌سازی؛ همان دستگاه را دوباره روشن می‌کنی، شاید با میوه‌ای دیگر. **تابع (function)** در برنامه‌نویسی دقیقاً همین دستگاه است: یک بار ساخته (تعریف) می‌شود و هر بار که لازم شد، با ورودی‌های تازه «روشن» (صدا زده) می‌شود.

تابع چیست؟ (اسلاید ۳۳)

اسلاید ۳۳ پنج جمله‌ی کلیدی درباره‌ی تابع دارد:

- تابع یک **بلوک کدِ قابل‌استفاده‌ی دوباره** (reusable block of code) است.
- کمک می‌کند از **تکرار** کد جلوگیری کنیم و کد **منظم** (organized) بماند.
- می‌تواند **پارامتر** (parameter) یعنی **ورودی** بگیرد.
- می‌تواند یک **مقدار برگرداند** (return a value) یعنی **خروجی** داشته باشد.
- **یک بار تعریف می‌شود و بارها استفاده می‌شود** (Defined once, used many times).

بیا ببینیم بدون تابع چه مشکلی داریم. می‌خواهیم مساحت سه مستطیل را چاپ کنیم:

```JavaScript
// Without a function: the same logic is written three times
console.log("Area: " + 3 * 4);
console.log("Area: " + 5 * 2);
console.log("Area: " + 7 * 7);
```

اگر فردا بخواهیم متن پیام را عوض کنیم، باید هر سه خط را دست بزنیم (و احتمال جا انداختن یکی‌شان کم نیست). با تابع، منطق را **یک جا** می‌نویسیم و **سه بار** صدا می‌زنیم:

```JavaScript
function printArea(width, height) {
  console.log("Area: " + width * height);
}

printArea(3, 4); // Area: 12
printArea(5, 2); // Area: 10
printArea(7, 7); // Area: 49
```

اجزای یک تابع

رایج‌ترین شکل نوشتن تابع، **function declaration** (اعلان تابع با کلمه‌ی کلیدی `function`) است:

JavaScript

```
function add(a, b) {   // keyword + name + parameter list
  return a + b;        // body: "return" sends a value back
}

const sum = add(2, 3); // call: 2 and 3 are the arguments
console.log(sum);      // 5
```

بخش	در این مثال	کارش چیست؟
کلمه‌ی کلیدی	`function`	به JavaScript می‌گوید «دارم یک تابع تعریف می‌کنم».
نام تابع	`add`	بعداً تابع را با همین نام صدا می‌زنیم.
فهرست پارامترها	`(a, b)`	نام ورودی‌ها، جدا شده با کاما؛ می‌تواند خالی باشد: `()`
بدنه (body)	`{ return a + b; }`	کدی که **هر بار** با صدا زدن تابع اجرا می‌شود.
مقدار بازگشتی	`a + b`	چیزی که `return` به جایی که تابع را صدا زده تحویل می‌دهد.

نکته‌ی مهم: **تعریف تابع آن را اجرا نمی‌کند.** بدنه فقط وقتی اجرا می‌شود که تابع را **صدا بزنیم** (call یا invoke، یعنی فراخوانی): نام تابع و بعد پرانتز، مثل `add(2, 3)`. روش کوتاه‌ترِ ES6 یعنی arrow function را در درس بعد («Function declaration در برابر Arrow function») می‌بینیم.

میم اسلاید ۳۳: تابع در زبان‌های مختلف

کنار اسلاید ۳۳ یک میم معروف از «پو خرسه»ی کت‌وشلواری هست. هر ردیف یک زبان را با کلمه‌ای که برای تعریف تابع به کار می‌برد نشان می‌دهد:

زبان	کلمه‌ی تعریف تابع	پو در میم
JavaScript	`function`	پوی معمولی با پیراهن قرمز
Go (Golang)	`func`	پوی کت‌وشلواری
Kotlin	`fun`	کت‌وشلوار و یک زینت گرد
Rust	`fn`	شیک‌ترین: کت‌وشلوار، عینک تک‌چشمی و سبیل
Python	`def`	پو با قیافه‌ی درهم و دندان‌های بیرون‌زده
Java	کلمه‌ی خاصی ندارد؛ امضای کامل متد را می‌نویسد	زشت‌ترین و گیج‌ترین پو

شوخی میم این است که هر چه کلمه‌ی کلیدی **کوتاه‌تر** باشد، زبان «شیک‌تر» به نظر می‌رسد: از `function` به `func`، بعد `fun` و در نهایت `fn`. کلمه‌ی `def` در Python و امضای طولانی متد در Java هم دست انداخته شده‌اند.

در ردیف آخر، یک متد Java برچسب‌گذاری شده است:

```
public int max(int x, int y) {
    if (x > y)
        return x;
    else
        return y;
}
```

برچسب در میم	بخش در متد Java	معادل در JavaScript
modifier (سطح دسترسی)	`public`	ندارد
return-type (نوع مقدار بازگشتی)	`int` اول (قبل از `max`)	ندارد؛ نوع خروجی نوشته نمی‌شود
method-name (نام متد)	`max`	نام تابع
parameter-list (فهرست پارامترها)	`(int x, int y)`	`(x, y)`، بدون نوع
body of the method (بدنه‌ی متد)	`{ ... }`	`{ ... }`

همین تابع در JavaScript این‌طور نوشته می‌شود (بدون modifier و بدون هیچ نوعی):

JavaScript

```
function max(x, y) {
  if (x > y) return x;
  else return y;
}

console.log(max(4, 9)); // 9
```

> **یادداشت**
>
> در JavaScript فقط `function` (و در درس بعد، `=>`) تابع می‌سازد. نوشتن `def`، `func`، `fun` یا `fn` به‌جای `function` در JavaScript خطای `SyntaxError` (خطای نحوی، یعنی «این نگارش را نمی‌فهمم») می‌دهد.

پارامتر در برابر آرگومان

- **parameter** (پارامتر): **نامی** که هنگام **تعریف** تابع داخل پرانتز می‌نویسیم؛ مثل یک جعبه‌ی خالیِ برچسب‌دار.
- **argument** (آرگومان): **مقدار واقعی** که هنگام **صدا زدن** تابع می‌فرستیم و آن جعبه را پر می‌کند.

در `function add(a, b)`، `a` و `b` پارامتر هستند و در `add(2, 3)`، `2` و `3` آرگومان. آرگومان‌ها **به ترتیب** در پارامترها ریخته می‌شوند: `a` می‌شود `2` و `b` می‌شود `3`. پارامترها مثل متغیرهایی هستند که فقط داخل همان تابع وجود دارند.

`return` : خروجی تابع

۱. مقداری که `return` می‌کنیم به **همان جایی که تابع صدا زده شده** برمی‌گردد؛ می‌توانیم آن را در متغیر بریزیم یا در یک عبارت به کار ببریم: `add(2, 3) * 10` برابر `50` است.

۲. `return` تابع را **همان لحظه تمام می‌کند**؛ خط‌های بعد از آن اجرا نمی‌شوند:

JavaScript

```
function checkAge(age) {
  if (age < 18) {
    return "Too young";
    console.log("never printed"); // unreachable: return already ended the function
  }
  return "Welcome";
}

console.log(checkAge(15)); // Too young
console.log(checkAge(20)); // Welcome
```

JavaScript برای کدِ «غیرقابل‌دسترس» بعد از `return` خطا نمی‌دهد (برخلاف Java که چنین کدی را کامپایل نمی‌کند)؛ آن خط فقط هیچ‌وقت اجرا نمی‌شود.

۳. تابعی که `return` ندارد، یا فقط یک `return;` خالی دارد، مقدار `undefined` برمی‌گرداند:

```JavaScript
function sayHi() {
  console.log("Hi!");   // prints something, but returns nothing
}
function stop() {
  return;               // bare return: ends the function, the value is undefined
}

const r = sayHi();      // Hi!
console.log(r);         // undefined
console.log(stop());    // undefined
```

چاپ کردن با برگرداندن فرق دارد

`console.log` مقدار را فقط **روی صفحه نشان می‌دهد** و برنامه چیزی تحویل نمی‌گیرد. `return` مقدار را **به برنامه تحویل می‌دهد** تا ذخیره یا دوباره استفاده شود:

```JavaScript
function doubleAndPrint(x) {
  console.log(x * 2);   // shows 10, returns undefined
}
function doubleAndReturn(x) {
  return x * 2;         // prints nothing, hands 10 back
}

const a = doubleAndPrint(5);  // 10
const b = doubleAndReturn(5); // (nothing is printed here)
console.log(a, b);            // undefined 10
```

آرگومان کمتر یا بیشتر: JavaScript خطا نمی‌دهد!

```JavaScript
function add(a, b) {
  return a + b;
}

console.log(add(2));       // NaN (b is undefined, and 2 + undefined is NaN)
console.log(add(2, 3, 4)); // 5   (the extra argument 4 is ignored)
```

- پارامتری که آرگومانی برایش فرستاده نشده، مقدار `undefined` می‌گیرد.
- آرگومان اضافه **بی‌صدا نادیده گرفته می‌شود**.
- در هیچ‌کدام از این دو حالت خطایی رخ نمی‌دهد (Java در هر دو حالت خطای کامپایل می‌داد).

Default parameters (مقدار پیش‌فرض پارامتر)

از ES6 می‌توانیم برای یک پارامتر مقدار پیش‌فرض بگذاریم تا اگر آرگومانی برایش نیامد، `undefined` نماند:

```JavaScript
function greet(name = "Guest") {
  return "Hello, " + name + "!";
}

console.log(greet("Sara"));    // Hello, Sara!
console.log(greet());          // Hello, Guest!
console.log(greet(undefined)); // Hello, Guest!
console.log(greet(null));      // Hello, null!
```

قانون: مقدار پیش‌فرض **فقط وقتی** استفاده می‌شود که آرگومانِ آن جایگاه `undefined` باشد (یعنی فرستاده نشده باشد یا خودمان `undefined` فرستاده باشیم). `null` ، `0` و `""` مقدار واقعی حساب می‌شوند و مقدار پیش‌فرض را کنار می‌زنند.

Function expression (تابع به‌عنوان یک مقدار)

تابع را می‌شود بدون نام هم نوشت و مثل هر مقدار دیگری در یک متغیر ریخت. به این شکل **function expression** (عبارت تابعی) می‌گویند:

```JavaScript
const square = function (n) {  // an anonymous function stored in a variable
  return n * n;
};

console.log(square(6)); // 36
```

این تابع را با نام متغیرش (`square`) صدا می‌زنیم. تفاوت مهم آن با function declaration در **hoisting** (اینکه آیا می‌شود تابع را قبل از خط تعریفش صدا زد) است که در درس بعد و درس «Hoisting و TDZ» می‌بینیم. arrow function هم در واقع شکل کوتاه‌تری از همین function expression است.

توابع first-class هستند (اسلاید ۱۷)

اسلاید ۱۷ می‌گوید:

- توابع **first-class object** هستند (شهروند درجه‌یک: می‌توان آن‌ها را **پاس داد، ذخیره کرد و برگرداند**).
- توابع می‌توانند **متد یک object** باشند یا **بلوک کد مستقل** (standalone).

«first-class» یعنی تابع در JavaScript یک **مقدار** است، درست مثل عدد و رشته. هر کاری با یک عدد می‌کنی، با یک تابع هم می‌توانی بکنی:

۱) ذخیره در متغیر، آرایه یا object:

```JavaScript
function hello() {
  return "Hello!";
}

const greetFn = hello;          // store the function itself (no parentheses!)
const tools = [hello];          // store it in an array
console.log(greetFn());         // Hello!
console.log(tools[0]());        // Hello!
console.log(greetFn === hello); // true (the same function, not a copy)
```

۲) پاس دادن به‌عنوان آرگومان (callback):

```JavaScript
function repeat(times, action) {  // action is a function (a callback)
  for (let i = 1; i <= times; i++) {
    action(i);                    // repeat decides when to call it
  }
}
function showStep(n) {
  console.log("Step " + n);
}

repeat(3, showStep);
// Step 1
// Step 2
// Step 3
```

۳) برگرداندن از یک تابع دیگر:

```JavaScript
function makeMultiplier(k) {
  return function (x) {   // a brand-new function is returned
    return x * k;
  };
}

const double = makeMultiplier(2);
const triple = makeMultiplier(3);
console.log(double(5));             // 10
console.log(triple(5));             // 15
console.log(makeMultiplier(10)(4)); // 40 (call the returned function right away)
```

تابعِ برگشتی هنوز `k` را «به یاد دارد»، حتی بعد از اینکه اجرای `makeMultiplier` تمام شده است. اسم این پدیده **closure** (بستار) است که در درس «Closure» مفصل می‌خوانیم.

۴) متد یک object یا تابع مستقل:

```JavaScript
const calculator = {
  brand: "Casio",
  add: function (a, b) {  // a function stored in an object = a method
    return a + b;
  },
};
function subtract(a, b) { // a standalone function
  return a - b;
}

console.log(calculator.add(2, 3));  // 5
console.log(subtract(9, 4));        // 5
console.log(typeof calculator.add); // function
```

تابع در JavaScript نوعی object است، ولی `typeof` برای آن رشته‌ی `"function"` را برمی‌گرداند (نه `"object"`).

Callback و Higher-order function

- **callback** (تابع بازخوان): تابعی که به تابع دیگری **پاس داده می‌شود** تا آن تابع، هر وقت لازم بود، **صدایش بزند** («بعداً خبرم کن!»). در مثال بالا `showStep` یک callback است.
- **higher-order function** (تابع مرتبه‌بالا): تابعی که یک تابع را **به‌عنوان آرگومان می‌گیرد** یا یک تابع را **برمی‌گرداند**. `repeat` و `makeMultiplier` تابع مرتبه‌بالا هستند.

callback لزوماً «بعداً» اجرا نمی‌شود؛ دقیقاً همان لحظه‌ای اجرا می‌شود که تابع گیرنده صدایش بزند. در `repeat`، هر بار که به خط `action(i)` می‌رسیم، callback همان‌جا و قبل از ادامه‌ی حلقه اجرا می‌شود. خود JavaScript پر از توابع مرتبه‌بالاست: `setTimeout(callback, ms)` (درس‌های برنامه‌نویسی ناهمگام)، متدهای آرایه مثل `map` و `forEach` (درس «متدهای آرایه») و `addEventListener` در فصل دوم.

`f` یا `f()`؟ ارجاع به تابع در برابر صدا زدن آن

	`hello`	`hello()`
یعنی چه؟	خودِ تابع (اشاره به دستگاه آب‌میوه‌گیری)	اجرای تابع و گرفتن مقدار بازگشتی‌اش (روشن کردن دستگاه)
`typeof` (برای `hello` بالا)	`"function"`	نوع خروجی؛ اینجا `"string"`
کاربرد	ذخیره در متغیر، پاس دادن به‌عنوان callback	گرفتن نتیجه

اشتباه رایج: پاس دادن `f()` به‌جای `f`. با پرانتز، تابع **همین الان** اجرا می‌شود و به‌جای خودِ تابع، **نتیجه‌اش** پاس داده می‌شود:

```JavaScript
function repeat(times, action) {
  for (let i = 1; i <= times; i++) action(i);
}
function hi() {
  console.log("hi");
}

repeat(2, hi);   // hi, hi (correct: repeat calls hi twice)
repeat(2, hi()); // hi, then TypeError: action is not a function
```

در خط آخر، `hi()` اول اجرا می‌شود (یک `hi` چاپ می‌شود) و چون چیزی برنمی‌گرداند، `repeat` به‌جای تابع مقدار `undefined` می‌گیرد و صدا زدن `undefined` خطای `TypeError` (خطای نوع) می‌دهد.

> **نکته**
> - **پارامتر** = نام در تعریف؛ **آرگومان** = مقدار در فراخوانی.
> - تابع بدون `return` (یا با `return;` خالی) مقدار `undefined` برمی‌گرداند و `return` تابع را همان لحظه تمام می‌کند.
> - تابع یک مقدار است: می‌شود ذخیره‌اش کرد، پاسش داد و برش گرداند. `typeof` آن `"function"` است و ریختنش در متغیر دیگر **کپی** نمی‌سازد (`greetFn === hello` برابر `true` است).
> - تابعی که تابع می‌گیرد یا برمی‌گرداند **higher-order** است و تابعی که پاس داده می‌شود **callback**.

⚠ دام تستی

- چاپ کردن با `console.log` داخل تابع، `return` نیست. اگر تابع فقط چاپ کند، `console.log(f())` اول چاپِ داخل تابع و بعد `undefined` را نشان می‌دهد.
- `f` خودِ تابع است و `f()` نتیجه‌ی اجرای آن. `const x = f;` تابع را اجرا **نمی‌کند**.
- آرگومان جاافتاده `undefined` می‌شود (و در محاسبه `NaN` می‌سازد) و آرگومان اضافه بی‌صدا نادیده گرفته می‌شود. هیچ‌کدام خطا نیست.
- مقدار پیش‌فرض فقط جای `undefined` را می‌گیرد؛ با `null` یا `0` استفاده نمی‌شود.
- کلمه‌هایی مثل `def`، `func` و `fn` در JavaScript وجود ندارند (`SyntaxError`).
- مقدار `return` را در **همان خط** بنویس. اگر بلافاصله بعد از `return` به خط بعد بروی، JavaScript خودش آنجا یک `;` می‌گذارد (automatic semicolon insertion، یعنی درج خودکار نقطه‌ویرگول) و تابع `undefined` برمی‌گرداند:

```javascript
function getScore() {
  return      // JavaScript reads this line as "return;"
    100;      // never reached
}
console.log(getScore()); // undefined
```

✓ جمع‌بندی

- تابع = بلوک کد قابل‌استفاده‌ی دوباره: یک بار تعریف، بارها صدا زدن با `name(args)`.
- `return` مقدار را برمی‌گرداند و تابع را تمام می‌کند؛ بدون آن، نتیجه `undefined` است.
- آرگومان جاافتاده `undefined` است و آرگومان اضافه نادیده گرفته می‌شود؛ مقدار پیش‌فرض فقط به‌جای `undefined` می‌نشیند.
- توابع first-class هستند: ذخیره، پاس دادن (callback) و برگرداندن (higher-order)؛ می‌توانند متد object یا مستقل باشند.
- کلمه‌ی تعریف تابع: Python `def`، Rust `fn`، Kotlin `fun`، Go `func`، JavaScript `function`؛ Java امضای کامل متد (modifier، return-type، نام، پارامترها و بدنه) را دارد.

✓ تست‌های تمرینی

۲۴ تست · شماره‌ی ۳۲۹ تا ۳۵۲

۳۲۹ آسان

طبق اسلاید ۳۳، کدام جمله درباره‌ی توابع **نادرست** است؟

۱) تابع یک بلوک کدِ قابل‌استفاده‌ی دوباره (reusable) است.

۲) تابع می‌تواند پارامتر (ورودی) بگیرد و یک مقدار (خروجی) برگرداند.

۳) تابع به جلوگیری از تکرار و منظم ماندن کد کمک می‌کند.

۴) برای هر بار استفاده از تابع، باید آن را دوباره تعریف کرد.

۳۳۰ آسان

در میم اسلاید ۳۳، هر زبان کنار کلمه‌ی کلیدی‌ای آمده که با آن تابع تعریف می‌کند. کدام جفت درست است؟

۱) `fn` — Go
۲) `fun` — Rust
۳) `def` — Python
۴) `func` — Kotlin

۳۳۱ آسان

کدام خط در JavaScript یک تابع را **درست** تعریف می‌کند و بدون خطا اجرا می‌شود؟

۱) `def greet() { return "hi"; }`
۲) `function greet() { return "hi"; }`
۳) `func greet() { return "hi"; }`
۴) `fn greet() { return "hi"; }`

۳۳۲ آسان

در تعریف `function add(a, b) { return a + b; }` و فراخوانی `add(2, 3)`، کدام گزینه درست است؟

۱) `a` و `b` آرگومان هستند و `2` و `3` پارامتر.
۲) `a` و `b` پارامتر هستند و `2` و `3` آرگومان.
۳) هر چهار مورد پارامترند؛ آرگومان یعنی مقدار بازگشتی تابع.
۴) `add` پارامتر است و `a`، `b`، `2` و `3` همگی آرگومان‌اند.

۳۳۳ آسان

خروجی کد زیر چیست؟

```javascript
function greet(name) {
  return "Hello, " + name + "!";
}
console.log(greet("Sara"));
console.log(greet("Ali"));
```

۱) `Hello, Sara!` `Hello, Ali!`
۲) `Hello, name!` `Hello, name!`
۳) `Hello, Sara!` `Hello, Sara!`
۴) `Hello, Sara!` `Uncaught TypeError`

۳۳۴ آسان

خروجی کد زیر چیست؟

```javascript
function square(x) {
  x * x;
}
console.log(square(4));
```

۱) `16`
۲) `NaN`
۳) `Uncaught SyntaxError`
۴) `undefined`

۳۳۵ آسان

فرض کن `hello` تابعی است که رشته‌ی `"hi"` را برمی‌گرداند. تفاوت `hello` و `hello()` چیست؟

۱) هر دو تابع را اجرا می‌کنند؛ پرانتز اختیاری است.

۲) `hello` تابع را اجرا می‌کند و `hello()` یک کپی از آن می‌سازد.

۳) `hello` به خودِ تابع اشاره می‌کند (بدون اجرا) و `hello()` تابع را اجرا می‌کند و مقدار بازگشتی‌اش را می‌دهد.

۴) `hello()` یک تابع جدید به نام `hello` تعریف می‌کند.

۳۳۶ آسان

به تابعی که یک تابع دیگر را **به‌عنوان آرگومان می‌گیرد** یا یک تابع را **برمی‌گرداند**، چه می‌گویند؟

۱) higher-order function

۲) callback

۳) method

۴) default parameter

۳۳۷ متوسط

در میم اسلاید ۳۳، بخش‌های متد Java زیر برچسب‌گذاری شده‌اند. کدام گزینه درست است؟

`public int max(int x, int y) { ... }`

۱) `public` همان return-type (نوع مقدار بازگشتی) است و در JavaScript هم نوشته می‌شود.

۲) `max` همان modifier است.

۳) `int` اول (قبل از `max`) نوع مقدار بازگشتی است و در تعریف تابع JavaScript نوع بازگشتی نوشته نمی‌شود.

۴) در JavaScript هم مثل Java نوع پارامترها نوشته می‌شود: `function max(int x, int y)` .

۳۳۸ متوسط

خروجی کد زیر چیست؟

```JavaScript
function check(n) {
  if (n > 0) {
    return "positive";
    console.log("after return");
  }
  return "not positive";
}
console.log(check(5));
console.log(check(-2));
```

۱) `after return` `positive` `not positive`

۲) `positive` `not positive`

۳) `not positive` `not positive`

۴) `Uncaught SyntaxError`

۳۳۹ متوسط

خروجی کد زیر چیست؟

```javascript
function double(x) {
  console.log(x * 2);
}
const result = double(5);
console.log(result);
```

۱) `10` `10`
۲) `10`
۳) `undefined` `10`
۴) `10` `undefined`

۳۴۰ متوسط

در جای خالی بدنه‌ی تابع زیر کدام گزینه قرار بگیرد تا خروجی برنامه **فقط** `12` باشد؟

```javascript
function area(w, h) {
  ------
}
const result = area(3, 4);
console.log(result);
```

۱) `console.log(w * h);`
۲) `let result = w * h;`
۳) `w * h;`
۴) `return w * h;`

۳۴۱ متوسط

خروجی کد زیر چیست؟

```javascript
function add(a, b) {
  return a + b;
}
console.log(add(2));
console.log(add(2, 3, 4));
```

۱) `2undefined` `5`
۲) `2` `9`
۳) `NaN` `5`
۴) `Uncaught TypeError`

۳۴۲ متوسط

کدام جمله درباره‌ی پارامترها و آرگومان‌ها **درست** است؟

۱) اگر تابعی با آرگومان‌های کمتر از تعداد پارامترهایش صدا زده شود، JavaScript خطا می‌دهد.
۲) پارامتری که آرگومانی برایش فرستاده نشده (و مقدار پیش‌فرض ندارد)، مقدار `undefined` می‌گیرد.
۳) فرستادن آرگومان اضافه خطای `TypeError` می‌دهد.
۴) هر تابع باید دست‌کم یک پارامتر داشته باشد.

۳۴۳ متوسط

خروجی کد زیر چیست؟

```JavaScript
function greet(name = "Guest") {
  return "Hi " + name;
}
console.log(greet());
console.log(greet("Reza"));
console.log(greet(undefined));
```

۱) `Hi Guest` `Hi Reza` `Hi Guest`

۲) `Hi Guest` `Hi Reza` `Hi undefined`

۳) `Hi undefined` `Hi Reza` `Hi undefined`

۴) `Hi Guest` `Hi Guest` `Hi Guest`

۳۴۴ متوسط

خروجی کد زیر چیست؟

```JavaScript
function add(a, b) {
  return a + b;
}
function sub(a, b) {
  return a - b;
}
const ops = { plus: add, minus: sub };
console.log(ops.minus(10, 4));
console.log(ops.plus === add);
```

۱) `14` `true`

۲) `6` `true`

۳) `6` `false`

۴) `Uncaught TypeError`

۳۴۵ متوسط

خروجی کد زیر چیست؟

```javascript
const calculator = {
  brand: "Casio",
  add: function (a, b) {
    return a + b;
  },
};
const plus = calculator.add;
console.log(calculator.add(2, 3));
console.log(plus(10, 20));
console.log(typeof calculator.add);
```

۱) `5` `30` `function`

۲) `5` `Uncaught TypeError`

۳) `5` `30` `object`

۴) `5` `undefined` `function`

۳۴۶ سخت

خروجی کد زیر چیست؟

```javascript
function processOrder(item, onDone) {
  console.log("Preparing " + item);
  onDone(item);
  console.log("Finished");
}
processOrder("pizza", function (x) {
  console.log(x + " is ready");
});
```

۱) `Preparing pizza` `Finished` `pizza is ready`

۲) `pizza is ready` `Preparing pizza` `Finished`

۳) `Preparing pizza` `pizza is ready` `Finished`

۴) `Preparing pizza` `Finished`

۳۴۷ سخت

خروجی کد زیر چیست؟

```JavaScript
function price(amount, tax = 10) {
  return amount + tax;
}
console.log(price(100));
console.log(price(100, 0));
console.log(price(100, null));
console.log(price(100, undefined));
```

۱) `110` `110` `110` `110`
۲) `110` `100` `110` `110`
۳) `110` `100` `100` `NaN`
۴) `110` `100` `100` `110`

۳۴۸ سخت

خروجی کد زیر چیست؟

```JavaScript
function makeMultiplier(k) {
  return function (x) {
    return x * k;
  };
}
const triple = makeMultiplier(3);
console.log(triple(5));
console.log(makeMultiplier(2)(7));
```

۱) `15` `14`
۲) `NaN` `NaN`
۳) `15` `Uncaught TypeError`
۴) `Uncaught SyntaxError`

۳۴۹ سخت

کدام خط کد زیر خطا می‌دهد؟

```JavaScript
1 function getName() {
2   return "Mina";
3 }
4 const userName = getName();
5 console.log(userName);
6 console.log(userName());
```

۱) خط ۴
۲) خط ۵
۳) خط ۶
۴) هیچ خطایی رخ نمی‌دهد

۳۵۰ سخت

خروجی کد زیر چیست؟

```javascript
function run(task) {
  console.log("running...");
  return task();
}
function hello() {
  console.log("hello");
  return "done";
}
console.log(run(hello));
console.log(run(hello()));
```

۱) `running...` `hello` `done` `running...` `hello` `done`

۲) `running...` `hello` `done` `running...` `hello` `Uncaught TypeError`

۳) `running...` `hello` `done` `hello` `running...` `done`

۴) `running...` `hello` `done` `hello` `running...` `Uncaught TypeError`

۳۵۱ سخت

خروجی کد زیر چیست؟

```javascript
let calls = 0;
function hello() {
  calls++;
  return "hi";
}
const a = hello;
const b = hello();
const c = a();
console.log(calls);
```

۱) `1` ۲) `2` ۳) `3` ۴) `0`

۳۵۲ سخت

خروجی کد زیر چیست؟

```javascript
function getScore() {
  return
    100;
}
console.log(getScore());
```

۱) `undefined` ۲) `100`

۳) `Uncaught SyntaxError` ۴) `null`

۱-۱۶ Function declaration در برابر Arrow function

منبع: Introduction to JS، اسلایدهای ۳۴ تا ۳۶

درسنامه

در درس قبل تابع را با کلمه‌ی کلیدی `function` تعریف کردیم. اسلایدهای ۳۴ تا ۳۶ دو روش اصلی تعریف تابع را کنار هم می‌گذارند: **روش اول: function declaration** (اعلان تابع) و **روش دوم: arrow function** (تابع پیکانی، با علامت `=>`). مثل نوشتن یک پیام به دو شکل است: یک نامه‌ی رسمی کامل و یک پیامک کوتاه. هر دو پیام را می‌رسانند، ولی قواعدشان کمی فرق دارد و همین فرق‌ها سؤال‌های امتحانی را می‌سازند.

روش اول: Function declaration (اسلاید ۳۴)

- روش **کلاسیک** تعریف تابع است.
- با کلمه‌ی کلیدی `function` شروع می‌شود.
- می‌توان آن را **قبل یا بعد از** خط تعریفش صدا زد، چون **hoisted** است.
- زیرنویس اسلاید: «Remember, this can be hoisted!» (یادت باشد این یکی hoist می‌شود!)

fn.js

```
function add(a, b) {
  return a + b;
}

console.log(add(2, 3)); // 5
```

صدا زدن **قبل از** خط تعریف هم بی‌دردسر کار می‌کند:

JavaScript

```
console.log(add(2, 3)); // 5 (works: function declarations are hoisted)

function add(a, b) {
  return a + b;
}
```

hoisting (بالا بردن اعلان‌ها به ابتدای محدوده) یعنی موتور JavaScript قبل از اجرای خط‌به‌خط، کل کد را یک دور مرور می‌کند و هر function declaration را **کامل** (نام و بدنه) از همان ابتدای محدوده‌اش آماده می‌کند. نتیجه این است که کد طوری اجرا می‌شود که انگار تابع بالای فایل نوشته شده است. قوانین کامل hoisting برای `var`، `let`، `const` و `class` را در درس «Hoisting و TDZ» می‌بینیم؛ اینجا فقط تفاوت دو روش تعریف تابع مهم است.

روش دوم: Arrow function در ES6 (اسلاید ۳۵)

- در **ES6** (یعنی JavaScript مدرن؛ نام دیگرش ES2015) معرفی شد.
- نحو (syntax) **کوتاه‌تری** دارد و اغلب برای **توابع کوچک و سریع** به کار می‌رود.
- **hoist نمی‌شود**: باید **قبل از استفاده** تعریف شود.
- زیرنویس اسلاید: «Remember, this **cannot** be hoisted!» (یادت باشد این یکی hoist **نمی‌شود**!)

fn.js

```
const multiply = (a, b) => {
  return a * b;
};

console.log(multiply(4, 5)); // 20
```

اینجا خبری از کلمه‌ی `function` نیست: اول فهرست پارامترها `(a, b)`، بعد علامت `=>` (پیکان، که به آن «fat arrow» هم می‌گویند) و بعد بدنه می‌آید. خودِ تابع اسمی ندارد و در متغیر `multiply` ذخیره شده است؛ یعنی arrow function نوعی function expression است (درس قبل) و با نام متغیرش صدا زده می‌شود.

حالا اگر arrow function را **قبل از** تعریفش صدا بزنیم چه می‌شود؟

JavaScript

```
console.log(multiply(4, 5)); // ReferenceError: Cannot access 'multiply' before initialization

const multiply = (a, b) => {
  return a * b;
};
```

و اگر همان arrow function را در یک `var` ریخته باشیم:

JavaScript

```
console.log(multiply(4, 5)); // TypeError: multiply is not a function

var multiply = (a, b) => a * b;
```

چرا دو خطای متفاوت؟ در هر دو حالت **خودِ تابع** فقط وقتی ساخته می‌شود که اجرا به خط `=>` برسد؛ قبل از آن فقط **نام متغیر** وجود دارد:

- متغیر `const` (و `let`) تا رسیدن به خط تعریفش «مقداردهی‌نشده» است و هر دسترسی به آن `ReferenceError` (خطای ارجاع) می‌دهد. به این بازه **TDZ** (Temporal Dead Zone، یعنی «منطقه‌ی مرده‌ی زمانی») می‌گویند.
- متغیر `var` از همان ابتدا با مقدار `undefined` وجود دارد؛ پس `multiply(4, 5)` یعنی صدا زدن `undefined` که `TypeError: multiply is not a function` (خطای نوع) می‌دهد.

همین قاعده برای function expression معمولی (`const f = function () {};`) هم برقرار است؛ hoisting کامل فقط مال **function declaration** است. جزئیات بیشتر در درس «Hoisting و TDZ».

شکل‌های مختلف نوشتن arrow function

حالت	مثال	قاعده
بدنه‌ی بلوکی (block body)	`(a, b) => { return a * b; }`	`return` صریح لازم است.
بدنه‌ی مختصر (concise body)	`(a, b) => a * b`	**return ضمنی** (implicit return): مقدار عبارت خودکار برمی‌گردد.
یک پارامتر	`x => x * 2`	پرانتز اختیاری است.
بدون پارامتر	`() => "Hi!"`	پرانتز خالی اجباری است.
برگرداندن object	`() => ({ a: 1 })`	object باید داخل پرانتز باشد.

JavaScript

```
const square = x => x * x;             // one parameter: parentheses are optional
const sayHi = () => "Hi!";             // no parameters: empty () is required
const add = (a, b) => a + b;           // concise body: the value is returned automatically
const addBlock = (a, b) => { a + b; }; // block body without return: returns undefined

console.log(square(4));      // 16
console.log(sayHi());        // Hi!
console.log(add(2, 3));      // 5
console.log(addBlock(2, 3)); // undefined
```

دو یا چند پارامتر **همیشه** پرانتز می‌خواهند: `const f = a, b => a + b;` خطای `SyntaxError` (خطای نحوی) می‌دهد و شکل درست `const f = (a, b) => a + b;` است.

دام برگرداندن object

JavaScript

```
const makeUser = () => { name: "Ali" };    // { } is read as a block, not as an object!
const makeUser2 = () => ({ name: "Ali" }); // ( ) makes it an object literal

console.log(makeUser());  // undefined
console.log(makeUser2()); // { name: 'Ali' }
```

در اولی، JavaScript آکولاد `{` بعد از `=>` را شروع **بدنه‌ی بلوکی** می‌داند، نه object. داخل این بلوک، `name:` یک «برچسب» (label، یک ویژگی کم‌کاربرد زبان) حساب می‌شود و `"Ali"` یک عبارت بی‌اثر است. چون `return` ی در کار نیست، نتیجه `undefined` است. پرانتز دور object به JavaScript می‌فهماند که این یک **مقدار** است.

arrow function برای callbackهای کوتاه

جای درخشش callback، arrow functionهای کوتاه است (callback یعنی تابعی که به تابع دیگر پاس داده می‌شود):

JavaScript

```
function applyToFive(fn) {
  return fn(5);
}

console.log(applyToFive(x => x * 10));                    // 50
console.log(applyToFive(function (x) { return x * 10; })); // 50 (same thing, longer)
```

مقایسه‌ی روش اول و دوم (اسلاید ۳۶)

- Function: با کلمه‌ی کلیدی `function` تعریف می‌شود.
- Arrow: نحو **کوتاه‌تر** با `=>` .
- Functionها **hoist می‌شوند** (قبل از تعریف قابل‌استفاده‌اند).
- Arrowها **hoist نمی‌شوند**.
- Functionها `this` **مخصوص خودشان** را دارند؛ arrowها `this` را **به ارث می‌برند** (بعداً بیشتر می‌گوییم!).

ویژگی	Function declaration	Arrow function
نحو	`function add(a, b) { ... }`	`const add = (a, b) => ...`
از چه زمانی؟	از اولین نسخه‌ی زبان	ES6 (ES2015)
صدا زدن قبل از خط تعریف	مجاز (hoisted)	خطا (hoist نمی‌شود)
return ضمنی	ندارد؛ همیشه `return` لازم است	در بدنه‌ی مختصر دارد
`this`	مخصوص خودش؛ به نحوه‌ی صدا زدن بستگی دارد	از کد بیرونی به ارث می‌برد
شیء `arguments`	دارد	ندارد
استفاده با `new`	مجاز	`TypeError`

`this` در یک نگاه

`this` (به معنی «این») کلمه‌ی ویژه‌ای است که داخل یک تابع به «objectی که تابع برایش کار می‌کند» اشاره می‌کند. در یک تابع معمولی که به شکل `user.regular()` صدا زده شود، `this` همانِ object قبل از نقطه یعنی `user` است. arrow function اما `this` مخصوص خودش را ندارد و آن را از کدِ اطرافش (جایی که نوشته شده) قرض می‌گیرد:

JavaScript

```
const user = {
  username: "Sara",
  regular: function () {
    return this.username;     // own this: the object before the dot (user)
  },
  arrow: () => this.username, // inherited this: from the code around the object
};

console.log(user.regular()); // Sara
console.log(user.arrow());   // undefined
```

arrow در سطح بالای فایل نوشته شده است (آکولادهای object محدوده‌ی جدیدی برای `this` نمی‌سازند)، پس `this` آن همان `this` سطح بالاست، نه `user` : در یک فایل معمولی Node.js (CommonJS) یک object خالی (`{}`) و در اسکریپت معمولی مرورگر شیء `window` . هیچ‌کدام `username` ندارند و نتیجه `undefined` است. قاعده‌ی عملی: **برای متدی که با `this` کار می‌کند از arrow function استفاده نکن.** همه‌ی قوانین `this` را در درس «کلمه‌ی کلیدی this» می‌خوانیم.

شیء `arguments`

هر تابع معمولی یک شیء پنهان به نام `arguments` دارد که **همه‌ی** آرگومان‌های فرستاده‌شده (حتی آرگومان‌های اضافه) را مثل یک آرایه نگه می‌دارد. arrow function این شیء را ندارد:

```JavaScript
function countArgs() {
  return arguments.length; // how many arguments were passed?
}
console.log(countArgs(1, 2, 3)); // 3

const arrowCount = () => arguments.length;
// In a browser: arrowCount(1, 2, 3) -> ReferenceError: arguments is not defined
```

arrow function، درست مثل `this` ، `arguments` تابع معمولیِ دربرگیرنده‌اش را می‌بیند:

```JavaScript
function outer() {
  const inner = () => arguments[0]; // the arguments of outer, not of inner
  return inner("x");
}
console.log(outer("a")); // a
```

> **بیشتر بدانیم**
>
> Node.js هر فایل CommonJS (فایل `.js` معمولی) را پشت پرده داخل یک تابع می‌پیچد. برای همین اگر در سطح بالای یک فایل Node.js داخل arrow function از `arguments` استفاده کنی، خطا نمی‌گیری و به آرگومان‌های همان تابعِ پنهان می‌رسی. در مرورگر همین کد `ReferenceError` می‌دهد.

arrow function و `new`

قبل از آمدن `class` در ES6، با یک تابع معمولی و کلمه‌ی `new` object می‌ساختند (به چنین تابعی constructor یا «سازنده» می‌گویند؛ کلاس‌ها را در درس «کلاس‌ها» می‌بینیم). arrow function نمی‌تواند constructor باشد:

```JavaScript
function Car(model) {
  this.model = model;
}
const Bike = (model) => {
  this.model = model;
};

const c = new Car("Pride");
console.log(c.model);        // Pride
const b = new Bike("Honda"); // TypeError: Bike is not a constructor
```

میم اسلاید ۳۶: سه درجه‌ی «شیک بودن»

میم پوی کت‌وشلواری این بار سه روش ساختن و صدا زدن یک تابع (خالی) را رتبه‌بندی می‌کند.

۱. پوی معمولی با پیراهن قرمز: function declaration و بعد صدا زدنش.

```JavaScript
function foo() {
  //
}
foo();
```

۲. پوی کت‌وشلواری: arrow function در یک `const` و بعد صدا زدنش.

```JavaScript
const foo = () => {
  //
}
foo();
```

۳. شیک‌ترین پو (با کلاه، عینک تک‌چشمی و سبیل): تابعی که همان لحظه‌ی تعریف صدا زده می‌شود.

```JavaScript
(() => {
  //
})();
```

هر سه کد بدون خطا اجرا می‌شوند و چون بدنه‌ها خالی‌اند، خروجی‌ای ندارند. شوخی میم: هر چه کوتاه‌تر و «مدرن‌تر»، شیک‌تر!

IIFE چیست؟

IIFE (Immediately Invoked Function Expression، یعنی «عبارت تابعی که بلافاصله صدا زده می‌شود») تابعی است که **تعریف می‌شود و همان لحظه اجرا می‌شود**:

```JavaScript
(() => {
  console.log("I run immediately!");
})();

(function () {
  console.log("Classic IIFE");
})();
// I run immediately!
// Classic IIFE
```

- پرانتز اول `( ... )` تابع را در بر می‌گیرد و آن را به یک **عبارت** (expression، تکه‌کدی که مقدار دارد) تبدیل می‌کند.
- `()` آخر همان تابع را **فوراً صدا می‌زند**.

چرا پرانتز دور تابع لازم است؟ اگر خطی با کلمه‌ی `function` شروع شود، JavaScript آن را function declaration می‌داند که **نام** می‌خواهد و نمی‌شود همان‌جا صدایش زد؛ `function () { ... }();` خطای `SyntaxError` می‌دهد. پرانتز این ابهام را برطرف می‌کند. (شکل `(function () { ... }());` که `()` داخل پرانتز بیرونی آمده هم درست است.)

چرا از IIFE استفاده می‌کنند؟

1. **ساختن محدوده‌ی (scope) خصوصی**: متغیرهای داخل IIFE بیرون آن وجود ندارند؛ پس محدوده‌ی سراسری (global) را شلوغ نمی‌کنند و با نام‌های کدهای دیگر تداخل پیدا نمی‌کنند. این کاربرد قبل از ES6 که فقط `var` داشتیم حیاتی بود (درس «مشکلات var»).
2. اجرای یک کد راه‌اندازی (setup) **فقط یک بار**، بدون اینکه نام تازه‌ای به محدوده اضافه شود.
3. حساب کردن یک مقدار در همان‌جا.

```JavaScript
const result = ((a, b) => a * b)(3, 4);
console.log(result); // 12

(() => {
  const secret = 42;              // exists only inside the IIFE
  console.log("inside:", secret); // inside: 42
})();
console.log(typeof secret);       // undefined (secret does not exist out here)
```

کِی کدام را استفاده کنیم؟

- **function declaration**: برای توابع اصلی و نام‌دار برنامه؛ وقتی می‌خواهی تابع را هر جای فایل (حتی قبل از تعریفش) صدا بزنی؛ وقتی `this` مخصوص تابع، `arguments` یا `new` لازم داری.
- **arrow function**: برای توابع کوتاه و callbackها (مثل `applyToFive(x => x * 10)`)؛ وقتی می‌خواهی `this` از بیرون به ارث برسد.
- arrow function **نه**: برای متد objectی که با `this` کار می‌کند، برای constructor (با `new`) و وقتی به `arguments` نیاز داری.

> **نکته**
>
> - پرانتز دور پارامتر arrow فقط وقتی **دقیقاً یک** پارامتر ساده داری اختیاری است؛ بدون پارامتر `()` و با دو پارامتر یا بیشتر `(a, b)` لازم است.
> - بدنه‌ی مختصر (بدون آکولاد) مقدارش را خودکار برمی‌گرداند؛ بدنه‌ی بلوکی (با آکولاد) `return` صریح می‌خواهد.
> - برای برگرداندن object از بدنه‌ی مختصر، آن را داخل پرانتز بگذار: `() => ({ a: 1 })` .
> - IIFE یعنی `(() => { ... })();` یا `(function () { ... })();` : تعریف و اجرا در یک قدم، همراه با یک محدوده‌ی خصوصی.

> **دام تستی**
>
> - صدا زدن arrow functionی که در `const` (یا `let`) ذخیره شده، قبل از خط تعریفش `ReferenceError: Cannot access ... before initialization` می‌دهد، نه `undefined` . اگر در `var` ذخیره شده باشد، `TypeError: ... is not a function` می‌گیری.
> - `typeof` روی یک function declaration قبل از خط تعریفش `"function"` است، ولی روی arrow function ذخیره‌شده در `var` برابر `"undefined"` .
> - `() => { a: 1 }` مقدار `undefined` برمی‌گرداند، نه object!
> - `(a, b) => { a + b }` هم `undefined` برمی‌گرداند؛ آکولاد یعنی بدنه‌ی بلوکی و `return` لازم است.
> - arrow function به‌عنوان متد object: `this` داخل آن، آن object نیست.
> - `new` روی arrow function خطای `TypeError: ... is not a constructor` می‌دهد و arrow function شیء `arguments` مخصوص خودش را ندارد.
> - `function () { ... }();` بدون پرانتز دور تابع، `SyntaxError` می‌دهد.

جمع‌بندی

- روش اول (function declaration): کلاسیک، با `function` ؛ hoist می‌شود و قبل و بعد از تعریف قابل صدا زدن است: «Remember, this can be hoisted!»
- روش دوم (arrow function): از ES6، کوتاه‌تر با `=>` ، برای توابع سریع؛ hoist نمی‌شود و باید قبل از استفاده تعریف شود: «Remember, this cannot be hoisted!»
- تابع معمولی `this` و `arguments` مخصوص خودش را دارد و با `new` کار می‌کند؛ arrow function (و `this` `arguments`) را از بیرون به ارث می‌برد و constructor نیست.
- IIFE یعنی `(() => { })();` : تابعی که همان لحظه اجرا می‌شود و محدوده‌ی خصوصی می‌سازد.

تست‌های تمرینی

۲۸ تست · شماره‌ی ۳۵۳ تا ۳۸۰

۳۵۳ آسان

طبق اسلایدهای ۳۴ تا ۳۶، کدام جمله درست است؟

۱) هر دو روش hoist می‌شوند و می‌توان هر دو را قبل از خط تعریفشان صدا زد.

۲) function declaration hoist می‌شود، ولی arrow function hoist نمی‌شود و باید قبل از استفاده تعریف شود.

۳) arrow function hoist می‌شود، ولی function declaration نه.

۴) هیچ‌کدام hoist نمی‌شوند.

۳۵۴ آسان

خروجی کد زیر چیست؟

```JavaScript
console.log(add(2, 3));

function add(a, b) {
  return a + b;
}
```

۱) `undefined`

۲) `Uncaught ReferenceError`

۳) `Uncaught TypeError`

۴) `5`

۳۵۵ آسان

طبق اسلاید ۳۵، arrow function در کدام نسخه‌ی ECMAScript به زبان اضافه شد؟

۱) ES1 (سال ۱۹۹۷)

۲) ES5 (سال ۲۰۰۹)

۳) ES6 (ES2015)

۴) ES2017

۳۵۶ آسان

میم پو در اسلاید ۳۶ سه روش ساختن و صدا زدن تابع را از «ساده» به «شیک» مرتب کرده است. ترتیب درست کدام است؟

۱) function declaration، سپس arrow function در یک `const`، سپس IIFE

۲) IIFE، سپس arrow function در یک `const`، سپس function declaration

۳) arrow function در یک `const`، سپس function declaration، سپس IIFE

۴) function declaration، سپس IIFE، سپس arrow function در یک `const`

۳۵۷ آسان

طبق اسلاید ۳۶، جای خالی را پر کن: «Functionها `this` مخصوص خودشان را دارند، ولی arrowها `this` را _____ می‌گیرند.»

۱) همیشه برابر خودِ تابع قرار می‌دهند.

۲) هر بار به یک object تازه وصل می‌کنند.

۳) فقط هنگام استفاده با `new` می‌سازند.

۴) از کدِ بیرونی (جایی که نوشته شده‌اند) به ارث می‌برند.

۳۵۸ آسان

کد `(() => { /* ... */ })();` در سومین پنل میم اسلاید ۳۶ چیست؟

۱) یک function declaration که hoist می‌شود.

۲) یک خطای نحوی، چون تابع اسم ندارد.

۳) یک IIFE: تابعی که همان لحظه‌ی تعریف، صدا زده (اجرا) می‌شود.

۴) یک arrow function که تعریف می‌شود ولی هیچ‌وقت اجرا نمی‌شود.

۳۵۹ آسان

در کدام حالت می‌توان پرانتز دور پارامترهای یک arrow function را حذف کرد؟

۱) وقتی تابع هیچ پارامتری ندارد.

۲) وقتی تابع دقیقاً یک پارامتر ساده دارد؛ مثل `x => x * 2`.

۳) وقتی تابع دو پارامتر دارد.

۴) همیشه؛ پرانتز در arrow function کاملاً اختیاری است.

۳۶۰ آسان

خروجی کد زیر چیست؟

```javascript
const double = x => x * 2;
const hello = () => "hello";
console.log(double(7), hello());
```

۱) `14 undefined`

۲) `Uncaught SyntaxError`

۳) `NaN hello`

۴) `14 hello`

۳۶۱ متوسط

خروجی کد زیر چیست؟

```javascript
console.log(multiply(4, 5));

const multiply = (a, b) => {
  return a * b;
};
```

۱) `20`
۲) `undefined`
۳) `Uncaught TypeError`
۴) `Uncaught ReferenceError`

۳۶۲ متوسط

کدام arrow function با `function (a, b) { return a * b; }` معادل است؛ یعنی اگر آن را در `const multiply = ______;` بگذاریم، دستور `console.log(multiply(4, 5))` عدد `20` را چاپ می‌کند؟

۱) `(a, b) => a * b`
۲) `(a, b) -> a * b`
۳) `(a, b) => { a * b }`
۴) `a, b => a * b`

۳۶۳ متوسط

خروجی کد زیر چیست؟

```javascript
const add = (a, b) => { a + b };
const sub = (a, b) => a - b;
console.log(add(5, 3), sub(5, 3));
```

۱) `8 2`
۲) `undefined 2`
۳) `8 undefined`
۴) `Uncaught SyntaxError`

۳۶۴ متوسط

کدام خط کد زیر خطا می‌دهد؟

```javascript
1 console.log(square(3));
2 console.log(cube(2));
3 function square(n) {
4   return n * n;
5 }
6 const cube = (n) => n ** 3;
```

۱) خط ۱
۲) خط ۶
۳) خط ۲
۴) هیچ خطایی رخ نمی‌دهد

۳۶۵ متوسط

خروجی کد زیر چیست؟

```JavaScript
const result = ((a, b) => a * b)(3, 4);
console.log(result);
(function () {
  console.log("IIFE ran");
})();
```

۱) `12` `IIFE ran`
۲) `Uncaught SyntaxError`
۳) `12`
۴) `undefined` `IIFE ran`

۳۶۶ متوسط

مهم‌ترین دلیل استفاده از IIFE (مثل `(() => { ... })();`) چیست؟

۱) ساختن یک محدوده‌ی (scope) خصوصی تا متغیرهای داخلش به بیرون درز نکنند و محدوده‌ی سراسری را شلوغ نکنند.
۲) اینکه تابع hoist شود و بتوان آن را قبل از تعریف صدا زد.
۳) اینکه تابع به‌طور خودکار هر چند ثانیه یک بار اجرا شود.
۴) اینکه بتوان تابع را بعداً با نامش بارها صدا زد.

۳۶۷ متوسط

کدام تعریف arrow function خطای نحوی (`SyntaxError`) می‌دهد؟

۱) `const f = x => x + 1;`
۲) `const f = () => 1;`
۳) `const f = x, y => x + y;`
۴) `const f = (x, y) => { return x + y; };`

۳۶۸ متوسط

کدام جمله درباره‌ی arrow function **نادرست** است؟

۱) arrow functionی که بدنه‌ی بلوکی `{ }` دارد، مقدار آخرین عبارتش را خودکار برمی‌گرداند.
۲) arrow function شیء `arguments` مخصوص خودش را ندارد.
۳) arrow function را نمی‌توان با `new` صدا زد.
۴) در arrow functionی که دقیقاً یک پارامتر ساده دارد، پرانتز دور پارامتر اختیاری است.

۳۶۹ متوسط

در کدام موقعیت **نباید** از arrow function استفاده کرد و تابع معمولی (`function`) لازم است؟

۱) متد یک object که باید با `this` به همان object دسترسی داشته باشد.

۲) یک تابع کوتاه یک‌خطی که عددی را دو برابر می‌کند.

۳) تابعی که هیچ پارامتری ندارد.

۴) callbackی که به تابع دیگری پاس داده می‌شود و فقط یک مقدار حساب می‌کند.

۳۷۰ متوسط

خروجی کد زیر چیست؟ (در یک فایل معمولی Node.js و در اسکریپت معمولی مرورگر نتیجه یکسان است.)

```JavaScript
const user = {
  username: "Sara",
  regular: function () {
    return this.username;
  },
  arrow: () => this.username,
};
console.log(user.regular());
console.log(user.arrow());
```

۱) `Sara` `Sara`

۲) `Sara` `undefined`

۳) `undefined` `Sara`

۴) `Sara` `Uncaught TypeError`

۳۷۱ متوسط

خروجی کد زیر چیست؟

```JavaScript
const add = (a) => (b) => a + b;
console.log(add(2)(3));
const add5 = add(5);
console.log(add5(10));
```

۱) `Uncaught SyntaxError`

۲) `NaN` `NaN`

۳) `Uncaught TypeError`

۴) `5` `15`

۳۷۲ متوسط

خروجی کد زیر چیست؟

```javascript
(() => {
  const secret = 42;
  console.log("inside:", secret);
})();
console.log(typeof secret);
```

۱) `inside: 42` `number`

۲) `inside: 42` `Uncaught ReferenceError`

۳) `inside: 42` `undefined`

۴) `Uncaught SyntaxError`

۳۷۳ سخت

خروجی کد زیر چیست؟

```javascript
console.log(multiply(4, 5));
var multiply = (a, b) => a * b;
```

۱) `20`

۲) `NaN`

۳) `Uncaught ReferenceError`

۴) `Uncaught TypeError`

۳۷۴ سخت

خروجی کد زیر چیست؟

```javascript
const makeUser = () => { name: "Ali" };
const makeUser2 = () => ({ name: "Ali" });
console.log(makeUser());
console.log(makeUser2());
```

۱) `{ name: 'Ali' }` `{ name: 'Ali' }`

۲) `undefined` `{ name: 'Ali' }`

۳) `Uncaught SyntaxError`

۴) `Ali` `{ name: 'Ali' }`

۳۷۵ سخت

خروجی کد زیر چیست؟

```JavaScript
console.log(typeof greet);
console.log(typeof welcome);
function greet() {}
var welcome = () => {};
```

۱) `function` `function`

۲) `undefined` `undefined`

۳) `function` `Uncaught ReferenceError`

۴) `function` `undefined`

۳۷۶ سخت

کدام یک از کدهای زیر خطای نحوی (`SyntaxError`) می‌دهد؟

۱) `(function () { console.log("hi"); })();`

۲) `function () { console.log("hi"); }();`

۳) `(() => { console.log("hi"); })();`

۴) `(function () { console.log("hi"); }());`

۳۷۷ سخت

در جای خالی کدام گزینه قرار بگیرد تا خروجی `Ali` باشد؟

```JavaScript
const toObj = (name) => ______;
console.log(toObj("Ali").name);
```

۱) `{ name: name }`

۲) `return { name: name }`

۳) `({ name: name })`

۴) `{ return name; }`

۳۷۸ سخت

خروجی کد زیر **در مرورگر** (یک اسکریپت معمولی `<script>`) چیست؟

```JavaScript
function regular() {
  return arguments.length;
}
const arrow = () => arguments.length;
console.log(regular(1, 2, 3));
console.log(arrow(1, 2, 3));
```

۱) `3` `Uncaught ReferenceError`

۲) `3` `3`

۳) `3` `0`

۴) `Uncaught ReferenceError`

۳۷۹ سخت

خروجی کد زیر چیست؟

```javascript
function outer() {
  const inner = () => arguments[0];
  return inner("x", "y");
}
console.log(outer("a", "b"));
```

۱) `x`
۲) `a`
۳) `undefined`
۴) `Uncaught ReferenceError`

۳۸۰ سخت

کدام خط کد زیر خطا می‌دهد؟

```javascript
function Car(model) {
  this.model = model;
}
const Bike = (model) => {
  this.model = model;
};
const c = new Car("Pride");
console.log(c.model);
const b = new Bike("Honda");
console.log(b.model);
```

۱) خط ۵
۲) خط ۷
۳) خط ۹
۴) هیچ خطایی رخ نمی‌دهد

۱-۱۷ مشکلات var

منبع: Introduction to JS، اسلاید ۲۷

درسنامه

قبل از ES6 (سال ۲۰۱۵) تنها راه تعریف متغیر در JavaScript کلمه‌ی `var` بود. `let` و `const` دقیقاً به این خاطر به زبان اضافه شدند که `var` چند رفتار دردسرساز دارد. امروز در کد جدید از `var` استفاده نمی‌کنیم، ولی در کدهای قدیمی، آموزش‌های اینترنتی و البته **سؤال‌های امتحان** زیاد می‌بینیمش؛ پس باید مشکلاتش را خوب بشناسیم. اسلاید ۲۷ این مشکلات را در پنج جمله خلاصه کرده است:

- **Function-scoped است، نه block-scoped**: می‌تواند از بلوک‌های `{}` بیرون بزند (leak کند، یعنی «نشت کند»).
- **Variable hoisting**: اعلان‌ها به بالا منتقل می‌شوند و `undefined` های غیرمنتظره می‌سازند.
- **اجازه‌ی تعریف دوباره (redeclaration)**: یک متغیر را می‌شود چند بار بدون خطا تعریف کرد.
- **آلوده کردن محدوده‌ی سراسری (global scope pollution)**: `var` در محدوده‌ی سراسری به `window` می‌چسبد.
- **رفتار ناسازگار (inconsistent behavior)**: دیباگ کردن و فهمیدن منطق کد را سخت‌تر می‌کند.

میم اسلاید ۲۷: حلقه‌ی قدرت

کنار اسلاید یک میم سه‌قسمتی از فیلم «ارباب حلقه‌ها» هست:

1. دستی که «حلقه‌ی یگانه» را گرفته و روی حلقه نوشته شده **VAR**.
2. اِلروند با برچسب **EVERYONE ELSE** (بقیه‌ی برنامه‌نویس‌ها) می‌گوید: «بیندازش توی آتش! نابودش کن!» (Cast it into the fire! Destroy it!)
3. ایسیلدور با برچسب **OLD JS DEVS** (برنامه‌نویس‌های قدیمی JavaScript) جواب می‌دهد: «نه.» (No.)

در فیلم، ایسیلدور فرصت داشت حلقه‌ی شیطانی را نابود کند، ولی دلش نیامد. شوخی میم: همه می‌دانند `var` باید کنار گذاشته شود، ولی برنامه‌نویس‌های قدیمی که به آن عادت کرده‌اند حاضر نیستند رهایش کنند. بیا ببینیم این «حلقه» چه بلاهایی سر کد می‌آورد.

مشکل ۱: function-scoped است، نه block-scoped

scope (محدوده) یعنی بخشی از کد که یک متغیر در آن شناخته می‌شود.

- متغیر **block-scoped** (مثل `let` و `const`) فقط داخل همان `{ ... }` ی وجود دارد که در آن تعریف شده است.
- متغیر **function-scoped** (یعنی `var`) در **کل تابعی** که در آن تعریف شده وجود دارد و آکولادهای `if`، `for` و `while` جلویش را نمی‌گیرند. اگر بیرون از هر تابعی تعریف شود، سراسری (global) است.

> **یک تشبیه ساده**
>
> `let` مثل کلید یک اتاق است که فقط درِ همان اتاق (بلوک) را باز می‌کند. `var` مثل شاه‌کلید خانه است که درِ همه‌ی اتاق‌های خانه (کل تابع) را باز می‌کند؛ ولی درِ خانه‌ی همسایه (یک تابع دیگر) را باز نمی‌کند.

```JavaScript
if (true) {
  var a = 1; // var: belongs to the surrounding function (or the global scope)
  let b = 2; // let: belongs only to this block
}
console.log(a); // 1 (var leaked out of the if block)
console.log(b); // ReferenceError: b is not defined
```

شمارنده‌ی حلقه هم بیرون می‌ریزد:

```JavaScript
for (var i = 0; i < 3; i++) {
  // loop body
}
console.log(i); // 3 (the counter leaked out of the loop)
```

چرا `3` و نه `2`؟ بدنه برای `i` برابر `0`، `1` و `2` اجرا می‌شود؛ بعد `i++` آن را `3` می‌کند، شرط `i < 3` نادرست می‌شود و حلقه تمام می‌شود. `i` با همان مقدار `3` بیرون حلقه باقی می‌ماند. اگر به‌جای `var` از `let` استفاده کنی، `i` بعد از حلقه وجود ندارد و `console.log(i)` خطای `ReferenceError` (خطای ارجاع) می‌دهد.

اما `var` از **تابع** بیرون نمی‌زند؛ متغیر `var` داخل یک تابع، محلیِ همان تابع است:

```JavaScript
function setup() {
  if (true) {
    var token = "abc";
  }
  console.log(token); // abc (visible everywhere inside setup)
}
setup();
console.log(token);   // ReferenceError: token is not defined
```

خطرناک‌ترین نتیجه‌ی این رفتار، **بازنویسی ناخواسته** است. ممکن است فکر کنی داخل بلوک یک متغیر تازه ساخته‌ای، در حالی که متغیر بیرونی را عوض کرده‌ای:

```JavaScript
var total = 10;
if (true) {
  var total = 99; // NOT a new variable: the same total is overwritten
}
console.log(total); // 99

let count = 10;
if (true) {
  let count = 99; // a new, separate variable that lives only in this block
}
console.log(count); // 10
```

مشکل ۲: hoisting و `undefined` های غیرمنتظره

hoisting (بالا بردن اعلان‌ها به ابتدای محدوده) یعنی موتور JavaScript قبل از اجرا، **اعلان** (declaration) هر `var` را به بالای محدوده‌اش (تابع یا محدوده‌ی سراسری) منتقل می‌کند و مقدار اولیه‌ی `undefined` به آن می‌دهد؛ ولی **مقداردهی** (assignment)

سر جای خودش می‌ماند. پس `var` را می‌شود قبل از خط تعریفش خواند، **بدون خطا**:

```JavaScript
console.log(score); // undefined (no error!)
var score = 50;
console.log(score); // 50
```

موتور این کد را عملاً این‌طور اجرا می‌کند:

```JavaScript
var score;          // the declaration is hoisted and starts as undefined
console.log(score); // undefined
score = 50;         // the assignment stays where it was
console.log(score); // 50
```

با `let` (و `const`) همین کار خطا می‌دهد، که در واقع **بهتر** است:

```JavaScript
console.log(level); // ReferenceError: Cannot access 'level' before initialization
let level = 1;
```

چرا `undefined` از خطا بدتر است؟ خطا همان لحظه و همان خط، محل اشکال را نشانت می‌دهد. اما `undefined` بی‌صدا در برنامه جلو می‌رود و جای دیگری دردسر می‌سازد (مثلاً `NaN` در محاسبه یا کلمه‌ی `undefined` وسط یک متن)؛ پیدا کردن ریشه‌ی چنین باگی سخت است.

غافلگیری معروف hoisting داخل تابع:

```JavaScript
var msg = "global";
function show() {
  console.log(msg); // undefined (not "global"!)
  var msg = "local";
  console.log(msg); // local
}
show();
```

داخل `show`، اعلان `var msg` به **بالای تابع** منتقل می‌شود؛ پس از همان خط اول تابع، یک `msg` محلی (با مقدار `undefined`) وجود دارد که `msg` سراسری را پنهان می‌کند. قوانین کامل hoisting (برای `let`، `const`، `class` و توابع) را در درس «Hoisting و TDZ» می‌خوانیم.

مشکل ۳: تعریف دوباره بدون خطا

```JavaScript
var x = 1;
var x = 2;      // no error: it is the same variable, now 2
console.log(x); // 2
```

```JavaScript
let y = 1;
let y = 2; // SyntaxError: Identifier 'y' has already been declared
```

فرض کن در یک فایل هزارخطی، دو بخش مختلف کد هر کدام یک `var data` تعریف کنند. با `var` هیچ هشداری نمی‌گیری و دومی بی‌صدا مقدار اولی را بازنویسی می‌کند. با `let` موتور **قبل از اجرای حتی یک خط** جلویت را می‌گیرد: `SyntaxError` (خطای نحوی) در مرحله‌ی خواندن (parse) کل فایل کشف می‌شود، پس هیچ خطی از آن فایل اجرا نمی‌شود. ترکیب `var` و `let` با یک نام در یک محدوده (مثل `var z = 1; let z = 2;`) هم `SyntaxError` است.

یک نکته‌ی ظریف: تعریف دوباره‌ی `var` **بدون مقدار**، مقدار قبلی را پاک نمی‌کند:

JavaScript

```
var count = 5;
var count;          // a redeclaration without a value is ignored
console.log(count); // 5 (not undefined!)
```

مشکل ۴: آلوده کردن محدوده‌ی سراسری (Global scope pollution)

در مرورگر، **شیء سراسری** (global object) اسمش `window` است؛ objectی که همه‌ی چیزهای سراسری مثل `alert`، `document` و `console` روی آن قرار دارند. `var`ی که در سطح بالای یک اسکریپت معمولی تعریف شود، **به یک ویژگی (property) از `window` تبدیل می‌شود**. `let` و `const` هم سراسری‌اند، ولی ویژگی `window` نمی‌شوند:

JavaScript

```
// In a browser (a classic <script>)
var city = "Tehran";
let country = "Iran";
console.log(window.city);    // Tehran
console.log(window.country); // undefined
```

(function declarationهای سطح بالا هم به همین شکل ویژگی `window` می‌شوند.)

چرا «آلودگی»؟ همه‌ی اسکریپت‌های یک صفحه (کد خودت، کتابخانه‌ها، تبلیغات و ...) **یک `window` مشترک** دارند. اگر دو اسکریپت با `var` نام یکسانی بسازند، دومی بی‌صدا اولی را بازنویسی می‌کند:

HTML

```
<script>
  // library.js
  var counter = 100;
  function getCounter() {
    return counter;
  }
</script>
<script>
  // app.js
  var counter = 0;           // overwrites the library's counter!
  console.log(getCounter()); // 0 (the library is now broken)
</script>
```

> **یادداشت**
>
> در **Node.js** شیء `window` وجود ندارد (`typeof window` برابر `"undefined"` است) و شیء سراسری `globalThis` (یا `global`) نام دارد. علاوه بر این، Node.js هر فایل `.js` معمولی (CommonJS) را یک ماژول با محدوده‌ی جداگانه اجرا می‌کند؛ پس `var` سطح بالای یک فایل Node.js ویژگی `globalThis` نمی‌شود:

```JavaScript
// In Node.js (a normal .js file)
var city = "Tehran";
console.log(globalThis.city); // undefined
console.log(typeof window);   // undefined
```

مشکل ۵: رفتار ناسازگار؛ دیباگ سخت‌تر

همه‌ی موارد بالا با هم یعنی رفتار `var` به **جای** تعریفش بستگی دارد: داخل تابع محلی است ولی داخل بلوک نه؛ قبل از تعریفش قابل خواندن است ولی مقدار ندارد؛ ممکن است بی‌صدا متغیر دیگری را بازنویسی کند یا خودش بازنویسی شود. نتیجه اینکه با نگاه به چند خط اطراف یک متغیر نمی‌شود با اطمینان گفت مقدارش چیست؛ یعنی دیباگ (پیدا کردن و رفع باگ) سخت‌تر می‌شود.

معروف‌ترین نمونه (پیش‌نمایش): `var` در حلقه همراه با `setTimeout` . تابع `setTimeout(fn, 0)` تابع `fn` را **کمی بعد**، یعنی بعد از تمام شدن کدهای فعلی، اجرا می‌کند:

```JavaScript
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}
// 3
// 3
// 3
```

```JavaScript
for (let j = 0; j < 3; j++) {
  setTimeout(() => console.log(j), 0);
}
// 0
// 1
// 2
```

با `var` کل حلقه فقط **یک** `i` دارد؛ وقتی callbackها بالاخره اجرا می‌شوند، حلقه تمام شده و `i` برابر `3` است. با `let` هر دور حلقه `j` **تازه‌ی خودش** را دارد. توضیح کامل این رفتار را در درس‌های «Closure» و «Event loop» می‌بینیم.

جدول مقایسه‌ی `var` ، `let` و `const`

ویژگی	`var`	`let`	`const`
محدوده (scope)	تابع (function-scoped)	بلوک (block-scoped)	بلوک (block-scoped)
استفاده قبل از خط تعریف	hoist می‌شود؛ مقدارش `undefined` است	hoist می‌شود ولی مقداردهی نمی‌شود: `ReferenceError` (TDZ)	مثل `let` : `ReferenceError`
تعریف دوباره در همان محدوده	مجاز (بی‌صدا)	`SyntaxError`	`SyntaxError`
انتساب دوباره (`x = ...`)	مجاز	مجاز	`TypeError`
ویژگی `window` در سطح بالای اسکریپت مرورگر	می‌شود	نمی‌شود	نمی‌شود
مقداردهی هنگام تعریف	اختیاری	اختیاری	اجباری

(TDZ یا Temporal Dead Zone یعنی «منطقه‌ی مرده‌ی زمانی»: بازه‌ی بین شروع محدوده تا خط تعریف `let` / `const` که دسترسی به متغیر در آن خطا می‌دهد.)

بهترین روش: در کد جدید هرگز از `var` استفاده نکن؛ به‌طور پیش‌فرض `const` و هر وقت انتساب دوباره لازم شد `let` . برای «تعمیر» کد قدیمی هم معمولاً کافی است `var` ها را با `let` / `const` عوض کنی.

نکته

- `var` تابع‌محور است: در کل تابع دیده می‌شود و بلوک‌های `if` / `for` نگهش نمی‌دارند؛ ولی از خودِ تابع بیرون نمی‌رود.
- شمارنده‌ی `for (var i = ...)` بعد از حلقه با مقدار نهایی‌اش (اولین مقداری که شرط را نادرست کرد) باقی می‌ماند.
- خواندن `var` قبل از تعریف: `undefined` ؛ خواندن `let` / `const` قبل از تعریف: `ReferenceError` .
- `var` سطح بالای اسکریپت معمولی مرورگر ویژگی `window` می‌شود؛ `let` و `const` نه. در Node.js اصلاً `window` نداریم.

دام تستی

- `var total = 99` داخل یک بلوک `if` متغیر جدید نمی‌سازد؛ همان `total` بیرونی را بازنویسی می‌کند. (با `let` یک متغیر جدید و جدا ساخته می‌شود.)
- داخل تابع، یک `var msg` در خط‌های پایین‌ترِ همان تابع باعث می‌شود `msg` در خط‌های بالاتر `undefined` باشد، نه مقدار سراسری.
- `var count;` (تعریف دوباره بدون مقدار) مقدار قبلی را پاک نمی‌کند.
- تعریف دوباره‌ی `let` در همان محدوده `SyntaxError` است و قبل از اجرای **هر** خطی رخ می‌دهد؛ پس هیچ خروجی‌ای چاپ نمی‌شود.
- در یک فایل Node.js، `var` سطح بالا ویژگی `globalThis` نمی‌شود؛ رفتار `window` فقط مخصوص اسکریپت معمولی مرورگر است.
- `typeof` روی نامی که تعریف نشده، `"undefined"` می‌دهد و خطا ندارد؛ ولی استفاده‌ی مستقیم از همان نام `ReferenceError` می‌دهد.

جمع‌بندی

- پنج مشکل `var` (اسلاید ۲۷): function-scoped بودن و نشت از بلوک‌ها، hoisting و `undefined` غیرمنتظره، تعریف دوباره بدون خطا، چسبیدن به `window` (آلودگی سراسری) و رفتار ناسازگار که دیباگ را سخت می‌کند.
- میم: همه می‌خواهند `var` نابود شود، ولی برنامه‌نویس‌های قدیمی JavaScript می‌گویند «نه».
- راه‌حل: `const` به‌طور پیش‌فرض، `let` برای انتساب دوباره و هرگز `var` .

تست‌های تمرینی

۲۰ تست · شماره‌ی ۳۸۱ تا ۴۰۰

۳۸۱ آسان

کدام مورد جزو «مشکلات `var`» در اسلاید ۲۷ **نیست**؟

۱) function-scoped است، نه block-scoped و می‌تواند از بلوک `{}` بیرون بزند.

۲) اجازه می‌دهد یک متغیر چند بار بدون خطا تعریف شود.

۳) `var` در محدوده‌ی سراسری به `window` می‌چسبد.

۴) نمی‌توان به متغیر `var` مقدار تازه داد.

۳۸۲ آسان

در میم «ارباب حلقه‌ها»ی اسلاید ۲۷، «EVERYONE ELSE» می‌گوید «حلقه‌ی VAR را در آتش بینداز!» و «OLD JS DEVS» جواب می‌دهند «نه.». منظور میم چیست؟

۱) همه می‌خواهند `var` کنار گذاشته شود، ولی برنامه‌نویس‌های قدیمی JavaScript حاضر نیستند دست از آن بکشند.

۲) برنامه‌نویس‌های قدیمی می‌خواهند `var` حذف شود، ولی بقیه مخالف‌اند.

۳) `var` جدیدترین ویژگی زبان است و برنامه‌نویس‌های قدیمی هنوز آن را یاد نگرفته‌اند.

۴) `var` سریع‌ترین روش تعریف متغیر است و برای همین همه آن را نگه می‌دارند.

۳۸۳ آسان

منظور از «آلوده کردن محدوده‌ی سراسری» (Global scope pollution) در اسلاید ۲۷ چیست؟

۱) متغیرهای `var` هرگز از حافظه پاک نمی‌شوند.

۲) `var` سطح بالای اسکریپت به ویژگی شیء سراسری `window` تبدیل می‌شود؛ همه‌ی اسکریپت‌های صفحه همین فضای مشترک را دارند و نام‌های تکراری همدیگر را بازنویسی می‌کنند.

۳) متغیرهای سراسری داخل توابع قابل خواندن نیستند.

۴) هر متغیر `var` یک کپی تازه از شیء `window` می‌سازد.

۳۸۴ آسان

خروجی کد زیر چیست؟

```javascript
if (true) {
  var color = "red";
}
console.log(color);
```

۱) `undefined`
۲) `Uncaught ReferenceError`
۳) `red`
۴) `Uncaught SyntaxError`

۳۸۵ آسان

خروجی کد زیر چیست؟

```javascript
console.log(score);
var score = 50;
console.log(score);
```

۱) `50` `50`
۲) `undefined` `50`
۳) `Uncaught ReferenceError`
۴) `undefined` `undefined`

۳۸۶ آسان

خروجی کد زیر چیست؟

```javascript
var x = 1;
var x = 2;
console.log(x);
```

۱) `1`
۲) `Uncaught SyntaxError`
۳) `undefined`
۴) `2`

۳۸۷ متوسط

خروجی کد زیر چیست؟

```javascript
if (true) {
  var a = 1;
  let b = 2;
}
console.log(a);
console.log(b);
```

۱) `1` `Uncaught ReferenceError`
۲) `1` `2`
۳) `Uncaught ReferenceError`
۴) `1` `undefined`

۳۸۸ متوسط

خروجی کد زیر چیست؟

```javascript
for (var i = 0; i < 3; i++) {}
console.log(i);
```

۱) `3`
۲) `2`
۳) `undefined`
۴) `Uncaught ReferenceError`

۳۸۹ متوسط

کدام خط کد زیر خطا می‌دهد؟

```javascript
var a = 1;
var a = 2;
let b = 3;
let b = 4;
console.log(a, b);
```

۱) خط ۲
۲) خط ۴
۳) خط ۵
۴) هیچ خطایی رخ نمی‌دهد

۳۹۰ متوسط

کدام خط کد زیر خطا می‌دهد؟

```javascript
console.log(a);
var a = 1;
console.log(b);
let b = 2;
```

۱) خط ۱
۲) خط ۴
۳) خط ۳
۴) هیچ خطایی رخ نمی‌دهد

۳۹۱ متوسط

خروجی کد زیر چیست؟

```javascript
function setup() {
  if (true) {
    var token = "abc";
  }
  console.log(token);
}
setup();
console.log(token);
```

۱) `abc` `abc`

۲) `undefined` `Uncaught ReferenceError`

۳) `Uncaught ReferenceError`

۴) `abc` `Uncaught ReferenceError`

۳۹۲ متوسط

خروجی کد زیر **در مرورگر** (یک اسکریپت معمولی `<script>`) چیست؟

```javascript
var city = "Tehran";
let country = "Iran";
console.log(window.city);
console.log(window.country);
```

۱) `Tehran` `Iran`

۲) `Tehran` `undefined`

۳) `undefined` `undefined`

۴) `undefined` `Iran`

۳۹۳ متوسط

کدام جمله **درست** است؟

۱) `let` را می‌توان در همان محدوده دوباره تعریف کرد، ولی `var` را نه.

۲) خواندن متغیر `let` قبل از خط تعریفش مقدار `undefined` می‌دهد.

۳) `var` محدوده‌ی تابعی (function-scoped) دارد، ولی `let` و `const` محدوده‌ی بلوکی (block-scoped) دارند.

۴) `const` سطح بالای یک اسکریپت مرورگر، ویژگی `window` می‌شود.

۳۹۴ متوسط

دستور `var x = 5;` در کدام حالت باعث می‌شود `window.x` برابر `5` شود؟

۱) وقتی در سطح بالای یک اسکریپت معمولی مرورگر (`<script>`) نوشته شود.

۲) وقتی در سطح بالای یک فایل Node.js نوشته شود.

۳) وقتی داخل بدنه‌ی یک تابع در مرورگر نوشته شود.

۴) وقتی داخل یک IIFE در مرورگر نوشته شود.

۳۹۵ سخت

خروجی کد زیر چیست؟

```JavaScript
var msg = "global";
function show() {
  console.log(msg);
  var msg = "local";
  console.log(msg);
}
show();
```

۱) `global` `local`

۲) `undefined` `local`

۳) `global` `global`

۴) `Uncaught ReferenceError`

۳۹۶ سخت

کد زیر را در یک فایل معمولی Node.js (مثلاً با `node app.js`) اجرا می‌کنیم. خروجی چیست؟

```JavaScript
var city = "Tehran";
console.log(globalThis.city);
console.log(typeof window);
```

۱) `Tehran` `object`

۲) `Tehran` `undefined`

۳) `undefined` `Uncaught ReferenceError`

۴) `undefined` `undefined`

۳۹۷ سخت

یک صفحه‌ی وب دو اسکریپت دارد: اول یک کتابخانه (اسکریپتی که در HTML زیر آمده) و بعد کد ما. خروجی کنسول **در مرورگر** چیست؟

HTML

```
<script>
  // library.js
  var counter = 100;
  function getCounter() {
    return counter;
  }
</script>
```

JavaScript

```
// app.js
var counter = 0;
console.log(getCounter());
```

۱) `0`
۲) `100`
۳) `undefined`
۴) `Uncaught ReferenceError`

۳۹۸ سخت

خروجی کد زیر چیست؟

JavaScript

```
var a = 1;
let b = 1;
if (true) {
  var a = 2;
  let b = 2;
}
console.log(a, b);
```

۱) `1 1`
۲) `2 2`
۳) `2 1`
۴) `Uncaught SyntaxError`

۳۹۹ سخت

در جای خالی کدام گزینه قرار بگیرد تا خروجی به ترتیب `0`، `1` و `2` (هر کدام در یک خط) باشد؟

JavaScript

```
for (___ i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}
```

۱) `var`
۲) `const`
۳) هیچ کلمه‌ای (فقط `i = 0`)
۴) `let`

۴۰۰ سخت

خروجی کد زیر چیست؟

```JavaScript
var count = 5;
var count;
console.log(count);
```

۱) `undefined`
۲) `Uncaught SyntaxError`
۳) `5`
۴) `null`

۱-۱۸ Hoisting و Temporal Dead Zone (TDZ)

منبع: Introduction to JS، اسلایدهای ۳۴ تا ۳۸

درسنامه

میم اسلاید ۳۷: «حس عجیبی دارد، ولی اجازه می‌دهم!»

در میم اسلاید ۳۷، پیتر پارکر (مرد عنکبوتی) با جمله‌ی **«Using variable before declaration»** (استفاده از متغیر قبل از تعریفش) سراغ دکتر استرنج می‌رود و دکتر استرنج که برچسب **Javascript** خورده، جواب می‌دهد: **«That feels weird, but I'll allow it»** (حس عجیبی دارد، ولی اجازه می‌دهم).

در بیشتر زبان‌ها (مثل C یا Java) اگر از متغیری قبل از تعریفش استفاده کنی، برنامه اصلاً اجرا نمی‌شود. ولی JavaScript در خیلی از موارد «اجازه می‌دهد»:

```JavaScript
console.log(city); // undefined (no error!)
var city = "Tehran";
console.log(city); // Tehran
```

خط اول متغیری را می‌خواند که هنوز به خط تعریفش نرسیده‌ایم، ولی خطا نمی‌گیرد و `undefined` چاپ می‌کند. دلیل این رفتار عجیب **hoisting** است. در این درس دقیق یاد می‌گیری کِی `undefined` می‌گیری، کِی خطا و کِی همه‌چیز درست کار می‌کند؛ این‌ها از پرتکرارترین تست‌های «خروجی کد» هستند.

Hoisting چیست؟ (اسلایدهای ۳۷ و ۳۸)

hoisting (بالا بردن اعلان‌ها به ابتدای محدوده؛ فعل انگلیسی hoist یعنی «بالا کشیدن»، مثل بالا کشیدن پرچم) طبق اسلاید ۳۷ یعنی:

- **Moving declarations before execution**: اعلان‌ها (declarations) **قبل از اجرای کد** جابه‌جا می‌شوند.
- **Common in languages like JavaScript**: رفتاری رایج در زبان‌هایی مثل JavaScript است.
- **Means code runs as if declared at top**: کد طوری اجرا می‌شود که **انگار** همه‌ی اعلان‌ها بالای محدوده نوشته شده‌اند.
- **Variables/functions exist before use**: متغیرها و توابع حتی قبل از خطی که در آن تعریف شده‌اند «وجود دارند».
- **Can cause unexpected behavior**: می‌تواند رفتارهای غیرمنتظره ایجاد کند.

اسلاید ۳۸ (Hoisting in JavaScript) دقیق‌ترش می‌کند:

- **JS moves declarations to the top of scope**: JavaScript اعلان‌ها را به **بالای scope** (محدوده؛ بخشی از کد که یک نام در آن معتبر است: کل فایل، بدنه‌ی یک تابع یا یک بلوک `{ }`) می‌برد.
- **Works for `var`, `function`, and `class` (partially)**: برای `var`، `function` و (به‌صورت ناقص) `class` کار می‌کند.
- **Variables are hoisted but not initialized**: متغیرها بالا برده می‌شوند ولی **مقداردهی اولیه (initialization)** نمی‌شوند.
- **Access before init gives `undefined` or error**: دسترسی قبل از مقداردهی، یا `undefined` می‌دهد یا خطا.
- **Makes code order confusing if misused**: اگر بد استفاده شود، ترتیب کد را گیج‌کننده می‌کند.

دو اصطلاح درس «متغیرها» را یادت بیاور: در `var score = 10;` بخش `var score` **اعلان (declaration)** است و `= 10` **مقداردهی (initialization)**. کل داستان hoisting در یک جمله: **فقط اعلان بالا می‌رود؛ مقداردهی سر جای خودش می‌ماند.**

پشت صحنه: مرحله‌ی ایجاد و مرحله‌ی اجرا

در واقعیت هیچ خطی از کد تو جابه‌جا نمی‌شود؛ «بالا رفتن» یک مدل ذهنی است. اتفاق واقعی این است که موتور JavaScript هر scope را در **دو مرحله** اجرا می‌کند:

1. **مرحله‌ی ایجاد (creation phase)**: قبل از اجرای حتی یک خط، موتور کل آن scope را مرور می‌کند و **نام** همه‌ی اعلان‌ها را ثبت می‌کند. hoisting همین‌جا رخ می‌دهد.
2. **مرحله‌ی اجرا (execution phase)**: حالا کد خط‌به‌خط از بالا به پایین اجرا می‌شود و هر مقداردهی **وقتی خطش برسد** انجام می‌شود.

یک تشبیه ساده

یک کلاس درس را تصور کن. قبل از شروع کلاس (مرحله‌ی ایجاد)، استاد فهرست دانشجوها را می‌خواند و برای هر اسم یک صندلی کنار می‌گذارد. بعد کلاس شروع می‌شود (مرحله‌ی اجرا) و دانشجوها یکی‌یکی از راه می‌رسند.

- صندلی `var` : رویش برگه‌ای با نوشته‌ی `undefined` گذاشته‌اند؛ اگر زودتر سراغش بروی، فقط همان برگه را می‌بینی.
- صندلی `let` ، `const` و `class` : رزرو شده، ولی دورش نوار «دست نزنید!» کشیده‌اند؛ تا صاحبش نرسد، نزدیک شدن به آن یعنی خطا.
- صندلی `function` (تابعِ اعلان‌شده): صاحبش از همان اول نشسته و آماده‌ی کار است.

برای `var` ، کد اول طوری اجرا می‌شود که انگار کد دوم را نوشته‌ای:

JavaScript

```
// What we write:
console.log(a); // undefined
var a = 10;
console.log(a); // 10
```

JavaScript

```
// How it behaves (mental model):
var a;          // the declaration is hoisted (a = undefined)
console.log(a); // undefined
a = 10;         // the assignment stays in its place
console.log(a); // 10
```

جدول اصلی: هر اعلان در مرحله‌ی ایجاد چه می‌شود؟

اعلان	در مرحله‌ی ایجاد	استفاده قبل از خط تعریف
`var x = 5;`	`x` ساخته می‌شود با مقدار `undefined`	`undefined` (بدون خطا)
`let x = 5;` و `const x = 5;`	`x` ساخته می‌شود ولی **مقدار اولیه نمی‌گیرد** (در TDZ است)	`ReferenceError`
`function f() {}`	`f` ساخته می‌شود و **کل تابع** در آن قرار می‌گیرد	کار می‌کند
`class A {}`	`A` ساخته می‌شود ولی مقدار اولیه نمی‌گیرد (در TDZ است)	`ReferenceError`
`var f = () => {};` یا `var f = function () {};`	مثل هر `var` : `f` برابر `undefined`	خواندن: `undefined` ؛ صدا زدن: `TypeError`
`const f = () => {};` یا `let f = function () {};`	مثل هر `let` / `const` : در TDZ	`ReferenceError`
(اصلاً اعلانی وجود ندارد)	—	`ReferenceError: x is not defined`

بیا یک مثال کامل را با این دو مرحله دنبال کنیم:

JavaScript

```
console.log(a);         // undefined
console.log(sum(1, 2)); // 3
var a = 1;
let b = 2;
function sum(x, y) {
  return x + y;
}
console.log(a, b);      // 1 2
```

- **مرحله‌ی ایجاد:** موتور سه اعلان پیدا می‌کند: `a` (با مقدار `undefined`)، `b` (بدون مقدار اولیه، در TDZ) و `sum` (همراه با کل بدنه‌اش).
- **مرحله‌ی اجرا:** خط ۱ مقدار فعلی `a` یعنی `undefined` را چاپ می‌کند. خط ۲ تابع `sum` را که از قبل آماده است صدا می‌زند و `3` چاپ می‌شود. خط ۳ عدد `1` را در `a` می‌ریزد. خط ۴ به TDZ متغیر `b` پایان می‌دهد و `2` را در آن می‌ریزد. خط‌های ۵ تا ۷ در مرحله‌ی اجرا کاری ندارند (تابع قبلاً ساخته شده است). خط ۸ چاپ می‌کند `1 2` .
- اگر در خط ۱ به جای `a` ، متغیر `b` را می‌خواندیم، خطای `ReferenceError` می‌گرفتیم و برنامه همان‌جا متوقف می‌شد.

`var` : بالا می‌رود، با مقدار `undefined`

اعلان `var` به بالای **نزدیک‌ترین تابع** می‌رود (یا اگر داخل هیچ تابعی نباشد، به بالای کل فایل). بلوک‌های `if` ، `for` و `{ }` برای `var` اهمیتی ندارند، چون `var` **function-scoped** (محدود به تابع، نه بلوک) است. نتیجه‌ی عجیبش: حتی اعلانی که داخل یک بلوکِ **هرگز اجرانشده** باشد هم hoist می‌شود:

```JavaScript
console.log(flag); // undefined
if (false) {
  var flag = true; // never runs, but the declaration is still hoisted
}
console.log(flag); // undefined
```

هیچ‌کدام از دو `console.log` خطا نمی‌دهد: `flag` در مرحله‌ی ایجاد ساخته شده، ولی چون مقداردهی‌اش هرگز اجرا نشد، تا آخر `undefined` می‌ماند. اسلاید ۲۷ همین را یکی از مشکلات `var` می‌داند: «declarations move to the top, causing unexpected undefined».

`let` و `const`: بالا می‌روند، ولی در TDZ

`let` و `const` هم hoist می‌شوند (موتور از همان ابتدای scope نامشان را می‌شناسد)، ولی **مقدار اولیه نمی‌گیرند**. از ابتدای scope تا لحظه‌ای که خط تعریفشان اجرا شود، در بازه‌ای به نام **Temporal Dead Zone** یا **TDZ** (منطقه‌ی مرده‌ی زمانی؛ بازه‌ای که متغیر وجود دارد ولی هنوز قابل استفاده نیست) هستند. هر دسترسی به متغیر در این بازه، خطای `ReferenceError` (خطای ارجاع) می‌دهد:

```JavaScript
console.log(score); // ReferenceError: Cannot access 'score' before initialization
let score = 10;
```

- **شروع TDZ**: ابتدای scope متغیر؛ برای `let` و `const` یعنی ابتدای همان بلوک `{ }`، بدنه‌ی تابع یا فایلی که در آن تعریف شده‌اند.
- **پایان TDZ**: لحظه‌ای که خط تعریف **اجرا** شود. حتی `let note;` بدون مقدار هم به TDZ پایان می‌دهد و متغیر را `undefined` می‌کند:

```JavaScript
let note;          // TDZ ends here, even without a value
console.log(note); // undefined
note = "saved";
console.log(note); // saved
```

چرا «Temporal» (زمانی)؟

چون TDZ به **زمان اجرا** بستگی دارد، نه به **جای** کد در فایل. تابعی که بالاتر از خط تعریف نوشته شده، اگر **بعد از** اجرای آن خط صدا زده شود، مشکلی ندارد:

```JavaScript
function show() {
  console.log(msg); // written ABOVE the declaration...
}
let msg = "hi";
show(); // hi  (...but it RUNS after the declaration line)
```

ولی اگر همین تابع **قبل از** اجرای خط تعریف صدا زده شود، هنوز در TDZ هستیم:

```JavaScript
function show() {
  console.log(msg);
}
show(); // ReferenceError: Cannot access 'msg' before initialization
let msg = "hi";
```

خطا هنگام اجرای خط ۲ (همان‌جا که `msg` خوانده می‌شود) پرتاب می‌شود و Node.js همین خط را گزارش می‌کند؛ خط ۴ فقط باعث اجرای آن شده است.

`typeof` هم از TDZ در امان نیست

`typeof` (عملگری که نوع یک مقدار را به‌صورت رشته برمی‌گرداند) برای نامی که **اصلاً تعریف نشده** خطا نمی‌دهد و `"undefined"` برمی‌گرداند. ولی برای متغیری که در TDZ است، خطا می‌دهد:

```JavaScript
console.log(typeof ghost); // "undefined" (ghost was never declared: typeof is safe)
console.log(typeof later); // ReferenceError: Cannot access 'later' before initialization
let later = 1;
```

سه پیام خطا که نباید قاطی کنی

پیام	معنی	چه زمانی؟
`ReferenceError: x is not defined`	اصلاً متغیری به نام `x` در scopeهای قابل‌دسترسی وجود ندارد	نامی که هیچ‌جا تعریف نشده
`ReferenceError: Cannot access 'x' before initialization`	`x` وجود دارد (hoist شده) ولی هنوز در TDZ است	`let` ، `const` یا `class` قبل از خط تعریف
`TypeError: f is not a function`	`f` وجود دارد ولی مقدارش تابع نیست (مثلاً `undefined` است) و آن را صدا زده‌ایم	`var f = ...` که قبل از خط تعریف صدا زده شود

متن پیام به موتور بستگی دارد: Chrome و Node.js (موتور V8) می‌نویسند `Cannot access 'x' before initialization` و Firefox می‌نویسد `can't access lexical declaration 'x' before initialization`؛ ولی **نوع خطا همیشه `ReferenceError` است.**

hoisting در هر scope جداگانه رخ می‌دهد

مرحله‌ی ایجاد فقط مخصوص کل فایل نیست: **هر بار که یک تابع صدا زده می‌شود**، بدنه‌ی آن تابع مرحله‌ی ایجاد خودش را دارد (و هر بلوک `{ }` هم برای `let` ، `const` و `class`). پس یک متغیر محلی، از **همان خط اول** تابع، متغیر هم‌نامِ بیرونی را **پنهان (shadow)** می‌کند؛ حتی در خط‌هایی که بالاتر از تعریف محلی نوشته شده‌اند.

با `var` نتیجه یک `undefined` غیرمنتظره است:

JavaScript

```
var user = "Ali";
function hello() {
  console.log(user); // undefined (the local user is hoisted to the top of hello)
  var user = "Sara";
  console.log(user); // Sara
}
hello();
console.log(user);   // Ali (the global user was never touched)
```

با `let` نتیجه خطاست، و این بهترین **اثبات** است که `let` هم hoist می‌شود:

JavaScript

```
let x = 1;
function f() {
  console.log(x); // ReferenceError: Cannot access 'x' before initialization
  let x = 2;
}
f();
```

اگر `let x` داخل `f` hoist نمی‌شد، خط ۳ باید `x` بیرونی را می‌دید و `1` چاپ می‌کرد. ولی `x` داخلی از ابتدای بدنه‌ی `f` ثبت شده، `x` بیرونی را پنهان کرده و هنوز در TDZ است؛ پس خطا می‌گیریم. پیام خطا هم همین را می‌گوید: `Cannot access 'x' before initialization`، نه `x is not defined`.

یک مثال ترکیبی از قاعده‌ی «`var` کل تابع را می‌بیند، `let` فقط بلوک خودش را»:

JavaScript

```
function demo() {
  console.log(a);        // undefined: var a belongs to the whole function
  {
    // the TDZ of b starts here (top of the block)
    let b = 2;           // the TDZ of b ends here
    var a = 1;           // var ignores the block
    console.log(b);      // 2
  }
  console.log(a);        // 1
  console.log(typeof b); // "undefined": b does not exist outside its block
}
demo();
```

توابع: declaration در برابر arrow (اسلایدهای ۳۴ تا ۳۶)

اسلاید ۳۴ درباره‌ی **function declaration** (اعلان تابع؛ تعریف تابع با کلمه‌ی کلیدی `function` و یک نام، به‌صورت یک دستور مستقل) می‌گوید: «Can be called before or after it's defined (hoisted)» و زیرش تأکید می‌کند: «Remember, this can be hoisted!». چنین تابعی در مرحله‌ی ایجاد **همراه با بدنه‌اش** ساخته می‌شود، پس قبل از خط تعریف هم قابل صدا زدن است:

fn.js

```
console.log(add(2, 3)); // 5 (called BEFORE the declaration: works)

function add(a, b) {
  return a + b;
}

console.log(add(2, 3)); // 5 (called after: works too)
```

اسلاید ۳۵ درباره‌ی **arrow function** (تابع پیکانی ES6 با نحو `=>`) می‌گوید: «Not hoisted (must be defined before use)» و «Remember, this **cannot** be hoisted!». دلیلش این است که arrow function خودش اعلان نیست؛ یک **مقدار** است که در یک متغیر ریخته می‌شود. پس رفتارش دقیقاً رفتار **همان متغیر** است و تابع فقط وقتی در متغیر قرار می‌گیرد که خط تعریف اجرا شود. کد اسلاید ۳۵، اگر صدا زدن را بالاتر ببریم:

fn.js

```
console.log(multiply(4, 5)); // ReferenceError: Cannot access 'multiply' before initialization

const multiply = (a, b) => {
  return a * b;
};
```

اگر همان arrow function را در `var` بریزیم، پیام خطا عوض می‌شود: متغیر hoist شده و `undefined` است، و صدا زدن `undefined` یعنی `TypeError` :

JavaScript

```
console.log(typeof mult2); // "undefined"
mult2(4, 5);               // TypeError: mult2 is not a function
var mult2 = (a, b) => a * b;
```

function expression (عبارت تابعی؛ تابعی که به‌صورت مقدار در یک متغیر ریخته می‌شود، مثل `const f = function () {};`) هم دقیقاً همین‌طور است: hoisting آن را کلمه‌ی کلیدی متغیر (`var`، `let` یا `const`) تعیین می‌کند، نه کلمه‌ی `function` .

خلاصه‌ی اسلاید ۳۶ (Method I vs. Method II): «Functions are **hoisted** (usable before definition)» و «Arrows are **not hoisted**». یک کاربرد عملی hoisting توابع این است که می‌توانی کد اصلی را بالای فایل بنویسی و تابع‌های کمکی را پایین آن:

JavaScript

```
// main code on top
console.log(formatPrice(1200)); // 1200 Toman

// helper functions at the bottom (function declarations are hoisted)
function formatPrice(n) {
  return n + " Toman";
}
```

hoisting توابع داخل توابع دیگر هم رخ می‌دهد؛ حتی تابعی که **بعد از** `return` نوشته شده، در مرحله‌ی ایجاد ساخته می‌شود:

```JavaScript
function outer() {
  return inner(); // works: inner is hoisted to the top of outer
  function inner() {
    return "inner works";
  }
}
console.log(outer()); // inner works
```

`class` : «تا حدی» hoist می‌شود

اسلاید ۳۸ می‌گوید hoisting برای `class` به‌صورت **partially** (ناقص) کار می‌کند: نام کلاس در مرحله‌ی ایجاد ثبت می‌شود، ولی مثل `let` و `const` تا خط تعریفش در TDZ می‌ماند. پس برخلاف function declaration، **نمی‌توانی قبل از خط تعریف از کلاس استفاده کنی** (کلاس‌ها را در درس «کلاس‌ها» کامل می‌خوانیم):

```JavaScript
const p = new Person("Ali"); // ReferenceError: Cannot access 'Person' before initialization

class Person {
  constructor(name) {
    this.name = name;
  }
}
```

راه درست این است که اول کلاس را تعریف کنی و بعد با `new` از آن شیء بسازی.

> **بیشتر بدانیم: وقتی چند اعلان هم‌نام داریم**
>
> - اگر **دو function declaration** هم‌نام باشند، هر دو در مرحله‌ی ایجاد پردازش می‌شوند و **دومی جای اولی را می‌گیرد**؛ پس حتی صدا زدن در بالای فایل هم نسخه‌ی دوم را اجرا می‌کند.
> - اگر یک `var` و یک function declaration هم‌نام باشند، در مرحله‌ی ایجاد **تابع برنده است** (اعلان `var` نامی را که از قبل وجود دارد `undefined` نمی‌کند). ولی در مرحله‌ی اجرا، وقتی به خط مقداردهی `var` برسیم، مقدار جدید جای تابع را می‌گیرد:
>
> ```JavaScript
> console.log(typeof value); // "function" (function declarations win in the creation phase)
> var value = 10;
> function value() {}
> console.log(typeof value); // "number" (the assignment ran on line 2)
> ```
>
> با `let`، `const` یا `class` اصلاً نمی‌شود نام تکراری تعریف کرد: `let value = 10; function value() {}` خطای `SyntaxError: Identifier 'value' has already been declared` می‌دهد.

چطور از دردسرهای hoisting دور بمانیم؟

اسلاید ۳۸ هشدار می‌دهد: «Makes code order confusing if misused». چند عادت ساده:

- متغیرها را با `const` (یا در صورت نیاز `let`) و **در بالای scope** تعریف کن و **قبل از استفاده** مقدار بده.

- از `var` استفاده نکن (درس «مشکلات var»)؛ `undefined` بی‌صدای آن باگ را پنهان می‌کند، ولی `ReferenceError` در `let` / `const` فوراً خبرت می‌کند.
- arrow function و function expression را **قبل از** اولین صدا زدن تعریف کن.
- به hoisting توابع فقط آگاهانه تکیه کن (مثل تابع‌های کمکی پایین فایل)، نه برای خواندن متغیرهایی که هنوز مقدار نگرفته‌اند.

نکته

- **فقط اعلان hoist می‌شود، نه مقداردهی.** `console.log(x); var x = 5;` چاپ می‌کند `undefined`، نه `5`.
- قبل از خط تعریف: `var` ← `undefined`؛ `let`، `const` و `class` ← `ReferenceError` (TDZ)؛ function declaration ← کامل و قابل صدا زدن.
- arrow function و function expression از متغیرشان پیروی می‌کنند: با `let` / `const` ← `ReferenceError` و با `var` ← `TypeError: ... is not a function`.
- TDZ از ابتدای scope شروع می‌شود و وقتی خط تعریف **اجرا** شود تمام می‌شود؛ زمان اجرا مهم است، نه جای کد در فایل.
- hoisting در هر scope جداگانه است: `var` به بالای تابع می‌رود و `let` / `const` / `class` به بالای بلوک خودشان.

دام تستی

- `typeof` فقط برای نامِ **تعریف‌نشده** امن است (`"undefined"`)؛ روی متغیرِ در TDZ خطای `ReferenceError` می‌دهد.
- متغیر محلیِ `let` از **اولین خط** تابع، متغیر هم‌نام بیرونی را پنهان می‌کند: `let x = 1; function f() { console.log(x); let x = 2; } f();` خطا می‌دهد، نه `1`. با `var` همین الگو `undefined` چاپ می‌کند، نه مقدار بیرونی.
- `var` داخل `if (false) { ... }` هم hoist می‌شود؛ خواندنش `undefined` است، نه `ReferenceError`.
- `let count = count + 1;` خطا می‌دهد: TDZ تا **پایان** اجرای خط تعریف ادامه دارد و سمت راست قبل از پایان TDZ خوانده می‌شود.
- پیام‌های `Cannot access 'x' before initialization` و `x is not defined` هر دو از نوع `ReferenceError` هستند، ولی صدا زدن متغیری که `undefined` است (`var f` قبل از تعریف) `TypeError` است.
- `var n;` بدون مقدار در مرحله‌ی اجرا هیچ کاری نمی‌کند؛ مقداری را که قبلاً در `n` ریخته شده پاک نمی‌کند.

جمع‌بندی

- hoisting یعنی ثبت اعلان‌ها در ابتدای scope قبل از اجرا؛ کد طوری اجرا می‌شود که انگار اعلان‌ها بالا نوشته شده‌اند (اسلاید ۳۷).
- طبق اسلاید ۳۸ برای `var`، `function` و `class` (ناقص) کار می‌کند؛ متغیرها hoist می‌شوند ولی مقداردهی نمی‌شوند.
- `var` ← `undefined`؛ `let` / `const` / `class` ← TDZ و `ReferenceError`؛ function declaration ← قابل صدا زدن پیش از تعریف؛ arrow function ← hoist نمی‌شود (مثل متغیرش رفتار می‌کند).
- TDZ: از ابتدای scope تا اجرای خط تعریف؛ حتی `typeof` هم در آن خطا می‌دهد.

تست‌های تمرینی

۳۲ تست · شماره‌ی ۴۰۱ تا ۴۳۲

۴۰۱ آسان

در میم اسلاید ۳۷، پیتر پارکر با جمله‌ی «Using variable before declaration» سراغ دکتر استرنج (با برچسب Javascript) می‌رود و جواب می‌شنود: «That feels weird, but I'll allow it». این میم به کدام رفتار JavaScript اشاره دارد؟

۱) Type coercion (تبدیل خودکار نوع داده‌ها)

۲) Hoisting (بالا بردن اعلان‌ها به ابتدای scope)

۳) Closure (به خاطر سپردن متغیرهای بیرونی توسط تابع)

۴) Garbage collection (پاک‌سازی خودکار حافظه)

۴۰۲ آسان

کدام گزینه تعریف درست **hoisting** در JavaScript است؟

۱) قبل از اجرا، کل خط‌های تعریف، یعنی هم اعلان و هم مقداردهی، به ابتدای فایل منتقل می‌شوند.

۲) هر متغیری که قبل از تعریفش استفاده شود، خودبه‌خود با مقدار `null` ساخته می‌شود.

۳) قبل از اجرای کد، اعلان‌ها (declarations) در ابتدای scope خودشان ثبت می‌شوند؛ انگار به بالای scope منتقل شده‌اند.

۴) موتور JavaScript همه‌ی `var` ها را خودکار به `let` تبدیل می‌کند تا خطایی رخ ندهد.

۴۰۳ آسان

طبق اسلاید ۳۸، hoisting برای کدام اعلان‌ها کار می‌کند؟

۱) فقط `function` ؛ متغیرها هرگز hoist نمی‌شوند.

۲) فقط `let` و `const` .

۳) `var` ، `function` و `class` (به‌صورت ناقص).

۴) فقط `var` ؛ توابع و کلاس‌ها هرگز hoist نمی‌شوند.

۴۰۴ آسان

در جای خالی جمله‌ی اسلاید ۳۸ کدام گزینه قرار می‌گیرد؟

«متغیرها hoist می‌شوند، ولی _____ نمی‌شوند.» (Variables are hoisted but not _____)

۱) اعلان (declared)

۲) مقداردهی اولیه (initialized)

۳) در scope ثبت (registered in the scope)

۴) برای موتور شناخته‌شده (known to the engine)

۴۰۵ آسان

خروجی کد زیر کدام است؟

```javascript
console.log(x);
var x = 5;
console.log(x);
```

۱) `undefined` `5`
۲) `5` `5`
۳) `Uncaught ReferenceError`
۴) `null` `5`

۴۰۶ متوسط

یک فایل این سه اعلان را دارد: `var a = 5;` ، `let b = 6;` و `function c() {}` . در پایان **مرحله‌ی ایجاد** (creation phase)، یعنی درست قبل از اجرای اولین خط، وضعیت این سه نام کدام است؟

۱) `a` برابر `5` ، `b` برابر `6` و `c` تابع کامل
۲) هر سه برابر `undefined`
۳) `a` برابر `undefined` ، `b` بدون مقدار اولیه (در TDZ) و `c` تابع کامل
۴) `a` در TDZ، `b` برابر `undefined` و `c` فقط یک نام بدون بدنه

۴۰۷ متوسط

خروجی کد زیر کدام است؟

```javascript
console.log(i);
for (var i = 0; i < 3; i++) {}
console.log(i);
```

۱) `undefined` `3`
۲) `Uncaught ReferenceError`
۳) `undefined` `2`
۴) `0` `3`

۴۰۸ سخت

خروجی کد زیر کدام است؟

```javascript
function check(n) {
  if (n > 10) {
    var label = "big";
  }
  return label;
}
console.log(check(50), check(5));
```

۱) `Uncaught ReferenceError`
۲) `big big`
۳) `big null`
۴) `big undefined`

۴۰۹ آسان

در جای خالی کدام گزینه قرار می‌گیرد؟

«به بازه‌ی زمانی از ابتدای scope تا اجرای خط تعریفِ یک متغیر `let` یا `const`، که در آن هر دسترسی به متغیر خطای `ReferenceError` می‌دهد، _____ می‌گویند.»

۱) Hoisting
۲) Global Scope
۳) Closure
۴) Temporal Dead Zone (TDZ)

۴۱۰ آسان

اجرای کدام قطعه‌کد **خطا** می‌دهد؟

۱) `console.log(b); let b = 1;`
۲) `console.log(a); var a = 1;`
۳) `hello(); function hello() {}`
۴) `var c; console.log(c);`

۴۱۱ متوسط

کدام عبارت درباره‌ی TDZ **درست** است؟

۱) TDZ از ابتدای scope متغیر شروع می‌شود و وقتی خط تعریف آن اجرا شود تمام می‌شود.
۲) TDZ فقط مخصوص متغیرهای `var` است.
۳) TDZ از خط تعریف متغیر شروع می‌شود و تا پایان بلوک ادامه دارد.
۴) خواندن متغیری که در TDZ است، مقدار `undefined` برمی‌گرداند.

۴۱۲ متوسط

کدام خط کد زیر خطا می‌دهد؟

```javascript
let price = 100;
let total = price + tax;
let tax = 9;
console.log(total);
```

۱) خط ۲
۲) خط ۳
۳) خط ۴
۴) هیچ خطایی رخ نمی‌دهد

۴۱۳ سخت

خروجی کد زیر کدام است؟

```javascript
function greet() {
  console.log("Hi " + userName);
}
let userName = "Sara";
greet();
```

۱) `Hi undefined`
۲) `Uncaught ReferenceError`
۳) `Hi Sara`
۴) `Hi`

۴۱۴ سخت

خروجی کد زیر کدام است؟

```javascript
let later = "outer";
console.log(typeof notDeclared);
{
  console.log(typeof later);
  let later = 5;
}
```

۱) `undefined` `string`
۲) `undefined` `Uncaught ReferenceError`
۳) `Uncaught ReferenceError`
۴) `undefined` `number`

۴۱۵ متوسط

کدام عبارت درباره‌ی پیام‌های خطای زیر **درست** است؟

`ReferenceError: x is not defined` و `ReferenceError: Cannot access 'x' before initialization` و `TypeError: x is not a function`

۱) `x is not defined` یعنی `x` در scope وجود دارد ولی هنوز در TDZ است.
۲) `Cannot access 'x' before initialization` یعنی `x` hoist شده و وجود دارد ولی هنوز در TDZ است؛ در حالی که `x is not defined` یعنی اصلاً متغیری با این نام وجود ندارد.
۳) `TypeError: x is not a function` فقط وقتی رخ می‌دهد که `x` با `const` تعریف شده باشد.
۴) این سه پیام در واقع یکی‌اند و فقط مرورگرهای مختلف متن متفاوتی برایشان نشان می‌دهند.

۴۱۶ سخت

کدام خط کد زیر خطا می‌دهد؟

```JavaScript
let count = 5;
{
  let count = count + 1;
  console.log(count);
}
console.log(count);
```

۱) هیچ خطایی رخ نمی‌دهد
۲) خط ۴
۳) خط ۳
۴) خط ۶

۴۱۷ آسان

خروجی کد زیر کدام است؟

```JavaScript
console.log(add(2, 3));

function add(a, b) {
  return a + b;
}
```

۱) `undefined`
۲) `Uncaught ReferenceError`
۳) `Uncaught TypeError`
۴) `5`

۴۱۸ آسان

کد زیر (کد اسلاید ۳۵ که صدا زدن تابع در آن بالاتر آمده) در Node.js اجرا می‌شود. کدام خط خطا می‌دهد؟

```JavaScript
console.log("Start");
console.log(multiply(4, 5));
const multiply = (a, b) => {
  return a * b;
};
console.log("End");
```

۱) هیچ خطایی رخ نمی‌دهد
۲) خط ۲
۳) خط ۳
۴) خط ۶

۴۱۹ متوسط

خروجی کد زیر کدام است؟

```JavaScript
console.log(typeof greet);
greet();
var greet = function () {
  console.log("Hello");
};
```

۱) `function` `Hello`

۲) `undefined` `Uncaught ReferenceError`

۳) `Uncaught ReferenceError`

۴) `undefined` `Uncaught TypeError`

۴۲۰ متوسط

اجرای کدام قطعه‌کد **بدون خطا** است؟

۱) `sayHi(); const sayHi = () => console.log("hi");`

۲) `sayHi(); var sayHi = () => console.log("hi");`

۳) `sayHi(); let sayHi = function () { console.log("hi"); };`

۴) `sayHi(); function sayHi() { console.log("hi"); }`

۴۲۱ متوسط

خروجی کد زیر کدام است؟

```JavaScript
console.log(typeof a, typeof b, typeof c);
var a = function () {};
function b() {}
var c = () => {};
```

۱) `undefined function undefined`

۲) `function function function`

۳) `undefined function function`

۴) `Uncaught ReferenceError`

۴۲۲ سخت

خروجی کد زیر کدام است؟

```javascript
function getMessage() {
  return format("hi");
  function format(s) {
    return s + suffix;
  }
  var suffix = "!";
}
console.log(getMessage());
```

۱) `hi!`
۲) `hiundefined`
۳) `Uncaught ReferenceError`
۴) `Uncaught TypeError`

۴۲۳ متوسط

کدام خط کد زیر خطا می‌دهد؟

```javascript
1 console.log("Creating a pet...");
2 const pet = new Animal("cat");
3 class Animal {
4   constructor(kind) {
5     this.kind = kind;
6   }
7 }
8 console.log(pet.kind);
```

۱) هیچ خطایی رخ نمی‌دهد
۲) خط ۲
۳) خط ۳
۴) خط ۸

۴۲۴ متوسط

چرا اسلاید ۳۸ می‌گوید hoisting برای `class` به‌صورت «partially» (ناقص) کار می‌کند؟

۱) چون فقط constructor کلاس hoist می‌شود و متدهایش نه.
۲) چون کلاس‌ها فقط در مرورگر hoist می‌شوند، نه در Node.js.
۳) چون قبل از خط تعریف، کلاس مثل `var` مقدار `undefined` دارد.
۴) چون نام کلاس از ابتدای scope ثبت می‌شود، ولی تا اجرای خط تعریف در TDZ است و نمی‌توان از آن استفاده کرد.

۴۲۵ متوسط

کدام خط کد زیر خطا می‌دهد؟

```JavaScript
let x = 1;
function f() {
  console.log(x);
  let x = 2;
}
f();
```

۱) هیچ خطایی رخ نمی‌دهد
۲) خط ۱
۳) خط ۳
۴) خط ۴

۴۲۶ سخت

خروجی کد زیر کدام است؟

```JavaScript
var a = 1;
function test() {
  console.log(a);
  if (true) {
    var a = 2;
  }
  console.log(a);
}
test();
console.log(a);
```

۱) `1` `2` `1`
۲) `1` `2` `2`
۳) `undefined` `2` `2`
۴) `undefined` `2` `1`

۴۲۷ سخت

خروجی کد زیر کدام است؟

```JavaScript
var result = calc(3);
console.log(result);
function calc(n) {
  return n * factor;
}
var factor = 2;
console.log(calc(3));
```

۱) `6` `6`
۲) `NaN` `6`
۳) `Uncaught ReferenceError`
۴) `undefined` `6`

۴۲۸ سخت

خروجی کد زیر کدام است؟

```javascript
greet();
function greet() {
  console.log("first");
}
greet();
function greet() {
  console.log("second");
}
greet();
```

۱) `second` `second` `second`

۲) `first` `first` `second`

۳) `first` `second` `second`

۴) `Uncaught SyntaxError`

۴۲۹ سخت

خروجی کد زیر کدام است؟

```javascript
var item;
console.log(typeof item);
function item() {}
item = "pen";
console.log(typeof item);
```

۱) `undefined` `string`

۲) `undefined` `function`

۳) `function` `string`

۴) `function` `function`

۴۳۰ سخت

کدام قطعه‌کد عدد `5` را چاپ می‌کند؟

۱) `console.log(n); var n = 5;`

۲) `var n; console.log(n); n = 5;`

۳) `n = 5; console.log(n); var n;`

۴) `console.log(n); let n = 5;`

۴۳۱ متوسط

کدام خط کد زیر خطا می‌دهد؟

```javascript
console.log(a);
console.log(b());
console.log(c);
var a = 1;
function b() {
  return "b";
}
let c = 3;
```

۱) هیچ خطایی رخ نمی‌دهد
۲) خط ۱
۳) خط ۲
۴) خط ۳

۴۳۲ آسان

اسلاید ۳۸ هشدار می‌دهد که hoisting اگر بد استفاده شود، ترتیب کد را گیج‌کننده می‌کند. کدام عادت بیشترین کمک را برای پرهیز از این مشکل می‌کند؟

۱) متغیرها را با `let` / `const` در بالای scope تعریف کنیم و هر چیزی را پیش از استفاده تعریف کنیم.
۲) همه‌ی متغیرها را با `var` تعریف کنیم تا هیچ‌وقت `ReferenceError` نگیریم.
۳) همه‌ی تعریف‌ها را عمداً پایین فایل بنویسیم، چون به هر حال hoist می‌شوند.
۴) قبل از خواندن هر متغیر، آن را با `typeof` بررسی کنیم؛ چون `typeof` هیچ‌وقت خطا نمی‌دهد.

۱-۱۹ Closure (بستار)

منبع: Introduction to JS، اسلاید ۲۸

درسنامه

تابعی با کوله‌پشتی

فرض کن تابعی داخل تابع دیگری «متولد» می‌شود. موقع تولد، یک **کوله‌پشتی** به او می‌دهند که داخلش متغیرهای اطرافش (متغیرهای تابع والد) قرار دارد. حالا این تابع هر جا برود (از تابع والد بیرون فرستاده شود، به `setTimeout` داده شود یا در یک آرایه ذخیره شود)، کوله‌پشتی‌اش را هم با خودش می‌برد و هر وقت لازم داشت از متغیرهای داخلش استفاده می‌کند؛ حتی اگر کار تابع والد مدت‌ها پیش تمام شده باشد. به این «تابع به‌همراه کوله‌پشتی‌اش» **closure** (بستار؛ تابعی که متغیرهای محیطِ تعریفش را به خاطر می‌سپارد) می‌گوییم.

پیش‌نیاز: lexical scope (محدوده‌ی واژگانی)

در درس‌های قبل دیدیم که هر متغیر فقط داخل scope (محدوده‌ی) خودش در دسترس است. وقتی تابع‌ها داخل هم نوشته می‌شوند، یک قاعده‌ی ساده حاکم است: **هر تابع به متغیرهای خودش، متغیرهای تابع‌هایی که داخلشان نوشته شده و متغیرهای سراسری (global) دسترسی دارد**؛ ولی برعکسش ممکن نیست و تابع بیرونی متغیرهای تابع درونی را نمی‌بیند.

JavaScript

```
const greeting = "Hello";             // global

function outer() {
  const name = "Sara";                // local to outer
  function inner() {
    const mark = "!";                 // local to inner
    console.log(greeting + ", " + name + mark); // inner sees all three
  }
  inner();
  console.log(typeof mark);           // "undefined": outer cannot see inside inner
}

outer();
// Hello, Sara!
// undefined
```

وقتی `inner` به نام `name` می‌رسد، اول بین متغیرهای خودش می‌گردد؛ پیدا نمی‌کند، پس یک لایه بیرون‌تر (`outer`) را نگاه می‌کند و پیدا می‌کند. برای `greeting` تا لایه‌ی سراسری بالا می‌رود. به این زنجیره‌ی جست‌وجو **scope chain** (زنجیره‌ی scopeها) می‌گویند.

کلمه‌ی **lexical** (واژگانی؛ یعنی مربوط به متن کد) نکته‌ی اصلی را می‌گوید: **جایی که تابع نوشته شده** تعیین می‌کند چه چیزهایی را می‌بیند، نه جایی که صدا زده می‌شود:

JavaScript

```
const x = "global";

function printX() {
  console.log(x);
}

function run() {
  const x = "local";
  printX(); // global (printX was WRITTEN at the top level, so it sees the global x)
}

run();
```

`printX` در سطح بالای فایل نوشته شده، پس فقط `x` سراسری را می‌شناسد؛ اینکه داخل `run` صدا زده شده، هیچ اهمیتی ندارد.

Closure چیست؟ (اسلاید ۲۸)

اسلاید ۲۸ این‌طور تعریف می‌کند:

- **A closure is when a function keeps access to its parent's variables** (A function that remembers variables from its outer scope): closure یعنی تابعی که دسترسی به متغیرهای تابع والدش را **نگه می‌دارد**؛ تابعی که متغیرهای scope بیرونی‌اش را «به خاطر می‌سپارد».
- **Keeps access even after the outer function ends**: این دسترسی **حتی بعد از تمام شدن تابع بیرونی** باقی می‌ماند.
- **Used for encapsulation, state, and callbacks**: برای کپسوله‌سازی (پنهان کردن داده)، نگه‌داری وضعیت (state) و callbackها استفاده می‌شود.
- **Common in JS and other functional languages**: در JavaScript و دیگر زبان‌های تابعی (functional) رایج است.
- **BEWARE OF PITFALL**: closureها وقتی با `var` ترکیب شوند، می‌توانند مشکل‌ساز شوند (آخر همین درس).

بند دوم مهم‌ترین بخش است. معمولاً وقتی اجرای یک تابع تمام می‌شود، متغیرهای محلی‌اش از بین می‌روند. ولی اگر تابعی که داخل آن ساخته شده هنوز زنده باشد و به آن متغیرها نیاز داشته باشد، JavaScript آن‌ها را نگه می‌دارد.

کد اسلاید ۲۸: شمارنده

sample.js

```
function counter() {
  let count = 0;
  return function() {
    count++;
    return count;
  };
}

const c = counter();
console.log(c()); // 1
console.log(c()); // 2
```

قدم‌به‌قدم:

1. `const c = counter();` : تابع `counter` اجرا می‌شود، متغیر `count` را با مقدار `0` می‌سازد، یک تابع بی‌نام (anonymous) می‌سازد و **همان تابع** را برمی‌گرداند. حالا کار `counter` تمام شده است.
2. `c` حالا **تابع درونی** است، نه یک عدد! تابعی که در کوله‌پشتی‌اش متغیر `count` را دارد (`typeof c` برابر `"function"` است).
3. اولین `c()` : `count` از `0` به `1` می‌رسد و `1` برگردانده می‌شود.
4. دومین `c()` : **همان** `count` از `1` به `2` می‌رسد و `2` برگردانده می‌شود. شمارنده یادش مانده است!

`count` نه سراسری است و نه با هر `c()` از نو ساخته می‌شود؛ فقط در کوله‌پشتی `c` زندگی می‌کند و هیچ کد دیگری مستقیم به آن دسترسی ندارد. اگر بیرون از `counter` بنویسی `console.log(count)` ، خطای `ReferenceError: count is not defined` می‌گیری.

هر بار صدا زدن، یک closure تازه و مستقل

هر بار که `counter()` را صدا بزنی، یک `count` **جدید** ساخته می‌شود و تابع جدیدی با کوله‌پشتی جدید برمی‌گردد. دو شمارنده هیچ وضعیتی را با هم شریک نیستند (با همان تابع `counter` اسلاید):

JavaScript

```
const c1 = counter();
const c2 = counter();
console.log(c1()); // 1
console.log(c1()); // 2
console.log(c2()); // 1 (c2 has its own count)
console.log(c1()); // 3
```

یک تشبیه ساده

`counter` مثل دستگاهی است که «شمارنده‌ی دستی» تولید می‌کند. هر بار دکمه‌ی دستگاه را بزنی (`counter()`)، یک شمارنده‌ی نو با عدد صفر تحویل می‌گیری. اگر روی شمارنده‌ی اول ده بار کلیک کنی، عدد شمارنده‌ی دوم تکان نمی‌خورد.

closure خودِ متغیر را نگه می‌دارد، نه عکسی از مقدارش

کوله‌پشتی یک **عکس (snapshot)** از مقدارها نیست؛ خودِ متغیرها داخلش هستند. پس اگر متغیر بعداً تغییر کند، closure **مقدار جدید** را می‌بیند:

JavaScript

```
let color = "red";
const showColor = () => console.log(color);
color = "blue";
showColor(); // blue (the closure reads the CURRENT value of color)
```

همین‌طور تابع درونی می‌تواند متغیر بیرونی را **تغییر** دهد (مثل `count++` در شمارنده) و این تغییر را همه‌ی توابعی که آن متغیر را در کوله‌پشتی دارند می‌بینند.

پارامترها هم متغیرند

پارامترهای یک تابع، متغیرهای محلی همان تابع‌اند. وقتی تابع را صدا می‌زنی، **مقدار فعلی** آرگومان در پارامتر ریخته می‌شود و closure به همان پارامتر وصل است، نه به متغیری که بیرون از تابع داشتی:

```JavaScript
function remember(value) {
  return () => value; // captures the PARAMETER variable "value"
}

let n = 1;
const getN = remember(n); // the current value of n (1) is copied into the parameter
n = 99;
console.log(getN()); // 1
```

کاربرد ۱: encapsulation (کپسوله‌سازی؛ پنهان کردن داده)

با closure می‌توانیم متغیری بسازیم که **فقط** از طریق چند تابع مشخص قابل خواندن و تغییر باشد؛ یعنی یک **متغیر خصوصی (private)**. در مثال زیر، `createCounter` یک شیء (object؛ مجموعه‌ای از جفت‌های «کلید: مقدار» که در درس «اشیاء» کامل می‌خوانیم) با دو تابع برمی‌گرداند:

```JavaScript
function createCounter() {
  let count = 0; // private: only the two functions below can reach it

  return {
    increment: () => {
      count++;
    },
    getValue: () => count,
  };
}

const counter = createCounter();
counter.increment();
counter.increment();
console.log(counter.getValue()); // 2
console.log(counter.count);      // undefined (no direct access to count)
```

هیچ کدی بیرون از `createCounter` نمی‌تواند `count` را مستقیم بخواند یا خراب کند (مثلاً منفی‌اش کند)؛ فقط `increment` و `getValue` کلید کوله‌پشتی را دارند. دقت کن که هر دو تابع در **یک** اجرای `createCounter` ساخته شده‌اند، پس **یک** `count` **مشترک** دارند.

کاربرد ۲: state (نگه‌داری وضعیت) و function factory

closure به تابع «حافظه» می‌دهد. یک الگوی رایج، **function factory** (کارخانه‌ی تابع؛ تابعی که تابع می‌سازد و برمی‌گرداند) است:

```JavaScript
function makeMultiplier(factor) {
  return (n) => n * factor; // remembers its own factor
}

const double = makeMultiplier(2);
const triple = makeMultiplier(3);
console.log(double(5)); // 10
console.log(triple(5)); // 15
```

`double` و `triple` از یک کارخانه آمده‌اند، ولی چون هر صدا زدن `makeMultiplier` یک closure جدید می‌سازد، هر کدام `factor` مخصوص خودش را دارد.

کاربرد ۳: callbackها

callback (تابعی که به تابع دیگری داده می‌شود تا بعداً صدا زده شود) تقریباً همیشه یک closure است. تابع `setTimeout(fn, ms)` تابع `fn` را **بعداً** اجرا می‌کند (حداقل `ms` میلی‌ثانیه بعد؛ جزئیاتش را در درس‌های بخش ناهمگام همین فصل می‌خوانیم). فعلاً همین یک نکته را بدان: callback حتی با تأخیر `0` هم فقط **بعد از تمام شدن کل کد همگام (synchronous) فعلی** اجرا می‌شود:

```JavaScript
setTimeout(() => console.log("timer"), 0);
console.log("sync");
// sync
// timer
```

وقتی callback بالاخره اجرا می‌شود، تابعی که آن را ساخته مدت‌هاست تمام شده؛ ولی callback به لطف closure هنوز متغیرهای آن را دارد:

```JavaScript
function greetLater(name) {
  setTimeout(() => {
    console.log("Hello, " + name); // the callback still remembers name
  }, 1000);
  console.log("greetLater finished");
}

greetLater("Ali");
// greetLater finished
// (one second later) Hello, Ali
```

«رایج در JS و زبان‌های تابعی»

closure در JavaScript کاملاً طبیعی است، چون توابع **first-class** هستند (یکی از ویژگی‌های JavaScript در اسلاید ۹): می‌شود آن‌ها را در متغیر ریخت، به تابع دیگر داد و از تابع برگرداند. زبان‌های **functional** (تابعی) مثل Haskell و Lisp و بیشتر زبان‌های مدرن (مثل Kotlin، Python و Swift) هم closure دارند.

> **بیشتر بدانیم: closure و حافظه**
>
> JavaScript حافظه را خودکار مدیریت می‌کند (garbage collection؛ یکی دیگر از ویژگی‌های اسلاید ۹). متغیری که در کوله‌پشتی یک closure زنده است، پاک نمی‌شود؛ وقتی دیگر هیچ ارجاعی به آن closure نماند، متغیرهایش هم قابل پاک شدن می‌شوند.

BEWARE OF PITFALL: closure و `var` در حلقه

این معروف‌ترین تست closure است:

```JavaScript
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}
// 3
// 3
// 3
```

انتظار داشتی `0` ، `1` ، `2` ببینی؟ دو دلیل دست به دست هم می‌دهند:

1. `var` **فقط یک متغیر می‌سازد.** `var` بلوک حلقه را نمی‌شناسد (function-scoped است)، پس در کل اجرای حلقه فقط **یک** `i` وجود دارد و هر سه callback **همان یک** `i` را در کوله‌پشتی دارند (یادت باشد: closure خودِ متغیر را نگه می‌دارد، نه عکسی از مقدارش).
2. **callbackها بعد از تمام شدن حلقه اجرا می‌شوند.** `setTimeout` فقط زمان‌بندی می‌کند؛ اجرای واقعی بعد از تمام شدن کد همگام است.

لحظه	مقدار `i`	چه اتفاقی می‌افتد؟
تکرار اول	`0`	callback اول زمان‌بندی می‌شود (اجرا نمی‌شود)
تکرار دوم	`1`	callback دوم زمان‌بندی می‌شود
تکرار سوم	`2`	callback سوم زمان‌بندی می‌شود
بعد از آخرین `i++`	`3`	شرط `3 < 3` نادرست است و حلقه تمام می‌شود
بعد از پایان کد همگام	`3`	هر سه callback اجرا می‌شوند و همان `i` مشترک را می‌خوانند: `3` ، `3` ، `3`

راه‌حل ۱: `let`

```JavaScript
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}
// 0
// 1
// 2
```

وقتی `let` در **سرآیند (header)** حلقه‌ی `for` باشد، JavaScript برای **هر تکرار یک binding (نسخه‌ی) جدید** از `i` می‌سازد که با مقدار همان تکرار شروع می‌شود. پس هر callback، `i` مخصوص تکرار خودش را در کوله‌پشتی دارد. این ساده‌ترین و توصیه‌شده‌ترین راه‌حل است.

راه‌حل ۲: IIFE (روش قدیمی، پیش از `let`)

IIFE (Immediately Invoked Function Expression؛ تابعی که همان لحظه‌ی تعریف صدا زده می‌شود، مثل پنل سوم میم اسلاید ۳۶) برای هر تکرار یک scope تازه می‌سازد:

```JavaScript
for (var i = 0; i < 3; i++) {
  (function (j) {
    setTimeout(() => console.log(j), 0); // j belongs to this IIFE call only
  })(i);
}
// 0
// 1
// 2
```

در هر تکرار، IIFE با مقدار فعلی `i` صدا زده می‌شود و آن مقدار در پارامتر `j` ریخته می‌شود. چون هر بار صدا زدن تابع، متغیرهای محلی جدید خودش را دارد، سه `j` جداگانه با مقدارهای `0`، `1` و `2` ساخته می‌شود.

همین دام، بدون `setTimeout`: آرایه‌ای از توابع

مشکل از خود `setTimeout` نیست؛ از «یک متغیر مشترک + اجرای دیرتر» است. اینجا توابع در یک آرایه ذخیره می‌شوند (`push` یک عنصر به انتهای آرایه اضافه می‌کند؛ درس «آرایه‌ها») و بعد از تمام شدن حلقه صدا زده می‌شوند:

```JavaScript
var fns = [];
for (var i = 0; i < 3; i++) {
  fns.push(function () {
    return i;
  });
}
console.log(fns[0](), fns[1](), fns[2]()); // 3 3 3
```

با `for (let i = 0; ...)` همین کد `0 1 2` چاپ می‌کند.

چه چیزهایی مشکل را حل نمی‌کنند؟

کپی کردن `i` در یک `var` دیگر فایده‌ای ندارد، چون `var j` هم فقط **یک** متغیر مشترک است:

```JavaScript
for (var i = 0; i < 3; i++) {
  var j = i; // still ONE shared j (var is function-scoped)
  setTimeout(() => console.log(j), 0);
}
// 2
// 2
// 2
```

`let` هم فقط وقتی binding جدید می‌سازد که **در سرآیند حلقه** تعریف شود؛ `let` بیرون از حلقه یعنی باز هم یک متغیر مشترک:

```JavaScript
let i; // ONE binding, declared outside the loop header
for (i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}
// 3
// 3
// 3
```

اما `let copy = i;` **داخل** بدنه‌ی حلقه مشکل را حل می‌کند، چون `let` در هر تکرار یک متغیر جدید در بلوک بدنه می‌سازد:

```JavaScript
for (var i = 0; i < 3; i++) {
  let copy = i; // a NEW copy in every iteration (block-scoped)
  setTimeout(() => console.log(copy), 0);
}
// 0
// 1
// 2
```

نکته

- closure = تابع + متغیرهای محیطی که در آن **نوشته** شده (lexical scope)؛ حتی بعد از تمام شدن تابع بیرونی.
- closure به **متغیر** وصل است، نه به مقدار لحظه‌ی ساخته شدن؛ تغییرات بعدی دیده می‌شوند.
- هر بار صدا زدن تابع بیرونی، متغیرهای جدید و closure مستقل می‌سازد؛ ولی توابعی که در **یک** اجرا ساخته شده‌اند، متغیرها را با هم شریک‌اند.
- برای ساختن متغیر خصوصی، آن را داخل یک تابع تعریف کن و فقط توابعی را که به آن نیاز دارند برگردان.
- در حلقه‌ها همیشه از `let` استفاده کن.

دام تستی

- حلقه‌ی `for (var i ...)` با `setTimeout` یا آرایه‌ای از توابع: همه مقدار نهایی `i` را چاپ می‌کنند. با `let` در سرآیند حلقه: `0` ، `1` ، `2` .
- مقدار نهایی را دقیق حساب کن: در `for (var i = 1; i <= 3; i++)` حلقه وقتی `i` برابر `4` شود تمام می‌شود، پس همه `4` می‌بینند، نه `3` .
- `var j = i;` داخل بدنه و `let i;` بیرون از حلقه (همراه با `for (i = 0; ...)`) مشکل را حل نمی‌کنند؛ `let copy = i;` داخل بدنه حل می‌کند.
- IIFE فقط وقتی کمک می‌کند که مقدار را به‌عنوان **پارامتر** بگیرد؛ IIFE بدون پارامتر که داخلش `i` را بخواند، باز همان `i` مشترک را می‌بیند.
- `const c = counter();` یعنی `c` یک **تابع** است، نه عدد. و `counter()()` همیشه `1` می‌دهد، چون هر بار شمارنده‌ی تازه‌ای ساخته می‌شود.
- متغیر خصوصی closure با ویژگی هم‌نام شیء فرق دارد: `obj.count = 100` یک ویژگی جدید می‌سازد و به `count` داخل closure دست نمی‌زند.

جمع‌بندی

- closure یعنی تابعی که متغیرهای scope بیرونی (والد) را به خاطر می‌سپارد و حتی بعد از پایان تابع بیرونی به آن‌ها دسترسی دارد (اسلاید ۲۸).
- کاربردها: encapsulation (متغیر خصوصی)، state (شمارنده، function factory) و callbackها.
- کد اسلاید ۲۸: `c()` اول `1` و بعد `2` برمی‌گرداند.
- دام `var`: یک متغیر مشترک + اجرای دیرتر = همه مقدار آخر را می‌بینند. راه‌حل: `let` در سرآیند حلقه یا IIFE با پارامتر.

تست‌های تمرینی

۳۱ تست · شماره‌ی ۴۳۳ تا ۴۶۳

۴۳۳ آسان

طبق اسلاید ۲۸، **closure** چیست؟

۱) تابعی که برنامه را می‌بندد و اجرای آن را متوقف می‌کند.

۲) تابعی که هنگام ساخته شدن، یک کپی ثابت (snapshot) از مقدار متغیرهای بیرونی برمی‌دارد و تغییرات بعدی آن‌ها را نمی‌بیند.

۳) تابعی که به متغیرهای scope بیرونی (والد) خود دسترسی دارد و این دسترسی را حتی بعد از تمام شدن تابع بیرونی هم حفظ می‌کند.

۴) تابعی که هیچ دسترسی‌ای به متغیرهای بیرون از خودش ندارد.

۴۳۴ آسان

در JavaScript، اینکه یک تابع به کدام متغیرها دسترسی دارد (**lexical scope**)، با چه چیزی تعیین می‌شود؟

۱) جایی که تابع **صدا زده** می‌شود.

۲) ترتیب زمانی صدا زدن توابع هنگام اجرا.

۳) نوع داده‌ی متغیرها.

۴) جایی که تابع در متن کد **نوشته (تعریف)** شده است.

۴۳۵ آسان

خروجی کد زیر کدام است؟

```javascript
const site = "Sharif";
function outer() {
  const course = "Web";
  function inner() {
    console.log(course + " @ " + site);
  }
  inner();
}
outer();
```

۱) `undefined @ Sharif`
۲) `Web @ Sharif`
۳) `Uncaught ReferenceError`
۴) `Web @ undefined`

۴۳۶ متوسط

خروجی کد زیر کدام است؟

```javascript
const label = "global";
function makePrinter() {
  const label = "maker";
  return function () {
    console.log(label);
  };
}
function run() {
  const label = "runner";
  const print = makePrinter();
  print();
}
run();
```

۱) `global`
۲) `maker`
۳) `runner`
۴) `undefined`

۴۳۷ آسان

خروجی کد اسلاید ۲۸ کدام است؟

```JavaScript
function counter() {
  let count = 0;
  return function() {
    count++;
    return count;
  };
}

const c = counter();
console.log(c());
console.log(c());
```

۱) `1` `1`
۲) `0` `1`
۳) `Uncaught ReferenceError`
۴) `1` `2`

۴۳۸ آسان

خروجی کد زیر کدام است؟

```JavaScript
function counter() {
  let count = 0;
  return function() {
    count++;
    return count;
  };
}

const c = counter();
console.log(typeof c);
```

۱) `number`
۲) `function`
۳) `undefined`
۴) `object`

۴۳۹ آسان

در کد اسلاید ۲۸، بعد از اجرای `const c = counter();` کار تابع `counter` تمام شده است. سرنوشت متغیر `count` چیست؟

۱) زنده می‌ماند، چون تابع برگردانده‌شده (`c`) هنوز به آن ارجاع دارد.
۲) حذف می‌شود و `c()` بعدی خطای `ReferenceError` می‌دهد.
۳) با هر بار صدا زدن `c()` دوباره از `0` شروع می‌شود.
۴) به یک متغیر سراسری (global) تبدیل می‌شود که همه‌جا در دسترس است.

۴۴۰ آسان

کدام خط کد زیر خطا می‌دهد؟

JavaScript

```
1  function counter() {
2    let count = 0;
3    return function () {
4      count++;
5      return count;
6    };
7  }
8  const c = counter();
9  c();
10 console.log(c());
11 console.log(count);
```

۱) هیچ خطایی رخ نمی‌دهد
۲) خط ۴
۳) خط ۱۰
۴) خط ۱۱

۴۴۱ متوسط

خروجی کد زیر کدام است؟

JavaScript

```
function counter() {
  let count = 0;
  return function () {
    count++;
    return count;
  };
}
const a = counter();
const b = counter();
a();
a();
console.log(a(), b(), counter()());
```

۱) `3 4 5`
۲) `1 1 1`
۳) `3 1 2`
۴) `3 1 1`

۴۴۲ متوسط

خروجی کد زیر کدام است؟

```JavaScript
function setup() {
  let msg = "first";
  const say = () => console.log(msg);
  msg = "second";
  return say;
}
const fn = setup();
fn();
```

۱) `first`
۲) `undefined`
۳) `second`
۴) `Uncaught ReferenceError`

۴۴۳ سخت

خروجی کد زیر کدام است؟

```JavaScript
function makeGreeter(greeting) {
  return (name) => greeting + ", " + name;
}
let g = "Hi";
const greeter = makeGreeter(g);
g = "Bye";
console.log(greeter("Ali"));
```

۱) `Hi, Ali`
۲) `Bye, Ali`
۳) `undefined, Ali`
۴) `Uncaught ReferenceError`

۴۴۴ متوسط

کدام عبارت درباره‌ی closureها **نادرست** است؟

۱) هر بار صدا زدن تابع بیرونی، مجموعه‌ی جدید و مستقلی از متغیرهای محلی (و در نتیجه closure جدیدی) می‌سازد.

۲) closure حتی بعد از تمام شدن تابع بیرونی، به متغیرهای آن دسترسی دارد.

۳) تابع درونی می‌تواند متغیر بیرونی را هم بخواند و هم تغییر دهد.

۴) closure هنگام ساخته شدن، یک کپی ثابت از مقدار متغیرهای بیرونی برمی‌دارد و تغییرات بعدی آن‌ها را نمی‌بیند.

۴۴۵ آسان

طبق اسلاید ۲۸، closureها برای کدام کارها استفاده می‌شوند؟

۱) encapsulation (کپسوله‌سازی)، نگه‌داری state (وضعیت) و callbackها

۲) تبدیل نوع داده‌ها و مقایسه با `==`

۳) hoisting متغیرها و جلوگیری از TDZ

۴) اجرای هم‌زمان چند thread و سریع‌تر کردن حلقه‌ها

۴۴۶ سخت

خروجی کد زیر کدام است؟

```JavaScript
function createWallet() {
  let balance = 0;
  return {
    deposit: (amount) => {
      balance += amount;
    },
    getBalance: () => balance,
  };
}
const w = createWallet();
w.deposit(50);
w.deposit(20);
w.balance = 1000;
console.log(w.getBalance(), w.balance);
```

۱) `1000 1000`

۲) `70 undefined`

۳) `1070 1000`

۴) `70 1000`

۴۴۷ متوسط

خروجی کد زیر کدام است؟

```javascript
function createCounter() {
  let count = 0;
  return {
    inc: () => {
      count++;
      return count;
    },
    reset: () => {
      count = 0;
    },
  };
}
const c = createCounter();
c.inc();
c.inc();
c.reset();
console.log(c.inc());
```

۱) `3`
۲) `0`
۳) `1`
۴) `Uncaught TypeError`

۴۴۸ متوسط

خروجی کد زیر کدام است؟

```javascript
function makeMultiplier(factor) {
  return function (n) {
    return n * factor;
  };
}
const double = makeMultiplier(2);
const triple = makeMultiplier(3);
console.log(double(5), triple(5), double(triple(2)));
```

۱) `10 15 12`
۲) `15 15 18`
۳) `10 10 8`
۴) `10 15 6`

۴۴۹ متوسط

خروجی کد زیر کدام است؟

```javascript
function once(fn) {
  let done = false;
  return () => {
    if (!done) {
      done = true;
      fn();
    }
  };
}
const init = once(() => console.log("init!"));
init();
init();
init();
```

۱) `init!` `init!` `init!`
۲) `init!`
۳) (بدون خروجی)
۴) `Uncaught TypeError`

۴۵۰ سخت

خروجی کد زیر کدام است؟

```javascript
let total = 0;
function makeAdder(step) {
  return () => {
    total += step;
    return total;
  };
}
const add5 = makeAdder(5);
const add10 = makeAdder(10);
add5();
add10();
console.log(add5(), total);
```

۱) `10 0`
۲) `20 15`
۳) `20 20`
۴) `10 20`

۴۵۱ متوسط

خروجی کد زیر کدام است؟

```JavaScript
function greetLater(name) {
  setTimeout(() => console.log("Hello, " + name), 0);
  console.log("scheduled: " + name);
}
greetLater("Ali");
greetLater("Sara");
```

۱) `scheduled: Ali` `Hello, Ali` `scheduled: Sara` `Hello, Sara`

۲) `scheduled: Ali` `scheduled: Sara` `Hello, Sara` `Hello, Sara`

۳) `scheduled: Ali` `scheduled: Sara` `Hello, Ali` `Hello, Sara`

۴) `scheduled: Ali` `scheduled: Sara` `Hello, undefined` `Hello, undefined`

۴۵۲ آسان

در جای خالی کدام گزینه قرار می‌گیرد؟

«طبق هشدار BEWARE OF PITFALL در اسلاید ۲۸، closureها وقتی با _____ ترکیب شوند، می‌توانند مشکل‌ساز شوند.»

۱) `let` ۲) `const` ۳) arrow function ۴) `var`

۴۵۳ متوسط

خروجی کد زیر کدام است؟

```JavaScript
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}
console.log("after loop:", i);
```

۱) `after loop: 3` `3` `3` `3`

۲) `after loop: 3` `0` `1` `2`

۳) `0` `1` `2` `after loop: 3`

۴) `after loop: 2` `2` `2` `2`

۴۵۴ متوسط

کد `for (var i = 0; i < 3; i++) { setTimeout(() => console.log(i), 0); }` سه بار عدد `3` را چاپ می‌کند. دلیل اصلی کدام است؟

۱) `setTimeout` همیشه آخرین مقدار هر متغیر را کپی می‌کند و به callback می‌دهد.

۲) arrow functionها به متغیرهای حلقه دسترسی ندارند و مقدار پیش‌فرض می‌گیرند.

۳) فقط **یک** متغیر `i` (function-scoped) وجود دارد که هر سه callback به آن دسترسی دارند، و callbackها بعد از تمام شدن حلقه اجرا می‌شوند؛ وقتی `i` به `3` رسیده است.

۴) تأخیر `0` میلی‌ثانیه مجاز نیست و حلقه سه بار دیگر تکرار می‌شود.

۴۵۵ متوسط

در جای خالی کدام گزینه قرار می‌گیرد؟

«وقتی `i` با `let` در سرآیند (header) حلقه‌ی `for` تعریف شود، JavaScript برای **هر تکرار** _____ می‌سازد؛ برای همین هر closure مقدار همان تکرار را می‌بیند.»

۱) یک binding (نسخه‌ی) جدید از `i`

۲) یک متغیر سراسری (global) مشترک برای همه‌ی تکرارها

۳) یک `var` جدید که به بالای تابع hoist می‌شود

۴) یک snapshot ثابت از مقدار `i` در لحظه‌ی پایان حلقه

۴۵۶ سخت

خروجی کد زیر کدام است؟

```JavaScript
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log("var", i), 0);
}
for (let j = 0; j < 3; j++) {
  setTimeout(() => console.log("let", j), 0);
}
```

۱) `var 3` `var 3` `var 3` `let 0` `let 1` `let 2`

۲) `var 0` `var 1` `var 2` `let 0` `let 1` `let 2`

۳) `var 3` `let 0` `var 3` `let 1` `var 3` `let 2`

۴) `var 3` `var 3` `var 3` `let 3` `let 3` `let 3`

۴۵۷ متوسط

خروجی کد زیر کدام است؟

```JavaScript
for (var i = 0; i < 3; i++) {
  ((n) => {
    setTimeout(() => console.log(n * 10), 0);
  })(i);
}
console.log("loop done, i =", i);
```

۱) `0` `10` `20` `loop done, i = 3`

۲) `loop done, i = 3` `30` `30` `30`

۳) `loop done, i = 2` `0` `10` `20`

۴) `loop done, i = 3` `0` `10` `20`

۴۵۸ متوسط

خروجی کد زیر کدام است؟

```javascript
var a = [];
var b = [];
for (var i = 0; i < 3; i++) {
  a.push(() => i);
}
for (let j = 0; j < 3; j++) {
  b.push(() => j);
}
console.log(a[1](), b[1]());
```

۱) `1 1`
۲) `3 1`
۳) `3 3`
۴) `1 3`

۴۵۹ سخت

خروجی کد زیر کدام است؟

```javascript
for (var i = 0; i < 3; i++) {
  const label = "item" + i;
  setTimeout(() => console.log(label, i), 0);
}
```

۱) `item0 0` `item1 1` `item2 2`
۲) `item0 3` `item1 3` `item2 3`
۳) `item2 3` `item2 3` `item2 3`
۴) `item3 3` `item3 3` `item3 3`

۴۶۰ سخت

کدام قطعه‌کد اعداد `0`، `1` و `2` را (هر کدام در یک خط) چاپ می‌کند؟

۱)
```JavaScript
for (var i = 0; i < 3; i++) {
  var j = i;
  setTimeout(() => console.log(j), 0);
}
```

۲)
```JavaScript
for (var i = 0; i < 3; i++) {
  (function () {
    setTimeout(() => console.log(i), 0);
  })();
}
```

۳)
```JavaScript
for (var i = 0; i < 3; i++) {
  (function (k) {
    setTimeout(() => console.log(k), 0);
  })(i);
}
```

۴)
```JavaScript
let i;
for (i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}
```

۴۶۱ سخت

خروجی کد زیر کدام است؟

```JavaScript
for (var i = 1; i <= 3; i++) {
  setTimeout(() => console.log(i), i * 100);
}
```

۱) `1` `2` `3`

۲) `3` `3` `3`

۳) `4` `4` `4`

۴) `0` `1` `2`

۴۶۲ سخت

خروجی کد زیر کدام است؟

```JavaScript
function a(x) {
  return function (y) {
    return function (z) {
      return x + y + z;
    };
  };
}
console.log(a(1)(2)(3));
console.log(a("1")(2)(3));
```

۱) `6` `123`
۲) `6` `6`
۳) `6` `15`
۴) `6` `NaN`

۴۶۳ سخت

خروجی کد زیر کدام است؟

```JavaScript
function makeCounter() {
  return increment;
  var count = 0;
  function increment() {
    count++;
    return count;
  }
}
const inc = makeCounter();
console.log(inc());
console.log(inc());
```

۱) `1` `2`
۲) `NaN` `NaN`
۳) `Uncaught ReferenceError`
۴) `undefined` `undefined`

۱-۲۰ اشیاء (Objects) و Object literal

منبع: Introduction to JS، اسلایدهای ۴۰ و ۴۱

درسنامه

تا این‌جا بیشتر با مقدارهای ساده (primitive) مثل عدد و رشته کار کردیم. اما داده‌های دنیای واقعی تنها نیستند: یک «کاربر» نام دارد، سن دارد، شهر دارد و کارهایی هم انجام می‌دهد (مثلاً سلام می‌کند). اگر برای هر کدام یک متغیر جدا بسازیم (`userName`، `userAge`، `userCity`، …) خیلی زود گم می‌شویم. **object** (شیء) راه JavaScript برای کنار هم نگه داشتن همه‌ی این‌هاست.

یک تشبیه ساده

یک شیء را مثل یک **کمد با کشوهای برچسب‌دار** تصور کن. روی هر کشو یک برچسب نوشته شده (مثل `name` یا `age`) که به آن **کلید** (key) می‌گوییم، و داخل کشو یک چیز هست (مثل `"Barry Allen"` یا `24`) که به آن **مقدار** (value) می‌گوییم. برای برداشتن چیزی فقط کافی است برچسب کشو را بدانی.

شیء چیست؟ (اسلاید ۴۰)

اسلاید ۴۰ چهار جمله‌ی کلیدی دارد:

1. شیءها **داده** (data) و **رفتار** (behavior) مرتبط را کنار هم گروه‌بندی می‌کنند.
2. هر شیء از **جفت‌های کلید-مقدار** (key-value pairs) ساخته شده است.
3. مقدارها می‌توانند **داده** باشند که به آن‌ها **property** (ویژگی) می‌گوییم، یا **تابع** باشند که به آن‌ها **method** (متد؛ تابعی که متعلق به یک شیء است) می‌گوییم.
4. **JSON** (مخفف JavaScript Object Notation؛ یک قالب متنی برای نوشتن داده‌ها) رایج‌ترین راه برای **ذخیره** و **به اشتراک گذاشتن** داده‌های یک شیء است؛ مثلاً وقتی سرور اطلاعات یک کاربر را برای مرورگر می‌فرستد. جزئیات JSON را در درس JSON می‌خوانیم.

```JavaScript
const user = {
  name: "Barry Allen", // property: key "name", value "Barry Allen"
  age: 24,             // property: key "age", value 24
  greet() {            // method: a function stored in the object
    console.log("Hi!");
  }
};
```

اصطلاح	یعنی	در مثال بالا
key (کلید)	نام یک خانه از شیء	`name`، `age` و `greet`
value (مقدار)	چیزی که در آن خانه است	`"Barry Allen"`، `24` و یک تابع
property	جفتی که مقدارش **داده** است	`name: "Barry Allen"`
method	جفتی که مقدارش **تابع** است	`greet() { ... }`

«تقریباً همه چیز شیء است» (میم اسلاید ۴۰)

در میم اسلاید ۴۰، کره‌ی زمین با لوگوی JS دیده می‌شود. فضانورد اول با تعجب می‌گوید: «صبر کن... همه‌اش **شیء** است؟!» و فضانورد دوم که از پشت سر به او اسلحه گرفته جواب می‌دهد: «همیشه همین‌طور بوده (Always has been)». زیرنویس اسلاید هم می‌گوید: «از نظر فنی، در JavaScript **تقریباً** همه چیز یک شیء است.»

منظور این است که آرایه‌ها، توابع، تاریخ‌ها (`Date`) و خیلی چیزهای دیگر در اصل شیء هستند. اما کلمه‌ی «**تقریباً**» مهم است: هفت نوع **primitive** (عدد، رشته، Symbol، `null`، `undefined`، boolean و BigInt) شیء **نیستند**.

```JavaScript
console.log(typeof { a: 1 });       // "object"
console.log(typeof [1, 2]);         // "object"   (arrays are objects)
console.log(typeof new Date());     // "object"
console.log(typeof function () {}); // "function" (functions are special objects)
console.log(typeof "hello");        // "string"   (a primitive, NOT an object)
```

پس چرا می‌توانیم بنویسیم `"hello".length` یا `"hello".toUpperCase()`، انگار رشته یک شیء با property و method است؟ چون وقتی بعد از یک primitive نقطه می‌گذاری، JavaScript **برای یک لحظه** آن را داخل یک **شیء پوشاننده** (wrapper object؛ مثلاً `String` برای رشته‌ها و `Number` برای عددها) می‌گذارد، property یا method را از آن شیء موقت می‌خواند و بلافاصله آن را دور می‌اندازد. برای همین primitiveها «مثل شیء رفتار می‌کنند» ولی شیء نیستند:

```JavaScript
const word = "hello";
console.log(word.length);        // 5
console.log(word.toUpperCase()); // "HELLO"
word.extra = 42;                 // stored on a temporary wrapper (in strict mode: TypeError)
console.log(word.extra);         // undefined -> that wrapper is already gone
```

`null` و `undefined` هیچ شیء پوشانده‌ای ندارند؛ برای همین `null.length` یا `undefined.x` خطای `TypeError` می‌دهند.

ساختن شیء با object literal (اسلاید ۴۱)

ساده‌ترین راه ساختن شیء، **object literal** است؛ یعنی نوشتن مستقیم خود شیء با آکولاد: داخل `{ }` جفت‌های `key: value` را می‌نویسیم و آن‌ها را با ویرگول از هم جدا می‌کنیم. اسلاید ۴۱ می‌گوید:

- شیءها با جفت‌های `{ key: value }` تعریف می‌شوند.
- داده و رفتار مرتبط را کنار هم نگه می‌دارند.
- **کلیدها رشته (string) هستند (یا Symbol)** و **مقدارها از هر نوعی** می‌توانند باشند.
- از **متد** و **شیء تودرتو** (nested object؛ شیئی که مقدارِ یک کلیدِ شیء دیگر است) پشتیبانی می‌کنند.
- **پایه‌ی ساختار JSON** در JavaScript هستند.

کد اسلاید ۴۱:

ol.js

```
const user = {
  name: "Barry Allen",
  age: 24,
  isTheFlash: true,
  greet() {
    console.log(`Hello, I'm ${this.name}`);
  }
};

user.greet(); // Hello, I'm Barry Allen
```

بیا خط‌به‌خط ببینیم:

- `user` یک شیء با چهار کلید است: سه property (`name` از نوع رشته، `age` عدد و `isTheFlash` boolean) و یک method (`greet`).
- `greet() { ... }` شکل کوتاه‌شده‌ی (shorthand) نوشتن متد است و دقیقاً یعنی `greet: function () { ... }`.
- داخل متد کلمه‌ی `this` آمده است: وقتی متدی را به شکل `user.greet()` صدا می‌زنیم، `this` همان شیءِ **قبل از نقطه**، یعنی `user` است؛ پس `this.name` می‌شود `"Barry Allen"`. (قاعده‌های کامل `this` را در درس `this` می‌خوانیم.)
- رشته‌ی داخل backtick یک template literal است و `${this.name}` مقدار را وسط رشته می‌گذارد؛ پس خروجی `Hello, I'm Barry Allen` است.

اگر کل یک شیء را چاپ کنیم، Node.js متدش را به شکل `[Function: greet]` نشان می‌دهد:

JavaScript

```
const hero = {
  name: "Barry Allen",
  age: 24,
  greet() {
    console.log(`Hello, I'm ${this.name}`);
  }
};
console.log(hero); // { name: 'Barry Allen', age: 24, greet: [Function: greet] }
```

مقدار می‌تواند هر چیزی باشد؛ حتی آرایه و شیء دیگر

JavaScript

```
const student = {
  name: "Sara",
  scores: [18, 20, 17],      // an array as a value
  address: {                 // a nested object
    city: "Tehran",
    zip: "12345"
  },
  "favorite color": "blue",  // a key with a space must be quoted
  isActive: true
};

console.log(student.address.city); // "Tehran"
console.log(student.scores[1]);    // 20
```

برای رسیدن به داده‌های تودرتو، نقطه‌ها (و براکت‌ها) را پشت سر هم می‌نویسیم: `student.address.city` یعنی «از `student` برو داخل `address` و از آن‌جا `city` را بردار».

خواندن propertyها: نقطه یا براکت؟

دو راه برای خواندن (و نوشتن) یک property داریم:

روش	مثال	کی؟
dot notation (نقطه)	`user.name`	نام کلید را موقع نوشتن کد می‌دانی و یک نام معمولی است
bracket notation (براکت)	`user["name"]`	کلید داخل یک **متغیر** است، یا فاصله یا خط تیره دارد، یا با رقم شروع می‌شود

JavaScript

```
const user = { name: "Barry", age: 24, "first-name": "Bartholomew" };

console.log(user.name);          // "Barry"
console.log(user["name"]);       // "Barry" (exactly the same thing)

const key = "age";
console.log(user[key]);          // 24        -> uses the VALUE of the variable key
console.log(user.key);           // undefined -> looks for a key literally named "key"

console.log(user["first-name"]); // "Bartholomew"
// user.first-name would mean (user.first) - name: a subtraction, not a key!
```

> ⚠ **دام تستی: `obj.key` با `obj[key]` فرق دارد**
>
> بعد از نقطه، JavaScript **خودِ کلمه** را نام کلید می‌گیرد؛ پس `user.key` یعنی «کلیدی به نام `"key"`». اما داخل براکت یک **عبارت** نوشته می‌شود و اول مقدارش حساب می‌شود؛ پس `user[key]` یعنی «کلیدی که نامش داخل متغیر `key` است». `user["key"]` (با کوتیشن) هم دوباره همان `user.key` است. اگر داخل براکت نام یک متغیرِ **تعریف‌نشده** را بی‌کوتیشن بنویسی (مثلاً `user[age]` وقتی متغیری به نام `age` نداری)، خطای `ReferenceError` می‌گیری.

کلیدها همیشه رشته‌اند (یا Symbol)

هر کلیدی که بنویسی، اگر Symbol نباشد، به **رشته** تبدیل می‌شود. پس کلید عددی `1` در واقع رشته‌ی `"1"` است و `obj[1]` و `obj["1"]` به **یک** خانه اشاره می‌کنند:

JavaScript

```
const obj = { 1: "one" };
console.log(obj[1]);           // "one"
console.log(obj["1"]);         // "one" (the same property)
console.log(Object.keys(obj)); // [ '1' ] -> the key is the string "1"
```

حتی اگر یک شیء را کلید کنی، به رشته‌ی `"[object Object]"` تبدیل می‌شود (پس دو شیءِ متفاوت به‌عنوان کلید، **یک** کلید می‌شوند!). تنها کلیدهای غیررشته‌ای، **Symbol**ها هستند: هر `Symbol()` یکتاست (اسلاید ۱۶) و برای کلیدهایی به کار می‌رود که نباید تصادفاً با کلید دیگری یکی شوند.

اگر داخل خودِ literal کلید را در براکت بنویسی (computed property name؛ کلید محاسبه‌شده)، **مقدارِ** عبارت داخل براکت کلید می‌شود:

```JavaScript
const id = Symbol("id");
const field = "score";
const item = { name: "Pen", [id]: 123, [field]: 95 };

console.log(item.score);        // 95  -> [field] became the key "score"
console.log(item[id]);          // 123
console.log(Object.keys(item)); // [ 'name', 'score' ] -> symbol keys are not listed
```

اگر property وجود نداشته باشد؟

خواندن کلیدی که وجود ندارد **خطا نمی‌دهد** و فقط `undefined` برمی‌گرداند. اما اگر روی همان `undefined` دوباره نقطه بگذاری، خطای `TypeError` می‌گیری:

```JavaScript
const user = { name: "Barry", address: { city: "Central City" } };
console.log(user.phone);        // undefined (no error)
console.log(user.address.city); // "Central City"
console.log(user.job?.title);   // undefined (optional chaining stops safely)
console.log(user.job.title);    // TypeError: Cannot read properties of undefined (reading 'title')
```

عملگر **optional chaining** یعنی `?.` (زنجیره‌ی اختیاری) می‌گوید: «اگر سمت چپ `undefined` یا `null` بود، همان‌جا بایست و `undefined` بده؛ وگرنه مثل نقطه‌ی معمولی ادامه بده.»

اضافه کردن، تغییر دادن و حذف کردن

شیءها **تغییرپذیر** (mutable) هستند؛ یعنی بعد از ساختن هم می‌توانی propertyهایشان را عوض کنی:

```JavaScript
const user = { name: "Barry", age: 24 };

user.city = "Central City"; // add: the key did not exist, so it is created
user.age = 25;              // update: the key exists, so its value is replaced
delete user.name;           // remove the whole property (key and value)

console.log(user);          // { age: 25, city: 'Central City' }
console.log(user.name);     // undefined
```

- انتساب به کلیدی که **نیست** آن را **می‌سازد**؛ انتساب به کلیدی که **هست** مقدارش را **عوض می‌کند**.
- عملگر `delete` خودِ کلید را حذف می‌کند، نه فقط مقدارش را.
- اگر در یک literal یک کلید دو بار بیاید، **آخرین مقدار برنده است**: `{ a: 1, a: 2 }` همان `{ a: 2 }` است.

`const` شیء را قفل نمی‌کند

همان‌طور که در درس متغیرها دیدیم، `const` فقط **انتساب دوباره‌ی خود متغیر** را منع می‌کند، نه تغییر **محتوای** شیء:

```JavaScript
const user = { name: "Barry" };
user.name = "Wally";     // OK: changing a property
user.age = 25;           // OK: adding a property
user = { name: "Iris" }; // TypeError: Assignment to constant variable.
```

آیا این کلید وجود دارد؟ `in` و `hasOwnProperty`

```JavaScript
const user = { name: "Barry", nickname: undefined };

console.log("name" in user);                  // true
console.log("email" in user);                 // false
console.log(user.hasOwnProperty("name"));     // true
console.log("nickname" in user);              // true  (the key exists; its value is undefined)
console.log(user.nickname === user.email);    // true  (both undefined: cannot tell them apart)
console.log("toString" in user);              // true  (inherited from Object.prototype)
console.log(user.hasOwnProperty("toString")); // false (not the object's OWN key)
```

- `"key" in obj` نام کلید را به‌صورت **رشته** می‌گیرد (کوتیشن را فراموش نکن) و می‌گوید آیا این کلید در شیء هست یا نه. این عملگر کلیدهای **ارث‌برده** را هم حساب می‌کند؛ هر شیء معمولی چند متد مثل `toString` را از یک شیء پایه به نام `Object.prototype` به ارث می‌برد.
- `obj.hasOwnProperty("key")` فقط کلیدهای **خودِ** شیء را حساب می‌کند. (شکل جدیدترش `Object.hasOwn(obj, "key")` است.)
- مقایسه با `undefined` فرقِ «کلید نیست» و «کلید هست ولی مقدارش `undefined` است» را نمی‌فهمد؛ `in` می‌فهمد.

گرفتن همه‌ی کلیدها و مقدارها

```JavaScript
const prices = { tea: 2, coffee: 3, cake: 5 };

console.log(Object.keys(prices));    // [ 'tea', 'coffee', 'cake' ]
console.log(Object.values(prices));  // [ 2, 3, 5 ]
console.log(Object.entries(prices)); // [ [ 'tea', 2 ], [ 'coffee', 3 ], [ 'cake', 5 ] ]

for (const key in prices) {          // for...in walks over the keys
  console.log(key, prices[key]);     // tea 2, then coffee 3, then cake 5
}
```

این سه تابع همیشه **آرایه** برمی‌گردانند (کلیدها در آن‌ها رشته‌اند)، پس بعداً می‌توانی متدهای آرایه مثل `map` و `filter` را رویشان صدا بزنی. `Object.entries` برای هر کلید یک آرایه‌ی دوتایی `[key, value]` می‌سازد. در حلقه‌ی `for...in` (درس حلقه‌ها) حتماً بنویس `prices[key]`، نه `prices.key`؛ چون `key` یک متغیر است.

نوشتن کوتاه: shorthand property

وقتی نام متغیر و نام کلید یکی است، لازم نیست دو بار بنویسی:

```JavaScript
const name = "Sara";
const age = 21;

const student1 = { name: name, age: age }; // long form
const student2 = { name, age };            // shorthand: exactly the same object
console.log(student2);                     // { name: 'Sara', age: 21 }
```

شیءها با reference کپی و مقایسه می‌شوند

متغیری که یک شیء در آن است، خودِ شیء را نگه نمی‌دارد؛ فقط **نشانیِ** آن را نگه می‌دارد که به آن **reference** (مرجع) می‌گوییم. پس `const b = a` شیء جدیدی **نمی‌سازد**؛ فقط نشانی را کپی می‌کند و حالا دو نام به **یک** شیء اشاره می‌کنند:

```JavaScript
const a = { score: 10 };
const b = a;          // copies the reference, not the object
b.score = 99;         // changes the ONE shared object
console.log(a.score); // 99
console.log(a === b); // true (the same object)
```

یک تشبیه ساده

reference مثل **آدرس یک خانه** است که روی کاغذ نوشته شده. `const b = a` یعنی همان آدرس را روی یک کاغذ دیگر هم بنویسی؛ هنوز فقط **یک** خانه داریم. اگر با کاغذ دوم بروی و دیوار را رنگ کنی، کسی که با کاغذ اول بیاید هم دیوار رنگ‌شده را می‌بیند.

همین اتفاق وقتی شیء را به یک تابع می‌دهیم هم می‌افتد: تابع همان شیء را می‌گیرد، نه یک کپی را. مقایسه‌ی شیءها با `==` یا `===` هم فقط می‌پرسد «آیا هر دو **همان یک** شیءاند؟»، نه «آیا محتوایشان یکی است؟»:

```JavaScript
console.log({} === {});             // false (two different objects)
console.log({ x: 1 } === { x: 1 }); // false (same content, different objects)
const p = { x: 1 };
const q = p;
console.log(p === q);               // true  (same reference)
```

کپی سطحی با spread: `{ ...obj }`

برای ساختن یک شیء **واقعاً جدید** با همان propertyها، از **spread** (عملگر پخش؛ سه نقطه‌ی `...`) استفاده می‌کنیم. `{ ...obj }` یعنی «همه‌ی propertyهای `obj` را در این شیء تازه بریز». اسلاید ۴۸ (درس متدهای آرایه) دقیقاً از همین روش استفاده می‌کند.

```JavaScript
const original = { name: "Ali", age: 20 };
const copy = { ...original };   // a NEW object with the same properties
copy.age = 30;
console.log(original.age);      // 20 (unchanged)
console.log(copy === original); // false
```

ترتیب مهم است: propertyها از چپ به راست در شیء جدید ریخته می‌شوند و اگر کلیدی تکراری باشد، **آخری برنده است**:

```JavaScript
const base = { name: "Ali", role: "user" };
console.log({ ...base, role: "admin" }); // { name: 'Ali', role: 'admin' } -> role overridden
console.log({ role: "admin", ...base }); // { role: 'user', name: 'Ali' } -> base wins
```

«سطحی» (shallow) یعنی چه؟ spread فقط **سطح اول** را کپی می‌کند. اگر مقدار یک property خودش یک شیء باشد، فقط **reference** آن کپی می‌شود و آن شیء داخلی بین نسخه‌ی اصلی و کپی **مشترک** می‌ماند:

```JavaScript
const user = { name: "Ali", address: { city: "Tehran" } };
const copy = { ...user };
copy.name = "Reza";             // top level: only the copy changes
copy.address.city = "Shiraz";   // nested object: SHARED by both
console.log(user.name);         // "Ali"
console.log(user.address.city); // "Shiraz"
```

شیءها و JSON

اسلاید ۴۱ می‌گوید object literal «پایه‌ی ساختار JSON» است. JSON شبیه object literal است ولی سخت‌گیرتر: کلیدها و رشته‌ها **حتماً** داخل دابل‌کوتیشن‌اند و تابع (متد) در آن جایی ندارد. با `JSON.stringify` یک شیء را به متن JSON تبدیل می‌کنیم:

```JavaScript
const user = { name: "Barry Allen", age: 24, greet() {} };
console.log(JSON.stringify(user)); // {"name":"Barry Allen","age":24} -> the method is dropped
```

نکته

- property یعنی مقدارِ داده‌ای و method یعنی مقدارِ تابعی. `greet() {}` کوتاه‌شده‌ی `greet: function () {}` است.
- کلیدی که فاصله یا خط تیره دارد یا با رقم شروع می‌شود، فقط با براکت و کوتیشن خوانده می‌شود: `obj["first-name"]` .
- خواندن کلیدِ ناموجود `undefined` می‌دهد، ولی خواندن property از `undefined` یا `null` خطای `TypeError` می‌دهد؛ `?.` جلوی این خطا را می‌گیرد.
- `Object.keys` ، `Object.values` و `Object.entries` همیشه آرایه برمی‌گردانند.
- `in` کلیدهای ارث‌برده (مثل `toString`) را هم می‌بیند؛ `hasOwnProperty` فقط کلیدهای خودِ شیء را.

دام‌های تستی شیءها

- `obj.key` دنبال کلیدی به نام `"key"` می‌گردد؛ برای کلیدی که نامش در متغیر `key` است بنویس `obj[key]` .
- `obj[1]` و `obj["1"]` یک خانه‌اند، چون کلیدها رشته‌اند.
- `{} === {}` و `{ a: 1 } === { a: 1 }` هر دو `false` هستند؛ شیءها با reference مقایسه می‌شوند.
- `const b = a` کپی نمی‌سازد؛ تغییر `b.x` همان `a.x` را عوض می‌کند. کپی با `{ ...a }` ساخته می‌شود، ولی شیءهای تودرتو در آن هنوز مشترک‌اند.
- شیءِ `const` را می‌توان تغییر داد (`user.age = 25`)، ولی نمی‌توان متغیر را به شیء دیگری وصل کرد (`user = {}` خطای `TypeError` می‌دهد).
- در `{ ...base, role: "admin" }` نتیجه `"admin"` است، ولی در `{ role: "admin", ...base }` مقدار `base.role` برنده است.
- رشته‌ها property دارند (`"hi".length`) ولی شیء نیستند: `typeof "hi"` برابر `"string"` است.

جمع‌بندی

- شیء = مجموعه‌ای از جفت‌های کلید-مقدار که داده (property) و رفتار (method) را کنار هم نگه می‌دارد.
- object literal: `{ key: value, method() { ... } }`؛ کلیدها رشته یا Symbol، مقدارها هر نوعی (حتی شیء و آرایه‌ی تودرتو).
- دسترسی: `obj.key` برای نام‌های ثابت و معمولی، `obj[expr]` برای کلیدهای پویا یا عجیب.
- شیءها تغییرپذیرند و با reference کپی و مقایسه می‌شوند؛ کپی سطحی با `{ ...obj }`.

تست‌های تمرینی

۲۷ تست · شماره‌ی ۴۶۴ تا ۴۹۰

۴۶۴ آسان

طبق اسلاید ۴۰، در یک شیء به مقدارهایی که **تابع** هستند و به مقدارهایی که **داده** هستند، به ترتیب چه می‌گوییم؟

۱) key و value
۲) property و method
۳) property و method
۴) literal و JSON

۴۶۵ آسان

طبق اسلایدهای ۴۰ و ۴۱، کدام گزینه **درست** است؟

۱) شیء از جفت‌های کلید-مقدار ساخته می‌شود؛ کلیدها رشته (یا Symbol) هستند و مقدارها می‌توانند از هر نوعی باشند.
۲) کلیدها می‌توانند از هر نوعی (مثلاً عدد یا شیء) باشند و نوعشان حفظ می‌شود، ولی مقدارها فقط عدد یا رشته‌اند.
۳) یک شیء نمی‌تواند شیء، آرایه یا تابع دیگری را به‌عنوان مقدار نگه دارد.
۴) JSON یک نوع داده‌ی primitive است که در JavaScript جای شیء را گرفته است.

۴۶۶ آسان

خروجی کد زیر (کد اسلاید ۴۱) چیست؟

```JavaScript
const user = {
  name: "Barry Allen",
  age: 24,
  isTheFlash: true,
  greet() {
    console.log(`Hello, I'm ${this.name}`);
  }
};

user.greet();
```

۱) `Hello, I'm undefined`
۲) `Hello, I'm ${this.name}`
۳) `Uncaught ReferenceError`
۴) `Hello, I'm Barry Allen`

۴۶۷ آسان

با فرض `const car = { brand: "BMW", year: 2020 };` حاصل `car.color` کدام است؟

۱) `null`
۲) `undefined`
۳) `""`
۴) خطای `ReferenceError`

۴۶۸ آسان

شیء `person` به شکل زیر تعریف شده است. کدام دستور، `Ali` را چاپ می‌کند؟

```JavaScript
const person = { "first-name": "Ali", age: 20 };
```

۱) `console.log(person.first-name);`
۲) `console.log(person.first_name);`
۳) `console.log(person["first-name"]);`
۴) `console.log(person[first-name]);`

۴۶۹ آسان

خروجی کد زیر چیست؟

```JavaScript
const user = { name: "Barry", age: 24 };
user.age = 25;
user.city = "Central City";
console.log(user.age, user.city);
```

۱) `25 Central City`
۲) `24 undefined`
۳) `Uncaught TypeError`
۴) `25 undefined`

۴۷۰ آسان

کدام مورد را می‌توان در یک **object literal** نوشت، ولی در **JSON** جایی ندارد؟

۱) یک رشته در دابل‌کوتیشن، مثل `"Ali"`
۲) یک متد، مثل `greet() { ... }`
۳) یک عدد، مثل `24`
۴) یک شیء تودرتو، مثل `{ "city": "Tehran" }`

۴۷۱ متوسط

خروجی کد زیر چیست؟

```JavaScript
const user = { name: "Barry", age: 24 };
const key = "age";
console.log(user.key);
console.log(user[key]);
console.log(user["key"]);
```

۱) `24` `24` `24`

۲) `undefined` `24` `undefined`

۳) `24` `undefined` `24`

۴) `undefined` `24` `24`

۴۷۲ متوسط

خروجی کد زیر چیست؟

```JavaScript
const obj = { 1: "one", "2": "two" };
console.log(obj[1]);
console.log(obj["2"]);
console.log(obj[2]);
console.log(Object.keys(obj));
```

۱) `one` `two` `undefined` `[1, '2']`

۲) `one` `two` `undefined` `['1', '2']`

۳) `undefined` `two` `two` `[1, 2]`

۴) `one` `two` `two` `['1', '2']`

۴۷۳ متوسط

خروجی کد زیر چیست؟

```JavaScript
const user = { name: "Barry", age: 24 };
user.city = "Central City";
user.age = 25;
delete user.name;
console.log(user);
console.log(user.name);
```

۱) `{ name: undefined, age: 25, city: 'Central City' }` `undefined`

۲) `{ age: 25, city: 'Central City' }` `undefined`

۳) `{ name: 'Barry', age: 25, city: 'Central City' }` `Barry`

۴) `Uncaught TypeError`

۴۷۴ متوسط

خروجی کد زیر چیست؟

```javascript
const a = { x: 1 };
const b = a;
const c = { x: 1 };
b.x = 5;
console.log(a.x, c.x);
console.log(a === b, a === c);
```

۱) `1 1` `true true`

۲) `5 5` `true true`

۳) `5 1` `true false`

۴) `1 1` `false false`

۴۷۵ متوسط

خروجی کد زیر چیست؟

```javascript
const base = { name: "Ali", role: "user" };
const a = { ...base, role: "admin" };
const b = { role: "admin", ...base };
console.log(a.role, b.role, base.role);
```

۱) `admin user user`

۲) `user admin user`

۳) `admin admin admin`

۴) `admin admin user`

۴۷۶ متوسط

خروجی کد زیر چیست؟

```javascript
const name = "Sara";
const age = 21;
const student = {
  name,
  age,
  intro() {
    return "I'm " + this.name;
  },
};
console.log(student.intro());
console.log(Object.keys(student));
```

۱) `I'm Sara` `['name', 'age', 'intro']`

۲) `I'm Sara` `['name', 'age']`

۳) `I'm undefined` `['name', 'age', 'intro']`

۴) `Uncaught SyntaxError`

۴۷۷ متوسط

خروجی کد زیر چیست؟

```JavaScript
const prices = { tea: 2, coffee: 3 };
console.log(Object.values(prices));
console.log(Object.entries(prices));
```

۱) `['tea', 'coffee']` `[['tea', 2], ['coffee', 3]]`

۲) `[2, 3]` `[['tea', 2], ['coffee', 3]]`

۳) `[2, 3]` `{ tea: 2, coffee: 3 }`

۴) `[2, 3]` `['tea', 2, 'coffee', 3]`

۴۷۸ متوسط

اجرای کد زیر در کدام خط با خطا متوقف می‌شود؟

```JavaScript
1 const user = { name: "Barry", address: { city: "Central City" } };
2 console.log(user.address.city);
3 console.log(user.phone);
4 console.log(user.address.zip);
5 console.log(user.job.title);
6 console.log(user.job?.title);
```

۱) خط ۳

۲) خط ۴

۳) خط ۵

۴) هیچ خطایی رخ نمی‌دهد

۴۷۹ متوسط

در جای خالی کدام گزینه قرار بگیرد تا خروجی کد `24` باشد؟

```JavaScript
const user = { name: "Barry", age: 24 };
const key = "age";
console.log(user___);
```

۱) `.key`

۲) `["key"]`

۳) `.[key]`

۴) `[key]`

۴۸۰ متوسط

اجرای کد زیر در کدام خط با خطا متوقف می‌شود؟

```javascript
const user = { name: "Barry" };
user.name = "Wally";
user["age"] = 25;
delete user.age;
user = { name: "Iris" };
console.log(user.name);
```

۱) هیچ خطایی رخ نمی‌دهد
۲) خط ۲
۳) خط ۴
۴) خط ۵

۴۸۱ متوسط

کدام تعریفِ شیء **خطا** می‌دهد؟

۱) `const o = { function greet() { return "hi"; } };`
۲) `const o = { greet() { return "hi"; } };`
۳) `const o = { greet: function () { return "hi"; } };`
۴) `const o = { greet: () => "hi" };`

۴۸۲ متوسط

خروجی کد زیر چیست؟

```javascript
function birthday(person) {
  person.age = person.age + 1;
}
const sara = { name: "Sara", age: 20 };
birthday(sara);
birthday(sara);
console.log(sara.age);
```

۱) `20`
۲) `21`
۳) `22`
۴) `Uncaught TypeError`

۴۸۳ سخت

خروجی کد زیر در حالت عادی (غیر strict) چیست؟

```javascript
const word = "hi";
word.extra = 42;
console.log(word.extra);
console.log(word.length, word.toUpperCase());
```

۱) `42` `2 HI`

۲) `Uncaught TypeError`

۳) `undefined` `2 HI`

۴) `undefined` `undefined undefined`

۴۸۴ سخت

خروجی کد زیر چیست؟

```javascript
const settings = { theme: "dark", font: undefined };
console.log(settings.font === settings.size);
console.log("font" in settings, "size" in settings);
console.log(settings.hasOwnProperty("toString"), "toString" in settings);
```

۱) `false` `true false` `false false`

۲) `true` `false false` `false true`

۳) `true` `true false` `true true`

۴) `true` `true false` `false true`

۴۸۵ سخت

خروجی کد زیر چیست؟

```javascript
const user = { name: "Ali", address: { city: "Tehran" } };
const copy = { ...user };
copy.name = "Reza";
copy.address.city = "Shiraz";
console.log(user.name, user.address.city);
console.log(copy.address === user.address);
```

۱) `Ali Tehran` `false`

۲) `Ali Shiraz` `true`

۳) `Reza Shiraz` `true`

۴) `Ali Tehran` `true`

۴۸۶ سخت

خروجی کد زیر چیست؟

```javascript
const a = { id: 1 };
const b = { id: 2 };
const labels = {};
labels[a] = "first";
labels[b] = "second";
console.log(labels[a]);
console.log(Object.keys(labels));
```

۱) `second` `['[object Object]']`

۲) `first` `['[object Object]', '[object Object]']`

۳) `first` `['a', 'b']`

۴) `undefined` `[]`

۴۸۷ سخت

خروجی کد زیر چیست؟

```javascript
const settings = { theme: "dark", size: 12, theme: "light" };
const updated = { ...settings, size: settings.size + 2 };
console.log(settings.theme, updated.size);
console.log(Object.keys(updated));
```

۱) `dark 14` `['theme', 'size', 'theme']`

۲) `Uncaught SyntaxError`

۳) `light 14` `['theme', 'size']`

۴) `light 12` `['size', 'theme']`

۴۸۸ سخت

کدام گزینه **نادرست** است؟

۱) حاصل `"hello".length` برابر `5` است، چون رشته برای یک لحظه در یک شیء پوشاننده (wrapper) قرار می‌گیرد.

۲) حاصل `typeof new Date()` برابر `"object"` است.

۳) حاصل `typeof function () {}` برابر `"function"` است، هرچند توابع هم نوعی شیء‌اند.

۴) حاصل `typeof "hello"` برابر `"object"` است، چون در JavaScript همه چیز شیء است.

۴۸۹ سخت

خروجی کد زیر چیست؟

```javascript
const user = {
  name: "Barry Allen",
  greet() {
    return `Hello, I'm ${this.name}`;
  },
};
const action = "greet";
console.log(user[action]());
console.log(typeof user.greet, typeof user.greet());
```

۱) `Uncaught TypeError`

۲) `Hello, I'm Barry Allen` `function string`

۳) `Hello, I'm undefined` `function string`

۴) `Hello, I'm Barry Allen` `object string`

۴۹۰ سخت

خروجی کد زیر چیست؟

```javascript
const id = Symbol("id");
const field = "score";
const item = { name: "Pen", [id]: 7, [field]: 20 };
console.log(Object.keys(item));
console.log(item[id], item.field, item.score);
```

۱) `['name', 'score']` `7 undefined 20`

۲) `['name', 'Symbol(id)', 'score']` `7 undefined 20`

۳) `['name', 'score']` `undefined undefined 20`

۴) `['name', 'id', 'field']` `7 20 20`

۱-۲۱ آرایه‌ها و متدهای push، pop، shift و unshift

منبع: Introduction to JS، اسلایدهای ۴۵ و ۴۶

درسنامه

فرض کن می‌خواهی نمره‌های ۳۰ دانشجو را نگه داری. ساختن ۳۰ متغیر (`score1`، `score2`، ...) کار عاقلانه‌ای نیست. **array** (آرایه؛ فهرستی **مرتب** از مقدارها) همه را زیر یک نام جمع می‌کند و به هر کدام با یک شماره دسترسی می‌دهد.

یک تشبیه ساده

آرایه مثل یک **قطار** است که واگن‌هایش پشت سر هم شماره خورده‌اند؛ فقط شماره‌گذاری از **صفر** شروع می‌شود. واگن شماره‌ی `0` اولین واگن است، واگن `1` دومی و... . هر واگن یک بار (مقدار) دارد و ترتیب واگن‌ها مهم است. به شماره‌ی هر واگن **index** (اندیس) می‌گوییم.

آرایه چیست؟ (اسلاید ۴۵)

اسلاید ۴۵ این پنج نکته را می‌گوید:

- آرایه‌ها مجموعه‌های **مرتبی** (ordered) از مقدارها را نگه می‌دارند.
- به عضوها (element) با **اندیس** (index) دسترسی داریم و اندیس **از ۰ شروع می‌شود**.
- یک آرایه می‌تواند **نوع‌های مختلف** را کنار هم نگه دارد (عدد، رشته، شیء و ...).
- آرایه‌ها متدهای **آماده** (built-in) برای **اضافه کردن**، **حذف کردن** و **تبدیل کردن** داده‌ها دارند.
- متدهای رایج: `push`، `pop`، `map`، `filter` و `forEach`. (در این درس `push` و `pop` و دوستانشان را می‌بینیم و `map`، `filter` و `forEach` را در درس بعد.)

آرایه را با براکت `[ ]` می‌سازیم و عضوها را با ویرگول جدا می‌کنیم:

```JavaScript
const empty = [];                                    // an empty array
const scores = [18, 20, 17];                         // three numbers
const mixed = [1, "two", true, null, { name: "Ali" }, [5, 6]]; // mixed types

console.log(scores[0]);     // 18  (the FIRST element has index 0)
console.log(scores[2]);     // 17  (the third element)
console.log(mixed[4].name); // "Ali" (an object inside the array)
console.log(mixed[5][1]);   // 6   (an array inside the array)
```

	عضو اول	عضو دوم	عضو سوم
اندیس (index)	`0`	`1`	`2`
مقدار در `scores`	18	20	17

طول آرایه و دسترسی به عضوها

property به نام `length` تعداد عضوهای آرایه را می‌دهد. چون اندیس از ۰ شروع می‌شود، **اندیس آخرین عضو همیشه `length - 1` است:**

```JavaScript
const colors = ["red", "green", "blue"];
console.log(colors.length);              // 3
console.log(colors[colors.length - 1]); // "blue"    (the last element)
console.log(colors[3]);                  // undefined (no element there, but NO error)
console.log(colors[-1]);                 // undefined (negative indexes do NOT count from the end)
console.log(colors.at(-1));              // "blue"    (at() does accept negative indexes)
```

- خواندن اندیسی که عضو ندارد **خطا نمی‌دهد** و فقط `undefined` برمی‌گرداند.
- برخلاف Python، `arr[-1]` آخرین عضو **نیست**؛ برای آخرین عضو بنویس `arr[arr.length - 1]` یا از متد جدیدتر `arr.at(-1)` استفاده کن.

عوض کردن عضوها با اندیس

```JavaScript
const colors = ["red", "green", "blue"];
colors[1] = "yellow";       // replace an element
colors[3] = "black";        // index 3 === length: adds at the end
console.log(colors);        // [ 'red', 'yellow', 'blue', 'black' ]

colors[6] = "white";        // jumping past the end leaves EMPTY slots
console.log(colors.length); // 7
console.log(colors);        // [ 'red', 'yellow', 'blue', 'black', <2 empty items>, 'white' ]
console.log(colors[5]);     // undefined
```

اگر به اندیسی جلوتر از انتهای آرایه مقدار بدهی، خانه‌های بین راه **خالی** (empty) می‌مانند و `length` برابر «بزرگ‌ترین اندیس + ۱» می‌شود. خود `length` را هم می‌توان عوض کرد: `arr.length = 2` آرایه را کوتاه می‌کند و `arr.length = 0` آن را کاملاً خالی می‌کند.

```JavaScript
const nums = [1, 2, 3, 4];
nums.length = 2;
console.log(nums); // [ 1, 2 ]
nums.length = 0;
console.log(nums); // []
```

سؤال اسلاید ۴۵: خروجی این کد چیست؟

```array.js
const numbers = [1, 2, 3];
numbers.push(4);
console.log(numbers[0]);
```

جواب: `1` . `push(4)` عدد ۴ را به **انتهای** آرایه اضافه می‌کند و آرایه `[1, 2, 3, 4]` می‌شود؛ عضوِ اندیس `0` هنوز همان `1` است. دو دام در این سؤال هست:

1. اگر فکر کنی `push` به **ابتدای** آرایه اضافه می‌کند، جواب را `4` می‌دهی (کار ابتدای آرایه با `unshift` است).
2. اگر فکر کنی آرایه‌ی `const` قابل تغییر نیست، جواب را «خطا» می‌دهی. اما فقط **انتساب دوباره** (`numbers = [...]`) را منع می‌کند؛ تغییر **محتوای** آرایه با `push` کاملاً مجاز است.

`push` و `pop` : کار با انتهای آرایه (اسلاید ۴۶)

اسلاید ۴۶ می‌گوید: `push` / `pop` به **انتها** اضافه می‌کنند / از **انتها** حذف می‌کنند.

```JavaScript
const fruits = ["apple", "banana"];

console.log(fruits.push("cherry")); // 3 -> push returns the NEW LENGTH
console.log(fruits);                // [ 'apple', 'banana', 'cherry' ]

console.log(fruits.pop());          // "cherry" -> pop returns the REMOVED element
console.log(fruits);                // [ 'apple', 'banana' ]
```

`shift` و `unshift` : کار با ابتدای آرایه (اسلاید ۴۶)

اسلاید ۴۶ می‌گوید: `shift` / `unshift` از **ابتدا** حذف می‌کنند / به **ابتدا** اضافه می‌کنند. (ترتیب کلمه‌ها با ترتیب متدها جور است: `shift` حذف می‌کند و `unshift` اضافه.)

```JavaScript
const fruits = ["apple", "banana"];

console.log(fruits.unshift("kiwi")); // 3 -> unshift also returns the NEW LENGTH
console.log(fruits);                 // [ 'kiwi', 'apple', 'banana' ]

console.log(fruits.shift());         // "kiwi" -> shift returns the REMOVED element
console.log(fruits);                 // [ 'apple', 'banana' ]
```

وقتی از ابتدا اضافه یا حذف می‌کنیم، اندیس همه‌ی عضوهای دیگر یکی جابه‌جا می‌شود: بعد از `unshift("kiwi")`، `"apple"` که در اندیس `0` بود به اندیس `1` می‌رود.

متد	کجا؟	چه می‌کند؟	چه برمی‌گرداند؟	آرایه‌ی اصلی را تغییر می‌دهد؟
`push(x, ...)`	انتها	یک یا چند عضو **اضافه** می‌کند	طول **جدید** آرایه	بله
`pop()`	انتها	آخرین عضو را **حذف** می‌کند	عضو **حذف‌شده**	بله
`unshift(x, ...)`	ابتدا	یک یا چند عضو **اضافه** می‌کند	طول **جدید** آرایه	بله
`shift()`	ابتدا	اولین عضو را **حذف** می‌کند	عضو **حذف‌شده**	بله

راه حفظ کردن

- `push` و `pop` جفتِ **انتهای** آرایه‌اند و `shift` و `unshift` جفتِ **ابتدای** آن.
- در هر جفت، کلمه‌ی **بلندتر** آرایه را **بلندتر** می‌کند: `push` در برابر `pop`، و `unshift` در برابر `shift`.
- متدهای «اضافه‌کن» (`push` و `unshift`) **طول جدید** را برمی‌گردانند و متدهای «حذف‌کن» (`pop` و `shift`) **عضو حذف‌شده** را.

چند جزئیات مهم دیگر:

```JavaScript
const nums = [3, 4];
nums.unshift(1, 2);           // several values keep their order: [ 1, 2, 3, 4 ]
nums.push(5, 6);              // [ 1, 2, 3, 4, 5, 6 ]
console.log(nums.join(", ")); // "1, 2, 3, 4, 5, 6"

const box = [1, 2];
box.push([3, 4]);             // pushes ONE element (a nested array)
console.log(box.length);      // 3
console.log(box[2]);          // [ 3, 4 ]

const empty = [];
console.log(empty.pop());     // undefined (nothing to remove, no error)
console.log(empty.shift());   // undefined
console.log(empty.length);    // 0
```

- `push` و `unshift` می‌توانند **چند** مقدار بگیرند و آن‌ها را **با همان ترتیب** اضافه می‌کنند؛ حاصل `unshift(1, 2)` روی `[3, 4]` می‌شود `[1, 2, 3, 4]` نه `[2, 1, 3, 4]`.
- اگر به `push` یک **آرایه** بدهی، کل آن آرایه **یک عضو** می‌شود (آرایه‌ی تودرتو)، نه این‌که عضوهایش تک‌تک اضافه شوند.
- `pop` و `shift` روی آرایه‌ی خالی خطا نمی‌دهند و `undefined` برمی‌گردانند.
- هر چهار متد آرایه‌ی **اصلی** را تغییر می‌دهند (mutate؛ «جهش» یا تغییر درجا) و `const` جلویشان را نمی‌گیرد.

پشته و صف

- `push` + `pop` یعنی **پشته** (stack)، مثل بشقاب‌های روی هم: آخرین بشقابی که گذاشتی، اولین بشقابی است که برمی‌داری (LIFO: Last In, First Out).
- `push` + `shift` یعنی **صف** (queue)، مثل صف نانوایی: هر کس اول آمده، اول می‌رود (FIFO: First In, First Out).

`for...of` : پیمایش مستقیم مقدارها (اسلاید ۴۶)

اسلاید ۴۶ می‌گوید `for...of` روی **مقدارهای** هر iterable (چیزی که می‌شود عضوهایش را یکی‌یکی پیمود؛ مثل آرایه و رشته) **مستقیم** حلقه می‌زند؛ یعنی دیگر لازم نیست با اندیس کار کنی:

```JavaScript
const scores = [18, 20, 17];

for (let i = 0; i < scores.length; i++) { // classic for: works with indexes
  console.log(scores[i]);                 // 18, then 20, then 17
}

for (const score of scores) {             // for...of: gives the values directly
  console.log(score);                     // 18, then 20, then 17
}

for (const ch of "Hi!") {                 // strings are iterable too
  console.log(ch);                        // "H", then "i", then "!"
}
```

داخل `for...of` می‌توانی از `break` (خروج از حلقه) و `continue` (رفتن به دور بعد) استفاده کنی. حواست باشد با `for...in` اشتباهش نگیری: `for...in` برای **کلیدهای شیء** است و روی آرایه، **اندیس‌ها** را آن هم به‌صورت **رشته** می‌دهد:

```JavaScript
const letters = ["x", "y"];
for (const i in letters) {
  console.log(i, typeof i); // 0 string, then 1 string
}
for (const v of letters) {
  console.log(v);           // x, then y
}
```

جست‌وجو در آرایه: `includes` و `indexOf`

```JavaScript
const ids = [10, 20, 30, 20];
console.log(ids.includes(20));   // true
console.log(ids.includes("20")); // false (strict comparison like ===, no coercion)
console.log(ids.indexOf(20));    // 1  (index of the FIRST match)
console.log(ids.indexOf(99));    // -1 (not found)
```

- `includes(x)` فقط جواب «هست / نیست» می‌دهد (`true` یا `false`).
- `indexOf(x)` **اندیسِ اولین** عضو برابر با `x` را می‌دهد و اگر پیدا نکند `-1`.
- هر دو مثل `===` مقایسه می‌کنند؛ پس رشته‌ی `"20"` با عدد `20` برابر نیست و دو شیء با محتوای یکسان هم برابر نیستند (مگر همان یک شیء باشند).

آرایه هم شیء است: `typeof` و `Array.isArray`

آرایه در واقع نوع خاصی از **شیء** است که کلیدهایش اندیس‌ها هستند؛ برای همین `typeof` نمی‌تواند آرایه را از شیء معمولی تشخیص دهد:

```JavaScript
console.log(typeof [1, 2]);                    // "object" (not "array"!)
console.log(Array.isArray([1, 2]));            // true
console.log(Array.isArray({ 0: "a", length: 1 })); // false (looks like an array, but is not)
console.log(Array.isArray("abc"));             // false
```

چند متد مفید دیگر: `slice`، `splice`، `join` و `reverse` و `concat`

- `slice(start, end)` : یک **تکه** از آرایه را در آرایه‌ی **جدید** کپی می‌کند؛ از اندیس `start` تا قبل از `end` (خودِ `end` شامل نمی‌شود). آرایه‌ی اصلی **تغییر نمی‌کند**. اندیس منفی از انتها می‌شمارد و `slice()` بدون ورودی کل آرایه را کپی می‌کند.
- `splice(start, deleteCount, ...items)` : **درجا** از اندیس `start` به تعداد `deleteCount` عضو حذف می‌کند و (اگر بدهی) عضوهای تازه را همان‌جا می‌گذارد. آرایه‌ی اصلی **تغییر می‌کند** و خروجی‌اش آرایه‌ای از عضوهای **حذف‌شده** است.

```JavaScript
const letters = ["a", "b", "c", "d", "e"];

console.log(letters.slice(1, 3));     // [ 'b', 'c' ] (index 3 is NOT included)
console.log(letters.slice(-2));       // [ 'd', 'e' ] (the last two)
console.log(letters);                 // [ 'a', 'b', 'c', 'd', 'e' ] (unchanged)

const removed = letters.splice(1, 2); // remove 2 elements starting at index 1
console.log(removed);                 // [ 'b', 'c' ]
console.log(letters);                 // [ 'a', 'd', 'e' ] (changed!)

letters.splice(1, 0, "x");            // remove nothing, insert "x" at index 1
console.log(letters);                 // [ 'a', 'x', 'd', 'e' ]
```

- `join(separator)` : عضوها را با جداکننده‌ی دلخواه به یک **رشته** می‌چسباند (جداکننده‌ی پیش‌فرض `","` است). آرایه را تغییر نمی‌دهد.
- `concat(...)` : آرایه‌ی **جدیدی** از عضوهای آرایه و ورودی‌ها می‌سازد. آرایه را تغییر نمی‌دهد.
- `reverse()` : ترتیب عضوها را **درجا** برعکس می‌کند و **همان آرایه** را برمی‌گرداند (نه یک کپی).

```JavaScript
const nums = [1, 2, 3];
console.log(nums.join());          // "1,2,3"
console.log(nums.join(" - "));     // "1 - 2 - 3"
const more = nums.concat([4, 5]);
console.log(more);                 // [ 1, 2, 3, 4, 5 ]
console.log(nums);                 // [ 1, 2, 3 ] (unchanged)
const back = nums.reverse();
console.log(nums);                 // [ 3, 2, 1 ] (changed in place!)
console.log(back === nums);        // true (the same array)
```

آرایه‌ی اصلی را تغییر می‌دهند (mutating)	آرایه‌ی اصلی را تغییر نمی‌دهند (non-mutating)
`push` ، `pop` ، `shift` ، `unshift` ، `splice` ، `reverse` ، `sort`	`slice` ، `concat` ، `join` ، `includes` ، `indexOf` ، `at`

(جدول کامل‌تر، همراه با `map` ، `filter` ، `reduce` و `sort` را در درس بعد می‌بینیم.)

آرایه‌ها هم با reference کپی می‌شوند

آرایه شیء است، پس مثل هر شیء دیگری متغیر فقط **reference** (نشانی) آن را نگه می‌دارد. `const b = a` آرایه‌ی تازه‌ای نمی‌سازد:

JavaScript

```
const a = [1, 2, 3];
const b = a;                    // the same array, two names
b.push(4);
console.log(a);                 // [ 1, 2, 3, 4 ]
console.log(a === b);           // true

const c = [...a];               // a real (shallow) copy with spread
const d = a.slice();            // another way to copy
c.push(5);
console.log(a.length, c.length); // 4 5
console.log([1, 2] === [1, 2]);  // false (two different arrays)
```

`[...a]` با spread (سه نقطه) یک آرایه‌ی **جدید** با همان عضوها می‌سازد؛ مثل `{ ...obj }` برای شیءها، این کپی هم **سطحی** است.

نکته

- آخرین عضو: `arr[arr.length - 1]` (یا `arr.at(-1)`).
- اندیسِ بدون عضو `undefined` می‌دهد و خطا نمی‌دهد.
- برای فهمیدن «آرایه است یا نه» از `Array.isArray(x)` استفاده کن، نه `typeof`.
- برای کپی آرایه: `[...arr]` یا `arr.slice()`.

دام‌های تستی آرایه‌ها

- `arr[-1]` آخرین عضو **نیست** و `undefined` می‌دهد.
- `push` و `unshift` **طول جدید** را برمی‌گردانند، نه آرایه را؛ `list = list.push(x)` آرایه‌ات را با یک **عدد** عوض می‌کند!
- `pop` و `shift` عضو حذف‌شده را برمی‌گردانند و روی آرایه‌ی خالی `undefined` می‌دهند (نه خطا).
- `arr.push([4, 5])` فقط **یک** عضو (یک آرایه‌ی تودرتو) اضافه می‌کند.
- `if (arr.indexOf(x))` غلط است: اگر `x` در اندیس `0` باشد، `0` falsy است و شرط اجرا نمی‌شود؛ و اگر نباشد، `-1` truthy است و شرط اجرا می‌شود! بنویس `arr.includes(x)` یا `arr.indexOf(x) !== -1`.
- `typeof []` برابر `"object"` است.
- `slice` آرایه را تغییر نمی‌دهد، ولی `splice` تغییر می‌دهد.
- آرایه‌ی `const` را می‌توان با `push`، `pop` و... تغییر داد؛ فقط `arr = [...]` ممنوع است.

جمع‌بندی

- آرایه = فهرست مرتب، با اندیس از ۰؛ `length` تعداد عضوهاست و نوع عضوها می‌تواند مختلف باشد.
- انتها: `push` (اضافه، طول جدید را برمی‌گرداند) و `pop` (حذف، عضو را برمی‌گرداند)؛ ابتدا: `unshift` (اضافه) و `shift` (حذف). هر چهار آرایه را تغییر می‌دهند.
- `for...of` مقدارها را مستقیم می‌دهد؛ `includes` و `indexOf` برای جست‌وجو هستند (`-1` یعنی پیدا نشد).
- `slice` و `concat` و `join` کپی/خروجی تازه می‌سازند؛ `splice` و `reverse` درجا تغییر می‌دهند.

تست‌های تمرینی

۲۷ تست · شماره‌ی ۴۹۱ تا ۵۱۷

۴۹۱ آسان

خروجی کد زیر (سؤال اسلاید ۴۵) چیست؟

```javascript
const numbers = [1, 2, 3];
numbers.push(4);
console.log(numbers[0]);
```

۱) `1`
۲) `4`
۳) `[1, 2, 3, 4]`
۴) `Uncaught TypeError`

۴۹۲ آسان

با فرض `const colors = ["red", "green", "blue"];` حاصل `colors[1]` کدام است؟

۱) `"red"`
۲) `"green"`
۳) `"blue"`
۴) `undefined`

۴۹۳ آسان

طبق اسلاید ۴۶، کدام متد یک عضو را به **ابتدای** آرایه اضافه می‌کند؟

۱) `push`
۲) `pop`
۳) `shift`
۴) `unshift`

۴۹۴ آسان

متد `pop()` چه چیزی را برمی‌گرداند؟

۱) طول جدید آرایه
۲) عضوی که حذف کرده است (آخرین عضو آرایه)
۳) یک آرایه‌ی جدید که عضو آخر را ندارد
۴) همیشه `undefined`، چون فقط آرایه را تغییر می‌دهد

۴۹۵ آسان

حاصل `typeof [1, 2, 3]` کدام است؟

۱) `"array"`
۲) `"list"`
۳) `"object"`
۴) `"number"`

۴۹۶ آسان

خروجی کد زیر چیست؟

```JavaScript
const fruits = ["apple", "kiwi"];
for (const f of fruits) {
  console.log(f);
}
```

۱) `apple` `kiwi`
۲) `0` `1`
۳) `['apple', 'kiwi']`
۴) `Uncaught TypeError`

۴۹۷ آسان

با فرض `const arr = [10, 20, 30];` حاصل `arr[3]` کدام است؟

۱) `30`
۲) `null`
۳) `undefined`
۴) خطای `RangeError`

۴۹۸ متوسط

خروجی کد زیر چیست؟

```JavaScript
const q = [2, 3];
console.log(q.push(4));
console.log(q.unshift(1));
console.log(q.pop());
console.log(q.shift());
console.log(q);
```

۱) `[2, 3]` `[2, 3]` `[1, 2, 3]` `[1, 2, 3, 4]` `[2, 3, 4]`
۲) `[2, 3]` `4` `1` `4` `3`
۳) `[2, 3]` `1` `4` `1` `4`
۴) `[2, 3]` `1` `4` `4` `3`

۴۹۹ متوسط

خروجی کد زیر چیست؟

```JavaScript
const arr = [5, 6, 7];
console.log(arr[-1]);
console.log(arr[arr.length - 1]);
console.log(arr.at(-1));
```

۱) `7` `7` `7`
۲) `undefined` `7` `7`
۳) `Uncaught RangeError`
۴) `undefined` `undefined` `7`

۵۰۰ متوسط

خروجی کد زیر چیست؟

```javascript
const arr = ["a", "b"];
arr[4] = "e";
console.log(arr.length);
console.log(arr[3]);
```

۱) `3` `undefined`
۲) `5` `null`
۳) `Uncaught RangeError`
۴) `5` `undefined`

۵۰۱ متوسط

خروجی کد زیر چیست؟

```javascript
const ids = [10, 20, 30];
console.log(ids.indexOf(20));
console.log(ids.indexOf(40));
console.log(ids.includes("10"));
```

۱) `1` `-1` `false`
۲) `2` `-1` `true`
۳) `1` `undefined` `true`
۴) `1` `-1` `true`

۵۰۲ متوسط

خروجی کد زیر چیست؟

```javascript
const a = [1, 2, 3, 4, 5];
const s = a.slice(1, 3);
console.log(s, a.length);
const r = a.splice(1, 2);
console.log(r, a);
```

۱) `[2, 3] 5` `[2, 3] [1, 4, 5]`
۲) `[2, 3, 4] 5` `[2, 3] [1, 4, 5]`
۳) `[2, 3] 3` `[2, 3] [1, 4, 5]`
۴) `[2, 3] 5` `[2, 3] [1, 2, 3, 4, 5]`

۵۰۳ متوسط

کدام گزینه **درست** است؟

۱) حاصل `typeof [1, 2]` برابر `"array"` است.

۲) حاصل `Array.isArray([1, 2])` برابر `true` و حاصل `Array.isArray({ length: 2 })` برابر `false` است.

۳) `typeof {}` و `typeof []` نتیجه‌های متفاوتی دارند.

۴) حاصل `Array.isArray("abc")` برابر `true` است، چون رشته هم اندیس و `length` دارد.

۵۰۴ متوسط

خروجی کد زیر چیست؟

```JavaScript
const a = [1, 2];
const b = a;
const c = [...a];
b.push(3);
console.log(a.length, c.length, a === b);
```

۱) `2 2 false`
۲) `3 3 true`
۳) `3 2 true`
۴) `2 2 true`

۵۰۵ متوسط

خروجی کد زیر چیست؟

```JavaScript
const letters = ["a", "b", "c"];
const joined = letters.join("-");
const reversed = letters.reverse();
console.log(joined);
console.log(letters);
console.log(reversed === letters);
```

۱) `a-b-c` `['a', 'b', 'c']` `false`
۲) `c-b-a` `['c', 'b', 'a']` `true`
۳) `a-b-c` `['c', 'b', 'a']` `true`
۴) `a-b-c` `['c', 'b', 'a']` `false`

۵۰۶ متوسط

خروجی کد زیر چیست؟

```JavaScript
let list = ["x"];
list = list.push("y");
console.log(list);
```

۱) `2`
۲) `['x', 'y']`
۳) `1`
۴) `Uncaught TypeError`

۵۰۷ متوسط

اجرای کد زیر در کدام خط با خطا متوقف می‌شود؟

```JavaScript
1 const arr = [1, 2, 3];
2 arr.push(4);
3 arr[0] = 100;
4 arr.length = 0;
5 arr = [5, 6];
6 console.log(arr);
```

۱) خط ۲
۲) خط ۴
۳) خط ۵
۴) هیچ خطایی رخ نمی‌دهد

۵۰۸ متوسط

خروجی کد زیر چیست؟

```JavaScript
const letters = ["x", "y"];
for (const i in letters) {
  console.log(i, typeof i);
}
for (const v of letters) {
  console.log(v);
}
```

۱) `x string` `y string` `x` `y`
۲) `0 string` `1 string` `x` `y`
۳) `0 number` `1 number` `x` `y`
۴) `0 string` `1 string` `0` `1`

۵۰۹ سخت

خروجی کد زیر چیست؟

```javascript
const stack = [];
console.log(stack.pop());
console.log(stack.shift());
console.log(stack.length);
console.log(stack.push("a", "b"));
```

۱) `Uncaught RangeError`

۲) `undefined` `undefined` `0` `2`

۳) `undefined` `undefined` `-2` `2`

۴) `null` `null` `0` `['a', 'b']`

۵۱۰ سخت

خروجی کد زیر چیست؟

```javascript
const a = [1, 2];
const n = a.push([3, 4]);
console.log(n, a.length);
console.log(a[2]);
```

۱) `4 4` `3`

۲) `4 4` `[3, 4]`

۳) `[1, 2, [3, 4]] 3` `[3, 4]`

۴) `3 3` `[3, 4]`

۵۱۱ سخت

خروجی کد زیر چیست؟

```javascript
const line = [];
line.push("Ali");
line.push("Sara");
line.unshift("VIP");
const first = line.shift();
line.push(first);
console.log(line.join(", "));
console.log(line.indexOf("Ali"));
```

۱) `Ali, Sara, VIP` `0`

۲) `VIP, Ali, Sara` `1`

۳) `Sara, VIP, Ali` `2`

۴) `Ali, Sara, VIP` `1`

۵۱۲ سخت

خروجی کد زیر چیست؟

```javascript
const mixed = [1, "two", { three: 3 }, [4, 5], null];
console.log(mixed.length);
console.log(typeof mixed[2], typeof mixed[3], typeof mixed[4]);
console.log(mixed[3][1] + mixed[2].three);
```

۱) `5` `object array null` `8`

۲) `6` `object object object` `8`

۳) `5` `object object object` `8`

۴) `5` `object object object` `53`

۵۱۳ سخت

خروجی کد زیر چیست؟

```javascript
const word = "JS!";
let result = "";
for (const ch of word) {
  if (ch === "!") break;
  result = ch + result;
}
console.log(result, word.length);
```

۱) `JS 3`

۲) `!SJ 3`

۳) `SJ 2`

۴) `SJ 3`

۵۱۴ سخت

خروجی کد زیر چیست؟

```javascript
const months = ["Jan", "Mar", "Apr"];
const removed = months.splice(1, 0, "Feb");
console.log(months);
console.log(removed);
```

۱) `['Jan', 'Feb', 'Mar', 'Apr']` `[]`

۲) `['Jan', 'Feb', 'Apr']` `['Mar']`

۳) `['Jan', 'Mar', 'Apr']` `['Feb']`

۴) `['Jan', 'Feb', 'Mar', 'Apr']` `4`

۵۱۵ سخت

خروجی کد زیر چیست؟

```javascript
const users = [{ id: 1 }, { id: 2 }];
const target = users[0];
console.log(users.includes({ id: 1 }));
console.log(users.includes(target));
console.log(users.indexOf(target));
```

۱) `true` `true` `0`
۲) `false` `true` `1`
۳) `false` `false` `-1`
۴) `false` `true` `0`

۵۱۶ سخت

خروجی کد زیر چیست؟

```javascript
const admins = ["root", "ali"];
if (admins.indexOf("root")) {
  console.log("root is admin");
} else {
  console.log("root is not admin");
}
if (admins.indexOf("guest")) {
  console.log("guest is admin");
}
```

۱) `root is admin`
۲) `root is admin` `guest is admin`
۳) `root is not admin` `guest is admin`
۴) `root is not admin`

۵۱۷ سخت

خروجی کد زیر چیست؟

```javascript
const nums = [3, 4];
nums.unshift(1, 2);
nums.push(5, 6);
console.log(nums.join(""));
console.log(nums.shift() + nums.pop());
```

۱) `213456` `7`
۲) `123456` `7`
۳) `123456` `16`
۴) `123465` `7`

۱-۲۲ متدهای reduce و forEach، map، filter، find، findIndex

منبع: Introduction to JS، اسلایدهای ۲۱ و ۴۶ تا ۴۹

درسنامه

در درس قبل با `push`، `pop`، `shift` و `unshift` عضو اضافه و کم کردیم. حالا به ابزارهایی می‌رسیم که احتمالاً بیشترین تست‌های کوییز از آن‌ها طرح می‌شود: متدهایی که روی **همه‌ی عضوهای** آرایه راه می‌روند و با هر عضو کاری انجام می‌دهند: `forEach`، `map`، `filter`، `find`، `findIndex` و `reduce` (اسلایدهای ۴۶ و ۴۷). بعد مثال اسلاید ۴۸ و فعالیت کلاسی اسلاید ۴۹ را کامل حل می‌کنیم و در آخر سراغ `sort` و میم معروف اسلاید ۲۱ می‌رویم.

پیش‌نیاز: callback یعنی چه؟

همه‌ی این متدها یک **تابع** را به‌عنوان ورودی می‌گیرند. به تابعی که به یک تابع (یا متد) دیگر می‌دهیم تا **خودش** آن را صدا بزند، **callback** (تابعِ فراخوانی‌شونده) می‌گوییم. متد، callback ما را **برای هر عضو یک بار** و به ترتیب صدا می‌زند و هر بار سه چیز به آن می‌دهد:

1. **element**: خودِ عضو؛
2. **index**: اندیس آن عضو؛
3. **array**: کل آرایه.

```JavaScript
const fruits = ["apple", "kiwi"];
fruits.forEach((fruit, index, arr) => {
  console.log(index, fruit, arr.length); // 0 apple 2, then 1 kiwi 2
});
```

بیشتر وقت‌ها فقط پارامتر اول را لازم داریم و بقیه را نمی‌نویسیم. ترتیب پارامترها مهم است، نه نامشان. callbackها را معمولاً با **arrow function** (تابع پیکانی؛ جزئیاتش در درس arrow functionها) می‌نویسیم:

نوشتار	یعنی
`n => n * 2`	تابعی که `n` را می‌گیرد و `n * 2` را **برمی‌گرداند** (بدون آکولاد، `return` خودکار است)
`(acc, n) => acc + n`	وقتی دو پارامتر یا بیشتر داریم، پرانتز لازم است
`n => { return n * 2; }`	با آکولاد (بدنه‌ی بلوکی)، `return` را **باید** خودت بنویسی

کارخانه‌ی آرایه‌ها

آرایه را مثل یک **نوار نقاله** پر از جعبه تصور کن:

- `forEach` یک **بازرس** است که سراغ تک‌تک جعبه‌ها می‌رود و کاری انجام می‌دهد (مثلاً چیزی یادداشت می‌کند)، ولی هیچ محصول تازه‌ای تحویل نمی‌دهد.
- `map` یک **دستگاه تبدیل** است: هر جعبه وارد می‌شود و یک جعبه‌ی **تازه** روی نوار نقاله‌ی دیگری بیرون می‌آید؛ تعداد جعبه‌ها همان است.
- `filter` یک **الک** است: فقط جعبه‌هایی که از آزمون رد می‌شوند روی نوار جدید می‌روند.
- `find` یک **کارآگاه** است: به اولین جعبه‌ای که با مشخصات جور است برسد، همان را برمی‌دارد و بقیه را نگاه نمی‌کند.
- `reduce` یک **مخلوط‌کن** است: همه‌ی جعبه‌ها را یکی‌یکی داخلش می‌ریزی و در آخر **یک** چیز تحویل می‌گیری (مثلاً یک عدد یا یک شیء).

`forEach` : برای هر عضو یک کار انجام بده (اسلاید ۴۶)

اسلاید ۴۶: `forEach` تابع داده‌شده را **برای هر عضو** اجرا می‌کند.

```JavaScript
const nums = [1, 2, 3];
const result = nums.forEach(n => console.log(n * 10)); // prints 10, 20, 30
console.log(result); // undefined -> forEach always returns undefined
```

- `forEach` همیشه `undefined` برمی‌گرداند و هر چه callback برگرداند نادیده گرفته می‌شود؛ پس برای «ساختن» چیزی از خروجی‌اش استفاده نکن.
- `forEach` را **نمی‌توان** با `break` متوقف کرد: `break` فقط داخل حلقه معنا دارد و نوشتنش داخل callback خطای `SyntaxError` می‌دهد. `return` داخل callback هم فقط **همان یک فراخوانی** را تمام می‌کند و سراغ عضو بعدی می‌رود؛ یعنی مثل `continue` عمل می‌کند:

```JavaScript
[1, 2, 3, 4].forEach(n => {
  if (n === 2) return; // ends ONLY this call, like "continue"
  console.log(n);      // 1, then 3, then 4
});
```

اسلاید ۴۶ کنار این متدها `for...of` را هم آورده (درس قبل)؛ اگر لازم است وسط کار متوقف شوی، `break` با `for...of` بهترین انتخاب است (یا `find` و `some` که خودشان زود متوقف می‌شوند).

`map` : هر عضو را به یک عضو جدید تبدیل کن (اسلاید ۴۷)

اسلاید ۴۷ درباره‌ی `map` سه نکته می‌گوید:

- هر عضو را به یک عضو تازه **تبدیل** می‌کند و نتیجه را در یک آرایه‌ی **جدید** برمی‌گرداند؛
- آرایه‌ی **اصلی را تغییر نمی‌دهد**؛
- اغلب برای **قالب‌بندی** (formatting) یا **محاسبه** به کار می‌رود.

```JavaScript
const prices = [100, 250, 40];
const doubled = prices.map(p => p * 2);   // calculation
const labels = prices.map(p => p + " $"); // formatting
console.log(doubled); // [ 200, 500, 80 ]
console.log(labels);  // [ '100 $', '250 $', '40 $' ]
console.log(prices);  // [ 100, 250, 40 ] (unchanged)
```

قانون طلایی `map` : **هر چه callback برگرداند، عضوِ آرایه‌ی جدید می‌شود** و طول آرایه‌ی جدید **همیشه برابر** طول آرایه‌ی اصلی است (یک خروجی به ازای هر ورودی). پس اگر callback چیزی برنگرداند، آن خانه `undefined` می‌شود:

```JavaScript
const nums = [1, 2, 3];
console.log(nums.map(n => n * 2));                    // [ 2, 4, 6 ] (no braces: automatic return)
console.log(nums.map(n => { return n * 2; }));        // [ 2, 4, 6 ] (braces + return)
console.log(nums.map(n => { n * 2; }));               // [ undefined, undefined, undefined ]
(forgot return!)
console.log(nums.map(n => { if (n > 1) return n; })); // [ undefined, 2, 3 ] (map cannot skip
elements)
```

`map` نمی‌تواند عضوی را «جا بیندازد»؛ برای حذف عضوها `filter` لازم است.

برگرداندن یک شیء از arrow function: پرانتز لازم است

اگر بلافاصله بعد از `=>` آکولاد بیاید، JavaScript آن را **بدنه‌ی تابع** می‌فهمد، نه یک object literal. برای برگرداندن شیء، آن را در **پرانتز** بگذار:

```JavaScript
const ids = [1, 2];
console.log(ids.map(id => ({ id }))); // [ { id: 1 }, { id: 2 } ]
console.log(ids.map(id => { id }));   // [ undefined, undefined ] -> { } was a function body!
```

در خط دوم، `{ id }` یک بدنه‌ی تابع است که فقط عبارت `id` را حساب می‌کند و چیزی برنمی‌گرداند. همین نکته دلیل پرانتزهای `({ ...s, passed: ... })` در اسلاید ۴۸ است.

`filter` : فقط عضوهایی که شرط را دارند (اسلاید ۴۶)

اسلاید ۴۶: `filter` فقط عضوهایی را **نگه می‌دارد** که با یک شرط جور باشند. callback برای هر عضو یک جواب «درست یا نادرست» برمی‌گرداند؛ اگر جواب truthy باشد (یعنی در شرط مثل `true` رفتار کند)، عضو در آرایه‌ی جدید می‌ماند.

```JavaScript
const nums = [1, 2, 3, 4, 5, 6];
console.log(nums.filter(n => n % 2 === 0));         // [ 2, 4, 6 ]
console.log(nums.filter(n => n > 10));              // [] (no match: an EMPTY array, not undefined)
console.log([0, 1, "", "hi", null].filter(v => v)); // [ 1, 'hi' ] (keeps truthy values)
console.log(nums);                                  // [ 1, 2, 3, 4, 5, 6 ] (unchanged)
```

- خروجی `filter` همیشه یک آرایه‌ی **جدید** است، با طول **کمتر یا مساوی** آرایه‌ی اصلی؛ اگر هیچ عضوی جور نباشد، `[]` .
- خودِ عضوها عوض نمی‌شوند، فقط انتخاب می‌شوند. خروجی callback فقط تعیین می‌کند «بماند یا نماند»؛ برای همین `[0, 1, 2].filter(n => n * 2)` حاصل `[1, 2]` را می‌دهد، نه عددهای دوبرابرشده را!

`find` و `findIndex` : اولین عضوی که شرط را دارد (اسلاید ۴۶)

اسلاید ۴۶: `find` / `findIndex` **اولین عضوی** را که با شرط جور است (یا **اندیس** آن را) می‌دهند.

```JavaScript
const users = [
  { name: "Ali", age: 17 },
  { name: "Sara", age: 22 },
  { name: "Reza", age: 30 },
];
console.log(users.find(u => u.age > 18));          // { name: 'Sara', age: 22 } (only the FIRST match)
console.log(users.findIndex(u => u.age > 18));     // 1
console.log(users.filter(u => u.age > 18).length); // 2 (filter returns ALL matches)
console.log(users.find(u => u.age > 50));          // undefined (no match)
console.log(users.findIndex(u => u.age > 50));     // -1 (no match)
```

	`find`	`findIndex`	`filter`
خروجی	**خودِ** اولین عضوِ جور	**اندیسِ** اولین عضوِ جور	**آرایه‌ی** همه‌ی عضوهای جور
اگر پیدا نشود	`undefined`	`-1`	`[]`
بعد از اولین پیدا شدن	متوقف می‌شود	متوقف می‌شود	تا آخر می‌رود

`find` خودِ عضو را برمی‌گرداند، نه کپی آن را؛ پس اگر عضو یک شیء باشد و آن را تغییر دهی، شیءِ داخل آرایه هم تغییر می‌کند (یادت هست؟ شیءها با reference دست‌به‌دست می‌شوند).

`reduce` : همه را به یک مقدار تبدیل کن (اسلاید ۴۷)

اسلاید ۴۷ درباره‌ی `reduce` می‌گوید:

- همه‌ی عضوها را **با هم ترکیب** می‌کند و **یک مقدار** تحویل می‌دهد؛
- از یک **accumulator** (انباشتگر؛ «نتیجه‌ی تا این لحظه») به‌همراه **مقدار فعلی** (current value) استفاده می‌کند؛
- برای **جمع**، **میانگین**، ساختن **شیء** و ... مفید است.

شکل کلی:

```JavaScript
array.reduce((acc, current) => newAcc, initialValue);
```

callback برای هر عضو یک بار اجرا می‌شود. `acc` نتیجه‌ی قبلی است و **هر چه callback برگرداند، `acc` دور بعد می‌شود.**
`initialValue` مقدار `acc` در دور اول است و آخرین `acc` خروجیِ `reduce` است.

```JavaScript
const total = [5, 10, 20].reduce((acc, n) => acc + n, 0);
console.log(total); // 35
```

دور	acc	n	خروجی callback (یعنی acc دور بعد)
۱	0 (مقدار اولیه)	5	5
۲	5	10	15
۳	15	20	35 (خروجی نهایی)

با مقدار اولیه یا بدون آن؟

اگر `initialValue` را ندهی، **اولین عضو** مقدار اولیه‌ی `acc` می‌شود و پیمایش از **عضو دوم** شروع می‌شود؛ یعنی callback یک بار کمتر اجرا می‌شود:

```JavaScript
const sum = [5, 10, 20].reduce((acc, n) => {
  console.log(acc, n); // 5 10, then 15 20 (only two calls)
  return acc + n;
});
console.log(sum);      // 35
```

اما روی آرایه‌ی **خالی** بدون مقدار اولیه، `reduce` چیزی برای شروع ندارد و خطا می‌دهد:

```JavaScript
console.log([].reduce((acc, n) => acc + n, 0)); // 0 (just the initial value)
console.log([].reduce((acc, n) => acc + n));    // TypeError: Reduce of empty array with no initial value
```

نوع مقدار اولیه هم مهم است، چون نوع `acc` را تعیین می‌کند:

```JavaScript
console.log([1, 2, 3, 4].reduce((acc, n) => acc + n, 0));  // 10
console.log([1, 2, 3, 4].reduce((acc, n) => acc + n, "")); // "1234" (string + number = concatenation)
```

قاعده‌ی عملی: همیشه مقدار اولیه بده؛ `0` برای جمع، `""` برای ساختن رشته، `{}` برای ساختن شیء و `[]` برای ساختن آرایه.

میانگین و بیشینه

```JavaScript
const scores = [18, 15, 20, 11];
const sum = scores.reduce((acc, s) => acc + s, 0);
console.log(sum / scores.length);                          // 16 (average = sum / count)
console.log(scores.reduce((max, s) => (s > max ? s : max))); // 20 (the maximum)
```

ساختن شیء با `reduce` : شمردن تکرارها

```JavaScript
const votes = ["yes", "no", "yes", "yes"];
const count = votes.reduce((acc, v) => {
  acc[v] = (acc[v] || 0) + 1; // first time: acc[v] is undefined, and (undefined || 0) is 0
  return acc;                 // never forget to return the accumulator!
}, {});
console.log(count); // { yes: 3, no: 1 }
```

این‌جا `acc` یک شیء است که کلیدهایش رأی‌ها و مقدارهایش تعداد آن‌هاست (`acc[v]` با براکت، چون کلید داخل متغیر `v` است). اگر `return acc` را فراموش کنی، در دور دوم `acc` برابر `undefined` است و خواندن `acc[v]` خطای `TypeError` می‌دهد.

مثال اسلاید ۴۸: `map` و `reduce` با هم

mapred.js

```
const students = [{ name: "Alice", score: 85 }, { name: "Bob", score: 72 }];

const scores = students.map(s => s.score);
console.log("Scores:", scores);

const total = scores.reduce((acc, s) => acc + s, 0);
const average = total / scores.length;

console.log("Total:", total);
console.log("Average:", average);

const results = students.map(s => ({
  ...s,
  passed: s.score >= 70
}));

console.log("Results:", results);
```

خروجی در Node.js:

```
Scores: [ 85, 72 ]
Total: 157
Average: 78.5
Results: [
  { name: 'Alice', score: 85, passed: true },
  { name: 'Bob', score: 72, passed: true }
]
```

گام‌به‌گام:

1. **استخراج یک فیلد با `map`**: `s => s.score` از هر شیءِ دانشجو فقط نمره‌اش را برمی‌گرداند؛ پس `scores` آرایه‌ی `[85, 72]` است.
2. **جمع با `reduce`**: با مقدار اولیه‌ی `0`: اول `0 + 85 = 85` و بعد `85 + 72 = 157`.
3. **میانگین**: `157 / 2 = 78.5` (`scores.length` برابر `2` است و تقسیم در JavaScript اعشاری است).
4. **ساختن شیءهای جدید با `map` و spread**: برای هر دانشجو یک شیء **تازه** ساخته می‌شود؛ `...s` (spread؛ درس شیءها) همه‌ی propertyهای `s` (یعنی `name` و `score`) را در شیء جدید کپی می‌کند و بعد `passed` به آن اضافه می‌شود. `s.score >= 70` یک boolean است: `85 >= 70` و `72 >= 70` هر دو `true` هستند، پس هر دو دانشجو قبول‌اند.
5. **پرانتز دور شیء**: بدون پرانتز (`s => { ...s, passed: ... }`) آکولاد بدنه‌ی تابع حساب می‌شد و این کد اصلاً اجرا نمی‌شد (`SyntaxError`).
6. آرایه‌ی `students` دست‌نخورده است و عضوهایش ندارند؛ `results` شیءهای **جدید** دارد. Node.js آرایه‌ی `results` را چون طولانی است در چند خط چاپ می‌کند.

فعالیت کلاسی اسلاید ۴۹

سؤال اسلاید: «خروجی این کد را پیش‌بینی کن. وضعیت عضوهای `nums` را در هر مرحله دنبال کن و صریحاً بگو چه اتفاقی برایشان می‌افتد.»

arraymethods.js

```
const nums = [1, 2, 3, 4];

nums.push(5);
nums.pop();

nums.unshift(0);
nums.shift();

const doubled = nums.map(n => n * 2);

const evens = nums.filter(n => n % 2 === 0);

nums.forEach(n => console.log(n));

console.log(nums.find(n => n > 2));
console.log(nums.findIndex(n => n > 2));
```

خط	دستور	مقدار برگشتی	`nums` بعد از این خط
۱	`const nums = [1, 2, 3, 4]`	—	`[1, 2, 3, 4]`
۳	`nums.push(5)`	`5` (طول جدید)	`[1, 2, 3, 4, 5]`
۴	`nums.pop()`	`5` (عضو حذف‌شده)	`[1, 2, 3, 4]`
۶	`nums.unshift(0)`	`5` (طول جدید)	`[0, 1, 2, 3, 4]`
۷	`nums.shift()`	`0` (عضو حذف‌شده)	`[1, 2, 3, 4]`
۹	`nums.map(n => n * 2)`	آرایه‌ی جدید `[2, 4, 6, 8]` در `doubled`	`[1, 2, 3, 4]` (بدون تغییر)
۱۱	`nums.filter(n => n % 2 === 0)`	آرایه‌ی جدید `[2, 4]` در `evens`	`[1, 2, 3, 4]` (بدون تغییر)
۱۳	`nums.forEach(n => console.log(n))`	`undefined` (و چاپ `1`، `2`، `3`، `4`)	`[1, 2, 3, 4]`
۱۵	`nums.find(n => n > 2)`	`3` (چاپ می‌شود)	`[1, 2, 3, 4]`
۱۶	`nums.findIndex(n => n > 2)`	`2` (چاپ می‌شود)	`[1, 2, 3, 4]`

خروجی نهایی (هر عدد در یک خط): `1`، `2`، `3`، `4`، `3`، `2`.

نکته‌های این فعالیت:

- چهار متد اول `nums` را **درجا تغییر می‌دهند**، ولی اثرشان همدیگر را خنثی می‌کند (`push` با `pop`، و `unshift` با `shift`) و `nums` به `[1, 2, 3, 4]` برمی‌گردد.
- `map` و `filter` آرایه‌ی **جدید** می‌سازند و `nums` را تغییر نمی‌دهند. `doubled` و `evens` ساخته می‌شوند ولی هیچ‌جا چاپ نمی‌شوند؛ پس در خروجی اثری ندارند.

- `forEach` هر عضو را در یک خط چاپ می‌کند و خودش `undefined` برمی‌گرداند.
- `find` **اولین عضوِ** بزرگ‌تر از ۲ یعنی `3` را می‌دهد و `findIndex` **اندیسِ** همان عضو یعنی `2` را. اگر کسی فکر کند `map` آرایه‌ی `nums` را دوبرابر کرده، به اشتباه `2` ، `4` ، `6` ، `8` و بعد `4` و `1` را جواب می‌دهد.

زنجیر کردن متدها (chaining)

چون `map` و `filter` آرایه برمی‌گردانند، می‌شود متد بعدی را مستقیم روی نتیجه‌شان صدا زد:

JavaScript

```
const orders = [120, 40, 300, 80];
const total = orders
  .filter(p => p >= 100)          // [ 120, 300 ]
  .map(p => p - 20)               // [ 100, 280 ] (a discount of 20)
  .reduce((sum, p) => sum + p, 0); // 380
console.log(total);               // 380
```

هر متد روی **خروجیِ متد قبلی** کار می‌کند، نه روی آرایه‌ی اصلی. `forEach` چون `undefined` برمی‌گرداند، فقط می‌تواند **آخرِ** زنجیر بیاید.

مقایسه‌ی `map` ، `forEach` و `reduce` (و بقیه)

متد	callback چه برمی‌گرداند؟	خروجی متد	کاربرد
`forEach`	مهم نیست (نادیده گرفته می‌شود)	همیشه `undefined`	یک کار برای هر عضو (چاپ، ارسال، ...)
`map`	عضو جدید	آرایه‌ی **جدید** با **همان طول**	تبدیل هر عضو (قالب‌بندی، محاسبه)
`filter`	شرط (truthy یا falsy)	آرایه‌ی **جدید** با طول کمتر یا مساوی	نگه داشتن عضوهای جور
`find`	شرط	**اولین** عضو جور یا `undefined`	پیدا کردن یک عضو
`findIndex`	شرط	اندیس اولین عضو جور یا `-1`	پیدا کردن جای یک عضو
`reduce`	`acc` جدید	**یک** مقدار از هر نوعی	جمع، میانگین، ساختن شیء
`some` / `every`	شرط	`true` یا `false`	«آیا حداقل یکی / همه جورند؟»

سه سؤال برای انتخاب متد

- فقط می‌خواهی برای هر عضو کاری انجام دهی و نتیجه‌ای لازم نداری؟ `forEach` .
- آرایه‌ای **هم‌اندازه** با عضوهای تبدیل‌شده می‌خواهی؟ `map` . آرایه‌ای با **بعضی** از عضوها؟ `filter` .
- **یک** مقدار می‌خواهی (عدد، رشته یا شیء)؟ `reduce` . فقط یک عضو را می‌خواهی؟ `find` .

جدول بزرگ: کدام متدها آرایه‌ی اصلی را تغییر می‌دهند؟

تغییر می‌دهند (mutating)	چه برمی‌گردانند؟
`push` ، `unshift`	طول جدید
`pop` ، `shift`	عضو حذف‌شده
`splice`	آرایه‌ای از عضوهای حذف‌شده
`sort` ، `reverse`	**همان** آرایه (مرتب‌شده / برعکس‌شده)

تغییر نمی‌دهند (non-mutating)	چه برمی‌گردانند؟
`map` ، `filter` ، `slice` ، `concat`	آرایه‌ی **جدید**
`find` / `findIndex`	عضو / اندیس (یا `undefined` / `-1`)
`reduce`	یک مقدار
`join`	رشته
`includes` ، `some` ، `every`	`true` یا `false`
`forEach`	`undefined`

متدهای non-mutating **خودشان** آرایه را عوض نمی‌کنند، ولی **callback**ِ **تو** می‌تواند: اگر عضوها شیء باشند و داخل `map` یا `forEach` بنویسی `item.qty = 10`، همان شیءهای داخل آرایه‌ی اصلی تغییر می‌کنند (چون شیءها با reference دست‌به‌دست می‌شوند). برای همین در اسلاید ۴۸ با `{ ...s }` شیء **جدید** ساخته شد.

`sort` : هم آرایه را تغییر می‌دهد، هم به‌طور پیش‌فرض رشته‌ای مقایسه می‌کند (اسلاید ۲۱)

میم اسلاید ۲۱ را یادت هست؟ کسی که تازه JavaScript یاد می‌گیرد می‌گوید: «با `array.sort()` این لیست عددها را مرتب می‌کنم!» و JavaScript با خنده جواب می‌دهد: `[1, 100000, 21, 30, 4]` .

دلیلش را در درس تبدیل نوع ضمنی دیدیم: `sort()` بدون **تابع مقایسه** (compare function) عضوها را به **رشته** تبدیل می‌کند و حرف‌به‌حرف، مثل کلمه‌های لغت‌نامه، مقایسه می‌کند؛ `"100000"` قبل از `"21"` می‌آید چون `"1"` از `"2"` کوچک‌تر است. دو نکته‌ی مهم دیگر: `sort` آرایه‌ی **اصلی** را تغییر می‌دهد و **همان آرایه** را برمی‌گرداند.

JavaScript

```
const nums = [4, 30, 1, 100000, 21];
const sorted = nums.sort();
console.log(sorted);          // [ 1, 100000, 21, 30, 4 ] (compared as strings!)
console.log(sorted === nums); // true (sort changed nums itself and returned it)

nums.sort((a, b) => a - b);   // numeric, ascending
console.log(nums);            // [ 1, 4, 21, 30, 100000 ]

const desc = [...nums].sort((a, b) => b - a); // sort a copy, so nums stays as it is
console.log(desc);            // [ 100000, 30, 21, 4, 1 ]
console.log(nums);            // [ 1, 4, 21, 30, 100000 ]
```

تابع مقایسه دو عضو `a` و `b` را می‌گیرد: اگر حاصلش **منفی** باشد `a` جلوتر می‌آید، اگر **مثبت** باشد `b` جلوتر می‌آید و اگر **صفر** باشد ترتیبشان عوض نمی‌شود. پس `(a, b) => a - b` صعودی و `(a, b) => b - a` نزولی مرتب می‌کند. برای شیءها با یک

فیلد مقایسه می‌کنیم: `students.sort((a, b) => b.score - a.score)` دانشجوها را از نمره‌ی بیشتر به کمتر مرتب می‌کند.

> **بیشتر بدانیم: نسخه‌های بدون تغییر**
>
> نسخه‌های جدیدتر JavaScript (ES2023) متدهای `toSorted()` و `toReversed()` را دارند که مثل `sort` و `reverse` کار می‌کنند، ولی آرایه‌ی **جدید** برمی‌گردانند و به آرایه‌ی اصلی دست نمی‌زنند. در کدهای قدیمی‌تر همین کار را با کپی کردن انجام می‌دهند: `[...arr].sort(...)` یا `arr.slice().sort(...)` .

`some` و `every` (به‌طور خلاصه)

```JavaScript
const ages = [16, 21, 30];
console.log(ages.some(a => a < 18));  // true  -> at least ONE element matches (16)
console.log(ages.every(a => a < 18)); // false -> NOT every element matches (21 fails)
console.log(ages.every(a => a > 10)); // true  -> every element matches
```

`some` به اولین جواب `true` که برسد متوقف می‌شود و `every` به اولین جواب `false` . روی آرایه‌ی خالی، همیشه `some` `false` و `every` همیشه `true` است.

> **نکته**
>
> - callbackها سه ورودی می‌گیرند: `(element, index, array)` ؛ ترتیبشان مهم است، نه نامشان.
> - `map` همیشه آرایه‌ای **هم‌طول** برمی‌گرداند، `filter` آرایه‌ای کوتاه‌تر یا مساوی، `find` یک عضو و `reduce` یک مقدار.
> - `find` ، `findIndex` ، `some` و `every` زود متوقف می‌شوند؛ `forEach` ، `map` و `filter` تا آخر می‌روند.
> - برای مرتب‌سازی عددی همیشه تابع مقایسه بده: `(a, b) => a - b` .

> **دام‌های تستی متدهای آرایه**
>
> - `forEach` همیشه `undefined` برمی‌گرداند؛ در `const r = arr.forEach(...)` مقدار `r` برابر `undefined` است.
> - `break` داخل callback `forEach` خطای `SyntaxError` می‌دهد و `return` فقط همان دور را رد می‌کند.
> - `map` با بدنه‌ی آکولاددار و بدون `return` ، آرایه‌ای پر از `undefined` می‌سازد؛ و `map` هیچ‌وقت عضوی را حذف نمی‌کند.
> - برای برگرداندن شیء از arrow function پرانتز لازم است: `x => ({ ... })` .
> - `find` اگر پیدا نکند `undefined` می‌دهد و `findIndex` عدد `-1` . `find` خودِ عضو را می‌دهد، `filter` یک آرایه.
> - `reduce` بدون مقدار اولیه از عضو اول شروع می‌کند و روی آرایه‌ی خالی `TypeError` می‌دهد. مقدار اولیه‌ی `""` جمع را به چسباندن رشته‌ای تبدیل می‌کند.
> - در `reduce` فراموش کردن `return acc` یعنی `acc` در دور بعد `undefined` است.
> - `sort()` بدون تابع مقایسه عددها را رشته‌ای مرتب می‌کند (`[1, 100000, 21, 30, 4]`) و آرایه‌ی اصلی را هم تغییر می‌دهد.
> - `map` ، `filter` ، `slice` و `reduce` آرایه را تغییر نمی‌دهند؛ `push` ، `pop` ، `shift` ، `unshift` ، `splice` و `sort` ، `reverse` تغییر می‌دهند.

جمع‌بندی

- `forEach` : یک کار برای هر عضو؛ خروجی `undefined` .
- `map` : تبدیل؛ آرایه‌ی جدیدِ هم‌طول. `filter` : انتخاب؛ آرایه‌ی جدیدِ کوتاه‌تر یا مساوی.
- `find` / `findIndex` : اولین عضو جور / اندیسش (`undefined` / `-1` اگر نباشد).
- `reduce` : ترکیب همه در یک مقدار با `acc` و مقدار فعلی؛ همیشه مقدار اولیه بده.
- `sort` ، `reverse` و `splice` و چهار متد درس قبل آرایه را تغییر می‌دهند؛ بقیه نه.

تست‌های تمرینی

۳۹ تست · شماره‌ی ۵۱۸ تا ۵۵۶

۵۱۸ آسان

طبق اسلاید ۴۷، کدام توصیف مربوط به متد `map` است؟

۱) آرایه‌ی اصلی را درجا تغییر می‌دهد و چیزی برنمی‌گرداند.

۲) همه‌ی عضوها را با یک accumulator و مقدار فعلی به یک مقدار تبدیل می‌کند.

۳) هر عضو را به یک عضو جدید تبدیل می‌کند و نتیجه را در یک آرایه‌ی **جدید** برمی‌گرداند؛ آرایه‌ی اصلی تغییر نمی‌کند.

۴) فقط عضوهایی را نگه می‌دارد که با یک شرط جور باشند.

۵۱۹ آسان

طبق اسلاید ۴۷، متد `reduce` چه می‌کند؟

۱) همه‌ی عضوها را با استفاده از یک accumulator (انباشتگر) و مقدار فعلی، در **یک مقدار** ترکیب می‌کند.

۲) همان‌طور که از نامش (کاهش) پیداست، عضوهای تکراری را حذف می‌کند و آرایه را کوچک‌تر می‌کند.

۳) آخرین عضو آرایه را حذف می‌کند و آن را برمی‌گرداند.

۴) برای هر عضو یک تابع اجرا می‌کند و همیشه `undefined` برمی‌گرداند.

۵۲۰ آسان

خروجی کد زیر چیست؟

```JavaScript
const nums = [1, 2, 3];
const result = nums.map(n => n * 10);
console.log(result);
console.log(nums);
```

۱) `[10, 20, 30]` `[10, 20, 30]`

۲) `[10, 20, 30]` `[1, 2, 3]`

۳) `60` `[1, 2, 3]`

۴) `undefined` `[10, 20, 30]`

۵۲۱ آسان

در جای خالی کدام متد قرار بگیرد تا خروجی `[ 2, 4 ]` باشد؟

```javascript
console.log([1, 2, 3, 4].___(n => n % 2 === 0));
```

۱) `map`
۲) `find`
۳) `forEach`
۴) `filter`

۵۲۲ آسان

حاصل عبارت زیر کدام است؟

```
[5, 12, 8, 130].find(n => n > 10)
```

۱) `12`
۲) `[12, 130]`
۳) `1`
۴) `true`

۵۲۳ آسان

حاصل `[5, 12, 8].findIndex(n => n > 100)` کدام است؟

۱) `undefined`
۲) `null`
۳) `3`
۴) `-1`

۵۲۴ آسان

خروجی کد زیر چیست؟

```javascript
const nums = [1, 2];
const r = nums.forEach(n => n * 2);
console.log(r);
```

۱) `[2, 4]`
۲) `undefined`
۳) `[1, 2]`
۴) `4`

۵۲۵ آسان

کدام متد آرایه‌ی **اصلی** را تغییر می‌دهد؟

۱) `map`
۲) `filter`
۳) `sort`
۴) `reduce`

۵۲۶ آسان

حاصل `[1, 2, 3, 4].reduce((acc, n) => acc + n, 0)` کدام است؟

۱) `"1234"`
۲) `24`
۳) `10`
۴) `[1, 3, 6, 10]`

۵۲۷ آسان

خروجی کد زیر چیست؟

```JavaScript
const letters = ["a", "b"];
letters.forEach((x, i) => console.log(i + ":" + x));
```

۱) `0:a` `1:b`
۲) `a:0` `b:1`
۳) `1:a` `2:b`
۴) `0:a` `0:b`

۵۲۸ متوسط

خروجی کد زیر (فعالیت کلاسی اسلاید ۴۹) چیست؟

```JavaScript
const nums = [1, 2, 3, 4];

nums.push(5);
nums.pop();

nums.unshift(0);
nums.shift();

const doubled = nums.map(n => n * 2);

const evens = nums.filter(n => n % 2 === 0);

nums.forEach(n => console.log(n));

console.log(nums.find(n => n > 2));
console.log(nums.findIndex(n => n > 2));
```

۱) `0` `1` `2` `3` `4` `3` `2`
۲) `2` `4` `6` `8` `4` `1`
۳) `1` `2` `3` `4` `[3, 4]` `2`
۴) `1` `2` `3` `4` `3` `2`

۵۲۹ متوسط

کد فعالیت کلاسی اسلاید ۴۹ را در نظر بگیر. کدام گزینه **نادرست** است؟

```JavaScript
const nums = [1, 2, 3, 4];

nums.push(5);
nums.pop();

nums.unshift(0);
nums.shift();

const doubled = nums.map(n => n * 2);

const evens = nums.filter(n => n % 2 === 0);

nums.forEach(n => console.log(n));
```

۱) مقدار برگشتی `nums.push(5)` عدد `5` (طول جدید آرایه) است.

۲) مقدار برگشتی `nums.shift()` عدد `0` (عضو حذف‌شده) است.

۳) بعد از خط `map`، خودِ `nums` برابر `[2, 4, 6, 8]` می‌شود.

۴) مقدار برگشتی `nums.forEach(...)` برابر `undefined` است.

۵۳۰ متوسط

خروجی کد زیر (بخش اول مثال اسلاید ۴۸) چیست؟

```JavaScript
const students = [{ name: "Alice", score: 85 }, { name: "Bob", score: 72 }];

const scores = students.map(s => s.score);
console.log("Scores:", scores);

const total = scores.reduce((acc, s) => acc + s, 0);
const average = total / scores.length;

console.log("Total:", total);
console.log("Average:", average);
```

۱) `Scores: [85, 72]` `Total: 157` `Average: 78.5`

۲) `Scores: [85, 72]` `Total: 157` `Average: 78`

۳) `Scores: [{ score: 85 }, { score: 72 }]` `Total: 157` `Average: 78.5`

۴) `Scores: [85, 72]` `Total: 8572` `Average: 4286`

۵۳۱ متوسط

خروجی کد زیر چیست؟

```javascript
const nums = [1, 2, 3];
const doubled = nums.map(n => { n * 2; });
console.log(doubled);
```

۱) `[2, 4, 6]`

۲) `[undefined, undefined, undefined]`

۳) `undefined`

۴) `[]`

۵۳۲ متوسط

خروجی کد زیر چیست؟

```javascript
const ids = [1, 2];
const a = ids.map(id => ({ id }));
const b = ids.map(id => { id });
console.log(a);
console.log(b);
```

۱) `[{ id: 1 }, { id: 2 }]` `[{ id: 1 }, { id: 2 }]`

۲) `[1, 2]` `[{ id: 1 }, { id: 2 }]`

۳) `[{ id: 1 }, { id: 2 }]` `[undefined, undefined]`

۴) `[{ id: 1 }, { id: 2 }]` `Uncaught SyntaxError`

۵۳۳ متوسط

خروجی کد زیر چیست؟

```javascript
const values = [0, 1, "", "hi", null, 5];
const kept = values.filter(v => v);
console.log(kept);
console.log(kept.length, values.length);
```

۱) `[1, 'hi', 5]` `3 6`

۲) `[1, 'hi', 5]` `3 3`

۳) `[0, 1, '', 'hi', null, 5]` `6 6`

۴) `[true, true, true]` `3 6`

۵۳۴ متوسط

خروجی کد زیر چیست؟

```javascript
const result = [10, 20, 30].reduce((acc, n) => {
  console.log(acc, n);
  return acc + n;
});
console.log(result);
```

۱) `0 10` `10 20` `30 30` `60`

۲) `10 20` `30 30` `60`

۳) `undefined 10` `NaN 20` `NaN 30` `NaN`

۴) `10 20` `20 30` `50`

۵۳۵ متوسط

خروجی کد زیر چیست؟

```javascript
const votes = ["yes", "no", "yes", "yes"];
const count = votes.reduce((acc, v) => {
  acc[v] = (acc[v] || 0) + 1;
  return acc;
}, {});
console.log(count);
```

۱) `{ yes: 1, no: 1 }`

۲) `4`

۳) `['yes', 'no']`

۴) `{ yes: 3, no: 1 }`

۵۳۶ متوسط

خروجی کد زیر چیست؟

```javascript
const scores = [18, 15, 20, 11];
const total = scores.reduce((sum, s) => sum + s, 0);
const best = scores.reduce((max, s) => (s > max ? s : max));
console.log(total / scores.length, best);
```

۱) `64 20`

۲) `16 18`

۳) `16 20`

۴) `16 11`

۵۳۷ متوسط

کدام گزینه درباره‌ی `map` و `forEach` **درست** است؟

۱) `forEach` یک آرایه‌ی جدید برمی‌گرداند و `map` چیزی برنمی‌گرداند.

۲) اگر از هر عضو یک عضو تازه بخواهیم و نتیجه را لازم داشته باشیم، `map` مناسب است؛ `forEach` فقط برای هر عضو کاری انجام می‌دهد و `undefined` برمی‌گرداند.

۳) `map` آرایه‌ی اصلی را تغییر می‌دهد، ولی `forEach` تغییر نمی‌دهد.

۴) خروجی `map` ممکن است از آرایه‌ی اصلی کوتاه‌تر باشد، چون عضوهایی را که callback برایشان `false` برگرداند حذف می‌کند.

۵۳۸ متوسط

خروجی کد زیر چیست؟

```JavaScript
const prices = [100, 250, 40, 300];
const total = prices
  .filter(p => p > 50)
  .map(p => p / 2)
  .reduce((sum, p) => sum + p, 0);
console.log(total);
```

۱) `345`

۲) `650`

۳) `[50, 125, 150]`

۴) `325`

۵۳۹ متوسط

خروجی کد زیر چیست؟

```JavaScript
const users = [
  { name: "Ali", age: 17 },
  { name: "Sara", age: 22 },
  { name: "Reza", age: 30 },
];
console.log(users.find(u => u.age > 18).name);
console.log(users.filter(u => u.age > 18).length);
console.log(users.findIndex(u => u.name === "Reza"));
```

۱) `Sara` `2` `2`

۲) `Reza` `2` `2`

۳) `Sara` `1` `3`

۴) `Sara` `2` `3`

۵۴۰ متوسط

خروجی کد زیر چیست؟

```javascript
const nums = [10, 9, 1, 100];
const sorted = nums.sort();
console.log(sorted);
console.log(nums === sorted);
```

۱) `[1, 9, 10, 100]` `false`
۲) `[1, 10, 100, 9]` `false`
۳) `[1, 10, 100, 9]` `true`
۴) `[1, 9, 10, 100]` `true`

۵۴۱ متوسط

خروجی کد زیر چیست؟

```javascript
const ages = [16, 21, 30];
console.log(ages.some(a => a < 18));
console.log(ages.every(a => a < 18));
console.log(ages.every(a => a > 10));
```

۱) `true` `false` `true`
۲) `false` `false` `true`
۳) `true` `true` `true`
۴) `true` `false` `false`

۵۴۲ متوسط

خروجی کد زیر چیست؟

```javascript
const out = [1, 2].map(n => {
  console.log("map", n);
  return n * 2;
});
console.log("done", out);
```

۱) `done [2, 4]` `map 1` `map 2`
۲) `map 1` `map 2` `done [2, 4]`
۳) `map 1` `done [2]` `map 2` `done [2, 4]`
۴) `map 1` `map 2` `done undefined`

۵۴۳ متوسط

در جای خالی کدام مقدار اولیه قرار بگیرد تا خروجی `10` باشد؟

```JavaScript
const nums = [1, 2, 3, 4];
console.log(nums.reduce((acc, n) => acc + n, ___));
```

۱) `0`
۲) `""`
۳) `[]`
۴) `1`

۵۴۴ سخت

خروجی کد زیر چیست؟ (یادآوری: `parseInt(str, radix)` رشته‌ی `str` را در مبنای `radix` به عدد صحیح تبدیل می‌کند؛ مبنای `0` یعنی «پیش‌فرض» (ده‌دهی) و مبنای `1` نامعتبر است.)

```JavaScript
const result = ["1", "2", "3"].map(parseInt);
console.log(result);
```

۱) `[1, 2, 3]`
۲) `[1, NaN, NaN]`
۳) `[NaN, NaN, NaN]`
۴) `['1', '2', '3']`

۵۴۵ سخت

اجرای کد زیر در کدام خط با خطا مواجه می‌شود؟

```JavaScript
1  const nums = [1, 2, 3];
2  for (const n of nums) {
3    if (n === 2) break;
4  }
5  nums.forEach(n => {
6    if (n === 2) break;
7  });
```

۱) خط ۳
۲) خط ۵
۳) خط ۶
۴) هیچ خطایی رخ نمی‌دهد

۵۴۶ سخت

خروجی کد زیر چیست؟

```JavaScript
const empty = [];
console.log(empty.reduce((a, b) => a + b, 0));
console.log(empty.reduce((a, b) => a + b));
```

۱) `0` `0`
۲) `0` `undefined`
۳) `Uncaught TypeError`
۴) `0` `Uncaught TypeError`

۵۴۷ سخت

خروجی کد زیر چیست؟

```javascript
const items = [{ qty: 1 }, { qty: 2 }];
const updated = items.map(item => {
  item.qty = item.qty * 10;
  return item;
});
console.log(items[0].qty, updated === items, updated[0] === items[0]);
```

۱) `1 false false`
۲) `10 true true`
۳) `10 false true`
۴) `1 false true`

۵۴۸ سخت

خروجی کد زیر چیست؟

```javascript
const students = [{ name: "Alice", score: 85 }, { name: "Bob", score: 62 }];
const results = students.map(s => ({ ...s, passed: s.score >= 70 }));
console.log(results[1].passed, students[1].passed, results.length);
```

۱) `false false 2`
۲) `false undefined 2`
۳) `true undefined 2`
۴) `false undefined 1`

۵۴۹ سخت

خروجی کد زیر چیست؟

```javascript
const nums = [1, 2, 3];
console.log(nums.reduce((acc, n) => acc + n, ""));
console.log(nums.reduce((acc, n) => acc + n, 10));
```

۱) `6` `16`
۲) `123` `10123`
۳) `6` `106`
۴) `123` `16`

۵۵۰ سخت

خروجی کد زیر چیست؟

```JavaScript
const products = [{ id: 1, stock: 5 }, { id: 2, stock: 0 }];
const p = products.find(x => x.id === 2);
p.stock = 10;
console.log(products[1].stock);
console.log(products.find(x => x.id === 3));
```

۱) `0` `undefined`

۲) `10` `undefined`

۳) `10` `-1`

۴) `0` `null`

۵۵۱ سخت

در کدام گزینه دستور آخر `3,1,2` را چاپ می‌کند (یعنی آرایه‌ی `arr` دست‌نخورده می‌ماند)؟

```JavaScript
const arr = [3, 1, 2];
```

۱) `arr.sort(); console.log(arr.join());`

۲) `arr.reverse(); console.log(arr.join());`

۳) `const sorted = arr.slice().sort(); console.log(arr.join());`

۴) `arr.splice(0, 1); console.log(arr.join());`

۵۵۲ سخت

کدام دستور `[ 1, 4, 21, 30, 100000 ]` را چاپ می‌کند؟

```JavaScript
const nums = [1, 100000, 21, 30, 4];
```

۱) `console.log(nums.sort());`

۲) `console.log(nums.sort((a, b) => b - a));`

۳) `console.log(nums.reverse());`

۴) `console.log(nums.sort((a, b) => a - b));`

۵۵۳ سخت

خروجی کد زیر چیست؟

```javascript
const nums = [1, 2, 3, 4];
const result = nums.map(n => {
  if (n % 2 === 0) return n * 10;
});
console.log(result);
```

۱) `[undefined, 20, undefined, 40]`
۲) `[20, 40]`
۳) `[1, 20, 3, 40]`
۴) `[false, 20, false, 40]`

۵۵۴ سخت

خروجی کد زیر چیست؟

```javascript
const arr = [1, 2, 3];
const r = arr.forEach((n, i, a) => {
  a[i] = n * n;
});
console.log(arr, r);
```

۱) `[1, 2, 3] undefined`
۲) `[1, 4, 9] [1, 4, 9]`
۳) `[1, 2, 3] [1, 4, 9]`
۴) `[1, 4, 9] undefined`

۵۵۵ سخت

خروجی کد زیر چیست؟

```javascript
const found = [];
[5, 8, 12, 3].forEach(n => {
  if (n > 10) return;
  found.push(n);
});
console.log(found);
```

۱) `[5, 8, 3]`
۲) `[5, 8]`
۳) `[12]`
۴) `[5, 8, 12, 3]`

۵۵۶ سخت

اجرای کد زیر در کدام خط با خطا متوقف می‌شود؟

JavaScript

```
1  const words = ["a", "b", "a"];
2  const count = words.reduce((acc, w) => {
3    acc[w] = (acc[w] || 0) + 1;
4  }, {});
5  console.log(count);
```

۱) خط ۲
۲) خط ۳
۳) خط ۵
۴) هیچ خطایی رخ نمی‌دهد

۱-۲۳ Destructuring (تجزیه‌ی ساختار) و Spread

منبع: Introduction to JS، اسلاید ۴۲

درسنامه

فرض کن از خرید برگشته‌ای و یک کیسه‌ی پر از خوراکی در دست داری. به‌جای این‌که هر بار دست در کیسه کنی و دنبال شیر بگردی، کیسه را **باز می‌کنی** و هر چیز را سر جای خودش می‌گذاری: شیر در یخچال، نان در سبد نان. **destructuring** (تجزیه‌ی ساختار؛ یعنی باز کردن یک شیء یا آرایه و ریختن تکه‌هایش در متغیرهای جداگانه) دقیقاً همین کار را با داده‌ها می‌کند.

> **درباره‌ی عنوان اسلاید ۴۲**
>
> عنوان اسلاید ۴۲ «Destructing» نوشته شده که یک غلط تایپی است؛ اصطلاح درست **destructuring** است (از structure به معنای «ساختار»). این کار هیچ ربطی به «نابود کردن» (destroy) ندارد: شیء یا آرایه‌ی اصلی **دست‌نخورده** باقی می‌ماند و فقط مقدارهایش در متغیرهای تازه ریخته می‌شوند.

چرا destructuring؟

بیا اول ببینیم بدون آن چه کار می‌کردیم:

```JavaScript
const person = { first: "Alice", last: "Doe", age: 30 };

// Without destructuring: one line for each property
const first = person.first;
const age = person.age;
console.log(first, age); // Alice 30
```

با destructuring همان دو خط در **یک خط** خلاصه می‌شود:

```JavaScript
const person = { first: "Alice", last: "Doe", age: 30 };

// With destructuring: one line for all of them
const { first, age } = person;
console.log(first, age); // Alice 30
```

اسلاید ۴۲ ویژگی‌های destructuring را این‌طور خلاصه کرده است:

- مقدارها را **به‌راحتی** از آرایه‌ها یا اشیاء بیرون می‌کشد (Extract values from arrays or objects easily).
- برای اشیاء از `{}` و برای آرایه‌ها از `[]` استفاده می‌کند.
- می‌توان هنگام destructuring نام متغیرها را **عوض کرد** (rename).
- اگر خاصیتی وجود نداشته باشد، می‌توان **مقدار پیش‌فرض** (default value) تعیین کرد.
- کد را **خواناتر** می‌کند (Makes code clearer).

کد اسلاید ۴۲

destructing.js

```
const person = {
  first: "Alice",
  last: "Doe",
  age: 30 };
const { first, age } = person;

const colors = ["red", "green", "blue"];
const [primary, secondary] = colors;

console.log(first, age);         // Alice 30
console.log(primary, secondary); // red green
```

- خط `const { first, age } = person;` دو متغیر جدید به نام‌های `first` و `age` می‌سازد و مقدار خاصیت‌های (property) هم‌نامشان را از `person` در آن‌ها می‌ریزد. خاصیت `last` را نخواسته‌ایم، پس متغیری برایش ساخته نمی‌شود.
- خط `const [primary, secondary] = colors;` اولین عضو آرایه را در `primary` و دومی را در `secondary` می‌ریزد. عضو سوم (`"blue"`) را نخواسته‌ایم، پس نادیده گرفته می‌شود.

> **سمت چپ یا سمت راست مساوی؟**
> آکولاد `{}` و کروشه `[]` وقتی **سمت راست** `=` باشند، شیء یا آرایه **می‌سازند** (literal). وقتی **سمت چپ** `=` (یا جای پارامترهای تابع) بیایند، شیء یا آرایه را **باز می‌کنند** (destructuring). به این شکلِ سمت چپ، **الگو** (pattern) می‌گویند.

destructuring شیء: بر اساس «نام»

در destructuring شیء، JavaScript برای هر نامی که داخل `{ }` نوشته‌ای، دنبال **خاصیتی با همان نام** می‌گردد. پس:

1. **ترتیب مهم نیست.** `const { age, first } = person;` دقیقاً همان نتیجه را می‌دهد.
2. **نام مهم است.** اگر خاصیتی با آن نام وجود نداشته باشد، متغیر ساخته می‌شود ولی مقدارش `undefined` است (خطایی رخ نمی‌دهد).

JavaScript

```
const person = { first: "Alice", last: "Doe", age: 30 };
const { age, first, nickname } = person;
console.log(first, age); // Alice 30
console.log(nickname);   // undefined (person has no "nickname")
```

destructuring آرایه: بر اساس «جایگاه»

اعضای آرایه نام ندارند؛ با **جایگاه** یا **index** (شماره‌ی خانه، از ۰) شناخته می‌شوند. پس متغیر اول عضو اول را می‌گیرد، متغیر دوم عضو دوم را و به همین ترتیب. نام متغیرها هر چیزی می‌تواند باشد.

```JavaScript
const colors = ["red", "green", "blue"];
const [a, b] = colors;        // a = "red", b = "green"
const [, , third] = colors;   // skip two items with empty slots: third = "blue"
const [x, y, z, w] = colors; // w = undefined (there is no 4th item)
console.log(a, b, third, w); // red green blue undefined
console.log(colors);         // [ 'red', 'green', 'blue' ] (the array is unchanged)
```

برای **رد شدن** (skip) از یک عضو، جای آن را خالی بگذار و فقط کاما بنویس: `[, second]` یعنی «اولی را نمی‌خواهم، دومی را در `second` بریز».

	destructuring شیء	destructuring آرایه
علامت	`{ }`	`[ ]`
تطبیق بر اساس	**نام** خاصیت	**جایگاه** (index)
نام متغیرها	همان نام خاصیت (مگر با تغییر نام)	دلخواه
ترتیب نوشتن	مهم نیست	مهم است
خاصیت یا عضوِ ناموجود	`undefined`	`undefined`

تغییر نام متغیر (rename)

گاهی نام خاصیت به کارمان نمی‌آید؛ مثلاً از قبل متغیری با همان نام داریم یا نام گویاتری می‌خواهیم. با `:` می‌شود نام متغیر را عوض کرد:

```JavaScript
const person = { first: "Alice", last: "Doe", age: 30 };
const { first: firstName } = person;
console.log(firstName); // Alice
console.log(first);     // ReferenceError: first is not defined
```

`{ first: firstName }` را این‌طور بخوان: «خاصیت `first` را بردار و در متغیر **جدیدی** به نام `firstName` بریز». سمت چپ `:` **نام خاصیت** (مبدأ) است و سمت راست آن **نام متغیر** (مقصد). مهم‌ترین نکته این است که بعد از تغییر نام، **فقط** `firstName` ساخته می‌شود و متغیری به نام `first` وجود ندارد؛ برای همین خط آخر خطای `ReferenceError` (خطای ارجاع به نامی که تعریف نشده) می‌دهد.

> **یک تشبیه ساده**
>
> در object literal، `{ first: "Alice" }` یعنی «کلید ← مقدار». در الگوی destructuring، `{ first: firstName }` یعنی «کلید ← متغیر مقصد». شکل هر دو یکی است، ولی جهت حرکت داده برعکس است: اولی داده را **در** شیء می‌گذارد و دومی داده را **از** شیء بیرون می‌آورد.

مقدار پیش‌فرض (default value)

اگر خاصیتی وجود نداشته باشد، می‌توانیم با `=` یک مقدار پیش‌فرض بدهیم. این مثال (که با اجرا بررسی شده) تغییر نام و مقدار پیش‌فرض را با هم نشان می‌دهد؛ همان دو قابلیتی که اسلاید ۴۲ نام برده ولی در کدش نیامده است:

```JavaScript
const person = { first: "Alice", last: "Doe", age: 30 };
const { first: firstName, middle = "N/A" } = person;
console.log(firstName, middle); // Alice N/A
```

`person` خاصیتی به نام `middle` ندارد، پس مقدار پیش‌فرض `"N/A"` (مخفف Not Available یعنی «موجود نیست») به کار می‌رود. تغییر نام و پیش‌فرض را می‌توان با هم نوشت: `const { middle: m = "-" } = person;` یعنی «`middle` را در `m` بریز و اگر نبود، `"-"`». آرایه‌ها هم پیش‌فرض دارند: `const [p = "none", q = "none"] = ["tea"];` به `p` مقدار `"tea"` و به `q` مقدار `"none"` می‌دهد.

قانون طلایی: مقدار پیش‌فرض **فقط وقتی** به کار می‌رود که مقدار برابر `undefined` باشد؛ یعنی خاصیت یا عضو وجود نداشته باشد یا صریحاً `undefined` باشد. `null`، `0`، `""` و `false` مقدارهای واقعی‌اند و پیش‌فرض را فعال **نمی‌کنند**:

```JavaScript
const { a = 1, b = 2, c = 3 } = { a: undefined, b: null };
console.log(a, b, c); // 1 null 3

const [x = 5, y = 5, z = 5] = [0, "", false];
console.log([x, y, z]); // [ 0, '', false ]
```

rest در destructuring: «بقیه‌اش مال تو»

سه نقطه‌ی `...` در الگوی destructuring یعنی **rest** (باقی‌مانده): «هر چه ماند را جمع کن و در این متغیر بریز».

```JavaScript
const [head, ...tail] = [1, 2, 3, 4];
console.log(head, tail); // 1 [ 2, 3, 4 ]

const { id, ...info } = { id: 7, name: "Ali", age: 20 };
console.log(id, info); // 7 { name: 'Ali', age: 20 }
```

- rest در آرایه همیشه یک **آرایه** می‌سازد (اگر چیزی نمانده باشد، آرایه‌ی خالی `[]`) و در شیء همیشه یک **شیء**.
- rest باید **آخرین** مورد الگو باشد. `const [...rest, last] = arr;` خطای `SyntaxError: Rest element must be last element` می‌دهد. چون (خطای نحوی) قبل از اجرا و هنگام خواندن فایل پیدا می‌شود، **هیچ خطی** از آن فایل اجرا نمی‌شود، حتی خط‌های قبل از آن.

جابه‌جا کردن دو متغیر (swap)

یکی از زیباترین کاربردهای destructuring، جابه‌جا کردن مقدار دو متغیر بدون متغیر کمکی است:

```JavaScript
let a = 1;
let b = 2;
[a, b] = [b, a];
console.log(a, b); // 2 1
```

اول سمت راست ساخته می‌شود: آرایه‌ی موقت `[2, 1]`. بعد این آرایه در `a` و `b` باز می‌شود. چون متغیرها از قبل وجود دارند، در خط سوم یا `let` نمی‌نویسیم (فقط انتساب است). متغیرها هم باید `let` باشند، چون به `const` نمی‌شود دوباره مقدار داد.

destructuring تودرتو (nested)

اگر داخل شیء، شیء یا آرایه‌ی دیگری باشد، الگو را هم تودرتو می‌نویسیم:

```JavaScript
const user = {
  name: "Sara",
  address: { city: "Tehran", zip: "12345" },
  skills: ["js", "css"],
};
const { address: { city }, skills: [firstSkill] } = user;
console.log(city, firstSkill); // Tehran js
```

این‌جا `address:` و `skills:` فقط «مسیر» را نشان می‌دهند و **متغیر نمی‌سازند**؛ تنها متغیرهای ساخته‌شده `city` و `firstSkill` هستند و استفاده از `address` خطای `ReferenceError` می‌دهد. اگر خود شیء والد وجود نداشته باشد، مثلاً `const { profile: { bio } } = {};` ، خطای `TypeError` (خطای نوع) می‌گیریم، چون نمی‌شود چیزی را از `undefined` بیرون کشید.

destructuring در پارامترهای تابع

می‌توان الگو را مستقیم جای پارامتر نوشت. این‌طوری تابع یک شیء می‌گیرد و همان اول، خاصیت‌های لازم را بیرون می‌کشد (حتی با مقدار پیش‌فرض):

```JavaScript
function printUser({ name, age = 18 }) {
  console.log(name + " is " + age);
}
printUser({ name: "Ali", age: 25 }); // Ali is 25
printUser({ name: "Sara" });         // Sara is 18
printUser(); // TypeError: Cannot destructure property 'name' of 'undefined' as it is undefined.
```

در فراخوانی آخر هیچ آرگومانی نداده‌ایم، پس پارامتر `undefined` است و باز کردن `undefined` ممکن نیست. راه‌حل رایج، دادن پیش‌فرضِ `{}` به **کل پارامتر** است:

```JavaScript
function createUser({ name = "Guest", role = "user" } = {}) {
  return `${name} (${role})`;
}
console.log(createUser({ name: "Sara" })); // Sara (user)
console.log(createUser());                 // Guest (user)
```

destructuring مقدار `null` یا `undefined`

`null` و `undefined` هیچ خاصیتی ندارند، پس باز کردن آن‌ها **همیشه** خطای `TypeError` می‌دهد:

```JavaScript
const settings = { theme: null };
const { color } = settings.theme;
// TypeError: Cannot destructure property 'color' of 'settings.theme' as it is null.
```

در مقابل، رشته‌ها مشکلی ندارند: `const [first] = "hi";` مقدار `"h"` را در `first` می‌ریزد، چون رشته مثل آرایه **iterable** (پیمایش‌پذیر؛ یعنی می‌شود اعضایش را یکی‌یکی گرفت) است. ولی الگوی آرایه روی یک شیء معمولی، مثل `const [a] = { a: 1 };`، خطای `TypeError: ... is not iterable` می‌دهد.

spread: همان سه نقطه، کار برعکس

spread (پخش کردن) همان `...` است ولی کارش برعکس rest است: rest چند چیز را **جمع** می‌کند و spread اعضای یک آرایه یا شیء را **بیرون می‌ریزد** و پخش می‌کند. spread را در سه جا می‌بینی:

۱. داخل آرایه (کپی و ادغام):

```JavaScript
const a = [1, 2];
const b = [...a, 3];          // [ 1, 2, 3 ]
const merged = [...a, ...b]; // [ 1, 2, 1, 2, 3 ]
const copy = [...a];          // a NEW array with the same items
console.log(copy === a);      // false
console.log([..."hi"]);       // [ 'h', 'i' ] (a string is iterable)
```

۲. داخل شیء (کپی، افزودن و بازنویسی):

```JavaScript
const base = { a: 1, b: 2 };
const x = { ...base, b: 3 }; // { a: 1, b: 3 } (the b written later wins)
const y = { b: 3, ...base }; // { b: 2, a: 1 } (base's b overwrites the earlier 3)
console.log(x, y);
```

وقتی یک کلید دو بار بیاید، **آخری برنده است**.

۳. در صدا زدن تابع (پخش آرگومان‌ها):

```JavaScript
const nums = [5, 1, 9];
console.log(Math.max(...nums)); // 9   (same as Math.max(5, 1, 9))
console.log(Math.max(nums));    // NaN (one argument: the whole array)
```

spread در اسلاید ۴۸

در مثال اسلاید ۴۸ (درس متدهای آرایه) این بخش را دیدیم:

```JavaScript
const students = [{ name: "Alice", score: 85 }, { name: "Bob", score: 72 }];
const results = students.map(s => ({
  ...s,
  passed: s.score >= 70
}));
console.log(results[0]);  // { name: 'Alice', score: 85, passed: true }
console.log(students[0]); // { name: 'Alice', score: 85 } (the original is unchanged)
```

`{ ...s, passed: ... }` یعنی «یک شیء **جدید** بساز، همه‌ی خاصیت‌های `s` را در آن کپی کن و خاصیت `passed` را هم اضافه کن». شیء اصلی دانشجو تغییری نمی‌کند. پرانتز دور `{ }` هم لازم است تا arrow function آکولاد را «شیء» بفهمد، نه بدنه‌ی تابع.

کپی با spread «سطحی» (shallow) است

spread فقط **یک لایه** کپی می‌کند. اگر داخل شیء، آرایه یا شیء دیگری باشد، کپی و اصلی هر دو به **همان** آرایه یا شیء درونی اشاره می‌کنند:

```JavaScript
const original = { name: "Ali", tags: ["x"] };
const clone = { ...original };
clone.name = "Sara";  // only the clone changes
clone.tags.push("y"); // the SAME inner array is shared
console.log(original.name, original.tags); // Ali [ 'x', 'y' ]
```

همین‌طور `const b = a;` برای آرایه **کپی نمی‌سازد** و فقط نام دومی برای همان آرایه است، اما `[...a]` یک آرایه‌ی جدید می‌سازد.

rest یا spread؟

	spread (پخش کردن)	rest (جمع کردن)
کار	اعضا را **بیرون می‌ریزد**	باقی‌مانده را **جمع می‌کند**
کجا؟	داخل `[ ]` یا `{ }` سمت راست `=`، و در آرگومان‌های فراخوانی تابع	در الگوی destructuring (سمت چپ `=`) و در پارامترهای تابع
مثال	`[...a, 4]`، `{ ...obj }`، `f(...args)`	`const [x, ...others] = a`، `function f(...args) {}`

نکته

- شیء ← بر اساس **نام**؛ آرایه ← بر اساس **جایگاه**.
- در `{ first: firstName }` جهت را از چپ به راست بخوان: خاصیتِ `first` ← متغیرِ `firstName`.
- پیش‌فرض فقط برای `undefined` (خاصیت یا عضوِ ناموجود) فعال می‌شود.
- rest همیشه آخرین مورد الگوست؛ در آرایه، آرایه می‌سازد و در شیء، شیء.
- برای پیدا کردن حاصل `{ ...a, ...b }` از چپ به راست جلو برو؛ کلید تکراری را آخرین مقدار تعیین می‌کند.

دام تستی

- بعد از `const { first: firstName } = person;` متغیری به نام `first` **وجود ندارد** و استفاده از آن `ReferenceError` است.
- `const { a = 5 } = { a: null };` مقدار `null` می‌دهد، نه `5`.
- باز کردن `null` یا `undefined` خطای `TypeError` می‌دهد؛ مثلاً تابعی با پارامتر `{ title }` که بدون آرگومان صدا زده شود. پیش‌فرض `= {}` برای کل پارامتر این مشکل را حل می‌کند.
- `[...rest, last]` خطای `SyntaxError` است و هیچ خطی از فایل اجرا نمی‌شود.
- `Math.max(nums)` برابر `NaN` است؛ باید بنویسی `Math.max(...nums)`.
- کپی با spread سطحی است: آرایه‌ها و اشیای درونی بین کپی و اصلی **مشترک** می‌مانند.
- destructuring چیزی را «نابود» نمی‌کند؛ آرایه یا شیء اصلی دست‌نخورده می‌ماند.

جمع‌بندی

- destructuring یعنی بیرون کشیدن مقدارها از شیء (`{}` ، بر اساس نام) یا آرایه (`[]` ، بر اساس جایگاه) و ریختن آن‌ها در متغیرهای جدا.
- تغییر نام: `{ prop: newName }` ؛ پیش‌فرض: `{ prop = value }` ؛ هر دو با هم: `{ prop: newName = value }` ؛ رد شدن از عضو: `[, second]` .
- rest (`...` در الگو) باقی‌مانده را جمع می‌کند؛ spread (`...` داخل literal یا آرگومان‌ها) اعضا را پخش می‌کند.
- swap با `[a, b] = [b, a]` ؛ الگوها می‌توانند تودرتو باشند یا جای پارامتر تابع بنشینند.

تست‌های تمرینی

۲۶ تست · شماره‌ی ۵۵۷ تا ۵۸۲

۵۵۷ آسان

destructuring در JavaScript چه کاری انجام می‌دهد؟

۱) شیء یا آرایه را از حافظه پاک (نابود) می‌کند.

۲) مقدارهای یک آرایه یا شیء را بیرون می‌کشد و در متغیرهای جداگانه می‌ریزد.

۳) یک شیء را به رشته‌ی JSON تبدیل می‌کند.

۴) از یک شیء، کپی عمیق (deep copy) کامل می‌سازد.

۵۵۸ آسان

طبق اسلاید ۴۲، در destructuring برای **اشیاء** از ..... و برای **آرایه‌ها** از ..... استفاده می‌شود. کدام گزینه جاهای خالی را به ترتیب درست پر می‌کند؟

۱) `[]` و `{}`　　۲) `()` و `[]`　　۳) `{}` و `[]`　　۴) `{}` و `()`

۵۵۹ آسان

کدام جمله درباره‌ی تفاوت destructuring شیء و آرایه **درست** است؟

۱) هر دو بر اساس جایگاه (index) پر می‌شوند و نام متغیرها اهمیتی ندارد.

۲) در شیء ترتیب نوشتن نام‌ها مهم است، ولی در آرایه نه.

۳) در آرایه، نام متغیرها باید با مقدارهای داخل آرایه یکی باشد.

۴) در شیء متغیرها بر اساس **نام** خاصیت پر می‌شوند و در آرایه بر اساس **جایگاه**.

۵۶۰ آسان

خروجی کد زیر چیست؟

```javascript
const person = { first: "Alice", last: "Doe", age: 30 };
const { age, first } = person;
console.log(first, age);
```

۱) `Alice 30`
۲) `30 Alice`
۳) `undefined undefined`
۴) `Doe Alice`

۵۶۱ آسان

خروجی کد زیر چیست؟

```javascript
const colors = ["red", "green", "blue"];
const [primary, secondary] = colors;
console.log(primary, secondary);
console.log(colors.length);
```

۱) `green blue` `3`
۲) `red green` `3`
۳) `red green` `1`
۴) `red blue` `3`

۵۶۲ متوسط

خروجی کد زیر چیست؟

```javascript
const scores = [10, 20, 30];
const [, second, , fourth] = scores;
console.log(second, fourth);
```

۱) `20 30`
۲) `10 30`
۳) `Uncaught TypeError`
۴) `20 undefined`

۵۶۳ متوسط

خروجی کد زیر چیست؟

```javascript
const person = { first: "Alice", last: "Doe", age: 30 };
const { first: firstName, middle = "N/A" } = person;
console.log(firstName, middle);
```

۱) `Alice N/A`
۲) `Alice undefined`
۳) `first N/A`
۴) `Uncaught ReferenceError`

۵۶۴ متوسط

اجرای کد زیر در کدام خط با خطا متوقف می‌شود؟

```JavaScript
const person = { first: "Alice", age: 30 };
const { first: firstName, age } = person;
console.log(firstName, age);
console.log(first);
```

۱) خط ۲
۲) خط ۳
۳) خط ۴
۴) هیچ خطایی رخ نمی‌دهد

۵۶۵ متوسط

خروجی کد زیر چیست؟

```JavaScript
const { a = 1, b = 2, c = 3 } = { a: undefined, b: null };
console.log(a, b, c);
```

۱) `1 2 3`
۲) `1 null 3`
۳) `undefined null 3`
۴) `undefined null undefined`

۵۶۶ سخت

خروجی کد زیر چیست؟

```JavaScript
const [x = 5, y = 5, z = 5, w = 5] = [0, "", false];
console.log([x, y, z, w]);
```

۱) `[5, 5, 5, 5]`
۲) `[0, '', false, undefined]`
۳) `[5, 5, 5, undefined]`
۴) `[0, '', false, 5]`

۵۶۷ متوسط

کدام گزاره **نادرست** است؟

۱) بعد از `const { x = 10 } = { x: null };` مقدار `x` برابر `10` است.
۲) بعد از `const [a, , b] = [1, 2, 3];` مقدار `b` برابر `3` است.
۳) بعد از `const [first] = "hi";` مقدار `first` برابر `"h"` است.
۴) بعد از `const { a, ...rest } = { a: 1, b: 2, c: 3 };` شیء `rest` دو کلید دارد.

۵۶۸ آسان

خروجی کد زیر چیست؟

```javascript
const [head, ...tail] = [1, 2, 3, 4];
console.log(head, tail);
```

۱) `1 [2, 3, 4]`
۲) `1 2`
۳) `[1] [2, 3, 4]`
۴) `1 4`

۵۶۹ متوسط

خروجی کد زیر چیست؟

```javascript
const product = { id: 7, title: "Pen", price: 20 };
const { id, ...details } = product;
console.log(id, details);
```

۱) `7 { id: 7, title: 'Pen', price: 20 }`
۲) `7 { title: 'Pen', price: 20 }`
۳) `7 ['Pen', 20]`
۴) `7 undefined`

۵۷۰ سخت

خروجی کد زیر چیست؟

```javascript
console.log("start");
const [...rest, last] = [1, 2, 3];
console.log(rest, last);
```

۱) `start` `[1, 2] 3`
۲) `start` `Uncaught SyntaxError`
۳) `Uncaught SyntaxError`
۴) `start` `[1, 2, 3] undefined`

۵۷۱ آسان

خروجی کد زیر چیست؟

```javascript
let a = 1;
let b = 2;
[a, b] = [b, a];
console.log(a, b);
```

۱) `2 1`
۲) `1 2`
۳) `2 2`
۴) `Uncaught SyntaxError`

۵۷۲ متوسط

با توجه به شیء زیر، کدام دستور مقدار `"Tehran"` را در متغیری به نام `city` می‌ریزد، به‌طوری که `console.log(city)` عبارت `Tehran` را چاپ کند؟

```JavaScript
const user = { name: "Sara", address: { city: "Tehran" } };
```

۱) `const { city } = user;`
۲) `const { address.city } = user;`
۳) `const [address, city] = user;`
۴) `const { address: { city } } = user;`

۵۷۳ سخت

خروجی کد زیر چیست؟

```JavaScript
const user = { id: 1, profile: { name: "Reza" } };
const {
  profile: { name: userName, city = "Tehran" },
  id: userId,
} = user;
console.log(userName, city, userId);
```

۱) `Reza undefined 1`
۲) `undefined Tehran 1`
۳) `Reza Tehran 1`
۴) `Uncaught ReferenceError`

۵۷۴ سخت

خروجی کد زیر چیست؟

```JavaScript
function createUser({ name = "Guest", role = "user" } = {}) {
  return `${name} (${role})`;
}
console.log(createUser({ name: "Sara" }));
console.log(createUser());
console.log(createUser({ role: null }));
```

۱) `Sara (user)` `Guest (user)` `Guest (null)`
۲) `Sara (user)` `Guest (user)` `Guest (user)`
۳) `Sara (user)` `Uncaught TypeError`
۴) `Sara (undefined)` `Guest (user)` `Guest (null)`

۵۷۵ سخت

خروجی کد زیر چیست؟

```javascript
function show({ title }) {
  console.log(title);
}
show({ title: "Book" });
show({});
show();
```

۱) `Book` `undefined` `undefined`

۲) `Book` `Uncaught TypeError`

۳) `Book` `undefined` `Uncaught TypeError`

۴) `Book` `Uncaught ReferenceError`

۵۷۶ سخت

خروجی کد زیر چیست؟

```javascript
const settings = { theme: null };
const { theme = "light" } = settings;
console.log(theme);
const { color } = settings.theme;
console.log(color);
```

۱) `light` `undefined`

۲) `null` `Uncaught TypeError`

۳) `light` `Uncaught TypeError`

۴) `null` `undefined`

۵۷۷ آسان

در کدام گزینه `...` نقش **rest** (جمع کردن باقی‌مانده) را دارد، نه spread (پخش کردن)؟

۱) `const copy = [...list];`

۲) `const [first, ...others] = list;`

۳) `Math.max(...list)`

۴) `const user2 = { ...user, age: 30 };`

۵۷۸ متوسط

خروجی کد زیر چیست؟

```javascript
const nums = [5, 1, 9];
console.log(Math.max(...nums));
console.log(Math.max(nums));
```

۱) `9` `NaN`

۲) `9` `9`

۳) `NaN` `9`

۴) `9` `Uncaught TypeError`

۵۷۹ متوسط

خروجی کد زیر چیست؟

```javascript
const base = { a: 1, b: 2 };
const x = { ...base, b: 3 };
const y = { b: 3, ...base };
console.log(x.b, y.b);
```

۱) `3 3`
۲) `2 2`
۳) `2 3`
۴) `3 2`

۵۸۰ متوسط

خروجی کد زیر چیست؟

```javascript
const a = [1, 2];
const b = a;
const c = [...a];
a.push(3);
console.log(b.length, c.length);
```

۱) `2 2`
۲) `3 3`
۳) `3 2`
۴) `2 3`

۵۸۱ سخت

خروجی کد زیر چیست؟

```javascript
const original = { name: "Ali", tags: ["x"] };
const clone = { ...original };
clone.name = "Sara";
clone.tags.push("y");
console.log(original.name, original.tags.length);
```

۱) `Ali 1`
۲) `Sara 2`
۳) `Sara 1`
۴) `Ali 2`

۵۸۲ سخت

کد زیر نسخه‌ی تغییریافته‌ی مثال اسلاید ۴۸ است. خروجی آن چیست؟

```javascript
const students = [
  { name: "Alice", score: 85 },
  { name: "Bob", score: 62 },
];
const results = students.map((s) => ({ ...s, passed: s.score >= 70 }));
console.log(results[1].passed, students[1].passed);
console.log(results[0] === students[0]);
```

۱) `false false` `true`
۲) `false undefined` `false`
۳) `true undefined` `false`
۴) `false false` `false`

۱-۲۴ کلاس‌ها (Classes) در JavaScript

منبع: Introduction to JS، اسلاید ۳۹

درسنامه

فرض کن یک شرکت ساختمانی **نقشه‌ی** (blueprint) یک خانه را کشیده است. خود نقشه خانه نیست و نمی‌شود در آن زندگی کرد؛ ولی با همین یک نقشه می‌شود ده خانه ساخت. همه‌ی خانه‌ها یک شکل‌اند (همان اتاق‌ها، همان درها)، ولی هر کدام صاحب، رنگ و وسایل **خودش** را دارد. در JavaScript، **class** (کلاس) همین نقشه است و هر شیئی که از روی آن ساخته می‌شود یک **instance** (نمونه) نام دارد.

چرا کلاس؟

فرض کن می‌خواهی چند کاربر بسازی. با object literal (که در درس اشیاء دیدیم) باید ساختار و متد هر کاربر را **تکرار** کنی:

```JavaScript
const kermit = { name: "Kermit", age: 22, greet() { console.log("Hello, " + this.name); } };
const piggy = { name: "Piggy", age: 30, greet() { console.log("Hello, " + this.name); } };
// ...and the same code again for every new user
```

اگر روزی بخواهی پیام `greet` را عوض کنی، باید همه‌جا را تغییر دهی. کلاس این مشکل را حل می‌کند: ساختار را **یک بار** تعریف می‌کنی و هر چند نمونه که بخواهی می‌سازی.

اسلاید ۳۹ درباره‌ی کلاس‌ها سه نکته می‌گوید:

- در **ES6** به‌عنوان راهی **تمیزتر** (cleaner) برای ساختن اشیاء معرفی شدند.
- با کلمه‌ی کلیدی `class` تعریف می‌شوند.
- کلاس‌ها یک **blueprint** (نقشه یا الگو) برای ساختن اشیاء تعریف می‌کنند.

کد اسلاید ۳۹

```classes.js
class User {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }
  greet() {
    console.log("Hello, " + this.name);
  }
}

const kermit = new User("Kermit", 22);
kermit.greet(); // Hello, Kermit
```

بیا خط‌به‌خط بخوانیم:

- `class User { ... }` : کلاسی به نام `User` تعریف می‌کند. طبق قرارداد، نام کلاس با حرف **بزرگ** شروع می‌شود (PascalCase).

- `constructor(name, age) { ... }` : **constructor** (سازنده) متد ویژه‌ای است که هر بار با `new` یک نمونه‌ی جدید ساخته شود، **خودکار** اجرا می‌شود. کارش مقداردهی اولیه به خاصیت‌های شیء تازه است.
- `this.name = name;` : این‌جا `this` همان **شیء تازه‌ای است که در حال ساخته شدن است**. این خط روی آن شیء خاصیتی (property) به نام `name` می‌سازد و پارامتر `name` را در آن می‌ریزد.
- `greet() { ... }` : یک **method** (متد؛ تابعی که متعلق به شیء است). در بدنه‌ی کلاس کلمه‌ی `function` نمی‌نویسیم و بین متدها **کاما نمی‌گذاریم**.
- `new User("Kermit", 22)` : یک نمونه‌ی جدید می‌سازد؛ آرگومان‌ها به `constructor` می‌روند.
- `kermit.greet()` : متد را روی `kermit` صدا می‌زند. چون `kermit` قبل از نقطه آمده، داخل `greet` مقدار `this` برابر `kermit` است و `Hello, Kermit` چاپ می‌شود.

اصطلاح	یعنی	در مثال
class	نقشه یا الگوی ساخت اشیاء	`User`
instance (نمونه)	شیئی که از روی کلاس ساخته شده	`kermit`
constructor (سازنده)	متدی که موقع `new` خودکار اجرا می‌شود	`constructor(name, age)`
property (خاصیت)	داده‌ی هر نمونه	`name` و `age`
method (متد)	رفتار مشترک نمونه‌ها	`greet`

`new` دقیقاً چه کار می‌کند؟

وقتی می‌نویسی `new User("Kermit", 22)`، این چهار قدم انجام می‌شود:

1. یک شیء خالی جدید `{}` ساخته می‌شود.
2. این شیء به `User.prototype` وصل می‌شود؛ جایی که متدهای کلاس (مثل `greet`) نگه‌داری می‌شوند.
3. `constructor` با `this` برابر همان شیء جدید اجرا می‌شود و خاصیت‌ها را پر می‌کند.
4. شیء ساخته‌شده **خودکار** برگردانده می‌شود (لازم نیست در `constructor` دستور `return` بنویسی).

JavaScript

```
class User {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }
  greet() {
    console.log("Hello, " + this.name);
  }
}
const kermit = new User("Kermit", 22);
console.log(kermit);                 // User { name: 'Kermit', age: 22 }
console.log(typeof kermit);          // object
console.log(kermit instanceof User); // true
```

عملگر `instanceof` می‌پرسد: «آیا این شیء از روی این کلاس (یا یکی از کلاس‌های فرزندش) ساخته شده است؟».

داده‌ها جدا، متدها مشترک

هر نمونه خاصیت‌های **خودش** را دارد و تغییر یکی روی بقیه اثر نمی‌گذارد. اما متدها برای هر نمونه کپی **نمی‌شوند**: فقط یک نسخه از `greet` در `User.prototype` هست و همه‌ی نمونه‌ها از همان استفاده می‌کنند.

```JavaScript
// (User is the class of slide 39)
const kermit = new User("Kermit", 22);
const piggy = new User("Piggy", 30);
piggy.name = "Miss Piggy";                  // changes only piggy
console.log(kermit.name);                   // Kermit
console.log(kermit.greet === piggy.greet); // true: one shared method
console.log(Object.keys(kermit));           // [ 'name', 'age' ] (greet is not an own property)
```

این همان **Prototype-based OO** (شیءگرایی مبتنی بر prototype) است که در درس «Transpile و ویژگی‌های زبان JavaScript» دیدیم: اگر شیء خاصیتی را خودش نداشته باشد، JavaScript آن را در **prototype** (الگوی والدِ شیء) جست‌وجو می‌کند. کلاس‌های ES6 در واقع **syntactic sugar** (ظاهر شیرین‌تر و تمیزتر) برای همین سازوکار قدیمی‌اند؛ برای همین `typeof User` برابر `"function"` است، نه `"class"`. به همین دلیل `Object.keys` و `JSON.stringify` فقط داده‌های خودِ نمونه (`name` و `age`) را نشان می‌دهند، نه متدها را.

چند قانون مهم

۱. بدون `new` نمی‌شود کلاس را صدا زد.

```JavaScript
// (User is the class of slide 39)
const piggy = User("Piggy", 30);
// TypeError: Class constructor User cannot be invoked without 'new'
```

۲. کلاس قبل از خط تعریفش قابل استفاده نیست. اعلان کلاس **hoist** (بالا برده) می‌شود ولی مقداردهی نمی‌شود و تا خط تعریفش در **TDZ** (Temporal Dead Zone، منطقه‌ی مرده‌ی زمانی) می‌ماند؛ درست مثل `let` و `const`. این همان hoisting «جزئی» (partially) است که اسلاید ۳۸ درباره‌ی `class` گفته بود:

```JavaScript
const a = new Animal("Rex"); // ReferenceError: Cannot access 'Animal' before initialization
class Animal {
  constructor(name) {
    this.name = name;
  }
}
```

در مقابل، یک **function declaration** (تابعی که با کلمه‌ی `function` اعلان شده) را می‌شود قبل از خط تعریفش صدا زد.

۳. آرگومانی که داده نشود، `undefined` است. `new User("Kermit")` نمونه‌ای می‌سازد که `age` آن `undefined` است؛ خطایی رخ نمی‌دهد.

۴. نوشتن `constructor` اختیاری است. اگر ننویسی، یک constructor خالی به‌طور پیش‌فرض در نظر گرفته می‌شود.

۵. بین متدها کاما نگذار. در object literal بین اعضا کاما لازم است، ولی در بدنه‌ی کلاس کاما خطای `SyntaxError` (خطای نحوی) است:

```JavaScript
class Bad {
  a() { return 1; },  // SyntaxError: Unexpected token ','
  b() { return 2; }
}
```

۶. بدنه‌ی کلاس همیشه در strict mode (حالت سخت‌گیرانه) است. اگر متدی را از نمونه جدا کنی و تنها صدا بزنی، `this` آن `undefined` می‌شود و خواندن `this.name` خطای `TypeError` (خطای نوع) می‌دهد. جزئیاتش را در درس «کلمه‌ی کلیدی this» می‌بینیم.

class field و static (آشنایی کوتاه)

- **class field** (فیلد کلاس): خاصیتی که مستقیم در بدنه‌ی کلاس (بیرون از `constructor`) تعریف می‌شود. هر نمونه نسخه‌ی **خودش** را می‌گیرد.
- **static** (ایستا): متد یا فیلدی که به **خود کلاس** تعلق دارد، نه به نمونه‌ها. با `ClassName.member` استفاده می‌شود و روی نمونه‌ها وجود ندارد.

```JavaScript
class Cart {
  items = [];       // class field: every instance gets its OWN array
  static count = 0; // static field: belongs to the class itself
  constructor(owner) {
    this.owner = owner;
    Cart.count++;
  }
  add(item) {
    this.items.push(item);
    return this;    // returning this allows chaining: cart.add(a).add(b)
  }
  static describe() {
    return "A shopping cart";
  }
}
const c1 = new Cart("Ali");
const c2 = new Cart("Sara");
c1.add("pen").add("book");
console.log(c1.items.length, c2.items.length); // 2 0
console.log(Cart.count, Cart.describe());      // 2 A shopping cart
console.log(typeof c1.describe);               // undefined (static is not on instances)
```

ارث‌بری: `extends` و `super`

گاهی یک کلاس نسخه‌ی خاص‌ترِ کلاس دیگری است: هر «سگ» یک «حیوان» است. با `extends` کلاس فرزند (child) همه‌ی متدهای کلاس والد (parent) را **ارث** می‌برد (inheritance):

JavaScript

```
class Animal {
  constructor(name) {
    this.name = name;
  }
  speak() {
    return this.name + " makes a sound";
  }
}

class Dog extends Animal {
  constructor(name, breed) {
    super(name);        // run the parent constructor FIRST
    this.breed = breed; // only now "this" can be used
  }
  speak() {             // overriding: replaces the parent's speak for dogs
    return super.speak() + " (woof)"; // super.speak() calls the parent's version
  }
}

const rex = new Dog("Rex", "Husky");
console.log(rex.speak());           // Rex makes a sound (woof)
console.log(rex.breed);             // Husky
console.log(rex instanceof Dog);    // true
console.log(rex instanceof Animal); // true (a Dog is also an Animal)
```

- `super(...)` در constructor فرزند، constructor والد را اجرا می‌کند. در کلاس فرزند، تا وقتی `super(...)` صدا زده نشده، **نمی‌شود از `this` استفاده کرد**؛ نوشتن `this.breed = breed;` قبل از `super(name);` خطای `ReferenceError: Must call super constructor in derived class before accessing 'this' ...` می‌دهد.
- اگر کلاس فرزند `constructor` نداشته باشد، constructor پیش‌فرضش همه‌ی آرگومان‌ها را به والد می‌دهد: با `class Cat extends Animal {}`، عبارت `new Cat("Tom").name` برابر `"Tom"` است.
- **override** (بازنویسی متد): اگر فرزند متدی هم‌نام با والد تعریف کند، برای نمونه‌های فرزند نسخه‌ی فرزند اجرا می‌شود. با `super.method()` می‌توان نسخه‌ی والد را هم صدا زد.
- `instanceof` کل زنجیره را بررسی می‌کند: `rex instanceof Animal` برابر `true` است، ولی `new Animal("x") instanceof Dog` برابر `false`.

> **بیشتر بدانیم**
>
> در فصل سوم (TypeScript) دوباره به کلاس‌ها برمی‌گردیم؛ آنجا نوع‌ها و access modifierهایی مثل `public` و `private` هم اضافه می‌شوند. این‌جا فقط کلاس‌های خودِ JavaScript را دیدیم.

نکته

- کلاس = نقشه؛ نمونه = شیئی که با `new` ساخته می‌شود. `typeof` کلاس `"function"` و `typeof` نمونه `"object"` است.
- `constructor` با هر `new` یک بار و خودکار اجرا می‌شود و `this` داخل آن، نمونه‌ی تازه است.
- داده‌ها (خاصیت‌ها و class fieldها) برای هر نمونه جداست؛ متدها در prototype هستند و بین همه‌ی نمونه‌ها مشترک‌اند.
- با `extends` متدها ارث می‌رسند و با `super` به constructor و متدهای والد دسترسی داری.

دام تستی

- صدا زدن کلاس بدون `new` خطای `TypeError` می‌دهد.
- استفاده از کلاس قبل از خط تعریفش `ReferenceError` (TDZ) است، در حالی که function declaration قبل از تعریف هم کار می‌کند.
- در constructor کلاس فرزند، استفاده از `this` قبل از `super(...)` خطای `ReferenceError` می‌دهد.
- کاما بین متدهای کلاس `SyntaxError` است.
- متد `static` روی نمونه وجود ندارد: `instance.staticMethod()` خطای `TypeError: ... is not a function` می‌دهد.
- متد جداشده از نمونه (`const g = kermit.greet; g();`) `this` را از دست می‌دهد و چون کلاس همیشه strict است، `TypeError` می‌دهد.

جمع‌بندی

- `class` (از ES6) راهی تمیزتر برای ساختن اشیای هم‌شکل است: نقشه یک بار تعریف می‌شود و با `new` هر تعداد نمونه ساخته می‌شود.
- `new` یک شیء خالی می‌سازد، آن را به prototype وصل می‌کند، `constructor` را با `this` برابر آن شیء اجرا می‌کند و شیء را برمی‌گرداند.
- کلاس روی prototypeها ساخته شده (`typeof` آن `"function"` است) و hoist می‌شود، ولی تا خط تعریفش در TDZ است.
- `extends` و `super` برای ارث‌بری، `static` برای اعضای خودِ کلاس و class field برای خاصیت‌های هر نمونه.

تست‌های تمرینی

۲۰ تست · شماره‌ی ۵۸۳ تا ۶۰۲

۵۸۳ آسان

خروجی کد زیر (کد اسلاید ۳۹) چیست؟

```JavaScript
class User {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }
  greet() {
    console.log("Hello, " + this.name);
  }
}

const kermit = new User("Kermit", 22);
kermit.greet();
```

۱) `Hello, undefined`
۲) `Hello, Kermit`
۳) `Hello, name`
۴) `Uncaught TypeError`

۵۸۴ آسان

طبق اسلاید ۳۹، کدام جمله درباره‌ی کلاس‌ها در JavaScript **درست** است؟

۱) کلاس‌ها در ES5 معرفی شدند و با کلمه‌ی کلیدی `object` تعریف می‌شوند.
۲) از هر کلاس فقط یک شیء می‌توان ساخت.
۳) کلاس‌ها در ES6 به‌عنوان راهی تمیزتر برای ساختن اشیاء معرفی شدند و یک blueprint (نقشه) برای ساختن اشیاء تعریف می‌کنند.
۴) خود کلاس یک نمونه (instance) است که با `new` ساخته شده است.

۵۸۵ آسان

در کد اسلاید ۳۹، شیء `kermit` که با `const kermit = new User("Kermit", 22);` ساخته شده، نسبت به `User` چیست؟

۱) یک نمونه (instance) از کلاس `User`
۲) یک کلاس فرزند (subclass) از `User`
۳) `User` کلاس constructor
۴) یک متد (method) از کلاس `User`

۵۸۶ آسان

در جای خالی کدام کلمه قرار بگیرد تا کد زیر یک نمونه از کلاس `User` بسازد و `Hello, Kermit` را چاپ کند؟

```JavaScript
class User {
  constructor(name) {
    this.name = name;
  }
  greet() {
    console.log("Hello, " + this.name);
  }
}
const kermit = ____ User("Kermit");
kermit.greet();
```

۱) `this`
۲) `create`
۳) `class`
۴) `new`

۵۸۷ آسان

متد `constructor` یک کلاس چه زمانی اجرا می‌شود؟

۱) یک بار، هنگام تعریف کلاس، حتی اگر هیچ نمونه‌ای ساخته نشود.
۲) هر بار که با `new` یک نمونه‌ی جدید ساخته می‌شود، به‌طور خودکار.
۳) فقط وقتی برنامه‌نویس آن را مستقیم با `kermit.constructor()` صدا بزند.
۴) هر بار که یکی از متدهای نمونه (مثل `greet`) صدا زده می‌شود.

۵۸۸ آسان

خروجی کد زیر چیست؟

```JavaScript
class User {
  constructor(name) {
    this.name = name;
  }
}
const kermit = new User("Kermit");
console.log(typeof User, typeof kermit, kermit instanceof User);
```

۱) `class object true`
۲) `object object true`
۳) `function object true`
۴) `function function false`

۵۸۹ سخت

درباره‌ی اجرای `new User("Kermit", 22)` (با کلاس اسلاید ۳۹) کدام جمله **نادرست** است؟

۱) یک شیء خالی جدید ساخته می‌شود.

۲) `constructor` با `this` برابر همان شیء جدید اجرا می‌شود.

۳) شیء ساخته‌شده خودکار برگردانده می‌شود، با اینکه `constructor` دستور `return` ندارد.

۴) متد `greet` برای این نمونه جداگانه کپی می‌شود و جزو خاصیت‌های خودِ (own) آن شیء است.

۵۹۰ متوسط

خروجی کد زیر چیست؟

```JavaScript
class User {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }
  greet() {
    console.log("Hello, " + this.name);
  }
}
const kermit = new User("Kermit", 22);
console.log(Object.keys(kermit));
```

۱) `['name', 'age']`

۲) `['name', 'age', 'greet']`

۳) `['constructor', 'greet']`

۴) `[]`

۵۹۱ متوسط

با فرض کد زیر، کدام گزاره **درست** است؟

```JavaScript
class User {
  constructor(name) {
    this.name = name;
  }
  greet() {
    return "Hello, " + this.name;
  }
}
const a = new User("Ali");
const b = new User("Sara");
```

۱) `a === b` برابر `true` است، چون هر دو از یک کلاس ساخته شده‌اند.

۲) `a.greet === b.greet` برابر `true` است.

۳) `typeof a` برابر `"function"` است.

۴) آرایه‌ی `Object.keys(a)` شامل `"greet"` است.

۵۹۲ متوسط

خروجی کد زیر چیست؟

```javascript
class Counter {
  constructor() {
    this.count = 0;
  }
  inc() {
    this.count++;
    return this.count;
  }
}
const a = new Counter();
const b = new Counter();
a.inc();
a.inc();
b.inc();
console.log(a.count, b.count);
```

۱) `3 3`
۲) `2 2`
۳) `1 1`
۴) `2 1`

۵۹۳ متوسط

خروجی کد زیر چیست؟

```javascript
class User {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }
  greet() {
    console.log("Hello, " + this.name);
  }
}
const kermit = new User("Kermit");
kermit.greet();
console.log(kermit.age);
```

۱) `Hello, Kermit` `undefined`
۲) `Hello, Kermit` `null`
۳) `Hello, Kermit` `0`
۴) `Uncaught TypeError`

۵۹۴ متوسط

اجرای کد زیر در کدام خط با خطا متوقف می‌شود؟

JavaScript

```
1  class User {
2    constructor(name) {
3      this.name = name;
4    }
5  }
6  const a = new User("Ali");
7  console.log(a.name);
8  const b = User("Sara");
9  console.log(b.name);
```

۱) خط ۶
۲) خط ۷
۳) خط ۸
۴) هیچ خطایی رخ نمی‌دهد

۵۹۵ سخت

اجرای کد زیر در کدام خط با خطا متوقف می‌شود؟

JavaScript

```
1  const a = makeUser("Ali");
2  console.log(a.name);
3  const b = new Person("Sara");
4  class Person {
5    constructor(name) {
6      this.name = name;
7    }
8  }
9  function makeUser(name) {
10   return { name };
11 }
```

۱) خط ۱
۲) خط ۳
۳) خط ۴
۴) هیچ خطایی رخ نمی‌دهد

۵۹۶ متوسط

اجرای کدام قطعه‌کد **خطا** می‌دهد؟

۱) `class A { a() { return 1; }, b() { return 2; } }`
۲) `const o = { a() { return 1; }, b() { return 2; } };`
۳) `class A { a() { return 1; } b() { return 2; } }`
۴) `class A { constructor() { this.x = 1; } }`

۵۹۷ متوسط

خروجی کد زیر چیست؟

```JavaScript
class MathUtil {
  static square(x) {
    return x * x;
  }
}
console.log(MathUtil.square(4));
const m = new MathUtil();
console.log(m.square(4));
```

۱) `16` `16`
۲) `16` `undefined`
۳) `Uncaught TypeError`
۴) `16` `Uncaught TypeError`

۵۹۸ متوسط

خروجی کد زیر چیست؟

```JavaScript
class Cart {
  items = [];
  add(item) {
    this.items.push(item);
    return this;
  }
}
const c1 = new Cart();
const c2 = new Cart();
c1.add("pen").add("book");
console.log(c1.items.length, c2.items.length);
```

۱) `2 2`
۲) `1 0`
۳) `2 0`
۴) `Uncaught TypeError`

۵۹۹ سخت

اجرای کد زیر در کدام خط با خطا متوقف می‌شود؟

JavaScript

```
1  class Animal {
2    constructor(name) {
3      this.name = name;
4    }
5  }
6  class Dog extends Animal {
7    constructor(name, breed) {
8      this.breed = breed;
9      super(name);
10   }
11 }
12 const d = new Dog("Rex", "Husky");
13 console.log(d.name);
```

۱) خط ۳
۲) خط ۸
۳) خط ۹
۴) هیچ خطایی رخ نمی‌دهد

۶۰۰ سخت

خروجی کد زیر چیست؟

JavaScript

```
class Animal {
  constructor(name) {
    this.name = name;
  }
  speak() {
    return this.name + " makes a sound";
  }
}
class Dog extends Animal {
  speak() {
    return super.speak() + " (woof)";
  }
}
const d = new Dog("Rex");
console.log(d.speak());
console.log(d instanceof Animal, d instanceof Dog);
```

۱) `Rex makes a sound` `true true`
۲) `undefined makes a sound (woof)` `false true`
۳) `Rex makes a sound (woof)` `true true`
۴) `Rex makes a sound (woof)` `false true`

۶۰۱ سخت

خروجی کد زیر چیست؟

```JavaScript
class Animal {
  constructor(name) {
    this.name = name;
  }
}
class Cat extends Animal {}
const tom = new Cat("Tom");
console.log(tom.name, tom instanceof Animal);
```

۱) `Tom true`
۲) `undefined true`
۳) `Tom false`
۴) `Uncaught ReferenceError`

۶۰۲ سخت

خروجی کد زیر چیست؟

```JavaScript
class Shape {}
class Circle extends Shape {}
const c = new Circle();
console.log(c instanceof Circle, c instanceof Shape, new Shape() instanceof Circle);
```

۱) `true false false`
۲) `true true true`
۳) `false true false`
۴) `true true false`

۱-۲۵ کلمه‌ی کلیدی this

منبع: Introduction to JS، اسلایدهای ۳۶، ۴۳ و ۴۴

درسنامه

به جمله‌ی «**من** گرسنه‌ام» فکر کن. اگر علی این را بگوید، یعنی علی گرسنه است؛ اگر سارا بگوید، یعنی سارا. خودِ جمله عوض نشده، ولی معنی «من» به **گوینده** بستگی دارد. کلمه‌ی کلیدی `this` (به معنای «این»؛ ارجاعی به شیئی که تابع را صدا زده) در JavaScript همین «من» است: داخل کدِ تابع نوشته می‌شود، ولی این‌که به چه شیئی اشاره کند به این بستگی دارد که **چه کسی و چطور** آن تابع را صدا زده است.

اسلایدها چه می‌گویند؟

اسلاید ۴۳:

- `this` به شیئی اشاره می‌کند که تابع را **صدا می‌زند** (Refers to the object that is calling the function).
- در اشیاء، `this` به **خود شیء** اشاره می‌کند.
- در کلاس‌ها، `this` به **نمونه‌ی (instance) ساخته‌شده** اشاره می‌کند.
- در arrow functionها، `this` از **محدوده‌ی بیرونی** (outer scope) به ارث می‌رسد.
- مقدار `this` به **نحوه‌ی صدا زدن** تابع بستگی دارد، **نه محل تعریف** آن.

اسلاید ۴۴ (ادامه):

- به شیئی اشاره می‌کند که تابع را صدا می‌زند؛ مقدارش به **بافتِ فراخوانی** (call context) بستگی دارد، نه به محل اعلان.
- در **محدوده‌ی سراسری** (global scope): `window` (در مرورگر) یا `undefined` (در strict mode).
- در **متدها** (method؛ تابعی که خاصیتِ یک شیء است): شیئی که **قبل از نقطه** آمده است.
- در **arrow functionها**: `this` را از محدوده‌ی بیرونی به ارث می‌برد (**lexical this**؛ یعنی `this`ی که از محل نوشته شدن کد تعیین می‌شود، نه از محل صدا زدن).
- می‌توان آن را به‌صورت دستی با `call`، `apply` یا `bind` تعیین کرد.

اسلاید ۳۶ هم در مقایسه‌ی دو روش تعریف تابع گفته بود: «Functions have their own `this`; arrows inherit `this`»؛ یعنی تابع‌های معمولی `this` مخصوص خودشان را دارند ولی arrow functionها آن را از بیرون به ارث می‌برند. در این درس همه‌ی این جمله‌ها را باز می‌کنیم.

کد اسلاید ۴۴

sample.js

```
const obj = {
  name: "Robert",
  say() {
    console.log(this.name);
  }
};
obj.say(); // "Robert"
const f = obj.say;
f(); // undefined (or global)
```

- `obj.say()` : قبل از نقطه `obj` آمده، پس `this` برابر `obj` است و `Robert` چاپ می‌شود.
- `const f = obj.say;` : فقط **خود تابع** در `f` ریخته می‌شود. تابع هیچ اتصال دائمی به `obj` ندارد.
- `f()` : این بار هیچ شیئی قبل از نقطه نیست؛ یک **صدا زدن ساده** (plain call) است و `this` دیگر `obj` نیست. نتیجه به محیط اجرا بستگی دارد:

محیط اجرا	`this` داخل `f()`	خروجی `f()`
Node.js، فایل معمولی `.js` (CommonJS، بدون strict)	شیء سراسری (`globalThis`)	`undefined` (شیء سراسری Node.js خاصیت `name` ندارد)
مرورگر، تگ `<script>` معمولی (بدون strict)	`window`	یک خط **خالی** (چون `window.name` یک رشته‌ی واقعی و معمولاً `""` است)
strict mode (`"use strict"`) یا ES module	`undefined`	`TypeError: Cannot read properties of undefined (reading 'name')`

منظور اسلاید از «(or global)» همین است: در حالت عادی `this` برابر **شیء سراسری** (global object) می‌شود؛ شیئی که متغیرها و توابع سراسری روی آن قرار دارند. نام استاندارد آن در همه‌ی محیط‌ها `globalThis` است (در مرورگر همان `window`). **strict mode** (حالت سخت‌گیرانه) با نوشتن `"use strict";` در ابتدای فایل یا تابع فعال می‌شود و بعضی رفتارهای خطرناک قدیمی را ممنوع می‌کند. **ES moduleها** (فایل‌هایی که با `import` / `export` کار می‌کنند) و بدنه‌ی کلاس‌ها **همیشه** strict هستند.

چهار قانون پیدا کردن `this`

برای یک تابع معمولی (غیر arrow)، به **محل صدا زدن** (call site) نگاه کن و این سؤال‌ها را **به همین ترتیب** بپرس؛ اولین جواب «بله» تعیین‌کننده است:

اولویت	شکل فراخوانی	`this` برابر است با	مثال
۱	با `new`	شیء تازه‌ای که ساخته می‌شود	`new User("Ali")`
۲	با `call` / `apply` / `bind`	شیئی که صریحاً داده شده	`f.call(obj)`
۳	به شکل متد: `obj.f()`	شیء قبل از نقطه	`obj.say()`
۴	صدا زدن ساده: `f()`	شیء سراسری (`window` یا `globalThis`)؛ در strict mode: `undefined`	`f()`

و **arrow function** از هیچ‌کدام از این قانون‌ها پیروی نمی‌کند: `this` آن همان `this` جایی است که arrow در آن **نوشته شده** است.

قانون ۳: متد؛ شیءِ قبل از نقطه

فقط شیئی مهم است که **دقیقاً قبل از آخرین نقطه** آمده است:

```JavaScript
const user = {
  name: "outer",
  profile: {
    name: "inner",
    show() {
      console.log(this.name);
    },
  },
};
user.profile.show(); // inner (the object right before the dot is user.profile)
```

و چون `this` به **فراخوانی** بستگی دارد نه به تعریف، یک تابع می‌تواند در اشیای مختلف `this` های مختلفی داشته باشد:

```JavaScript
const robert = {
  name: "Robert",
  say() {
    return "I am " + this.name;
  },
};
const anna = { name: "Anna" };
anna.say = robert.say;   // the SAME function is now stored in anna too
console.log(anna.say()); // I am Anna (called on anna)
```

قانون ۴: صدا زدن ساده و محدوده‌ی سراسری

وقتی تابعی بدون هیچ شیئی صدا زده شود (مثل `f()` در اسلاید ۴۴)، در حالت عادی `this` برابر شیء سراسری است و در strict mode برابر `undefined` . جمله‌ی اسلاید ۴۴ («In global scope: window (browser) or undefined (strict mode)») را دقیق‌تر این‌طور بخوان:

- در مرورگر، `this` در **بالاترین سطح** یک تگ `<script>` معمولی همیشه `window` است (حتی در strict mode).
- تابع معمولی‌ای که به‌صورت **ساده** صدا زده شود: بدون strict، `this` برابر `window` است و در strict mode برابر `undefined` . «undefined (strict mode)» اسلاید همین حالت است.
- در بالاترین سطح یک ES module، `this` برابر `undefined` است.
- در Node.js، در بالاترین سطح یک فایل CommonJS، `this` برابر `module.exports` است که در ابتدا یک شیء خالی است:

```JavaScript
// Node.js, a normal CommonJS file (app.js)
console.log(this);                  // {}
console.log(this === module.exports); // true
```

یک تفاوت مرورگر و Node.js: در مرورگر، `var` سراسری (و function declaration سراسری) خاصیتِ `window` می‌شود، ولی `let` و `const` نمی‌شوند:

JavaScript

```
// Browser, a classic <script> (not strict)
var city = "Tabriz";
function show() {
  console.log(this.city);
}
show();     // Tabriz (this = window, and var created window.city)
const obj = { city: "Shiraz", show };
obj.show(); // Shiraz (the same function, called as a method)
```

قانون ۲: `call` ، `apply` و `bind`

هر تابع سه متد دارد که با آن‌ها می‌توانی `this` را **دستی** تعیین کنی. اولین آرگومانِ هر سه، همان شیئی است که می‌خواهی `this` شود:

JavaScript

```
function introduce(greeting, punctuation) {
  return greeting + ", " + this.name + punctuation;
}
const robert = { name: "Robert" };

console.log(introduce.call(robert, "Hi", "!"));       // Hi, Robert!
console.log(introduce.apply(robert, ["Hello", "?"])); // Hello, Robert?
const bound = introduce.bind(robert); // nothing runs yet: a NEW function is returned
console.log(bound("Hey", "."));       // Hey, Robert.
```

متد	تابع را همان لحظه اجرا می‌کند؟	آرگومان‌ها	چه برمی‌گرداند؟
`call(thisArg, a, b)`	بله	جداجدا، با کاما	نتیجه‌ی تابع
`apply(thisArg, [a, b])`	بله	داخل **یک آرایه**	نتیجه‌ی تابع
`bind(thisArg)`	**خیر**	هنگام صدا زدن تابع جدید	یک **تابع جدید** با `this` دائمی

برای حفظ کردن: **C**all با **C**omma (کاما)، **A**pply با **A**rray (آرایه).

با همین ابزار، مشکل خط آخر اسلاید ۴۴ حل می‌شود (هر سه را اجرا کرده‌ایم):

JavaScript

```
const obj = {
  name: "Robert",
  say() {
    console.log(this.name);
  }
};
const f = obj.say;
f.call(obj);   // Robert
f.apply(obj);  // Robert
f.bind(obj)(); // Robert
```

تابعی که با `bind` ساخته شده، `this`ش **برای همیشه** قفل است: نه `call` و `apply` و نه یک `bind` دوباره نمی‌توانند آن را عوض کنند:

```JavaScript
function show() {
  console.log(this.name);
}
const a = { name: "A" };
const b = { name: "B" };
const f = show.bind(a);
f();         // A
f.call(b);   // A (a bound function ignores call/apply)
f.bind(b)(); // A (binding again has no effect)
```

قانون ۱: `new`

وقتی تابعی با `new` صدا زده شود، یک شیء تازه ساخته می‌شود و `this` به آن اشاره می‌کند. کلاس‌ها (درس «کلاس‌ها (Classes) در JavaScript») همین کار را می‌کنند و قبل از ES6 همین کار با تابع‌های معمولی انجام می‌شد:

```JavaScript
function Car(model) {
  this.model = model; // "this" is the brand-new object
}
const c = new Car("Pride");
console.log(c.model); // Pride
```

اگر چنین تابعی را **بدون** `new` صدا بزنی، یک صدا زدن ساده است: در حالت عادی `this` شیء سراسری می‌شود و `model` به‌اشتباه روی شیء سراسری ساخته می‌شود! کلاس‌ها برای جلوگیری از همین اشتباه، بدون `new` خطای `TypeError` می‌دهند.

arrow function و lexical this

arrow function `this` **مخصوص خودش را ندارد**؛ وقتی داخلش از `this` استفاده کنی، JavaScript آن را مثل یک متغیر معمولی از محدوده‌ی بیرونی (جایی که arrow **نوشته شده**) برمی‌دارد. نتیجه‌ها:

۱. arrow به‌عنوان متد شیء، شیء را نمی‌شناسد (دام معروف):

```JavaScript
const counter = {
  count: 10,
  regular() {
    return this.count; // this = counter
  },
  arrow: () => this.count, // this = the OUTER this, not counter
};
console.log(counter.regular()); // 10
console.log(counter.arrow());   // undefined (Node.js: the outer this is module.exports = {})
```

آکولادهای object literal محدوده‌ی جدیدی برای `this` نمی‌سازند؛ پس `this` در `arrow` همان `this` بیرون از شیء است: در Node.js شیء `module.exports` و در یک `<script>` مرورگر، `window` .

۲. arrow داخل متد، `this` متد را نگه می‌دارد (کاربرد درست):

```JavaScript
const team = {
  name: "Blue",
  members: ["Ali", "Sara"],
  list() {
    return this.members.map((m) => m + " of " + this.name); // the arrow uses list's this
  },
};
console.log(team.list()); // [ 'Ali of Blue', 'Sara of Blue' ]
```

در مقابل، یک تابع **معمولی** تودرتو `this` را گم می‌کند، چون به‌صورت ساده صدا زده می‌شود:

```JavaScript
const team2 = {
  name: "Blue",
  show() {
    function inner() {
      return this.name; // inner() is a plain call: this is NOT team2
    }
    return inner();
  },
};
console.log(team2.show()); // undefined in Node.js (TypeError in strict mode)
```

۳. `call`، `apply` و `bind` روی `this` arrow اثری ندارند:

```JavaScript
const arrow = () => this;
const obj = { name: "obj" };
console.log(arrow.call(obj) === obj); // false
```

۴. arrow function را نمی‌شود با `new` صدا زد (`TypeError: ... is not a constructor`).

callbackها و گم شدن `this`

callback (تابعی که به تابع دیگری داده می‌شود تا بعداً صدایش بزنند) یکی از رایج‌ترین جاهای گم شدن `this` است:

```JavaScript
const timer = {
  label: "T1",
  report() {
    console.log("label:", this.label);
  },
};
setTimeout(timer.report, 0);             // label: undefined (only the function is passed)
setTimeout(() => timer.report(), 0);     // label: T1 (fix 1: an arrow wrapper makes a real method
call)
setTimeout(timer.report.bind(timer), 0); // label: T1 (fix 2: bind)
```

`setTimeout(timer.report, 0)` فقط **خود تابع** را تحویل می‌دهد، نه `timer.` را. بعداً تابع بدون `timer` اجرا می‌شود و `this` دیگر `timer` نیست (در مرورگر `window` و در Node.js یک شیء داخلی به نام Timeout؛ در هر دو خاصیت `label`

وجود ندارد). اشتباه رایج دیگر نوشتن `setTimeout(timer.report(), 0)` است: این‌جا تابع **همان لحظه** اجرا می‌شود و به‌جای تابع، مقدار برگشتی‌اش (`undefined`) به `setTimeout` می‌رسد؛ Node.js در این حالت خطای `TypeError` می‌دهد.

همین داستان داخل یک متد (در Node.js):

```JavaScript
const user = {
  name: "Sara",
  later() {
    setTimeout(() => console.log("arrow:", this.name), 0); // arrow: Sara
    setTimeout(function () {
      console.log("function:", this.name);                 // function: undefined (Node.js)
    }, 0);
  },
};
user.later();
```

arrow داخل `later` نوشته شده و `this` آن همان `user` است؛ ولی `function` معمولی را تایمر صدا می‌زند، نه `user` ، پس `this` آن دیگر `user` نیست.

`this` در کلاس‌ها

در `constructor` و در متدهایی که به شکل `instance.method()` صدا زده می‌شوند، `this` همان **نمونه** است (اسلاید ۴۳). اما بدنه‌ی کلاس همیشه strict است؛ پس متدی که از نمونه جدا شود، `this` برابر `undefined` می‌گیرد:

```JavaScript
class User {
  constructor(name) {
    this.name = name;
  }
  greet() {
    return "Hello, " + this.name;
  }
}
const kermit = new User("Kermit");
console.log(kermit.greet()); // Hello, Kermit
const greet = kermit.greet;
greet(); // TypeError: Cannot read properties of undefined (reading 'name')
```

دو راه‌حل رایج: در `constructor` بنویسی `this.greet = this.greet.bind(this);` ، یا متد را به شکل یک class field با arrow function تعریف کنی: `greet = () => "Hello, " + this.name;` (این arrow، `this` را از نمونه‌ی در حال ساخت می‌گیرد).

پیش‌نمایش فصل دوم: `this` در event handlerها

در فصل دوم با **event handler** (تابعی که هنگام رخ دادن یک رویداد، مثل کلیک، اجرا می‌شود) کار می‌کنیم. اگر handler یک تابع معمولی باشد، `this` همان عنصری است که listener روی آن نصب شده؛ اگر arrow باشد، `this` بیرونی (در یک `<script>` معمولی، `window`) را نگه می‌دارد:

JavaScript

```
const btn = document.querySelector("#btn"); // <button id="btn">Save</button>
btn.addEventListener("click", function () {
  console.log(this.id);          // btn (this = the element)
});
btn.addEventListener("click", () => {
  console.log(this === window); // true (an arrow keeps the outer this)
});
```

نکته

- برای تابع معمولی فقط به **خط صدا زدن** نگاه کن، نه به جایی که تابع تعریف شده؛ به ترتیب: `new` ← `call` / `apply` / `bind` ← `obj.f()` ← `f()` .
- برای arrow function فقط به **جای نوشته شدن** نگاه کن: `this` آن همان `this` بیرونی است و با هیچ روشی عوض نمی‌شود.
- `call` و `apply` همان لحظه اجرا می‌کنند (Comma / Array)؛ `bind` اجرا نمی‌کند و تابع جدیدی با `this` دائمی برمی‌گرداند.
- برای متدهای شیء شکل کوتاه `say() {}` (یا `function`) بنویس، نه arrow؛ برای callbackهای داخل متد، arrow بهترین انتخاب است.

دام تستی

- `const f = obj.say; f();` شیء را گم می‌کند: Node.js (CommonJS، بدون strict) ← `undefined` ؛ مرورگر ← خط خالی (`window.name` برابر `""`)؛ strict mode یا ES module ← `TypeError` . **همیشه محیط اجرا را در صورت سؤال پیدا کن.**
- arrow function به‌عنوان متد شیء، `this` را از بیرون می‌گیرد، نه از خود شیء.
- `setTimeout(obj.say, 1000)` شیء را گم می‌کند و `setTimeout(obj.say(), 1000)` تابع را **همان لحظه** اجرا می‌کند. درستش: `setTimeout(() => obj.say(), 1000)` یا `setTimeout(obj.say.bind(obj), 1000)` .
- تابع معمولی تودرتو داخل متد، `this` متد را به ارث نمی‌برد؛ arrow تودرتو می‌برد.
- متد جداشده از نمونه‌ی کلاس، حتی در فایلِ بدون `"use strict"` ، خطای `TypeError` می‌دهد، چون بدنه‌ی کلاس همیشه strict است.
- در بالاترین سطح یک فایل CommonJS در Node.js، `this` برابر `{}` (یعنی `module.exports`) است، نه `window` یا `undefined` .

جمع‌بندی

فراخوانی	this
`new F()`	شیء تازه
`f.call(o)` / `f.apply(o)` / `f.bind(o)()`	`o`
`o.f()`	`o` (شیء قبل از نقطه)
`f()`	`window` یا `globalThis` (بدون strict)؛ `undefined` (strict)
arrow function	`this` محدوده‌ی بیرونی (lexical)؛ غیرقابل تغییر

تست‌های تمرینی

۳۳ تست · شماره‌ی ۶۰۳ تا ۶۳۵

۶۰۳ آسان

طبق اسلاید ۴۳، کلمه‌ی کلیدی `this` (در حالت کلی) به چه چیزی اشاره می‌کند؟

۱) به خودِ تابعی که `this` داخل آن نوشته شده است.

۲) همیشه به شیء سراسری `window`.

۳) به شیئی که تابع را صدا می‌زند.

۴) به اولین شیئی که در برنامه ساخته شده است.

۶۰۴ آسان

طبق اسلایدهای ۴۳ و ۴۴، مقدار `this` در یک تابع معمولی به چه چیزی بستگی دارد؟

۱) به نحوه‌ی صدا زدن تابع (call context)، نه محل تعریف آن.

۲) به محل تعریف تابع در کد، نه نحوه‌ی صدا زدن آن.

۳) به نام تابع و تعداد پارامترهایش.

۴) به نوع مقداری که تابع برمی‌گرداند.

۶۰۵ آسان

طبق اسلاید ۴۴، `this` در محدوده‌ی سراسری (global scope) چیست؟

۱) همیشه `null`

۲) در مرورگر `document` و در strict mode یک شیء خالی `{}`

۳) همیشه شیء `console`

۴) در مرورگر `window` و در strict mode برابر `undefined`

۶۰۶ آسان

خروجی کد زیر چیست؟

```JavaScript
const obj = {
  name: "Robert",
  say() {
    console.log(this.name);
  }
};
obj.say();
```

۱) `undefined`
۲) `Robert`
۳) `say`
۴) `Uncaught TypeError`

۶۰۷ آسان

اسلاید ۳۶ می‌گوید: «تابع‌های معمولی `this` مخصوص خودشان را دارند، ولی arrow functionها `this` را .....». جای خالی کدام است؟

۱) همیشه برابر `undefined` می‌گیرند
۲) از اولین آرگومانی که به آن‌ها داده می‌شود می‌گیرند
۳) از شیئی که قبل از نقطه آمده می‌گیرند
۴) از محدوده‌ی بیرونی به ارث می‌برند

۶۰۸ آسان

طبق اسلاید ۴۳، در کلاس‌ها `this` (مثلاً داخل `constructor`) به چه چیزی اشاره می‌کند؟

۱) به نمونه‌ی (instance) تازه‌ای که با `new` ساخته می‌شود
۲) به خود کلاس
۳) به شیء سراسری `window`
۴) به کلاس والد (parent)

۶۰۹ متوسط

فایل `sample.js` زیر (کد اسلاید ۴۴) با دستور `node sample.js` اجرا می‌شود؛ یک فایل معمولی CommonJS، بدون `"use strict"`. خروجی چیست؟

```JavaScript
const obj = {
  name: "Robert",
  say() {
    console.log(this.name);
  }
};
obj.say();
const f = obj.say;
f();
```

۱) `Robert` `Robert`

۲) `Robert` `undefined`

۳) `Robert` `Uncaught TypeError`

۴) `Robert` ``

۶۱۰ متوسط

کد اسلاید ۴۴ این بار با `"use strict"` در ابتدای فایل و در Node.js اجرا می‌شود. خروجی چیست؟

```JavaScript
"use strict";
const obj = {
  name: "Robert",
  say() {
    console.log(this.name);
  }
};
obj.say();
const f = obj.say;
f();
```

۱) `Robert` `undefined`

۲) `Robert` `Robert`

۳) `Uncaught TypeError`

۴) `Robert` `Uncaught TypeError`

۶۱۱ سخت

کد زیر در **مرورگر** و داخل یک تگ `<script>` معمولی (بدون strict mode) اجرا می‌شود. خروجی کنسول چیست؟

```JavaScript
const obj = {
  name: "Robert",
  say() {
    console.log(this.name);
  }
};
obj.say();
const f = obj.say;
f();
console.log("end");
```

۱) `Robert` `undefined` `end`

۲) `Robert` ` ` `end`

۳) `Robert` `Uncaught TypeError`

۴) `Robert` `Robert` `end`

۶۱۲ آسان

در **strict mode**، در کدام شکل فراخوانی، مقدار `this` داخل یک تابع معمولی `f` برابر `undefined` می‌شود؟

۱) `f()` (صدا زدن ساده، بدون شیء قبل از نقطه)

۲) `obj.f()`

۳) `f.call(obj)`

۴) `new f()`

۶۱۳ متوسط

خروجی کد زیر چیست؟

```JavaScript
const user = {
  name: "outer",
  profile: {
    name: "inner",
    show() {
      console.log(this.name);
    },
  },
};
user.profile.show();
```

۱) `outer`

۲) `undefined`

۳) `inner`

۴) `Uncaught TypeError`

۶۱۴ متوسط

خروجی کد زیر چیست؟

```JavaScript
const robert = {
  name: "Robert",
  say() {
    return "I am " + this.name;
  },
};
const anna = { name: "Anna" };
console.log(robert.say.call(anna));
anna.say = robert.say;
console.log(anna.say());
```

۱) `I am Robert` `I am Robert`
۲) `I am Anna` `I am Robert`
۳) `I am Anna` `I am Anna`
۴) `I am Robert` `I am Anna`

۶۱۵ متوسط

فایل `app.js` زیر با دستور `node app.js` (یک فایل CommonJS) اجرا می‌شود. خروجی چیست؟

```JavaScript
console.log(this);
console.log(this === module.exports);
```

۱) `undefined` `false`
۲) `{}` `true`
۳) `{}` `false`
۴) `undefined` `true`

۶۱۶ سخت

کد زیر در **مرورگر** داخل یک تگ `<script>` معمولی اجرا می‌شود. خروجی چیست؟

```JavaScript
"use strict";
function whoAmI() {
  return this;
}
console.log(whoAmI() === undefined);
console.log(this === window);
```

۱) `true` `true`
۲) `true` `false`
۳) `false` `true`
۴) `false` `false`

۶۱۷ متوسط

کد زیر در **مرورگر** داخل یک تگ `<script>` معمولی (بدون strict mode) اجرا می‌شود. خروجی چیست؟

```javascript
var city = "Tabriz";
function show() {
  console.log(this.city);
}
show();
const obj = { city: "Shiraz", show };
obj.show();
```

۱) `undefined` `Shiraz`
۲) `Shiraz` `Shiraz`
۳) `Uncaught TypeError`
۴) `Tabriz` `Shiraz`

۶۱۸ آسان

خروجی کد زیر چیست؟

```javascript
const obj = {
  name: "Robert",
  say() {
    console.log(this.name);
  }
};
const f = obj.say;
f.call(obj);
f.apply(obj);
f.bind(obj)();
```

۱) `Robert` `Robert` `Robert`
۲) `Robert` `Robert` `undefined`
۳) `undefined` `undefined` `undefined`
۴) `Robert` `Robert`

۶۱۹ آسان

کدام جمله درباره‌ی `call`، `apply` و `bind` **درست** است؟

۱) `bind` تابع را همان لحظه اجرا می‌کند، ولی `call` یک تابع جدید برمی‌گرداند.
۲) `apply` آرگومان‌ها را جداجدا با کاما می‌گیرد و `call` آن‌ها را داخل یک آرایه.
۳) `call` و `apply` تابع را همان لحظه اجرا می‌کنند، ولی `bind` آن را اجرا نمی‌کند و تابع جدیدی با `this` ثابت برمی‌گرداند.
۴) این سه متد فقط روی arrow functionها کار می‌کنند.

۶۲۰ متوسط

خروجی کد زیر چیست؟

```JavaScript
function introduce(greeting, punctuation) {
  return greeting + ", " + this.name + punctuation;
}
const robert = { name: "Robert" };
console.log(introduce.call(robert, "Hi", "!"));
console.log(introduce.apply(robert, ["Hello", "?"]));
```

۱) `Hi, undefined!` `Hello, undefined?`

۲) `Hi, Robert!` `Uncaught TypeError`

۳) `Hi, Robert!` `Hello,?, Robertundefined`

۴) `Hi, Robert!` `Hello, Robert?`

۶۲۱ متوسط

با فرض شیء زیر، کدام کد باعث می‌شود `f()` عبارت `Robert` را چاپ کند و هیچ خطایی هم رخ ندهد؟

```JavaScript
const obj = {
  name: "Robert",
  say() {
    console.log(this.name);
  }
};
```

۱) `const f = obj.say.call(obj); f();`

۲) `const f = obj.say.bind(obj); f();`

۳) `const f = obj.say; f.bind(obj); f();`

۴) `const f = obj.say; f();`

۶۲۲ سخت

خروجی کد زیر چیست؟

```JavaScript
function show() {
  console.log(this.name);
}
const a = { name: "A" };
const b = { name: "B" };
const f = show.bind(a);
const g = f.bind(b);
g();
f.call(b);
```

۱) `B` `B`

۲) `A` `B`

۳) `A` `A`

۴) `B` `A`

۶۲۳ متوسط

کدام جمله **نادرست** است؟

۱) در فراخوانی `obj.f()`، مقدار `this` داخل `f` برابر `obj` است.

۲) `f.bind(obj)` تابع `f` را بلافاصله اجرا می‌کند و نتیجه‌ی آن را برمی‌گرداند.

۳) مقدار `this` در arrow function را از محدوده‌ی بیرونی می‌گیرد.

۴) `apply` آرگومان‌های تابع را به شکل یک آرایه دریافت می‌کند.

۶۲۴ سخت

کد زیر در Node.js (فایل CommonJS، بدون strict mode) اجرا می‌شود. خروجی چیست؟

```JavaScript
function Car(model) {
  this.model = model;
}
const c = new Car("Pride");
console.log(c.model);
const d = Car("Tiba");
console.log(d);
console.log(globalThis.model);
```

۱) `Pride` `undefined` `Tiba`

۲) `Pride` `Car { model: 'Tiba' }` `undefined`

۳) `Pride` `Uncaught TypeError`

۴) `Pride` `undefined` `undefined`

۶۲۵ متوسط

کدام جمله درباره‌ی arrow functionها **درست** است؟

۱) اگر arrow function به‌عنوان متد یک شیء تعریف شود، `this` آن همان شیء است.

۲) می‌توان arrow function را با `new` صدا زد تا شیء جدیدی بسازد.

۳) `this` در arrow function هنگام صدا زدن و بر اساس شیء قبل از نقطه تعیین می‌شود.

۴) arrow function `this` مخصوص خودش را ندارد و `call`، `apply` و `bind` هم نمی‌توانند `this` آن را عوض کنند.

۶۲۶ سخت

کد زیر در Node.js (فایل CommonJS) اجرا می‌شود. خروجی چیست؟

```javascript
const counter = {
  count: 10,
  regular() {
    return this.count;
  },
  arrow: () => this.count,
};
console.log(counter.regular(), counter.arrow());
```

۱) `10 10`
۲) `10 undefined`
۳) `undefined undefined`
۴) `Uncaught TypeError`

۶۲۷ سخت

کد زیر در **مرورگر** داخل یک تگ `<script>` معمولی اجرا می‌شود. خروجی چیست؟

```javascript
var count = 99;
const counter = {
  count: 10,
  arrow: () => this.count,
};
console.log(counter.arrow());
```

۱) `10`
۲) `undefined`
۳) `99`
۴) `Uncaught TypeError`

۶۲۸ متوسط

خروجی کد زیر چیست؟

```javascript
const team = {
  name: "Blue",
  members: ["Ali", "Sara"],
  list() {
    return this.members.map((m) => m + " of " + this.name);
  },
};
console.log(team.list());
```

۱) `['Ali of Blue', 'Sara of Blue']`
۲) `['Ali of undefined', 'Sara of undefined']`
۳) `['Ali of ', 'Sara of ']`
۴) `Uncaught TypeError`

۶۲۹ متوسط

کد زیر در Node.js (فایل CommonJS، بدون strict mode) اجرا می‌شود. خروجی چیست؟

```javascript
const team = {
  name: "Blue",
  show() {
    function inner() {
      return this.name;
    }
    return inner();
  },
};
console.log(team.show());
```

۱) `Blue`
۲) `Uncaught TypeError`
۳) `Uncaught ReferenceError`
۴) `undefined`

۶۳۰ سخت

کد زیر در Node.js (فایل CommonJS) اجرا می‌شود. خروجی چیست؟

```javascript
const arrow = () => this;
const obj = { name: "obj" };
console.log(arrow.call(obj) === obj);
console.log(arrow.bind(obj)() === module.exports);
```

۱) `true` `false`
۲) `false` `true`
۳) `true` `true`
۴) `false` `false`

۶۳۱ سخت

کد زیر در Node.js اجرا می‌شود. خروجی چیست؟

```javascript
const timer = {
  label: "T1",
  report() {
    console.log("label:", this.label);
  },
};
setTimeout(timer.report, 0);
setTimeout(() => timer.report(), 0);
setTimeout(timer.report.bind(timer), 0);
```

۱) `label: T1` `label: T1` `label: T1`
۲) `label: undefined` `label: undefined` `label: T1`
۳) `label: undefined` `label: T1` `label: T1`
۴) `Uncaught TypeError`

۶۳۲ متوسط

با فرض شیء زیر، کدام دستور در Node.js با تأخیر (از طریق `setTimeout`) **فقط** عبارت `Robert` را چاپ می‌کند و هیچ خطایی هم نمی‌دهد؟

```JavaScript
const obj = {
  name: "Robert",
  say() {
    console.log(this.name);
  }
};
```

۱) `setTimeout(obj.say, 0);`

۲) `setTimeout(obj.say(), 0);`

۳) `setTimeout(() => obj.say(), 0);`

۴) `setTimeout(obj.say.call, 0);`

۶۳۳ سخت

کد زیر در Node.js اجرا می‌شود. خروجی چیست؟

```JavaScript
const user = {
  name: "Sara",
  later() {
    setTimeout(() => console.log("arrow:", this.name), 0);
    setTimeout(function () {
      console.log("function:", this.name);
    }, 0);
  },
};
user.later();
```

۱) `arrow: Sara` `function: Sara`

۲) `arrow: undefined` `function: Sara`

۳) `arrow: undefined` `function: undefined`

۴) `arrow: Sara` `function: undefined`

۶۳۴ سخت

کد زیر در Node.js (فایل CommonJS، **بدون** `"use strict"`) اجرا می‌شود. خروجی چیست؟

```javascript
class User {
  constructor(name) {
    this.name = name;
  }
  greet() {
    return "Hello, " + this.name;
  }
}
const kermit = new User("Kermit");
const greet = kermit.greet;
console.log(kermit.greet());
console.log(greet());
```

۱) `Hello, Kermit` `Hello, Kermit`

۲) `Hello, Kermit` `Hello, undefined`

۳) `Hello, Kermit` `Uncaught TypeError`

۴) `Uncaught TypeError`

۶۳۵ سخت

(پیش‌نمایش فصل دوم) کاربر یک بار روی دکمه کلیک می‌کند. خروجی کنسول چیست؟

```html
<button id="btn">Save</button>
```

```javascript
const btn = document.querySelector("#btn");
btn.addEventListener("click", function () {
  console.log("function:", this.id);
});
btn.addEventListener("click", () => {
  console.log("arrow:", this === window);
});
```

۱) `function: btn` `arrow: true`

۲) `function: undefined` `arrow: false`

۳) `function: btn` `arrow: false`

۴) `function: undefined` `arrow: true`

۱-۲۶ مدیریت خطا: try، catch، finally و throw

منبع: Introduction to JS، اسلاید ۵۰

درسنامه

خطا (Error) یعنی چه؟

تا اینجا فرض می‌کردیم کد همیشه بی‌دردسر اجرا می‌شود؛ اما در دنیای واقعی کاربر متن اشتباه وارد می‌کند، سرور جواب خراب می‌فرستد یا خودمان اسم یک متغیر را غلط تایپ می‌کنیم. طبق اسلاید ۵۰، **خطاها (errors) وقتی رخ می‌دهند که کد در زمان اجرا (runtime) خراب می‌شود**؛ یعنی کد از نظر دستور زبان درست نوشته شده، اما هنگام اجرا به کاری می‌رسد که انجامش ممکن نیست. چند نمونه:

- صدا زدن چیزی که تابع نیست؛
- خواندن یک ویژگی (property) از `null` یا `undefined`؛
- دادن یک متن خراب به `JSON.parse` (تابعی که متن JSON را به شیء تبدیل می‌کند و در درس بعد کاملش را می‌بینیم).

جمله‌ی دوم اسلاید ۵۰ مهم‌تر است: **اگر به خطا رسیدگی نکنیم (handle نکنیم)، کل برنامه متوقف می‌شود** و خط‌های بعدی اصلاً اجرا نمی‌شوند:

```JavaScript
console.log("Step 1");
JSON.parse("{ bad json }"); // throws a SyntaxError
console.log("Step 2");      // never runs
```

این کد فقط `Step 1` را چاپ می‌کند. بعد در Console مرورگر پیام قرمز `Uncaught SyntaxError: ...` ظاهر می‌شود و اجرای بقیه‌ی همان اسکریپت متوقف می‌شود (صفحه باز می‌ماند، ولی کدِ بعد از خطا اجرا نمی‌شود). Node.js هم پیام `SyntaxError: ...` را نشان می‌دهد و کل برنامه را می‌بندد. واژه‌ی **Uncaught** یعنی «گرفته‌نشده»: خطایی که هیچ کدی آن را نگرفت. در گزینه‌های تست‌های این کتاب هم چنین خطایی با برچسب «Uncaught» نشان داده می‌شود.

> **یک تشبیه ساده**
>
> بندبازی را تصور کن که روی طناب راه می‌رود. اگر زیرش تور ایمنی نباشد، یک لغزش یعنی پایان نمایش. `try...catch` همان تور ایمنی است: کار پرخطر را داخل `try` انجام می‌دهی و اگر لغزیدی، `catch` تو را می‌گیرد و نمایش ادامه پیدا می‌کند. `finally` هم مثل خاموش کردن نورافکن‌ها در پایان نمایش است: چه افتاده باشی چه نه، انجام می‌شود.

شیء خطا (Error object): `name` و `message`

وقتی خطایی رخ می‌دهد، JavaScript یک **شیء خطا** می‌سازد و آن را «پرتاب» (throw) می‌کند. مهم‌ترین ویژگی‌های این شیء:

- `name`: نوع خطا، مثل `"TypeError"` یا `"SyntaxError"`؛
- `message`: توضیح متنی خطا، مثل `"f is not a function"`؛
- `stack`: مسیر فراخوانی‌هایی که به خطا رسید (برای اشکال‌زدایی یا debugging).

خودمان هم می‌توانیم با `new Error("...")` یک شیء خطا بسازیم:

```JavaScript
const e = new Error("Disk full");
console.log(e.name);    // Error
console.log(e.message); // Disk full
console.log(String(e)); // Error: Disk full
console.log(typeof e);  // object
```

انواع خطاهای داخلی (built-in)

JavaScript چند نوع خطای آماده دارد. این جدول را خوب یاد بگیر؛ سؤال «این کد چه خطایی می‌دهد؟» بسیار پرتکرار است:

نوع خطا	چه زمانی رخ می‌دهد؟	مثال
`ReferenceError`	استفاده از اسمی که تعریف نشده، یا استفاده از متغیر `let` / `const` پیش از خط تعریفش (TDZ)	`console.log(totl)`
`TypeError`	کاری که با آن نوع داده ممکن نیست: صدا زدن چیزی که تابع نیست، خواندن ویژگی از `null` یا `undefined`، مقدار دادن دوباره به `const`	`null.length`
`SyntaxError`	دستور زبان (syntax) نامعتبر؛ چه در خود کد، چه در متنی که به `JSON.parse` داده‌ایم	`JSON.parse("{ bad json }")`
`RangeError`	عددی بیرون از بازه‌ی مجاز	`new Array(-1)`
`Error`	نوع پایه و عمومی همه‌ی خطاها؛ معمولاً خودمان با `throw new Error("...")` می‌سازیمش	`new Error("Disk full")`

پیام‌های واقعی V8 (موتور Chrome و Node.js) برای چند نمونه را ببین. هر خط را جداگانه در نظر بگیر، چون اولین خطا برنامه را متوقف می‌کند:

```JavaScript
null.length;        // TypeError: Cannot read properties of null (reading 'length')
const f = 5; f();   // TypeError: f is not a function
console.log(totl);  // ReferenceError: totl is not defined
new Array(-1);      // RangeError: Invalid array length
(1.5).toFixed(200); // RangeError: toFixed() digits argument must be between 0 and 100
```

تابعی که بی‌پایان خودش را صدا بزند هم به `RangeError: Maximum call stack size exceeded` می‌رسد.

دقت کن که **خواندن ویژگیِ ناموجود خطا نیست**؛ فقط `undefined` می‌دهد. خطا وقتی رخ می‌دهد که بخواهیم از خودِ `null` یا `undefined` ویژگی بخوانیم:

```JavaScript
const user = { name: "Sara", address: null };
console.log(user.age);          // undefined (no error)
console.log(user.address.city); // TypeError: Cannot read properties of null (reading 'city')
```

`try...catch` : اجرای امن کد پرخطر

طبق اسلاید ۵۰، از `try...catch` برای **اجرای امن کد پرخطر (risky code)** استفاده می‌کنیم. ساختارش این است:

```JavaScript
try {
  // risky code
} catch (err) {
  // runs only if something inside try threw an error
  // err is the error object that was thrown
}
```

کد اسلاید ۵۰ قدم‌به‌قدم

```errorhandling.js
try {
  JSON.parse("{ bad json }");
} catch (err) {
  console.log("Oops:", err.message);
} finally {
  console.log("Done!");
}
```

خروجی در Node.js (و Chrome، که هر دو موتور V8 دارند):

```
Oops: Expected property name or '}' in JSON at position 2 (line 1 column 3)
Done!
```

1. `JSON.parse` متن `{ bad json }` را می‌خواند. بعد از `{` باید اسم یک کلید داخل دابل‌کوتیشن بیاید، اما در **position 2** (شمارش از صفر: `{` جایگاه ۰ است و فاصله جایگاه ۱) به حرف `b` می‌رسد؛ پس یک `SyntaxError` پرتاب می‌کند.
2. بقیه‌ی بلوک `try` (اگر خطی مانده بود) رد می‌شود و اجرا به `catch` می‌پرد. `err` همان شیء `SyntaxError` است و `err.message` متن خطا را می‌دهد.
3. بلوک `finally` اجرا می‌شود و `Done!` چاپ می‌شود.
4. برنامه به‌طور عادی از خط بعد از کل این ساختار ادامه پیدا می‌کند؛ هیچ خطای Uncaughtای نداریم.

متن `message` به موتور بستگی دارد (مثلاً Firefox جمله‌ی دیگری می‌نویسد)، اما `err.name` همه‌جا `"SyntaxError"` است.

قواعد جریان اجرا

- اگر در `try` خطایی رخ ندهد، `try` تا آخر اجرا می‌شود و `catch` **کلاً رد می‌شود**.
- اگر در یک خط از `try` خطا رخ دهد، **بقیه‌ی خط‌های** `try` **اجرا نمی‌شوند** و اجرا مستقیم به `catch` می‌رود. بعد از `catch` هم دیگر به `try` برنمی‌گردد.
- `finally` (اگر باشد) در هر دو حالت، آخر از همه اجرا می‌شود.
- سپس برنامه از خط بعد از کل ساختار ادامه می‌دهد.

```JavaScript
console.log("A");
try {
  console.log("B");
  JSON.parse("oops"); // throws a SyntaxError
  console.log("C");   // skipped
} catch (err) {
  console.log("D:", err.name);
} finally {
  console.log("E");
}
console.log("F");     // the program goes on
// Output: A, B, D: SyntaxError, E, F
```

حالت	`try`	`catch`	`finally`	ادامه‌ی برنامه بعد از ساختار
خطایی رخ نمی‌دهد	کامل اجرا می‌شود	اجرا نمی‌شود	اجرا می‌شود	بله
خطا در `try` رخ می‌دهد	تا خطِ خطا اجرا می‌شود	اجرا می‌شود	اجرا می‌شود	بله
خطا رخ می‌دهد و `catch` نداریم (`try...finally`)	تا خطِ خطا اجرا می‌شود	—	اجرا می‌شود	نه؛ خطا به بیرون می‌رود

شکل‌های مجاز

- `try...catch`، `try...finally` و `try...catch...finally` مجازند. `try` تنها (بدون `catch` و `finally`) خطای `SyntaxError` دارد.
- اسم پارامتر `catch` دلخواه است (`err`، `e`، `error`).
- اگر به شیء خطا نیازی نداری، از ES2019 به بعد می‌توانی پرانتز را حذف کنی (**optional catch binding**، یعنی «پارامتر اختیاری catch»):

```JavaScript
try {
  JSON.parse("{");
} catch {
  console.log("invalid JSON"); // no error variable needed
}
```

`throw` : ساختن خطای خودمان

اسلاید ۵۰ می‌گوید `throw` **به ما اجازه می‌دهد خطای خودمان را بسازیم**. `throw` (پرتاب کردن) اجرای تابع فعلی را **همان لحظه** متوقف می‌کند (شبیه `return`، اما برای خطا) و اجرا را به نزدیک‌ترین `catch` می‌برد؛ حتی اگر آن `catch` در تابعِ صدازننده باشد.

```JavaScript
function withdraw(balance, amount) {
  if (amount > balance) {
    throw new Error("Insufficient funds");
  }
  return balance - amount;
}

try {
  console.log(withdraw(100, 30));  // 70
  console.log(withdraw(100, 500)); // throws
  console.log("never printed");
} catch (err) {
  console.log("Error:", err.message); // Error: Insufficient funds
}
```

می‌توانی نوع دقیق‌تری هم پرتاب کنی، مثل `throw new TypeError("Age must be a number")` یا `throw new RangeError("Age must be between 0 and 150")`.

در JavaScript هر مقداری را می‌شود پرتاب کرد (مثل `throw "oops"` یا `throw 404`)، اما این کار **توصیه نمی‌شود**، چون یک رشته `name`، `message` و `stack` ندارد:

```JavaScript
try {
  throw "Something bad"; // a string, not an Error object
} catch (err) {
  console.log(typeof err);  // string
  console.log(err.message); // undefined
}
```

کلاس خطای سفارشی (custom error)

با `extends` (که در درس کلاس‌ها دیدیم) می‌توانیم نوع خطای مخصوص برنامه‌ی خودمان را بسازیم:

```JavaScript
class ValidationError extends Error {
  constructor(message) {
    super(message); // the Error constructor sets this.message
    this.name = "ValidationError";
  }
}

try {
  throw new ValidationError("Email is required");
} catch (err) {
  console.log(err.name);                    // ValidationError
  console.log(err.message);                 // Email is required
  console.log(err instanceof ValidationError); // true
  console.log(err instanceof Error);           // true
}
```

اگر خط `this.name = ...` را ننویسی، `name` از `Error` به ارث می‌رسد و همان `"Error"` می‌ماند (هرچند `instanceof` درست کار می‌کند). با `instanceof` داخل `catch` می‌فهمیم دقیقاً با چه نوع خطایی روبه‌رو هستیم.

`finally` : کاری که همیشه انجام می‌شود

طبق اسلاید ۵۰، `finally` **چه خطا رخ بدهد چه ندهد** اجرا می‌شود. جای کارهای «جمع‌وجور کردن» (cleanup) است: بستن فایل یا اتصال، پنهان کردن نشانگر «در حال بارگذاری» و از این قبیل. حتی اگر `try` یا `catch` با `return` از تابع خارج شوند، `finally` قبل از خروج اجرا می‌شود:

JavaScript

```
function load() {
  try {
    console.log("loading");
    return "data";
  } finally {
    console.log("cleanup"); // runs before the function really returns
  }
}
console.log(load());
// loading
// cleanup
// data
```

⚠ **هرگز در finally از return استفاده نکن**

`return` داخل `finally` مقدار برگشتیِ `try` یا `catch` را **جایگزین** می‌کند (و حتی خطایی را که در حال پرتاب شدن است بی‌صدا از بین می‌برد):

JavaScript

```
function getValue() {
  try {
    throw new Error("fail");
  } catch (e) {
    return "from catch";
  } finally {
    return "from finally"; // overrides "from catch"
  }
}
console.log(getValue()); // from finally
```

BEST PRACTICE: فقط خطایی را بگیر که واقعاً می‌توانی رسیدگی کنی

کادر BEST PRACTICE اسلاید ۵۰ می‌گوید: **Catch only errors you can actually handle**. یعنی:

- خطایی را بگیر که **انتظارش را داری و می‌دانی با آن چه کنی**؛ مثلاً اگر متن تنظیمات کاربر JSON معتبری نیست، پیام بده و تنظیمات پیش‌فرض را بگذار.
- خطاهای دیگر (مثل باگ‌های برنامه) را **پنهان نکن**؛ آن‌ها را دوباره پرتاب کن (**rethrow**) تا لایه‌ی بالاتر به آن‌ها رسیدگی کند یا دست‌کم در Console دیده شوند.
- `catch` خالی (`catch (e) {}`) که همه‌ی خطاها را بی‌صدا می‌بلعد، باگ‌ها را پنهان می‌کند و پیدا کردنشان را خیلی سخت می‌کند.

```JavaScript
function readSettings(text) {
  try {
    return JSON.parse(text);
  } catch (err) {
    if (err instanceof SyntaxError) { // expected: bad user input
      console.log("Invalid settings, using defaults");
      return { theme: "light" };
    }
    throw err; // not ours to handle: rethrow it
  }
}
console.log(readSettings('{"theme": "dark"}').theme); // dark
console.log(readSettings("oops").theme);
// Invalid settings, using defaults
// light
```

خطا در زنجیره‌ی فراخوانی‌ها بالا می‌رود

اگر تابعی خطا را نگیرد، خطا به تابعِ صدازننده می‌رود، بعد به صدازننده‌ی او و همین‌طور بالا، تا اولین `catch` ی که پیدا شود (اگر هیچ `catch` ی نباشد، برنامه متوقف می‌شود). در `try` های تودرتو (nested) هم خطا به نزدیک‌ترین `catch` بیرونی می‌رود، اما `finally` های داخلی سر راه اجرا می‌شوند:

```JavaScript
try {
  try {
    null.length; // TypeError
  } finally {
    console.log("inner finally");
  }
} catch (err) {
  console.log("outer catch:", err.name);
}
// inner finally
// outer catch: TypeError
```

اگر خودِ `catch` هم خطا بدهد، آن خطای جدید به `catch` بیرونی‌تر می‌رود.

دو چیزی که try...catch نمی‌تواند بگیرد

۱. اشتباه دستوری در خود کد. موتور JavaScript قبل از اجرای یک اسکریپت، کل آن را می‌خواند و تجزیه (parse) می‌کند. اگر خود کد اشتباه دستوری داشته باشد، **هیچ خطی اجرا نمی‌شود**، حتی خط‌های قبل از `try` :

```JavaScript
console.log("start"); // does NOT run either!
try {
  let x = ;           // SyntaxError in the source code itself
} catch (err) {
  console.log("caught");
}
// Only: Uncaught SyntaxError: Unexpected token ';'
```

اما `SyntaxError` ِ `JSON.parse` فرق دارد: کدِ ما درست است و فقط **داده‌ای** که در زمان اجرا به `JSON.parse` داده‌ایم خراب است؛ پس یک خطای زمان اجراست و `catch` آن را می‌گیرد (همان کد اسلاید ۵۰).

۲. خطایی که بعداً در یک callback ناهمگام رخ می‌دهد. `try...catch` فقط خطاهایی را می‌گیرد که **در همان مدتی که بلوک** `try` **در حال اجراست** رخ دهند. تابعی که به `setTimeout` می‌دهیم (callback، یعنی تابعی که برای اجرای بعدی تحویل می‌دهیم) بعداً اجرا می‌شود؛ وقتی `try...catch` مدت‌ها تمام شده است (جزئیاتش را در درس «JavaScript ناهمگام (Asynchronous) و Callback» می‌بینیم):

JavaScript

```
try {
  setTimeout(() => {
    null.length; // TypeError, thrown 100 ms later
  }, 100);
} catch (err) {
  console.log("never caught here");
}
// Uncaught TypeError (the program crashes after 100 ms)
```

راه درست این است که `try...catch` را **داخل** همان callback بگذاری:

JavaScript

```
setTimeout(() => {
  try {
    null.length;
  } catch (err) {
    console.log("caught inside the callback:", err.name); // TypeError
  }
}, 100);
```

برای Promiseها از `.catch()` و برای `async/await` از `try...catch` دورِ `await` استفاده می‌کنیم که در درس‌های بعدی همین فصل می‌بینیم.

نکته

- `err.name` نوع خطا و `err.message` متن آن است؛ `String(err)` هر دو را به شکل `Name: message` می‌دهد.
- همیشه یک شیء `Error` (یا زیرکلاسی از آن) پرتاب کن، نه رشته یا عدد.
- `finally` برای کارهای پایانی (cleanup) است و همیشه اجرا می‌شود.
- در کلاس خطای سفارشی، `this.name` را خودت تنظیم کن.

⚠ دام‌های تستی

- بعد از خطِ خطا، **بقیه‌ی `try` اجرا نمی‌شود**؛ بعد از `catch` هم به `try` برنمی‌گردیم، بلکه برنامه از بعدِ کل ساختار ادامه پیدا می‌کند.
- `finally` حتی وقتی `try` یا `catch` با `return` خارج می‌شوند اجرا می‌شود (قبل از برگشتن مقدار)؛ و `return` داخل `finally` برنده است.
- `throw "text"` یک رشته پرتاب می‌کند؛ پس `err.message` برابر `undefined` است.
- `try` تنها، بدون `catch` یا `finally`، خطای `SyntaxError` دارد.
- خواندن ویژگیِ ناموجود (`user.age`) فقط `undefined` می‌دهد، اما خواندن ویژگی از `undefined` یا `null` (مثل `user.address.city` وقتی `address` برابر `null` است) `TypeError` می‌دهد.
- اشتباه دستوری در خود کد با `try...catch` گرفته نمی‌شود و هیچ خطی از آن اسکریپت اجرا نمی‌شود؛ اما خطای `JSON.parse` گرفته می‌شود.
- `try...catch` دورِ `setTimeout`، خطای داخل callback را نمی‌گیرد.

✓ جمع‌بندی

- خطای زمان اجرا (runtime error) اگر رسیدگی نشود، برنامه را متوقف می‌کند.
- `try` : کد پرخطر؛ `catch (err)` : رسیدگی به خطا؛ `finally` : همیشه اجرا می‌شود؛ `throw` : پرتاب خطای خودمان.
- نوع‌های مهم: `ReferenceError`، `TypeError`، `SyntaxError`، `RangeError` و نوع پایه‌ی `Error`.
- فقط خطایی را بگیر که می‌توانی رسیدگی کنی؛ بقیه را دوباره پرتاب کن (rethrow).

✓ تست‌های تمرینی

۲۵ تست · شماره‌ی ۶۳۶ تا ۶۶۰

۶۳۶ آسان

طبق اسلاید ۵۰، اگر در زمان اجرای برنامه خطایی رخ دهد و هیچ کدی به آن رسیدگی نکند (handle نشود)، چه اتفاقی می‌افتد؟

۱) JavaScript فقط همان خط را نادیده می‌گیرد و بقیه‌ی خط‌ها را اجرا می‌کند.

۲) اجرای برنامه متوقف می‌شود و خط‌های بعد از خطا اجرا نمی‌شوند.

۳) مقدار آن خط خودکار `undefined` می‌شود و برنامه بدون پیام خطا ادامه پیدا می‌کند.

۴) JavaScript برنامه را از اول دوباره اجرا می‌کند.

۶۳۷ آسان

طبق اسلاید ۵۰، کدهای کدام بلوک **چه خطا رخ بدهد چه ندهد** اجرا می‌شوند؟

۱) `catch` ۲) `try` ۳) `throw` ۴) `finally`

۶۳۸ آسان

در ساختار `try { ... } catch (err) { ... }`، پارامتر `err` چیست؟

۱) یک مقدار boolean که نشان می‌دهد خطا رخ داده است یا نه

۲) شماره‌ی خطی که خطا در آن رخ داده است

۳) همان چیزی که پرتاب شده است؛ برای خطاهای JavaScript یک شیء خطا با ویژگی‌هایی مثل `name` و `message`

۴) همیشه یک رشته (string) شامل متن خطا

۶۳۹ آسان

خروجی کد اسلاید ۵۰ در Node.js کدام است؟

```JavaScript
try {
  JSON.parse("{ bad json }");
} catch (err) {
  console.log("Oops:", err.message);
} finally {
  console.log("Done!");
}
```

۱) `Oops: Expected property name or '}' in JSON at position 2 (line 1 column 3)` `Done!`

۲) `Done!`

۳) `Oops: Expected property name or '}' in JSON at position 2 (line 1 column 3)`

۴) `Uncaught SyntaxError`

۶۴۰ آسان

خروجی کد زیر کدام است؟

```JavaScript
const e = new Error("Disk full");
console.log(e.name);
console.log(e.message);
```

۱) `Disk full` `Error`

۲) `Error: Disk full` `Disk full`

۳) `undefined` `Disk full`

۴) `Error` `Disk full`

۶۴۱ آسان

فرض کن به جای `total + 1` اشتباهی نوشته‌ایم `totl + 1` و اسم `totl` هیچ‌جا تعریف نشده است. هنگام اجرای این عبارت، مقدار `name` در خطای پرتاب‌شده کدام است؟

۱) `"TypeError"`

۲) `"ReferenceError"`

۳) `"SyntaxError"`

۴) `"RangeError"`

۶۴۲ آسان

در جای خالی کدام کلمه قرار بگیرد تا خروجی کد دقیقاً `Error: Age cannot be negative` شود؟

```javascript
function setAge(age) {
  if (age < 0) {
    ___ new RangeError("Age cannot be negative");
  }
  return age;
}

try {
  setAge(-5);
  console.log("Age saved");
} catch (err) {
  console.log("Error:", err.message);
}
```

۱) `return`　۲) `catch`　۳) `throw`　۴) `error`

۶۴۳ متوسط

خروجی کد زیر کدام است؟

```javascript
try {
  console.log("A");
  JSON.parse("{ bad json }");
  console.log("B");
} catch (err) {
  console.log("C:", err.name);
} finally {
  console.log("D");
}
console.log("E");
```

۱) `A` `C: SyntaxError` `D` `E`

۲) `A` `B` `C: SyntaxError` `D` `E`

۳) `A` `C: SyntaxError` `D`

۴) `A` `C: SyntaxError` `B` `D` `E`

۶۴۴ متوسط

خروجی کد زیر کدام است؟

```javascript
let step = 0;
try {
  step = 1;
  throw new Error("stop");
  step = 2;
} catch (e) {
  step = step + 10;
}
console.log(step);
```

۱) 12　۲) 11　۳) 1　۴) 2

۶۴۵ متوسط

کد زیر را در Node.js اجرا می‌کنیم. کدام خط خطا می‌دهد؟

```javascript
1  const user = { name: "Sara", address: null };
2  console.log(user.name.toUpperCase());
3  console.log(user.age);
4  console.log(user.address.city);
5  console.log("done");
```

۱) خط ۲　۲) خط ۳

۳) خط ۴　۴) هیچ خطایی رخ نمی‌دهد

۶۴۶ متوسط

کدام جمله **نادرست** است؟

۱) `JSON.parse("{name: 1}")` خطای `TypeError` می‌دهد.

۲) `null.length` خطای `TypeError` می‌دهد.

۳) `new Array(-1)` خطای `RangeError` می‌دهد.

۴) اگر اسم `price` هیچ‌جا تعریف نشده باشد، `price * 2` خطای `ReferenceError` می‌دهد.

۶۴۷ متوسط

خروجی کد زیر کدام است؟

```javascript
class ValidationError extends Error {
  constructor(message) {
    super(message);
    this.name = "ValidationError";
  }
}

try {
  throw new ValidationError("Email is required");
} catch (err) {
  console.log(err.name);
  console.log(err.message);
  console.log(err instanceof ValidationError, err instanceof Error);
}
```

۱) `Error` `Email is required` `true false`

۲) `ValidationError` `Email is required` `true true`

۳) `ValidationError` `Email is required` `true false`

۴) `ValidationError` `undefined` `true true`

۶۴۸ متوسط

خروجی کد زیر کدام است؟

```javascript
function load() {
  try {
    console.log("loading");
    return "data";
  } finally {
    console.log("cleanup");
  }
}
const result = load();
console.log(result);
```

۱) `loading` `data`

۲) `loading` `data` `cleanup`

۳) `cleanup` `loading` `data`

۴) `loading` `cleanup` `data`

۶۴۹ متوسط

خروجی کد زیر کدام است؟

```javascript
try {
  throw "Something bad";
} catch (err) {
  console.log(typeof err);
  console.log(err.message);
}
```

۱) `object` `Something bad`

۲) `string` `Something bad`

۳) `string` `undefined`

۴) `string` `Uncaught TypeError`

۶۵۰ متوسط

کادر BEST PRACTICE اسلاید ۵۰ می‌گوید: «Catch only errors you can actually handle». کدام روش با این توصیه سازگار است؟

۱) فقط خطایی را بگیریم که برایش راه‌حل مشخصی داریم (مثلاً متن JSON نامعتبرِ کاربر) و خطاهای دیگر را با `throw err` دوباره پرتاب کنیم.

۲) کل برنامه را داخل یک `try` بزرگ بگذاریم و با یک `catch` خالی همه‌ی خطاها را بی‌صدا نادیده بگیریم.

۳) از `try...catch` اصلاً استفاده نکنیم تا هر خطایی برنامه را متوقف کند.

۴) هر خطایی را بگیریم و به جایش `undefined` برگردانیم تا برنامه هیچ‌وقت متوقف نشود.

۶۵۱ متوسط

کدام کد **بدون خطا** اجرا می‌شود؟

۱) `try { JSON.parse("{"); }`

۲) `try { JSON.parse("{"); } finally (err) { console.log("invalid JSON"); }`

۳) `catch (err) { console.log("invalid JSON"); }`

۴) `try { JSON.parse("{"); } catch { console.log("invalid JSON"); }`

۶۵۲ متوسط

خروجی کد زیر کدام است؟

```javascript
function divide(a, b) {
  if (b === 0) throw new Error("Division by zero");
  return a / b;
}

try {
  console.log(divide(10, 2));
  console.log(divide(1, 0));
  console.log(divide(9, 3));
} catch (err) {
  console.log("Error:", err.message);
}
console.log("program continues");
```

۱) `5` `Error: Division by zero` `3` `program continues`

۲) `5` `Infinity` `3` `program continues`

۳) `5` `Error: Division by zero` `program continues`

۴) `5` `Error: Division by zero`

۶۵۳ سخت

خروجی کد زیر کدام است؟

```javascript
function getValue() {
  try {
    throw new Error("fail");
  } catch (e) {
    return "from catch";
  } finally {
    return "from finally";
  }
}
console.log(getValue());
```

۱) `from catch`

۲) `from finally`

۳) `from catch` `from finally`

۴) `Uncaught SyntaxError`

۶۵۴ سخت

خروجی کد زیر کدام است؟

JavaScript

```
function parseAge(text) {
  try {
    const age = JSON.parse(text);
    if (age < 0) throw new RangeError("negative age");
    return age;
  } catch (err) {
    if (err instanceof SyntaxError) return 0;
    throw err; // rethrow: we cannot handle it here
  }
}

try {
  console.log(parseAge("25"));
  console.log(parseAge("abc"));
  console.log(parseAge("-3"));
  console.log("end");
} catch (err) {
  console.log("outer:", err.name, err.message);
}
```

۱) `25` `0` `outer: RangeError negative age`

۲) `25` `0` `-3` `end`

۳) `25` `0` `outer: RangeError negative age` `end`

۴) `25` `0` `0` `end`

۶۵۵ سخت

خروجی کد زیر کدام است؟

```javascript
try {
  try {
    null.length;
  } finally {
    console.log("inner finally");
  }
  console.log("after inner");
} catch (err) {
  console.log("outer catch:", err.name);
} finally {
  console.log("outer finally");
}
```

۱) `outer catch: TypeError` `inner finally` `outer finally`

۲) `inner finally` `after inner` `outer finally`

۳) `inner finally` `Uncaught TypeError`

۴) `inner finally` `outer catch: TypeError` `outer finally`

۶۵۶ سخت

خروجی کد زیر کدام است؟

```javascript
console.log("start");
try {
  let x = ;
} catch (err) {
  console.log("caught:", err.name);
}
```

۱) `start` `caught: SyntaxError`

۲) `Uncaught SyntaxError`

۳) `caught: SyntaxError`

۴) `start` `Uncaught SyntaxError`

۶۵۷ سخت

خروجی کد زیر در Node.js کدام است؟

```javascript
try {
  setTimeout(() => {
    const user = null;
    console.log(user.name);
  }, 0);
  console.log("timer scheduled");
} catch (err) {
  console.log("caught:", err.name);
}
console.log("after try");
```

۱) `timer scheduled` `after try` `Uncaught TypeError`

۲) `timer scheduled` `caught: TypeError` `after try`

۳) `timer scheduled` `after try` `caught: TypeError`

۴) `caught: TypeError` `after try`

۶۵۸ سخت

کدام جمله درباره‌ی `SyntaxError` **درست** است؟

۱) هر `SyntaxError` ی را می‌توان با `try...catch` گرفت، چون همه از یک نوع‌اند.

۲) `JSON.parse` برای متن نامعتبر خطا نمی‌دهد و فقط `undefined` برمی‌گرداند.

۳) اشتباه دستوری در یک خط از کد فقط همان خط را از کار می‌اندازد و خط‌های دیگر اجرا می‌شوند.

۴) خطای `JSON.parse("{ bad json }")` قابل گرفتن است، چون در زمان اجرا و به‌خاطر داده‌ی خراب رخ می‌دهد؛ اما اشتباه دستوری در خود اسکریپت پیش از اجرای هر خطی گزارش می‌شود و `try...catch` همان اسکریپت نمی‌تواند آن را بگیرد.

۶۵۹ سخت

خروجی کد زیر کدام است؟

```javascript
class AppError extends Error {}

try {
  throw new AppError("Server down");
} catch (err) {
  console.log(err.name, err instanceof AppError, err.message);
}
```

۱) `AppError true Server down`

۲) `Error false Server down`

۳) `Error true Server down`

۴) `AppError false Server down`

۶۶۰ سخت

خروجی کد زیر کدام است؟

JavaScript

```
function c() {
  const settings = null;
  return settings.theme;
}
function b() {
  const theme = c();
  console.log("b got", theme);
}
function a() {
  try {
    b();
    console.log("a done");
  } catch (err) {
    console.log("a caught", err.name);
  }
}
a();
console.log("end");
```

۱) `b got undefined` `a done` `end`

۲) `Uncaught TypeError`

۳) `a caught TypeError` `a done` `end`

۴) `a caught TypeError` `end`

۱-۲۷ Promise

منبع: Introduction to JS، اسلاید ۵۵

درسنامه

Promise چیست؟

در درس قبل دیدیم که JavaScript برای کارهای زمان‌بر (مثل گرفتن داده از سرور یا صبر کردن برای یک تایمر) منتظر نمی‌ماند و به جای آن یک **callback** (تابعی که به تابع دیگر می‌دهیم تا بعداً صدا زده شود) ثبت می‌کند. مشکل این بود که وقتی چند کار ناهمگام پشت سر هم داریم، callbackها داخل هم می‌روند و کد شکل یک هرم تودرتو پیدا می‌کند (**callback hell**). اسلاید ۵۵ روش دوم را معرفی می‌کند: **Promise** (به معنای «قول» یا «وعده»).

یک تشبیه ساده

به یک فست‌فود می‌روی و ساندویچ سفارش می‌دهی. صندوق‌دار تو را جلوی پیشخوان نگه نمی‌دارد تا ساندویچ آماده شود؛ یک **رسید** با شماره‌ی سفارش به تو می‌دهد. این رسید خودِ ساندویچ نیست، **قولِ** ساندویچ است. تو می‌روی می‌نشینی و با دوستت حرف می‌زنی (برنامه به بقیه‌ی کارهایش می‌رسد). بعداً یکی از دو اتفاق می‌افتد: شماره‌ات را صدا می‌زنند و ساندویچ را تحویل می‌گیری (**fulfilled**)، یا می‌گویند «ببخشید، نان تمام شد» (**rejected**). تا وقتی هیچ‌کدام رخ نداده، سفارش «در انتظار» است (**pending**). و اگر آشپزخانه سفارشت را گم کند، شاید **هیچ‌وقت** صدایت نزنند!

اسلاید ۵۵ می‌گوید: «A `Promise` represents a value that may arrive now, later, or never»؛ یعنی یک Promise **نماینده‌ی مقداری است که ممکن است همین حالا، بعداً یا هرگز به دست برسد**. از نظر فنی، Promise یک **شیء** (object) است که نتیجه‌ی نهاییِ یک کار ناهمگام را در خودش نگه می‌دارد. به جای اینکه callback را به تابعِ زمان‌بر بدهیم، آن تابع یک Promise به ما **برمی‌گرداند** و ما روی همین Promise مشخص می‌کنیم که وقتی نتیجه آمد چه کنیم.

سه وضعیت (state) یک Promise

وضعیت	معنی	در مثال فست‌فود
`pending`	در انتظار؛ هنوز نتیجه‌ای در کار نیست	ساندویچ در حال آماده شدن است
`fulfilled`	کار با موفقیت تمام شد و Promise یک **مقدار** (value) دارد	ساندویچ تحویل شد
`rejected`	کار شکست خورد و Promise یک **دلیل** (reason، معمولاً یک خطا) دارد	«نان تمام شد!»

به Promiseی که دیگر pending نیست (یعنی fulfilled یا rejected شده) **settled** (تعیین‌تکلیف‌شده) می‌گویند. تغییر وضعیت فقط از یکی از این دو مسیر ممکن است:

```
pending --resolve(value)--> fulfilled   (final: never changes again)
pending --reject(reason)--> rejected    (final: never changes again)
```

اسلاید ۵۵ را درست بخوان

اسلاید ۵۵ وضعیت‌ها را به شکل «pending → fulfilled → rejected» نوشته است. این **یک دنباله نیست**! Promise از `pending` شروع می‌شود و فقط **یک بار** به `fulfilled` **یا** به `rejected` می‌رود و از آن به بعد برای همیشه همان‌جا می‌ماند. هیچ Promiseی از fulfilled به rejected (یا برعکس) نمی‌رود و به pending هم برنمی‌گردد.

ساختن Promise: `new Promise((resolve, reject) => …)`

کد اسلاید ۵۵ را خط به خط بخوانیم:

promises.js

```
const fetchData = new Promise((resolve, reject) => {
  setTimeout(() => {
    resolve("Data received!");
    // reject("Error occurred!");
  }, 1000);
});

fetchData
  .then(result => console.log(result))   // "Data received!" (after 1 sec)
  .catch(error => console.log(error));
```

- `new Promise(...)` یک Promise تازه می‌سازد. تابعی که به آن می‌دهیم **executor** (تابع اجراکننده) نام دارد.
- JavaScript دو تابع آماده به executor می‌دهد: `resolve` برای اعلام **موفقیت** (همراه با مقدار) و `reject` برای اعلام **شکست** (همراه با دلیل). این دو اسم قراردادی‌اند، ولی همه همین‌ها را می‌نویسند.
- داخل executor یک تایمر ۱۰۰۰ میلی‌ثانیه‌ای (۱ ثانیه) گذاشته‌ایم. بعد از ۱ ثانیه `resolve("Data received!")` اجرا می‌شود و Promise از `pending` به `fulfilled` با مقدار `"Data received!"` می‌رود.
- `.then(result => console.log(result))` یعنی «هر وقت fulfilled شد، این تابع را با مقدارش صدا بزن». خروجی بعد از حدود ۱ ثانیه: `Data received!`
- `.catch(error => console.log(error))` یعنی «اگر rejected شد، این تابع را با دلیلش صدا بزن». در این اجرا اصلاً صدا زده نمی‌شود.

حالا اگر خط `resolve` را کامنت کنیم و `reject` را از کامنت دربیاوریم، Promise بعد از ۱ ثانیه **rejected** می‌شود، تابعِ داخل `.then` کنار گذاشته می‌شود و `.catch` پیام را چاپ می‌کند:

JavaScript

```
const fetchData = new Promise((resolve, reject) => {
  setTimeout(() => {
    // resolve("Data received!");
    reject("Error occurred!");
  }, 1000);
});

fetchData
  .then(result => console.log(result))   // skipped
  .catch(error => console.log(error));   // "Error occurred!"
```

`fetchData` در اسلاید ۵۵ تابع نیست!

در کد اسلاید ۵۵، `fetchData` خودِ **شیء Promise** است، نه یک تابع؛ کارِ داخل آن (تایمر) همان لحظه‌ی ساخته شدن شروع شده است. برای همین می‌نویسیم `fetchData.then(...)`؛ نوشتن `fetchData()` خطای `TypeError: fetchData is not a function` می‌دهد. روش رایج‌تر این است که **تابعی بنویسیم که Promise برگرداند**؛ این‌طوری هر بار صدا زدنش یک کار تازه را شروع می‌کند:

JavaScript

```
function fetchData() {
  return new Promise((resolve) => {
    setTimeout(() => resolve("Data received!"), 1000);
  });
}

fetchData().then((result) => console.log(result)); // "Data received!"
```

گرفتن نتیجه: `.then()`، `.catch()` و `.finally()`

روی هر Promise سه متد اصلی داریم:

متد	کِی اجرا می‌شود؟	چه چیزی دریافت می‌کند؟
`.then(fn)`	وقتی Promise **fulfilled** شود	مقدار (value)
`.catch(fn)`	وقتی Promise **rejected** شود (یا در مرحله‌های قبلیِ زنجیره خطایی رخ دهد)	دلیل (reason، معمولاً شیء خطا)
`.finally(fn)`	در **هر دو** حالت، بعد از settled شدن	هیچ آرگومانی

`finally` برای کارهای «جمع‌وجور کردن» است؛ مثلاً مخفی کردن آیکون «در حال بارگذاری» که چه موفق شویم چه شکست بخوریم باید انجام شود:

JavaScript

```
function loadUser(ok) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (ok) resolve({ name: "Sara" });
      else reject(new Error("User not found"));
    }, 500);
  });
}

console.log("Loading...");
loadUser(true)
  .then((user) => console.log("Hello,", user.name))
  .catch((err) => console.log("Failed:", err.message))
  .finally(() => console.log("Hide spinner"));
// Loading...
// Hello, Sara
// Hide spinner
```

با `loadUser(false)` خروجی این می‌شود: `Loading...` ، `Failed: User not found` و `Hide spinner` . دقت کن که `finally` مقدار یا خطا را **تغییر نمی‌دهد** و فقط آن را دست‌نخورده به مرحله‌ی بعد می‌دهد؛ حتی اگر داخلش چیزی `return` کنی، نادیده گرفته می‌شود.

چهار قانون مهم که تست‌ها از آن‌ها طرح می‌شوند

قانون ۱: executor همان لحظه و همگام اجرا می‌شود

JavaScript

```
console.log("1: before");
const p = new Promise((resolve) => {
  console.log("2: inside executor");
  resolve("done");
});
p.then((value) => console.log("4: then got", value));
console.log("3: after");
// 1: before
// 2: inside executor
// 3: after
// 4: then got done
```

executor **بلافاصله** و به صورت همگام (synchronous) اجرا می‌شود. «ناهمگام» فقط کارهایی است که داخلش منتظر چیزی می‌مانند (مثل `setTimeout`) و callbackهای `then` ، `catch` و `finally` .

قانون ۲: فقط اولین `resolve` یا `reject` حساب می‌شود

JavaScript

```
const p = new Promise((resolve, reject) => {
  resolve("first");
  reject("second");   // ignored: the promise is already settled
  resolve("third");   // ignored
  console.log("executor keeps running");
});
p.then((v) => console.log("then:", v)).catch((e) => console.log("catch:", e));
// executor keeps running
// then: first
```

Promise فقط یک بار settled می‌شود. ضمناً `resolve` مثل `return` نیست: کدِ بعد از آن هم اجرا می‌شود؛ فقط دیگر وضعیت Promise عوض نمی‌شود.

قانون ۳: callbackهای `then` همیشه ناهمگام اجرا می‌شوند

حتی اگر Promise **از قبل** fulfilled شده باشد، تابع داخل `.then` فوراً اجرا نمی‌شود؛ اول کدهای همگامِ فعلی تمام می‌شوند:

JavaScript

```
const ready = Promise.resolve("X"); // already fulfilled
ready.then((v) => console.log(v));
console.log("Y");
// Y
// X
```

`Promise.resolve(v)` میان‌بُری برای ساختن Promiseی است که از همان اول با مقدار `v` fulfilled شده است و `Promise.reject(e)` Promiseی می‌سازد که از همان اول با دلیل `e` rejected شده است. دلیل دقیق قانون ۳ (صف microtask) را در درس «Event loop و ترتیب اجرای کدهای همگام و ناهمگام» می‌بینیم.

قانون ۴: `throw` داخل executor

اگر executor به صورت **همگام** خطا `throw` کند، Promise خودکار rejected می‌شود. ولی خطایی که **بعداً** داخلِ callback `setTimeout` پرتاب شود، دیگر به Promise ربطی ندارد و یک خطای مدیریت‌نشده است؛ آنجا باید `reject` را صدا بزنی:

JavaScript

```
const p1 = new Promise(() => {
  throw new Error("sync error"); // becomes a rejection
});
p1.catch((e) => console.log("caught:", e.message)); // caught: sync error

const p2 = new Promise((resolve, reject) => {
  setTimeout(() => {
    // throw new Error("late error"); // NOT caught by p2.catch: it crashes the program!
    reject(new Error("late error"));  // the correct way
  }, 100);
});
p2.catch((e) => console.log("caught:", e.message)); // caught: late error
```

زنجیره‌ی Promise (Promise chaining)

راز قدرت Promise این است: **هر** `.then()` **(و هر** `.catch()` **و** `.finally()` **) یک Promise جدید برمی‌گرداند**. پس می‌توان پشت سرش دوباره `.then` نوشت و یک **زنجیره** (chain) ساخت. قاعده‌های زنجیره:

1. هر چیزی که callback یک `then` با `return` برگرداند، ورودیِ `then` بعدی می‌شود.
2. اگر چیزی برنگرداند، `then` بعدی `undefined` می‌گیرد.
3. اگر یک **Promise** برگرداند، زنجیره **منتظر آن می‌ماند** و `then` بعدی مقدارِ نهایی آن را می‌گیرد (نه خودِ Promise را).
4. اگر خطایی `throw` شود (یا یک Promise rejected برگردانده شود)، `then`های بعدی **کنار گذاشته می‌شوند** و اجرا به نزدیک‌ترین `catch` می‌پرد.
5. `catch` خطا را مدیریت می‌کند و هر چیزی که برگرداند، زنجیره با همان مقدار **ادامه می‌یابد** (دوباره در مسیر موفقیت هستیم).

JavaScript

```
Promise.resolve(2)
  .then((x) => x * 10)                  // returns 20
  .then((x) => {
    console.log("got", x);              // got 20
    // no return here: the next then receives undefined
  })
  .then((x) => console.log("then got", x)) // then got undefined
  .then(() => {
    throw new Error("boom");            // jump to the nearest catch
  })
  .then(() => console.log("skipped"))    // never runs
  .catch((err) => {
    console.log("caught:", err.message); // caught: boom
    return "recovered";
  })
  .then((x) => console.log("after catch:", x)); // after catch: recovered
```

و قاعده‌ی ۳ (برگرداندن Promise از داخل `then`):

JavaScript

```
function wait(ms, value) {
  return new Promise((resolve) => setTimeout(() => resolve(value), ms));
}

wait(500, 1)
  .then((x) => wait(500, x + 1)) // returns a Promise: the chain waits for it
  .then((x) => console.log(x));   // 2 (after about 1 second), not a Promise
```

از callback hell به زنجیره‌ی Promise

فرض کن باید سه کار ناهمگام را پشت سر هم انجام دهیم: کاربر را بگیریم، بعد سفارش‌هایش را و بعد قیمت کل را حساب کنیم. با callback:

JavaScript

```
function getUser(id, callback) {
  setTimeout(() => callback({ id: id, name: "Sara" }), 300);
}
function getOrders(user, callback) {
  setTimeout(() => callback(["book", "pen"]), 300);
}
function getPrice(orders, callback) {
  setTimeout(() => callback(orders.length * 50), 300);
}

getUser(1, (user) => {
  getOrders(user, (orders) => {
    getPrice(orders, (price) => {
      console.log("Total:", price); // Total: 100
    });
  });
});
```

هر مرحله یک سطح تودرتوتر است. حالا همان کار با Promise؛ هر تابع به جای گرفتن callback، یک Promise **برمی‌گرداند**:

JavaScript

```
function getUser(id) {
  return new Promise((resolve) => {
    setTimeout(() => resolve({ id: id, name: "Sara" }), 300);
  });
}
function getOrders(user) {
  return new Promise((resolve) => {
    setTimeout(() => resolve(["book", "pen"]), 300);
  });
}
function getPrice(orders) {
  return new Promise((resolve) => {
    setTimeout(() => resolve(orders.length * 50), 300);
  });
}

getUser(1)
  .then((user) => getOrders(user))
  .then((orders) => getPrice(orders))
  .then((price) => console.log("Total:", price)) // Total: 100
  .catch((err) => console.log("Something failed:", err));
```

کد **صاف** شد (به جای هرم، یک ستون)، هر مرحله در یک خط است و فقط **یک** `.catch()` در انتها خطای هر سه مرحله را می‌گیرد. این همان جمله‌ی اسلاید ۵۵ است: «Avoids deep nesting from callbacks».

`Promise.all` و `Promise.race`

گاهی چند کار مستقل داریم که می‌توانند **هم‌زمان** پیش بروند. این دو متد (که در اسلایدهای ۵۸ تا ۶۰ هم به کار رفته‌اند) آرایه‌ای از Promiseها می‌گیرند و یک Promise برمی‌گردانند:

JavaScript

```
function task(name, ms) {
  return new Promise((resolve) => setTimeout(() => resolve(name), ms));
}

Promise.all([task("A", 300), task("B", 100), task("C", 200)])
  .then((results) => console.log(results)); // [ 'A', 'B', 'C' ] (after about 300 ms)
```

JavaScript

```
Promise.race([task("A", 300), task("B", 100), task("C", 200)])
  .then((first) => console.log(first)); // B (after about 100 ms)
```

	`Promise.all([...])`	`Promise.race([...])`
کِی settled می‌شود؟	وقتی **همه** fulfilled شوند، یا به محض اینکه **یکی** rejected شود	به محض اینکه **اولین** Promise settled شود (موفق یا ناموفق)
نتیجه	آرایه‌ی مقدارها **به ترتیب آرایه‌ی ورودی** (نه به ترتیب تمام شدن)	مقدار یا خطای همان اولین
زمان کل (وقتی کارها هم‌زمان‌اند)	به اندازه‌ی **کندترین** کار، نه مجموع زمان‌ها	به اندازه‌ی **سریع‌ترین** کار

اگر یکی از عضوها rejected شود، `Promise.all` **بلافاصله** با همان دلیل rejected می‌شود و منتظر بقیه نمی‌ماند (البته بقیه‌ی کارها متوقف نمی‌شوند؛ فقط نتیجه‌شان نادیده گرفته می‌شود). عضوهایی هم که Promise نیستند (مثل `"B"` یا `42`) همان‌طور در نتیجه قرار می‌گیرند:

```JavaScript
Promise.all([task("A", 300), Promise.reject(new Error("B failed")), task("C", 100)])
  .then((results) => console.log("results:", results))
  .catch((err) => console.log("error:", err.message)); // error: B failed

Promise.all([task("A", 100), "B", 42])
  .then((results) => console.log(results)); // [ 'A', 'B', 42 ]
```

> **بیشتر بدانیم**
>
> دو متد مشابه دیگر هم هست که در اسلایدها نیامده‌اند: `Promise.allSettled` (منتظر همه می‌ماند و نتیجه‌ی تک‌تک را، موفق یا ناموفق، گزارش می‌کند) و `Promise.any` (اولینِ Promise **موفق** را برمی‌گرداند).

BEST PRACTICE: همیشه `.catch()` بگذار

اسلاید ۵۵ تأکید می‌کند: «Always add `.catch()` to handle errors; unhandled rejections can crash apps». یعنی اگر Promiseی rejected شود و هیچ `.catch()` ی برایش نباشد (به این وضعیت **unhandled rejection** یعنی «ردشدنِ مدیریت‌نشده» می‌گویند)، برنامه ممکن است از کار بیفتد:

```JavaScript
const p = new Promise((resolve, reject) => {
  setTimeout(() => reject("Error occurred!"), 1000);
});

p.then((result) => console.log(result)); // no .catch() here!
setTimeout(() => console.log("still alive"), 2000);
```

- در **Node.js**: بعد از ۱ ثانیه برنامه با خطای `UnhandledPromiseRejection` متوقف می‌شود (crash) و `still alive` هرگز چاپ نمی‌شود.
- در **مرورگر**: صفحه بسته نمی‌شود، ولی خطای قرمز `Uncaught (in promise) Error occurred!` در کنسول می‌آید و خطا بی‌جواب می‌ماند.

دقت کن که `.then(...)` به‌تنهایی خطا را مدیریت **نمی‌کند**؛ چون `.then` یک Promise جدید برمی‌گرداند که آن هم rejected می‌شود و کسی آن را catch نکرده است. راه درست: یک `.catch()` در انتهای زنجیره.

> **نکته**
>
> - `.then`، `.catch` و `.finally` **همیشه** یک Promise **جدید** برمی‌گردانند؛ برای همین زنجیره ساخته می‌شود.
> - داخل `then` اگر آکولاد `{ }` گذاشتی، `return` را فراموش نکن؛ وگرنه مرحله‌ی بعدیِ زنجیره `undefined` می‌گیرد. (`x => x * 2` خودش مقدار را برمی‌گرداند، ولی `x => { x * 2; }` برنمی‌گرداند.)
> - یک `.catch()` در انتهای زنجیره خطای **هر** مرحله‌ی قبل از خودش را می‌گیرد؛ لازم نیست برای هر مرحله جدا catch بنویسی.
> - در `Promise.all` ترتیب نتیجه‌ها **همیشه** ترتیب آرایه‌ی ورودی است.

⚠ دام تستی

- `.then(console.log("done"))` اشتباه است: `console.log("done")` **همان لحظه** اجرا می‌شود و حاصلش (`undefined`) به `then` داده می‌شود که آن را نادیده می‌گیرد. باید یک **تابع** بدهی: `.then(() => console.log("done"))` .
- `resolve` اجرای executor را متوقف نمی‌کند؛ مثل `return` نیست.
- executor همگام اجرا می‌شود؛ فقط callbackهای `then` ، `catch` و `finally` ناهمگام‌اند (حتی اگر Promise از قبل settled باشد).
- Promise فقط یک بار settled می‌شود؛ `resolve` یا `reject` های بعدی بی‌اثرند.
- `throw` داخلِ callback `setTimeout` (درون executor) به `catch` نمی‌رسد؛ آنجا `reject` را صدا بزن.

✓ جمع‌بندی

- Promise یعنی شیئی برای مقداری که «الان، بعداً یا هرگز» می‌رسد. وضعیت‌ها: اول `pending` و بعد **فقط یک بار** `fulfilled` یا `rejected` .
- ساختن: `new Promise((resolve, reject) => { ... })` ؛ گرفتن نتیجه: `.then()` برای موفقیت، `.catch()` برای خطا و `.finally()` برای هر دو.
- هر `.then` یک Promise جدید می‌سازد: مقدارِ برگردانده‌شده به مرحله‌ی بعد می‌رود، Promise برگردانده‌شده باعث صبر می‌شود و `throw` اجرا را به `catch` می‌برد.
- `Promise.all` : منتظر همه (نتیجه به ترتیب ورودی؛ با اولین شکست rejected می‌شود). `Promise.race` : اولینِ settled شده.
- همیشه `.catch()` بگذار؛ unhandled rejection در Node.js برنامه را crash می‌کند.

✓ تست‌های تمرینی

۳۰ تست · شماره‌ی ۶۶۱ تا ۶۹۰

۶۶۱ آسان

طبق اسلاید ۵۵، یک `Promise` در JavaScript نماینده‌ی چیست؟

۱) تابعی که همیشه دقیقاً بعد از یک ثانیه اجرا می‌شود

۲) مقداری که ممکن است همین حالا، بعداً یا هرگز به دست برسد

۳) یک رشته‌ی اجرایی (thread) جداگانه که کد را موازی با برنامه‌ی اصلی اجرا می‌کند

۴) متغیری که مقدارش بعد از تعریف هرگز تغییر نمی‌کند

۶۶۲ آسان

کدام گزینه درباره‌ی وضعیت‌های (state) یک Promise **درست** است؟

۱) Promise همیشه به ترتیب از pending به fulfilled و سپس به rejected می‌رود

۲) یک Promise fulfilled اگر بعداً خطایی رخ دهد، به rejected تغییر می‌کند

۳) Promise در وضعیت pending شروع می‌شود و فقط یک بار به fulfilled یا rejected می‌رود و دیگر تغییر نمی‌کند

۴) Promise بعد از rejected شدن دوباره به pending برمی‌گردد تا کار را تکرار کند

۶۶۳ آسان

در جای خالی کدام گزینه قرار می‌گیرد؟

«طبق اسلاید ۵۵، برای گرفتن نتیجه‌ی موفقِ یک Promise از ..... و برای مدیریت خطای آن از ..... استفاده می‌کنیم.»

۱) `.catch()` و `.then()`
۲) `.resolve()` و `.reject()`
۳) `.finally()` و `.catch()`
۴) `.then()` و `.catch()`

۶۶۴ آسان

خروجی کد اسلاید ۵۵ (کد زیر) کدام است؟

```JavaScript
const fetchData = new Promise((resolve, reject) => {
  setTimeout(() => {
    resolve("Data received!");
    // reject("Error occurred!");
  }, 1000);
});

fetchData
  .then(result => console.log(result))
  .catch(error => console.log(error));
```

۱) `Data received!`
۲) `Error occurred!`
۳) `Data received!` `Error occurred!`
۴) `Promise { <pending> }`

۶۶۵ آسان

در کد اسلاید ۵۵، خط `resolve` را کامنت و خط `reject` را از حالت کامنت خارج کرده‌ایم. خروجی کدام است؟

```JavaScript
const fetchData = new Promise((resolve, reject) => {
  setTimeout(() => {
    // resolve("Data received!");
    reject("Error occurred!");
  }, 1000);
});

fetchData
  .then(result => console.log(result))
  .catch(error => console.log(error));
```

۱) `Data received!`
۲) `Data received!` `Error occurred!`
۳) `Error occurred!`
۴) `Uncaught UnhandledPromiseRejection`

۶۶۶ آسان

توصیه‌ی BEST PRACTICE اسلاید ۵۵ درباره‌ی Promiseها کدام است؟

۱) همیشه `.catch()` اضافه کن، چون rejectionهای مدیریت‌نشده می‌توانند برنامه را از کار بیندازند (crash)

۲) به جای Promise از callbackهای تودرتو استفاده کن تا کد خواناتر شود

۳) هیچ‌وقت داخل executor از `setTimeout` استفاده نکن

۴) همیشه بعد از `resolve` یک `reject` هم صدا بزن تا هر دو حالت پوشش داده شود

۶۶۷ آسان

طبق اسلاید ۵۵، مزیت اصلی Promise نسبت به callbackها چیست؟

۱) کد را چندرشته‌ای (multi-threaded) می‌کند تا سریع‌تر اجرا شود

۲) از تودرتو شدنِ عمیقِ callbackها جلوگیری می‌کند و مرحله‌ها با `.then()` پشت سر هم زنجیر می‌شوند

۳) کد ناهمگام را همگام می‌کند و برنامه تا آماده شدن نتیجه متوقف می‌ماند

۴) دیگر به مدیریت خطا نیازی نیست، چون Promise هیچ‌وقت شکست نمی‌خورد

۶۶۸ آسان

تابعی که به `new Promise(...)` می‌دهیم (executor) چه زمانی اجرا می‌شود؟

۱) وقتی برای اولین بار `.then()` روی Promise صدا زده شود

۲) بعد از اینکه همه‌ی کدهای همگامِ برنامه تمام شد

۳) فقط وقتی `resolve` صدا زده شود

۴) بلافاصله و به صورت همگام، همان لحظه‌ای که `new Promise` اجرا می‌شود

۶۶۹ متوسط

خروجی کد زیر کدام است؟

```JavaScript
console.log("A");
const p = new Promise((resolve) => {
  console.log("B");
  resolve("C");
});
p.then((value) => console.log(value));
console.log("D");
```

۱) A B C D

۲) A D B C

۳) A B D C

۴) A D C B

۶۷۰ متوسط

خروجی کد زیر کدام است؟

```JavaScript
const p = Promise.resolve("X");
p.then((v) => console.log(v));
console.log("Y");
```

۱) Y X
۲) X Y
۳) Y
۴) Promise { 'X' } Y

۶۷۱ متوسط

خروجی کد زیر کدام است؟

```JavaScript
Promise.resolve(2)
  .then((x) => x * 3)
  .then((x) => { x + 1; })
  .then((x) => console.log(x));
```

۱) 7
۲) 6
۳) NaN
۴) undefined

۶۷۲ متوسط

خروجی کد زیر کدام است؟

```JavaScript
Promise.resolve(1)
  .then((x) => {
    throw new Error("boom");
  })
  .then((x) => console.log("A", x))
  .catch((e) => {
    console.log("B", e.message);
    return 10;
  })
  .then((x) => console.log("C", x));
```

۱) A 1 C 10
۲) B boom
۳) B boom C 10
۴) A undefined B boom C 10

۶۷۳ متوسط

خروجی کد زیر کدام است؟

```javascript
Promise.reject(new Error("fail"))
  .finally(() => console.log("cleanup"))
  .catch((e) => console.log("caught:", e.message));
```

۱) `cleanup` `caught: fail`
۲) `caught: fail`
۳) `caught: fail` `cleanup`
۴) `cleanup`

۶۷۴ متوسط

خروجی کد زیر کدام است؟

```javascript
function wait(ms, value) {
  return new Promise((resolve) => setTimeout(() => resolve(value), ms));
}

wait(100, 1)
  .then((x) => wait(100, x + 1))
  .then((x) => console.log("got", x));
```

۱) `got Promise { <pending> }`
۲) `got 1`
۳) `got undefined`
۴) `got 2`

۶۷۵ متوسط

خروجی کد زیر کدام است؟

```javascript
function task(name, ms) {
  return new Promise((resolve) => setTimeout(() => resolve(name), ms));
}

Promise.all([task("A", 300), task("B", 100), task("C", 200)])
  .then((results) => console.log(results));
```

۱) `[ 'B', 'C', 'A' ]`
۲) `[ 'A', 'B', 'C' ]`
۳) `B`
۴) `[ 'B' ]`

۶۷۶ متوسط

خروجی کد زیر کدام است؟

```JavaScript
function task(name, ms) {
  return new Promise((resolve) => setTimeout(() => resolve(name), ms));
}

Promise.race([task("A", 300), task("B", 100), task("C", 200)])
  .then((winner) => console.log(winner));
```

۱) `A`
۲) `[ 'A', 'B', 'C' ]`
۳) `B`
۴) `[ 'B', 'C', 'A' ]`

۶۷۷ متوسط

کدام خط کد زیر خطا می‌دهد؟

```JavaScript
1  const fetchData = new Promise((resolve) => {
2    setTimeout(() => resolve("Data received!"), 100);
3  });
4
5  fetchData().then((result) => console.log(result));
```

۱) خط ۱
۲) خط ۵
۳) خط ۲
۴) هیچ خطایی رخ نمی‌دهد

۶۷۸ متوسط

خروجی کد زیر کدام است؟

```JavaScript
const p1 = Promise.resolve(1);
const p2 = p1.then((x) => x + 1);
console.log(p1 === p2);
p2.then((x) => console.log(x));
```

۱) `true` `2`
۲) `false` `1`
۳) `true` `1`
۴) `false` `2`

۶۷۹ متوسط

خروجی کد زیر کدام است؟

```javascript
Promise.resolve("ok")
  .catch(() => "recovered")
  .then((v) => console.log(v));
```

۱) `ok`
۲) `recovered`
۳) `undefined`
۴) `ok` `recovered`

۶۸۰ متوسط

سه تابع زیر هر کدام یک Promise برمی‌گردانند (نسخه‌ی Promiseیِ کدی که با callback به شکل callback hell نوشته شده بود). کدام گزینه دقیقاً `Total: 100` را چاپ می‌کند؟

```JavaScript
function getUser(id) {
  return new Promise((resolve) => {
    setTimeout(() => resolve({ id: id, name: "Sara" }), 50);
  });
}
function getOrders(user) {
  return new Promise((resolve) => {
    setTimeout(() => resolve(["book", "pen"]), 50);
  });
}
function getPrice(orders) {
  return new Promise((resolve) => {
    setTimeout(() => resolve(orders.length * 50), 50);
  });
}
```

۱

```JavaScript
getUser(1)
  .then((user) => getOrders(user))
  .then((orders) => getPrice(orders))
  .then((price) => console.log("Total:", price));
```

۲

```JavaScript
getUser(1)
  .then((user) => { getOrders(user); })
  .then((orders) => getPrice(orders))
  .then((price) => console.log("Total:", price));
```

۳

```JavaScript
const user = getUser(1);
const orders = getOrders(user);
const price = getPrice(orders);
console.log("Total:", price);
```

۴

```JavaScript
getUser(1)
  .then((user) => getOrders(user))
  .then((orders) => console.log("Total:", getPrice(orders)));
```

۶۸۱ متوسط

کدام گزینه درباره‌ی `Promise.all` **نادرست** است؟

۱) آرایه‌ی نتیجه به ترتیب آرایه‌ی ورودی است، نه به ترتیب تمام شدن کارها

۲) اگر یکی از Promiseها rejected شود، `Promise.all` بلافاصله با همان دلیل rejected می‌شود

۳) اگر عضوی از آرایه Promise نباشد (مثلاً عدد `42`)، همان مقدار در آرایه‌ی نتیجه قرار می‌گیرد

۴) کارها را یکی‌یکی اجرا می‌کند: کار دوم فقط بعد از تمام شدن کار اول شروع می‌شود

۶۸۲ سخت

خروجی کد زیر کدام است؟

```JavaScript
const p = new Promise((resolve, reject) => {
  resolve("first");
  reject("second");
  console.log("still running");
  resolve("third");
});

p.then((v) => console.log("then:", v))
  .catch((e) => console.log("catch:", e));
```

۱) `then: first`

۲) `still running` `then: first`

۳) `still running` `catch: second`

۴) `still running` `then: third`

۶۸۳ سخت

خروجی کد زیر کدام است؟

```JavaScript
Promise.resolve("data")
  .finally((v) => {
    console.log("finally:", v);
    return "changed";
  })
  .then((v) => console.log("then:", v));
```

۱) `finally: data` `then: changed`

۲) `finally: undefined` `then: data`

۳) `finally: data` `then: data`

۴) `finally: undefined` `then: changed`

۶۸۴ سخت

خروجی کد زیر کدام است؟

```JavaScript
const p = new Promise((resolve) => {
  setTimeout(() => resolve("ok"), 100);
});

p.then(console.log("done"));
console.log("end");
```

۱) `end` `done`
۲) `end` `ok`
۳) `done` `end`
۴) `end` `done` `ok`

۶۸۵ سخت

خروجی کد زیر کدام است؟

```JavaScript
Promise.resolve(1)
  .then((x) => x + 1)
  .then((x) => Promise.reject(x * 10))
  .then((x) => console.log("then", x))
  .catch((e) => {
    console.log("catch", e);
    return e + 1;
  })
  .then((x) => console.log("then2", x))
  .finally(() => console.log("finally"));
```

۱) `then 20` `then2 21` `finally`
۲) `catch 20` `finally`
۳) `catch 2` `then2 3` `finally`
۴) `catch 20` `then2 21` `finally`

۶۸۶ سخت

در **Node.js**، خروجی کد زیر کدام است؟

```JavaScript
const p = new Promise((resolve, reject) => {
  setTimeout(() => reject("Error occurred!"), 100);
});

p.then((result) => console.log(result));
setTimeout(() => console.log("still alive"), 500);
```

۱) `still alive`

۲) `Uncaught UnhandledPromiseRejection`

۳) `Error occurred!` `still alive`

۴) `still alive` `Uncaught UnhandledPromiseRejection`

۶۸۷ سخت

در **Node.js**، خروجی کد زیر کدام است؟

```JavaScript
const p = new Promise((resolve, reject) => {
  setTimeout(() => {
    const user = null;
    resolve(user.name);
  }, 100);
});

p.catch((e) => console.log("caught:", e.message));
setTimeout(() => console.log("after"), 200);
```

۱) `caught: Cannot read properties of null (reading 'name')` `after`

۲) `caught: Cannot read properties of null (reading 'name')`

۳) `after`

۴) `Uncaught TypeError`

۶۸۸ سخت

خروجی کد زیر کدام است؟

```javascript
const p = new Promise((resolve) => {
  console.log("start");
  throw new Error("sync boom");
  resolve("never");
});

p.then((v) => console.log("then:", v))
  .catch((e) => console.log("catch:", e.message));
console.log("end");
```

۱) `start` `end` `catch: sync boom`

۲) `start` `Uncaught Error`

۳) `start` `catch: sync boom` `end`

۴) `end` `start` `catch: sync boom`

۶۸۹ سخت

خروجی کد زیر کدام است؟

```javascript
function task(name, ms, fail) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      console.log(name, "finished");
      if (fail) reject(new Error(name + " failed"));
      else resolve(name);
    }, ms);
  });
}

Promise.all([task("A", 300), task("B", 100, true), task("C", 200)])
  .then((results) => console.log("results:", results))
  .catch((err) => console.log("error:", err.message));
```

۱) `B finished` `C finished` `A finished` `error: B failed`

۲) `B finished` `error: B failed`

۳) `B finished` `error: B failed` `C finished` `A finished`

۴) `A finished` `B finished` `C finished` `error: B failed`

۶۹۰ سخت

کدام گزینه **درست** است؟

۱) اگرِ callback داخل `.then()` یک Promise برگرداند، `.then` بعدی منتظر آن می‌ماند و مقدار نهایی‌اش را دریافت می‌کند

۲) `.then()` همان Promiseی را برمی‌گرداند که روی آن صدا زده شده است

۳) ِcallback داخل `.finally()` مقدار Promise را به عنوان آرگومان دریافت می‌کند

۴) اگر Promise از قبل fulfilled شده باشد، ِcallback داخل `.then()` همگام و فوراً اجرا می‌شود

فصل دوم

۲

JavaScript در HTML (کار با DOM)

بر اساس اسلایدهای «4 - JS in HTML.pdf»

حالا JavaScript را به صفحه‌ی وب وصل می‌کنیم. می‌بینیم مرورگر چطور HTML را به درخت DOM تبدیل می‌کند، چطور عنصرها را پیدا، اضافه، حذف و ویرایش کنیم، به رویدادها (event) واکنش نشان دهیم، با فرم‌ها کار کنیم و از امکانات شیء `window` و حافظه‌های `localStorage` و `sessionStorage` استفاده کنیم.

۱۳ درس · ۳۰۳ تست · آزمون جامع پایان فصل

۲-۱ مرور پیش‌نیازها: سه لایه‌ی وب، شیء، this و async
۲-۲ زبان‌های نشانه‌گذاری و درخت DOM
۲-۳ سلسله‌مراتب DOM در JavaScript و شیء document
۲-۴ پیدا کردن عناصر (Querying the DOM)
۲-۵ رویدادها (Events) و addEventListener
۲-۶ شیء event، انتشار رویداد (Bubbling) و Delegation
۲-۷ تغییر متن، HTML و استایل عناصر
۲-۸ ساختن و افزودن عناصر جدید
۲-۹ حذف و جایگزینی عناصر
۲-۱۰ ویژگی‌ها (Attributes) و کلاس‌ها: setAttribute و classList
۲-۱۱ کار با فرم‌ها (Forms)
۲-۱۲ شیء window: پنجره‌ی مرورگر
۲-۱۳ ذخیره‌سازی در مرورگر: localStorage و sessionStorage
آزمون جامع فصل دوم

۲-۱ مرور پیش‌نیازها: سه لایه‌ی وب، شیء، this و async

منبع: JS in HTML، اسلایدهای ۳ تا ۶

درسنامه

این درس، مرور چند پیش‌نیاز است که اسلایدهای ۳ تا ۶ پیش از شروع بحث DOM یادآوری می‌کنند. عنوان این جلسه «**Structure Meets Behaviour!**» است، یعنی «ساختار به رفتار می‌رسد»: قرار است JavaScript (لایه‌ی رفتار) را به HTML (لایه‌ی ساختار) وصل کنیم. طبق اسلاید ۲، این جلسه سه هدف دارد:

1. بفهمیم **DOM** چیست (از درس بعد کامل می‌بینیم).
2. راهی پیدا کنیم که صفحه‌هایمان **تعاملی (interactive)** شوند، یعنی به کارهای کاربر واکنش نشان دهند.
3. بفهمیم **برنامه‌نویسی ناهمگام (async)** در وب به چه کاری می‌آید.

۱) سه لایه‌ی یک وب‌سایت (اسلاید ۳)

یک تشبیه ساده

یک خانه را تصور کن. **اسکلت و دیوارها** مشخص می‌کنند خانه چند اتاق دارد و هر چیز کجاست. **رنگ و دکوراسیون** مشخص می‌کنند خانه چه شکلی دیده شود. **سیم‌کشی برق و کلیدها** مشخص می‌کنند وقتی کلیدی را می‌زنی چه اتفاقی بیفتد. یک صفحه‌ی وب هم دقیقاً همین سه لایه را دارد.

لایه	سؤالی که جواب می‌دهد (اسلاید ۳)	فناوری	مثال
Structure (ساختار)	What the **content** is: محتوا چیست؟	HTML	یک عنوان `<h1>` و یک دکمه‌ی `<button>`
Style (ظاهر)	How it **looks**: چه شکلی است؟	CSS	رنگ، اندازه‌ی فونت، فاصله‌ها و چیدمان
Behaviour (رفتار)	How it **reacts**: چه واکنشی نشان می‌دهد؟	JavaScript	با کلیک روی دکمه، متن عنوان عوض شود

اسلاید ۳ نام فناوری‌ها را ننوشته، ولی تناظر استاندارد (از جلسه‌های قبل) همین است: ساختار = HTML، ظاهر = CSS، رفتار = JavaScript. مثال کوچک زیر هر سه لایه را با هم دارد:

HTML

```
<!DOCTYPE html>
<html>
  <head>
    <!-- Style layer: CSS -->
    <style>
      h1 { color: navy; }
    </style>
  </head>
  <body>
    <!-- Structure layer: HTML -->
    <h1 id="title">Hello World</h1>
    <button id="btn">CLICK ME!</button>

    <!-- Behaviour layer: JavaScript -->
    <script>
      document.getElementById("btn").addEventListener("click", () => {
        document.getElementById("title").textContent = "Goodbye World";
      });
    </script>
  </body>
</html>
```

بدون JavaScript، صفحه فقط چیزی را **نمایش** می‌دهد؛ با JavaScript صفحه **واکنش** نشان می‌دهد (اینجا با کلیک، متن عنوان به `Goodbye World` تغییر می‌کند). البته لایه‌ی رفتار می‌تواند روی دو لایه‌ی دیگر هم اثر بگذارد؛ مثلاً در اسلاید ۱۹ با `title.style.color = 'red'` رنگ متن را از داخل JavaScript عوض می‌کنیم.

۲) اشیاء (Objects) — اسلاید ۴

شیء (object) مجموعه‌ای از داده‌ها و رفتارهای مرتبط است که کنار هم بسته‌بندی شده‌اند. شیء از **جفت‌های کلید–مقدار (key-value pairs)** ساخته می‌شود:

- اگر مقدارِ یک کلید **داده** باشد، به آن **property** (ویژگی) می‌گوییم.
- اگر مقدارِ یک کلید **تابع** باشد، به آن **method** (متد، یعنی تابعی که متعلق به شیء است) می‌گوییم.

JavaScript

```
const student = {
  name: "Ali",            // property (data)
  age: 20,                // property (data)
  greet() {               // method (function)
    return "Hi, I am " + this.name;
  },
};

console.log(student.name);    // "Ali"
console.log(student.greet()); // "Hi, I am Ali"
```

JSON (مخفف JavaScript Object Notation، یعنی «نشانه‌گذاری شیء JavaScript») یک قالب **متنی** رایج برای **ذخیره و به اشتراک گذاشتن** داده‌های شیء است؛ مثلاً برای فرستادن داده به سرور یا ذخیره در حافظه‌ی مرورگر. دو تابع اصلی آن:

```JavaScript
const text = JSON.stringify(student); // object -> JSON string
console.log(text);                    // {"name":"Ali","age":20}   (the method is gone!)

const copy = JSON.parse('{"name":"Sara","age":21}'); // JSON string -> object
console.log(copy.age + 1);            // 22   (age is a real number again)
```

در JSON کلیدها حتماً داخل `"` هستند و **تابع‌ها ذخیره نمی‌شوند**؛ JSON فقط داده نگه می‌دارد، نه رفتار.

«در JavaScript تقریباً همه‌چیز شیء است»

در اسلاید ۴ میم معروف دو فضانورد آمده: اولی به زمینی که رویش لوگوی JavaScript است نگاه می‌کند و می‌گوید «Wait it's all OBJECTS» (صبر کن، همه‌اش شیء است؟) و دومی پشت سرش می‌گوید «Always has been» (همیشه همین بوده). زیر آن نوشته: «از نظر فنی، در JavaScript **تقریباً** همه‌چیز شیء است». آرایه‌ها، تابع‌ها، تاریخ‌ها و (همان‌طور که در این فصل می‌بینیم) **همه‌ی عنصرهای صفحه** شیء هستند:

```JavaScript
console.log(typeof []);             // "object"
console.log(typeof function () {}); // "function"  (functions are special objects)
console.log(typeof null);           // "object"    (a famous historical bug!)
```

پس چرا «تقریباً»؟ چون مقدارهای **primitive** (مقدار اولیه: `string`، `number`، `boolean`، `null`، `undefined` و `bigint`، `symbol`) شیء **نیستند**. با این حال می‌توانی روی یک رشته property بخوانی یا متد صدا بزنی:

```JavaScript
console.log("abc".length);   // 3
console.log((5).toFixed(1)); // "5.0"
```

راز ماجرا **شیء پوششی (wrapper object)** است: وقتی روی یک primitive نقطه می‌گذاری، JavaScript برای یک لحظه یک شیء موقت (`String`، `Number` یا `Boolean`) دور آن می‌سازد، property یا متد را از آن می‌خواند و بلافاصله آن شیء موقت را دور می‌اندازد. به همین دلیل نمی‌توانی روی یک primitive چیزی ذخیره کنی:

```JavaScript
const s = "abc";
s.extra = 42;          // goes to a temporary wrapper (strict mode: TypeError)
console.log(s.extra);  // undefined
console.log(typeof s); // "string"  -> still a primitive
```

ارتباط با این فصل: مرورگر هر تگ HTML را به یک **شیء** تبدیل می‌کند که property دارد (مثل `id` و `textContent`) و method دارد (مثل `addEventListener`). پس هرچه درباره‌ی اشیاء بلدی، اینجا به کارت می‌آید.

۳) کلمه‌ی کلیدی `this` (اسلاید ۵)

قوانین اسلاید ۵ درباره‌ی `this`:

1. `this` به شیئی اشاره می‌کند که تابع را **صدا زده** است.
2. در متدِ یک شیء، `this` خودِ همان شیء است.
3. در کلاس‌ها، `this` همان **نمونه‌ی (instance)** ساخته‌شده با `new` است.

4. در arrow function، `this` از **scope (محدوده‌ی) بیرونی** به ارث می‌رسد؛ arrow function `this` مخصوص به خودش را ندارد.
5. مقدار `this` به **نحوه‌ی صدا زدن** تابع بستگی دارد، نه به جایی که تابع تعریف شده است.

نحوه‌ی فراخوانی	مقدار `this`
`obj.method()`	`obj` (شیءِ قبل از نقطه)
`new Person()` و متدهای کلاس	نمونه‌ی ساخته‌شده
arrow function	همان `this` بیرونِ arrow function
فراخوانی ساده‌ی `f()`	حالت عادی (sloppy): شیء سراسری (`window` در مرورگر، `globalThis` در Node.js)؛ حالت strict و بدنه‌ی کلاس‌ها: `undefined`
listener با `function` معمولی	همان عنصری که listener روی آن ثبت شده است

JavaScript

```
const obj = {
  name: "obj",
  regular() {
    return this.name;
  },
};
const other = { name: "other", regular: obj.regular }; // the same function!
const f = obj.regular;                                 // detached from obj

console.log(obj.regular());   // "obj"
console.log(other.regular()); // "other"  -> depends on the caller
console.log(f());             // Node.js: undefined | browser: "" (see the trap box)
```

arrow functionی که مستقیماً به‌عنوان property در یک object literal نوشته شود، `this` را از **بیرونِ** آن شیء می‌گیرد؛ ولی arrow functionی که **داخل یک متد** نوشته شود، `this` همان متد را به ارث می‌برد که معمولاً همان چیزی است که می‌خواهیم:

```JavaScript
const counter = {
  count: 10,
  regular() { return this.count; },
  arrow: () => this.count,   // `this` comes from OUTSIDE the object literal
};
console.log(counter.regular()); // 10
console.log(counter.arrow());   // undefined

class Timer {
  constructor(label) {
    this.label = label;            // `this` = the new instance
  }
  report() {
    const show = () => this.label; // arrow: inherits report's `this`
    return show();
  }
}
console.log(new Timer("T1").report()); // "T1"
```

`this` در صفحه‌ی وب: داخل event listener

در درس «رویدادها (Events) و addEventListener» می‌بینیم که با `addEventListener` به مرورگر می‌گوییم «هر وقت روی این عنصر کلیک شد، این تابع را اجرا کن». اینجا هم قانون «نحوه‌ی فراخوانی» برقرار است: مرورگر listener معمولی (`function`) را طوری صدا می‌زند که `this` همان **عنصری** باشد که listener روی آن ثبت شده (یعنی `event.currentTarget`). ولی arrow function `this` خودش را ندارد و `this` بیرونی را برمی‌دارد؛ در سطح بالای یک اسکریپت مرورگر، آن `this` همان `window` است.

```HTML
<button id="a">First</button>
<button id="b">Second</button>
<script>
  document.getElementById("a").addEventListener("click", function (event) {
    console.log(this.id, this === event.currentTarget); // a true
  });
  document.getElementById("b").addEventListener("click", (event) => {
    console.log(this === window, event.currentTarget.id); // true b
  });
</script>
```

> **نکته**
>
> اگر داخل listener به خودِ عنصر نیاز داری، مطمئن‌ترین راه `event.currentTarget` است؛ هم در `function` معمولی جواب می‌دهد و هم در arrow function.

> **دام تستی: `this.name` در مرورگر و Node.js**
>
> در کد بالا `f()` (متدی که از شیء جدا شده) در **Node.js** مقدار `undefined` می‌دهد، ولی در **مرورگر** رشته‌ی خالی `""`! چون در فراخوانی ساده (حالت sloppy)، `this` همان `window` است و `window` از قبل یک property به نام `name` دارد که مقدار پیش‌فرض `""` است. پس در تست‌ها همیشه دقت کن کد در کدام محیط اجرا می‌شود.

> ⚠ **دام تستی**
>
> arrow functionی که مستقیماً به‌عنوان property در object literal نوشته شود، `this` را **شیء** نمی‌گیرد. برای متدهایی که به `this` نیاز دارند از نحو `method() { }` یا `function` استفاده کن.

۴) یادآوری JavaScript ناهمگام (اسلاید ۶)

چرا async در وب این‌قدر مهم است؟

JavaScript در مرورگر روی یک **thread اصلی** (رشته‌ی اجرای اصلی) اجرا می‌شود و همین thread کارهای دیگری مثل رسم صفحه و پاسخ به کلیک و تایپ کاربر را هم انجام می‌دهد. اگر یک کد همگام (synchronous) چند ثانیه منتظر بماند، در تمام این مدت صفحه **یخ می‌زند**: دکمه‌ها کار نمی‌کنند و حتی اسکرول هم گیر می‌کند. در وب سه نوع کارِ «منتظرماندنی» زیاد داریم:

- **زمان‌سنج‌ها (timers)**: مثل `setTimeout` و `setInterval` (در درس «شیء window: پنجره‌ی مرورگر» می‌بینیم).
- **رویدادهای کاربر**: معلوم نیست کاربر کی کلیک می‌کند؛ پس تابعی می‌دهیم که «هر وقت کلیک شد» اجرا شود.
- **درخواست‌های شبکه**: گرفتن داده از سرور ممکن است از چند صدم ثانیه تا چند ثانیه طول بکشد.

راه‌حل این است که کار زمان‌بر را شروع کنیم، تابعی برای «بعداً» ثبت کنیم و فوراً به کار خودمان ادامه دهیم تا صفحه پاسخ‌گو بماند. این همان هدف سوم جلسه است: «Understand async programming usages in web».

سه سبک: Promise ،callback و async/await

۱. Callback (تابع برگشتی): تابعی را به تابع دیگر می‌دهیم تا **بعداً** اجرا شود. خطاها را باید **دستی** بررسی کنیم؛ معمولاً اولین پارامتر callback خطاست (`err`):

```JavaScript
function loadData(ok, callback) {
  setTimeout(() => {
    if (ok) callback(null, "data");
    else callback(new Error("failed"));
  }, 10);
}

loadData(false, (err, data) => {
  if (err) {                                  // manual error handling
    console.log("callback error:", err.message);
    return;
  }
  console.log("callback data:", data);
});
// callback error: failed
```

وقتی چند کار ناهمگام باید پشت سر هم انجام شوند، callbackها تو در تو می‌شوند و کد به شکل یک هرمِ کج درمی‌آید که خواندن و مدیریت خطایش سخت است. به این وضعیت **callback hell** («جهنم callbackها») می‌گویند:

```JavaScript
// Illustration only: getUser, getOrders and getDetails are imaginary functions.
getUser(1, (err, user) => {
  getOrders(user, (err, orders) => {
    getDetails(orders[0], (err, details) => {
      console.log(details); // deeper and deeper...
    });
  });
});
```

۲. Promise (وعده): شیئی که نماینده‌ی یک **مقدار آینده** است و یکی از سه **حالت (state)** را دارد: `pending` (در انتظار)، `fulfilled` (انجام‌شده) یا `rejected` (ردشده). نتیجه را با `.then()` و خطا را با `.catch()` می‌گیریم؛ زنجیره‌ی `.then()` ها کد را از حالت تو در تو بیرون می‌آورد:

```JavaScript
function fetchUser(ok) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (ok) resolve("user loaded");
      else reject(new Error("network down"));
    }, 30);
  });
}

fetchUser(true)
  .then((value) => console.log("then:", value))    // then: user loaded
  .catch((err) => console.log("catch:", err.message));
```

۳. async/await: بر پایه‌ی همان Promiseهاست، ولی نحو (syntax) تمیزتری دارد و کد را **شبیه کد همگام** نشان می‌دهد. `await` فقط اجرای **همان تابع async** را تا آماده شدن نتیجه متوقف می‌کند، نه کل برنامه را. خطاها با `try...catch` گرفته می‌شوند:

```JavaScript
async function main() {
  try {
    console.log(await fetchUser(true));  // user loaded
    console.log(await fetchUser(false)); // rejects -> jumps to catch
    console.log("after");                // never runs
  } catch (err) {
    console.log("caught:", err.message); // caught: network down
  }
}
main();
```

سبک	ایده	مدیریت خطا (اسلاید ۶)	ویژگی
Callback	تابعی را برای اجرا در آینده می‌دهیم	دستی (مثلاً بررسی `err`)	تو در تو شدن: callback hell
Promise	نماینده‌ی یک مقدار آینده با چند حالت	`.catch()`	زنجیره با `.then()` / `.catch()`
async/await	نحو ساده‌تر روی Promise	`try...catch`	ظاهر همگام و خوانایی بیشتر

ترتیب اجرا (یادآوری فصل اول)

ابتدا همه‌ی کد همگام اجرا می‌شود، بعد **microtask**ها (کارهای کوچکِ فوری، مثل callback `.then` و ادامه‌ی تابع بعد از `await`) و بعد **macrotask**ها (مثل callback `setTimeout`، حتی با تأخیر `0`). جزئیات event loop در فصل اول آمده است:

```JavaScript
async function load() {
  console.log("load start");
  const data = await Promise.resolve("data ready");
  console.log(data);
}
console.log("script start");
setTimeout(() => console.log("timeout"), 0);
load();
console.log("script end");
// script start -> load start -> script end -> data ready -> timeout
```

راستی، listenerهای رویداد هم نوعی callback هستند: در `btn.addEventListener("click", handler)` تابع `handler` «بعداً»، یعنی هر وقت کاربر کلیک کرد، اجرا می‌شود.

نکته

بدنه‌ی تابع async تا رسیدن به اولین `await` **همگام** اجرا می‌شود؛ برای همین در مثال بالا `load start` قبل از `script end` چاپ شد.

دام تستی

- `setTimeout(fn, 0)` یعنی «حداقل بعد از ۰ میلی‌ثانیه، **هر وقت نوبتش شد**»، نه «همین الان». callbackهای Promise (microtask) همیشه زودتر از آن اجرا می‌شوند.
- `JSON.stringify` متدها را حذف می‌کند، و `typeof null` برابر `"object"` است، با اینکه `null` شیء نیست.

جمع‌بندی

- ساختار = HTML (محتوا چیست)، ظاهر = CSS (چه شکلی است)، رفتار = JavaScript (چه واکنشی نشان می‌دهد).
- شیء = جفت‌های کلید–مقدار؛ داده = property و تابع = method؛ JSON قالب متنی برای ذخیره و تبادل داده است.
- primitiveها شیء نیستند، ولی با wrapper object موقت property و متد دارند (`"abc".length`).
- `this` به نحوه‌ی فراخوانی بستگی دارد؛ arrow function، `this` بیرونی را به ارث می‌برد؛ در listener معمولی، `this` همان عنصر است.
- callback (خطای دستی)، async/await، Promise (`.catch()`)، (`try...catch`)؛ ترتیب: همگام، microtask، macrotask.

تست‌های تمرینی

۱۷ تست · شماره‌ی ۶۹۱ تا ۷۰۷

۶۹۱ آسان

در مدل سه‌لایه‌ی یک وب‌سایت (اسلاید ۳)، لایه‌ای که به سؤال «صفحه چه واکنشی نشان می‌دهد؟» (How it reacts) پاسخ می‌دهد کدام است و معمولاً با چه فناوری‌ای ساخته می‌شود؟

۱) Structure، با HTML
۲) Style، با CSS
۳) Behaviour، با JavaScript
۴) Behaviour، با CSS

۶۹۲ آسان

کدام عبارت درباره‌ی سه لایه‌ی وب نادرست است؟

۱) HTML مشخص می‌کند محتوای صفحه چیست؛ مثلاً یک عنوان و یک دکمه.
۲) CSS مشخص می‌کند صفحه چه شکلی است؛ مثلاً رنگ، اندازه و چیدمان.
۳) JavaScript مشخص می‌کند صفحه به کارهای کاربر چه واکنشی نشان دهد.
۴) تغییر رنگ متن فقط با CSS ممکن است و JavaScript نمی‌تواند ظاهر عنصرها را تغییر دهد.

۶۹۳ آسان

خروجی کد زیر در Node.js کدام است؟

```JavaScript
const student = {
  name: "Ali",
  age: 20,
  greet() {
    return "Hi, I am " + this.name;
  },
};
console.log(typeof student.name, typeof student.greet);
console.log(student.greet());
```

۱) `string function` `Hi, I am Ali`
۲) `string object` `Hi, I am Ali`
۳) `property method` `Hi, I am Ali`
۴) `string function` `Hi, I am undefined`

۶۹۴ متوسط

خروجی کد زیر کدام است؟

```JavaScript
const student = {
  name: "Ali",
  age: 20,
  greet() {
    return "Hi";
  },
};
const text = JSON.stringify(student);
console.log(text);
console.log(typeof text, JSON.parse(text).age + 1);
```

۱) `{"name":"Ali","age":20,"greet":"Hi"}` `string 21`

۲) `{"name":"Ali","age":20}` `string 21`

۳) `{name:"Ali",age:20}` `string 21`

۴) `{"name":"Ali","age":20}` `string 201`

۶۹۵ متوسط

با توجه به جمله‌ی «در JavaScript تقریباً همه‌چیز شیء است» (اسلاید ۴)، کدام گزینه درست است؟

۱) `typeof "abc"` برابر `"object"` است.

۲) `typeof null` برابر `"null"` است.

۳) `typeof []` برابر `"object"` است.

۴) `typeof function () {}` برابر `"object"` است.

۶۹۶ سخت

خروجی کد زیر در Node.js (حالت عادی، نه strict) کدام است؟

```JavaScript
const s = "abc";
console.log(s.length, s.toUpperCase());
s.extra = 42;
console.log(s.extra, typeof s);
```

۱) `3 ABC` `undefined string`

۲) `3 ABC` `42 object`

۳) `3 ABC` `42 string`

۴) `Uncaught TypeError`

۶۹۷ متوسط

خروجی کد زیر در Node.js کدام است؟

```javascript
const obj = {
  name: "obj",
  regular() {
    return this.name;
  },
};
const other = { name: "other", regular: obj.regular };
const f = obj.regular;
console.log(obj.regular());
console.log(other.regular());
console.log(f());
```

۱) `obj` `obj` `obj`

۲) `obj` `other` `undefined`

۳) `obj` `obj` `undefined`

۴) `obj` `other` `obj`

۶۹۸ سخت

این کد در یک تگ `<script>` معمولی **در مرورگر** اجرا می‌شود (نه در Node.js). خروجی کدام است؟

```javascript
const user = {
  name: "Ali",
  getName() {
    return this.name;
  },
};
const f = user.getName;
console.log(user.getName());
console.log(JSON.stringify(f()));
```

۱) `Ali` `"Ali"`

۲) `Ali` `undefined`

۳) `Ali` `""`

۴) `Ali` `Uncaught TypeError`

۶۹۹ متوسط

خروجی کد زیر در Node.js کدام است؟

```JavaScript
const counter = {
  count: 10,
  regular() {
    return this.count;
  },
  arrow: () => this.count,
};
console.log(counter.regular(), counter.arrow());
```

۱) `10 10`
۲) `undefined 10`
۳) `Uncaught TypeError`
۴) `10 undefined`

۷۰۰ سخت

خروجی کد زیر کدام است؟

```JavaScript
class Timer {
  constructor(label) {
    this.label = label;
  }
  report() {
    const show = () => this.label;
    return show();
  }
}
const t = new Timer("T1");
console.log(t.report());
const r = t.report;
console.log(r());
```

۱) `T1` `Uncaught TypeError`
۲) `T1` `T1`
۳) `T1` `undefined`
۴) `undefined` `Uncaught TypeError`

۷۰۱ متوسط

در صفحه‌ی زیر، کاربر ابتدا روی دکمه‌ی First و سپس روی دکمه‌ی Second کلیک می‌کند. خروجی Console کدام است؟

HTML

```
<button id="a">First</button>
<button id="b">Second</button>
```

JavaScript

```
document.getElementById("a").addEventListener("click", function () {
  console.log(this.textContent);
});
document.getElementById("b").addEventListener("click", () => {
  console.log(this === window);
});
```

۱) `First` `false`
۲) `First` `true`
۳) `undefined` `true`
۴) `First` `Second`

۷۰۲ آسان

طبق اسلاید ۶، روش مدیریت خطا در هر یک از سه سبک برنامه‌نویسی ناهمگام کدام است؟

۱) Promise — `try...catch` :callback؛ دستی — async/await: `.catch()`
۲) async/await — `try...catch` :Promise؛ `.catch()` :callback؛ دستی
۳) هر سه سبک فقط با `try...catch` مدیریت می‌شوند.
۴) callback: دستی (مثلاً بررسی پارامتر `err`) — async/await — Promise: `.catch()`؛ `try...catch`

۷۰۳ سخت

خروجی کد زیر کدام است؟

JavaScript

```
async function load() {
  console.log("load start");
  const data = await Promise.resolve("data ready");
  console.log(data);
}
console.log("script start");
setTimeout(() => console.log("timeout"), 0);
load();
console.log("script end");
```

۱) `script start` `load start` `data ready` `script end` `timeout`
۲) `script start` `script end` `load start` `data ready` `timeout`
۳) `script start` `load start` `script end` `data ready` `timeout`
۴) `script start` `load start` `script end` `timeout` `data ready`

۷۰۴ متوسط

خروجی کد زیر کدام است؟

```JavaScript
function fetchUser(ok) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (ok) resolve("user loaded");
      else reject(new Error("network down"));
    }, 10);
  });
}
async function main() {
  try {
    console.log(await fetchUser(true));
    console.log(await fetchUser(false));
    console.log("after");
  } catch (err) {
    console.log("caught:", err.message);
  } finally {
    console.log("done");
  }
}
main();
```

۱) `user loaded` `caught: network down` `done`

۲) `user loaded` `caught: network down` `after` `done`

۳) `user loaded` `Uncaught Error`

۴) `caught: network down` `done`

۷۰۵ آسان

چرا در صفحه‌ی وب کارهای زمان‌بر مثل درخواست شبکه را به‌صورت ناهمگام (async) انجام می‌دهیم؟

۱) چون JavaScript در مرورگر هم‌زمان روی چند thread اجرا می‌شود و اجرای همگام در آن ممکن نیست.

۲) چون اگر کد همگام منتظر بماند، صفحه تا پایان آن کار «یخ می‌زند» و به کلیک و تایپ کاربر پاسخ نمی‌دهد.

۳) چون کد ناهمگام همیشه سریع‌تر از کد همگام اجرا می‌شود.

۴) چون مرورگر اجرای توابع همگام را بعد از بارگذاری کامل صفحه ممنوع می‌کند.

۷۰۶ آسان

کد زیر نمونه‌ای از کدام مشکل است و راه‌حل مدرن آن چیست؟

```JavaScript
getUser(1, (err, user) => {
  getOrders(user, (err, orders) => {
    getDetails(orders[0], (err, details) => {
      console.log(details);
    });
  });
});
```

۱) خطای TDZ؛ باید تعریف تابع‌ها را بالاتر برد.

۲) سایه‌اندازی متغیرها (shadowing)؛ باید نام پارامترهای `err` را عوض کرد.

۳) callback hell؛ بازنویسی با زنجیره‌ی Promise (`.then()`) یا `async/await`.

۴) نشت حافظه در DOM؛ باید listenerها را حذف کرد.

۷۰۷ متوسط

خروجی کد زیر کدام است؟

```JavaScript
Promise.resolve(5)
  .then((n) => n * 2)
  .then((n) => {
    if (n > 5) throw new Error("too big");
    return n;
  })
  .then((n) => console.log("value:", n))
  .catch((err) => console.log("error:", err.message));
console.log("sync");
```

۱) `value: 10` `sync`

۲) `sync` `value: 10`

۳) `error: too big` `sync`

۴) `sync` `error: too big`

۲-۲ زبان‌های نشانه‌گذاری و درخت DOM

منبع: JS in HTML، اسلایدهای ۷ تا ۱۱

درسنامه

تا اینجا HTML را فقط به چشم «متنی پر از تگ» دیده‌ایم. اما مرورگر وقتی صفحه را باز می‌کند، این متن را به یک **درخت از اشیاء** تبدیل می‌کند که JavaScript می‌تواند آن را بخواند و تغییر دهد. این درخت همان **DOM** است و پایه‌ی کل این فصل.

زبان نشانه‌گذاری (Markup language) چیست؟ (اسلاید ۷)

زبان نشانه‌گذاری (markup language) زبانی است که با **تگ‌ها (tags)** یا نشانه‌ها، **ساختار** یک سند را توصیف می‌کند: کدام بخش عنوان است، کدام بخش پاراگراف است و چه چیزی داخل چه چیزی قرار دارد. اسلاید ۷ سه نمونه نام می‌برد: XML، HTML و Markdown.

HTML

```
<!-- HTML: the structure of a web page -->
<h1>Shop</h1>
<p>Welcome to <b>our</b> shop!</p>
```

XML

```
<!-- XML: the structure of any data, with our own tag names -->
<student>
  <name>Ali</name>
  <age>20</age>
</student>
```

Markdown

```
# Shop
Welcome to **our** shop!
```

(Markdown به‌جای تگ‌های `<...>` از نشانه‌هایی مثل `#` برای عنوان و `**` برای متن پررنگ استفاده می‌کند، ولی کارش همان است: توصیف ساختار.) زبان نشانه‌گذاری، زبان برنامه‌نویسی **نیست**؛ شرط، حلقه و محاسبه ندارد و فقط می‌گوید هر بخش «چیست». منطق و رفتار کار JavaScript است.

سند، یک درخت سلسله‌مراتبی است

چون تگ‌ها **تو در تو (nested)** نوشته می‌شوند، سند به شکل یک **درخت سلسله‌مراتبی (hierarchical tree)** درمی‌آید: هر تگی که داخل تگ دیگری باشد **فرزند (child)** آن است و تگ بیرونی **والد (parent)** آن. به گفته‌ی اسلاید ۷، رابطه‌ی والد و فرزند نشان می‌دهد محتوا چطور تو در تو شده است، و همین مدل درختی **پایه‌ی DOM** است.

> **یک تشبیه ساده**
>
> شجره‌نامه‌ی یک خانواده یا ساختار پوشه‌های کامپیوترت را در نظر بگیر: داخل پوشه‌ی `Documents` پوشه‌ی `University` است و داخل آن چند فایل. هر پوشه (به‌جز پوشه‌ی ریشه) دقیقاً یک والد دارد و ممکن است چند فرزند داشته باشد. درخت یک سند HTML هم دقیقاً همین‌طور است.

نمودار اسلاید ۷ یک درخت عمومی است (A به B و C و D فلش دارد، B به E و F، و E و F هر کدام یک فرزند دارند):

```
        A
      /  |  \
     B   C   D
    / \
   E   F
   |   |
   G   H
```

واژه	معنی	در نمودار بالا
ریشه (root)	تنها گره‌ای که والد ندارد	`A`
والد (parent)	گره‌ی بلافاصله بالای یک گره	والدِ `E` گره‌ی `B` است
فرزند (child)	گره‌ی بلافاصله پایین یک گره	فرزندان `A` : `B` ، `C` و `D`
هم‌نیا (sibling)	گره‌هایی که والد مشترک دارند	`B` ، `C` و `D` ؛ همچنین `E` و `F`
جد (ancestor)	والد، والدِ والد و ... تا ریشه	اجداد `G` : `E` ، `B` و `A`
نواده (descendant)	فرزند، فرزندِ فرزند و ...	نوادگان `B` : `E` ، `F` ، `G` و `H`
برگ (leaf)	گره‌ای که هیچ فرزندی ندارد	`C` ، `D` ، `G` و `H`

از نشانه‌گذاری تا DOM (اسلاید ۸)

مرورگر متن HTML را **parse** می‌کند (یعنی می‌خواند و تجزیه می‌کند) و از آن یک **درخت از گره‌ها (nodes)** می‌سازد. قوانین اسلاید ۸:

- هر **تگ** به یک **گره‌ی عنصر (element node)** در درخت تبدیل می‌شود.
- **متنِ داخل تگ‌ها** به **گره‌ی متنی (text node)** تبدیل می‌شود.
- **attributeها** (ویژگی‌های داخل تگ، مثل `href` و `id`) به **propertyهای گره** تبدیل می‌شوند؛ یعنی به خودِ عنصر چسبیده‌اند و **فرزند آن نیستند**.
- همین درخت DOM است که JavaScript می‌تواند به آن دسترسی پیدا کند و تغییرش دهد.

برای HTML زیر:

```HTML
<html>
  <head>
    <title>My title</title>
  </head>
  <body>
    <a href="#">My link</a>
    <h1>My header</h1>
  </body>
</html>
```

نمودار W3Schools در اسلاید ۸ این درخت را نشان می‌دهد (اینجا به شکل متنی بازکشیده شده است):

```
Document
`-- Root element: <html>
    |-- Element: <head>
    |   `-- Element: <title>
    |       `-- Text: "My title"
    `-- Element: <body>
        |-- Element: <a>   ---- Attribute: "href"
        |   `-- Text: "My link"
        `-- Element: <h1>
            `-- Text: "My header"
```

دقت کن که در نمودار، جعبه‌ی `Attribute: "href"` **کنار** `<a>` و با یک خط افقی به آن وصل شده، نه زیر آن؛ چون attribute فرزند نیست، ویژگیِ خودِ عنصر است. جمله‌ی کنار نمودار هم همین را می‌گوید: «ببین درخت DOM عنصرها را چطور مرتب می‌کند».

DOM چیست؟ (اسلاید ۹)

DOM مخفف **Document Object Model** (مدل شیءِ سند) است. هر کلمه‌اش معنایی دارد:

- **Document**: سند، یعنی همان صفحه‌ی وب.
- **Object**: هر بخش صفحه (هر عنصر، متن و ...) به یک **شیء** JavaScript تبدیل می‌شود که property و method دارد.
- **Model**: این اشیاء با هم یک **مدل** (نمایش ساختاریافته) از صفحه می‌سازند که شکلش درخت است.

طبق اسلاید ۹، DOM:

- یک **نمایش درختی (tree-based representation)** از صفحه‌ی وب است؛
- **ساختار HTML را به JavaScript وصل می‌کند**؛
- به برنامه‌ها اجازه می‌دهد عنصرها را **بخوانند، تغییر دهند، اضافه یا حذف کنند**؛
- مثل یک **پل** بین محتوا و تعامل‌پذیری (interactivity) عمل می‌کند.

نمودار Milne Publishing در اسلاید ۹ همان ایده را با رنگ‌بندی نشان می‌دهد. این بار `<h1>` قبل از `<a>` آمده و ریشه با حروف کوچک `document` نوشته شده، درست مثل نام شیء در JavaScript:

```
document
`-- Root element: <html>
    |-- Element: <head>
    |   `-- Element: <title>
    |       `-- Text: "My title"
    `-- Element: <body>
        |-- Element: <h1>
        |   `-- Text: "A heading"
        `-- Element: <a>   ---- Attribute: href
            `-- Text: "Link text"
```

> **DOM همان فایل HTML نیست**
>
> فایل `.html` فقط **دستور ساخت** است. مرورگر از روی آن، درخت DOM را **در حافظه** می‌سازد. وقتی JavaScript عنصری را تغییر می‌دهد، فقط DOM (همان درختِ داخل حافظه) عوض می‌شود و فایل روی دیسک یا سرور دست نمی‌خورد؛ با refresh صفحه همه‌چیز از روی فایل اصلی دوباره ساخته می‌شود. در DevTools، زبانه‌ی Elements (یا Inspector) همین DOM زنده را نشان می‌دهد.

فعالیت کلاسی اسلاید ۱۰: درخت DOM این HTML را رسم کن

کد اسلاید ۱۰ (فایل `sample.html`) دقیقاً این است:

```HTML
<html>
    <head>
        <title>sample page here :)</sample>
        <link rel="stylesheet" href="./style.css">
    </head>
    <body>
        <h1>SAMPLE PAGE!</h1>
        <div class="sampleclass">
            <p>sample paragraph in sample class in sample page!</p>
            <h3 id="2223">wait wha-?</h3>
        </div>
    </body>
</html>
```

پاسخ مورد انتظار (با فرض اینکه خط ۳ با `</title>` بسته شده باشد؛ گره‌های متنیِ فقط‌فاصله را نشان نداده‌ایم و attributeها داخل `{ }` کنار عنصرشان آمده‌اند، چون فرزند نیستند):

```
#document
`-- html
    |-- head
    |   |-- title
    |   |   `-- #text "sample page here :)"
    |   `-- link   {rel="stylesheet", href="./style.css"}
    `-- body
        |-- h1
        |   `-- #text "SAMPLE PAGE!"
        `-- div   {class="sampleclass"}
            |-- p
            |   `-- #text "sample paragraph in sample class in sample page!"
            `-- h3   {id="2223"}
                `-- #text "wait wha-?"
```

چند نکته درباره‌ی این درخت:

- `head` و `body` هم‌نیا هستند و هر دو فرزند `html` اند؛ `p` و `h3` هم‌نیا و فرزند `div` اند؛ `div` والد `p` و جدِ متن `"wait wha-?"` است.
- `link` فرزندی ندارد (تگ خالی است) و `rel` و `href` ویژگی‌های آن هستند.
- در درخت واقعی، بین تگ‌ها گره‌های متنی‌ای هم هست که فقط شامل «خط جدید و فاصله» (whitespace) است. مثلاً `head` در واقع ۵ گره‌ی فرزند دارد: فاصله، `title`، فاصله، `link`، فاصله.

> ⚠ **دامِ تایپیِ اسلاید: `</sample>` به‌جای `</title>`**
>
> روی اسلاید، `<title>` با `</sample>` بسته شده که تگی بی‌معنی است. تگ `<title>` در HTML حالت خاصی دارد: parser همه‌ی محتوای آن را تا رسیدن به یک `</title>` واقعی **متن خام** حساب می‌کند و تگ‌های داخلش را تشخیص نمی‌دهد. چون در این فایل هیچ `</title>` ی وجود ندارد، **کل باقی فایل** (از `</sample>` تا `</html>`) به متن عنوان تبدیل می‌شود و `<body>` **خالی** می‌ماند: صفحه سفید دیده می‌شود و یک عنوان خیلی طولانی در زبانه‌ی مرورگر می‌نشیند.
>
> نکته‌ی مهم‌تر: **parser هیچ‌وقت به‌خاطر HTML نادرست خطا (exception) نمی‌دهد**؛ همیشه سعی می‌کند سند را «تعمیر» کند، حتی اگر نتیجه آن چیزی نباشد که انتظار داشتی.

دو نکته‌ی دیگر درباره‌ی این فایل: اول اینکه `<!DOCTYPE html>` ندارد، پس مرورگر صفحه را در «حالت سازگاری» (quirks mode) نشان می‌دهد؛ همیشه سند را با `<!DOCTYPE html>` شروع کن. دوم اینکه id عنصر `h3` با رقم شروع شده (`2223`)؛ در درس «پیدا کردن عناصر» می‌بینیم این موضوع برای `querySelector` دردسر می‌سازد.

ساختار DOM (اسلاید ۱۱)

خلاصه‌ی اسلاید ۱۱:

- **ریشه‌ی درخت (root node)، `document` است.**
- **عنصرها (elements)**: همان تگ‌ها، مثل `<div>` و `<p>`.
- **attributeها**: propertyهای عنصر، مثل `id` و `class`.
- **متن**: گره‌های متنی (text nodes).
- همه در یک درخت سلسله‌مراتبی به هم وصل‌اند.

دقت کن: `document` **گره‌ی ریشه** است، ولی **عنصر ریشه (root element)** تگ `<html>` است که فرزند `document` است و در JavaScript با `document.documentElement` به آن می‌رسیم.

سلسله‌مراتب کلاس‌ها (نمودار StackOverflow در اسلاید ۱۱)

هر گره در DOM نمونه‌ای از یک **کلاس** است و این کلاس‌ها از هم ارث می‌برند. نمودار اسلاید ۱۱ (از چپ به راست کشیده شده) را اینجا از بالا به پایین می‌بینی؛ `EventTarget` در نمودار اسلاید نیست ولی بالای همه قرار دارد:

```
EventTarget                 (not on the slide: the top of every chain)
`-- Node
    |-- Document
    |   `-- HTMLDocument
    |-- CharacterData
    |   |-- Text
    |   `-- Comment
    |-- Element
    |   `-- HTMLElement
    |       |-- HTMLHeadElement
    |       |-- HTMLBodyElement
    |       |-- HTMLTitleElement
    |       |-- HTMLParagraphElement
    |       |-- HTMLInputElement
    |       |-- HTMLTableElement
    |       `-- ... etc.  (many more element types)
    `-- Attr
```

رابطه‌ی ارث‌بری یعنی رابطه‌ی «**هست یک**»: یک `<p>` یک `HTMLParagraphElement` است، پس یک `HTMLElement`، یک `Element`، یک `Node` (و یک `EventTarget`) هم هست و همه‌ی propertyها و متدهای آن‌ها را دارد: از `Node` چیزهایی مثل `parentNode` و `childNodes`، از `Element` چیزهایی مثل `id` و `children`، از `HTMLElement` چیزهایی مثل `style`، و از `EventTarget` متد `addEventListener`. به همین دلیل روی هر گره‌ای می‌توان listener گذاشت.

HTML

```
<p id="p">Hi<!-- note --></p>
<script>
  const p = document.getElementById("p");
  console.log(p instanceof HTMLParagraphElement, p instanceof Element); // true true
  console.log(p instanceof Node, p.firstChild instanceof Text);         // true true
  console.log(p.firstChild instanceof CharacterData);                   // true
  console.log(p.firstChild instanceof Element);                         // false
  console.log(p.lastChild instanceof Comment);                          // true
  console.log(document instanceof HTMLDocument);                        // true
</script>
```

- `Text` و `Comment` (گره‌ی توضیح `<!-- ... -->`) هر دو زیرکلاس `CharacterData` هستند، **نه** `Element`.
- `Attr` (کلاس attributeها) مستقیماً از `Node` ارث می‌برد، ولی در `childNodes` هیچ عنصری ظاهر نمی‌شود.
- `document` نمونه‌ای از `HTMLDocument` است که از `Document` و آن هم از `Node` ارث می‌برد.

پیمایش درخت با JavaScript (پیش‌زمینه‌ی لازم)

هر گره دو property شناسایی دارد: `nodeType` (یک عدد) و `nodeName` (یک رشته):

گره	`nodeType`	`nodeName`
عنصر (مثلاً `<p>`)	`1`	نام تگ با حروف بزرگ، مثلاً `"P"`
attribute	`2`	نام attribute، مثلاً `"href"`
متن	`3`	`"#text"`
comment	`8`	`"#comment"`
`document`	`9`	`"#document"`

برای حرکت در درخت این propertyها را داریم:

property	چه برمی‌گرداند	گره‌های متنی و comment را هم حساب می‌کند؟
`parentNode`	گره‌ی والد	—
`childNodes`	**همه‌ی** گره‌های فرزند	بله
`children`	فقط **عنصرهای** فرزند	خیر
`firstChild` / `lastChild`	اولین / آخرین گره‌ی فرزند	بله
`firstElementChild` / `lastElementChild`	اولین / آخرین عنصر فرزند	خیر
`nextSibling` / `previousSibling`	گره‌ی هم‌نیای بعدی / قبلی	بله
`nextElementSibling` / `previousElementSibling`	عنصر هم‌نیای بعدی / قبلی	خیر

مهم‌ترین دام اینجاست: **خط جدید و فاصله‌هایی که برای مرتب نوشتن HTML می‌گذاریم، گره‌ی متنی می‌سازند.**

HTML

```
<ul id="list">
  <li>A</li>
  <li>B</li>
</ul>
<script>
  const ul = document.getElementById("list");
  console.log(ul.childNodes.length, ul.children.length);              // 5 2
  console.log(ul.firstChild.nodeName, ul.firstElementChild.nodeName); // #text LI
  console.log(ul.parentNode.nodeName);                                // BODY
  console.log(ul.children[1].previousElementSibling.textContent);     // A
</script>
```

چرا ۵؟ چون فرزندان `ul` این‌ها هستند: متنِ فاصله، `li`، متنِ فاصله، `li`، متنِ فاصله. اگر همین لیست را در یک خط و بدون فاصله بنویسیم (`<ul><li>A</li><li>B</li></ul>`)، `childNodes.length` هم `2` می‌شود.

> **نکته**
>
> اگر فقط با تگ‌ها کار داری، از نسخه‌های «Element» استفاده کن: `children`، `firstElementChild`، `nextElementSibling`. نسخه‌های بدون Element (`childNodes`، `firstChild`، `nextSibling`) گره‌های متنی، حتی فاصله‌ها، و commentها را هم می‌شمارند.

⚠ دام تستی

- attributeها فرزند نیستند: برای `<a href="#">Link</a>` مقدار `a.childNodes.length` برابر `1` است (فقط گره‌ی متنی)، نه `2`.
- گره‌ی ریشه `document` است ولی عنصر ریشه `<html>` (`document.documentElement`)؛ خودش `document` عنصر نیست (`nodeType` آن `9` است، نه `1`).
- `nodeName` عنصرها با حروف بزرگ است (`"LI"`)، و `nodeName` گره‌ی متنی `"#text"` است، نه `"text"`.
- `Text` زیرکلاس `CharacterData` است، نه `Element`؛ پس `instanceof Element` برای گره‌ی متنی `false` است.

جمع‌بندی

- زبان نشانه‌گذاری (HTML، XML، Markdown) ساختار را با تگ یا نشانه توصیف می‌کند؛ سند یک درخت است.
- تگ ← گره‌ی عنصر، متن ← گره‌ی متنی، property ← attribute عنصر (فرزند نیست).
- DOM = Document Object Model: نمایش درختیِ صفحه در حافظه، پل بین HTML و JavaScript.
- ریشه `document` است؛ کلاس‌ها: Document/CharacterData/Element/Attr ← Node و ← Element ← HTMLParagraphElement ← HTMLElement و ...
- `children` فقط عنصرها، `childNodes` همه‌ی گره‌ها (از جمله فاصله‌ها).

تست‌های تمرینی

۲۱ تست · شماره‌ی ۷۰۸ تا ۷۲۸

۷۰۸ آسان

کدام گزینه یک زبان نشانه‌گذاری (markup language) نیست؟

۱) HTML　۲) XML　۳) Markdown　۴) JavaScript

۷۰۹ آسان

DOM مخفف چیست؟

۱) Data Object Model　۲) Document Object Model
۳) Document Oriented Markup　۴) Dynamic Object Method

۷۱۰ آسان

در درخت زیر (نمودار اسلاید ۷)، کدام گره‌ها برگ (leaf) هستند؟

```
        A
      /  |  \
     B   C   D
    / \
   E   F
   |   |
   G   H
```

۱) فقط `G` و `H`　۲) `E`، `F`، `G` و `H`　۳) `C`، `D`، `G` و `H`　۴) `A` و `B`

۷۱۱ متوسط

در همان درخت اسلاید ۷، کدام عبارت نادرست است؟

```
        A
      /  |  \
     B   C   D
    / \
   E   F
   |   |
   G   H
```

۱) `B`، `C` و `D` هم‌نیا (sibling) هستند.

۲) `A` جدِ (ancestor) `H` است.

۳) `E` فرزند `B` و والد `G` است.

۴) `F` و `G` هم‌نیا هستند.

۷۱۲ آسان

طبق اسلاید ۸، وقتی مرورگر HTML را به DOM تبدیل می‌کند، attributeهای یک تگ (مثل `href` در `<a>`) به چه چیزی تبدیل می‌شوند؟

۱) به یک گره‌ی متنی (text node) که فرزند همان عنصر است

۲) به propertyهای همان گره‌ی عنصر

۳) به یک گره‌ی عنصر جداگانه که هم‌نیای همان عنصر است

۴) دور ریخته می‌شوند و در DOM باقی نمی‌مانند

۷۱۳ متوسط

خروجی کد زیر در مرورگر کدام است؟

HTML

```html
<a id="home" href="/">Home page</a>
```

JavaScript

```javascript
const a = document.getElementById("home");
console.log(a.childNodes.length, a.firstChild.nodeName, a.attributes.length);
```

۱) `1 #text 2`

۲) `2 #text 2`

۳) `3 A 2`

۴) `1 HREF 1`

۷۱۴ متوسط

خروجی کد زیر کدام است؟

HTML

```html
<p id="p">Hello <b>big</b> world</p>
```

JavaScript

```javascript
const p = document.getElementById("p");
console.log(p.childNodes.length, p.children.length);
```

۱) `1 1`

۲) `2 1`

۳) `3 3`

۴) `3 1`

۷۱۵ متوسط

خروجی کد زیر کدام است؟

HTML

```html
<ul id="list">
  <li>A</li>
  <li>B</li>
</ul>
```

JavaScript

```javascript
const ul = document.getElementById("list");
console.log(ul.childNodes.length, ul.children.length);
```

۱) `2 2`
۲) `5 2`
۳) `3 2`
۴) `5 5`

۷۱۶ سخت

خروجی کد زیر کدام است؟

HTML

```html
<ul id="list">
  <li>A</li>
  <li>B</li>
</ul>
```

JavaScript

```javascript
const ul = document.getElementById("list");
console.log(
  ul.firstChild.nodeName,
  ul.firstElementChild.nodeName,
  ul.lastElementChild.textContent
);
```

۱) `LI LI B`
۲) `#text LI A`
۳) `#text LI B`
۴) `LI #text B`

۷۱۷ متوسط

خروجی کد زیر کدام است؟

HTML

```html
<p id="p">Hi</p>
```

JavaScript

```javascript
const p = document.getElementById("p");
console.log(document.nodeType, document.body.nodeType, p.firstChild.nodeType);
```

۱) `9 1 3`
۲) `1 1 3`
۳) `0 1 2`
۴) `9 1 1`

۷۱۸ متوسط

خروجی کد زیر کدام است؟

HTML

```
<p id="p">Hi</p>
```

JavaScript

```
const p = document.getElementById("p");
console.log(
  document.nodeName,
  document.documentElement.nodeName,
  p.firstChild.nodeName
);
```

۱) `document html text`
۲) `#document BODY #text`
۳) `DOCUMENT HTML TEXT`
۴) `#document HTML #text`

۷۱۹ آسان

طبق اسلاید ۱۱، ریشه‌ی درخت DOM (root node) کدام است؟

۱) `window`
۲) عنصر `<html>`
۳) `document`
۴) عنصر `<body>`

۷۲۰ سخت

خروجی کد زیر کدام است؟

HTML

```
<!DOCTYPE html>
<html>
<head><title>Root</title></head>
<body><p>Hi</p></body>
</html>
```

JavaScript

```
const root = document.documentElement;
console.log(root.tagName, root.parentNode === document, document.childNodes.length);
```

۱) `HTML true 1`
۲) `HTML true 2`
۳) `HTML false 2`
۴) `BODY true 2`

۷۲۱ سخت

فعالیت کلاسی اسلاید ۱۰ را با یک اصلاح (بستن `<title>` با `</title>`) در مرورگر باز می‌کنیم و کد زیر در انتهای `<body>` اجرا می‌شود. خروجی کدام است؟

HTML

```
<html>
    <head>
        <title>sample page here :)</title>
        <link rel="stylesheet" href="./style.css">
    </head>
    <body>
        <h1>SAMPLE PAGE!</h1>
        <div class="sampleclass">
            <p>sample paragraph in sample class in sample page!</p>
            <h3 id="2223">wait wha-?</h3>
        </div>
    </body>
</html>
```

JavaScript

```
const div = document.querySelector(".sampleclass");
const h3 = document.getElementById("2223");
console.log(div.children.length, div.parentNode.tagName);
console.log(h3.previousElementSibling.tagName, document.head.children.length);
```

۱) `2 BODY` `P 2`

۲) `2 HTML` `P 2`

۳) `3 BODY` `H1 2`

۴) `2 BODY` `P 1`

۷۲۲ سخت

HTML اسلاید ۱۰ را **دقیقاً همان‌طور که روی اسلاید است** (با `</sample>` به‌جای `</title>`) در مرورگر باز می‌کنیم:

```HTML
<html>
    <head>
        <title>sample page here :)</sample>
        <link rel="stylesheet" href="./style.css">
    </head>
    <body>
        <h1>SAMPLE PAGE!</h1>
        <div class="sampleclass">
            <p>sample paragraph in sample class in sample page!</p>
            <h3 id="2223">wait wha-?</h3>
        </div>
    </body>
</html>
```

سپس در Console این دستورها را اجرا می‌کنیم. خروجی کدام است؟

```JavaScript
console.log(document.body.children.length);
console.log(document.getElementById("2223") === null);
console.log(document.title.includes("SAMPLE PAGE!"));
```

۱) `2` `false` `false`

۲) `0` `true` `true`

۳) `Uncaught SyntaxError`

۴) `2` `false` `true`

۷۲۳ متوسط

طبق نمودار سلسله‌مراتب کلاس‌ها در اسلاید ۱۱، کدام رابطه‌ی ارث‌بری نادرست است؟

۱) `HTMLParagraphElement` از `HTMLElement`، آن از `Element` و آن از `Node` ارث می‌برد.

۲) `Text` و `Comment` هر دو زیرکلاس `CharacterData` هستند.

۳) `Text` زیرکلاس `Element` است، چون متن هم داخل تگ‌ها قرار دارد.

۴) `Attr` (کلاس attributeها) مستقیماً از `Node` ارث می‌برد.

۷۲۴ سخت

خروجی کد زیر کدام است؟

```HTML
<p id="p">Hi</p>
```

```JavaScript
const p = document.getElementById("p");
console.log(
  p instanceof HTMLElement,
  p.firstChild instanceof Element,
  p.firstChild instanceof CharacterData,
  document instanceof Element
);
```

۱) `true true true false`
۲) `true false false true`
۳) `false false true false`
۴) `true false true false`

۷۲۵ متوسط

کدام عبارت درباره‌ی DOM درست است؟

۱) DOM نمایش درختیِ صفحه در حافظه‌ی مرورگر است که JavaScript از طریق آن می‌تواند عنصرها را بخواند، تغییر دهد، اضافه یا حذف کند.

۲) DOM یک زبان برنامه‌نویسی جدا از JavaScript است که برای ساخت صفحه به کار می‌رود.

۳) DOM همان فایل `.html` روی دیسک است و تغییر DOM با JavaScript، فایل را هم تغییر می‌دهد.

۴) DOM فقط برای خواندن صفحه است و نمی‌توان با آن عنصر جدیدی ساخت یا حذف کرد.

۷۲۶ آسان

در نمودار W3Schools (اسلاید ۸)، والدِ گره‌ی متنی `"My title"` کدام است؟

۱) عنصر `<title>`
۲) عنصر `<head>`
۳) عنصر `<html>`
۴) `Document`

۷۲۷ سخت

خروجی کد زیر کدام است؟

```HTML
<div id="box"><!-- note --><span>A</span>text</div>
```

```JavaScript
const box = document.getElementById("box");
console.log(box.childNodes.length, box.children.length, box.firstChild.nodeType);
```

۱) `2 1 8`
۲) `3 1 3`
۳) `3 1 8`
۴) `1 1 1`

۷۲۸ متوسط

خروجی کد زیر کدام است؟

HTML

```
<h1 id="head">A heading</h1><a id="link" href="#">Link text</a>
```

JavaScript

```
const h1 = document.getElementById("head");
const a = document.getElementById("link");
console.log(
  a.parentNode.tagName,
  a.previousElementSibling.id,
  h1.nextElementSibling.getAttribute("href")
);
```

۱) `BODY head #`

۲) `H1 head #`

۳) `BODY link #`

۴) `BODY head null`

۲-۳ سلسله‌مراتب DOM در JavaScript و شیء document

منبع: JS in HTML، اسلایدهای ۱۲ تا ۱۵

درسنامه

در درس قبل دیدیم مرورگر از HTML یک درخت می‌سازد. حالا ببینیم این درخت در JavaScript از کجا شروع می‌شود و چطور به آن دست پیدا می‌کنیم.

یک تشبیه ساده

مرورگر را یک **قاب عکس** فرض کن (`window`) که داخلش یک **نقاشی** قرار دارد (`document`)؛ هر جزء نقاشی (درخت، خانه، آسمان) یک **عنصر** است. قاب چیزهای دیگری هم دارد که به خودِ نقاشی ربطی ندارند: مثلاً برچسب آدرسِ پشت قاب (`location`) یا فهرست نقاشی‌هایی که قبلاً در این قاب بوده‌اند (`history`).

سلسله‌مراتب DOM در JavaScript (اسلاید ۱۲)

سطح	توضیح اسلاید ۱۲	در کد
`window`	بالاترین شیء مرورگر؛ همان **global context** (بافت سراسری) که در فصل اول دیدیم	`window`
`document`	صفحه‌ی وبِ داخل پنجره	`window.document` یا فقط `document`
عنصرها	تگ‌ها (`<div>`، `<p>`، `<h1>`) به‌صورت شیء در درخت DOM	`document.getElementById("title")`
attributeها و متن	propertyهای گره‌های عنصر	`h1.id`، `h1.className`، `h1.textContent`
رویدادها (events)	متدها و listenerهایی که JavaScript را به کارهای کاربر وصل می‌کنند	`btn.addEventListener("click", ...)`

`window` شیء سراسری (global object) مرورگر است؛ پس هر چیزی را که روی `window` باشد می‌توان بدون نوشتن `window.` هم استفاده کرد: `document` همان `window.document` است و `alert` همان `window.alert`. (جزئیات `window` را در درس «شیء window: پنجره‌ی مرورگر» می‌بینیم.)

HTML

```
<h1 id="title" class="big">Hello</h1>
<script>
  console.log(window.document === document);                    // true
  const h1 = document.querySelector("h1");
  console.log(h1.id, h1.className, h1.textContent, h1.tagName); // title big Hello H1
</script>
```

نمودار اسلاید ۱۲

نمودار اسلاید ۱۲ (از منبع CIS) را اینجا به شکل متنی می‌بینی. روی اسلاید، فرزندان `document` در دو ستون دو طرف یک خط عمودی کشیده شده‌اند (anchors و applets، links و embeds، plugins و frames، scripts و images، stylesheets و forms) و `forms` خودش به `elements` وصل است:

```
window
|-- document
|   |-- anchors
|   |-- links
|   |-- applets
|   |-- plugins
|   |-- embeds
|   |-- scripts
|   |-- frames
|   |-- stylesheets
|   |-- images
|   `-- forms
|       `-- elements
|-- event
|-- frames
|-- location
|-- history
|-- navigator
|   |-- plugins
|   `-- mimeTypes
`-- screen
```

فرزند `window` در نمودار	چیست؟
`document`	خودِ صفحه، یعنی درخت DOM
`event`	رویدادی که همین حالا در حال پردازش است (قدیمی؛ امروز از پارامتر `event` در listener استفاده می‌کنیم)
`frames`	قاب‌های داخلی صفحه (`<iframe>` ها)
`location`	آدرس (URL) صفحه‌ی فعلی
`history`	تاریخچه‌ی صفحه‌های بازشده در این زبانه (برای back و forward)
`navigator`	اطلاعات خودِ مرورگر؛ در نمودار دو فرزند دارد: `plugins` (افزونه‌ها) و `mimeTypes` (نوع فایل‌هایی که مرورگر می‌شناسد)
`screen`	اطلاعات صفحه‌نمایش (مانیتور) کاربر

فرزندان `document` در نمودار، **مجموعه‌هایی از عنصرهای خاص صفحه** هستند: `images` همه‌ی تصاویر، `links` لینک‌ها، `forms` فرم‌ها (و `elements` هر فرم، کادرهای ورودیِ آن فرم)، `scripts` تگ‌های `<script>`، `embeds` و `plugins` محتوای جاسازی‌شده، `anchors` لنگرها، و `stylesheets` شیوه‌نامه‌ها (فایل‌های CSS).

> **بیشتر بدانیم: این نمودار قدیمی است**
>
> این نمودار مربوط به «مدل کلاسیک» مرورگرهاست و پیام اصلی‌اش این است: `window` در بالاترین سطح است، `document` یکی از فرزندان آن است، و `window` فرزندان دیگری هم دارد. جزئیات نام‌ها را حفظ نکن: appletها سال‌هاست از مرورگرها حذف شده‌اند، نام property واقعی در مرورگرهای امروزی `document.styleSheets` (با S بزرگ) است، `frames` در عمل property خودِ `window` است و `window.event` هم منسوخ (deprecated) شده است.

شیء `document` (اسلاید ۱۳)

طبق اسلاید ۱۳، شیء `document` :

- نماینده‌ی **کل صفحه‌ی وب** در مرورگر است؛
- **نقطه‌ی ورود (entry point)** به درخت DOM است، یعنی تقریباً هر کاری با صفحه با `document.` شروع می‌شود؛
- متدهایی برای **پیدا کردن عنصرها** دارد، مثل `getElementById` و `querySelector` (درس بعد)؛
- به JavaScript اجازه می‌دهد گره‌ها را **بسازد، حذف کند یا به‌روزرسانی کند**.

چند property پرکاربرد آن:

property	چه چیزی را برمی‌گرداند
`document.documentElement`	عنصر `<html>` (عنصر ریشه)
`document.head`	عنصر `<head>`
`document.body`	عنصر `<body>`
`document.title`	متن عنوان صفحه (همان متنِ زبانه‌ی مرورگر)؛ قابل خواندن و تغییر

مثال اسلاید ۱۳ (فایل `doc.html`):

doc.html

```
<!DOCTYPE html>
<html>
  <body>
    <h1 id="title">Hello World</h1>
    <button id="btn">Change Text</button>

    <script>
      // Access an element
      const heading = document.getElementById("title");

      // Add event to button
      document.getElementById("btn").addEventListener("click", () => {
        heading.textContent = "Text Changed via document!";
      });
    </script>
  </body>
</html>
```

خط به خط:

1. `document.getElementById("title")` شیء عنصر `<h1>` را پیدا می‌کند و در `heading` نگه می‌دارد.
2. `document.getElementById("btn")` دکمه را پیدا می‌کند و `addEventListener("click", ...)` تابعی (اینجا یک arrow function) ثبت می‌کند که با هر کلیک اجرا شود.
3. با کلیک، `heading.textContent = "..."` متن `<h1>` را عوض می‌کند و صفحه فوراً به‌روز می‌شود.
4. کلیک دوم همان متن را دوباره می‌نویسد، پس تغییر دیگری دیده نمی‌شود.

`<script>` در **انتهای** `<body>` و بعد از عنصرهایی که لازم دارد آمده است؛ برای همین `getElementById` آن‌ها را پیدا می‌کند (پایین‌تر علتش را می‌بینیم). در ضمن تابعِ داخل listener متغیر `heading` را از بیرون به خاطر سپرده است (همان closure فصل

اول)، پس عنصر فقط یک بار جست‌وجو می‌شود.

پیش‌بینی خروجی Console (اسلاید ۱۴ و ۱۵)

اسلاید ۱۴ کد زیر را اجرا می‌کند؛ کدی که JavaScript را داخل HTML جاسازی کرده است، و می‌پرسد: «می‌توانی حدس بزنی چه چیزی در Console چاپ می‌شود؟»

doc.html

```
<!DOCTYPE html>
<html>
  <body>
    <h1 id="title">Hello World</h1>

    <script>
      const heading = document.getElementById("title");
      console.log(heading)
      </script>
  </body>
</html>
```

پاسخ اسلاید ۱۵: **خودِ عنصر** چاپ می‌شود، که منطقی است چون خودِ عنصر را به `console.log` داده‌ایم. در Firefox، زبانه‌ی Console سطری مثل `▶ <h1 id="title">` نشان می‌دهد و سمت راستش لینک `doc.html:8:15` آمده؛ یعنی «فایل `doc.html`، خط ۸، ستون ۱۵»، همان جایی که `console.log` نوشته شده است. با زدن مثلث `▶` فرزندان عنصر باز می‌شوند و با بردن ماوس روی آن، عنصر در صفحه هایلایت می‌شود.

- چیزی که چاپ شده **شیء گره** است، نه رشته‌ی `"Hello World"`. برای متن باید `heading.textContent` و برای نام تگ `heading.tagName` را چاپ کنیم.
- `null` هم چاپ نمی‌شود، چون اسکریپت بعد از `<h1>` آمده و عنصر از قبل در درخت وجود دارد.
- نبودن `;` بعد از `console.log(heading)` مشکلی ایجاد نمی‌کند؛ JavaScript خودش سمی‌کالن را درج می‌کند (automatic semicolon insertion). فاصله‌ی اضافه‌ی قبل از `</script>` هم فقط ظاهری است.

اسلاید ۱۵ با این سؤال تمام می‌شود: «`document` چه کارهای جالب دیگری می‌تواند بکند؟!»؛ جواب، باقی همین فصل است: پیدا کردن عنصرها، تغییر متن و استایل، رویدادها و ساختن و حذف عنصرها.

چرا جای تگ `<script>` مهم است؟ (پیش‌زمینه‌ی لازم)

مرورگر HTML را **از بالا به پایین** می‌خواند و درخت را کم‌کم می‌سازد. هر جا به یک `<script>` معمولی برسد، ساختن درخت را متوقف می‌کند، اسکریپت را اجرا می‌کند و بعد ادامه می‌دهد. پس اسکریپت فقط عنصرهایی را می‌بیند که **بالای آن** هستند:

```HTML
<!DOCTYPE html>
<html>
<head>
  <script>
    const heading = document.getElementById("title");
    console.log(heading);            // null  (the <h1> is not parsed yet)
    heading.textContent = "Changed"; // TypeError: Cannot set properties of null
  </script>
</head>
<body>
  <h1 id="title">Hello World</h1>
</body>
</html>
```

`getElementById` وقتی عنصری پیدا نکند `null` برمی‌گرداند، و هر تلاشی برای خواندن یا نوشتن property روی `null` خطای `TypeError` می‌دهد. سه راه‌حل:

راه‌حل	توضیح
`<script>` در انتهای `<body>`	روشی که اسلایدها استفاده می‌کنند؛ وقتی اسکریپت اجرا می‌شود همه‌ی عنصرهای بالایی ساخته شده‌اند
رویداد `DOMContentLoaded`	`document.addEventListener("DOMContentLoaded", () => { ... })`؛ کد وقتی اجرا می‌شود که کل HTML خوانده شده و درخت DOM آماده است
ویژگی `defer`	`<script src="app.js" defer></script>`؛ فایل خارجی هم‌زمان دانلود می‌شود ولی **بعد از** پایان parse اجرا می‌شود. `defer` فقط روی اسکریپت‌های **خارجی (با** `src` **)** اثر دارد

رویداد `load` روی `window` از این هم دیرتر است: منتظر می‌ماند تا **همه‌ی منابع** (تصاویر، فایل‌های CSS و ...) هم بارگیری شوند. ترتیب واقعی در این مثال:

```HTML
<!DOCTYPE html>
<html>
<head>
  <script>
    console.log("head", document.getElementById("title"));
    document.addEventListener("DOMContentLoaded", () => {
      console.log("DOMContentLoaded", document.getElementById("title").textContent);
    });
    window.addEventListener("load", () => console.log("load"));
  </script>
</head>
<body>
  <h1 id="title">Hello World</h1>
  <script>
    console.log("end of body", document.getElementById("title").tagName);
  </script>
</body>
</html>
```

```
head null
end of body H1
DOMContentLoaded Hello World
load
```

نکته

`console.log(element)` در مرورگر خودِ گره را (به‌صورت قابل باز شدن) نشان می‌دهد. اگر متن را می‌خواهی `element.textContent` و اگر نوع عنصر را می‌خواهی `element.tagName` را چاپ کن. تبدیل عنصر به رشته (`String(element)`) فقط چیزی مثل `"[object HTMLHeadingElement]"` می‌دهد.

دام تستی

- اسکریپتِ داخل `<head>` (بدون `defer` یا `DOMContentLoaded`) عنصرهای `<body>` را نمی‌بیند: `getElementById` مقدار `null` می‌دهد و استفاده از آن `TypeError` می‌دهد.
- گذاشتن `defer` روی یک اسکریپت **درون‌خطی** (بدون `src`) هیچ اثری ندارد.
- رابطه برعکس نیست: `document` یکی از propertyهای `window` است، نه اینکه `window` داخل `document` باشد.
- اگر عنصری را **بیرون** از `DOMContentLoaded` (زودتر از موعد) جست‌وجو کنی و فقط listener را داخل آن بگذاری، باز هم `null` در دست مانده است.

جمع‌بندی

- `window` (بالاترین شیء و global context) ← `document` (صفحه) ← عنصرها (اشیاء درخت)؛ attributeها و متن، propertyهای عنصرها هستند و رویدادها JavaScript را به کاربر وصل می‌کنند.
- `document` نقطه‌ی ورود به DOM است: پیدا کردن، ساختن، حذف و به‌روزرسانی گره‌ها.
- `console.log(heading)` خودِ عنصر را چاپ می‌کند، نه متنش را.
- اسکریپت فقط عنصرهای بالای خودش را می‌بیند؛ راه‌حل‌ها: انتهای `<body>` ، `DOMContentLoaded` یا `defer` (فقط برای فایل خارجی). ترتیب: `DOMContentLoaded` قبل از `load` .

تست‌های تمرینی

۱۸ تست · شماره‌ی ۷۲۹ تا ۷۴۶

۷۲۹ آسان

طبق اسلاید ۱۲، بالاترین شیء (top-level object) مرورگر که همان global context است کدام است؟

۱) `document`　۲) `window`　۳) عنصر `<html>`　۴) عنصر `<body>`

۷۳۰ آسان

طبق اسلاید ۱۳، شیء `document` نماینده‌ی چیست؟

۱) پنجره‌ی مرورگر، همراه با تاریخچه و آدرس صفحه

۲) فقط محتوای تگ `<body>`

۳) کل صفحه‌ی وبِ بارگذاری‌شده در پنجره؛ نقطه‌ی ورود به درخت DOM

۴) فایل HTML روی دیسک، که با تغییر DOM به‌روز می‌شود

۷۳۱ متوسط

در نمودار اسلاید ۱۲، کدام مورد فرزند مستقیم `window` نیست؟

۱) `location` ۲) `history` ۳) `navigator` ۴) `forms`

۷۳۲ متوسط

در نمودار اسلاید ۱۲، `mimeTypes` زیرمجموعه‌ی کدام شیء است؟

۱) `navigator` ۲) `document` ۳) `screen` ۴) `location`

۷۳۳ آسان

خروجی کد زیر در مرورگر کدام است؟

HTML

```html
<h1>Hi</h1>
```

JavaScript

```javascript
console.log(
  window.document === document,
  document.documentElement.tagName,
  document.body.tagName,
  document.head.tagName
);
```

۱) `false HTML BODY HEAD` ۲) `true BODY BODY HEAD`

۳) `true HTML BODY HEAD` ۴) `true DOCUMENT BODY HEAD`

۷۳۴ آسان

صفحه‌ی زیر همان مثال اسلاید ۱۳ است. کاربر **دو بار** روی دکمه کلیک می‌کند و سپس در Console دستور `console.log(heading.textContent)` را اجرا می‌کنیم. خروجی کدام است؟

HTML

```
<h1 id="title">Hello World</h1>
<button id="btn">Change Text</button>
```

JavaScript

```
// Access an element
const heading = document.getElementById("title");

// Add event to button
document.getElementById("btn").addEventListener("click", () => {
  heading.textContent = "Text Changed via document!";
});
```

۱) `Hello World`

۲) `Text Changed via document!`

۳) `Text Changed via document!Text Changed via document!`

۴) `Uncaught TypeError`

۷۳۵ متوسط

این کد اسلاید ۱۴ است. با باز کردن صفحه در Firefox، چه چیزی در Console نمایش داده می‌شود؟

HTML

```
<!DOCTYPE html>
<html>
  <body>
    <h1 id="title">Hello World</h1>

    <script>
      const heading = document.getElementById("title");
      console.log(heading)
      </script>
  </body>
</html>
```

۱) خودِ عنصر `<h1 id="title">`، به‌صورت یک گره‌ی قابل باز شدن

۲) رشته‌ی `"Hello World"`

۳) `null`، چون `heading` هنوز ساخته نشده است

۴) یک خطای نحوی (SyntaxError)، چون بعد از `console.log(heading)` سمی‌کالن نیامده است

۷۳۶ سخت

خروجی Console برای صفحه‌ی زیر کدام است؟

HTML

```
<!DOCTYPE html>
<html>
<head>
  <script>
    const heading = document.getElementById("title");
    console.log(heading);
    heading.textContent = "Changed";
    console.log("done");
  </script>
</head>
<body>
  <h1 id="title">Hello World</h1>
</body>
</html>
```

۱) `[object HTMLHeadingElement]` `done`

۲) `undefined` `done`

۳) `null` `Uncaught TypeError`

۴) `Uncaught ReferenceError`

۷۳۷ سخت

ترتیب خروجی‌های Console برای صفحه‌ی زیر کدام است؟

HTML

```
<!DOCTYPE html>
<html>
<head>
  <script>
    console.log("head", document.getElementById("title"));
    document.addEventListener("DOMContentLoaded", () => {
      console.log("DOMContentLoaded", document.getElementById("title").textContent);
    });
    window.addEventListener("load", () => console.log("load"));
  </script>
</head>
<body>
  <h1 id="title">Hello World</h1>
  <script>
    console.log("end of body", document.getElementById("title").tagName);
  </script>
</body>
</html>
```

۱) `head null` `DOMContentLoaded Hello World` `end of body H1` `load`

۲) `head null` `end of body H1` `DOMContentLoaded Hello World` `load`

۳) `head H1` `end of body H1` `DOMContentLoaded Hello World` `load`

۴) `head null` `end of body H1` `load` `DOMContentLoaded Hello World`

۷۳۸ متوسط

یک اسکریپت درون‌خطی (بدون `src`) داخل `<head>` قرار دارد و با `getElementById` دنبال عنصری در `<body>` می‌گردد، ولی `null` می‌گیرد. کدام تغییر به‌تنهایی مشکل را حل می‌کند؟

۱) اضافه کردن ویژگی `defer` به همان تگ `<script>` درون‌خطی

۲) تبدیل `const` به `var` در تعریف متغیر

۳) قرار دادن کد داخل `document.addEventListener("DOMContentLoaded", () => { ... })`

۴) نوشتن `getElementById("#title")` به‌جای `getElementById("title")`

۷۳۹ متوسط

خروجی کد زیر کدام است؟

HTML

```
<h1 id="title">Hello World</h1>
```

JavaScript

```
const heading = document.getElementById("title");
console.log(String(heading), typeof heading, heading.textContent);
```

۱) `Hello World string Hello World`

۲) `<h1 id="title">Hello World</h1> string Hello World`

۳) `[object HTMLHeadingElement] object Hello World`

۴) `[object HTMLHeadingElement] element Hello World`

۷۴۰ آسان

خروجی کد زیر کدام است؟

HTML

```
<h1 id="title" class="big">Hello</h1>
```

JavaScript

```
const h = document.getElementById("title");
console.log(h.id, h.className, h.textContent, h.tagName);
```

۱) `#title .big Hello h1`

۲) `title undefined Hello H1`

۳) `title big <h1 id="title" class="big">Hello</h1> H1`

۴) `title big Hello H1`

۷۴۱ متوسط

خروجی کد زیر کدام است؟

HTML

```
<!DOCTYPE html>
<html>
<head><title>Shop</title></head>
<body><h1>Home</h1></body>
</html>
```

JavaScript

```
console.log(document.title);
document.title = "Cart (2)";
console.log(document.title, document.querySelector("title").textContent);
```

۱) `Shop` `Cart (2) Cart (2)`

۲) `Home` `Cart (2) Shop`

۳) `Shop` `Cart (2) Shop`

۴) `undefined` `Cart (2) Cart (2)`

۷۴۲ آسان

کدام عبارت درباره‌ی شیء `document` نادرست است؟

۱) نقطه‌ی ورود (entry point) برای دسترسی به درخت DOM است.

۲) متدهایی مثل `getElementById` و `querySelector` برای پیدا کردن عنصرها دارد.

۳) با آن می‌توان گره‌ها را ساخت، حذف یا به‌روزرسانی کرد.

۴) بالاترین شیء مرورگر است و `window` یکی از propertyهای آن است.

۷۴۳ سخت

خروجی Console برای صفحه‌ی زیر کدام است؟

HTML

```
<h1 id="a">A</h1>
<script>
  const a = document.getElementById("a");
  const b = document.getElementById("b");
  console.log(a !== null, b !== null);
</script>
<h2 id="b">B</h2>
```

JavaScript

```
console.log(document.getElementById("b").textContent);
```

۱) `true false` `B`

۲) `true true` `B`

۳) `false false` `B`

۴) `true false` `Uncaught TypeError`

۷۴۴ متوسط

کدام عبارت درباره‌ی رویدادهای `DOMContentLoaded` و `load` درست است؟

۱) `load` زودتر از `DOMContentLoaded` رخ می‌دهد، چون فقط منتظر HTML است.

۲) `DOMContentLoaded` وقتی رخ می‌دهد که HTML کامل parse شده و درخت DOM آماده است؛ `load` علاوه بر آن منتظر بارگیری منابعی مثل تصاویر و stylesheetها هم می‌ماند.

۳) هر دو دقیقاً هم‌زمان رخ می‌دهند و فقط نامشان فرق دارد.

۴) `DOMContentLoaded` قبل از اجرای هر اسکریپتی در صفحه رخ می‌دهد.

۷۴۵ سخت

کاربر بعد از بارگذاری صفحه‌ی زیر روی دکمه‌ی Change Text کلیک می‌کند. خروجی Console کدام است؟

HTML

```
<!DOCTYPE html>
<html>
<head>
  <script>
    const heading = document.getElementById("title");
    document.addEventListener("DOMContentLoaded", () => {
      document.getElementById("btn").addEventListener("click", () => {
        heading.textContent = "Changed";
        console.log("changed");
      });
    });
  </script>
</head>
<body>
  <h1 id="title">Hello World</h1>
  <button id="btn">Change Text</button>
</body>
</html>
```

۱) `changed`

۲) (بدون خروجی)

۳) `null` `changed`

۴) `Uncaught TypeError`

۷۴۶ متوسط

کدام تگ، یک فایل JavaScript **خارجی** را در `<head>` طوری بارگذاری می‌کند که اجرای آن تا پایان parse شدن کل HTML عقب بیفتد؟

۱) `<script defer>console.log("hi")</script>`

۲) `<script src="app.js"></script>`

۳) `<script src="app.js" delay></script>`

۴) `<script src="app.js" defer></script>`

۲-۴ پیدا کردن عناصر (Querying the DOM)

منبع: JS in HTML، اسلایدهای ۱۶ و ۱۷

درسنامه

قبل از اینکه بتوانیم متن یک عنصر را عوض کنیم یا به آن listener بدهیم، باید آن را **پیدا کنیم**. اسلاید ۱۶ می‌گوید: «با JavaScript عنصرهای صفحه را پیدا کن» و پنج متد اصلی را معرفی می‌کند. به این کار **querying** (پرس‌وجو، یعنی جست‌وجوی عنصر در درخت DOM) می‌گویند.

یک تشبیه ساده

فرض کن در یک مدرسه دنبال دانش‌آموزها می‌گردی:

- با **شماره‌ی دانش‌آموزی** که برای هر نفر یکتاست، دقیقاً یک نفر پیدا می‌کنی: `getElementById`.
- با **نام کلاس** («همه‌ی دانش‌آموزان کلاس دهم ب») یک فهرست می‌گیری: `getElementsByClassName`.
- با **نقش** («همه‌ی معلم‌ها») هم یک فهرست می‌گیری: `getElementsByTagName`.
- با یک **توصیف دقیق و انعطاف‌پذیر** («اولین دانش‌آموزِ کلاس دهم ب که عینک دارد»): `querySelector` و `querySelectorAll` با CSS selector.

پنج متد اسلاید ۱۶ در یک نگاه

متد	ورودی	خروجی	اگر چیزی پیدا نشود
`getElementById("id")`	نام id، **بدون** `#`	**یک** عنصر	`null`
`getElementsByClassName("class")`	نام کلاس، **بدون** `.`	**همه**: یک HTMLCollection زنده	مجموعه‌ی خالی (`length` برابر `0`)
`getElementsByTagName("tag")`	نام تگ، مثل `"p"`	**همه**: یک HTMLCollection زنده	مجموعه‌ی خالی
`querySelector("selector")`	یک **CSS selector**	**اولین** عنصر منطبق	`null`
`querySelectorAll("selector")`	یک **CSS selector**	**همه**: یک NodeList ایستا	NodeList خالی

به نام‌ها دقت کن: `getElementById` مفرد است (Element، چون id یکتاست) ولی `getElementsByClassName` و `getElementsByTagName` جمع‌اند (Elements). همه‌ی نام‌ها camelCase هستند؛ مثلاً `getElementByID` (با D بزرگ) وجود ندارد و صدا زدنش `TypeError` می‌دهد.

۱) `getElementById`

- ورودی، **خودِ نام id** است، بدون `#`.
- خروجی یک عنصر است، یا `null` اگر عنصری با آن id نباشد.
- id باید در صفحه **یکتا** باشد؛ اگر به‌اشتباه تکراری باشد، این متد اولین عنصر را برمی‌گرداند.
- این متد فقط روی `document` وجود دارد، نه روی عنصرها.

```JavaScript
const title = document.getElementById("title"); // <h1 id="title">
console.log(document.getElementById("#title")); // null   (no # here!)
console.log(document.getElementById("nope"));   // null
```

۲) `getElementsByClassName` و `getElementsByTagName`

این دو متد یک **HTMLCollection** برمی‌گردانند: مجموعه‌ای **شبه‌آرایه (array-like)** که با اندیس (`[0]`) یا `.item(1)` به اعضایش می‌رسی و `.length` دارد، ولی **آرایه نیست** و متدهایی مثل `forEach` و `map` را ندارد.

```HTML
<button class="btn primary">A</button>
<button class="btn">B</button>
<script>
  const buttons = document.getElementsByClassName("btn");
  console.log(buttons.length);                                  // 2
  console.log(buttons[0].textContent, buttons.item(1).textContent);   // A B
  console.log(document.getElementsByClassName("btn primary").length); // 1
  console.log(Array.from(buttons).map((b) => b.textContent));         // ["A", "B"]
  for (const b of buttons) console.log(b.textContent);                // A, then B
  buttons.forEach((b) => console.log(b)); // TypeError: not a function
</script>
```

- نام کلاس را **بدون نقطه** می‌نویسیم. عنصری با `class="btn primary"` هم کلاس `btn` را دارد و پیدا می‌شود.
- اگر چند کلاس با فاصله بدهی (`"btn primary"`)، فقط عنصرهایی پیدا می‌شوند که **همه‌ی** آن کلاس‌ها را داشته باشند؛ ترتیب کلاس‌ها مهم نیست.
- این متدها هیچ‌وقت `null` برنمی‌گردانند؛ اگر چیزی پیدا نشود، مجموعه‌ای خالی با `length` برابر `0` می‌دهند.
- برای استفاده از متدهای آرایه، مجموعه را با `Array.from(...)` یا `[...collection]` به آرایه تبدیل کن؛ حلقه‌ی `for...of` هم مستقیم روی HTMLCollection کار می‌کند.
- `getElementsByTagName("*")` همه‌ی عنصرها را برمی‌گرداند.

۳) `querySelector` و `querySelectorAll` با CSS selector

این دو متد یک **CSS selector** (همان الگوهایی که در CSS برای انتخاب عنصر می‌نویسی) می‌گیرند و برای همین از همه انعطاف‌پذیرترند:

selector	معنی	مثال
`#id`	عنصری با این id	`#title`
`.class`	عنصرهایی با این کلاس	`.btn`
`tag`	عنصرهایی با این تگ	`button`
`tag.class`	تگ مشخص با کلاس مشخص	`a.nav-link`
`.a.b` (بدون فاصله)	عنصری که **هر دو** کلاس را دارد	`.btn.primary`
`[attr]`	عنصرهایی که این attribute را دارند	`[href]`
`[attr='value']`	attribute با مقدار مشخص	`input[type='text']`
`A B` (با فاصله)	Bهایی که **هر جایی داخل** A هستند (نواده)	`div p`
`A > B`	Bهایی که **فرزند مستقیم** A هستند	`div > p`
`A, B`	گروه: هر عنصری که A **یا** B باشد	`h1, .note`

- `querySelector` **اولین** عنصر منطبق را به ترتیب سند (از بالا به پایین) برمی‌گرداند، یا `null`.
- `querySelectorAll` **همه‌ی** عنصرهای منطبق را در یک **NodeList** برمی‌گرداند. NodeList هم شبه‌آرایه است، ولی برخلاف HTMLCollection متد `forEach` **دارد** (هنوز `map` و `filter` ندارد).
- نتیجه‌ها همیشه به **ترتیب سند** هستند، نه به ترتیب نوشتن selectorها در یک گروه.

HTML

```
<h1 id="title">Shop</h1>
<div id="menu">
  <a class="nav-link" href="#home">Home</a>
  <a class="nav-link active" href="#shop">Shop</a>
  <a class="btn" href="#buy">Buy</a>
</div>
<p class="note">First</p>
<input type="checkbox" id="agree">
<input type="text" id="user">
<script>
  console.log(document.querySelector("#title").textContent);          // "Shop"
  console.log(document.querySelector(".nav-link").textContent);       // "Home" (first)
  console.log(document.querySelectorAll("a.nav-link").length);        // 2
  console.log(document.querySelectorAll(".nav-link.active").length); // 1
  console.log(document.querySelector("input[type='text']").id);       // "user"
  console.log(document.querySelectorAll("#menu > a").length);         // 3
  console.log(document.querySelectorAll("h1, .note").length);         // 2
  console.log(document.querySelector(".nope"));                       // null
</script>
```

`#` بگذارم یا نه؟

متدهای `getElement...` **نام خالی** می‌گیرند (`getElementById("title")`، `getElementsByClassName("btn")`)، ولی `querySelector` و `querySelectorAll` **CSS selector** می‌گیرند و علامت لازم را می‌خواهند (`querySelector("#title")`، `querySelector(".btn")`). `querySelector("title")` دنبال **تگ** `<title>` می‌گردد، نه عنصری با id برابر `title`.

فعالیت کلاسی اسلاید ۱۷: هر خط چه می‌کند؟

sample.js

```
const title = document.getElementById("title");
const buttons = document.getElementsByClassName("btn");
const paragraphs = document.getElementsByTagName("p");
const firstInput = document.querySelector("input[type='text']");
const allLinks = document.querySelectorAll("a.nav-link");
```

خط	چه می‌کند	نتیجه
۱	عنصری که id آن `title` است (نام بدون `#`)	یک عنصر، یا `null`
۲	همه‌ی عنصرهایی که کلاس `btn` دارند، حتی اگر کلاس دیگری هم داشته باشند (`class="btn primary"`)	HTMLCollection زنده
۳	همه‌ی تگ‌های `<p>` صفحه	HTMLCollection زنده
۴	**اولین** `<input>`ی که attribute `type` آن `text` است (attribute selector)؛ چک‌باکس‌ها و inputهای دیگر رد می‌شوند	یک عنصر، یا `null`
۵	همه‌ی `<a>`هایی که کلاس `nav-link` دارند (selector از نوع `tag.class`)	NodeList ایستا

در خط ۴، رشته با `"` باز شده و مقدار attribute داخلش با `'` آمده (`"input[type='text']"`) تا دو نوع کوتیشن قاطی نشوند. نتیجه‌ها هم با `const` نگه داشته شده‌اند. اگر این کد را روی صفحه‌ی آزمایشی زیر اجرا کنیم:

HTML

```
<h1 id="title">Welcome</h1>
<button class="btn primary">A</button>
<button class="btn">B</button>
<p>First paragraph</p>
<p>Second paragraph</p>
<input type="checkbox" id="remember">
<input type="text" id="username">
<a class="nav-link" href="#home">Home</a>
<a class="nav-link" href="#about">About</a>
<a href="#help">Help</a>
```

نتیجه این است: `title` همان `<h1>` است، `buttons.length` برابر `2`، `paragraphs.length` برابر `2`، `firstInput.id` برابر `"username"` (چک‌باکس رد شد) و `allLinks.length` برابر `2` (لینک Help کلاس `nav-link` ندارد).

مجموعه‌ی زنده (live) در برابر ایستا (static)

HTMLCollectionای که `getElementsBy...` برمی‌گرداند **زنده (live)** است: اگر بعداً DOM تغییر کند، خودش به‌روز می‌شود. ولی NodeList `querySelectorAll` **ایستا (static)** است: یک عکس فوری (snapshot) از لحظه‌ی جست‌وجو.

```HTML
<ul id="list"><li class="item">a</li><li class="item">b</li></ul>
<script>
  const live = document.getElementsByClassName("item"); // HTMLCollection (live)
  const fixed = document.querySelectorAll(".item");     // NodeList (static)

  const li = document.createElement("li");              // a brand-new <li>
  li.className = "item";
  document.getElementById("list").appendChild(li);

  console.log(live.length, fixed.length);               // 3 2
</script>
```

همین زنده بودن یک دام معروف دارد: اگر در حلقه‌ای روی مجموعه‌ی زنده، کلاسی را که با آن جست‌وجو کرده‌ای حذف کنی، عنصرها از زیر دستت از مجموعه بیرون می‌روند و حلقه بعضی‌ها را جا می‌اندازد:

```JavaScript
// 4 paragraphs: <p class="red">1</p> ... <p class="red">4</p>
const reds = document.getElementsByClassName("red");
for (let i = 0; i < reds.length; i++) {
  reds[i].classList.remove("red"); // the element leaves the live collection immediately!
}
console.log(document.getElementsByClassName("red").length); // 2 (items 2, 4 skipped)
```

راه‌حل: روی یک کپی ثابت حلقه بزن (`for (const p of Array.from(reds))`) یا از `querySelectorAll` استفاده کن که ایستاست.

جست‌وجو داخل یک عنصر

`querySelector`، `querySelectorAll`، `getElementsByClassName` و `getElementsByTagName` روی هر **عنصری** هم صدا زده می‌شوند و آن وقت فقط **داخل همان عنصر** (بین نوادگانش) می‌گردند. ولی `getElementById` فقط روی `document` وجود دارد:

```HTML
<p class="note">First</p>
<div class="box">
  <p>Inside box</p>
  <section><p class="note">Deep</p></section>
</div>
<script>
  const box = document.querySelector(".box");
  console.log(box.querySelector("p").textContent);   // "Inside box"
  console.log(box.querySelectorAll(".note").length); // 1 (in the whole page: 2)
  console.log(typeof box.getElementById);            // "undefined"
</script>
```

چون `box.getElementById` وجود ندارد (`undefined` است)، صدا زدن `box.getElementById("x")` خطای `TypeError` می‌دهد.

id که با رقم شروع می‌شود (`id="2223"` در اسلاید ۱۰)

در CSS، شناسه‌ها (id و نام کلاس داخل selector) نمی‌توانند با رقم شروع شوند. برای همین `document.querySelector("#2223")` یک selector **نامعتبر** است و خطای `SyntaxError` (با پیامی شبیه `'#2223' is not a valid selector`) می‌دهد. ولی `getElementById` اصلاً CSS نمی‌فهمد و فقط id را مقایسه می‌کند:

```JavaScript
document.getElementById("2223");            // works: <h3 id="2223">
document.querySelector('[id="2223"]');      // works: attribute selector
document.querySelector("#" + CSS.escape("2223")); // works: escaped selector
document.querySelector("#2223");            // SyntaxError: not a valid selector
```

⚠ دام تستی

- `getElementById("#title")` همیشه `null` می‌دهد (چون `#` جزو نام id نیست) و استفاده از آن نتیجه (`.textContent`) خطای `TypeError` می‌دهد.
- HTMLCollection متد `forEach` ندارد (`TypeError`)، ولی NodeList دارد. هیچ‌کدام `map` ندارند.
- متدهای جمع (`getElementsBy...` و `querySelectorAll`) وقتی چیزی پیدا نکنند `null` نمی‌دهند، مجموعه‌ی خالی می‌دهند؛ متدهای مفرد (`getElementById` و `querySelector`) `null` می‌دهند.
- `querySelector` فقط **اولین** مورد را برمی‌گرداند؛ برای همه، `querySelectorAll` لازم است.
- `"div p"` (با فاصله: همه‌ی نوادگان) با `"div > p"` (فقط فرزند مستقیم) و `".a.b"` (هر دو کلاس) با `".a .b"` (عنصر `.b` داخل `.a`) فرق دارد.

✓ جمع‌بندی

- یکی با id: `getElementById("id")` (بدون `#`، عنصر یا `null`).
- همه با کلاس یا تگ: `getElementsByClassName` / `getElementsByTagName` (HTMLCollection زنده، بدون `forEach`).
- با CSS selector: `querySelector` (اولین مورد یا `null`) و `querySelectorAll` (NodeList ایستا با `forEach`).
- جست‌وجو داخل یک عنصر: `el.querySelector(...)`؛ `getElementById` فقط روی `document`.
- id با رقم اول: `querySelector("#2223")` خطای `SyntaxError` می‌دهد ولی `getElementById("2223")` کار می‌کند.

✓ تست‌های تمرینی

۲۳ تست · شماره‌ی ۷۴۷ تا ۷۶۹

۷۴۷ آسان

`document.getElementById("main")` چه چیزی برمی‌گرداند؟

۱. یک HTMLCollection از عنصرهایی که id آن‌ها `main` است

۲. یک NodeList

۳. یک عنصر، یا `null` اگر عنصری با این id نباشد

۴. آرایه‌ای از عنصرها

۷۴۸ آسان

خروجی کد زیر کدام است؟

HTML

```html
<h1 id="title">Shop</h1>
```

JavaScript

```javascript
console.log(document.getElementById("#title"));
console.log(document.querySelector("#title").textContent);
```

۱) `Shop` `Shop`
۲) `null` `Shop`
۳) `null` `null`
۴) `Uncaught SyntaxError`

۷۴۹ آسان

کدام متد عنصر را با یک **CSS selector** پیدا می‌کند؟

۱) `getElementById`
۲) `getElementsByClassName`
۳) `getElementsByTagName`
۴) `querySelector`

۷۵۰ متوسط

خروجی کد زیر کدام است؟

HTML

```html
<p class="note">First</p>
<div class="box">
  <p>Inside box</p>
  <section><p class="note">Deep</p></section>
</div>
```

JavaScript

```javascript
console.log(
  document.querySelectorAll("div p").length,
  document.querySelectorAll("div > p").length,
  document.getElementsByTagName("p").length
);
```

۱) `2 1 3`
۲) `2 2 3`
۳) `1 1 3`
۴) `3 1 3`

۷۵۱ سخت

خروجی کد زیر کدام است؟

HTML

```
<div class="card big">1</div>
<div class="big card">2</div>
<div class="card">3</div>
```

JavaScript

```
console.log(
  document.getElementsByClassName("card big").length,
  document.querySelectorAll(".card.big").length,
  document.querySelectorAll(".card .big").length
);
```

۱) `1 1 0`
۲) `2 2 0`
۳) `2 2 2`
۴) `3 2 0`

۷۵۲ متوسط

خروجی کد زیر کدام است؟

HTML

```
<button class="btn">A</button>
<button class="btn">B</button>
```

JavaScript

```
const a = document.getElementsByClassName("btn");
const b = document.querySelectorAll(".btn");
console.log(a.constructor.name, b.constructor.name);
console.log(typeof a.forEach, typeof b.forEach);
```

۱) `NodeList HTMLCollection` `function undefined`
۲) `Array Array` `function function`
۳) `HTMLCollection NodeList` `function function`
۴) `HTMLCollection NodeList` `undefined function`

۷۵۳ متوسط

خروجی کد زیر کدام است؟

HTML

```
<button class="btn">A</button>
<button class="btn">B</button>
<button class="btn">C</button>
```

JavaScript

```
const buttons = document.getElementsByClassName("btn");
console.log(buttons.length);
buttons.forEach((b) => console.log(b.textContent));
console.log("done");
```

۱) `3` `A` `B` `C` `done`

۲) `3` `done`

۳) `3` `Uncaught TypeError`

۴) `Uncaught TypeError`

۷۵۴ متوسط

خروجی کد زیر کدام است؟

HTML

```
<p class="item">A</p>
<p class="item">B</p>
```

JavaScript

```
const items = document.getElementsByClassName("item");
function test(label, fn) {
  try {
    fn();
    console.log(label, "ok");
  } catch (err) {
    console.log(label, err.name);
  }
}
test("forEach", () => items.forEach((el) => el.textContent));
test("Array.from", () => Array.from(items).forEach((el) => el.textContent));
test("spread", () => [...items].forEach((el) => el.textContent));
test("for...of", () => { for (const el of items) el.textContent; });
```

۱) `forEach ok` `Array.from ok` `spread ok` `for...of ok`

۲) `forEach TypeError` `Array.from ok` `spread ok` `for...of ok`

۳) `forEach TypeError` `Array.from ok` `spread TypeError` `for...of TypeError`

۴) `forEach TypeError` `Array.from TypeError` `spread TypeError` `for...of ok`

۷۵۵ سخت

خروجی کد زیر کدام است؟

HTML

```
<ul id="list"><li class="item">a</li><li class="item">b</li></ul>
```

JavaScript

```
const live = document.getElementsByClassName("item");
const fixed = document.querySelectorAll(".item");
const li = document.createElement("li");
li.className = "item";
document.getElementById("list").appendChild(li);
console.log(live.length, fixed.length);
```

۱) `3 3`
۲) `2 2`
۳) `3 2`
۴) `2 3`

۷۵۶ سخت

خروجی کد زیر کدام است؟

HTML

```
<p class="red">1</p>
<p class="red">2</p>
<p class="red">3</p>
<p class="red">4</p>
```

JavaScript

```
const reds = document.getElementsByClassName("red");
for (let i = 0; i < reds.length; i++) {
  reds[i].classList.remove("red");
}
console.log(document.getElementsByClassName("red").length);
```

۱) `0`
۲) `2`
۳) `1`
۴) `Uncaught TypeError`

۷۵۷ متوسط

خروجی کد زیر کدام است؟

HTML

```
<input type="checkbox" id="agree">
<input type="text" id="user">
<input type="text" id="city">
```

JavaScript

```
console.log(
  document.querySelector("input").id,
  document.querySelector("input[type='text']").id
);
```

۱) `agree user`
۲) `user user`
۳) `agree agree`
۴) `user city`

۷۵۸ سخت

خروجی کد زیر کدام است؟

HTML

```
<h1>Shop</h1>
<p class="note">First</p>
<p class="note">Second</p>
```

JavaScript

```
const list = document.querySelectorAll(".note, h1");
console.log(list.length, list[0].tagName);
```

۱) `3 P`
۲) `2 P`
۳) `3 H1`
۴) `1 P`

۷۵۹ آسان

خروجی کد زیر کدام است؟

HTML

```
<p class="x">Hi</p>
```

JavaScript

```
console.log(
  document.getElementById("nope"),
  document.querySelector(".nope"),
  document.querySelectorAll(".nope").length,
  document.getElementsByClassName("nope").length
);
```

۱) `null null null null`
۲) `undefined undefined 0 0`
۳) `null null 0 null`
۴) `null null 0 0`

۷۶۰ سخت

در فعالیت کلاسی اسلاید ۱۰، عنصر `<h3>` مقدار `id="2223"` دارد. خروجی کد زیر کدام است؟

HTML

```
<h3 id="2223">wait wha-?</h3>
```

JavaScript

```
console.log(document.getElementById("2223").textContent);
console.log(document.querySelector("#2223").textContent);
console.log("end");
```

۱) `wait wha-?` `wait wha-?` `end`

۲) `wait wha-?` `Uncaught TypeError`

۳) `Uncaught SyntaxError`

۴) `wait wha-?` `Uncaught SyntaxError`

۷۶۱ متوسط

خروجی کد زیر کدام است؟

HTML

```
<p class="note">First</p>
<div class="box">
  <p>Inside box</p>
  <section><p class="note">Deep</p></section>
</div>
```

JavaScript

```
const box = document.querySelector(".box");
console.log(box.querySelector("p").textContent, box.querySelectorAll(".note").length);
console.log(document.querySelector(".note").textContent);
```

۱) `Inside box 1` `First`

۲) `First 2` `First`

۳) `Inside box 2` `First`

۴) `Inside box 1` `Deep`

۷۶۲ سخت

خروجی کد زیر کدام است؟

```HTML
<div id="menu"><a id="home" href="#">Home</a></div>
```

```JavaScript
const menu = document.getElementById("menu");
console.log(menu.querySelector("#home").textContent);
console.log(menu.getElementById("home").textContent);
```

۱) `Home` `Home`

۲) `Home` `Uncaught TypeError`

۳) `Home` `null`

۴) `Uncaught TypeError`

۷۶۳ آسان

selector `input[type='text']` کدام عنصرها را توصیف می‌کند؟

۱) عنصرهای `<input>` ی که attribute `type` آن‌ها برابر `text` است

۲) عنصرهایی که کلاس `text` دارند

۳) عنصری که id آن برابر `text` است

۴) همه‌ی `<input>` ها، به‌همراه همه‌ی عنصرهایی که متن دارند

۷۶۴ متوسط

درباره‌ی کد فعالیت کلاسی اسلاید ۱۷، کدام عبارت نادرست است؟

```JavaScript
const title = document.getElementById("title");
const buttons = document.getElementsByClassName("btn");
const paragraphs = document.getElementsByTagName("p");
const firstInput = document.querySelector("input[type='text']");
const allLinks = document.querySelectorAll("a.nav-link");
```

۱) `title` یک عنصر است، یا `null` اگر عنصری با id برابر `title` وجود نداشته باشد.

۲) `buttons` یک HTMLCollection زنده است که عنصرهای دارای کلاس `btn` را، حتی با `class="btn primary"`، شامل می‌شود.

۳) `firstInput` همه‌ی inputهای متنی صفحه را به‌صورت یک NodeList برمی‌گرداند.

۴) `allLinks` یک NodeList ایستا از همه‌ی `<a>` هایی است که کلاس `nav-link` دارند.

۷۶۵ متوسط

در صفحه‌ی زیر، به‌اشتباه دو عنصر id یکسان دارند. خروجی کد کدام است؟

```HTML
<p id="x">first</p>
<p id="x">second</p>
```

```JavaScript
const first = document.getElementById("x");
console.log(first.textContent, document.querySelectorAll("#x").length);
```

۱) `second 2`
۲) `first 1`
۳) `first 2`
۴) `second 1`

۷۶۶ متوسط

می‌خواهیم فقط `<li>`هایی را انتخاب کنیم که **فرزند مستقیمِ** `<ul id="menu">` هستند (نه `<li>` های زیرمنوهای تو در تو). در جای خالی `document.querySelectorAll("______")` کدام گزینه قرار می‌گیرد؟

۱) `.menu > li`
۲) `menu > li`
۳) `#menu, li`
۴) `#menu > li`

۷۶۷ سخت

خروجی کد زیر کدام است؟

```HTML
<p class="red">1</p>
<p class="red">2</p>
<p class="red">3</p>
<p class="red">4</p>
```

```JavaScript
const reds = document.querySelectorAll(".red");
reds.forEach((p) => p.classList.remove("red"));
console.log(document.querySelectorAll(".red").length, reds.length);
```

۱) `0 4`
۲) `2 4`
۳) `0 0`
۴) `2 2`

۷۶۸ آسان

خروجی کد زیر کدام است؟

HTML

```
<button class="btn">A</button>
<button class="btn">B</button>
```

JavaScript

```
const buttons = document.getElementsByClassName("btn");
console.log(buttons.length, buttons[0].textContent);
console.log(buttons.item(1).textContent, buttons[5]);
```

۱) `2 A` `B null`
۲) `2 A` `B undefined`
۳) `2 B` `A undefined`
۴) `3 A` `B undefined`

۷۶۹ سخت

صفحه‌ای فقط HTML زیر را دارد:

HTML

```
<a class="nav-link" href="#home">Home</a>
<a class="nav-link" href="#shop">Shop</a>
```

کد JavaScript زیر در انتهای صفحه اجرا می‌شود. اجرای آن در کدام خط با خطا متوقف می‌شود؟

JavaScript

```
1  const links = document.getElementsByClassName("nav-link");
2  console.log(links.length);
3  const active = document.querySelector(".nav-link.active");
4  const firstLink = links[0];
5  console.log(firstLink.textContent);
6  console.log(active.textContent);
```

۱) خط ۳
۲) خط ۴
۳) خط ۵
۴) خط ۶

۲-۵ رویدادها (Events) و addEventListener

منبع: JS in HTML، اسلایدهای ۱۸ تا ۲۱

درسنامه

تا اینجا کدهای ما فقط یک بار، هنگام بارگذاری صفحه، اجرا می‌شدند. برای اینکه صفحه **تعاملی** شود (هدف دوم این جلسه در اسلاید ۲)، باید بتوانیم به کارهای کاربر واکنش نشان دهیم. ابزار این کار **رویدادها (events)** هستند.

> **زنگ در**
>
> روی در خانه یک زنگ نصب می‌کنی و می‌گویی «هر وقت کسی زنگ زد، در را باز کن». تا کسی زنگ نزند اتفاقی نمی‌افتد؛ وقتی زنگ زده شد، کاری که سپرده بودی انجام می‌شود. حتی می‌توانی چند کار را به همان زنگ وصل کنی (هم چراغ روشن شود، هم آهنگی پخش شود). در وب، «زنگ زدن» همان **رویداد** است (مثلاً کلیک) و «کاری که سپرده‌ای» یک تابع است که به آن **event handler** یا **event listener** (شنونده‌ی رویداد) می‌گوییم.

رویداد (event) یعنی اتفاقی که در صفحه رخ می‌دهد: کلیک، تایپ، ارسال فرم، فشردن کلید، تمام شدن بارگذاری صفحه و ... . **event listener** تابعی است که به مرورگر می‌سپاریم تا هر بار آن رویداد روی یک عنصر رخ داد، اجرایش کند. خلاصه‌ی اسلاید ۱۸:

- رویدادها به صفحه‌ی وب اجازه می‌دهند به **کارهای کاربر واکنش نشان دهد**.
- رویدادهای رایج: `click` ، `input` ، `submit` ، `keydown` ، `load` و ... .
- handlerها را می‌توان با **attributeهای HTML** یا با **JavaScript** تنظیم کرد.
- `addEventListener` روش **مدرن** وصل کردن رویداد است.
- برای یک رویداد می‌توان **چند handler** اضافه کرد.
- رویدادها تجربه‌ی کاربری **تعاملی و پویا** را ممکن می‌کنند.

رویدادهای رایج

رویداد	چه زمانی رخ می‌دهد	معمولاً روی
`click`	کلیک روی عنصر	دکمه، لینک یا هر عنصری
`input`	**هر بار** که مقدار یک کادر ورودی عوض شود (با هر حرفی که تایپ می‌شود)	`<input>` ، `<textarea>`
`change`	وقتی تغییر مقدار «قطعی» شود: برای کادر متنی وقتی از کادر بیرون می‌رویم؛ برای چک‌باکس و `<select>` بلافاصله	`<input>` ، `<select>`
`submit`	ارسال فرم (کلیک روی دکمه‌ی ارسال یا زدن Enter در فرم)	`<form>`
`keydown`	فشردن یک کلید صفحه‌کلید	کادرهای ورودی یا `document`
`load`	بارگذاری کامل صفحه همراه همه‌ی منابعش (تصاویر، CSS و ...)	`window`
`DOMContentLoaded`	آماده شدن درخت DOM (درس قبل)	`document`
`mouseover`	رفتن نشانگر ماوس روی عنصر	هر عنصری

سه راه وصل کردن handler

۱) attribute در HTML

```HTML
<button id="save" onclick="console.log('saved', this.id)">Save</button>
<!-- click -> saved save -->
```

کد به‌صورت یک **رشته** داخل HTML نوشته می‌شود و داخل آن `this` همان عنصر است. مرورگر این رشته را به یک تابع تبدیل می‌کند و در property `onclick` عنصر می‌گذارد (پس `typeof btn.onclick` برابر `"function"` است). عیبش این است که ساختار (HTML) و رفتار (JavaScript) را قاطی می‌کند؛ برای پروژه‌های واقعی توصیه نمی‌شود.

۲) property در JavaScript

```JavaScript
const btn = document.getElementById("btn");
btn.onclick = () => console.log("A");
btn.onclick = () => console.log("B"); // replaces A!
// click -> B
btn.onclick = null;                   // removes the handler
```

هر عنصر برای هر رویداد فقط **یک** property مثل `onclick` دارد؛ پس مقداردهی دوباره، handler قبلی را **بازنویسی** می‌کند. اگر عنصر از قبل یک `onclick="..."` در HTML داشته باشد، همین مقداردهی آن را هم جایگزین می‌کند، چون attribute و property در واقع یک خانه‌ی مشترک دارند.

۳) `addEventListener` (روش مدرن)

```JavaScript
btn.addEventListener("click", () => console.log("listener 1"));
btn.addEventListener("click", () => console.log("listener 2"));
// click -> listener 1, listener 2   (in the order they were added)
```

- نام رویداد **بدون** `on` نوشته می‌شود: `"click"`، نه `"onclick"`. اگر بنویسی `addEventListener("onclick", ...)` هیچ خطایی نمی‌گیری، ولی listener منتظر رویدادی به نام «onclick» می‌ماند که هرگز رخ نمی‌دهد.
- می‌توان **چند listener** برای یک رویداد ثبت کرد و همه، **به ترتیب ثبت**، اجرا می‌شوند.
- اگر **همان تابع** (همان مرجع) را دو بار برای یک رویداد ثبت کنی، فقط یک بار ثبت می‌شود.
- آرگومان سوم تنظیمات است، مثلاً `{ once: true }` (پایین‌تر) یا مرحله‌ی capture (درس بعد).
- با `removeEventListener` قابل حذف است.

روش	چند handler برای یک رویداد؟	نکته
`onclick="..."` در HTML	خیر	کد داخل رشته؛ قاطی شدن HTML و JavaScript
`el.onclick = fn`	خیر؛ آخرین مقدار می‌ماند	حذف با `el.onclick = null`
`el.addEventListener("click", fn)`	بله؛ به ترتیب ثبت	روش مدرن؛ حذف با `removeEventListener`

تابع را بده، صدا نزن!

```JavaScript
function sayHi() {
  console.log("hi");
}
btn.addEventListener("click", sayHi);   // correct: pass the function itself
btn.addEventListener("click", sayHi()); // WRONG: runs now, passes undefined
```

در خط دوم، پرانتزها باعث می‌شوند `sayHi` **همان لحظه** اجرا شود (و `hi` بلافاصله چاپ شود)؛ مقدار برگشتی‌اش (`undefined`) به‌عنوان listener ثبت می‌شود که یعنی هیچ‌چیز، پس کلیک کاری نمی‌کند. خطایی هم رخ نمی‌دهد و همین پیدا کردن این باگ را سخت می‌کند.

اگر لازم است به تابع آرگومان بدهی، آن را داخل یک arrow function بپیچ:

```JavaScript
function greet(name) {
  console.log("Hello " + name);
}
btn.addEventListener("click", () => greet("Sara")); // Hello Sara
btn.addEventListener("click", greet);               // Hello [object PointerEvent]
```

در خط دوم، مرورگر هنگام کلیک، `greet` را با **شیء رویداد** (event object) به‌عنوان اولین آرگومان صدا می‌زند؛ پس `name` دیگر یک رشته نیست، خودِ شیء رویداد است و در الحاق با رشته به چیزی مثل `[object PointerEvent]` تبدیل می‌شود. (در Chrome و مرورگرهای جدید، رویداد کلیک از نوع `PointerEvent` است که نوعی `MouseEvent` به حساب می‌آید؛ در بعضی محیط‌های قدیمی‌تر همان `[object MouseEvent]` چاپ می‌شود. شیء event را در درس بعد کامل می‌بینیم.)

حذف listener با `removeEventListener`

```JavaScript
function onClick() {
  console.log("clicked");
}
btn.addEventListener("click", onClick);
btn.removeEventListener("click", onClick); // removed: the SAME function reference

btn.addEventListener("click", () => console.log("anon"));
// NOT removed: the arrow function below is a different function object
btn.removeEventListener("click", () => console.log("anon"));
// click -> anon
```

برای حذف باید **همان نوع رویداد** و **همان شیء تابع** را بدهی. دو arrow function با متن یکسان، دو شیء متفاوت‌اند (همان‌طور که `{} === {}` برابر `false` است)؛ پس listener ناشناس (anonymous) را نمی‌توان حذف کرد. اگر قرار است بعداً حذفش کنی، اول آن را در یک متغیر یا تابع نام‌دار نگه دار. یک listener می‌تواند خودش را هم حذف کند (`removeEventListener` داخل بدنه‌اش)، و برای «فقط یک بار اجرا شو» راه ساده‌تری هم هست:

```JavaScript
btn.addEventListener("click", () => console.log("welcome!"), { once: true });
// first click -> welcome!   further clicks -> nothing
```

مثال اسلایدهای ۱۹ تا ۲۱: Button Action

doc.html

```
<!DOCTYPE html>
<html>
  <body>
    <h1 id="title">Hello World</h1>
    <button>CLICK ME!</button>

    <script>
      const btn = document.querySelector('button');

      btn.addEventListener('click', () => {
        const title = document.querySelector('#title');
        title.textContent = 'Goodbye World';
        title.style.color = 'red';
      });
    </script>
  </body>
</html>
```

- `document.querySelector('button')` با selector تگ، **اولین** `<button>` صفحه را پیدا می‌کند.
- `addEventListener('click', () => { ... })` یک arrow function برای کلیک ثبت می‌کند.
- داخل listener، `querySelector('#title')` عنوان را پیدا می‌کند (اینجا `#` لازم است چون ورودی CSS selector است؛ برخلاف `getElementById("title")`).
- `title.textContent = 'Goodbye World'` متن را عوض می‌کند و `title.style.color = 'red'` رنگ را قرمز می‌کند (درس «تغییر متن، HTML و استایل عناصر»).

اسلاید ۲۰ صفحه را **قبل** از کلیک نشان می‌دهد: عنوان سیاه «Hello World» و دکمه‌ی «CLICK ME!». اسلاید ۲۱ **بعد** از کلیک را نشان می‌دهد: عنوان «Goodbye World» به رنگ **قرمز** و دکمه بدون تغییر. در DOM، عنوان به `<h1 id="title" style="color: red;">Goodbye World</h1>` تبدیل شده است. کلیک‌های بعدی هم همین مقدارها را دوباره می‌نویسند و تغییر تازه‌ای دیده نمی‌شود.

`input` در برابر `change`، و `keydown` با `event.key`

HTML

```
<input id="name">
<script>
  const nameBox = document.getElementById("name");
  nameBox.addEventListener("input", (e) => console.log("input:", e.target.value));
  nameBox.addEventListener("change", (e) => console.log("change:", e.target.value));
</script>
```

اگر کاربر حرف `A` و بعد `l` را تایپ کند و سپس از کادر بیرون برود، خروجی این است: `input: A`، `input: Al`، `change: Al`. یعنی `input` با هر تغییر و `change` یک بار در پایان. (`e.target` همان عنصری است که رویداد رویش رخ داده؛ درس بعد.)

برای `keydown`، property `e.key` می‌گوید کدام کلید فشرده شده: `"a"`، `"Enter"`، `"Escape"`، `"ArrowUp"` و ... :

```JavaScript
searchBox.addEventListener("keydown", (e) => {
  if (e.key === "Enter") console.log("search for", searchBox.value);
});
```

`load` روی `window`

```JavaScript
window.addEventListener("load", () => console.log("page fully loaded"));
console.log("script runs first");
// script runs first -> page fully loaded
```

`load` وقتی رخ می‌دهد که صفحه همراه **همه‌ی** منابعش (تصاویر، فایل‌های CSS و ...) بارگذاری شده باشد؛ پس از `DOMContentLoaded` دیرتر است.

نکته

- همیشه نام رویداد را بدون `on` به `addEventListener` بده: `"click"` ، `"input"` ، `"keydown"` .
- اگر قرار است listener را بعداً حذف کنی، آن را تابع نام‌دار تعریف کن؛ اگر فقط یک بار لازم است، `{ once: true }` بگذار.

دام تستی

- `addEventListener("click", handler())` تابع را **همان لحظه** اجرا می‌کند و چیزی برای کلیک ثبت نمی‌شود.
- `el.onclick = ...` فقط یک handler نگه می‌دارد؛ مقداردهی دوم اولی را (و حتی `onclick` داخل HTML را) پاک می‌کند. ولی `addEventListener` چندتایی است.
- `removeEventListener` با یک تابع ناشناسِ تازه هیچ‌چیزی را حذف نمی‌کند.
- `addEventListener("onclick", ...)` نه خطا می‌دهد نه هیچ‌وقت اجرا می‌شود.
- `querySelector('button')` فقط **اولین** دکمه را برمی‌گرداند؛ اگر دکمه‌ی دیگری قبل از دکمه‌ی موردنظرت باشد، listener روی آن یکی ثبت می‌شود.

جمع‌بندی

- رویداد = اتفاقی در صفحه؛ listener = تابعی که با رخ دادن آن اجرا می‌شود.
- سه روش: `onclick="..."` در HTML، `el.onclick = fn` (فقط یکی)، `addEventListener` (مدرن، چندتایی، به ترتیب ثبت).
- مرجع تابع را بده (`fn`)، صدایش نزن (`fn()`)؛ برای آرگومان از `() => fn(arg)` استفاده کن.
- `removeEventListener` همان تابع را لازم دارد؛ `{ once: true }` برای اجرای یک‌باره.
- `input` با هر تغییر، `change` در پایان؛ برای کلید؛ `e.key` بعد از همه‌ی منابع. `load`

تست‌های تمرینی

۲۲ تست · شماره‌ی ۷۷۰ تا ۷۹۱

۷۷۰ آسان

طبق اسلاید ۱۸، روش **مدرن** وصل کردن رویداد که اجازه می‌دهد چند handler برای یک رویداد ثبت شود کدام است؟

۱) attribute `onclick` در HTML

۲) property `onclick` در JavaScript (`el.onclick = fn`)

۳) `addEventListener`

۴) `document.write`

۷۷۱ آسان

برای اجرای `handler` با کلیک روی دکمه، در جای خالی `btn.addEventListener("______", handler)` کدام گزینه قرار می‌گیرد؟

۱) `onclick`　۲) `click`　۳) `onClick`　۴) `clicked`

۷۷۲ متوسط

کاربر یک بار روی دکمه کلیک می‌کند. خروجی Console کدام است؟

```HTML
<button id="btn">Go</button>
```

```JavaScript
const btn = document.getElementById("btn");
btn.addEventListener("onclick", () => console.log("clicked"));
btn.addEventListener("click", () => console.log("click!"));
```

۱) `clicked` `click!`

۲) `clicked`

۳) (بدون خروجی)

۴) `click!`

۷۷۳ سخت

کاربر یک بار روی دکمه کلیک می‌کند. خروجی Console کدام است؟

```HTML
<button id="btn">Go</button>
```

```JavaScript
const btn = document.getElementById("btn");
btn.onclick = () => console.log("A");
btn.onclick = () => console.log("B");
btn.addEventListener("click", () => console.log("C"));
btn.addEventListener("click", () => console.log("D"));
```

۱) `A` `B` `C` `D`

۲) `B` `C` `D`

۳) `A` `C` `D`

۴) `C` `D` `B`

۷۷۴ سخت

کاربر یک بار روی دکمه کلیک می‌کند. خروجی Console کدام است؟

HTML

```
<button id="btn" onclick="console.log('attr')">Go</button>
```

JavaScript

```
const btn = document.getElementById("btn");
btn.addEventListener("click", () => console.log("listener"));
btn.onclick = () => console.log("property");
```

۱) `property` `listener`

۲) `attr` `listener` `property`

۳) `attr` `property` `listener`

۴) `listener` `property`

۷۷۵ متوسط

کاربر **دو بار** روی دکمه کلیک می‌کند. خروجی کامل Console کدام است؟

HTML

```
<button id="btn">Go</button>
```

JavaScript

```
function sayHi() {
  console.log("hi");
}
const btn = document.getElementById("btn");
btn.addEventListener("click", sayHi());
console.log("ready");
```

۱) `ready` `hi` `hi`

۲) `hi` `ready` `hi` `hi`

۳) `hi` `ready`

۴) `Uncaught TypeError`

۷۷۶ سخت

کاربر یک بار روی دکمه کلیک می‌کند. خروجی Console کدام است؟

HTML

```
<button id="btn">Go</button>
```

JavaScript

```
function sayHi() {
  console.log("hi");
}
const btn = document.getElementById("btn");
btn.addEventListener("click", sayHi);
btn.addEventListener("click", sayHi);
btn.addEventListener("click", () => console.log("arrow"));
btn.addEventListener("click", () => console.log("arrow"));
```

۱) `hi` `hi` `arrow` `arrow`

۲) `hi` `arrow`

۳) `hi` `hi` `arrow`

۴) `hi` `arrow` `arrow`

۷۷۷ متوسط

کاربر یک بار روی دکمه کلیک می‌کند. خروجی Console کدام است؟

HTML

```
<button id="btn">Go</button>
```

JavaScript

```
const btn = document.getElementById("btn");
btn.addEventListener("click", () => console.log("anon"));
btn.removeEventListener("click", () => console.log("anon"));
function named() {
  console.log("named");
}
btn.addEventListener("click", named);
btn.removeEventListener("click", named);
```

۱) (بدون خروجی)

۲) `anon`

۳) `anon` `named`

۴) `named`

۷۷۸ متوسط

کاربر **سه بار** روی دکمه کلیک می‌کند. خروجی Console کدام است؟

HTML

```
<button id="btn">Go</button>
```

JavaScript

```
const btn = document.getElementById("btn");
let n = 0;
btn.addEventListener("click", () => console.log("once", ++n), { once: true });
btn.addEventListener("click", () => console.log("always"));
```

۱) `once 1` `always`

۲) `once 1` `always` `once 2` `always` `once 3` `always`

۳) `once 1` `always` `always` `always`

۴) `always` `always` `always`

۷۷۹ متوسط

کاربر در کادر متنی حرف `A` و سپس حرف `l` را تایپ می‌کند و بعد بیرون کادر کلیک می‌کند (کادر از حالت فعال خارج می‌شود). خروجی Console کدام است؟

HTML

```
<input id="name">
```

JavaScript

```
const nameBox = document.getElementById("name");
nameBox.addEventListener("input", (e) => console.log("input:", e.target.value));
nameBox.addEventListener("change", (e) => console.log("change:", e.target.value));
```

۱) `input: A` `input: Al` `change: Al`

۲) `change: A` `change: Al` `input: Al`

۳) `input: Al` `change: Al`

۴) `input: A` `change: A` `input: Al` `change: Al`

۷۸۰ آسان

کاربر داخل کادر، کلید `a` و سپس کلید Enter را فشار می‌دهد. خروجی Console کدام است؟

HTML

```
<input id="box">
```

JavaScript

```
document.getElementById("box").addEventListener("keydown", (e) => {
  if (e.key === "Enter") console.log("submit search");
  else console.log("key:", e.key);
});
```

۱) `key: a` `submit search`

۲) `key: a` `key: Enter`

۳) `key: a` `submit search`

۴) `submit search`

۷۸۱ آسان

کدام جفتِ «رویداد: زمان رخ دادن» نادرست است؟

۱) `click` : وقتی کاربر روی عنصر کلیک می‌کند

۲) `input` : هر بار که مقدار یک کادر ورودی تغییر می‌کند

۳) `submit` : وقتی یک فرم ارسال می‌شود

۴) `load` : هر بار که کاربر کلیدی را روی صفحه‌کلید فشار می‌دهد

۷۸۲ سخت

کد مثال Button Action (اسلاید ۱۹) روی صفحه‌ی زیر اجرا می‌شود که یک دکمه‌ی اضافی دارد. کاربر روی دکمه‌ی **CLICK ME!** کلیک می‌کند و بعد در Console متن و رنگ inline عنوان را چاپ می‌کنیم. خروجی کدام است؟

HTML

```
<h1 id="title">Hello World</h1>
<button id="reset">Reset</button>
<button id="go">CLICK ME!</button>
```

JavaScript

```
const btn = document.querySelector('button');

btn.addEventListener('click', () => {
  const title = document.querySelector('#title');
  title.textContent = 'Goodbye World';
  title.style.color = 'red';
});
```

۱) `Goodbye World "red"`

۲) `Hello World ""`

۳) `Goodbye World ""`

۴) `Uncaught TypeError`

۷۸۳ متوسط

نسخه‌ی زیر همان مثال Button Action است که یک شمارنده هم دارد. کاربر **سه بار** روی دکمه کلیک می‌کند و سپس در Console دستور `console.log(clicks, document.querySelector("#title").textContent)` را اجرا می‌کنیم.

خروجی کدام است؟

HTML

```html
<h1 id="title">Hello World</h1>
<button>CLICK ME!</button>
```

JavaScript

```javascript
const btn = document.querySelector('button');
let clicks = 0;
btn.addEventListener('click', () => {
  clicks++;
  const title = document.querySelector('#title');
  title.textContent = 'Goodbye World';
  title.style.color = 'red';
});
```

۱) `3 Goodbye World`

۲) `1 Goodbye World`

۳) `3 Goodbye WorldGoodbye WorldGoodbye World`

۴) `0 Hello World`

۷۸۴ آسان

کدام عبارت درباره‌ی رویدادها (اسلاید ۱۸) نادرست است؟

۱) رویدادها به صفحه‌ی وب اجازه می‌دهند به کارهای کاربر واکنش نشان دهد.

۲) handlerها را می‌توان هم با attributeهای HTML و هم با JavaScript تنظیم کرد.

۳) با `addEventListener` برای یک نوع رویداد روی یک عنصر فقط یک handler می‌توان ثبت کرد.

۴) رویدادها تجربه‌ی کاربری تعاملی و پویا را ممکن می‌کنند.

۷۸۵ متوسط

برای اینکه `removeEventListener` یک listener را واقعاً حذف کند، کدام شرط لازم است؟

۱) همان نوع رویداد و **همان مرجع تابع** (همان شیء تابع) که به `addEventListener` داده شده بود

۲) فقط نوع رویداد کافی است و همه‌ی listenerهای آن نوع حذف می‌شوند

۳) تابعی با همان بدنه (همان متن کد)، حتی اگر شیء تابع جدیدی باشد

۴) listener حتماً باید arrow function باشد

۷۸۶ آسان

رویداد `load` روی `window` چه زمانی رخ می‌دهد؟

۱. بلافاصله بعد از اجرای اولین تگ `<script>`

۲. هر بار که کاربر روی صفحه کلیک کند

۳. قبل از اینکه مرورگر شروع به parse کردن HTML کند

۴. وقتی کل صفحه همراه همه‌ی منابعش (مثل تصاویر و فایل‌های CSS) کامل بارگذاری شده باشد

۷۸۷ سخت

کاربر یک بار روی دکمه کلیک می‌کند. خروجی Console کدام است؟

HTML

```
<button id="btn">Go</button>
```

JavaScript

```
function greet(name) {
  console.log(typeof name, name instanceof Event ? "an event object" : name);
}
const btn = document.getElementById("btn");
btn.addEventListener("click", () => greet("Sara"));
btn.addEventListener("click", greet);
```

۱. `string Sara` `undefined undefined`

۲. `string Sara` `string Sara`

۳. `string Sara` `object an event object`

۴. `string Sara` `Uncaught TypeError`

۷۸۸ متوسط

کاربر **سه بار** روی دکمه کلیک می‌کند. خروجی Console کدام است؟

HTML

```
<button id="btn">Go</button>
```

JavaScript

```
const btn = document.getElementById("btn");
function handler() {
  console.log("clicked");
  btn.removeEventListener("click", handler);
}
btn.addEventListener("click", handler);
```

۱. `clicked` `clicked` `clicked`

۲. `clicked`

۳. (بدون خروجی)

۴. `clicked` `Uncaught TypeError`

۷۸۹ آسان

کدام عبارت درباره‌ی `btn.onclick = fn` درست است؟

۱) فقط یک handler نگه می‌دارد و مقداردهی دوباره، handler قبلی را جایگزین می‌کند.

۲) مثل `addEventListener` همه‌ی handlerهای مقداردهی‌شده را پشت سر هم اجرا می‌کند.

۳) فقط داخل کد HTML قابل استفاده است.

۴) نام رویداد را باید بدون on نوشت، یعنی `btn.click = fn` .

۷۹۰ سخت

صفحه بارگذاری می‌شود و سپس کاربر یک بار روی دکمه کلیک می‌کند. خروجی کامل Console کدام است؟

HTML

```html
<button id="save" onclick="console.log(this.id)">Save</button>
```

JavaScript

```javascript
const btn = document.getElementById("save");
console.log(typeof btn.onclick, btn.getAttribute("onclick"));
btn.addEventListener("click", () => console.log("listener"));
```

۱) `string console.log(this.id)` `save` `listener`

۲) `function console.log(this.id)` `undefined` `listener`

۳) `function console.log(this.id)` `listener` `save`

۴) `function console.log(this.id)` `save` `listener`

۷۹۱ متوسط

صفحه فقط یک دکمه دارد:

HTML

```html
<button id="save">Save</button>
```

کد زیر در انتهای صفحه اجرا می‌شود. اجرای آن در کدام خط با خطا متوقف می‌شود؟

JavaScript

```javascript
const save = document.getElementById("save");
save.addEventListener("click", () => console.log("saved"));
const cancel = document.getElementById("cancel");
console.log("cancel is", cancel);
cancel.addEventListener("click", () => console.log("cancelled"));
console.log("ready");
```

۱) خط ۲ ۲) خط ۳ ۳) خط ۴ ۴) خط ۵

۲-۶ شیء event، انتشار رویداد (Bubbling) و Delegation

منبع: JS in HTML، پیش‌زمینه‌ی لازم برای اسلایدهای ۱۸ تا ۲۱ و ۳۵

درسنامه

در درس قبل listener ثبت کردیم. حالا دو سؤال مهم: مرورگر چه اطلاعاتی به listener می‌دهد؟ و وقتی روی یک عنصرِ تو در تو کلیک می‌شود، listenerهای والدها هم اجرا می‌شوند یا نه، و به چه ترتیبی؟ این درس پیش‌زمینه‌ای است که برای فهم واقعی رویدادهای اسلاید ۱۸ و جمله‌ی «Prevent page reload on submit with `event.preventDefault()`» در اسلاید ۳۵ لازم داری.

شیء رویداد (event object)

هر بار رویدادی رخ می‌دهد، مرورگر یک **شیء رویداد (event object)** می‌سازد و آن را به‌عنوان **اولین آرگومان** به همه‌ی listenerها می‌دهد. معمولاً اسمش را `event` یا `e` می‌گذاریم.

property یا متد	معنی
`event.type`	نام رویداد: `"click"`، `"input"`، `"submit"` و ...
`event.target`	عنصری که رویداد **در اصل** روی آن رخ داده (درونی‌ترین عنصری که کلیک شده)
`event.currentTarget`	عنصری که listener **در حال اجرا** روی آن ثبت شده است
`event.key`	در رویدادهای صفحه‌کلید: کلید فشرده‌شده (`"Enter"`، `"a"` و ...)
`event.preventDefault()`	لغو **رفتار پیش‌فرض** مرورگر (پایین‌تر)
`event.defaultPrevented`	اگر `preventDefault()` صدا زده شده باشد `true` است
`event.stopPropagation()`	جلوگیری از رسیدن رویداد به عنصرهای بعدی (پایین‌تر)

HTML

```
<div id="outer"><span id="inner">Hi</span></div>
<script>
  document.getElementById("outer").addEventListener("click", function (e) {
    // prints: click inner outer outer
    console.log(e.type, e.target.id, e.currentTarget.id, this.id);
  });
</script>
```

کاربر روی `span` کلیک کرده، پس `target` همان `inner` است؛ ولی listener روی `outer` ثبت شده، پس `currentTarget` و `this` (در `function` معمولی) برابر `outer` هستند. در arrow function، `this` همان `this` بیرونی است (در سطح بالای اسکریپت: `window`)؛ برای همین `event.currentTarget` مطمئن‌ترین راه رسیدن به عنصرِ صاحب listener است.

انتشار رویداد: bubbling و capturing

> **حباب در آب**
>
> حبابی که ته لیوان آب ساخته می‌شود، به سمت بالا حرکت می‌کند تا به سطح برسد. رویدادها هم همین‌طورند: کلیک روی درونی‌ترین عنصر رخ می‌دهد و بعد مثل حباب **بالا می‌رود (bubble)**: به والد، والدِ والد و ... تا `document` و `window`. هر کدام از این اجداد که برای آن رویداد listener داشته باشد، listenerاش اجرا می‌شود.

مسیر کامل یک رویداد سه **مرحله (phase)** دارد:

1. **capturing** (گرفتن): از `window` به سمت پایین، از اجداد عبور می‌کند تا به والدِ هدف برسد.
2. **target**: روی خودِ عنصر هدف.
3. **bubbling** (حباب‌وار): از والدِ هدف دوباره به سمت بالا تا `window`.

```
    (1) capturing               (3) bubbling
    window                      window
      |                           ^
    document                    document
      |                           ^
    <div id="outer">            <div id="outer">
      |                           ^
      +---->  <button id="btn">  --+
              (2) target: the click happens here
```

`addEventListener` به‌طور پیش‌فرض listener را برای مرحله‌ی **bubbling** ثبت می‌کند. اگر آرگومان سوم `true` یا `{ capture: true }` باشد، listener در مرحله‌ی **capturing** اجرا می‌شود.

HTML

```
<div id="outer">
  <p id="mid"><button id="btn">Go</button></p>
</div>
<script>
  for (const id of ["outer", "mid", "btn"]) {
    document.getElementById(id).addEventListener("click", (e) => {
      console.log(id, "| target:", e.target.id, "| currentTarget:", e.currentTarget.id);
    });
  }
  document.addEventListener("click", () => console.log("document"));
  window.addEventListener("click", () => console.log("window"));
</script>
```

با کلیک روی دکمه:

```
btn | target: btn | currentTarget: btn
mid | target: btn | currentTarget: mid
outer | target: btn | currentTarget: outer
document
window
```

`target` در همه‌ی سطرها `btn` است (کلیک واقعاً روی دکمه رخ داده)، ولی `currentTarget` در هر مرحله عنصری است که listener آن لحظه رویش ثبت شده. با capture:

```HTML
<div id="outer"><button id="btn">Go</button></div>
<script>
  const outer = document.getElementById("outer");
  const btn = document.getElementById("btn");
  outer.addEventListener("click", () => console.log("outer bubble"));
  outer.addEventListener("click", () => console.log("outer capture"), true);
  btn.addEventListener("click", () => console.log("btn capture"), { capture: true });
  btn.addEventListener("click", () => console.log("btn bubble"));
</script>
<!-- click on btn -> outer capture, btn capture, btn bubble, outer bubble -->
```

> **بیشتر بدانیم**
>
> بیشتر رویدادهای پرکاربرد (`click` ، `input` ، `change` ، `submit` ، `keydown`) bubble می‌کنند، ولی چند رویداد مثل `focus` و `blur` (فعال و غیرفعال شدن یک کادر) bubble نمی‌کنند.

`stopPropagation()` : «بقیه نشنوند!»

```HTML
<div id="outer"><button id="btn">Delete</button></div>
<script>
  const btn = document.getElementById("btn");
  btn.addEventListener("click", (e) => {
    e.stopPropagation();          // the ancestors will not hear this click
    console.log("btn 1");
  });
  btn.addEventListener("click", () => console.log("btn 2")); // still runs: same element
  document.getElementById("outer").addEventListener("click", () => console.log("outer"));
</script>
<!-- click on btn -> btn 1, btn 2   ("outer" is never printed) -->
```

`stopPropagation()` جلوی رفتن رویداد به عنصرهای **بعدی** را می‌گیرد، ولی listenerهای دیگرِ **همان عنصر** هنوز اجرا می‌شوند. (متد `stopImmediatePropagation()` آن‌ها را هم متوقف می‌کند.) کاربرد معمول: دکمه‌ی «حذف» داخل یک کارت که خودِ کارت هم با کلیک باز می‌شود. زیاده‌روی نکن؛ کدهای دیگری ممکن است به رسیدن رویداد به اجداد نیاز داشته باشند.

`preventDefault()` : لغو رفتار پیش‌فرض مرورگر

بعضی رویدادها یک **رفتار پیش‌فرض (default action)** دارند که مرورگر بعد از اجرای listenerها انجام می‌دهد:

رویداد روی عنصر	رفتار پیش‌فرض
`click` روی لینک `<a href="...">`	رفتن به آدرس لینک
`submit` روی `<form>`	فرستادن فرم و بارگذاری دوباره‌ی صفحه (یا رفتن به آدرس دیگر)
`click` روی چک‌باکس	تیک خوردن یا برداشتن تیک
`keydown` در کادر متنی	تایپ شدن حرف

`event.preventDefault()` این رفتار را لغو می‌کند و بعد از آن `event.defaultPrevented` برابر `true` است. مهم‌ترین کاربرد همان است که اسلاید ۳۵ می‌گوید: **جلوگیری از reload صفحه هنگام submit فرم**، تا بتوانیم داده‌ها را با JavaScript بررسی

کنیم و نتیجه را در همان صفحه نشان دهیم:

```HTML
<form id="signup">
  <input id="username" value="ali">
  <button>Sign up</button>
</form>
<p id="msg"></p>
<script>
  document.getElementById("signup").addEventListener("submit", (e) => {
    e.preventDefault(); // stay on this page: no reload
    const name = document.getElementById("username").value;
    document.getElementById("msg").textContent = "Welcome, " + name;
  });
</script>
```

بدون `preventDefault()`، صفحه بلافاصله دوباره بارگذاری می‌شود و پیام `Welcome, ali` همان لحظه پاک می‌شود. (جزئیات کار با فرم‌ها، مثل خواندن `.value` و `.checked`، در درس «کار با فرم‌ها (Forms)» آمده است.) به همین ترتیب:

- با `preventDefault()` در listener کلیکِ یک لینک، مرورگر به آدرس لینک نمی‌رود.
- در listener کلیکِ یک چک‌باکس، مرورگر **اول** تیک را عوض می‌کند (پس داخل listener، `checked` مقدار جدید را نشان می‌دهد) و اگر `preventDefault()` صدا زده شود، **بعد از** listener آن را به حالت قبل برمی‌گرداند.

	`preventDefault()`	`stopPropagation()`
کارش	لغو رفتار پیش‌فرض مرورگر (رفتن به لینک، ارسال فرم، تیک خوردن)	توقف حرکت رویداد به عنصرهای بعدی
آیا والدها رویداد را می‌شنوند؟	**بله**	خیر
آیا رفتار پیش‌فرض انجام می‌شود؟	خیر	**بله**

> **نماد خروجی در تست‌های این درس**
> در بعضی تست‌ها، بعد از هر رویداد سطری مثل `[default prevented]` یا `[default runs]` در گزینه‌ها آمده است. این سطر را کد چاپ نمی‌کند؛ نشان می‌دهد که آیا رفتار پیش‌فرض مرورگر برای آن رویداد لغو شد (`prevented`) یا انجام می‌شود (`runs`).

Event delegation (واگذاری رویداد)

فرض کن یک لیست ۱۰۰ آیتمی داری، یا آیتم‌ها بعداً با JavaScript اضافه می‌شوند. به‌جای اینکه به هر آیتم یک listener بدهی، **یک listener روی والد مشترک** می‌گذاری و چون کلیک‌ها bubble می‌کنند، همه به والد می‌رسند؛ بعد با `event.target` می‌فهمی روی کدام فرزند کلیک شده است. به این الگو **event delegation** می‌گویند.

HTML

```
<ul id="todo">
  <li>Buy milk</li>
  <li>Read chapter 2</li>
</ul>
<button id="add">Add</button>
<script>
  const todo = document.getElementById("todo");
  todo.addEventListener("click", (e) => {
    if (e.target.matches("li")) {            // was an <li> clicked?
      console.log("done:", e.target.textContent);
    }
  });
  document.getElementById("add").addEventListener("click", () => {
    const li = document.createElement("li");
    li.textContent = "New task";
    todo.append(li);                         // no new listener needed!
  });
</script>
```

کلیک روی `Buy milk` چاپ می‌کند `done: Buy milk`، و بعد از زدن Add، کلیک روی آیتم تازه هم چاپ می‌کند `done: New task`، بدون اینکه برای آن listener جدیدی ثبت کرده باشیم.

- `element.matches(selector)`: آیا این عنصر با selector جور است؟ (`true` یا `false`)
- `element.closest(selector)`: نزدیک‌ترین عنصر جور با selector، از **خودِ عنصر** شروع می‌کند و به سمت اجداد بالا می‌رود؛ اگر چیزی پیدا نشود `null` می‌دهد.

اگر داخل `<li>` عنصرهای دیگری باشد، `target` ممکن است همان عنصر داخلی باشد؛ آن وقت `matches("li")` برابر `false` می‌شود و باید از `closest` استفاده کنی:

HTML

```
<ul id="cart">
  <li><b>Milk</b> (2) <button class="remove">x</button></li>
  <li><b>Bread</b> (1) <button class="remove">x</button></li>
</ul>
<script>
  document.getElementById("cart").addEventListener("click", (e) => {
    const removeBtn = e.target.closest(".remove");
    if (!removeBtn) return;                  // the click was not on a remove button
    const item = removeBtn.closest("li");
    console.log("removing", item.querySelector("b").textContent);
    item.remove();                           // see the lesson on removing elements
  });
</script>
```

در delegation، `event.target` فرزندی است که کلیک شده و `event.currentTarget` همان والدی است که listener روی آن ثبت شده. مزایا: listenerهای کمتر (حافظه‌ی کمتر)، کار کردن برای عنصرهایی که بعداً اضافه می‌شوند، و کد ساده‌تر.

نکته

- برای رسیدن به عنصرِ صاحب listener از `event.currentTarget` استفاده کن و برای عنصری که واقعاً کلیک شده از `event.target`.
- در delegation، `closest(...)` معمولاً از `matches(...)` امن‌تر است، چون کلیک روی عنصرهای داخلی را هم درست تشخیص می‌دهد.

دام تستی

- `preventDefault()` جلوی bubbling را نمی‌گیرد و `stopPropagation()` رفتار پیش‌فرض را لغو نمی‌کند؛ دو کار کاملاً جدا هستند.
- `stopPropagation()` listenerهای دیگرِ **همان عنصر** را متوقف نمی‌کند.
- listener بدون آرگومان سوم در مرحله‌ی bubbling اجرا می‌شود، پس listener فرزند **قبل** از listener والد اجرا می‌شود؛ ولی listenerهای capture روی اجداد از همه زودترند.
- `return false` در listener `addEventListener` هیچ اثری ندارد؛ فقط در handlerهای `onclick` (attribute یا property) رفتار پیش‌فرض را لغو می‌کند. همیشه از `preventDefault()` استفاده کن.
- کلیک روی دکمه‌ی داخل فرم، علاوه بر `click` روی دکمه، رویداد `submit` را هم روی فرم ایجاد می‌کند.

جمع‌بندی

- شیء event: `type`، `target` (عنصر اصلی)، `currentTarget` (صاحب listener)، `key`، `defaultPrevented`.
- مسیر رویداد: capturing (پایین رفتن) ← target ← bubbling (بالا رفتن تا `window`)؛ پیش‌فرض `addEventListener` مرحله‌ی bubbling است و با `true` یا `{ capture: true }` مرحله‌ی capturing.
- `stopPropagation()` حرکت رویداد را متوقف می‌کند؛ `preventDefault()` رفتار پیش‌فرض (لینک، submit و reload، تیک) را لغو می‌کند.
- delegation: یک listener روی والد + `e.target.matches(...)` یا `closest(...)`؛ برای عنصرهای بعدی هم کار می‌کند.

تست‌های تمرینی

۲۲ تست · شماره‌ی ۷۹۲ تا ۸۱۳

۷۹۲ آسان

مرورگر هنگام اجرای یک event listener، چه چیزی را به‌عنوان اولین آرگومان به آن می‌دهد؟

۱. خودِ عنصری که listener روی آن ثبت شده است

۲. شیء رویداد (event object) که اطلاعاتی مثل `type` و `target` دارد

۳. رشته‌ای با نام رویداد، مثلاً `"click"`

۴. هیچ چیز؛ پارامتر اول همیشه `undefined` است

۷۹۳ آسان

در مرحله‌ی bubbling، رویداد به کدام سمت حرکت می‌کند؟

۱) از `window` به سمت پایین، تا عنصر هدف

۲) فقط روی خودِ عنصر هدف می‌ماند و به هیچ عنصر دیگری نمی‌رسد

۳) از عنصر هدف به سمت والدها و اجداد، تا `document` و `window`

۴) از عنصر هدف به سمت فرزندانش

۷۹۴ سخت

کاربر روی دکمه کلیک می‌کند. خروجی Console کدام است؟

HTML

```
<div id="outer"><p id="mid"><button id="btn">Go</button></p></div>
```

JavaScript

```
for (const id of ["outer", "mid", "btn"]) {
  document.getElementById(id).addEventListener("click", (e) => {
    console.log(id, e.target.id, e.currentTarget.id);
  });
}
document.addEventListener("click", () => console.log("document"));
window.addEventListener("click", () => console.log("window"));
```

۱) `window` `document` `outer btn outer` `mid btn mid` `btn btn btn`

۲) `btn btn btn` `mid btn mid` `outer btn outer` `document` `window`

۳) `btn btn btn` `mid mid mid` `outer outer outer` `document` `window`

۴) `btn btn btn`

۷۹۵ سخت

کاربر روی دکمه کلیک می‌کند. خروجی Console کدام است؟

```HTML
<div id="outer"><button id="btn">Go</button></div>
```

```JavaScript
const outer = document.getElementById("outer");
const btn = document.getElementById("btn");
outer.addEventListener("click", () => console.log("outer bubble"));
outer.addEventListener("click", () => console.log("outer capture"), true);
btn.addEventListener("click", () => console.log("btn capture"), { capture: true });
btn.addEventListener("click", () => console.log("btn bubble"));
```

۱) `btn capture` `btn bubble` `outer bubble` `outer capture`

۲) `outer bubble` `outer capture` `btn capture` `btn bubble`

۳) `btn bubble` `outer bubble` `outer capture` `btn capture`

۴) `outer capture` `btn capture` `btn bubble` `outer bubble`

۷۹۶ متوسط

کاربر روی متن `Hi` (عنصر `span`) کلیک می‌کند. خروجی Console کدام است؟

```HTML
<div id="outer"><span id="inner">Hi</span></div>
```

```JavaScript
document.getElementById("outer").addEventListener("click", function (e) {
  console.log(e.target.id, e.currentTarget.id, this.id);
});
```

۱) `outer outer outer`

۲) `inner inner outer`

۳) `inner outer outer`

۴) `inner outer undefined`

۷۹۷ متوسط

کاربر روی متن `Hi` کلیک می‌کند. خروجی Console کدام است؟

```HTML
<div id="outer"><span id="inner">Hi</span></div>
```

```JavaScript
document.getElementById("outer").addEventListener("click", (e) => {
  console.log(this === window, e.currentTarget.id, e.target.id);
});
```

۱) `false outer inner`

۲) `true outer inner`

۳) `true inner inner`

۴) `false inner outer`

۷۹۸ متوسط

کاربر روی دکمه کلیک می‌کند. خروجی Console کدام است؟

HTML

```
<div id="outer"><button id="btn">Go</button></div>
```

JavaScript

```
document.getElementById("outer").addEventListener("click", () => console.log("outer"));
const btn = document.getElementById("btn");
btn.addEventListener("click", (e) => {
  e.stopPropagation();
  console.log("btn 1");
});
btn.addEventListener("click", () => console.log("btn 2"));
```

۱) `btn 1` `btn 2`

۲) `btn 1`

۳) `btn 1` `btn 2` `outer`

۴) `btn 1` `outer`

۷۹۹ متوسط

رویداد `submit` روی فرم زیر رخ می‌دهد. در گزینه‌ها، سطر `[default prevented]` یعنی رفتار پیش‌فرض مرورگر (ارسال فرم و reload صفحه) لغو شد و `[default runs]` یعنی انجام می‌شود. خروجی کدام است؟

HTML

```
<form id="f"><input id="u" value="ali"><button>Send</button></form>
```

JavaScript

```
document.getElementById("f").addEventListener("submit", (e) => {
  e.preventDefault();
  console.log("submitted", e.defaultPrevented);
});
```

۱) `submitted false` `[default runs]`

۲) `submitted true` `[default prevented]`

۳) `submitted true` `[default runs]`

۴) `submitted false` `[default prevented]`

۸۰۰ متوسط

کاربر اول روی لینک Help و بعد روی لینک Contact کلیک می‌کند (هر دو لینک به بخشی از همین صفحه اشاره می‌کنند). سپس در Console دستور `console.log("hash:", location.hash)` را اجرا می‌کنیم. خروجی کامل کدام است؟

HTML

```
<a id="help" href="#help">Help</a>
<a id="contact" href="#contact">Contact</a>
```

JavaScript

```
document.getElementById("help").addEventListener("click", (e) => {
  e.preventDefault();
  console.log("help clicked");
});
document.getElementById("contact").addEventListener("click", () => {
  console.log("contact clicked");
});
```

۱) `help clicked` `contact clicked` `hash: #help`

۲) `contact clicked` `hash: #contact`

۳) `help clicked` `contact clicked` `hash:`

۴) `help clicked` `contact clicked` `hash: #contact`

۸۰۱ سخت

کاربر روی چک‌باکس (که در ابتدا تیک ندارد) کلیک می‌کند و بعد در Console مقدار `agree.checked` را چاپ می‌کنیم. سطر `[default prevented]` یعنی رفتار پیش‌فرض مرورگر برای این کلیک لغو شد. خروجی کامل کدام است؟

HTML

```
<input type="checkbox" id="agree">
```

JavaScript

```
const agree = document.getElementById("agree");
agree.addEventListener("click", (e) => {
  console.log("inside:", agree.checked);
  e.preventDefault();
});
```

۱) `inside: false` `[default prevented]` `after: false`

۲) `inside: true` `[default prevented]` `after: true`

۳) `inside: true` `[default prevented]` `after: false`

۴) `inside: false` `[default prevented]` `after: true`

۸۰۲ متوسط

کاربر روی لینک کلیک می‌کند. سطر `[default prevented]` یعنی رفتار پیش‌فرض (رفتن به آدرس لینک) لغو شد و `[default runs]` یعنی انجام می‌شود. خروجی کدام است؟

HTML

```
<div id="box"><a id="lnk" href="#x">Go</a></div>
```

JavaScript

```
document.getElementById("box").addEventListener("click", (e) => {
  console.log("box sees", e.defaultPrevented);
});
document.getElementById("lnk").addEventListener("click", (e) => e.preventDefault());
```

۱) `box sees true` `[default prevented]`

۲) `[default prevented]`

۳) `box sees false` `[default prevented]`

۴) `box sees true` `[default runs]`

۸۰۳ آسان

کدام جمله درباره‌ی `preventDefault()` و `stopPropagation()` درست است؟

۱) `preventDefault()` رفتار پیش‌فرض مرورگر را لغو می‌کند ولی جلوی bubbling را نمی‌گیرد؛ `stopPropagation()` جلوی bubbling را می‌گیرد ولی رفتار پیش‌فرض را لغو نمی‌کند.

۲) `preventDefault()` جلوی bubbling را می‌گیرد و `stopPropagation()` رفتار پیش‌فرض را لغو می‌کند.

۳) هر دو دقیقاً یک کار را انجام می‌دهند و فقط نامشان فرق دارد.

۴) `preventDefault()` listener را حذف می‌کند تا دوباره اجرا نشود.

۸۰۴ متوسط

کاربر ابتدا روی دکمه‌ی Add کلیک می‌کند، سپس روی آیتم تازه‌ی `C`، و در آخر روی فضای خالی خودِ لیست (روی `<ul>` و نه روی هیچ `<li>`ی). خروجی Console کدام است؟

HTML

```
<ul id="list"><li>A</li><li>B</li></ul>
<button id="add">Add</button>
```

JavaScript

```
const list = document.getElementById("list");
list.addEventListener("click", (e) => {
  if (e.target.matches("li")) {
    console.log("clicked", e.target.textContent, e.currentTarget.id);
  }
});
document.getElementById("add").addEventListener("click", () => {
  const li = document.createElement("li");
  li.textContent = "C";
  list.append(li);
});
```

۱) (بدون خروجی)

۲) `clicked C list`

۳) `clicked undefined list` `clicked C list`

۴) `clicked C C`

۸۰۵ سخت

کاربر روی کلمه‌ی `Milk` (که داخل تگ `<b>` است) کلیک می‌کند. خروجی Console کدام است؟

HTML

```
<ul id="list"><li><b>Milk</b> (2)</li><li>Bread</li></ul>
```

JavaScript

```
document.getElementById("list").addEventListener("click", (e) => {
  const li = e.target.closest("li");
  console.log(e.target.tagName, e.target.matches("li"), li.textContent);
});
```

۱) `LI true Milk (2)`

۲) `B true Milk (2)`

۳) `UL false Milk (2)`

۴) `B false Milk (2)`

۸۰۶ متوسط

کدام گزینه از مزایای event delegation نیست؟

۱) به‌جای ده‌ها listener فقط یک listener لازم است.

۲) برای عنصرهایی هم که بعداً به والد اضافه می‌شوند، بدون ثبت listener جدید کار می‌کند.

۳) باعث می‌شود رویداد دیگر bubble نکند و به `document` نرسد.

۴) کد ساده‌تر می‌شود و حافظه‌ی کمتری مصرف می‌شود.

۸۰۷ آسان

کاربر حرف `a` را در کادر تایپ می‌کند و بعد از کادر بیرون می‌رود (یک `input` و سپس یک `change`). خروجی Console کدام است؟

HTML

```
<input id="box">
```

JavaScript

```
const box = document.getElementById("box");
function log(e) {
  console.log(e.type, e.target.id);
}
box.addEventListener("input", log);
box.addEventListener("change", log);
```

۱) `input box` `change box`

۲) `change box` `input box`

۳) `log box` `log box`

۴) `input undefined` `change undefined`

۸۰۸ سخت

کاربر روی دکمه کلیک می‌کند. خروجی Console کدام است؟

HTML

```
<div id="mid"><button id="btn">Go</button></div>
```

JavaScript

```
document.addEventListener("click", () => console.log("doc capture"), true);
document.addEventListener("click", () => console.log("doc bubble"));
document.getElementById("mid").addEventListener("click", (e) => {
  e.stopPropagation();
  console.log("mid");
});
document.getElementById("btn").addEventListener("click", () => console.log("btn"));
```

۱) `btn` `mid` `doc bubble`

۲) `doc capture` `btn` `mid` `doc bubble`

۳) `btn` `mid`

۴) `doc capture` `btn` `mid`

۸۰۹ متوسط

کدام مورد یک «رفتار پیش‌فرض» (default action) مرورگر نیست که بتوان آن را با `preventDefault()` لغو کرد؟

۱) اجرای listenerهای همان رویداد روی عنصرهای والد

۲) رفتن به آدرس یک لینک بعد از کلیک روی آن

۳) ارسال فرم و بارگذاری دوباره‌ی صفحه بعد از submit

۴) تیک خوردن چک‌باکس بعد از کلیک روی آن

۸۱۰ سخت

کاربر اول روی لینک One و بعد روی لینک Two کلیک می‌کند و سپس `console.log(JSON.stringify(location.hash))` را اجرا می‌کنیم. سطرهای `[default ...]` نشان می‌دهند رفتار پیش‌فرض هر کلیک (رفتن به آدرس لینک) لغو شد (`prevented`) یا انجام شد (`runs`). خروجی کامل کدام است؟

HTML

```
<a id="a1" href="#one">One</a>
<a id="a2" href="#two">Two</a>
```

JavaScript

```
document.getElementById("a1").addEventListener("click", () => false);
document.getElementById("a2").onclick = () => false;
```

۱) `""` `[default prevented]` `[default prevented]`

۲) `"#two"` `[default runs]` `[default runs]`

۳) `"#one"` `[default prevented]` `[default runs]`

۴) `"#two"` `[default runs]` `[default prevented]`

۸۱۱ متوسط

کاربر روی دکمه‌ی Send (داخل فرم) کلیک می‌کند. خروجی Console کدام است؟

HTML

```
<form id="f"><input id="u" value="ali"><button id="send">Send</button></form>
```

JavaScript

```
document.getElementById("f").addEventListener("submit", (e) => {
  e.preventDefault();
  console.log("submit on", e.target.id);
});
document.getElementById("send").addEventListener("click", (e) => {
  console.log("click on", e.target.id);
});
```

۱) `click on send`

۲) `submit on f` `click on send`

۳) `click on send` `submit on send`

۴) `click on send` `submit on f`

۸۱۲ آسان

در listener، event delegation روی یک `<ul>` ثبت شده و کاربر روی یکی از `<li>`های آن کلیک می‌کند. `event.target` و `event.currentTarget` به ترتیب کدام‌اند؟

۱) `<li>` کلیک‌شده — `<ul>`

۲) `<ul>` — `<li>` کلیک‌شده

۳) هر دو `<ul>`

۴) هر دو `<li>` کلیک‌شده

۸۱۳ سخت

با HTML زیر:

```HTML
<ul id="list"><li>A</li><li>B</li></ul>
```

کد زیر اجرا می‌شود. کاربر ابتدا روی آیتم `A` و سپس روی فضای خالی خودِ لیست (روی `<ul>`، نه روی هیچ `<li>`ی) کلیک می‌کند. با کلیک دوم، کدام خط خطا می‌دهد؟

```JavaScript
1  const list = document.getElementById("list");
2  list.addEventListener("click", (e) => {
3    const li = e.target.closest("li");
4    console.log("clicked on", e.target.tagName);
5    console.log("item:", li.textContent);
6  });
```

۱) خط ۳
۲) خط ۴
۳) خط ۵
۴) هیچ خطایی رخ نمی‌دهد

۲-۷ تغییر متن، HTML و استایل عناصر

منبع: JS in HTML، اسلایدهای ۱۹ تا ۲۲

درسنامه

حالا که می‌توانیم عنصرها را پیدا کنیم و به رویدادها واکنش نشان دهیم، وقت آن است که **صفحه را تغییر دهیم**. اسلاید ۲۲ (Manipulating The DOM)، یعنی دستکاری DOM) این توانایی‌ها را خلاصه می‌کند:

- DOM صفحه را به شکل **درختی از اشیاء** نمایش می‌دهد.
- JavaScript می‌تواند **متن، HTML و استایل** را به‌صورت پویا (dynamic، یعنی هنگام اجرا و بدون reload صفحه) تغییر دهد. (موضوع همین درس)
- می‌توان عنصرها را در لحظه **ساخت، درج و حذف** کرد. (درس‌های «ساختن و افزودن عناصر جدید» و «حذف و جایگزینی عناصر»)
- attributeهایی مثل `id` ، `class` و `src` را به‌راحتی می‌توان تغییر داد. (اینجا مختصر، و کامل در درس «ویژگی‌ها (Attributes) و کلاس‌ها»)
- متدهایی مثل `getElementById` و `querySelector` به پیدا کردن عنصرها کمک می‌کنند. (درس «پیدا کردن عناصر»)
- دستکاری DOM همان چیزی است که صفحه‌های وب را **تعاملی و پویا** می‌کند.

تغییر متن و HTML: `textContent` ، `innerText` و `innerHTML`

property	هنگام خواندن	هنگام نوشتن
`textContent`	همه‌ی متن عنصر و نوادگانش، بدون تگ‌ها (حتی متن‌های مخفی)	متن ساده؛ تگ‌ها **تفسیر نمی‌شوند** و عیناً به‌صورت متن نمایش داده می‌شوند
`innerText`	متنی که **واقعاً روی صفحه دیده می‌شود** (به CSS بستگی دارد و متن مخفی را حساب نمی‌کند)	متن ساده، مثل `textContent`
`innerHTML`	کد HTML **داخل** عنصر، به‌صورت رشته	رشته به‌عنوان HTML **تفسیر (parse)** می‌شود و عنصرهای واقعی ساخته می‌شوند
`outerHTML`	کد HTML **خودِ عنصر** همراه با تگ خودش	(در این درس فقط برای خواندن و دیدن نتیجه از آن استفاده می‌کنیم)

خواندن:

```HTML
<p id="p">Hello <b>big</b> world</p>
<script>
  const p = document.getElementById("p");
  console.log(p.textContent); // "Hello big world"
  console.log(p.innerHTML);   // "Hello <b>big</b> world"
  console.log(p.outerHTML);   // "<p id="p">Hello <b>big</b> world</p>"
</script>
```

نوشتن:

```HTML
<div id="a"></div>
<div id="b"></div>
<script>
  const a = document.getElementById("a");
  const b = document.getElementById("b");
  a.innerHTML = "<b>Hi</b> there";   // a bold "Hi", then " there"
  b.textContent = "<b>Hi</b> there"; // the tags are shown as text
  console.log(a.children.length);    // 1 (a real <b> element)
  console.log(b.children.length);    // 0 (just text)
  console.log(b.innerHTML);          // "&lt;b&gt;Hi&lt;/b&gt; there"
</script>
```

در `div` دوم، کاربر دقیقاً نوشته‌ی `<b>Hi</b> there` را روی صفحه می‌بیند؛ مرورگر برای اینکه این علامت‌ها تگ حساب نشوند، آن‌ها را در کد HTML به شکل `<` و `>` نگه می‌دارد.

نوشتن، همه‌چیز را جایگزین می‌کند. هر سه property هنگام نوشتن، کل محتوای قبلی عنصر (از جمله عنصرهای فرزند) را پاک می‌کنند: اگر `card` یک `<h2>` و یک `<p>` داشته باشد، بعد از `card.textContent = "Empty"` فقط یک گره‌ی متنی در آن می‌ماند. هر مقداری هم که بدهی به رشته تبدیل می‌شود: `s.textContent = 5 + 5` متن `"10"` را می‌نویسد.

`innerText` چه فرقی با `textContent` دارد؟

```HTML
<p id="p">Hi<span style="display: none"> secret</span></p>
<script>
  const p = document.getElementById("p");
  console.log(p.textContent); // "Hi secret"  (hidden text is included)
  console.log(p.innerText);   // "Hi"         (only what is visible on the page)
</script>
```

`innerText` به چیدمان و CSS صفحه وابسته است (برای همین کمی کندتر است)، ولی `textContent` فقط درخت DOM را می‌خواند. برای **نوشتن** یک متن ساده (مثل `div.innerText = "Hello DOM!"` در مثال اسلاید ۲۴) هر دو تقریباً یکسان کار می‌کنند.

دام `innerHTML +=` (پیش‌زمینه)

`a += b` یعنی `a = a + b`. پس `box.innerHTML += "<p>New</p>"` اول کل HTML داخل `box` را به رشته تبدیل می‌کند، رشته‌ی جدید را به آن می‌چسباند و بعد **همه‌چیز را از نو parse می‌کند**. نتیجه: عنصرهای قبلی نابود و نسخه‌های تازه‌ای جایشان ساخته می‌شوند؛ listenerهای آن‌ها و متغیرهایی که به آن‌ها اشاره می‌کردند از دست می‌روند (متنی هم که کاربر در کادرها تایپ کرده بود پاک می‌شود).

```HTML
<div id="box"><button id="btn">Click</button></div>
<script>
  const btn = document.getElementById("btn");
  btn.addEventListener("click", () => console.log("clicked"));
  document.getElementById("box").innerHTML += "<p>New</p>";
  console.log(document.getElementById("btn") === btn); // false: a brand-new button
  // clicking the button now prints nothing: the new button has no listener
</script>
```

راه بهتر برای اضافه کردن: ساختن عنصر با `createElement` و اضافه کردنش با `append` (درس «ساختن و افزودن عناصر جدید»)، یا `insertAdjacentHTML("beforeend", "<p>New</p>")` که چیزهای قبلی را دست نمی‌زند.

امنیت: `innerHTML` و حمله‌ی XSS

اگر رشته‌ای که به `innerHTML` می‌دهی از **کاربر** آمده باشد (از یک کادر ورودی یا از آدرس صفحه)، مهاجم می‌تواند به‌جای متن، کد HTML و JavaScript بفرستد تا در صفحه‌ی کاربران دیگر اجرا شود. به این حمله **XSS** (Cross-Site Scripting، یعنی اجرای اسکریپت مهاجم در صفحه‌ی سایت دیگر) می‌گویند. تگ `<script>`ی که با `innerHTML` درج شود اجرا نمی‌شود، ولی ترفندهای دیگر کار می‌کنند؛ مثلاً یک `<img>` با آدرس خراب که کد `onerror` آن هنگام شکست بارگذاری تصویر اجرا می‌شود:

JavaScript

```
const comment = '<img src="x" onerror="alert(\'hacked\')">'; // typed by an attacker
box.innerHTML = comment;   // DANGEROUS: in a real browser the onerror code runs
box.textContent = comment; // safe: shown as plain text
```

قانون ساده: **متنی که از کاربر می‌آید را با `textContent` در صفحه بگذار**، نه با `innerHTML`.

تغییر استایل: `element.style`

در مثال اسلاید ۱۹ دیدیم `title.style.color = 'red'` رنگ عنوان را قرمز می‌کند. قواعد:

- `element.style.property = "value"` یک **استایل inline** می‌سازد، یعنی attribute `style` خودِ عنصر را تغییر می‌دهد.
- نام propertyهای CSS که خط تیره دارند، در JavaScript **camelCase** می‌شوند: `background-color` می‌شود `backgroundColor` و `font-size` می‌شود `fontSize`. (راه دیگر: `el.style["background-color"]`.)
- مقدارها **رشته** هستند و برای اندازه‌ها **واحد** لازم است: `"20px"`، `"1.5rem"`. مقدار نامعتبر (مثلاً عدد `20` بدون واحد برای `fontSize`) بی‌صدا نادیده گرفته می‌شود. (این قاعده برای صفحه‌ی استانداردی است که با `<!DOCTYPE html>` شروع شده؛ در «حالت سازگاری» یا quirks mode، مرورگر بعضی عددهای بی‌واحد را قبول می‌کند. یک دلیل دیگر برای اینکه همیشه DOCTYPE بنویسی.)
- `el.style.color = ""` آن استایل inline را حذف می‌کند.
- خواندن `el.style.color` **فقط استایل inline** را می‌بیند، نه قاعده‌هایی که در `<style>` یا فایل CSS نوشته شده‌اند. برای دیدن استایل نهایی که واقعاً اعمال شده، از `getComputedStyle(el)` استفاده کن.

HTML

```
<style>
  #p { color: green; }
</style>
<p id="p">Hi</p>
<script>
  const p = document.getElementById("p");
  console.log(p.style.color);              // ""  (the CSS rule is not inline)
  console.log(getComputedStyle(p).color); // "rgb(0, 128, 0)"
  p.style.backgroundColor = "yellow";
  p.style.fontSize = "20px";
  console.log(p.getAttribute("style"));   // "background-color: yellow; font-size: 20px;"
</script>
```

دقت کن که `p.style.background-color = "red"` (با خط تیره) از نظر JavaScript یعنی `p.style.background` منهای `color`، که سمت چپ انتساب نمی‌تواند باشد؛ این یک **خطای نحوی (SyntaxError)** است و چون خطاهای نحوی پیش از اجرا پیدا

می‌شوند، **هیچ خطی از آن اسکریپت** اجرا نمی‌شود. برای تغییرهای زیاد در ظاهر، بهتر است یک کلاس CSS را اضافه یا حذف کنی (با `classList`، در درس «ویژگی‌ها (Attributes) و کلاس‌ها»).

`className` و تغییر `id`، `class` و `src` (اسلاید ۲۲)

- `el.className` کل attribute `class` را به‌صورت یک رشته برمی‌گرداند، مثلاً `"note big"`. مقداردهی به آن **همه‌ی** کلاس‌های قبلی را جایگزین می‌کند و برای اضافه کردن باید با یک فاصله چسباند: `el.className += " bold"`. چون `class` در JavaScript کلمه‌ی رزروشده است (برای تعریف کلاس)، نام این property در DOM `className` انتخاب شده است.
- `el.id = "..."` و `img.src = "..."` هم attributeها را مستقیماً عوض می‌کنند.

HTML

```
<img id="pic" src="a.png" class="thumb">
<script>
  const pic = document.getElementById("pic");
  pic.id = "photo";
  pic.className = "big";
  pic.src = "b.png";
  console.log(pic.outerHTML); // <img id="photo" src="b.png" class="big">
  console.log(document.getElementById("pic") === null); // true: old id is gone
</script>
```

مثال Button Action، از نگاه این درس

دو خطِ داخل listener در اسلاید ۱۹ دقیقاً همین ابزارها هستند:

JavaScript

```
const title = document.querySelector('#title');
title.textContent = 'Goodbye World'; // change the text
title.style.color = 'red';           // change the style (inline)
```

بعد از کلیک، عنوان در DOM به این شکل درمی‌آید و مرورگر آن را فوراً با متن جدید و به رنگ قرمز نشان می‌دهد (اسلاید ۲۱):

HTML

```
<h1 id="title" style="color: red;">Goodbye World</h1>
```

نکته

- برای گذاشتن **متن** (به‌خصوص متنی که از کاربر آمده) از `textContent` استفاده کن؛ `innerHTML` را فقط وقتی به کار ببر که واقعاً می‌خواهی HTML مطمئنی را درج کنی.
- برای خواندن استایل نهایی یک عنصر، `getComputedStyle(el)` و برای نوشتن استایل، `el.style.prop` (camelCase، مقدار رشته‌ای با واحد).

⚠ دام تستی

- `textContent = "<b>Hi</b>"` تگ‌ها را **متن** می‌کند، ولی `innerHTML = "<b>Hi</b>"` یک عنصر `<b>` واقعی می‌سازد.
- `innerHTML +=` کل محتوا را از نو می‌سازد: listenerها و ارجاع‌های قبلی از بین می‌روند.
- `el.style.color` رنگی را که در فایل CSS تعیین شده نشان نمی‌دهد و `""` برمی‌گرداند.
- در صفحه‌ی استاندارد، `el.style.fontSize = 20` (بدون واحد) نادیده گرفته می‌شود؛ `"20px"` درست است.
- `el.style.background-color = ...` خطای نحوی است؛ `backgroundColor` درست است.
- `className = "x"` همه‌ی کلاس‌های قبلی را پاک می‌کند.

✓ جمع‌بندی

- `textContent` (متن خام، شامل متن مخفی)، `innerText` (متن قابل دیدن)، `innerHTML` (HTML تفسیرشده).
- نوشتن در این propertyها کل محتوای قبلی را جایگزین می‌کند؛ `innerHTML +=` همه‌چیز را از نو می‌سازد.
- ورودی کاربر را با `innerHTML` درج نکن (خطر XSS)؛ از `textContent` استفاده کن.
- `el.style.camelCase = "value"` استایل inline می‌سازد؛ خواندن استایل نهایی با `getComputedStyle` .
- `className` رشته‌ی کلاس‌هاست و مقداردهی‌اش همه را جایگزین می‌کند؛ `id` و `src` هم مستقیم قابل تغییرند.

✓ تست‌های تمرینی

۲۱ تست · شماره‌ی ۸۱۴ تا ۸۳۴

۸۱۴ آسان

خروجی کد زیر کدام است؟

HTML

```
<div id="a"></div>
<div id="b"></div>
```

JavaScript

```
const a = document.getElementById("a");
const b = document.getElementById("b");
a.innerHTML = "<b>Hi</b> there";
b.textContent = "<b>Hi</b> there";
console.log(a.children.length, b.children.length);
```

۱) `1 1`　۲) `0 0`　۳) `1 0`　۴) `0 1`

۸۱۵ متوسط

خروجی کد زیر کدام است؟

HTML
```
<p id="p">Hello <b>big</b> world</p>
```

JavaScript
```
const p = document.getElementById("p");
console.log(p.textContent);
console.log(p.innerHTML);
```

۱) `Hello <b>big</b> world` `Hello big world`

۲) `Hello big world` `Hello <b>big</b> world`

۳) `Hello big world` `Hello big world`

۴) `Hello` `Hello <b>big</b> world`

۸۱۶ آسان

خروجی کد زیر کدام است؟

HTML
```
<div id="card"><h2>Title</h2><p>Body</p></div>
```

JavaScript
```
const card = document.getElementById("card");
card.textContent = "Empty";
console.log(card.children.length, card.childNodes.length, card.innerHTML);
```

۱) `0 1 Empty`

۲) `2 3 Empty`

۳) `2 1 Empty`

۴) `0 1 <h2>Title</h2><p>Body</p>Empty`

۸۱۷ سخت

کد زیر اجرا می‌شود و سپس کاربر روی دکمه‌ی Click کلیک می‌کند. خروجی کامل Console کدام است؟

HTML

```
<div id="box"><button id="btn">Click</button></div>
```

JavaScript

```
const btn = document.getElementById("btn");
btn.addEventListener("click", () => console.log("clicked"));
const box = document.getElementById("box");
box.innerHTML += "<p>New</p>";
console.log(document.getElementById("btn") === btn);
```

۱) `true` `clicked`

۲) `false` `clicked`

۳) `false`

۴) `true`

۸۱۸ سخت

خروجی کد زیر کدام است؟

HTML

```
<div id="b"></div>
```

JavaScript

```
const b = document.getElementById("b");
b.textContent = "<b>Hi</b>";
console.log(b.textContent, "|", b.innerHTML);
```

۱) `Hi | <b>Hi</b>`

۲) `<b>Hi</b> | <b>Hi</b>`

۳) `<b>Hi</b> | <b>Hi</b>`

۴) `Hi | Hi`

۸۱۹ آسان

چرا نباید متنی را که کاربر وارد کرده است مستقیماً با `innerHTML` در صفحه قرار داد؟

۱) چون `innerHTML` رشته را به‌عنوان HTML تفسیر می‌کند و مهاجم می‌تواند کدی (مثلاً از طریق `onerror` یک تصویر) را در صفحه اجرا کند؛ برای متن کاربر از `textContent` استفاده می‌کنیم.

۲) چون `innerHTML` همه‌ی حروف متن را به حروف بزرگ تبدیل می‌کند.

۳) چون `innerHTML` فقط روی عنصر `<div>` کار می‌کند.

۴) چون `innerHTML` متن را فقط در Console نشان می‌دهد و چیزی در صفحه نمایش نمی‌دهد.

۸۲۰ متوسط

خروجی کد زیر کدام است؟

HTML

```
<p id="p">Hi</p>
```

JavaScript

```
const p = document.getElementById("p");
p.style.color = "red";
p.style.backgroundColor = "yellow";
p.style.fontSize = "20px";
console.log(p.getAttribute("style"));
console.log(p.style.color, p.style.fontSize, p.style.backgroundColor);
```

۱) `color: red; backgroundColor: yellow; fontSize: 20px;` `red 20px yellow`

۲) `color: red; background-color: yellow; font-size: 20px;` `red 20px yellow`

۳) `null` `red 20px yellow`

۴) `color: red; background-color: yellow; font-size: 20px;` `red 20 yellow`

۸۲۱ سخت

در یک صفحه‌ی استاندارد (که با `<!DOCTYPE html>` شروع می‌شود)، خروجی کد زیر کدام است؟

HTML

```
<p id="p">Hi</p>
```

JavaScript

```
const p = document.getElementById("p");
p.style.fontSize = 20;
console.log(JSON.stringify(p.style.fontSize));
p.style.fontSize = "20px";
console.log(JSON.stringify(p.style.fontSize));
```

۱) `"20"` `"20px"`

۲) `"20px"` `"20px"`

۳) `""` `"20px"`

۴) `20` `"20px"`

۸۲۲ سخت

خروجی کد زیر کدام است؟

HTML

```
<p id="p">Hi</p>
```

JavaScript

```
const p = document.getElementById("p");
console.log("start");
p.style.background-color = "red";
console.log("end");
```

۱) `start` `Uncaught SyntaxError`

۲) `start` `end`

۳) `start` `Uncaught ReferenceError`

۴) `Uncaught SyntaxError`

۸۲۳ متوسط

خروجی کد زیر کدام است؟

HTML

```
<style>#p { color: green; }</style>
<p id="p">Hi</p>
```

JavaScript

```
const p = document.getElementById("p");
console.log(JSON.stringify(p.style.color));
console.log(getComputedStyle(p).color);
```

۱) `"green"` `rgb(0, 128, 0)`

۲) `""` `rgb(0, 128, 0)`

۳) `"green"` `green`

۴) `""` `""`

۸۲۴ متوسط

خروجی کد زیر کدام است؟

```HTML
<p id="p" class="note big">Hi</p>
```

```JavaScript
const p = document.getElementById("p");
p.className = "warning";
console.log(p.className, p.className.includes("note"));
p.className += " bold";
console.log(p.getAttribute("class"));
```

۱) `note big warning true` `note big warning bold`

۲) `warning true` `warning bold`

۳) `warning false` `warningbold`

۴) `warning false` `warning bold`

۸۲۵ آسان

چرا برای کار با کلاس‌های یک عنصر در JavaScript از `el.className` استفاده می‌کنیم و نه از `el.class`؟

۱) چون `class` در JavaScript یک کلمه‌ی رزروشده (برای تعریف کلاس) است و به همین دلیل property DOM با نام `className` تعریف شده است.

۲) چون `el.class` فقط در Node.js کار می‌کند.

۳) چون `className` فقط کلاس اولِ عنصر را برمی‌گرداند.

۴) چون attribute `class` در HTML وجود ندارد.

۸۲۶ متوسط

این کد مثال Button Action (اسلاید ۱۹) است. کاربر یک بار روی دکمه کلیک می‌کند و سپس در Console دستور `console.log(document.querySelector("#title").outerHTML)` را اجرا می‌کنیم. خروجی کدام است؟

HTML

```
<h1 id="title">Hello World</h1>
<button>CLICK ME!</button>
```

JavaScript

```
const btn = document.querySelector('button');

btn.addEventListener('click', () => {
  const title = document.querySelector('#title');
  title.textContent = 'Goodbye World';
  title.style.color = 'red';
});
```

۱) `<h1 id="title" color="red">Goodbye World</h1>`

۲) `<h1 id="title">Goodbye World</h1>`

۳) `<h1 id="title" style="color: red;">Goodbye World</h1>`

۴) `<h1 id="title" style="color: red;">Hello World</h1>`

۸۲۷ متوسط

خروجی کد زیر کدام است؟

HTML

```
<span id="s"></span>
```

JavaScript

```
const s = document.getElementById("s");
s.textContent = 5 + 5;
console.log(s.textContent, typeof s.textContent);
s.textContent = "5" + 5;
console.log(s.textContent);
```

۱) `10 number` `55`

۲) `10 string` `10`

۳) `5+5 string` `55`

۴) `10 string` `55`

۸۲۸ سخت

کد زیر **در مرورگر** اجرا می‌شود. مقدار `p.textContent` و `p.innerText` به ترتیب کدام است؟

```HTML
<p id="p">Hi<span style="display: none"> secret</span></p>
```

```JavaScript
const p = document.getElementById("p");
console.log(p.textContent);
console.log(p.innerText);
```

۱) `"Hi secret"` و `"Hi"`

۲) `"Hi"` و `"Hi secret"`

۳) هر دو `"Hi secret"`

۴) هر دو `"Hi"`

۸۲۹ آسان

کدام عبارت درباره‌ی دستکاری DOM (اسلاید ۲۲) نادرست است؟

۱) DOM صفحه را به شکل درختی از اشیاء نمایش می‌دهد.

۲) JavaScript می‌تواند متن، HTML و استایل عنصرها را به‌صورت پویا تغییر دهد.

۳) attributeهایی مثل `id`، `class` و `src` را می‌توان به‌راحتی تغییر داد.

۴) تغییراتی که JavaScript در DOM می‌دهد، در فایل HTML روی سرور هم ذخیره می‌شود.

۸۳۰ سخت

خروجی کد زیر کدام است؟

```HTML
<p id="p" style="color: blue">Hi</p>
```

```JavaScript
const p = document.getElementById("p");
p.style.fontWeight = "bold";
console.log(p.getAttribute("style"));
p.style.color = "";
console.log(p.getAttribute("style"));
```

۱) `color: blue; font-weight: bold;` `font-weight: bold;`

۲) `font-weight: bold;` `font-weight: bold;`

۳) `color: blue; font-weight: bold;` `color: ; font-weight: bold;`

۴) `color: blue; fontWeight: bold;` `fontWeight: bold;`

۸۳۱ متوسط

خروجی کد زیر کدام است؟

```HTML
<img id="pic" src="a.png" class="thumb">
```

```JavaScript
const pic = document.getElementById("pic");
pic.id = "photo";
pic.className = "big";
pic.src = "b.png";
console.log(pic.outerHTML);
console.log(document.getElementById("pic") === null);
```

۱) `<img id="pic" src="a.png" class="thumb">` `false`

۲) `<img id="photo" src="b.png" class="thumb big">` `true`

۳) `<img id="photo" src="b.png" class="big">` `true`

۴) `<img id="photo" src="b.png" class="big">` `false`

۸۳۲ سخت

خروجی کد زیر کدام است؟

```HTML
<div id="out"></div>
```

```JavaScript
const out = document.getElementById("out");
out.innerHTML = "<script>console.log('script ran')<\/script>";
console.log("children:", out.children.length);
```

۱) `script ran` `children: 1`

۲) `children: 1` `script ran`

۳) `children: 0`

۴) `children: 1`

۸۳۳ آسان

برای اینکه رنگ پس‌زمینه‌ی عنصر `box` زرد شود، در جای خالی `box.style._____ = "yellow";` کدام گزینه قرار می‌گیرد؟

۱) `background-color`

۲) `backgroundColor`

۳) `BackgroundColor`

۴) `bgColor`

۸۳۴ سخت

با HTML زیر:

```HTML
<p class="note">A</p>
<p class="note">B</p>
```

کد زیر در انتهای صفحه اجرا می‌شود. اجرای آن در کدام خط با خطا متوقف می‌شود؟

```JavaScript
1 const notes = document.getElementsByClassName("note");
2 notes[0].textContent = "First note";
3 document.querySelectorAll(".note").textContent = "All notes";
4 notes[1].style.color = "red";
5 notes.style.color = "blue";
6 console.log("done");
```

۱) خط ۲
۲) خط ۳
۳) خط ۵
۴) هیچ خطایی رخ نمی‌دهد

۲-۸ ساختن و افزودن عناصر جدید

منبع: JS in HTML، اسلایدهای ۲۳ تا ۲۶

درسنامه

تا این‌جا یاد گرفتیم عنصرهایی را که **از قبل** در صفحه هستند پیدا کنیم و متن یا ظاهرشان را عوض کنیم. اما در خیلی از صفحه‌ها باید چیزی را **از صفر** بسازیم: یک آیتم تازه در لیست کارها، یک پیام خطا زیر فرم، یک کارت محصول جدید و... . در این درس می‌بینیم JavaScript چطور یک عنصر (element) تازه می‌سازد و آن را در درخت DOM (همان مدل درختیِ صفحه که مرورگر از روی HTML می‌سازد) جا می‌دهد.

یک تشبیه ساده

ساختن عنصر جدید مثل ساختن یک قطعه‌ی لگو در دستت است. قطعه وجود دارد و رنگ و شکل دارد، ولی تا آن را به ساختمان لگو **نچسبانی** جزو ساختمان نیست و کسی آن را در ساختمان نمی‌بیند. در DOM هم `createElement` قطعه را می‌سازد و متدهایی مثل `appendChild` آن را به درخت صفحه می‌چسبانند.

چهار قدمِ ساختن یک عنصر

اسلاید ۲۳ کل کار را این‌طور خلاصه می‌کند:

1. **ساختن:** `document.createElement("tag")`
2. **گذاشتن محتوا:** `.innerText` یا `.innerHTML` (یا `.textContent`)
3. **تنظیم attributeها** (ویژگی‌های داخل تگ، مثل `id` و `class`): `.setAttribute("name", "value")`
4. **افزودن به صفحه:** `.appendChild()` یا `.append()`؛ و برای گذاشتن قبل از یک عنصر دیگر: `.insertBefore()`

و در آخر این کارها را با رویدادها (event) ترکیب می‌کنیم تا صفحه با کلیک و تایپ کاربر تغییر کند.

قدم ۱: ساختن با `createElement`

```JavaScript
const p = document.createElement("p");   // a brand-new <p> element
console.log(p.tagName);        // "P"
console.log(p.isConnected);    // false -> it is not in the page yet
console.log(p.parentNode);     // null  -> it has no parent yet
```

- ورودی فقط **نام تگ** است، بدون `<` و `>`. `createElement("div")` درست است ولی `createElement("<div>")` خطای `InvalidCharacterError` می‌دهد.
- خروجی یک شیء عنصر واقعی است (مثلاً `HTMLParagraphElement`) که همه‌ی propertyها (خاصیت‌ها) و متدهای یک عنصر معمولی را دارد.
- این عنصر فعلاً فقط در **حافظه** است: پدر ندارد، `isConnected` (آیا به سند وصل است؟) برابر `false` است و روی صفحه دیده نمی‌شود.

قدم ۲: گذاشتن محتوا

```JavaScript
const div = document.createElement("div");
div.innerText = "Hello DOM!";          // plain text (this is what slide 24 uses)
div.textContent = "Hello DOM!";        // plain text as well
div.innerHTML = "<b>Hello</b> DOM!";   // parsed as HTML: a real <b> element is created
```

همان‌طور که در درس مربوط به `textContent` و `innerHTML` دیدیم، `innerHTML` رشته را **به‌عنوان کد HTML تفسیر می‌کند** و تگ‌ها را به عنصر تبدیل می‌کند، ولی `innerText` و `textContent` متن را **همان‌طور که هست** می‌گذارند (اگر در متن `<b>` باشد، خودِ این علامت‌ها روی صفحه دیده می‌شوند). برای متنی که کاربر وارد کرده همیشه `textContent` (یا `innerText`) را انتخاب کن تا کسی نتواند HTML یا اسکریپت مخرب در صفحه‌ات تزریق کند.

قدم ۳: تنظیم attributeها

```JavaScript
div.setAttribute("class", "box");   // <div class="box">
div.setAttribute("id", "msg");      // <div class="box" id="msg">
```

برای کلاس سه راه رایج داریم (جزئیاتشان در درس «ویژگی‌ها و کلاس‌ها» می‌آید):

روش	کار
`div.setAttribute("class", "box")`	کل attribute کلاس را برابر `"box"` می‌گذارد
`div.className = "box"`	همان کار، با property رشته‌ای `className`
`div.classList.add("box")`	کلاس `box` را **اضافه** می‌کند و کلاس‌های قبلی را نگه می‌دارد

property مربوط به کلاس `className` نام دارد، نه `class`، چون `class` در JavaScript یک کلمه‌ی رزروشده است.

قدم ۴: افزودن به درخت

`appendChild` : **یک گره، به‌عنوان آخرین فرزند.** `parent.appendChild(node)` گره (node) را **آخرین فرزند** `parent` می‌کند و **همان گره را برمی‌گرداند**:

```JavaScript
const list = document.querySelector("ul");
const li = document.createElement("li");
li.textContent = "Milk";
const result = list.appendChild(li);
console.log(result === li);   // true
```

`appendChild` فقط **یک** آرگومان می‌گیرد و آن هم باید یک **گره** باشد. اگر رشته بدهی (`list.appendChild("Milk")`) خطای `TypeError` می‌گیری.

`append` : **چند گره و رشته با هم.**

```JavaScript
const box = document.querySelector("#box");
const b = document.createElement("b");
b.textContent = "World";
box.append("Hello ", b, "!");   // strings become text nodes
console.log(box.innerHTML);     // Hello <b>World</b>!
```

`append` جدیدتر و راحت‌تر است: هر تعداد آرگومان می‌گیرد، هر رشته را به یک **گره‌ی متنی** (text node؛ گره‌ای که فقط متن است و تگ ندارد) جداگانه تبدیل می‌کند و **چیزی برنمی‌گرداند** (`undefined`).

	`appendChild(node)`	`append(...items)`
تعداد آرگومان	دقیقاً یکی	هر تعداد
رشته قبول می‌کند؟	خیر (`TypeError`)	بله؛ هر رشته یک text node می‌شود
مقدار برگشتی	همان گره‌ی اضافه‌شده	`undefined`
محل افزودن	آخرین فرزند	آخرِ فرزندان، به همان ترتیب آرگومان‌ها

`document.body` میان‌بُری برای عنصر `<body>` است؛ پس `document.body.appendChild(div)` یعنی «div را آخرین فرزند body کن».

`insertBefore` : افزودن قبل از یک فرزند مشخص.

```JavaScript
parent.insertBefore(newNode, referenceNode);
```

- روی **پدر** صدا زده می‌شود، نه روی گره‌ی مرجع.
- ترتیب آرگومان‌ها را این‌طور به خاطر بسپار: «**این** (گره‌ی جدید) را بگذار **قبلِ آن** (گره‌ی مرجع)».
- گره‌ی مرجع باید **فرزند مستقیم** همان پدر باشد؛ وگرنه خطای `NotFoundError` رخ می‌دهد.
- اگر مرجع `null` باشد، `insertBefore` درست مثل `appendChild` گره را به **انتها** اضافه می‌کند.
- مقدار برگشتی همان گره‌ی درج‌شده است.

```HTML
<ul id="list"><li>A</li><li id="c">C</li></ul>
```

```JavaScript
const list = document.getElementById("list");
const b = document.createElement("li");
b.textContent = "B";
list.insertBefore(b, document.getElementById("c"));   // A, B, C

const first = document.createElement("li");
first.textContent = "Start";
list.insertBefore(first, list.firstChild);            // Start, A, B, C
```

متدهای مدرن: `prepend`، `before` و `after`

متد	گره‌ی جدید کجا می‌رود؟
`parent.prepend(x)`	**اولین** فرزند `parent`
`parent.append(x)`	**آخرین** فرزند `parent`
`el.before(x)`	هم‌سطحِ (sibling) `el`، درست **قبل** از آن
`el.after(x)`	هم‌سطحِ `el`، درست **بعد** از آن

این چهار متد مثل `append` چند گره یا رشته می‌گیرند و `undefined` برمی‌گردانند. توجه کن که `prepend` و `append` روی **پدر** صدا زده می‌شوند ولی `before` و `after` روی **خودِ عنصر مرجع**.

گره‌ی متنی با `createTextNode`

JavaScript

```
const text = document.createTextNode("<b>Hi</b>");
document.body.appendChild(text);   // shows the characters <b>Hi</b>, no bold element
```

`document.createTextNode` یک گره‌ی متنی می‌سازد. چون یک **گره** است، `appendChild` آن را قبول می‌کند. محتوایش هیچ‌وقت HTML تفسیر نمی‌شود؛ این همان کاری است که `append("…")` پشت صحنه با رشته‌ها می‌کند.

جابه‌جا کردن، نه کپی کردن!

هر گره در هر لحظه فقط **یک جا** در درخت می‌تواند باشد. اگر گره‌ای را که الان در صفحه است دوباره append یا insert کنی، از جای قبلی **برداشته می‌شود** و به جای جدید **منتقل** می‌شود؛ کپی ساخته نمی‌شود.

HTML

```
<ul id="todo"><li id="task">Read slides</li></ul>
<ul id="done"></ul>
```

JavaScript

```
const task = document.getElementById("task");
document.getElementById("done").appendChild(task);   // moved, not copied
console.log(document.getElementById("todo").children.length); // 0
console.log(document.getElementById("done").children.length); // 1
```

برای کپی واقعی از `cloneNode` استفاده کن:

- `el.cloneNode(true)` : کپی **عمیق** (deep)؛ خود عنصر به‌همراه همه‌ی فرزندان و متن داخلش.
- `el.cloneNode()` یا `el.cloneNode(false)` : کپی **سطحی** (shallow)؛ فقط خود تگ و attributeهایش، بدون فرزندان و حتی بدون متن.

کپی، attributeها و از جمله `id` را هم کپی می‌کند؛ پس اگر عنصر اصلی `id` داشت، کپی را عوض کن تا دو عنصر با یک `id` نداشته باشی. listenerهایی که با `addEventListener` به عنصر اصلی وصل شده‌اند کپی **نمی‌شوند**.

مثال اسلایدهای ۲۴ تا ۲۶: New Div!

doc.html

```
<!DOCTYPE html>
<html>
<body>
    <h1 id="title">Hello World</h1>
    <button>CLICK ME!</button>
    <script>
        const btn = document.querySelector('button');
        btn.addEventListener('click', () => {
            const div = document.createElement("div");
            div.innerText = "Hello DOM!";
            div.setAttribute("class", "box");
            document.body.appendChild(div);
        });
    </script>
</body>
</html>
```

خط به خط:

- **خط ۷:** اولین `<button>` صفحه را پیدا می‌کنیم.
- **خط ۸:** به رویداد `click` آن یک listener (تابع شنونده) وصل می‌کنیم.
- **خط ۹:** در **هر** کلیک یک `<div>` **تازه** ساخته می‌شود (هنوز روی صفحه نیست).
- **خط ۱۰:** متن داخلش `Hello DOM!` می‌شود.
- **خط ۱۱:** کلاس `box` را به آن می‌دهیم؛ حالا `<div class="box">Hello DOM!</div>` داریم.
- **خط ۱۲:** آن را آخرین فرزند `<body>` می‌کنیم و از همین لحظه روی صفحه دیده می‌شود.

نتیجه (اسلایدهای ۲۵ و ۲۶): قبل از کلیک فقط عنوان «Hello World» و دکمه‌ی «CLICK ME!» دیده می‌شود. بعد از کلیک، درست زیر دکمه یک خط تازه با متن «Hello DOM!» ظاهر می‌شود. چون `<div>` عنصری بلوکی (block؛ عنصری که از خط جدید شروع می‌شود و کل عرض را می‌گیرد) است، در خط جدید می‌آید.

دو نکته‌ی ظریف این مثال:

- div در واقع **بعد از تگ** `<script>` به body اضافه می‌شود (آخرین فرزند body)، ولی چون `<script>` چیزی روی صفحه نشان نمی‌دهد، div درست زیر دکمه دیده می‌شود.
- چون `createElement` **داخل** listener است، هر کلیک یک div **جدید** می‌سازد: بعد از ۳ کلیک، ۳ خط «Hello DOM!» داریم. برای `.box` در این مثال CSSی تعریف نشده، پس کلاس فعلاً اثر ظاهری ندارد و فقط نشان می‌دهد attribute چطور تنظیم می‌شود.

ترکیب با رویدادها: یک لیست کار ساده

HTML

```
<input id="task" placeholder="New task">
<button id="add">Add</button>
<ul id="list"></ul>
```

```JavaScript
const input = document.getElementById("task");
const list = document.getElementById("list");

document.getElementById("add").addEventListener("click", () => {
  if (input.value.trim() === "") return;   // ignore empty tasks
  const li = document.createElement("li");
  li.textContent = input.value;            // safe: plain text
  list.appendChild(li);
  input.value = "";                        // clear the box for the next task
});
```

ساختن لیست از روی یک آرایه

```JavaScript
const fruits = ["Apple", "Banana", "Cherry"];
const ul = document.createElement("ul");

fruits.forEach((name) => {
  const li = document.createElement("li");
  li.textContent = name;
  ul.appendChild(li);
});

document.body.appendChild(ul);
console.log(ul.children.length);   // 3
```

راه کوتاه‌تر: با `map` آرایه‌ای از عنصرها بساز و همه را با spread (`...`؛ پخش کردن عضوهای آرایه به‌صورت آرگومان‌های جدا) یک‌جا به `append` بده:

```JavaScript
const items = fruits.map((name) => {
  const li = document.createElement("li");
  li.textContent = name;
  return li;
});
ul.append(...items);   // append accepts many nodes at once
```

راه سوم ساختن یک رشته‌ی HTML و دادن آن به `innerHTML` است: `ul.innerHTML = fruits.map((f) => "<li>" + f + "</li>").join("")`. کوتاه است، ولی اگر داده از کاربر بیاید خطرناک است و هر بار کل محتوای قبلی را دور می‌ریزد و از نو می‌سازد.

> **نکته**
>
> - `createElement` فقط **می‌سازد**؛ تا `append`، `appendChild`، `insertBefore`، `prepend` یا `before` یا `after` صدا نزنی، چیزی روی صفحه نمی‌آید.
> - `appendChild` : دقیقاً یک **گره**، و خروجی‌اش همان گره است. `append` : چند گره یا رشته، و خروجی‌اش `undefined` است.
> - `parent.insertBefore(جدید, مرجع)` روی **پدر** صدا زده می‌شود؛ مرجعِ `null` یعنی «به انتها اضافه کن».
> - برای «اولین فرزند کردن» دو راه داری: `parent.prepend(x)` یا `parent.insertBefore(x, parent.firstChild)`.

⚠ دام تستی

- اگر `createElement` را **بیرون** از listener بنویسی و داخل listener فقط `appendChild` کنی، با هر کلیک **همان یک عنصر** جابه‌جا می‌شود و تعداد عنصرها زیاد نمی‌شود!
- `list.appendChild("text")` خطای `TypeError` می‌دهد، ولی `list.append("text")` درست کار می‌کند.
- `const x = list.append(li); x.textContent = "…";` خطا می‌دهد، چون `x` برابر `undefined` است. با `appendChild` همین کد درست است.
- `createElement("<p>")` غلط است (`InvalidCharacterError`)؛ فقط نام تگ را بده: `createElement("p")`.
- اگر خروجی `map` را **بدون** `join("")` به `innerHTML` بدهی، آرایه با **کاما** به رشته تبدیل می‌شود و کاماها هم روی صفحه می‌آیند.
- append کردنِ گره‌ای که در صفحه هست آن را **منتقل** می‌کند، نه کپی؛ برای کپی `cloneNode(true)` لازم است.

✓ جمع‌بندی

ساختن: `createElement` (یا `createTextNode` برای متن خالی) ← محتوا: `textContent` / `innerText` / `innerHTML` ← attribute: `setAttribute` یا `className` / `classList` ← افزودن: `appendChild` / `append` (آخر)، `prepend` (اول)، `before` / `insertBefore` / `after` (وسط). هر گره فقط یک جا در درخت دارد و افزودن دوباره یعنی جابه‌جایی.

✓ تست‌های تمرینی

۲۵ تست · شماره‌ی ۸۳۵ تا ۸۵۹

۸۳۵ آسان

بلافاصله بعد از اجرای `const li = document.createElement("li");` کدام گزینه درست است؟

۱) یک `<li>` به انتهای `<body>` اضافه می‌شود و روی صفحه دیده می‌شود.

۲) اولین `<li>` موجود در صفحه پیدا و در `li` ذخیره می‌شود.

۳) یک عنصر `<li>` تازه در حافظه ساخته می‌شود که تا به درخت DOM اضافه نشود روی صفحه نمی‌آید.

۴) یک `<li>` خالی به‌عنوان آخرین فرزندِ اولین `<ul>` صفحه قرار می‌گیرد.

۸۳۶ آسان

خروجی کد زیر در مرورگر کدام است؟

HTML

```
<p>Old paragraph</p>
```

JavaScript

```
const p = document.createElement("p");
p.textContent = "New paragraph";
console.log(p.isConnected, document.querySelectorAll("p").length);
```

۱) `true 2`　۲) `false 1`　۳) `false 2`　۴) `true 1`

۸۳۷ متوسط

کدام دستور هنگام اجرا در مرورگر **خطا** می‌دهد؟

۱) `document.createElement("section")`

۲) `document.createElement("<div>")`

۳) `document.body.append("Hi", "!")`

۴) `document.body.insertBefore(document.createElement("p"), null)`

۸۳۸ آسان

می‌خواهیم عنصر `div` که با `createElement` ساخته‌ایم به‌عنوان آخرین فرزند `<body>` روی صفحه بیاید. در جای خالی `document.body._____(div);` کدام گزینه قرار می‌گیرد؟

۱) `createElement`　۲) `querySelector`　۳) `setAttribute`　۴) `appendChild`

۸۳۹ آسان

کدام جمله درباره‌ی `append` و `appendChild` **درست** است؟

۱) `append` می‌تواند چند گره و رشته را یک‌جا اضافه کند، ولی `appendChild` فقط یک گره می‌گیرد.

۲) `appendChild` رشته‌ها را خودکار به گره‌ی متنی تبدیل می‌کند.

۳) `append` گره‌ی اضافه‌شده را برمی‌گرداند، پس می‌توان پشت آن `.textContent` نوشت.

۴) هر دو متد گره را به‌عنوان اولین فرزند پدر اضافه می‌کنند.

۸۴۰ متوسط

خروجی کد زیر کدام است؟

HTML

```html
<ul id="list"></ul>
```

JavaScript

```javascript
const list = document.querySelector("#list");
list.append(document.createElement("li"), "note", "!");
console.log(list.children.length, list.childNodes.length);
```

۱) `1 3`　۲) `3 3`　۳) `1 1`　۴) `1 2`

۸۴۱ سخت

خروجی کد زیر کدام است؟

HTML

```
<ul></ul>
```

JavaScript

```
const ul = document.querySelector("ul");
const first = ul.appendChild(document.createElement("li"));
first.textContent = "One";
console.log(ul.textContent);
const second = ul.append(document.createElement("li"));
second.textContent = "Two";
console.log(ul.textContent);
```

۱) `One` `OneTwo`

۲) `One` `Uncaught TypeError`

۳) `Uncaught TypeError`

۴) `One` `One`

۸۴۲ متوسط

اجرای کدام خط از کد زیر باعث خطا می‌شود؟

HTML

```
<div id="box"></div>
```

JavaScript

```
1  const box = document.getElementById("box");
2  const p = document.createElement("p");
3  box.append("Title: ", p);
4  console.log("A");
5  box.insertBefore(document.createTextNode("!"), null);
6  console.log("B");
7  box.appendChild("Done");
8  console.log("C");
```

۱) خط ۳

۲) خط ۵

۳) خط ۷

۴) هیچ خطایی رخ نمی‌دهد

۸۴۳ آسان

کدام دستور یک **گره‌ی متنی** (text node) می‌سازد که بعداً بتوان آن را با `appendChild` به صفحه اضافه کرد؟

۱) `document.createElement("text")`

۲) `document.createText("Hello")`

۳) `document.textNode("Hello")`

۴) `document.createTextNode("Hello")`

۸۴۴ متوسط

خروجی کد زیر کدام است؟

```HTML
<div id="box"></div>
```

```JavaScript
const box = document.getElementById("box");
box.appendChild(document.createTextNode("<b>Hi</b>"));
console.log(box.children.length, box.textContent);
```

۱) `1 Hi`
۲) `1 <b>Hi</b>`
۳) `0 Hi`
۴) `0 <b>Hi</b>`

۸۴۵ آسان

`ul` یک لیست است و `oldLi` یکی از فرزندان مستقیم آن. کدام دستور، `newLi` را درست **قبل از** `oldLi` قرار می‌دهد؟

۱) `ul.insertBefore(oldLi, newLi)`
۲) `ul.insertBefore(newLi, oldLi)`
۳) `oldLi.insertBefore(newLi)`
۴) `newLi.insertBefore(ul, oldLi)`

۸۴۶ متوسط

خروجی کد زیر کدام است؟

```HTML
<ul id="list"><li>A</li><li>C</li></ul>
```

```JavaScript
const list = document.getElementById("list");
const b = document.createElement("li");
b.textContent = "B";
list.insertBefore(b, list.children[1]);
console.log(list.textContent);
```

۱) `BAC`
۲) `ACB`
۳) `AC`
۴) `ABC`

۸۴۷ متوسط

می‌خواهیم `newItem` **اولین** فرزندِ `list` شود. در جای خالی `list.insertBefore(newItem, ______);` کدام گزینه قرار می‌گیرد؟

۱) `null`
۲) `list.firstChild`
۳) `list.lastChild`
۴) `list`

۸۴۸ سخت

خروجی کد زیر کدام است؟

HTML

```
<div id="box"><p>P1</p></div>
<span id="s">S</span>
```

JavaScript

```
const box = document.getElementById("box");
const s = document.getElementById("s");
box.insertBefore(document.createElement("em"), null);
console.log(box.children.length);
box.insertBefore(document.createElement("b"), s);
console.log(box.children.length);
```

۱) `2` `Uncaught NotFoundError`

۲) `2` `3`

۳) `Uncaught TypeError`

۴) `1` `Uncaught NotFoundError`

۸۴۹ متوسط

خروجی کد زیر کدام است؟

HTML

```
<ul id="list"><li id="b">B</li></ul>
```

JavaScript

```
const list = document.getElementById("list");
const b = document.getElementById("b");
const make = (t) => {
  const li = document.createElement("li");
  li.textContent = t;
  return li;
};
b.before(make("A"));
b.after(make("C"));
list.prepend(make("0"));
console.log(list.textContent);
```

۱) `A0BC`

۲) `ABC0`

۳) `0ABC`

۴) `0BAC`

۸۵۰ متوسط

خروجی کد زیر کدام است؟

```HTML
<ul id="todo"><li id="task">Read slides</li></ul>
<ul id="done"></ul>
```

```JavaScript
const task = document.getElementById("task");
document.getElementById("done").appendChild(task);
console.log(
  document.getElementById("todo").children.length,
  document.getElementById("done").children.length
);
```

۱) `0 1`
۲) `1 1`
۳) `1 0`
۴) `0 0`

۸۵۱ متوسط

کدام جمله درباره‌ی افزودن عناصر **نادرست** است؟

۱) اگر عنصری را که الان در صفحه است دوباره با `appendChild` به جای دیگری اضافه کنیم، یک کپی از آن ساخته می‌شود و نسخه‌ی قبلی سر جایش می‌ماند.

۲) `insertBefore(node, null)` گره را به انتهای فرزندان پدر اضافه می‌کند.

۳) `appendChild` همان گره‌ای را که اضافه کرده برمی‌گرداند.

۴) `append` رشته‌ها را به گره‌ی متنی تبدیل می‌کند و اضافه می‌کند.

۸۵۲ سخت

خروجی کد زیر کدام است؟

```HTML
<ul id="a"><li id="x">X</li></ul>
<ul id="b"></ul>
```

```JavaScript
const x = document.getElementById("x");
const deep = x.cloneNode(true);
const shallow = x.cloneNode();
document.getElementById("b").append(deep, shallow);
console.log(
  document.getElementById("a").children.length,
  document.getElementById("b").children.length
);
console.log(deep.textContent, shallow.textContent === "");
```

۱) `0 2` `X true`
۲) `1 2` `X false`
۳) `0 2` `X false`
۴) `1 2` `X true`

۸۵۳ سخت

خروجی کد زیر کدام است؟ (یادآوری `children` یک HTMLCollection **زنده** است؛ یعنی با هر تغییر در DOM به‌روز می‌شود.)

```HTML
<ul id="list"><li>A</li><li>B</li><li>C</li></ul>
```

```JavaScript
const list = document.getElementById("list");
const items = list.children;
const z = document.createElement("li");
z.textContent = "Z";
list.insertBefore(z, items[1]);
list.insertBefore(items[0], null);
console.log([...list.children].map((li) => li.textContent).join(""));
```

۱) `AZBCA`
۲) `AZBC`
۳) `ZBCA`
۴) `ZABC`

۸۵۴ آسان

در مثال «New Div!» اسلاید ۲۴، کاربر **یک بار** روی دکمه‌ی «CLICK ME!» کلیک می‌کند. طبق تصویر اسلاید ۲۶، صفحه چه شکلی می‌شود؟

۱) عنوان «Hello World» با «Hello DOM!» جایگزین می‌شود.
۲) «Hello DOM!» بالای عنوان «Hello World» ظاهر می‌شود.
۳) عنوان و دکمه سر جایشان می‌مانند و زیر دکمه یک خط تازه با متن «Hello DOM!» ظاهر می‌شود.
۴) هیچ تغییری دیده نمی‌شود، چون برای کلاس `box` هیچ CSSی تعریف نشده است.

۸۵۵ متوسط

صفحه‌ی زیر همان کد اسلاید ۲۴ است. کاربر **سه بار** روی دکمه کلیک می‌کند. خروجی کدام است؟

```HTML
<h1 id="title">Hello World</h1>
<button>CLICK ME!</button>
```

```JavaScript
const btn = document.querySelector('button');
btn.addEventListener('click', () => {
    const div = document.createElement("div");
    div.innerText = "Hello DOM!";
    div.setAttribute("class", "box");
    document.body.appendChild(div);
});
```

بعد از سه کلیک، این دستور در کنسول اجرا می‌شود:

```
console.log(document.querySelectorAll(".box").length, document.body.lastElementChild.tagName);
```

۱) `1 DIV`
۲) `3 DIV`
۳) `3 SCRIPT`
۴) `0 SCRIPT`

۸۵۶ سخت

کاربر **سه بار** روی دکمه کلیک می‌کند. خروجی کد زیر کدام است؟

HTML

```
<button>Add</button>
```

JavaScript

```
const btn = document.querySelector("button");
const div = document.createElement("div");
div.textContent = "Hello DOM!";
btn.addEventListener("click", () => {
  document.body.appendChild(div);
  console.log(document.querySelectorAll("div").length);
});
```

۱) `1` `2` `3`

۲) `0` `1` `2`

۳) `1` `1` `1`

۴) `1` `Uncaught TypeError`

۸۵۷ سخت

کاربر این کارها را به ترتیب انجام می‌دهد: در کادر متن `Read` را تایپ می‌کند و Add را می‌زند؛ دوباره Add را می‌زند (کادر خالی است)؛ `Write` را تایپ می‌کند و Add را می‌زند. خروجی کدام است؟

HTML

```
<input id="task">
<button id="add">Add</button>
<ul id="list"></ul>
```

JavaScript

```
const input = document.getElementById("task");
const list = document.getElementById("list");
document.getElementById("add").addEventListener("click", () => {
  if (input.value.trim() === "") return;
  const li = document.createElement("li");
  li.textContent = input.value;
  list.prepend(li);
  input.value = "";
});
```

در پایان این دستور اجرا می‌شود: `console.log(list.children.length, list.firstElementChild.textContent);`

۱) `3 Write`

۲) `3 Read`

۳) `2 Read`

۴) `2 Write`

۸۵۸ متوسط

خروجی کد زیر کدام است؟

HTML

```
<ul id="menu"></ul>
```

JavaScript

```
const data = ["HTML", "CSS", "JS"];
const ul = document.getElementById("menu");
data.forEach((item, i) => {
  const li = document.createElement("li");
  li.textContent = i + 1 + ". " + item;
  ul.appendChild(li);
});
console.log(ul.children.length, ul.lastElementChild.textContent);
```

۱) `3 21. JS`
۲) `3 3. JS`
۳) `1 3. JS`
۴) `3 2. JS`

۸۵۹ سخت

خروجی کد زیر کدام است؟

HTML

```
<ul></ul>
```

JavaScript

```
const fruits = ["Apple", "Banana"];
const ul = document.querySelector("ul");
ul.innerHTML = fruits.map((f) => "<li>" + f + "</li>");
console.log(ul.children.length, ul.textContent);
```

۱) `2 Apple,Banana`
۲) `2 AppleBanana`
۳) `1 Apple,Banana`
۴) `0 <li>Apple</li>,<li>Banana</li>`

۲-۹ حذف و جایگزینی عناصر

منبع: JS in HTML، اسلایدهای ۲۷ تا ۳۰

درسنامه

در درس قبل عنصر ساختیم و به صفحه اضافه کردیم. حالا نوبت برعکسش است: **برداشتن** عنصر از صفحه یا **عوض کردن** آن با عنصری دیگر. بستن یک اعلان (notification)، حذف یک کار از لیست کارها (todo list) و پاک کردن پیام خطا بعد از اصلاح فرم همه از همین جنس‌اند.

یک تشبیه ساده

درخت DOM را مثل یک قفسه‌ی کتاب در نظر بگیر. وقتی کتابی را از قفسه برمی‌داری، کتاب **نابود نمی‌شود**؛ فقط دیگر در قفسه نیست و اگر هنوز در دستت باشد (یعنی در یک متغیر نگهش داشته باشی) می‌توانی دوباره سر جایش یا جای دیگری بگذاری. حذف عنصر از DOM هم دقیقاً همین است.

سه ابزار اصلی (اسلاید ۲۷)

اسلاید ۲۷ می‌گوید:

- با `.remove()` عنصر را **مستقیم** حذف کن؛
- یا `.removeChild()` را روی **عنصر پدر** صدا بزن؛
- `replaceChild()` یک عنصر را با عنصر دیگری **عوض** می‌کند؛
- **همیشه اول عنصر را پیدا کن** (query کن)، بعد حذفش کن؛
- برای رابط‌های پویا مثل لیست کارها و اعلان‌ها کاربرد دارد؛
- با رویدادها ترکیبش کن تا کاربر با کلیک چیزی را حذف کند.

`remove()` : عنصر خودش را حذف می‌کند

```JavaScript
const note = document.getElementById("note");
note.remove();   // the element removes itself, together with all its children
```

- به پدر احتیاجی ندارد و آرگومان هم نمی‌گیرد.
- عنصر را **همراه با همه‌ی فرزندانش** (کل زیردرخت) از صفحه برمی‌دارد.
- مقدار برگشتی‌اش `undefined` است.
- اگر عنصر از قبل در صفحه نباشد (مثلاً قبلاً حذف شده یا هیچ‌وقت اضافه نشده)، **هیچ اتفاقی نمی‌افتد** و خطایی هم رخ نمی‌دهد.

`removeChild()` : پدر فرزندش را بیرون می‌کند

```JavaScript
const list = document.getElementById("list");
const first = list.firstElementChild;
const removed = list.removeChild(first);
console.log(removed === first);    // true  -> it returns the removed node
console.log(removed.isConnected);  // false -> no longer in the page
```

- روی **پدر** صدا زده می‌شود و فرزند را به‌عنوان آرگومان می‌گیرد.
- **گره‌ی حذف‌شده را برمی‌گرداند.**
- آرگومان باید **فرزند مستقیم** همان پدر باشد؛ وگرنه خطای `NotFoundError` رخ می‌دهد (مثلاً اگر قبلاً حذف شده باشد یا نوه باشد نه فرزند).
- روش قدیمی‌تر است؛ الگوی معروف `el.parentNode.removeChild(el)` همان کار `el.remove()` را می‌کند.

`replaceChild()` : عوض کردن یک فرزند با گره‌ای دیگر

```JavaScript
parent.replaceChild(newChild, oldChild);
```

- روی **پدر** صدا زده می‌شود.
- ترتیب آرگومان‌ها مثل `insertBefore` است: **اول جدید، بعد قدیمی**.
- **گره‌ی قدیمی** (همان که بیرون رفت) را برمی‌گرداند.
- اگر `oldChild` فرزند این پدر نباشد، خطای `NotFoundError` می‌گیری.
- اگر `newChild` از قبل جای دیگری در صفحه باشد، به جای جدید **منتقل** می‌شود (کپی نمی‌شود).

```HTML
<ul id="list"><li>A</li><li id="b">B</li><li>C</li></ul>
```

```JavaScript
const list = document.getElementById("list");
const fresh = document.createElement("li");
fresh.textContent = "X";
const old = list.replaceChild(fresh, document.getElementById("b"));
console.log(list.textContent);  // "AXC"
console.log(old.textContent);   // "B"  -> the replaced node is returned
```

نسخه‌ی مدرن‌تر `oldEl.replaceWith(newEl)` است که روی **خودِ عنصر قدیمی** صدا زده می‌شود و `undefined` برمی‌گرداند؛ یعنی `old.replaceWith(fresh)` همان `old.parentNode.replaceChild(fresh, old)` است.

متد	روی چه چیزی صدا زده می‌شود؟	مقدار برگشتی	اگر گره فرزندِ مورد انتظار نباشد
`el.remove()`	خود عنصر	`undefined`	خطا نمی‌دهد؛ کاری نمی‌کند
`parent.removeChild(child)`	پدر	گره‌ی حذف‌شده	`NotFoundError`
`parent.replaceChild(newC, oldC)`	پدر	گره‌ی قدیمی	`NotFoundError`
`oldEl.replaceWith(newEl)`	عنصر قدیمی	`undefined`	—

اول پیدا کن، بعد حذف کن!

`getElementById` و `querySelector` اگر چیزی پیدا نکنند `null` برمی‌گردانند. صدا زدن متد روی `null` خطاست:

```JavaScript
document.getElementById("nope").remove();
// Uncaught TypeError: Cannot read properties of null (reading 'remove')
```

پس قبل از حذف مطمئن شو عنصر پیدا شده:

```JavaScript
const el = document.getElementById("nope");
if (el) el.remove();   // classic guard
el?.remove();          // optional chaining: does nothing when el is null
```

`?.` (optional chaining؛ «اگر سمت چپ `null` یا `undefined` بود، ادامه نده») در این حالت بدون خطا `undefined` برمی‌گرداند.

مثال اسلایدهای ۲۸ تا ۳۰: Title Goes Away!

doc.html

```1 <!DOCTYPE html>
2 <html>
3 <body>
4     <h1 id="title">Hello World</h1>
5     <button>CLICK ME!</button>
6     <script>
7         const btn = document.querySelector('button');
8         btn.addEventListener('click', () => {
9             const element = document.getElementById("title");
10            element.remove();
11        });
12    </script>
13 </body>
14 </html>
```

- **خط ۷:** دکمه را پیدا می‌کنیم.
- **خط ۸:** listener کلیک را وصل می‌کنیم.
- **خط ۹:** در **هر** کلیک، عنوانی با `id="title"` را جست‌وجو می‌کنیم (همان «اول پیدا کن»).
- **خط ۱۰:** عنوان را حذف می‌کنیم.

قبل از کلیک (اسلاید ۲۹): عنوان بزرگ «Hello World» و زیرش دکمه‌ی «CLICK ME!». **بعد از کلیک (اسلاید ۳۰):** عنوان ناپدید شده و فقط دکمه مانده که حالا بالای صفحه قرار گرفته (چون جای خالیِ عنوان هم از بین رفته است).

کلیک دوم چه می‌شود؟ این‌بار `getElementById("title")` چیزی پیدا نمی‌کند و `null` برمی‌گرداند؛ پس `element.remove()` در کنسول خطای `TypeError: Cannot read properties of null (reading 'remove')` می‌دهد. ظاهر صفحه تغییری نمی‌کند و دکمه هم سر جایش است؛ خطا فقط همان اجرای listener را متوقف می‌کند و با هر کلیک بعدی دوباره تکرار می‌شود. راه‌های رفع:

- `element?.remove();` یا `if (element) element.remove();`

- غیرفعال کردن دکمه بعد از کلیک اول: `btn.disabled = true;`
- اجرای listener فقط یک بار: `btn.addEventListener('click', handler, { once: true });` (گزینه‌ی `once` باعث می‌شود listener بعد از اولین اجرا خودبه‌خود حذف شود.)

حذف‌شده یعنی نابودشده؟ نه!

```JavaScript
const msg = document.getElementById("msg");
msg.remove();                  // gone from the page...
console.log(msg.textContent);  // ...but still alive in the variable
document.body.append(msg);     // put it back (an "undo" feature)
```

بعد از `remove()` متغیر `msg` **نمی‌شود**؛ `null` همان شیء با همه‌ی متن، attributeها و فرزندانش در حافظه است و فقط به صفحه وصل نیست (`isConnected` برابر `false`). هر وقت هیچ متغیری دیگر به آن اشاره نکند، garbage collector (بخش خودکار پاک‌سازی حافظه‌ی JavaScript) آن را پاک می‌کند.

خالی کردن یک عنصر (حذف همه‌ی فرزندان)

```JavaScript
list.innerHTML = "";           // 1) simplest
list.textContent = "";         // 2) also removes every child
list.replaceChildren();        // 3) modern: no arguments = no children
while (list.firstChild) {      // 4) classic loop
  list.removeChild(list.firstChild);
}
```

در هر چهار روش خودِ `list` سر جایش می‌ماند و فقط **محتوایش** خالی می‌شود؛ اما `list.remove()` خودِ لیست را از صفحه برمی‌دارد. در روش ۴ دقت کن که `firstChild` گره‌های متنیِ فاصله و خط جدید بین تگ‌ها را هم شامل می‌شود؛ حلقه آن‌قدر ادامه می‌دهد تا هیچ فرزندی (از هر نوع) نماند.

دام حلقه روی مجموعه‌ی زنده

`getElementsByClassName` یک HTMLCollection **زنده** (live؛ با هر تغییر DOM خودش به‌روز می‌شود) برمی‌گرداند. حذف در یک حلقه‌ی معمولی روی آن **نصف عنصرها را جا می‌اندازد**:

```JavaScript
const items = document.getElementsByClassName("item"); // live collection of 4 items
for (let i = 0; i < items.length; i++) {
  items[i].remove();   // BUG: after each removal the collection shrinks and shifts
}
// only items 1 and 3 are removed; 2 and 4 stay
```

بعد از حذف `items[0]` ، عنصر دوم به خانه‌ی ۰ می‌لغزد ولی `i` به ۱ رفته است؛ پس از رویش می‌پرد. راه درست:

```JavaScript
document.querySelectorAll(".item").forEach((el) => el.remove()); // static NodeList
// or
while (items.length > 0) items[0].remove();
```

کاربردها: لیست کارها و اعلان‌ها (ترکیب با رویداد)

دکمه‌ی حذف برای هر کار با event delegation (یک listener روی پدر برای همه‌ی فرزندان، که در درس رویدادها دیدیم):

HTML

```
<ul id="todos">
  <li>Read slides <button class="del">x</button></li>
  <li>Do homework <button class="del">x</button></li>
</ul>
```

JavaScript

```
document.getElementById("todos").addEventListener("click", (e) => {
  if (e.target.classList.contains("del")) {
    e.target.parentElement.remove();   // remove the <li> that holds this button
  }
});
```

`e.target` خودِ دکمه است؛ اگر `e.target.remove()` بنویسیم فقط دکمه حذف می‌شود و متن کار می‌ماند. برای همین سراغ پدرش (`parentElement`) یا نزدیک‌ترین `<li>` با `e.target.closest("li")` می‌رویم.

اعلانی که خودش بعد از ۳ ثانیه بسته می‌شود:

JavaScript

```
function showToast(text) {
  const toast = document.createElement("div");
  toast.className = "toast";
  toast.textContent = text;
  document.body.append(toast);
  setTimeout(() => toast.remove(), 3000);   // auto-dismiss after 3 seconds
}
showToast("Saved!");
```

پنهان کردن با حذف کردن فرق دارد: `el.style.display = "none"` یا `el.hidden = true` فقط عنصر را **نامرئی** می‌کنند؛ عنصر هنوز در DOM است و `querySelector` پیدایش می‌کند. `remove()` واقعاً آن را از درخت بیرون می‌برد.

> **نکته**
>
> - `el.remove()` ساده‌ترین راه است: نه پدر می‌خواهد نه آرگومان، و روی عنصرِ از قبل حذف‌شده هم خطا نمی‌دهد.
> - `removeChild` و `replaceChild` روی **پدر** صدا زده می‌شوند و هر دو گره‌ای را که بیرون رفته **برمی‌گردانند**.
> - `replaceChild(جدید, قدیمی)` ؛ ترتیبش مثل `insertBefore(جدید, مرجع)` است: «اول جدید».
> - گره‌ی حذف‌شده در متغیرت زنده می‌ماند و می‌شود دوباره اضافه‌اش کرد.

⚠ دام تستی

- در مثال «Title Goes Away!»، کلیک دوم خطای `TypeError` (خواندن `remove` از `null`) می‌دهد، چون عنوان دیگر وجود ندارد.
- `removeChild` روی گره‌ای که فرزند مستقیم آن پدر نیست (یا قبلاً حذف شده) خطای `NotFoundError` می‌دهد، ولی `remove()` در همین وضعیت بی‌صدا هیچ کاری نمی‌کند.
- جابه‌جا نوشتن آرگومان‌ها در `replaceChild(old, fresh)` معمولاً به `NotFoundError` می‌رسد، چون `fresh` فرزند آن پدر نیست.
- حذف در حلقه‌ی `for` روی HTMLCollection زنده فقط نصف عنصرها را حذف می‌کند.
- حذف پدر یعنی حذف کل زیردرختش؛ فرزندان جداگانه در صفحه نمی‌مانند.

✓ جمع‌بندی

حذف: `el.remove()` (ساده) یا `parent.removeChild(child)` (برمی‌گرداند، سخت‌گیر). جایگزینی: `parent.replaceChild(newC, oldC)` یا `oldC.replaceWith(newC)` . خالی کردن: `innerHTML = ""` ، `textContent = ""` ، `replaceChildren()` یا حلقه‌ی `while (firstChild)` . قبل از حذف، عنصر را پیدا کن و `null` بودنش را بررسی کن.

✓ تست‌های تمرینی

۲۱ تست · شماره‌ی ۸۶۰ تا ۸۸۰

۸۶۰ آسان

کدام متد عنصر را **بدون نیاز به دسترسی به پدرش** و بدون هیچ آرگومانی از صفحه حذف می‌کند؟

۱) `el.removeChild()`
۲) `el.remove()`
۳) `el.replaceChild()`
۴) `delete el`

۸۶۱ آسان

خروجی کد زیر کدام است؟

HTML

```
<ul id="list"><li id="a">A</li><li>B</li></ul>
```

JavaScript

```
const list = document.getElementById("list");
const a = document.getElementById("a");
const removed = list.removeChild(a);
console.log(removed === a, list.children.length, removed.textContent);
```

۱) `true 1 A`
۲) `Uncaught TypeError`
۳) `true 1`
۴) `false 1 AB`

۸۶۲ آسان

در جای خالی `parent.replaceChild(_____)` برای این‌که `oldChild` با `newChild` عوض شود، کدام گزینه قرار می‌گیرد؟

۱) `oldChild, newChild`
۲) `parent, newChild`
۳) `newChild, oldChild`
۴) `newChild`

۸۶۳ متوسط

خروجی کد زیر کدام است؟

HTML

```
<ul id="list"><li id="a">A</li></ul>
<p id="note">N</p>
```

JavaScript

```
const list = document.getElementById("list");
const note = document.getElementById("note");
list.removeChild(document.getElementById("a"));
console.log(list.children.length);
list.removeChild(note);
console.log("done");
```

۱) `0` `done`
۲) `Uncaught NotFoundError`
۳) `0` `Uncaught TypeError`
۴) `0` `Uncaught NotFoundError`

۸۶۴ متوسط

خروجی کد زیر کدام است؟

HTML

```
<ul id="list"><li>A</li><li id="b">B</li><li>C</li></ul>
```

JavaScript

```
const list = document.getElementById("list");
const old = document.getElementById("b");
const fresh = document.createElement("li");
fresh.textContent = "X";
const r = list.replaceChild(fresh, old);
console.log(list.textContent, r.textContent);
```

۱) `AXC X`
۲) `AXC B`
۳) `ABC X`
۴) `AXBC B`

۸۶۵ سخت

خروجی کد زیر کدام است؟

```HTML
<ul id="list"><li>A</li><li>B</li></ul>
```

```JavaScript
const list = document.getElementById("list");
const old = list.querySelector("li");
const fresh = document.createElement("li");
fresh.textContent = "NEW";
list.replaceChild(old, fresh);
console.log(list.textContent);
```

۱) `NEWB`
۲) `AB`
۳) `Uncaught NotFoundError`
۴) `Uncaught TypeError`

۸۶۶ سخت

خروجی کد زیر کدام است؟

```HTML
<ul id="list"><li id="a">A</li><li id="b">B</li><li id="c">C</li></ul>
```

```JavaScript
const list = document.getElementById("list");
const a = document.getElementById("a");
const c = document.getElementById("c");
list.replaceChild(a, c);
console.log(list.textContent, list.children.length);
```

۱) `BA 2`
۲) `ABA 3`
۳) `ABC 3`
۴) `AB 2`

۸۶۷ آسان

دستور `old.replaceWith(fresh)` معادل کدام دستور است؟

۱) `old.parentNode.replaceChild(old, fresh)`
۲) `fresh.parentNode.replaceChild(old, fresh)`
۳) `old.removeChild(fresh)`
۴) `old.parentNode.replaceChild(fresh, old)`

۸۶۸ آسان

در مثال «Title Goes Away!» (اسلاید ۲۸)، طبق تصویر اسلاید ۳۰ بعد از **اولین** کلیک روی دکمه چه چیزی روی صفحه دیده می‌شود؟

۱) عنوان «Hello World» و دکمه، بدون تغییر

۲) فقط دکمه‌ی «CLICK ME!»

۳) فقط عنوان «Hello World»

۴) یک صفحه‌ی کاملاً خالی

۸۶۹ متوسط

صفحه‌ی زیر همان کد اسلاید ۲۸ است. کاربر **دو بار** روی دکمه کلیک می‌کند. خروجی کنسول کدام است؟

HTML

```html
<h1 id="title">Hello World</h1>
<button>CLICK ME!</button>
```

JavaScript

```javascript
const btn = document.querySelector('button');
btn.addEventListener('click', () => {
    const element = document.getElementById("title");
    element.remove();
});
```

بعد از دو کلیک این دستور اجرا می‌شود:

```
console.log(document.querySelectorAll("h1").length, document.querySelectorAll("button").length);
```

۱) `0 1`

۲) `0 0`

۳) `Uncaught TypeError` `0 1`

۴) `Uncaught TypeError` `0 0`

۸۷۰ متوسط

در مثال اسلاید ۲۸، برای این‌که کلیک‌های دوم به بعد **هیچ خطایی** ندهند، خط `element.remove();` را با کدام گزینه عوض کنیم؟

۱) `element?.remove();`

۲) `element.remove(true);`

۳) `document.removeChild(element);`

۴) `element.innerHTML = "";`

۸۷۱ سخت

اجرای کدام خط از کد زیر باعث خطا می‌شود؟

HTML

```
<ul id="list"><li>1</li><li>2</li></ul>
```

JavaScript

```
const list = document.querySelector("#list");
const first = list.firstElementChild;
first.remove();
console.log("A");
first.remove();
console.log("B");
list.removeChild(first);
console.log("C");
```

۱) خط ۳
۲) خط ۵
۳) خط ۷
۴) هیچ خطایی رخ نمی‌دهد

۸۷۲ متوسط

خروجی کد زیر کدام است؟

HTML

```
<div id="box"><p id="msg">Saved!</p></div>
```

JavaScript

```
const box = document.getElementById("box");
const msg = document.getElementById("msg");
msg.remove();
console.log(box.children.length, msg.textContent);
box.append(msg);
console.log(box.children.length, document.getElementById("msg") === msg);
```

۱) `0` `Uncaught TypeError`
۲) `1 Saved!` `1 true`
۳) `0 Saved!` `1 false`
۴) `0 Saved!` `1 true`

۸۷۳ سخت

خروجی کد زیر کدام است؟

```HTML
<div id="card"><h2>Title</h2><p>Text</p></div>
```

```JavaScript
const card = document.getElementById("card");
const title = card.querySelector("h2");
card.remove();
console.log(document.querySelectorAll("h2, p").length, title.isConnected,
card.children.length);
```

۱) `2 true 0`
۲) `0 false 2`
۳) `0 false 0`
۴) `0 true 2`

۸۷۴ متوسط

کدام جمله **نادرست** است؟

۱) `el.remove()` به دسترسی به پدر نیاز ندارد.
۲) `parent.removeChild(child)` گره‌ی حذف‌شده را برمی‌گرداند.
۳) `parent.replaceChild(newC, oldC)` گره‌ی قدیمی را برمی‌گرداند.
۴) بعد از اجرای `el.remove()`، متغیر `el` خودبه‌خود برابر `null` می‌شود.

۸۷۵ آسان

بعد از اجرای کدام دستور، `document.querySelector("#ad")` دیگر عنصر `#ad` را **پیدا نمی‌کند**؟ (`ad` همان عنصر `#ad` است.)

۱) `ad.remove()`
۲) `ad.style.display = "none"`
۳) `ad.hidden = true`
۴) `ad.textContent = ""`

۸۷۶ آسان

هدف این است که همه‌ی فرزندان لیست `list` حذف شوند ولی خودِ `<ul>` در صفحه بماند. کدام گزینه این کار را **انجام نمی‌دهد**؟

۱) `list.innerHTML = ""`
۲) `list.replaceChildren()`
۳) `while (list.firstChild) list.removeChild(list.firstChild);`
۴) `list.remove()`

۸۷۷ سخت

خروجی کد زیر کدام است؟ (به فاصله‌ها و خطوط جدیدِ داخل HTML دقت کن.)

HTML

```
<ul id="list">
  <li>A</li>
  <li>B</li>
</ul>
```

JavaScript

```
const list = document.getElementById("list");
let count = 0;
while (list.firstChild) {
  list.removeChild(list.firstChild);
  count++;
}
console.log(count, list.childNodes.length);
```

۱) `2 0`
۲) `5 0`
۳) `2 3`
۴) `3 0`

۸۷۸ سخت

خروجی کد زیر کدام است؟

HTML

```
<ul>
<li class="item">1</li><li class="item">2</li><li class="item">3</li><li class="item">4</li>
</ul>
```

JavaScript

```
const items = document.getElementsByClassName("item");
for (let i = 0; i < items.length; i++) {
  items[i].remove();
}
console.log(items.length, document.querySelector("ul").textContent.trim());
```

۱) `2 24`
۲) `0`
۳) `2 13`
۴) `3 234`

۸۷۹ متوسط

کاربر روی دکمه‌ی x **دومین** آیتم لیست کلیک می‌کند. خروجی کدام است؟

HTML

```
<ul id="todos">
  <li>Read <button class="del">x</button></li>
  <li>Code <button class="del">x</button></li>
  <li>Sleep <button class="del">x</button></li>
</ul>
```

JavaScript

```
const todos = document.getElementById("todos");
todos.addEventListener("click", (e) => {
  if (e.target.classList.contains("del")) {
    e.target.parentElement.remove();
  }
});
```

بعد از کلیک این دستور اجرا می‌شود: `console.log(todos.children.length, todos.querySelectorAll(".del").length);`

۱) `3 2`
۲) `2 3`
۳) `2 2`
۴) `0 0`

۸۸۰ متوسط

خروجی کد زیر کدام است؟

JavaScript

```
const note = document.createElement("div");
note.className = "toast";
note.textContent = "Saved!";
document.body.append(note);
setTimeout(() => note.remove(), 0);
console.log(document.querySelectorAll(".toast").length);
setTimeout(() => console.log(document.querySelectorAll(".toast").length), 10);
```

۱) `0` `0`
۲) `1` `1`
۳) `0` `1`
۴) `1` `0`

۲-۱۰ ویژگی‌ها (Attributes) و کلاس‌ها: setAttribute و classList

منبع: JS in HTML، اسلایدهای ۳۱ تا ۳۴

درسنامه

عنوان اسلاید ۳۱ این است: «Everything ...Modifying!»؛ یعنی با JavaScript تقریباً **همه‌چیزِ** یک عنصر را می‌شود عوض کرد. در این درس سراغ attributeها (ویژگی‌ها؛ همان جفت‌های `name="value"` که داخل تگ HTML می‌نویسیم) و مهم‌ترینشان، یعنی `class`، می‌رویم:

```HTML
<img src="cat.png" alt="A cat" width="300">
<a href="/about" target="_blank">About</a>
<input type="text" value="Ali" disabled>
```

یک تشبیه ساده

هر عنصر را مثل یک جعبه با چند **برچسب** در نظر بگیر: `src="cat.png"` و `width="300"`. `setAttribute` برچسب می‌چسباند (یا برچسب قبلی را عوض می‌کند)، `getAttribute` برچسب را می‌خواند و `removeAttribute` آن را می‌کَنَد. برچسب `class` هم یک برچسب ویژه است که فهرستی از کلمه‌ها را نگه می‌دارد و ابزار مخصوص خودش، `classList`، را دارد.

چهار متد کار با attributeها

```JavaScript
const link = document.querySelector("a");
link.setAttribute("href", "https://sharif.edu");  // add or change
console.log(link.getAttribute("href"));           // "https://sharif.edu"
link.removeAttribute("target");                   // delete it
console.log(link.hasAttribute("target"));         // false
console.log(link.getAttribute("title"));          // null -> no such attribute
```

متد	کار	خروجی
`setAttribute(name, value)`	اگر attribute نباشد **اضافه** می‌کند و اگر باشد مقدارش را **عوض** می‌کند	`undefined`
`getAttribute(name)`	مقدار را **می‌خواند**	همیشه **رشته**، یا `null` اگر وجود نداشته باشد
`removeAttribute(name)`	attribute را **حذف** می‌کند؛ اگر نباشد خطایی نمی‌دهد	`undefined`
`hasAttribute(name)`	**وجود** attribute را بررسی می‌کند	`true` یا `false`

مقداری که به `setAttribute` می‌دهی همیشه **به رشته تبدیل** می‌شود: `setAttribute("title", 42)` مقدار `"42"` را ذخیره می‌کند و `setAttribute("title", null)` مقدار رشته‌ی `"null"` را؛ یعنی `null` دادن، attribute را **حذف نمی‌کند**.

attributeهای بولی (Boolean attributes)

attributeهایی مثل `disabled` ، `checked` ، `hidden` ، `required` و `readonly` فقط با **حضورشان** کار می‌کنند و مقدارشان مهم نیست. پس:

```JavaScript
btn.setAttribute("disabled", "false");  // STILL disabled! the attribute exists
console.log(btn.disabled);              // true
btn.removeAttribute("disabled");        // now it is enabled
btn.disabled = false;                   // same thing, using the property
```

attribute یا property؟

مرورگر برای بیشترِ attributeهای استاندارد یک **property** (خاصیت) هم‌نام روی شیء عنصر می‌سازد: `a.href` ، `img.src` ، `el.id` ، `input.value` و... . اما این دو همیشه یکی نیستند:

موضوع	`getAttribute(...)`	property
نوع مقدار	همیشه رشته (یا `null`)	نوع واقعی: `img.width` عدد `300` ، `btn.disabled` بولی
آدرس‌ها (`src` ، `href`)	همان متنی که نوشته شده: `"dambooldor2.png"`	آدرس کامل و مطلق: `"file:///…/dom/dambooldor2.png"`
`value` در `<input>`	مقدار **اولیه** (نوشته‌شده در HTML)	مقدار **فعلی** (چیزی که کاربر تایپ کرده)
نام	`class` ، `for`	`className` ، `htmlFor`

```HTML
<input id="name" value="guest">
```

```JavaScript
const input = document.getElementById("name");
// ...the user types "Sara" into the box...
console.log(input.value);                  // "Sara"  (current value)
console.log(input.getAttribute("value"));  // "guest" (initial value from HTML)
```

attributeهای ساختگی (غیراستاندارد) مثل `<div foo="1">` property نمی‌سازند (`div.foo` برابر `undefined` است)؛ برای آن‌ها یا `getAttribute` به کار ببر یا از `data-*` استفاده کن.

attributeهای `data-*` و `dataset`

برای چسباندن داده‌ی دلخواه به یک عنصر، استاندارد این است که نام attribute با `data-` شروع شود. JavaScript همه را در شیء `dataset` می‌بیند و نامِ خطتیره‌دار را به camelCase (چسباندن کلمه‌ها و بزرگ کردن حرف اول هر کلمه‌ی بعدی) تبدیل می‌کند:

```HTML
<li id="item" data-product-id="42" data-price="9.5">Pen</li>
```

```JavaScript
const item = document.getElementById("item");
console.log(item.dataset.productId);   // "42"  (a string!)
console.log(item.dataset.price);       // "9.5"
item.dataset.inStock = "yes";          // adds data-in-stock="yes" to the tag
```

`classList` : ابزار مخصوص کلاس‌ها

```JavaScript
const box = document.querySelector(".card");   // <div class="card">
box.classList.add("active");              // class="card active"
box.classList.add("big", "shadow");       // several at once: "card active big shadow"
box.classList.remove("shadow");           // "card active big" (no error if missing)
box.classList.toggle("dark");             // added -> returns true
box.classList.toggle("dark");             // removed -> returns false
box.classList.contains("active");         // true
box.classList.replace("big", "small");    // "card active small"
console.log(box.classList.length);        // 3
```

- `add` کلاس تکراری اضافه نمی‌کند و می‌تواند چند کلاس را یک‌جا بگیرد.
- `remove` هم چندتایی کار می‌کند و اگر کلاس وجود نداشته باشد خطا نمی‌دهد.
- `toggle(name)` اگر کلاس نباشد اضافه می‌کند و اگر باشد حذف؛ **خروجی‌اش وضعیت جدید است**: `true` یعنی «الان هست» و `false` یعنی «الان نیست».
- `toggle(name, force)` : با `force` برابر `true` فقط اضافه می‌کند و با `false` فقط حذف. مثلاً `el.classList.toggle("hidden", !isVisible)` کلاس را دقیقاً مطابق یک شرط تنظیم می‌کند.
- `contains(name)` فقط بررسی می‌کند و چیزی را تغییر نمی‌دهد.
- نام کلاس را **بدون نقطه** بنویس: `add("active")` . نقطه مال انتخابگرهای CSS است؛ `add(".active")` کلاسی به نام عجیبِ `.active` می‌سازد!
- هر آرگومان فقط **یک** کلاس است: `add("btn primary")` خطای `InvalidCharacterError` می‌دهد؛ درستش `add("btn", "primary")` است.

`className` ، `classList` و `setAttribute("class", …)`

	نوع	نوشتن با آن
`el.className`	یک رشته مثل `"card active"`	**همه‌ی** کلاس‌ها را جایگزین می‌کند
`el.setAttribute("class", …)`	—	**همه‌ی** کلاس‌ها را جایگزین می‌کند
`el.classList`	شیئی فهرست‌مانند (DOMTokenList) با متدهای بالا	فقط همان کلاس را اضافه/حذف می‌کند

```JavaScript
// <p class="note big">
p.setAttribute("class", "red");   // class="red"  (note and big are gone!)
p.classList.add("bold");          // class="red bold"
p.className += " small";          // class="red bold small" (do not forget the space)
```

استایل پویا و تغییر وضعیت با کلاس‌ها

آخرین نکته‌ی اسلاید ۳۱ می‌گوید این ابزارها برای **استایل پویا** و **تغییر وضعیت** (state) به کار می‌روند. به‌جای این‌که در JavaScript ده‌ها استایل درون‌خطی (inline style؛ مثل `el.style.color = "red"`) بنویسی، ظاهر را در CSS به شکل کلاس تعریف کن و در JavaScript فقط کلاس را اضافه یا حذف کن:

CSS

```
.dark   { background: #222; color: #eee; }
.hidden { display: none; }
.error  { border: 2px solid red; }
```

JavaScript

```
themeBtn.addEventListener("click", () => {
  const isDark = document.body.classList.toggle("dark");  // state: dark on or off
  themeBtn.textContent = isDark ? "Light mode" : "Dark mode";
});
```

این کار سه لایه‌ی صفحه (ساختار با HTML، ظاهر با CSS، رفتار با JavaScript؛ یادآوری اسلاید ۳) را از هم جدا نگه می‌دارد، برگرداندنش با یک `remove` یا `toggle` ساده است و یک کلاس را می‌شود روی عنصرهای زیادی به کار برد. کلاس‌ها **وضعیت** را هم نشان می‌دهند: تب `active`، منوی `open`، آیتم `selected`، فیلد `error`؛ و `contains` می‌گوید الان در کدام وضعیت هستیم.

مثال اسلایدهای ۳۲ تا ۳۴: Change Image

doc.html

```
1  <!DOCTYPE html>
2  <html>
3  <body>
4      <img src="dambooldor.png" id="dambooldor" width="300" height="300">
5      <button>clickme</button>
6      <script>
7          const btn = document.querySelector('button');
8          btn.addEventListener('click', () => {
9              const img = document.getElementById("dambooldor");
10             img.setAttribute("src", "dambooldor2.png");
11         });
12     </script>
13 </body>
14 </html>
```

- **خط ۴:** تصویری ۳۰۰ در ۳۰۰ با `src="dambooldor.png"` و `id="dambooldor"` (شوخیِ اسلاید با نام «دامبلدور»).
- **خطوط ۷ و ۸:** دکمه را پیدا می‌کنیم و listener کلیک را وصل می‌کنیم.
- **خط ۹:** تصویر را با `id` پیدا می‌کنیم.
- **خط ۱۰:** مقدار attribute `src` را عوض می‌کنیم؛ مرورگر بلافاصله تصویر جدید را بار می‌کند و نشان می‌دهد.

قبل از کلیک (اسلاید ۳۳): تصویر اول (دامبلدور با رنگ‌های قرمز) و دکمه‌ی «clickme» درست کنار آن، چون `<img>` و `<button>` هر دو عنصرهای inline (عنصرهایی که در یک خط کنار هم می‌نشینند) هستند. **بعد از کلیک (اسلاید ۳۴):** تصویر دیگری (`dambooldor2.png`، با رنگ‌های آبی‌سبز) به‌جایش آمده؛ اندازه همچنان ۳۰۰ در ۳۰۰ است، چون به attributeهای `width` و `height` دست نزدیم، و دکمه هم سر جایش است.

چند نکته درباره‌ی همین مثال:

- `img.src = "dambooldor2.png";` (کار با property) همین اثر را دارد.
- بعد از کلیک، `img.getAttribute("src")` برابر `"dambooldor2.png"` است ولی `img.src` آدرس کامل را می‌دهد: `file:///home/mani/Projects/University/web-ta/sampleprojects/dom/dambooldor2.png`.
- کلیک دوم همان مقدار را دوباره می‌نویسد؛ نه تغییری دیده می‌شود و نه خطایی رخ می‌دهد.

برای اینکه هر کلیک تصویر را **بین دو حالت** عوض کند، باید با `getAttribute` (نه `img.src`) مقایسه کنیم:

```JavaScript
btn.addEventListener("click", () => {
  const img = document.getElementById("dambooldor");
  const next = img.getAttribute("src") === "dambooldor.png" ? "dambooldor2.png" : "dambooldor.png";
  img.setAttribute("src", next);
});
```

نکته

- `setAttribute` هم اضافه می‌کند هم تغییر می‌دهد؛ `getAttribute` همیشه رشته یا `null` برمی‌گرداند؛ `removeAttribute` روی attribute ناموجود خطا نمی‌دهد.
- برای کلاس‌ها `classList` را ترجیح بده: `add`، `remove` و `toggle` فقط همان کلاس را دست می‌زنند و بقیه را نگه می‌دارند.
- `toggle` وضعیت جدید را برمی‌گرداند؛ پس `const isDark = body.classList.toggle("dark")` هم وضعیت را عوض می‌کند و هم می‌گوید الان در چه وضعیتی هستیم.
- ظاهر را در CSS تعریف کن و در JavaScript فقط کلاس را عوض کن.

دام تستی

- `getAttribute` برای attribute ناموجود `null` برمی‌گرداند، نه `undefined` و نه رشته‌ی خالی.
- `getAttribute("width")` رشته‌ی `"300"` است ولی `img.width` عدد `300`؛ مقدار `dataset` هم همیشه رشته است (`"42" + 1` می‌شود `"421"`).
- `img.src` و `a.href` آدرس **کامل** را برمی‌گردانند؛ مقایسه‌ی `img.src === "a.png"` تقریباً هیچ‌وقت درست نیست.
- `setAttribute("disabled", "false")` دکمه را غیرفعال **می‌کند**؛ برای فعال کردن `removeAttribute("disabled")` لازم است.
- `setAttribute("class", …)` و `className = …` همه‌ی کلاس‌های قبلی را پاک می‌کنند.
- `classList.add(".active")` کلاسی با نقطه می‌سازد و `classList.add("a b")` خطای `InvalidCharacterError` می‌دهد.
- بعد از تایپ کاربر، `input.value` و `input.getAttribute("value")` با هم فرق دارند.

جمع‌بندی

attributeها: `setAttribute` (افزودن/تغییر)، `getAttribute` (خواندن؛ رشته یا `null`)، `removeAttribute` (حذف)، `hasAttribute` (وجود). property همان مقدار را با نوع واقعی و وضعیت فعلی نشان می‌دهد (`img.src` مطلق، `input.value` فعلی). کلاس‌ها: `classList.add/remove/toggle/contains/replace` ؛ `className` و `setAttribute("class")` همه را جایگزین می‌کنند. استایل پویا = تعریف کلاس در CSS + عوض کردن کلاس در JavaScript.

تست‌های تمرینی

۲۱ تست · شماره‌ی ۸۸۱ تا ۹۰۱

۸۸۱ آسان

خروجی کد زیر کدام است؟

HTML

```
<img id="pic" src="cat.png">
```

JavaScript

```
const pic = document.getElementById("pic");
console.log(pic.getAttribute("alt"));
```

۱) `undefined`
۲) `null`
۳) `false`
۴) `Uncaught TypeError`

۸۸۲ آسان

کدام دستور attribute `title` را به‌طور **کامل** از عنصر `el` حذف می‌کند؟

۱) `el.setAttribute("title", null)`
۲) `el.setAttribute("title", "")`
۳) `el.removeAttribute("title")`
۴) `el.classList.remove("title")`

۸۸۳ آسان

خروجی کد زیر کدام است؟

HTML

```
<a id="link" href="/home">Home</a>
```

JavaScript

```
const link = document.getElementById("link");
link.setAttribute("href", "/about");
link.setAttribute("target", "_blank");
console.log(link.getAttribute("href"), link.getAttribute("target"), link.attributes.length);
```

۱) `/home _blank 3`
۲) `/about _blank 2`
۳) `/about null 3`
۴) `/about _blank 3`

۸۸۴ متوسط

خروجی کد زیر کدام است؟

HTML
```
<div id="box">Box</div>
```

JavaScript
```
const box = document.getElementById("box");
box.removeAttribute("title");
console.log(box.hasAttribute("title"), box.hasAttribute("id"));
```

۱) `Uncaught NotFoundError`
۲) `false true`
۳) `true true`
۴) `undefined true`

۸۸۵ متوسط

خروجی کد زیر کدام است؟

HTML
```
<img src="a.png" width="300">
```

JavaScript
```
const img = document.querySelector("img");
console.log(typeof img.getAttribute("width"), typeof img.width);
```

۱) `string number`
۲) `number number`
۳) `string string`
۴) `number string`

۸۸۶ سخت

خروجی کد زیر کدام است؟

HTML
```
<button id="send">Send</button>
```

JavaScript
```
const send = document.getElementById("send");
send.setAttribute("disabled", "false");
console.log(send.disabled);
send.removeAttribute("disabled");
console.log(send.disabled);
```

۱) `false` `false`
۲) `false` `true`
۳) `true` `false`
۴) `true` `true`

۸۸۷ آسان

در مثال «Change Image» (اسلاید ۳۲) بعد از کلیک روی دکمه‌ی «clickme»، طبق اسلاید ۳۴ چه تغییری روی صفحه دیده می‌شود؟

۱) تصویر و دکمه هر دو حذف می‌شوند و تصویر جدیدی جای آن‌ها می‌آید.

۲) فقط تصویر عوض می‌شود؛ اندازه‌ی ۳۰۰ در ۳۰۰ و دکمه بدون تغییر می‌مانند.

۳) تصویر عوض می‌شود ولی اندازه‌اش به اندازه‌ی واقعی فایل جدید برمی‌گردد.

۴) تصویر جدید زیر تصویر قبلی اضافه می‌شود.

۸۸۸ متوسط

صفحه‌ی زیر همان کد اسلاید ۳۲ است. کاربر روی دکمه کلیک می‌کند. خروجی کدام است؟

HTML

```html
<img src="dambooldor.png" id="dambooldor" width="300" height="300">
<button>clickme</button>
```

JavaScript

```javascript
const btn = document.querySelector('button');
btn.addEventListener('click', () => {
    const img = document.getElementById("dambooldor");
    img.setAttribute("src", "dambooldor2.png");
});
```

بعد از کلیک این دستور اجرا می‌شود: `console.log(pic.getAttribute("src"), pic.getAttribute("width"));` که در آن `pic` همان تصویر `#dambooldor` است.

۱) `dambooldor2.png 300`

۲) `dambooldor.png 300`

۳) `dambooldor2.png null`

۴) `dambooldor.png dambooldor2.png`

۸۸۹ سخت

صفحه از آدرس `https://quiz.example/gallery/index.html` باز شده است. خروجی کد زیر کدام است؟

HTML

```html
<img src="day.png">
```

JavaScript

```javascript
const img = document.querySelector("img");
img.setAttribute("src", "cat.png");
console.log(img.getAttribute("src"));
console.log(img.src);
```

۱) `cat.png` `cat.png`

۲) `cat.png` `https://quiz.example/gallery/cat.png`

۳) `https://quiz.example/gallery/cat.png` `cat.png`

۴) `https://quiz.example/cat.png` `cat.png`

۸۹۰ متوسط

`<input>` زیر را در نظر بگیر. کاربر محتوای کادر را پاک می‌کند و `Sara` را تایپ می‌کند. خروجی کدام است؟

```HTML
<input id="name" value="guest">
```

```JavaScript
const input = document.getElementById("name");
input.addEventListener("input", () => {
  console.log(input.value, input.getAttribute("value"));
});
```

۱) `Sara Sara`
۲) `guest guest`
۳) `guest Sara`
۴) `Sara guest`

۸۹۱ سخت

صفحه از آدرس `https://quiz.example/` باز شده و کاربر **سه بار** روی دکمه کلیک می‌کند. خروجی کدام است؟

```HTML
<img id="pic" src="day.png">
<button id="switch">Switch</button>
```

```JavaScript
const pic = document.getElementById("pic");
document.getElementById("switch").addEventListener("click", () => {
  if (pic.src === "day.png") {
    pic.src = "night.png";
  } else {
    pic.src = "day.png";
  }
  console.log(pic.getAttribute("src"));
});
```

۱) `night.png` `day.png` `night.png`
۲) `night.png` `night.png` `night.png`
۳) `day.png` `night.png` `day.png`
۴) `day.png` `day.png` `day.png`

۸۹۲ سخت

خروجی کد زیر کدام است؟

HTML

```
<li id="item" data-product-id="42" data-price="9.5">Pen</li>
```

JavaScript

```
const item = document.getElementById("item");
console.log(item.dataset.productId + 1);
item.dataset.inStock = "yes";
console.log(item.getAttribute("data-in-stock"));
```

۱) `43` `yes`

۲) `NaN` `null`

۳) `421` `null`

۴) `421` `yes`

۸۹۳ آسان

خروجی کد زیر کدام است؟

HTML

```
<div id="box" class="card"></div>
```

JavaScript

```
const box = document.getElementById("box");
box.classList.add("active", "big");
box.classList.add("active");
console.log(box.className, box.classList.length);
```

۱) `card active big 3`

۲) `card active big active 4`

۳) `active big 2`

۴) `active big active 3`

۸۹۴ متوسط

خروجی کد زیر کدام است؟

HTML

```
<p class="a b c">Text</p>
```

JavaScript

```
const el = document.querySelector("p");
el.classList.remove("ghost");
el.classList.remove("a", "c");
console.log(el.className, el.classList.contains("b"));
```

۱) `Uncaught NotFoundError`

۲) `a b c true`

۳) `b false`

۴) `b true`

۸۹۵ متوسط

خروجی کد زیر کدام است؟

```javascript
const body = document.body;
console.log(body.classList.toggle("dark"));
console.log(body.classList.toggle("dark"));
console.log(body.classList.toggle("dark", true));
console.log(body.className);
```

۱) `undefined` `undefined` `undefined` `dark`

۲) `true` `false` `true` `dark`

۳) `false` `true` `false` `dark`

۴) `true` `true` `true` `dark`

۸۹۶ متوسط

خروجی کد زیر کدام است؟

```html
<p id="p" class="note big">Hi</p>
```

```javascript
const p = document.getElementById("p");
p.setAttribute("class", "red");
p.classList.add("bold");
p.className += " small";
console.log(p.className);
```

۱) `note big red bold small`

۲) `red small`

۳) `red bold small`

۴) `bold small`

۸۹۷ سخت

خروجی کد زیر کدام است؟

```html
<div id="tab">Home</div>
```

```javascript
const tab = document.querySelector("#tab");
tab.classList.add(".active");
console.log(tab.classList.contains("active"), tab.className);
```

۱) `false .active`

۲) `true active`

۳) `true .active`

۴) `Uncaught SyntaxError`

۸۹۸ سخت

خروجی کد زیر کدام است؟

```HTML
<button>Save</button>
```

```JavaScript
const btn = document.querySelector("button");
btn.classList.add("btn primary");
console.log(btn.className);
```

۱) `btn primary`
۲) `Uncaught InvalidCharacterError`
۳) `btn`
۴) `btn,primary`

۸۹۹ متوسط

می‌خواهیم هر کلیک روی دکمه، حالت تاریک صفحه را **روشن یا خاموش** کند (اگر کلاس `dark` هست برداشته شود و اگر نیست اضافه شود). در جای خالی `document.body.classList._____("dark");` کدام گزینه قرار می‌گیرد؟

۱) `add`
۲) `contains`
۳) `toggle`
۴) `replace`

۹۰۰ سخت

کدام جمله **نادرست** است؟

۱) `el.classList.contains("x")` اگر کلاس `x` وجود نداشته باشد، آن را اضافه می‌کند.
۲) `el.className = "x"` همه‌ی کلاس‌های قبلی را با `x` جایگزین می‌کند.
۳) `el.classList.add("x")` کلاس‌های قبلی را نگه می‌دارد.
۴) `el.classList.toggle("x")` اگر `x` وجود داشته باشد آن را حذف می‌کند و `false` برمی‌گرداند.

۹۰۱ آسان

طبق اسلاید ۳۱ کلاس‌ها برای «استایل پویا و تغییر وضعیت» به کار می‌روند. کدام گزینه بهترین دلیل برای اضافه و حذف کردن **کلاس‌های CSS** به‌جای نوشتن چندین `el.style.…` در JavaScript است؟

۱) تغییر `style` از طریق JavaScript در مرورگرهای جدید کار نمی‌کند.
۲) `classList` تنها راه تغییر رنگ یک عنصر است.
۳) با کلاس‌ها، صفحه بدون CSS هم ظاهر درستی خواهد داشت.
۴) ظاهر در CSS تعریف می‌شود و JavaScript فقط وضعیت را عوض می‌کند؛ برگرداندن آن ساده است و یک کلاس را می‌شود روی عنصرهای زیادی به کار برد.

۲-۱۱ کار با فرم‌ها (Forms)

منبع: JS in HTML، اسلاید ۳۵

درسنامه

فرم‌ها (form) راه اصلی گرفتن اطلاعات از کاربرند: ورود به سایت، جست‌وجو، ثبت‌نام، نوشتن نظر و... . نقش JavaScript این است که مقدارها را **بخواند**، **بررسی کند** (اعتبارسنجی یا validation)، به کاربر **بازخورد** بدهد و نگذارد صفحه بی‌دلیل دوباره بارگذاری شود. اسلاید ۳۵ همین را در شش نکته می‌گوید:

1. به عنصرهای فرم با `getElementById` یا `querySelector` دسترسی پیدا کن.
2. ورودی کاربر را با `.value` بخوان؛ برای checkbox و radio از `.checked` استفاده کن.
3. برای تعامل لحظه‌ای، رویدادهای `submit` یا `input` را وصل کن.
4. با `event.preventDefault()` جلوی بارگذاری دوباره‌ی صفحه هنگام submit را بگیر.
5. ورودی کاربر را **قبل از پردازش** اعتبارسنجی کن.
6. با متدهای DOM بازخورد پویا نشان بده.

یک تشبیه ساده

فرم مثل برگه‌ای است که مشتری در بانک پر می‌کند. JavaScript کارمند باجه است: برگه را **قبل از** فرستادن به اتاق پشتی (سرور) نگاه می‌کند، اگر جایی خالی یا غلط باشد همان‌جا به مشتری تذکر می‌دهد و اگر همه‌چیز درست بود کار را ادامه می‌دهد. اما بانک عاقل در اتاق پشتی هم برگه را دوباره بررسی می‌کند؛ چون ممکن است کسی از کنار باجه رد شده باشد!

یک فرم نمونه

HTML

```
<form id="signup">
  <input type="text" id="username" name="username" placeholder="Username">
  <input type="number" id="age" name="age" value="20">
  <input type="email" id="email" name="email">
  <label><input type="checkbox" id="terms" name="terms"> I accept the terms</label>
  <label><input type="radio" name="plan" value="free" checked> Free</label>
  <label><input type="radio" name="plan" value="pro"> Pro</label>
  <select id="city" name="city">
    <option value="thr">Tehran</option>
    <option value="isf">Isfahan</option>
  </select>
  <textarea id="bio" name="bio"></textarea>
  <button type="submit" id="submitBtn">Sign up</button>
  <p id="feedback"></p>
</form>
```

۱) دسترسی به عنصرهای فرم

```JavaScript
const form = document.getElementById("signup");
const username = document.getElementById("username");
const email = document.querySelector("#email");
const terms = document.querySelector('input[name="terms"]');   // attribute selector

// background: form.elements holds every control of the form, by name or id
const city = form.elements.city;        // same as form.elements["city"]
console.log(form.elements.length);      // number of controls inside the form
```

`form.elements` (مجموعه‌ی همه‌ی کنترل‌های یک فرم) همان شاخه‌ی `forms → elements` در نمودار اسلاید ۱۲ است؛ `document.forms` هم فهرست همه‌ی فرم‌های صفحه را دارد.

۲) خواندن مقدارها: `.value` و `.checked`

کنترل	چه چیزی را بخوانیم؟	نمونه‌ی نتیجه
`<input type="text">` و `<textarea>`	`.value`	`"ali"`
`<input type="number">`	`.value` (**رشته!**)	`"20"`
`<select>`	`.value` = مقدار `value` گزینه‌ی انتخاب‌شده	`"isf"`
checkbox	`.checked`	`true` یا `false`
radio	`.checked` هر کدام، یا پیدا کردن گزینه‌ی تیک‌خورده	`"free"`

`.value` همیشه رشته است

```JavaScript
const age = document.getElementById("age");   // <input type="number" value="20">
console.log(age.value + 1);           // "201" -> string concatenation!
console.log(Number(age.value) + 1);   // 21
console.log(age.valueAsNumber + 1);   // 21 (only for number and date inputs)
```

حتی وقتی `type="number"` است، `.value` رشته برمی‌گرداند. قبل از هر محاسبه با `Number()` تبدیلش کن؛ و یادت باشد `Number("")` برابر `0` و `Number("abc")` برابر `NaN` (Not a Number؛ «عدد نیست») است. پس **اول خالی بودن** را بررسی کن.

checkbox: وضعیت در `.checked` است

`.checked` یک مقدار بولی است: تیک خورده `true`، نخورده `false`. یک checkbox به‌طور پیش‌فرض رشته‌ی `"on"` است، **چه تیک خورده باشد چه نه**؛ پس هیچ‌وقت وضعیت را از `.value` نخوان.

radio: یک گروه، یک انتخاب

radioهایی که `name` یکسان دارند یک گروه‌اند و در هر لحظه فقط **یکی** از آن‌ها می‌تواند انتخاب شده باشد. برای پیدا کردن گزینه‌ی انتخاب‌شده از شبه‌کلاس `:checked` در CSS استفاده می‌کنیم:

```JavaScript
const chosen = document.querySelector('input[name="plan"]:checked');
console.log(chosen ? chosen.value : "nothing selected");   // "free"
```

اگر هیچ گزینه‌ای انتخاب نشده باشد `chosen` برابر `null` است و `chosen.value` خطای `TypeError` می‌دهد؛ به همین دلیل اول `null` بودن را بررسی کرده‌ایم.

۳) رویدادهای فرم: `submit`، `input` و `change`

رویداد	روی چه چیزی؟	کِی رخ می‌دهد؟
`submit`	خودِ `<form>`	وقتی فرم ارسال می‌شود: کلیک روی دکمه‌ی ارسال یا زدن Enter داخل یک کادر متن
`input`	کنترل‌ها	با **هر** تغییر مقدار: تایپ هر حرف، پاک کردن، paste
`change`	کنترل‌ها	وقتی تغییر «نهایی» شد: در کادر متن وقتی کاربر از کادر خارج شود (blur) و مقدار عوض شده باشد؛ در checkbox، radio و select بلافاصله بعد از انتخاب

listener رویداد `submit` را به **فرم** وصل کن، نه به دکمه: رویداد `submit` روی خودِ فرم رخ می‌دهد (`event.target` آن فرم است)، همه‌ی راه‌های ارسال را پوشش می‌دهد و جایی است که `preventDefault()` جلوی ارسال را می‌گیرد. listener `submit` روی دکمه هرگز اجرا نمی‌شود.

برای بازخورد لحظه‌ای از `input` استفاده کن؛ مثلاً یک شمارنده‌ی کاراکتر:

```JavaScript
const bio = document.getElementById("bio");
const counter = document.getElementById("counter");   // e.g. <span id="counter"></span>
bio.addEventListener("input", () => {
  counter.textContent = bio.value.length + " / 100";
});
```

۴) جلوگیری از reload با `preventDefault`

رفتار پیش‌فرض مرورگر هنگام submit این است که داده‌ها را به آدرسِ attribute `action` فرم (یا اگر نبود، به همان صفحه) بفرستد و **صفحه‌ی جدیدی بار کند**. یعنی صفحه دوباره بارگذاری (reload) می‌شود و همه‌ی متغیرهای JavaScript و پیام‌هایی که در DOM نشان داده بودیم از بین می‌روند. همان‌طور که در درس شیء رویداد دیدیم، `event.preventDefault()` رفتار پیش‌فرض را لغو می‌کند تا خودمان داده را با JavaScript پردازش کنیم:

```JavaScript
form.addEventListener("submit", (event) => {
  event.preventDefault();   // stop the page reload
  console.log("Form handled by JavaScript");
});
```

۵ و ۶) اعتبارسنجی قبل از پردازش و بازخورد پویا

قاعده‌های رایج اعتبارسنجی: حذف فاصله‌های اول و آخر با `trim()`، بررسی خالی بودن، طول، یک بررسی ساده‌ی ایمیل، و بازه‌ی عدد. اگر چیزی غلط بود پیام بده و با `return` از ادامه‌ی کار جلوگیری کن:

JavaScript

```
const feedback = document.getElementById("feedback");

function showMessage(text, isError) {
  feedback.textContent = text;                    // dynamic feedback with the DOM
  feedback.classList.toggle("error", isError);
  feedback.classList.toggle("success", !isError);
}

form.addEventListener("submit", (event) => {
  event.preventDefault();
  const name = username.value.trim();
  const mail = email.value.trim();
  const ageNum = Number(age.value);

  if (name === "") {
    showMessage("Username is required", true);
    return;
  }
  if (name.length < 3) {
    showMessage("Username must be at least 3 characters", true);
    return;
  }
  if (!mail.includes("@") || !mail.includes(".")) {
    showMessage("Please enter a valid email", true);
    return;
  }
  if (age.value === "" || ageNum < 1) {
    showMessage("Please enter your age", true);
    return;
  }
  if (!terms.checked) {
    showMessage("Please accept the terms", true);
    return;
  }
  showMessage("Welcome, " + name + "!", false);
  form.reset();   // clear the form for the next user
});
```

بازخورد پویا فقط پیام نیست؛ مثلاً می‌شود دکمه‌ی ارسال را تا وقتی قوانین پذیرفته نشده غیرفعال نگه داشت:

JavaScript

```
const submitBtn = document.getElementById("submitBtn");
submitBtn.disabled = true;
terms.addEventListener("change", () => {
  submitBtn.disabled = !terms.checked;   // enable only when the box is checked
});
```

ابزارهای خودِ HTML و یک هشدار امنیتی

attributeهایی مثل `required` (اجباری)، `minlength`، `maxlength`، `min`، `max` و `type="email"` را مرورگر **قبل از** رخ دادن `submit` بررسی می‌کند؛ اگر کاربر با دکمه یا Enter فرم را بفرستد و یکی از این قیدها رعایت نشده باشد، مرورگر پیام خودش را نشان می‌دهد و رویداد `submit` اصلاً رخ نمی‌دهد. اعتبارسنجی با JavaScript قاعده‌ها و پیام‌های دلخواه را اضافه می‌کند. اما

یادت باشد: اعتبارسنجی سمت کاربر فقط برای **راحتی کاربر** است و کاربرِ بدخواه می‌تواند دورش بزند؛ سرور **همیشه** باید داده را دوباره بررسی کند.

`form.reset()` و زمانِ خواندن مقدار

`form.reset()` همه‌ی کنترل‌ها را به **مقدار اولیه**‌شان برمی‌گرداند؛ یعنی همان مقدار attribute `value` یا `checked` نوشته‌شده در HTML، نه لزوماً خالی (در فرم نمونه، `age` دوباره `"20"` می‌شود).

مقدار را **داخل** listener بخوان، نه یک بار هنگام بارگذاری صفحه:

```JavaScript
const cityName = document.getElementById("cityName").value;   // read ONCE at load: ""
form.addEventListener("submit", (e) => {
  e.preventDefault();
  console.log(cityName);   // always the old (empty) value, never what the user typed
});
```

نکته

- هر چیزی که از `.value` می‌آید رشته است؛ برای حساب و مقایسه‌ی عددی `Number()` را فراموش نکن.
- وضعیت checkbox و radio را با `.checked` بخوان؛ گزینه‌ی انتخاب‌شده‌ی یک گروه radio را با `querySelector('input[name="..."]:checked')`.
- `submit` روی فرم + `preventDefault()` + اعتبارسنجی + پیام با `textContent` و `classList` = الگوی استاندارد کار با فرم.
- `input` برای واکنش لحظه‌ای و `change` برای وقتی که انتخاب کاربر نهایی شد.

دام تستی

- `"2" + "3"` می‌شود `"23"`، نه `5`؛ مقدار دو کادر را بدون `Number()` جمع نکن.
- `Number("")` برابر `0` است، پس کادر خالی بدون بررسی، «صفر» حساب می‌شود.
- `.value` یک checkbox همیشه `"on"` است، چه تیک خورده باشد چه نه.
- اگر هیچ radioی انتخاب نشده باشد، `querySelector('...:checked')` مقدار `null` می‌دهد و `.value` روی آن `TypeError` است.
- listener `submit` را به دکمه وصل نکن؛ رویداد `submit` روی فرم رخ می‌دهد.
- بدون `preventDefault()` صفحه reload می‌شود و پیام‌ها و داده‌های JavaScript از بین می‌روند.
- مقداری که بیرون از listener (هنگام بارگذاری) خوانده شده، با تایپ کاربر به‌روز نمی‌شود.
- `form.reset()` به مقدار attribute `value` برمی‌گرداند، نه لزوماً به رشته‌ی خالی.

جمع‌بندی

پیدا کن (`getElementById` / `querySelector` / `form.elements`) ← بخوان (`.value` رشته‌ای، `.checked` بولی) ← در `submit` فرم `preventDefault()` کن ← با `trim` و شرط‌ها اعتبارسنجی کن ← با DOM پیام بده. `input` = هر تغییر، `change` = تغییر نهایی. اعتبارسنجی مرورگر برای راحتی است و سرور هم باید بررسی کند.

تست‌های تمرینی

۲۲ تست · شماره‌ی ۹۰۲ تا ۹۲۳

۹۰۲ آسان

کاربر در کادر زیر عدد ۵ را تایپ می‌کند. خروجی کدام است؟

```html
<input id="age" type="number">
```

```javascript
const age = document.getElementById("age");
age.addEventListener("input", () => {
  console.log(typeof age.value, age.value);
});
```

۱) `number 5`
۲) `string 5`
۳) `number NaN`
۴) `undefined 5`

۹۰۳ آسان

خروجی کد زیر کدام است؟

```html
<input id="a" value="2">
<input id="b" value="3">
```

```javascript
const a = document.getElementById("a").value;
const b = document.getElementById("b").value;
console.log(a + b, Number(a) + Number(b));
```

۱) `5 5`
۲) `23 23`
۳) `23 5`
۴) `5 23`

۹۰۴ سخت

خروجی کد زیر کدام است؟

```html
<input id="price" value="">
<input id="qty" type="number" value="3">
```

```javascript
const price = Number(document.getElementById("price").value);
const qty = document.getElementById("qty").value;
console.log(price, qty * 2, qty + 2);
```

۱) `0 6 32`
۲) `NaN 6 32`
۳) `0 6 5`
۴) `NaN 6 5`

۹۰۵ آسان

برای فهمیدن این‌که کاربر یک checkbox را تیک زده است یا نه، کدام را باید خواند؟

۱) `checkbox.value`
۲) `checkbox.selected`
۳) `checkbox.checked()`
۴) `checkbox.checked`

۹۰۶ متوسط

خروجی کد زیر کدام است؟ (کاربر هنوز به checkbox دست نزده است.)

HTML
```
<input type="checkbox" id="agree">
```

JavaScript
```
const agree = document.getElementById("agree");
console.log(agree.value, agree.checked);
```

۱) `false false`
۲) `on false`
۳) `on true`
۴) `undefined false`

۹۰۷ سخت

خروجی کد زیر کدام است؟

HTML
```
<input type="radio" name="plan" value="free" checked>
<input type="radio" name="plan" value="pro">
```

JavaScript
```
const first = document.querySelector('input[name="plan"]:checked');
console.log(first.value);
document.querySelector('input[value="pro"]').checked = true;
console.log(document.querySelector('input[name="plan"]:checked').value);
console.log(document.querySelectorAll('input[name="plan"]:checked').length);
```

۱) `free` `pro` `1`
۲) `free` `pro` `2`
۳) `on` `on` `1`
۴) `free` `free` `1`

۹۰۸ سخت

خروجی کد زیر کدام است؟ (کاربر هیچ گزینه‌ای را انتخاب نکرده است.)

HTML

```html
<input type="radio" name="size" value="S">
<input type="radio" name="size" value="M">
```

JavaScript

```javascript
const chosen = document.querySelector('input[name="size"]:checked');
console.log(chosen);
console.log(chosen.value);
```

۱) `undefined` `undefined`
۲) `null` `undefined`
۳) `null` `Uncaught TypeError`
۴) `S` `S`

۹۰۹ متوسط

خروجی کد زیر کدام است؟

HTML

```html
<select id="city">
  <option value="thr">Tehran</option>
  <option value="isf" selected>Isfahan</option>
</select>
<textarea id="bio">Hello</textarea>
```

JavaScript

```javascript
const city = document.getElementById("city");
const bio = document.getElementById("bio");
console.log(city.value, bio.value.length);
```

۱) `Isfahan 5`
۲) `thr 5`
۳) `isf undefined`
۴) `isf 5`

۹۱۰ آسان

کدام رویداد با **هر بار** تغییر محتوای یک کادر متن (مثلاً با تایپ هر حرف) رخ می‌دهد؟

۱) `input`
۲) `change`
۳) `submit`
۴) `load`

۹۱۱ متوسط

خروجی کد زیر کدام است؟

HTML

```
<input id="name">
```

JavaScript

```
const nameInput = document.getElementById("name");
nameInput.addEventListener("input", () => console.log("input:", nameInput.value));
nameInput.addEventListener("change", () => console.log("change:", nameInput.value));
```

کاربر حرف‌های `A`، `l` و `i` را یکی‌یکی تایپ می‌کند و سپس روی جای دیگری از صفحه کلیک می‌کند تا کادر از حالت انتخاب خارج شود.

۱) `input: Ali` `change: Ali`

۲) `input: A` `input: Al` `input: Ali` `change: Ali`

۳) `change: A` `change: Al` `change: Ali` `input: Ali`

۴) `input: A` `change: A` `input: Al` `change: Al` `input: Ali` `change: Ali`

۹۱۲ آسان

رویداد `submit` روی کدام عنصر **رخ می‌دهد** (یعنی `event.target` آن کدام عنصر است)؟

۱) دکمه‌ی ارسال (`<button>`)

۲) شیء `window`

۳) خودِ عنصر `<form>`

۴) هر یک از `<input>`های داخل فرم

۹۱۳ سخت

کاربر فرم زیر را ارسال می‌کند. خروجی کدام است؟

HTML

```
<form id="f">
  <input id="user" value="ali">
  <button id="go">Go</button>
</form>
```

JavaScript

```
document.getElementById("go").addEventListener("submit", (e) => {
  e.preventDefault();
  console.log("button handler");
});
document.getElementById("f").addEventListener("submit", () => {
  console.log("form handler");
});
```

رویداد `submit` روی فرم رخ می‌دهد. در خروجی، سطر `[default runs]` یعنی رفتار پیش‌فرض (ارسال فرم و بارگذاری دوباره‌ی صفحه) **لغو نشده** و `[default prevented]` یعنی لغو شده است.

۱) `button handler` `form handler` `[default prevented]`

۲) `button handler` `[default prevented]`

۳) `form handler` `[default prevented]`

۴) `form handler` `[default runs]`

۹۱۴ آسان

در listener رویداد `submit` یک فرم، دستور `event.preventDefault()` چه می‌کند؟

۱) همه‌ی کادرهای فرم را خالی می‌کند.

۲) جلوی رفتار پیش‌فرض مرورگر، یعنی ارسال فرم و بارگذاری دوباره‌ی صفحه، را می‌گیرد تا خودمان داده را با JavaScript پردازش کنیم.

۳) کادرهای فرم را به‌طور خودکار اعتبارسنجی می‌کند.

۴) جلوی اجرای listenerهای دیگرِ همین فرم را می‌گیرد.

۹۱۵ متوسط

کاربر فرم زیر را ارسال می‌کند. خروجی کدام است؟

HTML

```
<form id="search">
  <input name="q" value="js">
  <button>Search</button>
</form>
```

JavaScript

```
const form = document.getElementById("search");
form.addEventListener("submit", (e) => {
  e.preventDefault();
  console.log("searching for", form.elements.q.value);
});
```

سطر `[default runs]` یعنی رفتار پیش‌فرض (ارسال و reload) لغو نشده و `[default prevented]` یعنی لغو شده است.

۱. `searching for js` `[default prevented]`

۲. `searching for js` `[default runs]`

۳. `[default prevented]`

۴. `searching for undefined` `[default prevented]`

۹۱۶ سخت

خروجی کد زیر کدام است؟

HTML

```
<form id="signup">
  <input id="username" value="   ">
  <p id="msg"></p>
  <button>Sign up</button>
</form>
```

JavaScript

```
const form = document.getElementById("signup");
const msg = document.getElementById("msg");
function show(text) {
  msg.textContent = text;
  console.log(text);
}
form.addEventListener("submit", (e) => {
  e.preventDefault();
  const username = document.getElementById("username").value.trim();
  if (username === "") {
    show("Username is required");
    return;
  }
  if (username.length < 3) {
    show("Too short");
    return;
  }
  show("Welcome, " + username + "!");
});
```

کاربر سه بار فرم را ارسال می‌کند: بار اول با همان مقدار اولیه (سه فاصله)، بار دوم بعد از نوشتن `" al "` و بار سوم بعد از نوشتن `"  sara "` در کادر (نقل‌قول‌ها فقط برای نشان دادن فاصله‌ها هستند).

۱) `Username is required` `Welcome, al!` `Welcome, sara!`

۲) `Username is required` `Username is required` `Welcome, sara!`

۳) `Username is required` `Too short` `Welcome, sara!`

۴) `Too short` `Too short` `Welcome, sara!`

۹۱۷ سخت

کاربر `Tehran` را در کادر تایپ می‌کند و سپس فرم را ارسال می‌کند. خروجی کدام است؟

HTML

```
<form id="f">
  <input id="city">
  <button>Save</button>
</form>
```

JavaScript

```
const city = document.getElementById("city").value;
document.getElementById("f").addEventListener("submit", (e) => {
  e.preventDefault();
  console.log("City: " + city);
});
```

۱) `City: undefined`
۲) `City: null`
۳) `City: Tehran`
۴) `City:`

۹۱۸ متوسط

کاربر ابتدا `Hello` و سپس، با ادامه‌ی تایپ، `Hello world!` را در کادر می‌نویسد. در پایان کد `console.log(left.textContent, left.className);` اجرا می‌شود. خروجی آن کدام است؟

HTML

```
<textarea id="tweet"></textarea>
<span id="left">10</span>
```

JavaScript

```
const tweet = document.getElementById("tweet");
const left = document.getElementById("left");
tweet.addEventListener("input", () => {
  const remaining = 10 - tweet.value.length;
  left.textContent = remaining;
  left.classList.toggle("over", remaining < 0);
});
```

۱) `-2 over`
۲) `5`
۳) `-2`
۴) `12 over`

۹۱۹ متوسط

کاربر یک بار تیک checkbox را می‌زند و بعد دوباره آن را برمی‌دارد. خروجی کدام است؟

HTML

```
<form>
  <input type="checkbox" id="terms">
  <button id="send" disabled>Send</button>
</form>
```

JavaScript

```
const terms = document.getElementById("terms");
const send = document.getElementById("send");
terms.addEventListener("change", () => {
  send.disabled = !terms.checked;
  console.log(send.disabled);
});
```

۱) `true` `false`

۲) `false` `true`

۳) `false` `false`

۴) `on` `off`

۹۲۰ متوسط

خروجی کد زیر در Node.js کدام است؟

JavaScript

```
function validateEmail(value) {
  const email = value.trim();
  if (email === "") return "Email is required";
  if (!email.includes("@") || !email.includes(".")) return "Invalid email";
  return "OK";
}
console.log(validateEmail("   "));
console.log(validateEmail("ali@mail"));
console.log(validateEmail(" ali@mail.com "));
```

۱) `Invalid email` `Invalid email` `OK`

۲) `Email is required` `OK` `OK`

۳) `Email is required` `Invalid email` `OK`

۴) `Email is required` `Invalid email` `Invalid email`

۹۲۱ سخت

خروجی کد زیر کدام است؟

HTML

```
<form id="f">
  <input id="nameInput" value="guest">
</form>
```

JavaScript

```
const form = document.getElementById("f");
const nameInput = document.getElementById("nameInput");
nameInput.value = "Sara";
form.reset();
console.log(nameInput.value === "" ? "(empty)" : nameInput.value);
```

۱) `(empty)`
۲) `Sara`
۳) `undefined`
۴) `guest`

۹۲۲ متوسط

کدام جمله درباره‌ی فرم‌ها **نادرست** است؟

۱) `.value` یک `<input type="number">` همیشه رشته است.
۲) رویداد `submit` روی خودِ عنصر `<form>` رخ می‌دهد.
۳) رویداد `input` هنگام تایپ، با هر تغییر محتوای کادر رخ می‌دهد.
۴) اعتبارسنجی با JavaScript در مرورگر برای امنیت کافی است و لازم نیست سرور داده را دوباره بررسی کند.

۹۲۳ آسان

کاربر در کادر `ageInput` عدد ۲۰ را وارد کرده است. در جای خالی `const next = ______(ageInput.value) + 1;` کدام گزینه قرار گیرد تا `next` برابر **عدد** ۲۱ شود؟

۱) `Number`
۲) `String`
۳) `Boolean`
۴) `typeof`

۲-۱۲ شیء window: پنجره‌ی مرورگر

منبع: JS in HTML، اسلایدهای ۳۶ و ۳۷

درسنامه

تا این‌جا بیشتر با `document` (خودِ صفحه) کار کردیم. اما `document` داخل شیء بزرگ‌تری زندگی می‌کند: `window`، یعنی پنجره (یا تب) مرورگری که صفحه در آن باز شده است. در نمودار اسلاید ۱۲ دیدیم که `window` در بالاترین سطح است و `document` فقط یکی از فرزندان آن است.

یک تشبیه ساده

`window` را یک آپارتمان در نظر بگیر و `document` را اتاق نشیمنش که مبلمان (محتوای صفحه) در آن است. آپارتمان اتاق‌ها و وسایل دیگری هم دارد: پلاک آدرس روی در (`location`)، راهرویی که اتاق‌های قبلاً دیده‌شده را به یاد دارد (`history`)، شناسنامه‌ی ساختمان (`navigator`)، اطلاعات پنجره‌ی بزرگ رو به خیابان (`screen`)، کمد (`localStorage`) و ساعت زنگ‌دار (تایمرها).

```
window
├── document ── anchors, links, images, scripts, forms ── elements, ...
├── event
├── frames
├── location
├── history
├── navigator ── plugins, mimeTypes
└── screen
```

اسلاید ۳۶ درباره‌ی `window` می‌گوید:

- پنجره‌ی مرورگری را نشان می‌دهد که صفحه‌ی وب در آن است.
- همه‌ی اشیا، توابع و متغیرهای **سراسری** (global) JavaScript به آن تعلق دارند.
- متدهایی مثل `alert()`، `confirm()` و `prompt()` دارد.
- به `location`، `history`، `navigator` و موارد دیگر دسترسی می‌دهد.
- تایمرهای `setTimeout()` و `setInterval()` را فراهم می‌کند.
- propertyهای `innerWidth`، `innerHeight`، `scrollX` و `scrollY` را دارد.

۱) `window` همان شیء سراسری است

در مرورگر، `window` شیء سراسری (global object) است؛ یعنی هر چیزی که «همه‌جا در دسترس است» در واقع property آن است:

JavaScript

```
var a = 1;            // global var          -> becomes window.a
function greet() {}   // global function     -> becomes window.greet
let b = 2;            // global let          -> NOT a window property
const c = 3;          // global const        -> NOT a window property
class Car {}          // global class        -> NOT a window property
console.log(window.a, window.b, window.c, typeof window.greet);  // 1 undefined undefined
"function"
```

- `var` و اعلان تابع (function declaration) در سطح بالای یک اسکریپت، property شیء `window` می‌شوند.
- `let` ، `const` و `class` هم سراسری‌اند (از همه‌جا قابل استفاده‌اند) ولی در جای جداگانه‌ای نگه داشته می‌شوند و property `window` **نمی‌شوند**.
- همه‌ی چیزهای آماده‌ی مرورگر روی `window` هستند: `window.document` ، `window.console` ، `window.alert` ، `window.setTimeout` ، `window.localStorage` و... . به همین دلیل نوشتن `window.` **اختیاری** است: `alert("Hi")` همان `window.alert("Hi")` است و `alert === window.alert` برابر `true` .
- در سطح بالای یک اسکریپت معمولی `this === window` است؛ و در حالت غیر strict، تابعی که به‌تنهایی صدا زده شود (`show()`) هم `this` را برابر `window` می‌بیند (یادآوری `this` از فصل اول).
- `globalThis` نام استاندارد شیء سراسری در **هر** محیطی است: در مرورگر همان `window` و در Node.js شیء `global` . در Node.js اصلاً `window` وجود ندارد (`ReferenceError: window is not defined`) و `var` سطح بالای یک فایل هم property شیء سراسری نمی‌شود.

۲) پنجره‌های گفت‌وگو: `alert` ، `confirm` و `prompt`

```JavaScript
alert("Saved!");                          // message + OK          -> undefined
const sure = confirm("Delete this item?");    // OK / Cancel           -> true / false
const name = prompt("Your name?", "Guest");   // text box + OK/Cancel  -> string or null
```

تابع	کاربر چه می‌بیند؟	مقدار برگشتی
`alert(msg)`	پیام + دکمه‌ی OK	`undefined`
`confirm(msg)`	پیام + OK و Cancel	Cancel ، `true` ← OK ← `false`
`prompt(msg, default)`	پیام + کادر متن (با مقدار پیش‌فرض اختیاری) + OK و Cancel	متن تایپ‌شده به‌صورت **رشته** (حتی اگر عدد تایپ شود؛ کادر خالی ← `""`)، و با Cancel ← `null`

هر سه پنجره **مودال و مسدودکننده** (modal و blocking) هستند: تا کاربر جواب ندهد، صفحه قفل است و اجرای JavaScript **در همان خط منتظر می‌ماند**؛ حتی callbackهای تایمرها و رویدادها هم تا بعد از تمام شدن کد فعلی صبر می‌کنند. ظاهرشان را هم نمی‌شود با CSS تغییر داد؛ برای همین سایت‌های امروزی معمولاً پنجره‌های دلخواه با HTML و CSS می‌سازند، ولی این سه برای یادگیری و آزمایش خیلی راحت‌اند.

```JavaScript
const input = prompt("How old are you?");
if (input === null) {
  console.log("User cancelled");
} else {
  const age = Number(input);   // "20" -> 20
  console.log("Next year you will be " + (age + 1));
}
```

۳) `location` : آدرس صفحه

`window.location` شیئی است که **آدرس (URL) صفحه‌ی فعلی** را نگه می‌دارد؛ نه موقعیت جغرافیایی کاربر (آن کارِ `navigator.geolocation` است). برای آدرس `https://shop.example.com:8080/products/list.html?page=2#reviews` :

property	مقدار
`href`	کل آدرس
`protocol`	`"https:"`
`host`	`"shop.example.com:8080"` (نام میزبان + پورت)
`hostname`	`"shop.example.com"`
`port`	`"8080"`
`pathname`	`"/products/list.html"`
`search`	`"?page=2"` (رشته‌ی پرس‌وجو، query string)
`hash`	`"#reviews"`
`origin`	`"https://shop.example.com:8080"`

`location` فقط خواندنی نیست:

```JavaScript
location.href = "https://example.com";    // navigate to a new page (adds a history entry)
location.assign("https://example.com");   // the same as above
location.replace("https://example.com"); // navigate WITHOUT a history entry (Back will not return here)
location.reload();                        // reload the current page
```

۴) `history`، `navigator` و `screen`

- **`history`** (تاریخچه‌ی همین تب): `history.back()` مثل دکمه‌ی Back مرورگر، `history.forward()` مثل دکمه‌ی Forward، `history.go(-2)` دو صفحه به عقب و `history.go(1)` یک صفحه به جلو؛ `history.length` تعداد صفحه‌های تاریخچه‌ی این تب است. برای حفظ حریم خصوصی، آدرس صفحه‌های تاریخچه را **نمی‌شود خواند**.
- **`navigator`** (اطلاعات مرورگر): `navigator.userAgent` (رشته‌ای که مرورگر و سیستم‌عامل را معرفی می‌کند)، `navigator.language` (زبان مرورگر، مثلاً `"fa-IR"` یا `"en-US"`)، `navigator.onLine` (آیا به اینترنت وصل است؟) و امکاناتی مثل `navigator.geolocation` و `navigator.clipboard`. `plugins` و `mimeTypes` نمودار اسلاید ۱۲ هم زیرمجموعه‌ی همین شیءاند.
- **`screen`** (اطلاعات **مانیتور** کاربر): `screen.width`، `screen.height` و `screen.availWidth`. این‌ها اندازه‌ی کل صفحه‌نمایش است، نه اندازه‌ی پنجره‌ی مرورگر.

۵) تایمرها: `setTimeout` و `setInterval`

```JavaScript
const id = setTimeout(() => {
  console.log("runs once, after about 2 seconds");
}, 2000);
clearTimeout(id);   // cancel it before it runs

let count = 0;
const timer = setInterval(() => {
  count++;
  console.log("tick", count);
  if (count === 3) clearInterval(timer);   // stop after 3 ticks
}, 1000);
```

- `setTimeout(fn, ms)` تابع را **یک بار**، دست‌کم `ms` میلی‌ثانیه بعد اجرا می‌کند و یک **شناسه** (id؛ در مرورگر یک عدد) برمی‌گرداند. `clearTimeout(id)` آن را قبل از اجرا لغو می‌کند.
- `setInterval(fn, ms)` تابع را هر `ms` میلی‌ثانیه **تکرار** می‌کند تا با `clearInterval(id)` متوقف شود.
- callback تایمر فقط **بعد از تمام شدن کد همگامِ فعلی** اجرا می‌شود (event loop در فصل اول)؛ پس تأخیر `0` هم یعنی «بعداً»، نه «فوراً».
- خودِ تابع را بده، نه نتیجه‌ی صدا زدنش را: `setTimeout(sayHi, 1000)` درست است؛ `setTimeout(sayHi(), 1000)` تابع را **همین الان** اجرا می‌کند و نتیجه‌اش (`undefined`) را به `setTimeout` می‌دهد.

۶) اندازه و اسکرول: `innerWidth`، `innerHeight`، `scrollX` و `scrollY`

- `innerWidth` و `innerHeight`: عرض و ارتفاع **viewport** (ناحیه‌ای از پنجره‌ی مرورگر که صفحه در آن دیده می‌شود، شامل نوار اسکرول) بر حسب پیکسل. با تغییر اندازه‌ی پنجره عوض می‌شوند.
- `outerWidth` و `outerHeight`: اندازه‌ی **کل** پنجره‌ی مرورگر همراه با نوار ابزار و حاشیه‌ها. `screen.width` هم اندازه‌ی مانیتور است.
- `scrollX` و `scrollY`: صفحه چند پیکسل به‌ترتیب به **راست** و به **پایین** اسکرول شده است (بالای صفحه برابر `0` است).

```JavaScript
const backToTop = document.getElementById("backToTop");
window.addEventListener("scroll", () => {
  backToTop.hidden = window.scrollY < 300;   // show the button only after scrolling down
});
```

۷) صفحه‌ی واکنش‌گرا (responsive) با `window`

```JavaScript
function applyLayout() {
  if (window.innerWidth < 600) {
    document.body.classList.add("mobile");
  } else {
    document.body.classList.remove("mobile");
  }
}
applyLayout();                                  // once when the page loads
window.addEventListener("resize", applyLayout);  // and every time the window size changes
```

رویداد `resize` روی `window` هر بار که اندازه‌ی پنجره تغییر کند رخ می‌دهد. البته برای تغییرات **فقط ظاهری**، ابزار معمول CSS media query است (قاعده‌ای در CSS که فقط در اندازه‌های مشخصی از صفحه اعمال می‌شود، مثل `@media (max-width: 600px) { … }`) و JavaScript برای تغییر **رفتار** به کار می‌رود (مثلاً جمع کردن منو یا نمایش تعداد کمتری آیتم). `window.matchMedia("(max-width: 600px)").matches` همان شرط media query را در JavaScript بررسی می‌کند.

فعالیت کلاسی اسلاید ۳۷

۱. `alert` چه می‌کند و چه فرقی با `prompt` و `confirm` دارد؟ هر سه متد `window` هستند و پنجره‌ای مسدودکننده باز می‌کنند. `alert` فقط پیام و دکمه‌ی OK نشان می‌دهد و `undefined` برمی‌گرداند. `confirm` دکمه‌های OK و Cancel دارد و `true` یا `false` برمی‌گرداند. `prompt` یک کادر متن هم دارد و متن تایپ‌شده را به‌صورت رشته، یا با Cancel مقدار `null` را برمی‌گرداند.

۲. `location` چه چیزی را نشان می‌دهد؟ آدرس (URL) صفحه‌ی فعلی را، با بخش‌هایش (`href`، `protocol`، `host`، `hostname`، `port`، `pathname`، `search`، `hash`) و متدهایی برای رفتن به آدرس دیگر یا بارگذاری دوباره (`assign`، `replace`، `reload`). مقداردهی `location.href` کاربر را به صفحه‌ی جدید می‌برد.

۳. چطور با `window` صفحه را واکنش‌گرا کنیم؟ `window.innerWidth` و `innerHeight` را بخوانیم، به رویداد `resize` روی `window` گوش بدهیم و بر اساس اندازه کلاس‌ها یا استایل‌ها را عوض کنیم یا عنصرهایی را نشان دهیم یا پنهان کنیم (مثل کد بالا). برای رفتارهای وابسته به اسکرول هم `scrollX` و `scrollY` به کار می‌آیند.

۴. آیا `window` به‌جز `document` فرزندان دیگری دارد؟ بله. طبق نمودار اسلاید ۱۲: `event`، `frames`، `location`، `history`، `navigator` (با `plugins` و `mimeTypes`) و `screen`. علاوه بر این‌ها `localStorage`، `sessionStorage`، `console`، توابع تایمر و هر متغیر `var` یا تابع سراسری هم روی `window` هستند.

> **نکته**
>
> - هر چیز سراسری در مرورگر property `window` است، پس `window.` را می‌توانی ننویسی: `alert === window.alert`.
> - `confirm` بولی برمی‌گرداند و `prompt` رشته یا `null`؛ خروجی `prompt` را قبل از محاسبه با `Number()` تبدیل کن و `null` را جدا بررسی کن.
> - `location` = آدرس صفحه؛ `history` = عقب و جلو؛ `navigator` = اطلاعات مرورگر؛ `screen` = مانیتور؛ `innerWidth` = viewport.
> - هر `setInterval` را با `clearInterval` و شناسه‌ی خودش متوقف کن.

⚠ دام تستی

- `let`، `const` و `class` سراسری property `window` نمی‌شوند: `window.b` برای `let b = 2` برابر `undefined` است.
- `prompt("Age?") + 1` با ورودی ۲۰ می‌شود `"201"`؛ و اگر کاربر Cancel بزند، `Number(null)` برابر `0` است، نه `NaN`.
- `alert` مقدار `undefined` برمی‌گرداند، نه `true`.
- `location` موقعیت جغرافیایی نیست؛ آدرس صفحه است.
- `innerWidth` اندازه‌ی viewport است، نه مانیتور (`screen.width`) و نه کل پنجره (`outerWidth`).
- `setTimeout(sayHi(), 1000)` تابع را فوراً اجرا می‌کند؛ و حتی `setTimeout(fn, 0)` بعد از کد همگام اجرا می‌شود.
- در Node.js شیء `window` وجود ندارد؛ نام مشترک شیء سراسری `globalThis` است.

جمع‌بندی

`window` = پنجره‌ی مرورگر + شیء سراسری. فرزندانش: `document`، `location` (آدرس)، `history` (عقب و جلو)، `navigator` (مرورگر)، `screen` (مانیتور)، `localStorage` و... . پنجره‌های گفت‌وگو: `alert` ← `undefined`، `confirm` ← بولی، `prompt` ← رشته یا `null` (همه مسدودکننده). تایمرها: `setTimeout` / `clearTimeout` و `setInterval` / `clearInterval` . اندازه‌ها: `innerWidth` / `innerHeight` (viewport)، `scrollX` / `scrollY` (اسکرول) و رویداد `resize` برای صفحه‌ی واکنش‌گرا.

تست‌های تمرینی

۲۳ تست · شماره‌ی ۹۲۴ تا ۹۴۶

۹۲۴ آسان

طبق اسلاید ۳۶، شیء `window` چه چیزی را نشان می‌دهد؟

۱) پنجره (یا تب) مرورگر که صفحه در آن باز است؛ همه‌ی متغیرها، توابع و اشیای سراسری JavaScript به آن تعلق دارند.

۲) فقط محتوای HTML صفحه؛ دقیقاً همان چیزی که `document` نشان می‌دهد.

۳) پنجره‌های سیستم‌عامل (مثلاً همه‌ی پنجره‌های باز روی دسکتاپ) را مدیریت می‌کند.

۴) فقط عنصر `<body>` صفحه را نشان می‌دهد.

۹۲۵ آسان

طبق فعالیت کلاسی اسلاید ۳۷، آیا `window` به‌جز `document` فرزندان (اشیای زیرمجموعه‌ی) دیگری هم دارد؟

۱) خیر؛ `document` تنها فرزند `window` است.

۲) فقط `frames` (برای iframeها).

۳) بله؛ مثلاً `location`، `history`، `navigator` و `screen` (و همچنین `localStorage`، `console` و توابع تایمر).

۴) فقط در Node.js فرزند دیگری دارد.

۹۲۶ متوسط

خروجی کد زیر در مرورگر کدام است؟

```JavaScript
var a = 1;
let b = 2;
const c = 3;
function d() {}
console.log(window.a, window.b, window.c, typeof window.d);
```

۱) `1 2 3 function`

۲) `1 undefined undefined function`

۳) `undefined undefined undefined undefined`

۴) `1 2 undefined function`

۹۲۷ آسان

خروجی کد زیر در مرورگر کدام است؟

```JavaScript
console.log(alert === window.alert, window.document === document, globalThis === window);
```

۱) `false true true`

۲) `true false true`

۳) `true true false`

۴) `true true true`

۹۲۸ سخت

خروجی کد زیر **در مرورگر** (یک اسکریپت معمولی، بدون `"use strict"`) کدام است؟

```JavaScript
var color = "red";
function show() {
  console.log(this.color);
}
show();
console.log(this === window);
```

۱) `red` `true`

۲) `undefined` `true`

۳) `undefined` `false`

۴) `red` `false`

۹۲۹ متوسط

کدام جمله **درست** است؟

۱) در Node.js هم شیء `window` وجود دارد و همان کار مرورگر را انجام می‌دهد.

۲) `globalThis` در مرورگر به `window` و در Node.js به شیء سراسری Node (`global`) اشاره می‌کند.

۳) متغیر سراسری‌ای که با `let` تعریف شود، در مرورگر به property شیء `window` تبدیل می‌شود.

۴) برای صدا زدن `alert` حتماً باید `window.alert(...)` نوشت.

۹۳۰ آسان

کدام تابع پنجره‌ای با دو دکمه‌ی OK و Cancel نشان می‌دهد و مقدار بولی (`true` یا `false`) برمی‌گرداند؟

۱) `alert`　۲) `prompt`　۳) `confirm`　۴) `console.log`

۹۳۱ متوسط

کدام گزینه مقدار برگشتی سه تابع `alert`، `confirm` و `prompt` را درست بیان می‌کند؟

۱) `alert` ← `true`؛ `confirm` ← `true` / `false`؛ `prompt` ← عدد یا `null`

۲) `alert` ← `undefined`؛ `confirm` ← `"OK"` یا `"Cancel"`؛ `prompt` ← رشته یا `""`

۳) هر سه یک Promise برمی‌گردانند که بعد از جواب کاربر resolve می‌شود.

۴) `alert` ← `undefined`؛ `confirm` ← `true` / `false`؛ `prompt` ← رشته‌ی تایپ‌شده، یا `null` اگر Cancel زده شود

۹۳۲ متوسط

کاربر در پنجره‌ی confirm روی **Cancel** کلیک می‌کند. خروجی کدام است؟ (سطر `[confirm] …` یعنی پنجره‌ی confirm با آن پیام نمایش داده شده است.)

```javascript
const ok = confirm("Delete this item?");
if (ok) {
  console.log("deleted");
} else {
  console.log("kept");
}
console.log(typeof ok);
```

۱) `[confirm] Delete this item?` `kept` `boolean`

۲) `[confirm] Delete this item?` `deleted` `boolean`

۳) `[confirm] Delete this item?` `kept` `string`

۴) `[confirm] Delete this item?` `kept` `undefined`

۹۳۳ متوسط

کاربر در پنجره‌ی prompt عدد ۲۰ را تایپ می‌کند و OK را می‌زند. خروجی کدام است؟ (سطر `[prompt] …` یعنی پنجره‌ی prompt با آن پیام نمایش داده شده است.)

```JavaScript
const age = prompt("How old are you?");
console.log(age + 1);
console.log(Number(age) + 1);
```

۱) `[prompt] How old are you?` `21` `21`

۲) `[prompt] How old are you?` `201` `21`

۳) `[prompt] How old are you?` `201` `201`

۴) `[prompt] How old are you?` `NaN` `21`

۹۳۴ سخت

کاربر در پنجره‌ی prompt روی **Cancel** کلیک می‌کند. خروجی کدام است؟ (سطر `[prompt] …` یعنی پنجره‌ی prompt نمایش داده شده است.)

```JavaScript
const userName = prompt("Your name?");
console.log("Hello " + userName);
console.log(Number(userName));
```

۱) `[prompt] Your name?` `Hello` `0`

۲) `[prompt] Your name?` `Hello undefined` `NaN`

۳) `[prompt] Your name?` `Hello null` `0`

۴) `[prompt] Your name?` `Hello null` `NaN`

۹۳۵ سخت

کاربر پنج ثانیه به پنجره‌ی confirm نگاه می‌کند و سپس OK را می‌زند. خروجی کد زیر کدام است؟

```JavaScript
setTimeout(() => console.log("timer"), 0);
const ok = confirm("Continue?");
console.log("answer:", ok);
```

۱) `timer` `[confirm] Continue?` `answer: true`

۲) `[confirm] Continue?` `timer` `answer: true`

۳) `[confirm] Continue?` `answer: OK` `timer`

۴) `[confirm] Continue?` `answer: true` `timer`

۹۳۶ آسان

طبق فعالیت کلاسی اسلاید ۳۷، `window.location` چه چیزی را نشان می‌دهد؟

۱) آدرس (URL) صفحه‌ی فعلی، با بخش‌هایی مثل `hostname` و `pathname`؛ با مقداردهی `location.href` هم مرورگر به صفحه‌ی جدید می‌رود.

۲) موقعیت جغرافیایی (طول و عرض جغرافیایی) کاربر

۳) مختصات نشانگر ماوس روی صفحه

۴) محل قرار گرفتن پنجره‌ی مرورگر روی مانیتور

۹۳۷ متوسط

صفحه‌ای با آدرس `https://shop.example.com:8080/products/list.html?page=2#reviews` باز است. خروجی کد زیر کدام است؟

```JavaScript
console.log(location.host);
console.log(location.pathname);
console.log(location.search + location.hash);
```

۱) `shop.example.com` `/products/list.html` `?page=2#reviews`

۲) `shop.example.com:8080` `/products/list.html` `?page=2#reviews`

۳) `shop.example.com:8080` `/products/list.html?page=2` `#reviews`

۴) `https://shop.example.com:8080` `products/list.html` `page=2reviews`

۹۳۸ آسان

کدام دستور معادل `history.go(-1)` است؟

۱) `history.forward()`

۲) `history.go(1)`

۳) `history.back()`

۴) `location.reload()`

۹۳۹ متوسط

کدام جمله **نادرست** است؟

۱) `navigator.userAgent` رشته‌ای است که مرورگر و سیستم‌عامل کاربر را معرفی می‌کند.

۲) `screen.width` عرض صفحه‌نمایش (مانیتور) کاربر است.

۳) `history.back()` مثل دکمه‌ی Back مرورگر یک صفحه به عقب برمی‌گردد.

۴) `location` موقعیت جغرافیایی کاربر (طول و عرض جغرافیایی) را نگه می‌دارد.

۹۴۰ متوسط

خروجی کد زیر **در مرورگر** کدام است؟

```javascript
console.log("A");
const id = setTimeout(() => console.log("B"), 0);
console.log(typeof id);
console.log("C");
```

۱) A number C B

۲) A B number C

۳) A undefined C B

۴) A object C B

۹۴۱ متوسط

خروجی کد زیر کدام است؟

```javascript
const t1 = setTimeout(() => console.log("first"), 10);
const t2 = setTimeout(() => console.log("second"), 20);
clearTimeout(t1);
console.log("scheduled");
```

۱) scheduled first second

۲) scheduled second

۳) first scheduled

۴) scheduled

۹۴۲ سخت

خروجی کد زیر کدام است؟

```javascript
let count = 0;
const id = setInterval(() => {
  count++;
  console.log("tick", count);
  if (count === 3) clearInterval(id);
}, 10);
console.log("started");
```

۱) tick 1 tick 2 tick 3 started

۲) started tick 1 tick 2

۳) started tick 1 tick 2 tick 3

۴) started tick 1 tick 2 tick 3 tick 4

۹۴۳ سخت

خروجی کد زیر **در مرورگر** کدام است؟

```JavaScript
function hello() {
  console.log("hello");
}
setTimeout(hello(), 50);
console.log("end");
```

۱) `end` `hello`
۲) `end`
۳) `hello` `end` `hello`
۴) `hello` `end`

۹۴۴ متوسط

`window.innerWidth` چه چیزی را نشان می‌دهد؟

۱) عرض viewport، یعنی ناحیه‌ای از پنجره‌ی مرورگر که صفحه در آن دیده می‌شود، بر حسب پیکسل
۲) عرض صفحه‌نمایش (مانیتور) کاربر
۳) عرض کل پنجره‌ی مرورگر همراه با نوار ابزار و حاشیه‌ها
۴) میزان اسکرول افقی صفحه

۹۴۵ آسان

کدام property نشان می‌دهد صفحه چند پیکسل به **پایین** اسکرول شده است؟

۱) `window.scrollX`
۲) `window.scrollY`
۳) `window.innerHeight`
۴) `screen.height`

۹۴۶ سخت

کاربر صفحه را در پنجره‌ای به عرض ۱۲۰۰ پیکسل باز می‌کند، سپس پنجره را تا عرض ۵۰۰ پیکسل کوچک و بعد تا ۸۰۰ پیکسل بزرگ می‌کند. مقدار `document.body.classList.contains("mobile")` در این سه وضعیت به‌ترتیب کدام است؟

```JavaScript
function applyLayout() {
  if (window.innerWidth < 600) {
    document.body.classList.add("mobile");
  } else {
    document.body.classList.remove("mobile");
  }
}
applyLayout();
window.addEventListener("resize", applyLayout);
```

۱) `false` ، `false` ، `false`
۲) `true` ، `false` ، `true`
۳) `false` ، `true` ، `false`
۴) `true` ، `true` ، `false`

۲-۱۳ ذخیره‌سازی در مرورگر: localStorage و sessionStorage

منبع: JS in HTML، اسلایدهای ۳۸ تا ۴۰

درسنامه

وقتی صفحه را reload (بارگذاری دوباره) می‌کنی، همه‌ی متغیرهای JavaScript از نو ساخته می‌شوند و صفحه همه‌چیز را «فراموش» می‌کند. اما خیلی وقت‌ها می‌خواهیم چیزی **بماند**: تم تیره‌ای که کاربر انتخاب کرده، زبان سایت، سبد خرید یا مرحله‌ای از فرم که کاربر در آن بود. مرورگر برای این کار دو شیء آماده دارد: `localStorage` و `sessionStorage` . هر دو property شیء `window` هستند (پس `window.localStorage` همان `localStorage` است) و به آن‌ها Web Storage (حافظه‌ی وب) می‌گویند.

یک تشبیه ساده

`localStorage` مثل کمد شخصی‌ات در یک باشگاه است: وسایلت تا وقتی خودت خالی‌اش نکنی می‌ماند، حتی اگر به خانه بروی و هفته‌ی بعد برگردی؛ و هر باشگاه (هر سایت) کمدهای خودش را دارد. `sessionStorage` مثل سینی روی میز رستوران است: تا پشت همان میز (همان تب) نشسته‌ای در دسترس است، ولی وقتی بلند شوی (تب را ببندی) جمعش می‌کنند؛ و میز کناری (تب دیگر) سینی جداگانه‌ی خودش را دارد.

آنچه اسلایدهای ۳۸ و ۳۹ می‌گویند

ویژگی	`localStorage` (اسلاید ۳۸)	`sessionStorage` (اسلاید ۳۹)
هدف	ذخیره‌ی **ماندگار** (persistent) داده در مرورگر	ذخیره‌ی داده **فقط برای تب فعلی**
شکل داده	جفت‌های کلید-مقدار با `setItem()` و `getItem()`	همان
عمر داده	بعد از بستن تب و حتی بستن مرورگر **می‌ماند**	با بستن تب یا پنجره **پاک می‌شود**
حجم	معمولاً حدود ۵ تا ۱۰ مگابایت برای هر سایت	معمولاً همان حدود ۵ تا ۱۰ مگابایت
امنیت	سیاست same-origin: داده‌ی هر دامنه جداست	همان
محدودیت‌ها	فقط رشته، همگام (synchronous)، **برای اطلاعات حساس امن نیست**	فقط رشته، همگام، برای اطلاعات حساس نیست

متدها (برای هر دو یکسان است)

```JavaScript
localStorage.setItem("theme", "dark");   // save: add a new key or overwrite it
localStorage.getItem("theme");           // "dark"
localStorage.getItem("lang");            // null -> there is no such key
localStorage.removeItem("theme");        // delete one key (no error if it is missing)
localStorage.clear();                    // delete ALL keys of this site (localStorage only)
localStorage.length;                     // how many keys are saved
localStorage.key(0);                     // the NAME of the key at index 0 (or null)
```

عضو	کار	خروجی
`setItem(key, value)`	ذخیره (یا بازنویسی)	`undefined`
`getItem(key)`	خواندن	رشته، یا `null` اگر کلید نباشد
`removeItem(key)`	حذف یک کلید	`undefined`
`clear()`	حذف همه‌ی کلیدهای همان حافظه	`undefined`
`key(i)`	نامِ کلیدِ شماره‌ی `i`	رشته یا `null`
`length`	تعداد کلیدها	عدد

`localStorage` و `sessionStorage` دو انبار **جدا** هستند: `localStorage.clear()` به sessionStorage دست نمی‌زند و `sessionStorage.setItem("step", "2")` در localStorage دیده نمی‌شود. نوشتن به شکل property (`localStorage.theme = "dark"`) هم کار می‌کند، ولی بهتر است همیشه از `setItem` و `getItem` استفاده کنی تا با نام متدها (مثل `length` و `key`) قاطی نشود.

فقط رشته!

هر مقداری که به `setItem` بدهی **قبل از ذخیره به رشته تبدیل می‌شود** و `getItem` همیشه رشته (یا `null`) برمی‌گرداند:

```JavaScript
localStorage.setItem("count", 5);
console.log(localStorage.getItem("count") + 1);    // "51" -> string concatenation

localStorage.setItem("darkMode", false);
if (localStorage.getItem("darkMode")) {
  console.log("runs!");   // "false" is a non-empty string, so it is truthy
}

localStorage.setItem("user", { name: "Ali" });
console.log(localStorage.getItem("user"));         // "[object Object]" -> the data is lost

localStorage.setItem("nums", [1, 2, 3]);
console.log(localStorage.getItem("nums"));         // "1,2,3"
```

چیزی که ذخیره می‌کنی	چیزی که برمی‌گردد
`5`	`"5"`
`false`	`"false"` (یک رشته‌ی غیرخالی، پس truthy؛ یعنی در شرط «درست» حساب می‌شود)
`null`	`"null"`
`{ name: "Ali" }`	`"[object Object]"`
`[1, 2, 3]`	`"1,2,3"`

الگوی درست: JSON

برای شیء و آرایه، همان‌طور که در درس JSON فصل اول دیدیم، قبل از ذخیره `JSON.stringify` و بعد از خواندن `JSON.parse` می‌کنیم:

```JavaScript
const user = { name: "Ali", age: 20 };
localStorage.setItem("user", JSON.stringify(user));      // '{"name":"Ali","age":20}'
const saved = JSON.parse(localStorage.getItem("user"));  // a real object again
console.log(saved.age + 1);                              // 21 -> age is a number again
```

وقتی معلوم نیست کلید وجود دارد یا نه:

```JavaScript
const cart = JSON.parse(localStorage.getItem("cart")) || [];  // JSON.parse(null) is null -> []
```

برای عدد از `Number()` و برای بولی از مقایسه با رشته استفاده کن:

```JavaScript
const visits = Number(localStorage.getItem("visits")) + 1;   // Number(null) is 0
localStorage.setItem("visits", visits);
console.log("You have visited this page " + visits + " times");

const isDark = localStorage.getItem("darkMode") === "true";  // compare with the string
```

همگام (synchronous)

`setItem` و `getItem` همگام‌اند: کارشان قبل از رفتن به خط بعد تمام می‌شود و برای خواندن مقدار، نه `await` لازم است نه callback. اما چون روی رشته‌ی اصلی (main thread) اجرا می‌شوند، ذخیره‌ی داده‌ی خیلی بزرگ می‌تواند صفحه را لحظه‌ای کُند کند؛ داده را کوچک نگه دار.

سیاست same-origin

origin (مبدأ) یعنی ترکیب **پروتکل + میزبان (دامنه) + پورت**. هر origin انبار localStorage و sessionStorage **جداگانه**‌ی خودش را دارد و هیچ سایتی نمی‌تواند داده‌ی سایت دیگر را بخواند. مسیر (path) صفحه مهم نیست:

دو آدرس	origin یکسان؟
`https://site.com/a.html` و `https://site.com/blog/b.html`	**بله**؛ فقط مسیر فرق دارد
`http://site.com` و `https://site.com`	خیر؛ پروتکل فرق دارد
`https://site.com` و `https://shop.site.com`	خیر؛ میزبان فرق دارد
`https://site.com` و `https://site.com:8080`	خیر؛ پورت فرق دارد

عمر داده و تب‌ها

موقعیت	`localStorage`	`sessionStorage`
reload در همان تب	می‌ماند	**می‌ماند**
بستن تب یا پنجره	می‌ماند	پاک می‌شود
بستن کل مرورگر و باز کردن فردا	می‌ماند	پاک شده است
باز کردن همان سایت در تب جدید (با تایپ آدرس)	**مشترک** است؛ همان داده دیده می‌شود	تب جدید sessionStorage **خالیِ خودش** را دارد
تا کی؟	تا وقتی کد یا خودِ کاربر (پاک کردن داده‌های مرورگر) حذفش کند؛ تاریخ انقضا ندارد	تا وقتی آن تب باز است

امنیت: جای اطلاعات حساس نیست

- داده‌ها **رمزنگاری نمی‌شوند** و هر کسی که به مرورگر دسترسی دارد می‌تواند آن‌ها را در DevTools (ابزار توسعه‌دهنده‌ی مرورگر، بخش Application یا Storage) ببیند یا تغییر دهد.
- **هر** کد JavaScriptی که در صفحه اجرا شود می‌تواند همه‌ی داده‌ها را بخواند؛ از جمله کدی که مهاجم با حمله‌ی XSS (Cross-Site Scripting؛ تزریق اسکریپت مخرب به صفحه‌ی یک سایت) وارد صفحه کرده است.
- پس رمز عبور، شماره‌ی کارت بانکی و اطلاعات محرمانه را آن‌جا نگذار و به مقدارهایش (مثلاً `isAdmin: true`) برای تصمیم‌های امنیتی اعتماد نکن.

نمونه‌ی کاربردی ۱: به خاطر سپردن تم تیره (localStorage)

JavaScript

```
const btn = document.getElementById("toggle");
if (localStorage.getItem("theme") === "dark") {
  document.body.classList.add("dark");   // restore the saved choice when the page loads
}
btn.addEventListener("click", () => {
  const isDark = document.body.classList.toggle("dark");
  localStorage.setItem("theme", isDark ? "dark" : "light");
});
```

نمونه‌ی کاربردی ۲: فرم چندمرحله‌ای (sessionStorage)

JavaScript

```
// remember the progress of this tab only
sessionStorage.setItem("signup-step", "2");
sessionStorage.setItem("signup-name", "Sara");

// after a reload in the same tab, continue where the user left off
const step = sessionStorage.getItem("signup-step") || "1";
console.log("Show step " + step);   // "Show step 2"
```

فعالیت کلاسی اسلاید ۴۰

۱. تفاوت اصلی localStorage و sessionStorage چیست؟ عمر و دامنه‌ی داده. داده‌ی localStorage تاریخ انقضا ندارد، بعد از بستن تب و مرورگر هم می‌ماند و بین همه‌ی تب‌های همان origin مشترک است. داده‌ی sessionStorage فقط مال همان یک تب است و با بستن تب یا پنجره پاک می‌شود. بقیه‌ی ویژگی‌ها مشترک‌اند: API یکسان، سیاست same-origin، فقط رشته، همگام، حدود ۵ تا ۱۰ مگابایت و نامناسب برای داده‌ی حساس.

۲. یک مثال برای localStorage: به خاطر سپردن تم تیره یا روشن، زبان انتخابی کاربر، سبد خرید یا لیست کارهایی که باید تا بازدید بعدی بماند، یا گزینه‌ی «این راهنما را دوباره نشان نده».

۳. یک مثال برای sessionStorage: پیشرفت یک فرم چندمرحله‌ای در همان تب، فیلترها و جست‌وجوی موقتی که با بستن تب باید پاک شود، پیام خوش‌آمدی که در هر بازدید فقط یک بار نمایش داده شود، یا وضعیتی که هر تب باید جدا داشته باشد.

نکته

- `setItem` و `getItem`، و برای حذف `removeItem` و `clear`؛ کلید ناموجود ← `null`.
- شیء و آرایه را با `JSON.stringify` ذخیره و با `JSON.parse` بخوان؛ برای مقدار شاید ناموجود: `JSON.parse(localStorage.getItem("key")) || []`.
- ماندگار و مشترک بین تب‌ها ← `localStorage`؛ موقت و مخصوص یک تب ← `sessionStorage`.

دام تستی

- `setItem("count", 5)` ← `getItem("count") + 1` می‌شود `"51"`.
- `setItem("flag", false)` ← `"false"` که در `if` **درست** (truthy) حساب می‌شود.
- ذخیره‌ی مستقیم شیء ← `"[object Object]"` و داده از دست می‌رود.
- `getItem` برای کلید ناموجود `null` برمی‌گرداند، نه `undefined`.
- sessionStorage با reload پاک **نمی‌شود**؛ با بستن تب پاک می‌شود.
- localStorage تاریخ انقضا ندارد؛ خودبه‌خود (مثلاً بعد از ۲۴ ساعت) پاک نمی‌شود.
- `http://` و `https://` یک سایت، یا دو پورت متفاوت، origin جدا و انبار جدا دارند.
- رمز عبور و اطلاعات حساس را هرگز در Web Storage نگذار (XSS می‌تواند همه را بخواند).

جمع‌بندی

دو انبار کلید-مقدار در `window`، با API یکسان (`setItem`، `getItem`، `removeItem`، `clear`، `key`، `length`)، فقط رشته، همگام، حدود ۵ تا ۱۰ مگابایت، جدا برای هر origin و ناامن برای داده‌ی حساس. فرق اصلی: `localStorage` ماندگار و مشترک بین تب‌ها؛ `sessionStorage` فقط برای همان تب و تا بسته شدن آن.

تست‌های تمرینی

۲۳ تست · شماره‌ی ۹۴۷ تا ۹۶۹

۹۴۷ آسان

طبق فعالیت کلاسی اسلاید ۴۰، تفاوت **اصلی** `localStorage` و `sessionStorage` چیست؟

۱) `localStorage` فقط عدد ذخیره می‌کند ولی `sessionStorage` فقط رشته.

۲) داده‌ی `localStorage` بعد از بستن تب و حتی مرورگر می‌ماند و بین تب‌های همان origin مشترک است، ولی داده‌ی `sessionStorage` فقط مال همان تب است و با بستن تب پاک می‌شود.

۳) `localStorage` ناهمگام (asynchronous) است ولی `sessionStorage` همگام.

۴) `sessionStorage` از سیاست same-origin پیروی نمی‌کند ولی `localStorage` می‌کند.

۹۴۸ آسان

کاربری در یک سایت مقدار `theme` را هم در `localStorage` و هم در `sessionStorage` ذخیره می‌کند. سپس **کل مرورگر را می‌بندد** و فردا دوباره همان سایت را باز می‌کند. کدام مقدار هنوز وجود دارد؟

۱) فقط مقدار `localStorage`
۲) فقط مقدار `sessionStorage`
۳) هر دو
۴) هیچ‌کدام

۹۴۹ متوسط

کاربر در یک تب مقداری را در `sessionStorage` ذخیره می‌کند و سپس صفحه را در **همان تب** reload (بارگذاری دوباره) می‌کند. آن مقدار چه می‌شود؟

۱) پاک می‌شود، چون `sessionStorage` فقط تا reload بعدی دوام دارد.

۲) به `localStorage` منتقل می‌شود.

۳) باقی می‌ماند، چون تب هنوز بسته نشده است.

۴) فقط در صورتی می‌ماند که با `JSON.stringify` ذخیره شده باشد.

۹۵۰ سخت

کاربر سایت `https://shop.example.com` را در تب A باز کرده و این دو دستور را اجرا می‌کند: `localStorage.setItem("y", "1")` و `sessionStorage.setItem("x", "1")`. سپس یک تب **جدید** (B) باز می‌کند و همان آدرس را **تایپ** می‌کند. در تب B کدام مقدار قابل خواندن است؟

۱) هم `x` و هم `y`
۲) هیچ‌کدام
۳) فقط `x`
۴) فقط `y`

۹۵۱ آسان

خروجی کد زیر کدام است؟ (فرض کن localStorage این سایت در ابتدا خالی است.)

```javascript
localStorage.setItem("theme", "dark");
console.log(localStorage.getItem("theme"));
console.log(localStorage.getItem("lang"));
```

۱) `dark` `undefined`
۲) `dark` `null`
۳) `theme` `null`
۴) `dark` `false`

۹۵۲ آسان

خروجی کد زیر کدام است؟ (هر دو حافظه در ابتدا خالی‌اند.)

```javascript
sessionStorage.setItem("step", "2");
console.log(sessionStorage.getItem("step"), localStorage.getItem("step"));
```

۱) `2 2`
۲) `null 2`
۳) `2 null`
۴) `2 undefined`

۹۵۳ متوسط

خروجی کد زیر کدام است؟

```javascript
localStorage.setItem("x", "1");
console.log(localStorage.getItem("x"));
console.log(localStorage.setItem("y", "2"));
```

۱) `1` `undefined`
۲) `null` `undefined`
۳) `1` `true`
۴) `1` `y`

۹۵۴ متوسط

خروجی کد زیر کدام است؟ (هر دو حافظه در ابتدا خالی‌اند.)

```javascript
localStorage.setItem("a", "1");
localStorage.setItem("b", "2");
sessionStorage.setItem("c", "3");
localStorage.removeItem("a");
console.log(localStorage.length, localStorage.key(0));
localStorage.clear();
console.log(localStorage.length, sessionStorage.length);
```

۱) `1 b` `0 0`
۲) `2 a` `0 1`
۳) `1 a` `0 1`
۴) `1 b` `0 1`

۹۵۵ متوسط

فقط کلید `token` باید از localStorage حذف شود و بقیه‌ی کلیدها بمانند. در جای خالی `localStorage.______("token");` کدام گزینه قرار می‌گیرد؟

۱) `clear`
۲) `removeItem`
۳) `deleteItem`
۴) `setItem`

۹۵۶ آسان

طبق اسلاید ۳۸، در `localStorage` چه نوع مقدارهایی ذخیره می‌شود؟

۱) هر نوع داده‌ی JavaScript، از جمله شیء و آرایه، دقیقاً همان‌طور که هست
۲) فقط عدد و رشته
۳) فقط شیءهای JSON
۴) فقط رشته؛ هر مقدار دیگری قبل از ذخیره به رشته تبدیل می‌شود.

۹۵۷ متوسط

خروجی کد زیر کدام است؟

```javascript
localStorage.setItem("count", 5);
const count = localStorage.getItem("count");
console.log(count + 1, typeof count);
```

۱) `6 number`
۲) `6 string`
۳) `51 string`
۴) `51 number`

۹۵۸ سخت

خروجی کد زیر کدام است؟

```javascript
localStorage.setItem("darkMode", false);
if (localStorage.getItem("darkMode")) {
  console.log("dark theme");
} else {
  console.log("light theme");
}
```

۱) `dark theme`
۲) `light theme`
۳) (بدون خروجی)
۴) `Uncaught TypeError`

۹۵۹ متوسط

خروجی کد زیر کدام است؟

```javascript
const user = { name: "Ali", age: 20 };
localStorage.setItem("user", user);
const saved = localStorage.getItem("user");
console.log(saved);
console.log(saved.name);
```

۱) `{ name: 'Ali', age: 20 }` `Ali`

۲) `[object Object]` `undefined`

۳) `[object Object]` `Uncaught TypeError`

۴) `{"name":"Ali","age":20}` `Ali`

۹۶۰ سخت

خروجی کد زیر کدام است؟

```javascript
localStorage.setItem("nums", [1, 2, 3]);
const nums = localStorage.getItem("nums");
console.log(nums.length, nums.split(",").map(Number)[2] + 1);
```

۱) `3 4`

۲) `5 31`

۳) `5 4`

۴) `3 31`

۹۶۱ متوسط

خروجی کد زیر کدام است؟

```javascript
const cart = [{ id: 1, qty: 2 }];
localStorage.setItem("cart", JSON.stringify(cart));
console.log(localStorage.getItem("cart"));
const loaded = JSON.parse(localStorage.getItem("cart"));
console.log(loaded[0].qty + 1);
```

۱) `[object Object]` `3`

۲) `[{"id":1,"qty":2}]` `21`

۳) `[{ id: 1, qty: 2 }]` `3`

۴) `[{"id":1,"qty":2}]` `3`

۹۶۲ سخت

خروجی کد زیر کدام است؟ (کلید `todos` هنوز در localStorage وجود ندارد.)

```javascript
const todos = JSON.parse(localStorage.getItem("todos")) || [];
todos.push("learn DOM");
localStorage.setItem("todos", JSON.stringify(todos));
console.log(todos.length, localStorage.getItem("todos"));
```

۱) `1 ["learn DOM"]`
۲) `Uncaught SyntaxError`
۳) `1 learn DOM`
۴) `Uncaught TypeError`

۹۶۳ سخت

خروجی کد زیر کدام است؟ (کلید `visits` هنوز در localStorage وجود ندارد.)

```javascript
let visits = Number(localStorage.getItem("visits")) + 1;
localStorage.setItem("visits", visits);
console.log(visits, typeof localStorage.getItem("visits"));
visits = localStorage.getItem("visits") + 1;
console.log(visits);
```

۱) `NaN string` `NaN1`
۲) `1 string` `11`
۳) `1 number` `2`
۴) `1 string` `2`

۹۶۴ سخت

خروجی کد زیر کدام است؟ (localStorage در ابتدا خالی است.)

```javascript
localStorage.setItem("score", 10);
localStorage.score = 20;
console.log(localStorage.getItem("score") * 2, localStorage.length);
```

۱) `20 1`
۲) `40 2`
۳) `40 1`
۴) `2020 1`

۹۶۵ متوسط

کدام دو صفحه localStorage **مشترک** دارند (یعنی origin یکسانی دارند)؟

۱) `http://site.com/index.html` و `https://site.com/index.html`
۲) `https://site.com/` و `https://shop.site.com/`
۳) `https://site.com/` و `https://site.com:8080/`
۴) `https://site.com/a.html` و `https://site.com/blog/b.html`

۹۶۶ آسان

چرا نباید رمز عبور یا اطلاعات حساس را در `localStorage` ذخیره کرد؟

۱) داده‌ها رمزنگاری نمی‌شوند و هر کد JavaScriptی که در صفحه اجرا شود (مثلاً کدی که مهاجم با حمله‌ی XSS تزریق کرده) می‌تواند آن‌ها را بخواند.

۲) چون `localStorage` فقط عدد ذخیره می‌کند.

۳) چون داده‌های `localStorage` بعد از ۲۴ ساعت خودبه‌خود پاک می‌شوند.

۴) چون همه‌ی سایت‌های دیگر هم می‌توانند `localStorage` این سایت را بخوانند.

۹۶۷ سخت

کدام جمله درباره‌ی `localStorage` و `sessionStorage` **نادرست** است؟

۱) هر دو معمولاً حدود ۵ تا ۱۰ مگابایت فضا برای هر سایت دارند.

۲) داده‌های `localStorage` بعد از ۲۴ ساعت به‌طور خودکار منقضی و پاک می‌شوند.

۳) هر دو همگام (synchronous) هستند و فقط رشته ذخیره می‌کنند.

۴) `localStorage.clear()` روی داده‌های `sessionStorage` اثری ندارد.

۹۶۸ متوسط

کدام گزینه برای هر دو حافظه کاربرد **مناسبی** ذکر کرده است؟

۱) `localStorage` ← ذخیره‌ی رمز عبور کاربر برای ورود خودکار؛ `sessionStorage` ← تم تیره برای همه‌ی بازدیدهای آینده

۲) `localStorage` ← مرحله‌ی فعلی فرم ثبت‌نام در همین تب؛ `sessionStorage` ← سبد خریدی که باید تا هفته‌ی بعد بماند

۳) `localStorage` ← زبان انتخابی کاربر؛ `sessionStorage` ← مرحله‌ی فعلی یک فرم چندمرحله‌ای در همین تب

۴) `localStorage` ← شماره‌ی کارت بانکی؛ `sessionStorage` ← رمز عبور

۹۶۹ متوسط

صفحه برای اولین بار باز شده (localStorage خالی است) و کاربر **سه بار** روی دکمه کلیک می‌کند. خروجی کدام است؟

HTML

```
<button id="toggle">Theme</button>
```

JavaScript

```
const saved = localStorage.getItem("theme");
if (saved === "dark") document.body.classList.add("dark");

document.getElementById("toggle").addEventListener("click", () => {
  const isDark = document.body.classList.toggle("dark");
  localStorage.setItem("theme", isDark ? "dark" : "light");
  console.log(localStorage.getItem("theme"));
});
```

۱) `dark` `dark` `dark`

۲) `true` `false` `true`

۳) `light` `dark` `light`

۴) `dark` `light` `dark`

آزمون جامع فصل دوم

۲۴ تست ترکیبی · شماره‌ی ۹۷۰ تا ۹۹۳

این آزمون ۲۴ تست **ترکیبی** دارد؛ هر تست دو یا چند درس از این فصل را با هم درگیر می‌کند (مثلاً event delegation همراه با ساختن عنصر، یا ارسال فرم همراه با localStorage و JSON). پیشنهاد می‌کنیم مثل جلسه‌ی آزمون واقعی و بدون نگاه کردن به درسنامه‌ها، هر ۲۴ تست را در حدود **۴۰ دقیقه** حل کنی و بعد پاسخ‌هایت را با کلید و پاسخ تشریحی مقایسه کنی. برای هر تستی که اشتباه زدی، درس‌هایی را که در آن تست ترکیب شده‌اند دوباره مرور کن.

یادآوری: در گزینه‌ها سطر «Uncaught …» یعنی یک خطای مهارنشده در Console چاپ شده است.

۹۷۰ سخت

صفحه‌ی زیر در مرورگر باز می‌شود و بعد از کامل شدن بارگذاری، کاربر یک بار روی دکمه‌ی Save کلیک می‌کند. خروجی کامل Console کدام است؟

HTML

```
<!DOCTYPE html>
<html>
<head>
  <script>
    console.log("head:", document.getElementById("save"));
    document.addEventListener("DOMContentLoaded", () => {
      console.log("ready:", document.getElementById("save").textContent);
    });
    document.getElementById("save").addEventListener("click", () => {
      console.log("saved");
    });
    console.log("head end");
  </script>
</head>
<body>
  <button id="save">Save</button>
  <script>
    console.log("body script");
  </script>
</body>
</html>
```

۱) `head: null` `Uncaught TypeError`

۲) `head: null` `Uncaught TypeError` `ready: Save` `body script`

۳) `head: null` `Uncaught TypeError` `body script` `ready: Save` `saved`

۴) `head: null` `Uncaught TypeError` `body script` `ready: Save`

۹۷۱ سخت

خروجی کد زیر کدام است؟ (به فاصله‌ها و خط‌های جدیدِ داخل HTML دقت کن.)

HTML

```
<ul id="list">
  <li>A</li>
</ul>
```

JavaScript

```
const list = document.getElementById("list");
const li = document.createElement("li");
li.textContent = "B";
list.insertBefore(li, list.lastChild);
console.log(list.lastChild.nodeName, list.lastElementChild.textContent);
console.log(list.children.length, list.childNodes.length);
```

۱) `#text B` `2 4`

۲) `LI A` `2 2`

۳) `LI B` `2 4`

۴) `#text A` `2 4`

۹۷۲ سخت

خروجی کد زیر کدام است؟

HTML

```
<p class="x">1</p>
<p class="x">2</p>
<p class="x">3</p>
<p class="x">4</p>
```

JavaScript

```
const live = document.getElementsByClassName("x");
const fixed = document.querySelectorAll(".x");
for (let i = 0; i < live.length; i++) {
  live[i].classList.remove("x");
}
console.log(live.length, fixed.length, live[0].textContent);
```

۱) `4 4 1`

۲) `2 2 2`

۳) `2 4 2`

۴) `Uncaught TypeError`

۹۷۳ سخت

کد زیر اجرا می‌شود و سپس کاربر روی آیتم `A` در صفحه کلیک می‌کند. خروجی کامل Console کدام است؟

HTML

```
<ul id="list"><li>A</li><li>B</li></ul>
```

JavaScript

```
const list = document.getElementById("list");
const items = list.getElementsByTagName("li");
const first = items[0];
first.addEventListener("click", () => console.log("A clicked"));
list.innerHTML += "<li>C</li>";
console.log(items.length, items[0] === first, first.isConnected);
```

۱) `3 true true` `A clicked`

۲) `3 false false`

۳) `2 false false`

۴) `3 false false` `A clicked`

۹۷۴ سخت

HTML صفحه این است:

HTML

```
<ul id="menu"><li class="item">Home</li><li class="item">Blog</li></ul>
```

کد زیر در انتهای صفحه اجرا می‌شود. **اولین** خطا در کدام خط رخ می‌دهد؟

JavaScript

```
1 const items = document.querySelectorAll(".item");
2 items.forEach((li) => li.classList.add("ready"));
3 const ready = document.getElementsByClassName("ready");
4 console.log(ready.length);
5 ready.forEach((li) => li.remove());
6 items.classList.remove("item");
```

۱) خط ۲

۲) خط ۵

۳) خط ۶

۴) هیچ خطایی رخ نمی‌دهد.

۹۷۵ سخت

کد زیر **در مرورگر** (اسکریپت معمولی، بدون `"use strict"`) اجرا می‌شود و کاربر یک بار روی دکمه کلیک می‌کند. خروجی Console کدام است؟

HTML

```html
<button id="btn">Go</button>
```

JavaScript

```javascript
var label = "global";
const btn = document.getElementById("btn");
btn.label = "button";
btn.addEventListener("click", function () {
  console.log(this.label);
});
btn.addEventListener("click", () => {
  console.log(this.label);
});
btn.onclick = function () {
  console.log(this === btn);
};
```

۱) `button` `button` `true`

۲) `button` `undefined` `true`

۳) `true` `button` `global`

۴) `button` `global` `true`

۹۷۶ سخت

کاربر روی دکمه‌ی Go کلیک می‌کند. خروجی Console کدام است؟

HTML

```
<div id="outer">
  <div id="inner">
    <button id="btn">Go</button>
  </div>
</div>
```

JavaScript

```
const outer = document.getElementById("outer");
const inner = document.getElementById("inner");
const btn = document.getElementById("btn");
document.addEventListener("click", () => console.log("document"));
outer.addEventListener("click", () => console.log("outer capture"), true);
outer.addEventListener("click", () => console.log("outer bubble"));
inner.addEventListener("click", (e) => {
  console.log("inner");
  e.stopPropagation();
});
btn.addEventListener("click", () => console.log("btn"));
```

۱) `outer capture` `btn` `inner`

۲) `btn` `inner`

۳) `document` `outer capture` `btn` `inner`

۴) `outer capture` `btn` `inner` `document`

۹۷۷ سخت

کاربر ابتدا روی دکمه‌ی Add و سپس روی آیتمی که تازه به لیست اضافه شده کلیک می‌کند. خروجی کامل Console کدام است؟

HTML

```
<ul id="list"><li>A</li></ul>
<button id="add">Add</button>
```

JavaScript

```
const list = document.getElementById("list");
list.addEventListener("click", function (e) {
  console.log(e.target.textContent, this.tagName);
});
document.getElementById("add").addEventListener("click", () => {
  const li = document.createElement("li");
  li.textContent = "B" + list.children.length;
  list.appendChild(li);
});
```

۱) `B2 UL`

۲) `B1 LI`

۳) `B1 UL`

۴) (بدون خروجی)

۹۷۸ متوسط

HTML صفحه این است:

```HTML
<ul id="list">
  <li>Milk <button class="del"><i>x</i></button></li>
  <li>Bread <button class="del"><i>x</i></button></li>
</ul>
```

می‌خواهیم با کلیک روی دکمه‌ی حذف هر آیتم (چه روی خودِ دکمه و چه روی `<i>` داخل آن) همان آیتم حذف شود و کلیک روی متن آیتم (مثلاً روی کلمه‌ی Milk) هیچ کاری نکند. در جای خالی چه باید نوشت؟

```JavaScript
const list = document.getElementById("list");
list.addEventListener("click", (e) => {
  const btn = ________;
  if (!btn) return;
  btn.closest("li").remove();
});
```

۱) `this.querySelector(".del")`
۲) `e.target.matches(".del")`
۳) `e.currentTarget.closest(".del")`
۴) `e.target.closest(".del")`

۹۷۹ سخت

کاربر روی دکمه‌ی `x` کنار Bread کلیک می‌کند و در پنجره‌ی confirm دکمه‌ی OK را می‌زند؛ سپس روی کلمه‌ی Milk کلیک می‌کند. خروجی کامل Console کدام است؟ (سطر `[confirm] …` یعنی پنجره‌ی confirm با آن پیام نمایش داده شده است.)

```HTML
<ul id="cart">
  <li><span>Milk</span> <button class="del">x</button></li>
  <li><span>Bread</span> <button class="del">x</button></li>
</ul>
```

```JavaScript
const cart = document.getElementById("cart");
cart.addEventListener("click", (e) => {
  const btn = e.target.closest(".del");
  if (!btn) return;
  const li = btn.closest("li");
  if (confirm("Delete " + li.querySelector("span").textContent + "?")) {
    li.remove();
  }
  console.log(cart.children.length, li.textContent);
});
```

۱) `[confirm] Delete Bread?` `1 Bread x`
۲) `[confirm] Delete Bread?` `1 Bread`
۳) `[confirm] Delete Bread?` `Uncaught TypeError`
۴) `[confirm] Delete Bread?` `2 Bread x`

۹۸۰ سخت

کد زیر اجرا می‌شود و سپس کاربر روی دکمه‌ی B کلیک می‌کند. خروجی Console کدام است؟

HTML

```
<button id="a" onclick="console.log('inline', this.id)">A</button>
```

JavaScript

```
const a = document.getElementById("a");
a.addEventListener("click", () => console.log("listener"));
const b = a.cloneNode(true);
b.id = "b";
b.textContent = "B";
document.body.append(b);
```

۱) `inline b` `listener`

۲) `inline b`

۳) `listener`

۴) (بدون خروجی)

۹۸۱ سخت

کاربر در پنجره‌ی prompt عبارت `<b>Sara</b>` را تایپ می‌کند و OK را می‌زند. خروجی کد زیر کدام است؟ (سطر `[prompt] …` یعنی پنجره‌ی prompt با آن پیام نمایش داده شده است.)

HTML

```
<p id="a"></p>
<p id="b"></p>
```

JavaScript

```
const userName = prompt("Your name?");
const a = document.getElementById("a");
const b = document.getElementById("b");
a.textContent = "Hi " + userName;
b.innerHTML = "Hi " + userName;
console.log(a.children.length, b.children.length);
console.log(a.textContent.length, b.textContent.length);
```

۱) `[prompt] Your name?` `1 1` `7 7`

۲) `[prompt] Your name?` `0 0` `14 14`

۳) `[prompt] Your name?` `0 1` `14 7`

۴) `[prompt] Your name?` `0 1` `14 14`

۹۸۲ سخت

HTML صفحه این است:

```HTML
<ul id="list">
  <li id="a">A</li>
  <li id="b">B</li>
</ul>
```

کد زیر در انتهای صفحه اجرا می‌شود. **اولین** خطا در کدام خط رخ می‌دهد؟

```JavaScript
1 const list = document.getElementById("list");
2 const b = document.getElementById("b");
3 b.remove();
4 b.remove();
5 console.log(document.getElementById("b"), b.textContent);
6 list.appendChild(b);
7 document.body.removeChild(b);
```

۱) خط ۴
۲) خط ۵
۳) خط ۶
۴) خط ۷

۹۸۳ سخت

کاربر روی checkbox کلیک می‌کند و آن را تیک می‌زند. خروجی Console کدام است؟

```HTML
<input type="checkbox" id="agree">
<button id="send" class="btn" disabled>Send</button>
```

```JavaScript
const agree = document.getElementById("agree");
const send = document.getElementById("send");
agree.addEventListener("change", () => {
  send.disabled = !agree.checked;
  send.classList.toggle("ready");
  console.log(
    send.disabled,
    send.hasAttribute("disabled"),
    agree.hasAttribute("checked"),
    send.className
  );
});
```

۱) `false false false btn ready`
۲) `false true false btn ready`
۳) `false false true btn ready`
۴) `false false false ready`

۹۸۴ متوسط

خروجی کد زیر در مرورگر کدام است؟

HTML

```
<div id="box" class="card"></div>
```

JavaScript

```
const box = document.getElementById("box");
let n = 0;
const id = setInterval(() => {
  n++;
  box.classList.toggle("on");
  if (n === 3) {
    clearInterval(id);
    console.log(n, box.className);
  }
}, 10);
console.log("start", box.className);
```

۱) `start card` `3 card on`

۲) `3 card on` `start card`

۳) `start card` `3 on`

۴) `start card on` `3 card on`

۹۸۵ سخت

کاربر سه حرف `d`، `o` و `m` را پشت سر هم در کادر تایپ می‌کند (فاصله‌ی زمانی بین حرف‌ها کمتر از ۵۰۰ میلی‌ثانیه است) و بعد یک ثانیه صبر می‌کند. خروجی Console کدام است؟

HTML

```
<input id="q">
```

JavaScript

```
const q = document.getElementById("q");
let timer;
q.addEventListener("input", () => {
  clearTimeout(timer);
  timer = setTimeout(() => {
    console.log("search:", q.value);
  }, 500);
});
```

۱) `search: d` `search: do` `search: dom`

۲) `search: dom` `search: dom` `search: dom`

۳) `search: dom`

۴) (بدون خروجی)

۹۸۶ سخت

localStorage این سایت در ابتدا خالی است. کاربر `pen` را در کادر تایپ می‌کند و فرم را ارسال می‌کند؛ سپس `book` را تایپ می‌کند و دوباره فرم را ارسال می‌کند. خروجی کامل Console کدام است؟

HTML

```html
<form id="f">
  <input id="item">
  <button>Add</button>
</form>
```

JavaScript

```javascript
const form = document.getElementById("f");
const input = document.getElementById("item");
form.addEventListener("submit", (e) => {
  e.preventDefault();
  const items = JSON.parse(localStorage.getItem("items")) || [];
  items.push(input.value);
  localStorage.setItem("items", items);
  console.log(localStorage.getItem("items"));
});
```

۱) `["pen"]` `["pen","book"]`

۲) `pen` `pen,book`

۳) `pen` `book`

۴) `pen` `Uncaught SyntaxError`

۹۸۷ متوسط

HTML صفحه این است:

```HTML
<form id="todo-form">
  <input id="todo">
  <button>Add</button>
</form>
```

می‌خواهیم با ارسال فرم صفحه دوباره بارگذاری نشود و آرایه‌ی `todos` طوری ذخیره شود که بعداً `JSON.parse(localStorage.getItem("todos"))` دقیقاً همان آرایه را برگرداند. در جاهای خالی (۱) و (۲) چه باید نوشت؟

```JavaScript
const form = document.getElementById("todo-form");
const input = document.getElementById("todo");
const todos = [];
form.addEventListener("submit", (e) => {
  e.___(1)___();
  todos.push(input.value);
  localStorage.setItem("todos", ___(2)___);
});
```

۱) (۱) `preventDefault` و (۲) `JSON.stringify(todos)`

۲) (۱) `stopPropagation` و (۲) `JSON.stringify(todos)`

۳) (۱) `preventDefault` و (۲) `todos.toString()`

۴) (۱) `preventDefault` و (۲) `JSON.parse(todos)`

۹۸۸ متوسط

خروجی کد زیر کدام است؟ (هر دو حافظه در ابتدا خالی‌اند.)

```HTML
<span id="v"></span>
```

```JavaScript
localStorage.setItem("visits", 1);
sessionStorage.visits = Number(localStorage.getItem("visits")) + 1;
const v = document.getElementById("v");
v.textContent = sessionStorage.getItem("visits") + 1;
console.log(v.textContent, typeof sessionStorage.visits, localStorage.getItem("visits"));
```

۱) `3 number 1`　۲) `21 string 1`　۳) `21 string 2`　۴) `3 string 1`

۹۸۹ سخت

localStorage این سایت در ابتدا خالی است. کد زیر در مرورگر اجرا می‌شود. **اولین** خطا در کدام خط رخ می‌دهد؟

```JavaScript
localStorage.setItem("user", { name: "Ali" });
const raw = localStorage.getItem("user");
console.log(raw.name);
const cart = JSON.parse(localStorage.getItem("cart"));
console.log(cart);
const user = JSON.parse(raw);
console.log(user.name);
```

۱) خط ۱
۲) خط ۳
۳) خط ۴
۴) خط ۶

۹۹۰ سخت

HTML صفحه این است:

```HTML
<ul id="tasks">
  <li class="done">A</li>
  <li class="done">B</li>
  <li>C</li>
  <li class="done">D</li>
</ul>
```

می‌خواهیم همه‌ی `<li>` هایی که کلاس `done` دارند حذف شوند و فقط `C` در لیست بماند. با کدام گزینه در جای خالی، کد زیر درست کار می‌کند؟

```JavaScript
const done = ________;
for (let i = 0; i < done.length; i++) {
  done[i].remove();
}
```

۱) `document.querySelectorAll(".done")`
۲) `document.getElementsByClassName("done")`
۳) `document.querySelector(".done")`
۴) `document.getElementsByTagName("li")`

۹۹۱ متوسط

کدام عبارت **درست** است؟

۱) NodeListی که `document.querySelectorAll` برمی‌گرداند **زنده** است؛ یعنی اگر بعداً عنصر جدیدی با همان selector به صفحه اضافه شود، `length` آن خودبه‌خود بیشتر می‌شود.

۲) `localStorage.getItem` برای کلیدی که وجود ندارد، رشته‌ی خالی `""` برمی‌گرداند.

۳) بعد از `el.remove()`، اگر متغیری هنوز به `el` اشاره کند، می‌توان همان عنصر را با `appendChild` دوباره به صفحه برگرداند.

۴) صدا زدن `e.stopPropagation()` در listener رویداد `submit`، ارسال فرم و بارگذاری دوباره‌ی صفحه را هم متوقف می‌کند.

۹۹۲ سخت

کدام عبارت **نادرست** است؟

۱) یک اسکریپت معمولی (بدون `defer`) داخل `<head>` که `document.getElementById("btn")` را صدا می‌زند، برای دکمه‌ای که در `<body>` تعریف شده `null` می‌گیرد.

۲) در یک اسکریپت معمولی مرورگر، `var x = 1` در سطح بالا باعث می‌شود `window.x` برابر `1` شود؛ ولی `let` و `const` چنین propertyای روی `window` نمی‌سازند.

۳) `setTimeout(fn, 0)` تابع `fn` را بلافاصله، قبل از اجرای خط بعدیِ همان اسکریپت، اجرا می‌کند.

۴) داده‌ی `sessionStorage` با بارگذاری دوباره‌ی (reload) همان تب باقی می‌ماند، ولی با بستن تب پاک می‌شود.

۹۹۳ سخت

کدام عبارت **درست** است؟

۱) اجرای `el.innerHTML = el.innerHTML` هیچ اثری ندارد و listenerهایی که با `addEventListener` روی فرزندان `el` ثبت شده‌اند حفظ می‌شوند.

۲) در listenerای که با یک تابع معمولی (`function`) ثبت شده، `this` همان `event.currentTarget` است؛ یعنی عنصری که listener روی آن ثبت شده، نه لزوماً عنصری که کلیک شده.

۳) `el.classList.toggle("on")` چیزی برنمی‌گرداند (`undefined`) و فقط کلاس را اضافه یا حذف می‌کند.

۴) `el.style.color` رنگی را که در فایل CSS (مثلاً با یک selector کلاس) برای عنصر تعیین شده برمی‌گرداند.

فصل سوم

۳

Node.js و TypeScript

بر اساس اسلایدهای «5- NodeJS & TS.pdf»

در این فصل از مرورگر بیرون می‌آییم: Node.js چرا و چطور به وجود آمد، npm و `package.json` و اسکریپت‌ها چه هستند، bundler به چه دردی می‌خورد و پکیج‌های معروف کدام‌اند. نیمه‌ی دوم فصل TypeScript است: نوع‌ها، interface، توابع، کلاس‌ها، generic، ماژول‌ها و بهترین شیوه‌ها.

۱۵ درس · ۳۰۲ تست · آزمون جامع پایان فصل

۳-۱ محدودیت‌های JavaScript و تولد Node.js
۳-۲ ویژگی‌های Node.js
۳-۳ پکیج‌های خارجی و npm
۳-۴ دستورات npm و تنظیم mirror
۳-۵ اسکریپت‌های npm و فایل package.json
۳-۶ Bundling برای مرورگر
۳-۷ پکیج‌های محبوب
۳-۸ مشکلات تایپ در JavaScript و معرفی TypeScript
۳-۹ متغیرها و انواع داده در TypeScript
۳-۱۰ Interface در TypeScript
۳-۱۱ توابع در TypeScript
۳-۱۲ کلاس‌ها در TypeScript
۳-۱۳ Generics در TypeScript
۳-۱۴ ماژول‌ها (Modules)
۳-۱۵ بهترین شیوه‌ها (Best Practices) در TypeScript

۳-۱ محدودیت‌های JavaScript و تولد Node.js

منبع: NodeJS & TS، اسلایدهای ۲ تا ۵

درسنامه

تا اینجای کتاب، JavaScript را فقط **داخل مرورگر** دیدیم: دکمه‌ها را کلیک‌پذیر کردیم، فرم‌ها را بررسی کردیم و DOM را تغییر دادیم. عنوان اسلاید اول این جلسه «**Beyond JavaScript**» است، یعنی «فراتر از JavaScript». در این درس می‌بینیم JavaScript اولش چه ضعف‌هایی داشت، چرا برنامه‌نویس‌ها خواستند آن را از مرورگر بیرون بیاورند و **Node.js** چطور این کار را ممکن کرد.

در این جلسه چه یاد می‌گیریم؟ (اسلاید ۲)

اسلاید ۲ سه هدف را روی سه کارت نشان می‌دهد:

کارت	متن اسلاید	یعنی…	کجای این فصل؟
a	Make console applications with JS	ساختن **برنامه‌های کنسولی** (console application: برنامه‌ای که در ترمینال اجرا می‌شود و صفحه‌ی گرافیکی ندارد) با JavaScript	Node.js و npm
b	Learn how JS runs on servers	یاد گرفتن اینکه JavaScript چطور روی **سرور** اجرا می‌شود	Node.js
c	Solve the old JS typing issues	حل مشکلات قدیمی **نوع‌دهی** (typing) در JavaScript	TypeScript (نیمه‌ی دوم فصل)

چند مفهوم پایه قبل از شروع

زبان، موتور و محیط اجرا

این سه کلمه را زیاد می‌شنوی و نباید قاطی‌شان کنی:

- **زبان (language)**: خودِ JavaScript؛ یعنی قواعد نوشتن کد (`let`، تابع، حلقه و…).
- **موتور (engine)**: برنامه‌ای که کد JavaScript را می‌خواند، به زبان ماشین تبدیل و اجرا می‌کند. معروف‌ترینش **V8** است که Google برای مرورگر Chrome ساخت.
- **محیط اجرا (runtime)**: موتور به‌اضافه‌ی ابزارهای آماده‌ای که آن محیط در اختیار کد می‌گذارد. مرورگر به کد ما `document` و `window` و `alert` می‌دهد؛ Node.js به جای آن‌ها ابزارهایی مثل `fs` (کار با فایل)، `http` (ساختن سرور) و `process` (اطلاعات برنامه‌ی در حال اجرا) می‌دهد.

> **یک تشبیه ساده**
>
> زبان مثل **دستور پخت** است، موتور مثل **آشپزی** است که دستور را می‌خواند و انجام می‌دهد، و محیط اجرا مثل **کل آشپزخانه** است: آشپز به‌اضافه‌ی وسایلش. آشپزی به اسم V8 هم در آشپزخانه‌ی Chrome کار می‌کند و هم در آشپزخانه‌ی Node.js؛ آشپز یکی است ولی وسایل دو آشپزخانه فرق دارد. برای همین در Node.js خبری از `document` نیست، ولی می‌توانی فایل بخوانی.

	مرورگر (مثلاً Chrome)	Node.js
زبان	JavaScript	JavaScript
موتور	V8	V8
ابزارهای مخصوص	`window`، `document`، `alert`، `localStorage`	`fs`، `http`، `process`، `require`
کاربرد اصلی	رابط کاربری صفحه‌ی وب	سرور، برنامه‌ی کنسولی، ابزارهای برنامه‌نویسی

client و server، frontend و backend

وقتی آدرس یک سایت را در مرورگر می‌زنی، مرورگرت یک **client** (کلاینت: برنامه‌ای که درخواست می‌فرستد) است و کامپیوتری که جواب می‌دهد یک **server** (سرور: برنامه یا کامپیوتری که به درخواست‌ها پاسخ می‌دهد). بخشی از برنامه که کاربر می‌بیند و در مرورگر اجرا می‌شود **frontend** (سمت کاربر) است و بخش پشت صحنه که روی سرور اجرا می‌شود (ذخیره در پایگاه داده، ورود کاربر، پرداخت و...) **backend** (سمت سرور).

I/O یعنی چه؟

I/O (Input/Output، ورودی/خروجی) هر کاری است که برنامه با «بیرون از خودش» انجام می‌دهد: خواندن و نوشتن فایل، فرستادن درخواست در شبکه، پرسیدن از پایگاه داده. این کارها در مقایسه با محاسبه‌ی خودِ CPU **خیلی کُند** هستند؛ پس یک سرور بیشتر وقتش را **منتظر I/O** است، نه مشغول حساب کردن. این نکته کلید فهمیدن Node.js است.

محدودیت‌های JavaScript (اسلاید ۳)

JavaScript در سال ۱۹۹۵ برای کارهای ساده‌ی صفحه‌ی وب ساخته شد. اسلاید ۳ پنج محدودیت آن را می‌شمارد:

۱. فقط برای کارهای ساده‌ی مرورگر طراحی شده بود. هدفش چیزهایی مثل بررسی یک فرم یا واکنش به کلیک بود، نه **برنامه‌های کامل و بزرگ** (full-scale applications) یا **منطق سمت سرور** (backend logic).

۲. اجرای تک‌نخی (single-threaded) باعث مسدود شدن (blocking) می‌شد. **thread** (نخ اجرا) را مثل یک کارگر در نظر بگیر. JavaScript فقط **یک** کارگر دارد که در هر لحظه یک کار انجام می‌دهد. اگر این کارگر درگیر یک محاسبه‌ی سنگین یا یک عملیات I/O طولانی شود، بقیه‌ی کارها **منتظر می‌مانند**؛ به این وضعیت **blocking** (مسدود شدن) می‌گوییم. مثال زیر را ببین:

```JavaScript
setTimeout(() => console.log("timer"), 0);

const start = Date.now();
while (Date.now() - start < 200) {} // busy wait: blocks the only thread for 200 ms
console.log("loop done");
// Output:
// loop done
// timer
```

با اینکه زمان تایمر `0` است، `timer` **بعد از** `loop done` چاپ می‌شود؛ چون تنها thread برنامه ۲۰۰ میلی‌ثانیه درگیر حلقه بود و نوبت به callback تایمر نمی‌رسید. (جزئیات صف‌ها و event loop را در فصل اول خواندیم.)

۳. نوع‌دهی پویا (dynamic typing) خطاهای غیرقابل‌پیش‌بینی می‌ساخت. در JavaScript نوع متغیر هنگام اجرا مشخص می‌شود و می‌تواند عوض شود. در پروژه‌های بزرگ این یعنی باگ‌هایی که فقط **هنگام اجرا** (runtime) پیدا می‌شوند و نگه‌داری کد را سخت می‌کنند:

```JavaScript
function total(price, count) {
  return price + count;
}
console.log(total(5, 2));   // 7
console.log(total("5", 2)); // "52" (a hidden bug: string concatenation!)
```

جواب این مشکل **TypeScript** است که در نیمه‌ی دوم همین فصل می‌خوانیم.

۴. نبودِ سیستم ماژول (module system). **ماژول** (module) یعنی یک فایل کد با مرز مشخص که فقط چیزهایی را که خودش بخواهد به بقیه نشان می‌دهد. JavaScript اولیه چنین چیزی نداشت: همه‌ی فایل‌های `<script>` یک **محدوده‌ی سراسری** (global scope) مشترک داشتند و اسم‌ها با هم تداخل می‌کردند:

```HTML
<script src="cart.js"></script>  <!-- var count = 5; -->
<script src="likes.js"></script> <!-- var count = 0; overwrites the cart count! -->
```

پس سازمان‌دهی کد و استفاده‌ی دوباره از آن سخت بود. بعدها Node.js سیستم ماژول CommonJS (`require`) را آورد و خود زبان هم ES Modules (`import` و `export`) را به دست آورد.

۵. ابزارهای محدود و مقیاس‌پذیری ضعیف. ابزار درست‌وحسابی برای مدیریت کتابخانه‌ها، ساختن (build) و تست پروژه نبود و کد با بزرگ شدن پروژه به‌سختی قابل نگه‌داری بود؛ یعنی **scalability** (مقیاس‌پذیری: توانایی رشد کردن بدون به هم ریختن) ضعیفی داشت.

محدودیت (اسلاید ۳)	جوابش در همین فصل
فقط مرورگر، نه برنامه‌ی کامل یا backend	Node.js (اجرای JS روی سرور و در ترمینال)
تک‌نخی و blocking در I/O	مدل event-driven و non-blocking در Node.js
dynamic typing	TypeScript
نبودِ module system	ماژول‌های CommonJS و ES Modules، پکیج‌های npm
ابزار محدود و مقیاس‌پذیری ضعیف	npm، bundlerها، TypeScript

جرقه‌ی اولیه (Inception، اسلاید ۴)

چه شد که کسی به فکر بردن JavaScript به بیرون از مرورگر افتاد؟ اسلاید ۴ پنج دلیل می‌آورد:

۱. برنامه‌نویس‌ها می‌خواستند مهارت JavaScript خود را **بیرون از مرورگر** هم به کار ببرند، مخصوصاً برای منطق **backend**؛ به جای اینکه برای سرور یک زبان دیگر (مثل PHP یا Java) هم یاد بگیرند.

۲. برنامه‌های وب **پویاتر** شدند و تبادل داده بین client و server باید **سریع‌تر و یکپارچه‌تر** (unified) انجام می‌شد؛ اگر هر دو طرف یک زبان و یک قالب داده داشته باشند، این کار خیلی ساده‌تر است.

۳. ظهور **AJAX** و **JSON** نشان داد JavaScript می‌تواند کارهای **ناهمگام** (asynchronous: کاری که شروع می‌شود و نتیجه‌اش بعداً می‌رسد، بدون اینکه برنامه منتظر بماند) را به‌خوبی مدیریت کند.

- (Asynchronous JavaScript And XML) **AJAX**: روشی که صفحه‌ی وب **بدون بارگذاری دوباره** و در پس‌زمینه از سرور داده بگیرد؛ مثلاً وقتی در نقشه‌ی آنلاین جابه‌جا می‌شوی و تکه‌های جدید نقشه بدون refresh ظاهر می‌شوند.

- (JavaScript Object Notation) **JSON**: یک قالب متنی برای جابه‌جایی داده که دقیقاً شبیه شیءهای JavaScript است:

```JSON
{ "name": "Ali", "age": 21, "courses": ["Web", "AI"] }
```

۴. **بهبود موتورها** (engine improvements)، مخصوصاً **V8** ساخت Google، JavaScript را آن‌قدر سریع کرد که برای بار کاری سرور هم کافی بود. V8 کد JavaScript را هنگام اجرا به **کد ماشین** کامپایل می‌کند، نه اینکه فقط خط‌به‌خط تفسیرش کند.

۵. همه‌ی این‌ها به ایده‌ی اجرای JavaScript **بیرون از مرورگر** رسید؛ یعنی شروع حرکت به سمت **server-side JavaScript** (JavaScript سمت سرور).

> **تاریخچه‌ی کوتاه**
>
> این تاریخ‌ها در اسلایدها نیست و فقط برای تصویر ذهنی بهتر آمده است: ۱۹۹۵ تولد JavaScript در شرکت Netscape؛ ۲۰۰۵ رواج نام AJAX؛ ۲۰۰۸ عرضه‌ی Chrome و موتور V8؛ ۲۰۰۹ معرفی Node.js توسط Ryan Dahl؛ ۲۰۱۰ عرضه‌ی npm.

سرانجام Node.js آمد (اسلاید ۵)

Node.js یک **runtime** (محیط اجرا) برای JavaScript است که روی موتور V8 ساخته شده و اجازه می‌دهد کد JavaScript را **بیرون از مرورگر** (روی سرور یا در ترمینال کامپیوتر خودت) اجرا کنی. دقت کن: Node.js **زبان جدید نیست** (زبانش همان JavaScript است)، **فریم‌ورک** (framework) هم نیست و **مرورگر** هم نیست.

اسلاید ۵ پنج ویژگی آن را می‌گوید:

۱. **Powered by Google's fast V8 engine:** همان موتور سریع Chrome.

۲. **Event-driven, non-blocking I/O model:** مهم‌ترین ایده‌ی Node.js.

- **non-blocking I/O** (ورودی/خروجی غیرمسدودکننده): Node.js کار I/O (مثلاً خواندن فایل) را **شروع می‌کند و منتظر نمی‌ماند**؛ می‌رود سراغ ادامه‌ی کد. وقتی نتیجه آماده شد، خبرش می‌کنند.
- **event-driven** (رویدادمحور): کد ما به **رویدادها** (event) واکنش نشان می‌دهد: «درخواستی رسید»، «خواندن فایل تمام شد»، «زمان تایمر رسید». ما از قبل یک تابع **callback** (تابعی که بعداً صدا زده می‌شود) ثبت می‌کنیم و Node.js هر وقت آن رویداد رخ داد، آن را اجرا می‌کند.

> **گارسون رستوران**
>
> یک رستوران را با **یک گارسون** تصور کن (همان thread واحد).
>
> - **گارسون blocking**: سفارش میز ۱ را می‌گیرد، به آشپزخانه می‌دهد و **دم در آشپزخانه می‌ایستد** تا غذا آماده شود؛ در این مدت میزهای دیگر معطل‌اند.
> - **گارسون non-blocking (مثل Node.js)**: سفارش میز ۱ را به آشپزخانه می‌دهد و **فوراً** سراغ میز ۲ و ۳ می‌رود. هر وقت آشپزخانه **زنگ زد** (رویداد)، غذای آماده را می‌برد (اجرای callback).
>
> با همان یک گارسون، رستوران دوم به مشتری‌های خیلی بیشتری می‌رسد. آشپزخانه اینجا همان سیستم‌عامل و بخش‌های داخلی Node.js است که کار I/O را در پس‌زمینه انجام می‌دهند.

این تفاوت را در کد ببین. `fs.readFile` غیرمسدودکننده است و `fs.readFileSync` مسدودکننده:

non-blocking.js

```
const fs = require("fs");

console.log("start");
fs.readFile(__filename, "utf8", () => {
  console.log("file read"); // runs later, when the file is ready
});
console.log("end");
// Output: start, end, file read
```

blocking.js

```
const fs = require("fs");

console.log("start");
const text = fs.readFileSync(__filename, "utf8"); // waits here until the file is read
console.log("file read");
console.log("end");
// Output: start, file read, end
```

متغیر `__filename` در Node.js مسیر همین فایلِ در حال اجراست؛ یعنی در این دو مثال برنامه دارد خودش را می‌خواند.

۳. Same language for frontend and backend: یک زبان برای هر دو سمت. یک برنامه‌نویس می‌تواند هم frontend و هم backend بنویسد (به چنین کسی full-stack می‌گویند) و حتی کد مشترک (مثلاً تابع بررسی ایمیل) را در هر دو سمت استفاده کند.

۴. Massive npm ecosystem for packages: اکوسیستم عظیم **npm** (مدیر بسته‌ی Node) با میلیون‌ها پکیج آماده. در درس‌های بعدی مفصل می‌خوانیمش.

۵. Turned JavaScript into a full platform: JavaScript از «زبان اسکریپت‌نویسی مرورگر» به یک **پلتفرم کامل** تبدیل شد: سرور، برنامه‌ی کنسولی، ابزارهای ساخت پروژه و حتی برنامه‌ی دسکتاپ.

اولین برنامه‌ها با Node.js

یک فایل بساز و با دستور `node` اجرایش کن:

hello.js

```
console.log("Hello from Node.js!");
```

Terminal

```
node hello.js
# Hello from Node.js!
```

این یک **برنامه‌ی کنسولی** است؛ همان هدف a در اسلاید ۲. حالا یک وب‌سرور خیلی کوچک (هدف b):

server.js

```
const http = require("http");

const server = http.createServer((req, res) => {
  res.end("Hello from a Node.js server!"); // the answer sent to the browser
});

server.listen(3000, () => {
  console.log("Server running at http://localhost:3000");
});
```

بعد از `node server.js` برنامه تمام نمی‌شود؛ منتظر **رویداد** «رسیدن درخواست» می‌ماند. هر بار که آدرس `http://localhost:3000` را در مرورگر باز کنی، callback داده‌شده به `createServer` اجرا می‌شود و متن را برمی‌گرداند. این یعنی event-driven بودن.

در Node.js چه چیزهایی نیست و چه چیزهایی هست؟

Node.js صفحه‌ی وب ندارد، پس **DOM** هم ندارد: `window` ، `document` و `alert` تعریف نشده‌اند. به جایش چیزهای دیگری دارد:

- `globalThis` : شیء سراسری که در **هر دو** محیط هست (در Node.js همان `global` است و در مرورگر همان `window`).
- `process` : اطلاعات برنامه‌ی در حال اجرا؛ مثل `process.argv` (آرگومان‌های خط فرمان)، `process.env` (متغیرهای محیطی) و `process.platform` (نام سیستم‌عامل).
- ماژول‌های داخلی مثل `fs` (فایل)، `http` (سرور)، `path` (مسیر فایل‌ها) و `os` (اطلاعات سیستم‌عامل).

JavaScript

```
// Running in Node.js
console.log(typeof window);     // "undefined"
console.log(typeof globalThis); // "object"
console.log(typeof process);    // "object"
console.log(document.title);    // ReferenceError: document is not defined
```

نکته

Node.js **زبان** نیست؛ یک **runtime** است. زبانش همان JavaScript است و موتورش همان V8 مرورگر Chrome. فرق Node.js با مرورگر در **ابزارهای اطراف موتور** است: مرورگر DOM دارد و Node.js فایل و شبکه و `process` .

دام تستی

«Node.js تک‌نخی است، پس در هر لحظه فقط به یک کاربر جواب می‌دهد.» این جمله **غلط** است. چون I/O در Node.js غیرمسدودکننده است، همان یک thread هر درخواست را راه می‌اندازد و سراغ درخواست بعدی می‌رود؛ پس هزاران درخواست هم‌زمان را مدیریت می‌کند. ولی دقت کن: کارِ **محاسباتی سنگین** (مثل حلقه‌ی طولانی بالا) هنوز همان یک thread را **مسدود می‌کند** و همه منتظر می‌مانند.

⚠ دام تستی

`typeof window` در Node.js خطا **نمی‌دهد** و فقط رشته‌ی `"undefined"` برمی‌گرداند (`typeof` روی متغیر تعریف‌نشده خطا نمی‌دهد). ولی استفاده‌ی واقعی از آن، مثل `document.title` یا `window.innerWidth`، خطای `ReferenceError` می‌دهد.

جمع‌بندی

- اهداف جلسه: برنامه‌ی کنسولی با JS، اجرای JS روی سرور، حل مشکلات نوع‌دهی (با TypeScript).
- محدودیت‌های JS: فقط مرورگر، تک‌نخی و blocking، dynamic typing، نبودِ module system، ابزار و مقیاس‌پذیری ضعیف.
- جرقه‌ی اولیه: استفاده‌ی دوباره از مهارت JS در backend، تبادل داده‌ی یکپارچه، AJAX و JSON، موتور V8، JavaScript سمت سرور.
- Node.js: موتور I/O، V8 رویدادمحور و غیرمسدودکننده، یک زبان برای frontend و backend، اکوسیستم npm، پلتفرم کامل.

تست‌های تمرینی

۱۷ تست · شماره‌ی ۹۹۴ تا ۱۰۱۰

۹۹۴ آسان

کدام گزینه جزو اهداف جلسه‌ی «Beyond JavaScript» (اسلاید ۲) **نیست**؟

۱) ساختن برنامه‌های کنسولی (console application) با JavaScript

۲) یاد گرفتن اینکه JavaScript چطور روی سرور اجرا می‌شود

۳) طراحی ظاهر صفحه‌های وب با CSS

۴) حل مشکلات قدیمی نوع‌دهی (typing) در JavaScript

۹۹۵ آسان

طبق اسلاید ۳، کدام مورد از محدودیت‌های JavaScript در سال‌های اول **نبود**؟

۱) اجرای تک‌نخی (single-threaded) که در محاسبات سنگین یا عملیات I/O باعث مسدود شدن (blocking) می‌شد

۲) نوع‌دهی پویا (dynamic typing) که در پروژه‌های بزرگ خطاهای غیرمنتظره‌ی زمان اجرا می‌ساخت

۳) نبودِ یک سیستم ماژول (module system) درست برای سازمان‌دهی و استفاده‌ی دوباره از کد

۴) اینکه JavaScript در مرورگر اجرا نمی‌شد و فقط روی سرور کار می‌کرد

۹۹۶ آسان

طبق اسلایدهای ۴ و ۵، بهبود کدام موتور JavaScript آن را برای کارهای سمت سرور به‌اندازه‌ی کافی سریع کرد و Node.js هم بر پایه‌ی همان ساخته شد؟

۱) SpiderMonkey (موتور مرورگر Firefox)

۲) V8 ساخت Google

۳) JavaScriptCore (موتور مرورگر Safari)

۴) Chakra (موتور قدیمی مرورگر Edge)

۹۹۷ متوسط

یک رستوران فقط **یک گارسون** دارد. کدام رفتار گارسون، مدل **non-blocking I/O** در Node.js را درست نشان می‌دهد؟

۱) سفارش میز ۱ را به آشپزخانه می‌دهد، فوراً سراغ میزهای دیگر می‌رود و هر وقت آشپزخانه زنگ زد، غذای آماده را می‌برد.

۲) سفارش میز ۱ را به آشپزخانه می‌دهد و دم در آشپزخانه می‌ایستد تا غذا آماده شود؛ بعد سراغ میز ۲ می‌رود.

۳) برای هر مشتری تازه یک گارسون جدید استخدام می‌شود تا هیچ مشتری‌ای منتظر نماند.

۴) گارسون غذای همه‌ی میزها را خودش می‌پزد تا به آشپزخانه نیازی نباشد.

۹۹۸ متوسط

خروجی کد زیر در Node.js کدام است؟

```JavaScript
const fs = require("fs");

console.log("start");
fs.readFile(__filename, "utf8", () => {
  console.log("file read");
});
console.log("end");
```

۱) `start` `file read` `end`

۲) `start` `end` `file read`

۳) `file read` `start` `end`

۴) `start` `end`

۹۹۹ متوسط

در هر چهار گزینه، ماژول `fs` قبلاً با `const fs = require("fs");` بارگذاری شده است. کدام گزینه در Node.js **دقیقاً** خروجی زیر را چاپ می‌کند؟

```
start
end
done
```

۱

```JavaScript
console.log("start");
fs.readFile(__filename, () => console.log("done"));
console.log("end");
```

۲

```JavaScript
console.log("start");
fs.readFileSync(__filename);
console.log("done");
console.log("end");
```

۳

```JavaScript
fs.readFile(__filename, () => console.log("start"));
console.log("end");
console.log("done");
```

۴

```JavaScript
console.log("start");
fs.readFile(__filename, () => console.log("end"));
console.log("done");
```

۱۰۰۰ سخت

خروجی کد زیر در Node.js کدام است؟

```JavaScript
setTimeout(() => console.log("timer"), 0);
console.log("before loop");

const start = Date.now();
while (Date.now() - start < 100) {} // busy wait for about 100 ms
console.log("after loop");
```

۱ `before loop` `after loop` `timer`

۲ `timer` `before loop` `after loop`

۳ `before loop` `timer` `after loop`

۴ `before loop` `after loop`

۱۰۰۱ آسان

کدام تعریف درباره‌ی **Node.js** درست است؟

۱) یک زبان برنامه‌نویسی جدید که جایگزین JavaScript شده است

۲) یک فریم‌ورک (framework) برای ساختن رابط کاربری، مثل React

۳) یک محیط اجرا (runtime) برای JavaScript که روی موتور V8 ساخته شده و JS را بیرون از مرورگر اجرا می‌کند

۴) یک مرورگر وب سبک که Google برای سرورها ساخته است

۱۰۰۲ متوسط

خروجی کد زیر در **Node.js** کدام است؟

```JavaScript
console.log(typeof window);
console.log(typeof globalThis);
console.log(typeof process);
```

۱) `object` `object` `undefined`

۲) `Uncaught ReferenceError`

۳) `undefined` `undefined` `object`

۴) `undefined` `object` `object`

۱۰۰۳ سخت

اگر کد زیر را در **Node.js** اجرا کنیم، اولین خطا در کدام خط رخ می‌دهد؟

```JavaScript
1 console.log(typeof document);
2 const os = require("os");
3 console.log(typeof os.platform());
4 console.log(window.innerWidth);
5 console.log("finished");
```

۱) خط ۱ ۲) خط ۴

۳) خط ۵ ۴) هیچ خطایی رخ نمی‌دهد

۱۰۰۴ متوسط

طبق اسلاید ۴، ظهور **AJAX** و **JSON** چه چیزی را درباره‌ی JavaScript نشان داد؟

۱) اینکه JavaScript می‌تواند چند thread هم‌زمان داشته باشد.

۲) اینکه JavaScript بدون XML نمی‌تواند داده‌ای از سرور بگیرد.

۳) اینکه JavaScript می‌تواند عملیات ناهمگام (asynchronous) را به‌خوبی مدیریت کند.

۴) اینکه JavaScript فقط باید در مرورگر بماند و برای سرور مناسب نیست.

۱۰۰۵ متوسط

کدام گزینه منظور از **event-driven** (رویدادمحور) بودن Node.js را درست بیان می‌کند؟

۱) برنامه خط‌به‌خط اجرا می‌شود و برای هر عملیات I/O آن‌قدر صبر می‌کند تا تمام شود.

۲) برای هر درخواستی که به سرور می‌رسد، یک thread جدید ساخته می‌شود.

۳) برنامه فقط به کلیک‌ها و رویدادهای موس کاربر روی صفحه واکنش نشان می‌دهد.

۴) کد از قبل callbackهایی ثبت می‌کند و Node.js هر وقت رویدادی مثل «رسیدن درخواست» یا «تمام شدن خواندن فایل» رخ داد، آن‌ها را اجرا می‌کند.

۱۰۰۶ سخت

کدام عبارت درباره‌ی Node.js **نادرست** است؟

۱) چون Node.js روی سرور اجرا می‌شود، اشیای `document` و `window` در آن در دسترس‌اند تا صفحه را روی سرور بسازیم.

۲) Node.js از همان موتور V8 استفاده می‌کند که در مرورگر Chrome هست.

۳) با Node.js می‌توان برای frontend و backend از یک زبان استفاده کرد.

۴) وقتی یک حلقه‌ی محاسباتی سنگین در حال اجراست، callbackهای دیگر (مثل تایمرها) باید منتظر تمام شدن آن بمانند.

۱۰۰۷ متوسط

جای خالی را کامل کن:

«Node.js به لطف مدل I/O ..... و .....، فقط با یک thread به تعداد زیادی درخواست هم‌زمان پاسخ می‌دهد.»

۱) چندنخی (multi-threaded) و مسدودکننده (blocking)

۲) همگام (synchronous) و مسدودکننده (blocking)

۳) رویدادمحور (event-driven) و غیرمسدودکننده (non-blocking)

۴) همگام (synchronous) و بدون callback

۱۰۰۸ سخت

کدام جفتِ «محدودیت قدیمی JavaScript ← راه‌حلی که در این فصل می‌خوانیم» **نادرست** است؟

۱) نوع‌دهی پویا (dynamic typing) ← TypeScript

۲) اجرای تک‌نخی ← Node.js برای هر درخواست یک thread جدید می‌سازد تا هیچ کاری مسدود نشود

۳) نبودِ module system ← ماژول‌های CommonJS و ES Modules و پکیج‌های npm

۴) محدود بودن به مرورگر ← Node.js که JavaScript را روی سرور و در ترمینال اجرا می‌کند

۱۰۰۹ آسان

کدام مورد جزو ویژگی‌هایی که اسلاید ۵ («Then Came Node.js») برای Node.js می‌شمارد **نیست**؟

۱) اکوسیستم عظیم npm برای پکیج‌ها

۲) مدل I/O رویدادمحور و غیرمسدودکننده

۳) استفاده از یک زبان برای frontend و backend

۴) بررسی نوع‌ها (type checking) در زمان کامپایل

۱۰۱۰ سخت

خروجی کد زیر در Node.js کدام است؟ (در این برنامه هیچ درخواستی به سرور فرستاده نمی‌شود. پورت `0` یعنی «هر پورت آزادی»، و `server.close` سرور را می‌بندد تا برنامه تمام شود.)

```JavaScript
const http = require("http");

const server = http.createServer((req, res) => {
  console.log("request received");
  res.end("Hello");
});

server.listen(0, () => {
  console.log("listening");
  server.close(() => console.log("closed"));
});
console.log("setup done");
```

۱) `listening` `setup done` `closed`

۲) `setup done` `listening` `request received` `closed`

۳) `setup done` `listening` `closed`

۴) `request received` `listening` `setup done` `closed`

۳-۲ ویژگی‌های Node.js

منبع: NodeJS & TS، اسلایدهای ۶ و ۷

درسنامه

در درس قبل دیدیم Node.js چرا به وجود آمد. حالا وقت آن است که با آن کار کنیم و ویژگی‌هایش را یکی‌یکی بشناسیم. اسلاید ۶ فقط یک **تصویر** است (یک نمودار با هشت برچسب دور لوگوی Node.js) و اسلاید ۷ پنج ویژگی دیگر و یک میم دارد؛ هر دو را کامل اینجا آورده‌ایم.

اجرای Node.js در عمل

بعد از نصب Node.js، در ترمینال سه کار اصلی می‌توانی انجام دهی:

```
Terminal
node -v        # print the installed version, e.g. v24.22.2
node app.js    # run the file app.js
node           # open the REPL (interactive mode)
```

اجرای یک فایل

هر فایل `.js` را با `node <file>` اجرا می‌کنی؛ خروجی `console.log` در همان **ترمینال** چاپ می‌شود، نه در کنسول مرورگر. یک برنامه‌ی کنسولی کوچک که اسم کاربر را از **آرگومان‌های خط فرمان** (command-line arguments: کلمه‌هایی که بعد از نام فایل تایپ می‌کنی) می‌گیرد:

```
greet.js
const name = process.argv[2] || "stranger";
console.log("Hello, " + name + "!");
```

```
Terminal
node greet.js Ali   # Hello, Ali!
node greet.js       # Hello, stranger!
```

`process.argv` آرایه‌ای است که خانه‌ی `0` آن مسیر خودِ برنامه‌ی `node`، خانه‌ی `1` مسیر فایل و از خانه‌ی `2` به بعد آرگومان‌های ما هستند.

REPL

اگر فقط `node` را بزنی، وارد **REPL** (Read-Eval-Print Loop: حلقه‌ی «بخوان، ارزیابی کن، چاپ کن») می‌شوی؛ هر عبارتی تایپ کنی، فوراً اجرا و نتیجه‌اش نشان داده می‌شود. برای امتحان سریع یک تکه کد عالی است:

```
> 1 + 2
3
> "a" + 1
'a1'
> const x = 5
undefined
> x * 2
10
> .exit
```

REPL رشته‌ها را با کوتیشن نشان می‌دهد (`'a1'`) و برای دستوری که مقدار برنمی‌گرداند (مثل تعریف متغیر) `undefined` می‌نویسد. با `.exit` یا دو بار `Ctrl+C` بیرون می‌آیی.

هشت ویژگی Node.js (اسلاید ۶)

این اسلاید یک نمودار تصویری است: لوگوی Node.js در وسط و هشت برچسب رنگی دورش، چهار تا در هر طرف:

ستون چپ نمودار	ستون راست نمودار
Fast Data Streaming	Single-Threaded
Highly Scalable	Event-Driven
JavaScript	No Buffering
Cross-Platform Compatibility	Asynchronous

اسلاید هیچ توضیحی نمی‌دهد؛ معنی هر کدام را با هم ببینیم:

ویژگی	معنی ساده
Asynchronous (ناهمگام)	توابع Node.js منتظر تمام شدن کارهای کُند (فایل، شبکه) نمی‌مانند؛ کار را شروع می‌کنند و نتیجه بعداً با callback یا Promise می‌رسد.
Event-Driven (رویدادمحور)	برنامه حول رویدادها می‌چرخد: «درخواست رسید»، «داده آمد»، «کار تمام شد». برای هر رویداد یک تابع شنونده (listener) ثبت می‌کنیم.
Single-Threaded (تک‌نخی)	کد JavaScript ما روی **یک** thread اجرا می‌شود و event loop کارها را نوبت‌بندی می‌کند.
Highly Scalable (مقیاس‌پذیری بالا)	چون منتظر I/O نمی‌ماند، با منابع کم به تعداد خیلی زیادی اتصال هم‌زمان جواب می‌دهد.
No Buffering (بدون بافر کردن)	داده را **تکه‌تکه** (chunk) خروجی می‌دهد، نه اینکه اول همه‌اش را در حافظه جمع کند (**buffer**: حافظه‌ی موقتی که داده در آن جمع می‌شود).
Fast Data Streaming (جریان سریع داده)	موتور سریع V8 به‌اضافه‌ی پردازش داده به‌صورت **stream** (جریان: داده‌ای که ذره‌ذره می‌رسد و ذره‌ذره پردازش می‌شود).
JavaScript	همان زبان مرورگر؛ برنامه‌نویس frontend بدون یاد گرفتن زبان جدید، backend هم می‌نویسد.
Cross-Platform Compatibility (سازگاری چندسکویی)	همان کد روی Linux، Windows و macOS اجرا می‌شود. `process.platform` نام سیستم‌عامل را می‌دهد (`"win32"`، `"linux"`، `"darwin"`).

Asynchronous و Single-Threaded با هم چطور جور درمی‌آیند؟

شاید بپرسی «اگر فقط یک thread هست، چطور کارها هم‌زمان پیش می‌روند؟» جواب: کد **JavaScript** ما روی یک thread اجرا می‌شود، ولی کارهای I/O را سیستم‌عامل و کتابخانه‌ی داخلی Node.js (به نام libuv) **در پس‌زمینه** انجام می‌دهند. thread اصلی فقط کار را سفارش می‌دهد و وقتی نتیجه آماده شد، callback را اجرا می‌کند. برای همین Node.js با یک thread است؛ **Highly Scalable** ولی یادت باشد که یک محاسبه‌ی سنگین JavaScript همان thread را مسدود می‌کند.

Event-Driven در عمل: EventEmitter

قلب رویدادمحور بودن Node.js کلاسی به نام `EventEmitter` است. با `on` برای یک رویداد شنونده ثبت می‌کنی و با `emit` آن رویداد را «اعلام» می‌کنی:

```JavaScript
const EventEmitter = require("events");
const shop = new EventEmitter();

shop.on("order", (item) => console.log(`Preparing ${item}`)); // register a listener
console.log("Waiting...");
shop.emit("order", "pizza"); // fire the event
console.log("Done");
// Output:
// Waiting...
// Preparing pizza
// Done
```

دقت کن `Preparing pizza` **قبل از** `Done` چاپ شد: `emit` شنونده‌ها را **همان لحظه و همگام** صدا می‌زند و تا کارشان تمام نشود، خط بعد اجرا نمی‌شود. اگر برای رویدادی هیچ شنونده‌ای ثبت نشده باشد، `emit` هیچ کاری نمی‌کند (فقط `false` برمی‌گرداند) و خطا هم نمی‌دهد.

No Buffering و Fast Data Streaming در عمل: stream

فرض کن می‌خواهی یک فیلم ۲ گیگابایتی را برای کاربر بفرستی. راه بد: کل فایل را در حافظه بخوانی و بعد بفرستی. راه Node.js: فایل را **تکه‌تکه** بخوانی و هر تکه را فوراً بفرستی؛ درست مثل تماشای آنلاین فیلم که لازم نیست صبر کنی کل فیلم دانلود شود. در مثال زیر، `highWaterMark: 4` اندازه‌ی هر تکه را ۴ بایت (اینجا ۴ حرف) گذاشته است:

```JavaScript
const fs = require("fs");
fs.writeFileSync("data.txt", "abcdefghij");

const stream = fs.createReadStream("data.txt", { encoding: "utf8", highWaterMark: 4 });
stream.on("data", (chunk) => console.log("chunk:", chunk)); // fired for every chunk
stream.on("end", () => console.log("done"));                // fired once at the end
console.log("reading...");
// Output:
// reading...
// chunk: abcd
// chunk: efgh
// chunk: ij
// done
```

اول `reading...` آمد، چون خواندن stream ناهمگام است. بعد رویداد `data` سه بار (برای هر تکه) و در آخر رویداد `end` یک بار رخ داد. یعنی stream هم رویدادمحور است.

ادامه‌ی ویژگی‌ها (اسلاید ۷)

۱. Built on async/await for cleaner, modern concurrency. API های جدید Node.js به جای callback، **Promise** برمی‌گردانند، پس می‌شود با `async/await` کد ناهمگام را مثل کد معمولی و خطبه‌خط نوشت. (**concurrency** یعنی هم‌روندی: پیش بردن چند کار در یک بازه‌ی زمانی.)

JavaScript

```
const fs = require("fs/promises"); // the Promise-based version of fs

async function main() {
  await fs.writeFile("note.txt", "Hello file");
  const text = await fs.readFile("note.txt", "utf8");
  console.log(text);
}

main();
console.log("main started");
// Output:
// main started
// Hello file
```

`main()` تا اولین `await` جلو می‌رود و بعد کنترل را پس می‌دهد؛ برای همین `main started` زودتر چاپ می‌شود.

۲. Node.js **Uses ES modules and import/export syntax natively.** سیستم ماژول استاندارد JavaScript یعنی **ES Modules** (ESM) را مستقیم و بدون ابزار اضافه پشتیبانی می‌کند (پایین‌تر مفصل می‌بینیم).

۳. Node.js **Embraces TypeScript, ESM, and modern tooling by default.** با TypeScript، ESM و ابزارهای مدرن به‌خوبی کنار می‌آید. مثلاً Node.js نسخه‌ی ۲۴ می‌تواند یک فایل `.ts` را مستقیم اجرا کند: فقط **نوع‌ها را پاک می‌کند** (type stripping) و اجرا می‌کند، ولی آن‌ها را **بررسی نمی‌کند**. پس برای پیدا کردن خطاهای نوع هنوز به کامپایلر TypeScript (`tsc`) نیاز داریم.

Terminal

```
node app.ts   # Node.js 24 removes the types and runs the code (no type checking!)
```

۴. framework **Supports powerful frameworks like Express, NestJS, and Fastify.** (چارچوب: اسکلت آماده‌ای که ساختار برنامه را تعیین می‌کند و ما فقط بخش‌های خودمان را در آن می‌نویسیم) کار ساختن سرور را خیلی ساده می‌کند:

- **Express**: کمینه‌گرا (minimalist) و پرطرفدارترین فریم‌ورک وب Node.js (در درس «پکیج‌های محبوب» دوباره می‌بینیمش).
- **NestJS**: فریم‌ورکی ساختاریافته برای پروژه‌های بزرگ که با TypeScript نوشته می‌شود.
- **Fastify**: فریم‌ورکی با تمرکز بر سرعت بالا و سربار کم.

۵. Integrates seamlessly with containers, cloud, and edge runtimes. برنامه‌ی Node.js را راحت می‌شود داخل **container** (بسته‌ای که برنامه را با همه‌ی نیازهایش یکجا نگه می‌دارد تا همه‌جا یکسان اجرا شود؛ مثل Docker) گذاشت، روی سرویس‌های **cloud** (ابری) اجرا کرد یا در **edge runtime**ها (محیط‌هایی که کد را روی سرورهای نزدیک به محل کاربر اجرا می‌کنند) استفاده کرد.

میم اسلاید ۷

کنار این اسلاید یک میم با عنوان «**JavaScript programming**» هست: مردی که روی خودش نوشته «**app 300kb**» یک کوله‌پشتی نظامی غول‌پیکر به دوش دارد که رویش نوشته «**Node_modules, 12GB**». شوخی‌اش این است که کد خود برنامه کوچک است، ولی پوشه‌ی `node_modules` (جایی که npm همه‌ی پکیج‌های نصب‌شده را می‌گذارد) به‌خاطر **وابستگی‌ها** (dependency: پکیج‌هایی که برنامه به آن‌ها نیاز دارد) و وابستگی‌های آن وابستگی‌ها، **خیلی** بزرگ می‌شود. اسم این مشکل dependency bloat (پف کردن وابستگی‌ها) است؛ در درس بعد، «dependency overload» در اسلاید ۸ و «size» در اسلاید ۹ به همین اشاره دارند.

دو سیستم ماژول در Node.js: CommonJS و ES Modules

Node.js دو روش برای تقسیم کد به فایل‌های جدا (ماژول) دارد.

CommonJS (روش قدیمی و پیش‌فرض Node.js)

با `module.exports` چیزی را **صادر** (export) می‌کنی و با `require` **وارد** (import) می‌کنی:

math.js

```
console.log("math.js loaded");
function add(a, b) {
  return a + b;
}
module.exports = { add, PI: 3.14 };
```

app.js

```
const math = require("./math");       // the ".js" extension is optional here
const { add } = require("./math.js"); // destructuring works too
console.log(math.add(2, 3), add(1, 1), math.PI);
// Output:
// math.js loaded
// 5 2 3.14
```

`math.js loaded` فقط **یک بار** چاپ شد، با اینکه دو بار `require` کردیم: Node.js هر ماژول را فقط بار اول اجرا می‌کند و نتیجه را **cache** (ذخیره) می‌کند؛ `require` های بعدی همان شیء ذخیره‌شده را برمی‌گردانند.

ES Modules (روش استاندارد و جدید)

با `export` و `import` کار می‌کنی؛ همان سیستمی که مرورگرها و TypeScript هم استفاده می‌کنند:

math.mjs

```
export function add(a, b) {
  return a + b;
}
export const PI = 3.14;
export default function greet(name) {
  return "Hello " + name;
}
```

app.mjs

```
import greet, { add, PI } from "./math.mjs"; // the extension is required in ESM
console.log(add(2, 3), PI, greet("Ali"));    // 5 3.14 Hello Ali
```

Node.js از کجا می‌فهمد فایل CommonJS است یا ESM؟

فایل	Node.js آن را چطور اجرا می‌کند
`name.mjs`	همیشه ES Module
`name.cjs`	همیشه CommonJS
`name.js` و در `package.json` نوشته شده `"type": "module"`	ES Module
`name.js` و در `package.json` نوشته شده `"type": "commonjs"`	CommonJS (دستور `npm init` در نسخه‌های جدید همین را می‌نویسد)
`name.js` بدون فیلد `type`	CommonJS؛ ولی Node.js جدید اگر در فایل `import` / `export` ببیند، آن را ESM اجرا می‌کند (و اگر `package.json` وجود داشته باشد، یک هشدار هم می‌دهد)

	CommonJS	ES Modules
وارد کردن	`require("./math")`	`import { add } from "./math.mjs"`
صادر کردن	`module.exports = …` یا `exports.x = …`	`export` و `export default`
پسوند در مسیرهای نسبی	اختیاری	اجباری
`require` و `__dirname` و `__filename`	تعریف شده‌اند	تعریف **نشده‌اند**
`this` در بالاترین سطح فایل	`{}` (همان `module.exports`)	`undefined`
`await` در بالاترین سطح فایل (top-level await)	مجاز نیست	مجاز است
استاندارد بودن	مخصوص Node.js	استاندارد رسمی JavaScript (در مرورگر هم کار می‌کند)

⚠ دام تستی

- در CommonJS، `exports` فقط یک **میان‌بُر** به `module.exports` است. `exports.hello = "hi"` درست کار می‌کند، ولی `exports = { hello: "hi" }` فقط متغیر محلی `exports` را عوض می‌کند و چیزی صادر **نمی‌شود** (`require` همان شیء خالی `{}` را برمی‌گرداند). برای صادر کردن یک شیء کامل از `module.exports = {...}` استفاده کن.
- اگر در `package.json` نوشته باشد `"type": "module"`، استفاده از `require` در یک فایل `.js` خطای `ReferenceError: require is not defined in ES module scope` می‌دهد.
- برعکس، اگر `"type": "commonjs"` باشد (پیش‌فرض `npm init` جدید)، نوشتن `import` در فایل `.js` خطای `SyntaxError: Cannot use import statement outside a module` می‌دهد.

Node.js در برابر مرورگر

امکان	مرورگر	Node.js
`window`، `document`، `alert`، DOM	دارد	**ندارد**
`process`، `require`، `fs`، `http`	ندارد	دارد
`globalThis`	دارد (همان `window`)	دارد (همان `global`)
`console.log`	در کنسول DevTools	در ترمینال
`setTimeout`، `Promise`، `JSON`، `Math`، `fetch`	دارد	دارد
`import` / `export`	دارد (`<script type="module">`)	دارد (ESM)

JavaScript

```
// A CommonJS file running in Node.js
console.log(typeof window);         // "undefined"
console.log(globalThis === global); // true
console.log(typeof setTimeout);     // "function"
console.log(this);                  // {}
```

نکته

- **JavaScript** همان زبان است؛ چیزی که بین مرورگر و Node.js فرق می‌کند **ابزارهای محیط** است. قبل از حل هر تست، اول ببین کد در کدام محیط اجرا می‌شود.
- `typeof` روی چیزی که وجود ندارد خطا نمی‌دهد و `"undefined"` برمی‌گرداند؛ این ترفند معروف برای فهمیدن محیط است: `typeof window === "undefined"` یعنی احتمالاً داخل Node.js هستیم.

دام تستی

`emit` در `EventEmitter` **همگام** است: شنونده‌ها همان لحظه اجرا می‌شوند، نه «بعداً» مثل `setTimeout`. پس در تست‌های ترتیب خروجی، خروجی شنونده را درست در جای خط `emit` بنویس.

جمع‌بندی

- اجرای فایل با `node app.js`، REPL با `node` خالی، آرگومان‌ها در `process.argv`.
- هشت ویژگی اسلاید ۶: Fast، No Buffering، Highly Scalable، Single-Threaded، Event-Driven، Asynchronous، Cross-Platform Compatibility، JavaScript، Data Streaming.
- اسلاید ۷: TypeScript، ES Modules، async/await و ابزارهای مدرن، فریم‌ورک‌های Express و NestJS و Fastify، container و cloud و edge.
- میم اسلاید ۷: برنامه‌ی ۳۰۰ کیلوبایتی و `node_modules` دوازده‌گیگابایتی، یعنی پف کردن وابستگی‌ها.
- CommonJS (`require`، `module.exports`) در برابر ESM (`import`، `export`، `.mjs` یا `"type": "module"`).

تست‌های تمرینی

۲۰ تست · شماره‌ی ۱۰۱۱ تا ۱۰۳۰

۱۰۱۱ آسان

در نمودار تصویری اسلاید ۶ («Node.js Features») هشت ویژگی دور لوگوی Node.js نوشته شده است. کدام گزینه جزو این هشت ویژگی **نیست**؟

۱) No Buffering
۲) Multi-Threaded
۳) Cross-Platform Compatibility
۴) Highly Scalable

۱۰۱۲ آسان

منظور از ویژگی **No Buffering** در Node.js چیست؟

۱) Node.js اصلاً از حافظه‌ی RAM استفاده نمی‌کند.
۲) Node.js نمی‌تواند فایل‌های بزرگ را بخواند.
۳) داده‌ای که خوانده می‌شود هرگز در هیچ متغیری نگه داشته نمی‌شود.
۴) برنامه‌ها داده را تکه‌تکه (chunk) و به‌محض آماده شدن خروجی می‌دهند، نه اینکه اول همه‌ی داده را در حافظه جمع کنند.

۱۰۱۳ متوسط

در کد زیر `highWaterMark: 4` یعنی اندازه‌ی هر تکه (chunk) حداکثر ۴ بایت است (اینجا ۴ حرف). خروجی کد در Node.js کدام است؟

```JavaScript
const fs = require("fs");
fs.writeFileSync("data.txt", "abcdefghij");

const stream = fs.createReadStream("data.txt", { encoding: "utf8", highWaterMark: 4 });
stream.on("data", (chunk) => console.log("chunk:", chunk));
stream.on("end", () => console.log("done"));
console.log("reading...");
```

۱) `reading...` `chunk: abcd` `chunk: efgh` `chunk: ij` `done`
۲) `reading...` `chunk: abcdefghij` `done`
۳) `chunk: abcd` `chunk: efgh` `chunk: ij` `done` `reading...`
۴) `chunk: abcdefghij` `reading...` `done`

۱۰۱۴ سخت

خروجی کد زیر در Node.js کدام است؟

```javascript
const EventEmitter = require("events");
const shop = new EventEmitter();

shop.on("order", (item) => console.log(`Preparing ${item}`));
console.log("Waiting...");
shop.emit("order", "pizza");
shop.emit("cancel", "pizza");
console.log("Done");
```

۱) `Waiting...` `Done` `Preparing pizza`

۲) `Preparing pizza` `Waiting...` `Done`

۳) `Waiting...` `Preparing pizza` `Done`

۴) `Waiting...` `Preparing pizza` `Uncaught Error`

۱۰۱۵ آسان

ویژگی **Cross-Platform Compatibility** در نمودار اسلاید ۶ یعنی چه؟

۱) همان کد Node.js را می‌توان روی سیستم‌عامل‌های مختلف مثل Windows، Linux و macOS اجرا کرد.

۲) هر برنامه‌ی Node.js هم‌زمان در مرورگر کاربر و روی سرور اجرا می‌شود.

۳) برنامه‌ی Node.js را باید برای هر سیستم‌عامل از نو نوشت.

۴) Node.js کد زبان‌های دیگر مثل Python و Java را هم اجرا می‌کند.

۱۰۱۶ متوسط

خروجی کد زیر در Node.js کدام است؟

```javascript
const fs = require("fs/promises");

async function main() {
  await fs.writeFile("note.txt", "Hello file");
  const text = await fs.readFile("note.txt", "utf8");
  console.log(text);
}

main();
console.log("main started");
```

۱) `Hello file` `main started`

۲) `main started`

۳) `Hello file`

۴) `main started` `Hello file`

۱۰۱۷ آسان

فایلی به نام `app.js` داریم. با کدام دستور در ترمینال آن را با Node.js اجرا می‌کنیم؟

۱) `npm app.js`
۲) `run app.js`
۳) `node app.js`
۴) `js app.js`

۱۰۱۸ متوسط

اگر در ترمینال فقط دستور `node` را (بدون نام فایل) بزنیم چه اتفاقی می‌افتد؟

۱) نسخه‌ی نصب‌شده‌ی Node.js چاپ می‌شود و برنامه تمام می‌شود.
۲) محیط تعاملی REPL باز می‌شود که هر عبارت تایپ‌شده را می‌خواند، ارزیابی می‌کند و نتیجه را چاپ می‌کند.
۳) یک فایل `package.json` جدید در پوشه‌ی فعلی ساخته می‌شود.
۴) خطا می‌دهد، چون Node.js همیشه به نام یک فایل نیاز دارد.

۱۰۱۹ سخت

با اجرای `node app.js`، خروجی کدام است؟

math.js

```
console.log("math.js loaded");
function add(a, b) {
  return a + b;
}
module.exports = { add, PI: 3.14 };
```

app.js

```
const math = require("./math");
const { add } = require("./math.js");
console.log(math.add(2, 3), add(1, 1), math.PI);
```

۱) `math.js loaded` `math.js loaded` `5 2 3.14`
۲) `5 2 3.14`
۳) `math.js loaded` `5 2 3.14`
۴) `Uncaught Error`

۱۰۲۰ سخت

با اجرای `node app.js`، خروجی کدام است؟

bad.js

```
exports = { hello: "hi" };
```

good.js

```
exports.hello = "hi";
```

app.js

```
const bad = require("./bad");
const good = require("./good");
console.log(bad);
console.log(good);
```

۱) `{}` `{ hello: 'hi' }`

۲) `{ hello: 'hi' }` `{ hello: 'hi' }`

۳) `{ hello: 'hi' }` `{}`

۴) `undefined` `{ hello: 'hi' }`

۱۰۲۱ متوسط

با اجرای `node app.js` در پوشه‌ای که این دو فایل در آن هستند، چه اتفاقی می‌افتد؟

package.json

```
{
  "name": "demo",
  "type": "module"
}
```

app.js

```
const fs = require("fs");
console.log("file system ready");
```

۱) `file system ready`

۲) `Uncaught SyntaxError`

۳) `Uncaught TypeError`

۴) `Uncaught ReferenceError`

۱۰۲۲ سخت

پروژه را با `npm init -y` ساخته‌ایم (نسخه‌های جدید npm فیلد `"type": "commonjs"` را هم در `package.json` می‌نویسند). با اجرای `node index.js` چه اتفاقی می‌افتد؟

```package.json
{
  "name": "demo",
  "version": "1.0.0",
  "type": "commonjs"
}
```

```index.js
import fs from "fs";
console.log("file system ready");
```

۱) `file system ready`

۲) `Uncaught SyntaxError`

۳) `Uncaught ReferenceError`

۴) `Uncaught TypeError`

۱۰۲۳ متوسط

با کدام کار، Node.js یک فایل را به‌عنوان **ES Module** (با پشتیبانی از `import` و `export`) اجرا می‌کند؟

۱) گذاشتن پسوند `.cjs` برای فایل

۲) نوشتن `"type": "commonjs"` در `package.json`

۳) گذاشتن پسوند `.mjs` برای فایل، یا نوشتن `"type": "module"` در `package.json` برای فایل‌های `.js`

۴) نوشتن `"main": "index.js"` در `package.json`

۱۰۲۴ متوسط

فایل زیر با پسوند `.mjs` (یعنی به‌صورت ES Module) در Node.js اجرا می‌شود. خروجی کدام است؟

```JavaScript
console.log(typeof require);
console.log(this);
console.log(typeof setTimeout);
```

۱) `undefined` `undefined` `function`

۲) `function` `{}` `function`

۳) `undefined` `{}` `undefined`

۴) `Uncaught ReferenceError`

۱۰۲۵ متوسط

در میم اسلاید ۷ با عنوان «JavaScript programming»، روی مرد نوشته شده «app 300kb» و روی کوله‌پشتی غول‌پیکرش «Node_modules, 12GB». این میم به کدام واقعیت اشاره دارد؟

۱) برنامه‌های Node.js همیشه دقیقاً ۱۲ گیگابایت حجم دارند.

۲) پوشه‌ی `node_modules` کد اصلی برنامه‌ی خود ما را نگه می‌دارد.

۳) Node.js اجازه نمی‌دهد حجم کد برنامه از ۳۰۰ کیلوبایت بیشتر شود.

۴) کد خود برنامه کوچک است، ولی وابستگی‌ها و وابستگی‌های آن‌ها در `node_modules` حجم بسیار بزرگی پیدا می‌کنند.

۱۰۲۶ متوسط

اسلاید ۷ می‌گوید Node.js از فریم‌ورک‌های قدرتمندی پشتیبانی می‌کند. کدام گزینه جزو فریم‌ورک‌های سمت سروری که این اسلاید نام می‌برد **نیست**؟

۱) Express ۲) React ۳) NestJS ۴) Fastify

۱۰۲۷ سخت

کدام عبارت درباره‌ی تفاوت Node.js و مرورگر **نادرست** است؟

۱) `globalThis` هم در مرورگر و هم در Node.js وجود دارد.

۲) در Node.js با ماژول `fs` می‌توان فایل‌های روی دیسک را با داشتن مسیرشان خواند؛ کاری که کد یک صفحه‌ی وب اجازه‌اش را ندارد.

۳) `setTimeout` و `Promise` مخصوص مرورگرند و در Node.js وجود ندارند.

۴) در Node.js عبارت `typeof window` برابر `"undefined"` است.

۱۰۲۸ سخت

اسلاید ۷ می‌گوید «Node.js Embraces TypeScript … by default». در **Node.js نسخه‌ی ۲۴** فایل `app.ts` زیر را مستقیم با دستور `node app.ts` اجرا می‌کنیم. چه اتفاقی می‌افتد؟

app.ts

```
let count: number = "ten";
console.log(count);
```

۱) برنامه اجرا نمی‌شود و خطای نوع (type error) می‌گیریم، چون `"ten"` از نوع `number` نیست.

۲) خطای `SyntaxError` می‌گیریم، چون Node.js نوشتن نوع (`: number`) را نمی‌شناسد.

۳) `NaN` چاپ می‌شود، چون `"ten"` به عدد تبدیل می‌شود.

۴) `ten` چاپ می‌شود؛ Node.js فقط نوع‌ها را پاک می‌کند و آن‌ها را بررسی نمی‌کند.

۱۰۲۹ آسان

در سیستم ماژول **CommonJS** (سیستم قدیمی و پیش‌فرض Node.js)، برای وارد کردن یک ماژول از چه چیزی و برای صادر کردن از چه چیزی استفاده می‌کنیم؟

۱) `require` برای وارد کردن و `module.exports` برای صادر کردن

۲) `import` برای وارد کردن و `export default` برای صادر کردن

۳) `include` برای وارد کردن و `module.import` برای صادر کردن

۴) `fetch` برای وارد کردن و `window.exports` برای صادر کردن

۱۰۳۰ متوسط

Node.js با اینکه **تک‌نخی** (single-threaded) است، چرا **Highly Scalable** شمرده می‌شود؟

۱) چون برای هر درخواست یک thread جدید می‌سازد و کارها را بین آن‌ها پخش می‌کند.

۲) چون I/O غیرمسدودکننده و رویدادمحور است و آن یک thread هیچ‌وقت بیکار منتظر فایل یا شبکه نمی‌ماند؛ پس به اتصال‌های هم‌زمان زیادی می‌رسد.

۳) چون کد سرور را در مرورگر هر کاربر اجرا می‌کند تا بار سرور کم شود.

۴) چون همه‌ی داده‌ها را اول کامل در حافظه buffer می‌کند و بعد یکجا می‌فرستد.

۳-۳ پکیج‌های خارجی و npm

منبع: NodeJS & TS، اسلایدهای ۸ و ۹

درسنامه

فرض کن می‌خواهی یک خانه‌ی اسباب‌بازی بسازی. می‌توانی تک‌تک آجرها را خودت از پلاستیک بتراشی، یا یک جعبه **LEGO** بخری و قطعه‌های آماده را کنار هم بگذاری. برنامه‌نویسی امروز بیشتر شبیه حالت دوم است: به‌جای اینکه هر قابلیتی را از صفر بنویسیم (به قول معروف «چرخ را از نو اختراع کنیم»)، از کدهای آماده‌ای استفاده می‌کنیم که دیگران نوشته و منتشر کرده‌اند. این درس درباره‌ی همین قطعه‌های آماده (**پکیج**) و ابزاری است که آن‌ها را مدیریت می‌کند (**npm**).

چند تعریف پایه

- **package** (پکیج یا بسته): یک پوشه کد آماده و قابل استفاده‌ی دوباره که یک فایل `package.json` دارد و خودش را در آن معرفی می‌کند (اسم، نسخه و...). پکیج‌ها معمولاً در مخزن npm منتشر می‌شوند تا هر کسی بتواند نصبشان کند.
- **library** (کتابخانه): مجموعه‌ای از توابع آماده برای یک کار مشخص، مثل کار با تاریخ یا فرستادن درخواست HTTP. بیشتر کتابخانه‌های JavaScript به شکل پکیج npm منتشر می‌شوند.
- **module** در برابر package: یک ماژول معمولاً **یک فایل** است؛ یک پکیج می‌تواند **چندین** ماژول (فایل) داشته باشد.
- **dependency** (وابستگی): پکیجی که پروژه برای کار کردن به آن نیاز دارد.
  - **direct dependency** (وابستگی مستقیم): پکیجی که خودت نصب کرده‌ای و اسمش در `package.json` پروژه‌ات آمده است.
  - **transitive dependency** (وابستگی غیرمستقیم): پکیجی که یکی از وابستگی‌های تو به آن نیاز دارد. npm آن را خودکار نصب می‌کند، ولی در `package.json` تو نوشته **نمی‌شود**.

یک مثال واقعی: پکیج کوچک `is-odd` (بررسی فرد بودن یک عدد) خودش از پکیج `is-number` استفاده می‌کند. در یک پروژه‌ی تازه فقط `is-odd` را نصب می‌کنیم:

```
Terminal
```

```
npm install is-odd
# added 2 packages in 311ms
npm ls --all
# my-app@1.0.0 /home/ali/my-app
# └─┬ is-odd@3.0.1
#   └── is-number@6.0.0
```

ما یک پکیج خواستیم، ولی **دو** پکیج نصب شد. در `package.json` فقط `is-odd` آمده است، ولی در پوشه‌ی `node_modules` هر دو پکیج هستند. این درخت وابستگی‌ها در پروژه‌های واقعی می‌تواند صدها شاخه داشته باشد؛ همان کوله‌پشتی ۱۲ گیگابایتی میم اسلاید ۷!

پکیج‌های خارجی (اسلاید ۸)

اسلاید ۸ داستان پکیج‌های خارجی (external packages: کدی که دیگران نوشته‌اند و ما در پروژه‌مان استفاده می‌کنیم) را در پنج جمله می‌گوید:

۱. **Open-source community began sharing reusable JS libraries.** جامعه‌ی **open-source** (متن‌باز: نرم‌افزاری که کدش عمومی است و هر کسی می‌تواند از آن استفاده کند و بهترش کند) شروع کرد به اشتراک گذاشتن کتابخانه‌های **قابل استفاده‌ی دوباره**

(reusable).

۲. **Developers could import external code instead of reinventing features.** برنامه‌نویس‌ها به‌جای نوشتن دوباره‌ی قابلیت‌های تکراری (مثل قالب‌بندی تاریخ)، کد آماده را **import** (وارد) می‌کنند.

۳. **Enabled rapid development through modular building blocks.** با کنار هم گذاشتن **قطعه‌های ماژولار** (modular building blocks)، برنامه خیلی **سریع‌تر** ساخته می‌شود؛ درست مثل LEGO.

۴. **Popular libraries shaped best practices and common patterns.** کتابخانه‌های محبوب روی **بهترین شیوه‌ها** (best practices) و الگوهای رایج اثر گذاشتند. مثلاً کار با Promise سال‌ها قبل از اینکه رسماً وارد زبان شود، در کتابخانه‌ها رایج شد؛ یا الگوی «کامپوننت» را React همه‌گیر کرد.

۵. **Also raised issues of trust, maintenance, and dependency overload.** این راحتی هزینه هم داشت:

- **trust** (اعتماد): کد یک غریبه را وارد پروژه‌ات می‌کنی؛ از کجا مطمئنی کار مخربی نمی‌کند؟
- **maintenance** (نگه‌داری): اگر نویسنده‌ی پکیج آن را رها کند، باگ‌ها و مشکلات امنیتی‌اش دیگر درست نمی‌شوند.
- **dependency overload** (بار اضافه‌ی وابستگی‌ها): وابستگی‌های زیاد یعنی `node_modules` حجیم، نصب کُند و نقاط آسیب‌پذیر بیشتر.

مزیت‌ها (اسلاید ۸)	دردسرها (اسلاید ۸)
استفاده‌ی دوباره از کد (reusable)	اعتماد (trust)
اختراع نکردن دوباره‌ی چرخ	نگه‌داری (maintenance)
توسعه‌ی سریع با قطعه‌های ماژولار	بار اضافه‌ی وابستگی‌ها (dependency overload)
شکل گرفتن بهترین شیوه‌ها و الگوها	

> **ماجرای left-pad**
>
> در سال ۲۰۱۶ نویسنده‌ی پکیج کوچکی به نام `left-pad` (حدود ۱۱ خط کد برای اضافه کردن فاصله به ابتدای یک رشته) آن را از npm حذف کرد. چون هزاران پروژه به‌طور مستقیم یا غیرمستقیم به آن وابسته بودند، ساختِ (build) خیلی از آن‌ها، از جمله ابزارهای معروف، ناگهان خراب شد. این ماجرا نمونه‌ی واقعی «dependency overload» و خطر وابستگی‌های غیرمستقیم است.

npm (اسلاید ۹)

npm مخفف **N**ode **P**ackage **M**anager (مدیر بسته‌ی Node) است و همراه Node.js نصب می‌شود. npm در واقع سه بخش دارد:

- **registry** (مخزن): یک پایگاه داده‌ی آنلاین عظیم از پکیج‌ها که آدرس پیش‌فرضش `https://registry.npmjs.org/` است.
- **CLI** (رابط خط فرمان): همان دستور `npm` در ترمینال که با آن پکیج نصب و حذف می‌کنیم.
- **وب‌سایت** `npmjs.com` : برای جست‌وجو و خواندن مستندات پکیج‌ها.

اسلاید ۹ پنج نکته درباره‌ی npm می‌گوید:

۱. **Introduced a central registry to manage and distribute packages.** یک **مخزن مرکزی** (central registry) برای مدیریت و توزیع پکیج‌ها ساخت؛ همه پکیج‌ها را در یک جا منتشر می‌کنند و از همان‌جا دانلود می‌کنند.

۲. **Made installing and updating dependencies effortless.** نصب و به‌روزرسانی وابستگی‌ها را **بی‌زحمت** کرد: یک دستور `npm install` به‌جای دانلود دستی فایل‌ها.

۳. **Standardized project setup with package.json.** ساختار پروژه را با فایل `package.json` **استاندارد** کرد؛ هر پروژه‌ی Node.js یک شناسنامه‌ی یکسان دارد.

۴. **Fueled explosive growth of the JavaScript ecosystem.** باعث رشد **انفجاری** اکوسیستم JavaScript شد؛ امروز میلیون‌ها پکیج در npm هست.

۵. **Later faced challenges with security, size, and version chaos.** بعدها با چالش‌های **امنیت** (پکیج‌های مخرب یا آسیب‌پذیر)، **حجم** (`node_modules` غول‌آسا) و **آشفتگی نسخه‌ها** (version chaos: ناسازگاری نسخه‌های مختلف یک پکیج با هم) روبه‌رو شد.

سه چیز مهم در هر پروژه

۱. `package.json` : شناسنامه‌ی پروژه

اسم و نسخه‌ی پروژه، اسکریپت‌ها و فهرست وابستگی‌های مستقیم در این فایل است (جزئیاتش را در درس «اسکریپت‌های npm و فایل package.json» می‌خوانیم):

package.json

```
{
  "name": "my-app",
  "version": "1.0.0",
  "dependencies": {
    "express": "^5.3.0"
  },
  "devDependencies": {
    "jest": "^30.5.2"
  }
}
```

۲. `package-lock.json` : قفلِ نسخه‌های دقیق

در `package.json` معمولاً یک **بازه** از نسخه‌ها نوشته می‌شود (مثل `^3.0.1`). ولی `package-lock.json` نسخه‌ی **دقیقِ** همه‌ی پکیج‌های نصب‌شده را ثبت می‌کند، حتی وابستگی‌های غیرمستقیم، به‌اضافه‌ی آدرس دانلود (`resolved`) و یک کد بررسی سلامت فایل (`integrity`). npm این فایل را خودکار می‌سازد و به‌روز نگه می‌دارد. این تکه از lock فایل همان مثال بالاست:

package-lock.json

```
"node_modules/is-odd": {
  "version": "3.0.1",
  "resolved": "https://registry.npmjs.org/is-odd/-/is-odd-3.0.1.tgz",
  "integrity": "sha512-CQpnWPrDwmP1+SMHXZhtLtJv90yiyVfluGsX5iNC...",
  "dependencies": { "is-number": "^6.0.0" }
}
```

(مقدار `integrity` اینجا کوتاه شده است.)

نتیجه: هر کسی که پروژه را بگیرد و `npm install` بزند، **دقیقاً همان نسخه‌ها** را می‌گیرد و جمله‌ی معروف «روی سیستم من کار می‌کرد!» کمتر شنیده می‌شود.

۳. پوشه‌ی `node_modules`

کدِ واقعی همه‌ی پکیج‌های نصب‌شده (مستقیم و غیرمستقیم) اینجاست. این پوشه معمولاً خیلی بزرگ است و چون همیشه می‌شود آن را با `npm install` از روی دو فایل بالا **دوباره ساخت**، در git ثبت (commit) نمی‌شود و اسمش را در فایل `.gitignore` می‌نویسند.

	در git ثبت شود؟	چه کسی می‌سازدش؟
`package.json`	بله	ما (و npm هنگام نصب و حذف پکیج)
`package-lock.json`	بله	npm، خودکار
`node_modules/`	**خیر**	npm، هنگام `npm install`

نسخه‌بندی معنایی (Semantic Versioning)

نسخه‌ی پکیج‌های npm سه عدد دارد: **MAJOR.MINOR.PATCH**؛ مثلاً `4.17.21` . به این قرارداد **SemVer** (Semantic Versioning: نسخه‌بندی معنایی) می‌گویند، چون هر عدد معنی دارد:

بخش	کی زیاد می‌شود؟	مثال از `4.17.21`
PATCH (وصله)	فقط رفع باگ، بدون تغییر در نحوه‌ی استفاده	`4.17.22`
MINOR (جزئی)	قابلیت جدید که با کدهای قبلی **سازگار** است	`4.18.0` (PATCH صفر می‌شود)
MAJOR (اصلی)	**تغییر ناسازگار** (breaking change): ممکن است کد قبلی تو را خراب کند	`5.0.0` (بقیه صفر می‌شوند)

در `package.json` می‌شود به‌جای یک نسخه‌ی ثابت، یک **بازه** نوشت:

نوشته در `package.json`	یعنی	مجاز	غیرمجاز
`"1.2.3"`	دقیقاً همین نسخه	`1.2.3`	`1.2.4`
`"~1.2.3"`	فقط PATCH جدیدتر: از `1.2.3` تا قبل از `1.3.0`	`1.2.9`	`1.3.0`
`"^1.2.3"`	MINOR و PATCH جدیدتر: از `1.2.3` تا قبل از `2.0.0`	`1.9.0`	`2.0.0`
`"^0.2.3"`	برای نسخه‌های `0.x` محتاط‌تر: از `0.2.3` تا قبل از `0.3.0`	`0.2.9`	`0.3.0`
`"*"`	هر نسخه‌ای	`9.9.9`	—

وقتی `npm install <package>` می‌زنی، npm به‌طور پیش‌فرض آخرین نسخه را نصب می‌کند و آن را با علامت `^` در `package.json` ذخیره می‌کند (مثلاً `"is-number": "^7.0.0"`). اگر بعداً پروژه را بدون lock فایل نصب کنی، npm **جدیدترین نسخه‌ای که در بازه جا می‌شود** را برمی‌دارد.

dependencies در برابر devDependencies

- **dependencies**: پکیج‌هایی که برنامه **هنگام اجرا** لازم دارد؛ مثل `express` (سرور)، `axios` (درخواست HTTP) یا `react` (رابط کاربری).
- **devDependencies**: پکیج‌هایی که فقط **هنگام توسعه** لازم‌اند و برنامه‌ی در حال اجرا به آن‌ها نیاز ندارد؛ مثل `jest` (تست)، `webpack` (bundler)، `typescript` (کامپایلر TypeScript). با `npm install -D <package>` نصب می‌شوند.

روی سرور اصلی (محیط production) می‌شود با `npm install --omit=dev` فقط dependencies را نصب کرد تا حجم کمتر شود.

خطرهای زنجیره‌ی تأمین (supply chain)

supply chain (زنجیره‌ی تأمین نرم‌افزار) یعنی همه‌ی کدهایی که از بیرون وارد پروژه‌ات می‌شود، به‌اضافه‌ی آدم‌ها و ابزارهایی که آن کد را می‌سازند و منتشر می‌کنند. چند خطر رایج:

- **typosquatting** (سوءاستفاده از اشتباه تایپی): کسی پکیج مخربی با اسمی شبیه یک پکیج معروف منتشر می‌کند (مثلاً `expresss` به‌جای `express`) و منتظر اشتباه تایپی تو می‌ماند.
- **حساب نفوذشده**: اگر حساب کاربری نویسنده‌ی یک پکیج محبوب هک شود، نسخه‌ی جدید و مخربی منتشر می‌شود و با یک `npm update` وارد هزاران پروژه می‌شود.
- **اسکریپت‌های نصب**: بعضی پکیج‌ها هنگام نصب، خودکار کد اجرا می‌کنند.
- **پکیج رهاشده**: پکیجی که دیگر نگه‌داری نمی‌شود و آسیب‌پذیری‌هایش درست نمی‌شود.

راه‌های کم کردن خطر: قبل از نصب، اسم، محبوبیت و فعال بودن پکیج را بررسی کن؛ وابستگی‌ها را کم نگه دار؛ `package-lock.json` را commit کن؛ و با `npm audit` آسیب‌پذیری‌های شناخته‌شده را پیدا کن.

> **نکته**
>
> `package.json` و `package-lock.json` را در git ثبت کن، ولی `node_modules` را نه. هم‌تیمی تو بعد از گرفتن پروژه فقط یک `npm install` می‌زند و دقیقاً همان نسخه‌ها را می‌گیرد.

> **دام تستی**
>
> - علامت `^` اجازه‌ی رفتن به MAJOR بعدی را **نمی‌دهد**: `^1.2.3` شامل `2.0.0` نیست. علامت `~` از این هم سخت‌گیرتر است و فقط PATCH را باز می‌گذارد: `~1.2.3` شامل `1.3.0` نیست.
> - در نسخه‌های `0.x` علامت `^` مثل `~` رفتار می‌کند: `^0.2.3` شامل `0.3.0` نیست.

> **دام تستی**
>
> وابستگی غیرمستقیم در `package.json` تو نوشته **نمی‌شود**، ولی در `node_modules` و `package-lock.json` حضور دارد. پس «هر پکیجی که در `node_modules` هست، در `package.json` هم آمده» غلط است.

> **جمع‌بندی**
>
> - پکیج = کد آماده‌ی قابل استفاده‌ی دوباره؛ وابستگی‌ها مستقیم یا غیرمستقیم‌اند.
> - اسلاید ۸: اشتراک کتابخانه‌های متن‌باز، اختراع نکردن دوباره‌ی چرخ، توسعه‌ی سریع با قطعه‌های ماژولار، شکل گرفتن بهترین شیوه‌ها؛ و در مقابل: اعتماد، نگه‌داری، بار اضافه‌ی وابستگی‌ها.
> - اسلاید ۹: npm (Node Package Manager) = مخزن مرکزی، نصب و به‌روزرسانی آسان، استاندارد شدن با `package.json`، رشد انفجاری؛ و بعدها چالش امنیت، حجم و آشفتگی نسخه‌ها.
> - `package-lock.json` نسخه‌های دقیق را قفل می‌کند؛ `node_modules` ساختنی است و commit نمی‌شود.
> - SemVer: MAJOR.MINOR.PATCH؛ `^` تا قبل از MAJOR بعدی، `~` تا قبل از MINOR بعدی.

تست‌های تمرینی

۱۸ تست · شماره‌ی ۱۰۳۱ تا ۱۰۴۸

۱۰۳۱ آسان

npm مخفف چیست؟

۱) New Programming Module
۲) Node Package Manager
۳) Node Project Maker
۴) Network Package Monitor

۱۰۳۲ آسان

طبق اسلاید ۸، کدام مورد از **مزیت‌های** استفاده از پکیج‌های خارجی **نیست**؟

۱) استفاده از کد آماده‌ی دیگران به‌جای اختراع دوباره‌ی همان قابلیت‌ها
۲) توسعه‌ی سریع‌تر با کنار هم گذاشتن قطعه‌های ماژولار (modular building blocks)
۳) شکل گرفتن بهترین شیوه‌ها و الگوهای رایج به کمک کتابخانه‌های محبوب
۴) تضمین اینکه پروژه هیچ مشکل امنیتی و نگه‌داری نخواهد داشت

۱۰۳۳ آسان

کدِ پکیج‌هایی که با `npm install` دانلود می‌شوند، در کجای پروژه قرار می‌گیرد؟

۱) پوشه‌ی `node_modules`
۲) فایل `package.json`
۳) فایل `package-lock.json`
۴) فایل `.npmrc`

۱۰۳۴ متوسط

نقش فایل `package-lock.json` چیست؟

۱) فهرست اسکریپت‌های پروژه را نگه می‌دارد.
۲) یک نسخه‌ی پشتیبان از `package.json` است که فقط برای بازیابی استفاده می‌شود.
۳) نسخه‌ی دقیق همه‌ی پکیج‌های نصب‌شده (حتی وابستگی‌های غیرمستقیم) و محل دانلودشان را ثبت می‌کند تا نصب در همه‌ی سیستم‌ها یکسان باشد.
۴) پروژه را قفل می‌کند تا دیگر نتوان پکیج جدیدی نصب کرد.

۱۰۳۵ متوسط

در `package.json` نوشته شده `"lodash": "^4.17.20"` . npm هنگام نصب، مجاز به نصب کدام نسخه است؟

۱) `5.0.0`
۲) `4.16.0`
۳) `4.17.19`
۴) `4.18.1`

۱۰۳۶ متوسط

بازه‌ی `"~2.3.1"` در `package.json` شامل کدام نسخه می‌شود؟

۱) `2.3.9`
۲) `2.4.0`
۳) `3.0.0`
۴) `2.3.0`

۱۰۳۷ سخت

بازه‌ی `"^0.4.2"` در `package.json` شامل کدام نسخه می‌شود؟

۱) `0.4.9`
۲) `0.5.0`
۳) `1.0.0`
۴) `0.4.1`

۱۰۳۸ متوسط

نسخه‌ی یک کتابخانه از `3.4.2` به `4.0.0` رسیده است. طبق نسخه‌بندی معنایی (SemVer) این یعنی چه؟

۱) فقط یک باگ رفع شده و هیچ تغییر دیگری نیست.

۲) تغییر ناسازگار (breaking change) رخ داده و ممکن است کدی که با نسخه‌ی قبلی نوشته شده، خراب شود.

۳) قابلیت جدیدی اضافه شده که کاملاً با نسخه‌ی قبل سازگار است.

۴) عددهای نسخه معنای خاصی ندارند و تصادفی زیاد می‌شوند.

۱۰۳۹ متوسط

یک کتابخانه در نسخه‌ی `2.5.3` است. نویسنده فقط یک باگ را رفع کرده و هیچ قابلیت جدیدی اضافه نکرده است. طبق SemVer، نسخه‌ی بعدی کدام است؟

۱) `2.6.0`
۲) `3.0.0`
۳) `2.5.4`
۴) `2.5.3.1`

۱۰۴۰ سخت

در یک پروژه‌ی تازه فقط دستور `npm install is-odd` را اجرا کرده‌ایم. پکیج `is-odd` برای کار کردن به پکیج `is-number` نیاز دارد. کدام گزینه درست است؟

۱) npm خطا می‌دهد و می‌خواهد اول `is-number` را دستی نصب کنیم.

۲) `is-number` هم در `node_modules` نصب می‌شود و هم خودکار در بخش `dependencies` فایل `package.json` ما نوشته می‌شود.

۳) `is-number` نصب نمی‌شود و `is-odd` هر بار هنگام اجرا آن را از اینترنت دانلود می‌کند.

۴) `is-number` به‌عنوان وابستگی غیرمستقیم (transitive) خودکار در `node_modules` نصب می‌شود، ولی در `dependencies` فایل `package.json` ما نوشته نمی‌شود.

۱۰۴۱ متوسط

در یک پروژه‌ی سرور Node.js، کدام پکیج معمولاً در **devDependencies** (و نه dependencies) قرار می‌گیرد؟

۱) `express` ؛ فریم‌ورکی که سرور با آن ساخته شده است

۲) `typescript` ؛ کامپایلری که کد TypeScript را پیش از اجرا به JavaScript تبدیل می‌کند

۳) `axios` ؛ کلاینت HTTP که سرور با آن به APIهای دیگر درخواست می‌فرستد

۴) `socket.io` ؛ برای ارتباط بلادرنگ سرور با کاربران

۱۰۴۲ سخت

کدام عبارت **نادرست** است؟

۱) پوشه‌ی `node_modules` باید حتماً در git ثبت (commit) شود، وگرنه هم‌تیمی‌ها نمی‌توانند پروژه را اجرا کنند.

۲) پوشه‌ی `node_modules` را همیشه می‌توان با `npm install` از روی `package.json` و `package-lock.json` دوباره ساخت.

۳) فایل `package-lock.json` باید در git ثبت شود تا همه دقیقاً همان نسخه‌ها را نصب کنند.

۴) پوشه‌ی `node_modules` علاوه بر وابستگی‌های مستقیم، وابستگی‌های غیرمستقیم را هم در خود دارد.

۱۰۴۳ متوسط

طبق اسلاید ۹، npm بعدها با کدام چالش‌ها روبه‌رو شد؟

۱) نبودِ یک مخزن مرکزی برای پکیج‌ها

۲) کُندی موتور V8 و پشتیبانی نکردن از async/await

۳) امنیت، حجم و آشفتگی نسخه‌ها (version chaos)

۴) اینکه فقط روی سیستم‌عامل Linux کار می‌کرد

۱۰۴۴ سخت

برنامه‌نویسی می‌خواست `express` را نصب کند، ولی اشتباهی تایپ کرد `npm install expresss` (با سه s). اگر کسی پکیجی با همین اسم اشتباه منتشر کرده باشد، چه خطری وجود دارد؟

۱) هیچ خطری ندارد؛ npm اسم اشتباه را خودکار به نزدیک‌ترین پکیج معروف یعنی `express` اصلاح می‌کند.

۲) ممکن است پکیج دیگری که شاید کد مخرب دارد نصب شود؛ به این ترفند typosquatting می‌گویند و یکی از خطرهای زنجیره‌ی تأمین است.

۳) npm فقط پکیج‌هایی را نصب می‌کند که از قبل در `package.json` نوشته شده‌اند، پس این دستور اصلاً اجرا نمی‌شود.

۴) خطری ندارد، چون npm قبل از انتشار هر پکیج کد آن را کامل از نظر امنیتی بررسی می‌کند.

۱۰۴۵ آسان

جای خالی را کامل کن:

«npm برای مدیریت و توزیع پکیج‌ها یک ..... مرکزی معرفی کرد.»

۱) کامپایلر (compiler)
۲) مرورگر (browser)
۳) bundler
۴) مخزن (registry)

۱۰۴۶ سخت

در `package.json` نوشته شده `"chalk": "^4.1.0"` و هنوز `package-lock.json` نداریم. در مخزن npm نسخه‌های `4.1.0`، `4.1.2`، `4.2.0` و `5.0.0` از این پکیج هست و آخرین نسخه‌ی منتشرشده (latest) هم `5.0.0` است. با `npm install` کدام نسخه نصب می‌شود؟

۱) `4.1.0`
۲) `4.1.2`
۳) `4.2.0`
۴) `5.0.0`

۱۰۴۷ آسان

کدام فایل «شناسنامه»ی یک پروژه‌ی Node.js است و نام، نسخه، اسکریپت‌ها و فهرست وابستگی‌های مستقیم پروژه در آن نوشته می‌شود؟

۱) `package.json`
۲) `package-lock.json`
۳) `node_modules`
۴) `.npmrc`

۱۰۴۸ متوسط

«نویسنده‌ی پکیجی که پروژه‌ات به آن وابسته است، کار روی آن را رها کرده و دیگر باگ‌ها و مشکلات امنیتی‌اش را رفع نمی‌کند.» این موقعیت نمونه‌ای از کدام دردسرِ اسلاید ۸ است؟

۱) trust (اعتماد)
۲) maintenance (نگه‌داری)
۳) dependency overload (بار اضافه‌ی وابستگی‌ها)
۴) rapid development (توسعه‌ی سریع)

۳-۴ دستورات npm و تنظیم mirror

منبع: NodeJS & TS، اسلاید ۱۰

درسنامه

در درس قبل دیدیم npm چیست. حالا با دستورهایش کار می‌کنیم. همه‌ی این دستورها را در **ترمینال** و داخل **پوشه‌ی پروژه** اجرا می‌کنیم. اسلاید ۱۰ پنج دستور اصلی و دو دستور برای تنظیم mirror را نشان می‌دهد:

دستور (اسلاید ۱۰)	توضیح اسلاید	یعنی...
`npm init`	Create a new project and generate a package.json file	پروژه‌ی جدید بساز و `package.json` تولید کن
`npm install <package>`	Add a dependency to your project	یک وابستگی به پروژه اضافه کن
`npm uninstall <package>`	Remove a previously installed package	پکیجی را که قبلاً نصب شده حذف کن
`npm update`	Update all dependencies to newer versions	همه‌ی وابستگی‌ها را به نسخه‌های جدیدتر به‌روز کن
`npm run <script>`	Execute custom scripts defined in package.json	اسکریپت‌های تعریف‌شده در `package.json` را اجرا کن
`npm config set registry <url>`	Setting npm mirror (or set it in .npmrc file)	آدرس mirror را تنظیم کن
`npm config get registry`	Setting npm mirror	آدرس registry فعلی را نشان بده

چیزهایی مثل `<package>`، `<script>` و `<url>` **جای‌نگهدار** (placeholder) هستند؛ یعنی به جایشان اسم واقعی می‌نویسی، مثلاً `npm install express` .

`npm init` : ساختن پروژه

`npm init` چند سؤال می‌پرسد (اسم پکیج، نسخه، توضیح، فایل ورودی، دستور تست، مخزن git، کلمه‌های کلیدی، نویسنده، مجوز و نوع ماژول) و بعد فایل `package.json` را می‌سازد. جلوی هر سؤال، جواب پیش‌فرض داخل پرانتز آمده است:

```
package name: (my-app)
version: (1.0.0)
description:
entry point: (index.js)
test command:
git repository:
keywords:
author:
license: (ISC)
type: (commonjs)
```

اگر حوصله‌ی سؤال و جواب نداری، از `npm init -y` (یا `--yes`) استفاده کن تا **همه‌ی جواب‌های پیش‌فرض** را بپذیرد. نتیجه در پوشه‌ای به اسم `my-app` :

package.json

```
{
  "name": "my-app",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

- `name` به‌طور پیش‌فرض **اسم پوشه** است و `version` از `1.0.0` شروع می‌شود.
- `main` فایل ورودی پکیج است (پیش‌فرض: `index.js`).
- `scripts.test` یک اسکریپت نمونه است که فقط پیام خطا چاپ می‌کند (در درس بعد می‌بینیمش).
- `license` پیش‌فرض `ISC` است.
- نسخه‌های جدید npm فیلد `"type": "commonjs"` را هم می‌نویسند؛ یعنی فایل‌های `.js` به‌صورت CommonJS اجرا می‌شوند (درس «ویژگی‌های Node.js»).

`npm install` : نصب پکیج

`npm install <package>` (یا کوتاهش `npm i <package>`) این کارها را با هم انجام می‌دهد:

- **اول:** آخرین نسخه‌ی پکیج را از registry دانلود می‌کند و در `node_modules` می‌گذارد (همراه وابستگی‌هایش).
- **دوم:** اسم پکیج را با یک بازه‌ی `^` در بخش `dependencies` فایل `package.json` می‌نویسد.
- **سوم:** فایل `package-lock.json` را می‌سازد یا به‌روز می‌کند.

Terminal

```
npm install is-number
# added 1 package in 304ms
```

بعد از این دستور، فیلدهای قبلی `package.json` سر جایشان می‌مانند و این بخش تازه به آخرش اضافه می‌شود:

package.json

```
"dependencies": {
  "is-number": "^7.0.0"
}
```

حالت‌های دیگر `npm install` :

دستور	کار
`npm install` (بدون اسم)	**همه‌ی** وابستگی‌های نوشته‌شده در `package.json` (طبق `package-lock.json`) را نصب می‌کند؛ مثلاً بعد از گرفتن پروژه از git
`npm install express axios`	چند پکیج را با هم نصب می‌کند
`npm install lodash@4.17.21`	نسخه‌ی مشخصی را نصب می‌کند (ولی به شکل `"^4.17.21"` ذخیره می‌شود)
`npm install -E lodash@4.17.21`	`-E` یا `--save-exact`: نسخه را دقیق و بدون `^` ذخیره می‌کند (`"4.17.21"`)
`npm install -D jest`	`-D` یا `--save-dev`: پکیج را در `devDependencies` ثبت می‌کند
`npm install -g typescript`	`-g` یا `--global`: نصب **سراسری** روی سیستم؛ معمولاً برای ابزارهای خط فرمان (مثل دستور `tsc`)؛ به پروژه `package.json` دست نمی‌زند

`npm uninstall`: حذف پکیج

`npm uninstall <package>` پکیج را از `node_modules` پاک می‌کند و اسمش را از `package.json` (و `package-lock.json`) برمی‌دارد. اسم‌های کوتاه‌ترش `npm remove`، `npm rm` و `npm un` هستند.

```
Terminal
npm uninstall is-even
# removed 5 packages in 212ms   (the package and its own dependencies)
```

`npm update`: به‌روزرسانی

`npm update` هر پکیج را به **جدیدترین نسخه‌ای که در بازه‌ی `package.json` جا می‌شود** می‌رساند و `package-lock.json` را به‌روز می‌کند. یعنی با `^6.0.0`، هیچ‌وقت خودکار به `7.0.0` (MAJOR جدید) نمی‌رود. دستور `npm outdated` نشان می‌دهد کدام پکیج‌ها نسخه‌ی جدیدتری دارند:

```
Terminal
npm outdated
# Package    Current  Wanted  Latest  Location                Depended by
# is-number    6.0.0   6.0.0   7.0.0  node_modules/is-number  my-app
npm update
# up to date in 209ms
```

ستون `Wanted` بالاترین نسخه‌ی داخل بازه است و `Latest` آخرین نسخه‌ی منتشرشده. اینجا `npm update` کاری نکرد، چون `7.0.0` بیرون از بازه‌ی `^6.0.0` است. برای رفتن به MAJOR جدید باید آگاهانه `npm install is-number@7` بزنی.

`npm run <script>`

اسکریپت‌های بخش `scripts` فایل `package.json` را اجرا می‌کند؛ مثلاً `npm run build`. جزئیاتش را در درس بعد می‌خوانیم.

تنظیم mirror (Setting npm mirror)

به‌طور پیش‌فرض npm پکیج‌ها را از registry رسمی یعنی `https://registry.npmjs.org/` دانلود می‌کند. **mirror** (آینه) یک کپی از همین مخزن روی سرور دیگری است که همان پکیج‌ها را دارد. چرا از mirror استفاده می‌کنند؟

- **سرعت**: سروری که به تو نزدیک‌تر است، سریع‌تر جواب می‌دهد.
- **در دسترس بودن**: وقتی registry اصلی کُند است، قطع شده یا از شبکه‌ی تو به‌خوبی در دسترس نیست (مثلاً به‌خاطر محدودیت‌های دسترسی).
- **کنترل در شرکت‌ها**: بعضی شرکت‌ها registry داخلی خودشان را دارند.

اسلاید ۱۰ دو راه برای تنظیم آدرس registry نشان می‌دهد:

```
Terminal
```

```
npm config get registry
# https://registry.npmjs.org/          (the default)

npm config set registry https://registry.npmmirror.com/
npm config get registry
# https://registry.npmmirror.com/
```

دستور `npm config set registry <url>` در عمل یک خط به فایل تنظیمات کاربر (`~/.npmrc` در پوشه‌ی خانه) اضافه می‌کند:

```
~/.npmrc
```

```
registry=https://registry.npmmirror.com/
```

راه دوم (به قول اسلاید «or set it in .npmrc file») این است که همین خط را خودت در یک فایل `.npmrc` بنویسی؛ مثلاً در پوشه‌ی پروژه، کنار `package.json`، تا فقط برای همان پروژه اعمال شود. هر دو راه یک نتیجه دارند.

چند نکته‌ی تکمیلی:

- `npm config delete registry` تنظیم را پاک می‌کند و npm به registry پیش‌فرض برمی‌گردد.
- برای فقط **یک** دستور: `npm install express --registry=https://registry.npmmirror.com/`
- اگر چند جا تنظیم شده باشد، اولویت با این ترتیب است: گزینه‌ی خود دستور (`--registry=…`)، بعد `.npmrc` پروژه، بعد `~/.npmrc` کاربر، و در آخر مقدار پیش‌فرض.

npx : اجرا بدون نصب دائمی

بعضی پکیج‌ها یک **دستور خط فرمان** (CLI) دارند. `npx <command>` آن دستور را اجرا می‌کند؛ اگر پکیج در پروژه نصب نباشد، آن را موقتاً در cache دانلود می‌کند (اول اجازه می‌گیرد؛ `--yes` این سؤال را حذف می‌کند) و دیگر به پروژه یا سیستم اضافه‌اش نمی‌کند. مثلاً پکیج `semver` می‌گوید کدام نسخه‌ها در یک بازه جا می‌شوند:

```
Terminal
```

```
npx --yes semver -r "^1.2.3" 1.2.2 1.2.3 1.9.0 2.0.0
# 1.2.3
# 1.9.0
```

`npx` ابزارهای نصب‌شده در `node_modules/.bin` همان پروژه را هم اجرا می‌کند؛ مثلاً `npx jest` .

جمع‌بندی دستورها

می‌خواهم…	دستور
پروژه‌ی جدید بسازم	`npm init` یا `npm init -y`
پکیجی برای اجرای برنامه نصب کنم	`npm install <package>` یا `npm i <package>`
پکیجی فقط برای توسعه نصب کنم	`npm install -D <package>`
ابزاری را سراسری نصب کنم	`npm install -g <package>`
همه‌ی وابستگی‌های پروژه را نصب کنم	`npm install`
پکیجی را حذف کنم	`npm uninstall <package>`
پکیج‌ها را در محدوده‌ی بازه‌شان به‌روز کنم	`npm update`
یک اسکریپت را اجرا کنم	`npm run <script>`
آدرس mirror را تنظیم کنم یا ببینم	`npm config set registry <url>` و `npm config get registry`
دستور یک پکیج را بدون نصب دائمی اجرا کنم	`npx <command>`

نکته

اسم‌های کوتاه را بشناس: `npm i` یعنی `npm install`، `-D` یعنی `--save-dev`، `-g` یعنی `--global` و `-E` یعنی `--save-exact`. در تست‌ها معمولاً یکی از این‌ها را به‌جای دیگری می‌آورند.

دام تستی

- `npm install` **بدون اسم** بیکار نیست: همه‌ی وابستگی‌های `package.json` را نصب می‌کند.
- `npm install -g` به `package.json` پروژه **دست نمی‌زند**؛ نصب سراسری مال سیستم است، نه پروژه.
- `npm config get registry` فقط **می‌خواند** و چیزی را عوض نمی‌کند؛ تنظیم کردن کار `set` است.
- `npm update` از بازه‌ی `package.json` بیرون نمی‌رود؛ با `^` هرگز MAJOR جدید نصب نمی‌کند.

تست‌های تمرینی

۱۹ تست · شماره‌ی ۱۰۴۹ تا ۱۰۶۷

۱۰۴۹ آسان

طبق اسلاید ۱۰، کدام دستور یک پروژه‌ی جدید می‌سازد و فایل `package.json` را تولید می‌کند؟

۱) `npm init`
۲) `npm start`
۳) `npm update`
۴) `npm config get registry`

۱۰۵۰ آسان

در دستور `npm init -y`، گزینه‌ی `-y` چه کاری می‌کند؟

۱) همه‌ی وابستگی‌های پروژه را نصب می‌کند.

۲) فایل `package.json` قبلی را پاک می‌کند و پروژه را از نو می‌سازد.

۳) به همه‌ی سؤال‌ها جواب پیش‌فرض می‌دهد و `package.json` را بدون پرسیدن سؤال می‌سازد.

۴) پروژه را به‌جای npm با ابزار yarn می‌سازد.

۱۰۵۱ متوسط

در پوشه‌ای به نام `my-app` دستور `npm init -y` را اجرا می‌کنیم. کدام مورد در `package.json` ساخته‌شده درست است؟

۱) `"name": "my-project"`

۲) `"version": "1.0.0"`

۳) `"main": "app.js"`

۴) `"license": "MIT"`

۱۰۵۲ متوسط

در پروژه‌ای که `package.json` دارد، دستور `npm install axios` را می‌زنیم. کدام گزینه نتیجه را کامل و درست بیان می‌کند؟

۱) فقط پوشه‌ی `node_modules/axios` ساخته می‌شود و `package.json` تغییری نمی‌کند.

۲) `axios` با بازه‌ای مثل `^1.21.1` به بخش `devDependencies` اضافه می‌شود.

۳) `axios` به‌صورت سراسری (global) روی سیستم نصب می‌شود.

۴) `axios` با بازه‌ای مثل `^1.21.1` به بخش `dependencies` اضافه می‌شود، کدش در `node_modules` قرار می‌گیرد و `package-lock.json` ساخته یا به‌روز می‌شود.

۱۰۵۳ آسان

دستور کوتاه `npm i` همان کدام دستور است؟

۱) `npm init`

۲) `npm install`

۳) `npm info`

۴) `npm import`

۱۰۵۴ متوسط

می‌خواهیم `jest` (فریم‌ورک تست) را فقط به‌عنوان وابستگی **زمان توسعه** نصب کنیم. کدام دستور درست است؟

۱) `npm install -D jest`

۲) `npm install -g jest`

۳) `npm install jest`

۴) `npm update jest`

۱۰۵۵ متوسط

هم‌تیمی‌ات پروژه را از git گرفته است. فایل‌های `package.json` و `package-lock.json` هستند، ولی پوشه‌ی `node_modules` نیست. برای نصب همه‌ی وابستگی‌ها کدام دستور را بزند؟

۱) `npm init`

۲) `npm update -g`

۳) `npm install`

۴) `npm run node_modules`

۱۰۵۶ سخت

دستور `npm install -g typescript` را داخل پوشه‌ی یک پروژه اجرا می‌کنیم. کدام گزینه درست است؟

۱) `typescript` به بخش `dependencies` فایل `package.json` همین پروژه اضافه می‌شود.

۲) `typescript` فقط در پوشه‌ی `node_modules` همین پروژه نصب می‌شود و فقط در همین پروژه قابل استفاده است.

۳) `-g` یعنی نسخه‌ی پکیج از GitHub دانلود شود.

۴) `typescript` به‌صورت سراسری (global) نصب می‌شود؛ دستور `tsc` در ترمینال همه‌جا در دسترس است و `package.json` پروژه تغییری نمی‌کند.

۱۰۵۷ آسان

کدام دستور پکیج `lodash` را هم از `node_modules` و هم از `package.json` حذف می‌کند؟

۱) `npm uninstall lodash`

۲) `npm update lodash`

۳) `npm delete lodash`

۴) `npm install lodash@0`

۱۰۵۸ متوسط

`package.json` شامل `"is-number": "^6.0.0"` است و نسخه‌ی نصب‌شده `6.0.0` است. در registry، آخرین نسخه `7.0.0` است و نسخه‌ی دیگری با MAJOR برابر ۶ وجود ندارد. با اجرای `npm update` چه اتفاقی می‌افتد؟

۱) `is-number` به `7.0.0` ارتقا پیدا می‌کند، چون `npm update` همیشه آخرین نسخه را نصب می‌کند.

۲) خود برنامه‌ی npm به آخرین نسخه به‌روز می‌شود.

۳) نسخه همان `6.0.0` می‌ماند، چون `7.0.0` بیرون از بازه‌ی `^6.0.0` است.

۴) `is-number` حذف می‌شود، چون نسخه‌اش قدیمی است.

۱۰۵۹ آسان

اگر هیچ تنظیمی تغییر نکرده باشد، خروجی دستور `npm config get registry` کدام است؟

۱) `https://npmjs.com/`

۲) `https://registry.npm.com/`

۳) `https://nodejs.org/registry/`

۴) `https://registry.npmjs.org/`

۱۰۶۰ متوسط

کدام دستور npm را تنظیم می‌کند که از این به بعد پکیج‌ها را از آدرس `https://registry.npmmirror.com/` دانلود کند؟

۱) `npm config get registry https://registry.npmmirror.com/`

۲) `npm config set registry https://registry.npmmirror.com/`

۳) `npm install registry https://registry.npmmirror.com/`

۴) `npm mirror https://registry.npmmirror.com/`

۱۰۶۱ متوسط

به‌جای دستور `npm config set registry <url>` می‌توان آدرس mirror را در یک فایل هم نوشت. کدام گزینه درست است؟

۱) نوشتن خط `registry=<url>` در فایل `.npmrc`

۲) نوشتن `"registry": "<url>"` در بخش `dependencies` فایل `package.json`

۳) نوشتن خط `registry=<url>` در فایل `.gitignore`

۴) نوشتن `"registry": "<url>"` در فایل `package-lock.json`

۱۰۶۲ سخت

دستور `npm config set registry https://mirror.example.com/` را اجرا کرده‌ایم. کدام عبارت درست است؟

۱) همه‌ی پکیج‌های پروژه فوراً از mirror دوباره دانلود و نصب می‌شوند.

۲) آدرس mirror در فایل `package.json` پروژه ذخیره می‌شود.

۳) خط `registry=https://mirror.example.com/` در فایل `.npmrc` کاربر ذخیره می‌شود، نصب‌های بعدی از mirror انجام می‌شوند و `npm config get registry` هم همین آدرس را نشان می‌دهد.

۴) این تنظیم فقط برای همان یک دستور است و دستور بعدی دوباره از registry اصلی دانلود می‌کند.

۱۰۶۳ متوسط

معمولاً چرا از یک **mirror** (نسخه‌ی آینه‌ی registry) برای npm استفاده می‌کنند؟

۱) برای منتشر کردن پکیج بدون نیاز به حساب کاربری

۲) برای حذف خودکار وابستگی‌های بلااستفاده

۳) برای اجرای سریع‌تر اسکریپت‌های `package.json`

۴) برای دانلود سریع‌تر و در دسترس بودن پکیج‌ها، وقتی registry اصلی کُند است یا از شبکه‌ی ما به‌خوبی در دسترس نیست

۱۰۶۴ سخت

در فایل `~/.npmrc` کاربر نوشته شده `registry=https://user-mirror.example.com/` و در فایل `.npmrc` پروژه (کنار `package.json`) نوشته شده `registry=https://project-mirror.example.com/`. داخل پوشه‌ی پروژه، خروجی `npm config get registry` کدام است؟

۱) `https://user-mirror.example.com/`

۲) `https://project-mirror.example.com/`

۳) `https://registry.npmjs.org/`

۴) خطا می‌دهد، چون دو تنظیم با هم تداخل دارند.

۱۰۶۵ متوسط

`cowsay` پکیجی است که یک دستور خط فرمان دارد و در پروژه نصب نشده است. دستور `npx cowsay hello` چه کاری انجام می‌دهد؟

۱) پکیج `cowsay` را به `dependencies` فایل `package.json` اضافه می‌کند.

۲) پکیج `cowsay` را در registry منتشر (publish) می‌کند.

۳) دستورِ پکیج `cowsay` را بدون نصب دائمی در پروژه اجرا می‌کند و اگر لازم باشد، پکیج را موقتاً در cache دانلود می‌کند.

۴) اسکریپتی به اسم `cowsay` را از بخش `scripts` فایل `package.json` اجرا می‌کند.

۱۰۶۶ سخت

کدام عبارت درباره‌ی دستورهای npm **نادرست** است؟

۱) `npm i lodash` همان `npm install lodash` است.

۲) `npm install -D webpack` پکیج را در `devDependencies` ثبت می‌کند.

۳) `npm uninstall lodash` اسم پکیج را از `package.json` هم حذف می‌کند.

۴) `npm install` بدون اسم پکیج هیچ کاری انجام نمی‌دهد.

۱۰۶۷ سخت

با تنظیمات پیش‌فرض npm، دستور `npm install lodash@4.17.21` را اجرا می‌کنیم. در بخش `dependencies` فایل `package.json` چه نوشته می‌شود؟

۱) `"lodash": "4.17.21"`

۲) `"lodash": "^4.17.21"`

۳) `"lodash": "~4.17.21"`

۴) `"lodash": "latest"`

۳-۵ اسکریپت‌های npm و فایل package.json

منبع: NodeJS & TS، اسلاید ۱۱

درسنامه

فرض کن هر بار که می‌خواهی تلویزیون را روی شبکه‌ی خبر ببری، باید عدد فرکانس را دستی وارد کنی. خیلی راحت‌تر است که روی کنترل یک دکمه به اسم «خبر» داشته باشی. **اسکریپت‌های npm** همین دکمه‌ها هستند: به دستورهای طولانی و تکراری پروژه یک **اسم کوتاه** می‌دهیم و با آن اسم اجرایشان می‌کنیم. جای این دکمه‌ها فایل `package.json` است؛ پس اول خود این فایل را کامل بشناسیم.

آناتومی فایل `package.json`

یک `package.json` واقعی و نسبتاً کامل:

package.json

```
{
  "name": "shop-api",
  "version": "2.1.0",
  "description": "A small online shop",
  "main": "server.js",
  "type": "commonjs",
  "scripts": {
    "start": "node server.js",
    "dev": "node --watch server.js",
    "test": "jest",
    "build": "webpack --mode production"
  },
  "dependencies": {
    "express": "^5.3.0"
  },
  "devDependencies": {
    "jest": "^30.5.2",
    "webpack": "^5.112.2"
  },
  "license": "ISC"
}
```

فیلد	معنی
`name`	اسم پکیج؛ با حروف کوچک و بدون فاصله (مثل `shop-api`)
`version`	نسخه با قالب SemVer یعنی MAJOR.MINOR.PATCH
`description`	توضیح کوتاه پروژه
`main`	**فایل ورودی پکیج**: وقتی کس دیگری این پکیج را با `require("shop-api")` وارد کند، این فایل بارگذاری می‌شود. `npm start` از این فیلد استفاده **نمی‌کند**.
`type`	`"commonjs"` یا `"module"` : فایل‌های `.js` به‌صورت CommonJS اجرا شوند یا ES Module
`scripts`	دستورهای میان‌بُر پروژه (موضوع اصلی این درس)
`dependencies`	وابستگی‌های زمان **اجرا**
`devDependencies`	وابستگی‌های زمان **توسعه** (تست، build و...)
`license`	مجوز استفاده از کد (پیش‌فرض `npm init` : `ISC`)

دستور `node --watch server.js` در اسکریپت `dev` یعنی «سرور را اجرا کن و با هر تغییر فایل، خودکار دوباره راهش بینداز».

قانون‌های JSON

`package.json` باید **JSON واقعی** باشد، نه کد JavaScript. یعنی:

- همه‌ی کلیدها و رشته‌ها داخل کوتیشن **دوتایی** (`"`) باشند؛ کوتیشن تکی (`'`) و کلید بدون کوتیشن ممنوع است.
- **توضیح (comment)** مثل `//` یا `/* */` ممنوع است.
- بعد از آخرین عضو یک شیء یا آرایه **ویرگول** نباید (trailing comma ممنوع).
- مقدارها فقط می‌توانند رشته، عدد، `true` / `false` ، `null` ، آرایه یا شیء باشند (نه تابع، نه `undefined`).

در فایل خراب زیر، بعد از آخرین عضو یک ویرگول اضافه آمده است:

package.json

```
{
  "name": "shop-api",
  "scripts": { "start": "node server.js" },
}
```

همین یک ویرگول کافی است تا هر دستور npm خطا بدهد:

```
npm error code EJSONPARSE
npm error JSON.parse Invalid package.json: JSONParseError: Expected double-quoted property name in
JSON at position 68 (line 4 column 1) while parsing near "...\"node server.js\" },\n}\n"
npm error JSON.parse Failed to parse JSON data.
npm error JSON.parse Note: package.json must be actual JSON, not just JavaScript.
```

اسکریپت‌های npm (اسلاید ۱۱)

اسلاید ۱۱ پنج نکته دارد:

۱. **Define custom commands in the scripts section of package.json.** دستورهای دلخواه را در بخش `scripts` فایل `package.json` تعریف می‌کنیم: کلید، اسم اسکریپت است و مقدار، دستوری که در ترمینال اجرا می‌شود.

۲. **Run scripts with `npm run <script>` from the terminal**. اسکریپت را با `npm run <اسم>` اجرا می‌کنیم.

۳. **Can automate tasks like building, testing, or starting servers**. کارهای تکراری مثل **ساختن** (build)، **تست** و **راه‌اندازی سرور** را خودکار می‌کنیم.

۴. **Supports chaining commands and using environment variables**. می‌توان چند دستور را پشت هم **زنجیر** کرد (chaining) و از **متغیرهای محیطی** (environment variables: مقدارهایی که سیستم‌عامل در اختیار برنامه‌ها می‌گذارد و در Node.js با `process.env` خوانده می‌شوند) استفاده کرد.

۵. **Widely used to standardize project workflows across teams**. همه‌ی اعضای تیم با دستورهای **یکسان** (`npm test`، `npm run build`، ...) کار می‌کنند و لازم نیست جزئیات ابزارهای پشت هر دستور را حفظ کنند.

خود اسلاید مثال کد ندارد؛ مثال‌های زیر را ما ساخته و با npm نسخه‌ی ۱۱ اجرا کرده‌ایم. فرض کن `package.json` این است و `server.js` فقط `console.log("Server is running");` را دارد:

package.json

```
{
  "name": "shop-api",
  "version": "2.1.0",
  "scripts": {
    "start": "node server.js",
    "test": "echo \"Error: no test specified\" && exit 1",
    "dev": "echo dev mode",
    "build": "echo step 1: clean && echo step 2: compile",
    "ci": "npm test && npm run build",
    "info": "node -e \"console.log(process.env.npm_package_name, process.env.npm_package_version,
process.env.npm_lifecycle_event)\""
  }
}
```

(داخل رشته‌ی JSON، کوتیشن دوتایی را باید با `\"` نوشت.)

اجرای یک اسکریپت و شکل خروجی

npm قبل از اجرای هر اسکریپت، دو خط **سرآیند** با علامت `>` چاپ می‌کند: اسم و نسخه‌ی پکیج به‌اضافه‌ی اسم اسکریپت، و خود دستور. بعد خروجی دستور می‌آید:

Terminal

```
npm run build
# > shop-api@2.1.0 build
# > echo step 1: clean && echo step 2: compile
#
# step 1: clean
# step 2: compile
```

اگر `--silent` اضافه کنی (`npm run build --silent`)، خط‌های سرآیند چاپ نمی‌شوند و فقط خروجی خود دستور دیده می‌شود.

`start` و `test` بدون `run`؛ بقیه با `run`

چند اسم اسکریپت برای npm معنای خاص دارند و **بدون** `run` هم اجرا می‌شوند؛ مهم‌ترین‌هایشان `start` و `test` هستند (`npm t` هم کوتاه‌شده‌ی `npm test` است). بقیه‌ی اسکریپت‌ها حتماً `run` می‌خواهند:

```
Terminal
npm start        # runs "start": Server is running
npm test         # runs "test"
npm run dev      # runs "dev": dev mode
npm dev
# Unknown command: "dev"
#
# Did you mean this?
#   npm run dev # run the "dev" package script
```

فهرست اسکریپت‌ها و اسکریپتِ ناموجود

`npm run` **بدون اسم** فهرست اسکریپت‌ها را نشان می‌دهد؛ اول اسکریپت‌های خاص (Lifecycle scripts مثل `start` و `test`) و بعد بقیه («available via `npm run`»). اگر اسم اسکریپتی را بزنی که وجود ندارد:

```
Terminal
npm run deploy
# npm error Missing script: "deploy"
# npm error
# npm error To see a list of scripts, run:
# npm error   npm run
```

دو پیام را با هم قاطی نکن: `Unknown command: "dev"` یعنی خود **npm** دستوری به اسم `dev` ندارد (چون `run` را جا انداخته‌ای)؛ `Missing script: "deploy"` یعنی **اسکریپتی** به اسم `deploy` در `package.json` نیست.

زنجیر کردن دستورها با `&&`

در اسکریپت `build`، عملگر `&&` (و منطقی در ترمینال) دو دستور را پشت هم اجرا کرد. قانونش این است: **دستور دوم فقط وقتی اجرا می‌شود که دستور اول موفق باشد** (یعنی با کد خروج ۰ تمام شود). اسکریپت `ci` را ببین:

```
Terminal
npm run ci
# > shop-api@2.1.0 ci
# > npm test && npm run build
#
# > shop-api@2.1.0 test
# > echo "Error: no test specified" && exit 1
#
# Error: no test specified
```

اسکریپت `test` پیش‌فرضِ `npm init` با `exit 1` تمام می‌شود، یعنی «شکست». پس `&&` اجازه‌ی اجرای `npm run build` را نمی‌دهد و کل `npm run ci` هم با کد خروج ۱ شکست می‌خورد. (خود اسکریپت `test` پیش‌فرض هم یک نمونه‌ی `&&` است: اول پیام را چاپ می‌کند و بعد با `exit 1` شکست را اعلام می‌کند.)

عملگر	دستور دوم کی اجرا می‌شود؟
`a && b`	فقط اگر `a` **موفق** باشد
`a || b`	فقط اگر `a` **شکست** بخورد
`a ; b`	در هر حال (در ترمینال‌های Linux و macOS)

متغیرهای محیطی

npm هنگام اجرای اسکریپت چند متغیر محیطی آماده می‌کند؛ مثلاً `npm_package_name` (اسم پکیج)، `npm_package_version` (نسخه) و `npm_lifecycle_event` (اسم اسکریپتی که الان اجرا می‌شود):

```
Terminal
npm run info
# > shop-api@2.1.0 info
# > node -e "console.log(process.env.npm_package_name, process.env.npm_package_version,
process.env.npm_lifecycle_event)"
#
# shop-api 2.1.0 info
```

متغیر دلخواه خودت را هم می‌توانی جلوی دستور بنویسی (این نحو مال ترمینال‌های Linux و macOS است؛ در Windows معمولاً از پکیج `cross-env` کمک می‌گیرند). با اسکریپت `"greet": "node -e \"console.log('Hello ' + (process.env.NAME || 'guest'))\""`:

```
Terminal
npm run greet              # Hello guest
NAME=Sara npm run greet    # Hello Sara
```

ابزارهای `node_modules/.bin` و فرستادن آرگومان

- npm هنگام اجرای اسکریپت، پوشه‌ی `node_modules/.bin` پروژه را **اول PATH** می‌گذارد. پس اسکریپتی مثل `"test": "jest"` کار می‌کند، حتی اگر `jest` فقط داخل پروژه (با `npm install -D jest`) نصب شده باشد و سراسری نصب نباشد.
- برای فرستادن آرگومان به خود اسکریپت، آن را بعد از `--` بنویس: `npm start -- --port=4000` در عمل `node server.js --port=4000` را اجرا می‌کند و `process.argv.slice(2)` در `server.js` می‌شود `[ '--port=4000' ]`. بدون `--`، npm خودش آن گزینه را برمی‌دارد و به اسکریپت نمی‌رسد.

اسکریپت‌های pre و post

اگر اسکریپتی به اسم `prebuild` یا `postbuild` تعریف کنی، `npm run build` خودکار اول `prebuild`، بعد `build` و در آخر `postbuild` را اجرا می‌کند؛ ترتیب نوشتنشان در فایل مهم نیست:

```
JSON
"scripts": {
  "postbuild": "echo C",
  "build": "echo B",
  "prebuild": "echo A"
}
```

```
Terminal
npm run build --silent
# A
# B
# C
```

نکته

- `npm start` و `npm test` بدون `run` هم کار می‌کنند؛ هر اسکریپت دیگری (`dev`، `build`، `lint`، ...) با `npm run <اسم>` اجرا می‌شود.
- اگر `start` تعریف نشده باشد ولی فایلی به اسم `server.js` در پروژه باشد، `npm start` به‌طور پیش‌فرض `node server.js` را اجرا می‌کند؛ در غیر این صورت `Missing script: "start"` می‌گیری. فیلد `main` در این تصمیم هیچ نقشی ندارد.

دام تستی

- `npm build` یا `npm dev` خطای `Unknown command` می‌دهند؛ `run` را فراموش نکن.
- `package.json` توضیح (comment)، ویرگول اضافه‌ی آخر و کوتیشن تکی را **نمی‌پذیرد**؛ هر کدام خطای `EJSONPARSE` می‌دهد.
- در `a && b`، اگر `a` شکست بخورد `b` **اجرا نمی‌شود**؛ در `a || b` برعکس است.

جمع‌بندی

- `package.json`: `name`، `version`، `main` (فایل ورودی پکیج)، `type`، `scripts`، `dependencies`، `devDependencies`، `license`؛ و باید JSON معتبر باشد.
- اسلاید ۱۱: تعریف در `scripts`، اجرا با `npm run <script>`، خودکارسازی build و test و راه‌اندازی سرور، زنجیر کردن دستورها و متغیرهای محیطی، یکسان‌سازی روند کار تیم.
- `npm run` به‌تنهایی فهرست می‌دهد؛ اسکریپت ناموجود: `Missing script`؛ دستور ناموجود npm: `Unknown command`.
- npm متغیرهایی مثل `npm_package_name` و `npm_lifecycle_event` می‌سازد و `node_modules/.bin` را به PATH اضافه می‌کند.

تست‌های تمرینی

۲۰ تست · شماره‌ی ۱۰۶۸ تا ۱۰۸۷

۱۰۶۸ آسان

طبق اسلاید ۱۱، اسکریپت‌های npm در کجا تعریف می‌شوند؟

۱) فایل `package-lock.json`
۲) فایل `.npmrc`
۳) بخش `scripts` فایل `package.json`
۴) پوشه‌ی `node_modules/.bin`

۱۰۶۹ آسان

در `package.json` اسکریپتی به اسم `build` تعریف شده است. با کدام دستور اجرایش می‌کنیم؟

۱) `npm run build`
۲) `npm build`
۳) `node build`
۴) `npm install build`

۱۰۷۰ متوسط

در `package.json` چهار اسکریپت `dev`، `lint`، `build` و `test` تعریف شده است. کدام دستور **بدون** `run` اسکریپت مربوطش را اجرا می‌کند؟

۱) `npm dev`
۲) `npm lint`
۳) `npm build`
۴) `npm test`

۱۰۷۱ متوسط

در `package.json` هیچ اسکریپتی به اسم `deploy` تعریف نشده است. با اجرای `npm run deploy` چه پیامی می‌گیریم؟

۱) `Unknown command: "deploy"`
۲) `npm error Missing script: "deploy"`
۳) هیچ پیامی چاپ نمی‌شود و دستور بی‌صدا تمام می‌شود.
۴) npm یک اسکریپت خالی به اسم `deploy` می‌سازد و اجرایش می‌کند.

۱۰۷۲ آسان

دستور `npm run` (بدون اسم اسکریپت) چه کاری انجام می‌دهد؟

۱) همه‌ی اسکریپت‌های `package.json` را به ترتیب اجرا می‌کند.
۲) اسکریپت `start` را اجرا می‌کند.
۳) فهرست اسکریپت‌های تعریف‌شده در `package.json` را نشان می‌دهد.
۴) خطای `Missing script` می‌دهد.

۱۰۷۳ سخت

`package.json` یک پروژه به شکل زیر است. با اجرای `npm run build`، خود دستورها (بدون در نظر گرفتن خط‌های سرآیندِ `>` که npm چاپ می‌کند) چه چیزی چاپ می‌کنند؟

```JSON
{
  "name": "site",
  "version": "1.0.0",
  "scripts": {
    "postbuild": "echo C",
    "build": "echo B",
    "prebuild": "echo A"
  }
}
```

۱) `A` `B` `C`
۲) `C` `B` `A`
۳) `B`
۴) `B` `A` `C`

۱۰۷۴ سخت

با `package.json` زیر، دستور `npm run ci` را اجرا می‌کنیم. چه اتفاقی می‌افتد؟

```JSON
{
  "name": "shop-api",
  "version": "2.1.0",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "build": "echo building...",
    "ci": "npm test && npm run build"
  }
}
```

۱) هر دو اسکریپت اجرا می‌شوند و `building...` هم چاپ می‌شود، چون `&&` فقط دستورها را پشت سر هم می‌چیند.

۲) اول `building...` و بعد پیام خطای تست چاپ می‌شود.

۳) npm خطای `Missing script: "ci"` می‌دهد.

۴) پیام `Error: no test specified` چاپ می‌شود، تست با کد خروج ۱ شکست می‌خورد و `npm run build` اجرا نمی‌شود.

۱۰۷۵ متوسط

هنگام اجرای یک اسکریپت npm، کدام متغیر محیطی **اسم اسکریپتی** را که الان اجرا می‌شود (مثلاً `build`) نگه می‌دارد؟

۱) `npm_package_name`

۲) `npm_lifecycle_event`

۳) `npm_script_name`

۴) `NODE_ENV`

۱۰۷۶ سخت

کدام گزینه می‌تواند محتوای **معتبر** یک فایل `package.json` باشد؟

۱) `{ "name": "shop", }`

۲) `{ 'name': 'shop' }`

۳) `{ "name": "shop", "private": true }`

۴) `{ name: "shop" }`

۱۰۷۷ سخت

کدام عبارت درباره‌ی `package.json` **نادرست** است؟

۱) می‌توان با `//` داخل `package.json` توضیح (comment) نوشت تا اسکریپت‌ها خواناتر شوند.

۲) کلیدها و رشته‌ها در `package.json` باید داخل کوتیشن دوتایی (`"`) باشند.

۳) بعد از آخرین عضو یک شیء یا آرایه در `package.json` نباید ویرگول بیاید.

۴) اگر `package.json` خراب باشد، npm خطای `EJSONPARSE` می‌دهد و یادآوری می‌کند که این فایل باید JSON واقعی باشد، نه JavaScript.

۱۰۷۸ متوسط

فرق `npm run greet --silent` با `npm run greet` چیست؟

۱) `--silent` اسکریپت را اجرا نمی‌کند و فقط نشان می‌دهد چه دستوری اجرا می‌شد.

۲) `--silent` خروجی خود اسکریپت را هم پنهان می‌کند و هیچ چیزی چاپ نمی‌شود.

۳) `--silent` اسکریپت را در پس‌زمینه اجرا می‌کند.

۴) `--silent` خط‌های سرآیند npm (که با `>` شروع می‌شوند) را چاپ نمی‌کند و فقط خروجی خود اسکریپت دیده می‌شود.

۱۰۷۹ سخت

دو فایل زیر در پروژه هستند. در ترمینال دستور `npm start -- --port=4000` را اجرا می‌کنیم. فایل `server.js` (بعد از خط‌های سرآیند npm) چه چیزی چاپ می‌کند؟

package.json

```
{
  "name": "shop-api",
  "version": "2.1.0",
  "scripts": {
    "start": "node server.js"
  }
}
```

server.js

```
console.log(process.argv.slice(2));
```

۱) `[]`

۲) `[ '--port=4000' ]`

۳) `[ '--', '--port=4000' ]`

۴) خطای `Unknown command`، چون `start` بدون `run` آمده است.

۱۰۸۰ متوسط

`jest` در یک پروژه فقط با `npm install -D jest` نصب شده (سراسری نصب نیست) و در `package.json` داریم `"test": "jest"`. چرا `npm test` بدون خطا `jest` را پیدا و اجرا می‌کند؟

۱) چون npm هنگام نصب، `jest` را خودکار به‌صورت سراسری هم نصب می‌کند.

۲) چون `jest` جزو دستورهای داخلی خود npm است.

۳) چون npm هنگام اجرای اسکریپت‌ها، پوشه‌ی `node_modules/.bin` پروژه را به PATH اضافه می‌کند.

۴) چون Node.js خودش `jest` را همراه دارد.

۱۰۸۱ متوسط

در `package.json` فقط `"main": "app.js"` آمده و بخش `scripts` هیچ اسکریپت `start`ی ندارد. فایلی به اسم `server.js` هم در پروژه نیست. با اجرای `npm start` چه اتفاقی می‌افتد؟

۱) خطای `npm error Missing script: "start"` می‌گیریم.

۲) فایل `app.js` اجرا می‌شود، چون `main` فایل اصلی پروژه است.

۳) فایل `index.js` اجرا می‌شود.

۴) خطای `Unknown command: "start"` می‌گیریم.

۱۰۸۲ آسان

جای خالی را کامل کن:

«در اسکریپت `"build": "npm run clean …… npm run compile"`، برای اینکه دستور دوم **فقط در صورت موفقیت** دستور اول اجرا شود، به جای نقطه‌چین عملگر ..... را می‌نویسیم.»

۱) `||`　۲) `|`　۳) `;`　۴) `&&`

۱۰۸۳ متوسط

اسلاید ۱۱ می‌گوید اسکریپت‌های «Widely used to standardize project workflows across teams in npm». منظور چیست؟

۱) هر عضو تیم باید اسکریپت‌های جداگانه‌ی خودش را بنویسد.

۲) همه‌ی اعضای تیم کارهای رایج را با دستورهای یکسانی مثل `npm test` و `npm run build` انجام می‌دهند، بدون اینکه لازم باشد جزئیات ابزارهای پشت آن‌ها را حفظ کنند.

۳) اسکریپت‌ها فقط روی سیستم‌عامل Linux اجرا می‌شوند تا محیط همه یکسان باشد.

۴) npm خودکار کد همه‌ی اعضای تیم را یکسان‌سازی و ادغام (merge) می‌کند.

۱۰۸۴ سخت

با `package.json` زیر، دستور `npm run info` را اجرا می‌کنیم. خط آخر خروجی (بعد از سرآیندهای npm) کدام است؟

```JSON
{
  "name": "shop-api",
  "version": "2.1.0",
  "scripts": {
    "info": "node -e \"console.log(process.env.npm_package_name,
process.env.npm_package_version, process.env.npm_lifecycle_event)\""
  }
}
```

۱) `undefined undefined undefined`

۲) `shop-api 2.1.0 start`

۳) `shop-api 2.1.0 info`

۴) `info 2.1.0 shop-api`

۱۰۸۵ آسان

در `package.json`، وابستگی‌هایی که فقط هنگام **توسعه** لازم‌اند (مثل ابزار تست) در کدام فیلد نوشته می‌شوند؟

۱) `devDependencies`　۲) `dependencies`　۳) `scripts`　۴) `main`

۱۰۸۶ متوسط

فیلد `"type": "module"` در `package.json` چه اثری دارد؟

۱) پروژه را خصوصی می‌کند تا به‌اشتباه در registry منتشر نشود.

۲) اعلام می‌کند که پروژه یک ماژول npm است و دیگران می‌توانند نصبش کنند.

۳) فایل‌های `.mjs` پروژه را به CommonJS تبدیل می‌کند.

۴) فایل‌های `.js` پروژه به‌صورت ES Module اجرا می‌شوند و می‌توان در آن‌ها از `import` و `export` استفاده کرد.

۱۰۸۷ متوسط

پروژه‌ای را با `npm init -y` ساخته‌ایم و اسکریپت `test` آن را تغییر نداده‌ایم. با اجرای `npm test` چه اتفاقی می‌افتد؟

۱) npm همه‌ی فایل‌های تست پروژه را پیدا و اجرا می‌کند.

۲) پیام `Error: no test specified` چاپ می‌شود و دستور با کد خروج ۱ (یعنی شکست) تمام می‌شود.

۳) خطای `Missing script: "test"` می‌گیریم.

۴) خطای `Unknown command: "test"` می‌گیریم، چون باید `npm run test` بنویسیم.

۳-۶ Bundling برای مرورگر

منبع: NodeJS & TS، اسلاید ۱۲

درسنامه

فرض کن برای یک سفر، وسایلت را در پنجاه کیسه‌ی کوچک جدا گذاشته‌ای. حمل کردنشان عذاب است. راه بهتر: همه را در **یک چمدان** بگذاری، چیزهایی را که اصلاً استفاده نمی‌کنی **بیرون بیاوری** و لباس‌ها را **وکیوم کنی** تا جای کمتری بگیرند. **Bundling** (بسته‌بندی) برای کد JavaScript دقیقاً همین کار را می‌کند: یکی کردن فایل‌ها (چمدان)، **tree-shaking** (بیرون آوردن چیزهای بی‌استفاده) و **minification** (وکیوم کردن).

مشکل: برنامه‌ی مرورگر با فایل‌های زیاد

یک برنامه‌ی واقعی ده‌ها یا صدها فایل JavaScript دارد. اگر بخواهیم همه را مستقیم به مرورگر بدهیم:

```HTML
<script src="utils.js"></script>
<script src="cart.js"></script>
<script src="ui.js"></script>
<script src="app.js"></script> <!-- order matters: app.js needs the others -->
```

- برای **هر فایل** یک **درخواست HTTP** (HTTP request: پیامی که مرورگر برای گرفتن یک فایل به سرور می‌فرستد) جدا لازم است و هر درخواست سربار و تأخیر خودش را دارد.
- **ترتیب** تگ‌ها مهم است و با زیاد شدن فایل‌ها، مدیریتش سخت می‌شود.
- پکیج‌های npm در پوشه‌ی `node_modules` هستند، ولی مرورگر از `node_modules` خبر ندارد و `require` را هم نمی‌شناسد. حتی `import _ from "lodash"` در مرورگر کار نمی‌کند، چون مرورگر فقط آدرس‌هایی را می‌فهمد که با `./`، `../` یا `/` شروع شوند (یا آدرس کامل باشند). Chrome در این حالت خطای `Failed to resolve module specifier "lodash"` می‌دهد.

راه‌حل، یک **bundler** (بسته‌بند) است.

Bundling برای مرورگرها (اسلاید ۱۲)

bundler از یک فایل شروع (**entry point**: نقطه‌ی ورود، مثلاً `src/app.js`) را می‌خواند، همه‌ی `import` ها و `require` هایش را دنبال می‌کند، **گراف وابستگی‌ها** (dependency graph: نقشه‌ی اینکه کدام فایل به کدام فایل نیاز دارد) را می‌سازد و همه را در یک فایل خروجی به اسم **bundle** می‌ریزد. اسلاید ۱۲ پنج نکته دارد:

۱. **Combines multiple JS files into a single output for browsers.** چند فایل JavaScript را در **یک خروجی** برای مرورگر ترکیب می‌کند.

۲. **Reduces HTTP requests and improves load performance.** تعداد درخواست‌های HTTP کم و سرعت بارگذاری صفحه بیشتر می‌شود.

۳. **Handles module resolution, imports, and exports automatically.** کار **module resolution** (پیدا کردن فایل واقعی هر `import`؛ چه مسیر نسبی مثل `./math.js` باشد و چه اسم یک پکیج مثل `lodash` در `node_modules`) و وصل کردن `import` ها و `export` ها را **خودکار** انجام می‌دهد.

۴. **Often includes optimizations like minification and tree-shaking.** معمولاً **بهینه‌سازی‌هایی** مثل minification و tree-shaking را هم انجام می‌دهد (پایین‌تر می‌بینیم).

۵. **Popular tools: Webpack, Rollup, and Parcel.** ابزارهای معروف: **Webpack** (قدیمی‌ترینِ این سه و پرامکانات‌ترینشان)، **Rollup** (محبوب برای ساختن کتابخانه‌ها و معروف به tree-shaking) و **Parcel** (تقریباً بدون نیاز به تنظیمات).

یک مثال واقعی

دو فایل زیر را با bundler سریع **esbuild** (یک bundler مدرن) bundle کرده‌ایم. خروجی Webpack ظاهر متفاوتی دارد، ولی ایده همین است:

src/math.js

```
// Math helpers
export function add(a, b) {
  return a + b;
}

export function multiply(a, b) {
  return a * b;
}
```

src/app.js

```
import { add } from "./math.js";

// Show the result of a small calculation
const result = add(2, 3);
console.log("Result:", result);
```

dist/bundle.js

```
(() => {
  // src/math.js
  function add(a, b) {
    return a + b;
  }

  // src/app.js
  var result = add(2, 3);
  console.log("Result:", result);
})();
```

به خروجی دقت کن:

- هر دو فایل در **یک** فایل آمده‌اند و `export` / `import` دیگر لازم نیست؛ bundler خودش آن‌ها را به هم وصل کرده است.
- تابع `multiply` **حذف شده**، چون هیچ‌جا استفاده نشده بود (tree-shaking).
- توضیح‌های اصلی (`// Math helpers`) حذف شده‌اند؛ خط‌هایی مثل `// src/math.js` را خود bundler گذاشته تا معلوم باشد هر تکه از کجا آمده.
- کل کد داخل یک تابع `(() => { … })()` پیچیده شده تا اسم‌هایش وارد محدوده‌ی سراسری (global scope) نشوند.

حالا در HTML فقط یک خط لازم است:

```HTML
<script src="dist/bundle.js"></script>
```

Minification (کوچک‌سازی)

minification یعنی کوچک کردن حجم کد **بدون تغییر رفتارش**: فاصله‌ها، خط‌های خالی و توضیح‌ها حذف و اسم متغیرهای محلی کوتاه می‌شوند. مرورگر به خوانایی کد اهمیتی نمی‌دهد، ولی فایل کوچک‌تر سریع‌تر دانلود می‌شود.

قبل از minification:

```total.js
// Calculate the total price
function calculateTotal(price, quantity) {
  const total = price * quantity;
  return total;
}

console.log(calculateTotal(10, 3));
```

بعد از bundle و minify با esbuild (گزینه‌های `--bundle --minify`):

```total.min.js
(()=>{function l(t,o){return t*o}console.log(l(10,3));})();
```

هر دو نسخه `30` چاپ می‌کنند. اسم `calculateTotal` به `l` و پارامترها به `t` و `o` تغییر کرده‌اند؛ چون همه‌ی استفاده‌ها **با هم** عوض شده‌اند، رفتار برنامه دست نخورده است.

Tree-shaking (تکاندن درخت)

درختی را تصور کن که برگ‌های خشک دارد؛ اگر تکانش بدهی، برگ‌های مرده می‌ریزند. **tree-shaking** کدی را که **export شده ولی هیچ‌جا استفاده نشده** (dead code: کد مرده) از bundle نهایی حذف می‌کند؛ مثل `multiply` در مثال بالا. حتی اگر تابعی را `import` کنی ولی هیچ‌وقت صدایش نزنی، باز هم حذف می‌شود.

tree-shaking به **ES Modules** تکیه دارد، چون `import` و `export` **ایستا** (static) هستند: باید در بالاترین سطح فایل و با اسم‌های ثابت نوشته شوند. پس bundler **قبل از اجرای برنامه**، فقط با خواندن کد، دقیقاً می‌فهمد چه چیزی از کجا استفاده شده است. اما `require` در CommonJS یک فراخوانی معمولی تابع است و می‌تواند **پویا** باشد:

```JavaScript
const lib = require(useFast ? "./fast.js" : "./slow.js"); // decided only at runtime
exports[name] = value;                                     // export name decided at runtime
```

bundler در این حالت نمی‌تواند مطمئن باشد چه چیزی لازم است و معمولاً همه را نگه می‌دارد. ما همان `math.js` را به شکل CommonJS (`exports.add = …` و `exports.multiply = …`) نوشتیم و bundle کردیم: `multiply` این بار در bundle **باقی ماند**.

Transpiling، Bundling و Minifying را قاطی نکن

کار	چه می‌کند؟	ابزار نمونه
Transpiling (ترجمه‌ی کد)	کد جدید (ES2022، TypeScript، JSX) را به JavaScript قدیمی‌تر یا ساده‌تر تبدیل می‌کند؛ تعداد فایل‌ها عوض نمی‌شود	Babel و SWC (فصل اول)، `tsc`
Bundling (یکی کردن)	چند ماژول را در یک (یا چند) فایل خروجی ترکیب می‌کند	Webpack، Rollup، Parcel
Minifying (کوچک‌سازی)	حجم کد را بدون تغییر رفتار کم می‌کند	Terser، esbuild

در عمل، bundler معمولاً transpiler و minifier را هم در مسیر کارش صدا می‌زند و همه با یک اسکریپت npm اجرا می‌شوند؛ مثلاً `"build": "webpack --mode production"` که در حالت production، minification و tree-shaking را هم روشن می‌کند.

Source map

اشکال‌زدایی (debug) کد minifyشده تقریباً غیرممکن است: خطا در «خط ۱، ستون ۸۴۳۲» یک فایل یک‌خطی! **source map** (نقشه‌ی منبع) فایل جداگانه‌ای مثل `bundle.js.map` است که هر تکه از کد bundle را به **فایل و خط اصلی‌اش** وصل می‌کند. ابزار توسعه‌دهنده‌ی مرورگر (DevTools) با کمک آن، کد اصلی و خوانا را نشان می‌دهد. وقتی مثال بالا را با گزینه‌ی source map ساختیم، کنار `bundle.js` فایل `bundle.js.map` هم ساخته شد که در آن آمده بود این bundle از `src/math.js` و `src/app.js` ساخته شده است.

ابزارهای مدرن‌تر

- bundler :**esbuild** بسیار سریعی که با زبان Go نوشته شده است.
- **Vite**: ابزار محبوبی که در زمان توسعه خیلی سریع است و برای خروجی نهایی هم bundle می‌سازد.
- در پروژه‌های بزرگ، bundler می‌تواند خروجی را به چند تکه (chunk) هم تقسیم کند تا هر صفحه فقط کد لازم خودش را دانلود کند (code splitting).

نکته

bundler کد **ما** را برای مرورگر آماده می‌کند، ولی پکیج‌ها را دانلود نمی‌کند؛ دانلود هنوز کار `npm install` است و bundler از چیزهایی استفاده می‌کند که در `node_modules` هست. bundler معمولاً یک devDependency است و با اسکریپتی مثل `npm run build` اجرا می‌شود.

دام تستی

- **tree-shaking** کدِ استفاده‌نشده را از **bundle** حذف می‌کند، **نه** پکیج‌ها را از `node_modules` .
- **minification** کد را کوچک می‌کند ولی کد مرده را لزوماً حذف نمی‌کند؛ حذف کد مرده کار tree-shaking است. این دو را با هم عوض نکن.
- Babel یک **transpiler** است، نه bundler؛ Webpack یک **bundler** است، نه transpiler.

جمع‌بندی

- bundler: چند فایل ← یک خروجی برای مرورگر، درخواست HTTP کمتر و بارگذاری سریع‌تر، module resolution و `import` / `export` خودکار.
- بهینه‌سازی‌ها: minification (کوچک‌سازی بدون تغییر رفتار) و tree-shaking (حذف کد مرده؛ به `import` / `export` ایستای ES Modules تکیه دارد).
- ابزارها: Webpack، Rollup، Parcel (و ابزارهای مدرن‌تری مثل esbuild و Vite).
- minifying ≠ bundling ≠ transpiling؛ source map برای debug کد bundleشده است.

تست‌های تمرینی

۱۵ تست · شماره‌ی ۱۰۸۸ تا ۱۱۰۲

۱۰۸۸ آسان

کار اصلی یک **bundler** چیست؟

۱) تبدیل کد TypeScript به JavaScript

۲) ترکیب چند فایل JavaScript (ماژول) در یک فایل خروجی برای مرورگر

۳) اجرای کد JavaScript روی سرور

۴) دانلود و نصب پکیج‌ها از registry

۱۰۸۹ آسان

کدام گزینه جزو ابزارهای bundling که اسلاید ۱۲ نام می‌برد **نیست**؟

۱) Webpack ۲) Rollup ۳) Parcel ۴) Express

۱۰۹۰ متوسط

چرا bundling سرعت بارگذاری صفحه را بیشتر می‌کند؟

۱) چون مرورگر به‌جای ده‌ها درخواست HTTP برای فایل‌های جدا، فقط یک (یا چند) فایل دانلود می‌کند.

۲) چون bundle کد را چندنخی (multi-threaded) می‌کند.

۳) چون با bundle دیگر به سرور نیازی نیست.

۴) چون مرورگر کد bundleشده را بدون خواندن و پردازش (parse) اجرا می‌کند.

۱۰۹۱ متوسط

کدام گزینه **minification** را درست توصیف می‌کند؟

۱) حذف توابعی که در هیچ‌جای برنامه استفاده نشده‌اند

۲) فشرده کردن تصاویر و فونت‌های سایت

۳) حذف فاصله‌ها و توضیح‌ها و کوتاه کردن اسم متغیرهای محلی، بدون تغییر رفتار کد

۴) تبدیل کد JavaScript به TypeScript

۱۰۹۲ متوسط

کدام گزینه **tree-shaking** را درست توصیف می‌کند؟

۱) حذف پکیج‌های بلااستفاده از پوشه‌ی `node_modules`

۲) حذف کدهای مرده (exportهایی که هیچ‌جا استفاده نشده‌اند) از bundle نهایی

۳) مرتب کردن فایل‌های پروژه به شکل درخت پوشه‌ها

۴) کوتاه کردن اسم متغیرها در کد خروجی

۱۰۹۳ سخت

دو ماژول زیر را با یک bundler که tree-shaking انجام می‌دهد (مثل esbuild یا Rollup) bundle می‌کنیم و `app.js` نقطه‌ی ورود است. کدام تابع(ها) در bundle نهایی باقی می‌مانند؟

math.js

```
export function add(a, b) {
  return a + b;
}
export function subtract(a, b) {
  return a - b;
}
export function multiply(a, b) {
  return a * b;
}
```

app.js

```
import { add, multiply } from "./math.js";

console.log(add(2, 3));
```

۱) هر سه تابع `add`، `subtract` و `multiply`

۲) `add` و `multiply`

۳) هیچ‌کدام؛ فقط `console.log` باقی می‌ماند

۴) فقط `add`

۱۰۹۴ سخت

چرا tree-shaking با **ES Modules** خیلی بهتر از **CommonJS** کار می‌کند؟

۱) چون `import` و `export` ایستا (static) هستند و bundler قبل از اجرای برنامه می‌فهمد دقیقاً چه چیزی استفاده شده؛ ولی `require` یک فراخوانی معمولی تابع است و می‌تواند پویا باشد.

۲) چون فایل‌های ES Module همیشه از فایل‌های CommonJS کوچک‌ترند.

۳) چون در CommonJS اصلاً نمی‌توان چیزی را export کرد.

۴) چون مرورگرها فقط ES Module را اجرا می‌کنند و کد CommonJS را نادیده می‌گیرند.

۱۰۹۵ متوسط

کد اصلی زیر `30` چاپ می‌کند. کدام گزینه یک نسخه‌ی minify‌شده‌ی **درست** از آن است، یعنی دقیقاً همان خروجی `30` را چاپ می‌کند؟

```JavaScript
// Calculate the total price
function calculateTotal(price, quantity) {
  const total = price * quantity;
  return total;
}
console.log(calculateTotal(10, 3));
```

۱) `function c(t,n){return t+n}console.log(c(10,3));`

۲) `function c(t,n){return t*n}console.log(calculateTotal(10,3));`

۳) `function c(t,n){return t*n}console.log(c(10,3));`

۴) `function c(t,n){t*n}console.log(c(10,3));`

۱۰۹۶ متوسط

کدام جفت «ابزار ← کار اصلی» درست است؟

۱) bundler ← Babel (یکی کردن فایل‌ها)؛ transpiler ← Webpack (تبدیل نسخه‌ی کد)

۲) transpiler ← Babel (تبدیل کد جدید به JavaScript قدیمی‌تر)؛ bundler ← Webpack (یکی کردن ماژول‌ها)

۳) Babel و Webpack هر دو موتور اجرای JavaScript هستند، مثل V8.

۴) Babel ← مدیر بسته (مثل npm)؛ Webpack ← فریم‌ورک سرور (مثل Express)

۱۰۹۷ متوسط

منظور از **module resolution** که bundler انجام می‌دهد چیست؟

۱) پیدا کردن فایل واقعی هر `import` یا `require`؛ چه مسیر نسبی مثل `./math.js` باشد و چه اسم یک پکیج مثل `lodash` در `node_modules`

۲) دانلود پکیج‌ها از registry در زمان ساختن bundle

۳) بررسی نوع متغیرها، مثل کاری که TypeScript انجام می‌دهد

۴) حل کردن تداخل‌ها (conflict) هنگام ادغام کد در git

۱۰۹۸ سخت

کدام عبارت **نادرست** است؟

۱) bundle نهایی را معمولاً با یک تگ `<script>` در HTML بارگذاری می‌کنیم.

۲) bundlerها معمولاً می‌توانند minification و tree-shaking را هم انجام دهند.

۳) bundler جای `npm install` را می‌گیرد و خودش پکیج‌ها را از registry دانلود می‌کند.

۴) source map کمک می‌کند خطاهای کد minify‌شده را در فایل و خط اصلی‌شان ببینیم.

۱۰۹۹ آسان

فایل **source map** (مثلاً `bundle.js.map`) به چه دردی می‌خورد؟

۱) فهرست همه‌ی پکیج‌های نصب‌شده‌ی پروژه را نگه می‌دارد.

۲) نقشه‌ی صفحه‌های سایت را برای موتورهای جست‌وجو می‌سازد.

۳) ترتیب بارگذاری تگ‌های `<script>` را مشخص می‌کند.

۴) هر بخش از کد bundle یا minifyشده را به فایل و خط اصلی‌اش وصل می‌کند تا اشکال‌زدایی (debug) ساده شود.

۱۱۰۰ متوسط

bundler خروجی را در فایل `dist/bundle.js` ساخته است. برای بارگذاری این خروجی در `index.html` چه می‌نویسیم؟

۱) `<link rel="stylesheet" href="dist/bundle.js">`

۲) `<script src="dist/bundle.js"></script>`

۳) برای هر فایل پوشه‌ی `src` یک تگ `<script>` جداگانه، مثل قبل از bundling

۴) `require("dist/bundle.js")` را مستقیم داخل HTML می‌نویسیم.

۱۱۰۱ سخت

بدون هیچ bundler و تنظیم اضافه‌ای، فایل `app.js` زیر را با `<script type="module" src="app.js"></script>` در مرورگر بارگذاری می‌کنیم. پکیج `lodash` با `npm install` در `node_modules` نصب شده است. چه اتفاقی می‌افتد؟

```JavaScript
import _ from "lodash";
console.log(_.chunk([1, 2, 3, 4], 2));
```

۱) مرورگر نمی‌داند `"lodash"` کجاست (فقط آدرس‌هایی را که با `./` ، `../` یا `/` شروع می‌شوند یا آدرس کامل‌اند می‌فهمد و از `node_modules` خبر ندارد)، پس بارگذاری ماژول با خطا متوقف می‌شود.

۲) مرورگر خودکار پوشه‌ی `node_modules` را جست‌وجو می‌کند و آرایه‌ی تکه‌تکه‌شده را چاپ می‌کند.

۳) مرورگر پکیج `lodash` را خودش از registry npm دانلود و اجرا می‌کند.

۴) مرورگر اصلاً از `import` پشتیبانی نمی‌کند، پس هر فایلی که `import` داشته باشد خطا می‌دهد.

۱۱۰۲ آسان

جای خالی را کامل کن:

«اسلاید ۱۲ ابزارهای معروف bundling را این‌طور نام می‌برد: Webpack، Rollup و .....»

۱) Babel ۲) Jest ۳) Parcel ۴) Lodash

۳-۷ پکیج‌های محبوب

منبع: NodeJS & TS، اسلایدهای ۱۳ و ۱۴

درسنامه

در npm میلیون‌ها پکیج هست، ولی چند تا از آن‌ها آن‌قدر معروف‌اند که تقریباً هر برنامه‌نویس JavaScript اسمشان را می‌شناسد. اسلایدهای ۱۳ و ۱۴ ده پکیج محبوب را با یک جمله‌ی کوتاه معرفی می‌کنند (اسلاید ۱۳ کنار متن، لوگوی Lodash، React، Express و socket.io را هم دارد). در تست‌ها معمولاً یک **نیاز** را می‌گویند و می‌پرسند «کدام پکیج؟»؛ پس جمله‌ی هر پکیج را دقیق یاد بگیر.

پکیج	توصیف اسلاید	یعنی…	بیشتر کجا؟
Express	minimalist web framework for building APIs and servers	فریم‌ورک وب کمینه‌گرا برای ساختن API و سرور	سرور (Node.js)
React	library for building interactive UIs with components	کتابخانه‌ی ساختن رابط کاربری تعاملی با کامپوننت‌ها	مرورگر (frontend)
Lodash	utility functions for arrays, objects, and more	توابع کمکی برای آرایه‌ها، شیءها و…	هر دو
Axios	promise-based HTTP client for API requests	کلاینت HTTP مبتنی بر Promise برای درخواست به APIها	هر دو
Socket.io	real-time bidirectional communication for web apps	ارتباط دوطرفه‌ی بلادرنگ برای برنامه‌های وب	سرور + مرورگر
Moment.js / Day.js	date and time manipulation utilities	ابزارهای کار با تاریخ و زمان	هر دو
Jest	testing framework for JavaScript and Node.js	فریم‌ورک تست	ابزار توسعه
Three.js	3D graphics and animations in the browser	گرافیک و انیمیشن سه‌بعدی در مرورگر	مرورگر
Chalk	style and colorize terminal output	رنگی و قالب‌دار کردن خروجی ترمینال	برنامه‌ی کنسولی
Electron	build cross-platform desktop apps with web technologies	ساختن برنامه‌ی دسکتاپ چندسکویی با فناوری‌های وب	دسکتاپ

کدهای زیر فقط برای آشنایی‌اند و برای اجرا باید اول پکیج را نصب کنی (مثلاً `npm install express`)؛ ما همه‌ی آن‌ها را (به‌جز Three.js و Electron که به محیط گرافیکی نیاز دارند) با نسخه‌های فعلی پکیج‌ها اجرا کرده‌ایم.

پکیج‌های اسلاید ۱۳

Express: ساختن سرور و API

API (Application Programming Interface، رابط برنامه‌نویسی) اینجا یعنی مجموعه‌ای از آدرس‌ها روی سرور که برنامه‌های دیگر (مثلاً frontend) با فرستادن درخواست به آن‌ها داده (معمولاً JSON) می‌گیرند یا می‌فرستند. Express «کمینه‌گرا» (minimalist)

است: هسته‌ی کوچکی دارد و بقیه‌ی امکانات را با پکیج‌های دیگر اضافه می‌کنی. نسبت به ماژول خام `http` (درس اول) کار تعریف **route** (مسیر: ترکیب یک آدرس و یک نوع درخواست مثل GET) و فرستادن JSON را خیلی ساده می‌کند:

server.js

```
const express = require("express");
const app = express();

app.get("/api/hello", (req, res) => {
  res.json({ message: "Hello from Express!" }); // send JSON to the client
});

app.listen(3000, () => console.log("Server running on port 3000"));
```

با باز کردن `http://localhost:3000/api/hello` جواب `{"message":"Hello from Express!"}` برمی‌گردد.

React: رابط کاربری با کامپوننت

component (کامپوننت یا مؤلفه) یک تکه‌ی مستقل و قابل استفاده‌ی دوباره از رابط کاربری است؛ مثل یک دکمه، یک کارت محصول یا یک منو. در React هر کامپوننت یک تابع است که ظاهر آن تکه را برمی‌گرداند. نحوی که شبیه HTML داخل JavaScript است **JSX** نام دارد و قبل از اجرا با یک transpiler (مثل Babel) به JavaScript معمولی تبدیل می‌شود:

JavaScript

```
function Greeting({ name }) {
  return <h1>Hello, {name}!</h1>;
}

// Used like an HTML tag:
<Greeting name="Sara" />
```

React یک **کتابخانه** (library) است، نه سرور؛ در مرورگر اجرا می‌شود و صفحه را با تغییر داده‌ها خودکار به‌روز می‌کند.

Lodash: جعبه‌ابزار توابع کمکی

Lodash ده‌ها تابع آماده برای کارهای رایج با آرایه‌ها، شیءها، رشته‌ها و... دارد و طبق قرارداد با اسم `_` (زیرخط) استفاده می‌شود:

JavaScript

```
const _ = require("lodash");

console.log(_.chunk([1, 2, 3, 4, 5], 2)); // [ [ 1, 2 ], [ 3, 4 ], [ 5 ] ]
console.log(_.uniq([1, 1, 2, 3, 3]));     // [ 1, 2, 3 ]

const original = { user: { name: "Ali" } };
const copy = _.cloneDeep(original); // a deep copy: nested objects are copied too
copy.user.name = "Sara";
console.log(original.user.name);    // Ali
```

Axios: درخواست HTTP با Promise

HTTP client کدی است که از طرف برنامه به سرورها درخواست می‌فرستد. Axios **promise-based** است، یعنی هر درخواست یک Promise برمی‌گرداند و می‌شود با `then` یا `async/await` از آن استفاده کرد. هم در مرورگر کار می‌کند و هم در Node.js:

JavaScript

```
const axios = require("axios");

async function loadUsers() {
  try {
    const response = await axios.get("https://api.example.com/users");
    console.log(response.data); // the JSON body, already parsed
  } catch (error) {
    console.log("Request failed:", error.message);
  }
}

loadUsers();
```

Axios در برابر fetch

`fetch` داخلی مرورگر و Node.js هم Promise برمی‌گرداند، ولی دو فرق مهم با Axios دارد: بدنه‌ی JSON را باید خودت با `await res.json()` بخوانی (در Axios آماده در `response.data` است)، و برای پاسخ‌های خطا مثل Promise 404 را **رد نمی‌کند** (فقط `res.ok` برابر `false` می‌شود). Axios برای چنین پاسخی خطای `Request failed with status code 404` می‌دهد.

Socket.io: ارتباط دوطرفه و بلادرنگ

در HTTP معمولی همیشه **کلاینت شروع می‌کند**: درخواست می‌فرستد و جواب می‌گیرد؛ سرور نمی‌تواند خودش سر صحبت را باز کند. Socket.io یک اتصال **دوطرفه** (bidirectional) و **بلادرنگ** (real-time: لحظه‌ای و بدون تأخیر محسوس) بین مرورگر و سرور برقرار می‌کند (بر پایه‌ی فناوری WebSocket)؛ هر طرف هر وقت بخواهد می‌تواند پیام بفرستد. کاربردها: چت، اعلان‌های زنده، بازی‌های آنلاین، ویرایش هم‌زمان یک سند.

server.js

```
const { Server } = require("socket.io");
const io = new Server(3000);

io.on("connection", (socket) => {
  socket.on("chat message", (msg) => {
    io.emit("chat message", msg); // send it to everyone
  });
});
```

و در مرورگر (با کتابخانه‌ی کلاینت Socket.io):

client.js

```
const socket = io("http://localhost:3000");
socket.on("chat message", (msg) => console.log("New message:", msg));
socket.emit("chat message", "Hi!"); // New message: Hi!
```

به شباهتش با `EventEmitter` درس «ویژگی‌های Node.js» دقت کن: باز هم `on` و `emit`؛ این بار رویدادها از روی شبکه جابه‌جا می‌شوند.

پکیج‌های اسلاید ۱۴

Moment.js / Day.js: تاریخ و زمان

کار با تاریخ (قالب‌بندی، اضافه کردن چند روز، فاصله‌ی دو تاریخ) با شیء `Date` خام JavaScript دردسر دارد. Moment.js سال‌ها کتابخانه‌ی استاندارد این کار بود؛ امروز خود تیمش آن را پروژه‌ی قدیمی (legacy) در حالت فقط نگه‌داری اعلام کرده است و **Day.js** جایگزین بسیار سبک‌تری با API تقریباً یکسان است:

```JavaScript
const dayjs = require("dayjs");

const exam = dayjs("2026-11-17");
console.log(exam.format("YYYY/MM/DD"));              // 2026/11/17
console.log(exam.add(7, "day").format("YYYY-MM-DD")); // 2026-11-24
console.log(exam.diff(dayjs("2026-11-13"), "day"));   // 4
```

Jest: تست خودکار

test (آزمون خودکار) کدی است که درستیِ کد دیگری را بررسی می‌کند. Jest یک فریم‌ورک تست برای JavaScript و Node.js است. با `test` یک تست تعریف می‌کنی و با `expect(...).toBe(...)` می‌گویی چه نتیجه‌ای انتظار داری:

```sum.test.js
const sum = require("./sum");

test("adds 1 + 2 to equal 3", () => {
  expect(sum(1, 2)).toBe(3);
});
```

Jest را با `npm install -D jest` (یک **devDependency**) نصب می‌کنی، در `package.json` می‌نویسی `"test": "jest"` و با `npm test` اجرا می‌کنی:

```
PASS ./sum.test.js
  ✓ adds 1 + 2 to equal 3 (2 ms)

Tests:       1 passed, 1 total
```

Three.js: سه‌بعدی در مرورگر

Three.js ساختن **گرافیک و انیمیشن سه‌بعدی** را در مرورگر (با فناوری WebGL) ساده می‌کند: یک صحنه (scene)، یک دوربین (camera)، یک رندرکننده (renderer) و اجسام (mesh) می‌سازی:

JavaScript

```
import * as THREE from "three";

const scene = new THREE.Scene();
const camera = new THREE.PerspectiveCamera(75, innerWidth / innerHeight, 0.1, 1000);
const renderer = new THREE.WebGLRenderer();
renderer.setSize(innerWidth, innerHeight);
document.body.appendChild(renderer.domElement); // a <canvas> element

const cube = new THREE.Mesh(
  new THREE.BoxGeometry(1, 1, 1),
  new THREE.MeshBasicMaterial({ color: 0x00ff00 })
);
scene.add(cube);
camera.position.z = 5;

function animate() {
  cube.rotation.y += 0.01; // rotate a little in every frame
  renderer.render(scene, camera);
  requestAnimationFrame(animate);
}
animate();
```

Chalk: رنگی کردن خروجی ترمینال

برای **برنامه‌های کنسولی** (هدف a در اسلاید ۲) خروجی رنگی خیلی خواناتر است. Chalk متن را با کدهای رنگ ترمینال رنگی و قالب‌دار (پررنگ، زیرخط‌دار و…) می‌کند. نسخه‌های جدید Chalk فقط به‌صورت ES Module منتشر می‌شوند، پس با `import` واردش می‌کنیم:

JavaScript

```
import chalk from "chalk";

console.log(chalk.green("Success!"));
console.log(chalk.red.bold("Error: file not found"));
console.log(chalk.blue("Info:"), "server started");
```

Electron: برنامه‌ی دسکتاپ با HTML و CSS و JavaScript

Electron موتور مرورگر **Chromium** (برای نمایش رابط کاربری با HTML و CSS و JavaScript) را با **Node.js** (برای دسترسی به فایل‌ها و سیستم‌عامل) ترکیب می‌کند تا با همان مهارت‌های وب، برنامه‌ی دسکتاپ **چندسکویی** (Windows، macOS، Linux) بسازی. برنامه‌هایی مثل VS Code، Slack و Discord با Electron ساخته شده‌اند:

main.js

```
const { app, BrowserWindow } = require("electron");

app.whenReady().then(() => {
  const win = new BrowserWindow({ width: 800, height: 600 });
  win.loadFile("index.html"); // the UI is a normal web page
});
```

دسته‌بندی: کدام پکیج کجا به کار می‌رود؟

دسته	پکیج‌ها
سمت سرور (backend، در Node.js)	Express، بخش سرور Socket.io
سمت مرورگر (frontend)	React، Three.js، بخش کلاینت Socket.io
هر دو سمت	Axios، Lodash، Moment.js / Day.js
ابزار توسعه (devDependency)	Jest
برنامه‌ی کنسولی (ترمینال)	Chalk
برنامه‌ی دسکتاپ	Electron

نیاز	پکیج
ساختن API یا وب‌سرور	Express
رابط کاربری با تکه‌های قابل استفاده‌ی دوباره	React
تکه‌تکه کردن آرایه، حذف تکراری‌ها، کپی عمیق شیء	Lodash
گرفتن داده از یک API دیگر	Axios
چت یا اعلان زنده که سرور خودش به کاربر بفرستد	Socket.io
اضافه کردن چند روز به یک تاریخ و قالب‌بندی آن	Moment.js / Day.js
نوشتن تست خودکار	Jest
مکعب چرخان سه‌بعدی در صفحه‌ی وب	Three.js
پیام سبز «Success» در ترمینال	Chalk
برنامه‌ی دسکتاپ برای Windows و macOS و Linux	Electron

نکته

برای هر پکیج یک «کلمه‌ی کلیدی» به خاطر بسپار: Express ← سرور، React ← کامپوننت، Lodash ← توابع کمکی (`_`)، Axios ← درخواست HTTP با Promise، Socket.io ← بلادرنگ و دوطرفه، Day.js ← تاریخ، Jest ← تست، Three.js ← سه‌بعدی، Chalk ← رنگ ترمینال، Electron ← دسکتاپ.

دام تستی

- Axios و Socket.io هر دو با شبکه کار می‌کنند، ولی Axios **درخواست و پاسخ** است (همیشه کلاینت شروع می‌کند) و Socket.io **دوطرفه و بلادرنگ** است (سرور هم می‌تواند هر وقت خواست پیام بفرستد).
- React کتابخانه‌ی **رابط کاربری** است، نه سرور و نه برنامه‌ی دسکتاپ؛ برنامه‌ی دسکتاپ با وب کار Electron است.
- Jest ابزار زمان توسعه است و در `devDependencies` می‌نشیند؛ Express و Axios و Socket.io هنگام اجرا لازم‌اند.

تست‌های تمرینی

۱۶ تست · شماره‌ی ۱۱۰۳ تا ۱۱۱۸

۱۱۰۳ آسان

طبق اسلاید ۱۳، **Express** چیست؟

۱) فریم‌ورک وب کمینه‌گرا (minimalist) برای ساختن API و سرور

۲) کتابخانه‌ای برای ساختن رابط کاربری تعاملی با کامپوننت‌ها

۳) فریم‌ورک تست برای JavaScript و Node.js

۴) ابزاری برای رنگی کردن خروجی ترمینال

۱۱۰۴ آسان

برای نوشتن **تست خودکار** برای کد JavaScript و Node.js، کدام پکیج از اسلاید ۱۴ مناسب است؟

۱) Chalk ۲) Axios ۳) Jest ۴) Lodash

۱۱۰۵ آسان

می‌خواهیم در یک برنامه‌ی کنسولی، پیام‌های موفقیت را **سبز** و پیام‌های خطا را **قرمز** در ترمینال نشان دهیم. کدام پکیج مناسب است؟

۱) Three.js ۲) Chalk ۳) React ۴) Electron

۱۱۰۶ متوسط

یک برنامه‌ی **چت** می‌سازیم که در آن پیام هر کاربر باید **همان لحظه** و بدون refresh برای بقیه نمایش داده شود و سرور هم بتواند خودش پیام را به کاربران بفرستد. کدام پکیج از اسلاید ۱۳ برای این کار ساخته شده است؟

۱) Axios ۲) Lodash ۳) Day.js ۴) Socket.io

۱۱۰۷ متوسط

می‌خواهیم با همان HTML و CSS و JavaScript یک برنامه‌ی **دسکتاپ** بسازیم که روی Windows، macOS و Linux اجرا شود. کدام پکیج مناسب است؟

۱) Electron ۲) React ۳) Three.js ۴) Express

۱۱۰۸ متوسط

«**promise-based HTTP client for API requests**» توصیف کدام پکیج در اسلاید ۱۳ است؟

۱) Express ۲) Socket.io ۳) Axios ۴) Day.js

۱۱۰۹ متوسط

کدام جفت «پکیج ← کاربرد» **نادرست** است؟

۱) Three.js ← گرافیک و انیمیشن سه‌بعدی در مرورگر

۲) Lodash ← ارتباط دوطرفه‌ی بلادرنگ برای برنامه‌های وب

۳) React ← ساختن رابط کاربری تعاملی با کامپوننت‌ها

۴) Moment.js / Day.js ← کار با تاریخ و زمان

۱۱۱۰ متوسط

کدام پکیج اساساً یک **فریم‌ورک سمت سرور** برای Node.js است؟

۱) React ۲) Three.js ۳) Chalk ۴) Express

۱۱۱۱ سخت

یک پروژه‌ی سرور با Express ساخته شده، با Axios به APIهای دیگر درخواست می‌فرستد، با Socket.io پیام بلادرنگ می‌فرستد و با Jest تست می‌شود. کدام پکیج باید با `npm install -D` (یعنی در `devDependencies`) نصب شود؟

۱) Jest ۲) Express ۳) Axios ۴) Socket.io

۱۱۱۲ سخت

در کد زیر اسم پکیج با `???` پوشانده شده است. این توابع کمکی (که طبق قرارداد با اسم `_` استفاده می‌شوند) مال کدام پکیج از اسلاید ۱۳ است؟

```JavaScript
const _ = require("???");

console.log(_.chunk([1, 2, 3, 4, 5], 2)); // [ [ 1, 2 ], [ 3, 4 ], [ 5 ] ]
console.log(_.uniq([1, 1, 2, 3, 3]));     // [ 1, 2, 3 ]
```

۱) Axios ۲) React ۳) Lodash ۴) Socket.io

۱۱۱۳ متوسط

کدام عبارت درباره‌ی **React** درست است؟

۱) React یک فریم‌ورک سمت سرور برای ساختن API است.

۲) React رابط کاربری را از کامپوننت‌های قابل استفاده‌ی دوباره می‌سازد.

۳) React یک پایگاه داده برای ذخیره‌ی اطلاعات کاربران است.

۴) React ابزاری برای bundle کردن فایل‌های JavaScript است.

۱۱۱۴ متوسط

می‌خواهیم به تاریخ امتحان **۷ روز اضافه کنیم** و نتیجه را با قالب `YYYY-MM-DD` نمایش دهیم. کدام پکیج برای این کار ساخته شده است؟

۱) Chalk
۲) Jest
۳) Socket.io
۴) Day.js یا Moment.js

۱۱۱۵ سخت

کدام عبارت **نادرست** است؟

۱) در Socket.io ارتباط یک‌طرفه است: فقط کلاینت می‌تواند به سرور پیام بفرستد.
۲) Electron با ترکیب موتور مرورگر Chromium و Node.js، برنامه‌ی دسکتاپ با HTML و CSS و JavaScript می‌سازد.
۳) Axios مبتنی بر Promise است، پس می‌توان آن را با `async/await` استفاده کرد.
۴) Lodash توابع کمکی برای کار با آرایه‌ها، شیءها و… دارد.

۱۱۱۶ سخت

تیمی می‌خواهد: (الف) یک API روی سرور بسازد، (ب) برای کدش تست خودکار بنویسد و (ج) یک ابزار خط فرمان با خروجی رنگی برای ترمینال بسازد. کدام سه‌تایی به ترتیب (الف، ب، ج) مناسب است؟

۱) React، Jest، Three.js
۲) Express، Axios، Electron
۳) Express، Jest، Chalk
۴) Socket.io، Lodash، React

۱۱۱۷ آسان

برای نمایش یک **مکعب سه‌بعدی چرخان** در صفحه‌ی وب، کدام پکیج از اسلاید ۱۴ مناسب است؟

۱) Electron
۲) Chalk
۳) Express
۴) Three.js

۱۱۱۸ متوسط

کد زیر از توابع `test` و `expect` استفاده می‌کند (`sum` تابعی است که دو عدد را جمع می‌کند). این کد برای کدام پکیج نوشته شده و معمولاً چطور اجرا می‌شود؟

```javascript
test("adds 2 + 3", () => {
  expect(sum(2, 3)).toBe(5);
});
```

۱) Chalk؛ با `npm start` اجرا می‌شود.
۲) Jest؛ معمولاً با `npm test` اجرا می‌شود (وقتی در `package.json` نوشته باشیم `"test": "jest"`).
۳) Express؛ با `npm run build` اجرا می‌شود.
۴) React؛ با `npm test` اجرا می‌شود.

۳-۸ مشکلات تایپ در JavaScript و معرفی TypeScript

منبع: NodeJS & TS، اسلایدهای ۱۵ تا ۱۷

درسنامه

تا اینجای کتاب با خودِ JavaScript کار کردیم. در این درس می‌بینیم JavaScript در موضوع **type** (نوع داده؛ مثل عدد، رشته یا boolean) چه ضعف‌هایی دارد و چرا Microsoft زبانی به نام **TypeScript** ساخت. اسلاید ۱۵ فقط صفحه‌ی عنوانِ بخش TypeScript است و درس اصلی از اسلاید ۱۶ شروع می‌شود. این همان هدف سومی است که اسلاید ۲ برای این جلسه گفته بود: حل مشکلات قدیمی تایپ در JavaScript.

> **یک تشبیه ساده**
>
> فرض کن در یک انبار، روی هیچ جعبه‌ای برچسب نیست. هر چیزی را می‌شود در هر جعبه‌ای گذاشت و تا جعبه را باز نکنی (یعنی تا برنامه اجرا نشود) نمی‌فهمی داخلش چیست. JavaScript شبیه همین انبار است. TypeScript روی جعبه‌ها **برچسب** می‌زند و قبل از ارسال بار، یک بازرس همه‌ی برچسب‌ها را چک می‌کند.

۱. دو مفهوم پایه: زمان اجرا و زمان کامپایل

- **runtime** (زمان اجرا): لحظه‌ای که برنامه واقعاً اجرا می‌شود؛ مثلاً وقتی مرورگر یا Node.js کد را خط به خط اجرا می‌کند.
- **compile time** (زمان کامپایل): مرحله‌ای **قبل از اجرا** که یک ابزار به نام **compiler** (کامپایلر؛ برنامه‌ای که کد را می‌خواند، بررسی می‌کند و به شکل دیگری تبدیل می‌کند) روی کد کار می‌کند. خطایی که در این مرحله پیدا شود، هیچ‌وقت به دست کاربر نمی‌رسد.

بر همین اساس زبان‌ها از نظر نوع‌دهی دو دسته‌اند:

	dynamic typing (نوع‌دهی پویا)	static typing (نوع‌دهی ایستا)
نوع به چه چیزی تعلق دارد؟	به **مقدار**؛ یک متغیر هر بار می‌تواند مقداری با نوع دیگر بگیرد	به **متغیر**؛ نوع متغیر از قبل معلوم و ثابت است
خطای نوع کی پیدا می‌شود؟	در زمان اجرا (runtime)	در زمان کامپایل، قبل از اجرا
نمونه	JavaScript	TypeScript، Java، C#

۲. مشکلات تایپ در JavaScript (اسلاید ۱۶)

اسلاید ۱۶ پنج مشکل را نام می‌برد. بیا هر کدام را با مثال ببینیم.

مشکل ۱: نوع‌دهی پویا (dynamically typed)

در JavaScript نوعِ یک متغیر ثابت نیست و **در زمان اجرا می‌تواند عوض شود:**

```JavaScript
let x = 5;              // x holds a number
console.log(typeof x); // number
x = "hello";            // now a string: no complaint
x = true;               // now a boolean
console.log(typeof x); // boolean
```

این آزادی در برنامه‌ی کوچک راحت است، اما در برنامه‌ی بزرگ نمی‌دانی یک متغیر در لحظه‌ی استفاده چه نوعی دارد.

مشکل ۲: تبدیل نوع خودکار (type coercion)

وقتی دو مقدار با نوع متفاوت با هم ترکیب می‌شوند، JavaScript بی‌صدا یکی را به نوع دیگر تبدیل می‌کند (جزئیات کامل این قاعده‌ها را در فصل اول دیدیم). مثال خود اسلاید: `"5" + 2 → "52"`؛ علامت `→` یعنی «حاصلش می‌شود».

```JavaScript
console.log("5" + 2); // 52 (a string: "5" + "2")
console.log("5" - 2); // 3  (a number: "5" becomes 5)

function add(a, b) {
  return a + b;
}
console.log(add(2, 3));   // 5
console.log(add("2", 3)); // 23  <- a hidden bug, but no error!
```

مثلاً مقداری که از یک فیلد فرم HTML خوانده می‌شود همیشه رشته است؛ اگر آن را به این تابع بدهی، به‌جای جمع، دو مقدار به هم چسبانده می‌شوند و هیچ خطایی هم نمی‌بینی.

مشکل ۳: باگ‌های پنهان نوع در کدهای بزرگ

در پروژه‌ای با صدها فایل و چند برنامه‌نویس، هیچ‌چیز تضمین نمی‌کند که یک تابع با داده‌ی درست صدا زده شود. حتی یک غلط املایی ساده خطا نمی‌دهد؛ فقط `undefined` برمی‌گرداند و باگ جای دیگری از برنامه خودش را نشان می‌دهد. به همین دلیل نگه‌داری (maintain) کدهای بزرگ JavaScript سخت است:

```JavaScript
const user = { name: "Ali" };
console.log(user.nmae); // undefined (a typo, but no error)
```

مشکل ۴: نبودن بررسی‌های زمان کامپایل (compile-time checks)

JavaScript قبل از اجرا کد را از نظر نوع بررسی نمی‌کند؛ پس بسیاری از خطاها **فقط وقتی همان خط اجرا شود** پیدا می‌شوند:

```JavaScript
function shout(msg) {
  return msg.toUppercase(); // typo: the real method is toUpperCase
}
console.log("app started");
console.log(shout("hi"));   // TypeError: msg.toUppercase is not a function
```

خط `app started` بی‌مشکل چاپ می‌شود و برنامه تازه در خط آخر از کار می‌افتد. اگر `shout` فقط در یک حالت نادر صدا زده شود، این باگ ممکن است ماه‌ها پنهان بماند و یک روز برای کاربر واقعی رخ دهد.

مشکل ۵: رفتار ناسازگار در موتورها و نسخه‌های مختلف

کد JavaScript روی موتورهای مختلف (مثلاً V8 در Chrome و Node.js، SpiderMonkey در Firefox و JavaScriptCore در Safari) و نسخه‌های مختلفِ آن‌ها اجرا می‌شود. قابلیتی که در نسخه‌ی جدید هست ممکن است در مرورگر قدیمی نباشد و گاهی حتی رفتار یک تابع فرق کرده است؛ مثلاً در مرورگرهای خیلی قدیمی `parseInt("08")` به‌جای ۸ عدد ۰ را برمی‌گرداند. این تفاوت‌ها باگ‌های ظریفی می‌سازند که فقط روی بعضی سیستم‌ها دیده می‌شوند.

۳. معرفی TypeScript (اسلاید ۱۷)

- **سازنده و سال:** شرکت Microsoft در سال ۲۰۱۲ آن را معرفی کرد.
- **طراح:** طراحی آن را **Anders Hejlsberg** رهبری کرد؛ کسی که معمار اصلی (lead architect) زبان‌های **C#** و **Delphi** هم هست.
- **هدف:** اضافه کردن **static typing** (نوع‌دهی ایستا) و **tooling** (ابزارهای کمکی برنامه‌نویسی) به JavaScript برای پروژه‌های بزرگ.
- **superset تدریجی و اختیاری** (gradual, optional superset): TypeScript یک «ابرمجموعه»ی JavaScript است و می‌شود آن را کم‌کم وارد یک پروژه کرد.
- **محبوبیت:** خیلی زود در نرم‌افزارهای سازمانی (enterprise) و وب‌اپلیکیشن‌های پیچیده محبوب شد؛ برای نمونه ویرایشگر VS Code و فریم‌ورک Angular با TypeScript نوشته شده‌اند.
- **مسیر اجرا:** TypeScript می‌نویسی ← به JavaScript تبدیل می‌شود ← هر جایی که JavaScript اجرا شود (مرورگر، Node.js و ...) اجرا می‌شود.

superset یعنی چه؟

یعنی هر کد JavaScript از نظر **syntax** (نحو؛ قواعد نوشتن کد) یک کد TypeScript هم هست. TypeScript چیزی از JavaScript کم نمی‌کند، فقط **نوع‌ها** را به آن اضافه می‌کند: TypeScript = JavaScript + types. نوع با یک **type annotation** (برچسب نوع؛ دونقطه و نام نوع بعد از اسم متغیر) نوشته می‌شود:

```TypeScript
let age: number = 25; // ": number" is a type annotation
age = 26;             // OK
age = "twenty";       // Error TS2322: Type 'string' is not assignable to type 'number'.
```

اما «نحوِ معتبر داشتن» با «بی‌خطا بودن» فرق دارد. کد زیر هیچ نوعی ندارد و یک JavaScript کاملاً درست است که در Node.js بدون خطا اجرا می‌شود؛ با این حال tsc برایش خطا گزارش می‌کند، چون از روی مقدار اولیه فهمیده `count` عدد است:

```TypeScript
let count = 0;     // plain JavaScript: no types written
count = count + 1; // OK
count = "zero";    // Error TS2322: Type 'string' is not assignable to type 'number'.
```

پس **هر کد JavaScript نحوِ معتبر TypeScript را دارد، ولی ممکن است tsc در آن خطای نوع پیدا کند.**

تدریجی و اختیاری یعنی چه؟

- **اختیاری (optional):** نوشتن نوع اجباری نیست؛ هر جا ننویسی، TypeScript نوع را از روی مقدار تشخیص می‌دهد (به این کار type inference می‌گویند و در درس بعد کامل می‌آید).
- **تدریجی (gradual):** لازم نیست کل پروژه را یک‌جا بازنویسی کنی؛ می‌توانی فایل‌ها را یکی‌یکی از `.js` به `.ts` تغییر نام بدهی و کم‌کم نوع اضافه کنی.

tooling یعنی چه؟

چون ویرایشگر (مثلاً VS Code) نوع هر متغیر را می‌داند، می‌تواند:

- هنگام تایپ، متدها و ویژگی‌های درست را پیشنهاد بدهد (autocomplete)؛
- خطاها را همان لحظه زیر کد خط قرمز بکشد، قبل از اینکه برنامه را اجرا کنی؛
- نام یک متغیر یا تابع را در کل پروژه با خیال راحت عوض کند (rename / refactor) و با یک کلیک به محل تعریف هر چیز برود (go to definition).

۴. کامپایلر tsc: از TypeScript تا JavaScript

مرورگرها TypeScript را نمی‌فهمند. ابزاری به نام (TypeScript Compiler) **tsc** فایل `.ts` را بررسی می‌کند و به فایل `.js` تبدیل می‌کند:

app.ts

```ts
let age: number = 25;
function greet(name: string): string {
  return "Hello, " + name;
}
console.log(greet("Ali"), age);
```

Terminal

```
npm install -D typescript   # install the compiler in the project
npx tsc app.ts              # check app.ts and create app.js
node app.js                 # prints: Hello, Ali 25
```

app.js

```js
"use strict";
let age = 25;
function greet(name) {
    return "Hello, " + name;
}
console.log(greet("Ali"), age);
```

در خروجی **هیچ اثری از نوع‌ها نیست**؛ به این کار **type erasure** (پاک شدن نوع‌ها) می‌گویند. نوع‌ها فقط برای کامپایلر و ویرایشگرند: هیچ بررسی‌ای در زمان اجرا به کد اضافه نمی‌شود و برنامه نه سریع‌تر می‌شود و نه کندتر. (خط `"use strict";` را خود tsc در حالت strict اضافه می‌کند.)

خواندن پیام خطای tsc

اگر در فایل خطای نوع باشد، tsc چیزی شبیه این چاپ می‌کند:

```
app.ts(2,1): error TS2322: Type 'string' is not assignable to type 'number'.
```

یعنی در فایل `app.ts`، خط ۲، ستون ۱، خطایی با کد `TS2322` هست: «نوع string را نمی‌شود به نوع number نسبت داد». لازم نیست کد خطاها را حفظ کنی؛ در درس‌های بعد فقط برای آشنایی کنار مثال‌ها می‌آیند.

فایل tsconfig.json و حالت strict

در یک پروژه‌ی واقعی، تنظیمات کامپایلر در فایلی به نام `tsconfig.json` نوشته می‌شود. دستور `npx tsc --init` این فایل را با تنظیمات پیشنهادی می‌سازد که یکی از آن‌ها `"strict": true` است. بعد از آن کافی است فقط `npx tsc` (بدون نام فایل) را اجرا کنی تا همه‌ی فایل‌های پروژه طبق همین تنظیمات کامپایل شوند. بخشی از این فایل:

tsconfig.json

```ts
{
  "compilerOptions": {
    "target": "esnext",
    "strict": true
  }
}
```

حالت **strict** (سخت‌گیرانه) چند بررسی مهم را با هم روشن می‌کند؛ مثلاً اجازه نمی‌دهد پارامتر یک تابع بدون نوع بماند یا `null` در متغیری از نوع `number` قرار بگیرد. **در این کتاب همه‌جا فرض می‌کنیم strict روشن است** (همان پیش‌فرض `tsc --init`) و همه‌ی مثال‌ها و تست‌ها با همین فرض بررسی شده‌اند.

۵. TypeScript چطور مشکلات اسلاید ۱۶ را حل می‌کند؟

مشکل در JavaScript	پاسخ TypeScript
متغیر در زمان اجرا نوع عوض می‌کند	نوع متغیر ثابت می‌ماند؛ `count = "zero"` خطای کامپایل دارد
تبدیل نوع خودکار و نتایج عجیب	عملیات مشکوک مثل `"5" - 2`، `true + true` و `null + 1` گزارش می‌شوند
باگ‌های پنهان در کدهای بزرگ	غلط‌های املایی و فراخوانی‌های اشتباه قبل از اجرا پیدا می‌شوند؛ نوع‌ها مثل مستندات عمل می‌کنند
خطاها فقط در زمان اجرا دیده می‌شوند	tsc و ویرایشگر خطا را قبل از اجرا نشان می‌دهند
تفاوت موتورها و نسخه‌ها	با تنظیم `target` خروجی برای نسخه‌ی موردنظر JavaScript ساخته می‌شود

TypeScript

```
const a = "5" + 2;     // OK: type string (concatenation is allowed)
const b = "5" - 2;     // Error TS2362: The left-hand side of an arithmetic operation must be of type 'any', 'number', 'bigint' or an enum type.
const c = true + true; // Error TS2365: Operator '+' cannot be applied to types 'boolean' and 'boolean'.
const d = null + 1;    // Error TS18050: The value 'null' cannot be used here.
```

و همان باگ تابع `shout` این بار **قبل از اجرا** پیدا می‌شود، حتی اگر `shout` هیچ‌وقت صدا زده نشود:

TypeScript

```
function shout(msg: string): string {
  return msg.toUppercase(); // Error TS2551: Property 'toUppercase' does not exist on type 'string'. Did you mean 'toUpperCase'?
}
```

نکته

نوع‌ها فقط برای کامپایلر و ویرایشگرند. بعد از کامپایل هیچ اثری از آن‌ها در JavaScript نمی‌ماند؛ پس TypeScript نه برنامه را سریع‌تر می‌کند و نه در زمان اجرا نوع‌ها را چک می‌کند. تمام کمکش **قبل از اجرا** است.

دام تستی

- `"5" + 2` در TypeScript **خطا نیست**! چون چسباندن رشته کار مجازی است، tsc آن را قبول می‌کند و نوع حاصل را `string` می‌داند. اما `"5" - 2` خطای کامپایل دارد.
- superset یعنی هر کد JavaScript نحوِ معتبر TypeScript را دارد؛ **نه** اینکه حتماً بدون خطای نوع کامپایل شود.
- tsc به‌طور پیش‌فرض حتی وقتی خطای نوع پیدا کند، باز هم فایل `.js` را می‌سازد و فقط خطا را گزارش می‌کند.

بیشتر بدانیم

- اگر بخواهی در صورت وجود خطا هیچ فایل `.js` ساخته نشود، گزینه‌ی `noEmitOnError` را در `tsconfig.json` روشن کن.
- با گزینه‌ی `target` تعیین می‌کنی خروجی برای کدام نسخه‌ی JavaScript ساخته شود؛ مثلاً با `"target": "es2015"` عملگر جدید `??` به یک عبارت شرطی معمولی تبدیل می‌شود تا موتورهای قدیمی‌تر هم آن را بفهمند.
- نسخه‌های جدید Node.js (مثلاً نسخه‌ی ۲۴) فایل `.ts` را مستقیم هم اجرا می‌کنند (`node app.ts`)، اما فقط نوع‌ها را پاک می‌کنند و **هیچ بررسی نوعی انجام نمی‌دهند**؛ برای بررسی نوع همچنان tsc لازم است.

جمع‌بندی

JavaScript نوع‌دهی پویا دارد، تبدیل نوع خودکار انجام می‌دهد و خطاهای نوع را فقط در زمان اجرا نشان می‌دهد. TypeScript (Microsoft، سال ۲۰۱۲، با طراحی Anders Hejlsberg) یک superset تدریجی و اختیاری از JavaScript است که static typing و tooling را اضافه می‌کند. کد TypeScript با tsc بررسی و به JavaScript ساده تبدیل می‌شود، نوع‌ها در خروجی پاک می‌شوند و نتیجه هر جا که JavaScript اجرا شود، اجرا می‌شود.

تست‌های تمرینی

۲۱ تست · شماره‌ی ۱۱۱۹ تا ۱۱۳۹

۱۱۱۹ آسان

TypeScript در چه سالی و توسط کدام شرکت معرفی شد؟

۱) Google، سال ۲۰۰۹
۲) Facebook، سال ۲۰۱۲
۳) Microsoft، سال ۲۰۱۲
۴) Mozilla، سال ۲۰۱۵

۱۱۲۰ آسان

در جای خالی کدام گزینه قرار می‌گیرد؟ «طراحی TypeScript را Anders Hejlsberg رهبری کرد؛ کسی که معمار اصلی زبان‌های ..... است.»

۱) Java و Kotlin
۲) C# و Delphi
۳) Python و Ruby
۴) C و C++

۱۱۲۱ آسان

جمله‌ی «JavaScript یک زبان **dynamically typed** است» یعنی چه؟

۱) نوع متغیرها قبل از اجرا توسط کامپایلر بررسی می‌شود.
۲) در JavaScript مقدارها اصلاً نوع ندارند.
۳) نوع یک متغیر فقط با توابع تبدیل مثل `Number()` قابل تغییر است.
۴) نوع به مقدار تعلق دارد و یک متغیر می‌تواند در زمان اجرا مقدارهایی با نوع‌های مختلف بگیرد.

۱۱۲۲ آسان

در JavaScript حاصل `"5" + 2` و `"5" - 2` به ترتیب کدام است؟ (دو حاصل در قالب یک آرایه نوشته شده‌اند.)

۱) `["52", 3]`
۲) `[7, 3]`
۳) `["52", "3"]`
۴) `["7", 3]`

۱۱۲۳ آسان

«TypeScript یک **superset** (ابرمجموعه) از JavaScript است.» منظور از این جمله کدام است؟

۱) هر کد JavaScript از نظر نحو (syntax) یک کد TypeScript هم هست و TypeScript نوع‌ها را به آن اضافه می‌کند.

۲) TypeScript زبانی کاملاً جدا با نحو متفاوت است که باید آن را از صفر یاد گرفت.

۳) هر کد TypeScript بدون هیچ تغییری مستقیماً در مرورگر اجرا می‌شود.

۴) TypeScript بخش کوچکی از JavaScript است که قسمت‌های خطرناک آن حذف شده است.

۱۱۲۴ آسان

دستور `npx tsc --init` چه کاری انجام می‌دهد؟

۱) TypeScript را روی سیستم نصب می‌کند.

۲) همه‌ی فایل‌های `.ts` پروژه را اجرا می‌کند.

۳) فایل `tsconfig.json` را با تنظیمات پیشنهادی (از جمله `"strict": true`) می‌سازد.

۴) فایل `package.json` پروژه را می‌سازد.

۱۱۲۵ متوسط

خروجی کد JavaScript زیر در Node.js کدام است؟

```JavaScript
let data = 10;
console.log(typeof data);
data = "ten";
console.log(typeof data);
data = data + 5;
console.log(data);
```

۱) `number` `number` `15`

۲) `number` `string` `ten5`

۳) `number` `Uncaught TypeError`

۴) `number` `string` `NaN`

۱۱۲۶ متوسط

کدام خط از کد JavaScript زیر باعث خطا می‌شود؟

```JavaScript
1  const user = { name: "Ali" };
2  console.log(user.nmae);
3  console.log(user.name.toUppercase());
4  console.log("done");
```

۱) خط ۲

۲) خط ۳

۳) خط ۴

۴) هیچ خطایی رخ نمی‌دهد

۱۱۲۷ سخت

خروجی کد JavaScript زیر در Node.js چیست؟

```javascript
function shout(msg) {
  return msg.toUppercase();
}
console.log("start");
if (Math.random() > 2) {
  console.log(shout("hi"));
}
console.log("end");
```

۱) `start` `Uncaught TypeError`
۲) `Uncaught SyntaxError`
۳) `start` `end`
۴) `start` `HI` `end`

۱۱۲۸ متوسط

کد TypeScript زیر را با tsc بررسی می‌کنیم. کدام خط خطای کامپایل دارد؟

```typescript
function shout(msg: string): string {
  return msg.toUppercase();
}
console.log("start");
if (Math.random() > 2) {
  console.log(shout("hi"));
}
console.log("end");
```

۱) خط ۲
۲) خط ۵
۳) خط ۶
۴) هیچ خطایی رخ نمی‌دهد

۱۱۲۹ سخت

در TypeScript (حالت strict) کدام دستور **بدون خطا** کامپایل می‌شود؟

۱) `const b = "5" - 2;`
۲) `const c = true + true;`
۳) `const d = null + 1;`
۴) `const a = "5" + 2;`

۱۱۳۰ متوسط

هر چهار کد زیر در JavaScript بدون خطا اجرا می‌شوند. tsc برای کدام‌یک خطای نوع گزارش می‌کند؟

۱) `let n = 5; n = n + 1;`
۲) `let s = "a"; s = s + 1;`
۳) `const list = [1, 2]; list.push(3);`
۴) `let flag = true; flag = "yes";`

۱۱۳۱ متوسط

کدام گزینه درباره‌ی «تدریجی و اختیاری» (gradual, optional) بودن TypeScript **نادرست** است؟

۱) برای استفاده از TypeScript باید کل پروژه یک‌جا بازنویسی شود و هیچ فایل JavaScript نباید باقی بماند.

۲) می‌توان یک پروژه‌ی JavaScript را فایل به فایل به TypeScript تبدیل کرد.

۳) نوشتن type annotation اجباری نیست و TypeScript نوع بسیاری از متغیرها را خودش تشخیص می‌دهد.

۴) کد TypeScript در نهایت به JavaScript تبدیل می‌شود و هر جا JavaScript اجرا شود، اجرا می‌شود.

۱۱۳۲ متوسط

کدام گزینه با مطالب اسلاید ۱۷ درباره‌ی TypeScript **مطابقت ندارد**؟

۱) خیلی زود در نرم‌افزارهای سازمانی (enterprise) و وب‌اپلیکیشن‌های پیچیده محبوب شد.

۲) TypeScript می‌نویسی، به JavaScript تبدیل می‌شود و هر جا JavaScript اجرا شود، اجرا می‌شود.

۳) فقط برای برنامه‌های کوچک و ساده طراحی شده و در پروژه‌های بزرگ کاربردی ندارد.

۴) برای اضافه کردن static typing و tooling به JavaScript ساخته شد.

۱۱۳۳ متوسط

TypeScript برای اضافه کردن static typing و **tooling** به JavaScript ساخته شد. کدام مورد از مزایای این tooling **نیست**؟

۱) پیشنهاد خودکار متدها و ویژگی‌ها هنگام تایپ (autocomplete)

۲) اجرای سریع‌تر برنامه در مرورگر به خاطر دانستن نوع‌ها

۳) نمایش خطا در ویرایشگر، پیش از اجرای برنامه

۴) تغییر نام امن یک تابع در همه‌ی فایل‌های پروژه

۱۱۳۴ متوسط

کدام گزینه جزو «مشکلات تایپ در JavaScript» که در اسلاید ۱۶ آمده **نیست**؟

۱) متغیرها می‌توانند در زمان اجرا نوع عوض کنند.

۲) تبدیل نوع خودکار نتایج غیرمنتظره می‌دهد؛ مثل `"5" + 2`.

۳) نبودن بررسی‌های زمان کامپایل باعث می‌شود بسیاری از خطاها فقط هنگام اجرا دیده شوند.

۴) اجرای تک‌نخی (single-threaded) باعث می‌شود محاسبات سنگین، اجرای برنامه را متوقف کنند.

۱۱۳۵ متوسط

کدام مثال، مشکل «رفتار ناسازگار در موتورها و نسخه‌های مختلف» (بولت آخر اسلاید ۱۶) را نشان می‌دهد؟

۱) حاصل `"5" + 2` در JavaScript رشته‌ی `"52"` است.

۲) متغیری که اول عدد بوده، بعداً رشته می‌شود و JavaScript اعتراضی نمی‌کند.

۳) غلط املایی `user.nmae` به‌جای خطا، فقط `undefined` برمی‌گرداند.

۴) در مرورگرهای خیلی قدیمی `parseInt("08")` عدد ۰ را برمی‌گرداند، ولی در موتورهای امروزی عدد ۸ را.

۱۱۳۶ سخت

فایل `app.ts` یک خطای نوع دارد. در پوشه‌ی پروژه فایل `tsconfig.json` نیست و هیچ تنظیم خاصی (مثل `noEmitOnError`) هم فعال نشده است. با اجرای `npx tsc app.ts` چه اتفاقی می‌افتد؟

۱) خطا گزارش می‌شود، اما فایل `app.js` باز هم ساخته می‌شود.

۲) خطا گزارش می‌شود و فایل `app.js` ساخته نمی‌شود.

۳) هیچ خطایی گزارش نمی‌شود؛ خطا فقط هنگام اجرای `app.js` دیده می‌شود.

۴) tsc خطا را خودش اصلاح می‌کند و `app.js` درست را می‌سازد.

۱۱۳۷ متوسط

کدام گزینه درباره‌ی رابطه‌ی TypeScript و زمان اجرا **درست** است؟

۱) نوع‌هایی که در TypeScript می‌نویسیم در فایل JavaScript خروجی باقی می‌مانند و مرورگر آن‌ها را بررسی می‌کند.

۲) tsc نوع‌ها را پاک می‌کند و هیچ بررسی‌ای به کد خروجی اضافه نمی‌کند؛ پس در زمان اجرا چیزی جلوی مقدار نادرست را نمی‌گیرد.

۳) چون موتور JavaScript نوع‌ها را از قبل می‌داند، برنامه‌ی TypeScript همیشه سریع‌تر اجرا می‌شود.

۴) مرورگرها فایل `.ts` را مستقیماً و بدون هیچ تبدیلی اجرا می‌کنند.

۱۱۳۸ سخت

کد TypeScript زیر بدون خطا کامپایل می‌شود (نوع `any` یعنی «هر نوعی؛ بررسی نکن»). خروجی اجرای آن چیست؟

```TypeScript
function double(n: number): number {
  return n * 2;
}
const input: any = "21";
console.log(double(input));
```

۱) `2121`

۲) `Uncaught TypeError`

۳) `42`

۴) `NaN`

۱۱۳۹ سخت

tsc در کدام خط کد زیر خطا گزارش می‌کند؟

```TypeScript
1  const qty = 3;
2  const price = 2;
3  const msg = "Total: " + qty * price;
4  const half = msg / 2;
5  console.log(msg, half);
```

۱) خط ۳

۲) خط ۵

۳) خط ۴

۴) هیچ خطایی رخ نمی‌دهد

۳-۹ متغیرها و انواع داده در TypeScript

منبع: NodeJS & TS، اسلاید ۱۸

درسنامه

در درس قبل دیدیم TypeScript همان JavaScript است به‌علاوه‌ی نوع‌ها. حالا وقتش است ببینیم این نوع‌ها را دقیقاً کجا و چطور می‌نویسیم. موضوع اسلاید ۱۸ متغیرهاست. یادآوری: در این کتاب فرض می‌کنیم حالت **strict** (سخت‌گیرانه) روشن است؛ همان پیش‌فرض `tsc --init` .

> **یک تشبیه ساده**
>
> متغیر را مثل یک ظرف با برچسب تصور کن. در JavaScript ظرف‌ها برچسب ندارند و هر چیزی داخلشان می‌رود. در TypeScript روی ظرف برچسب «فقط عدد» یا «فقط متن» می‌خورد و کامپایلر اجازه نمی‌دهد چیز دیگری داخلش بریزی.

۱. let و const با type annotation

طبق اسلاید ۱۸، متغیرها را دقیقاً مثل JavaScript با `let` و `const` تعریف می‌کنیم؛ فقط می‌توانیم (به‌طور اختیاری) بعد از نام متغیر یک دونقطه و نام نوع بنویسیم. به این بخش **type annotation** (برچسب نوع) می‌گویند:

```
let variableName: type = value;
```

مثال خود اسلاید، `let age: number = 25;` ، تضمین می‌کند که `age` همیشه عدد بماند:

```TypeScript
let age: number = 25;
age = 26;   // OK: still a number
age = "26"; // Error TS2322: Type 'string' is not assignable to type 'number'.
```

پیام خطا را این‌طور بخوان: «نوع string را نمی‌شود به نوع number نسبت داد». این رایج‌ترین خطای TypeScript است و کدش `TS2322` است.

۲. نوع‌های پایه

نوع	مقدارهای مجاز	مثال
`number`	همه‌ی عددها: صحیح، اعشاری، منفی و حتی `NaN`	`let price: number = 9.5;`
`string`	متن، با `"…"` یا `'…'` یا `` `…` ``	`let city: string = "Tehran";`
`boolean`	فقط `true` یا `false`	`let isOpen: boolean = true;`
`null` و `undefined`	فقط خودِ همان مقدار	توضیح در بخش ۹

> **یادداشت**
>
> نام نوع‌ها را با حرف **کوچک** بنویس: `number` ، `string` و `boolean` . نوع‌های `Number` و `String` (با حرف بزرگ) مربوط به اشیای پوششی (wrapper) JavaScript‌اند و نباید به‌جای این‌ها به کار بروند.

۳. const: اتصال تغییرناپذیر (immutable binding)

اسلاید ۱۸ می‌گوید `const` یک **immutable binding** (اتصال تغییرناپذیر بین نام و مقدار) می‌سازد و جلوی **reassignment** (انتساب دوباره) را می‌گیرد. در JavaScript این اشتباه در زمان اجرا `TypeError` می‌داد؛ در TypeScript همان موقع کامپایل گزارش می‌شود:

```TypeScript
const PI = 3.14;
PI = 3; // Error TS2588: Cannot assign to 'PI' because it is a constant.
```

دقت کن: `const` فقط خودِ متغیر را قفل می‌کند، **نه محتوای** آرایه یا شیئی که متغیر به آن اشاره می‌کند:

```TypeScript
const list = [1, 2];
list.push(3);  // OK: changing the contents is allowed
list[0] = 10;  // OK
list = [4, 5]; // Error TS2588: Cannot assign to 'list' because it is a constant.
```

هر `const` هم باید هنگام تعریف مقدار بگیرد؛ نوشتن `const x: number;` این خطا را دارد: `'const' declarations must be initialized`.

۴. استنتاج نوع (type inference)

آخرین بولت اسلاید ۱۸: اگر نوع را ننویسی، TypeScript **خودش آن را از روی مقدار اولیه تشخیص می‌دهد**. به این کار **type inference** (استنتاج نوع) می‌گویند. پس نوشتن نوع اختیاری است، ولی بررسی نوع همچنان انجام می‌شود:

```TypeScript
let city = "Tehran"; // inferred type: string
city = "Shiraz";     // OK
city = 42;           // Error TS2322: Type 'number' is not assignable to type 'string'.
```

جدول زیر نوع استنتاج‌شده را در حالت‌های مهم نشان می‌دهد:

تعریف	نوع استنتاج‌شده	توضیح
`let x = 5;`	`number`	با `let` نوع «عمومی» انتخاب می‌شود
`const y = 5;`	`5`	با `const` نوع **literal** (خودِ همان مقدار)
`let s = "hi";`	`string`	
`const g = "hi";`	`"hi"`	literal type
`let flag = true;`	`boolean`	
`let nums = [1, 2, 3];`	`number[]`	آرایه‌ی عددی
`const arr = [1, 2, 3];`	`number[]`	`const` روی آرایه هم `number[]` می‌دهد؛ محتوا قابل تغییر است
`let mixed = [1, "a"];`	`(string | number)[]`	آرایه‌ای که هر خانه‌اش عدد یا رشته است
`const user = { name: "Ali" };`	`{ name: string }`	ویژگی‌های شیء هم نوع عمومی می‌گیرند

چرا `const y = 5` نوع `5` می‌گیرد؟ چون `y` هرگز عوض نمی‌شود، TypeScript دقیق‌ترین نوع ممکن یعنی خودِ مقدار `5` را برایش در نظر می‌گیرد. اما `let x = 5` بعداً می‌تواند عدد دیگری بگیرد، پس نوعش `number` است.

کِی نوع را خودمان بنویسیم؟ (۱) وقتی متغیر را **بدون مقدار اولیه** تعریف می‌کنیم؛ (۲) وقتی نوعی **بازتر** از مقدار اولیه می‌خواهیم (مثلاً union). اگر متغیری بدون مقدار تعریف شود و پیش از مقداردهی خوانده شود، خطا می‌گیریم:

```TypeScript
let later: number;  // declared without a value
console.log(later); // Error TS2454: Variable 'later' is used before being assigned.
later = 10;
```

۵. آرایه‌ها

نوع آرایه را به دو شکل می‌نویسیم که **کاملاً معادل‌اند**: `number[]` و `Array<number>`.

```TypeScript
let scores: number[] = [18, 20];
let tags: Array<string> = ["ts", "js"];
scores.push(19.5); // OK
tags.push("node"); // OK
scores.push("19"); // Error TS2345: Argument of type 'string' is not assignable to parameter of type 'number'.
```

۶. Union type (نوع اجتماع)

طبق اسلاید ۱۸، TypeScript از **union type** (نوع اجتماع؛ «این یا آن») پشتیبانی می‌کند: `let id: number | string;` یعنی `id` می‌تواند عدد **یا** رشته باشد. علامت `|` را «یا» بخوان.

```TypeScript
let id: number | string;
id = 101;     // OK
id = "A-101"; // OK
id = true;    // Error TS2322: Type 'boolean' is not assignable to type 'string | number'.
```

tsc در پیام خطا این union را `string | number` چاپ می‌کند؛ ترتیب اعضای union اهمیتی ندارد.

استفاده از مقدار union: روی یک مقدار union فقط کارهایی مجاز است که برای **همه‌ی** اعضای آن درست باشد. مثلاً `toString()` هم روی عدد هست هم روی رشته، اما `toUpperCase()` فقط مال رشته است:

```TypeScript
function show(id: number | string) {
  console.log(id.toString());    // OK: both types have toString
  console.log(id.toUpperCase()); // Error TS2339: Property 'toUpperCase' does not exist on type
'string | number'.
}
```

راه‌حل، **narrowing** (باریک کردن نوع) است: با یک شرط مثل `typeof` به TypeScript نشان می‌دهیم در هر شاخه کدام نوع را داریم:

```TypeScript
function format(id: number | string): string {
  if (typeof id === "string") {
    return id.toUpperCase(); // here id is a string
  }
  return id.toFixed(1);      // here id is a number
}
console.log(format("a-7")); // A-7
console.log(format(5));     // 5.0
```

union فقط در زمان کامپایل وجود دارد. در زمان اجرا، `typeof` نوع **واقعیِ** مقدار فعلی را برمی‌گرداند (`"number"` یا `"string"`)، نه چیزی مثل `"number | string"`.

> **بیشتر بدانیم**
>
> TypeScript آخرین مقداری را که به یک متغیر union داده‌ای هم دنبال می‌کند. مثلاً بعد از `let id: number | string = 5;` در خط‌های بعدی می‌داند `id` **فعلاً** عدد است و `id.toFixed(1)` را قبول می‌کند، ولی همچنان می‌توانی بعداً رشته در آن بگذاری. برای همین مثال‌های union را با پارامتر تابع می‌نویسیم که مقدارش از قبل معلوم نیست.

۷. Literal type (نوع لفظی)

نوع می‌تواند یک **مقدار مشخص** باشد؛ به آن **literal type** (نوع لفظی) می‌گویند. وقتی چند literal را با `|` ترکیب کنیم، فهرستی از مقدارهای مجاز ساخته‌ایم:

```TypeScript
let role: "admin" | "user" = "user";
role = "admin"; // OK
role = "guest"; // Error TS2322: Type '"guest"' is not assignable to type '"admin" | "user"'.
let dice: 1 | 2 | 3 | 4 | 5 | 6 = 3;
```

نتیجه‌ی جالب: چون `const status = "ok"` نوع `"ok"` دارد، می‌شود آن را در متغیری از نوع `"ok" | "error"` گذاشت؛ اما `let message = "ok"` نوع `string` دارد و نمی‌شود.

۸. any و unknown

- `any` (هر نوعی): بررسی نوع را برای آن مقدار **کاملاً خاموش** می‌کند. هر کاری با آن مجاز است و خطاها دوباره به زمان اجرا برمی‌گردند. فقط به‌عنوان راه فرار موقت (مثلاً هنگام تبدیل تدریجی یک پروژه) از آن استفاده کن.
- `unknown` (ناشناخته): نسخه‌ی امنِ `any` . هر مقداری را می‌پذیرد، اما **تا نوعش را با یک شرط مشخص نکنی** اجازه‌ی استفاده از آن را نمی‌دهد.

```TypeScript
let a: any = "hello";
a.toUpperCase(); // OK: no checking at all
a.foo.bar();     // also accepted by tsc, but crashes at runtime with a TypeError

let u: unknown = "hello";
u.toUpperCase(); // Error TS18046: 'u' is of type 'unknown'.
if (typeof u === "string") {
  console.log(u.toUpperCase()); // OK: narrowed to string
}
```

	`any`	`unknown`
چه مقداری می‌گیرد؟	هر مقداری	هر مقداری
استفاده‌ی مستقیم (صدا زدن متد و ...)	مجاز، بدون هیچ بررسی	ممنوع تا وقتی narrowing نشود
نسبت دادن به متغیر نوع‌دار (`let n: number = …`)	مجاز	خطا (TS2322)
ایمنی	بررسی خاموش است	امن

۹. null و undefined در حالت strict

در حالت strict، `null` و `undefined` نوع‌های جداگانه‌اند و **خودبه‌خود** داخل `number` یا `string` جا نمی‌شوند. اگر متغیری ممکن است خالی باشد، باید صریحاً با union بگویی:

```TypeScript
let n1: number = null;         // Error TS2322: Type 'null' is not assignable to type 'number'.
let n2: number = undefined;    // Error TS2322: Type 'undefined' is not assignable to type
'number'.
let n3: number | null = null;  // OK
let nick: string | undefined;  // OK
```

۱۰. Tuple (چندتایی)

tuple (چندتایی) آرایه‌ای با **طول ثابت** است که نوع هر خانه‌اش جداگانه مشخص شده است:

```typescript
let pair: [string, number] = ["Ali", 20];
pair[0] = "Sara";     // OK: index 0 is a string
pair = [20, "Ali"];   // Error TS2322 (twice): the order of the types is wrong
console.log(pair[2]); // Error TS2493: Tuple type '[string, number]' of length '2' has no element at index '2'.
```

نکته

هر جا مقدار اولیه داری، بگذار TypeScript خودش نوع را تشخیص دهد (`let city = "Tehran";`). نوع را وقتی بنویس که مقدار اولیه نداری یا نوعی بازتر مثل `number | string` می‌خواهی.

دام تستی

- `const` جلوی **انتساب دوباره** را می‌گیرد، نه تغییر محتوا: `const list = [1]; list.push(2);` مجاز است.
- نوع `const y = 5` برابر `5` (literal) است، نه `number`؛ اما `const arr = [1, 2]` باز هم `number[]` است.
- `string | number[]` یعنی «یک رشته **یا** یک آرایه‌ی عددی»، اما `(string | number)[]` یعنی «آرایه‌ای که هر خانه‌اش عدد یا رشته است». پرانتز مهم است!
- `any` بررسی را خاموش می‌کند؛ `unknown` تا قبل از narrowing اجازه‌ی هیچ استفاده‌ای نمی‌دهد.
- در حالت strict، `let n: number = null;` خطاست؛ باید بنویسی `number | null`.

جمع‌بندی

در TypeScript متغیرها را مثل JavaScript با `let` و `const` تعریف می‌کنیم و می‌توانیم با type annotation نوعشان را بنویسیم (`let age: number = 25;`). اگر ننویسیم، TypeScript نوع را از مقدار اولیه استنتاج می‌کند (`let` نوع عمومی می‌گیرد و `const` نوع literal). `const` جلوی انتساب دوباره را می‌گیرد، نه تغییر محتوا. آرایه‌ها `number[]` یا `Array<number>` نوشته می‌شوند. union مثل `number | string` چند نوع را مجاز می‌کند و با `typeof` باریک می‌شود. `any` بررسی را خاموش می‌کند و `unknown` نسخه‌ی امن آن است. در حالت strict، `null` و `undefined` فقط با union مجازند و tuple آرایه‌ای با طول و نوعِ ثابت برای هر خانه است.

تست‌های تمرینی

۲۵ تست · شماره‌ی ۱۱۴۰ تا ۱۱۶۴

۱۱۴۰ آسان

در دستور `let age: number = 25;` بخش `: number` چه نام دارد؟

۱) type inference (استنتاج نوع)
۲) type annotation (برچسب نوع)
۳) union type (نوع اجتماع)
۴) type coercion (تبدیل نوع خودکار)

۱۱۴۱ آسان

با فرض تعریف زیر، کدام دستور خطای کامپایل دارد؟

```TypeScript
let age: number = 25;
```

۱) `age = "26";`
۲) `age = 26;`
۳) `age = age + 1;`
۴) `age = 25.5;`

۱۱۴۲ آسان

tsc در کدام خط کد زیر خطا گزارش می‌کند؟

```TypeScript
let count = 1;
count = 2;
const PI = 3.14;
PI = 3;
console.log(count, PI);
```

۱) خط ۲
۲) خط ۳
۳) خط ۴
۴) هیچ خطایی رخ نمی‌دهد

۱۱۴۳ متوسط

tsc در کدام خط کد زیر خطا گزارش می‌کند؟

```TypeScript
const list = [1, 2];
list.push(3);
list[0] = 10;
list = [4, 5];
```

۱) خط ۴
۲) خط ۲
۳) خط ۳
۴) هیچ خطایی رخ نمی‌دهد

۱۱۴۴ آسان

با فرض تعریف زیر (مثال اسلاید ۱۸)، کدام دستور خطای کامپایل دارد؟

```TypeScript
let id: number | string;
```

۱) `id = 101;`
۲) `id = "A-101";`
۳) `id = true;`
۴) `id = -5;`

۱۱۴۵ متوسط

tsc در کدام خط کد زیر خطا گزارش می‌کند؟

```TypeScript
let city = "Tehran";
city = "Shiraz";
let year = 1403;
city = year;
```

۱) خط ۲
۲) خط ۳
۳) هیچ خطایی رخ نمی‌دهد
۴) خط ۴

۱۱۴۶ سخت

کدام گزینه **بدون خطا** کامپایل می‌شود؟ (نوع `5` یعنی «فقط و فقط عدد ۵».)

۱) `let x = 5; const five: 5 = x;`
۲) `let s = "hi"; const h: "hi" = s;`
۳) `const y = 5; const five: 5 = y;`
۴) `const n = 5; n = 6;`

۱۱۴۷ متوسط

TypeScript نوع متغیر `x` را در کد زیر چه تشخیص می‌دهد؟

```TypeScript
let x = [1, 2, 3];
```

۱) `[1, 2, 3]`
۲) `any[]`
۳) `number`
۴) `number[]`

۱۱۴۸ آسان

کدام نوع **دقیقاً** با `number[]` معادل است؟

۱) `[number]`
۲) `Array<number>`
۳) `number | []`
۴) `Array[number]`

۱۱۴۹ متوسط

با فرض تعریف‌های زیر، کدام دستور خطای کامپایل دارد؟

```TypeScript
let scores: number[] = [18, 20];
let tags: Array<string> = ["ts"];
```

۱) `scores.push("19");`
۲) `scores.push(19.5);`
۳) `tags.push("node");`
۴) `tags = [];`

۱۱۵۰ سخت

tsc در کدام خط کد زیر خطا گزارش می‌کند؟

```TypeScript
function show(id: number | string) {
  console.log(id.toString());
  console.log(id.toUpperCase());
  if (typeof id === "string") {
    console.log(id.toUpperCase());
  }
}
```

۱) خط ۲
۲) خط ۳
۳) خط ۵
۴) هیچ خطایی رخ نمی‌دهد

۱۱۵۱ متوسط

خروجی کد TypeScript زیر پس از کامپایل و اجرا چیست؟

```TypeScript
function format(value: number | string): string {
  if (typeof value === "number") {
    return value.toFixed(1);
  }
  return value.toUpperCase();
}
console.log(format(5));
console.log(format("ok"));
```

۱) `5` `OK`
۲) `5.0` `ok`
۳) `5.0` `OK`
۴) `5` `ok`

۱۱۵۲ متوسط

با فرض تعریف زیر، کدام دستور خطای کامپایل دارد؟

```TypeScript
let role: "admin" | "user" = "user";
```

۱) `role = "admin";`
۲) `role = "guest";`
۳) `const r: string = role;`
۴) `role = "user";`

۱۱۵۳ متوسط

کدام گزینه درباره‌ی `any` و `unknown` **درست** است؟

۱) هر دو بررسی نوع را به‌طور کامل خاموش می‌کنند.

۲) `unknown` فقط می‌تواند مقدار `undefined` بگیرد.

۳) هر مقداری را می‌توان در هر دو گذاشت؛ اما از مقدار `unknown` فقط پس از مشخص کردن نوعش (مثلاً با `typeof`) می‌شود استفاده کرد، در حالی که `any` هر استفاده‌ای را بدون بررسی می‌پذیرد.

۴) `any` امن‌تر از `unknown` است، چون خطاهای بیشتری را پیدا می‌کند.

۱۱۵۴ متوسط

tsc در کدام خط کد زیر خطا گزارش می‌کند؟

```TypeScript
let a: any = "hello";
let u: unknown = "hello";
console.log(a.toUpperCase());
console.log(u.toUpperCase());
```

۱) خط ۲

۲) خط ۳

۳) خط ۴

۴) هیچ خطایی رخ نمی‌دهد

۱۱۵۵ سخت

کد TypeScript زیر بدون خطا کامپایل می‌شود. خروجی اجرای آن چیست؟

```TypeScript
let data: any = 42;
console.log("before");
data.toUpperCase();
console.log("after");
```

۱) `before` `Uncaught TypeError`

۲) `before` `after`

۳) `before` `42` `after`

۴) `Uncaught TypeError`

۱۱۵۶ متوسط

در حالت strict، کدام تعریف **بدون خطا** کامپایل می‌شود؟

۱) `let c: number | null = null;`

۲) `let a: number = null;`

۳) `let b: string = undefined;`

۴) `let d: boolean = null;`

۱۱۵۷ متوسط

tsc در کدام خط کد زیر خطا گزارش می‌کند؟

```TypeScript
let pair: [string, number] = ["Ali", 20];
pair[0] = "Sara";
pair[1] = 21;
console.log(pair[2]);
```

۱) خط ۲
۲) خط ۴
۳) خط ۳
۴) هیچ خطایی رخ نمی‌دهد

۱۱۵۸ آسان

با فرض تعریف زیر، کدام دستور خطای کامپایل دارد؟

```TypeScript
let user: [string, number];
```

۱) `user = ["Ali", 20];`
۲) `user = ["", 0];`
۳) `user = ["Sara", -1];`
۴) `user = [20, "Ali"];`

۱۱۵۹ سخت

کدام گزینه **بدون خطا** کامپایل می‌شود؟

۱) `let b: (string | number)[] = [1, "x"];`
۲) `let a: string | number[] = [1, "x"];`
۳) `let c: string[] | number[] = [1, "x"];`
۴) `let d: number[] = [1, "x"];`

۱۱۶۰ سخت

tsc در کدام خط کد زیر خطا گزارش می‌کند؟

```TypeScript
let total: number;
let count = 0;
count++;
console.log(total + count);
total = 100;
```

۱) خط ۱
۲) خط ۳
۳) هیچ خطایی رخ نمی‌دهد
۴) خط ۴

۱۱۶۱ سخت

کدام گزینه **نادرست** است؟

۱) در `let x = 5;` نوع استنتاج‌شده‌ی `x` برابر `number` است.

۲) در `const y = 5;` نوع استنتاج‌شده‌ی `y` برابر `5` است.

۳) در `let names = ["Ali", "Sara"];` نوع استنتاج‌شده‌ی `names` برابر `string[]` است.

۴) در `const arr = [1, 2];` چون `arr` با `const` تعریف شده، نمی‌توان با `push` عضوی به آن اضافه کرد.

۱۱۶۲ آسان

در جای خالی کدام نوع قرار گیرد تا کد زیر **بدون خطا** کامپایل شود؟

```TypeScript
let id: ______;
id = 101;
id = "A-101";
```

۱) `number` ۲) `string` ۳) `boolean` ۴) `number | string`

۱۱۶۳ سخت

tsc در کدام خط کد زیر خطا گزارش می‌کند؟

```TypeScript
1  const status = "ok";
2  let state: "ok" | "error" = status;
3  let message = "ok";
4  state = message;
```

۱) خط ۴ ۲) خط ۲

۳) هیچ خطایی رخ نمی‌دهد ۴) خط ۱

۱۱۶۴ متوسط

خروجی کد TypeScript زیر پس از کامپایل و اجرا چیست؟

```TypeScript
let id: number | string = 7;
console.log(typeof id);
id = "seven";
console.log(typeof id);
```

۱) `number | string` `number | string`

۲) `number` `string`

۳) `number` `number`

۴) `string` `string`

۳-۱۰ Interface در TypeScript

منبع: NodeJS & TS، اسلاید ۱۹

درسنامه

تا اینجا برای متغیرهای ساده (عدد، رشته و ...) نوع نوشتیم. اما در برنامه‌های واقعی بیشتر با **object** (شیء) کار می‌کنیم: کاربر، محصول، سفارش. اسلاید ۱۹ ابزار اصلی TypeScript برای توصیف شیءها را معرفی می‌کند: **interface** (رابط؛ قراردادی که شکل یک شیء را مشخص می‌کند).

یک تشبیه ساده

interface مثل یک **فرم استخدام** است: بعضی خانه‌ها اجباری‌اند (نام، کد ملی)، بعضی اختیاری (ایمیل) و برای هر خانه نوع جواب مشخص است (عدد یا متن). هر شیئی که این فرم را درست پر کند، با آن interface «جور» است. خود فرم هیچ کاری انجام نمی‌دهد؛ فقط مشخص می‌کند جواب‌ها چه شکلی باید باشند.

۱. تعریف شکل یک شیء (اسلاید ۱۹)

بولت اول اسلاید: interface **شکل یک شیء** (shape of an object) را تعریف می‌کند؛ یعنی شیء چه **property**هایی (ویژگی‌ها) با چه **type**هایی باید داشته باشد. کد خود اسلاید (فایل `weee.ts`):

weee.ts

```ts
interface User {
  id: number;
  name: string;
  email?: string;
}
```

خط به خط:

- `interface User { … }` یک نوع جدید به نام `User` تعریف می‌کند (نام interfaceها را معمولاً با حرف بزرگ شروع می‌کنیم).
- `id: number;` یعنی هر `User` **باید** ویژگی `id` از نوع عدد داشته باشد.
- `name: string;` یعنی ویژگی اجباری `name` از نوع رشته.
- `email?: string;` علامت `?` یعنی این ویژگی **اختیاری** است: ممکن است نباشد، ولی اگر بود باید رشته باشد.

tsc برای کد اسلاید هیچ خطایی گزارش نمی‌کند و چون این کد فقط یک تعریف نوع است، با اجرا هیچ خروجی‌ای هم چاپ نمی‌شود. حالا از آن استفاده کنیم:

```TypeScript
interface User {
  id: number;
  name: string;
  email?: string;
}
const u1: User = { id: 1, name: "Ali" };                              // OK: email is optional
const u2: User = { id: 2, name: "Sara", email: "sara@example.com" }; // OK
const u3: User = { name: "Reza" };                                    // Error TS2741: Property 'id'
is missing
const u4: User = { id: "5", name: "Navid" };                          // Error TS2322: Type 'string'
is not assignable to type 'number'.
const u5: User = { id: 4, name: "Mina", age: 20 };                    // Error TS2353: 'age' does
not exist in type 'User'
```

سه خطای مهم:

پیام خطا	چه زمانی؟
`Property 'id' is missing …`	یک ویژگی **اجباری** جا افتاده باشد (TS2741)
`Type 'string' is not assignable to type 'number'`	نوع یک ویژگی اشتباه باشد (TS2322)
`Object literal may only specify known properties …`	یک **object literal** (شیئی که همان‌جا با `{ … }` نوشته شده) ویژگی اضافه داشته باشد (TS2353)

interface را می‌شود مثل هر نوع دیگری برای پارامتر تابع هم به کار برد:

```TypeScript
interface User {
  id: number;
  name: string;
  email?: string;
}
function printUser(user: User): void {
  console.log(`${user.id}: ${user.name}`);
}
printUser({ id: 7, name: "Ali" }); // 7: Ali
```

۲. ویژگی اختیاری (optional property)

ویژگی‌ای که با `?` تعریف شود، نوعش در عمل `string | undefined` است، چون ممکن است اصلاً وجود نداشته باشد. پس TypeScript اجازه نمی‌دهد بدون بررسی از آن استفاده کنی:

TypeScript

```ts
interface User {
  id: number;
  name: string;
  email?: string;
}
const u: User = { id: 1, name: "Ali" };
console.log(u.email);          // undefined (just reading it is fine)
console.log(u.email.length);   // Error TS18048: 'u.email' is possibly 'undefined'.
if (u.email !== undefined) {
  console.log(u.email.length); // OK: inside this block email is a string
}
console.log(u.email?.length);  // OK: "?." gives undefined when email is missing
```

عملگر `?.` (optional chaining، زنجیره‌ی اختیاری) یعنی «اگر سمت چپ `undefined` یا `null` بود، ادامه نده و `undefined` برگردان».

۳. ویژگی فقط‌خواندنی (readonly)

با کلمه‌ی `readonly` ویژگی‌ای می‌سازیم که فقط **هنگام ساختن شیء** مقدار می‌گیرد و بعد قابل تغییر نیست:

TypeScript

```ts
interface Point {
  readonly x: number;
  y: number;
}
const p: Point = { x: 1, y: 2 };
p.y = 20; // OK
p.x = 10; // Error TS2540: Cannot assign to 'x' because it is a read-only property.
```

اسلاید ۱۹ می‌گوید ویژگی‌های optional و readonly به ترتیب برای **انعطاف** (flexibility) و **ایمنی** (safety) هستند. `readonly` هم مثل بقیه‌ی نوع‌ها فقط در زمان کامپایل بررسی می‌شود.

فرق `readonly` با `const`: `const` خودِ **متغیر** را قفل می‌کند (نمی‌شود شیء دیگری به آن داد)، اما `readonly` یک **ویژگی** از شیء را قفل می‌کند. حتی اگر شیء در یک متغیر `let` باشد، ویژگی readonly آن قابل تغییر نیست.

۴. متد در interface

interface علاوه بر داده می‌تواند **متدها** را هم توصیف کند؛ یعنی بگوید شیء چه تابعی با چه پارامترها و چه خروجی‌ای دارد:

TypeScript

```ts
interface Greeter {
  greet(name: string): string;        // method signature
  farewell: (name: string) => string; // a property that holds a function
}
const g: Greeter = {
  greet(name) {
    return `Hi ${name}`;
  },
  farewell: (name) => `Bye ${name}`,
};
console.log(g.greet("Ali"));    // Hi Ali
console.log(g.farewell("Ali")); // Bye Ali
```

اینجا لازم نیست نوع `name` را دوباره بنویسیم؛ TypeScript آن را از روی interface می‌فهمد.

۵. گسترش interface با extends

بولت آخر اسلاید ۱۹: interface را می‌شود **extend** (گسترش) داد تا از نوع‌های ساده‌تر، نوع‌های پیچیده‌تر ساخته شود:

```TypeScript
interface User {
  id: number;
  name: string;
  email?: string;
}
interface Admin extends User {
  role: string; // Admin = everything in User + role
}
const a1: Admin = { id: 6, name: "Neda", role: "owner" }; // OK
const a2: Admin = { id: 7, name: "Omid" };                // Error TS2741: Property 'role' is
missing
```

- یک interface می‌تواند چند interface را با هم گسترش دهد: `interface Person extends HasId, HasName { … }` .
- چون `Admin` همه‌ی ویژگی‌های `User` را دارد، هر جا `User` لازم باشد می‌شود یک `Admin` داد؛ اما برعکسش نه، چون `User` ویژگی `role` را ندارد.

۶. پیاده‌سازی interface در کلاس (implements)

بولت دوم اسلاید ۱۹: کلاس‌ها می‌توانند interface را **implement** (پیاده‌سازی) کنند تا ساختارشان تضمین شود. با کلمه‌ی `implements` کلاس قول می‌دهد همه‌ی اعضای اجباری interface را داشته باشد (جزئیات خودِ کلاس‌ها در درس کلاس‌ها می‌آید):

```TypeScript
interface User {
  id: number;
  name: string;
  email?: string;
}
class Customer implements User { // OK: email is optional, so it may be left out
  id = 8;
  name = "Hamed";
}
class Broken implements User {   // Error TS2420: Class 'Broken' incorrectly implements interface
'User'.
  id = 9;
}
```

- کلاس می‌تواند **عضو اضافه** هم داشته باشد (مثلاً `phone`)؛ مهم این است که چیزی از interface کم نباشد.
- نوع اعضا هم باید درست باشد؛ اگر در کلاس `id = "x"` بنویسی، خطای TS2416 می‌گیری.
- یک کلاس می‌تواند چند interface را با هم پیاده‌سازی کند: `class Admin implements HasId, HasName { … }` .
- کلاس نمی‌تواند interface را `extends` کند؛ tsc می‌گوید:
  `Cannot extend an interface 'User'. Did you mean 'implements'?`

۷. structural typing: شکل مهم است، نه نام

TypeScript نوع‌ها را با **شکلشان** مقایسه می‌کند، نه با نامشان؛ به این روش **structural typing** (نوع‌دهی ساختاری) می‌گویند. هر شیئی که ویژگی‌های لازم را با نوع درست داشته باشد، `User` حساب می‌شود؛ حتی اگر هیچ‌جا اسمی از `User` نیاورده باشد.

اما یک استثنا هست: **excess property check** (بررسی ویژگی اضافه). اگر یک object literal را **مستقیم** به نوع `User` بدهی و ویژگی اضافه داشته باشد، TypeScript احتمال می‌دهد غلط تایپی کرده‌ای و خطا می‌دهد. ولی اگر همان شیء اول در یک متغیر جدا گذاشته شده باشد، فقط ویژگی‌های لازم بررسی می‌شوند:

TypeScript

```
interface User {
  id: number;
  name: string;
  email?: string;
}
const extra = { id: 10, name: "Leila", age: 30 };
const u6: User = extra;                              // OK: extra has id and name
const u7: User = { id: 11, name: "Omid", age: 25 }; // Error TS2353: 'age' does not exist in type
'User'
```

نوع‌ها چیزی را از شیء **حذف نمی‌کنند**: در زمان اجرا `age` هنوز در شیء هست؛ فقط TypeScript اجازه نمی‌دهد از طریق متغیر `User` به آن دسترسی داشته باشی:

TypeScript

```
interface User {
  id: number;
  name: string;
}
const data = { id: 10, name: "Leila", age: 30 };
const u: User = data;
console.log(u); // { id: 10, name: 'Leila', age: 30 }
```

(اگر در همین کد بنویسی `u.age`، خطای `Property 'age' does not exist on type 'User'` می‌گیری.)

۸. interface در زمان اجرا وجود ندارد

interface فقط برای کامپایلر است و در JavaScript خروجی **هیچ اثری** از آن نمی‌ماند. اگر فایل `weee.ts` اسلاید را با tsc کامپایل کنی، کل خروجی همین است:

weee.js

```
"use strict";
```

به همین دلیل نمی‌شود interface را مثل یک مقدار به کار برد:

```TypeScript
interface User {
  id: number;
  name: string;
}
const u: User = { id: 1, name: "Ali" };
console.log(typeof u);          // OK: at runtime this prints "object"
console.log(u instanceof User); // Error TS2693: 'User' only refers to a type, but is being used as a value here.
```

۹. type alias در برابر interface

راه دیگرِ نام‌گذاری یک نوع، **type alias** (نام مستعار نوع) با کلمه‌ی `type` است:

```TypeScript
type Point = { x: number; y: number };          // an object shape, like an interface
type ID = number | string;                      // a union: impossible with an interface
type Admin = { name: string } & { role: string }; // "&" combines two types
```

	`interface`	`type`
توصیف شکل شیء	بله	بله
نام دادن به union یا نوع ساده (`number | string`)	خیر	بله
گسترش	با `extends`	با `&` (intersection)، ترکیب نوع‌ها)
دو تعریف هم‌نام	با هم ادغام می‌شوند (declaration merging)	خطای `Duplicate identifier`

قاعده‌ی ساده: برای توصیف شکل شیء‌ها و قرارداد کلاس‌ها از `interface` استفاده کن و برای union و ترکیب نوع‌ها از `type` .

> **نکته**
>
> - interface یک **قرارداد** است، نه کد اجرایی: فقط در زمان کامپایل بررسی می‌شود و در JavaScript خروجی اثری از آن نیست.
> - پیش از استفاده از یک ویژگی اختیاری، `undefined` نبودنش را بررسی کن (با `if`) یا از `?.` استفاده کن.
> - `extends` برای گسترش interface از interface است؛ `implements` برای وقتی است که یک **کلاس** قرارداد یک interface را رعایت می‌کند.

⚠ دام تستی

- object literal با ویژگی اضافه خطای TS2353 می‌گیرد، اما **همان شیء از طریق یک متغیر** بدون خطا پذیرفته می‌شود (structural typing).
- `readonly` با `const` فرق دارد: اولی یک ویژگی را قفل می‌کند و دومی خودِ متغیر را.
- در `implements` ویژگی **اختیاری** را می‌شود حذف کرد، اما ویژگی اجباری را نه.
- `typeof User` یا `x instanceof User` برای یک interface خطای TS2693 دارد، چون interface در زمان اجرا وجود ندارد.

جمع‌بندی

interface شکل یک شیء را تعریف می‌کند: ویژگی‌های اجباری، اختیاری (`?`) و فقط‌خواندنی (`readonly`) و نوع هر کدام، و حتی متدها. interface را می‌شود با `extends` گسترش داد و کلاس‌ها با `implements` آن را پیاده‌سازی می‌کنند. TypeScript نوع‌ها را با شکلشان مقایسه می‌کند (structural typing)، ولی برای object literal ویژگی اضافه را خطا می‌داند. interface در JavaScript خروجی هیچ اثری ندارد و برای union و ترکیب نوع‌ها از `type` استفاده می‌کنیم.

تست‌های تمرینی

۲۲ تست · شماره‌ی ۱۱۶۵ تا ۱۱۸۶

۱۱۶۵ آسان

کاربرد اصلی interface در TypeScript چیست؟

۱) تعریف شکل یک شیء؛ یعنی شیء چه ویژگی‌هایی با چه نوع‌هایی باید داشته باشد.

۲) ساختن شیء جدید در زمان اجرا، مثل یک کلاس.

۳) تبدیل کد TypeScript به JavaScript.

۴) بررسی نوع داده‌ها در زمان اجرا داخل مرورگر.

۱۱۶۶ آسان

در interface اسلاید ۱۹، معنای علامت `?` در `email?: string;` چیست؟

۱) `email` فقط می‌تواند مقدار `null` بگیرد.

۲) `email` فقط‌خواندنی است و بعد از ساخت شیء تغییر نمی‌کند.

۳) ویژگی `email` اختیاری است؛ ممکن است وجود نداشته باشد، اما اگر وجود داشت باید رشته باشد.

۴) نوع `email` می‌تواند هر چیزی باشد (مثل `any`).

۱۱۶۷ آسان

با فرض interface اسلاید ۱۹ (کد زیر)، کدام دستور خطای کامپایل دارد؟

```TypeScript
interface User {
  id: number;
  name: string;
  email?: string;
}
```

۱) `const a: User = { id: 1, name: "Ali" };`

۲) `const b: User = { id: 2, name: "Sara", email: "sara@example.com" };`

۳) `const d: User = { id: 4, name: "" };`

۴) `const c: User = { name: "Reza" };`

۱۱۶۸ متوسط

tsc در کدام خط کد زیر خطا گزارش می‌کند؟

```TypeScript
1  interface User {
2    id: number;
3    name: string;
4    email?: string;
5  }
6  const a: User = { id: 1, name: "Ali" };
7  const b: User = { id: 2, name: "Sara", age: 20 };
8  const c: User = { id: 3, name: "Reza", email: "reza@example.com" };
```

۱) خط ۶

۲) خط ۷

۳) خط ۸

۴) هیچ خطایی رخ نمی‌دهد

۱۱۶۹ سخت

tsc در کدام خط کد زیر خطا گزارش می‌کند؟

```TypeScript
1  interface Point {
2    x: number;
3    y: number;
4  }
5  function print(p: Point): void {
6    console.log(p.x, p.y);
7  }
8  const p3 = { x: 1, y: 2, z: 3 };
9  print(p3);
10 print({ x: 1, y: 2, z: 3 });
```

۱) خط ۸

۲) خط ۹

۳) هیچ خطایی رخ نمی‌دهد

۴) خط ۱۰

۱۱۷۰ متوسط

tsc در کدام خط کد زیر خطا گزارش می‌کند؟

TypeScript

```
1  interface User {
2    id: number;
3    name: string;
4    email?: string;
5  }
6  const u: User = { id: 1, name: "Ali" };
7  console.log(u.name.length);
8  console.log(u.email?.length);
9  console.log(u.email.length);
```

۱) خط ۹
۲) خط ۷
۳) خط ۸
۴) هیچ خطایی رخ نمی‌دهد

۱۱۷۱ متوسط

خروجی کد TypeScript زیر پس از کامپایل و اجرا چیست؟

TypeScript

```
interface User {
  id: number;
  name: string;
  email?: string;
}
const u: User = { id: 1, name: "Ali" };
console.log(u.email);
console.log(u.email === undefined);
console.log(Object.keys(u).length);
```

۱) `undefined` `true` `3`
۲) `undefined` `true` `2`
۳) `null` `false` `2`
۴) `undefined` `false` `3`

۱۱۷۲ آسان

tsc در کدام خط کد زیر خطا گزارش می‌کند؟

TypeScript

```
1  interface Point {
2    readonly x: number;
3    y: number;
4  }
5  const p: Point = { x: 1, y: 2 };
6  p.y = 20;
7  p.x = 10;
```

۱) خط ۵
۲) خط ۶
۳) هیچ خطایی رخ نمی‌دهد
۴) خط ۷

۱۱۷۳ سخت

با فرض تعریف زیر، کدام گزینه خطای کامپایل دارد؟

```TypeScript
interface Config {
  readonly url: string;
  retries: number;
}
```

۱) `const c: Config = { url: "a.com", retries: 3 }; c.retries = 5;`

۲) `let c: Config = { url: "a.com", retries: 3 }; c = { url: "b.com", retries: 1 };`

۳) `let c: Config = { url: "a.com", retries: 3 }; c.url = "b.com";`

۴) `const c: Config = { url: "a.com", retries: 3 }; console.log(c.url);`

۱۱۷۴ متوسط

با فرض تعریف‌های زیر، کدام دستور خطای کامپایل دارد؟

```TypeScript
interface User {
  id: number;
  name: string;
  email?: string;
}
interface Admin extends User {
  role: string;
}
```

۱) `const a: Admin = { id: 1, name: "Neda", role: "owner" };`

۲) `const b: Admin = { id: 2, name: "Omid" };`

۳) `const c: User = { id: 3, name: "Sara" };`

۴) `const d: Admin = { id: 4, name: "Mina", role: "editor", email: "mina@example.com" };`

۱۱۷۵ متوسط

با فرض تعریف‌های زیر، کدام دستور **بدون خطا** کامپایل می‌شود؟

```TypeScript
interface HasId {
  id: number;
}
interface HasName {
  name: string;
}
interface Person extends HasId, HasName {
  age: number;
}
```

۱) `const p: Person = { id: 1, name: "Ali", age: 20 };`

۲) `const p: Person = { id: 1, name: "Ali" };`

۳) `const p: Person = { id: 1, age: 20 };`

۴) `const p: Person = { name: "Ali", age: 20 };`

۱۱۷۶ سخت

tsc در کدام خط کد زیر خطا گزارش می‌کند؟

```TypeScript
interface User {
  id: number;
  name: string;
}
interface Admin extends User {
  role: string;
}
function greet(u: User): string {
  return `Hello, ${u.name}`;
}
const admin: Admin = { id: 1, name: "Neda", role: "owner" };
const user: User = { id: 2, name: "Omid" };
greet(admin);
greet(user);
const boss: Admin = user;
```

۱) خط ۱۵

۲) خط ۱۳

۳) خط ۱۴

۴) هیچ خطایی رخ نمی‌دهد

۱۱۷۷ آسان

کدام گزینه درباره‌ی کلمه‌های `extends` و `implements` درست است؟

۱) کلاس با `extends` یک interface را پیاده‌سازی می‌کند و با `implements` از interface دیگر گسترش می‌یابد.

۲) interface با `extends` از interface دیگر گسترش می‌یابد و کلاس با `implements` قرارداد یک interface را پیاده‌سازی می‌کند.

۳) این دو کلمه دقیقاً یک معنی دارند و می‌شود هر کدام را به‌جای دیگری نوشت.

۴) `implements` فقط در JavaScript وجود دارد و `extends` فقط در TypeScript.

۱۱۷۸ متوسط

tsc در کدام خط کد زیر خطا گزارش می‌کند؟

TypeScript

```
1  interface User {
2    id: number;
3    name: string;
4    email?: string;
5  }
6  class Customer implements User {
7    id = 8;
8    name = "Hamed";
9  }
10 class Broken implements User {
11   id = 9;
12 }
```

۱) خط ۱۰
۲) خط ۶
۳) خط ۱۱
۴) هیچ خطایی رخ نمی‌دهد

۱۱۷۹ سخت

با فرض interface زیر، کدام کلاس **بدون خطا** کامپایل می‌شود؟

```TypeScript
interface Shape {
  name: string;
  area(): number;
}
```

۱

```TypeScript
class Square implements Shape {
  name = "square";
}
```

۲

```TypeScript
class Square implements Shape {
  name = "square";
  area() {
    return "4";
  }
}
```

۳

```TypeScript
class Square implements Shape {
  name = "square";
  side = 2;
  area() {
    return this.side * this.side;
  }
}
```

۴

```TypeScript
class Square implements Shape {
  name = 4;
  area() {
    return 4;
  }
}
```

۱۱۸۰ متوسط

tsc در کدام خط کد زیر خطا گزارش می‌کند؟

```TypeScript
interface User {
  id: number;
  name: string;
}
const u: User = { id: 1, name: "Ali" };
console.log(typeof u);
console.log(u instanceof User);
```

۱) خط ۷
۲) خط ۵
۳) خط ۶
۴) هیچ خطایی رخ نمی‌دهد

۱۱۸۱ متوسط

فایلی که فقط شامل interface اسلاید ۱۹ (یعنی `interface User { id: number; name: string; email?: string; }`) است با tsc کامپایل می‌شود. در فایل JavaScript خروجی چه می‌بینیم؟

۱) یک کلاس به نام `User` با یک constructor
۲) یک شیء به نام `User` که نوع هر ویژگی را نگه می‌دارد
۳) تابعی که نوع شیءها را در زمان اجرا بررسی می‌کند
۴) هیچ اثری از interface؛ فقط خطی مثل `"use strict";`

۱۱۸۲ سخت

کد TypeScript زیر بدون خطا کامپایل می‌شود. خروجی اجرای آن چیست؟

```TypeScript
interface User {
  id: number;
  name: string;
}
const data = { id: 1, name: "Ali", age: 30 };
const u: User = data;
console.log(u);
```

۱) `{ id: 1, name: 'Ali' }`
۲) `User { id: 1, name: 'Ali' }`
۳) `{ id: 1, name: 'Ali', age: 30 }`
۴) `Uncaught TypeError`

۱۱۸۳ سخت

با فرض interface زیر، کدام گزینه خطای کامپایل دارد؟

```TypeScript
interface Greeter {
  greet(name: string): string;
}
```

۱) `const g: Greeter = { greet: (n) => "Hi " + n };`

۲) `const g: Greeter = { greet(n: string) { return "Hi " + n; } };`

۳) `const g: Greeter = { greet: () => "Hi" };`

۴) `const g: Greeter = { greet: (n: string) => n.length };`

۱۱۸۴ متوسط

کدام گزینه درباره‌ی `type` و `interface` **نادرست** است؟

۱) هر دو می‌توانند شکل یک شیء را توصیف کنند.

۲) با `type` می‌شود به یک union نام داد؛ مثل `type ID = number | string;`.

۳) با interface هم می‌شود مستقیماً یک union از نوع‌های ساده مثل `number | string` تعریف کرد.

۴) یک interface می‌تواند با `extends` از interface دیگری گسترش پیدا کند.

۱۱۸۵ سخت

کدام گزینه **بدون خطا** کامپایل می‌شود؟

۱) `type A = { x: number }; type A = { y: number };`

۲) `type C = { x: number }; const c: C = { x: 1, y: 2 };`

۳) `interface D { x: number } const d: D = { y: 2 };`

۴) `interface B { x: number } interface B { y: number } const b: B = { x: 1, y: 2 };`

۱۱۸۶ آسان

در جای خالی کد زیر کدام گزینه قرار گیرد تا کد **بدون خطا** کامپایل شود؟

```TypeScript
interface Product {
  title: string;
  price_____ number;
}
const p: Product = { title: "Book" };
```

۱) `:`　۲) `?:`　۳) `!:`　۴) `=`

۳-۱۱ توابع در TypeScript

منبع: NodeJS & TS، اسلاید ۲۰

درسنامه

تابع‌ها قلب هر برنامه‌اند. در JavaScript هر تابعی را می‌شد با هر تعداد آرگومان و با هر نوعی صدا زد و اشتباه‌ها فقط در زمان اجرا معلوم می‌شد. اسلاید ۲۰ نشان می‌دهد TypeScript چطور به تابع‌ها نوع اضافه می‌کند. در این درس هم حالت strict را فرض می‌کنیم.

یک تشبیه ساده

یک تابع را مثل **دستگاه آب‌میوه‌گیری** تصور کن: روی دهانه‌ی ورودی نوشته «فقط پرتقال» و روی خروجی نوشته «آب پرتقال». TypeScript نگهبانی است که نمی‌گذارد کسی سنگ در دستگاه بیندازد و به کسی که منتظر آب پرتقال است، چیز دیگری تحویل بدهد.

۱. پارامترهای نوع‌دار و نوع خروجی

بولت اول اسلاید: تابع‌ها **typed parameters** (پارامترهای نوع‌دار) و **return type** (نوع مقدار برگشتی) دارند تا ایمن باشند. نوع هر پارامتر بعد از نامش و نوع خروجی بعد از پرانتز بسته نوشته می‌شود:

```TypeScript
function add(a: number, b: number): number {
  return a + b;
}
console.log(add(2, 3)); // 5
```

حالا tsc هر فراخوانی را بررسی می‌کند:

```TypeScript
function add(a: number, b: number): number {
  return a + b;
}
add("2", 3);  // Error TS2345: Argument of type 'string' is not assignable to parameter of type 'number'.
add(1);       // Error TS2554: Expected 2 arguments, but got 1.
add(1, 2, 3); // Error TS2554: Expected 2 arguments, but got 3.
```

برخلاف JavaScript (که آرگومانِ جاافتاده را `undefined` می‌کرد و آرگومان اضافه را نادیده می‌گرفت)، در TypeScript **تعداد آرگومان‌ها** هم بررسی می‌شود.

بدنه‌ی تابع هم باید با نوع خروجی جور باشد:

```TypeScript
function label(n: number): string {
  return n; // Error TS2322: Type 'number' is not assignable to type 'string'.
}
function sign(x: number): string { // Error TS2366: Function lacks ending return statement and return type does not include 'undefined'.
  if (x > 0) return "positive";
  if (x < 0) return "negative";
}
```

در `sign` اگر `x` صفر باشد، تابع به هیچ `return` ی نمی‌رسد و `undefined` برمی‌گرداند؛ tsc همین را تشخیص می‌دهد و خطا را روی سطر تعریف تابع (جایی که نوع خروجی نوشته شده) گزارش می‌کند.

نوع void

اگر تابع هیچ مقداری برنمی‌گرداند، نوع خروجی‌اش `void` («هیچ مقداری») است:

```TypeScript
function logIt(msg: string): void {
  console.log("LOG: " + msg);
}
const result = logIt("hi"); // prints: LOG: hi
console.log(result);        // undefined
```

در زمان اجرا، تابعی که `return` ندارد `undefined` برمی‌گرداند. برگرداندن یک مقدار از تابع `void` هم خطاست؛ مثلاً `return 5;` در چنین تابعی خطای `Type 'number' is not assignable to type 'void'` دارد.

استنتاج نوع خروجی (return type inference)

اگر نوع خروجی را ننویسی، TypeScript آن را از روی `return` ها **استنتاج** می‌کند:

```TypeScript
function multiply(a: number, b: number) { // inferred return type: number
  return a * b;
}
const n: number = multiply(2, 3); // OK
const s: string = multiply(2, 3); // Error TS2322: Type 'number' is not assignable to type 'string'.
```

اما نوع **پارامترهای** یک تابع معمولی را نمی‌تواند حدس بزند (یک استثنا را در بخش ۶ می‌بینیم). در حالت strict، پارامتر بی‌نوع خطا دارد، چون به‌طور ضمنی `any` می‌شد (به این قاعده noImplicitAny می‌گویند):

```TypeScript
function half(n) { // Error TS7006: Parameter 'n' implicitly has an 'any' type.
  return n / 2;
}
```

۲. مثال اسلاید ۲۰: تابع createUser

کد خود اسلاید (فایل `weee.ts`):

weee.ts

```ts
function createUser(
  name: string,
  age: number,
  ...hobbies: string[]
): { name: string; age: number; hobbies: string[] } {
  return {
    name,
    age,
    hobbies,
  };
}

const user = createUser("Alice", 30, "Reading", "Cycling");
console.log(user);
```

خروجی:

```
{ name: 'Alice', age: 30, hobbies: [ 'Reading', 'Cycling' ] }
```

خط به خط:

- `name: string` و `age: number` دو پارامتر اجباری‌اند.
- `...hobbies: string[]` یک **rest parameter** است (توضیح کامل در بخش ۵): همه‌ی آرگومان‌های بعدی را در یک آرایه‌ی رشته‌ای جمع می‌کند. اینجا `"Reading"` و `"Cycling"` در `hobbies` قرار می‌گیرند.
- `): { name: string; age: number; hobbies: string[] }` نوع خروجی است: شیئی با سه ویژگی. این نوع همان‌جا (inline) نوشته شده و اعضایش با `;` از هم جدا شده‌اند؛ به‌جایش می‌شد یک interface هم تعریف کرد.
- `return { name, age, hobbies, };` از **shorthand property** (خلاصه‌نویسی ویژگی) استفاده می‌کند: `name` یعنی `name: name`. ویرگول آخر (trailing comma) هم مجاز است.

JavaScriptی که tsc از این فایل می‌سازد، همان کد بدون نوع‌هاست:

weee.js

```js
"use strict";
function createUser(name, age, ...hobbies) {
    return {
        name,
        age,
        hobbies,
    };
}
const user = createUser("Alice", 30, "Reading", "Cycling");
console.log(user);
```

چند فراخوانی دیگر از همین تابع:

فراخوانی	نتیجه
`createUser("Bob", 25)`	`{ name: 'Bob', age: 25, hobbies: [] }`؛ بدون آرگومان اضافه، `hobbies` آرایه‌ی **خالی** است
`createUser("Ali")`	خطای TS2555: `Expected at least 2 arguments, but got 1.`
`createUser("Ali", "20")`	خطای TS2345: `"20"` رشته است، نه عدد
`createUser("Ali", 20, "Chess", 5)`	خطای TS2345: هر آرگومانِ rest باید رشته باشد

به کلمه‌ی `at least` (دست‌کم) در پیام دقت کن: چون rest parameter هر تعدادی آرگومان می‌پذیرد، tsc فقط حداقل تعداد را می‌گوید.

۳. پارامتر اختیاری (optional parameter)

بولت دوم اسلاید: تابع‌ها می‌توانند پارامتر **اختیاری** و **پیش‌فرض** داشته باشند. پارامتر اختیاری با `?` بعد از نامش مشخص می‌شود و اگر فرستاده نشود، مقدارش `undefined` است؛ پس نوعش داخل تابع `string | undefined` است:

TypeScript

```
function greet(person: string, greeting?: string): string {
  if (greeting === undefined) {
    return `Hello, ${person}`;
  }
  return `${greeting}, ${person}`;
}
console.log(greet("Ali"));          // Hello, Ali
console.log(greet("Ali", "Salam")); // Salam, Ali
```

- `greet()` خطا دارد: `Expected 1-2 arguments, but got 0.` (یعنی «یک تا دو آرگومان»).
- استفاده‌ی مستقیم از پارامتر اختیاری بدون بررسی (مثلاً `greeting.toUpperCase()`) خطای `'greeting' is possibly 'undefined'` دارد.
- **پارامتر اختیاری باید بعد از پارامترهای اجباری بیاید:**

TypeScript

```
function wrongOrder(a?: number, b: number) {} // Error TS1016: A required parameter cannot follow an optional parameter.
```

۴. پارامتر پیش‌فرض (default parameter)

به پارامتر می‌شود یک **مقدار پیش‌فرض** داد. اگر آرگومان فرستاده نشود (یا `undefined` فرستاده شود)، همان مقدار به کار می‌رود:

TypeScript

```
function greet2(person: string, greeting = "Hi"): string {
  return `${greeting}, ${person}`;
}
console.log(greet2("Ali"));            // Hi, Ali
console.log(greet2("Ali", "Salam"));   // Salam, Ali
console.log(greet2("Ali", undefined)); // Hi, Ali  (undefined triggers the default)
console.log(greet2("Ali", ""));        // , Ali    (an empty string is a real value)
```

- پارامتر پیش‌فرض خودبه‌خود **اختیاری** است و نوعش از روی مقدار پیش‌فرض استنتاج می‌شود؛ پس `greet2("Ali", 5)` خطا دارد، چون `5` رشته نیست.
- یک پارامتر نمی‌تواند هم `?` داشته باشد و هم مقدار پیش‌فرض: `b?: number = 5` خطای `Parameter cannot have question mark and initializer` دارد.
- برخلاف `?` که در خروجی پاک می‌شود، مقدار پیش‌فرض یک امکان خودِ JavaScript است و در خروجی می‌ماند: `function greet2(person, greeting = "Hi")`.

> **بیشتر بدانیم**
>
> اگر پارامتر پیش‌فرض **قبل از** یک پارامتر اجباری بیاید، دیگر اختیاری حساب نمی‌شود و برای استفاده از مقدار پیش‌فرض باید صریحاً `undefined` بفرستی:

```TypeScript
function f(a = 1, b: number): number {
  return a + b;
}
f(undefined, 5); // OK: returns 6
f(5);            // Error TS2554: Expected 2 arguments, but got 1.
```

۵. پارامتر rest (rest parameter)

بولت سوم اسلاید: **rest parameter** (پارامترِ «بقیه») برای تابع‌هایی است که **تعداد آرگومان‌هایشان متغیر است** (variable-length arguments). سه نقطه‌ی `...` قبل از نام پارامتر یعنی «همه‌ی آرگومان‌های باقی‌مانده را در یک آرایه جمع کن»:

```TypeScript
function sum(...nums: number[]): number {
  return nums.reduce((total, n) => total + n, 0);
}
console.log(sum());            // 0  (nums is an empty array)
console.log(sum(1, 2, 3));     // 6
console.log(sum(...[10, 20])); // 30 (spread an array into the arguments)
```

قاعده‌ها:

- نوع rest parameter باید **آرایه** باشد (`number[]`)؛ نوشتن `...nums: number` خطای `A rest parameter must be of an array type` دارد.
- rest parameter باید **آخرین** پارامتر باشد و هر تابع فقط یکی از آن می‌تواند داشته باشد:

```TypeScript
function badRest(...hobbies: string[], name: string) {} // Error TS1014: A rest parameter must be last in a parameter list.
```

- اگر هیچ آرگومان اضافه‌ای فرستاده نشود، مقدار rest parameter آرایه‌ی **خالی** `[]` است، نه `undefined`.
- `...` در **تعریف** تابع یعنی rest (جمع کردن آرگومان‌ها در آرایه)، اما `...` در **فراخوانی** (مثل `sum(...[10, 20])`) یعنی spread (پخش کردن آرایه به آرگومان‌های جدا).

۶. arrow function و استنتاج نوع

بولت آخر اسلاید: **arrow function**ها (تابع‌های پیکانی) نحو کوتاه‌تری دارند و از **type inference** (استنتاج نوع) خوب استفاده می‌کنند:

```TypeScript
const add = (a: number, b: number) => a + b; // return type inferred: number
const square = (n: number): number => n * n; // explicit return type
console.log(add(2, 3), square(4));           // 5 16
```

```TypeScript
const add = (a: number, b: number) => a + b;
const s: string = add(1, 2); // Error TS2322: Type 'number' is not assignable to type 'string'.
```

contextual typing (نوع‌دهی از روی زمینه): وقتی یک arrow function را جایی می‌دهی که نوعش معلوم است (مثلاً به `map`)، لازم نیست نوع پارامترش را بنویسی و خطای implicit any هم نمی‌گیری:

```TypeScript
const nums = [1, 2, 3];
const doubled = nums.map((n) => n * 2); // n is inferred as number
console.log(doubled);                   // [ 2, 4, 6 ]
```

دقت کن: arrow function با بدنه‌ی `{ … }` باید `return` صریح داشته باشد؛ وگرنه چیزی برنمی‌گرداند:

```TypeScript
const triple = (x: number): number => { // Error TS2355: A function whose declared type is neither 'undefined', 'void', nor 'any' must return a value.
  x * 3;
};
```

نوع تابع (function type)

می‌شود نوع یک متغیر را «تابع» تعیین کرد. نحوش شبیه arrow function است: `(پارامترها) => نوع خروجی` .

```TypeScript
let op: (a: number, b: number) => number;
op = (x, y) => x * y;  // OK: x and y get the type number from the context
op = (x, y) => x > y;  // Error TS2322: Type 'boolean' is not assignable to type 'number'.
op = (x: string) => 1; // Error TS2322: the parameter type does not match
op = (x) => x;         // OK: using fewer parameters is allowed
```

۷. جمع‌بندی خطاهای رایج تابع‌ها

موقعیت	پیام tsc
نوع آرگومان اشتباه	`Argument of type 'string' is not assignable to parameter of type 'number'.`
تعداد آرگومان اشتباه	`Expected 2 arguments, but got 1.` یا `Expected 1-2 arguments, but got 0.`
آرگومان کم در تابعی که rest دارد	`Expected at least 2 arguments, but got 1.`
پارامتر اجباری بعد از اختیاری	`A required parameter cannot follow an optional parameter.`
rest parameter در وسط پارامترها	`A rest parameter must be last in a parameter list.`
برگرداندن مقدار با نوع اشتباه	`Type 'number' is not assignable to type 'string'.`
بعضی مسیرها `return` ندارند	`Function lacks ending return statement and return type does not include 'undefined'.`
پارامتر بی‌نوع (در strict)	`Parameter 'n' implicitly has an 'any' type.`

نکته

- نوع پارامترها را همیشه بنویس؛ نوع خروجی را TypeScript معمولاً خودش درست استنتاج می‌کند، ولی نوشتنش کد را خواناتر و امن‌تر می‌کند.
- ترتیب درست پارامترها: اول اجباری‌ها، بعد اختیاری‌ها یا پیش‌فرض‌ها و در آخر rest parameter.

دام تستی

- فقط `undefined` (یا نفرستادن آرگومان) مقدار پیش‌فرض را فعال می‌کند؛ رشته‌ی خالی یا `0` مقدار واقعی‌اند و جایگزین نمی‌شوند.
- rest parameter بدون آرگومان اضافه آرایه‌ی خالی `[]` است، نه `undefined`.
- `createUser("Ali")` پیام «at least» می‌دهد (به خاطر rest)، اما `add(1)` پیام `Expected 2 arguments` را.
- arrow function با بدنه‌ی `{ }` و بدون `return`، چیزی برنمی‌گرداند.
- در حالت strict، پارامتر بی‌نوع خطای implicit any دارد؛ مگر در callbackهایی مثل `map` که نوعشان از زمینه معلوم است.

جمع‌بندی

در TypeScript نوع پارامترها و نوع خروجی تابع نوشته می‌شود و tsc نوع و **تعداد** آرگومان‌ها را بررسی می‌کند. پارامتر اختیاری (`?`) و پیش‌فرض (`= value`) باید بعد از پارامترهای اجباری بیایند و فقط `undefined` مقدار پیش‌فرض را فعال می‌کند. rest parameter (`...name: T[]`) آرگومان‌های متغیر را در یک آرایه جمع می‌کند، باید آخرین پارامتر باشد و بدون آرگومان اضافه `[]` است. arrow functionها نوع خروجی (و در callbackها نوع پارامترها) را استنتاج می‌کنند و تابعی که چیزی برنمی‌گرداند نوع خروجی `void` دارد.

تست‌های تمرینی

۲۵ تست · شماره‌ی ۱۱۸۷ تا ۱۲۱۱

۱۱۸۷ آسان

با فرض تابع زیر، کدام فراخوانی خطای کامپایل دارد؟

```TypeScript
function add(a: number, b: number): number {
  return a + b;
}
```

۱) `add(2, 3);`
۲) `add("2", 3);`
۳) `add(2.5, -1);`
۴) `add(0, 0);`

۱۱۸۸ آسان

خروجی کد اسلاید ۲۰ (کد زیر) پس از کامپایل و اجرا چیست؟

```TypeScript
function createUser(
  name: string,
  age: number,
  ...hobbies: string[]
): { name: string; age: number; hobbies: string[] } {
  return {
    name,
    age,
    hobbies,
  };
}

const user = createUser("Alice", 30, "Reading", "Cycling");
console.log(user);
```

۱) `{ name: 'Alice', age: 30, hobbies: [ 'Reading', 'Cycling' ] }`
۲) `{ name: 'Alice', age: 30, hobbies: 'Reading' }`
۳) `{ name: 'Alice', age: 30, hobbies: [ 'Reading' ] }`
۴) `[ 'Alice', 30, 'Reading', 'Cycling' ]`

۱۱۸۹ متوسط

خروجی کد زیر پس از کامپایل و اجرا چیست؟

TypeScript

```
function createUser(
  name: string,
  age: number,
  ...hobbies: string[]
): { name: string; age: number; hobbies: string[] } {
  return { name, age, hobbies };
}
const u = createUser("Bob", 25);
console.log(u.hobbies, u.hobbies.length);
```

۱) `[ undefined ] 1`
۲) `Uncaught TypeError`
۳) `[] 0`
۴) `[ '' ] 1`

۱۱۹۰ متوسط

tsc در کدام خط کد زیر خطا گزارش می‌کند؟

TypeScript

```
1  function createUser(
2    name: string,
3    age: number,
4    ...hobbies: string[]
5  ): { name: string; age: number; hobbies: string[] } {
6    return { name, age, hobbies };
7  }
8  createUser("Ali", 20);
9  createUser("Sara", 22, "Chess");
10 createUser("Reza", 25, "Music", 3);
```

۱) خط ۸
۲) خط ۹
۳) خط ۱۰
۴) هیچ خطایی رخ نمی‌دهد

۱۱۹۱ سخت

با فرض تابع زیر (مثال اسلاید ۲۰)، کدام فراخوانی خطای کامپایل دارد؟

```TypeScript
function createUser(
  name: string,
  age: number,
  ...hobbies: string[]
): { name: string; age: number; hobbies: string[] } {
  return { name, age, hobbies };
}
```

۱) `createUser("Ali", 20);`

۲) `createUser("Ali", 20, "A", "B", "C");`

۳) `createUser("Ali", 20, ...["A", "B"]);`

۴) `createUser("Ali");`

۱۱۹۲ آسان

در تعریف `function greet(person: string, greeting?: string)`، علامت `?` چه معنایی دارد؟

۱) `greeting` می‌تواند از هر نوعی باشد.

۲) `greeting` اختیاری است؛ اگر فرستاده نشود مقدارش `undefined` است و نوعش داخل تابع `string | undefined` است.

۳) `greeting` مقدار پیش‌فرض `""` (رشته‌ی خالی) دارد.

۴) `greeting` اجباری است ولی می‌تواند `null` باشد.

۱۱۹۳ آسان

tsc در کدام خط کد زیر خطا گزارش می‌کند؟

```TypeScript
1  function a(x: number, y?: number) {}
2  function b(x?: number, y?: number) {}
3  function c(x?: number, y: number) {}
4  function d(x: number, y = 5) {}
```

۱) خط ۱ ۲) خط ۲ ۳) خط ۳ ۴) خط ۴

۱۱۹۴ سخت

خروجی کد زیر پس از کامپایل و اجرا چیست؟

```typescript
function greet(person: string, greeting = "Hi"): string {
  return `${greeting}, ${person}`;
}
console.log(greet("Ali"));
console.log(greet("Sara", undefined));
console.log(greet("Reza", ""));
```

۱) `Hi, Ali` `Hi, Sara` `, Reza`

۲) `Hi, Ali` `undefined, Sara` `, Reza`

۳) `Hi, Ali` `Hi, Sara` `Hi, Reza`

۴) `Hi, Ali` `undefined, Sara` `Hi, Reza`

۱۱۹۵ متوسط

با فرض تابع زیر، کدام فراخوانی خطای کامپایل دارد؟

```typescript
function repeat(text: string, times = 2): string {
  return text.repeat(times);
}
```

۱) `repeat("ab");`

۲) `repeat("ab", 3);`

۳) `repeat("ab", undefined);`

۴) `repeat("ab", "3");`

۱۱۹۶ آسان

کدام گزینه درباره‌ی rest parameter (مثل `...hobbies: string[]`) **درست** است؟

۱) می‌تواند در هر جای فهرست پارامترها (اول، وسط یا آخر) بیاید.

۲) اگر هیچ آرگومان اضافه‌ای فرستاده نشود، مقدارش `undefined` است.

۳) همه‌ی آرگومان‌های باقی‌مانده را در یک آرایه جمع می‌کند و باید آخرین پارامتر باشد.

۴) یک تابع می‌تواند چند rest parameter داشته باشد.

۱۱۹۷ متوسط

خروجی کد زیر پس از کامپایل و اجرا چیست؟

```typescript
function sum(...nums: number[]): number {
  return nums.reduce((total, n) => total + n, 0);
}
console.log(sum());
console.log(sum(1, 2, 3));
console.log(sum(...[10, 20]));
```

۱) `undefined` `6` `30`
۲) `0` `6` `30`
۳) `NaN` `6` `30`
۴) `0` `6` `Uncaught TypeError`

۱۱۹۸ متوسط

tsc در کدام خط کد زیر خطا گزارش می‌کند؟

```typescript
function log(level: string, ...messages: string[]) {}
function tag(...names: string[], color: string) {}
function count(...values: number[]) {}
function show(title: string, size = 1, ...items: string[]) {}
```

۱) خط ۱
۲) خط ۲
۳) خط ۳
۴) خط ۴

۱۱۹۹ آسان

tsc در کدام خط کد زیر خطا گزارش می‌کند؟

```typescript
function log(msg: string): void {
  console.log(msg);
}
function double(n: number): number {
  return n * 2;
}
function label(n: number): string {
  return n;
}
```

۱) خط ۲
۲) خط ۵
۳) خط ۸
۴) هیچ خطایی رخ نمی‌دهد

۱۲۰۰ سخت

tsc در کدام خط کد زیر خطا گزارش می‌کند؟

TypeScript

```
1 function sign(x: number): string {
2   if (x > 0) {
3     return "positive";
4   } else if (x < 0) {
5     return "negative";
6   }
7 }
8 console.log(sign(5));
```

۱) خط ۱
۲) خط ۶
۳) خط ۸
۴) هیچ خطایی رخ نمی‌دهد

۱۲۰۱ سخت

در حالت strict، tsc در کدام خط کد زیر خطا گزارش می‌کند؟

TypeScript

```
1 function square(n: number) {
2   return n * n;
3 }
4 function half(n) {
5   return n / 2;
6 }
7 const nums = [1, 2, 3].map((n) => n * 2);
```

۱) خط ۱
۲) خط ۴
۳) خط ۷
۴) هیچ خطایی رخ نمی‌دهد

۱۲۰۲ متوسط

خروجی کد زیر پس از کامپایل و اجرا چیست؟

TypeScript

```
function logIt(msg: string): void {
  console.log("LOG: " + msg);
}
const result = logIt("hi");
console.log(result);
```

۱) `LOG: hi` `void`
۲) `LOG: hi` `null`
۳) `undefined` `LOG: hi`
۴) `LOG: hi` `undefined`

۱۲۰۳ متوسط

با فرض تعریف زیر، کدام دستور خطای کامپایل دارد؟

```TypeScript
const add = (a: number, b: number) => a + b;
```

۱) `const s: string = add(1, 2);`

۲) `const r: number = add(1, 2);`

۳) `const t = add(1, 2) * 10;`

۴) `const u: string = String(add(1, 2));`

۱۲۰۴ سخت

با فرض تعریف زیر، کدام انتساب خطای کامپایل دارد؟

```TypeScript
let op: (a: number, b: number) => number;
```

۱) `op = (x, y) => x * y;`

۲) `op = (x: number) => x;`

۳) `op = function (a, b) { return a - b; };`

۴) `op = (x, y) => x > y;`

۱۲۰۵ سخت

tsc در کدام خط کد زیر خطا گزارش می‌کند؟

```TypeScript
1  const double = (n: number): number => n * 2;
2  const triple = (n: number): number => {
3    n * 3;
4  };
5  const square = (n: number) => {
6    return n * n;
7  };
```

۱) خط ۱ ۲) خط ۲ ۳) خط ۳ ۴) خط ۵

۱۲۰۶ آسان

در جای خالی کدام گزینه قرار می‌گیرد؟ «نوع خروجی تابعی که هیچ مقداری برنمی‌گرداند (مثل تابعی که فقط چیزی چاپ می‌کند)، ..... است.»

۱) `void` ۲) `null` ۳) `empty` ۴) `none`

۱۲۰۷ سخت

با فرض تابع زیر، کدام فراخوانی **بدون خطا** کامپایل می‌شود؟

```TypeScript
function f(a = 1, b: number): number {
  return a + b;
}
```

۱) `f(5);`
۲) `f();`
۳) `f(undefined, 5);`
۴) `f(5, "5");`

۱۲۰۸ متوسط

کدام گزینه **نادرست** است؟

۱) پارامتر اختیاری با `?` بعد از نامش مشخص می‌شود و می‌توان آن را نفرستاد.

۲) پارامتری که مقدار پیش‌فرض دارد، خودبه‌خود اختیاری است.

۳) اگر نوع خروجی تابع نوشته نشود، TypeScript آن را از روی `return` ها استنتاج می‌کند.

۴) یک پارامتر می‌تواند هم `?` داشته باشد و هم مقدار پیش‌فرض؛ مثل `b?: number = 5` .

۱۲۰۹ متوسط

خروجی کد زیر پس از کامپایل و اجرا چیست؟

```TypeScript
function info(name: string, age?: number): string {
  if (age === undefined) {
    return `${name}: unknown`;
  }
  return `${name}: ${age}`;
}
console.log(info("Sara"));
console.log(info("Sara", 0));
```

۱) `Sara: unknown` `Sara: 0`
۲) `Sara: undefined` `Sara: 0`
۳) `Sara: unknown` `Sara: unknown`
۴) `Sara: undefined` `Sara: unknown`

۱۲۱۰ متوسط

tsc در کدام خط کد زیر خطا گزارش می‌کند؟

TypeScript

```
1  function shout(text: string, suffix?: string): string {
2    if (suffix) {
3      return text.toUpperCase() + suffix.toUpperCase();
4    }
5    return text.toUpperCase() + suffix.length;
6  }
```

۱) خط ۱
۲) خط ۳
۳) هیچ خطایی رخ نمی‌دهد
۴) خط ۵

۱۲۱۱ متوسط

با فرض تابع زیر، کدام فراخوانی **بدون خطا** کامپایل می‌شود؟

TypeScript

```
function add(a: number, b: number): number {
  return a + b;
}
```

۱) `add(1);`
۲) `add(1, 2, 3);`
۳) `add(1, parseInt("2"));`
۴) `add("1", 2);`

۳-۱۲ کلاس‌ها در TypeScript

منبع: NodeJS & TS، اسلاید ۲۱

درسنامه

کلاس یعنی چه؟ (یک یادآوری سریع)

فرض کن در یک کارخانه‌ی خودروسازی کار می‌کنی. قبل از ساختن خودروها یک **نقشه** می‌کشی: «هر خودرو رنگ دارد، مدل دارد و می‌تواند حرکت کند». بعد از روی همین یک نقشه، صدها خودروی واقعی می‌سازی. در برنامه‌نویسی به این نقشه **class** (کلاس) و به هر چیزی که از روی آن ساخته می‌شود **object** یا **instance** (شیء یا نمونه) می‌گویند.

> **یک تشبیه ساده**
>
> class مثل نقشه‌ی یک ساختمان است و هر object یک ساختمانِ واقعی که از روی آن نقشه ساخته شده. خودِ نقشه جای زندگی نیست؛ تا وقتی با `new` چیزی از روی آن نسازی، هیچ اتفاقی نمی‌افتد.

کلاس‌ها از نسخه‌ی ES6 بخشی از خودِ JavaScript هستند (در فصل اول با آن‌ها و با `this` آشنا شدی). TypeScript همان کلاس‌های JavaScript را برمی‌دارد و چند امکان مهم به آن‌ها اضافه می‌کند: نوع (type) برای propertyها و پارامترها، **access modifier**ها، `implements` ، `readonly` و چند امکان دیگر. برای همین زیر کد اسلاید ۲۱ نوشته شده: **«Just like JS, but better!»** (درست مثل JS، ولی بهتر!).

اسلاید ۲۱ چهار نکته‌ی اصلی دارد:

1. کلاس‌ها از **OOP** (Object-Oriented Programming، برنامه‌نویسی شیءگرا) با **constructor** (سازنده)، **property** (ویژگی) و **method** (متد) پشتیبانی می‌کنند.
2. کلاس می‌تواند یک **interface را implement کند** (پیاده‌سازی کند) تا ساختارش تضمین شود.
3. **inheritance** (وراثت) با `extends` و **method overriding** (بازنویسی متد) پشتیبانی می‌شود.
4. **access modifier**ها (تعیین‌کننده‌های سطح دسترسی: `public` ، `private` ، `protected`) مشخص می‌کنند هر property و method از کجا قابل دسترسی است.

کد اسلاید ۲۱: سه جزء اصلی یک کلاس

weee.ts

```ts
class Person {
  name: string;
  constructor(name: string) {
    this.name = name;
  }
  greet() {
    console.log(`Hello, ${this.name}`);
  }
}
```

بیا خط به خط جلو برویم:

- `class Person { ... }` : کلاسی به اسم `Person` تعریف می‌کند. رسم است اسم کلاس با حرف بزرگ شروع شود.

- `name: string;` : **اعلان property**. در TypeScript هر property را باید (همراه با نوعش) در بدنه‌ی کلاس معرفی کنی. این خط یعنی «هر `Person` یک `name` از نوع `string` دارد».
- `constructor(name: string)` : **constructor** تابع ویژه‌ای است که **هر بار که با** `new` **یک شیء جدید ساخته می‌شود** اجرا می‌شود و شیء را مقداردهی اولیه می‌کند. پارامترش هم نوع دارد.
- `this.name = name;` : `this` یعنی «همین شیئی که الان در حال ساخته شدن است». مقدار پارامتر را در property آن شیء می‌گذاریم.
- `greet() { ... }` : یک **method** (تابعی که مال شیء است). داخلش با **template literal** (رشته‌ای بین دو بک‌تیک که `${...}` را با مقدار واقعی جایگزین می‌کند) سلام می‌کند.

> ⚠ **کد اسلاید به‌تنهایی هیچ خروجی‌ای ندارد**
>
> این کد فقط **نقشه** را تعریف می‌کند: نه شیئی ساخته شده و نه `greet` صدا زده شده است. پس اجرای آن هیچ چیزی چاپ نمی‌کند. برای دیدن خروجی باید شیء بسازیم و متد را صدا بزنیم.

TypeScript

```
class Person {
  name: string;
  constructor(name: string) {
    this.name = name;
  }
  greet() {
    console.log(`Hello, ${this.name}`);
  }
}
const p = new Person("Alice");     // the constructor runs here
p.greet();                         // Hello, Alice
console.log(p.name);               // Alice
console.log(p instanceof Person); // true
```

اسم کلاس خودش یک **نوع** هم هست؛ یعنی می‌توانی بنویسی `const p: Person = new Person("Ali");` یا تابعی بنویسی که پارامترش از نوع `Person` باشد.

وقتی `tsc` (کامپایلر TypeScript) این کد را به JavaScript تبدیل می‌کند، نوع‌ها پاک می‌شوند و یک کلاس معمولی JavaScript باقی می‌ماند (خروجی با target ES2022):

JavaScript

```
class Person {
    name;
    constructor(name) {
        this.name = name;
    }
    greet() {
        console.log(`Hello, ${this.name}`);
    }
}
```

«Just like JS, but better!» یعنی چه؟

در JavaScript معمولی اشتباه‌های زیر تا لحظه‌ی اجرا دیده نمی‌شوند (یا اصلاً خطا نمی‌دهند و فقط نتیجه‌ی غلط می‌دهند). TypeScript همان موقعِ نوشتن، یعنی در **compile time** (زمان کامپایل)، جلویشان را می‌گیرد:

```TypeScript
class Person {
  name: string;
  constructor(name: string) {
    this.name = name;
  }
  greet() {
    console.log(`Hello, ${this.name}`);
  }
}
const p = new Person("Ali");
p.gret();              // Error TS2551: Property 'gret' does not exist. Did you mean 'greet'?
const q = new Person(); // Error TS2554: Expected 1 arguments, but got 0.
p.name = 42;           // Error TS2322: Type 'number' is not assignable to type 'string'.
```

اعلان و مقداردهی propertyها (خطای TS2564)

در حالت **strict** (سخت‌گیرانه؛ این کتاب همیشه آن را فرض می‌کند) TypeScript مطمئن می‌شود هیچ propertyای بی‌مقدار نمی‌ماند: هر property اعلان‌شده باید یا **کنار اعلانش** مقدار اولیه بگیرد، یا **داخل constructor** مقداردهی شود. در غیر این صورت خطای **TS2564** می‌گیری.

```TypeScript
class Product {
  title: string;       // OK: assigned in the constructor
  price: number = 0;   // OK: has an initializer
  stock?: number;      // OK: optional (number | undefined)
  code: string;        // Error TS2564: no initializer and not assigned in the constructor
  constructor(title: string) {
    this.title = title;
  }
}
```

روش	مثال	توضیح
مقدار اولیه کنار اعلان	`price: number = 0;`	ساده‌ترین راه
مقداردهی در constructor	`this.title = title;`	باید در **همه‌ی** مسیرهای constructor انجام شود
property اختیاری	`stock?: number;`	نوعش `number | undefined` می‌شود
علامت `!`	`code!: string;`	به TypeScript قول می‌دهی بعداً مقدار می‌گیرد؛ فقط وقتی واقعاً مطمئنی

> ⚠ **دام تستی: مقداردهی «گاهی‌اوقات» کافی نیست**
>
> اگر property را فقط داخل یک `if` در constructor مقدار بدهی (`if (n) this.name = n;`) یا آن را در یک متد معمولی (مثلاً `init()`) مقدار بدهی، باز هم خطای TS2564 می‌گیری؛ چون ممکن است شیء ساخته شود و آن property بی‌مقدار بماند.

پیاده‌سازی interface با `implements`

یادت هست interface (اسلاید ۱۹) **شکل** یک شیء را تعریف می‌کرد. کلاس می‌تواند با کلیدواژه‌ی `implements` اعلام کند: «من این قرارداد را رعایت می‌کنم». آن‌وقت TypeScript بررسی می‌کند همه‌ی اعضای interface، با نوع درست، در کلاس وجود داشته باشند.

TypeScript

```
interface Shape {
  name: string;
  area(): number;
}
class Square implements Shape {   // OK: has name and area()
  name = "square";
  side: number;
  constructor(side: number) {
    this.side = side;
  }
  area(): number {
    return this.side * this.side;
  }
}
class Circle implements Shape {   // Error TS2420: Property 'area' is missing in type 'Circle'
  name = "circle";
}
class Blob implements Shape {
  name = "blob";
  area(): string {                // Error TS2416: '() => string' is not assignable to '() =>
number'
    return "big";
  }
}
```

- یک کلاس می‌تواند چند interface را با هم پیاده کند: `class Duck implements Walker, Swimmer { ... }`.
- `implements` فقط یک **بررسی** است؛ هیچ کدی به کلاس اضافه نمی‌کند و متدها را باید خودت بنویسی. کلاس می‌تواند اعضای اضافه هم داشته باشد (مثل `side` در `Square`).

وراثت با `extends` و بازنویسی متد

با `class Student extends Person`، کلاس `Student` (به آن **subclass** یا کلاس فرزند می‌گویند) همه‌ی propertyها و متدهای `Person` (**superclass** یا کلاس والد) را **به ارث می‌برد** و می‌تواند چیزهای تازه هم اضافه کند. رابطه‌ی وراثت یعنی «**is-a**»: هر دانشجو **یک** انسان است.

TypeScript

```typescript
interface Greeter {
  greet(): void;
}
class Person implements Greeter {
  public name: string;
  private secret: string;
  protected age: number;
  constructor(name: string, secret: string, age: number) {
    this.name = name;
    this.secret = secret;
    this.age = age;
  }
  greet() {
    console.log(`Hello, ${this.name}`);
  }
}
class Student extends Person {
  school: string;
  constructor(name: string, age: number, school: string) {
    super(name, "hidden", age);   // call the parent constructor first
    this.school = school;
  }
  greet() {                       // overrides Person.greet
    console.log(`Hi, I am ${this.name} (${this.age}) from ${this.school}`);
  }
}
const people: Person[] = [new Person("Ali", "s3cr3t", 30), new Student("Sara", 20, "Sharif")];
for (const x of people) x.greet();
// Hello, Ali
// Hi, I am Sara (20) from Sharif
console.log(people[1] instanceof Person); // true
```

قانون‌های constructor در کلاس فرزند:

1. اگر کلاس فرزند constructor خودش را دارد، **باید** `super(...)` را صدا بزند (`super` یعنی constructor کلاس والد). در غیر این صورت: خطای **TS2377**.
2. `super(...)` باید **قبل از هر استفاده از** `this` صدا زده شود؛ وگرنه خطای **TS17009**. (در JavaScript خالی همین اشتباه در زمان اجرا `ReferenceError` می‌دهد.)
3. اگر کلاس فرزند اصلاً constructor ننویسد، constructor والد خودکار استفاده می‌شود: با `class Cat extends Animal {}` می‌توانی بنویسی `new Cat("Tom")`.

TypeScript

```
class Animal {
  name: string;
  constructor(name: string) {
    this.name = name;
  }
}
class Dog extends Animal {
  breed: string;
  constructor(name: string, breed: string) {
    this.breed = breed;   // Error TS17009: 'super' must be called before accessing 'this'
    super(name);
  }
}
class Fish extends Animal {
  constructor(name: string) {   // Error TS2377: Constructors for derived classes must contain a
'super' call.
    console.log("creating a fish");
  }
}
class Cat extends Animal {}     // OK: uses Animal's constructor
const tom = new Cat("Tom");
```

Method overriding (بازنویسی متد) یعنی کلاس فرزند متدی با **همان اسم** متد والد تعریف کند؛ آن‌وقت برای اشیای فرزند، نسخه‌ی فرزند اجرا می‌شود. در مثال بالا با این‌که نوع آرایه `Person[]` است، برای `Sara` متد `greet` `Student` اجرا شد؛ چون مهم این است که **شیء واقعاً از چه کلاسی ساخته شده** (به این رفتار **polymorphism** یا چندریختی می‌گویند). اگر بخواهی داخل نسخه‌ی جدید، نسخه‌ی والد را هم اجرا کنی، از `super.method()` استفاده کن:

TypeScript

```
class Animal {
  name: string;
  constructor(name: string) {
    this.name = name;
  }
  speak(): string {
    return `${this.name} makes a sound`;
  }
}
class Dog extends Animal {
  override speak(): string {
    return `${super.speak()} - Woof`;
  }
}
console.log(new Dog("Rex").speak()); // Rex makes a sound - Woof
```

کلیدواژه‌ی `override` اختیاری است و به TypeScript می‌گوید «عمداً دارم متد والد را بازنویسی می‌کنم». فایده‌اش این است که اگر والد چنین متدی نداشته باشد (مثلاً اسم را غلط نوشته باشی)، خطای **TS4113** می‌گیری. (گزینه‌ی `noImplicitOverride` نوشتن `override` را اجباری می‌کند؛ این گزینه جزو `strict` نیست.) همچنین متد بازنویسی‌شده باید با نسخه‌ی والد **سازگار** باشد؛ مثلاً اگر والد `string` برمی‌گرداند و فرزند `number`، خطای **TS2416** می‌گیری.

Access modifierها: `public` ، `private` ، `protected`

modifier	داخل خودِ کلاس	داخل کلاس فرزند	بیرون کلاس (روی شیء)
`public` (پیش‌فرض)	مجاز	مجاز	مجاز
`protected`	مجاز	مجاز	ممنوع (TS2445)
`private`	مجاز	ممنوع (TS2341)	ممنوع (TS2341)

- اگر هیچ modifierی ننویسی، عضو `public` است؛ پس `name: string` همان `public name: string` است.
- `private` یعنی «فقط خودِ همین کلاس». حتی کلاس فرزند هم نمی‌تواند به آن دست بزند.
- `protected` یعنی «خودِ کلاس و فرزندانش»، ولی نه بیرون.
- متدها هم می‌توانند `private` یا `protected` باشند (مثلاً یک متد کمکی که نباید از بیرون صدا زده شود).

با همان کلاس‌های `Person` و `Student` بالا:

TypeScript

```
class Person {
  public name: string;
  private secret: string;
  protected age: number;
  constructor(name: string, secret: string, age: number) {
    this.name = name;
    this.secret = secret;
    this.age = age;
  }
}
class Student extends Person {
  showAge() {
    return this.age;      // OK: protected is visible in subclasses
  }
  leak() {
    return this.secret;   // Error TS2341: 'secret' is private and only accessible within class
'Person'
  }
}
const p = new Person("Ali", "s3cr3t", 30);
console.log(p.name);      // OK: public
console.log(p.secret);    // Error TS2341: 'secret' is private
console.log(p.age);       // Error TS2445: 'age' is protected
```

`private` فقط در زمان کامپایل است!

TypeScript بعد از بررسی، همه‌ی modifierها را پاک می‌کند. یعنی در JavaScript خروجی، `secret` یک property کاملاً معمولی است و در **runtime** (زمان اجرا) هر کسی می‌تواند آن را ببیند:

TypeScript

```
class Account {
  public owner: string;
  private pin: string;
  #balance = 100;          // a real JavaScript private field
  constructor(owner: string, pin: string) {
    this.owner = owner;
    this.pin = pin;
  }
}
const acc = new Account("Sara", "1234");
console.log(Object.keys(acc));    // [ 'owner', 'pin' ]
console.log(JSON.stringify(acc)); // {"owner":"Sara","pin":"1234"}
console.log(acc["pin"]);          // 1234 (bracket access is allowed by TypeScript)
```

اگر **حریم خصوصی واقعی در زمان اجرا** بخواهی، از فیلدهای `#` خودِ JavaScript استفاده کن: `#balance` نه در `Object.keys` دیده می‌شود، نه در `JSON.stringify`، و نوشتن `acc.#balance` بیرون از کلاس هم در TypeScript خطای **TS18013** دارد و هم در JavaScript خالی `SyntaxError` است.

	`private` (TypeScript)	`#field` (JavaScript)
چه کسی بررسی می‌کند؟	فقط کامپایلر TypeScript	خودِ موتور JavaScript
در JS خروجی	property معمولی	فیلد خصوصی واقعی
`Object.keys` / `JSON.stringify`	دیده می‌شود	دیده نمی‌شود

`readonly` : فقط‌خواندنی

propertyای که `readonly` باشد **فقط دو جا** می‌تواند مقدار بگیرد: کنار اعلانش یا داخل constructor. هر جای دیگری (حتی داخل یک متد از همان کلاس) خطای **TS2540** می‌دهد.

TypeScript

```
class Config {
  readonly apiUrl: string;
  readonly retries = 3;
  constructor(url: string) {
    this.apiUrl = url;      // OK: inside the constructor
  }
  setUrl(url: string) {
    this.apiUrl = url;      // Error TS2540: Cannot assign to 'apiUrl' because it is a read-only
property.
  }
}
const cfg = new Config("https://api.example.com");
cfg.retries = 5;            // Error TS2540: Cannot assign to 'retries' because it is a read-only
property.
```

Parameter properties: خلاصه‌نویسی در constructor

الگوی «اعلان property + گرفتن پارامتر + `this.x = x`» آن‌قدر رایج است که TypeScript یک میان‌بر دارد: اگر **جلوی پارامتر constructor** یک **modifier** (`public`، `private`، `protected` یا `readonly`) بنویسی، آن پارامتر خودکار به property تبدیل و مقداردهی می‌شود. این دو کلاس دقیقاً یک کار می‌کنند:

```TypeScript
class UserLong {
  private name: string;
  public age: number;
  constructor(name: string, age: number) {
    this.name = name;
    this.age = age;
  }
}
class UserShort {
  constructor(private name: string, public age: number) {}
}
```

⚠ دام تستی: پارامتر بدون modifier، property نیست

در `constructor(public name: string, age: number) {}` فقط `name` property می‌شود. `age` یک پارامتر معمولی است که بعد از تمام شدن constructor از بین می‌رود؛ پس `this.age` در یک متد خطای **TS2339** (`Property 'age' does not exist`) می‌دهد.

`static` : عضوِ خودِ کلاس، نه شیء

عضو `static` به **خودِ کلاس** تعلق دارد، نه به تک‌تک اشیاء؛ همه‌ی اشیاء یک نسخه‌ی مشترک از آن دارند. آن را با **اسم کلاس** صدا می‌زنیم:

```TypeScript
class Counter {
  static count = 0;
  id: number;
  constructor() {
    Counter.count++;          // shared by all instances
    this.id = Counter.count;
  }
}
const a = new Counter();
const b = new Counter();
console.log(a.id, b.id, Counter.count); // 1 2 2
console.log(a.count);  // Error TS2576: Did you mean to access the static member 'Counter.count' instead?
```

💡 نکته: کلاس TypeScript بعد از کامپایل همان کلاس JavaScript است

نوع‌ها، `implements`، `public` / `private` / `protected` و `readonly` فقط برای بررسی در زمان کامپایل هستند و در JavaScript خروجی اثری از آن‌ها نمی‌ماند. ولی `extends`، `super`، `static`، متدها و فیلدهای `#` امکانات خودِ JavaScript هستند و در زمان اجرا هم کار می‌کنند.

⚠ دام تستی: گم شدن `this` وقتی متد را جدا می‌کنی

اگر متد را از شیء جدا کنی و تنها صدا بزنی (`const f = p.greet; f();`)، دیگر شیئی قبل از نقطه نیست و `this` برابر `undefined` می‌شود (بدنه‌ی کلاس همیشه در strict mode اجرا می‌شود). نتیجه: `TypeError: Cannot read properties of undefined`. TypeScript این اشتباه را **نمی‌گیرد** و کد بدون خطا کامپایل می‌شود.

جمع‌بندی

- کلاس = نقشه؛ object = نمونه‌ای که با `new` ساخته می‌شود. constructor هنگام `new` اجرا می‌شود.
- در strict هر property باید کنار اعلان یا در constructor مقدار بگیرد (وگرنه TS2564).
- `implements` = تعهد به یک interface (TS2420 اگر عضوی کم باشد، TS2416 اگر نوعی ناسازگار باشد).
- `extends` = وراثت؛ constructor فرزند باید قبل از `this`، `super(...)` را صدا بزند (TS17009/TS2377). بازنویسی متد + `super.method()` + `override` (TS4113).
- `public` (پیش‌فرض) همه‌جا؛ `protected` خود و فرزندان (بیرون: TS2445)؛ `private` فقط خودِ کلاس (TS2341) و آن هم فقط در زمان کامپایل؛ حریم واقعی = `#`.
- `readonly` فقط کنار اعلان یا در parameter property = modifier جلوی پارامتر constructor (TS2540). `static` با اسم کلاس صدا زده می‌شود.

تست‌های تمرینی

۲۶ تست · شماره‌ی ۱۲۱۲ تا ۱۲۳۷

۱۲۱۲ آسان

کد زیر همان کد اسلاید ۲۱ است. اگر این فایل را کامپایل و اجرا کنیم، چه چیزی چاپ می‌شود؟

```TypeScript
class Person {
  name: string;
  constructor(name: string) {
    this.name = name;
  }
  greet() {
    console.log(`Hello, ${this.name}`);
  }
}
```

۱) `Hello, undefined`
۲) `Hello, name`
۳) (بدون خروجی)
۴) `Uncaught ReferenceError`

۱۲۱۳ آسان

خروجی کد زیر چیست؟

```typescript
class Person {
  name: string;
  constructor(name: string) {
    this.name = name;
  }
  greet() {
    console.log(`Hello, ${this.name}`);
  }
}
const p = new Person("Alice");
p.greet();
```

۱) `Hello, Alice`
۲) `Hello, ${this.name}`
۳) `Hello, undefined`
۴) `Hello, name`

۱۲۱۴ آسان

اگر برای یک property یا method در کلاس TypeScript هیچ access modifierی ننویسیم، آن عضو چه وضعیتی دارد؟

۱) `private` است و فقط داخل همان کلاس قابل استفاده است.
۲) `protected` است و فقط در همان کلاس و کلاس‌های فرزند قابل استفاده است.
۳) `readonly` است و بعد از ساخته شدن شیء قابل تغییر نیست.
۴) `public` است و از همه‌جا (حتی بیرون کلاس) قابل دسترسی است.

۱۲۱۵ آسان

کدام گزینه درباره‌ی access modifierهای TypeScript **درست** است؟

۱) عضو `private` در کلاس فرزند (کلاسی که با `extends` از آن ارث برده) قابل استفاده است.
۲) عضو `protected` داخل کلاس‌های فرزند قابل استفاده است، ولی از بیرون کلاس (روی شیء) نه.
۳) عضو `protected` را از بیرون کلاس می‌توان خواند، ولی نمی‌توان تغییر داد.
۴) نوشتن `public` اجباری است و اگر آن را ننویسیم خطای کامپایل می‌گیریم.

۱۲۱۶ آسان

زیر کد اسلاید ۲۱ نوشته شده «Just like JS, but better!». کدام قابلیت زیر در کلاس‌های JavaScript معمولی **وجود ندارد** و TypeScript آن را اضافه کرده است؟

۱) تابع ویژه‌ی `constructor` برای مقداردهی اولیه‌ی شیء
۲) وراثت با کلیدواژه‌ی `extends`
۳) کلیدواژه‌های `public`، `private` و `protected` برای تعیین سطح دسترسی
۴) تعریف متد (مثل `greet()`) داخل بدنه‌ی کلاس

۱۲۱۷ آسان

می‌خواهیم کلاس `Student` از `Person` **ارث ببرد** تا property `name` را از آن بگیرد و کد بدون خطا کامپایل شود. در جای خالی کدام کلیدواژه قرار می‌گیرد؟

```TypeScript
class Person {
  constructor(public name: string) {}
}
class Student ____ Person {}
const s = new Student("Sara");
console.log(s.name);
```

۱) `extends`
۲) `implements`
۳) `inherits`
۴) `super`

۱۲۱۸ آسان

نوشتن `class Person implements Greeter` (که `Greeter` یک interface است) چه کاری انجام می‌دهد؟

۱) TypeScript هنگام کامپایل بررسی می‌کند که کلاس همه‌ی اعضای `Greeter` را با نوع درست داشته باشد.

۲) کد متدهای `Greeter` به‌طور خودکار به کلاس `Person` اضافه می‌شود.

۳) در زمان اجرا (runtime) بررسی می‌شود که هر شیء ساخته‌شده شکل `Greeter` را داشته باشد.

۴) دقیقاً مثل `extends` است و `Person` از `Greeter` ارث می‌برد.

۱۲۱۹ متوسط

در حالت strict، کدام خط کد زیر خطای کامپایل دارد؟

```TypeScript
1  class Product {
2    title: string;
3    price: number = 0;
4    stock?: number;
5    constructor(title: string) {
6      this.title = title;
7    }
8  }
9  class Order {
10   id: number;
11   items: string[] = [];
12 }
```

۱) خط ۲
۲) خط ۴
۳) خط ۱۰
۴) خط ۱۱

۱۲۲۰ متوسط

کدام خط کد زیر خطای کامپایل دارد؟

```TypeScript
class Person {
  public name: string;
  private secret: string;
  protected age: number;
  constructor(name: string, secret: string, age: number) {
    this.name = name;
    this.secret = secret;
    this.age = age;
  }
}
class Student extends Person {
  info() {
    return `${this.name} (${this.age})`;
  }
}
const s = new Student("Sara", "x", 20);
console.log(s.name);
console.log(s.info());
console.log(s.age);
```

۱) خط ۱۳
۲) خط ۱۷
۳) خط ۱۸
۴) خط ۱۹

۱۲۲۱ متوسط

کدام خط کد زیر خطای کامپایل دارد؟

```TypeScript
class Account {
  private balance = 0;
  protected owner = "Ali";
  deposit(amount: number) {
    this.balance += amount;
  }
}
class Savings extends Account {
  addInterest() {
    this.balance *= 1.1;
  }
  whoOwns() {
    return this.owner;
  }
}
const acc = new Savings();
acc.deposit(100);
```

۱) خط ۱۰
۲) خط ۱۳
۳) خط ۱۷
۴) هیچ خطایی رخ نمی‌دهد

۱۲۲۲ متوسط

کدام خط کد زیر خطای کامپایل دارد؟

TypeScript

```
1  class Config {
2    readonly apiUrl: string;
3    readonly retries = 3;
4    constructor(url: string) {
5      this.apiUrl = url;
6    }
7    setUrl(url: string) {
8      this.apiUrl = url;
9    }
10 }
11 const cfg = new Config("https://api.example.com");
12 console.log(cfg.apiUrl, cfg.retries);
```

۱) خط ۵
۲) خط ۸
۳) خط ۱۱
۴) هیچ خطایی رخ نمی‌دهد

۱۲۲۳ متوسط

کدام خط کد زیر خطای کامپایل دارد؟

TypeScript

```
1  interface Shape {
2    name: string;
3    area(): number;
4  }
5  class Square implements Shape {
6    name = "square";
7    constructor(public side: number) {}
8    area(): number {
9      return this.side * this.side;
10   }
11 }
12 class Circle implements Shape {
13   name = "circle";
14   constructor(public r: number) {}
15 }
```

۱) خط ۵
۲) خط ۷
۳) خط ۸
۴) خط ۱۲

۱۲۲۴ متوسط

خروجی کد زیر چیست؟

```typescript
class Counter {
  static count = 0;
  id: number;
  constructor() {
    Counter.count++;
    this.id = Counter.count;
  }
}
const a = new Counter();
const b = new Counter();
const c = new Counter();
console.log(a.id, c.id, Counter.count);
```

۱) `1 1 1`
۲) `1 3 3`
۳) `3 3 3`
۴) `0 2 3`

۱۲۲۵ متوسط

خروجی کد زیر چیست؟

```typescript
class Point {
  constructor(public x: number, private y: number, readonly z: number = 0) {}
  sum() {
    return this.x + this.y + this.z;
  }
}
const pt = new Point(1, 2);
console.log(pt.x, pt.z, pt.sum());
console.log(JSON.stringify(pt));
```

۱) `1 0 3` `{"x":1,"z":0}`
۲) `1 undefined NaN` `{}`
۳) `1 0 3` `{}`
۴) `1 0 3` `{"x":1,"y":2,"z":0}`

۱۲۲۶ متوسط

در حالت strict، کدام کلاس **بدون خطا** کامپایل می‌شود؟

۱
```TypeScript
class A {
  name: string;
}
```

۲
```TypeScript
class B {
  name: string;
  constructor(n?: string) {
    if (n) this.name = n;
  }
}
```

۳
```TypeScript
class C {
  name: string = "";
}
```

۴
```TypeScript
class D {
  name: string;
  init() {
    this.name = "x";
  }
}
```

۱۲۲۷ متوسط

کدام گزینه درباره‌ی `readonly` در کلاس‌های TypeScript **نادرست** است؟

۱ propertyای که `readonly` است را می‌توان همان‌جا که اعلان می‌شود مقدار داد.

۲ propertyای که `readonly` است را می‌توان داخل constructor مقدار داد.

۳ اگر یک property `readonly` آرایه باشد، باز هم می‌توان با `push` عضو جدیدی به همان آرایه اضافه کرد.

۴ propertyای که `readonly` است را می‌توان داخل یک متد معمولی از همان کلاس دوباره مقداردهی کرد.

۱۲۲۸ متوسط

کدام دستور را می‌توان در جای خالی نوشت تا کد **بدون خطا** کامپایل شود و شمارنده‌ی `static` یکی زیاد شود؟

```TypeScript
class Counter {
  static count = 0;
}
const c = new Counter();
// ____
```

۱) `Counter.count++;`
۲) `c.count++;`
۳) `this.count++;`
۴) `count++;`

۱۲۲۹ متوسط

با کلاس `Animal` زیر، کدام تعریف **خطای کامپایل** دارد؟

```TypeScript
class Animal {
  constructor(public name: string) {}
}
```

۱) `class Cat extends Animal {}`
۲) `class Fish extends Animal { constructor(name: string) { console.log(name); } }`
۳) `class Dog extends Animal { constructor(name: string) { super(name); } }`
۴) `class Bird extends Animal { constructor() { super("bird"); } }`

۱۲۳۰ سخت

کدام خط کد زیر خطای کامپایل دارد؟

```TypeScript
1  class Animal {
2    constructor(public name: string) {}
3  }
4  class Dog extends Animal {
5    breed: string;
6    constructor(name: string, breed: string) {
7      this.breed = breed;
8      super(name);
9    }
10 }
11 const d = new Dog("Rex", "Husky");
```

۱) خط ۵
۲) خط ۷
۳) خط ۸
۴) هیچ خطایی رخ نمی‌دهد

۱۲۳۱ سخت

کدام خط کد زیر خطای کامپایل دارد؟

```TypeScript
class User {
  constructor(public name: string, age: number) {}
  describe() {
    return `${this.name} is ${this.age} years old`;
  }
}
const u = new User("Ali", 20);
console.log(u.name);
```

۱) خط ۲
۲) خط ۴
۳) خط ۷
۴) خط ۸

۱۲۳۲ سخت

با کلاس `Animal` زیر، کدام تعریف **خطای کامپایل** دارد؟

```TypeScript
class Animal {
  speak(): string {
    return "...";
  }
}
```

۱) `class Dog extends Animal { speak(): string { return "Woof"; } }`
۲) `class Cat extends Animal { override speak(): string { return "Meow"; } }`
۳) `class Bird extends Animal { fly(): string { return "flying"; } }`
۴) `class Fish extends Animal { override swim(): string { return "swimming"; } }`

۱۲۳۳ سخت

با کلاس `Animal` زیر، کدام تعریف **خطای کامپایل** دارد؟

```TypeScript
class Animal {
  speak(): string {
    return "...";
  }
}
```

۱) `class Fish extends Animal { speak(): number { return 0; } }`
۲) `class Dog extends Animal { speak(): string { return "Woof"; } }`
۳) `class Cow extends Animal { speak() { return "Moo"; } }`
۴) `class Cat extends Animal { override speak(): string { return super.speak() + "!"; } }`

۱۲۳۴ سخت

خروجی کد زیر چیست؟

```typescript
class Animal {
  constructor(public name: string) {}
  speak(): string {
    return `${this.name} makes a sound`;
  }
}
class Dog extends Animal {
  speak(): string {
    return `${super.speak()} - Woof`;
  }
}
const pets: Animal[] = [new Animal("Generic"), new Dog("Rex")];
for (const p of pets) console.log(p.speak());
```

۱) `Generic makes a sound` `Rex makes a sound`

۲) `Generic makes a sound` `Rex makes a sound - Woof`

۳) `Generic makes a sound` `Woof`

۴) `Generic makes a sound` `Uncaught TypeError`

۱۲۳۵ سخت

خروجی کد زیر چیست؟

```typescript
class Account {
  public owner: string;
  private pin: string;
  #balance = 100;
  constructor(owner: string, pin: string) {
    this.owner = owner;
    this.pin = pin;
  }
}
const acc = new Account("Sara", "1234");
console.log(Object.keys(acc));
console.log(JSON.stringify(acc));
```

۱) `['owner']` `{"owner":"Sara"}`

۲) `['owner', 'pin', '#balance']` `{"owner":"Sara","pin":"1234","#balance":100}`

۳) `['owner', 'pin']` `{"owner":"Sara","pin":"1234"}`

۴) `['owner', 'pin', 'balance']` `{"owner":"Sara","pin":"1234","balance":100}`

۱۲۳۶ سخت

کد زیر بدون خطا کامپایل می‌شود. خروجی آن در زمان اجرا چیست؟

```typescript
class Person {
  constructor(public name: string) {}
  greet() {
    console.log(`Hello, ${this.name}`);
  }
}
const p = new Person("Alice");
p.greet();
const f = p.greet;
f();
```

۱) `Hello, Alice` `Hello, Alice`

۲) `Hello, Alice` `Hello, undefined`

۳) `Uncaught TypeError`

۴) `Hello, Alice` `Uncaught TypeError`

۱۲۳۷ سخت

کدام گزینه درباره‌ی `private` در TypeScript و فیلدهای `#` در JavaScript **درست** است؟

۱) `private` باعث می‌شود آن property در کد JavaScript خروجی حذف شود.

۲) فیلد `#balance` در خروجی `Object.keys` دیده می‌شود، ولی property `private` دیده نمی‌شود.

۳) `private` فقط هنگام کامپایل بررسی می‌شود؛ برای پنهان ماندن واقعی در زمان اجرا باید از فیلدهای `#` خودِ JavaScript استفاده کرد.

۴) اگر در زمان اجرا از بیرون کلاس به یک property `private` دسترسی پیدا کنیم (مثلاً با `obj["pin"]`)، خطای `TypeError` رخ می‌دهد.

۳-۱۳ Generics در TypeScript

منبع: NodeJS & TS، اسلاید ۲۲

درسنامه

یک تشبیه: جعبه‌ی برچسب‌دار

یک تشبیه ساده

فرض کن یک کارخانه‌ی جعبه‌سازی داری. نمی‌خواهی برای کتاب، کفش و لیوان سه خط تولید جدا بسازی؛ یک **جعبه‌ی عمومی** می‌سازی و روی هر جعبه یک **برچسب** می‌زنی: «جعبه‌ی کتاب» یا «جعبه‌ی کفش». وقتی برچسب «کتاب» خورد، فقط کتاب داخلش می‌رود و وقتی بازش کنی مطمئنی کتاب بیرون می‌آید. **Generics** همین است: یک کد عمومی که با یک «برچسب نوع» برای هر نوع داده‌ای کار می‌کند، بدون این‌که TypeScript نوع را گم کند.

پارامتر معمولی تابع، جای خالی برای یک **مقدار** است. **generic** (عام) یعنی تابع، کلاس یا interface یک جای خالی هم برای **نوع** داشته باشد که هنگام استفاده پر می‌شود.

مشکل: یا تکرار کد، یا گم شدن نوع

فرض کن تابعی می‌خواهیم که هر چه بگیرد، همان را برگرداند. بدون generics دو راه بد داریم:

TypeScript

```
// Way 1: one copy per type (code duplication)
function identityNumber(value: number): number {
  return value;
}
function identityString(value: string): string {
  return value;
}

// Way 2: any (type information is lost)
function identityAny(value: any): any {
  return value;
}
const y = identityAny("hi");  // y: any
console.log(y.length);        // 2
console.log(y.toFixed(2));    // compiles, but crashes at runtime: TypeError: y.toFixed is not a function
```

راه اول برای هر نوع یک نسخه‌ی تکراری می‌خواهد. راه دوم از `any` (نوعی که یعنی «هر چیزی») و بررسی نوع را خاموش می‌کند) استفاده می‌کند: TypeScript دیگر نمی‌داند `y` رشته است، پس خطای `y.toFixed` را نمی‌گیرد و برنامه در زمان اجرا می‌ترکد.

راه‌حل: تابع generic (کد اسلاید ۲۲)

weee.ts

```ts
function identity<T>(value: T): T
{
  return value;
}

let num = identity<number>(42);
let str = identity<string>("Hello");
```

- `<T>` یک **type parameter** (پارامتر نوع) است: جای خالی برای یک نوع. اسمش دلخواه است؛ رسم است از `T` (اول کلمه‌ی Type) استفاده شود.
- `(value: T): T` یعنی «ورودی از نوع `T` است و خروجی **همان** نوع `T`». نوع خروجی به نوع ورودی **وصل** است.
- در `identity<number>(42)`، عبارت `<number>` یک **type argument** (آرگومان نوع) است: این‌جا `T` برابر `number` می‌شود، پس `num` از نوع `number` است. به همین ترتیب `str` از نوع `string` است.
- کد اسلاید هیچ `console.log` ی ندارد، پس **خروجی ندارد**. اگر `console.log(num, str, typeof num, typeof str);` را اضافه کنیم، `42 Hello number string` چاپ می‌شود.

فراخوانی	`T` چه می‌شود؟	نوع نتیجه
`identity<number>(42)`	`number`	`number`
`identity<string>("Hello")`	`string`	`string`
`identity<boolean>(true)`	`boolean`	`boolean`

generics فقط برای بررسی در زمان کامپایل هستند؛ در JavaScript خروجی اثری از آن‌ها نمی‌ماند:

JavaScript

```js
function identity(value) {
    return value;
}
let num = identity(42);
let str = identity("Hello");
```

چهار نکته‌ی اسلاید ۲۲ حالا معنا پیدا می‌کنند:

1. generics امکان ساختن کامپوننت‌ها و تابع‌های **قابل استفاده‌ی دوباره (reusable)** و **type-safe** (ایمن از نظر نوع) را می‌دهند: یک `identity` برای همه‌ی نوع‌ها.
2. روی **تابع‌ها، کلاس‌ها و interfaceها** به کار می‌روند (پایین‌تر هر سه را می‌بینیم).
3. **انعطاف** می‌دهند و در عین حال **ایمنی نوع** را حفظ می‌کنند.
4. با کار کردن روی هر نوع داده، **جلوی تکرار کد** را می‌گیرند **بدون این‌که بررسی نوع از دست برود** (برخلاف `any`).

بررسی نوع کاملاً برقرار است:

```TypeScript
function identity<T>(value: T): T {
  return value;
}
let num = identity<number>(42);
let str = identity<string>("Hello");
let wrong = identity<number>("Hello"); // Error TS2345: Argument of type 'string' is not assignable to parameter of type 'number'.
num = str;                             // Error TS2322: Type 'string' is not assignable to type 'number'.
```

استنتاج نوع (type inference): لازم نیست همیشه `<number>` بنویسی

اگر آرگومان نوع را ننویسی، TypeScript آن را از روی آرگومان‌های تابع **استنتاج** (حدس دقیق) می‌کند. یک ظرافت مهم: نتیجه به `let` و `const` بستگی دارد.

```TypeScript
function identity<T>(value: T): T {
  return value;
}
let a = identity(42);          // a: number
const b = identity(42);        // b: 42 (a literal type)
let c = identity("Hello");     // c: string
const d = identity("Hello");   // d: "Hello"
a = 100;                       // OK: a is a number
let e: 42 = b;                 // OK: b is exactly 42
let f: 42 = a;                 // Error TS2322: Type 'number' is not assignable to type '42'.
```

چرا؟ متغیر `let` بعداً می‌تواند عوض شود، پس TypeScript نوع را **گسترده** (`number`) در نظر می‌گیرد. اما `const` هرگز عوض نمی‌شود، پس TypeScript همان **نوع دقیق (literal type)** `42` را نگه می‌دارد (مثل `const PI = 3.14` که نوعش `3.14` است).

آرگومان نوع را وقتی صریح بنویس که نوعی **گسترده‌تر** از حدس TypeScript بخواهی:

```TypeScript
function identity<T>(value: T): T {
  return value;
}
let g = identity<number | string>(5); // g: string | number
g = "five";                           // OK
```

چرا generic بهتر از `any` است؟

	`any`	`<T>` **generic**
نوع خروجی	`any` (اطلاعات نوع گم می‌شود)	همان نوع ورودی
اشتباهی مثل `toFixed` روی رشته	در **زمان اجرا** (`TypeError`)	در **زمان کامپایل**
پیشنهاد خودکار (autocomplete) ویرایشگر	ندارد	دارد

```typescript
function identity<T>(value: T): T {
  return value;
}
const x = identity("hi");
x.toUpperCase();  // OK: x is a string
x.toFixed(2);     // Error TS2551: Property 'toFixed' does not exist on type '"hi"'.
```

آرایه‌ها: `Array<T>`

همین حالا هم بدون این‌که بدانی از generics استفاده می‌کردی: `number[]` همان `Array<number>` است.

```typescript
let nums: Array<number> = [1, 2, 3];   // same as number[]
nums.push(4);                          // OK
nums.push("5");                        // Error TS2345: Argument of type 'string' is not assignable
to parameter of type 'number'.

function firstItem<T>(items: T[]): T | undefined {
  return items[0];
}
const h = firstItem([10, 20]);    // h: number | undefined
const i = firstItem(["x", "y"]);  // i: string | undefined
```

(`Promise` فصل اول هم generic است: `Promise<string>` یعنی Promiseای که در پایان یک `string` می‌دهد.)

interface generic: `Box<T>`

```typescript
interface Box<T> {
  value: T;
}
const b1: Box<string> = { value: "hi" };
const b2: Box<number[]> = { value: [1, 2] };
const b3: Box<number> = { value: "hi" };  // Error TS2322: Type 'string' is not assignable to type
'number'.
const b4: Box = { value: 1 };             // Error TS2314: Generic type 'Box<T>' requires 1 type
argument(s).
```

چند پارامتر نوع: `<K, V>`

می‌توان بیش از یک پارامتر نوع داشت؛ رسم است برای کلید (Key) از `K` و برای مقدار (Value) از `V` استفاده شود:

```TypeScript
interface Pair<K, V> {
  key: K;
  value: V;
}
const p1: Pair<string, number> = { key: "age", value: 30 };    // OK
const p2: Pair<string, number> = { key: "age", value: "30" };  // Error TS2322: Type 'string' is
not assignable to type 'number'.

function makePair<K, V>(key: K, value: V): Pair<K, V> {
  return { key, value };
}
const p3 = makePair("id", 7);                    // Pair<string, number>
const p4 = makePair<string, number>("id", "7");  // Error TS2345: Argument of type 'string' is not
assignable to parameter of type 'number'.
```

کلاس generic: `Stack<T>`

Stack (پشته) مثل یک دسته بشقاب است: آخرین بشقابی که می‌گذاری، اولین بشقابی است که برمی‌داری (LIFO: Last In, First Out). یک بار کلاس را می‌نویسیم و برای هر نوعی استفاده می‌کنیم:

```TypeScript
class Stack<T> {
  private items: T[] = [];
  push(item: T): void {
    this.items.push(item);
  }
  pop(): T | undefined {
    return this.items.pop();
  }
  size(): number {
    return this.items.length;
  }
}
const s = new Stack<string>();
s.push("a");
s.push("b");
s.push("c");
console.log(s.pop(), s.size());  // c 2
const n = new Stack<number>();
console.log(n.pop(), n.size());  // undefined 0
n.push(1);                        // OK
n.push("2");                      // Error TS2345: Argument of type 'string' is not assignable to
parameter of type 'number'.
```

`pop` نوع `T | undefined` برمی‌گرداند، چون پشته ممکن است خالی باشد (`pop` آرایه‌ی خالی `undefined` می‌دهد).

قید (constraint) با `extends`

داخل یک تابع generic، TypeScript هیچ چیزی درباره‌ی `T` نمی‌داند (ممکن است عدد باشد، ممکن است رشته). پس اجازه نمی‌دهد به property خاصی دست بزنی:

```TypeScript
function size<T>(x: T): number {
  return x.length;  // Error TS2339: Property 'length' does not exist on type 'T'.
}
```

راه‌حل، **قید** گذاشتن روی `T` است: `T extends { length: number }` یعنی «`T` هر نوعی می‌تواند باشد، **به شرط این‌که** یک `length` عددی داشته باشد».

```TypeScript
function logLength<T extends { length: number }>(x: T): T {
  console.log(x.length);
  return x;
}
logLength("hello");        // 5
logLength([1, 2, 3]);      // 3
logLength({ length: 7 });  // 7
logLength(42);             // Error TS2345: Argument of type 'number' is not assignable to
parameter of type '{ length: number; }'.
```

چون خروجی `T` است، `logLength("hello")` همچنان یک `string` برمی‌گرداند و می‌توانی روی نتیجه `toUpperCase()` را صدا بزنی.

«You remember generic from Java, right?»

زیر کد اسلاید ۲۲ نوشته «generic را از Java یادت هست، نه؟». ایده دقیقاً همان است:

	Java	TypeScript
نوشتن	`List<Integer>` ، `class Box<T>`	`Array<number>` ، `class Box<T>`
استنتاج آرگومان نوع	محدودتر (مثلاً `new ArrayList<>()` در `<>`)	قوی؛ معمولاً لازم نیست بنویسی
نوع‌های پایه	`List<int>` مجاز نیست (باید `Integer` نوشت)	`Array<number>` مجاز است
در زمان اجرا	type erasure (اطلاعات generic پاک می‌شود)	کاملاً پاک می‌شود؛ در JS خروجی هیچ اثری نیست

نکته: اسم پارامتر نوع و شکل‌های دیگر

- اسم پارامتر نوع دلخواه است: `T` (Type)، `K` (Key)، `V` (Value)، `E` (Element) یا حتی اسم توصیفی مثل `TItem` .
- arrow function هم می‌تواند generic باشد: `const identity2 = <T>(value: T): T => value;` (در فایل `.ts` معمولی).
- generic روی تابع، کلاس و interface یک شکل دارد: `<...>` بعد از اسم.

دام تستی: generic مقدار را تبدیل نمی‌کند

`identity<number>("5")` رشته‌ی `"5"` را به عدد `5` تبدیل **نمی‌کند**؛ فقط یک خطای کامپایل (TS2345) است. generics فقط بررسی نوع در زمان کامپایل‌اند و در زمان اجرا وجود ندارند؛ پس `typeof T` یا هر بررسی دیگری روی `T` در زمان اجرا ممکن نیست.

⚠ دام تستی: `let` یا `const`؟ و معنای `extends`

- `let a = identity(42)` نوع `number` می‌گیرد، ولی `const b = identity(42)` نوع دقیق `42` .
- `extends` در `<T extends { length: number }>` به معنی **قید** است («باید این شکل را داشته باشد»)، نه وراثت کلاس.

جمع‌بندی

- generic = جای خالی برای **نوع**: `function identity<T>(value: T): T` .
- `<number>` در `identity<number>(42)` آرگومان نوع است؛ اگر ننویسی، استنتاج می‌شود (`let` → `number` ، `const` → `42`).
- آرگومان ناسازگار با آرگومان نوع صریح → generic .TS2345 اطلاعات نوع را حفظ می‌کند، `any` نه.
- روی تابع (`identity<T>`)، interface (`Box<T>` ، `Pair<K, V>`) و کلاس (`Stack<T>`) به کار می‌رود؛ `Array<T>` همان `T[]` است.
- قید: `<T extends { length: number }>` ؛ بدون قید، `x.length` خطای TS2339 دارد.
- در JavaScript خروجی هیچ اثری از generics نمی‌ماند.

تست‌های تمرینی

۲۱ تست · شماره‌ی ۱۲۳۸ تا ۱۲۵۸

۱۲۳۸ آسان

طبق اسلاید ۲۲، مهم‌ترین فایده‌ی generics چیست؟

۱) نوشتن کدی که برای انواع مختلف داده دوباره قابل استفاده است، **بدون** از دست رفتن بررسی نوع

۲) سریع‌تر شدن اجرای برنامه در زمان اجرا

۳) خاموش کردن بررسی نوع برای انعطاف بیشتر، درست مثل `any`

۴) تبدیل خودکار مقدارها به نوع `T` در زمان اجرا

۱۲۳۹ آسان

در تعریف `function identity<T>(value: T): T` ، `T` چیست؟

۱) اسم کلاسی که باید از قبل جایی تعریف شده باشد.

۲) اسم دیگری برای همان پارامتر `value` .

۳) یک **پارامتر نوع** (type parameter)؛ جای خالی برای نوعی که هنگام فراخوانی مشخص می‌شود.

۴) کلیدواژه‌ای هم‌معنی `any` که بررسی نوع را خاموش می‌کند.

۱۲۴۰ آسان

خروجی کد زیر (کد اسلاید ۲۲ به همراه یک `console.log`) چیست؟

```typescript
function identity<T>(value: T): T
{
  return value;
}

let num = identity<number>(42);
let str = identity<string>("Hello");
console.log(num, str, typeof num, typeof str);
```

۱) `42 Hello T T`
۲) `42 Hello number string`
۳) `42 Hello object object`
۴) `42 Hello any any`

۱۲۴۱ آسان

طبق اسلاید ۲۲، generics را روی چه چیزهایی می‌توان به کار برد؟

۱) فقط روی تابع‌ها
۲) فقط روی کلاس‌ها (مثل Java)
۳) فقط روی interfaceها
۴) روی تابع‌ها، کلاس‌ها و interfaceها

۱۲۴۲ آسان

کدام خط کد زیر خطای کامپایل دارد؟

```typescript
function identity<T>(value: T): T {
  return value;
}
let num = identity<number>(42);
let str = identity<string>("Hello");
let wrong = identity<number>("Hello");
let auto = identity(true);
```

۱) خط ۴
۲) خط ۵
۳) خط ۶
۴) خط ۷

۱۲۴۳ آسان

می‌خواهیم `names` آرایه‌ای از رشته‌ها باشد (معادل `string[]`) و کد بدون خطا کامپایل شود. در جای خالی کدام گزینه قرار می‌گیرد؟

```typescript
let names: ____ = ["Ali", "Sara"];
```

۱) `Array[string]`
۲) `Array<string>`
۳) `Array(string)`
۴) `string<Array>`

۱۲۴۴ متوسط

کدام خط کد زیر خطای کامپایل دارد؟

```TypeScript
function identity<T>(value: T): T {
  return value;
}
let a = identity(42);
const b = identity(42);
a = 100;
let c: 42 = b;
let d: 42 = a;
```

۱) خط ۵
۲) خط ۶
۳) خط ۷
۴) خط ۸

۱۲۴۵ متوسط

TypeScript برای متغیر `s` در کد زیر چه نوعی استنتاج می‌کند؟

```TypeScript
function identity<T>(value: T): T {
  return value;
}
let s = identity("Hello");
```

۱) `string`
۲) `"Hello"`
۳) `any`
۴) `T`

۱۲۴۶ متوسط

خروجی کد زیر چیست؟

```typescript
class Stack<T> {
  private items: T[] = [];
  push(item: T): void {
    this.items.push(item);
  }
  pop(): T | undefined {
    return this.items.pop();
  }
  size(): number {
    return this.items.length;
  }
}
const s = new Stack<string>();
s.push("a");
s.push("b");
s.push("c");
console.log(s.pop(), s.size());
const e = new Stack<number>();
console.log(e.pop(), e.size());
```

۱) `a 2` `undefined 0`

۲) `c 3` `undefined 0`

۳) `c 2` `undefined 0`

۴) `c 2` `Uncaught TypeError`

۱۲۴۷ متوسط

کدام خط کد زیر خطای کامپایل دارد؟

```typescript
1  class Stack<T> {
2    private items: T[] = [];
3    push(item: T): void {
4      this.items.push(item);
5    }
6    pop(): T | undefined {
7      return this.items.pop();
8    }
9  }
10 const s = new Stack<number>();
11 s.push(1);
12 s.push(2 * 3);
13 s.push("4");
14 const top = s.pop();
```

۱) خط ۱۰

۲) خط ۱۲

۳) خط ۱۳

۴) خط ۱۴

۱۲۴۸ متوسط

کدام خط کد زیر خطای کامپایل دارد؟

```TypeScript
function logLength<T extends { length: number }>(x: T): T {
  console.log(x.length);
  return x;
}
logLength("hello");
logLength([1, 2, 3]);
logLength({ length: 7 });
logLength(42);
```

۱) خط ۲ ۲) خط ۶ ۳) خط ۷ ۴) خط ۸

۱۲۴۹ متوسط

خروجی کد زیر چیست؟

```TypeScript
function logLength<T extends { length: number }>(x: T): T {
  console.log(x.length);
  return x;
}
const r = logLength("hello");
logLength([1, 2, 3]);
console.log(r.toUpperCase());
```

۱) `5` `3` `HELLO`
۲) `5` `3` `Uncaught TypeError`
۳) `hello` `1,2,3` `HELLO`
۴) `5` `undefined` `HELLO`

۱۲۵۰ متوسط

کدام گزینه درباره‌ی مقایسه‌ی `function f(x: any): any` با `function f<T>(x: T): T` **درست** است؟

۱) از نظر بررسی نوع هیچ فرقی با هم ندارند؛ هر دو هر ورودی‌ای را می‌پذیرند.

۲) نسخه‌ی generic فقط وقتی کار می‌کند که آرگومان نوع را صریحاً بنویسیم (مثل `f<number>(5)`).

۳) نسخه‌ی generic در زمان اجرا نوع مقدار را بررسی می‌کند و اگر نادرست باشد خطا می‌دهد.

۴) در نسخه‌ی generic نوع خروجی به نوع ورودی وصل می‌ماند؛ ولی با `any` اطلاعات نوع گم می‌شود و بعضی خطاها تا زمان اجرا دیده نمی‌شوند.

۱۲۵۱ متوسط

با interface زیر، کدام تعریف **خطای کامپایل** دارد؟

```TypeScript
interface Pair<K, V> {
  key: K;
  value: V;
}
```

۱) `const p: Pair<string, number> = { key: "age", value: 30 };`

۲) `const p: Pair<string, number> = { key: "age", value: "30" };`

۳) `const p: Pair<number, string> = { key: 1, value: "one" };`

۴) `const p: Pair<boolean, boolean> = { key: true, value: false };`

۱۲۵۲ متوسط

کدام خط کد زیر خطای کامپایل دارد؟

```TypeScript
1 interface Box<T> {
2   value: T;
3 }
4 const b1: Box<string> = { value: "hi" };
5 const b2: Box<number[]> = { value: [1, 2] };
6 const b3: Box<boolean> = { value: "true" };
7 const b4: Box<number> = { value: 0 };
```

۱) خط ۴ ۲) خط ۵ ۳) خط ۶ ۴) خط ۷

۱۲۵۳ سخت

کدام خط کد زیر خطای کامپایل دارد؟

```TypeScript
1 function first<T>(items: T[]): T | undefined {
2   return items[0];
3 }
4 function size<T>(x: T): number {
5   return x.length;
6 }
7 const n = first([10, 20]);
8 const s = first(["a", "b"]);
```

۱) خط ۲ ۲) خط ۵ ۳) خط ۷ ۴) خط ۸

۱۲۵۴ سخت

کد زیر **بدون خطا** کامپایل می‌شود. خروجی آن در زمان اجرا چیست؟

```typescript
function identityAny(value: any): any {
  return value;
}
const y = identityAny("hi");
console.log(y.length);
console.log(y.toFixed(2));
console.log("done");
```

۱) `2` `Uncaught TypeError`

۲) `2` `hi` `done`

۳) `2` `NaN` `done`

۴) `2` `undefined` `done`

۱۲۵۵ سخت

کدام خط کد زیر خطای کامپایل دارد؟

```typescript
function identity<T>(value: T): T {
  return value;
}
function identityAny(value: any): any {
  return value;
}
const x = identity("hi");
const y = identityAny("hi");
console.log(x.toUpperCase());
console.log(y.toFixed(2));
console.log(x.toFixed(2));
```

۱) خط ۹

۲) خط ۱۰

۳) خط ۱۱

۴) هیچ خطایی رخ نمی‌دهد

۱۲۵۶ سخت

با تابع زیر، کدام فراخوانی **خطای کامپایل** دارد؟

```typescript
function longest<T extends { length: number }>(a: T, b: T): T {
  return a.length >= b.length ? a : b;
}
```

۱) `const r = longest("abc", "de");`

۲) `const r = longest([1, 2], [3]);`

۳) `const r = longest<string | number[]>("abc", [1, 2]);`

۴) `const r = longest("abc", [1, 2]);`

۱۲۵۷ سخت

زیر کد اسلاید ۲۲ نوشته شده «You remember generic from Java, right?». کدام گزینه درباره‌ی generics در TypeScript **نادرست** است؟

۱) مثل Java، پارامتر یا آرگومان نوع داخل `< >` نوشته می‌شود؛ مثل `List<Integer>` در Java و `Array<number>` در TypeScript.

۲) می‌توان در زمان اجرا با `typeof T` فهمید `T` چه نوعی است، چون اطلاعات generic در JavaScript خروجی باقی می‌ماند.

۳) اگر آرگومان نوع را ننویسیم، کامپایلر معمولاً آن را از روی آرگومان‌های تابع استنتاج می‌کند.

۴) مثل Java می‌توان کلاس generic (مانند `class Box<T>`) تعریف کرد.

۱۲۵۸ سخت

TypeScript برای متغیر `g` در کد زیر چه نوعی در نظر می‌گیرد؟

```typescript
function identity<T>(value: T): T {
  return value;
}
let g = identity<number | string>(5);
```

۱) `string | number`　۲) `number`　۳) `5`　۴) `any`

۳-۱۴ ماژول‌ها (Modules)

منبع: NodeJS & TS، اسلاید ۲۳

درسنامه

چرا به ماژول نیاز داریم؟

در اسلاید ۳ (محدودیت‌های JavaScript) دیدیم که JavaScript اولیه «سیستم ماژول درست‌وحسابی نداشت». یعنی چه؟ در صفحه‌های وب قدیمی همه‌ی کدها با تگ‌های `<script>` معمولی به صفحه وصل می‌شدند و **همه‌شان یک scope سراسری (global) مشترک** داشتند؛ متغیری که یک اسکریپت تعریف می‌کرد در اسکریپت بعدی هم دیده می‌شد و اگر دو اسکریپت یک اسم را تعریف می‌کردند، با هم تداخل پیدا می‌کردند:

HTML

```
<script>
  let counter = 1;
  console.log("first", counter);   // first 1
</script>
<script>
  let counter = 2;                 // SyntaxError: Identifier 'counter' has already been declared
  console.log("second", counter);  // never runs
</script>
<script>
  console.log("third", counter);   // third 1 (the first script's variable is global)
</script>
```

یک تشبیه ساده

فرض کن همه‌ی اعضای یک خانواده‌ی پرجمعیت در **یک اتاق** زندگی کنند: وسایل همه قاطی می‌شود و اگر دو نفر یک اسم روی وسیله‌شان بنویسند، دعوا می‌شود. ماژول‌ها به هر فایل یک **اتاق جدا با در بسته** می‌دهند: هر چیزی داخل اتاق خصوصی است، مگر این‌که خودت آن را دمِ در بگذاری (**export**) تا دیگران بتوانند برش دارند (**import**).

ماژول چیست؟ (اسلاید ۲۳)

اسلاید ۲۳ شش نکته دارد:

1. **A module is simply a file that has imports or exports**: ماژول یعنی **فایلی که import یا export دارد**.
2. TypeScript (و JavaScript مدرن) از **ES module system** استفاده می‌کند؛ یعنی **ES Modules** یا **ESM** (سیستم ماژول استانداردِ خودِ JavaScript که از ES2015 یا همان ES6 به زبان اضافه شد).
3. **هر فایل scope خصوصی (private) خودش را دارد**؛ متغیرها و تابع‌هایش به‌طور پیش‌فرض بیرون دیده نمی‌شوند.
4. با **export** چیزی را برای فایل‌های دیگر قابل دیدن می‌کنی.
5. با **import** از کدِ فایل‌های دیگر استفاده می‌کنی.
6. **انواع export**: Named Exports (نام‌دار)، Default Export (پیش‌فرض) و Mixed Exports (ترکیبی).

فایل بدون «import/export: «script

TypeScript فایلی را که هیچ `import` یا `export` ی ندارد **script** (اسکریپت سراسری) حساب می‌کند؛ متغیرهای سطح بالای همه‌ی scriptها در یک scope مشترک قرار می‌گیرند. پس اگر این دو فایل را با هم کامپایل کنیم:

a.ts

```
let counter = 1;
```

b.ts

```
let counter = 2;
```

`tsc` (با تنظیم رایج `--module esnext`) خطای تعریف دوباره می‌دهد:

```
a.ts(1,5): error TS2451: Cannot redeclare block-scoped variable 'counter'.
b.ts(1,5): error TS2451: Cannot redeclare block-scoped variable 'counter'.
```

اگر به آخر هر دو فایل `export {};` (یک export خالی) اضافه کنیم، هر فایل **ماژول** می‌شود، scope خصوصی خودش را می‌گیرد و خطا از بین می‌رود.

> **بیشتر بدانیم**
>
> بعضی تنظیمات tsc این تشخیص را عوض می‌کنند: گزینه‌ی `"moduleDetection": "force"` هر فایلی را (حتی بدون import/export) ماژول حساب می‌کند، و در TypeScript 7 (نسخه‌ای که مثال‌های این کتاب با آن بررسی شده‌اند) تنظیم `"module": "nodenext"` هم همین رفتار را دارد. ولی قاعده‌ی کلی و تعریف اسلاید همان «فایلِ دارای import یا export» است.

۱) Named export (export نام‌دار)

هر چیزی را که جلویش `export` بنویسی، **با همان اسم** صادر می‌شود. یک فایل می‌تواند **هر تعداد** export نام‌دار داشته باشد:

math.ts

```
export const PI = 3.14;
export function add(a: number, b: number): number {
  return a + b;
}
const secret = 42; // not exported: private to this file
export function useSecret(): number {
  return secret;
}
```

در فایل دیگر، export نام‌دار را **داخل آکولاد** `{ }` **و دقیقاً با همان اسم** import می‌کنیم. اگر اسم دیگری بخواهی، از `as` استفاده کن:

TypeScript

```
import { PI, add } from "./math.js";
import { add as sum } from "./math.js";   // rename while importing
```

می‌توان exportها را آخر فایل هم به‌صورت فهرست نوشت و حتی هنگام export اسم را عوض کرد:

`export { shout, whisper as quiet };` .

`secret` export نشده، پس **خصوصی** است: `useSecret()` (که داخل همان فایل است) به آن دسترسی دارد، ولی هیچ فایل دیگری نه.

۲) Default export (export پیش‌فرض)

هر فایل می‌تواند **حداکثر یک** export پیش‌فرض داشته باشد؛ معمولاً «چیز اصلی» آن فایل:

logger.ts

```
export default function log(message: string): void {
  console.log(`[LOG] ${message}`);
}
```

export پیش‌فرض **بدون آکولاد** و **با هر اسمی که دوست داری** import می‌شود:

TypeScript

```
import log from "./logger.js";
import printer from "./logger.js";   // same function, different local name
```

- دو `export default` در یک فایل مجاز نیست (TypeScript خطا می‌دهد و Node هم هنگام بارگذاری `SyntaxError: Duplicate export of 'default'` می‌دهد).
- بعد از `export default` یک **عبارت** (مقدار، تابع یا کلاس) می‌آید، نه تعریف متغیر:

TypeScript

```
export default const settings = { port: 3000 }; // Error TS1109: Expression expected.
```

TypeScript

```
const settings = { port: 3000 };
export default settings;   // OK: export an existing value
```

۳) Mixed exports (export ترکیبی)

یک فایل می‌تواند هم یک export پیش‌فرض و هم چند export نام‌دار داشته باشد:

shapes.ts

```
export default class Circle {
  radius: number;
  constructor(radius: number) {
    this.radius = radius;
  }
}
export function area(c: Circle): number {
  return Math.round(Math.PI * c.radius * c.radius);
}
```

هنگام import، **اول پیش‌فرض (بدون آکولاد)، بعد ویرگول، بعد نام‌دارها داخل آکولاد**:

`import Circle, { area } from "./shapes.js";`

`import * as` : همه‌ی exportها در یک شیء

`import * as MathUtils from "./math.js";` همه‌ی exportهای نام‌دار را در یک شیء به اسم `MathUtils` جمع می‌کند (به آن **namespace import** می‌گویند): `MathUtils.PI` و `MathUtils.add(2, 3)`. چیزهای exportنشده (مثل `secret`) در این شیء هم نیستند (`MathUtils.secret` در TypeScript خطای TS2339 دارد و در JavaScript برابر `undefined` است). اگر فایل export پیش‌فرض داشته باشد، با اسم `default` در همین شیء می‌آید.

مثال کامل

main.ts

```ts
import { PI, add, useSecret } from "./math.js";
import log from "./logger.js";
import Circle, { area } from "./shapes.js";
import * as MathUtils from "./math.js";
import { add as sum } from "./math.js";
import printer from "./logger.js";

log(`PI = ${PI}`);
log(`add(2, 3) = ${add(2, 3)}`);
log(`sum(4, 5) = ${sum(4, 5)}`);
log(`MathUtils.PI = ${MathUtils.PI}`);
log(`useSecret() = ${useSecret()}`);
log(`area = ${area(new Circle(2))}`);
printer("default import can use any name");
```

خروجی اجرای `main` بعد از کامپایل:

```
[LOG] PI = 3.14
[LOG] add(2, 3) = 5
[LOG] sum(4, 5) = 9
[LOG] MathUtils.PI = 3.14
[LOG] useSecret() = 42
[LOG] area = 13
[LOG] default import can use any name
```

و این‌ها اشتباه‌های رایج import با خطای TypeScript هر کدام‌اند:

wrong.ts

```ts
import { secret } from "./math.js";   // Error TS2459: 'secret' is declared locally, but it is not exported
import { missing } from "./math.js";  // Error TS2305: Module has no exported member 'missing'
import { log } from "./logger.js";    // Error TS2614: no exported member 'log'. Did you mean 'import log from'?
import add from "./math.js";          // Error TS2613: Module has no default export
```

نوع	در فایل مبدأ	در فایل مقصد
Named	`export const PI = 3.14;`	`import { PI } from "./math.js";`
Named با اسم جدید	(همان)	`import { add as sum } from "./math.js";`
Default	`export default function log() {}`	`import log from "./logger.js";` (هر اسمی)
Mixed	یک default + چند named	`import Circle, { area } from "./shapes.js";`
همه با هم	(هر نوع)	`import * as MathUtils from "./math.js";`

نکته: پسوند `.js` در import فایل‌های TypeScript

در مثال‌ها فایل `math.ts` است ولی می‌نویسیم `from "./math.js"`. دلیلش این است که بعد از کامپایل، فایلی که واقعاً اجرا و import می‌شود `math.js` است و `tsc` خودش می‌فهمد منظور `math.ts` است. در بعضی پروژه‌ها (مثلاً با bundler) نوشتن پسوند لازم نیست.

دام تستی: آکولاد یا بدون آکولاد؟

- export **پیش‌فرض** ← **بدون** آکولاد و با **هر** اسمی: `import log from ...`
- export **نام‌دار** ← **با** آکولاد و با **همان** اسم (یا `as`): `import { add } from ...`
- `import { log } from "./logger.js"` برای یک export پیش‌فرض خطای TS2614 و `import add from "./math.js"` برای فایلی که پیش‌فرض ندارد خطای TS2613 می‌دهد.

ماژول‌ها هنگام اجرا: چهار واقعیت مهم

counter.mjs

```
console.log("counter.mjs runs");
export let count = 0;
export function increment() {
  count++;
}
```

helper.mjs

```
import { increment } from "./counter.mjs";
console.log("helper.mjs runs");
increment();
```

app.mjs

```
console.log("app starts");
import "./helper.mjs";
import { count, increment } from "./counter.mjs";
increment();
console.log("count =", count);
```

اجرای `node app.mjs` چاپ می‌کند:

```
counter.mjs runs
helper.mjs runs
app starts
count = 2
```

1. **importها اول اجرا می‌شوند:** مهم نیست خط import کجای فایل باشد؛ همه‌ی ماژول‌های importشده **قبل از** کدِ خودِ فایل اجرا می‌شوند. برای همین `app starts` سوم چاپ شد.
2. **هر ماژول فقط یک بار اجرا می‌شود:** `counter.mjs` را هم `helper.mjs` و هم `app.mjs` import کرده‌اند، ولی فقط یک بار اجرا شد و هر دو **یک نسخه‌ی مشترک** از `count` را می‌بینند.
3. **import زنده ولی فقط‌خواندنی است:** `count` در `app.mjs` مقدار به‌روز (`2`) را نشان می‌دهد، چون هر دو `increment` روی همان متغیر اثر گذاشته‌اند. اما خودِ import کننده نمی‌تواند به آن مقدار بدهد: `count = 5` در `app.mjs` خطای

`TypeError: Assignment to constant variable.` می‌دهد (و در TypeScript خطای TS2632).

4. **اسم اشتباه، یعنی هیچ خطی اجرا نمی‌شود:** اگر چیزی را import کنی که export نشده (مثلاً `import { decrement } from "./counter.mjs"`)، Node قبل از اجرای حتی یک خط، `SyntaxError: The requested module './counter.mjs' does not provide an export named 'decrement'` می‌دهد.

کد ESM همیشه در **strict mode** اجرا می‌شود و `this` در سطح بالای یک ماژول `undefined` است.

CommonJS در برابر ES Modules در Node.js

Node.js قبل از استاندارد شدن ESM، سیستم ماژول خودش را داشت به اسم **CommonJS** که هنوز در خیلی از پروژه‌ها و پکیج‌ها دیده می‌شود. در CommonJS با `module.exports` صادر و با تابع `require()` وارد می‌کنیم:

calc.cjs

```
function add(a, b) {
  return a + b;
}
const secret = 42; // not exported
module.exports = { add, PI: 3.14 };
```

index.cjs

```
const { add, PI } = require("./calc.cjs");
const calc = require("./calc.cjs");
console.log(add(2, 3), PI, calc.secret); // 5 3.14 undefined
```

	CommonJS	ES Modules (ESM)
صادر کردن	`module.exports = ...`	`export default` / `export`
وارد کردن	`require("./x")`	`import ... from "./x.js"`
در Node چه فایلی؟	`.cjs`، یا `.js` وقتی `package.json` فیلد `"type": "module"` ندارد	`.mjs`، یا `.js` وقتی `package.json` فیلد `"type": "module"` دارد
در مرورگر	پشتیبانی نمی‌شود	`<script type="module">`
strict mode	فقط با `"use strict"`	همیشه

برای این‌که فایل‌های `.js` یک پروژه‌ی Node.js به‌صورت ESM اجرا شوند، در `package.json` می‌نویسیم:

JSON

```
{
  "name": "my-app",
  "type": "module"
}
```

در یک فایل ESM تابع `require` وجود ندارد: نوشتن `require("fs")` در یک فایل `.mjs` خطای `ReferenceError: require is not defined in ES module scope` می‌دهد. (نسخه‌های جدید Node.js اگر در یک فایل `.js` بدون فیلد `"type"` دستور import/export ببینند، خودشان آن را ESM اجرا می‌کنند؛ ولی بهتر است همیشه صریحاً `"type": "module"` یا `.mjs` را به کار ببری.)

`<script type="module">` در مرورگر

در مرورگر هم برای استفاده از `import` / `export` باید به تگ script بگوییم که ماژول است:

```HTML
<script type="module" src="main.js"></script>
```

چنین اسکریپتی scope خصوصی خودش را دارد (متغیرهای سطح بالایش روی `window` نمی‌روند)، همیشه strict است، به‌طور خودکار مثل `defer` بعد از ساخته شدن کامل HTML اجرا می‌شود، و فقط وقتی صفحه از طریق یک سرور (http) باز شود کار می‌کند؛ باز کردن مستقیم فایل با `file://` معمولاً به خاطر قوانین امنیتی (CORS) خطا می‌دهد.

⚠ دام تستی: ترتیب اجرا

اگر در وسط فایل `console.log("start")` و بعد از آن یک `import` بنویسی، باز هم ماژول importشده **قبل از** `start` اجرا می‌شود؛ importها همیشه اول بارگذاری و اجرا می‌شوند.

جمع‌بندی

- ماژول = فایلی که `import` یا `export` دارد؛ هر ماژول scope خصوصی دارد. فایل بدون import/export یک script با scope مشترک است (TS2451 برای اسم تکراری).
- Named: `export const x` ← `import { x } from` (همان اسم، یا `as`). Default: حداکثر یکی ← `import anyName from` . Mixed: `import Def, { named } from` . همه: `import * as NS from` .
- خطاها: TS2459 (export نشده)، TS2305 (وجود ندارد)، TS2614 (پیش‌فرض با آکولاد)، TS2613 (فایل پیش‌فرض ندارد).
- هنگام اجرا: importها اول اجرا می‌شوند، هر ماژول یک بار اجرا می‌شود، importها فقط‌خواندنی‌اند، اسم export اشتباه یعنی `SyntaxError` قبل از اجرا.
- CommonJS: `require` و `module.exports` (`.cjs`). ESM در Node: `"type": "module"` یا `.mjs` ؛ در مرورگر: `<script type="module">` .

تست‌های تمرینی

۲۲ تست · شماره‌ی ۱۲۵۹ تا ۱۲۸۰

۱۲۵۹ آسان

طبق اسلاید ۲۳، کدام فایل یک **ماژول** است؟

۱) فایلی که فقط `let total = 0;` و یک تابع معمولی دارد.

۲) فایلی که یک کلاس تعریف کرده، ولی هیچ `import` یا `export` ی ندارد.

۳) فایلی که حداقل یک `import` یا `export` دارد.

۴) هر فایلی که با تگ `<script>` معمولی به صفحه‌ی HTML وصل شده باشد.

۱۲۶۰ آسان

یک فایل حداکثر چند **export پیش‌فرض** (`export default`) می‌تواند داشته باشد؟

۱) حداکثر یکی

۲) هر تعداد که بخواهیم

۳) دقیقاً یکی؛ فایلی که export پیش‌فرض ندارد ماژول حساب نمی‌شود

۴) دو تا: یکی برای تابع‌ها و یکی برای کلاس‌ها

۱۲۶۱ آسان

فایل `utils.ts` تابعی به اسم `formatDate` دارد. برای این‌که فایل `app.ts` بتواند از آن استفاده کند، چه باید کرد؟

۱) کافی است در `app.ts` بنویسیم `import { formatDate } from "./utils.js";` ؛ در `utils.ts` کاری لازم نیست.

۲) تابع را با `var` تعریف کنیم تا سراسری (global) شود.

۳) جلوی تابع کلیدواژه‌ی `public` بنویسیم.

۴) در `utils.ts` تابع را `export` کنیم و در `app.ts` آن را `import` کنیم.

۱۲۶۲ آسان

طبق اسلاید ۲۳، TypeScript و JavaScript مدرن از کدام سیستم ماژول استفاده می‌کنند؟

۱) CommonJS با `require` و `module.exports`

۲) ES Modules با `import` و `export`

۳) هیچ سیستم ماژولی ندارند و همه‌ی فایل‌ها یک scope مشترک دارند

۴) فقط متغیرهای سراسری روی شیء `window`

۱۲۶۳ آسان

با اجرای `node main.mjs` چه چیزی چاپ می‌شود؟

math.mjs

```
export const PI = 3.14;
export function double(x) {
  return x * 2;
}
```

logger.mjs

```
export default function log(message) {
  console.log("[LOG] " + message);
}
```

main.mjs

```
import { PI, double } from "./math.mjs";
import log from "./logger.mjs";
log(double(PI));
```

۱) `6.28`

۲) `[LOG] NaN`

۳) `[LOG] 6.28`

۴) `Uncaught SyntaxError`

۱۲۶۴ آسان

فایل `shapes.ts` یک export پیش‌فرض (کلاس `Circle`) و یک export نام‌دار (تابع `area`) دارد. کدام دستور import هر دو را **درست** وارد می‌کند؟

۱) `import { Circle, area } from "./shapes.js";`

۲) `import { area }, Circle from "./shapes.js";`

۳) `import Circle, area from "./shapes.js";`

۴) `import Circle, { area } from "./shapes.js";`

۱۲۶۵ متوسط

این دو فایل با هم کامپایل می‌شوند. خطای کامپایل در کدام خط است؟

math.ts

```ts
export const PI = 3.14;
export function add(a: number, b: number): number {
  return a + b;
}
const secret = 42;
```

main.ts

```ts
import { PI } from "./math.js";
import { add } from "./math.js";
import { secret } from "./math.js";
console.log(PI, add(1, 2), secret);
```

۱) خط ۵ از فایل `math.ts`

۲) خط ۱ از فایل `main.ts`

۳) خط ۳ از فایل `main.ts`

۴) خط ۴ از فایل `main.ts`

۱۲۶۶ متوسط

این دو فایل با هم کامپایل می‌شوند. خطای کامپایل در کدام خط است؟

logger.ts

```ts
export default function log(message: string): void {
  console.log(`[LOG] ${message}`);
}
```

main.ts

```ts
import { log } from "./logger.js";
import printer from "./logger.js";
import write from "./logger.js";
printer("a");
write("b");
```

۱) خط ۱ از فایل `main.ts`

۲) خط ۲ از فایل `main.ts`

۳) خط ۳ از فایل `main.ts`

۴) هیچ خطایی رخ نمی‌دهد

۱۲۶۷ متوسط

این دو فایل با هم کامپایل می‌شوند. خطای کامپایل در کدام خط است؟

math.ts

```ts
export const PI = 3.14;
export function add(a: number, b: number): number {
  return a + b;
}
```

main.ts

```ts
import * as M from "./math.js";
import add from "./math.js";
import { PI as pi } from "./math.js";
console.log(M.PI, pi);
```

۱) خط ۱ از فایل `main.ts`
۲) خط ۲ از فایل `main.ts`
۳) خط ۳ از فایل `main.ts`
۴) هیچ خطایی رخ نمی‌دهد

۱۲۶۸ متوسط

این دو فایل با هم کامپایل می‌شوند. خطای کامپایل در کدام خط است؟

math.ts

```ts
export const PI = 3.14;
export function add(a: number, b: number): number {
  return a + b;
}
```

main.ts

```ts
import { add as plus } from "./math.js";
import * as M from "./math.js";
import { PI, subtract } from "./math.js";
console.log(plus(1, 2), PI, M.add(3, 4));
```

۱) خط ۱ از فایل `main.ts`
۲) خط ۲ از فایل `main.ts`
۳) خط ۳ از فایل `main.ts`
۴) خط ۴ از فایل `main.ts`

۱۲۶۹ متوسط

با اجرای `node main.mjs` چه چیزی چاپ می‌شود؟

math.mjs

```
export const PI = 3.14;
export function add(a, b) {
  return a + b;
}
const secret = 42;
export function useSecret() {
  return secret;
}
```

main.mjs

```
import * as MathUtils from "./math.mjs";
import { add as sum } from "./math.mjs";
console.log(MathUtils.PI, sum(4, 5));
console.log(MathUtils.useSecret(), MathUtils.secret);
```

۱) `3.14 9` `42 42`

۲) `3.14 9` `Uncaught ReferenceError`

۳) `undefined 9` `42 undefined`

۴) `3.14 9` `42 undefined`

۱۲۷۰ متوسط

کدام گزینه درباره‌ی CommonJS و ES Modules در Node.js **درست** است؟

۱) `require` و `module.exports` مربوط به سیستم ES Modules هستند.

۲) با نوشتن `"type": "module"` در `package.json` (یا استفاده از پسوند `.mjs`)، فایل‌ها به‌صورت ES Module اجرا می‌شوند.

۳) پسوند `.cjs` یعنی فایل حتماً به‌صورت ES Module اجرا می‌شود.

۴) در یک ES Module می‌توان بدون هیچ تنظیمی از `require` استفاده کرد.

۱۲۷۱ متوسط

با اجرای `node main.cjs` چه چیزی چاپ می‌شود؟

math.cjs

```
function add(a, b) {
  return a + b;
}
const PI = 3.14;
const secret = 42;
module.exports = { add, PI };
```

main.cjs

```
const { add, PI } = require("./math.cjs");
const math = require("./math.cjs");
console.log(add(2, 3), math.PI, math.secret);
```

۱) `5 3.14 undefined`
۲) `5 3.14 42`
۳) `23 3.14 undefined`
۴) `Uncaught ReferenceError`

۱۲۷۲ متوسط

با اجرای `node main.mjs`، اولین خطا در کدام خط رخ می‌دهد؟

math.mjs

```
1  export const PI = 3.14;
```

main.mjs

```
1  import { PI } from "./math.mjs";
2  const fs = require("fs");
3  console.log(PI);
```

۱) خط ۱ از فایل `main.mjs`
۲) خط ۲ از فایل `main.mjs`
۳) خط ۳ از فایل `main.mjs`
۴) هیچ خطایی رخ نمی‌دهد

۱۲۷۳ متوسط

کدام خط کد زیر خطای کامپایل دارد؟

TypeScript

```
1  export default const settings = { port: 3000 };
2  export const host = "localhost";
3  export function url(): string {
4    return host;
5  }
```

۱) خط ۱
۲) خط ۲
۳) خط ۳
۴) هیچ خطایی رخ نمی‌دهد

۱۲۷۴ سخت

با اجرای `node main.mjs` چه چیزی چاپ می‌شود؟

counter.mjs

```
console.log("counter.mjs runs");
export let count = 0;
export function increment() {
  count++;
}
```

a.mjs

```
import { increment } from "./counter.mjs";
console.log("a.mjs runs");
increment();
```

main.mjs

```
console.log("main starts");
import "./a.mjs";
import { count, increment } from "./counter.mjs";
increment();
console.log("count =", count);
```

۱) `main starts` `counter.mjs runs` `a.mjs runs` `count = 2`

۲) `counter.mjs runs` `a.mjs runs` `counter.mjs runs` `main starts` `count = 1`

۳) `counter.mjs runs` `a.mjs runs` `main starts` `count = 1`

۴) `counter.mjs runs` `a.mjs runs` `main starts` `count = 2`

۱۲۷۵ سخت

با اجرای `node main.mjs` چه چیزی چاپ می‌شود؟

counter.mjs

```
export let count = 0;
export function increment() {
  count++;
}
```

main.mjs

```
import { count, increment } from "./counter.mjs";
increment();
console.log("before", count);
count = 5;
console.log("after", count);
```

۱) `before 0` `after 5`

۲) `before 1` `after 5`

۳) `before 1` `Uncaught TypeError`

۴) `before 0` `Uncaught TypeError`

۱۲۷۶ سخت

با اجرای `node main.mjs` چه چیزی چاپ می‌شود؟

counter.mjs

```
export let count = 0;
export function increment() {
  count++;
}
```

main.mjs

```
console.log("start");
import { decrement } from "./counter.mjs";
console.log("end");
```

۱) `Uncaught SyntaxError`

۲) `start` `Uncaught SyntaxError`

۳) `start` `Uncaught ReferenceError`

۴) `start` `end`

۱۲۷۷ سخت

با اجرای `node main.mjs`، خطا در کدام خط گزارش می‌شود؟

logger.mjs

```
1 export default function log(msg) {
2   console.log(msg);
3 }
4 export default function warn(msg) {
5   console.log("WARN: " + msg);
6 }
```

main.mjs

```
1 import log from "./logger.mjs";
2 log("hi");
```

۱) خط ۱ از فایل `main.mjs`

۲) خط ۲ از فایل `main.mjs`

۳) خط ۱ از فایل `logger.mjs`

۴) خط ۴ از فایل `logger.mjs`

۱۲۷۸ سخت

دو فایل زیر هیچ `import` یا `export` ی ندارند و با دستور `tsc --strict --module esnext a.ts b.ts` با هم کامپایل می‌شوند. نتیجه چیست؟

a.ts

```ts
let counter = 1;
```

b.ts

```ts
let counter = 2;
```

۱) هیچ خطایی رخ نمی‌دهد، چون هر فایل scope خصوصی خودش را دارد.

۲) خطایی در کامپایل رخ نمی‌دهد و فقط هنگام اجرا `ReferenceError` می‌گیریم.

۳) خطای TS2451 (تعریف دوباره‌ی متغیر `counter`) گزارش می‌شود، چون فایل‌های بدون «import/export script» هستند و scope مشترک دارند.

۴) خطایی رخ نمی‌دهد و مقدار `counter` در فایل دوم، بی‌صدا مقدار فایل اول را عوض می‌کند.

۱۲۷۹ سخت

این دو فایل با هم کامپایل می‌شوند. خطای کامپایل در کدام خط است؟

math.ts

```ts
export const PI = 3.14;
export function add(a: number, b: number): number {
  return a + b;
}
const secret = 42;
export function useSecret(): number {
  return secret;
}
```

main.ts

```ts
import * as M from "./math.js";
console.log(M.PI);
console.log(M.add(2, 3));
console.log(M.useSecret());
console.log(M.secret);
```

۱) خط ۵ از فایل `math.ts`

۲) خط ۱ از فایل `main.ts`

۳) خط ۴ از فایل `main.ts`

۴) خط ۵ از فایل `main.ts`

۱۲۸۰ سخت

در مرورگر، صفحه‌ی زیر سه تگ `<script>` معمولی (غیر ماژول) دارد. در کنسول چه چیزی چاپ می‌شود؟

HTML

```
<script>
  let counter = 1;
  console.log("first", counter);
</script>
<script>
  let counter = 2;
  console.log("second", counter);
</script>
<script>
  console.log("third", counter);
</script>
```

۱) `first 1` `second 2` `third 2`

۲) `first 1` `Uncaught SyntaxError` `third 1`

۳) `first 1` `second 2` `Uncaught ReferenceError`

۴) `first 1` `Uncaught SyntaxError`

۳-۱۵ بهترین شیوه‌ها (Best Practices) در TypeScript

منبع: NodeJS & TS، اسلاید ۲۴

درسنامه

بهترین شیوه یعنی چه؟

تا این‌جا TypeScript **قانون‌های** را یاد گرفتیم؛ قانون‌هایی که اگر رعایت نشوند، کامپایلر خطا می‌دهد. **Best practice** (بهترین شیوه) فرق دارد: کامپایلر مجبورت نمی‌کند و کدِ بد هم ممکن است بدون خطا کامپایل شود، ولی رعایتش باعث می‌شود پروژه‌ی بزرگ **قابل فهم، قابل تغییر و کم‌خطا** بماند (یادت هست در اسلایدهای ۳ و ۱۶ گفته شد نگه‌داری پروژه‌های بزرگ JavaScript سخت است).

اسلاید ۲۴ پنج توصیه دارد:

1. Use **interfaces** to define contracts for classes and objects.
2. Prefer `readonly` and `private` for encapsulation and immutability.
3. Use inheritance sparingly; prefer composition when possible.
4. Leverage generics for reusable, type-safe code.
5. Keep classes small and focused; follow the Single Responsibility Principle (SRP).

۱) interface به‌عنوان قرارداد (contract)

یک تشبیه ساده

پریز برق یک **استاندارد** دارد: هر وسیله‌ای که دوشاخه‌ی استاندارد داشته باشد (اتو، شارژر، یخچال) به آن وصل می‌شود و پریز اهمیتی نمی‌دهد سازنده‌ی وسیله کیست. interface همان استاندارد است: می‌گوید «هر کسی این اعضا را با این نوع‌ها داشته باشد، قابل استفاده است» و کاری به جزئیات پیاده‌سازی ندارد.

TypeScript

```typescript
interface PaymentMethod {
  pay(amount: number): string;
}
class CardPayment implements PaymentMethod {
  pay(amount: number): string {
    return `Paid ${amount} by card`;
  }
}
class WalletPayment implements PaymentMethod {
  pay(amount: number): string {
    return `Paid ${amount} from wallet`;
  }
}
// checkout depends on the contract, not on a concrete class
function checkout(method: PaymentMethod, total: number): void {
  console.log(method.pay(total));
}
checkout(new CardPayment(), 100);                          // Paid 100 by card
checkout(new WalletPayment(), 50);                         // Paid 50 from wallet
checkout({ pay: (a: number) => `Paid ${a} in cash` }, 20); // Paid 20 in cash
```

- تابع `checkout` فقط به **قرارداد** (`PaymentMethod`) وابسته است، نه به یک کلاس خاص. فردا اگر `CryptoPayment` اضافه شود، لازم نیست `checkout` حتی یک خط تغییر کند.
- قرارداد برای **اشیاء** هم هست: هر شیئی که متد `pay` با همین نوع را داشته باشد قبول است (خط آخر)، حتی اگر از هیچ کلاسی ساخته نشده باشد. شیئی که این قرارداد را رعایت نکند (مثلاً به‌جای `pay` متد `charge` داشته باشد) خطای کامپایل می‌گیرد.

۲) `readonly` و `private` برای encapsulation و immutability

- **Encapsulation** (کپسوله‌سازی): داده‌ی داخلی شیء را پنهان کن و فقط از راه متدهای کنترل‌شده اجازه‌ی تغییر بده. با `private` .
- **Immutability** (تغییرناپذیری): چیزی که بعد از ساخته شدن نباید عوض شود، از اول غیرقابل‌تغییر تعریف شود. با `readonly` .

TypeScript

```
class BankAccount {
  private balance = 0;         // hidden: only changed through deposit()
  readonly owner: string;      // fixed after construction
  constructor(owner: string) {
    this.owner = owner;
  }
  deposit(amount: number): void {
    if (amount <= 0) throw new Error("Invalid amount");
    this.balance += amount;
  }
  getBalance(): number {
    return this.balance;
  }
}
const acc = new BankAccount("Sara");
acc.deposit(200);
console.log(acc.owner, acc.getBalance()); // Sara 200
acc.balance = 1000000;  // Error TS2341: Property 'balance' is private and only accessible within
class 'BankAccount'.
acc.owner = "Hacker";   // Error TS2540: Cannot assign to 'owner' because it is a read-only
property.
```

اگر `balance` عمومی بود، هر جای برنامه می‌توانست مقدارش را منفی یا نامعقول کند؛ با `private` تنها راه تغییرش `deposit` است که مبلغ نامعتبر را رد می‌کند.

برای آرایه‌ها هم نسخه‌ی فقط‌خواندنی داریم: `readonly string[]` (یا `ReadonlyArray<string>`). روی چنین آرایه‌ای فقط متدهایی مجازند که آرایه را **تغییر نمی‌دهند** و آرایه‌ی جدید می‌سازند:

TypeScript

```
const days: readonly string[] = ["Sat", "Sun"];
const more = days.concat(["Mon"]);           // OK: returns a new array
const upper = days.map((d) => d.toUpperCase()); // OK: returns a new array
const sorted = [...days].sort();             // OK: sorts a copy
days.push("Mon");   // Error TS2339: Property 'push' does not exist on type 'readonly string[]'.
days[0] = "Fri";    // Error TS2542: Index signature in type 'readonly string[]' only permits
reading.
days.sort();        // Error TS2339: Property 'sort' does not exist on type 'readonly string[]'.
```

> ⚠ **دام تستی: `readonly` سطحی است**
>
> `private readonly items: string[] = []` فقط جلوی **عوض کردن خودِ property** (`this.items = []`، خطای TS2540) را می‌گیرد؛ تغییر محتوای همان آرایه (`this.items.push(...)` یا `this.items.length = 0`) مجاز است. برای قفل کردن محتوا باید نوع را هم `readonly string[]` کنی. یادت هم باشد `private` فقط در زمان کامپایل بررسی می‌شود.

۳) وراثت کم، ترکیب (composition) بیشتر

دو جور رابطه بین کلاس‌ها داریم:

رابطه	معنی	مثال	ابزار
is-a («... یک ... است»)	زیرنوعِ واقعی	دانشجو یک انسان است	وراثت (`extends`)
has-a («... یک ... دارد»)	جزئی از آن است	خودرو یک موتور دارد	ترکیب (composition)

Composition (ترکیب) یعنی به‌جای ارث بردن از یک کلاس، **یک شیء از آن را داخل کلاس خودت نگه داری** و کارها را به آن بسپاری.
چرا اسلاید می‌گوید «از وراثت با احتیاط استفاده کن»؟

- `class Car extends Engine` یعنی «خودرو **یک** موتور است» که از نظر معنا غلط است و خودرو همه‌ی متدهای موتور را هم بی‌دلیل به ارث می‌برد.
- وراثت کلاس‌ها را **محکم به هم می‌چسباند**: تغییر کلاس والد ممکن است همه‌ی فرزندان را خراب کند، و زنجیره‌های عمیق وراثت فهمیدن کد را سخت می‌کنند.
- هر کلاس فقط **از یک کلاس** می‌تواند `extends` کند، ولی می‌تواند هر تعداد شیء را در خودش ترکیب کند.

TypeScript

```typescript
class Engine {
  start(): string {
    return "engine started";
  }
}
class Radio {
  play(): string {
    return "music on";
  }
}
// Composition: a Car HAS an Engine and a Radio
class Car {
  private engine = new Engine();
  private radio = new Radio();
  drive(): void {
    console.log(this.engine.start());
    console.log(this.radio.play());
    console.log("driving");
  }
}
const car = new Car();
car.drive();                          // engine started / music on / driving
console.log(car instanceof Engine);  // false: a Car is not an Engine
```

مثال کلاسیک دیگر: `class Stack<T> extends Array<T> {}` بدون خطا کامپایل می‌شود، ولی آن‌وقت پشته همه‌ی متدهای آرایه (`splice`، `shift`، `unshift` و ...) را دارد و هر کسی می‌تواند قانون LIFO را بشکند. با ترکیب (`private items: T[] = []` داخل کلاس، مثل `Stack<T>` در درس Generics) فقط `push` و `pop` بیرون دیده می‌شوند.

۴) generics برای کد قابل استفاده‌ی دوباره و type-safe

به‌جای نوشتن `ProductRepository`، `UserRepository` و ... با کد تقریباً یکسان (تکرار)، یا یک کلاس با `any[]` (گم شدن نوع)، **یک** کلاس generic بنویس:

TypeScript

```
interface Entity {
  id: number;
}
class Repository<T extends Entity> {
  private items: T[] = [];
  add(item: T): void {
    this.items.push(item);
  }
  findById(id: number): T | undefined {
    return this.items.find((x) => x.id === id);
  }
}
interface Product extends Entity {
  title: string;
}
const products = new Repository<Product>();
products.add({ id: 1, title: "Book" });
console.log(products.findById(1)?.title); // Book
console.log(products.findById(2));        // undefined
products.add({ id: 2 });                  // Error TS2741: Property 'title' is missing
const names = new Repository<string>();   // Error TS2344: Type 'string' does not satisfy the
constraint 'Entity'.
```

(`?.` یعنی «اگر نتیجه `undefined` یا `null` نبود، `title` را بخوان؛ وگرنه `undefined` بده».) این‌جا سه توصیه با هم کار می‌کنند: interface (`Entity`) قرارداد است، generic (`<T extends Entity>`) کد را برای هر نوع موجودیتی قابل استفاده می‌کند و `private items` داده را کپسوله می‌کند.

۵) کلاس‌های کوچک و متمرکز: Single Responsibility Principle (SRP)

SRP (اصل تک‌مسئولیتی) می‌گوید: **هر کلاس فقط یک مسئولیت داشته باشد**؛ یا به بیان دیگر، فقط **یک دلیل** برای تغییر کردن داشته باشد. کلاسی که همه‌کار می‌کند (به آن «god class» می‌گویند) بزرگ، شکننده و سخت‌آزمون می‌شود.

TypeScript

```typescript
// Bad: one class with four unrelated jobs
class UserManager {
  validateEmail(email: string): boolean { return email.includes("@"); }
  saveToDatabase(name: string): void { /* SQL code */ }
  sendWelcomeEmail(email: string): void { /* SMTP code */ }
  renderProfileHtml(name: string): string { return `<h1>${name}</h1>`; }
}

// Good: small, focused classes (each has one reason to change)
class EmailValidator {
  isValid(email: string): boolean {
    return email.includes("@");
  }
}
class UserRepository {
  save(name: string): void { /* database code only */ }
}
class WelcomeMailer {
  send(email: string): void { /* email code only */ }
}
```

اگر روزی فرمت ایمیل‌ها یا نوع پایگاه داده عوض شود، فقط **یک** کلاس کوچک تغییر می‌کند. کلاس‌های کوچک را بعداً می‌توان با **ترکیب** کنار هم گذاشت (مثلاً یک `UserService` که این سه را در خودش نگه می‌دارد).

نکته: پنج توصیه در یک نگاه

توصیه	چرا؟	نمونه
interface = قرارداد	وابستگی به «شکل»، نه کلاس خاص	`checkout(method: PaymentMethod)`
`readonly` و `private`	جلوگیری از تغییر ناخواسته	`private balance` ، `readonly owner`
ترکیب به‌جای وراثت	رابطه‌ی has-a، وابستگی کمتر	`Car` یک `Engine` دارد
generics	بدون تکرار و بدون `any`	`Repository<T>` ، `Stack<T>`
SRP	کلاس کوچک، یک دلیل برای تغییر	`EmailValidator` ، `UserRepository`

دام تستی: «کامپایل می‌شود» یعنی «خوب طراحی شده»؟ نه!

`class Car extends Engine` ، یک `UserManager` همه‌کاره یا یک `Stack` با `any[]` همگی **بدون خطا** کامپایل می‌شوند. در تست‌های این بخش سؤال این است که کدام طراحی **با توصیه‌های اسلاید ۲۴ سازگارتر** است، نه کدام کد خطا دارد.

جمع‌بندی

- interface را به‌عنوان قرارداد کلاس‌ها و اشیاء به کار ببر؛ کد را به قرارداد وابسته کن، نه به کلاس خاص.
- داده‌ی داخلی را `private` و چیزهای ثابت را `readonly` کن (TS2341 و TS2540 از بیرون)؛ `readonly` سطحی است و برای آرایه `readonly T[]` داریم.
- is-a → وراثت (با احتیاط)؛ has-a → ترکیب (ترجیح اسلاید).
- generics (مثل `Repository<T extends Entity>`) جای کد تکراری و `any` را می‌گیرد.
- SRP: هر کلاس یک مسئولیت؛ کلاس همه‌کاره را به چند کلاس کوچک بشکن.

تست‌های تمرینی

۱۵ تست · شماره‌ی ۱۲۸۱ تا ۱۲۹۵

۱۲۸۱ آسان

اسلاید ۲۴ می‌گوید «از interfaceها برای تعریف **قرارداد (contract)** برای کلاس‌ها و اشیاء استفاده کنید». منظور از قرارداد چیست؟

۱) interface مشخص می‌کند یک کلاس یا شیء **چه اعضایی با چه نوعی** باید داشته باشد، بدون این‌که پیاده‌سازی آن‌ها را تعیین کند.

۲) interface کد مشترک متدها را می‌نویسد تا کلاس‌ها آن را به ارث ببرند.

۳) interface در زمان اجرا جلوی ساخته شدن اشیای نادرست را می‌گیرد.

۴) interface کلاسی است که نمی‌توان از آن شیء ساخت، ولی متدهایش کامل پیاده‌سازی شده‌اند.

۱۲۸۲ آسان

در کدام جفت، رابطه‌ی درست «has-a» (… یک … دارد) است و طبق اسلاید ۲۴ بهتر است با **ترکیب (composition)** پیاده شود، نه وراثت؟

۱) `Student` و `Person` (دانشجو یک انسان است)

۲) `Dog` و `Animal` (سگ یک حیوان است)

۳) `Car` و `Engine` (خودرو یک موتور دارد)

۴) `Square` و `Shape` (مربع یک شکل است)

۱۲۸۳ آسان

Single Responsibility Principle (SRP) که در اسلاید ۲۴ آمده، چه می‌گوید؟

۱) هر کلاس باید فقط یک متد داشته باشد.

۲) هر کلاس باید فقط **یک مسئولیت** (و در نتیجه فقط یک دلیل برای تغییر) داشته باشد.

۳) از هر کلاس فقط یک شیء باید ساخته شود.

۴) هر کلاس فقط باید از یک کلاس دیگر ارث ببرد.

۱۲۸۴ آسان

در کلاس `BankAccount`، property `balance` را `private` کرده‌ایم و فقط از طریق متد `deposit(amount)` (که مبلغ منفی یا صفر را رد می‌کند) تغییرش می‌دهیم. این کار نمونه‌ی کدام اصل است؟

۱) وراثت؛ تا کلاس‌های فرزند بتوانند `balance` را مستقیم تغییر دهند.

۲) generics؛ تا `balance` بتواند هر نوعی داشته باشد.

۳) SRP؛ چون کلاس فقط یک property دارد.

۴) کپسوله‌سازی (encapsulation)؛ تا کد بیرونی نتواند مستقیماً مقدار نامعتبری در `balance` بگذارد.

۱۲۸۵ متوسط

کدام خط کد زیر خطای کامپایل دارد؟

```TypeScript
class BankAccount {
  private balance = 0;
  readonly owner: string;
  constructor(owner: string) {
    this.owner = owner;
  }
  deposit(amount: number): void {
    if (amount <= 0) throw new Error("Invalid amount");
    this.balance += amount;
  }
  getBalance(): number {
    return this.balance;
  }
}
const acc = new BankAccount("Sara");
acc.deposit(200);
console.log(acc.owner, acc.getBalance());
acc.balance = 1000000;
```

۱) خط ۹ ۲) خط ۱۶ ۳) خط ۱۷ ۴) خط ۱۸

۱۲۸۶ متوسط

کدام دستور را می‌توان در جای خالی نوشت تا کد **بدون خطا** کامپایل شود؟

```TypeScript
const days: readonly string[] = ["Sat", "Sun"];
// ____
```

۱) `days.push("Mon");` ۲) `days[0] = "Fri";`

۳) `const more = days.concat(["Mon"]);` ۴) `days.sort();`

۱۲۸۷ متوسط

کلاس `UserManager` این چهار کار را انجام می‌دهد: بررسی درستی ایمیل، ذخیره‌ی کاربر در پایگاه داده، ارسال ایمیل خوش‌آمد و ساختن HTML صفحه‌ی پروفایل. طبق اسلاید ۲۴، بهترین اصلاح کدام است؟

۱) کلاس را به چند کلاس کوچک و متمرکز تقسیم کنیم؛ مثلاً `EmailValidator`، `UserRepository`، `WelcomeMailer` و `ProfileView`.

۲) همه را در همان کلاس نگه داریم ولی همه‌ی متدها را `private` کنیم.

۳) یک کلاس پایه بسازیم و `UserManager` را از آن `extends` کنیم تا کد کوتاه‌تر شود.

۴) همه‌ی propertyهای کلاس را `readonly` کنیم.

۱۲۸۸ متوسط

در یک پروژه به یک پشته (stack) برای عددها و یک پشته برای رشته‌ها نیاز داریم. طبق توصیه‌های اسلاید ۲۴ کدام طراحی بهتر است؟

۱) دو کلاس جدا به اسم‌های `NumberStack` و `StringStack` با کد تقریباً یکسان.

۲) یک کلاس `Stack` که آرایه‌ی داخلی‌اش از نوع `any[]` است.

۳) یک کلاس `NumberStack` و یک کلاس `StringStack extends NumberStack`.

۴) یک کلاس generic به شکل `class Stack<T>` و ساختن `new Stack<number>()` و `new Stack<string>()`.

۱۲۸۹ متوسط

خروجی کد زیر چیست؟

```TypeScript
class Engine {
  start(): string {
    return "engine started";
  }
}
class Car {
  private engine = new Engine();
  drive(): void {
    console.log(this.engine.start());
    console.log("driving");
  }
}
const car = new Car();
car.drive();
console.log(car instanceof Engine);
```

۱) `engine started` `driving` `true`

۲) `engine started` `driving` `false`

۳) `driving` `false`

۴) `Uncaught TypeError`

۱۲۹۰ متوسط

کدام خط کد زیر خطای کامپایل دارد؟

```TypeScript
class Engine {
  start(): string {
    return "engine started";
  }
}
class Car {
  private engine = new Engine();
  drive(): string {
    return this.engine.start() + ", driving";
  }
}
const car = new Car();
console.log(car.drive());
console.log(car.start());
```

۱) خط ۹
۲) خط ۱۳
۳) خط ۱۴
۴) هیچ خطایی رخ نمی‌دهد

۱۲۹۱ متوسط

کدام خط کد زیر خطای کامپایل دارد؟

```TypeScript
interface PaymentMethod {
  pay(amount: number): string;
}
class CardPayment implements PaymentMethod {
  pay(amount: number): string {
    return `Paid ${amount} by card`;
  }
}
function checkout(method: PaymentMethod, total: number): void {
  console.log(method.pay(total));
}
checkout(new CardPayment(), 100);
checkout({ pay: (a: number) => `Paid ${a} in cash` }, 20);
checkout({ charge: (a: number) => `Charged ${a}` }, 30);
```

۱) خط ۴
۲) خط ۱۲
۳) خط ۱۳
۴) خط ۱۴

۱۲۹۲ سخت

کدام خط کد زیر خطای کامپایل دارد؟

```TypeScript
class Cart {
  private readonly items: string[] = [];
  add(item: string): void {
    this.items.push(item);
  }
  clear(): void {
    this.items.length = 0;
  }
  reset(): void {
    this.items = [];
  }
}
```

۱) خط ۴
۲) خط ۷
۳) خط ۱۰
۴) هیچ خطایی رخ نمی‌دهد

۱۲۹۳ سخت

با تعریف‌های زیر، کدام گزینه **خطای کامپایل** دارد؟

```TypeScript
interface Entity {
  id: number;
}
class Repository<T extends Entity> {
  private items: T[] = [];
  add(item: T): void {
    this.items.push(item);
  }
}
interface Product extends Entity {
  title: string;
}
```

۱) `const r = new Repository<string>();`
۲) `new Repository<Product>().add({ id: 1, title: "Book" });`
۳) `new Repository<Entity>().add({ id: 2 });`
۴) `new Repository<{ id: number; name: string }>().add({ id: 3, name: "Pen" });`

۱۲۹۴ سخت

کدام طراحی با توصیه‌های اسلاید ۲۴ **در تضاد** است؟

۱) تعریف interface `Logger` و نوشتن تابع‌هایی که به‌جای یک کلاس مشخص، هر `Logger` ی را بپذیرند.

۲) ساختن زنجیره‌ی وراثت عمیق (`D extends C` ، `C extends B` ، `B extends A`) فقط برای این‌که چند متد دوباره استفاده شود.

۳) `readonly` کردن شناسه‌ی (`id`) یک شیء که بعد از ساخته شدن نباید عوض شود.

۴) نوشتن یک کلاس `Cache<T>` به‌جای دو کلاس جداگانه‌ی `StringCache` و `NumberCache` .

۱۲۹۵ سخت

برنامه‌نویسی پشته را این‌طور نوشته است: `class Stack<T> extends Array<T> {}` . این کد بدون خطا کامپایل می‌شود. طبق توصیه‌های اسلاید ۲۴، مشکل اصلی این طراحی چیست؟

۱) `Stack` همه‌ی متدهای آرایه (مثل `splice` ، `shift` و `unshift`) را به ارث می‌برد و هر کسی می‌تواند قانون LIFO را بشکند؛ ترکیب (یک آرایه‌ی `private` داخل کلاس) فقط متدهای لازم را بیرون نشان می‌دهد.

۲) در TypeScript نمی‌توان از کلاس‌های آماده‌ای مثل `Array` ارث برد.

۳) وراثت باعث می‌شود generic بودن `Stack<T>` از بین برود و همه‌چیز `any` شود.

۴) با `extends Array` دیگر نمی‌توان `push` و `pop` را روی پشته صدا زد.

بخش پایانی

پاسخنامه‌ی تشریحی

برای هر تست، گزینه‌ی درست همراه با دلیل درستی آن و دلیل نادرستی گزینه‌های وسوسه‌انگیز آمده است. شماره‌ها با شماره‌ی تست‌ها در متن کتاب یکی است؛ روی شماره‌ی هر پاسخ بزنید تا به خود تست برگردید.

پاسخ‌های فصل اول: مقدمه‌ای بر JavaScript

کلید سریع فصل اول

۱ ۳	۲ ۳	۳ ۲	۴ ۱	۵ ۲	۶ ۴	۷ ۲	۸ ۱	۹ ۴	۱۰ ۱
۱۱ ۳	۱۲ ۲	۱۳ ۴	۱۴ ۴	۱۵ ۳	۱۶ ۲	۱۷ ۴	۱۸ ۲	۱۹ ۳	۲۰ ۳
۲۱ ۱	۲۲ ۴	۲۳ ۴	۲۴ ۱	۲۵ ۳	۲۶ ۱	۲۷ ۲	۲۸ ۱	۲۹ ۲	۳۰ ۴
۳۱ ۳	۳۲ ۱	۳۳ ۳	۳۴ ۴	۳۵ ۲	۳۶ ۳	۳۷ ۲	۳۸ ۴	۳۹ ۱	۴۰ ۳
۴۱ ۴	۴۲ ۲	۴۳ ۳	۴۴ ۴	۴۵ ۱	۴۶ ۴	۴۷ ۲	۴۸ ۱	۴۹ ۳	۵۰ ۲
۵۱ ۱	۵۲ ۲	۵۳ ۴	۵۴ ۴	۵۵ ۳	۵۶ ۴	۵۷ ۳	۵۸ ۲	۵۹ ۴	۶۰ ۳
۶۱ ۱	۶۲ ۳	۶۳ ۲	۶۴ ۳	۶۵ ۴	۶۶ ۱	۶۷ ۱	۶۸ ۲	۶۹ ۴	۷۰ ۱
۷۱ ۱	۷۲ ۱	۷۳ ۲	۷۴ ۲	۷۵ ۴	۷۶ ۴	۷۷ ۳	۷۸ ۱	۷۹ ۲	۸۰ ۱
۸۱ ۳	۸۲ ۲	۸۳ ۴	۸۴ ۲	۸۵ ۳	۸۶ ۴	۸۷ ۴	۸۸ ۲	۸۹ ۳	۹۰ ۱
۹۱ ۳	۹۲ ۳	۹۳ ۴	۹۴ ۲	۹۵ ۴	۹۶ ۱	۹۷ ۳	۹۸ ۱	۹۹ ۳	۱۰۰ ۱
۱۰۱ ۳	۱۰۲ ۲	۱۰۳ ۱	۱۰۴ ۲	۱۰۵ ۴	۱۰۶ ۲	۱۰۷ ۳	۱۰۸ ۴	۱۰۹ ۱	۱۱۰ ۳
۱۱۱ ۴	۱۱۲ ۲	۱۱۳ ۳	۱۱۴ ۲	۱۱۵ ۴	۱۱۶ ۳	۱۱۷ ۱	۱۱۸ ۱	۱۱۹ ۴	۱۲۰ ۲
۱۲۱ ۳	۱۲۲ ۱	۱۲۳ ۴	۱۲۴ ۱	۱۲۵ ۲	۱۲۶ ۴	۱۲۷ ۱	۱۲۸ ۳	۱۲۹ ۲	۱۳۰ ۴
۱۳۱ ۳	۱۳۲ ۱	۱۳۳ ۲	۱۳۴ ۱	۱۳۵ ۲	۱۳۶ ۴	۱۳۷ ۳	۱۳۸ ۳	۱۳۹ ۱	۱۴۰ ۲
۱۴۱ ۴	۱۴۲ ۴	۱۴۳ ۳	۱۴۴ ۲	۱۴۵ ۱	۱۴۶ ۳	۱۴۷ ۲	۱۴۸ ۴	۱۴۹ ۴	۱۵۰ ۱
۱۵۱ ۱	۱۵۲ ۴	۱۵۳ ۱	۱۵۴ ۲	۱۵۵ ۲	۱۵۶ ۳	۱۵۷ ۳	۱۵۸ ۲	۱۵۹ ۳	۱۶۰ ۱
۱۶۱ ۴	۱۶۲ ۲	۱۶۳ ۳	۱۶۴ ۴	۱۶۵ ۱	۱۶۶ ۲	۱۶۷ ۲	۱۶۸ ۴	۱۶۹ ۳	۱۷۰ ۱
۱۷۱ ۱	۱۷۲ ۳	۱۷۳ ۴	۱۷۴ ۳	۱۷۵ ۳	۱۷۶ ۴	۱۷۷ ۲	۱۷۸ ۲	۱۷۹ ۲	۱۸۰ ۱
۱۸۱ ۳	۱۸۲ ۴	۱۸۳ ۲	۱۸۴ ۲	۱۸۵ ۱	۱۸۶ ۱	۱۸۷ ۴	۱۸۸ ۲	۱۸۹ ۴	۱۹۰ ۳
۱۹۱ ۱	۱۹۲ ۱	۱۹۳ ۳	۱۹۴ ۲	۱۹۵ ۳	۱۹۶ ۴	۱۹۷ ۲	۱۹۸ ۴	۱۹۹ ۳	۲۰۰ ۲
۲۰۱ ۱	۲۰۲ ۱	۲۰۳ ۴	۲۰۴ ۳	۲۰۵ ۱	۲۰۶ ۲	۲۰۷ ۴	۲۰۸ ۱	۲۰۹ ۳	۲۱۰ ۲
۲۱۱ ۴	۲۱۲ ۱	۲۱۳ ۳	۲۱۴ ۲	۲۱۵ ۲	۲۱۶ ۳	۲۱۷ ۱	۲۱۸ ۲	۲۱۹ ۴	۲۲۰ ۳
۲۲۱ ۴	۲۲۲ ۱	۲۲۳ ۴	۲۲۴ ۲	۲۲۵ ۲	۲۲۶ ۳	۲۲۷ ۴	۲۲۸ ۲	۲۲۹ ۱	۲۳۰ ۴
۲۳۱ ۲	۲۳۲ ۳	۲۳۳ ۱	۲۳۴ ۱	۲۳۵ ۳	۲۳۶ ۲	۲۳۷ ۱	۲۳۸ ۴	۲۳۹ ۳	۲۴۰ ۲
۲۴۱ ۴	۲۴۲ ۲	۲۴۳ ۴	۲۴۴ ۳	۲۴۵ ۳	۲۴۶ ۴	۲۴۷ ۳	۲۴۸ ۱	۲۴۹ ۴	۲۵۰ ۱
۲۵۱ ۲	۲۵۲ ۴	۲۵۳ ۱	۲۵۴ ۲	۲۵۵ ۱	۲۵۶ ۳	۲۵۷ ۴	۲۵۸ ۲	۲۵۹ ۱	۲۶۰ ۳
۲۶۱ ۴	۲۶۲ ۲	۲۶۳ ۳	۲۶۴ ۱	۲۶۵ ۴	۲۶۶ ۲	۲۶۷ ۱	۲۶۸ ۳	۲۶۹ ۴	۲۷۰ ۳
۲۷۱ ۲	۲۷۲ ۱	۲۷۳ ۴	۲۷۴ ۳	۲۷۵ ۲	۲۷۶ ۱	۲۷۷ ۴	۲۷۸ ۳	۲۷۹ ۴	۲۸۰ ۳
۲۸۱ ۲	۲۸۲ ۴	۲۸۳ ۱	۲۸۴ ۲	۲۸۵ ۳	۲۸۶ ۱	۲۸۷ ۴	۲۸۸ ۲	۲۸۹ ۱	۲۹۰ ۳
۲۹۱ ۴	۲۹۲ ۱	۲۹۳ ۲	۲۹۴ ۳	۲۹۵ ۴	۲۹۶ ۳	۲۹۷ ۲	۲۹۸ ۱	۲۹۹ ۴	۳۰۰ ۳
۳۰۱ ۲	۳۰۲ ۴	۳۰۳ ۱	۳۰۴ ۳	۳۰۵ ۲	۳۰۶ ۳	۳۰۷ ۴	۳۰۸ ۴	۳۰۹ ۱	۳۱۰ ۱
۳۱۱ ۲	۳۱۲ ۴	۳۱۳ ۳	۳۱۴ ۱	۳۱۵ ۲	۳۱۶ ۴	۳۱۷ ۳	۳۱۸ ۲	۳۱۹ ۴	۳۲۰ ۱
۳۲۱ ۳	۳۲۲ ۲	۳۲۳ ۴	۳۲۴ ۱	۳۲۵ ۳	۳۲۶ ۲	۳۲۷ ۱	۳۲۸ ۳	۳۲۹ ۴	۳۳۰ ۳

۳۳۱ ۲	۳۳۲ ۲	۳۳۳ ۱	۳۳۴ ۴	۳۳۵ ۳	۳۳۶ ۱	۳۳۷ ۳	۳۳۸ ۲	۳۳۹ ۴	۳۴۰ ۴
۳۴۱ ۳	۳۴۲ ۲	۳۴۳ ۱	۳۴۴ ۲	۳۴۵ ۱	۳۴۶ ۳	۳۴۷ ۴	۳۴۸ ۱	۳۴۹ ۳	۳۵۰ ۴
۳۵۱ ۲	۳۵۲ ۱	۳۵۳ ۲	۳۵۴ ۴	۳۵۵ ۳	۳۵۶ ۱	۳۵۷ ۴	۳۵۸ ۳	۳۵۹ ۲	۳۶۰ ۴
۳۶۱ ۴	۳۶۲ ۱	۳۶۳ ۲	۳۶۴ ۳	۳۶۵ ۱	۳۶۶ ۱	۳۶۷ ۳	۳۶۸ ۱	۳۶۹ ۱	۳۷۰ ۲
۳۷۱ ۴	۳۷۲ ۳	۳۷۳ ۴	۳۷۴ ۲	۳۷۵ ۴	۳۷۶ ۲	۳۷۷ ۳	۳۷۸ ۱	۳۷۹ ۲	۳۸۰ ۳
۳۸۱ ۴	۳۸۲ ۱	۳۸۳ ۲	۳۸۴ ۳	۳۸۵ ۲	۳۸۶ ۴	۳۸۷ ۱	۳۸۸ ۱	۳۸۹ ۲	۳۹۰ ۳
۳۹۱ ۴	۳۹۲ ۲	۳۹۳ ۳	۳۹۴ ۱	۳۹۵ ۲	۳۹۶ ۴	۳۹۷ ۱	۳۹۸ ۳	۳۹۹ ۴	۴۰۰ ۳
۴۰۱ ۲	۴۰۲ ۳	۴۰۳ ۳	۴۰۴ ۲	۴۰۵ ۱	۴۰۶ ۳	۴۰۷ ۱	۴۰۸ ۴	۴۰۹ ۴	۴۱۰ ۱
۴۱۱ ۱	۴۱۲ ۱	۴۱۳ ۳	۴۱۴ ۲	۴۱۵ ۲	۴۱۶ ۳	۴۱۷ ۴	۴۱۸ ۲	۴۱۹ ۴	۴۲۰ ۴
۴۲۱ ۱	۴۲۲ ۲	۴۲۳ ۲	۴۲۴ ۴	۴۲۵ ۳	۴۲۶ ۴	۴۲۷ ۲	۴۲۸ ۱	۴۲۹ ۳	۴۳۰ ۳
۴۳۱ ۴	۴۳۲ ۱	۴۳۳ ۳	۴۳۴ ۴	۴۳۵ ۲	۴۳۶ ۲	۴۳۷ ۴	۴۳۸ ۲	۴۳۹ ۱	۴۴۰ ۴
۴۴۱ ۴	۴۴۲ ۳	۴۴۳ ۱	۴۴۴ ۴	۴۴۵ ۱	۴۴۶ ۴	۴۴۷ ۳	۴۴۸ ۱	۴۴۹ ۲	۴۵۰ ۳
۴۵۱ ۳	۴۵۲ ۴	۴۵۳ ۱	۴۵۴ ۳	۴۵۵ ۱	۴۵۶ ۱	۴۵۷ ۴	۴۵۸ ۲	۴۵۹ ۲	۴۶۰ ۳
۴۶۱ ۳	۴۶۲ ۱	۴۶۳ ۲	۴۶۴ ۳	۴۶۵ ۱	۴۶۶ ۴	۴۶۷ ۲	۴۶۸ ۳	۴۶۹ ۱	۴۷۰ ۲
۴۷۱ ۲	۴۷۲ ۴	۴۷۳ ۲	۴۷۴ ۳	۴۷۵ ۱	۴۷۶ ۱	۴۷۷ ۲	۴۷۸ ۳	۴۷۹ ۴	۴۸۰ ۴
۴۸۱ ۱	۴۸۲ ۳	۴۸۳ ۳	۴۸۴ ۴	۴۸۵ ۲	۴۸۶ ۱	۴۸۷ ۳	۴۸۸ ۴	۴۸۹ ۲	۴۹۰ ۱
۴۹۱ ۱	۴۹۲ ۲	۴۹۳ ۴	۴۹۴ ۲	۴۹۵ ۳	۴۹۶ ۱	۴۹۷ ۳	۴۹۸ ۴	۴۹۹ ۲	۵۰۰ ۴
۵۰۱ ۱	۵۰۲ ۱	۵۰۳ ۲	۵۰۴ ۳	۵۰۵ ۳	۵۰۶ ۱	۵۰۷ ۳	۵۰۸ ۲	۵۰۹ ۲	۵۱۰ ۴
۵۱۱ ۱	۵۱۲ ۳	۵۱۳ ۴	۵۱۴ ۱	۵۱۵ ۴	۵۱۶ ۳	۵۱۷ ۲	۵۱۸ ۳	۵۱۹ ۱	۵۲۰ ۲
۵۲۱ ۴	۵۲۲ ۱	۵۲۳ ۴	۵۲۴ ۲	۵۲۵ ۳	۵۲۶ ۳	۵۲۷ ۱	۵۲۸ ۴	۵۲۹ ۳	۵۳۰ ۱
۵۳۱ ۲	۵۳۲ ۳	۵۳۳ ۱	۵۳۴ ۲	۵۳۵ ۴	۵۳۶ ۳	۵۳۷ ۲	۵۳۸ ۴	۵۳۹ ۱	۵۴۰ ۳
۵۴۱ ۱	۵۴۲ ۲	۵۴۳ ۱	۵۴۴ ۲	۵۴۵ ۳	۵۴۶ ۴	۵۴۷ ۳	۵۴۸ ۲	۵۴۹ ۴	۵۵۰ ۲
۵۵۱ ۳	۵۵۲ ۴	۵۵۳ ۱	۵۵۴ ۴	۵۵۵ ۱	۵۵۶ ۲	۵۵۷ ۲	۵۵۸ ۳	۵۵۹ ۴	۵۶۰ ۱
۵۶۱ ۲	۵۶۲ ۴	۵۶۳ ۱	۵۶۴ ۳	۵۶۵ ۲	۵۶۶ ۴	۵۶۷ ۱	۵۶۸ ۱	۵۶۹ ۲	۵۷۰ ۳
۵۷۱ ۱	۵۷۲ ۴	۵۷۳ ۳	۵۷۴ ۱	۵۷۵ ۳	۵۷۶ ۲	۵۷۷ ۲	۵۷۸ ۱	۵۷۹ ۴	۵۸۰ ۳
۵۸۱ ۴	۵۸۲ ۲	۵۸۳ ۲	۵۸۴ ۳	۵۸۵ ۱	۵۸۶ ۴	۵۸۷ ۲	۵۸۸ ۳	۵۸۹ ۴	۵۹۰ ۱
۵۹۱ ۲	۵۹۲ ۴	۵۹۳ ۱	۵۹۴ ۳	۵۹۵ ۲	۵۹۶ ۱	۵۹۷ ۴	۵۹۸ ۳	۵۹۹ ۲	۶۰۰ ۳
۶۰۱ ۱	۶۰۲ ۴	۶۰۳ ۳	۶۰۴ ۱	۶۰۵ ۴	۶۰۶ ۲	۶۰۷ ۴	۶۰۸ ۱	۶۰۹ ۲	۶۱۰ ۴
۶۱۱ ۲	۶۱۲ ۱	۶۱۳ ۳	۶۱۴ ۳	۶۱۵ ۲	۶۱۶ ۱	۶۱۷ ۴	۶۱۸ ۱	۶۱۹ ۳	۶۲۰ ۴
۶۲۱ ۲	۶۲۲ ۳	۶۲۳ ۲	۶۲۴ ۱	۶۲۵ ۴	۶۲۶ ۲	۶۲۷ ۳	۶۲۸ ۱	۶۲۹ ۴	۶۳۰ ۲
۶۳۱ ۳	۶۳۲ ۳	۶۳۳ ۴	۶۳۴ ۳	۶۳۵ ۱	۶۳۶ ۲	۶۳۷ ۴	۶۳۸ ۳	۶۳۹ ۱	۶۴۰ ۴
۶۴۱ ۲	۶۴۲ ۳	۶۴۳ ۱	۶۴۴ ۲	۶۴۵ ۳	۶۴۶ ۱	۶۴۷ ۲	۶۴۸ ۴	۶۴۹ ۳	۶۵۰ ۱
۶۵۱ ۴	۶۵۲ ۳	۶۵۳ ۲	۶۵۴ ۱	۶۵۵ ۴	۶۵۶ ۲	۶۵۷ ۱	۶۵۸ ۴	۶۵۹ ۳	۶۶۰ ۴
۶۶۱ ۲	۶۶۲ ۳	۶۶۳ ۴	۶۶۴ ۱	۶۶۵ ۳	۶۶۶ ۱	۶۶۷ ۲	۶۶۸ ۴	۶۶۹ ۳	۶۷۰ ۱
۶۷۱ ۴	۶۷۲ ۳	۶۷۳ ۱	۶۷۴ ۴	۶۷۵ ۲	۶۷۶ ۳	۶۷۷ ۲	۶۷۸ ۴	۶۷۹ ۱	۶۸۰ ۱
۶۸۱ ۴	۶۸۲ ۲	۶۸۳ ۲	۶۸۴ ۳	۶۸۵ ۴	۶۸۶ ۲	۶۸۷ ۴	۶۸۸ ۱	۶۸۹ ۳	۶۹۰ ۱

۱-۱ JavaScript چیست؟ سه لایه‌ی وب، تاریخچه و ECMAScript

۱ **گزینه‌ی ۳** آسان

اسلاید ۲ سه هدف دارد: JavaScript چیست، ویژگی‌ها و محدودیت‌هایش کدام‌اند و برنامه‌نویسی async در آن چطور کار می‌کند؛ یعنی گزینه‌های ۱، ۲ و ۴ دقیقاً همین سه هدف‌اند. استایل‌دهی با CSS موضوع جلسه‌های قبل بود (اسلاید ۴ آن را جزو «کارهایی که تا الان بلدیم» آورده)، نه هدف این جلسه.

۲ **گزینه‌ی ۳** آسان

Behaviour یعنی «How it reacts»: صفحه چطور به کار کاربر واکنش نشان می‌دهد، و این کار JavaScript است. گزینه‌ی ۱ تعریف لایه‌ی Structure (HTML) و گزینه‌ی ۲ تعریف لایه‌ی Style (CSS) است. گزینه‌ی ۴ سؤال را درست گفته ولی زبان را اشتباه؛ CSS فقط ظاهر را تعیین می‌کند و نمی‌تواند منطق و واکنش واقعی بسازد.

۳ **گزینه‌ی ۲** متوسط

شمردن کلیک‌ها یعنی «به خاطر سپردن یک عدد و تغییر صفحه در واکنش به کار کاربر»؛ این منطق و interactivity است و فقط با JavaScript ممکن است (همان مثال دکمه‌ی درسنامه). گزینه‌های ۱ و ۴ کار HTML (لایه‌ی Structure) و گزینه‌ی ۳ کار CSS (لایه‌ی Style) هستند که طبق اسلاید ۴ قبلاً یاد گرفته‌ایم.

۴ **گزینه‌ی ۱** آسان

Brendan Eich در مه ۱۹۹۵ و در شرکت Netscape، فقط در ۱۰ روز اولین نسخه را ساخت. Microsoft بعداً فقط از آن کپی کرد (JScript)، پس گزینه‌ی ۲ غلط است. James Gosling سازنده‌ی زبان Java در شرکت Sun است (دام شباهت اسم) و Tim Berners-Lee مخترع وب (WWW) در CERN است، نه سازنده‌ی JavaScript.

۵ **گزینه‌ی ۲** متوسط

طبق اسلاید ۶، نسخه‌ی اول (۱۹۹۵) Mocha نام داشت، سپس به LiveScript و در نهایت به JavaScript تغییر نام داد. گزینه‌های ۱ و ۳ ترتیب را جابه‌جا کرده‌اند. JScript هم هیچ‌وقت نام قبلی JavaScript نبود؛ نسخه‌ی کپی‌شده‌ی Microsoft برای Internet Explorer بود.

۶ **گزینه‌ی ۴** آسان

اسم JavaScript یک تصمیم تبلیغاتی برای جذب برنامه‌نویس‌های Java بود و اسلاید با خط تأکید می‌گوید «JavaScript has almost nothing to do with Java». پس گزینه‌های ۱ و ۲ (نسخه‌ی ساده‌ی Java بودن یا ترجمه شدن به Java) غلط‌اند. سازنده هم Netscape (Brendan Eich) بود، نه Sun؛ پس گزینه‌ی ۳ هم نادرست است.

۷ **گزینه‌ی ۲** آسان

طبق اسلاید ۶، در سال ۱۹۹۶ Microsoft در Internet Explorer از JavaScript شرکت Netscape کپی کرد و اسمش را JScript گذاشت. اولین استاندارد رسمی ES1 بود (گزینه‌ی ۱)، نام اولیه Mocha بود (گزینه‌ی ۳) و ابزار تبدیل کد جدید به قدیمی، transpilerهایی مثل Babel هستند (گزینه‌ی ۴).

۸ **گزینه‌ی ۱** متوسط

اسلاید ۶ صریحاً می‌گوید: «ECMAScript is the standard, JavaScript the language in practice». ECMAScript جای JavaScript را نگرفت (گزینه‌ی ۲)؛ ES2015 فقط یک نسخه از همین استاندارد است. نسخه‌ی Microsoft اسمش JScript بود (گزینه‌ی ۳) و ECMAScript سند قواعد زبان است، نه کتابخانه‌ای که نصب شود (گزینه‌ی ۴).

۹ **گزینه‌ی ۴** متوسط

در سال ۱۹۹۷، به خاطر نیاز به استاندارد شدن زبان، سازمان ECMA استاندارد ECMAScript 1 (ES1) را منتشر کرد. سال ۱۹۹۵ سال ساخت Mocha در Netscape بود، نه انتشار استاندارد (گزینه‌ی ۱). ES5 (۲۰۰۹) اولین استاندارد نبود (گزینه‌ی ۲) و JScript اصلاً استاندارد نبود، بلکه کپی Microsoft بود (گزینه‌ی ۳).

۱۰ **گزینه‌ی ۱** متوسط

طبق اسلاید ۷، ES5 در ۲۰۰۹ و ES6 (همان ES2015) در ۲۰۱۵ منتشر شد و بزرگ‌ترین به‌روزرسانی تاریخ زبان بود. گزینه‌ی ۲ غلط است چون ES6 و ES2015 دو اسم یک نسخه‌اند و ES6 مال ۲۰۰۹ نیست. ES1 مال ۱۹۹۷ است (گزینه‌ی ۳) و ES2016 نسخه‌ی سال ۲۰۱۶ است، نه ۲۰۱۵ (گزینه‌ی ۴).

۱۱ **گزینه‌ی ۳** متوسط

اسلاید ۷ دلیل را صریحاً می‌گوید: «in order to ship less features per update»؛ یعنی به جای یک بسته‌ی بزرگ هر چند سال، هر سال یک بسته‌ی کوچک‌تر. انتشار سالانه برای حذف ویژگی‌های قدیمی نیست (گزینه‌ی ۱)، ربطی به ایراد داشتن ES2015 ندارد (گزینه‌ی ۲) و هدفش از کار انداختن مرورگرهای قدیمی هم نیست (گزینه‌ی ۴).

۱۲ **گزینه‌ی ۲** سخت

از ES6 به بعد، سال = عدد نسخه + ۲۰۰۹؛ پس ES7 یعنی ES2016 (و ES2017 همان ES8 است). بنابراین گزینه‌ی ۲ نادرست است. گزینه‌ی ۱ درست است (ES6 = ES2015)، گزینه‌ی ۳ همان چرخه‌ی انتشار سالانه‌ی اسلاید ۷ است و گزینه‌ی ۴ هم طبق اسلاید ۷ درست است.

۱۳ **گزینه‌ی ۴** سخت

سال‌ها: JScript در ۱۹۹۶ (ب)، ES1 در ۱۹۹۷ (ج)، ES5 در ۲۰۰۹ (الف) و ES2015 در ۲۰۱۵ (د)؛ پس ترتیب «ب، ج، الف، د» است. گزینه‌ی ۱ فکر کرده استاندارد قبل از JScript آمده؛ در حالی که دقیقاً به خاطر وجود نسخه‌های مختلف (مثل JScript) بود که در ۱۹۹۷ به استاندارد نیاز شد. گزینه‌های ۲ و ۳ هم ES5 را قبل از ES1 گذاشته‌اند.

۱۴ **گزینه‌ی ۴** سخت

قاعده: سال = عدد نسخه + ۲۰۰۹؛ پس ۲۰۱۷ - ۲۰۰۹ = ۸ و ES2017 همان ES8 است. ES7 دام معروف است و در واقع ES2016 است. ES6 همان ES2015 است و اسمی مثل ES17 اصلاً وجود ندارد؛ دو رقم آخر سال، عدد نسخه نیست.

۱۵ **گزینه‌ی ۳** سخت

اسلاید ۵ می‌گوید «an actual programming language JavaScript» است و «in your browser AND your machine (via Node.js)» اجرا می‌شود. گزینه‌ی ۱ نقش‌ها را جابه‌جا کرده: Netscape سازنده بود و Microsoft کپی‌کننده (JScript). بزرگ‌ترین به‌روزرسانی ES6/ES2015 بود نه ES5 (گزینه‌ی ۲) و ECMAScript استاندارد JavaScript است، نه زبانی که جایش را گرفته باشد (گزینه‌ی ۴).

۱-۲ Transpile و ویژگی‌های زبان JavaScript

۱۶ **گزینه‌ی ۲** آسان

طبق اسلاید ۸، transpile «Converts new JS to older syntax» و «Keeps code working on old browsers». گزینه‌ی ۱ تعریف compile به زبان ماشین است که کار خود موتور (مثلاً V8) است. گزینه‌ی ۳ فشرده‌سازی (minify) است، نه ترجمه. گزینه‌ی ۴ هم بی‌معناست؛ JavaScript و Java دو زبان جدا هستند.

۱۷ **گزینه‌ی ۴** آسان

اسلاید ۸ می‌گوید: «Uses tools like Babel or SWC». V8 و SpiderMonkey موتورهای اجرای JavaScript در Chrome و Firefox هستند (اسلاید ۱۰)، CodePen و JSFiddle محیط‌های آنلاین نوشتن کد هستند (اسلاید ۱۳) و Node.js یک محیط اجرا و npm مدیر بسته‌های آن است (فصل ۳).

۱۸ **گزینه‌ی ۲** آسان

در meme، قاضی همان Browser است و ES5+ (کد جدید) ساکت ایستاده؛ Babel کنارش حرف می‌زند و مثل مترجم، کد جدید را به syntax قدیمی که مرورگر می‌فهمد تبدیل می‌کند. پس گزینه‌ی ۱ نقش مرورگر است و گزینه‌ی ۳ هم غلط است چون کسی که مترجم لازم دارد کد **جدید** است. Babel مرورگر را تغییر نمی‌دهد، خودِ کد را ترجمه می‌کند (رد گزینه‌ی ۴).

۱۹ **گزینه‌ی ۳** متوسط

اسلاید ۸ دقیقاً برعکس گزینه‌ی ۳ را می‌گوید: «Adds a small build step»؛ یعنی transpile یک مرحله‌ی build **اضافه** می‌کند. گزینه‌های ۱ («Lets devs use future JS features»)، ۲ («Keeps code working on old browsers») و ۴ («Converts new JS to older syntax») عیناً از اسلاید هستند.

۲۰ **گزینه‌ی ۳** متوسط

build step یعنی بین «نوشتن کد» و «رسیدن کد به مرورگر» یک مرحله اضافه می‌شود: دستوری مثل `npx babel src --out-dir dist` اجرا می‌شود و مرورگر نسخه‌ی `dist` را می‌گیرد. این کار روی سیستم برنامه‌نویس انجام می‌شود، نه کاربر (رد گزینه‌ی ۱). نسخه‌ی قدیمی را خود ابزار خودکار می‌سازد و لازم نیست دو بار کد بنویسیم (رد گزینه‌ی ۲). هدف transpile اجرای کد روی مرورگرهای **قدیمی** است، پس گزینه‌ی ۴ برعکس است.

۲۱ **گزینه‌ی ۱** متوسط

transpile یعنی تبدیل کد به زبان یا نسخه‌ی **هم‌سطح**؛ ES2015 و ES5 هر دو JavaScript هستند، پس گزینه‌ی ۱ درست است. گزینه‌ی ۲ compile است (تبدیل به کد سطح پایین)، گزینه‌ی ۳ تفسیر (interpret) است که خروجی جداگانه‌ای نمی‌سازد و گزینه‌ی ۴ فقط فشرده‌سازی فایل است و ربطی به ترجمه‌ی کد ندارد.

۲۲ **گزینه‌ی ۴** سخت

arrow function یک‌خطی مقدارش را خودکار برمی‌گرداند، پس در نسخه‌ی ES5 باید `return` صریح نوشت و متغیر `name` را با `+` به رشته چسباند؛ این دقیقاً گزینه‌ی ۴ است (Babel همین را با `"Hello, ".concat(name, "!")` می‌نویسد). گزینه‌ی ۱ `return` ندارد و `undefined` چاپ می‌کند. در گزینه‌ی ۲، `${name}` داخل کوتیشن معمولی است و فقط داخل backtick کار می‌کند؛ پس همان متن `Hello, ${name}!` چاپ می‌شود. گزینه‌ی ۳ کلمه‌ی `"name"` را در کوتیشن گذاشته و `Hello, name!` چاپ می‌کند.

۲۳ **گزینه‌ی ۴** آسان

«سطح بالا» یعنی فاصله گرفتن از سخت‌افزار: موتور جای داده‌ها در حافظه و جزئیات پردازنده را مدیریت می‌کند. ربطی به قدرت کامپیوتر ندارد (گزینه‌ی ۱) و زبان‌های سطح بالا معمولاً سریع‌ترین زبان‌ها هم نیستند (گزینه‌ی ۲). گزینه‌ی ۳ توصیف زبان‌هایی مثل C است؛ JavaScript بدون کامپایل جداگانه و مستقیماً توسط موتور اجرا می‌شود.

۲۴ **گزینه‌ی ۱** آسان

Garbage Collector بخشی از موتور است که حافظه‌ی مقدارهای غیرقابل‌دسترس را خودکار آزاد می‌کند. First-class functions یعنی تابع یک مقدار است، Dynamic یعنی نوع متغیرها هنگام اجرا تعیین می‌شود و Single threaded یعنی در هر لحظه فقط یک کار انجام می‌شود؛ هیچ‌کدام درباره‌ی آزادسازی حافظه نیستند.

۲۵ **گزینه‌ی ۳** آسان

«First-class» یعنی تابع هم مثل عدد و رشته یک مقدار معمولی است؛ پس می‌شود ذخیره‌اش کرد، پاسش داد و برش گرداند. Prototype-based OO درباره‌ی ارث‌بری اشیاء از یکدیگر است، Garbage Collected درباره‌ی آزادسازی خودکار حافظه و Single threaded درباره‌ی انجام یک کار در هر لحظه.

۲۶ **گزینه‌ی ۱** متوسط

paradigm یعنی سبک برنامه‌نویسی و JavaScript هر سه سبک OOP، procedural و functional را پشتیبانی می‌کند (مثل مثال جمع قیمت‌ها در درسنامه). گزینه‌ی ۲ درباره‌ی محیط‌های اجراست، نه سبک برنامه‌نویسی. گزینه‌ی ۳ با Single threaded بودن JavaScript تناقض دارد و گزینه‌ی ۴ تعریف ویژگی Dynamic است.

۲۷ **گزینه‌ی ۲** متوسط

در JavaScript نوع به **مقدار** وابسته است، نه به متغیر؛ پس با هر انتساب، `typeof x` نوع مقدار فعلی را می‌دهد: `number`، سپس `string` و در آخر `boolean`. گزینه‌ی ۱ فرض کرده نوع متغیر ثابت می‌ماند و گزینه‌ی ۳ فکر کرده تغییر نوع مثل زبان‌های static خطا می‌دهد. گزینه‌ی ۴ هم `typeof` را با خودِ مقدار اشتباه گرفته است.

۲۸ **گزینه‌ی ۱** متوسط

به خاطر **First-class functions**، تابع‌های `shout` و `whisper` مثل یک مقدار به پارامتر `style` داده می‌شوند و همان `style(text)` تابع را صدا می‌زند: `toUpperCase` متن را بزرگ و `toLowerCase` آن را کوچک می‌کند؛ پس `HELLO!` و `hello...` . گزینه‌ی ۲ فراموش کرده که تابع‌ها حروف را عوض می‌کنند. پاس دادن تابع به تابع دیگر کاملاً مجاز است و خطا نمی‌دهد (رد گزینه‌ی ۳). `speak` نتیجه‌ی **صدا زدن** تابع را برمی‌گرداند، نه خودِ تابع را (رد گزینه‌ی ۴).

---

۲۹ **گزینه‌ی ۲** متوسط

به خاطر JavaScript، **non-blocking event loop** منتظر تایمر نمی‌ماند: `setTimeout` تابع را به محیط اجرا می‌سپارد و برنامه سراغ خط بعد می‌رود. تابع تایمر فقط وقتی اجرا می‌شود که کدهای عادی تمام شده باشند؛ پس `start` ، `end` و بعد `timeout` . زمان صفر یعنی «در اولین فرصت»، نه «همین الان» (رد گزینه‌های ۱ و ۳). تایمر هم حذف نمی‌شود و بالاخره اجرا می‌شود (رد گزینه‌ی ۴).

---

۳۰ **گزینه‌ی ۴** سخت

`Object.create(animal)` شیئی می‌سازد که prototype آن `animal` است (**Prototype-based OO**). `rabbit` خودش `eats` ندارد، ولی JavaScript آن را در prototype پیدا می‌کند؛ پس `true true` (رد گزینه‌ی ۱). `Object.keys` فقط کلیدهای **خودِ** شیء را برمی‌گرداند، پس فقط `['jumps']` (رد گزینه‌ی ۲). هیچ خطایی هم رخ نمی‌دهد (رد گزینه‌ی ۳).

---

۳۱ **گزینه‌ی ۳** سخت

JIT یعنی کامپایل «درست در لحظه‌ی نیاز»: موتور کد را parse می‌کند، اجرایش را شروع می‌کند و قسمت‌های پرتکرار را حین اجرا بهینه و به زبان ماشین کامپایل می‌کند (اسلاید ۱۰: «parsed, optimized, and compiled to machine code»). برنامه‌نویس هیچ مرحله‌ی کامپایل جداگانه‌ای انجام نمی‌دهد (رد گزینه‌ی ۱) و برخلاف گزینه‌ی ۲، کد به زبان ماشین تبدیل می‌شود. JavaScript هیچ ربطی به Java و JVM ندارد (رد گزینه‌ی ۴).

---

۳۲ **گزینه‌ی ۱** سخت

بعد از `user = null` دیگر هیچ متغیری به آن شیء اشاره نمی‌کند؛ پس شیء «غیرقابل‌دسترس» است و **Garbage Collector** حافظه‌اش را خودکار آزاد می‌کند، آن هم در زمانی که خودش انتخاب می‌کند. در JavaScript تابعی مثل `free` وجود ندارد؛ آن روش زبان C است (رد گزینه‌ی ۲). `let` اجازه‌ی انتساب دوباره را می‌دهد و `null` یک مقدار معمولی است (رد گزینه‌ی ۳). گزینه‌ی ۴ دقیقاً همان memory leak است که Garbage Collector جلویش را می‌گیرد.

---

۳۳ **گزینه‌ی ۳** سخت

JavaScript **single threaded** است: تنها thread به مدت ۲۰۰ میلی‌ثانیه درگیر حلقه‌ی `while` است و تابع تایمر، با اینکه زمانش صفر است و زمانش هم مدت‌ها پیش رسیده، باید صبر کند تا thread آزاد شود. پس اول `loop done` و بعد `timer` . گزینه‌ی ۱ فرض کرده تایمر می‌تواند وسط حلقه اجرا شود. تایمر هم گم نمی‌شود (رد گزینه‌ی ۲) و حلقه هم بالاخره تمام می‌شود (رد گزینه‌ی ۴).

---

۳۴ **گزینه‌ی ۴** سخت

Single threaded دقیقاً یعنی JavaScript فقط یک thread اصلی دارد و در هر لحظه **فقط یک** تکه کد را اجرا می‌کند؛ پس گزینه‌ی ۴ نادرست است. گزینه‌های ۱، ۲ و ۳ تعریف درست سه ویژگی Dynamic، First-class functions و Garbage Collected هستند.

---

۳۵ **گزینه‌ی ۲** سخت

«یک کار در هر لحظه» تعریف single threaded است و چیزی که باعث می‌شود برنامه منتظر کارهای زمان‌بر نماند، non-blocking event loop است. گزینه‌ی ۱ خلاف ویژگی تک‌نخی است. JIT compiler سرعت اجرا را بالا می‌برد ولی ربطی به منتظر نماندن ندارد (رد گزینه‌ی ۳). dynamic درباره‌ی نوع متغیرهاست و prototype chain درباره‌ی ارث‌بری (رد گزینه‌ی ۴).

۳-۱ JavaScript چطور اجرا می‌شود؟ موتورها و APIها

---

۳۶ **گزینه‌ی ۳** آسان

اسلاید ۱۰: «Chrome uses V8, Firefox uses SpiderMonkey». گزینه‌های ۱ و ۲ جای دو موتور را عوض کرده‌اند. Babel اصلاً موتور نیست؛ یک ابزار transpile است که کد جدید را به syntax قدیمی تبدیل می‌کند (رد گزینه‌ی ۴).

---

۳۷ **گزینه‌ی ۲** آسان

طبق نمودار اسلاید ۱۱، هر دو جعبه (مرورگر و Node JS) موتور V8 دارند؛ یعنی Node.js همان موتور Chrome را به کار می‌برد. SpiderMonkey موتور Firefox است، پس گزینه‌ی ۳ هم غلط است چون Node.js موتور مشترک با Chrome دارد. JScript هم موتور نیست؛ نسخه‌ی کپی‌شده‌ی JavaScript توسط Microsoft بود.

۳۸ **گزینه‌ی ۴** آسان

اسلاید ۱۰ می‌گوید: «Handles async tasks (timers, promises, user events) without blocking». تعریف متغیر، حلقه و محاسبات (گزینه‌های ۱ و ۲) کارهای همگام (synchronous) هستند که همان لحظه انجام می‌شوند. نوشتن تگ‌های HTML هم اصلاً کار JavaScript نیست (گزینه‌ی ۳).

۳۹ **گزینه‌ی ۱** آسان

اسلاید ۱۱: محیط‌ها «APIs (Application Programming Interfaces)» فراهم می‌کنند و APIها به JavaScript اجازه می‌دهند با دنیای بیرون ارتباط بگیرد (پیام نشان دهد، داده بگیرد، صفحه را تغییر دهد). سه گزینه‌ی دیگر عبارت‌هایی ساختگی‌اند؛ موتور، نصب‌کننده و ابزار transpile هیچ‌کدام تعریف API نیستند.

۴۰ **گزینه‌ی ۳** متوسط

جمله‌ی اسلاید: «Code is parsed, optimized, and compiled to machine code». موتور کد را به Java تبدیل نمی‌کند و transpile هم کار ابزارهایی مثل Babel است، نه موتور (رد گزینه‌ی ۱). خروجی موتور HTML نیست (رد گزینه‌ی ۲) و ECMAScript هم استاندارد زبان است، نه مقصد کامپایل (رد گزینه‌ی ۴).

۴۱ **گزینه‌ی ۴** متوسط

طبق نمودار اسلاید ۱۱، Node.js یعنی موتور V8 به‌علاوه‌ی System APIها (Server Side)؛ پس یک محیط اجرا (runtime) است. زبانش همان JavaScript است، نه زبانی جدید (رد گزینه‌ی ۱). موتور SpiderMonkey، Mozilla است و Node.js از V8 استفاده می‌کند (رد گزینه‌ی ۲). Node.js مرورگر نیست و صفحه نمایش نمی‌دهد (رد گزینه‌ی ۳).

۴۲ **گزینه‌ی ۲** متوسط

در نمودار، جعبه‌ی نارنجی (JS، سمت Client) شامل V8 و Web APIs است و جعبه‌ی سبز (Node JS، سمت Server) شامل V8 و System APIs. گزینه‌ی ۱ جای این دو را عوض کرده. Babel ابزار transpile است، نه API (رد گزینه‌ی ۳) و SpiderMonkey یک موتور است، نه مجموعه‌ای از APIها؛ ضمن اینکه Node.js از V8 استفاده می‌کند (رد گزینه‌ی ۴).

۴۳ **گزینه‌ی ۳** متوسط

اسلاید ۱۱: «Without these APIs, JavaScript alone couldn't do much beyond calculations». تغییر صفحه (DOM) و نمایش پیام هر دو از طریق APIهای محیط انجام می‌شوند، پس بدون API هیچ‌کدام ممکن نیستند (رد گزینه‌های ۱ و ۲). APIها را محیط (مرورگر یا Node.js) می‌دهد، نه خود زبان (رد گزینه‌ی ۴).

۴۴ **گزینه‌ی ۴** متوسط

`document` بخشی از DOM است که **مرورگر** به‌عنوان Web API می‌دهد؛ برای همین در Node.js وجود ندارد. `Math` ، `JSON` و متدهای رشته مثل `toUpperCase` جزو خود زبان (استاندارد ECMAScript) هستند و در هر محیطی، چه مرورگر و چه Node.js، کار می‌کنند.

۴۵ **گزینه‌ی ۱** متوسط

کد خطبه‌خط اجرا می‌شود: خط ۱ `first` را چاپ می‌کند، خط ۲ از اسم `total` استفاده می‌کند که هیچ‌جا تعریف نشده و خطای `ReferenceError` می‌دهد و برنامه همان‌جا متوقف می‌شود؛ پس `last` چاپ نمی‌شود. گزینه‌ی ۲ فراموش کرده خط ۱ قبل از خطا اجرا شده است. اسم تعریف‌نشده `undefined` نیست، خطاست (رد گزینه‌ی ۳) و `total` بدون کوتیشن یک اسم است، نه متن (رد گزینه‌ی ۴).

۴۶ **گزینه‌ی ۴** سخت

طبق اسلاید ۱۰، کد اول **parse** می‌شود و بعد اجرا. خطای syntax در همین مرحله‌ی parse پیدا می‌شود، یعنی **پیش از اجرای هر خطی**؛ پس حتی `one` و `two` هم چاپ نمی‌شوند و فقط `SyntaxError` گزارش می‌شود. گزینه‌ی ۱ فکر کرده خطای syntax مثل خطای هنگام اجرا فقط از خط ۳ به بعد را متوقف می‌کند. موتور پرانتز جاافتاده را نادیده نمی‌گیرد (رد گزینه‌ی ۲) و گزینه‌ی ۳ هم هیچ منطقی ندارد.

۴۷ **گزینه‌ی ۲** سخت

`Math` جزو خود زبان است و `console` را هم Node.js فراهم می‌کند؛ پس هر دو `object` هستند. `document` جزو DOM است و فقط مرورگر آن را دارد؛ در Node.js تعریف نشده و `typeof` برای آن `undefined` برمی‌گرداند، اما `process` یک System API در Node.js است (`object`). گزینه‌ی ۱ نتیجه‌ی مرورگر است. گزینه‌ی ۳ فکر کرده `console` فقط مال مرورگر است. `typeof` روی اسم تعریف‌نشده خطا نمی‌دهد (رد گزینه‌ی ۴).

۴۸ **گزینه‌ی ۱** سخت

در مرورگر، `document` (درخت DOM) و `window` (شیء سراسری مرورگر) Web APIهایی هستند که مرورگر می‌دهد، پس هر دو `object` هستند. `process` مخصوص Node.js (System API) است و در مرورگر تعریف نشده؛ `typeof` آن `undefined` است و خطا نمی‌دهد (رد گزینه‌ی ۴). گزینه‌ی ۲ نتیجه را با Node.js قاطی کرده و گزینه‌ی ۳ فکر کرده همه‌ی APIها همه‌جا هستند.

۴۹ **گزینه‌ی ۳** سخت

`Math` و `JSON` جزو خود زبان JavaScript هستند و در Node.js هم کار می‌کنند، پس خط‌های ۱ تا ۴ بدون مشکل اجرا می‌شوند (`max: 9` و `{"max":9}` چاپ می‌شود). اما `document` یک Web API مرورگر است و در Node.js وجود ندارد؛ پس خط ۵ خطای `ReferenceError: document is not defined` می‌دهد. گزینه‌ی «هیچ خطایی رخ نمی‌دهد» فقط در مرورگر درست بود.

۵۰ **گزینه‌ی ۲** سخت

DOM یک Web API است که **مرورگر** فراهم می‌کند و جزو خود زبان نیست؛ برای همین در Node.js شیء `document` وجود ندارد و گزینه‌ی ۲ نادرست است. گزینه‌ی ۱ طبق نمودار اسلاید ۱۱ درست است (هر دو جعبه V8 دارند). گزینه‌ی ۳ همان نمودار اسلاید ۱۱ است و گزینه‌ی ۴ همان جمله‌ی اسلاید ۱۰.

۴-۱ Console API و محل نوشتن کد JavaScript

۵۱ **گزینه‌ی ۱** آسان

اسلاید ۱۲: «Console «Accessible through the global console object است و «Provides many methods (log, warn, error, table,.etc)». `console` یک شیء معمولی است، نه کلمه‌ی کلیدی زبان (`typeof console` برابر `"object"` است). هیچ فایلی هم لازم نیست به صفحه اضافه شود، چون محیط (مرورگر یا Node.js) خودش آن را می‌دهد، و تگ HTML هم نیست.

۵۲ **گزینه‌ی ۲** آسان

اسلاید ۱۲: «Used mainly for debugging and inspecting values». طراحی ظاهر کار CSS است (گزینه‌ی ۱). Console فقط پیام نشان می‌دهد و نه چیزی را برای همیشه ذخیره می‌کند (گزینه‌ی ۳) و نه صفحه را به سرور می‌فرستد (گزینه‌ی ۴).

۵۳ **گزینه‌ی ۴** آسان

`console.table` (که اسلاید ۱۲ در فهرست متدها آورده) آرایه یا شیء را به شکل جدول با ستون `(index)` و ستون‌هایی برای هر کلید نشان می‌دهد. `console.log` و `console.info` داده را به شکل متنی معمولی چاپ می‌کنند و متدی به اسم `console.grid` اصلاً وجود ندارد.

۵۴ **گزینه‌ی ۴** متوسط

در تصویر اسلاید ۱۴ هشدار (warn) زرد با آیکون مثلث و خطا (error) قرمز با علامت تعجب و فلشی برای باز کردن stack trace است. گزینه‌ی ۱ رنگ‌ها را جابه‌جا کرده. اسلاید صریحاً می‌گوید «there is an obvious difference between different types of messages» (رد گزینه‌ی ۲). هیچ متدی از console برنامه را متوقف نمی‌کند (رد گزینه‌ی ۳).

۵۵ **گزینه‌ی ۳** متوسط

وقتی چند مقدار را با **کاما** به `console.log` بدهی، با یک فاصله بینشان چاپ می‌شوند: `Name: Sara 20` . اما `+` دو مقدار را **بدون فاصله** به هم می‌چسباند: `Age:20` . گزینه‌ی ۱ برای `+` هم فاصله گذاشته، گزینه‌ی ۲ برای کاماها فاصله نگذاشته و گزینه‌ی ۴ فکر کرده خود کاماها هم چاپ می‌شوند.

۵۶ **گزینه‌ی ۴** متوسط

خط ۱ دو عدد را جمع می‌کند: `5` . خط ۲ دو **رشته** را به هم می‌چسباند: `23` . در خط ۳ کل عبارت داخل کوتیشن است، پس فقط یک متن است و همان‌طور چاپ می‌شود: `2 + 3` . گزینه‌های ۱ و ۲ فکر کرده‌اند متن داخل کوتیشن محاسبه می‌شود و گزینه‌ی ۳ فراموش کرده که `console.log` رشته‌ها را **بدون کوتیشن** چاپ می‌کند.

۵۷ **گزینه‌ی ۳** سخت

ورودی هر تابع قبل از اجرای آن محاسبه می‌شود؛ پس اول `console.log("hi")` درونی اجرا می‌شود و `hi` را چاپ می‌کند. متدهای console هیچ مقداری برنمی‌گردانند، پس مقدار برگشتی آن `undefined` است و `console.log` بیرونی همین `undefined` را چاپ می‌کند. گزینه‌ی ۱ فکر کرده `console.log` متن را برمی‌گرداند، گزینه‌ی ۲ ترتیب را برعکس کرده و گزینه‌ی ۴ چاپِ بیرونی را از قلم انداخته است.

۵۸ **گزینه‌ی ۲** سخت

`console.log("x")` اول `x` را چاپ می‌کند و `undefined` برمی‌گرداند؛ پس `x` برابر `undefined + 1` است که حاصلش `NaN` (Not a Number، «عدد نامعتبر») است (این دام را در درس تبدیل نوع مفصل می‌بینیم). گزینه‌ی ۱ فکر کرده `console.log` متن `"x"` را برمی‌گرداند. گزینه‌ی ۳ با `null + 1` که برابر `1` است قاطی کرده و گزینه‌ی ۴ چاپ `x` در خط اول را فراموش کرده است.

۵۹ **گزینه‌ی ۴** متوسط

JavaScript به بزرگی و کوچکی حروف حساس است. `Console` (با C بزرگ) اصلاً تعریف نشده و `ReferenceError: Console is not defined` می‌دهد. شیء `console` متدی به اسم `Log` (با L بزرگ) یا `print` ندارد؛ پس صدا زدنشان `TypeError: ... is not a function` می‌دهد. فقط `console.log` درست است.

۶۰ **گزینه‌ی ۳** سخت

`console.warn` و `console.error` فقط پیام هشدار یا خطا **نمایش می‌دهند** و برنامه را متوقف نمی‌کنند؛ پس خط‌های ۲ و ۳ با وجود ظاهر ترسناکشان مشکلی ایجاد نمی‌کنند. خط ۴ از اسم `total` استفاده می‌کند که هیچ‌جا تعریف نشده و خطای واقعی `ReferenceError: total is not defined` می‌دهد. پس گزینه‌ی «هیچ خطایی رخ نمی‌دهد» هم غلط است.

۶۱ **گزینه‌ی ۱** آسان

اسلاید ۱۳ چهار راه را می‌گوید: Console مرورگر (گزینه‌ی ۲)، داخل HTML با `<script>` (گزینه‌ی ۳)، فایل `.js` جداگانه با `<script src="...">` و محیط‌های آنلاین (گزینه‌ی ۴). فایل `.css` فقط برای استایل است و مرورگر محتوای آن را به‌عنوان JavaScript اجرا نمی‌کند؛ اسلاید فقط برای **شیوه‌ی وصل کردن** فایل `.js` آن را با CSS مقایسه کرده است.

۶۲ **گزینه‌ی ۳** متوسط

طبق اسلاید ۱۳، فایل `.js` با `<script src="...">` وصل می‌شود (و تگ باید با `</script>` بسته شود). تگ `<link>` با `href` روش وصل کردن **CSS** است و برای اجرای JavaScript کار نمی‌کند (گزینه‌ی ۱). ویژگی درست برای `<script>` اسم `src` است، نه `href` (گزینه‌ی ۲) و تگی به اسم `<js>` در HTML وجود ندارد (گزینه‌ی ۴).

۶۳ **گزینه‌ی ۲** سخت

وقتی تگ `<script>` ویژگی `src` دارد، مرورگر فقط فایل خارجی (`app.js`) را اجرا می‌کند و کدِ بین `<script src="app.js">` و `</script>` را **کاملاً نادیده می‌گیرد**؛ پس فقط `external` چاپ می‌شود. گزینه‌های ۱ و ۴ فکر کرده‌اند هر دو اجرا می‌شوند (فقط ترتیبشان فرق دارد) و گزینه‌ی ۳ فکر کرده کد درون‌خطی بر فایل خارجی اولویت دارد. برای داشتن هر دو، باید دو تگ `<script>` جداگانه نوشت.

۶۴ **گزینه‌ی ۳** آسان

اسلاید ۱۳ این‌ها را «Online Playgrounds» می‌نامد: سایت‌هایی برای نوشتن و اجرای HTML، CSS و JavaScript بدون نصب هیچ برنامه‌ای. موتورهای JavaScript چیزهایی مثل V8 و SpiderMonkey هستند (گزینه‌ی ۱)، ابزارهای transpile Babel و SWC هستند (گزینه‌ی ۲) و Node.js برای اجرای JavaScript به هیچ افزونه‌ای نیاز ندارد (گزینه‌ی ۴).

۶۵ **گزینه‌ی ۴** متوسط

Console (مثل Python shell) اول دستور را اجرا می‌کند، پس `Hello` چاپ می‌شود؛ بعد **مقدار برگشتی** دستور را نشان می‌دهد که برای متدهای console برابر `undefined` است (اسلاید ۱۴). گزینه‌ی ۱ نمایش مقدار برگشتی را فراموش کرده و گزینه‌ی ۲ چاپ پیام را. برنگرداندن مقدار هم خطا نیست؛ فقط یعنی مقدار برگشتی `undefined` است (رد گزینه‌ی ۳).

۶۶ **گزینه‌ی ۱** سخت

تعریف متغیر با `let` مقداری ندارد، پس Console برای خط ۱ `undefined` نشان می‌دهد (رد گزینه‌های ۲ و ۴). انتساب `a = "Hello!"` یک عبارت است که مقدارش همان مقدار جدید است، پس `"Hello!"` نمایش داده می‌شود (رد گزینه‌ی ۳). در خط ۳، `console.log` متن `Hello!` را چاپ می‌کند و بعد مقدار برگشتی‌اش، `undefined`، نشان داده می‌شود؛ دقیقاً مثل تصویر اسلاید ۲۳.

۶۷ **گزینه‌ی ۱** سخت

نمایش خودکار مقدار هر خط فقط مخصوص Console و REPL است؛ وقتی یک **فایل** اجرا می‌شود، فقط `console.log` چیزی چاپ می‌کند. پس `2 + 3;` و انتساب‌ها چیزی چاپ نمی‌کنند و تنها خروجی `Hello!` است. گزینه‌ی ۲ رفتار Console را به فایل نسبت داده و گزینه‌ی ۳ هم فکر کرده `2 + 3;` به‌تنهایی چاپ می‌شود. `console.log` رشته را بدون کوتیشن چاپ می‌کند (رد گزینه‌ی ۴).

۶۸ **گزینه‌ی ۲** آسان

Node.js هم شیء `console` را دارد و `console.log` در آن، متن را در **ترمینال** چاپ می‌کند. Node.js ربطی به مرورگر ندارد، پس پیام در Console مرورگر دیده نمی‌شود (گزینه‌ی ۱). خودبه‌خود هم در هیچ فایلی ذخیره نمی‌شود (گزینه‌ی ۳). `console` در هر دو محیط، یعنی مرورگر و Node.js، وجود دارد (رد گزینه‌ی ۴).

۱-۵ متغیرها: const، let و var

۶۹ **گزینه‌ی ۴** آسان

`const` (مخفف constant) انتساب دوباره را ممنوع می‌کند و هر تلاشی برای آن خطای `TypeError` می‌دهد. `let` و `var` هر دو اجازه می‌دهند بعداً مقدار تازه‌ای در متغیر بریزیم. `final` کلمه‌ی کلیدی زبان Java است و در JavaScript وجود ندارد؛ خطای `final x = 1` `SyntaxError` می‌دهد.

۷۰ **گزینه‌ی ۱** آسان

اسلاید ۲۳ درباره‌ی `var` می‌گوید: outdated (قدیمی)، function-scoped و «در کد مدرن از آن پرهیز کن». گزینه‌ی ۲ برعکس است: `let` و `const` در ES6 آمدند و `var` از اول زبان وجود داشت. گزینه‌ی ۳ غلط است چون `var` انتساب دوباره (و حتی تعریف دوباره) را اجازه می‌دهد. گزینه‌ی ۴ ویژگی `let` و `const` است، نه `var`.

۷۱ **گزینه‌ی ۱** آسان

متغیر `let` قابل انتساب دوباره است: اول `21`، بعد `22` و در خط سوم مقدار فعلی (`22`) با ۱ جمع می‌شود و حاصل یعنی `23` دوباره در `age` ریخته می‌شود. گزینه‌ی ۴ مخصوص `const` است و گزینه‌های ۲ و ۳ فرض کرده‌اند انتساب‌های بعدی اثری ندارند.

۷۲ **گزینه‌ی ۱** آسان

متغیری که تعریف شده ولی هنوز مقدار نگرفته، مقدارش `undefined` است؛ بعد از `score = 10` مقدارش `10` می‌شود. `null` را JavaScript خودش در متغیر نمی‌گذارد (آن را برنامه‌نویس عمداً می‌گذارد) و مقدار پیش‌فرض `0` هم مخصوص زبان‌هایی مثل Java است. `ReferenceError` فقط برای متغیری است که اصلاً تعریف نشده باشد.

۷۳ **گزینه‌ی ۲** آسان

خط ۴ تلاش می‌کند به `pi` که با `const` تعریف شده مقدار تازه بدهد، پس `TypeError: Assignment to constant variable.` رخ می‌دهد و برنامه متوقف می‌شود. خط ۲ مشکلی ندارد چون `age` با `let` تعریف شده است. خط ۵ هرگز اجرا نمی‌شود، چون برنامه قبلاً روی خط ۴ متوقف شده است.

---

۷۴ **گزینه‌ی ۲** آسان

متن بنر اسلاید ۲۴: «Use `const` by default, switch to `let` when reassignment is needed». با این روش اگر جایی به‌اشتباه مقدار متغیری را عوض کنی، فوراً `TypeError` می‌گیری. گزینه‌ی ۱ این محافظت را از بین می‌برد، گزینه‌ی ۳ خلاف توصیه‌ی اسلاید ۲۳ (پرهیز از `var`) است و گزینه‌ی ۴ کاربرد `const` را بی‌دلیل محدود می‌کند؛ هر متغیری که قرار نیست دوباره مقدار بگیرد باید `const` باشد.

---

۷۵ **گزینه‌ی ۴** متوسط

در گزینه‌ی ۴ به متغیر `const` مقدار تازه داده شده (انتساب دوباره)، پس `TypeError` رخ می‌دهد. در گزینه‌های ۱ و ۲ خود متغیر عوض نشده، فقط **محتوای** شیء یا آرایه‌ای که به آن اشاره می‌کند تغییر کرده و این برای `const` مجاز است. گزینه‌ی ۳ هم یک `let` است که انتساب دوباره‌اش آزاد است.

---

۷۶ **گزینه‌ی ۴** متوسط

`const` فقط اجازه نمی‌دهد `nums` به آرایه‌ی **دیگری** وصل شود؛ تغییر محتوای همان آرایه آزاد است. `push(4)` عدد ۴ را به انتها اضافه می‌کند و `nums[0] = 10` خانه‌ی اول را عوض می‌کند، پس `[10, 2, 3, 4]` چاپ می‌شود. گزینه‌ی ۳ همان تصور غلطِ «const یعنی محتوای غیرقابل‌تغییر» است و گزینه‌های ۱ و ۲ فرض کرده‌اند بعضی تغییرها نادیده گرفته می‌شوند.

---

۷۷ **گزینه‌ی ۳** سخت

خط ۲ محتوای همان شیء را تغییر می‌دهد که برای `const` مجاز است؛ پس خط ۳ مقدار `light` را چاپ می‌کند. خط ۴ می‌خواهد `settings` را به یک شیء **جدید** وصل کند؛ این انتساب دوباره است و `TypeError` می‌دهد. این خطا در زمان اجرا رخ می‌دهد، پس خروجی خط ۳ قبل از آن چاپ شده است (رد گزینه‌ی ۴). گزینه‌ی ۲ تغییر ویژگی را هم ممنوع فرض کرده و گزینه‌ی ۱ اصلاً ممنوعیت انتساب دوباره را ندیده است.

---

۷۸ **گزینه‌ی ۱** سخت

`const` باید همان لحظه‌ی تعریف مقدار بگیرد؛ `const b;` در خط ۲ خطای `SyntaxError: Missing initializer in const declaration` می‌دهد. این خطا هنگام خواندن (parse) کل فایل و **قبل از اجرای هر خطی** کشف می‌شود. گزینه‌ی ۲ وسوسه‌انگیز است، چون خط ۳ هم انتساب دوباره به `const` به نظر می‌رسد، ولی برنامه اصلاً به مرحله‌ی اجرا نمی‌رسد.

---

۷۹ **گزینه‌ی ۲** متوسط

یک دستور تعریف (declaration) مقداری تولید نمی‌کند و کنسول برای چنین دستوری `undefined` نشان می‌دهد. این `undefined` مقدار `a` **نیست**؛ مقدار `a` همان `12` است و اگر `a` را تایپ کنی، `12` می‌بینی. گزینه‌ی ۱ همان تصور غلطِ «کنسول مقدار متغیر را نشان می‌دهد» است و `null` و نام متغیر هم هیچ‌وقت نتیجه‌ی یک تعریف نیستند.

---

۸۰ **گزینه‌ی ۱** متوسط

انتساب برخلاف تعریف، یک **عبارت** (expression) است و مقدارش همان مقداری است که منتسب شد؛ برای همین کنسول اسلاید ۲۳ جلوی `a = "Hello!"` مقدار `"Hello!"` را نشان داد. نتیجه‌ی دستورهای تعریف مثل `let a = 12` همیشه `undefined` است، `12` مقدار قبلی `a` است و انتساب هیچ‌وقت `true` (به معنای «موفق بود») برنمی‌گرداند.

---

۸۱ **گزینه‌ی ۳** سخت

انتساب دوباره به `const` در همه‌ی موتورها `TypeError` است و فقط متن پیام فرق می‌کند: SpiderMonkey در Firefox می‌گوید `invalid assignment to const 'b'` و V8 در Chrome و Node.js می‌گوید `Assignment to constant variable.`. گزینه‌ی ۱ غلط است چون Node.js هم جلوی این کار را می‌گیرد. `SyntaxError` برای `const` بدون مقدار اولیه است، نه انتساب دوباره، و `ReferenceError` برای متغیر تعریف‌نشده است، در حالی که `b` تعریف شده است.

---

۸۲ **گزینه‌ی ۲** متوسط

اسلاید ۲۴: «Both are block-scoped (only exist inside `{ ... }`)». گزینه‌ی ۱ غلط است چون `const` فقط انتساب دوباره را منع می‌کند و `push` یا تغییر خانه‌های آرایه مجاز است. گزینه‌ی ۳ توصیف `const` است، نه `let`. گزینه‌ی ۴ از دو جهت غلط است: `var` قدیمی‌تر است و function-scoped است.

---

۸۳ **گزینه‌ی ۴** سخت

`var` بلوک `{ }` را نادیده می‌گیرد (function-scoped است)، پس `a` بیرون از بلوک هم وجود دارد و `1` چاپ می‌شود. اما `let` block-scoped است و `b` فقط داخل بلوک وجود دارد؛ خواندن آن بیرون از بلوک `ReferenceError` می‌دهد. چون خطا روی خط آخر رخ می‌دهد، خروجی خط قبل چاپ شده است (رد گزینه‌ی ۲). گزینه‌ی ۱ هر دو را مثل `var` فرض کرده است.

---

۸۴ **گزینه‌ی ۲** سخت

`let level` داخل بلوک، تعریف دوباره نیست؛ یک متغیر **جدید** است که فقط در همان بلوک زندگی می‌کند و در آن‌جا متغیر بیرونی را می‌پوشاند. داخل بلوک `inner` و بیرون از آن متغیر اصلی یعنی `outer` چاپ می‌شود. `SyntaxError` فقط وقتی رخ می‌داد که هر دو `let` در **یک** محدوده بودند. گزینه‌ی ۱ فرض کرده متغیر داخلی، بیرونی را عوض کرده است.

---

۸۵ **گزینه‌ی ۳** سخت

`var` تعریف دوباره را بی‌صدا اجازه می‌دهد، پس خط ۲ مشکلی ندارد. خط ۴ انتساب دوباره به `let` است که مجاز است. خط ۵ متغیر `y` را در همان محدوده **دوباره تعریف** می‌کند و `SyntaxError: Identifier 'y' has already been declared` می‌دهد. چون این یک `SyntaxError` است، در واقع هیچ خطی از فایل اجرا نمی‌شود.

---

۸۶ **گزینه‌ی ۴** سخت

تعریف دوباره‌ی `let total` در همان محدوده یک `SyntaxError` است. خطاهای نحوی هنگام خواندن (parse) فایل و **قبل از اجرای اولین خط** کشف می‌شوند، پس حتی `start` هم چاپ نمی‌شود. گزینه‌ی ۲ فرض کرده خطا در زمان اجرا رخ می‌دهد (مثل `TypeError` انتساب به `const`). گزینه‌ی ۱ رفتار `var` را در نظر گرفته که تعریف دوباره را اجازه می‌دهد و گزینه‌ی ۳ فرض کرده تعریف دوباره بی‌صدا نادیده گرفته می‌شود.

---

۸۷ **گزینه‌ی ۴** متوسط

نام متغیر می‌تواند حرف، رقم، `_` و `$` داشته باشد، پس `_total$` کاملاً مجاز است. در گزینه‌ی ۱ خط تیره عملگر منها تفسیر می‌شود، در گزینه‌ی ۲ نام با رقم شروع شده و در گزینه‌ی ۳ `class` یک کلمه‌ی رزروشده است؛ هر سه `SyntaxError` می‌دهند.

---

۸۸ **گزینه‌ی ۲** متوسط

JavaScript به بزرگی و کوچکی حروف حساس است (case-sensitive)؛ پس `userName` و `username` دو متغیر جدا هستند و تعریف دوباره‌ای رخ نداده است (رد گزینه‌ی ۴). خط ۳ فقط `userName` را عوض می‌کند و `username` همان `Reza` می‌ماند. گزینه‌ی ۳ دو نام را یکی فرض کرده است.

---

۸۹ **گزینه‌ی ۳** آسان

در camelCase کلمه‌ی اول با حرف کوچک و هر کلمه‌ی بعدی با حرف بزرگ شروع می‌شود: `totalOrders`. هر چهار نام از نظر زبان مجازند و خطا نمی‌دهند، ولی `total_orders` سبک snake_case (رایج در Python) است، `TotalOrders` سبک PascalCase است که برای نام کلاس‌ها به کار می‌رود و `totalorders` مرز کلمه‌ها را نشان نمی‌دهد و خواندنش سخت است.

---

۹۰ **گزینه‌ی ۱** متوسط

در JavaScript نوع به **مقدار** تعلق دارد، نه به متغیر (dynamic typing). وقتی `data` عدد است `typeof` می‌گوید `number` و بعد از انتساب رشته می‌گوید `string`. گزینه‌ی ۲ فرض کرده نوع متغیر هنگام تعریف قفل می‌شود و گزینه‌ی ۳ رفتار زبان‌هایی مثل Java را در نظر گرفته که ریختن رشته در متغیر عددی را منع می‌کنند.

---

۹۱ **گزینه‌ی ۳** متوسط

خط دوم به `count` مقدار تازه می‌دهد، پس متغیر باید قابل انتساب دوباره باشد: `let` . با `const` همان خط دوم `TypeError` می‌دهد. `int` و `final` کلمه‌های Java هستند و در JavaScript برای تعریف متغیر وجود ندارند؛ هر دو `SyntaxError` می‌دهند.

۶-۱ انواع داده: Primitiveها و Objectها

۹۲ **گزینه‌ی ۳** آسان

هفت primitive داریم: `number` ، `string` ، `boolean` ، `undefined` ، `null` ، `Symbol` و `BigInt` . عدد ۸ تعداد کل انواع داده است، یعنی ۷ primitive به‌علاوه‌ی object. عدد ۶ مربوط به قبل از اضافه شدن BigInt (ES2020) و عدد ۵ مربوط به قبل از Symbol (ES6) است.

۹۳ **گزینه‌ی ۴** آسان

طبق اسلاید ۱۵، آرایه‌ها (و توابع و تاریخ‌ها) objectهای خاص هستند، نه primitive؛ برای همین `typeof [1, 2]` برابر `"object"` است. `Symbol` ، `BigInt` و `null` هر سه جزو ۷ primitive اسلاید ۱۶ هستند؛ این‌که `typeof null` برابر `"object"` است فقط یک اشتباه تاریخی است و `null` را object نمی‌کند.

۹۴ **گزینه‌ی ۲** آسان

اسلاید ۱۵: تبدیل نوع یا خودکار است (**coercion**) یا صریح (**casting**). در `"5" * 2` ما چیزی نخواسته‌ایم و خود موتور رشته را به عدد تبدیل کرده، پس coercion است. casting یعنی تبدیل صریح مثل `Number("5")` . hoisting مربوط به بالا بردن اعلان‌ها است و parsing یعنی خواندن و تجزیه‌ی کد؛ هیچ‌کدام تبدیل نوع نیستند.

۹۵ **گزینه‌ی ۴** آسان

اسلاید ۱۶: `null` یعنی «نبودِ **عمدیِ** مقدار» و آن را برنامه‌نویس می‌گذارد. `undefined` را خود JavaScript برای متغیری که تعریف شده ولی مقدار نگرفته قرار می‌دهد. `NaN` نتیجه‌ی محاسبه‌ی عددیِ بی‌معنی است و `0` یک عدد واقعی است، نه «نبودِ مقدار».

۹۶ **گزینه‌ی ۱** آسان

`"42"` داخل علامت نقل‌قول است، پس یک **رشته** است، نه عدد (گزینه‌ی ۲ همین را اشتباه گرفته). `42` از نوع `number` است؛ JavaScript نوعی به نام `int` ندارد (رد گزینه‌ی ۴). `true` یک مقدار `boolean` است، نه رشته (رد گزینه‌ی ۳).

۹۷ **گزینه‌ی ۳** آسان

پسوند `n` یک **BigInt** می‌سازد و `typeof` برای آن `"bigint"` را **با حروف کوچک** برمی‌گرداند. گزینه‌ی ۲ نام نوع را با حرف بزرگ نوشته، ولی خروجی `typeof` همیشه حروف کوچک است. گزینه‌ی ۱ پسوند `n` را نادیده گرفته و نوعی به نام `integer` هم وجود ندارد.

۹۸ **گزینه‌ی ۱** آسان

`Number.MAX_SAFE_INTEGER` برابر `9007199254740991` یعنی `2 ** 53 - 1` است و BigInt برای اعداد صحیح بزرگ‌تر از آن ساخته شده (اسلاید ۱۶). `2147483647` بیشینه‌ی نوع `int` در Java است، `9007199254740992` یکی بیشتر از حد امن است و `Infinity` بزرگ‌ترین عدد «امن» نیست، بلکه مقدار ویژه‌ی بی‌نهایت است.

۹۹ **گزینه‌ی ۳** متوسط

`null` یک primitive است، اما `typeof null` به‌خاطر یک اشتباه قدیمی در نسخه‌ی اول JavaScript، `"object"` برمی‌گرداند و برای سازگاری با کدهای قدیمی هیچ‌وقت اصلاح نشد. گزینه‌ی ۱ منطقی به نظر می‌رسد ولی واقعیت ندارد. گزینه‌ی ۴ هم غلط است چون `typeof` همیشه یک **رشته** برمی‌گرداند. برای تشخیص `null` بنویس `value === null` .

۱۰۰ **گزینه‌ی ۱** متوسط

`10n` یک BigInt است و `typeof` آن `"bigint"` است؛ BigInt نوع جداگانه‌ای است، نه بخشی از `number` . سه مقدار دیگر همه از نوع `number` هستند: `3.14` یک عدد اعشاری معمولی است و `NaN` و `Infinity` مقدارهای ویژه‌ی همین نوع‌اند. دام اصلی `NaN` است: با این‌که نامش مخفف Not a Number است، `typeof NaN` برابر `"number"` است.

۱۰۱ **گزینه‌ی ۳** متوسط

`typeof` روی متغیر تعریف‌نشده **خطا نمی‌دهد** و رشته‌ی `"undefined"` را برمی‌گرداند، پس خط اول `undefined` چاپ می‌کند. اما خواندن مستقیم `price` در خط دوم خطای `ReferenceError: price is not defined` می‌دهد. گزینه‌ی ۱ «تعریف‌نشده» را با «بی‌مقدار» یکی گرفته و گزینه‌ی ۲ فکر کرده `typeof` هم خطا می‌دهد.

۱۰۲ **گزینه‌ی ۲** متوسط

آرایه‌ها object‌های خاص‌اند، پس `typeof list` برابر `"object"` است (نوعی به نام `"array"` در خروجی `typeof` وجود ندارد). برای تشخیص آرایه باید از `Array.isArray` استفاده کرد که برای `list` مقدار `true` و برای رشته‌ی `"123"` مقدار `false` می‌دهد؛ یک رشته آرایه نیست، حتی اگر شبیه فهرستی از کاراکترها باشد.

۱۰۳ **گزینه‌ی ۱** متوسط

`a` تعریف شده ولی مقدار نگرفته، پس `undefined` است؛ `b` عمداً `null` شده است. در خط دوم `typeof undefined` برابر `"undefined"` و `typeof null` (به‌خاطر اشتباه تاریخی) برابر `"object"` است. گزینه‌ی ۲ فکر کرده `typeof null` می‌شود `"null"` و گزینه‌های ۳ و ۴ تفاوت `undefined` و `null` را نادیده گرفته‌اند.

۱۰۴ **گزینه‌ی ۲** متوسط

هر بار `Symbol(...)` صدا زده شود یک مقدار **کاملاً یکتا** ساخته می‌شود و متن `"id"` فقط یک برچسب توضیحی است؛ پس دو Symbol با برچسب یکسان برابر نیستند و `false` چاپ می‌شود. `typeof` برای Symbol رشته‌ی `"symbol"` را برمی‌گرداند، نه `"string"`؛ برچسب رشته‌ای، Symbol را رشته نمی‌کند.

۱۰۵ **گزینه‌ی ۴** سخت

`[id]: 123` یعنی Symbol ذخیره‌شده در `id` را کلید ویژگی قرار بده؛ پس `user[id]` مقدار `123` را می‌دهد. رشته‌ی `"id"` کلید کاملاً متفاوتی است و چنین کلیدی در شیء نداریم، پس `user["id"]` برابر `undefined` است. گزینه‌ی ۱ برچسب Symbol را با یک کلید رشته‌ای یکی گرفته، در حالی که دقیقاً به همین دلیل از Symbol برای کلیدها استفاده می‌شود: با هیچ کلید دیگری تداخل نمی‌کند.

۱۰۶ **گزینه‌ی ۲** سخت

این اعداد از `Number.MAX_SAFE_INTEGER` (یعنی `9007199254740991`) بزرگ‌ترند و نوع `number` آن‌ها را دقیق نگه نمی‌دارد؛ `9007199254740993` در حافظه همان `9007199254740992` ذخیره می‌شود و مقایسه `true` می‌دهد. با پسوند `n` اعداد BigInt هستند که دقیق‌اند، پس مقایسه‌ی دوم `false` است. همین مشکل دلیل وجود BigInt است.

۱۰۷ **گزینه‌ی ۳** سخت

جمع دو BigInt یک BigInt است و `console.log` آن را با `n` نشان می‌دهد: `15n`. اما BigInt و number را نمی‌شود مستقیم با هم قاطی کرد: `big + 5` خطای `TypeError: Cannot mix BigInt and other types` می‌دهد. گزینه‌های ۱ و ۲ فرض کرده‌اند JavaScript خودش تبدیل می‌کند؛ راه درست تبدیل صریح است، مثل `big + BigInt(5)` یا `Number(big) + 5`.

۱۰۸ **گزینه‌ی ۴** آسان

اسلاید ۱۶: string یعنی دنباله‌ای از کاراکترها داخل علامت نقل‌قول، و هر سه شکل `'...'`، `"..."` و `` `...` `` یک `string` می‌سازند. JavaScript نوعی به نام `char` ندارد (برخلاف Java) و `'a'` فقط یک رشته به طول ۱ است. backtick هم فقط شکل دیگری از نوشتن رشته (template literal) است و `typeof` آن `"string"` است.

۱۰۹ **گزینه‌ی ۱** متوسط

متغیرها در JavaScript نوع ثابت ندارند (dynamic typing)، پس ریختن رشته در متغیری که عدد بود خطا ندارد (رد گزینه‌ی ۳). در خط سوم `value` رشته‌ی `"seven"` است و `+` با یک رشته، **الحاق** (چسباندن) انجام می‌دهد: `"seven1"` که نوعش `string` است. گزینه‌ی ۲ مقدار قبلی `7` را در نظر گرفته و گزینه‌ی ۴ فکر کرده رشته به عدد تبدیل می‌شود.

۱۱۰ **گزینه‌ی ۳** متوسط

اسلاید ۱۷: «Functions are first-class objects (can be passed, stored, and returned)»؛ یعنی تابع هم مثل هر مقدار دیگری قابل ذخیره، پاس دادن و برگرداندن است. گزینه‌ی ۱ ربطی به first-class بودن ندارد. گزینه‌ی ۲ غلط است چون طبق همان اسلاید، تابع می‌تواند متد یا کد مستقل (standalone) باشد. گزینه‌ی ۴ هم غلط است: تابع object است، نه primitive.

۱۱۱ **گزینه‌ی ۴** متوسط

تابع `shout` در متغیر `myFn` **ذخیره** شده و به‌عنوان آرگومان به `apply` **پاس داده** شده است؛ این همان first-class بودن توابع است و خطایی ندارد. `apply` آن را با `"hi"` صدا می‌زند و `toUpperCase()` حروف را بزرگ می‌کند: `HI!` . تابع object است، ولی `typeof` برای آن `"function"` برمی‌گرداند، نه `"object"` .

۱۱۲ **گزینه‌ی ۲** متوسط

اندیس آرایه‌ها از ۰ شروع می‌شود، پس `courses[1]` دومین خانه یعنی `AI` است (گزینه‌ی ۱ از ۱ شمرده). `address` یک شیء تودرتو است و `address.city` برابر `Tehran` است. خواندن کلیدی که در شیء وجود ندارد خطا نمی‌دهد و `undefined` برمی‌گرداند؛ `null` را فقط برنامه‌نویس می‌گذارد و `ReferenceError` مخصوص **متغیر** تعریف‌نشده است، نه کلید ناموجود.

۱۱۳ **گزینه‌ی ۳** متوسط

`"2026-11-13"` داخل علامت نقل‌قول است، پس یک **رشته** (primitive) است، هرچند شکل تاریخ دارد. طبق اسلاید ۱۵، تاریخ‌ها (`new Date()`)، آرایه‌ها و توابع همه objectهای خاص هستند و primitive حساب نمی‌شوند.

۱۱۴ **گزینه‌ی ۲** متوسط

تاریخ یک object خاص است و `typeof new Date()` برابر `"object"` است؛ `typeof` هیچ‌وقت `"date"` برنمی‌گرداند. بقیه‌ی گزینه‌ها درست‌اند: تابع با این‌که object است، `typeof` آن `"function"` است؛ `typeof undefined` برابر `"undefined"` و `typeof` یک Symbol برابر `"symbol"` است.

۱۱۵ **گزینه‌ی ۴** سخت

از راست به چپ حساب کن: `typeof 10n` رشته‌ی `"bigint"` را می‌دهد. حالا `typeof "bigint"` نوعِ یک **رشته** را می‌پرسد که `"string"` است. خروجی `typeof` همیشه رشته است، پس `typeof typeof` هر چیزی همیشه `"string"` می‌شود. گزینه‌ی ۱ فقط `typeof` اول را حساب کرده است.

۱۱۶ **گزینه‌ی ۳** سخت

`b = a` **مقدارِ** `10` را کپی می‌کند؛ عوض کردن `b` روی `a` اثری ندارد، پس `a` همان `10` می‌ماند. اما `obj2 = obj1` فقط **نشانی (reference)** شیء را کپی می‌کند و هر دو نام به یک شیء اشاره می‌کنند؛ پس تغییر `obj2.score` همان شیئی را عوض می‌کند که `obj1` به آن اشاره دارد و `20` چاپ می‌شود. گزینه‌ی ۱ objectها را هم مثل primitiveها کپی‌شونده فرض کرده و گزینه‌ی ۲ primitiveها را مثل objectها.

۱۱۷ **گزینه‌ی ۱** سخت

primitiveها (مثل رشته) با **مقدار** مقایسه می‌شوند، پس دو رشته‌ی یکسان برابرند. اما objectها (از جمله آرایه‌ها) با **نشانی** مقایسه می‌شوند: `[1, 2]` اول و دوم دو آرایه‌ی جداگانه در حافظه‌اند و `{}` اول و دوم هم دو شیء جدا؛ پس هر دو مقایسه `false` است، حتی با محتوای یکسان.

۱۱۸ **گزینه‌ی ۱** سخت

رشته‌ها primitive و **تغییرناپذیر** هستند؛ `word[0] = "b"` نمی‌تواند حرفی از رشته را عوض کند و در حالت عادی بی‌صدا نادیده گرفته می‌شود (در حالت strict خطای `TypeError` می‌دهد). پس اول `cat` چاپ می‌شود. اما `word = "bat"` خود متغیر `let` را به رشته‌ی **جدیدی** وصل می‌کند که مجاز است. گزینه‌ی ۲ رشته را قابل‌تغییر فرض کرده و گزینه‌ی ۴ انتساب دوباره را هم ممنوع پنداشته است.

۱۱۹ **گزینه‌ی ۴** سخت

اسلاید ۱۶: `number` شامل اعداد صحیح **و** اعشاری است؛ JavaScript نوع جداگانه‌ی `int` یا `float` ندارد و `5 === 5.0` برابر `true` است (رد گزینه‌های ۱ و ۲). نوشتن عدد بزرگ‌تر از حد امن خطا نمی‌دهد، فقط **دقتش از دست می‌رود** (`9007199254740993` به `9007199254740992` گرد می‌شود)؛ برای همین گزینه‌ی ۳ غلط است و برای دقت باید از BigInt استفاده کرد.

۱-۷ عملگرهای حسابی و انتساب

۱۲۰ **گزینه‌ی ۲** آسان

در JavaScript فقط یک نوع عدد (`number`) داریم و تقسیم همیشه نتیجه‌ی واقعی می‌دهد: `3.5`. گزینه‌ی ۱ رفتار تقسیم صحیح در Java و C است و گزینه‌ی ۳ نتیجه را گرد کرده است. نتیجه یک عدد است، نه رشته، پس گزینه‌ی ۴ هم غلط است.

۱۲۱ **گزینه‌ی ۳** آسان

`%` **باقی‌مانده‌ی** تقسیم را می‌دهد: ۱۷ تقسیم بر ۵ می‌شود ۳ و ۲ باقی می‌ماند (`5 * 3 + 2 = 17`). گزینه‌ی ۱ خارج‌قسمت است، گزینه‌ی ۲ حاصل تقسیم `17 / 5` و گزینه‌ی ۴ حاصل تفریق `17 - 5`.

۱۲۲ **گزینه‌ی ۱** آسان

`**` عملگر توان است: `2 ** 4` یعنی ۲ به توان ۴، یعنی `2 * 2 * 2 * 2` که می‌شود `16`. گزینه‌ی ۲ حاصل `2 * 4` است، گزینه‌ی ۳ حاصل `2 + 4` و گزینه‌ی ۴ حاصل `2 ** 5`.

۱۲۳ **گزینه‌ی ۴** آسان

وقتی یک طرف `+` رشته باشد، طرف دیگر هم به رشته تبدیل و دو رشته به هم **چسبانده** می‌شوند (concatenation): `"1" + "1"` یعنی رشته‌ی `"11"`. گزینه‌ی ۲ عدد یازده است، نه رشته، و گزینه‌های ۱ و ۳ جمع ریاضی را فرض کرده‌اند. یادت باشد `1 + 1` (بدون رشته) در JavaScript هم همان `2` است.

۱۲۴ **گزینه‌ی ۱** آسان

در برنامه‌نویسی `=` عملگر **انتساب** است، نه برابری: اول سمت راست (`x + 1`) حساب می‌شود و بعد نتیجه در `x` ریخته می‌شود؛ یعنی همان `x += 1`. گزینه‌ی ۲ معنای مقایسه را فرض کرده که در JavaScript با `===` انجام می‌شود. این دستور کاملاً مجاز است (رد گزینه‌ی ۳) و `x` را یکی زیاد می‌کند، نه برابر ۱ (رد گزینه‌ی ۴).

۱۲۵ **گزینه‌ی ۲** آسان

عملگر توان در JavaScript `**` است و `2 ** 3` برابر `8` است. گزینه‌ی ۱ دام معروف است: `^` در JavaScript توان نیست بلکه عملگر بیتی XOR است و `2 ^ 3` برابر `1` می‌شود. گزینه‌ی ۳ ضرب است (`6`) و در گزینه‌ی ۴ پایه و توان جابه‌جا شده‌اند (`9`).

۱۲۶ **گزینه‌ی ۴** آسان

در JavaScript تقسیم بر صفر **خطا نمی‌دهد** (رد گزینه‌ی ۲)؛ نتیجه یکی از مقدارهای ویژه‌ی نوع `number` است: عدد مثبت بر صفر `Infinity`، عدد منفی بر صفر `-Infinity`، و `0 / 0` که معنای ریاضی ندارد `NaN`. گزینه‌ی ۱ علامت منفی و حالت `0 / 0` را اشتباه گرفته و گزینه‌ی ۳ همه را بی‌معنی فرض کرده است.

۱۲۷ **گزینه‌ی ۱** متوسط

قدم‌به‌قدم: `10 + 5 = 15`، سپس `15 - 3 = 12`، سپس `12 * 2 = 24`، سپس `24 / 4 = 6` و در آخر `6 % 4 = 2` (باقی‌مانده‌ی ۶ بر ۴). گزینه‌ی ۲ آخرین خط (`%=`) را فراموش کرده، گزینه‌ی ۳ به‌جای باقی‌مانده دوباره تقسیم کرده (`6 / 4`) و گزینه‌ی ۴ باقی‌مانده را با بخش‌پذیری اشتباه گرفته است.

۱۲۸ **گزینه‌ی ۳** متوسط

در JavaScript علامت نتیجه‌ی `%` از **عدد سمت چپ** (مقسوم) می‌آید: `7 % 3` برابر `1` است، پس `-7 % 3` برابر `-1` است. گزینه‌ی ۱ جواب «modulo ریاضی» است که همیشه نامنفی است (برخی زبان‌ها مثل Python همین را می‌دهند). گزینه‌ی ۲ علامت را نادیده گرفته است.

۱۲۹ **گزینه‌ی ۲** متوسط

عملگر `**` **از راست به چپ** حساب می‌شود: اول `3 ** 2 = 9` و بعد `2 ** 9 = 512` . گزینه‌ی ۱ از چپ به راست حساب کرده (`(2 ** 3) ** 2 = 64`)، گزینه‌ی ۳ توان را با ضرب اشتباه گرفته (`2 * 3 * 2`) و گزینه‌ی ۴ حاصل `(2 * 3) ** 2` است.

۱۳۰ **گزینه‌ی ۴** سخت

نوشتن منفیِ یک‌طرفه درست قبل از `**` مبهم است (منظور `(-2) ** 2` است یا `-(2 ** 2)`؟)، برای همین JavaScript آن را با `SyntaxError` رد می‌کند و مجبورت می‌کند پرانتز بگذاری. گزینه‌های ۱ و ۲ با پرانتز ابهام را رفع کرده‌اند (`4` و `-4`). در گزینه‌ی ۳ منفی روی **توان** است، نه پایه، و ابهامی ندارد: `2 ** -2` برابر `0.25` است.

۱۳۱ **گزینه‌ی ۳** متوسط

ترتیب اولویت: اول توان `4 ** 2 = 16` ، بعد ضرب `3 * 16 = 48` و در آخر جمع `2 + 48 = 50` . گزینه‌ی ۱ همه را از چپ به راست حساب کرده (`((2 + 3) * 4) ** 2`)، گزینه‌ی ۲ جمع را قبل از ضرب انجام داده (`(2 + 3) * 16`) و گزینه‌ی ۴ ضرب را قبل از توان (`2 + 12 ** 2`).

۱۳۲ **گزینه‌ی ۱** متوسط

`/` و `*` هم‌سطح هستند و **از چپ به راست** حساب می‌شوند: `20 / 4 = 5` و بعد `5 * 2 = 10` . گزینه‌ی ۲ ضرب را جلو انداخته (`20 / (4 * 2)`)؛ این اشتباه رایجِ «ضرب همیشه قبل از تقسیم» است. گزینه‌ی ۴ عمل دوم را جا انداخته و گزینه‌ی ۳ هیچ ترتیب درستی ندارد.

۱۳۳ **گزینه‌ی ۲** سخت

اول توان: `3 ** 2 = 9` . حالا `2 * 9 % 5` داریم و `*` و `%` هم‌سطح‌اند، پس از چپ به راست: `2 * 9 = 18` و `18 % 5 = 3` . گزینه‌ی ۱ `%` را قبل از ضرب انجام داده (`2 * (9 % 5)`)، گزینه‌ی ۳ ضرب را قبل از توان (`(2 * 3) ** 2 % 5 = 36 % 5`) و گزینه‌ی ۴ عمل `%` را جا انداخته است.

۱۳۴ **گزینه‌ی ۱** سخت

`b = a++` (postfix): اول مقدار **قدیمی** `a` یعنی `5` در `b` ریخته می‌شود و بعد `a` می‌شود `6` . `c = ++a` (prefix): اول `a` می‌شود `7` و بعد مقدار **جدید** یعنی `7` در `c` ریخته می‌شود. پس `7 5 7` . گزینه‌ی ۲ تفاوت postfix را ندیده، گزینه‌ی ۳ افزایش دوم را جا انداخته و گزینه‌ی ۴ prefix را مثل postfix حساب کرده است.

۱۳۵ **گزینه‌ی ۲** متوسط

دو بار `++` و یک بار `--` ، پس `count` برابر `1` است. در `console.log(count++)` چون postfix است، **اول** مقدار فعلی (`1`) چاپ می‌شود و **بعد** `count` به `2` می‌رسد؛ خط آخر `2` را نشان می‌دهد. گزینه‌ی ۱ postfix را مثل prefix فرض کرده و گزینه‌ی ۴ فکر کرده `count++` داخل `console.log` اثری ندارد.

۱۳۶ **گزینه‌ی ۴** سخت

در عملگرهای ترکیبی **اول کل سمت راست** حساب می‌شود: `x *= 2 + 3` یعنی `x = x * (2 + 3)` که می‌شود `4 * 5 = 20` . گزینه‌ی ۱ آن را `x * 2 + 3` خوانده، گزینه‌ی ۲ عملگر را `+=` فرض کرده (`4 + 5`) و گزینه‌ی ۳ هر دو عدد را در `x` ضرب کرده است.

۱۳۷ **گزینه‌ی ۳** متوسط

`a += 2` یعنی `a = "8" + 2` و چون یک طرف `+` رشته است، الحاق انجام می‌شود: `"82"` . اما `-` فقط معنای عددی دارد، پس `b -= 2` یعنی `"8" - 2` و رشته‌ی `"8"` به عدد تبدیل می‌شود: `6` . گزینه‌ی ۱ هر دو را عددی حساب کرده و گزینه‌ی ۲ فکر کرده تفریق از رشته `NaN` می‌دهد؛ `NaN` فقط وقتی پیش می‌آمد که رشته عددی نبود (مثل `"abc"`).

۱۳۸ **گزینه‌ی ۳** متوسط

اعداد اعشاری مثل `0.1` در مبنای ۲ دقیق ذخیره نمی‌شوند، پس حاصل جمع `0.30000000000000004` است و با `0.3` برابر نیست. گزینه‌های ۱ و ۲ فرض کرده‌اند حاصل دقیقاً `0.3` است. گزینه‌ی ۴ هم ممکن نیست: وقتی حاصل `0.30000000000000004` است، مقایسه با `0.3` نمی‌تواند `true` باشد. این رفتار مخصوص JavaScript نیست و طبق استاندارد IEEE 754 در بیشتر زبان‌ها دیده می‌شود.

۱۳۹ **گزینه‌ی ۱** متوسط

تقسیم بر صفر در JavaScript هیچ خطایی نمی‌دهد: `5 / 0` برابر `Infinity` است. بقیه‌ی گزینه‌ها درست‌اند: باقی‌مانده‌ی ۱۰ بر ۲ صفر است، تقسیم همیشه اعشاری است (`3.5`) و `Math.floor` قسمت اعشاری را به سمت پایین حذف می‌کند (`3`).

۱۴۰ **گزینه‌ی ۲** سخت

`3 % 2` برابر `1` است، پس `isOdd(3)` درست کار می‌کند. اما علامت `%` از عدد سمت چپ می‌آید: `-3 % 2` برابر `-1` است، نه `1`؛ پس `isOdd(-3)` به‌اشتباه `false` می‌دهد. گزینه‌های ۱ و ۴ این دام را ندیده‌اند و گزینه‌ی ۳ با خط سوم تناقض دارد. راه درست تشخیص فرد بودن `n % 2 !== 0` است.

۱۴۱ **گزینه‌ی ۴** متوسط

`/` در JavaScript همیشه اعشاری است (`3.5`)، پس باید قسمت اعشاری را حذف کنیم: `Math.floor(3.5)` برابر `3` است. گزینه‌ی ۲ باقی‌مانده (`1`) را می‌دهد، نه خارج‌قسمت. گزینه‌ی ۳ دام است: `Math.round` به نزدیک‌ترین عدد صحیح گرد می‌کند و `3.5` را `4` می‌کند.

۱۴۲ **گزینه‌ی ۴** سخت

`-7 / 2` برابر `-3.5` است. `Math.floor` همیشه به سمت **پایین** (به سمت منفی بی‌نهایت) گرد می‌کند و `-3.5` را `-4` می‌کند. `Math.trunc` فقط قسمت اعشاری را **می‌بُرد** و `-3` می‌دهد. برای اعداد مثبت این دو یکسان‌اند، ولی برای اعداد منفی فرق دارند؛ گزینه‌های ۱ و ۲ همین تفاوت را ندیده‌اند و گزینه‌ی ۳ آن دو را جابه‌جا کرده است.

۱۴۳ **گزینه‌ی ۳** سخت

هر انتساب یک عبارت است و مقدارش **مقدار جدید** متغیر است؛ پس `console.log(score += 5)` عدد `15` را چاپ می‌کند (و نوشتن انتساب داخل `console.log` مجاز است؛ رد گزینه‌ی ۴). `score++` مقدار **قدیمی** (`15`) را برمی‌گرداند و بعد `score` را `16` می‌کند؛ خط آخر `16` است. گزینه‌ی ۱ postfix را مثل prefix حساب کرده و گزینه‌ی ۲ فکر کرده انتساب، مقدارِ قبلی را برمی‌گرداند.

۱۴۴ **گزینه‌ی ۲** سخت

`bonus++` یعنی `bonus = bonus + 1` که انتساب دوباره به یک `const` است و `TypeError: Assignment to constant variable.` می‌دهد. خط ۴ فقط **مقدار** `bonus` را می‌خواند و `points` را (که `let` است) عوض می‌کند، پس مجاز است. خط ۶ هرگز اجرا نمی‌شود. گزینه‌ی «بدون خطا» هم غلط است، چون `++` و همه‌ی عملگرهای ترکیبی روی `const` ممنوع‌اند.

۱۴۵ **گزینه‌ی ۱** متوسط

`+` با یک رشته الحاق می‌کند: `1 + "5"` می‌شود `"51"`. اما `-` فقط معنای عددی دارد و `"5"` را به ۵ تبدیل می‌کند: `4`. در خط سوم ارزیابی از چپ به راست است: `1 + 2` (دو عدد) می‌شود `3` و بعد `3 + "3"` می‌شود `"33"`. گزینه‌ی ۴ فرض کرده رشته از اول ساخته می‌شود و گزینه‌ی ۳ تفریق را هم الحاق پنداشته است.

۱-۸ رشته‌ها و Template literals

۱۴۶ **گزینه‌ی ۳** آسان

طبق اسلاید ۵۱، template literalها با **backtick** نوشته می‌شوند. تک‌کوتیشن و دابل‌کوتیشن رشته‌ی معمولی می‌سازند که نه جاسازی `${}` دارد و نه چندخطی مستقیم. علامت `$` فقط بخشی از `${...}` داخل template literal است و رشته را باز یا بسته نمی‌کند.

۱۴۷ **گزینه‌ی ۲** آسان

اسلاید ۵۱ می‌گوید template literalها در **ES6** (که نام دیگرش ES2015 است) معرفی شدند؛ همان نسخه‌ای که بزرگ‌ترین به‌روزرسانی زبان بود و `let`، `const` و توابع arrow را هم آورد. ES1 اولین استاندارد بود، ES5 هنوز template literal نداشت و ES2020 خیلی بعدتر آمد (مثلاً BigInt را اضافه کرد).

۱۴۸ **گزینه‌ی ۴** آسان

اسلاید ۵۱ سه ویژگی را می‌گوید: با backtick نوشته می‌شود (ES6)، رشته‌ی چندخطی بدون `\n` را ممکن می‌کند و از جاسازی با `${expression}` پشتیبانی می‌کند. template literal همیشه یک **رشته** می‌سازد و کاری با تبدیل رشته به عدد ندارد؛ برعکس، مقدار داخل `${}` را به رشته تبدیل می‌کند.

۱۴۹ **گزینه‌ی ۴** آسان

template literal فقط **روش دیگری برای نوشتن رشته** است و نتیجه‌اش یک رشته‌ی معمولی (`"Hello 2"`) است؛ پس `typeof` آن `"string"` است. نوع جداگانه‌ای به نام `"template"` وجود ندارد، template literal یک object نیست و با این‌که داخلش محاسبه‌ی عددی انجام شده، نتیجه‌ی نهایی رشته است، نه عدد.

۱۵۰ **گزینه‌ی ۱** آسان

جاسازی با `${}` فقط داخل **backtick** انجام می‌شود؛ پس خط اول `Hello, Sara!` چاپ می‌کند. در خط دوم رشته با دابل‌کوتیشن نوشته شده و `${name}` فقط چند کاراکتر معمولی است که عیناً چاپ می‌شود. این خط خطا هم نمی‌دهد (رد گزینه‌ی ۴)؛ یک رشته‌ی کاملاً معتبر است.

۱۵۱ **گزینه‌ی ۱** آسان

هر `${}` جداگانه حساب می‌شود: `${a}` می‌شود `5`، `${b}` می‌شود `10` و `${a + b}` جمع **عددی** دو عدد یعنی `15` است. علامت `+` و `=` که بیرون از `${}` نوشته شده‌اند فقط متن هستند. گزینه‌ی ۳ فکر کرده جمع داخل `${}` به‌صورت رشته‌ای انجام می‌شود، در حالی که `a` و `b` عددند و تبدیل به رشته فقط **بعد از** محاسبه رخ می‌دهد.

۱۵۲ **گزینه‌ی ۴** متوسط

`\n` با این‌که با دو علامت نوشته می‌شود، **یک** کاراکتر (خط جدید) است: ۵ حرف `Hello` + ۱ کاراکتر خط جدید + ۵ حرف `World` = ۱۱. گزینه‌ی ۱ بک‌اسلش و `n` را دو کاراکتر شمرده و گزینه‌ی ۲ خط جدید را اصلاً نشمرده است.

۱۵۳ **گزینه‌ی ۱** متوسط

در گزینه‌ی ۱ رشته با تک‌کوتیشن باز شده و تک‌کوتیشنِ داخل `It's` رشته را زودتر می‌بندد؛ بقیه‌ی متن (`s broken'`) برای JavaScript بی‌معنی است و `SyntaxError` می‌دهد. راه درست `'It\'s broken'` یا `"It's broken"` است. در گزینه‌های ۲ و ۳ علامت داخل متن با علامت دور رشته فرق دارد و در گزینه‌ی ۴ با backtick، هر دو نوع کوتیشن داخل متن آزادند.

۱۵۴ **گزینه‌ی ۲** متوسط

رشته‌ی معمولی (تک‌کوتیشن یا دابل‌کوتیشن) نمی‌تواند Enter خام داشته باشد؛ پس گزینه‌ی ۲ خطای `SyntaxError: Invalid or unexpected token` می‌دهد. template literal (گزینه‌ی ۱) دقیقاً برای همین ساخته شده: رشته‌ی چندخطی بدون `\n`. گزینه‌ی ۳ خط جدید را با `\n` نوشته و گزینه‌ی ۴ سه رشته را با `+` چسبانده که هر دو درست‌اند.

۱۵۵ **گزینه‌ی ۲** متوسط

داخل backtick هر `${...}` حساب و جایگزین می‌شود: `2`، `Laptop` و حاصل **ضرب** `price * quantity` یعنی `2400` (رد گزینه‌ی ۱ که جاسازی را ندیده و گزینه‌ی ۳ که عبارت را حساب نکرده). در `$${...}` علامت `$` اول یک دلار معمولی است که چاپ می‌شود، پس `$2400` درست است و گزینه‌ی ۴ آن را جا انداخته. چون رشته با Enter شروع و تمام شده، خروجی یک خط خالی در ابتدا و یک خط خالی در انتها دارد.

۱۵۶ **گزینه‌ی ۳** سخت

رشته‌ی `message` بلافاصله بعد از backtick بازکننده یک Enter دارد و قبل از backtick بسته‌کننده هم یک Enter؛ پس با `\n` شروع و تمام می‌شود. برای همین علامت `|` اول تنها در خط اول و علامت `|` دوم تنها در خط آخر می‌آید. گزینه‌ی ۱ این دو Enter را ندیده، گزینه‌ی ۲ فکر کرده Enterهای داخل template literal حذف می‌شوند و گزینه‌ی ۴ علامت `$` معمولیِ قبل از `${...}` را جا انداخته است.

۱۵۷ **گزینه‌ی ۳** متوسط

در گزینه‌ی ۳، `$` اول یک علامت دلار معمولی است و `${price * qty}` حاصل `30` را جاسازی می‌کند: `Total: $30` (درست مثل `$$` در کد اسلاید ۵۱). گزینه‌ی ۱ علامت دلار را ندارد و `Total: 30` چاپ می‌کند. گزینه‌ی ۲ با دابل‌کوتیشن نوشته شده و اصلاً جاسازی نمی‌کند. در گزینه‌ی ۴ آکولاد `{ }` نیامده، پس هیچ جاسازی‌ای رخ نمی‌دهد و متن `Total: $$price * qty` عیناً چاپ می‌شود.

۱۵۸ **گزینه‌ی ۲** متوسط

داخل `${}` هر **عبارتی** مجاز است: عملگر سه‌تایی (`20 >= 18` درست است، پس `adult`)، محاسبه (`age + 1` جمع عددی است و `21` می‌دهد) و صدا زدن متد (`toUpperCase()` حروف را بزرگ می‌کند). گزینه‌ی ۱ فکر کرده `age` قبل از جمع به رشته تبدیل می‌شود، گزینه‌ی ۳ صدا زدن متد را متن فرض کرده و گزینه‌ی ۴ استفاده از `? :` یا متد را داخل `${}` غیرمجاز پنداشته است.

۱۵۹ **گزینه‌ی ۳** سخت

داخل `${}` فقط **عبارت (expression)** مجاز است، یعنی چیزی که مقدار تولید می‌کند. `if` یک **دستور (statement)** است و مقداری تولید نمی‌کند، پس گزینه‌ی ۳ خطای `SyntaxError` می‌دهد. عملگر سه‌تایی (گزینه‌ی ۱)، صدا زدن تابع (گزینه‌ی ۲) و محاسبه‌ی پرانتزدار (گزینه‌ی ۴) همه عبارت‌اند و مجازند؛ برای شرط داخل template literal باید از `? :` استفاده کرد.

۱۶۰ **گزینه‌ی ۱** متوسط

وقتی آرایه به رشته تبدیل می‌شود، عضوهایش با **کاما و بدون فاصله** به هم چسبانده می‌شوند و کروشه‌ها حذف می‌شوند: `"1,2,3"`. گزینه‌ی ۲ شکل نمایش آرایه در کنسول است، نه تبدیلش به رشته. گزینه‌ی ۳ جداکننده را فاصله فرض کرده و `"[object Array]"` هم شکل تبدیل آرایه نیست (چیزی شبیه آن، یعنی `"[object Object]"`، برای شیءهای معمولی است).

۱۶۱ **گزینه‌ی ۴** متوسط

`${}` هر مقداری را به **رشته** تبدیل می‌کند. یک شیء معمولی به رشته‌ی `[object Object]` تبدیل می‌شود (محتوایش نمایش داده نمی‌شود) و آرایه به عضوهایش با کاما و بدون کروشه: `js,web`. گزینه‌ی ۱ شکل نمایش `console.log(user)` است که با تبدیل به رشته فرق دارد. گزینه‌ی ۳ کروشه‌ها را نگه داشته و گزینه‌ی ۲ فرض کرده JavaScript خودش مقدار `name` را پیدا می‌کند. برای دیدن محتوای شیء باید از `JSON.stringify(user)` استفاده کرد.

۱۶۲ **گزینه‌ی ۲** متوسط

`city` تعریف شده ولی مقدار ندارد، پس `undefined` است؛ `${}` آن را به رشته‌ی `"undefined"` تبدیل می‌کند و `null` و `false` هم به `"null"` و `"false"`. template literal مقدارهای «خالی» را حذف نمی‌کند (رد گزینه‌ی ۱) و `null` را با `undefined` یکی نمی‌کند (رد گزینه‌ی ۳). `ReferenceError` فقط برای متغیر تعریف‌نشده است، در حالی که `city` با `let` تعریف شده است.

۱۶۳ **گزینه‌ی ۳** سخت

خط اول از چپ به راست حساب می‌شود: `"Sum: " + 2` رشته‌ی `"Sum: 2"` می‌شود و بعد `+ 3` آن را به `"Sum: 23"` می‌رساند. در خط دوم `a + b` داخل `${}` اول به‌صورت **عددی** حساب می‌شود (`5`) و بعد در متن قرار می‌گیرد. در خط سوم پرانتز باعث می‌شود جمع عددی اول انجام شود: `"Sum: 5"`. گزینه‌ی ۱ دام خط اول را ندیده و گزینه‌های ۲ و ۴ جاسازی یا پرانتز را بی‌اثر فرض کرده‌اند.

۱۶۴ **گزینه‌ی ۴** سخت

داخل `${}` بیرونی یک عملگر سه‌تایی است؛ چون `isOnline` برابر `true` است، شاخه‌ی اول انتخاب می‌شود که خودش یک template literal **تودرتو** است و `${2 + 3}` داخلش به `5` تبدیل می‌شود. backtickهای تودرتو داخل `${}` کاملاً مجازند (رد گزینه‌ی ۳)، template داخلی هم جاسازی می‌کند (رد گزینه‌ی ۲) و شرط درست است (رد گزینه‌ی ۱).

۱۶۵ **گزینه‌ی ۱** سخت

`\`` یک backtick معمولی داخل template literal می‌گذارد و رشته را نمی‌بندد (رد گزینه‌ی ۳). `\${` جاسازی را غیرفعال می‌کند و `${price}` عیناً چاپ می‌شود؛ چون `price` اصلاً خوانده نمی‌شود، `ReferenceError` هم رخ نمی‌دهد (رد گزینه‌ی ۲). `\"` هم یک دابل‌کوتیشن معمولی است. خود بک‌اسلش‌ها فقط علامت گریز هستند و چاپ نمی‌شوند (رد گزینه‌ی ۴).

۱۶۶ **گزینه‌ی ۲** سخت

در template literal چندخطی، هر Enter و هر فاصله جزو رشته است: `A` (۱) + خط جدید (۱) + دو فاصله (۲) + `B` (۱) = ۵. گزینه‌ی ۳ فقط حروف را شمرده، گزینه‌ی ۱ فاصله‌ها را نادیده گرفته و گزینه‌ی ۴ خط جدید را نشمرده است.

۹-۱ تبدیل نوع صریح (Casting)

۱۶۷ **گزینه‌ی ۲** آسان

Casting یعنی تبدیل نوع یک مقدار، مثل `Number("42")` که رشته را به عدد `42` تبدیل می‌کند. ذخیره در متغیر (گزینه‌ی ۱) فقط انتساب است و نوع را عوض نمی‌کند؛ `typeof` (گزینه‌ی ۳) فقط نوع را **گزارش** می‌کند و چیزی را تبدیل نمی‌کند؛ `console.log` (گزینه‌ی ۴) هم فقط مقدار را نمایش می‌دهد.

۱۶۸ **گزینه‌ی ۴** آسان

در تبدیل صریح، خود برنامه‌نویس با صدا زدن `Number()`، `String()` یا `Boolean()` تبدیل را درخواست می‌کند؛ `String(99)` عدد را صریحاً به رشته‌ی `"99"` تبدیل می‌کند. سه گزینه‌ی دیگر نمونه‌ی **تبدیل ضمنی** (type coercion) هستند: در آن‌ها JavaScript خودش وسط عملیات نوع را عوض می‌کند و به ترتیب `"51"`، `6` و `4` به دست می‌آید.

۱۶۹ **گزینه‌ی ۳** آسان

`Number` رشته‌ی عددیِ معتبر `"42"` را به **عدد** `42` تبدیل می‌کند (مثال اسلاید ۲۰). گزینه‌ی ۱ همان رشته‌ی ورودی است و تبدیلی در آن رخ نداده؛ `NaN` (گزینه‌ی ۲) فقط وقتی به دست می‌آید که رشته عدد معتبری نباشد، مثل `"42px"`؛ و `undefined` (گزینه‌ی ۴) هیچ ربطی به این تبدیل ندارد.

۱۷۰ **گزینه‌ی ۱** آسان

عملگر `-` برای رشته‌ها معنایی ندارد، پس JavaScript رشته‌ی `"5"` را به عدد `5` تبدیل می‌کند و حاصل **عدد** `4` است. گزینه‌ی ۲ رفتار `+` است (`"5" + 1` می‌شود `"51"`)؛ گزینه‌ی ۳ مقدار درست را دارد ولی نوعش را اشتباهاً رشته فرض کرده؛ `NaN` (گزینه‌ی ۴) وقتی پیش می‌آمد که رشته عددی نبود، مثل `"abc" - 1` .

۱۷۱ **گزینه‌ی ۱** متوسط

`Number` رشته‌ی خالی را `0` می‌کند و فاصله‌های ابتدا و انتها را نادیده می‌گیرد، پس `"  12  "` می‌شود `12`؛ اما `"12px"` به خاطر `px` عدد معتبری نیست و `NaN` می‌شود. گزینه‌ی ۲ دقیقاً رفتار `parseInt` است، نه `Number` (`parseInt("")` برابر `NaN` و `parseInt("12px")` برابر `12` است). گزینه‌ی ۳ فاصله‌ها را نامعتبر و `px` را بی‌اثر فرض کرده و گزینه‌ی ۴ فکر کرده رشته‌ی خالی و رشته‌ی فاصله‌دار هم عدد نمی‌شوند.

۱۷۲ **گزینه‌ی ۳** متوسط

`undefined` به `NaN` تبدیل می‌شود، نه `0`؛ پس گزینه‌ی ۳ نادرست است. سه گزینه‌ی دیگر درست‌اند: `null` (یعنی «عمداً هیچ») صفر می‌شود، `true` یک می‌شود و آرایه‌ی خالی اول به رشته‌ی `""` و بعد به `0` تبدیل می‌شود. همین تفاوت `null` و `undefined` ریشه‌ی دام معروف `null + 1` برابر `1` ولی `undefined + 1` برابر `NaN` است.

۱۷۳ **گزینه‌ی ۴** متوسط

آرایه‌ها و شیءها اول به رشته تبدیل می‌شوند: `[] -> "" -> 0` و `[5] -> "5" -> 5` . آرایه‌ی `[1, 2]` رشته‌ی `"1,2"` می‌شود که به خاطر ویرگول عدد معتبری نیست، پس `NaN`؛ و `{} -> "[object Object]" -> NaN` . گزینه‌ی ۱ فکر کرده عناصر آرایه با هم جمع می‌شوند، گزینه‌ی ۲ فکر کرده ویرگول حذف می‌شود و شیء خالی صفر است، و گزینه‌ی ۳ فکر کرده هیچ آرایه‌ای به عدد تبدیل نمی‌شود.

۱۷۴ **گزینه‌ی ۳** سخت

رشته‌ای که فقط فاصله دارد، بعد از کنار گذاشتن فاصله‌ها رشته‌ی خالی است و `Number` آن را `0` می‌کند (هر چند `Boolean(" ")` برابر `true` است!). بقیه همگی `NaN` می‌دهند: `"12px"` کاراکتر غیرعددی دارد، `undefined` همیشه `NaN` می‌شود و `{}` اول به `"[object Object]"` و بعد به `NaN` تبدیل می‌شود.

۱۷۵ **گزینه‌ی ۳** متوسط

`parseInt` و `parseFloat` از ابتدای رشته می‌خوانند و در اولین کاراکتر نامعتبر می‌ایستند، پس `"12px"` می‌شود `12`. `parseInt` به نقطه که برسد می‌ایستد، پس `"3.9"` می‌شود `3` (گرد نمی‌کند؛ گزینه‌ی ۱ اشتباهاً گرد کرده). `parseFloat` نقطه را می‌پذیرد، پس `3.9`. و `"abc"` از همان کاراکتر اول نامعتبر است، پس `NaN` (نه `0`؛ گزینه‌ی ۴). گزینه‌ی ۲ رفتار سخت‌گیرانه‌ی `Number` را به این دو تابع نسبت داده است.

۱۷۶ **گزینه‌ی ۴** متوسط

`parseFloat` از ابتدای رشته `3.9` را می‌خواند و به `k` که می‌رسد متوقف می‌شود. `parseInt` همان اول در نقطه‌ی اعشار می‌ایستد و `3` می‌دهد؛ `Number` به خاطر `kg` کل رشته را نامعتبر می‌داند و `NaN` می‌دهد؛ `String` هم همان رشته‌ی `3.9kg` را بدون تغییر برمی‌گرداند.

۱۷۷ **گزینه‌ی ۲** سخت

`Number` سخت‌گیر است و کل رشته را می‌سنجد: `""` می‌شود `0`، `"3.9"` می‌شود `3.9` و `"12px"` می‌شود `NaN`. `parseInt` از ابتدا می‌خواند: رشته‌ی خالی چیزی برای خواندن ندارد، پس `NaN`؛ در `"3.9"` به نقطه می‌رسد، پس `3`؛ و در `"12px"` تا قبل از `p` را می‌خواند، پس `12`. گزینه‌ی ۱ رشته‌ی خالی را برای `parseInt` صفر و `3.9` را گرد کرده، گزینه‌ی ۳ رفتار دو تابع را جابه‌جا کرده و گزینه‌ی ۴ فکر کرده `Number` گرد می‌کند و `parseInt` هم مثل `Number` سخت‌گیر است.

۱۷۸ **گزینه‌ی ۲** آسان

`String(99)` عدد `99` را به رشته‌ی `"99"` تبدیل می‌کند و `typeof` هر رشته `"string"` است. گزینه‌ی ۱ فرض کرده تبدیلی رخ نداده؛ `"object"` (گزینه‌ی ۳) فقط برای `new String(99)` درست بود که یک شیء پوشاننده می‌سازد، نه برای `String(99)` بدون `new`؛ و نتیجه هم تعریف‌نشده نیست (گزینه‌ی ۴).

۱۷۹ **گزینه‌ی ۲** متوسط

`String` هر مقداری را، حتی `null` و `undefined`، بی‌خطر به متن تبدیل می‌کند: `String(null)` می‌شود `"null"`. گزینه‌ی ۱ «هیچ» را رشته‌ی خالی فرض کرده؛ گزینه‌ی ۳ خود مقدار `null` است و تبدیلی در آن رخ نداده؛ و خطای `TypeError` (گزینه‌ی ۴) مخصوص `null.toString()` است، نه `String(null)`.

۱۸۰ **گزینه‌ی ۱** متوسط

آرایه به رشته‌ای تبدیل می‌شود که عناصرش با ویرگول (بدون فاصله و بدون کروشه) به هم چسبیده‌اند: `"1,2"`. هر شیء ساده به `"[object Object]"` تبدیل می‌شود. آرایه‌ی خالی هم به رشته‌ی خالی `""` تبدیل می‌شود که `length` آن (تعداد کاراکترها) `0` است. گزینه‌ی ۲ فکر کرده شکل نوشتاری کد حفظ می‌شود، گزینه‌ی ۳ ویرگول را حذف کرده و گزینه‌ی ۴ هم حاصل تبدیل شیء و هم طول رشته‌ی خالی را اشتباه گرفته است.

۱۸۱ **گزینه‌ی ۳** سخت

`String(null)` و `String(undefined)` بی‌خطرند و `"null"` و `"undefined"` می‌دهند، پس خط ۱ (و خط ۳) مشکلی ندارد. در خط ۲ عدد داخل پرانتز است، پس `.toString()` درست کار می‌کند و `"123"` می‌دهد. اما `null` هیچ متدی ندارد و `null.toString()` در خط ۴ خطای `TypeError: Cannot read properties of null` می‌دهد؛ پس گزینه‌ی «بدون خطا» هم نادرست است.

۱۸۲ **گزینه‌ی ۴** آسان

`NaN` یکی از ۸ مقدار falsy است (`false`، `0`، `-0`، `0n`، `""`، `null`، `undefined`، `NaN`). `"0"` و `" "` رشته‌های **غیرخالی**‌اند (اولی یک کاراکتر صفر و دومی یک کاراکتر فاصله دارد)، پس truthy هستند؛ آرایه‌ی خالی `[]` هم مثل هر شیء دیگری truthy است.

۱۸۳ **گزینه‌ی ۲** متوسط

رشته‌ی `" "` یک کاراکتر (فاصله) دارد، پس خالی نیست و truthy است. `"false"` هم یک رشته‌ی غیرخالی است و `Boolean` آن `true` می‌شود؛ محتوای متن اصلاً مهم نیست (گزینه‌ی ۱). `0n` (صفرِ BigInt) یکی از falsyهاست (گزینه‌ی ۳) و هر شیء، حتی شیء خالی `{}`، truthy است (گزینه‌ی ۴).

۱۸۴ **گزینه‌ی ۲** آسان

عملگر `+` تک‌عملوندی (unary plus) مقدار را دقیقاً با همان قواعد `Number` به عدد تبدیل می‌کند؛ مثلاً `+"42"` می‌شود `42` و `+""` می‌شود `0`. `!!x` معادل `Boolean(x)` است (گزینه‌ی ۱) و `x + ""` معادل `String(x)` (گزینه‌ی ۳). `-x` (گزینه‌ی ۴) هم عدد می‌دهد، اما **قرینه‌ی** آن را؛ مثلاً `-"42"` برابر `-42` است.

---

۱۸۵ **گزینه‌ی ۱** متوسط

`!!x` همان `Boolean(x)` است. `"0"` رشته‌ای غیرخالی است، پس `true`؛ `""` خالی است، پس `false`؛ `[]` مثل هر آرایه‌ای truthy است، پس `true`؛ و `NaN` falsy است، پس `false`. گزینه‌ی ۳ رشته‌ی `"0"` را با عدد `0` اشتباه گرفته، گزینه‌ی ۴ آرایه‌ی خالی را falsy فرض کرده و گزینه‌ی ۲ فکر کرده `!!` همیشه `false` می‌دهد.

---

۱۸۶ **گزینه‌ی ۱** متوسط

unary plus (`+x`) دقیقاً مثل `Number(x)` عمل می‌کند: `""` می‌شود `0`، `"3.5"` می‌شود `3.5`، `"3.5px"` می‌شود `NaN` (سخت‌گیر است و برخلاف `parseFloat` تکه‌ی اول را برنمی‌دارد)، `true` می‌شود `1` و `[] -> "" -> 0`. گزینه‌های ۲ و ۴ رشته‌ی خالی را `NaN` فرض کرده‌اند (و گزینه‌ی ۲ رفتار `parseInt` را هم به `+` نسبت داده)؛ گزینه‌ی ۳ رفتار `parseFloat` را به `+` نسبت داده و `true` را تبدیل‌نشده چاپ کرده است.

---

۱۸۷ **گزینه‌ی ۴** سخت

`"0"` یک رشته‌ی غیرخالی است، پس truthy است؛ اما وقتی به عدد تبدیل شود `0` می‌شود. `""` و `null` هم به `0` تبدیل می‌شوند، ولی خودشان falsy هستند؛ `"1"` truthy است، ولی به `1` تبدیل می‌شود. همین دوگانگی (truthy بودن ولی صفر شدن) ریشه‌ی دام‌هایی مثل `"0" == false` برابر `true` در درس بعد است.

---

۱۸۸ **گزینه‌ی ۲** آسان

`NaN` (Not a Number) حاصل تبدیل یا محاسبه‌ی عددی‌ای است که شکست خورده: `Number("abc")`، `0 / 0` و `"abc" * 2` همگی `NaN` می‌دهند. گزینه‌ی ۱ غلط است، چون `NaN` یک مقدار خاص از نوع number است و `typeof NaN` برابر `"number"` است. گزینه‌ی ۳ غلط است، چون `NaN` با هیچ چیز، حتی خودش، برابر نیست و `NaN === NaN` برابر `false` است (برای تشخیصش `Number.isNaN` لازم است). گزینه‌ی ۴ هم غلط است، چون `NaN` «مسری» است و `NaN + 1` باز هم `NaN` می‌شود.

---

۱۸۹ **گزینه‌ی ۴** سخت

`isNaN` اول ورودی را به عدد تبدیل می‌کند: `"abc"` می‌شود `NaN`، پس `true`؛ اما `""` می‌شود `0` که `NaN` نیست، پس `false` (گزینه‌ی ۲ همین را اشتباه کرده). `Number.isNaN` هیچ تبدیلی انجام نمی‌دهد: `"abc"` یک رشته است نه مقدار `NaN`، پس `false` (گزینه‌ی ۱ این تفاوت را نادیده گرفته)؛ فقط خودِ `NaN` جواب `true` می‌گیرد. گزینه‌ی ۳ هم فکر کرده `isNaN` مثل `Number.isNaN` بدون تبدیل کار می‌کند.

---

۱۹۰ **گزینه‌ی ۳** سخت

`Number("abc")` برابر `NaN` است و `typeof NaN` برابر `"number"`؛ پس گزینه‌ی ۳ درست است. `"12"` به عدد معتبر `12` تبدیل می‌شود، پس `isNaN("12")` برابر `false` است (گزینه‌ی ۱). `Number.isNaN` رشته را تبدیل نمی‌کند، پس برای `"abc"` جواب `false` است (گزینه‌ی ۲). `undefined` به `NaN` تبدیل می‌شود، پس `isNaN(undefined)` برابر `true` است (گزینه‌ی ۴).

---

۱۹۱ **گزینه‌ی ۱** سخت

در اولین چاپ، `age` رشته است و `+` با رشته یعنی چسباندن، پس `"201"` (گزینه‌ی ۲ فکر کرده JavaScript خودش جمع ریاضی انجام می‌دهد). در چاپ دوم و سوم، `Number(age)` و `+age` هر دو عدد `20` می‌دهند و جمع با `1` می‌شود `21` (گزینه‌ی ۳ فکر کرده `+age` رشته می‌ماند). در چاپ آخر، `parseInt(age)` عدد `20` است، اما با **رشته‌ی** `"1"` جمع شده، پس دوباره چسباندن رخ می‌دهد و `"201"` چاپ می‌شود (گزینه‌ی ۴ این را ندیده است).

۱-۱۰ تبدیل نوع ضمنی (Type coercion) و دام‌هایش

۱۹۲ **گزینه‌ی ۱** آسان

اسلاید ۲۲ coercion را «تبدیل خودکار یک نوع داده به نوعی دیگر وقتی عملیات شامل مقدارهایی از نوع‌های مختلف است» تعریف می‌کند. گزینه‌ی ۲ تعریف casting **صریح** (explicit) است؛ گزینه‌ی ۳ غلط است، چون JavaScript برای `"5" + 1` خطا نمی‌دهد و `"51"` می‌سازد؛ گزینه‌ی ۴ هم برعکسِ dynamic typing است: در JavaScript متغیرها نوع ثابتی ندارند.

۱۹۳ **گزینه‌ی ۳** آسان

در مقایسه‌ی عدد با رشته، `==` همیشه **رشته را به عدد** تبدیل می‌کند (`string -> number` در اسلاید ۲۲)، پس `5 == 5` و نتیجه `true` است. گزینه‌ی ۱ جهت تبدیل را برعکس گفته (این کار `+` است، نه `==`)؛ گزینه‌ی ۲ تبدیل `if` را با `==` قاطی کرده؛ گزینه‌ی ۴ رفتار `===` را توصیف می‌کند.

۱۹۴ **گزینه‌ی ۲** آسان

طبق قاعده‌ی اسلاید ۲۲ (`String + Anything -> String`)، چون یک طرف `+` رشته است، `5` هم به رشته تبدیل می‌شود و دو رشته به هم می‌چسبند: `"105"`. گزینه‌های ۱ و ۳ جمع ریاضی انجام داده‌اند (که برای `+` با رشته رخ نمی‌دهد) و گزینه‌ی ۴ مقدار درست را دارد ولی نوعش را عدد فرض کرده است.

۱۹۵ **گزینه‌ی ۳** آسان

`-` همیشه در «بافت عددی» کار می‌کند (اسلاید ۲۲)، پس `'10'` به عدد `10` تبدیل می‌شود و حاصل **عدد** `5` است. گزینه‌ی ۱ رفتار `+` را به `-` نسبت داده، گزینه‌ی ۲ نوع نتیجه را رشته فرض کرده و گزینه‌ی ۴ فکر کرده تفریق از رشته ممکن نیست؛ اما `'10'` یک رشته‌ی عددی معتبر است.

۱۹۶ **گزینه‌ی ۴** آسان

هیچ‌کدام از دو طرف رشته نیست، پس `+` جمع عددی است و booleanها به عدد تبدیل می‌شوند: `1 + 1 = 2` (اسلاید ۲۱: booleans become numbers). چسباندن (گزینه‌ی ۱) فقط وقتی رخ می‌دهد که یک طرف رشته باشد؛ `+` منطق «و/یا» نیست که `true` بدهد (گزینه‌ی ۲)؛ و `true` به `NaN` تبدیل نمی‌شود، بلکه `1` می‌شود (گزینه‌ی ۳).

۱۹۷ **گزینه‌ی ۲** آسان

`11` فقط وقتی ساخته می‌شود که یک طرف `+` رشته باشد: `"1" + 1` یعنی چسباندن `"1"` و `"1"`. در JavaScript هم `1 + 1` برابر `2` است (گزینه‌ی ۱) و میم اغراق می‌کند. `"1" - 1` عددی است و `0` می‌دهد (گزینه‌ی ۳). در گزینه‌ی ۴، `+"1"` رشته را به عدد `1` تبدیل می‌کند و حاصل `2` است.

۱۹۸ **گزینه‌ی ۴** آسان

فقط `+` برای رشته معنا دارد (چسباندن)، پس `"8" + 2` می‌شود رشته‌ی `"82"`. عملگرهای `-`، `*` و `/` همیشه عددی‌اند و رشته‌ی `"8"` را به عدد `8` تبدیل می‌کنند؛ حاصل آن‌ها به ترتیب عددهای `6`، `16` و `4` است.

۱۹۹ **گزینه‌ی ۳** آسان

هیچ طرفی رشته نیست، پس جمع عددی است و `undefined` به `NaN` تبدیل می‌شود؛ هر محاسبه با `NaN` هم `NaN` است (اسلاید ۲۱). گزینه‌ی ۱ رفتار `null` را به `undefined` نسبت داده (`null + 1` برابر `1` است)؛ گزینه‌ی ۲ فقط وقتی درست بود که طرف دیگر رشته‌ی `"1"` بود؛ و `+` هیچ‌وقت خودِ `undefined` را برنمی‌گرداند.

۲۰۰ **گزینه‌ی ۲** آسان

طبق «قانون ویژه»ی اسلاید ۲۲، `null == undefined` برابر `true` است. اما `===` هیچ تبدیلی انجام نمی‌دهد و چون نوع `null` و `undefined` متفاوت است، `null === undefined` برابر `false` است. گزینه‌ی ۱ تفاوت `==` و `===` را نادیده گرفته و گزینه‌های ۳ و ۴ قانون ویژه را نمی‌دانند.

۲۰۱ **گزینه‌ی ۱** آسان

اسلاید ۲۱: «`==` می‌تواند coercion غیرمنتظره ایجاد کند، پس `===` را ترجیح بده». `===` نوع و مقدار را بدون هیچ تبدیلی مقایسه می‌کند. `==` و `!=` (گزینه‌های ۲ و ۴) دقیقاً همان عملگرهایی‌اند که تبدیل انجام می‌دهند و `=` (گزینه‌ی ۳) اصلاً مقایسه نیست، بلکه انتساب (assignment) است.

---

۲۰۲ **گزینه‌ی ۱** متوسط

وقتی یک طرف `+` رشته باشد، **چسباندن** رخ می‌دهد: `"5" + 1` می‌شود `"51"` (اسلاید ۲۱) و `null` و `undefined` هم به متن‌های `"null"` و `"undefined"` تبدیل می‌شوند. فقط در خط دوم رشته‌ای در کار نیست، پس جمع عددی است و `null` به `0` تبدیل می‌شود؛ یعنی `1` . گزینه‌ی ۲ فکر کرده `+` همیشه رشته را عدد می‌کند، گزینه‌ی ۳ فکر کرده `null` و `undefined` حتی کنار رشته هم به عدد تبدیل می‌شوند و گزینه‌ی ۴ فکر کرده `null` در جمع عددی `NaN` می‌شود.

---

۲۰۳ **گزینه‌ی ۴** متوسط

خط اول از چپ: `1 + 2 = 3` ، بعد `3 + "3" = "33"` و از اینجا به بعد همه‌چیز چسباندن است: `"334"` و `"3345"` (گزینه‌ی ۱ فکر کرده از همان اول چسباندن است). در خط دوم پرانتز اول حساب می‌شود: `2 + 3 = 5` و `"1" + 5 = "15"` (گزینه‌ی ۲ پرانتز را نادیده گرفته). در خط سوم `1 + "2" = "12"` و چون `-` عددی است، `"12" - 1 = 11` (گزینه‌ی ۳ فکر کرده تفریق از رشته `NaN` می‌دهد).

---

۲۰۴ **گزینه‌ی ۳** متوسط

`===` نوع را هم مقایسه می‌کند و رشته و عدد هیچ‌وقت «سخت‌گیرانه» برابر نیستند، پس `'5' === 5` برابر `false` است و گزینه‌ی ۳ نادرست است. سه گزینه‌ی دیگر دقیقاً مثال‌های اسلاید ۲۲ هستند: `'5' == 5` (رشته به عدد)، `false == 0` (boolean به عدد) و `null == undefined` (قانون ویژه)، که همگی `true` می‌دهند.

---

۲۰۵ **گزینه‌ی ۱** متوسط

دام اسلاید ۲۱: در `[] == false` اول `false -> 0` ، بعد `[] -> "" -> 0` ، پس `0 == 0` یعنی `true` . در `[] == 0` هم `[] -> "" -> 0` ، پس `true` . اما `===` تبدیل انجام نمی‌دهد و آرایه با boolean هم‌نوع نیست، پس `false` . گزینه‌ی ۲ فکر کرده آرایه‌ی truthy نمی‌تواند با `false` برابر باشد، گزینه‌ی ۳ فرق `==` و `===` را ندیده و گزینه‌ی ۴ مرحله‌ی تبدیل `false` به `0` را فراموش کرده است.

---

۲۰۶ **گزینه‌ی ۲** متوسط

`*` ، `/` و `%` همیشه عددی‌اند، حتی وقتی **هر دو** طرف رشته باشند: `2 * 3 = 6` و `10 * 2 = 20` (مثال‌های اسلایدهای ۲۰ و ۲۲)، `6 / 2 = 3` و باقی‌مانده‌ی `7` بر `2` برابر `1` است. اما `+` با رشته یعنی چسباندن: `"52"` . گزینه‌ی ۱ فکر کرده رشته‌ها در ضرب و تقسیم شرکت نمی‌کنند، گزینه‌ی ۳ برای `+` هم جمع عددی انجام داده و گزینه‌ی ۴ رفتار Python را فرض کرده که در آن ضرب رشته در عدد، رشته را تکرار می‌کند (در Python، `"10" * 2` می‌شود `"1010"`).

---

۲۰۷ **گزینه‌ی ۴** متوسط

در `true + "1"` یک طرف رشته است، پس `true` به متن `"true"` تبدیل و چسبانده می‌شود: `"true1"` . در گزینه‌های ۱ و ۲ هیچ رشته‌ای وجود ندارد، پس جمع عددی است: `false + 1 = 0 + 1 = 1` و `null + null = 0 + 0 = 0` . در گزینه‌ی ۳ عملگر `*` همیشه عددی است: `3 * 1 = 3` .

---

۲۰۸ **گزینه‌ی ۱** متوسط

`+"a"` یک unary plus است که اولویت بالاتری دارد و `"a"` را به عدد تبدیل می‌کند؛ نتیجه `NaN` است. بعد از چپ: `"b" + "a" = "ba"` ، سپس `"ba" + NaN = "baNaN"` و در آخر `"baNaN" + "a" = "baNaNa"` . گزینه‌ی ۲ فکر کرده `+` اضافه نادیده گرفته می‌شود، گزینه‌ی ۳ آن را یک کاراکتر معمولی فرض کرده و گزینه‌ی ۴ فراموش کرده که وجود رشته، کل عبارت را چسباندن می‌کند.

---

۲۰۹ **گزینه‌ی ۳** متوسط

`if (list)` فقط truthy بودن را می‌پرسد و هر آرایه‌ای، حتی خالی، truthy است، پس `A` چاپ می‌شود. طبق الگوریتم `list == false` به `==` می‌رسد و `0 == 0` است، پس `B` هم چاپ می‌شود. اما `list === false` بدون تبدیل `false` است، پس `C` چاپ نمی‌شود. گزینه‌ی ۱ آرایه‌ی خالی را falsy فرض کرده، گزینه‌ی ۲ فکر کرده چیزی که truthy است نمی‌تواند `== false` باشد و گزینه‌ی ۴ فرق `==` و `===` را ندیده است.

۲۱۰ **گزینه‌ی ۲** متوسط

`sort()` بدون تابع مقایسه، عناصر را به رشته تبدیل می‌کند و حرف‌به‌حرف مقایسه می‌کند: `"1"` < `"100000"` < `"21"` < `"30"` < `"4"`؛ یعنی همان `[1, 100000, 21, 30, 4]` میم اسلاید ۲۱. گزینه‌ی ۱ ترتیب عددی است که فقط با `(a, b) => a - b` به دست می‌آید؛ گزینه‌ی ۳ ترتیب عددی نزولی است؛ و گزینه‌ی ۴ فرض کرده `sort()` بدون تابع مقایسه هیچ کاری انجام نمی‌دهد.

۲۱۱ **گزینه‌ی ۴** متوسط

به عنوان رشته، `"10"` با `"1"` شروع می‌شود و از `"9"` کوچک‌تر است؛ پس ترتیب لغت‌نامه‌ای `"1"`، `"10"`، `"9"` است. ترتیب نزولی (گزینه‌ی ۱) `[10, 9, 1]` می‌شد؛ اگر تعداد رقم ملاک بود (گزینه‌ی ۲) `[1, 9, 10]` به دست می‌آمد؛ و گزینه‌ی ۳ هم غلط است، چون با `(a, b) => a - b` همین آرایه به‌درستی `[1, 9, 10]` می‌شود.

۲۱۲ **گزینه‌ی ۱** متوسط

تابع مقایسه‌ی `(a, b) => b - a` وقتی `b` بزرگ‌تر است عدد مثبت برمی‌گرداند، یعنی «`b` جلوتر بیاید»؛ پس عددها به‌صورت عددی و **نزولی** مرتب می‌شوند. گزینه‌ی ۲ نتیجه‌ی `a - b` (صعودی) است و گزینه‌ی ۴ نتیجه‌ی `sort()` بدون تابع مقایسه (ترتیب رشته‌ای). گزینه‌ی ۳ هم ترتیب رشته‌ای **نزولی** است (برعکسِ ترتیب لغت‌نامه‌ای)؛ اما `b - a` با عملگر `-` مقایسه می‌کند که همیشه عددی است، پس رشته‌ای در کار نیست.

۲۱۳ **گزینه‌ی ۳** متوسط

در `Number("5") + 1` تبدیل به‌صورت **صریح** (explicit casting) با `Number` انجام شده و بعد دو عدد با هم جمع می‌شوند؛ پس JavaScript خودش هیچ نوعی را عوض نمی‌کند. در گزینه‌ی ۱ عملگر `*` رشته را خودکار به عدد تبدیل می‌کند، در گزینه‌ی ۲ عملگر `==` رشته را به عدد تبدیل می‌کند و در گزینه‌ی ۴ عملگر `+` عدد را به رشته تبدیل می‌کند.

۲۱۴ **گزینه‌ی ۲** متوسط

اینجا `{}` داخل یک عبارت (سمت راست `=`) آمده، پس یک شیء است و به `"[object Object]"` تبدیل می‌شود؛ `[]` هم به رشته‌ی خالی تبدیل می‌شود و حاصل چسباندن آن‌ها `"[object Object]"` است. جواب `0` (گزینه‌ی ۱) فقط وقتی پیش می‌آید که `{} + []` به‌تنهایی در ابتدای یک دستور نوشته شود و `{}` بلوک کد خالی فهمیده شود. `+` با رشته `NaN` نمی‌سازد (گزینه‌ی ۳) و `[]` به `"[]"` تبدیل نمی‌شود (گزینه‌ی ۴).

۲۱۵ **گزینه‌ی ۲** سخت

آرایه‌ها اول به رشته تبدیل می‌شوند: `1 + "2" = "12"` (چسباندن). در `[2] * [3]` عملگر `*` عددی است: `"2" * "3" = 6`. در `[] - 1` هم `0` -> `""` -> `[]` و `0 - 1 = -1`. در خط آخر `"1,2" + "3"` می‌شود `"1,23"`؛ دقت کن که `+` آرایه‌ها را ادغام نمی‌کند. گزینه‌های ۱ و ۴ فکر کرده‌اند آرایه‌ی تک‌عضوی مستقیم عدد است و `+` آرایه‌ها را یکی می‌کند؛ گزینه‌ی ۳ هم `*` را چسباندن و `[] - 1` را `NaN` فرض کرده است.

۲۱۶ **گزینه‌ی ۳** سخت

`[]` به `""` و `{}` به `"[object Object]"` تبدیل می‌شوند و `+` آن‌ها را می‌چسباند: `[] + []`. برابر رشته‌ی **خالی** `""` است، نه `"[]"` (گزینه‌ی ۱). در `[] + 1` یک طرف (بعد از تبدیل) رشته‌ی `""` است، پس حاصل رشته‌ی `"1"` است، نه عدد `1` (گزینه‌ی ۲). و `-` همیشه عددی است: `0 - 0 = 0`، نه رشته‌ی خالی (گزینه‌ی ۴).

۲۱۷ **گزینه‌ی ۱** سخت

`"0"` رشته‌ای غیرخالی است، پس truthy است (`true`)؛ اما در `"0" == false` اول `false -> 0` و بعد `"0" -> 0`، پس `true`. `null` falsy است (`false`)، ولی `==` برای `null` قانون ویژه دارد و `null` فقط با `undefined` برابر است، پس `null == false` برابر است. گزینه‌های ۲ و ۴ رشته‌ی `"0"` را falsy گرفته‌اند و گزینه‌های ۲ و ۳ فکر کرده‌اند هر مقدار falsy با `false` برابر است.

۲۱۸ **گزینه‌ی ۲** سخت

boolean همیشه اول به عدد تبدیل می‌شود. `true == "1"` : یعنی `1 == "1"` و بعد `1 == 1`، پس `true`. `true == "true"` : یعنی `1 == "true"`، و `"true"` به `NaN` تبدیل می‌شود، پس `false`. `"2" == true` : یعنی `"2" == 1` و بعد `2 == 1`، پس `false`. گزینه‌ی ۱ فکر کرده هر رشته‌ی truthy با `true` برابر است، گزینه‌ی ۳ فکر کرده boolean و رشته هیچ‌وقت برابر نمی‌شوند و گزینه‌ی ۴ متنِ `"true"` را با مقدار `true` یکی گرفته است.

۲۱۹ **گزینه‌ی ۴** سخت

طبق قانون ویژه، `null` در `==` فقط با `undefined` (و خودش) برابر است؛ نه با `0` و نه با `false`. پس `null == false` برابر `false` است و گزینه‌ی ۴ نادرست است، هرچند `null` falsy است. سه گزینه‌ی دیگر درست‌اند: `null == 0` و `undefined == 0` هر دو به خاطر همین قانون ویژه `false` هستند و `NaN` با هیچ چیز، حتی خودش، برابر نیست.

۲۲۰ **گزینه‌ی ۳** سخت

JavaScript تقریباً همیشه به جای خطا دادن تبدیل می‌کند: خط ۱ عدد `10` و خط ۲ عدد `1` می‌دهد (`0 + 1`)، و `undefined + 1` در خط ۳ هم فقط `NaN` می‌شود، نه خطا. تنها استثنا BigInt است: در خط ۴، `10n + 1` خطای `TypeError: Cannot mix BigInt and other types` می‌دهد، چون BigInt با عدد معمولی coercion نمی‌شود و باید صریحاً تبدیل شود (`10n + 1n`). پس گزینه‌ی «بدون خطا» هم نادرست است.

۲۲۱ **گزینه‌ی ۴** سخت

از چپ به راست و دوبه‌دو: `true + null` رشته ندارد، پس عددی است: `1 + 0 = 1`. بعد `1 + "1"` چسباندن است: `"11"`. در آخر `"11" + undefined` هم چسباندن است: `"11undefined"`. گزینه‌ی ۱ فکر کرده از همان اول چسباندن است؛ گزینه‌ی ۲ فکر کرده `undefined` همه‌چیز را `NaN` می‌کند، در حالی که اینجا با رشته طرف است؛ گزینه‌ی ۳ هم `null` را `1` حساب کرده است.

۲۲۲ **گزینه‌ی ۱** سخت

خط اول: `"3" - 1` عددی است و `2` می‌شود، بعد `2 + "1"` چسباندن است: `"21"`. خط دوم: ضرب اولویت دارد، `4 * 5 = 20` و بعد `"3" + 20 = "320"`. خط سوم: هر `+"4"` یک unary plus است که رشته را به عدد `4` تبدیل می‌کند، پس `4 + 4 = 8`. گزینه‌ی ۲ فکر کرده JavaScript همه‌ی رشته‌ها را به عدد تبدیل می‌کند (`2 + 1 = 3` و `3 + 20 = 23`). گزینه‌ی ۳ اولویت ضرب را نادیده گرفته و از چپ حساب کرده (`("3" + 4) * 5` یعنی `"34" * 5 = 170`) و فکر کرده unary plus رشته را عدد نمی‌کند (`"4" + "4" = "44"`). گزینه‌ی ۴ فقط همین اشتباه آخر را دارد.

۲۲۳ **گزینه‌ی ۴** سخت

در `[0] == false` اول `false -> 0` و بعد `[0] -> "0" -> 0`، پس `0 == 0` یعنی `true`. گزینه‌ی ۱: `"abc"` به `NaN` تبدیل می‌شود، ولی `NaN` با هیچ چیز (حتی `NaN`) برابر نیست. گزینه‌ی ۲: دو آرایه‌ی جداگانه‌اند و `==` بین دو شیء فقط مرجع را مقایسه می‌کند. گزینه‌ی ۳: طبق قانون ویژه، `undefined` فقط با `null` برابر است.

۲۲۴ **گزینه‌ی ۲** سخت

در `"" == 0` رشته‌ی خالی به `0` تبدیل می‌شود، پس `true`. در `" " == 0` هم رشته‌ای که فقط فاصله دارد به `0` تبدیل می‌شود، پس `true` (گزینه‌ی ۱ این را نمی‌داند). اما در `"0" == ""` **هر دو طرف رشته‌اند**، پس هیچ تبدیلی رخ نمی‌دهد و دو رشته‌ی متفاوت مقایسه می‌شوند، پس `false` (گزینه‌ی ۳ فکر کرده چون هر دو با `0` برابرند، با هم هم برابرند). گزینه‌ی ۴ هم coercion را کاملاً نادیده گرفته است.

۱-۱۱ عملگرهای مقایسه و منطقی

۲۲۵ **گزینه‌ی ۲** آسان

`===` مقدار **و** نوع را مقایسه می‌کند و هیچ type coercion انجام نمی‌دهد؛ برای همین `5 === "5"` برابر `false` است. گزینه‌ی ۱ تعریف `==` است؛ گزینه‌ی ۳ غلط است چون `5 === 6` با وجود نوع یکسان `false` است؛ و گزینه‌ی ۴ هم غلط است، چون طبق اسلاید ۳۱ primitiveها با مقدارشان مقایسه می‌شوند، نه با مرجع.

۲۲۶ **گزینه‌ی ۳** آسان

`5 == "5"` رشته را به عدد تبدیل می‌کند، پس `true`؛ `5 === "5"` نوع‌ها را هم می‌سنجد، پس `false`. `!=` نقیضِ `==` است، پس `!==` نقیضِ `===` است، پس `true`. گزینه‌ی ۱ فرق `==` و `===` را ندیده، گزینه‌ی ۲ نقیض‌ها را برعکس جفت کرده و گزینه‌ی ۴ فکر کرده `==` هم تبدیل نوع انجام نمی‌دهد.

۲۲۷ **گزینه‌ی ۴** آسان

`7 !== 7` می‌پرسد «آیا `7` و `7` (از نظر مقدار یا نوع) متفاوت‌اند؟» و جوابش `false` است؛ پس گزینه‌ی ۴ نادرست است. گزینه‌ی ۱ درست است، چون primitiveها (از جمله رشته‌ها) با مقدارشان مقایسه می‌شوند؛ `>=` در حالت تساوی هم `true` می‌دهد (گزینه‌ی ۲)؛ و `3 < 5` هم بدیهی است.

۲۲۸ **گزینه‌ی ۲** متوسط

`!==` بدون تبدیل مقایسه می‌کند و چون نوع رشته و عدد فرق دارد، `"1" !== 1` برابر `true` است. `null == undefined` درست است، پس نقیضش `null != undefined` برابر `false` است (گزینه‌ی ۱). `NaN` با خودش برابر نیست، پس `NaN != NaN` برابر `true` است (گزینه‌ی ۳). و `"1" == 1` با تبدیل نوع `true` است، پس `"1" != 1` برابر `false` است (گزینه‌ی ۴).

۲۲۹ **گزینه‌ی ۱** سخت

`a` و `b` دو آرایه‌ی **جدا** با محتوای یکسان‌اند و `==` بین دو آرایه فقط مرجع را مقایسه می‌کند، پس `false`؛ اما `c` به **همان** آرایه‌ی `a` اشاره می‌کند، پس `true`. در خط بعد، `[] == 1` یک طرف primitive دارد، پس آرایه به `"1"` و بعد به `1` تبدیل می‌شود و نتیجه `true` است؛ ولی `[] == []` دو آرایه‌ی جداست، پس `false`. گزینه‌های ۲ و ۴ آرایه‌ها را با محتوا مقایسه کرده‌اند و گزینه‌ی ۳ نمی‌داند که آرایه در برابر عدد به primitive تبدیل می‌شود.

۲۳۰ **گزینه‌ی ۴** سخت

`p1` و `p2` دو شیء جداگانه‌اند (دو «خانه»ی جدا با دکوراسیون یکسان)، پس `p1 == p2` برابر `false` است. `p3` همان مرجعِ `p1` را دارد، پس `p1 === p3` برابر `true` است. اما `p1.name` و `p2.name` رشته‌اند و primitiveها با **مقدار** مقایسه می‌شوند، پس `"Ali" === "Ali"` برابر `true` است. گزینه‌ی ۱ شیءها را با محتوا مقایسه کرده، گزینه‌ی ۲ مرجع مشترک را ندیده و گزینه‌ی ۳ فکر کرده رشته‌ها هم با مرجع مقایسه می‌شوند.

۲۳۱ **گزینه‌ی ۲** سخت

دو رشته حرف‌به‌حرف و با کد Unicode مقایسه می‌شوند: `"a"` از `"b"` کوچک‌تر است، پس `true`. کد `"Z"` (۹۰) از کد `"a"` (۹۷) کوچک‌تر است، پس `true` (گزینه‌های ۱ و ۳ فکر کرده‌اند ترتیب الفبایی به بزرگی و کوچکی حروف ربطی ندارد). در `"10" < "9"` اولین حرف‌ها `"1"` و `"9"` هستند، پس `true` (گزینه‌ی ۴ عددی مقایسه کرده). اما در `"10" < 9` یک طرف عدد است، پس مقایسه عددی است: `10 < 9` یعنی `false`.

۲۳۲ **گزینه‌ی ۳** سخت

در `2 > "12"` یک طرف عدد است، پس `"12"` به عدد `12` تبدیل می‌شود و `2 > 12` برابر `false` است؛ پس گزینه‌ی ۳ نادرست است. بقیه درست‌اند: `"app"` ابتدای `"apple"` است و رشته‌ی کوتاه‌تر کوچک‌تر است (گزینه‌ی ۱)؛ کد `"A"` از کد `"a"` کوچک‌تر است (گزینه‌ی ۲)؛ و در `"2" > "12"` هر دو طرف رشته‌اند و `"2"` از `"1"` بزرگ‌تر است (گزینه‌ی ۴).

۲۳۳ **گزینه‌ی ۱** متوسط

نوع دو طرف در هر سه مقایسه فرق دارد یا boolean است، پس همه به عدد تبدیل می‌شوند: `1 > 0` یعنی `true`؛ `5 > 3` یعنی `true`؛ و `"a"` به `NaN` تبدیل می‌شود و هر مقایسه‌ای با `NaN` برابر `false` است. گزینه‌ی ۲ فکر کرده booleanها قابل مقایسه نیستند، گزینه‌ی ۳ رشته‌ی `"5"` را به عدد تبدیل نکرده و گزینه‌ی ۴ فکر کرده حروف از رقم‌ها بزرگ‌ترند (که فقط در مقایسه‌ی دو **رشته** معنا دارد).

۲۳۴ **گزینه‌ی ۱** سخت

`==` برای `null` قانون ویژه دارد و `null` فقط با `undefined` برابر است، پس `null == 0` برابر `false` است. اما `>` و `>=` مقدار `null` را به `0` تبدیل می‌کنند: `0 > 0` یعنی `false` و `0 >= 0` یعنی `true`. در `==` هم فقط با `null` برابر است و `undefined` هم فقط با `null` برابر است و در `>=` به `NaN` تبدیل می‌شود، پس هر دو `false` هستند. گزینه‌های ۲ و ۴ فکر کرده‌اند `null == 0` است، و گزینه‌ی ۳ نمی‌داند که `>=` مقدار `null` را صفر می‌کند.

۲۳۵ **گزینه‌ی ۳** متوسط

`NaN` با هیچ چیز، حتی خودش، برابر نیست؛ پس `x == NaN` و `x === x` هر دو `false` هستند و `x !== x` برابر `true` است (یعنی `NaN` تنها مقداری است که با خودش «نابرابر» است!). `Number.isNaN(x)` راه درست تشخیص است و `true` می‌دهد. گزینه‌های ۱ و ۲ فکر کرده‌اند `NaN` با خودش برابر است و گزینه‌ی ۴ فکر کرده `Number.isNaN` هم نمی‌تواند `NaN` را تشخیص دهد.

۲۳۶ **گزینه‌ی ۲** سخت

مقایسه‌ها دوبه‌دو از چپ انجام می‌شوند. در خط اول، `1 < 2` برابر `true` است و بعد `true < 3` یعنی `1 < 3` که `true` است. در خط دوم، `3 > 2` برابر `true` است و بعد `true > 1` یعنی `1 > 1` که `false` است. گزینه‌ی ۱ عبارت‌ها را مثل ریاضی خوانده، گزینه‌ی ۳ فکر کرده مقایسه‌ی boolean با عدد همیشه `false` است و گزینه‌ی ۴ ترتیب را برعکس حساب کرده است.

۲۳۷ **گزینه‌ی ۱** آسان

طبق اسلاید ۳۱، عملگرهای منطقی مثل زبان C هستند: `&&` (AND)، `||` (OR) و `!` (NOT). `OR` و `or` (گزینه‌های ۲ و ۳) کلمه‌ی کلیدی JavaScript نیستند (`or` در Python است)؛ و `|` تکی (گزینه‌ی ۴) یک عملگر بیتی (bitwise) است، نه OR منطقی.

۲۳۸ **گزینه‌ی ۴** آسان

`&&` فقط وقتی `true` است که هر دو طرف `true` باشند، پس `false` . `||` وقتی `true` است که حداقل یک طرف `true` باشد، پس `true` و `!true` برابر `false` است. گزینه‌ی ۱ تعریف `&&` را با `||` قاطی کرده، گزینه‌ی ۲ نتیجه‌ی `||` و `!` را برعکس گرفته و گزینه‌ی ۳ جای نتیجه‌ی `&&` و `||` را عوض کرده است.

۲۳۹ **گزینه‌ی ۳** آسان

«هر دو شرط با هم» یعنی AND، که در JavaScript با `&&` نوشته می‌شود. `||` (گزینه‌ی ۱) با برقرار بودن **یکی** از دو شرط هم `true` می‌دهد؛ `!` (گزینه‌ی ۲) عملگری تک‌عملوندی است و دو شرط را به هم وصل نمی‌کند؛ و `==` (گزینه‌ی ۴) نتیجه‌ی دو مقایسه را با هم مقایسه می‌کند، نه اینکه هر دو را لازم بداند (مثلاً اگر هر دو `false` باشند، `false == false` برابر `true` می‌شود!).

۲۴۰ **گزینه‌ی ۲** متوسط

اولویت `&&` از `||` بیشتر است، پس خط اول یعنی `true || (false && false)` که `true || false` و در نتیجه `true` است. در خط دوم پرانتز اول حساب می‌شود: `true && false` یعنی `false` . در خط سوم `!` اولویت بالاتری دارد: `(!false) && false` یعنی `true && false` که `false` است. گزینه‌های ۱ و ۴ خط اول را از چپ به راست حساب کرده‌اند و گزینه‌های ۳ و ۴ در خط سوم `!` را روی کل عبارت اعمال کرده‌اند.

۲۴۱ **گزینه‌ی ۴** متوسط

طبق قانون دمورگان، وقتی `!` را به داخل پرانتز می‌بریم هر دو طرف نقیض می‌شوند و `&&` به `||` تبدیل می‌شود: `!(a && b)` همان `!a || !b` است. «نه هر دو» یعنی «حداقل یکی نه». گزینه‌ی ۱ اشتباه رایج است: مثلاً برای `a = true` و `b = false` مقدار `!(a && b)` برابر `true` است، ولی `!a && !b` برابر `false` . گزینه‌ی ۲ برای `a = b = true` و گزینه‌ی ۳ برای `a = true` و `b = false` جواب متفاوتی می‌دهند.

۲۴۲ **گزینه‌ی ۲** آسان

این همان اتصال کوتاه (short-circuit) است: وقتی `a` falsy است، جواب `&&` از قبل معلوم است، پس JavaScript سراغ `b` نمی‌رود و خودِ `a` را برمی‌گرداند؛ مثلاً `0 && f()` تابع `f` را اجرا نمی‌کند و حاصلش `0` است. گزینه‌ی ۱ از دو جهت غلط است: `b` ارزیابی نمی‌شود و حاصل هم لزوماً `false` نیست. JavaScript برای مقدارهای غیر boolean خطا نمی‌دهد (گزینه‌ی ۳)، و گزینه‌ی ۴ رفتار حالتی است که `a` **truthy** باشد.

---

۲۴۳ **گزینه‌ی ۴** متوسط

در `&&` طرف راست فقط وقتی اجرا می‌شود که طرف چپ truthy باشد، پس `A` چاپ نمی‌شود ولی `B` چاپ می‌شود. در `||` طرف راست فقط وقتی اجرا می‌شود که طرف چپ falsy باشد، پس `C` چاپ نمی‌شود ولی `D` چاپ می‌شود. گزینه‌ی ۱ اتصال کوتاه را نمی‌شناسد و گزینه‌های ۲ و ۳ قاعده‌ی `||` (و در گزینه‌ی ۳ قاعده‌ی `&&`) را برعکس گرفته‌اند.

---

۲۴۴ **گزینه‌ی ۳** متوسط

در خط ۲، `user` برابر `null` و falsy است، پس `&&` اتصال کوتاه می‌کند و اصلاً به `user.name` نمی‌رسد (حاصل `null` است). خط ۳ هم بی‌خطر است و `"Guest"` می‌دهد. اما در خط ۴ اول باید خودِ `user.name` حساب شود و خواندن ویژگی از `null` خطای `TypeError` می‌دهد؛ `||` بعد از آن می‌آید و دیگر فرصتی برای نجات ندارد. پس گزینه‌ی «بدون خطا» هم نادرست است.

---

۲۴۵ **گزینه‌ی ۳** سخت

در خط ۱، `"x"` truthy است و `||` اتصال کوتاه می‌کند، پس `y` اصلاً خوانده نمی‌شود. در خط ۲ هم `false` باعث می‌شود `&&` سراغ `y` نرود. اما در خط ۳ عملوند **اول** خودِ `y` است و باید خوانده شود؛ چون `y` تعریف نشده، خطای `ReferenceError: y is not defined` رخ می‌دهد. پس گزینه‌ی «بدون خطا» هم نادرست است.

---

۲۴۶ **گزینه‌ی ۴** متوسط

`&&` اگر عملوند اول truthy باشد، **خودِ عملوند دوم** را برمی‌گرداند؛ `"a"` رشته‌ای غیرخالی و truthy است، پس حاصل `"b"` است. برخلاف زبان C، حاصل لزوماً boolean نیست (گزینه‌ی ۱). عملوند اول فقط وقتی برگردانده می‌شد که falsy بود (گزینه‌ی ۲)، و `&&` هیچ‌وقت رشته‌ها را نمی‌چسباند (گزینه‌ی ۳).

---

۲۴۷ **گزینه‌ی ۳** متوسط

`null` falsy است، پس `||` عملوند دوم یعنی `"x"` را برمی‌گرداند. در گزینه‌ی ۱، `""` falsy است و `&&` همان را برمی‌گرداند، پس حاصل `""` است. در گزینه‌ی ۲، `1` truthy است، پس `&&` عملوند دوم یعنی `0` را برمی‌گرداند. در گزینه‌ی ۴، `0` falsy است، پس `||` عملوند دوم یعنی `null` را برمی‌گرداند (حتی با اینکه `null` هم falsy است).

---

۲۴۸ **گزینه‌ی ۱** سخت

خط اول: `&&` اولویت دارد؛ `0 && "A"` حاصلش `0` است (اولین falsy) و بعد `0 || "B"` حاصلش `"B"` است. خط دوم: اول `"B" && "C"` حساب می‌شود که `"C"` است و بعد `"A" || "C"` چون `"A"` truthy است، `"A"` را برمی‌گرداند. خط سوم: `||` اولین مقدار truthy را برمی‌گرداند و چون `null`، `0` و `""` همه falsy هستند، حاصل `"last"` است. گزینه‌ی ۲ فکر کرده حاصل همیشه boolean است، و گزینه‌های ۳ و ۴ اولویت `&&` یا قاعده‌ی برگرداندن عملوند را اشتباه گرفته‌اند.

---

۲۴۹ **گزینه‌ی ۴** متوسط

`&&` و `||` در JavaScript یکی از **خود عملوندها** را برمی‌گردانند؛ مثلاً `"a" && "b"` برابر `"b"` و `0 || "x"` برابر `"x"` است؛ پس گزینه‌ی ۴ نادرست است. سه گزینه‌ی دیگر درست‌اند: اتصال کوتاه دقیقاً مثل C است، `!` همیشه boolean واقعی می‌دهد (برای همین `!!x` همان `Boolean(x)` است) و `&&` قبل از `||` انجام می‌شود.

---

۲۵۰ **گزینه‌ی ۱** آسان

`"hello"` رشته‌ای غیرخالی و truthy است، پس `!"hello"` برابر `false` و `!!"hello"` (دو بار نقیض) برابر `true` است. `0` falsy است، پس `!0` برابر `true` است. گزینه‌ی ۲ truthy و falsy را برعکس گرفته (`"hello"` را falsy و `0` را truthy فرض کرده)، گزینه‌ی ۳ نقیض دوم را نادیده گرفته و گزینه‌ی ۴ فقط صفر را truthy حساب کرده است.

---

۲۵۱ **گزینه‌ی ۲** سخت

اول `[]!` حساب می‌شود: آرایه‌ی خالی truthy است، پس `[]!` برابر `false` است. عبارت تبدیل می‌شود به `[] == false` که همان دام اسلاید ۲۱ است: `false -> 0` و `[] -> "" -> 0` ، پس `0 == 0` یعنی `true` . به نظر می‌رسد «یک چیز با نقیض خودش برابر است»، اما در واقع دو تبدیل متفاوت (truthy بودن برای `!` و تبدیل عددی برای `==`) رخ داده است. JavaScript اینجا خطا نمی‌دهد و `==` همیشه boolean برمی‌گرداند، نه `undefined` .

۲۵۲ **گزینه‌ی ۴** متوسط

`0` falsy است، پس `volume || 50` عملوند دوم یعنی `50` را برمی‌گرداند؛ این همان دام «مقدار پیش‌فرض با `||` » است که مقدار معتبرِ `0` را دور می‌ریزد. اما `??` فقط وقتی طرف راست را برمی‌گرداند که طرف چپ `null` یا `undefined` باشد؛ `0` هیچ‌کدام نیست، پس `b` برابر `0` می‌ماند. گزینه‌ی ۱ فکر کرده `||` صفر را نگه می‌دارد، گزینه‌ی ۲ فکر کرده `??` هم مثل `||` با falsyها رفتار می‌کند و گزینه‌ی ۳ رفتار دو عملگر را جابه‌جا کرده است.

۲۵۳ **گزینه‌ی ۱** متوسط

`??` (nullish coalescing) طرف راست را **فقط** وقتی برمی‌گرداند که طرف چپ `null` یا `undefined` باشد؛ پس `undefined ?? "x"` برابر `"x"` است. `0` ، `""` و `false` با اینکه falsy هستند، nullish (یعنی `null` یا `undefined`) نیستند؛ پس `??` خودِ آن‌ها را برمی‌گرداند و حاصل گزینه‌های ۲ تا ۴ به ترتیب `0` ، `""` و `false` است. اگر به جای `??` از `||` استفاده می‌شد، هر چهار عبارت `"x"` می‌دادند.

۱-۱۲ کنترل جریان و دستورهای شرطی (if و switch)

۲۵۴ **گزینه‌ی ۲** آسان

اسلاید ۲۹ می‌گوید: «Control flow decides which code runs and when» و «By default, JS runs top to bottom». شرط‌ها (`if` ، `else`) مسیر را دوشاخه می‌کنند و حلقه‌ها (`for` ، `while`) کد را تکرار می‌کنند؛ گزینه‌ی ۳ نقش این دو را جابه‌جا گفته است. گزینه‌های ۱ و ۴ هم با «اجرای خط‌به‌خط از بالا به پایین» در تضادند؛ ترتیب نوشتن خط‌ها همان ترتیب اجرای آن‌هاست.

۲۵۵ **گزینه‌ی ۱** آسان

`18 >= 18` درست است (بزرگ‌تر **یا مساوی**)، پس فقط بلوک `if` اجرا می‌شود. گزینه‌ی ۲ عملگر `>=` را با `>` اشتباه گرفته و گزینه‌ی ۴ فکر کرده هر دو بلوک اجرا می‌شوند، در حالی که در `if / else` دقیقاً یکی اجرا می‌شود. حلقه از `i = 1` شروع می‌شود و تا وقتی `i <= 3` است ادامه دارد، پس ۱، ۲ و ۳ چاپ می‌شوند؛ گزینه‌ی ۳ شمارش را از صفر گرفته است.

۲۵۶ **گزینه‌ی ۳** آسان

طبق اسلاید ۳۰: `if` یعنی «اگر شرط درست است کدی را اجرا کن» (فقط یک مسیر که یا اجرا می‌شود یا نه)، `if / else` یعنی «انتخاب بین دو مسیر»، `else if` یعنی «زنجیر کردن چند شرط» و `switch` یعنی «مدیریت چند حالت مشخص». در `if / else` همیشه دقیقاً یکی از دو بلوک اجرا می‌شود.

۲۵۷ **گزینه‌ی ۴** آسان

شرط `if` با قانون truthy/falsy سنجیده می‌شود و `NaN` یکی از ۸ مقدار falsy است، پس شاخه‌ی `else` اجرا می‌شود. `"0"` و `"false"` رشته‌های **غیرخالی** هستند و truthyاند (مهم نیست داخلشان چه نوشته شده) و آرایه‌ی خالی `[]` هم مثل هر شیء دیگری truthy است؛ پس گزینه‌های ۱ تا ۳ هر سه `yes` چاپ می‌کنند.

۲۵۸ **گزینه‌ی ۲** سخت

`===` تبدیل نوع انجام نمی‌دهد و رشته‌ی `"0"` با عدد `0` برابر نیست، پس شاخه‌ی اول رد می‌شود (گزینه‌ی ۱ رفتار `==` را در نظر گرفته است). در شاخه‌ی دوم، `"0"` یک رشته‌ی غیرخالی و در نتیجه truthy است، پس `non-empty text` چاپ می‌شود؛ گزینه‌ی ۳ فکر کرده `"0"` مثل عدد صفر falsy است. گزینه‌ی ۴ هم غلط است، چون در زنجیره‌ی `else if` حداکثر یک شاخه اجرا می‌شود.

۲۵۹ **گزینه‌ی ۱** متوسط

شرط‌ها به ترتیب بررسی می‌شوند: `28 > 30` نادرست و `28 > 20` درست است؛ پس `Warm` چاپ می‌شود و بقیه‌ی زنجیره **اصلاً بررسی نمی‌شود**، با این‌که `28 > 10` هم درست است. گزینه‌های ۲ و ۳ زنجیره‌ی `if` `else if` را با چند جدا اشتباه گرفته‌اند و گزینه‌ی ۴ مقایسه‌ی اول را اشتباه حساب کرده است.

۲۶۰ **گزینه‌ی ۳** سخت

اولین شرطِ درست برنده است: `1200 > 100` درست است، پس `discount = 5` اجرا می‌شود و بقیه‌ی زنجیره رد می‌شود. با این ترتیب، شاخه‌های `total > 500` و `total > 1000` **هیچ‌وقت** به نوبت نمی‌رسند، چون هر عددی که از ۵۰۰ یا ۱۰۰۰ بزرگ‌تر باشد از ۱۰۰ هم بزرگ‌تر است. گزینه‌ی ۱ فرض کرده «دقیق‌ترین» شرط انتخاب می‌شود (یا همه‌ی شاخه‌ها اجرا شده‌اند و آخری مانده است). راه درست این است که شرط‌ها را از سخت‌گیرانه‌ترین به کلی‌ترین بچینیم: اول `> 1000`، بعد `> 500` و بعد `> 100`.

۲۶۱ **گزینه‌ی ۴** سخت

این کد **دو** ساختار مستقل دارد. `if` خط ۲ تنهاست و چون `15 > 10` است، `A` چاپ می‌شود. خط ۳ شروع یک زنجیره‌ی جداست: `15 > 5` درست است، پس `B` چاپ می‌شود و `else if` و `else` بعدی رد می‌شوند. گزینه‌ی ۱ هر چهار خط را یک زنجیره فرض کرده، گزینه‌ی ۳ فکر کرده `else if` هم با درست بودن شرطش اجرا می‌شود و گزینه‌ی ۲ `if` اول را نادیده گرفته است.

۲۶۲ **گزینه‌ی ۲** متوسط

شرط بیرونی (`loggedIn`) درست است، پس وارد بلوک اول می‌شویم و `else` بیرونی (گزینه‌ی ۳) کنار می‌رود. شرط درونی `itemCount` برابر `0` است که falsy است، پس `else` درونی اجرا می‌شود و `Cart is empty` چاپ می‌شود؛ گزینه‌ی ۱ صفر را truthy فرض کرده است. گزینه‌ی ۴ هم غلط است، چون از هر `if / else` دقیقاً یک شاخه اجرا می‌شود.

۲۶۳ **گزینه‌ی ۳** آسان

مقدار `light` با caseها مقایسه می‌شود و با `"yellow"` جور است؛ `Slow down` چاپ می‌شود و `break` اجرا را از `switch` بیرون می‌برد. گزینه‌های ۲ و ۴ فقط وقتی درست بودند که `break` نوشته نشده بود (fall-through). `default` (گزینه‌ی ۱) فقط وقتی انتخاب می‌شود که هیچ caseی مطابق نباشد.

۲۶۴ **گزینه‌ی ۱** آسان

`default` بخش «در هر حالت دیگر» در `switch` است: اگر هیچ caseی با مقدار برابر نباشد، اجرا از `default` شروع می‌شود. چون همه‌ی caseها `break` دارند، بعد از اجرای case مطابق از `switch` خارج می‌شویم و به `default` نمی‌رسیم (رد گزینه‌ی ۲). گزینه‌ی ۳ پایه‌ای ندارد و گزینه‌ی ۴ هم غلط است، چون `default` کد واقعی است و اجرا می‌شود.

۲۶۵ **گزینه‌ی ۴** متوسط

`switch` مقدار را با caseها **سخت‌گیرانه** (مثل `===`) مقایسه می‌کند و هیچ تبدیل نوعی انجام نمی‌دهد. `"2" === 2` نادرست است (یکی رشته و دیگری عدد)، پس هیچ caseی جور نمی‌شود و `default` اجرا می‌شود. گزینه‌ی ۱ فرض کرده مقایسه مثل `==` است و گزینه‌ی ۲ نقش `break` را ندیده است. گزینه‌ی ۳ هم غلط است، چون مقایسه‌ی دو مقدار ناهم‌نوع خطا نمی‌دهد، فقط نتیجه‌اش `false` است.

۲۶۶ **گزینه‌ی ۲** متوسط

اجرا از case مطابق یعنی `"banana"` **شروع** می‌شود؛ caseهای **بالاتر** اجرا نمی‌شوند، پس `red` چاپ نمی‌شود (رد گزینه‌ی ۴). case `"banana"` با `break` تمام نشده، پس اجرا بدون بررسی مقدار به case بعدی سقوط می‌کند (fall-through) و `purple` هم چاپ می‌شود. آن‌جا `break` هست و از `switch` خارج می‌شویم، پس `default` اجرا نمی‌شود (رد گزینه‌ی ۳). گزینه‌ی ۱ فرض کرده هر case خودبه‌خود تمام می‌شود.

۲۶۷ **گزینه‌ی ۱** متوسط

caseهای `12`، `1` و `2` پشت سر هم و بدون کد آمده‌اند؛ این روش رایجِ **گروه‌بندی** چند مقدار با یک کد مشترک است. مقدار `2` با `case 2` جور می‌شود، `Winter` چاپ می‌شود و `break` اجرا را از `switch` بیرون می‌برد (رد گزینه‌ی ۲). case بدون کد کاملاً مجاز است (رد گزینه‌ی ۴) و `default` فقط وقتی اجرا می‌شود که هیچ caseی جور نباشد (رد گزینه‌ی ۳).

۲۶۸ **گزینه‌ی ۳** متوسط

`break` اجرا را از `switch` بیرون می‌برد. اجرا از `case 2` شروع می‌شود، `Tue` چاپ می‌شود و `break` جلوی سقوط به `case 3` را می‌گیرد. `continue` فقط داخل حلقه معنا دارد و این‌جا `SyntaxError` می‌دهد. `default` یک برچسب است و باید به شکل `default:` بیاید؛ `default;` خطای نحوی است. `stop` هم کلمه‌ی کلیدی نیست؛ JavaScript آن را نام یک متغیر تعریف‌نشده می‌بیند و بعد از چاپ `Tue` خطای `ReferenceError` می‌دهد.

---

۲۶۹ **گزینه‌ی ۴** سخت

`default` لازم نیست آخرین بخش باشد (رد گزینه‌ی ۲). مقدار `7` با هیچ caseی جور نیست، پس اجرا از `default` شروع می‌شود و `unknown` چاپ می‌شود. چون این `default` با `break` تمام نشده، اجرا به `case 1` سقوط می‌کند و `start` هم چاپ می‌شود؛ `break` آن‌جا اجرا را از `switch` بیرون می‌برد، پس `stop` چاپ نمی‌شود (رد گزینه‌ی ۳). گزینه‌ی ۱ فراموش کرده که fall-through برای `default` هم اتفاق می‌افتد.

---

۲۷۰ **گزینه‌ی ۳** متوسط

`default` می‌تواند هر جای `switch` بیاید و این خطا نیست؛ فقط اگر وسط باشد و `break` نداشته باشد، اجرا از آن به caseهای پایینی سقوط می‌کند. سه گزینه‌ی دیگر درست‌اند: مقایسه مثل `===` است (پس `"1"` با `1` جور نمی‌شود)، نبودن `break` باعث fall-through می‌شود و گروه‌بندی caseها (مثل `case "Thursday": case "Friday":`) کاملاً رایج است.

---

۲۷۱ **گزینه‌ی ۲** آسان

عملگر سه‌تایی (ternary) این‌طور خوانده می‌شود: «شرط `?` مقدارِ حالت درست `:` مقدارِ حالت نادرست». `0 > 0` نادرست است، پس مقدار بعد از `:` یعنی `"Your wallet is empty"` در `message` ریخته می‌شود. گزینه‌ی ۱ فقط وقتی درست بود که `balance` از صفر بزرگ‌تر بود، گزینه‌ی ۳ فکر کرده خودِ نتیجه‌ی شرط برگردانده می‌شود و گزینه‌ی ۴ این نحو (syntax) کاملاً مجاز را خطا دانسته است.

---

۲۷۲ **گزینه‌ی ۱** متوسط

زنجیره‌ی ternary مثل یک زنجیره‌ی `else if` خوانده می‌شود: «اگر `n > 0` پس `positive`، وگرنه اگر `n < 0` پس `negative`، وگرنه `zero`». `-3 > 0` نادرست است، پس سراغ ternary دوم می‌رویم و چون `-3 < 0` درست است، `negative` انتخاب می‌شود. زنجیر کردن ternaryها مجاز است (رد گزینه‌ی ۴)، هرچند زیاده‌روی در آن خوانایی را کم می‌کند. گزینه‌ی ۳ فقط برای `n = 0` درست بود.

---

۲۷۳ **گزینه‌ی ۴** متوسط

نحو درست ternary «شرط `?` مقدار درست `:` مقدار نادرست» است. `7 % 2 === 0` نادرست است، پس `"odd"` انتخاب می‌شود. در گزینه‌ی ۱ جای `?` و `:` عوض شده و `SyntaxError` می‌گیریم. گزینه‌ی ۳ هم `SyntaxError` می‌دهد، چون `if` یک **دستور** است و نمی‌تواند سمت راست `=` بنشیند؛ فقط **عبارت‌ها** (مثل ternary) مقدار تولید می‌کنند. گزینه‌ی ۲ بدون خطا اجرا می‌شود ولی `even` چاپ می‌کند: `7 % 2` برابر `1` و truthy است، پس مقدار قبل از `:` انتخاب می‌شود.

---

۲۷۴ **گزینه‌ی ۳** متوسط

بدون آکولاد، فقط **یک** دستورِ بعد از `if` (خط ۳) به آن تعلق دارد. تورفتگی خط ۴ در JavaScript هیچ معنایی ندارد (برخلاف Python)، پس خط ۴ بیرون از `if` است و همیشه اجرا می‌شود. چون `isAdmin` نادرست است، فقط خط ۳ رد می‌شود. گزینه‌ی ۱ تورفتگی را مثل Python معنادار فرض کرده، گزینه‌ی ۲ شرط را نادیده گرفته و گزینه‌ی ۴ غلط است، چون `if` بدون آکولاد مجاز است.

---

۲۷۵ **گزینه‌ی ۲** سخت

`if` بدون آکولاد فقط دستور خط ۳ را در بر می‌گیرد؛ خط ۴ (با وجود تورفتگی) یک دستور مستقل است و ساختار `if` قبل از آن تمام شده است. پس `else` در خط ۵ به هیچ `if` ی نمی‌چسبد و `SyntaxError: Unexpected token 'else'` رخ می‌دهد. چون این خطای نحوی است، قبل از اجرای هر خطی کشف می‌شود و هیچ چیزی (حتی `positive`) چاپ نمی‌شود. خط‌های ۲ و ۴ (گزینه‌های ۱ و ۳) به‌تنهایی کاملاً مجازند و گزینه‌ی ۴ («هیچ خطایی رخ نمی‌دهد») هم غلط است.

---

۲۷۶ **گزینه‌ی ۱** سخت

داخل شرط `=` آمده، نه `===`. پس `count = 0` اول صفر را **در** `count` **می‌ریزد** و مقدار کل عبارتِ انتساب هم `0` است که falsy است؛ بنابراین شاخه‌ی `else` اجرا می‌شود و بعد از آن `count` واقعاً `0` شده است. گزینه‌ی ۲ فکر کرده این یک مقایسه بوده و مقدار متغیر عوض نشده، گزینه‌ی ۳ فکر کرده «انتسابِ انجام‌شده» یعنی شرط درست، و گزینه‌ی ۴ غلط است، چون انتساب داخل شرط از نظر نحوی مجاز است (و همین آن را به یک باگ پنهان تبدیل می‌کند).

۲۷۷ **گزینه‌ی ۴** متوسط

در خط ۴ به جای مقایسه (`===`)، انتساب (`=`) نوشته شده و چون `limit` با `const` تعریف شده، انتساب دوباره به آن `TypeError: Assignment to constant variable.` می‌دهد. خط ۳ یک مقایسه‌ی معمولی است و `ok` را چاپ می‌کند. خط ۱ یک تعریف عادی است و خط ۵ هرگز اجرا نمی‌شود، چون برنامه روی خط ۴ متوقف شده است.

۲۷۸ **گزینه‌ی ۳** سخت

سمی‌کالنِ بلافاصله بعد از `if (score > 100)` یک **دستور خالی** است و همان، بدنه‌ی `if` حساب می‌شود؛ یعنی `if` همان‌جا تمام شده است. بلوک `{ ... }` بعدی یک بلوک مستقل و معمولی است که همیشه اجرا می‌شود؛ پس با وجود این‌که `40 > 100` نادرست است، `Score too high!` و بعد `Checked` چاپ می‌شوند. گزینه‌ی ۱ این سمی‌کالن را ندیده، گزینه‌ی ۲ غلط است چون این کد از نظر نحوی مجاز است و گزینه‌ی ۴ خط آخر را جا انداخته است.

۲۷۹ **گزینه‌ی ۴** سخت

عملگر `||` مقایسه را تکرار نمی‌کند؛ شرط یعنی `(color === "red") || "blue"`. بخش اول `false` است، پس `||` مقدار دوم یعنی رشته‌ی `"blue"` را برمی‌گرداند که truthy است؛ در نتیجه شرط **همیشه** درست است. گزینه‌ی ۱ همان نتیجه‌ای است که برنامه‌نویس می‌خواست و با شکل درست `color === "red" || color === "blue"` به دست می‌آید. گزینه‌ی ۲ غلط است چون این کد مجاز است و گزینه‌ی ۳ هم غلط است، چون از `if / else` فقط یک شاخه اجرا می‌شود.

۱-۱۳ حلقه‌ها (Loops)

۲۸۰ **گزینه‌ی ۳** آسان

طبق اسلاید ۳۲، `do...while` «اول یک بار اجرا می‌کند و بعد شرط را بررسی می‌کند» (run at least once, then check condition). در `for` و `while` شرط **قبل از** هر دور بررسی می‌شود، پس با شرطِ نادرست بدنه هیچ‌وقت اجرا نمی‌شود. `for...in` هم برای پیمایش کلیدهای یک شیء است و اگر شیء کلیدی نداشته باشد، بدنه‌اش اصلاً اجرا نمی‌شود.

۲۸۱ **گزینه‌ی ۲** آسان

اسلاید ۳۲: `for...in` حلقه روی **کلیدهای** یک شیء است و `for...of` حلقه روی **مقدارهای** (items) یک آرایه. `for...of` روی یک شیء معمولی حتی خطای `TypeError` می‌دهد، چون شیء معمولی iterable نیست. `while` و `do...while` حلقه‌های شرطی عمومی‌اند و به‌تنهایی کلیدها را به ما نمی‌دهند.

۲۸۲ **گزینه‌ی ۴** آسان

`while` «تا وقتی شرط درست است» تکرار می‌کند و شرط را **قبل از** هر دور می‌سنجد؛ پس ممکن است صفر بار اجرا شود. گزینه‌ی ۱ توصیف `do...while` است. گزینه‌ی ۲ بی‌پایه است: تعداد دورها به شرط بستگی دارد، نه یک عدد ثابت. گزینه‌ی ۳ هم غلط است؛ `while` یک حلقه‌ی شرطی عمومی است و برای پیمایش آرایه معمولاً `for...of` به کار می‌رود.

۲۸۳ **گزینه‌ی ۱** آسان

`i` از ۰ شروع می‌شود و تا وقتی `i < 3` است بدنه اجرا می‌شود؛ پس `i` برابر ۰، ۱ و ۲ است و سه خط چاپ می‌شود. گزینه‌ی ۲ شروع را از ۱ گرفته، گزینه‌ی ۳ شرط را `<=` فرض کرده (یک دور اضافه) و گزینه‌ی ۴ فکر کرده فقط مقدار پایانی چاپ می‌شود، در حالی که `console.log` در **هر** دور اجرا می‌شود.

۲۸۴ **گزینه‌ی ۲** آسان

`i` مقدارهای ۰ تا ۹ را می‌گیرد، یعنی ۱۰ دور، و در هر دور `count` یکی زیاد می‌شود؛ پس `10` چاپ می‌شود. گزینه‌ی ۱ فکر کرده چون آخرین مقدار `i` برابر ۹ است، ۹ دور داریم و گزینه‌ی ۳ شرط را `<=` فرض کرده است. `count` بیرون از حلقه تعریف شده و داخل حلقه هم در دسترس است، پس `ReferenceError` (گزینه‌ی ۴) رخ نمی‌دهد.

۲۸۵ **گزینه‌ی ۳** متوسط

شرط `i <= 4` است، پس خودِ ۴ هم شامل می‌شود: `1 + 2 + 3 + 4 = 10` . گزینه‌ی ۱ عدد ۴ را جا انداخته (انگار شرط `i < 4` بوده)، گزینه‌ی ۲ تا ۵ جمع زده و گزینه‌ی ۴ فکر کرده `+=` هر بار مقدار قبلی را پاک می‌کند و فقط آخرین `i` می‌ماند.

۲۸۶ **گزینه‌ی ۱** متوسط

`i` از ۱۰ شروع می‌شود و بعد از هر دور ۳ تا کم می‌شود: ۱۰، ۷، ۴ و ۱ همه از صفر بزرگ‌ترند و چاپ می‌شوند. مقدار بعدی `-2` است و شرط `i > 0` را نادرست می‌کند، پس چاپ نمی‌شود (رد گزینه‌ی ۳). گزینه‌ی ۴ فکر کرده update قبل از اولین دور اجرا می‌شود و گزینه‌ی ۲ عدد ۱ را جا انداخته، در حالی که `1 > 0` درست است.

۲۸۷ **گزینه‌ی ۴** سخت

در هر دور `i` **دو بار** زیاد می‌شود: یک بار با `i++` داخل بدنه و یک بار با update حلقه. ردیابی: `i = 0` چاپ و بعد ۲؛ `i = 2` چاپ و بعد ۴؛ `i = 4` چاپ و بعد ۶؛ و `6 < 5` نادرست است. پس `0` ، `2` و `4` چاپ می‌شوند. گزینه‌ی ۱ تغییر داخل بدنه را ندیده، گزینه‌ی ۲ فراموش کرده که ۶ شرط را نادرست می‌کند و گزینه‌ی ۳ فکر کرده `4 < 5` نادرست است.

۲۸۸ **گزینه‌ی ۲** سخت

`var` به بلوک حلقه اهمیت نمی‌دهد و بعد از حلقه هم وجود دارد؛ مقدارش همان مقداری است که شرط را نادرست کرد، یعنی `3` (نه آخرین مقدارِ داخل بدنه، `2` ؛ رد گزینه‌ی ۱). اما `j` که با `let` تعریف شده فقط داخل حلقه وجود دارد و خواندنش بعد از حلقه `ReferenceError` می‌دهد (رد گزینه‌ی ۳). این خطا در زمان اجرا و روی خط آخر رخ می‌دهد، پس `3` قبل از آن چاپ شده است (رد گزینه‌ی ۴).

۲۸۹ **گزینه‌ی ۱** آسان

قبل از هر دور شرط `n > 0` بررسی می‌شود: برای ۳، ۲ و ۱ درست است و همان مقدار چاپ می‌شود و بعد `n--` آن را یکی کم می‌کند. وقتی `n` صفر شد شرط نادرست می‌شود و حلقه تمام می‌شود، پس `0` چاپ نمی‌شود (رد گزینه‌ی ۲). گزینه‌ی ۳ فکر کرده کم شدن قبل از چاپ اتفاق می‌افتد و گزینه‌ی ۴ فکر کرده بدنه اصلاً اجرا نمی‌شود.

۲۹۰ **گزینه‌ی ۳** متوسط

`x` به ترتیب ۱، ۳، ۹، ۲۷، ۸۱ و ۲۴۳ می‌شود. وقتی `x` برابر ۸۱ است شرط `81 < 100` هنوز درست است، پس یک دور دیگر اجرا می‌شود و `x` برابر ۲۴۳ می‌شود؛ حالا شرط نادرست است و حلقه تمام می‌شود. گزینه‌ی ۱ آخرین مقدارِ کمتر از ۱۰۰ را گفته (اما حلقه بعد از سه برابر کردن تمام می‌شود)، گزینه‌ی ۲ فکر کرده حلقه دقیقاً روی حدِ شرط می‌ایستد و گزینه‌ی ۴ یک دور اضافه حساب کرده است.

۲۹۱ **گزینه‌ی ۴** متوسط

شرط هر دو حلقه از اول نادرست است (`5 < 5` نادرست است). `while` شرط را **قبل از** اجرای بدنه می‌سنجد، پس بدنه‌اش هیچ‌وقت اجرا نمی‌شود. `do...while` اول بدنه را اجرا می‌کند و **بعد** شرط را می‌سنجد، پس یک بار `do 5` چاپ می‌شود و بعد حلقه تمام می‌شود. گزینه‌ی ۲ این ویژگی `do...while` را ندیده و گزینه‌های ۱ و ۳ فرض کرده‌اند `while` هم حداقل یک بار اجرا می‌شود.

۲۹۲ **گزینه‌ی ۱** آسان

`for...of` مقدارهای (items) آرایه را یکی‌یکی و به ترتیب می‌دهد و در هر دور یک مقدار چاپ می‌شود. گزینه‌ی ۲ خروجیِ `for...in` است که indexها را می‌دهد. گزینه‌ی ۳ فکر کرده کل آرایه یک بار چاپ می‌شود. گزینه‌ی ۴ هم غلط است؛ تعریف متغیر حلقه با `const` مجاز است، چون در هر دور یک متغیر تازه ساخته می‌شود.

۲۹۳ **گزینه‌ی ۲** متوسط

`for...of` روی رشته هم کار می‌کند و حرف‌های آن را یکی‌یکی می‌دهد: `b` ، `a` ، `a` ، `n` ، `a` ، `n` ، `a` . سه حرف از این شش حرف `a` است، پس `count` برابر ۳ می‌شود. گزینه‌ی ۱ تعداد کل حرف‌ها را شمرده و شرط `if` را نادیده گرفته، گزینه‌ی ۳ فکر کرده شمارش بعد از اولین `a` متوقف می‌شود و گزینه‌ی ۴ رشته را iterable ندانسته است؛ اما `for...of` روی رشته کاملاً مجاز است و خطایی نمی‌دهد.

---

۲۹۴ **گزینه‌ی ۳** متوسط

`for...of` مقدارهای آرایه را می‌دهد، پس `a` ، `b` و `c` چاپ می‌شوند. گزینه‌ی ۱ (`for...in`) کلیدها یعنی indexهای `"0"` ، `"1"` و `"2"` را چاپ می‌کند. گزینه‌ی ۲ هم خودِ شمارنده (۰، ۱، ۲) را چاپ می‌کند؛ برای چاپ حرف‌ها باید `letters[item]` نوشته می‌شد. در گزینه‌ی ۴، `letters.length` عدد `3` است و عدد iterable نیست، پس `TypeError` رخ می‌دهد.

---

۲۹۵ **گزینه‌ی ۴** متوسط

`for...in` **کلیدهای** شیء را می‌دهد، نه مقدارها را؛ پس `brand` و `year` چاپ می‌شوند. برای رسیدن به مقدارها باید `car[key]` را چاپ کنیم (گزینه‌ی ۱). گزینه‌ی ۲ فکر کرده شیء مثل آرایه index عددی دارد و گزینه‌ی ۳ رفتار `for...of` را در نظر گرفته که روی شیء معمولی خطا می‌دهد؛ `for...in` روی شیء کاملاً مجاز است.

---

۲۹۶ **گزینه‌ی ۳** سخت

`for...in` روی آرایه کلیدها یعنی indexها را می‌دهد، آن هم به‌صورت **رشته**: `"0"` ، `"1"` و `"2"` . حاصل `"0" + 1` الحاق رشته‌ای است و `"01"` می‌شود (نه جمع عددی). گزینه‌ی ۱ indexها را عدد فرض کرده و گزینه‌های ۲ و ۴ فکر کرده‌اند `for...in` مقدارها (۱۰، ۲۰، ۳۰) را می‌دهد؛ یکی با جمع عددی و دیگری با الحاق رشته‌ای. این همان دلیلی است که برای آرایه‌ها `for...of` را ترجیح می‌دهیم.

---

۲۹۷ **گزینه‌ی ۲** سخت

`for...of` فقط روی چیزهای iterable (مثل آرایه و رشته) کار می‌کند. یک شیء معمولی iterable نیست، پس خط ۵ خطای `TypeError: student is not iterable` می‌دهد. خط ۲ (گزینه‌ی ۱) `for...of` روی آرایه است و درست کار می‌کند و خط ۴ (گزینه‌ی ۳) `for...in` روی شیء است که کاملاً مجاز است و کلیدهای `name` و `score` را چاپ می‌کند. این خطا در زمان اجرا رخ می‌دهد، پس خروجی خط‌های قبل از آن چاپ شده است.

---

۲۹۸ **گزینه‌ی ۱** متوسط

وقتی `i` به ۴ می‌رسد، `break` **قبل از** `console.log` اجرا می‌شود و کل حلقه را تمام می‌کند؛ پس ۴ چاپ نمی‌شود (رد گزینه‌ی ۲). `break` فقط از حلقه بیرون می‌رود، نه از کل برنامه؛ پس خط بعد از حلقه اجرا می‌شود و `done` چاپ می‌شود (رد گزینه‌ی ۳). گزینه‌ی ۴ رفتار `continue` است که فقط همان یک دور را رد می‌کند.

---

۲۹۹ **گزینه‌ی ۴** متوسط

`continue` فقط **بقیه‌ی همان دور** را رد می‌کند و حلقه با `i` بعدی ادامه پیدا می‌کند. برای `i` برابر ۳ و ۶ (مضرب‌های ۳) چاپ رد می‌شود و بقیه چاپ می‌شوند. گزینه‌ی ۱ رفتار `break` را فرض کرده که کل حلقه را تمام می‌کند. گزینه‌ی ۲ برعکس فکر کرده فقط همان دورهای شرط‌دار چاپ می‌شوند و گزینه‌ی ۳ اثر `continue` را ندیده است.

---

۳۰۰ **گزینه‌ی ۳** متوسط

در گزینه‌ی ۳ شمارنده به جای زیاد شدن **کم** می‌شود (`i--`): `i` برابر ۰، ۱-، ۲- و ... می‌شود و همیشه از ۳ کوچک‌تر می‌ماند، پس شرط هیچ‌وقت نادرست نمی‌شود. گزینه‌ی ۱ یک حلقه‌ی معمولی سه‌دوره است. در گزینه‌ی ۲، `i` از ۵ کم می‌شود تا به ۰ برسد و شرط `i > 0` نادرست شود. در گزینه‌ی ۴، `i` به ترتیب ۲، ۴، ۶ و ۸ می‌شود و با رسیدن به ۸ شرط `i < 7` نادرست می‌شود.

---

۳۰۱ **گزینه‌ی ۲** سخت

`0` و `1` چاپ می‌شوند. وقتی `k` برابر ۲ می‌شود، `continue` اجرا را مستقیم به بررسی شرط برمی‌گرداند و `k++` **هیچ‌وقت** اجرا نمی‌شود؛ پس `k` برای همیشه ۲ می‌ماند و شرط `k < 5` همیشه درست است. گزینه‌ی ۱ رفتار `continue` در حلقه‌ی `for` را فرض کرده که در آن بخش update (مثل `k++`) حتی بعد از `continue` هم اجرا می‌شود. گزینه‌ی ۳ اثر `continue` را ندیده و گزینه‌ی ۴ غلط است، چون `continue` در هر حلقه‌ای (از جمله `while`) مجاز است.

---

۳۰۲ **گزینه‌ی ۴** متوسط

به ازای **هر** دورِ حلقه‌ی بیرونی (۳ دور)، حلقه‌ی درونی **کامل** اجرا می‌شود (۴ دور)؛ پس `count++` در مجموع `3 × 4 = 12` بار اجرا می‌شود. گزینه‌ی ۱ تعداد دورها را جمع زده (۳ + ۴)، گزینه‌ی ۲ فقط حلقه‌ی درونی را یک بار حساب کرده و گزینه‌ی ۳ هر دو حلقه را ۴ دوره فرض کرده است.

۳۰۳ **گزینه‌ی ۱** سخت

`break` فقط **نزدیک‌ترین** حلقه، یعنی حلقه‌ی درونی، را تمام می‌کند. در هر دورِ بیرونی، حلقه‌ی درونی `j = 1` را چاپ می‌کند و با رسیدن به `j = 2` تمام می‌شود؛ بعد حلقه‌ی بیرونی دور بعدی‌اش را شروع می‌کند. گزینه‌ی ۲ فکر کرده `break` هر دو حلقه را تمام می‌کند، گزینه‌ی ۳ رفتار `continue` را فرض کرده و گزینه‌ی ۴ فکر کرده `j = 2` قبل از `break` چاپ می‌شود.

۳۰۴ **گزینه‌ی ۳** سخت

در `do...while` بدنه **اول** اجرا می‌شود و شرط **بعد از** هر دور بررسی می‌شود؛ برای همین حداقل یک بار اجرا تضمین شده است. پس گزینه‌ی ۳ نادرست است. سه جمله‌ی دیگر درست‌اند: رشته iterable است و `for...of` حرف‌به‌حرف آن را می‌دهد، `for...in` روی آرایه indexهای رشته‌ای می‌دهد و `break` حلقه‌ای را که داخلش هست کامل تمام می‌کند.

۱-۱۴ Block، Scope و Global scope

۳۰۵ **گزینه‌ی ۲** آسان

اسلاید ۲۵: «A block is code inside { ... }». بدنه‌ی `if` و `else`، حلقه‌ها، توابع و حتی یک جفت آکولادِ تنها همه بلوک‌اند (رد گزینه‌ی ۴). `;` پایان یک دستور است، نه مرز یک بلوک (گزینه‌ی ۱) و پرانتز برای شرط‌ها، فراخوانی تابع و گروه‌بندی عبارت‌ها به کار می‌رود (گزینه‌ی ۳).

۳۰۶ **گزینه‌ی ۳** آسان

داخل بلوک `x` وجود دارد و `10` چاپ می‌شود. بعد از بسته شدن آکولاد، `x` (که با `let` تعریف شده) دیگر وجود ندارد و خط آخر `ReferenceError: x is not defined` می‌دهد. این خطا در زمان اجرا رخ می‌دهد، پس `10` قبل از آن چاپ شده است (رد گزینه‌ی ۲). گزینه‌ی ۱ رفتار `var` را در نظر گرفته که از بلوک بیرون می‌زند و گزینه‌ی ۴ فکر کرده متغیرِ ناموجود `undefined` می‌دهد؛ اما خواندن نامی که اصلاً پیدا نمی‌شود خطاست.

۳۰۷ **گزینه‌ی ۴** آسان

متن اسلاید ۲۵: «blocks aren't just syntax, they're a tool to organize and protect variables». چون `let` و `const` داخل بلوک فقط همان‌جا وجود دارند، بلوک متغیرهای موقت را در جای خودشان نگه می‌دارد (سازمان‌دهی) و جلوی دسترسی یا تغییر ناخواسته از بیرون و تداخل نام‌ها را می‌گیرد (محافظت). گزینه‌ی ۱ دقیقاً برعکس این جمله است، گزینه‌ی ۲ ادعایی بی‌پایه درباره‌ی سرعت است و گزینه‌ی ۳ غلط است، چون بلوک‌ها همه‌جا هستند (حتی یک جفت آکولادِ تنها).

۳۰۸ **گزینه‌ی ۴** آسان

`tax` داخل تابع تعریف شده و فقط داخل همان تابع وجود دارد (function scope). خط ۳ داخل تابع است و بی‌مشکل `Tax: 9` را چاپ می‌کند و فراخوانی تابع در خط ۵ هم خطایی ندارد. بعد از چاپ `Done`، خط ۷ از بیرونِ تابع سراغ `tax` می‌رود و `ReferenceError: tax is not defined` می‌گیرد؛ پس گزینه‌ی ۳ («هیچ خطایی رخ نمی‌دهد») هم غلط است.

۳۰۹ **گزینه‌ی ۱** متوسط

`token` فقط داخل بلوک وجود دارد. `typeof` روی نامی که پیدا نمی‌شود خطا نمی‌دهد و رشته‌ی `"undefined"` را برمی‌گرداند؛ ولی خواندن مستقیم `token` در خط آخر `ReferenceError` می‌دهد. گزینه‌ی ۲ فرض کرده `const` از بلوک بیرون می‌زند، گزینه‌ی ۳ فکر کرده `typeof` هم خطا می‌دهد و گزینه‌ی ۴ فکر کرده خواندن مستقیم یک متغیرِ ناموجود هم `undefined` می‌دهد.

۳۱۰ **گزینه‌ی ۱** آسان

`message` بیرون از هر بلوکی تعریف شده، پس سراسری (global) است و «از هر جای کد» در دسترس است: هم داخل تابع `greet` و هم در سطح بالای برنامه. گزینه‌های ۲ و ۳ فکر کرده‌اند تابع به متغیرهای بیرون از خودش دسترسی ندارد و گزینه‌ی ۴ فکر کرده بعد از اجرای تابع، متغیر از بین می‌رود؛ در حالی که متغیر سراسری تا پایان برنامه زنده است.

۳۱۱ **گزینه‌ی ۲** آسان

اسلاید ۲۶ سه ویژگی می‌گوید: متغیری که **بیرون** از هر بلوکی تعریف شود سراسری است، **از هر جای کد** قابل دسترسی است و **تا وقتی برنامه اجرا می‌شود** زنده می‌ماند. گزینه‌ی ۱ برعکسِ «از هر جا» است و گزینه‌ی ۳ با «زنده ماندن تا پایان برنامه» تضاد دارد. گزینه‌ی ۴ هم غلط است: در خودِ مثال اسلاید ۲۶، `let message` سراسری است.

۳۱۲ **گزینه‌ی ۴** آسان

متن اسلاید ۲۶: «Too many globals can cause naming conflicts and bugs. They make code harder to maintain and debug.» همه‌ی اسکریپت‌ها و توابع به globalها دسترسی دارند، پس نام‌ها ممکن است با هم تداخل کنند و هر جای برنامه ممکن است مقدارشان را عوض کند. محدودیت عددی (گزینه‌ی ۱) وجود ندارد، globalها داخل توابع قابل خواندن‌اند (رد گزینه‌ی ۲) و در هر دو محیط مرورگر و Node.js کار می‌کنند (رد گزینه‌ی ۳).

۳۱۳ **گزینه‌ی ۳** متوسط

`score` سراسری است و تابع `addPoint` نه‌تنها آن را می‌خواند بلکه **تغییر** هم می‌دهد (چون داخل تابع هیچ `let score` جدیدی تعریف نشده). دو بار فراخوانی، `score` را از ۰ به ۲ می‌رساند. گزینه‌ی ۱ فکر کرده تغییرِ داخل تابع روی متغیر بیرونی اثری ندارد، گزینه‌ی ۲ فقط یکی از دو فراخوانی را حساب کرده و گزینه‌ی ۴ فکر کرده تابع به متغیر سراسری دسترسی ندارد. همین که «هر تابعی می‌تواند متغیر سراسری را عوض کند» یکی از دلیل‌های سخت شدن دیباگ با globalهای زیاد است.

۳۱۴ **گزینه‌ی ۱** متوسط

`inner` اول `c` را در محدوده‌ی خودش پیدا می‌کند، `b` را در محدوده‌ی بیرونی‌اش (`outer`) و `a` را در global scope. این جست‌وجوی رو به بیرون همان scope chain است. گزینه‌های ۲ و ۴ فکر کرده‌اند تابع درونی فقط متغیرهای خودش را می‌بیند (یکی با نادیده گرفتن بقیه و دیگری با `undefined`) و گزینه‌ی ۳ فکر کرده دسترسی به متغیرهای بیرونی خطا می‌دهد.

۳۱۵ **گزینه‌ی ۲** متوسط

`inner` به `x` از محدوده‌ی بیرونی دسترسی دارد، پس خط ۵ عدد `3` را چاپ می‌کند (رد گزینه‌ی ۱). اما جست‌وجو فقط رو به **بیرون** است و `outer` نمی‌تواند داخل `inner` را ببیند؛ پس `y` در خط ۸ پیدا نمی‌شود و `ReferenceError: y is not defined` رخ می‌دهد. خط ۱۰ فقط تابع را صدا می‌زند و دستورِ خطادار، خط ۸ داخل تابع است.

۳۱۶ **گزینه‌ی ۴** متوسط

`inner` متغیری به نام `level` ندارد، پس جست‌وجو یک قدم بیرون می‌رود، در `outer` به `level = "outer"` می‌رسد و همان‌جا **متوقف** می‌شود؛ به global نمی‌رسد (رد گزینه‌ی ۱). `level` داخل `outer` یک متغیر جدید در محدوده‌ای دیگر است که روی متغیر سراسری سایه انداخته (shadowing)، نه تعریف دوباره در همان محدوده؛ پس `SyntaxError` رخ نمی‌دهد (رد گزینه‌ی ۳). متغیر سراسری هم دست‌نخورده می‌ماند و خط آخر `global` را چاپ می‌کند (رد گزینه‌ی ۲).

۳۱۷ **گزینه‌ی ۳** متوسط

`reset` داخل خودش یک `let total` **جدید** تعریف می‌کند که فقط در همان تابع زندگی می‌کند (shadowing)؛ پس `total` سراسری دست‌نخورده می‌ماند و `10` چاپ می‌شود. `clear` اما `let` ندارد و با `total = 0` همان متغیر سراسری را تغییر می‌دهد؛ پس بعد از آن `0` چاپ می‌شود. گزینه‌ی ۱ فکر کرده `let total = 0` هم متغیر سراسری را عوض می‌کند، گزینه‌ی ۲ فکر کرده هیچ تابعی نمی‌تواند متغیر سراسری را تغییر دهد و گزینه‌ی ۴ نقش دو تابع را جابه‌جا گرفته است.

۳۱۸ **گزینه‌ی ۲** سخت

`x = 3` (بدون `let`) متغیر جدیدی نمی‌سازد؛ JavaScript **نزدیک‌ترین** `x` را در scope chain پیدا می‌کند، یعنی `x`ِ بلوکِ میانی (که ۲ بود)، و همان را ۳ می‌کند. پس داخلی‌ترین بلوک و بلوک میانی هر دو `3` چاپ می‌کنند. `x`ِ بیرونی (سطح بالا) که `let x = 2` رویش سایه انداخته بود دست‌نخورده می‌ماند و `1` چاپ می‌شود. گزینه‌ی ۱ فکر کرده `x = 3` در داخلی‌ترین بلوک یک متغیر محلیِ تازه ساخته، گزینه‌ی ۳ فکر کرده `let x = 2` متغیر جدیدی نمی‌سازد و در کل کد فقط یک `x` داریم، و گزینه‌ی ۴ فکر کرده انتسابِ بدون `let` همیشه به متغیرِ سطح بالا می‌رسد و `x`ِ نزدیک‌تر را نادیده می‌گیرد.

۳۱۹ **گزینه‌ی ۴** متوسط

برای این‌که بیرون از بلوک هم `inner` چاپ شود، باید **همان** متغیر بیرونی را تغییر دهیم؛ یعنی یک انتساب ساده بدون کلمه‌ی کلیدی. `let` و `const` (گزینه‌های ۱ و ۲) یک متغیر **جدید** در بلوک می‌سازند که روی بیرونی سایه می‌اندازد؛ پس بیرون از بلوک همان `outer` چاپ می‌شود. `var` (گزینه‌ی ۳) به بلوک اهمیت نمی‌دهد و در عمل در همان محدوده‌ی `let label` تعریف می‌شود؛ این تعریف دوباره است و `SyntaxError: Identifier 'label' has already been declared` می‌دهد.

۳۲۰ **گزینه‌ی ۱** متوسط

پارامتر `price` یک متغیر محلیِ تابع است که روی `price` سراسری سایه می‌اندازد. داخل تابع، `price` همان ۵۰ ورودی است و `50 - 20 = 30` برگردانده می‌شود. `price` سراسری اصلاً دیده یا تغییر داده نمی‌شود و `100` می‌ماند. گزینه‌ی ۲ فکر کرده تغییر پارامتر به متغیر سراسریِ هم‌نام می‌رسد و گزینه‌های ۳ و ۴ فکر کرده‌اند داخل تابع، `price` سراسری (۱۰۰) خوانده می‌شود.

۳۲۱ **گزینه‌ی ۳** سخت

`var i` function-scoped است و بعد از حلقه هم در کل تابع `report` دیده می‌شود؛ پس خط ۵ عدد `2` را چاپ می‌کند (رد گزینه‌ی ۱). `doubled` با `let` داخل بلوک حلقه تعریف شده و بیرون از آن وجود ندارد؛ پس خط ۶ `ReferenceError: doubled is not defined` می‌دهد. خط ۹ هم اگر اجرا می‌شد خطا می‌داد (چون `var i` فقط داخل `report` است، نه سراسری)، ولی برنامه قبل از رسیدن به آن، روی خط ۶ متوقف شده است.

۳۲۲ **گزینه‌ی ۲** سخت

`message` سراسری است و داخل `greet` هم دیده می‌شود، پس `Hello` چاپ می‌شود. اما جمله‌ی اسلاید ۲۶ («global‌ها property شیء `window` می‌شوند») فقط برای `var` و اعلان تابع درست است: `let` و `const` سراسری‌اند ولی روی `window` نمی‌نشینند. پس `window.message` برابر `undefined` و `window.counter` برابر `1` است. گزینه‌ی ۱ جمله‌ی اسلاید را برای همه‌ی متغیرها تعمیم داده، گزینه‌ی ۳ فکر کرده هیچ متغیری روی `window` نمی‌نشیند و گزینه‌ی ۴ نقش `let` و `var` را برعکس گرفته است.

۳۲۳ **گزینه‌ی ۴** آسان

`window` شیء سراسریِ **مرورگر** است و در Node.js وجود ندارد. `typeof` روی نامی که پیدا نمی‌شود خطا نمی‌دهد و `"undefined"` برمی‌گرداند. در مرورگر جواب `"object"` بود (گزینه‌ی ۱). شیء سراسری Node.js با نام `globalThis` (یا `global`) در دسترس است، ولی `typeof` هیچ‌وقت نام یک متغیر را برنمی‌گرداند (رد گزینه‌ی ۲) و `"function"` (گزینه‌ی ۳) هم مخصوص توابع است.

۳۲۴ **گزینه‌ی ۱** سخت

همه‌ی اسکریپت‌های معمولیِ یک صفحه **یک global scope مشترک** دارند. `var count` و `function init` در `cart.js` بی‌صدا (و بدون هیچ خطایی) همان نام‌های `menu.js` را بازنویسی می‌کنند؛ پس `init()` نسخه‌ی `cart` را اجرا می‌کند و `count` برابر ۵ است. این دقیقاً همان «تداخل نام‌ها» است که اسلاید ۲۶ درباره‌اش هشدار می‌دهد. گزینه‌ی ۲ فکر کرده اولین تعریف می‌ماند، گزینه‌ی ۳ فکر کرده هر دو تابع اجرا می‌شوند و گزینه‌ی ۴ فقط تابع را بازنویسی‌شده دانسته است.

۳۲۵ **گزینه‌ی ۳** سخت

`let` سطح بالا هم سراسری است، پس اسکریپت دوم می‌خواهد `count` را در **همان** global scope دوباره تعریف کند. نتیجه `SyntaxError: Identifier 'count' has already been declared` است و این خطا **قبل از اجرای اولین خطِ** اسکریپت دوم رخ می‌دهد؛ پس هیچ خطی از آن (حتی `cart loading`) اجرا نمی‌شود (رد گزینه‌ی ۲). اسکریپت‌ها مستقل‌اند و اسکریپت سوم اجرا می‌شود و `count` اسکریپت اول یعنی `0` را چاپ می‌کند. گزینه‌های ۱ و ۴ فکر کرده‌اند `let` هم مثل `var` تعریف دوباره را بی‌صدا قبول می‌کند.

۳۲۶ **گزینه‌ی ۲** متوسط

`counter` هیچ‌جا با `let`، `const` یا `var` تعریف نشده؛ در حالت عادی (sloppy) انتساب به چنین نامی بی‌صدا یک متغیر **سراسری** می‌سازد، حتی از داخل تابع. پس بیرون از تابع `5` چاپ می‌شود. اما `local` با `let` تعریف شده و فقط داخل تابع وجود دارد؛ `typeof` روی آن بیرون از تابع `"undefined"` می‌دهد (بدون خطا). گزینه‌ی ۱ فکر کرده `local` هم بیرون می‌آید، گزینه‌ی ۳ رفتار حالت strict را در نظر گرفته و گزینه‌ی ۴ فکر کرده متغیری که داخل تابع ساخته شده بیرون دیده نمی‌شود.

۳۲۷ گزینه‌ی ۱ سخت

در حالت strict، انتساب به نامی که تعریف نشده دیگر متغیر سراسری نمی‌سازد و همان لحظه `ReferenceError: total is not defined` می‌دهد؛ دستورِ خطادار خط ۳ (داخل تابع) است. خط ۵ فقط تابع را صدا می‌زند (رد گزینه‌ی ۳) و خط ۶ هرگز اجرا نمی‌شود، چون برنامه روی خط ۳ متوقف شده است (رد گزینه‌ی ۲). گزینه‌ی ۴ («هیچ خطایی رخ نمی‌دهد») رفتار حالت عادی (sloppy) است که بی‌صدا یک global می‌سازد.

۳۲۸ گزینه‌ی ۳ متوسط

جست‌وجوی نام‌ها (scope chain) فقط از داخل به بیرون است و محدوده‌ی بیرونی هیچ‌وقت متغیرهای داخل یک تابع را نمی‌بیند. صدا زدن تابع هم چیزی را عوض نمی‌کند؛ متغیرهای محلیِ هر فراخوانی فقط برای کدِ داخل همان تابع قابل دسترسی‌اند. پس گزینه‌ی ۳ نادرست است. گزینه‌ی ۱ همان دسترسی رو به بیرون است، گزینه‌ی ۲ تعریف block scope است و گزینه‌ی ۴ همان shadowing است که تعریف دوباره حساب نمی‌شود.

۱-۱۵ توابع: پارامتر، مقدار بازگشتی و توابع درجه‌ی یک

۳۲۹ گزینه‌ی ۴ آسان

اسلاید ۳۳ می‌گوید تابع «Defined once, used many times» است: **یک بار** تعریف می‌شود و بارها صدا زده می‌شود؛ پس گزینه‌ی ۴ نادرست است. گزینه‌های ۱، ۲ و ۳ دقیقاً جمله‌های همان اسلاید هستند (reusable، پارامتر و مقدار بازگشتی، جلوگیری از تکرار و organized بودن کد).

۳۳۰ گزینه‌ی ۳ آسان

در Python تابع با `def` تعریف می‌شود. بقیه‌ی جفت‌ها جابه‌جا شده‌اند: Go از `func`، Kotlin از `fun` و Rust از `fn` استفاده می‌کند. JavaScript هم کلمه‌ی کامل `function` را دارد که در میم «ساده‌ترین» پو را گرفته است.

۳۳۱ گزینه‌ی ۲ آسان

در JavaScript تابع با کلمه‌ی کلیدی `function` تعریف می‌شود. `def` (Python)، `func` (Go) و `fn` (Rust) کلمه‌های زبان‌های دیگرند و JavaScript آن‌ها را نمی‌شناسد؛ هر سه خطای `SyntaxError` می‌دهند و حتی یک خط از برنامه اجرا نمی‌شود.

۳۳۲ گزینه‌ی ۲ آسان

پارامتر نامی است که در **تعریف** تابع می‌آید (`a` و `b`) و **آرگومان** مقدار واقعی است که هنگام **صدا زدن** فرستاده می‌شود (`2` و `3`). گزینه‌ی ۱ دقیقاً برعکس است. مقدار بازگشتی (اینجا `5`) نه پارامتر است نه آرگومان (رد گزینه‌ی ۳) و `add` نام تابع است، نه پارامتر (رد گزینه‌ی ۴).

۳۳۳ گزینه‌ی ۱ آسان

تابع یک بار تعریف شده و دو بار با آرگومان‌های مختلف صدا زده می‌شود؛ هر فراخوانی پارامتر `name` را با آرگومان خودش پر می‌کند. گزینه‌ی ۲ فکر کرده `name` متن ثابت است، در حالی که بیرون از کوتیشن یک متغیر است. گزینه‌ی ۳ تصور کرده تابع اولین ورودی را «یادش می‌ماند» و گزینه‌ی ۴ فکر کرده یک تابع را فقط یک بار می‌شود صدا زد.

۳۳۴ گزینه‌ی ۴ آسان

`x * x` حساب می‌شود ولی چون `return` ندارد، نتیجه دور ریخته می‌شود. تابعی که `return` ندارد مقدار `undefined` برمی‌گرداند. گزینه‌ی ۱ فرض کرده آخرین عبارت خودکار برگردانده می‌شود (چنین چیزی در تابع معمولی وجود ندارد). کد از نظر نحوی کاملاً درست است (رد گزینه‌ی ۳) و محاسبه‌ای روی `undefined` هم انجام نشده که `NaN` بسازد (رد گزینه‌ی ۲).

۳۳۵ گزینه‌ی ۳ آسان

نام تابع بدون پرانتز یعنی **خودِ تابع** به‌عنوان یک مقدار (مثلاً برای ذخیره یا پاس دادن به‌عنوان callback) و با پرانتز یعنی **صدا زدن** آن؛ پس `hello()` برابر `"hi"` است. پرانتز اختیاری نیست (رد گزینه‌ی ۱)، نوشتن نام تابع چیزی اجرا یا کپی نمی‌کند (رد گزینه‌ی ۲) و تعریف تابع با کلمه‌ی `function` انجام می‌شود، نه با پرانتز (رد گزینه‌ی ۴).

۳۳۶ **گزینه‌ی ۱** آسان

تابعی که با توابع دیگر مثل داده رفتار می‌کند (تابع می‌گیرد یا برمی‌گرداند) **higher-order function** (تابع مرتبه‌بالا) است. گزینه‌ی ۲ وسوسه‌انگیز است، ولی callback تابعی است که **پاس داده می‌شود**، نه تابعی که آن را می‌گیرد. method تابعی است که داخل یک object قرار دارد و default parameter مقدار پیش‌فرض یک پارامتر است.

۳۳۷ **گزینه‌ی ۳** متوسط

برچسب‌های میم: `public` = modifier، `int` = اول parameter-list = `(int x, int y)`، `max` = method-name، و return-type. `{ ... }` = بدنه. JavaScript نه modifier دارد و نه نوع؛ معادلش `function max(x, y) { ... }` است. گزینه‌ی ۱ modifier را با return-type قاطی کرده، گزینه‌ی ۲ نام متد را modifier دانسته و گزینه‌ی ۴ در JavaScript خطای `SyntaxError` می‌دهد.

۳۳۸ **گزینه‌ی ۲** متوسط

`return` تابع را **همان لحظه** تمام می‌کند؛ پس برای `5` مقدار `"positive"` برمی‌گردد و خط `console.log("after return")` هرگز اجرا نمی‌شود (رد گزینه‌ی ۱). برای `-2` شرط برقرار نیست و `"not positive"` برمی‌گردد. گزینه‌ی ۳ فکر کرده اجرا بعد از اولین `return` ادامه پیدا می‌کند. گزینه‌ی ۴ از Java آمده که کد غیرقابل‌دسترس را خطا می‌داند؛ JavaScript چنین خطایی ندارد.

۳۳۹ **گزینه‌ی ۴** متوسط

`double(5)` در خط ۴ اجرا می‌شود و `10` را **چاپ** می‌کند، ولی چیزی `return` نمی‌کند؛ پس `result` برابر `undefined` است و خط ۵ آن را چاپ می‌کند. گزینه‌ی ۱ چاپ کردن را با برگرداندن یکی گرفته است. گزینه‌ی ۲ خط آخر را فراموش کرده و گزینه‌ی ۳ ترتیب را برعکس فرض کرده است.

۳۴۰ **گزینه‌ی ۴** متوسط

فقط `return` حاصل را به بیرون تحویل می‌دهد تا در `result` ذخیره و یک بار چاپ شود. با گزینه‌ی ۱، `12` داخل تابع چاپ می‌شود ولی `result` برابر `undefined` است؛ پس خروجی دو خط `12` و `undefined` است. در گزینه‌ی ۲، `result` داخل تابع یک متغیر محلی جداست و به `result` بیرونی ربطی ندارد؛ خروجی `undefined` است. گزینه‌ی ۳ هم حاصل را دور می‌ریزد و `undefined` چاپ می‌کند.

۳۴۱ **گزینه‌ی ۳** متوسط

در `add(2)` پارامتر `b` آرگومانی نگرفته و `undefined` است؛ `2 + undefined` جمع عددی است و `NaN` می‌شود (الحاق رشته‌ای فقط وقتی رخ می‌دهد که یک طرف رشته باشد؛ رد گزینه‌ی ۱). در `add(2, 3, 4)` آرگومان اضافه‌ی `4` بی‌صدا نادیده گرفته می‌شود و حاصل `5` است (رد گزینه‌ی ۲). JavaScript برای تعداد نادرست آرگومان‌ها خطا نمی‌دهد (رد گزینه‌ی ۴).

۳۴۲ **گزینه‌ی ۲** متوسط

پارامتر جاافتاده بی‌صدا `undefined` می‌شود؛ پس گزینه‌ی ۲ درست است. JavaScript نه برای آرگومان کمتر خطا می‌دهد (رد گزینه‌ی ۱) و نه برای آرگومان اضافه که فقط نادیده گرفته می‌شود (رد گزینه‌ی ۳). تابع بدون پارامتر هم کاملاً مجاز است، مثل `function hello() { ... }` (رد گزینه‌ی ۴).

۳۴۳ **گزینه‌ی ۱** متوسط

مقدار پیش‌فرض وقتی به کار می‌رود که آرگومان آن جایگاه `undefined` باشد؛ چه آرگومانی نفرستیم (خط ۴) و چه خودمان صریحاً `undefined` بفرستیم (خط ۶). گزینه‌ی ۲ فکر کرده `undefined` صریح جلوی مقدار پیش‌فرض را می‌گیرد. گزینه‌ی ۳ مقدار پیش‌فرض را کلاً نادیده گرفته و گزینه‌ی ۴ فکر کرده پیش‌فرض حتی آرگومان واقعی `"Reza"` را هم کنار می‌زند.

۳۴۴ **گزینه‌ی ۲** متوسط

توابع first-class هستند و می‌شود آن‌ها را در یک object ذخیره کرد؛ `ops.minus` همان `sub` است و `10 - 4` برابر `6`. ذخیره کردن تابع کپی نمی‌سازد؛ `ops.plus` و `add` **همان یک** تابع‌اند، پس `===` برابر `true` است (رد گزینه‌ی ۳). گزینه‌ی ۱ تابع‌ها را جابه‌جا گرفته و گزینه‌ی ۴ فکر کرده تابعِ ذخیره‌شده در object را نمی‌شود صدا زد.

۳۴۵ **گزینه‌ی ۱** متوسط

طبق اسلاید ۱۷ تابع می‌تواند متد یک object باشد یا مستقل استفاده شود. `calculator.add` یک متد است و `plus` همان تابع را در یک متغیر جدا نگه می‌دارد؛ چون بدنه‌ی تابع به object وابسته نیست (از `this` استفاده نمی‌کند)، `plus(10, 20)` هم درست `30` می‌دهد (رد گزینه‌های ۲ و ۴). تابع نوعی object است ولی `typeof` برایش `"function"` برمی‌گرداند (رد گزینه‌ی ۳).

۳۴۶ **گزینه‌ی ۳** سخت

تابع بی‌نام به‌عنوان callback به پارامتر `onDone` پاس داده شده است. callback دقیقاً همان لحظه‌ای اجرا می‌شود که `processOrder` آن را با `onDone(item)` صدا می‌زند، یعنی بین دو `console.log`. گزینه‌ی ۱ تصور کرده callback همیشه «آخر کار» اجرا می‌شود. گزینه‌ی ۲ فکر کرده تابع هنگام پاس داده شدن اجرا می‌شود و گزینه‌ی ۴ فراموش کرده که callback واقعاً صدا زده می‌شود.

۳۴۷ **گزینه‌ی ۴** سخت

مقدار پیش‌فرض **فقط** جای `undefined` را می‌گیرد. پس `price(100)` و `price(100, undefined)` هر دو از `tax = 10` استفاده می‌کنند و `110` می‌دهند (رد گزینه‌ی ۳). `0` یک مقدار واقعی است و حاصل `100` است (رد گزینه‌ی ۱). `null` هم مقدار واقعی است و پیش‌فرض را کنار می‌زند؛ در جمع عددی `null` به `0` تبدیل می‌شود و `100 + null` برابر `100` است (رد گزینه‌ی ۲).

۳۴۸ **گزینه‌ی ۱** سخت

`makeMultiplier(3)` یک **تابع** برمی‌گرداند که `k = 3` را به خاطر دارد؛ پس `triple(5)` برابر `15` است. در `makeMultiplier(2)(7)` پرانتز اول تابع را می‌سازد و پرانتز دوم بلافاصله همان تابع برگشتی را با `7` صدا می‌زند: `14`. دو پرانتز پشت سر هم نحو کاملاً درستی است (رد گزینه‌های ۳ و ۴). گزینه‌ی ۲ فکر کرده `k` بعد از تمام شدن `makeMultiplier` از بین می‌رود، در حالی که تابع برگشتی آن را نگه می‌دارد (closure).

۳۴۹ **گزینه‌ی ۳** سخت

در خط ۴ تابع **صدا زده شده** و `userName` مقدار برگشتی یعنی رشته‌ی `"Mina"` را دارد، نه خودِ تابع را. خط ۵ همین رشته را چاپ می‌کند. در خط ۶ می‌خواهیم یک رشته را مثل تابع صدا بزنیم و خطای `TypeError: userName is not a function` می‌گیریم. گزینه‌ی ۱ فکر کرده صدا زدن تابع هنگام ریختن در `const` مشکل دارد و گزینه‌ی ۴ `userName` را با خودِ تابع `getName` اشتباه گرفته است.

۳۵۰ **گزینه‌ی ۴** سخت

`run(hello)` خودِ تابع را می‌فرستد: اول `running...`، بعد `run` آن را صدا می‌زند (`hello`) و `"done"` برمی‌گردد و چاپ می‌شود. اما در `run(hello())` آرگومان **قبل از** ورود به `run` حساب می‌شود: `hello()` همین الان اجرا می‌شود (`hello`) و رشته‌ی `"done"` را می‌دهد. سپس `run` چاپ می‌کند `running...` و می‌خواهد رشته‌ی `"done"` را صدا بزند: `TypeError`. گزینه‌ی ۱ فرقی بین `hello` و `hello()` قائل نشده، گزینه‌ی ۲ ترتیب ارزیابی آرگومان را اشتباه گرفته و گزینه‌ی ۳ فکر کرده رشته را می‌شود مثل تابع صدا زد.

۳۵۱ **گزینه‌ی ۲** سخت

`const a = hello` فقط **خودِ تابع** را در `a` می‌ریزد و آن را اجرا نمی‌کند. `hello()` یک بار تابع را اجرا می‌کند و `a()` هم همان تابع را (با نام دیگر) یک بار دیگر اجرا می‌کند؛ پس `calls` برابر `2` است. گزینه‌ی ۳ فکر کرده `const a = hello` هم تابع را صدا می‌زند. گزینه‌ی ۱ تصور کرده `a` یک کپی جداست که شمارنده‌ی اصلی را عوض نمی‌کند و گزینه‌ی ۴ اثر فراخوانی‌ها روی متغیر بیرونی را ندیده است.

۳۵۲ **گزینه‌ی ۱** سخت

وقتی بلافاصله بعد از `return` به خط بعد می‌رویم، JavaScript همان‌جا یک `;` خودکار می‌گذارد (automatic semicolon insertion)؛ پس کد مثل `return;` خوانده می‌شود و تابع `undefined` برمی‌گرداند. خط `100;` هرگز اجرا نمی‌شود (رد گزینه‌ی ۲). این کد از نظر نحوی مجاز است و خطا نمی‌دهد (رد گزینه‌ی ۳) و `return;` خالی همیشه `undefined` می‌دهد، نه `null` (رد گزینه‌ی ۴).

۱-۱۶ Function declaration در برابر Arrow function

۳۵۳ **گزینه‌ی ۲** آسان

زیرنویس اسلاید ۳۴ می‌گوید «Remember, this can be hoisted!» و زیرنویس اسلاید ۳۵ می‌گوید «Remember, this cannot be hoisted!»؛ اسلاید ۳۶ هم همین را جمع‌بندی می‌کند: functionها hoist می‌شوند و arrowها نه. گزینه‌ی ۳ دقیقاً برعکس است و گزینه‌های ۱ و ۴ تفاوت دو روش را نادیده گرفته‌اند.

۳۵۴ **گزینه‌ی ۴** آسان

hoist function declaration می‌شود: موتور قبل از اجرای خط ۱ تابع `add` را کامل (با بدنه‌اش) آماده کرده است؛ پس صدا زدن آن قبل از خط تعریف، `5` را چاپ می‌کند. `ReferenceError` (گزینه‌ی ۲) مخصوص متغیرهای `let` / `const` قبل از تعریف است و `TypeError` (گزینه‌ی ۳) وقتی رخ می‌دهد که یک `var` با مقدار `undefined` را مثل تابع صدا بزنیم. گزینه‌ی ۱ هم فکر کرده تابع هنوز بدنه ندارد.

۳۵۵ **گزینه‌ی ۳** آسان

اسلاید ۳۵: «ES6 (modern JS) Introduced in ES6». همان ES2015 است؛ بزرگ‌ترین به‌روزرسانی زبان که `let` ، `const` و arrow function را آورد. ES1 اولین استاندارد بود، ES5 هنوز arrow function نداشت و ES2017 بعدها `async` / `await` را اضافه کرد.

۳۵۶ **گزینه‌ی ۱** آسان

پنل اول (پوی ساده) `function foo() {} foo();` است، پنل دوم (پوی کت‌وشلواری) `const foo = () => {}; foo();` و پنل سوم (شیک‌ترین پو) `(() => {})();` یعنی IIFE. بقیه‌ی گزینه‌ها ترتیب پنل‌ها را به هم ریخته‌اند.

۳۵۷ **گزینه‌ی ۴** آسان

اسلاید ۳۶: «arrow function. arrows **inherit** `this`; Functions have their own `this` هیچ `this` مخصوص خودش ندارد و همان `this` محیطی را می‌بیند که در آن نوشته شده است. هیچ‌کدام از گزینه‌های ۱ تا ۳ درست نیست؛ به‌خصوص arrow function اصلاً با `new` کار نمی‌کند.

۳۵۸ **گزینه‌ی ۳** آسان

پرانتز اول arrow function را به یک عبارت تبدیل می‌کند و `()` آخر بلافاصله صدایش می‌زند؛ به این الگو IIFE (Immediately Invoked Function Expression) می‌گویند. این کد declaration نیست (رد گزینه‌ی ۱)، چون داخل پرانتز است اسم لازم ندارد و خطا نمی‌دهد (رد گزینه‌ی ۲) و به خاطر `()` آخر حتماً اجرا می‌شود (رد گزینه‌ی ۴).

۳۵۹ **گزینه‌ی ۲** آسان

فقط با **یک** پارامتر ساده می‌شود پرانتز را حذف کرد. تابع بدون پارامتر حتماً `()` می‌خواهد (مثل `() => "Hi!"`؛ رد گزینه‌ی ۱) و دو پارامتر یا بیشتر هم باید داخل پرانتز باشند (`(a, b) => a + b`؛ رد گزینه‌های ۳ و ۴).

۳۶۰ **گزینه‌ی ۴** آسان

هر دو arrow function بدنه‌ی مختصر (بدون آکولاد) دارند، پس مقدارشان **خودکار** برمی‌گردد (implicit return): `double(7)` برابر `14` و `hello()` برابر `"hello"`. حذف پرانتز دور تنها پارامتر `x` و `()` خالی برای تابع بدون پارامتر هر دو مجازند (رد گزینه‌ی ۲). گزینه‌ی ۱ فکر کرده بدون `return` چیزی برنمی‌گردد و گزینه‌ی ۳ تصور کرده `x` مقدار نگرفته است.

۳۶۱ **گزینه‌ی ۴** متوسط

arrow function hoist نمی‌شود و `multiply` یک `const` است که تا رسیدن به خط تعریفش در TDZ قرار دارد؛ پس خط ۱ خطای `ReferenceError: Cannot access 'multiply' before initialization` می‌دهد. گزینه‌ی ۱ رفتار function declaration را فرض کرده است. `undefined` و `TypeError` (گزینه‌های ۲ و ۳) مربوط به حالتی است که تابع در `var` ذخیره شده باشد.

۳۶۲ **گزینه‌ی ۱** متوسط

بدنه‌ی مختصر `a * b` بدون آکولاد مقدارش را خودکار برمی‌گرداند؛ پس گزینه‌ی ۱ دقیقاً همان تابع است. `->` نحو Java است و در JavaScript `SyntaxError` می‌دهد (گزینه‌ی ۲). در گزینه‌ی ۳ آکولاد بدنه‌ی بلوکی می‌سازد و بدون `return` نتیجه `undefined` است. در گزینه‌ی ۴ دو پارامتر بدون پرانتز آمده‌اند که `SyntaxError` می‌دهد.

۳۶۳ **گزینه‌ی ۲** متوسط

`add` بدنه‌ی بلوکی دارد (آکولاد) و `return` ندارد، پس `undefined` برمی‌گرداند؛ `sub` بدنه‌ی مختصر دارد و `5 - 3` یعنی `2` را خودکار برمی‌گرداند. گزینه‌ی ۱ فکر کرده آکولاد هم return ضمنی دارد و گزینه‌ی ۳ قاعده را برعکس فهمیده است. هر دو تعریف از نظر نحوی درست‌اند (رد گزینه‌ی ۴).

---

۳۶۴ **گزینه‌ی ۳** متوسط

`square` یک function declaration است و hoist می‌شود؛ پس خط ۱ بدون مشکل `9` را چاپ می‌کند (رد گزینه‌ی ۱). `cube` یک arrow function در `const` است و در خط ۲ هنوز در TDZ قرار دارد؛ خطای `ReferenceError: Cannot access 'cube' before initialization` همین‌جا رخ می‌دهد و اجرا متوقف می‌شود. خط ۶ خودش مشکلی ندارد (رد گزینه‌ی ۲) و گزینه‌ی ۴ فکر کرده هر دو تابع hoist می‌شوند.

---

۳۶۵ **گزینه‌ی ۱** متوسط

هر دو، IIFE هستند: تابع داخل پرانتز تعریف و با `()` بعدی فوراً اجرا می‌شود. اولی با `3` و `4` صدا زده می‌شود و `12` را برمی‌گرداند که در `result` ذخیره می‌شود. دومی هم بلافاصله `IIFE ran` را چاپ می‌کند (رد گزینه‌ی ۳). نحو هر دو درست است (رد گزینه‌ی ۲) و arrow function با بدنه‌ی مختصر مقدارش را برمی‌گرداند (رد گزینه‌ی ۴).

---

۳۶۶ **گزینه‌ی ۱** متوسط

IIFE یک بار و همان لحظه اجرا می‌شود و متغیرهایش فقط داخل خودش زنده‌اند؛ این راهی کلاسیک برای جلوگیری از آلودگی محدوده‌ی سراسری (به‌خصوص در دوران `var`) است. IIFE ربطی به hoisting ندارد (رد گزینه‌ی ۲)، تکرار زمان‌دار کار `setInterval` است نه IIFE (رد گزینه‌ی ۳) و چون نامی در محدوده ثبت نمی‌کند، نمی‌شود بعداً دوباره صدایش زد (رد گزینه‌ی ۴).

---

۳۶۷ **گزینه‌ی ۳** متوسط

دو پارامتر حتماً باید داخل پرانتز باشند؛ در گزینه‌ی ۳ موتور `x, y => ...` را درست نمی‌فهمد و `SyntaxError` می‌دهد. گزینه‌ی ۱ یک پارامتر بدون پرانتز دارد که مجاز است، گزینه‌ی ۲ تابع بدون پارامتر با `()` خالی است و گزینه‌ی ۴ شکل کامل با بدنه‌ی بلوکی و `return` است؛ هر سه درست‌اند.

---

۳۶۸ **گزینه‌ی ۱** متوسط

return ضمنی فقط در **بدنه‌ی مختصر** (بدون آکولاد) وجود دارد؛ با آکولاد باید `return` را صریح نوشت، وگرنه نتیجه `undefined` است. پس گزینه‌ی ۱ نادرست است. سه جمله‌ی دیگر درست‌اند: arrow شیء `arguments` ندارد، با `new` خطای `TypeError: ... is not a constructor` می‌دهد و با یک پارامتر ساده، پرانتز اختیاری است.

---

۳۶۹ **گزینه‌ی ۱** متوسط

arrow function `this` مخصوص خودش را ندارد و آن را از بیرون به ارث می‌برد؛ پس در متدی که با `this` کار می‌کند، `this` به object اشاره نمی‌کند. توابع کوتاه، توابع بدون پارامتر (`() => ...`) و callbackهای ساده دقیقاً جاهایی هستند که arrow function انتخاب خوبی است (رد گزینه‌های ۲ تا ۴).

---

۳۷۰ **گزینه‌ی ۲** متوسط

تابع معمولی `this` مخصوص خودش را دارد و در `user.regular()` برابر `user` است: `Sara`. arrow function `this` را از کد بیرونی به ارث می‌برد؛ اینجا سطح بالای فایل (در فایل معمولی Node.js یک object خالی و در مرورگر `window`) که `username` ندارد، پس `undefined` چاپ می‌شود و خطایی رخ نمی‌دهد (رد گزینه‌ی ۴). گزینه‌ی ۱ فکر کرده arrow هم `this` را از object قبل از نقطه می‌گیرد و گزینه‌ی ۳ قاعده را برعکس کرده است.

---

۳۷۱ **گزینه‌ی ۴** متوسط

`add` یک arrow function است که با بدنه‌ی مختصر **یک arrow function دیگر** برمی‌گرداند. `add(2)` تابع `b => 2 + b` را می‌دهد و `(3)` همان را صدا می‌زند: `5`. `add(5)` تابعی می‌سازد که `a = 5` را نگه داشته و `add5(10)` برابر `15` است. دو `=>` پشت سر هم نحو درستی است (رد گزینه‌ی ۱)، تابع برگشتی واقعاً تابع است (رد گزینه‌ی ۳) و `a` بعد از پایان `add` از بین نمی‌رود (رد گزینه‌ی ۲).

---

۳۷۲ **گزینه‌ی ۳** متوسط

IIFE فوراً اجرا می‌شود و `inside: 42` را چاپ می‌کند. `secret` فقط داخل همان تابع وجود دارد، پس بیرون از آن تعریف نشده است (رد گزینه‌ی ۱). `typeof` روی نامی که اصلاً تعریف نشده خطا نمی‌دهد و `"undefined"` برمی‌گرداند (رد گزینه‌ی ۲؛ استفاده‌ی مستقیم مثل `console.log(secret)` بود که `ReferenceError` می‌داد). نحو IIFE هم درست است (رد گزینه‌ی ۴).

---

۳۷۳ **گزینه‌ی ۴** سخت

متغیر `var` hoist می‌شود و از ابتدا مقدار `undefined` دارد، ولی خودِ arrow function فقط وقتی ساخته و در آن ریخته می‌شود که اجرا به خط ۲ برسد. پس در خط ۱ داریم `undefined(4, 5)` که خطای `TypeError: multiply is not a function` می‌دهد.
`ReferenceError` (گزینه‌ی ۳) برای حالت `let` / `const` است، گزینه‌ی ۱ رفتار function declaration را فرض کرده و گزینه‌ی ۲ تصور کرده تابع اجرا می‌شود ولی با پارامترهای بی‌مقدار.

---

۳۷۴ **گزینه‌ی ۲** سخت

در `makeUser`، آکولاد بعد از `=>` شروع **بدنه‌ی بلوکی** است، نه object؛ `name:` یک label (برچسب) و `"Ali"` یک عبارت بی‌اثر است و بدون `return` نتیجه `undefined` می‌شود. این کد از نظر نحوی مجاز است (رد گزینه‌ی ۳). در `makeUser2` پرانتز دور آکولاد آن را به یک object تبدیل می‌کند که با return ضمنی برمی‌گردد. گزینه‌ی ۱ آکولاد را object فرض کرده و گزینه‌ی ۴ فکر کرده مقدار برچسب برمی‌گردد.

---

۳۷۵ **گزینه‌ی ۴** سخت

`greet` یک function declaration است و کامل hoist می‌شود، پس از همان خط ۱ نوعش `"function"` است. `var welcome` یک `undefined` است؛ تابع arrow هنوز ساخته نشده، پس `typeof` برابر `"undefined"` است که hoist می‌شود ولی تا خط ۴ مقدارش است (رد گزینه‌ی ۱). گزینه‌ی ۳ فکر کرده `var` هم مثل `let` / `const` در TDZ است و گزینه‌ی ۲ hoisting کامل function declaration را نادیده گرفته است.

---

۳۷۶ **گزینه‌ی ۲** سخت

خطی که با کلمه‌ی `function` شروع شود، function declaration حساب می‌شود؛ declaration نام می‌خواهد و نمی‌شود همان‌جا با `()` صدایش زد، پس گزینه‌ی ۲ خطای `SyntaxError: Function statements require a function name` می‌دهد. در گزینه‌های ۱، ۳ و ۴ تابع داخل پرانتز آمده و به یک عبارت تبدیل شده است؛ هر سه IIFEهای درستی هستند و `hi` را چاپ می‌کنند (در گزینه‌ی ۴ فقط جای `()` داخل پرانتز بیرونی است).

---

۳۷۷ **گزینه‌ی ۳** سخت

برای برگرداندن object از بدنه‌ی مختصر باید آن را داخل پرانتز گذاشت (گزینه‌ی ۳). در گزینه‌ی ۱ آکولاد بدنه‌ی بلوکی است و تابع `undefined` برمی‌گرداند؛ خواندن `.name` از `undefined` خطای `TypeError` می‌دهد. در گزینه‌ی ۲، `return` بدون آکولاد بعد از `=>` مجاز نیست و `SyntaxError` می‌گیریم. گزینه‌ی ۴ رشته‌ی `"Ali"` را برمی‌گرداند و رشته ویژگی `name` ندارد، پس `undefined` چاپ می‌شود.

---

۳۷۸ **گزینه‌ی ۱** سخت

تابع معمولی شیء `arguments` مخصوص خودش را دارد و `3` را برمی‌گرداند. arrow function چنین شیئی ندارد و در سطح بالای اسکریپت مرورگر هم تابع معمولیِ دربرگیرنده‌ای نیست که `arguments` را از آن به ارث ببرد؛ پس `ReferenceError: arguments is not defined` می‌گیریم. گزینه‌های ۲ و ۳ فکر کرده‌اند arrow هم `arguments` دارد و گزینه‌ی ۴ مشکل را به تابع معمولی هم نسبت داده است. (در سطح بالای یک فایل Node.js نتیجه فرق می‌کند، چون Node.js فایل را داخل یک تابع پنهان می‌پیچد.)

---

۳۷۹ **گزینه‌ی ۲** سخت

arrow function شیء `arguments` مخصوص خودش را ندارد و، مثل `this`، آن را از تابع معمولیِ دربرگیرنده یعنی `outer` به ارث می‌برد. پس `arguments[0]` اولین آرگومان `outer` یعنی `"a"` است، نه `"x"` (رد گزینه‌ی ۱). چون `outer` یک تابع معمولی است، `arguments` وجود دارد و خطایی رخ نمی‌دهد (رد گزینه‌های ۳ و ۴).

---

۳۸۰ **گزینه‌ی ۳** سخت

تابع معمولی `Car` می‌تواند با `new` به‌عنوان constructor استفاده شود؛ پس خط ۷ درست است و خط ۸ `Pride` را چاپ می‌کند. arrow function نمی‌تواند constructor باشد و خط ۹ خطای `TypeError: Bike is not a constructor` می‌دهد. خط ۵ هیچ‌وقت اجرا نمی‌شود، چون بدنه‌ی `Bike` اصلاً صدا زده نمی‌شود (رد گزینه‌ی ۱).

۱-۱۷ مشکلات var

۳۸۱ **گزینه‌ی ۴** آسان

`var` انتساب دوباره را کاملاً اجازه می‌دهد (`var x = 1; x = 2;` مشکلی ندارد)؛ ممنوعیت انتساب دوباره ویژگی `const` است. پس گزینه‌ی ۴ جزو مشکلات `var` نیست. گزینه‌های ۱، ۲ و ۳ سه مورد از پنج مشکلی هستند که اسلاید ۲۷ نام می‌برد (دو مورد دیگر hoisting و رفتار ناسازگارند).

۳۸۲ **گزینه‌ی ۱** آسان

«بقیه» (EVERYONE ELSE) می‌خواهند `var` نابود شود و «برنامه‌نویس‌های قدیمی» (OLD JS DEVS) مثل ایسیلدور که حلقه را نگه داشت، می‌گویند «نه». گزینه‌ی ۲ نقش‌ها را برعکس کرده است. `var` قدیمی‌ترین روش تعریف متغیر است، نه جدیدترین (رد گزینه‌ی ۳) و میم هیچ ادعایی درباره‌ی سرعت ندارد (رد گزینه‌ی ۴).

۳۸۳ **گزینه‌ی ۲** آسان

اسلاید ۲۷: «`var` in global context attaches to window». چون همه‌ی اسکریپت‌های صفحه یک `window` مشترک دارند، `var`های هم‌نام از اسکریپت‌های مختلف بی‌صدا همدیگر را بازنویسی می‌کنند. گزینه‌ی ۱ درباره‌ی مدیریت حافظه است و ربطی به این مشکل ندارد. متغیرهای سراسری داخل توابع قابل خواندن‌اند (رد گزینه‌ی ۳) و `var` هیچ کپی‌ای از `window` نمی‌سازد (رد گزینه‌ی ۴).

۳۸۴ **گزینه‌ی ۳** آسان

`var` function-scoped است و آکولادهای `if` جلویش را نمی‌گیرند؛ `color` از بلوک بیرون می‌زند و `red` چاپ می‌شود. گزینه‌ی ۲ رفتار `let` / `const` است که block-scoped هستند. گزینه‌ی ۱ فکر کرده مقدار داخل بلوک بیرون نمی‌آید (ولی خط ۲ قبل از چاپ اجرا شده است) و کد هیچ خطای نحوی‌ای ندارد (رد گزینه‌ی ۴).

۳۸۵ **گزینه‌ی ۲** آسان

اعلان `var score` به بالا hoist می‌شود و از ابتدا مقدار `undefined` دارد، ولی مقداردهی `= 50` سر جای خودش (خط ۲) انجام می‌شود. پس خط ۱ `undefined` و خط ۳ `50` چاپ می‌کند. گزینه‌ی ۱ فکر کرده مقدار هم hoist می‌شود، گزینه‌ی ۳ رفتار `let` را فرض کرده و گزینه‌ی ۴ فراموش کرده خط ۲ قبل از خط ۳ اجرا شده است.

۳۸۶ **گزینه‌ی ۴** آسان

`var` اجازه‌ی تعریف دوباره را می‌دهد و خطایی نمی‌گیریم (رد گزینه‌ی ۲؛ این خطا مخصوص `let` / `const` است). هر دو خط به **همان یک** متغیر اشاره می‌کنند و خط ۲ مقدار `2` را در آن می‌ریزد. گزینه‌ی ۱ فکر کرده تعریف دوم نادیده گرفته می‌شود و گزینه‌ی ۳ تصور کرده تعریف دوباره متغیر را خالی می‌کند.

۳۸۷ **گزینه‌ی ۱** متوسط

`a` با `var` تعریف شده و از بلوک `if` بیرون می‌زند، پس `1` چاپ می‌شود. `b` با `let` تعریف شده و فقط داخل بلوک وجود دارد؛ خط ۶ خطای `ReferenceError: b is not defined` می‌دهد. گزینه‌ی ۲ هر دو را مثل `var` فرض کرده، گزینه‌ی ۳ هر دو را مثل `let` و گزینه‌ی ۴ تصور کرده متغیر تعریف‌نشده `undefined` چاپ می‌شود.

۳۸۸ **گزینه‌ی ۱** متوسط

شمارنده‌ی `var` بعد از حلقه باقی می‌ماند (نشت از بلوک). حلقه برای `0`، `1` و `2` اجرا می‌شود، بعد `i++` مقدار را `3` می‌کند و شرط `3 < 3` نادرست می‌شود؛ پس `i` با مقدار `3` بیرون می‌ماند. گزینه‌ی ۲ آخرین مقدارِ اجرای بدنه را با مقدار نهایی اشتباه گرفته و گزینه‌ی ۴ رفتار `let` را فرض کرده است.

۳۸۹ گزینه‌ی ۲ متوسط

تعریف دوباره‌ی `var` مجاز است، پس خط ۲ مشکلی ندارد (رد گزینه‌ی ۱). اما تعریف دوباره‌ی `let` در همان محدوده خطای `SyntaxError: Identifier 'b' has already been declared` در خط ۴ می‌دهد. این خطا هنگام خواندن (parse) فایل کشف می‌شود؛ یعنی هیچ خطی، حتی خط ۱، اجرا نمی‌شود و به خط ۵ هم نمی‌رسیم (رد گزینه‌های ۳ و ۴).

۳۹۰ گزینه‌ی ۳ متوسط

خواندن `var` قبل از تعریفش خطا نمی‌دهد و `undefined` چاپ می‌کند؛ پس خط ۱ سالم است (رد گزینه‌ی ۱). `b` یک `let` است که تا خط ۴ در TDZ قرار دارد؛ خط ۳ خطای `ReferenceError: Cannot access 'b' before initialization` می‌دهد. خط ۴ خودش مشکلی ندارد و اصلاً به آن نمی‌رسیم (رد گزینه‌ی ۲) و گزینه‌ی ۴ فکر کرده `let` هم مثل `var` مقدار `undefined` می‌گیرد.

۳۹۱ گزینه‌ی ۴ متوسط

`var` function-scoped است: از بلوک `if` بیرون می‌زند و داخل `setup` همه‌جا دیده می‌شود، پس خط ۵ `abc` را چاپ می‌کند (رد گزینه‌های ۲ و ۳). اما از خودِ **تابع** بیرون نمی‌رود؛ بیرون از `setup` متغیری به نام `token` نیست و خط ۸ خطای `ReferenceError` می‌دهد. گزینه‌ی ۱ تصور کرده `var` از تابع هم نشت می‌کند.

۳۹۲ گزینه‌ی ۲ متوسط

`var` سطح بالای اسکریپت معمولی مرورگر به ویژگی شیء سراسری `window` تبدیل می‌شود (همان global scope pollution اسلاید ۲۷)، پس `window.city` برابر `Tehran` است. `let` و `const` با اینکه سراسری‌اند، ویژگی `window` نمی‌شوند؛ پس `window.country` برابر `undefined` است. گزینه‌ی ۱ فکر کرده هر متغیر سراسری روی `window` می‌رود و گزینه‌های ۳ و ۴ رفتار `var` را نادیده گرفته‌اند.

۳۹۳ گزینه‌ی ۳ متوسط

گزینه‌ی ۳ همان تفاوت اصلی است. گزینه‌ی ۱ برعکس است: `var` تعریف دوباره را اجازه می‌دهد و `let` خطای `SyntaxError` می‌دهد. گزینه‌ی ۲ رفتار `var` است؛ `let` قبل از تعریف در TDZ است و `ReferenceError` می‌دهد. گزینه‌ی ۴ هم غلط است: فقط `var` (و function declaration) سطح بالا ویژگی `window` می‌شوند.

۳۹۴ گزینه‌ی ۱ متوسط

فقط `var` سطح بالا در اسکریپت معمولی مرورگر به `window` می‌چسبد. در Node.js اصلاً `window` وجود ندارد و `var` سطح بالای فایل هم محلیِ همان ماژول است (رد گزینه‌ی ۲). داخل یک تابع، از جمله IIFE که خودش یک تابع است، `var` محلیِ همان تابع است و به `window` نمی‌رسد (رد گزینه‌های ۳ و ۴)؛ دقیقاً به همین دلیل قدیمی‌ها کدشان را داخل IIFE می‌گذاشتند.

۳۹۵ گزینه‌ی ۲ سخت

اعلان `var msg` داخل `show` به **بالای همان تابع** hoist می‌شود؛ پس از خط اول تابع یک `msg` محلی با مقدار `undefined` وجود دارد که `msg` سراسری را پنهان می‌کند. خط ۳ `undefined` و بعد از مقداردهی، خط ۵ `local` چاپ می‌کند. گزینه‌ی ۱ (پرطرفدارترین اشتباه) hoisting را نادیده گرفته، گزینه‌ی ۳ فکر کرده متغیر محلی ساخته نمی‌شود و گزینه‌ی ۴ رفتار `let` را فرض کرده است.

۳۹۶ گزینه‌ی ۴ سخت

هر فایل `.js` در Node.js معمولی را یک ماژول (CommonJS) با محدوده‌ی جداگانه اجرا می‌کند؛ پس `var` سطح بالا ویژگی شیء سراسری `globalThis` نمی‌شود و `undefined` چاپ می‌شود (رد گزینه‌های ۱ و ۲؛ این رفتار مخصوص اسکریپت معمولی مرورگر است). Node.js شیء `window` ندارد، ولی `typeof` روی نام تعریف‌نشده خطا نمی‌دهد و `"undefined"` برمی‌گرداند (رد گزینه‌ی ۳).

۳۹۷ گزینه‌ی ۱ سخت

هر دو اسکریپت در محدوده‌ی سراسری مشترک صفحه اجرا می‌شوند و `var counter` در هر دو، **همان یک** ویژگی `window.counter` است. اسکریپت دوم بی‌صدا آن را `0` می‌کند و `getCounter` که مقدار فعلی `counter` را می‌خواند، `0` برمی‌گرداند؛ این همان global scope pollution است. گزینه‌ی ۲ فکر کرده هر اسکریپت متغیر جداگانه‌ی خودش را دارد. `getCounter` هم سراسری است و از اسکریپت دوم قابل صدا زدن است (رد گزینه‌ی ۴).

۳۹۸ **گزینه‌ی ۳** سخت

`var a` داخل بلوک متغیر جدیدی نمی‌سازد؛ چون `var` بلوک را نمی‌شناسد، این همان `a` بیرونی است که دوباره تعریف و `2` می‌شود. اما `let b` داخل بلوک یک متغیر **جدید و جدا** است که فقط در همان بلوک زنده است و `b` بیرونی همان `1` می‌ماند. هیچ تعریف دوباره‌ای در یک محدوده‌ی واحد برای `let` رخ نداده است (رد گزینه‌ی ۴). گزینه‌ی ۱ فکر کرده `var` هم بلوکی است و گزینه‌ی ۲ فکر کرده `let` هم مثل `var` بیرونی را بازنویسی می‌کند.

۳۹۹ **گزینه‌ی ۴** سخت

callbackهای `setTimeout` بعد از تمام شدن حلقه اجرا می‌شوند. با `let` هر دور حلقه `i` تازه‌ی خودش را دارد، پس `0` ، `1` و `2` چاپ می‌شود. با `var` کل حلقه یک `i` مشترک دارد که در پایان `3` است و سه بار `3` چاپ می‌شود (گزینه‌ی ۱). بدون کلمه‌ی کلیدی هم (در حالت عادی و غیر strict) یک متغیر سراسری مشترک ساخته می‌شود و باز هم سه بار `3` می‌گیریم (گزینه‌ی ۳). با `const` دستور `i++` بعد از دور اول خطای `TypeError` می‌دهد (گزینه‌ی ۲). جزئیات این دام را در درس «Closure» می‌بینیم.

۴۰۰ **گزینه‌ی ۳** سخت

`var count;` در خط ۲ فقط یک تعریف دوباره‌ی بدون مقدار است؛ چنین تعریفی متغیر را از نو نمی‌سازد و مقدار فعلی را پاک نمی‌کند، پس `count` همان `5` می‌ماند. گزینه‌ی ۱ فکر کرده تعریف بدون مقدار، متغیر را به `undefined` برمی‌گرداند. تعریف دوباره‌ی `var` خطا ندارد (رد گزینه‌ی ۲) و `null` هیچ‌وقت خودکار به متغیری داده نمی‌شود (رد گزینه‌ی ۴).

۱۸-۱ Hoisting و Temporal Dead Zone (TDZ)

۴۰۱ **گزینه‌ی ۲** آسان

میم دقیقاً درباره‌ی «استفاده از متغیر قبل از تعریفش» است و JavaScript به خاطر **hoisting** این کار را اجازه می‌دهد؛ مثلاً `console.log(x); var x = 5;` خطا نمی‌دهد و `undefined` چاپ می‌کند. type coercion درباره‌ی تبدیل نوع داده‌هاست، closure درباره‌ی دسترسی تابع به متغیرهای scope بیرونی و garbage collection درباره‌ی پاک‌سازی حافظه؛ هیچ‌کدام ربطی به استفاده‌ی زودتر از تعریف ندارند.

۴۰۲ **گزینه‌ی ۳** آسان

در مرحله‌ی ایجاد (قبل از اجرای کد)، موتور نام همه‌ی اعلان‌های هر scope را ثبت می‌کند؛ اسلاید ۳۷ این را «Moving declarations before execution» و «code runs as if declared at top» می‌نامد. گزینه‌ی ۱ اشتباه رایج است: **مقداردهی** بالا نمی‌رود و سر جای خودش اجرا می‌شود (برای همین `var` قبل از تعریف `undefined` است، نه مقدار نهایی). گزینه‌ی ۲ غلط است، چون `var` قبل از تعریف `undefined` است، نه `null` ، و `let` اصلاً مقدار اولیه نمی‌گیرد. گزینه‌ی ۴ ساختگی است؛ هیچ تبدیل خودکاری بین `var` و `let` وجود ندارد.

۴۰۳ **گزینه‌ی ۳** آسان

متن اسلاید ۳۸: «(partially) Works for `var` , `function` , and `class`». گزینه‌ی ۱ غلط است، چون `var` هم hoist می‌شود (و قبل از تعریف `undefined` است). گزینه‌ی ۲ نادرست است، چون `var` و `function` را که قطعاً hoist می‌شوند کنار گذاشته است (`let` و `const` هم مثل `class` فقط به‌صورت ناقص و در TDZ hoist می‌شوند). گزینه‌ی ۴ هم function declaration را نادیده گرفته که اسلاید ۳۴ صریحاً می‌گوید hoist می‌شود.

۴۰۴ **گزینه‌ی ۲** آسان

جمله‌ی اسلاید ۳۸: «Variables are hoisted but not **initialized**». یعنی نام متغیر از ابتدای scope ثبت و شناخته می‌شود، ولی مقداری که در خط تعریف آمده هنوز در آن ریخته نشده است: `var` موقتاً `undefined` است و `let` / `const` اصلاً مقدار اولیه ندارند (TDZ). گزینه‌های ۱، ۳ و ۴ همه به معنی «اعلان یا ثبت شدن» هستند؛ این دقیقاً کاری است که hoisting **انجام می‌دهد**، پس نمی‌توانند در جای خالی بنشینند.

۴۰۵ **گزینه‌ی ۱** آسان

اعلان `var x` hoist می‌شود و `x` از ابتدا با مقدار `undefined` وجود دارد، ولی مقداردهی `= 5` سر جای خودش (خط ۲) اجرا می‌شود. پس خط ۱ `undefined` و خط ۳ `5` چاپ می‌کند. گزینه‌ی ۲ فکر کرده مقداردهی هم بالا می‌رود؛ گزینه‌ی ۳ رفتار `let` را به `var` نسبت داده؛ گزینه‌ی ۴ هم اشتباه است، چون مقدار پیش‌فرض `undefined` است، نه `null`.

۴۰۶ **گزینه‌ی ۳** متوسط

در مرحله‌ی ایجاد: `var a` ساخته و با `undefined` مقداردهی می‌شود؛ `let b` ساخته می‌شود ولی مقدار اولیه نمی‌گیرد (در TDZ است)؛ function declaration `c` همراه با کل بدنه‌اش ساخته می‌شود. مقدارهای `5` و `6` فقط در مرحله‌ی اجرا و وقتی خطشان برسد ریخته می‌شوند (رد گزینه‌ی ۱). گزینه‌ی ۲ تفاوت `let` و `function` را نادیده گرفته و گزینه‌ی ۴ رفتار `var` و `let` را برعکس گفته و بدنه‌ی تابع را هم جا انداخته است.

۴۰۷ **گزینه‌ی ۱** متوسط

`var i` در سرآیند حلقه به بالای scope (اینجا کل فایل) hoist می‌شود، چون `var` بلوک حلقه را نمی‌شناسد. پس خط ۱ `undefined` چاپ می‌کند، نه خطا (رد گزینه‌ی ۲). حلقه تا وقتی `i < 3` است ادامه دارد و وقتی `i` به `3` می‌رسد متوقف می‌شود؛ `i` بیرون از حلقه هم در دسترس است و `3` چاپ می‌شود. گزینه‌ی ۳ آخرین مقدارِ «داخل» بدنه‌ی حلقه را گفته و گزینه‌ی ۴ فکر کرده hoisting مقدار اولیه‌ی `0` را هم بالا می‌برد.

۴۰۸ **گزینه‌ی ۴** سخت

`var label` به بالای تابع `check` hoist می‌شود و بلوک `if` برایش اهمیتی ندارد. در `check(50)` شرط درست است و مقدار `label` `"big"` می‌گیرد. در `check(5)` شرط نادرست است، ولی `label` باز هم وجود دارد (با مقدار `undefined`)، پس `return label` خطا نمی‌دهد (رد گزینه‌ی ۱). هر بار صدا زدن تابع متغیرهای محلی تازه‌ی خودش را دارد، پس `"big"` از صدا زدن قبلی باقی نمی‌ماند (رد گزینه‌ی ۲)، و مقدار پیش‌فرض `undefined` است نه `null` (رد گزینه‌ی ۳).

۴۰۹ **گزینه‌ی ۴** آسان

این تعریف **Temporal Dead Zone (TDZ)** است: متغیر `let` / `const` از ابتدای scope وجود دارد ولی تا اجرای خط تعریفش قابل استفاده نیست. hoisting نام کلی پدیده‌ی ثبت اعلان‌ها در ابتدای scope است، نه آن بازه‌ی ممنوع؛ global scope محدوده‌ی سراسری برنامه است و closure به خاطر سپردن متغیرهای بیرونی توسط یک تابع.

۴۱۰ **گزینه‌ی ۱** آسان

در گزینه‌ی ۱، `b` با `let` تعریف شده و `console.log(b)` در TDZ آن اجرا می‌شود، پس `ReferenceError: Cannot access 'b' before initialization` رخ می‌دهد. گزینه‌ی ۲ فقط `undefined` چاپ می‌کند (رفتار `var`)، گزینه‌ی ۳ تابعی را صدا می‌زند که به‌صورت function declaration کامل hoist شده است، و گزینه‌ی ۴ متغیری را می‌خواند که قبلاً اعلان شده و مقدارش `undefined` است.

۴۱۱ **گزینه‌ی ۱** متوسط

TDZ از ابتدای scope (بلوک، بدنه‌ی تابع یا فایل) شروع می‌شود و لحظه‌ای که خط تعریف اجرا شود تمام می‌شود. گزینه‌ی ۲ برعکس است: `var` اصلاً TDZ ندارد و TDZ مخصوص `let`، `const` و `class` است. گزینه‌ی ۳ جهت را اشتباه گفته؛ بعد از خط تعریف، متغیر کاملاً قابل استفاده است. گزینه‌ی ۴ هم رفتار `var` است؛ دسترسی در TDZ خطای `ReferenceError` می‌دهد.

۴۱۲ **گزینه‌ی ۱** متوسط

در خط ۲ برای محاسبه‌ی `price + tax` باید `tax` خوانده شود، ولی `tax` با `let` در خط ۳ تعریف شده و هنوز در TDZ است؛ پس `ReferenceError: Cannot access 'tax' before initialization` روی خط ۲ رخ می‌دهد. خط ۳ خودش مشکلی ندارد و خط ۴ هم به دلیل توقف برنامه هرگز اجرا نمی‌شود. کسی که «هیچ خطایی» را انتخاب کند، `let` را با `var` اشتباه گرفته است (با `var` حاصل `100 + undefined` یعنی `NaN` می‌شد).

۴۱۳ **گزینه‌ی ۳** سخت

TDZ به **زمان اجرا** بستگی دارد، نه به جای نوشتن کد. تابع `greet` گرچه بالاتر از `let userName` نوشته شده، فقط در خط ۵ و **بعد از** اجرای خط ۴ صدا زده می‌شود؛ در آن لحظه TDZ تمام شده و `userName` برابر `"Sara"` است. اگر `greet()` قبل از خط ۴ صدا زده می‌شد، `ReferenceError` می‌گرفتیم (گزینه‌ی ۲). گزینه‌ی ۱ رفتار `var` قبل از مقداردهی است و گزینه‌ی ۴ هم هیچ پایه‌ای ندارد.

۴۱۴ **گزینه‌ی ۲** سخت

`typeof` برای نامی که **اصلاً تعریف نشده** امن است و رشته‌ی `"undefined"` را برمی‌گرداند (خط ۲). داخل بلوک، `let later = 5` یک `later` **جدید** می‌سازد که از ابتدای بلوک ثبت شده و `later` بیرونی را پنهان (shadow) می‌کند؛ پس خط ۴، `later` داخلی را می‌بیند که هنوز در TDZ است و حتی `typeof` هم `ReferenceError` می‌دهد. گزینه‌ی ۱ فکر کرده خط ۴ متغیر بیرونی را می‌بیند (یعنی `let` داخلی hoist نمی‌شود)، گزینه‌ی ۴ مقداردهی داخلی را هم hoistشده فرض کرده، و گزینه‌ی ۳ فکر کرده `typeof` روی نام تعریف‌نشده هم خطا می‌دهد.

۴۱۵ **گزینه‌ی ۲** متوسط

پیام `Cannot access 'x' before initialization` یعنی موتور متغیر را می‌شناسد (hoist شده) ولی در TDZ است؛ `x is not defined` یعنی هیچ متغیری با این نام در scopeهای قابل‌دسترسی نیست. گزینه‌ی ۱ این دو را جابه‌جا کرده است. گزینه‌ی ۳ غلط است، چون `TypeError: x is not a function` وقتی رخ می‌دهد که چیزی را که تابع نیست صدا بزنیم؛ مثلاً `var x` قبل از مقداردهی که `undefined` است. گزینه‌ی ۴ هم غلط است: این‌ها سه وضعیت متفاوت‌اند (نوع خطای سومی حتی `TypeError` است)؛ فقط **متن** یک پیام ممکن است بین موتورها فرق کند.

۴۱۶ **گزینه‌ی ۳** سخت

داخل بلوک، `let count` جدیدی تعریف می‌شود که از ابتدای بلوک، `count` بیرونی را پنهان (shadow) می‌کند. برای اجرای خط ۳ اول باید سمت راست، یعنی `count + 1`، حساب شود؛ ولی TDZ متغیر داخلی تا **پایان** اجرای همین خط ادامه دارد، پس خواندن `count` در سمت راست `ReferenceError` می‌دهد. انتخاب «هیچ خطایی» (یعنی انتظار چاپ `6` و `5`) از این تصور غلط می‌آید که سمت راست، `count` بیرونی را می‌بیند. خط‌های ۴ و ۶ هرگز اجرا نمی‌شوند.

۴۱۷ **گزینه‌ی ۴** آسان

`add` یک function declaration است و در مرحله‌ی ایجاد **همراه با بدنه‌اش** ساخته می‌شود؛ اسلاید ۳۴: «Can be called before or after it's defined (hoisted)». پس صدا زدن قبل از خط تعریف، `5` چاپ می‌کند. `ReferenceError` و `TypeError` رفتار arrow function یا function expression در `const` / `var` است، و `undefined` فقط وقتی چاپ می‌شد که تابع چیزی برنمی‌گرداند.

۴۱۸ **گزینه‌ی ۲** آسان

`multiply` با `const` تعریف شده و arrow function فقط در خط ۳ در آن ریخته می‌شود. در خط ۲ هنوز در TDZ است، پس `ReferenceError: Cannot access 'multiply' before initialization` رخ می‌دهد؛ اسلاید ۳۵: «Not hoisted (must be defined before use)». خط ۱ قبل از خطا اجرا شده و `Start` را چاپ کرده است. خط ۳ خودِ تعریف است و مشکلی ندارد، و خط ۶ به دلیل توقف برنامه هرگز اجرا نمی‌شود.

۴۱۹ **گزینه‌ی ۴** متوسط

`greet` با `var` تعریف شده؛ پس hoist می‌شود و تا اجرای خط ۳ مقدارش `undefined` است. خط ۱ رشته‌ی `"undefined"` را چاپ می‌کند. خط ۲ سعی می‌کند `undefined` را مثل تابع صدا بزند، پس `TypeError: greet is not a function` رخ می‌دهد؛ نه `ReferenceError`، چون متغیر وجود دارد (رد گزینه‌ی ۲). گزینه‌ی ۱ فکر کرده function expression هم مثل function declaration کامل hoist می‌شود و گزینه‌ی ۳ فکر کرده خط ۱ هم خطا می‌دهد.

۴۲۰ **گزینه‌ی ۴** متوسط

فقط function declaration (گزینه‌ی ۴) کامل hoist می‌شود و قبل از خط تعریف قابل صدا زدن است. در گزینه‌های ۱ و ۳، `sayHi` با `const` / `let` تعریف شده و هنگام صدا زدن در TDZ است: `ReferenceError`. در گزینه‌ی ۲، `sayHi` با `var` تعریف شده و `undefined` است؛ صدا زدنش `TypeError: sayHi is not a function` می‌دهد. دقت کن که در گزینه‌ی ۳ هم کلمه‌ی `function` آمده، ولی این یک function expression در `let` است و رفتارش را `let` تعیین می‌کند.

۴۲۱ **گزینه‌ی ۱** متوسط

در مرحله‌ی ایجاد، `a` و `c` (هر دو `var`) با `undefined` ساخته می‌شوند؛ اینکه بعداً یک function expression یا arrow function در آن‌ها ریخته می‌شود، در خط ۱ هنوز اهمیتی ندارد. فقط `b` که function declaration است همراه با بدنه‌اش ساخته می‌شود. پس خروجی `undefined function undefined` است. گزینه‌ی ۲ فکر کرده هر چیزی که تابع است hoist می‌شود؛ گزینه‌ی ۳ arrow function را برخلاف اسلاید ۳۵ hoistشده فرض کرده؛ و گزینه‌ی ۴ غلط است، چون اینجا نه `typeof` و نه `var` خطایی ایجاد نمی‌کنند.

۴۲۲ **گزینه‌ی ۲** سخت

بدنه‌ی `getMessage` مرحله‌ی ایجاد خودش را دارد: `format` (function declaration) همراه با بدنه ساخته می‌شود و `suffix` (`var`) با مقدار `undefined`. بعد خط `return format("hi")` اجرا می‌شود و تابع همان‌جا تمام می‌شود؛ پس خط `var suffix = "!"` **هرگز** اجرا نمی‌شود و حاصل `"hi" + undefined` رشته‌ی `"hiundefined"` است. گزینه‌ی ۱ فرض کرده مقداردهی هم hoist می‌شود. گزینه‌های ۳ و ۴ فکر کرده‌اند چیزی که بعد از `return` نوشته شده (تابع `format` یا متغیر `suffix`) اصلاً وجود ندارد؛ ولی hoisting اعلان‌ها را صرف‌نظر از جایشان در بدنه ثبت می‌کند.

۴۲۳ **گزینه‌ی ۲** متوسط

`class` طبق اسلاید ۳۸ «hoist partially» می‌شود: نامش از ابتدای scope ثبت می‌شود ولی تا اجرای خط تعریف (خط ۳) در TDZ است. پس `new Animal("cat")` در خط ۲ خطای `ReferenceError: Cannot access 'Animal' before initialization` می‌دهد. خط ۱ قبل از خطا اجرا شده و پیامش را چاپ کرده است؛ خط ۳ و خط ۸ هم به دلیل توقف برنامه اجرا نمی‌شوند. کسی که «هیچ خطایی» را انتخاب کند، کلاس را مثل function declaration کاملاً hoistشده فرض کرده است.

۴۲۴ **گزینه‌ی ۴** متوسط

«partially» یعنی hoisting کلاس نیمه‌کاره است: نام کلاس از ابتدای scope شناخته می‌شود (برای همین پیام خطا `Cannot access 'X' before initialization` است، نه `X is not defined`)، ولی مثل `const`/`let` تا خط تعریف در TDZ می‌ماند و قابل استفاده نیست. گزینه‌ی ۱ ساختگی است (کلاس به‌صورت یک واحد تعریف می‌شود)، گزینه‌ی ۲ غلط است چون رفتار در مرورگر و Node.js یکسان است، و گزینه‌ی ۳ رفتار `var` است، نه `class`.

۴۲۵ **گزینه‌ی ۳** متوسط

`let x = 2` داخل `f` باعث می‌شود یک `x` محلی از **ابتدای** بدنه‌ی `f` ثبت شود و `x` بیرونی را پنهان کند. پس خط ۳، `x` محلی را می‌خواند که هنوز در TDZ است: `ReferenceError: Cannot access 'x' before initialization` (خطا روی همین خط پرتاب می‌شود؛ خط ۶ فقط تابع را صدا زده است). خط ۱ مشکلی ندارد، چون `x` داخلی در scope دیگری است و تعریف دوباره حساب نمی‌شود. خط ۴ هرگز اجرا نمی‌شود. اگر `let` اصلاً hoist نمی‌شد، خط ۳ عدد `1` را چاپ می‌کرد و خطایی رخ نمی‌داد (گزینه‌ی ۱).

۴۲۶ **گزینه‌ی ۴** سخت

داخل `test`، اعلان `var a` (هرچند داخل بلوک `if` نوشته شده) به بالای **تابع** hoist می‌شود و `a` سراسری را پنهان می‌کند. پس اولین `console.log(a)` مقدار `a` محلی یعنی `undefined` را چاپ می‌کند. بعد `if` اجرا می‌شود و `a` محلی `2` می‌شود؛ دومین چاپ `2` است. `a` سراسری هرگز تغییر نکرده، پس خط آخر `1` چاپ می‌کند. گزینه‌های ۱ و ۲ hoisting داخل تابع را نادیده گرفته‌اند و گزینه‌های ۲ و ۳ فکر کرده‌اند `var a = 2` متغیر سراسری را تغییر می‌دهد.

۴۲۷ **گزینه‌ی ۲** سخت

`calc` یک function declaration است و در خط ۱ قابل صدا زدن است؛ ولی در آن لحظه `factor` (یک `var`) هنوز `undefined` است، پس `3 * undefined` برابر `NaN` می‌شود. بعد از اجرای خط ۶، `factor` برابر `2` است و صدا زدن دوباره `6` می‌دهد. گزینه‌ی ۱ فرض کرده مقداردهی `factor` هم hoist می‌شود، گزینه‌ی ۳ رفتار `let` را به `var` نسبت داده، و گزینه‌ی ۴ فراموش کرده که ضرب در `undefined` عدد `NaN` می‌دهد، نه `undefined`.

۴۲۸ **گزینه‌ی ۱** سخت

هر دو function declaration در مرحله‌ی ایجاد پردازش می‌شوند و دومی جای اولی را می‌گیرد. در مرحله‌ی اجرا، خط‌های تعریف تابع کاری انجام نمی‌دهند، پس هر سه صدا زدن، نسخه‌ی دوم را اجرا می‌کنند. گزینه‌های ۲ و ۳ فکر کرده‌اند تعریف‌ها در مرحله‌ی اجرا و به ترتیب خط‌ها جایگزین می‌شوند. گزینه‌ی ۴ هم غلط است: در این کد تعریف دوباره‌ی یک function declaration (مثل `var`) خطا نمی‌دهد؛ این `let`، `const` و `class` هستند که با تعریف دوباره `SyntaxError` می‌دهند.

---

۴۲۹ **گزینه‌ی ۳** سخت

در مرحله‌ی ایجاد، function declaration برنده است: `item` همان تابع است و اعلان `var item` (چون این نام از قبل وجود دارد) آن را نمی‌کند. در مرحله‌ی اجرا، خط ۱ (`var item;` بدون مقدار) هیچ کاری انجام نمی‌دهد، پس خط ۲ `function` چاپ می‌کند. خط ۴ رشته‌ی `"pen"` را در `item` می‌ریزد و خط ۵ `string` چاپ می‌کند. گزینه‌های ۱ و ۲ فکر کرده‌اند `var item;` مقدار را `undefined` می‌کند و گزینه‌ی ۴ فکر کرده خط تعریف تابع در مرحله‌ی اجرا دوباره `item` را تابع می‌کند.

---

۴۳۰ **گزینه‌ی ۳** سخت

در گزینه‌ی ۳، اعلان `var n` در مرحله‌ی ایجاد بالا می‌رود؛ در مرحله‌ی اجرا اول `n = 5` اجرا می‌شود، بعد `console.log(n)` عدد `5` را چاپ می‌کند و `var n;` (بدون مقدار) کاری انجام نمی‌دهد. گزینه‌های ۱ و ۲ هر دو `undefined` چاپ می‌کنند، چون موقع چاپ هنوز مقداری در `n` ریخته نشده است. گزینه‌ی ۴ به دلیل TDZ خطای `ReferenceError` می‌دهد.

---

۴۳۱ **گزینه‌ی ۴** متوسط

خط ۱ یک `var` را قبل از تعریف می‌خواند: `undefined` و بدون خطا. خط ۲ یک function declaration را صدا می‌زند که کامل hoist شده است و `b` چاپ می‌شود. خط ۳ متغیر `c` را می‌خواند که با `let` در خط ۸ تعریف شده و هنوز در TDZ است: `ReferenceError`. پس اولین (و تنها) خطا روی خط ۳ است.

---

۴۳۲ **گزینه‌ی ۱** آسان

تعریف همه‌چیز با `let` / `const` در بالای scope و پیش از استفاده باعث می‌شود ترتیب نوشتن کد با ترتیب واقعی اجرا یکی باشد، و اگر جایی اشتباهاً زودتر از متغیری استفاده کنی، خطای واضح `ReferenceError` بگیری. گزینه‌ی ۲ خطا را حذف نمی‌کند، بلکه آن را به یک `undefined` بی‌صدا و باگ پنهان تبدیل می‌کند (همان مشکل `var` در اسلاید ۲۷). گزینه‌ی ۳ ترتیب کد را گیج‌کننده‌تر می‌کند و برای `let`، `const` و `class` خطا می‌دهد. گزینه‌ی ۴ غلط است، چون `typeof` روی متغیری که در TDZ است خطا می‌دهد.

۱-۱۹ Closure (بستار)

---

۴۳۳ **گزینه‌ی ۳** آسان

طبق اسلاید ۲۸، closure یعنی تابعی که دسترسی به متغیرهای تابع والدش (scope بیرونی) را **نگه می‌دارد**، حتی بعد از اینکه اجرای تابع بیرونی تمام شده باشد. گزینه‌ی ۲ تصور غلط رایجی است: closure از مقدارها عکس (snapshot) نمی‌گیرد، بلکه به خودِ متغیرها وصل است و تغییرات بعدی را می‌بیند. گزینه‌ی ۱ از معنی لغوی close (بستن) گمراه شده و گزینه‌ی ۴ دقیقاً برعکس تعریف است.

---

۴۳۴ **گزینه‌ی ۴** آسان

کلمه‌ی lexical (واژگانی) یعنی «مربوط به متن کد»: اینکه یک تابع کدام متغیرها را می‌بیند، با **جایی که در کد نوشته شده** تعیین می‌شود. جایی که صدا زده می‌شود (گزینه‌ی ۱) یا ترتیب صدا زدن‌ها در زمان اجرا (گزینه‌ی ۲) تأثیری در scope آن ندارد، و نوع داده‌ی متغیرها (گزینه‌ی ۳) هم ربطی به دسترسی ندارد.

---

۴۳۵ **گزینه‌ی ۲** آسان

`inner` داخل `outer` نوشته شده، پس به متغیرهای خودش، متغیرهای `outer` (یعنی `course`) و متغیرهای سراسری (`site`) دسترسی دارد؛ جست‌وجوی هر نام از درونی‌ترین scope شروع می‌شود و لایه‌به‌لایه بیرون می‌رود (scope chain). پس `Web @ Sharif` چاپ می‌شود. گزینه‌های ۱ و ۴ فکر کرده‌اند تابع درونی فقط یکی از این دو لایه را می‌بیند و گزینه‌ی ۳ فکر کرده تابع درونی اصلاً به متغیرهای بیرونی دسترسی ندارد.

---

۴۳۶ **گزینه‌ی ۲** متوسط

تابع بی‌نامی که `makePrinter` برمی‌گرداند، داخل `makePrinter` **نوشته شده**، پس `label` را در scope همان تابع پیدا می‌کند: `"maker"` . اینکه بعداً داخل `run` صدا زده شده (جایی که `label` برابر `"runner"` است) اهمیتی ندارد، چون scope با محل نوشتن تعیین می‌شود (lexical scope؛ رد گزینه‌ی ۳). `label` سراسری هم با `label` نزدیک‌تر (داخل `makePrinter`) پنهان شده است (رد گزینه‌ی ۱). `undefined` هم غلط است، چون نام در زنجیره‌ی scope پیدا می‌شود.

۴۳۷ **گزینه‌ی ۴** آسان

`counter()` متغیر `count` را با `0` می‌سازد و تابع درونی را برمی‌گرداند. اولین `c()` همان `count` را `1` می‌کند و برمی‌گرداند؛ دومین `c()` **همان** متغیر را `2` می‌کند. گزینه‌ی ۱ فکر کرده `count` با هر `c()` از نو ساخته می‌شود (در حالی که فقط یک بار، هنگام اجرای `counter()` ، ساخته شده است). گزینه‌ی ۲ فکر کرده مقدار قبل از افزایش برگردانده می‌شود، ولی `return count` بعد از `count++` اجرا می‌شود. گزینه‌ی ۳ فکر کرده `count` با تمام شدن `counter` از بین رفته است.

۴۳۸ **گزینه‌ی ۲** آسان

`counter()` یک **تابع** برمی‌گرداند (همان `function() { count++; return count; }`)، نه عدد. پس `c` تابعی است که `count` را در کوله‌پشتی‌اش دارد و `typeof c` برابر `"function"` است. گزینه‌ی ۱ فکر کرده `c` همان مقدار `count` است؛ ولی `count` فقط با صدا زدن `c()` برگردانده می‌شود. `undefined` و `object` هم غلط‌اند: `counter` واقعاً چیزی برگردانده و `typeof` برای توابع `"function"` می‌دهد.

۴۳۹ **گزینه‌ی ۱** آسان

اسلاید ۲۸: «Keeps access even after the outer function ends». وقتی `counter` تمام می‌شود، معمولاً متغیرهای محلی‌اش پاک می‌شوند؛ ولی چون تابع برگردانده‌شده (`c`) هنوز به `count` نیاز دارد، JavaScript آن را زنده نگه می‌دارد. برای همین `c()` بار دوم `2` می‌دهد، نه خطا (رد گزینه‌ی ۲) و نه دوباره `1` (رد گزینه‌ی ۳). `count` سراسری هم نمی‌شود (رد گزینه‌ی ۴): بیرون از `counter` با نام `count` قابل دسترسی نیست و فقط از راه `c` به آن می‌رسیم.

۴۴۰ **گزینه‌ی ۴** آسان

`count` متغیر محلی `counter` است و فقط تابع درونی (از طریق closure) به آن دسترسی دارد. در scope سراسری هیچ متغیری به نام `count` وجود ندارد، پس خط ۱۱ خطای `ReferenceError: count is not defined` می‌دهد. خط‌های ۴ و ۱۰ مشکلی ندارند: تابع درونی `count` را از کوله‌پشتی‌اش می‌خواند و خط ۱۰ عدد `2` را چاپ می‌کند. این همان encapsulation است: داده فقط از راه تابع در دسترس است.

۴۴۱ **گزینه‌ی ۴** متوسط

هر بار صدا زدن `counter()` یک `count` **جدید** و یک closure مستقل می‌سازد. `a` سه بار صدا زده شده (دو بار بیرون و یک بار داخل `console.log`)، پس `3` برمی‌گرداند. `b` شمارنده‌ی خودش را دارد و اولین صدا زدنش `1` است. `counter()()` یک شمارنده‌ی کاملاً تازه می‌سازد و همان لحظه یک بار صدایش می‌زند: `1` . گزینه‌ی ۱ فکر کرده همه یک `count` مشترک دارند، گزینه‌ی ۲ صدا زدن‌های `a()` بیرون از `console.log` را نادیده گرفته، و گزینه‌ی ۳ فکر کرده `counter()()` شمارنده‌ی `b` را ادامه می‌دهد.

۴۴۲ **گزینه‌ی ۳** متوسط

closure از مقدار متغیر عکس نمی‌گیرد؛ به **خودِ متغیر** `msg` وصل است. `say` وقتی ساخته شد که `msg` برابر `"first"` بود، ولی قبل از اینکه صدا زده شود، `msg` به `"second"` تغییر کرد. وقتی `fn()` اجرا می‌شود، مقدار **فعلی** `msg` را می‌خواند: `second` . گزینه‌ی ۱ همان تصور غلط «snapshot» است و گزینه‌های ۲ و ۴ فکر کرده‌اند بعد از تمام شدن `setup` متغیرهایش از بین رفته‌اند.

۴۴۳ **گزینه‌ی ۱** سخت

`greeting` پارامتر `makeGreeter` است؛ یعنی یک متغیر محلی جداگانه که هنگام صدا زدن، **مقدار فعلی** `g` (یعنی `"Hi"`) در آن ریخته شده است. closure به همین پارامتر وصل است، نه به `g` ؛ پس تغییر بعدی `g` به `"Bye"` روی `greeting` اثری ندارد و `Hi, Ali` چاپ می‌شود. گزینه‌ی ۲ فکر کرده closure به `g` وصل است (به متغیرِ داخل scope خودش، یعنی `greeting` ، وصل است). گزینه‌های ۳ و ۴ هم فکر کرده‌اند پارامتر بعد از تمام شدن `makeGreeter` از بین رفته است.

۴۴۴ **گزینه‌ی ۴** متوسط

عبارت ۴ نادرست است: closure از مقدارها کپی ثابت (snapshot) نمی‌گیرد؛ به خودِ متغیرها وصل است و اگر متغیر بعداً تغییر کند، closure مقدار جدید را می‌بیند. بقیه درست‌اند: هر بار صدا زدن تابع بیرونی متغیرهای جدیدی می‌سازد (مثل دو شمارنده‌ی مستقل)، دسترسی بعد از تمام شدن تابع بیرونی باقی می‌ماند (اسلاید ۲۸)، و تابع درونی می‌تواند متغیر بیرونی را تغییر دهد (مثل `count++` در کد اسلاید).

۴۴۵ **گزینه‌ی ۱** آسان

اسلاید ۲۸: «Used for encapsulation, state, and callbacks»؛ یعنی پنهان کردن داده (متغیر خصوصی)، نگه‌داری وضعیت (مثل شمارنده) و callbackها (مثل تابعی که به `setTimeout` داده می‌شود). تبدیل نوع و `==` (گزینه‌ی ۲) موضوع coercion است، hoisting و TDZ (گزینه‌ی ۳) موضوع درس دیگری است، و JavaScript تک‌نخی (single threaded) است و closure هیچ thread جدیدی نمی‌سازد (گزینه‌ی ۴).

۴۴۶ **گزینه‌ی ۴** سخت

`balance` متغیر محلی `createWallet` است و فقط `deposit` و `getBalance` از طریق closure به آن دسترسی دارند: بعد از دو واریز، `70` . خط `w.balance = 1000` یک **ویژگی (property) جدید** به نام `balance` روی شیء `w` می‌سازد که هیچ ارتباطی با متغیر خصوصی ندارد؛ پس `w.balance` برابر `1000` است و `getBalance()` همچنان `70` برمی‌گرداند. گزینه‌ی ۱ فکر کرده این خط متغیر خصوصی را عوض می‌کند (دقیقاً همان چیزی که encapsulation جلویش را می‌گیرد)، گزینه‌ی ۲ خط ساختن ویژگی را نادیده گرفته، و گزینه‌ی ۳ این دو مقدار را با هم جمع کرده است.

۴۴۷ **گزینه‌ی ۳** متوسط

`inc` و `reset` در **یک** اجرای `createCounter` ساخته شده‌اند، پس هر دو به **یک** `count` مشترک دسترسی دارند. دو بار `inc` آن را `2` می‌کند، `reset` همان متغیر را `0` می‌کند و `inc` بعدی آن را `1` می‌کند و برمی‌گرداند. گزینه‌ی ۱ فکر کرده هر تابع کپی جداگانه‌ای از `count` دارد و `reset` روی شمارنده‌ی `inc` اثری ندارد؛ گزینه‌ی ۲ فکر کرده `inc` مقدار قبل از افزایش را برمی‌گرداند؛ گزینه‌ی ۴ هم غلط است، چون هر دو ویژگی واقعاً تابع‌اند و صدا زدنشان مشکلی ندارد.

۴۴۸ **گزینه‌ی ۱** متوسط

هر صدا زدن `makeMultiplier` یک closure جدید با `factor` مخصوص خودش می‌سازد: `double` همیشه `factor = 2` و `triple` همیشه `factor = 3` را می‌بیند. پس `double(5) = 10` و `triple(5) = 15` . در عبارت آخر اول `triple(2) = 6` حساب می‌شود و بعد `double(6) = 12` . گزینه‌ی ۲ فکر کرده آخرین `factor` (یعنی `3`) بین همه مشترک شده، گزینه‌ی ۳ فکر کرده اولین `factor` باقی مانده، و گزینه‌ی ۴ صدا زدن بیرونی `double` را جا انداخته است.

۴۴۹ **گزینه‌ی ۲** متوسط

`once` یک closure برمی‌گرداند که متغیر `done` (و پارامتر `fn`) را به خاطر می‌سپارد. در اولین `init()` ، هنوز `done` است؛ پس `true` می‌شود، `fn` اجرا می‌شود و `init!` چاپ می‌شود. در صدا زدن‌های دوم و سوم، **همان** `done` حالا `true` است و چیزی چاپ نمی‌شود. گزینه‌ی ۱ فکر کرده `done` با هر صدا زدن دوباره `false` می‌شود (در حالی که فقط یک بار، هنگام اجرای `once` ، ساخته شده)، گزینه‌ی ۳ فکر کرده `fn` بعد از تمام شدن `once` دیگر در دسترس نیست، و گزینه‌ی ۴ هم غلط است، چون `fn` یک تابع واقعی است که در کوله‌پشتی closure مانده است.

۴۵۰ **گزینه‌ی ۳** سخت

`total` یک متغیر سراسری است و **هر دو** closure به همان یک `total` دسترسی دارند؛ ولی هر کدام `step` مخصوص خودش را دارد (`5` و `10`). مراحل: `add5()` ← `total = 5` ؛ `add10()` ← `total = 15` ؛ داخل `console.log` اول `add5()` اجرا می‌شود ← `total` و `20` برمی‌گرداند؛ بعد `total` خوانده می‌شود که `20` است. گزینه‌ی ۱ فکر کرده هر closure کپی جداگانه‌ای از `total` گرفته و متغیر سراسری دست‌نخورده می‌ماند. گزینه‌ی ۲ فکر کرده `total` قبل از اجرای `add5()` خوانده می‌شود، در حالی که آرگومان‌ها از چپ به راست ارزیابی می‌شوند. گزینه‌ی ۴ فکر کرده `add5` مجموع جداگانه‌ی خودش را دارد.

۴۵۱ **گزینه‌ی ۳** متوسط

هر صدا زدن `greetLater` اول یک callback را برای بعد زمان‌بندی می‌کند و بلافاصله `scheduled: ...` را چاپ می‌کند. callbackهای `setTimeout` (حتی با تأخیر `0`) فقط بعد از تمام شدن کل کد همگام اجرا می‌شوند؛ پس اول هر دو `scheduled` و بعد هر دو `Hello` چاپ می‌شوند (رد گزینه‌ی ۱). هر صدا زدن `greetLater` پارامتر `name` مخصوص خودش را دارد و هر callback، `name` همان صدا زدن را به خاطر می‌سپارد: اول `Ali` و بعد `Sara` (رد گزینه‌ی ۲). گزینه‌ی ۴ هم فکر کرده بعد از تمام شدن `greetLater` پارامترهایش از بین رفته‌اند؛ در حالی که این دقیقاً همان کاری است که closure انجام می‌دهد: «Keeps access even after the outer function ends».

---

۴۵۲ **گزینه‌ی ۴** آسان

متن اسلاید ۲۸: «BEWARE OF PITFALL: But closures can cause problems when combined with *var*». دلیلش این است که `var` بلوک (مثلاً بدنه‌ی حلقه) را نمی‌شناسد و فقط یک متغیر مشترک می‌سازد، پس همه‌ی closureها همان یک متغیر را می‌بینند. `let` و `const` برعکس، block-scoped هستند و دقیقاً راه‌حل این مشکل‌اند؛ arrow function هم مشکلی ایجاد نمی‌کند.

---

۴۵۳ **گزینه‌ی ۱** متوسط

حلقه سه callback را **زمان‌بندی** می‌کند ولی اجرا نمی‌کند. بعد از حلقه، `i` (که با `var` تعریف شده و بیرون حلقه هم در دسترس است) برابر `3` است و خط آخر `after loop: 3` را چاپ می‌کند. فقط بعد از تمام شدن کد همگام، سه callback اجرا می‌شوند و هر سه **همان یک** `i` مشترک را می‌خوانند: `3`. گزینه‌ی ۲ فکر کرده هر callback مقدار `i` لحظه‌ی ساخته شدنش را نگه داشته (رفتار `let`)، گزینه‌ی ۳ فکر کرده callbackها با تأخیر `0` فوراً اجرا می‌شوند، و گزینه‌ی ۴ آخرین مقدار داخل حلقه (`2`) را با مقدار نهایی `i` اشتباه گرفته است.

---

۴۵۴ **گزینه‌ی ۳** متوسط

دو دلیل با هم: (۱) `var` function-scoped است، پس در کل حلقه فقط **یک** `i` وجود دارد و هر سه callback همان را در closure دارند؛ (۲) callbackهای `setTimeout` بعد از تمام شدن حلقه اجرا می‌شوند، وقتی `i` به `3` رسیده است. گزینه‌ی ۱ غلط است: چیزی را کپی نمی‌کند و اتفاقاً مشکل این است که **هیچ** کپی‌ای ساخته نمی‌شود. گزینه‌ی ۲ غلط است، چون arrow function مثل هر تابع دیگری به متغیرهای بیرونی دسترسی دارد. گزینه‌ی ۴ هم ساختگی است؛ تأخیر `0` مجاز است و حلقه فقط سه بار اجرا می‌شود.

---

۴۵۵ **گزینه‌ی ۱** متوسط

وقتی `let` در سرآیند `for` باشد، JavaScript برای هر تکرار یک **binding (نسخه‌ی) جدید** از `i` می‌سازد که با مقدار همان تکرار شروع می‌شود؛ برای همین با `let` خروجی `0`، `1`، `2` است. گزینه‌ی ۲ دقیقاً برعکس است (یک متغیر مشترک، رفتار `var` است). گزینه‌ی ۳ غلط است، چون `let` هرگز به `var` تبدیل نمی‌شود و block-scoped می‌ماند. گزینه‌ی ۴ هم غلط است: closure از مقدارها عکس نمی‌گیرد، و اگر همه یک snapshot از مقدار پایانی داشتند، همه یک عدد را چاپ می‌کردند.

---

۴۵۶ **گزینه‌ی ۱** سخت

همه‌ی callbackها تأخیر `0` دارند و به همان ترتیبی که زمان‌بندی شده‌اند اجرا می‌شوند: اول سه callback حلقه‌ی اول، بعد سه callback حلقه‌ی دوم (رد گزینه‌ی ۳ که آن‌ها را درهم چیده است). callbackهای حلقه‌ی `var` همگی یک `i` مشترک دارند که بعد از حلقه `3` است؛ ولی حلقه‌ی `let` برای هر تکرار یک `j` جدید می‌سازد، پس `0`، `1`، `2` چاپ می‌شود. گزینه‌ی ۲ فکر کرده `var` هم مقدار هر تکرار را نگه می‌دارد و گزینه‌ی ۴ فکر کرده `let` هم مثل `var` یک متغیر مشترک دارد.

---

۴۵۷ **گزینه‌ی ۴** متوسط

در هر تکرار، IIFE (تابعی که همان لحظه صدا زده می‌شود) با مقدار فعلی `i` اجرا می‌شود و آن مقدار در پارامتر `n` ریخته می‌شود؛ هر صدا زدن `n` مخصوص خودش را دارد (`0`، `1`، `2`) و callback همان `n` را به خاطر می‌سپارد. اول کد همگام تمام می‌شود: `i` (یک `var` که بیرون حلقه هم در دسترس است) برابر `3` است و `loop done, i = 3` چاپ می‌شود؛ بعد callbackها `0`، `10` و `20` را چاپ می‌کنند. گزینه‌ی ۱ فکر کرده callbackها قبل از خط آخر اجرا می‌شوند، گزینه‌ی ۲ فکر کرده IIFE مشکل `var` را حل نمی‌کند، و گزینه‌ی ۳ مقدار نهایی `i` را اشتباه گفته است.

---

۴۵۸ **گزینه‌ی ۲** متوسط

در حلقه‌ی اول، همه‌ی توابع آرایه‌ی `a` یک `i` مشترک (`var`) دارند که بعد از حلقه `3` شده است؛ پس `a[1]()` هم `3` برمی‌گرداند. در حلقه‌ی دوم، `let` برای هر تکرار یک `j` جدید می‌سازد و تابع دوم (`b[1]`) تکرار دوم یعنی `1` را دارد. گزینه‌ی ۱ فکر کرده `var` هم مقدار هر تکرار را نگه می‌دارد، گزینه‌ی ۳ فکر کرده `let` هم مشترک است، و گزینه‌ی ۴ رفتار دو حلقه را جابه‌جا کرده است. این دام به `setTimeout` ربطی ندارد: کافی است تابع **بعد از** تمام شدن حلقه صدا زده شود.

---

۴۵۹ **گزینه‌ی ۲** سخت

هر callback دو متغیر را در closure دارد که رفتار متفاوتی دارند. `label` با `const` **داخل بدنه‌ی حلقه** تعریف شده، پس در هر تکرار یک `label` **جدید** (block-scoped) با مقدار همان تکرار ساخته می‌شود: `item0`، `item1`، `item2`. ولی `i` با `var` تعریف شده و فقط **یک** `i` مشترک وجود دارد که وقتی callbackها اجرا می‌شوند `3` است. گزینه‌ی ۱ فکر کرده `i` هم برای هر تکرار جدا ساخته می‌شود (این رفتار `let` در سرآیند حلقه است)؛ گزینه‌ی ۳ فکر کرده `label` هم مثل `var` مشترک است و آخرین مقدارش را نگه می‌دارد؛ و گزینه‌ی ۴ فکر کرده `label` هم هنگام اجرای callback از روی `i` نهایی ساخته می‌شود، در حالی که مقدارش همان لحظه‌ی تکرار حساب شده است.

---

۴۶۰ **گزینه‌ی ۳** سخت

فقط در گزینه‌ی ۳ هر تکرار متغیر مستقلی می‌سازد: IIFE مقدار فعلی `i` را به‌عنوان **پارامتر** `k` می‌گیرد و هر callback، `k` مخصوص خودش را دارد. گزینه‌ی ۱ `2`، `2`، `2` چاپ می‌کند: `var j` هم function-scoped است و فقط یک `j` مشترک وجود دارد که آخرین مقدارش `2` است. گزینه‌ی ۲ با اینکه IIFE دارد، `i` را به‌عنوان پارامتر نگرفته و همان `i` مشترک را می‌خواند: `3`، `3`، `3`. گزینه‌ی ۴ هم `3`، `3`، `3` چاپ می‌کند: `let i` بیرون از سرآیند حلقه تعریف شده، پس فقط یک binding وجود دارد.

---

۴۶۱ **گزینه‌ی ۳** سخت

شرط حلقه `i <= 3` است، پس حلقه با `i = 1`، `2` و `3` اجرا می‌شود و وقتی `i` به `4` برسد متوقف می‌شود. تأخیرها (`100`، `200` و `300` میلی‌ثانیه) همان لحظه‌ی زمان‌بندی و با مقدار فعلی `i` حساب می‌شوند، ولی **بدنه‌ی** callbackها دیرتر اجرا می‌شود و `i` مشترک را می‌خواند که حالا `4` است. گزینه‌ی ۱ فکر کرده هر callback مقدار تکرار خودش را دارد (رفتار `let`)، گزینه‌ی ۲ مقدار نهایی را مثل حلقه‌ی معروف `i < 3` حساب کرده، و گزینه‌ی ۴ شروع حلقه را `0` فرض کرده است.

---

۴۶۲ **گزینه‌ی ۱** سخت

هر لایه یک closure است: درونی‌ترین تابع به `x` (از `a`)، `y` (از تابع وسط) و `z` (پارامتر خودش) دسترسی دارد، حتی بعد از اینکه دو تابع بیرونی تمام شده‌اند. `a(1)(2)(3)` یعنی `1 + 2 + 3 = 6`. در `a("1")(2)(3)` جمع از چپ به راست انجام می‌شود: رشته‌ی `"1" + 2` و بعد `"12" + 3` رشته‌ی `"123"` می‌شود (type coercion). گزینه‌ی ۲ coercion را نادیده گرفته، گزینه‌ی ۳ فرض کرده `2 + 3` اول حساب می‌شود، و گزینه‌ی ۴ فکر کرده جمع رشته و عدد `NaN` می‌دهد.

---

۴۶۳ **گزینه‌ی ۲** سخت

ترکیب hoisting و closure: داخل `makeCounter`، تابع `increment` (function declaration) کامل hoist می‌شود و `count` (`var`) با مقدار `undefined`. خط `return increment` تابع را برمی‌گرداند و خط `var count = 0` **هرگز** اجرا نمی‌شود. `increment` از طریق closure به همان `count` دسترسی دارد که `undefined` است؛ `count++` آن را به عدد تبدیل می‌کند و حاصل `NaN` است. بار بعد هم `NaN + 1` همان `NaN` است. گزینه‌ی ۱ فرض کرده `count = 0` اجرا شده، گزینه‌ی ۳ فکر کرده `increment` یا `count` به خاطر قرار گرفتن بعد از `return` وجود ندارند، و گزینه‌ی ۴ فراموش کرده که `++` مقدار را به عدد تبدیل می‌کند.

۱-۲۰ اشیاء (Objects) و Object literal

---

۴۶۴ **گزینه‌ی ۳** آسان

اسلاید ۴۰: «Values can be data (properties) or functions (methods)»؛ پس مقدار تابعی **method** و مقدار داده‌ای **property** نام دارد. گزینه‌ی ۲ همین دو را برعکس گفته است. key و value (گزینه‌ی ۱) دو بخشِ **هر** جفت‌اند (نام خانه و محتوای آن)، نه دو نوع مقدار. literal یعنی «نوشتن مستقیم شیء با آکولاد» و JSON یک قالب متنی برای ذخیره‌ی داده است (گزینه‌ی ۴).

---

۴۶۵ **گزینه‌ی ۱** آسان

اسلاید ۴۱: «Keys are strings (or symbols), values any type» و «Support methods and nested objects». گزینه‌ی ۲ برعکس است: مقدارها هر نوعی می‌توانند باشند و کلیدِ عددی یا شیء به **رشته** تبدیل می‌شود (نوعش حفظ نمی‌شود). گزینه‌ی ۳ با «nested objects» و متدها در تضاد است. JSON هم نوع داده نیست؛ یک قالب **متنی** برای ذخیره و تبادل داده‌های شیء است (اسلاید ۴۰).

۴۶۶ **گزینه‌ی ۴** آسان

`greet` با `user.greet()` صدا زده شده، پس داخل آن `this` همان شیءِ قبل از نقطه یعنی `user` است و `this.name` برابر `"Barry Allen"` می‌شود. گزینه‌ی ۱ فرض کرده `this` به شیء اشاره نمی‌کند، که فقط وقتی متد **بدون** شیءِ قبل از نقطه صدا زده شود پیش می‌آید. گزینه‌ی ۲ فراموش کرده رشته با backtick نوشته شده و `${...}` در آن جایگزین می‌شود. `ReferenceError` هم رخ نمی‌دهد چون `name` با `this.name` خوانده شده، نه به‌عنوان یک متغیر.

۴۶۷ **گزینه‌ی ۲** آسان

خواندن کلیدی که در شیء وجود ندارد **خطا نمی‌دهد** و فقط `undefined` برمی‌گرداند. `null` (گزینه‌ی ۱) را JavaScript خودش برنمی‌گرداند و فقط برنامه‌نویس آن را عمداً می‌گذارد؛ رشته‌ی خالی (گزینه‌ی ۳) هم مقدار پیش‌فرضی نیست. `ReferenceError` مخصوص **متغیرِ** تعریف‌نشده است، نه **کلیدِ** ناموجود در یک شیء.

۴۶۸ **گزینه‌ی ۳** آسان

کلیدی که خط تیره دارد فقط با براکت و کوتیشن خوانده می‌شود: `person["first-name"]`. در گزینه‌ی ۱، JavaScript عبارت را `person.first - name` می‌فهمد (یک تفریق!)؛ چون متغیری به نام `name` نداریم، در Node.js خطای `ReferenceError` می‌گیریم. گزینه‌ی ۲ دنبال کلید دیگری (`first_name`) می‌گردد و `undefined` می‌دهد. در گزینه‌ی ۴ کوتیشن نیست، پس داخل براکت عبارتِ `first - name` حساب می‌شود و چون متغیر `first` وجود ندارد، `ReferenceError` رخ می‌دهد.

۴۶۹ **گزینه‌ی ۱** آسان

`const` فقط اجازه نمی‌دهد متغیر `user` به شیء **دیگری** وصل شود؛ تغییر محتوای همان شیء آزاد است. `user.age = 25` مقدار کلید موجود را عوض می‌کند و `user.city = ...` چون این کلید وجود نداشت آن را **می‌سازد**. گزینه‌ی ۳ همان تصور غلطِ «شیءِ `const` تغییرناپذیر است» را دارد؛ گزینه‌ی ۲ فرض کرده تغییرها نادیده گرفته می‌شوند و گزینه‌ی ۴ فکر کرده نمی‌توان کلید جدید اضافه کرد.

۴۷۰ **گزینه‌ی ۲** آسان

JSON فقط **داده** نگه می‌دارد (رشته، عدد، boolean، `null`، آرایه و شیء) و تابع در آن جایی ندارد؛ برای همین `JSON.stringify` متدها را کنار می‌گذارد. رشته‌ی داخل دابل‌کوتیشن، عدد و شیء تودرتو (گزینه‌های ۱، ۳ و ۴) همه در JSON مجازند؛ اسلاید ۴۱ هم می‌گوید object literal «پایه‌ی ساختار JSON» است.

۴۷۱ **گزینه‌ی ۲** متوسط

بعد از نقطه، خودِ کلمه نام کلید است: `user.key` دنبال کلیدی به نام `"key"` می‌گردد که وجود ندارد، پس `undefined`. داخل براکت یک **عبارت** است: `user[key]` مقدار متغیر `key` یعنی `"age"` را کلید می‌کند و `24` می‌دهد. `user["key"]` (با کوتیشن) دقیقاً همان `user.key` است و `undefined` می‌دهد. گزینه‌ی ۴ همین سومی را با `user[key]` اشتباه گرفته است.

۴۷۲ **گزینه‌ی ۴** متوسط

کلیدهای شیء همیشه رشته‌اند: کلید `1` در واقع `"1"` است، پس `obj[1]` و `obj["1"]` یک خانه‌اند و `one` چاپ می‌شود. به همین دلیل `obj[2]` هم عدد `2` را به `"2"` تبدیل می‌کند و `two` می‌دهد (رد گزینه‌های ۱ و ۲). `Object.keys` هم کلیدها را به‌صورت **رشته** برمی‌گرداند: `['1', '2']`؛ پس `[1, '2']` و `[1, 2]` (گزینه‌های ۱ و ۳) غلط‌اند.

۴۷۳ **گزینه‌ی ۲** متوسط

`user.city = ...` کلید تازه‌ای می‌سازد، `user.age = 25` مقدار کلید موجود را عوض می‌کند و `delete user.name` property **کل** را (کلید و مقدار) حذف می‌کند؛ پس در چاپ شیء اصلاً اثری از `name` نیست و خواندنش `undefined` می‌دهد. گزینه‌ی ۱ فکر کرده `delete` فقط مقدار را خالی می‌کند و کلید می‌ماند؛ گزینه‌ی ۳ فرض کرده `delete` کاری نمی‌کند؛ و گزینه‌ی ۴ تغییر محتوای شیءِ `const` را ممنوع پنداشته است.

۴۷۴ **گزینه‌ی ۳** متوسط

`const b = a` شیء تازه‌ای نمی‌سازد و فقط **reference** (نشانی) همان شیء را کپی می‌کند؛ پس `b.x = 5` همان شیئی را عوض می‌کند که `a` به آن اشاره دارد و `a.x` می‌شود `5`. اما `c` یک شیء **جداگانه** است (هرچند محتوای اولیه‌اش یکی بود) و `1` می‌ماند. برای `===` شیءها فقط می‌پرسد «همان یک شیء‌اند؟»: `a === b` درست و `a === c` نادرست است. گزینه‌ی ۱ کپی را مستقل فرض کرده و گزینه‌ی ۲ شیءها را با محتوا مقایسه کرده است.

---

۴۷۵ **گزینه‌ی ۱** متوسط

spread propertyها را **از چپ به راست** در شیء جدید می‌ریزد و برای کلید تکراری، **آخری برنده است**. در `a` اول `role: "user"` از `base` می‌آید و بعد `role: "admin"` جایش را می‌گیرد. در `b` اول `"admin"` نوشته می‌شود و بعد `...base` آن را با `"user"` بازنویسی می‌کند. `base` هم دست‌نخورده می‌ماند، چون spread یک شیء **جدید** می‌سازد (رد گزینه‌ی ۳). گزینه‌ی ۴ ترتیب را نادیده گرفته و گزینه‌ی ۲ نتیجه‌ی `a` و `b` را جابه‌جا کرده است.

---

۴۷۶ **گزینه‌ی ۱** متوسط

`{ name, age }` شکل کوتاه (shorthand) `{ name: name, age: age }` است و کاملاً مجاز است (رد گزینه‌ی ۴). `intro` با `student.intro()` صدا زده شده، پس `this` همان `student` است و `I'm Sara` برمی‌گردد (رد گزینه‌ی ۳). متد هم یک کلید معمولی شیء است، پس `Object.keys` آن را هم می‌آورد: `['name', 'age', 'intro']`؛ گزینه‌ی ۲ متدها را از کلیدها جدا فرض کرده است.

---

۴۷۷ **گزینه‌ی ۲** متوسط

`Object.values` آرایه‌ی **مقدارها** را می‌دهد: `[2, 3]` (گزینه‌ی ۱ آن را با `Object.keys` اشتباه گرفته). `Object.entries` برای هر کلید یک آرایه‌ی دوتایی `[key, value]` می‌سازد و همه را در یک آرایه برمی‌گرداند: `[['tea', 2], ['coffee', 3]]`. خروجی‌اش شیء نیست (رد گزینه‌ی ۳) و جفت‌ها هم در یک آرایه‌ی تخت پشت سر هم نمی‌آیند (رد گزینه‌ی ۴).

---

۴۷۸ **گزینه‌ی ۳** متوسط

خواندن کلیدِ ناموجود خطا نمی‌دهد: `user.phone` (خط ۳) و `user.address.zip` (خط ۴) فقط `undefined` می‌دهند. اما در خط ۵، `user.job` برابر `undefined` است و خواندن `title` از `undefined` خطای `TypeError: Cannot read properties of undefined (reading 'title')` می‌دهد و برنامه همان‌جا می‌ایستد. خط ۶ با `?.` امن نوشته شده و به‌تنهایی `undefined` می‌داد، ولی اصلاً به آن نمی‌رسیم.

---

۴۷۹ **گزینه‌ی ۴** متوسط

نام کلید (`"age"`) داخل متغیر `key` است، پس باید از براکت بدون کوتیشن استفاده کنیم تا مقدارِ متغیر حساب شود: `user[key]` همان `user["age"]` است. گزینه‌های ۱ و ۲ هر دو دنبال کلیدی به نام `"key"` می‌گردند و `undefined` چاپ می‌کنند. گزینه‌ی ۳ اصلاً نوشتار درستی ندارد و `SyntaxError` می‌دهد (`?.[key]` درست است، ولی `.[key]` نه).

---

۴۸۰ **گزینه‌ی ۴** متوسط

خط‌های ۲ تا ۴ فقط **محتوای** شیء را تغییر می‌دهند (عوض کردن، اضافه کردن و حذف کردن property)، که برای شیءِ `const` کاملاً مجاز است. اما خط ۵ می‌خواهد خودِ متغیر `user` را به یک شیء **دیگر** وصل کند؛ این انتساب دوباره است و خطای `TypeError: Assignment to constant variable.` می‌دهد. پس گزینه‌ی «بدون خطا» هم نادرست است.

---

۴۸۱ **گزینه‌ی ۱** متوسط

داخل object literal هر عضو باید شکل `key: value` (یا shorthand) داشته باشد. گزینه‌ی ۲ شکل کوتاهِ متد است (مثل `greet()` در اسلاید ۴۱)، گزینه‌ی ۳ شکل کامل آن و گزینه‌ی ۴ یک arrow function را به‌عنوان مقدار نگه می‌دارد؛ هر سه مجازند. اما نوشتن کلمه‌ی `function` قبل از نام متد (گزینه‌ی ۱) در object literal معتبر نیست و `SyntaxError` می‌دهد.

---

۴۸۲ **گزینه‌ی ۳** متوسط

وقتی یک شیء را به تابع می‌دهیم، تابع **همان شیء** را می‌گیرد (reference کپی می‌شود، نه خود شیء). پس هر بار `person.age` یکی زیاد می‌شود و بعد از دو فراخوانی، `sara.age` برابر `22` است. گزینه‌ی ۱ فرض کرده تابع روی یک کپی کار می‌کند، گزینه‌ی ۲ اثر فراخوانی دوم را جا انداخته و گزینه‌ی ۴ تغییر محتوای شیءِ `const` را ممنوع پنداشته است.

---

۴۸۳ **گزینه‌ی ۳** سخت

رشته primitive است، نه شیء. وقتی روی آن نقطه می‌گذاریم، JavaScript برای یک لحظه یک شیء پوشاننده (wrapper) می‌سازد؛ `extra` روی همان شیء موقت نوشته می‌شود که بلافاصله دور انداخته می‌شود، پس خواندن بعدی `undefined` می‌دهد (رد گزینه‌ی ۱). خواندن `length` و صدا زدن `toUpperCase` از همین مکانیزم استفاده می‌کنند و کار می‌کنند: `2 HI` (رد گزینه‌ی ۴). خطای `TypeError` (گزینه‌ی ۲) فقط در حالت strict رخ می‌داد: `Cannot create property 'extra' on string 'hi'`.

---

۴۸۴ **گزینه‌ی ۴** سخت

`settings.font` مقدارش `undefined` است و `settings.size` اصلاً وجود ندارد و آن هم `undefined` می‌دهد؛ پس خط اول `true` است و مقایسه با `undefined` فرق این دو حالت را نمی‌فهمد (رد گزینه‌ی ۱). `in` فقط می‌پرسد «کلید هست؟»: `font` هست (هرچند مقدارش `undefined` است) و `size` نیست (رد گزینه‌ی ۲). `toString` از `Object.prototype` به ارث رسیده: `in` آن را می‌بیند، ولی `hasOwnProperty` فقط کلیدهای **خودِ** شیء را حساب می‌کند و `false` می‌دهد (رد گزینه‌ی ۳).

---

۴۸۵ **گزینه‌ی ۲** سخت

spread یک کپی **سطحی** (shallow) می‌سازد: `copy` شیء جدیدی است، پس عوض کردن `copy.name` روی `user.name` اثری ندارد (رد گزینه‌ی ۳). اما مقدار `address` یک شیء است و فقط **reference** آن کپی شده؛ هر دو به **همان** شیء آدرس اشاره می‌کنند (`true` در خط آخر)، پس تغییر `copy.address.city` در `user` هم دیده می‌شود. گزینه‌های ۱ و ۴ spread را کپی کامل (deep) فرض کرده‌اند.

---

۴۸۶ **گزینه‌ی ۱** سخت

کلیدهای شیء فقط رشته (یا Symbol) هستند؛ پس شیءهای `a` و `b` موقع کلید شدن به رشته تبدیل می‌شوند و رشته‌ی هر دو `"[object Object]"` است. یعنی `labels[a]` و `labels[b]` **یک** خانه‌اند: اول `"first"` در آن نوشته می‌شود و بعد `"second"` جایش را می‌گیرد. برای همین `labels[a]` مقدار `second` را می‌دهد و فقط یک کلید وجود دارد. گزینه‌ی ۲ دو کلید جدا فرض کرده و گزینه‌ی ۳ فکر کرده نام متغیرها کلید می‌شوند.

---

۴۸۷ **گزینه‌ی ۳** سخت

کلید تکراری در object literal خطا نیست (رد گزینه‌ی ۲) و **آخرین مقدار برنده است**، پس `theme` برابر `"light"` می‌شود و شیء فقط دو کلید دارد (رد گزینه‌ی ۱). در `updated`، اول همه‌ی propertyهای `settings` ریخته می‌شوند و بعد `size` با `12 + 2` یعنی `14` بازنویسی می‌شود؛ بازنویسی جای کلید را عوض نمی‌کند، پس ترتیب کلیدها `['theme', 'size']` است. گزینه‌ی ۴ بازنویسی را نادیده گرفته است.

---

۴۸۸ **گزینه‌ی ۴** سخت

زیرنویس اسلاید ۴۰ می‌گوید «**تقریباً** همه چیز شیء است»، نه همه چیز: رشته یک primitive است و `typeof "hello"` برابر `"string"` است. گزینه‌ی ۱ درست است، چون primitiveها از طریق شیء پوشاننده‌ی موقت property و method دارند. تاریخ‌ها شیء‌اند (گزینه‌ی ۲) و توابع شیءهای خاصی هستند که `typeof` برایشان `"function"` می‌دهد (گزینه‌ی ۳).

---

۴۸۹ **گزینه‌ی ۲** سخت

`user[action]` همان `user["greet"]` یعنی متد `greet` است و با `()` بلافاصله صدا زده می‌شود؛ این هنوز «صدا زدن روی شیء» است، پس `this` همان `user` است و نام درست چاپ می‌شود (رد گزینه‌های ۱ و ۳). در خط آخر، `user.greet` **خودِ تابع** است و `typeof` آن `"function"` است (نه `"object"`؛ رد گزینه‌ی ۴)، ولی `user.greet()` تابع را **اجرا** می‌کند و رشته‌ی برگشتی را می‌دهد، پس `"string"`.

---

۴۹۰ **گزینه‌ی ۱** سخت

براکت داخل literal یعنی «**مقدارِ** این عبارت را کلید کن»: `[field]` کلید `"score"` می‌سازد (نه `"field"`) و `[id]` خودِ Symbol را کلید می‌کند. `Object.keys` فقط کلیدهای رشته‌ای را برمی‌گرداند و Symbolها را نشان نمی‌دهد: `['name', 'score']` (رد گزینه‌های ۲ و ۴). `item[id]` با همان Symbol مقدار `7` را پیدا می‌کند (رد گزینه‌ی ۳)، `item.field` دنبال کلیدی به نام `"field"` می‌گردد که نیست (`undefined`) و `item.score` برابر `20` است.

۱-۲۱ آرایه‌ها و متدهای push، pop، shift و unshift

۴۹۱ **گزینه‌ی ۱** آسان

`push` عدد ۴ را به **انتهای** آرایه اضافه می‌کند و آرایه `[1, 2, 3, 4]` می‌شود؛ عضو اندیس `0` هنوز `1` است. گزینه‌ی ۲ فکر کرده `push` به ابتدا اضافه می‌کند (کار `unshift`)، گزینه‌ی ۳ فرض کرده `numbers[0]` کل آرایه را می‌دهد، و گزینه‌ی ۴ همان تصور غلطِ «آرایه‌ی `const` تغییرناپذیر است» را دارد؛ `const` فقط انتساب دوباره را منع می‌کند.

۴۹۲ **گزینه‌ی ۲** آسان

طبق اسلاید ۴۵، اندیس از ۰ شروع می‌شود: `colors[0]` اولین عضو (`"red"`)، `colors[1]` دومین عضو (`"green"`) و `colors[2]` سومین عضو است. گزینه‌ی ۱ اندیس را از ۱ شمرده و گزینه‌ی ۳ اندیس ۱ را «آخرین» فرض کرده است. `undefined` فقط برای اندیسی است که عضوی ندارد، مثل `colors[3]` .

۴۹۳ **گزینه‌ی ۴** آسان

اسلاید ۴۶: «`shift` / `unshift` : remove/add at the start»؛ یعنی `shift` از ابتدا **حذف** می‌کند و `unshift` به ابتدا **اضافه** می‌کند. `push` به **انتها** اضافه می‌کند و `pop` از **انتها** حذف می‌کند. راه حفظ کردن: در هر جفت، کلمه‌ی بلندتر (`push` و `unshift`) آرایه را بلندتر می‌کند.

۴۹۴ **گزینه‌ی ۲** آسان

`pop` آخرین عضو را از آرایه‌ی **اصلی** حذف می‌کند و **همان عضو حذف‌شده** را برمی‌گرداند (`shift` هم برای ابتدای آرایه همین کار را می‌کند). «طول جدید» (گزینه‌ی ۱) خروجی `push` و `unshift` است. `pop` آرایه‌ی جدیدی نمی‌سازد و همان آرایه را درجا تغییر می‌دهد (رد گزینه‌ی ۳). `undefined` (گزینه‌ی ۴) فقط وقتی برمی‌گردد که آرایه خالی باشد.

۴۹۵ **گزینه‌ی ۳** آسان

آرایه در JavaScript نوع خاصی از **شیء** است، پس `typeof` برایش `"object"` برمی‌گرداند. نوعی به نام `"array"` یا `"list"` در خروجی `typeof` وجود ندارد (گزینه‌های ۱ و ۲) و نوع عضوها هم (`"number"`، گزینه‌ی ۴) ربطی به نوع خود آرایه ندارد. برای تشخیص آرایه باید از `Array.isArray(x)` استفاده کرد.

۴۹۶ **گزینه‌ی ۱** آسان

طبق اسلاید ۴۶، `for...of` روی **مقدارهای** آرایه حلقه می‌زند؛ پس در هر دور `f` یکی از عضوهاست و هر کدام در یک خط چاپ می‌شود. گزینه‌ی ۲ کار `for...in` است که **اندیس‌ها** را می‌دهد. گزینه‌ی ۳ فرض کرده کل آرایه یک بار چاپ می‌شود و گزینه‌ی ۴ فکر کرده تعریف `const` در حلقه مشکل دارد؛ در `for...of` هر دور یک متغیر تازه ساخته می‌شود، پس `const` مجاز است.

۴۹۷ **گزینه‌ی ۳** آسان

آرایه سه عضو دارد و اندیس‌هایش `0`، `1` و `2` است؛ در اندیس `3` عضوی نیست. خواندن اندیسِ بدون عضو در JavaScript **خطا نمی‌دهد** و `undefined` برمی‌گرداند (برخلاف زبان‌هایی مثل Java که خطای «خارج از محدوده» می‌دهند؛ رد گزینه‌ی ۴). گزینه‌ی ۱ اندیس را از ۱ شمرده و `null` (گزینه‌ی ۲) را JavaScript خودش برنمی‌گرداند.

۴۹۸ **گزینه‌ی ۴** متوسط

`push(4)` آرایه را `[2, 3, 4]` می‌کند و **طول جدید** یعنی `3` را برمی‌گرداند؛ `unshift(1)` آرایه را `[1, 2, 3, 4]` می‌کند و طول جدید `4` را می‌دهد. `pop()` آخرین عضو یعنی `4` را حذف می‌کند و **همان** را برمی‌گرداند و `shift()` اولین عضو یعنی `1` را. در پایان `[2, 3]` می‌ماند. گزینه‌ی ۱ فرض کرده این متدها آرایه را برمی‌گردانند، گزینه‌ی ۲ کار `pop` و `shift` را جابه‌جا کرده و گزینه‌ی ۳ خروجی متدهای «اضافه‌کن» را عضو اضافه‌شده پنداشته است.

۴۹۹ **گزینه‌ی ۲** متوسط

برخلاف Python، در JavaScript اندیس منفی از انتها نمی‌شمارد: `arr[-1]` دنبال کلیدی به نام `"-1"` می‌گردد که وجود ندارد، پس `undefined` می‌دهد (بدون خطا؛ رد گزینه‌های ۱ و ۳). `arr.length - 1` برابر `2` است و `arr[2]` آخرین عضو یعنی `7` است (رد گزینه‌ی ۴). متد `at` اندیس منفی را می‌فهمد و `at(-1)` هم `7` می‌دهد.

۵۰۰ **گزینه‌ی ۴** متوسط

دادن مقدار به اندیسی جلوتر از انتها مجاز است: `"e"` در اندیس `4` قرار می‌گیرد و خانه‌های `2` و `3` **خالی** (empty) می‌مانند. `length` همیشه «بزرگ‌ترین اندیس + ۱» است، پس `5` (رد گزینه‌ی ۱ که فقط عضوهای واقعی را شمرده). خواندن خانه‌ی خالی `undefined` می‌دهد، نه `null` (رد گزینه‌ی ۲)؛ و هیچ خطایی هم رخ نمی‌دهد (رد گزینه‌ی ۳).

۵۰۱ **گزینه‌ی ۱** متوسط

`indexOf` اندیس اولین عضو برابر را می‌دهد: `20` در اندیس `1` است (گزینه‌ی ۲ از ۱ شمرده). اگر عضو پیدا نشود، `indexOf` عدد `-1` برمی‌گرداند، نه `undefined` (رد گزینه‌ی ۳). `includes` مثل `===` مقایسه می‌کند و رشته‌ی `"10"` را به عدد تبدیل نمی‌کند، پس `false` (رد گزینه‌ی ۴).

۵۰۲ **گزینه‌ی ۱** متوسط

`slice(1, 3)` از اندیس `1` تا **قبل از** اندیس `3` را در آرایه‌ی **جدید** کپی می‌کند: `[2, 3]` (گزینه‌ی ۲ اندیس پایانی را هم حساب کرده) و `a` دست‌نخورده با طول `5` می‌ماند (رد گزینه‌ی ۳). اما `splice(1, 2)` از اندیس `1`، **دو** عضو را از خودِ `a` حذف می‌کند و آن‌ها را برمی‌گرداند؛ پس `a` می‌شود `[1, 4, 5]`. گزینه‌ی ۴ فرض کرده `splice` هم مثل `slice` آرایه را تغییر نمی‌دهد.

۵۰۳ **گزینه‌ی ۲** متوسط

`Array.isArray` تنها راه مطمئن تشخیص آرایه است: برای آرایه‌ی واقعی `true` و برای شیئی که فقط شبیه آرایه است (مثل `{ length: 2 }`) `false` می‌دهد. `typeof` برای آرایه `"object"` می‌دهد (رد گزینه‌ی ۱)؛ به همین دلیل `typeof []` و `typeof {}` هر دو `"object"` و **یکسان**‌اند (رد گزینه‌ی ۳). رشته هم با این‌که اندیس و `length` دارد آرایه نیست (رد گزینه‌ی ۴).

۵۰۴ **گزینه‌ی ۳** متوسط

`const b = a` آرایه‌ی تازه‌ای نمی‌سازد و `a` و `b` دو نام برای **یک** آرایه‌اند؛ پس `b.push(3)` طول همان آرایه را `3` می‌کند و `a === b` هم `true` است (رد گزینه‌های ۱ و ۴). اما `[...a]` یک آرایه‌ی **جدید** (کپی) ساخته بود که از این تغییر خبر ندارد و طولش `2` می‌ماند (رد گزینه‌ی ۲).

۵۰۵ **گزینه‌ی ۳** متوسط

`join("-")` یک **رشته** می‌سازد و آن لحظه آرایه هنوز `["a", "b", "c"]` است، پس `joined` برابر `a-b-c` می‌ماند و بعداً عوض نمی‌شود (رد گزینه‌ی ۲). `reverse()` آرایه را **درجا** برعکس می‌کند (رد گزینه‌ی ۱) و **همان آرایه** را برمی‌گرداند، نه یک کپی؛ پس `reversed === letters` برابر `true` است (رد گزینه‌ی ۴).

۵۰۶ **گزینه‌ی ۱** متوسط

`push` آرایه را درجا تغییر می‌دهد و **طول جدید** (`2`) را برمی‌گرداند، نه خود آرایه را. چون نتیجه دوباره در `list` ریخته شده، `list` حالا عدد `2` است و آرایه گم شده! گزینه‌ی ۲ همان تصور غلطِ «`push` آرایه را برمی‌گرداند» است و گزینه‌ی ۳ طول قبلی را گفته است. خطایی هم رخ نمی‌دهد، چون `list` با `let` تعریف شده و انتساب دوباره‌اش مجاز است.

۵۰۷ **گزینه‌ی ۳** متوسط

خط‌های ۲ تا ۴ فقط **محتوای** همان آرایه را تغییر می‌دهند (اضافه کردن، عوض کردن عضو و خالی کردن با `length = 0`) که برای آرایه‌ی `const` مجاز است. خط ۵ می‌خواهد متغیر `arr` را به آرایه‌ی **دیگری** وصل کند، یعنی انتساب دوباره؛ پس `TypeError: Assignment to constant variable.` رخ می‌دهد.

۵۰۸ **گزینه‌ی ۲** متوسط

`for...in` برای کلیدهای شیء ساخته شده و روی آرایه **اندیس‌ها** را می‌دهد (رد گزینه‌ی ۱)؛ و چون کلیدهای هر شیء رشته‌اند، اندیس‌ها هم به‌صورت **رشته** (`"0"` و `"1"`) می‌آیند، نه عدد (رد گزینه‌ی ۳). اما `for...of` طبق اسلاید ۴۶ خودِ **مقدارها** را می‌دهد: `x` و `y` (رد گزینه‌ی ۴).

۵۰۹ **گزینه‌ی ۲** سخت

`pop` و `shift` روی آرایه‌ی خالی خطا نمی‌دهند (رد گزینه‌ی ۱) و چون چیزی برای حذف نیست `undefined` برمی‌گردانند، نه `null` (رد گزینه‌ی ۴). طول هم منفی نمی‌شود و `0` می‌ماند (رد گزینه‌ی ۳). `push("a", "b")` هر دو مقدار را اضافه می‌کند و **طول جدید** یعنی `2` را برمی‌گرداند، نه آرایه را.

---

۵۱۰ **گزینه‌ی ۴** سخت

وقتی به `push` یک **آرایه** می‌دهیم، کل آن آرایه **یک عضو** می‌شود: `a` برابر `[1, 2, [3, 4]]` با طول `3` است و `a[2]` همان آرایه‌ی تودرتوی `[3, 4]` است. گزینه‌های ۱ و ۲ فرض کرده‌اند عضوهای `[3, 4]` تک‌تک اضافه می‌شوند (برای این کار باید نوشت `a.push(...[3, 4])`). گزینه‌ی ۳ فکر کرده `push` آرایه را برمی‌گرداند، در حالی که طول جدید را برمی‌گرداند.

---

۵۱۱ **گزینه‌ی ۱** سخت

گام‌به‌گام: بعد از دو `push` داریم `["Ali", "Sara"]`؛ `unshift("VIP")` آن را `["VIP", "Ali", "Sara"]` می‌کند. `shift()` عضو اول یعنی `"VIP"` را حذف می‌کند و برمی‌گرداند، پس `first` برابر `"VIP"` و آرایه `["Ali", "Sara"]` است؛ سپس `push(first)` آن را به انتها می‌برد: `["Ali", "Sara", "VIP"]`. حالا در اندیس `0` `"Ali"` است (رد گزینه‌ی ۴). گزینه‌ی ۲ اثر `shift` و `push` آخر را نادیده گرفته و گزینه‌ی ۳ فرض کرده `shift` عضو آخر را برمی‌دارد.

---

۵۱۲ **گزینه‌ی ۳** سخت

آرایه پنج عضو دارد؛ آرایه‌ی تودرتوی `[4, 5]` فقط **یک** عضو حساب می‌شود (رد گزینه‌ی ۲). `typeof` برای شیء، آرایه و حتی `null` (یک اشکال تاریخی JavaScript) `"object"` می‌دهد؛ نوعی به نام `"array"` یا `"null"` وجود ندارد (رد گزینه‌ی ۱). `mixed[3][1]` عدد `5` و `mixed[2].three` عدد `3` است و جمع دو عدد `8` می‌شود، نه چسباندن رشته‌ای (رد گزینه‌ی ۴).

---

۵۱۳ **گزینه‌ی ۴** سخت

`for...of` روی رشته هم کار می‌کند و حرف‌ها را یکی‌یکی می‌دهد (اسلاید ۴۶: «arrays, strings, etc.»). چون هر حرف **جلوی** `result` چسبانده می‌شود، `"J"` و بعد `"S"` نتیجه را `"SJ"` می‌کنند (رد گزینه‌ی ۱). با رسیدن به `"!"`، `break` از حلقه خارج می‌شود و `"!"` اضافه نمی‌شود (رد گزینه‌ی ۲). حلقه رشته‌ی اصلی را تغییر نمی‌دهد و طولش `3` است (رد گزینه‌ی ۳).

---

۵۱۴ **گزینه‌ی ۱** سخت

`splice(1, 0, "Feb")` یعنی «از اندیس ۱، **صفر** عضو حذف کن و `"Feb"` را همان‌جا بگذار»؛ پس `"Feb"` قبل از `"Mar"` درج می‌شود و چیزی حذف نمی‌شود (رد گزینه‌ی ۲ که یک عضو را جایگزین کرده). `splice` آرایه‌ی اصلی را تغییر می‌دهد (رد گزینه‌ی ۳) و همیشه **آرایه‌ای از عضوهای حذف‌شده** را برمی‌گرداند که این‌جا خالی است؛ طول جدید را برنمی‌گرداند (رد گزینه‌ی ۴).

---

۵۱۵ **گزینه‌ی ۴** سخت

`includes` و `indexOf` مثل `===` مقایسه می‌کنند و شیءها با **reference** مقایسه می‌شوند. `{ id: 1 }` در خط سوم یک شیء **تازه** است که فقط محتوایش شبیه عضو اول است، پس `false` (رد گزینه‌ی ۱). اما `target` **همان** شیء اندیس `0` است: `includes` برایش `true` می‌دهد (رد گزینه‌ی ۳) و `indexOf` اندیس `0` را (رد گزینه‌ی ۲ که از ۱ شمرده).

---

۵۱۶ **گزینه‌ی ۳** سخت

`indexOf` اندیس را برمی‌گرداند، نه `true` / `false`. `"root"` در اندیس `0` است و `0` **falsy** است، پس شرط اول برقرار نیست و `root is not admin` چاپ می‌شود (رد گزینه‌های ۱ و ۲). `"guest"` پیدا نمی‌شود و `indexOf` عدد `-1` را می‌دهد که **truthy** است، پس شرط دوم برقرار است و `guest is admin` هم چاپ می‌شود (رد گزینه‌ی ۴). راه درست: `admins.includes("root")` یا `admins.indexOf("root") !== -1`.

---

۵۱۷ **گزینه‌ی ۲** سخت

`unshift(1, 2)` و `push(5, 6)` مقدارها را **با همان ترتیبی که نوشته شده‌اند** اضافه می‌کنند، پس آرایه `[1, 2, 3, 4, 5, 6]` می‌شود و `join("")` حاصل `123456` را می‌دهد (رد گزینه‌های ۱ و ۴ که ترتیب را برعکس کرده‌اند). `shift()` عدد `1` و `pop()` عدد `6` را برمی‌گرداند و جمع دو **عدد** `7` است؛ گزینه‌ی ۳ آن‌ها را مثل رشته به هم چسبانده است.

۱-۲۲ متدهای findIndex، find، filter، map، forEach و reduce

۵۱۸ گزینه‌ی ۳ — آسان

اسلاید ۴۷: `map` «Transforms each element into a new array» و «Does not change the original array». گزینه‌ی ۲ توصیف `reduce` و گزینه‌ی ۴ توصیف `filter` (اسلاید ۴۶) است. گزینه‌ی ۱ برعکس است: `map` آرایه را تغییر نمی‌دهد و آرایه‌ی جدید برمی‌گرداند؛ «چیزی برنگرداندن» ویژگی `forEach` است.

۵۱۹ گزینه‌ی ۱ — آسان

اسلاید ۴۷: `reduce` «Combines all elements into a single value» و «Uses an accumulator + current value»؛ برای جمع، میانگین و ساختن شیء مفید است. گزینه‌ی ۲ از معنای لغوی reduce گمراه شده؛ `reduce` نه عضو تکراری حذف می‌کند و نه آرایه را تغییر می‌دهد. گزینه‌ی ۳ کار `pop` و گزینه‌ی ۴ توصیف `forEach` است.

۵۲۰ گزینه‌ی ۲ — آسان

`map` برای هر عضو callback را اجرا می‌کند و خروجی‌ها را در یک آرایه‌ی **جدید** می‌گذارد: `[10, 20, 30]`؛ آرایه‌ی اصلی `nums` دست‌نخورده می‌ماند (رد گزینه‌ی ۱). `map` همه را در یک عدد جمع نمی‌کند؛ `60` کار `reduce` است (رد گزینه‌ی ۳). گزینه‌ی ۴ رفتار `forEach` (برگرداندن `undefined`) را به `map` نسبت داده و تازه فرض کرده آرایه‌ی اصلی عوض شده است.

۵۲۱ گزینه‌ی ۴ — آسان

`filter` عضوهایی را نگه می‌دارد که callback برایشان `true` برگرداند: `[2, 4]`. `map` خودِ جواب‌های callback را در آرایه می‌گذارد: `[false, true, false, true]`. `find` فقط **اولین** عضو جور را برمی‌گرداند (عدد `2`، نه آرایه) و `forEach` همیشه `undefined` برمی‌گرداند.

۵۲۲ گزینه‌ی ۱ — آسان

`find` **اولین عضوی** را که شرط را دارد برمی‌گرداند و همان‌جا متوقف می‌شود: `12`. آرایه‌ی همه‌ی عضوهای جور (`[12, 130]`) خروجی `filter` است، `1` (اندیسِ همان عضو) خروجی `findIndex` است و `true` (آیا عضو جوری هست؟) خروجی `some`.

۵۲۳ گزینه‌ی ۴ — آسان

هیچ عضوی بزرگ‌تر از ۱۰۰ نیست و `findIndex` وقتی چیزی پیدا نکند `-1` برمی‌گرداند (مثل `indexOf`). جوابِ «پیدا نشد» در `find` است، نه `undefined`. `findIndex` را این متدها هیچ‌وقت برنمی‌گردانند و `null` طول آرایه است، نه یک اندیس معتبر.

۵۲۴ گزینه‌ی ۲ — آسان

`forEach` فقط callback را برای هر عضو اجرا می‌کند و **همیشه** `undefined` برمی‌گرداند؛ مقدار `n * 2` که callback برمی‌گرداند نادیده گرفته می‌شود. خروجی `map` است (گزینه‌ی ۱). `forEach` خودِ آرایه را هم برنمی‌گرداند (گزینه‌ی ۳) و آخرین جواب callback را هم نه (گزینه‌ی ۴).

۵۲۵ گزینه‌ی ۳ — آسان

`sort` عضوها را **درجا** مرتب می‌کند و همان آرایه را برمی‌گرداند؛ پس آرایه‌ی اصلی تغییر می‌کند. `map` و `filter` آرایه‌ی **جدید** می‌سازند و `reduce` یک مقدار تازه برمی‌گرداند؛ هیچ‌کدام به آرایه‌ی اصلی دست نمی‌زنند.

۵۲۶ گزینه‌ی ۳ — آسان

`acc` از مقدار اولیه‌ی `0` شروع می‌شود و هر عضو به آن اضافه می‌شود: `0 + 1 + 2 + 3 + 4 = 10`؛ یک **عدد**. وقتی به `"1234"` دست می‌آمد که مقدار اولیه `""` بود (چسباندن رشته‌ای). `24` حاصل ضرب است (با `*` و مقدار اولیه‌ی `1`). `[1, 3, 6, 10]` مقدارهای میانیِ `acc` است؛ `reduce` فقط **آخرین** مقدار را برمی‌گرداند.

۵۲۷ گزینه‌ی ۱ — آسان

callback به ترتیب `(element, index, array)` را می‌گیرد؛ پس `x` خودِ عضو و `i` اندیس آن است که از `0` شروع می‌شود. گزینه‌ی ۲ ترتیب پارامترها را برعکس فرض کرده، گزینه‌ی ۳ اندیس را از ۱ شمرده و گزینه‌ی ۴ فکر کرده اندیس در همه‌ی دورها ثابت است.

۵۲۸ گزینه‌ی ۴ متوسط

`push(5)` و `pop()` همدیگر را خنثی می‌کنند و `unshift(0)` و `shift()` هم؛ پس `nums` به `[1, 2, 3, 4]` برمی‌گردد (گزینه‌ی ۱ اثر `shift` را جا انداخته). `map` و `filter` آرایه‌ی **جدید** می‌سازند و `nums` را عوض نمی‌کنند (گزینه‌ی ۲ فرض کرده `map` خودِ `nums` را دوبرابر کرده). `forEach` چهار عدد را چاپ می‌کند، `find` اولین عضو بزرگ‌تر از ۲ یعنی `3` را می‌دهد و `findIndex` اندیس آن یعنی `2` را. گزینه‌ی ۳ `find` را با `filter` اشتباه گرفته است.

۵۲۹ گزینه‌ی ۳ متوسط

`map` آرایه‌ی **جدید** `[2, 4, 6, 8]` را می‌سازد و در `doubled` می‌گذارد؛ خودِ `nums` همان `[1, 2, 3, 4]` می‌ماند (اسلاید ۴۷: «Does not change the original array»). بقیه درست‌اند: `push` طول جدید (`5`) را برمی‌گرداند، `shift` عضو حذف‌شده (`0`) را، و `forEach` همیشه `undefined` را.

۵۳۰ گزینه‌ی ۱ متوسط

`s => s.score` از هر شیء فقط **عدد** نمره را برمی‌گرداند، نه یک شیء (رد گزینه‌ی ۳). `reduce` با مقدار اولیه‌ی **عددی** `0` عددها را جمع می‌کند: `157` (گزینه‌ی ۴ فرض کرده عددها به هم چسبانده می‌شوند، که فقط با مقدار اولیه‌ی `""` رخ می‌داد). تقسیم در JavaScript اعشاری است، پس `157 / 2` برابر `78.5` است، نه `78` (رد گزینه‌ی ۲).

۵۳۱ گزینه‌ی ۲ متوسط

بعد از `=>` آکولاد آمده، پس این یک **بدنه‌ی بلوکی** است و `return` خودکار ندارد؛ `n * 2` حساب می‌شود ولی برگردانده نمی‌شود و callback در هر دور `undefined` برمی‌گرداند. `map` همچنان آرایه‌ای **هم‌طول** می‌سازد: سه `undefined`. گزینه‌ی ۱ با `n => n * 2` یا نوشتن `return` درست می‌شد. خود `map` همیشه آرایه برمی‌گرداند (رد گزینه‌ی ۳) و خانه‌های `undefined` را حذف نمی‌کند (رد گزینه‌ی ۴).

۵۳۲ گزینه‌ی ۳ متوسط

در `a` پرانتز باعث می‌شود `{ id }` یک **object literal** باشد (شکل کوتاه `{ id: id }`)، پس `[{ id: 1 }, { id: 2 }]` ساخته می‌شود. در `b` آکولادِ بعد از `=>` **بدنه‌ی تابع** است؛ داخلش فقط عبارت `id` حساب می‌شود و چیزی برنمی‌گردد، پس دو `undefined` (رد گزینه‌های ۱ و ۲). این بدنه از نظر نوشتاری معتبر است و `SyntaxError` نمی‌دهد (رد گزینه‌ی ۴)؛ برخلاف `{ ...s, passed: ... }` بدون پرانتز در اسلاید ۴۸ که واقعاً `SyntaxError` می‌داد.

۵۳۳ گزینه‌ی ۱ متوسط

`filter` عضوهایی را نگه می‌دارد که callback برایشان مقدار **truthy** برگرداند؛ `0`، `""` و `null` falsy‌اند و حذف می‌شوند (رد گزینه‌ی ۳). عضوهای نگه‌داشته‌شده **همان‌طور که هستند** در آرایه‌ی جدید می‌روند و به `true` تبدیل نمی‌شوند (رد گزینه‌ی ۴). آرایه‌ی اصلی `filter` را تغییر نمی‌دهد، پس طول `values` همان `6` است (رد گزینه‌ی ۲).

۵۳۴ گزینه‌ی ۲ متوسط

مقدار اولیه داده نشده، پس `acc` از **عضو اول** (`10`) شروع می‌شود و پیمایش از عضو دوم؛ callback فقط **دو** بار اجرا می‌شود: `(10, 20)` که `30` برمی‌گرداند و `(30, 30)` که `60`. گزینه‌ی ۱ فرض کرده مقدار اولیه‌ی پیش‌فرض `0` است و گزینه‌ی ۳ فکر کرده از `acc` `undefined` شروع می‌شود. گزینه‌ی ۴ فراموش کرده `acc` در دور دوم **خروجی دور قبل** (`30`) است، نه عضو قبلی.

۵۳۵ گزینه‌ی ۴ متوسط

`acc` از شیء خالی `{}` شروع می‌شود. بار اولی که یک رأی دیده می‌شود، `acc[v]` برابر `undefined` است و `(undefined || 0) + 1` می‌شود `1`؛ دفعه‌های بعد همان عدد یکی زیاد می‌شود. چون `acc` در هر دور برگردانده می‌شود، شمارش جمع می‌شود و حاصل `{ yes: 3, no: 1 }` است. گزینه‌ی ۱ انباشته شدن را نادیده گرفته، گزینه‌ی ۲ فقط تعداد کل را شمرده و گزینه‌ی ۳ فهرست کلیدهاست، نه خروجی این `reduce`.

۵۳۶ گزینه‌ی ۳ متوسط

`total` برابر `18 + 15 + 20 + 11 = 64` است و میانگین `64 / 4 = 16` (گزینه‌ی ۱ جمع را به جای میانگین چاپ کرده). در `reduce` دوم، `max` از عضو اول (`18`) شروع می‌شود و هر بار عدد بزرگ‌تر نگه داشته می‌شود: `18`، `18`، `20`، `20`؛ پس `best` برابر `20` است. گزینه‌ی ۲ فکر کرده `reduce` بدون مقدار اولیه همان عضو اول را برمی‌گرداند و گزینه‌ی ۴ آخرین عضو را خروجی فرض کرده است.

۵۳۷ **گزینه‌ی ۲** متوسط

`map` آرایه‌ی **جدیدی** از جواب‌های callback برمی‌گرداند و `forEach` همیشه `undefined` ؛ گزینه‌ی ۱ دقیقاً برعکس است. هیچ‌کدام از این دو متد خودشان آرایه‌ی اصلی را تغییر نمی‌دهند (رد گزینه‌ی ۳). طول خروجی `map` **همیشه** برابر طول آرایه‌ی اصلی است؛ حذف عضوها کار `filter` است (رد گزینه‌ی ۴).

۵۳۸ **گزینه‌ی ۴** متوسط

هر متد روی خروجی متد قبلی کار می‌کند: `filter` عدد `40` را حذف می‌کند و `[100, 250, 300]` می‌ماند؛ `map` آن‌ها را نصف می‌کند: `[50, 125, 150]` ؛ و `reduce` جمعشان را می‌دهد: `325` . گزینه‌ی ۱ اثر `filter` را نادیده گرفته (`20` اضافه شده)، گزینه‌ی ۲ اثر `map` را، و گزینه‌ی ۳ قبل از `reduce` متوقف شده است.

۵۳۹ **گزینه‌ی ۱** متوسط

`find` **اولین** عضو جور را برمی‌گرداند و متوقف می‌شود: `Sara` (نه آخرین عضو جور؛ رد گزینه‌ی ۲). `filter` **همه‌ی** عضوهای جور را می‌دهد: Sara و Reza، پس طول `2` (رد گزینه‌ی ۳). اندیس از ۰ شمرده می‌شود، پس `Reza` در اندیس `2` است (گزینه‌های ۳ و ۴ از ۱ شمرده‌اند).

۵۴۰ **گزینه‌ی ۳** متوسط

`sort()` بدون تابع مقایسه عضوها را به **رشته** تبدیل می‌کند و حرف‌به‌حرف مقایسه می‌کند: `"1"` < `"10"` < `"100"` < `"9"` (همان میم اسلاید ۲۱)؛ پس ترتیب عددی گزینه‌های ۱ و ۴ غلط است. `sort` آرایه را **درجا** مرتب می‌کند و **همان آرایه** را برمی‌گرداند، نه یک کپی؛ پس `nums === sorted` برابر `true` است (رد گزینه‌ی ۲).

۵۴۱ **گزینه‌ی ۱** متوسط

`some` می‌پرسد «آیا **حداقل یک** عضو جور است؟»؛ `16` کمتر از ۱۸ است، پس `true` (رد گزینه‌ی ۲). `every` می‌پرسد «آیا **همه** جورند؟»؛ `21` و `30` کمتر از ۱۸ نیستند، پس `false` (رد گزینه‌ی ۳). همه‌ی سن‌ها بیشتر از ۱۰ هستند، پس آخری `true` است (رد گزینه‌ی ۴).

۵۴۲ **گزینه‌ی ۲** متوسط

`map` callback را **همان لحظه** و پشت سر هم برای همه‌ی عضوها اجرا می‌کند و بعد آرایه‌ی جدید را برمی‌گرداند؛ پس اول `map 1` و `map 2` و بعد `done [2, 4]` چاپ می‌شود. گزینه‌ی ۱ فکر کرده callback بعداً (مثل `setTimeout`) اجرا می‌شود، گزینه‌ی ۳ خط آخر را چند بار اجرا کرده و گزینه‌ی ۴ رفتار `forEach` را به `map` نسبت داده است.

۵۴۳ **گزینه‌ی ۱** متوسط

با مقدار اولیه‌ی `0` ، `acc` عدد است و جمع عددی `10` می‌دهد. با `""` ، `acc` رشته است و `"" + 1` می‌شود `"1"` ، پس در آخر `1234` چاپ می‌شود. `[]` هم در `+` به رشته‌ی خالی تبدیل می‌شود و همان `1234` را می‌دهد. مقدار اولیه‌ی `1` خودش هم در جمع حساب می‌شود و حاصل `11` است.

۵۴۴ **گزینه‌ی ۲** سخت

`map` به callback **سه** ورودی می‌دهد: `(element, index, array)` ؛ پس `parseInt` به ترتیب با `("1", 0)` ، `("2", 1)` و `("3", 2)` صدا زده می‌شود. مبنای `0` یعنی پیش‌فرض، پس `1` ؛ مبنای `1` نامعتبر است، پس `NaN` ؛ و در مبنای ۲ رقم `3` وجود ندارد، پس باز `NaN` . گزینه‌ی ۱ فرض کرده فقط خودِ عضو به callback داده می‌شود؛ راه درست `.map(s => parseInt(s))` یا `.map(Number)` است.

۵۴۵ **گزینه‌ی ۳** سخت

`break` داخل حلقه‌ی `for...of` (خط ۳) کاملاً مجاز است. اما در خط ۶، `break` داخل یک **تابع** (callback) است که خودش حلقه‌ای ندارد؛ پس `SyntaxError: Illegal break statement` رخ می‌دهد. این خطا هنگام خواندن کل فایل کشف می‌شود و هیچ خطی اجرا نمی‌شود. `forEach` را نمی‌توان با `break` متوقف کرد؛ برای توقف زودهنگام از `for...of` یا `some` / `find` استفاده کن.

۵۴۶ **گزینه‌ی ۴** سخت

با مقدار اولیه، `reduce` روی آرایه‌ی خالی callback را اصلاً صدا نمی‌زند و همان مقدار اولیه (`0`) را برمی‌گرداند. بدون مقدار اولیه، عضو اولی وجود ندارد که `acc` از آن شروع شود؛ پس `TypeError: Reduce of empty array with no initial value` رخ می‌دهد (رد گزینه‌های ۱ و ۲). خط اول قبل از خطا اجرا شده و `0` چاپ شده است (رد گزینه‌ی ۳).

۵۴۷ **گزینه‌ی ۳** سخت

`map` یک آرایه‌ی **جدید** می‌سازد (`updated !== items` ؛ رد گزینه‌ی ۲)، ولی عضوهای آن **همان شیءهای قبلی** هستند، چون callback خودِ `item` (یک reference) را برگردانده است. callback هم همان شیءها را تغییر داده، پس `items[0].qty` برابر `10` است (رد گزینه‌های ۱ و ۴). «`map` آرایه را تغییر نمی‌دهد» درباره‌ی خودِ آرایه است؛ برای دست نزدن به شیءها باید مثل اسلاید ۴۸ شیء جدید ساخت: `({ ...item, qty: item.qty * 10 })` .

۵۴۸ **گزینه‌ی ۲** سخت

نمره‌ی Bob برابر `62` است و `62 >= 70` نادرست است، پس `passed` او `false` است (رد گزینه‌ی ۳). spread برای هر دانشجو یک شیء **جدید** می‌سازد و `passed` فقط به آن شیء جدید اضافه می‌شود؛ شیءهای `students` این کلید را ندارند و `undefined` می‌دهند (رد گزینه‌ی ۱). `map` هیچ عضوی را حذف نمی‌کند و `results` هم دو عضو دارد (رد گزینه‌ی ۴).

۵۴۹ **گزینه‌ی ۴** سخت

در خط اول `acc` از رشته‌ی خالی شروع می‌شود و «رشته + عدد» یعنی چسباندن: `"1"` ، `"12"` و `"123"` (رد گزینه‌های ۱ و ۳). در خط دوم `acc` از عدد `10` شروع می‌شود و جمع عددی است: `10 + 1 + 2 + 3 = 16` . گزینه‌ی ۲ فرض کرده رفتار خط اول در خط دوم هم تکرار می‌شود؛ نوع `acc` را **مقدار اولیه** تعیین می‌کند.

۵۵۰ **گزینه‌ی ۲** سخت

`find` **خودِ** عضو (reference همان شیء) را برمی‌گرداند، نه یک کپی؛ پس تغییر `p.stock` همان شیءِ داخل آرایه را عوض می‌کند و `10` چاپ می‌شود (رد گزینه‌های ۱ و ۴). وقتی هیچ عضوی جور نباشد، `find` مقدار `undefined` برمی‌گرداند؛ `-1` جواب `findIndex` است (رد گزینه‌ی ۳) و `null` را هیچ‌کدام برنمی‌گردانند.

۵۵۱ **گزینه‌ی ۳** سخت

`arr.slice()` یک **کپی** می‌سازد و `sort` فقط همان کپی را مرتب می‌کند؛ پس `arr` همان `3,1,2` می‌ماند. سه گزینه‌ی دیگر متدهای **mutating** را مستقیم روی `arr` صدا می‌زنند: `sort` آن را `1,2,3` می‌کند، `reverse` آن را `2,1,3` و `splice(0, 1)` عضو اول را حذف می‌کند و `1,2` می‌ماند.

۵۵۲ **گزینه‌ی ۴** سخت

فقط تابع مقایسه‌ی `(a, b) => a - b` مقایسه‌ی **عددی صعودی** انجام می‌دهد. `sort()` بدون تابع مقایسه رشته‌ای مقایسه می‌کند و همان ترتیب میم اسلاید ۲۱ یعنی `[1, 100000, 21, 30, 4]` را نگه می‌دارد. `b - a` نزولی مرتب می‌کند و `reverse` اصلاً مرتب نمی‌کند، فقط ترتیب فعلی را برعکس می‌کند: `[4, 30, 21, 100000, 1]` .

۵۵۳ **گزینه‌ی ۱** سخت

`map` همیشه آرایه‌ای **هم‌طول** می‌سازد و برای هر عضو، **هر چه callback برگرداند** را می‌گذارد. برای عددهای فرد شرط برقرار نیست و callback به انتها می‌رسد بدون این‌که چیزی برگرداند، پس `undefined` . گزینه‌ی ۲ کار ترکیب `filter` و `map` است؛ `map` نمی‌تواند عضو حذف کند. گزینه‌ی ۳ فرض کرده عضوهای بی‌جواب دست‌نخورده می‌مانند و گزینه‌ی ۴ فکر کرده جوابِ شرط برگردانده می‌شود.

۵۵۴ **گزینه‌ی ۴** سخت

پارامتر سوم callback (`a`) **همان آرایه‌ی** `arr` است (reference)، پس `a[i] = n * n` خانه‌های خودِ `arr` را درجا عوض می‌کند و `arr` می‌شود `[1, 4, 9]` (رد گزینه‌های ۱ و ۳). `forEach` خودش آرایه را تغییر نمی‌دهد، ولی callback ما می‌تواند. خروجی `forEach` هم همیشه `undefined` است (رد گزینه‌ی ۲).

۵۵۵ **گزینه‌ی ۱** سخت

`return` داخل callback فقط **همان یک فراخوانی** را تمام می‌کند (مثل `continue`)؛ پس `12` رد می‌شود ولی `forEach` به عضو بعدی یعنی `3` می‌رود و آن را هم اضافه می‌کند. گزینه‌ی ۲ فرض کرده `return` کل `forEach` را متوقف می‌کند (مثل `break`)، گزینه‌ی ۳ شرط را برعکس خوانده و گزینه‌ی ۴ اثر `return` را نادیده گرفته است.

۵۵۶ **گزینه‌ی ۲** سخت

دور اول درست اجرا می‌شود (`acc` همان `{}` است)، ولی callback چیزی **برنمی‌گرداند**؛ پس `acc` در دور دوم `undefined` است و خواندن `acc[w]` در خط ۳ خطای `TypeError: Cannot read properties of undefined (reading 'b')` می‌دهد. به خط ۵ نمی‌رسیم و «بدون خطا» هم درست نیست. راه درست: بعد از خط ۳ بنویس `return acc;` .

۱-۲۳ Destructuring (تجزیه‌ی ساختار) و Spread

۵۵۷ **گزینه‌ی ۲** آسان

اسلاید ۴۲ می‌گوید «Extract values from arrays or objects easily destructuring»؛ یعنی مقدارها را از آرایه یا شیء بیرون می‌کشد و در متغیرهای جدا قرار می‌دهد. گزینه‌ی ۱ از غلط تایپی عنوان اسلاید («Destructing») و شباهتش به destroy آمده، در حالی که شیء اصلی دست‌نخورده می‌ماند. تبدیل به JSON کار `JSON.stringify` است (رد گزینه‌ی ۳) و destructuring اصلاً شیء جدیدی نمی‌سازد، چه برسد به کپی عمیق (رد گزینه‌ی ۴).

۵۵۸ **گزینه‌ی ۳** آسان

اسلاید ۴۲: «Uses {} for objects, [] for arrays». درست مثل literalها: شیء با آکولاد ساخته و با آکولاد هم باز می‌شود (`const { first, age } = person`) و آرایه با کروشه (`const [primary, secondary] = colors`). گزینه‌ی ۱ جای این دو را برعکس کرده و پرانتز `()` (گزینه‌های ۲ و ۴) اصلاً الگوی destructuring نمی‌سازد.

۵۵۹ **گزینه‌ی ۴** آسان

شیء خاصیت‌های نام‌دار دارد، پس JavaScript برای هر متغیر دنبال خاصیتِ هم‌نامش می‌گردد و ترتیب نوشتن مهم نیست. اعضای آرایه نام ندارند، پس متغیر اول عضو اول را می‌گیرد، دومی عضو دوم را و الی آخر. گزینه‌ی ۱ فقط درباره‌ی آرایه درست است، گزینه‌ی ۲ دقیقاً برعکسِ واقعیت است و در گزینه‌ی ۳، نام متغیرهای آرایه کاملاً دلخواه است.

۵۶۰ **گزینه‌ی ۱** آسان

در destructuring شیء تطبیق بر اساس **نام** است، نه ترتیب: `age` مقدار `person.age` (یعنی `30`) و `first` مقدار `person.first` (یعنی `"Alice"`) را می‌گیرد و چاپ به ترتیب `first, age` انجام می‌شود. گزینه‌ی ۲ ترتیب چاپ را با ترتیب الگو اشتباه گرفته، گزینه‌ی ۳ فکر کرده جابه‌جا نوشتن نام‌ها تطبیق را خراب می‌کند و گزینه‌ی ۴ مثل آرایه بر اساس جایگاه فکر کرده است (`age` اولین مقدار و `first` دومین مقدار را بگیرد).

۵۶۱ **گزینه‌ی ۲** آسان

این همان کد اسلاید ۴۲ است: در destructuring آرایه، متغیر اول عضو اول (`"red"`) و متغیر دوم عضو دوم (`"green"`) را می‌گیرد. destructuring فقط مقدارها را در متغیرها می‌ریزد و آرایه را تغییر نمی‌دهد، پس طول آن همچنان `3` است. گزینه‌ی ۳ فکر کرده اعضای برداشته‌شده از آرایه حذف می‌شوند، گزینه‌ی ۱ شمارش را از ۱ index شروع کرده و گزینه‌ی ۴ تصور کرده متغیر دوم آخرین عضو را می‌گیرد.

۵۶۲ **گزینه‌ی ۴** متوسط

هر جای خالیِ بین کاماها یعنی «از این جایگاه رد شو». جایگاه ۰ (`10`) رد می‌شود، `second` جایگاه ۱ یعنی `20` را می‌گیرد، جایگاه ۲ (`30`) هم رد می‌شود و `fourth` باید جایگاه ۳ را بگیرد که وجود ندارد؛ پس `undefined` می‌شود، بدون هیچ خطایی. گزینه‌ی ۱ جای خالی دوم را نادیده گرفته، گزینه‌ی ۲ جای خالی اول را نشمرده و گزینه‌ی ۳ فکر کرده عضوِ ناموجود خطا می‌دهد.

۵۶۳ **گزینه‌ی ۱** متوسط

`first: firstName` یعنی «خاصیت `first` را در متغیر جدید `firstName` بریز»، پس `firstName` برابر `"Alice"` است. `person` خاصیتی به نام `middle` ندارد، پس مقدار پیش‌فرض `"N/A"` به کار می‌رود. گزینه‌ی ۲ مقدار پیش‌فرض را نادیده گرفته، گزینه‌ی ۳ فکر کرده نام خاصیت (نه مقدارش) در متغیر ریخته می‌شود و گزینه‌ی ۴ تصور کرده خاصیتِ ناموجود خطا می‌دهد؛ در حالی که فقط `undefined` است و پیش‌فرض جایش را می‌گیرد.

---

۵۶۴ **گزینه‌ی ۳** متوسط

خط ۲ کاملاً معتبر است: خاصیت `first` در متغیر **جدید** `firstName` و خاصیت `age` در متغیر `age` ریخته می‌شود؛ خط ۳ هم `Alice 30` را چاپ می‌کند. اما بعد از تغییر نام، متغیری به نام `first` ساخته نشده، پس خط ۴ خطای `ReferenceError: first is not defined` می‌دهد. گزینه‌ی «خط ۲» فکر کرده ترکیب تغییر نام با یک نام معمولی مجاز نیست و گزینه‌ی «هیچ خطایی رخ نمی‌دهد» تصور کرده هر دو نامِ `first` و `firstName` ساخته می‌شوند.

---

۵۶۵ **گزینه‌ی ۲** متوسط

مقدار پیش‌فرض فقط وقتی به کار می‌رود که مقدار برابر `undefined` باشد. `a` صریحاً `undefined` است، پس `1` می‌گیرد؛ `c` اصلاً وجود ندارد (یعنی `undefined`)، پس `3` می‌گیرد؛ ولی `b` برابر `null` است که یک مقدار واقعی است و پیش‌فرض را فعال نمی‌کند. گزینه‌ی ۱ با `null` مثل `undefined` رفتار کرده، گزینه‌ی ۳ فکر کرده `undefined` صریح جلوی پیش‌فرض را می‌گیرد و گزینه‌ی ۴ همه‌ی پیش‌فرض‌ها را نادیده گرفته است.

---

۵۶۶ **گزینه‌ی ۴** سخت

`0`، `""` و `false` مقدارهای falsy هستند (در شرط مثل `false` رفتار می‌کنند)، ولی `undefined` **نیستند**؛ پس پیش‌فرض برای آن‌ها به کار نمی‌رود. عضو چهارم وجود ندارد (`undefined`)، پس فقط `w` مقدار `5` می‌گیرد. گزینه‌های ۱ و ۳ پیش‌فرض را با رفتار عملگر `||` اشتباه گرفته‌اند (که هر مقدار falsy را جایگزین می‌کند) و گزینه‌ی ۲ پیش‌فرضِ `w` را فراموش کرده است.

---

۵۶۷ **گزینه‌ی ۱** متوسط

`null` یک مقدار واقعی است و پیش‌فرض را فعال نمی‌کند؛ پس `x` برابر `null` می‌شود، نه `10`، و گزاره‌ی ۱ نادرست است. گزاره‌ی ۲ درست است، چون جای خالی بین دو کاما عضو `2` را رد می‌کند و `b` عضو سوم را می‌گیرد. گزاره‌ی ۳ درست است، چون رشته iterable است و حرف اولش در `first` می‌نشیند. گزاره‌ی ۴ هم درست است: `a` جدا شده و `rest` شامل `b` و `c` است.

---

۵۶۸ **گزینه‌ی ۱** آسان

`head` عضو اول (`1`) را می‌گیرد و `...tail` (rest) **همه‌ی** اعضای باقی‌مانده را در یک **آرایه‌ی جدید** جمع می‌کند: `[ 2, 3, 4 ]`. گزینه‌ی ۲ فکر کرده rest فقط عضو بعدی را می‌گیرد، گزینه‌ی ۴ تصور کرده rest آخرین عضو را برمی‌دارد و گزینه‌ی ۳ متغیر اول را هم آرایه فرض کرده است.

---

۵۶۹ **گزینه‌ی ۲** متوسط

`id` جدا بیرون کشیده می‌شود و rest در الگوی شیء، **بقیه‌ی** خاصیت‌ها را در یک **شیء جدید** جمع می‌کند؛ پس `details` فقط `title` و `price` را دارد. گزینه‌ی ۱ فکر کرده rest کل شیء را کپی می‌کند (اما `id` قبلاً برداشته شده)، گزینه‌ی ۳ تصور کرده rest همیشه آرایه می‌سازد (فقط در الگوی آرایه این‌طور است) و گزینه‌ی ۴ فکر کرده خاصیتی به نام `details` جست‌وجو می‌شود.

---

۵۷۰ **گزینه‌ی ۳** سخت

rest باید **آخرین** مورد الگو باشد؛ `[...rest, last]` خطای `SyntaxError: Rest element must be last element` است. خطای نحوی هنگام خواندن (parse) کل فایل و **قبل از اجرای اولین خط** پیدا می‌شود، پس حتی `start` هم چاپ نمی‌شود. گزینه‌ی ۲ آن را با یک خطای زمان اجرا (runtime) اشتباه گرفته که خط‌های قبلش اجرا می‌شوند. گزینه‌های ۱ و ۴ فکر کرده‌اند JavaScript چنین الگویی را می‌پذیرد.

---

۵۷۱ **گزینه‌ی ۱** آسان

سمت راست اول ارزیابی می‌شود و آرایه‌ی موقت `[2, 1]` ساخته می‌شود؛ بعد این آرایه در `a` و `b` باز می‌شود. نتیجه جابه‌جایی (swap) مقدارهاست: `2 1`. گزینه‌ی ۳ فکر کرده انتساب‌ها یکی‌یکی انجام می‌شوند (اول `a = b` و بعد `b = a`) که فقط در جابه‌جاییِ اشتباهِ بدون متغیر کمکی پیش می‌آید. destructuring در انتساب به متغیرهای موجود (بدون `let`) کاملاً مجاز است، پس گزینه‌ی ۴ غلط است؛ گزینه‌ی ۲ هم اثر خط سوم را نادیده گرفته است.

۵۷۲ **گزینه‌ی ۴** متوسط

`city` داخل `user.address` است، پس الگو هم باید تودرتو باشد: `{ address: { city } }` یعنی «برو داخل `address` و `city` را بیرون بیاور». گزینه‌ی ۱ دنبال `user.city` می‌گردد که وجود ندارد و `undefined` چاپ می‌کند. نقطه داخل الگو مجاز نیست، پس گزینه‌ی ۲ خطای `SyntaxError` می‌دهد. گزینه‌ی ۳ الگوی آرایه را روی یک شیء معمولی به کار برده که iterable نیست و خطای `TypeError` می‌دهد.

۵۷۳ **گزینه‌ی ۳** سخت

`profile: { ... }` یعنی «داخل `user.profile` برو». آنجا `name` در متغیر جدید `userName` (یعنی `"Reza"`) ریخته می‌شود و چون `city` وجود ندارد، پیش‌فرض `"Tehran"` را می‌گیرد. `id: userId` هم `1` را در `userId` می‌ریزد. گزینه‌ی ۱ پیش‌فرض را نادیده گرفته، گزینه‌ی ۲ فکر کرده `name` باید در سطح اول `user` باشد و گزینه‌ی ۴ تصور کرده چون `profile` و `id` متغیر نشده‌اند خطا رخ می‌دهد، در حالی که در کد اصلاً از آن دو نام استفاده نشده است.

۵۷۴ **گزینه‌ی ۱** سخت

در فراخوانی اول `name` داده شده و `role` پیش‌فرض `"user"` را می‌گیرد. فراخوانی دوم آرگومانی ندارد، پس کل پارامتر پیش‌فرض `{}` را می‌گیرد و هر دو خاصیت پیش‌فرض می‌شوند؛ به همین دلیل `TypeError` رخ نمی‌دهد (رد گزینه‌ی ۳). در فراخوانی سوم `role` برابر `null` است که پیش‌فرض را فعال نمی‌کند (رد گزینه‌ی ۲). گزینه‌ی ۴ فکر کرده پیش‌فرض خاصیت‌ها فقط وقتی کار می‌کند که اصلاً آرگومانی داده نشود.

۵۷۵ **گزینه‌ی ۳** سخت

فراخوانی اول `Book` را چاپ می‌کند. در فراخوانی دوم شیء خالی است؛ باز کردن آن مشکلی ندارد و فقط `title` برابر `undefined` می‌شود (رد گزینه‌ی ۲). در فراخوانی سوم پارامتر `undefined` است و باز کردن `undefined` ممکن نیست:
`TypeError: Cannot destructure property 'title' of 'undefined'`. گزینه‌ی ۱ این خطا را ندیده (برای جلوگیری از آن باید می‌نوشتیم `{ title } = {}`) و گزینه‌ی ۴ نوع خطا را اشتباه گفته است.

۵۷۶ **گزینه‌ی ۲** سخت

`theme` وجود دارد و مقدارش `null` است؛ پیش‌فرض فقط برای `undefined` به کار می‌رود، پس `null` چاپ می‌شود (رد گزینه‌های ۱ و ۳). در خط چهارم می‌خواهیم `null` را باز کنیم؛ `null` هیچ خاصیتی ندارد و این کار خطای
`TypeError: Cannot destructure property 'color' of 'settings.theme' as it is null` می‌دهد. گزینه‌ی ۴ فکر کرده باز کردن `null` هم مثل یک شیء خالی، فقط `undefined` می‌دهد.

۵۷۷ **گزینه‌ی ۲** آسان

rest در **الگوی destructuring** (سمت چپ `=`) یا در پارامترهای تابع می‌آید و باقی‌مانده را جمع می‌کند؛ در گزینه‌ی ۲، `others` همه‌ی اعضا به‌جز اولی را در یک آرایه جمع می‌کند. در گزینه‌های ۱ و ۴، `...` داخل یک آرایه یا شیء جدید (سمت راست `=`) آمده و اعضا را پخش می‌کند و در گزینه‌ی ۳ اعضای آرایه را به شکل آرگومان‌های جدا به تابع می‌دهد؛ هر سه spread هستند.

۵۷۸ **گزینه‌ی ۱** متوسط

`Math.max(...nums)` با spread همان `Math.max(5, 1, 9)` است و `9` می‌دهد. اما `Math.max(nums)` فقط **یک** آرگومان دارد که خودِ آرایه است؛ `Math.max` آن را به عدد تبدیل می‌کند و آرایه‌ی سه‌عضوی عدد معتبری نمی‌شود، پس حاصل `NaN` است. گزینه‌ی ۲ فکر کرده `Math.max` خودش آرایه را باز می‌کند، گزینه‌ی ۳ نقش spread را برعکس فهمیده و گزینه‌ی ۴ تصور کرده دادن آرایه خطا می‌دهد.

۵۷۹ گزینه‌ی ۴ متوسط

در object literal اگر یک کلید دو بار مقدار بگیرد، **آخرین** مقدار برنده است. در `x` اول `b: 2` از `base` کپی می‌شود و بعد `b: 3` آن را بازنویسی می‌کند، پس `3` . در `y` اول `b: 3` نوشته می‌شود و بعد spread، مقدار `2` را از `base` روی آن می‌ریزد، پس `2` . گزینه‌ی ۱ فکر کرده مقدار صریح همیشه برنده است، گزینه‌ی ۲ تصور کرده مقدار `base` همیشه برنده است و گزینه‌ی ۳ ترتیب را برعکس خوانده است.

۵۸۰ گزینه‌ی ۳ متوسط

`const b = a;` آرایه‌ی جدیدی نمی‌سازد؛ `b` فقط نام دیگری برای **همان** آرایه است، پس `push` روی `a` در `b` هم دیده می‌شود و طولش `3` است. `[...a]` با spread یک آرایه‌ی **جدید** با اعضای آن لحظه (`1` و `2`) می‌سازد که به `a` وابسته نیست، پس طولش `2` می‌ماند. گزینه‌ی ۱ فکر کرده `b` هم کپی است، گزینه‌ی ۲ فکر کرده spread هم فقط ارجاع را کپی می‌کند و گزینه‌ی ۴ این دو را برعکس کرده است.

۵۸۱ گزینه‌ی ۴ سخت

spread یک کپی **سطحی** (shallow) می‌سازد: `clone` شیء جدیدی است، پس تغییر `clone.name` روی `original` اثری ندارد و نام همان `Ali` می‌ماند. اما خاصیت `tags` در هر دو شیء به **همان** آرایه اشاره می‌کند؛ `push` روی آرایه‌ی مشترک انجام می‌شود و طول آن در `original` هم `2` است. گزینه‌ی ۱ کپی را عمیق فرض کرده و گزینه‌های ۲ و ۳ فکر کرده‌اند `clone` اصلاً شیء جدیدی نیست.

۵۸۲ گزینه‌ی ۲ سخت

`{ ...s, passed: ... }` برای هر دانشجو یک شیء **جدید** می‌سازد که خاصیت‌های `s` به‌علاوه‌ی `passed` را دارد. نمره‌ی Bob برابر `62` است، پس `passed` برابر `false` است؛ ولی شیء اصلی Bob تغییری نکرده و خاصیت `passed` ندارد، پس `undefined` . چون شیءها جدیدند، `results[0] === students[0]` برابر `false` است. گزینه‌های ۱ و ۴ فکر کرده‌اند spread شیء اصلی را تغییر می‌دهد و گزینه‌ی ۳ نتیجه‌ی مقایسه‌ی `62 >= 70` را اشتباه حساب کرده است.

۱-۲۴ کلاس‌ها (Classes) در JavaScript

۵۸۳ گزینه‌ی ۲ آسان

`new User("Kermit", 22)` یک نمونه می‌سازد و constructor با `this` برابر همان نمونه اجرا می‌شود، پس `kermit.name` برابر `"Kermit"` است. در `kermit.greet()` شیء قبل از نقطه `kermit` است، پس `this.name` همان `"Kermit"` است. گزینه‌ی ۱ فکر کرده خاصیت‌ها در شیء ذخیره نمی‌شوند، گزینه‌ی ۳ رشته‌ی `"name"` را با مقدار خاصیت اشتباه گرفته و گزینه‌ی ۴ تصور کرده متدِ بدون کلمه‌ی `function` خطا می‌دهد.

۵۸۴ گزینه‌ی ۳ آسان

اسلاید ۳۹ دقیقاً همین را می‌گوید: «Introduced in ES6 as a cleaner way to create objects» و «Classes define a blueprint for creating objects». کلاس‌ها در ES6 (نه ES5) آمدند و کلمه‌ی کلیدی آن‌ها `class` است (رد گزینه‌ی ۱). نقشه برای ساختن **هر تعداد** شیء است (رد گزینه‌ی ۲). نمونه‌ها با `new` از روی کلاس ساخته می‌شوند و خود کلاس نمونه نیست (رد گزینه‌ی ۴).

۵۸۵ گزینه‌ی ۱ آسان

هر شیئی که با `new` از روی یک کلاس ساخته شود، یک **instance** (نمونه) از آن کلاس است؛ مثل خانه‌ای که از روی نقشه ساخته شده. کلاس فرزند با `class X extends User` تعریف می‌شود (رد گزینه‌ی ۲)، constructor متد سازنده‌ی داخل کلاس است (رد گزینه‌ی ۳) و متد، تابعی مثل `greet` است (رد گزینه‌ی ۴).

۵۸۶ گزینه‌ی ۴ آسان

نمونه‌ها با کلمه‌ی کلیدی `new` ساخته می‌شوند: `new User("Kermit")` شیء تازه را می‌سازد و constructor را اجرا می‌کند. `create` در JavaScript کلمه‌ی کلیدی نیست، `this` و `class` هم در این جایگاه معنایی ندارند؛ هر سه گزینه‌ی دیگر خطای `SyntaxError` می‌دهند.

۵۸۷ **گزینه‌ی ۲** آسان

constructor (سازنده) با **هر** `new` یک بار و خودکار اجرا می‌شود و کارش مقداردهی اولیه به خاصیت‌های نمونه‌ی تازه است. تعریف کلاس به‌تنهایی آن را اجرا نمی‌کند (رد گزینه‌ی ۱)، لازم نیست دستی صدایش بزنیم (رد گزینه‌ی ۳) و صدا زدن متدهای دیگر هم آن را دوباره اجرا نمی‌کند (رد گزینه‌ی ۴).

۵۸۸ **گزینه‌ی ۳** آسان

کلاس در پشت صحنه یک تابع ویژه است (syntactic sugar روی prototypeها)، پس `typeof User` برابر `"function"` است؛ در JavaScript نوعی به نام `"class"` نداریم (رد گزینه‌ی ۱). نمونه‌ی ساخته‌شده یک شیء معمولی است: `"object"` (رد گزینه‌ی ۴). `kermit` با `new User` ساخته شده، پس `instanceof User` برابر `true` است. گزینه‌ی ۲ کلاس را هم شیء فرض کرده است.

۵۸۹ **گزینه‌ی ۴** سخت

متدهای کلاس فقط **یک بار** در `User.prototype` ذخیره می‌شوند و همه‌ی نمونه‌ها از طریق prototype به همان نسخه دسترسی دارند؛ برای همین `Object.keys(kermit)` فقط `name` و `age` را نشان می‌دهد و گزینه‌ی ۴ نادرست است. سه گزینه‌ی دیگر قدم‌های `new` هستند: ساختن شیء خالی، اجرای constructor با `this` برابر آن شیء و برگرداندن خودکار شیء.

۵۹۰ **گزینه‌ی ۱** متوسط

`Object.keys` فقط کلیدهای **خودِ** شیء را برمی‌گرداند. `name` و `age` در constructor روی خود نمونه ساخته شده‌اند، ولی `greet` (و `constructor`) در `User.prototype` هستند و بین همه‌ی نمونه‌ها مشترک‌اند. گزینه‌ی ۲ فکر کرده متد در هر نمونه کپی می‌شود، گزینه‌ی ۳ اعضای prototype را با خاصیت‌های نمونه اشتباه گرفته و گزینه‌ی ۴ تصور کرده خاصیت‌های ساخته‌شده با `this` جزو شیء حساب نمی‌شوند.

۵۹۱ **گزینه‌ی ۲** متوسط

`greet` فقط یک بار در `User.prototype` تعریف شده و هر دو نمونه به **همان** تابع می‌رسند، پس `a.greet === b.greet` برابر `true` است. اما `a` و `b` دو شیء جدا هستند (هر `new` شیء تازه‌ای می‌سازد)، پس `a === b` برابر `false` است (رد گزینه‌ی ۱). `typeof` یک نمونه `"object"` است، نه `"function"` (رد گزینه‌ی ۳)، و چون `greet` خاصیتِ خودِ `a` نیست، در `Object.keys(a)` نمی‌آید (رد گزینه‌ی ۴).

۵۹۲ **گزینه‌ی ۴** متوسط

هر `new Counter()` شیء جدیدی با `count` **خودش** می‌سازد. `a.inc()` دو بار روی `a` اجرا می‌شود (`this` برابر `a`)، پس `a.count` برابر `2` است؛ `b.inc()` یک بار، پس `b.count` برابر `1` . متد `inc` مشترک است ولی داده‌ها جدا هستند. گزینه‌ی ۱ فکر کرده `count` بین نمونه‌ها مشترک است، گزینه‌ی ۲ تعداد فراخوانی‌ها را اشتباه شمرده و گزینه‌ی ۳ تصور کرده هر فراخوانی از صفر شروع می‌شود.

۵۹۳ **گزینه‌ی ۱** متوسط

در JavaScript تعداد آرگومان‌ها لازم نیست با پارامترها برابر باشد؛ پارامتری که مقدار نگیرد `undefined` است. پس `this.age = age` مقدار `undefined` را در `age` می‌ریزد و `greet` هم بدون مشکل `Hello, Kermit` چاپ می‌کند. گزینه‌ی ۴ فکر کرده کمبود آرگومان خطا می‌دهد و گزینه‌های ۲ و ۳ مقدار پیش‌فرض را `null` یا `0` تصور کرده‌اند.

۵۹۴ **گزینه‌ی ۳** متوسط

خط‌های ۶ و ۷ درست‌اند و `Ali` چاپ می‌شود. در خط ۸ کلاس **بدون** `new` صدا زده شده و JavaScript خطای `TypeError: Class constructor User cannot be invoked without 'new'` می‌دهد؛ پس اجرا به خط ۹ نمی‌رسد. برخلاف تابع‌های معمولی، کلاس را نمی‌شود مثل یک تابع ساده صدا زد، پس گزینه‌ی «هیچ خطایی رخ نمی‌دهد» غلط است.

۵۹۵ **گزینه‌ی ۲** سخت

`makeUser` یک function declaration است و کاملاً hoist می‌شود، پس صدا زدنش در خط ۱ قبل از تعریف مشکلی ندارد و خط ۲ `Ali` را چاپ می‌کند (رد گزینه‌ی خط ۱). اعلان کلاس هم hoist می‌شود ولی مقداردهی نمی‌شود و تا خط ۴ در TDZ است؛ پس خط ۳ خطای `ReferenceError: Cannot access 'Person' before initialization` می‌دهد. خود تعریف کلاس در خط ۴ خطایی ندارد و گزینه‌ی «هیچ خطایی» هم فرض کرده کلاس مثل function declaration کامل hoist می‌شود.

---

۵۹۶ **گزینه‌ی ۱** متوسط

در بدنه‌ی کلاس متدها پشت سر هم و **بدون کاما** نوشته می‌شوند؛ کاما در گزینه‌ی ۱ خطای `SyntaxError: Unexpected token ','` می‌دهد. گزینه‌ی ۲ یک object literal است که در آن کاما بین اعضا **لازم** است. گزینه‌ی ۳ همان کلاس بدون کاماست و گزینه‌ی ۴ یک constructor معمولی است؛ هر دو درست‌اند.

---

۵۹۷ **گزینه‌ی ۴** متوسط

متد `static` به **خود کلاس** تعلق دارد، پس `MathUtil.square(4)` برابر `16` است. اما روی نمونه‌ها وجود ندارد: `m.square` برابر `undefined` است و صدا زدن `undefined` خطای `TypeError: m.square is not a function` می‌دهد. گزینه‌ی ۱ فکر کرده متدهای static به نمونه‌ها هم می‌رسند، گزینه‌ی ۲ فراموش کرده که صدا زدنِ `undefined` خطاست و گزینه‌ی ۳ تصور کرده خط اول هم خطا می‌دهد.

---

۵۹۸ **گزینه‌ی ۳** متوسط

`items = [];` یک class field است و **هر نمونه آرایه‌ی خودش** را می‌گیرد. `add` در پایان `this` (همان `c1`) را برمی‌گرداند، پس `.add("book")` دوباره روی `c1` اجرا می‌شود و `c1` دو آیتم دارد؛ `c2` دست‌نخورده می‌ماند. گزینه‌ی ۱ فکر کرده آرایه بین نمونه‌ها مشترک است، گزینه‌ی ۲ زنجیره‌ی دوم را نادیده گرفته و گزینه‌ی ۴ تصور کرده زنجیر کردن متدها (chaining) خطا می‌دهد؛ در حالی که چون `add` خودِ شیء را برمی‌گرداند، کاملاً مجاز است.

---

۵۹۹ **گزینه‌ی ۲** سخت

در constructor کلاس فرزند (کلاسی که `extends` دارد)، تا `super(...)` صدا زده نشده، شیء هنوز ساخته نشده و `this` **قابل استفاده نیست**. خط ۸ قبل از `super` از `this` استفاده کرده و خطای `ReferenceError: Must call super constructor in derived class before accessing 'this' ...` می‌دهد؛ اجرا به خط ۹ نمی‌رسد. خط ۳ (constructor والد) اصلاً اجرا نمی‌شود و ترتیب این دو خط مهم است، پس گزینه‌ی «هیچ خطایی» هم غلط است.

---

۶۰۰ **گزینه‌ی ۳** سخت

`Dog` constructor ندارد، پس constructor پیش‌فرضش آرگومان `"Rex"` را به constructor والد می‌دهد و `name` درست مقدار می‌گیرد (رد گزینه‌ی ۲). `speak` در `Dog` بازنویسی (override) شده و با `super.speak()` نسخه‌ی والد را صدا می‌زند، سپس `(woof)` را اضافه می‌کند (رد گزینه‌ی ۱). یک `Dog` هم‌زمان نمونه‌ای از `Animal` هم هست، چون `instanceof` کل زنجیره‌ی ارث‌بری را بررسی می‌کند (رد گزینه‌ی ۴).

---

۶۰۱ **گزینه‌ی ۱** سخت

کلاس فرزندی که `constructor` ندارد، یک constructor پیش‌فرض می‌گیرد که همه‌ی آرگومان‌ها را با `super(...)` به والد می‌دهد؛ پس `name` برابر `"Tom"` می‌شود (رد گزینه‌ی ۲). چون `Cat` از `Animal` ارث می‌برد، `tom instanceof Animal` برابر `true` است (رد گزینه‌ی ۳). خطای `ReferenceError` فقط وقتی رخ می‌دهد که خودمان constructor بنویسیم و قبل از `super` از `this` استفاده کنیم یا اصلاً `super` را صدا نزنیم (رد گزینه‌ی ۴).

---

۶۰۲ **گزینه‌ی ۴** سخت

`c` با `new Circle()` ساخته شده، پس نمونه‌ی `Circle` است؛ چون `Circle` از `Shape` ارث می‌برد، نمونه‌ی `c` هم `Shape` هست (هر دایره یک شکل است). اما یک `Shape` معمولی دایره نیست، پس سومی `false` است. گزینه‌ی ۱ زنجیره‌ی ارث‌بری را نادیده گرفته، گزینه‌ی ۲ رابطه را دوطرفه فرض کرده و گزینه‌ی ۳ فکر کرده نمونه فقط به کلاس والد تعلق دارد.

۱-۲۵ کلمه‌ی کلیدی this

۶۰۳ **گزینه‌ی ۳** آسان

اسلاید ۴۳: «Refers to the object that is calling the function»؛ یعنی شیئی که تابع را صدا زده است (مثلاً در `obj.say()` ، شیء `obj`). `this` به خود تابع اشاره نمی‌کند (رد گزینه‌ی ۱). فقط در صدا زدن ساده و بدون strict mode در مرورگر برابر `window` می‌شود، نه «همیشه» (رد گزینه‌ی ۲). ترتیب ساخته شدن اشیاء هم هیچ ربطی به آن ندارد (رد گزینه‌ی ۴).

۶۰۴ **گزینه‌ی ۱** آسان

اسلاید ۴۳: «The value of this depends on how a function is called, not where it's defined» و اسلاید ۴۴: «Value depends on call context, not declaration». گزینه‌ی ۲ دقیقاً برعکس است؛ «محل نوشته شدن» فقط برای arrow functionها تعیین‌کننده است. نام تابع، تعداد پارامترها و مقدار برگشتی هم هیچ اثری روی `this` ندارند (رد گزینه‌های ۳ و ۴).

۶۰۵ **گزینه‌ی ۴** آسان

اسلاید ۴۴: «In global scope: window (browser) or undefined (strict mode)». دقیق‌ترش این است که تابعی که به‌صورت ساده و بدون شیء صدا زده شود، در مرورگر `window` و در strict mode مقدار `undefined` می‌گیرد (بالاترین سطح خود تگ `<script>` همیشه `window` است). `null` و `console` هیچ‌کدام مقدار پیش‌فرض `this` نیستند (رد گزینه‌های ۱ و ۳) و `document` فقط یکی از خاصیت‌های `window` است (رد گزینه‌ی ۲).

۶۰۶ **گزینه‌ی ۲** آسان

در `obj.say()` شیء `obj` قبل از نقطه آمده، پس داخل `say` مقدار `this` برابر `obj` است و `this.name` همان `"Robert"` است. گزینه‌ی ۱ جواب حالتی است که متد از شیء جدا شود (`const f = obj.say; f();`)، گزینه‌ی ۳ نام متد را با `this.name` اشتباه گرفته و در فراخوانی متد به این شکل هیچ خطایی رخ نمی‌دهد (رد گزینه‌ی ۴).

۶۰۷ **گزینه‌ی ۴** آسان

اسلاید ۳۶: «arrows inherit; `this` Functions have their own `this` » و اسلاید ۴۴ آن را lexical this می‌نامد: arrow function `this` را از محدوده‌ای که در آن نوشته شده می‌گیرد. `this` arrow همیشه `undefined` نیست (مثلاً داخل یک متد، همان شیء است؛ رد گزینه‌ی ۱)، آرگومان‌ها روی آن اثری ندارند (رد گزینه‌ی ۲) و «شیء قبل از نقطه» قانونِ تابع‌های معمولی است، نه arrow (رد گزینه‌ی ۳).

۶۰۸ **گزینه‌ی ۱** آسان

اسلاید ۴۳: «In classes, `this` points to the created instance». وقتی `new User("Kermit")` اجرا می‌شود، شیء تازه‌ای ساخته می‌شود و `constructor` با `this` برابر همان شیء اجرا می‌شود؛ برای همین `this.name = name` روی نمونه خاصیت می‌سازد. `this` به خود کلاس (گزینه‌ی ۲)، شیء سراسری (گزینه‌ی ۳) یا کلاس والد (گزینه‌ی ۴) اشاره نمی‌کند؛ کلاس والد فقط از طریق `super` در دسترس است.

۶۰۹ **گزینه‌ی ۲** متوسط

`obj.say()` یک فراخوانی متد است و `Robert` چاپ می‌کند. `f()` صدا زدن ساده است و تابع اتصالش به `obj` را ندارد؛ بدون strict mode، `this` برابر شیء سراسری Node.js است که خاصیت `name` ندارد، پس `undefined` چاپ می‌شود. گزینه‌ی ۱ فکر کرده تابع `obj` را به خاطر می‌سپارد، گزینه‌ی ۳ جواب strict mode است و گزینه‌ی ۴ (خط خالی) جواب مرورگر است که در آن `window.name` برابر `""` است.

۶۱۰ **گزینه‌ی ۴** متوسط

`obj.say()` مثل قبل `Robert` چاپ می‌کند. در strict mode، صدا زدن ساده‌ی `f()` به `this` مقدار `undefined` می‌دهد و خواندن `undefined.name` خطای `TypeError: Cannot read properties of undefined (reading 'name')` می‌دهد. گزینه‌ی ۱ رفتار حالت عادی (بدون strict) است، گزینه‌ی ۲ فکر کرده `this` حفظ می‌شود و گزینه‌ی ۳ فراموش کرده که خط `obj.say()` قبل از خطا اجرا شده و خروجی‌اش باقی می‌ماند.

۶۱۱ **گزینه‌ی ۲** سخت

در `f()` شیئی قبل از نقطه نیست، پس در یک اسکریپت معمولی مرورگر `this` برابر `window` است. برخلاف Node.js، `window` واقعاً خاصیتی به نام `name` دارد (نام پنجره) که معمولاً رشته‌ی خالی `""` است؛ پس `console.log` یک خط خالی چاپ می‌کند و بعد `end`. گزینه‌ی ۱ جواب Node.js است، گزینه‌ی ۳ جواب strict mode و گزینه‌ی ۴ فکر کرده تابع اتصالش به `obj` را حفظ می‌کند.

۶۱۲ **گزینه‌ی ۱** آسان

در صدا زدن ساده هیچ شیئی برای `this` وجود ندارد؛ بدون strict mode شیء سراسری جای آن را می‌گیرد، ولی در strict mode `this` همان `undefined` می‌ماند. در `obj.f()` مقدار `this` برابر `obj` است (رد گزینه‌ی ۲)، در `f.call(obj)` صریحاً `obj` داده شده (رد گزینه‌ی ۳) و با `new` شیء تازه‌ای ساخته می‌شود که `this` به آن اشاره می‌کند (رد گزینه‌ی ۴).

۶۱۳ **گزینه‌ی ۳** متوسط

قانون متد می‌گوید `this` شیئی است که **دقیقاً قبل از آخرین نقطه** آمده؛ در `user.profile.show()` این شیء `user.profile` است، پس `this.name` برابر `"inner"` است. گزینه‌ی ۱ فکر کرده بیرونی‌ترین شیء (`user`) تعیین‌کننده است، گزینه‌ی ۲ تصور کرده در اشیای تودرتو `this` گم می‌شود و در این فراخوانی هیچ خطایی رخ نمی‌دهد (رد گزینه‌ی ۴).

۶۱۴ **گزینه‌ی ۳** متوسط

`this` به **نحوه‌ی صدا زدن** بستگی دارد، نه به جایی که تابع تعریف شده. در `robert.say.call(anna)` مقدار `this` صریحاً `anna` است. در خط بعد همان تابع در `anna.say` هم ذخیره می‌شود و `anna.say()` یک فراخوانی متد روی `anna` است. پس هر دو بار `I am Anna`. گزینه‌های ۱، ۲ و ۴ فرض کرده‌اند تابع برای همیشه به `robert`، یعنی جایی که تعریف شده، وابسته است.

۶۱۵ **گزینه‌ی ۲** متوسط

در بالاترین سطح یک فایل CommonJS در Node.js، `this` برابر `module.exports` است که در ابتدا یک شیء خالی است و `{}` چاپ می‌شود. گزینه‌های ۱ و ۴ رفتار ES module (که در آن `this` در بالاترین سطح `undefined` است) را در نظر گرفته‌اند و گزینه‌ی ۳ نفهمیده که این شیء خالی دقیقاً همان `module.exports` است.

۶۱۶ **گزینه‌ی ۱** سخت

در strict mode، تابعی که به‌صورت ساده صدا زده شود `this` برابر `undefined` می‌گیرد؛ پس خط اول `true` است (رد گزینه‌های ۳ و ۴). اما `this` در **بالاترین سطح** یک `<script>` معمولی، حتی در strict mode، همان `window` است؛ پس خط دوم هم `true` است (رد گزینه‌ی ۲). «undefined (strict mode)» در اسلاید ۴۴ مربوط به همین صدا زدن ساده‌ی تابع است.

۶۱۷ **گزینه‌ی ۴** متوسط

در مرورگر، `var` سراسری خاصیتِ `window` می‌شود (`window.city`). `show()` صدا زدن ساده است و بدون strict، `this` برابر `window` است؛ پس `Tabriz` چاپ می‌شود. `{ city: "Shiraz", show }` همان `show: show` است و `obj.show()` همان تابع را به شکل متد صدا می‌زند، پس `Shiraz`. گزینه‌ی ۱ جواب Node.js است (آنجا `var` سراسری روی شیء سراسری نمی‌نشیند)، گزینه‌ی ۲ فکر کرده تابع بعد از قرار گرفتن در `obj` به آن وابسته می‌شود و گزینه‌ی ۳ جواب strict mode است.

۶۱۸ **گزینه‌ی ۱** آسان

طبق اسلاید ۴۴، `this` را می‌شود دستی با `call`، `apply` یا `bind` تعیین کرد. `call` و `apply` تابع را همان لحظه با `this` برابر `obj` اجرا می‌کنند و `bind` تابع جدیدی می‌سازد که `()` بعد از آن، همان تابع جدید را اجرا می‌کند؛ پس هر سه `Robert` چاپ می‌کنند. گزینه‌ی ۲ فکر کرده `bind` اثری ندارد، گزینه‌ی ۴ پرانتزِ اجرای تابعِ ساخته‌شده با `bind` را نادیده گرفته و گزینه‌ی ۳ تصور کرده متدِ جداشده را دیگر نمی‌شود به شیء وصل کرد.

۶۱۹ **گزینه‌ی ۳** آسان

`call(thisArg, a, b)` و `apply(thisArg, [a, b])` تابع را بلافاصله اجرا می‌کنند و `bind(thisArg)` فقط یک تابع جدید با `this` دائمی می‌سازد. گزینه‌ی ۱ نقش `bind` و `call` را جابه‌جا کرده و گزینه‌ی ۲ نقش `apply` و `call` را (Call با Comma، Apply با Array). گزینه‌ی ۴ برعکس است: این متدها `this` تابع‌های معمولی را تعیین می‌کنند و روی arrow function `this` اصلاً اثری ندارند.

۶۲۰ **گزینه‌ی ۴** متوسط

`call` آرگومان‌ها را جداجدا و `apply` داخل یک آرایه می‌گیرد؛ هر دو `this` را برابر `robert` می‌کنند، پس `Hi, Robert!` و `Hello, Robert?` . گزینه‌ی ۱ فکر کرده تابعِ بیرون از شیء نمی‌تواند `this` بگیرد. گزینه‌ی ۳ فرض کرده `apply` آرایه را به‌عنوان **یک** آرگومان می‌دهد (این رفتارِ `call` با آرایه است: `greeting` کل آرایه می‌شود و `punctuation` برابر `undefined`). گزینه‌ی ۲ تصور کرده دادن آرایه به `apply` خطاست.

---

۶۲۱ **گزینه‌ی ۲** متوسط

`bind` تابع جدیدی با `this` دائمی برابر `obj` می‌سازد و آن را در `f` می‌ریزیم؛ پس `f()` چاپ می‌کند `Robert` . در گزینه‌ی ۱، `call` تابع را **همان لحظه** اجرا می‌کند (`Robert` چاپ می‌شود) و مقدار برگشتی‌اش (`undefined`) در `f` می‌نشیند، پس `f()` خطای `TypeError: f is not a function` می‌دهد. در گزینه‌ی ۳ تابعِ ساخته‌شده با `bind` دور ریخته می‌شود و خود `f` تغییری نمی‌کند، پس مثل گزینه‌ی ۴ در Node.js مقدار `undefined` چاپ می‌شود.

---

۶۲۲ **گزینه‌ی ۳** سخت

`show.bind(a)` تابعی می‌سازد که `this` آن **برای همیشه** `a` است. `bind` دوباره (`f.bind(b)`) و `call` (`f.call(b)`) نمی‌توانند `this` یک تابعِ bindشده را عوض کنند؛ پس هر دو خط `A` چاپ می‌کنند. گزینه‌های ۱، ۲ و ۴ فرض کرده‌اند آخرین `bind` یا `call` برنده است.

---

۶۲۳ **گزینه‌ی ۲** متوسط

`bind` تابع را اجرا **نمی‌کند**؛ فقط تابع جدیدی با `this` ثابت برمی‌گرداند که بعداً باید صدایش بزنی (`f.bind(obj)()`). اجرای فوری کار `call` و `apply` است؛ پس گزینه‌ی ۲ نادرست است. گزینه‌ی ۱ قانون متد (شیء قبل از نقطه)، گزینه‌ی ۳ lexical this و گزینه‌ی ۴ تفاوت `apply` با `call` است که همگی درست‌اند.

---

۶۲۴ **گزینه‌ی ۱** سخت

با `new` ، شیء تازه‌ای ساخته می‌شود و `this` به آن اشاره می‌کند، پس `c.model` برابر `Pride` است. `Car("Tiba")` بدون `new` یک صدا زدن ساده است: تابع چیزی برنمی‌گرداند (`d` برابر `undefined`) و چون strict نیست، `this` شیء سراسری است و `model` به‌اشتباه روی `globalThis` ساخته می‌شود. گزینه‌ی ۲ فکر کرده بدون `new` هم شیء ساخته می‌شود، گزینه‌ی ۳ تابع معمولی را با کلاس (که بدون `new` خطا می‌دهد) اشتباه گرفته و گزینه‌ی ۴ اثر جانبی روی شیء سراسری را ندیده است.

---

۶۲۵ **گزینه‌ی ۴** متوسط

arrow function `this` را از محدوده‌ای که در آن **نوشته شده** می‌گیرد (lexical this) و هیچ روشی آن را عوض نمی‌کند. گزینه‌ی ۱ دام معروف است: arrow به‌عنوان متد، `this` بیرون از شیء را می‌گیرد، نه خود شیء را. گزینه‌ی ۲ غلط است، چون arrow constructor نیست و `new` روی آن خطای `TypeError` می‌دهد. گزینه‌ی ۳ قانونِ تابع‌های معمولی است، نه arrow.

---

۶۲۶ **گزینه‌ی ۲** سخت

`regular` یک متد معمولی است و در `counter.regular()` مقدار `this` برابر `counter` است، پس `10` . اما arrow یک function است و `this` را از بیرونِ object literal می‌گیرد؛ در بالاترین سطح فایل CommonJS، `this` برابر `module.exports` (یعنی `{}`) است که `count` ندارد، پس `undefined` . گزینه‌ی ۱ دام اصلی است (فکر کرده arrow هم شیء را می‌شناسد). گزینه‌ی ۳ متد معمولی را هم بی‌`this` فرض کرده و خطایی هم رخ نمی‌دهد، چون `this` در این‌جا `undefined` نیست (رد گزینه‌ی ۴).

---

۶۲۷ **گزینه‌ی ۳** سخت

arrow function `this` را از بیرونِ object literal می‌گیرد؛ در بالاترین سطح یک `<script>` مرورگر، `this` برابر `window` است. در مرورگر خاصیت `var count = 99` را می‌سازد، پس `window.count` برابر `99` است. گزینه‌ی ۱ فکر کرده arrow شیء `counter` را می‌شناسد، گزینه‌ی ۲ جواب Node.js است (آنجا `this` بیرونی `module.exports` است و `var` روی آن نمی‌نشیند) و خطایی هم رخ نمی‌دهد (رد گزینه‌ی ۴).

---

۶۲۸ **گزینه‌ی ۱** متوسط

`team.list()` یک فراخوانی متد است، پس داخل `list` مقدار `this` برابر `team` است. callback ِ`map` یک arrow function است و `this` را از `list` به ارث می‌برد؛ پس `this.name` داخل آن هم `"Blue"` است. گزینه‌ی ۲ رفتار یک `function` معمولی را به arrow نسبت داده، گزینه‌ی ۳ جواب همان حالت در مرورگر است (`window.name` برابر `""`) و گزینه‌ی ۴ فکر کرده `this.members` در دسترس نیست.

---

۶۲۹ **گزینه‌ی ۴** متوسط

داخل `show` مقدار `this` برابر `team` است، ولی `inner` یک تابع **معمولی** است که به‌صورت ساده (`inner()`) صدا زده شده؛ پس `this` خودش را دارد که بدون strict، شیء سراسری Node.js است و `name` ندارد: `undefined`. تابع معمولیِ تودرتو `this` متد بیرونی را به ارث نمی‌برد (رد گزینه‌ی ۱؛ این رفتار arrow است). گزینه‌ی ۲ جواب strict mode است و گزینه‌ی ۳ هم غلط است، چون خواندن خاصیتِ ناموجود هیچ‌وقت `ReferenceError` نمی‌دهد.

---

۶۳۰ **گزینه‌ی ۲** سخت

`this` ِarrow function همان `this` محل تعریفش است: بالاترین سطح فایل CommonJS، یعنی `module.exports`. `call` و `bind` روی `this` ِarrow هیچ اثری ندارند؛ پس `arrow.call(obj)` باز هم `module.exports` را برمی‌گرداند (نه `obj`) و خط اول `false` است، ولی خط دوم `true`. گزینه‌های ۱ و ۳ فکر کرده‌اند `call` می‌تواند `this` ِarrow را عوض کند و گزینه‌ی ۴ بالاترین سطح Node.js را چیزی جز `module.exports` فرض کرده است.

---

۶۳۱ **گزینه‌ی ۳** سخت

در خط اول فقط **خود تابع** به `setTimeout` داده می‌شود و بعداً بدون `timer.` اجرا می‌شود؛ `this` دیگر `timer` نیست (در Node.js یک شیء داخلی Timeout و در مرورگر `window`) و `label` ندارد، پس `undefined`. در خط دوم arrow function، `timer.report()` را به شکل یک فراخوانی متد واقعی اجرا می‌کند و در خط سوم `bind` مقدار `this` را قفل کرده؛ هر دو `T1` چاپ می‌کنند. گزینه‌ی ۱ گم شدن `this` را ندیده، گزینه‌ی ۲ تصور کرده arrow wrapper هم `this` را گم می‌کند و گزینه‌ی ۴ فکر کرده دادن متد به `setTimeout` خطاست.

---

۶۳۲ **گزینه‌ی ۳** متوسط

arrow wrapper تابعی است که بعداً `obj.say()` را **به شکل متد** صدا می‌زند، پس `this` برابر `obj` است. گزینه‌ی ۱ فقط تابع را می‌دهد و `this` گم می‌شود (`undefined` چاپ می‌شود). گزینه‌ی ۲ تابع را **همان لحظه** اجرا می‌کند (`Robert` بدون تأخیر چاپ می‌شود) و مقدار برگشتی‌اش (`undefined`) به `setTimeout` می‌رسد که در Node.js خطای `TypeError` می‌دهد. گزینه‌ی ۴ متد `call` را بدون تابعی که باید رویش صدا زده شود جدا کرده و آن هم `TypeError` می‌دهد.

---

۶۳۳ **گزینه‌ی ۴** سخت

`user.later()` یک فراخوانی متد است و داخل `later` مقدار `this` برابر `user` است. callback ِاول arrow است و `this` را از `Sara` به ارث می‌برد: callback ِدوم یک `function` معمولی است که بعداً توسط تایمر صدا زده می‌شود، نه به شکل `user.something()`؛ پس `this` خودش را دارد (در Node.js شیء Timeout) و `name` ندارد: `undefined`. گزینه‌ی ۱ فکر کرده تابع معمولی هم `this` را به ارث می‌برد، گزینه‌ی ۲ نقش arrow و function را برعکس کرده و گزینه‌ی ۳ تصور کرده arrow هم `this` را گم می‌کند.

---

۶۳۴ **گزینه‌ی ۳** سخت

`kermit.greet()` فراخوانی متد است و `Hello, Kermit` چاپ می‌کند. `greet()` صدا زدن ساده‌ی همان تابع است؛ بدنه‌ی کلاس **همیشه** strict است (حتی اگر فایل `"use strict"` نداشته باشد)، پس `this` برابر `undefined` است و خواندن `this.name` خطای `TypeError` می‌دهد. گزینه‌ی ۲ رفتار یک تابع معمولیِ غیر strict را به متد کلاس نسبت داده، گزینه‌ی ۱ فکر کرده متد به نمونه وصل می‌ماند و گزینه‌ی ۴ فراموش کرده که خط اول قبل از خطا اجرا شده است.

---

۶۳۵ **گزینه‌ی ۱** سخت

وقتی handler یک `function` معمولی باشد، مرورگر آن را با `this` برابر **عنصری** صدا می‌زند که listener روی آن نصب شده؛ پس `this.id` برابر `btn` است (رد گزینه‌های ۲ و ۴). arrow function `this` خودش را ندارد و `this` بیرونی، یعنی بالاترین سطح `<script>` که `window` است، را نگه می‌دارد؛ پس `true` (رد گزینه‌ی ۳).

۱-۲۶ مدیریت خطا: try، catch، finally و throw

۶۳۶ گزینه‌ی ۲ آسان

اسلاید ۵۰ صریحاً می‌گوید: «Without handling, they stop the whole program». خطای رسیدگی‌نشده (Uncaught) اجرای برنامه را همان‌جا متوقف می‌کند و خط‌های بعدی اجرا نمی‌شوند. گزینه‌های ۱ و ۳ رفتار زبانی را توصیف می‌کنند که خطا را نادیده می‌گیرد؛ JavaScript چنین کاری نمی‌کند و پیام قرمز Uncaught نشان می‌دهد. گزینه‌ی ۴ هم ساختگی است؛ هیچ اجرای دوباره‌ای در کار نیست.

۶۳۷ گزینه‌ی ۴ آسان

طبق اسلاید ۵۰، `finally` «whether an error happened or not» اجرا می‌شود. `catch` فقط وقتی اجرا می‌شود که در `try` خطایی رخ داده باشد. `try` هم اگر وسطش خطا رخ دهد، بقیه‌ی خط‌هایش اجرا نمی‌شود. `throw` اصلاً بلوک نیست؛ دستوری برای پرتاب کردن خطاست.

۶۳۸ گزینه‌ی ۳ آسان

وقتی در `try` چیزی پرتاب می‌شود، همان چیز به پارامتر `catch` داده می‌شود؛ برای خطاهای خود JavaScript (و `new Error(...)`) یک شیء خطاست که `err.name` (نوع) و `err.message` (متن) دارد، مثل `err.message` در کد اسلاید ۵۰. اگر `err` یک boolean بود (گزینه‌ی ۱)، همیشه `true` می‌شد و فایده‌ای نداشت. شماره‌ی خط (گزینه‌ی ۲) فقط داخل `err.stack` دیده می‌شود. گزینه‌ی ۴ هم غلط است: متن خطا در `err.message` است و `err` فقط وقتی رشته می‌شود که کسی خودش یک رشته پرتاب کند (`throw "..."`).

۶۳۹ گزینه‌ی ۱ آسان

`JSON.parse` روی متن نامعتبر `{ bad json }` یک `SyntaxError` پرتاب می‌کند؛ پس اجرا به `catch` می‌رود و `Oops:` همراه `err.message` چاپ می‌شود. بعد `finally` هم `Done!` را چاپ می‌کند. گزینه‌ی ۲ فرض کرده `catch` اجرا نمی‌شود، گزینه‌ی ۳ فراموش کرده `finally` همیشه اجرا می‌شود و گزینه‌ی ۴ فراموش کرده که خطا داخل `try` رخ داده و گرفته می‌شود، پس Uncaught نیست.

۶۴۰ گزینه‌ی ۴ آسان

`name` نوع خطاست و برای شیئی که با `new Error(...)` ساخته شده، `Error` است. `message` همان متنی است که به سازنده دادیم: `Disk full`. گزینه‌ی ۱ جای این دو را عوض کرده؛ `Error: Disk full` (گزینه‌ی ۲) حاصل `String(e)` است، نه `e.name`؛ و `name` برای شیء خطا همیشه مقدار دارد، پس `undefined` (گزینه‌ی ۳) هم غلط است.

۶۴۱ گزینه‌ی ۲ آسان

استفاده از اسمی که هیچ‌جا تعریف نشده، `ReferenceError` می‌دهد (`totl is not defined`). `TypeError` برای کار نامعتبر روی یک مقدارِ موجود است (مثل `null.length`)، `SyntaxError` برای اشتباه در دستور زبان است (عبارت `totl + 1` از نظر نگارشی کاملاً درست است) و `RangeError` برای عدد خارج از بازه (مثل `new Array(-1)`).

۶۴۲ گزینه‌ی ۳ آسان

`throw` شیء خطا را پرتاب می‌کند؛ اجرای `setAge` همان‌جا قطع می‌شود، خط `Age saved` رد می‌شود و `catch` پیام را چاپ می‌کند. با `return`، شیء `RangeError` فقط به‌عنوان یک مقدار معمولی برگردانده می‌شود و هیچ خطایی پرتاب نمی‌شود؛ پس چاپ می‌شود. `catch` فقط بعد از بلوک `try` معنا دارد و `error` هم کلمه‌ی کلیدی نیست؛ هر دو `SyntaxError` می‌دهند.

۶۴۳ گزینه‌ی ۱ متوسط

`A` چاپ می‌شود، بعد `JSON.parse` خطا می‌دهد و بقیه‌ی `try` (یعنی `B`) رد می‌شود. `catch` نام خطا را چاپ می‌کند، `finally` حرف `D` را، و چون خطا گرفته شده، برنامه ادامه پیدا می‌کند و `E` هم چاپ می‌شود. گزینه‌ی ۲ فکر کرده خط‌های بعد از خطا در `try` هم اجرا می‌شوند، گزینه‌ی ۴ فکر کرده بعد از `catch` اجرا به `try` برمی‌گردد و گزینه‌ی ۳ فکر کرده بعد از خطا برنامه حتماً متوقف می‌شود؛ در حالی که خطای گرفته‌شده برنامه را متوقف نمی‌کند.

۶۴۴ **گزینه‌ی ۲** متوسط

`step = 1` اجرا می‌شود، بعد `throw` اجرای `try` را قطع می‌کند و خط `step = 2` هرگز اجرا نمی‌شود. در `catch` مقدار `1 + 10` یعنی `11` ذخیره و در آخر چاپ می‌شود. `12` یعنی فکر کنیم `step = 2` هم اجرا شده، `1` یعنی فکر کنیم `catch` اجرا نشده و `2` یعنی فکر کنیم `throw` هیچ اثری نداشته است.

---

۶۴۵ **گزینه‌ی ۳** متوسط

خط ۲ مشکلی ندارد (`SARA` چاپ می‌شود). خط ۳ ویژگی‌ای را می‌خواند که وجود ندارد؛ این خطا نیست و فقط `undefined` چاپ می‌شود. اما در خط ۴، `user.address` برابر `null` است و خواندن ویژگی از `null` خطای `TypeError: Cannot read properties of null (reading 'city')` می‌دهد؛ پس برنامه همان‌جا متوقف می‌شود و `done` هرگز چاپ نمی‌شود.

---

۶۴۶ **گزینه‌ی ۱** متوسط

متن `{name: 1}` یک JSON نامعتبر است (کلید باید داخل دابل‌کوتیشن باشد) و `JSON.parse` برای متن نامعتبر `SyntaxError` می‌دهد، نه `TypeError`؛ پس جمله‌ی گزینه‌ی ۱ نادرست است. سه جمله‌ی دیگر درست‌اند: خواندن ویژگی از `null` یعنی `TypeError`، طول منفی برای آرایه یعنی `RangeError` و اسم تعریف‌نشده یعنی `ReferenceError`.

---

۶۴۷ **گزینه‌ی ۲** متوسط

`super(message)` سازنده‌ی `Error` را صدا می‌زند و `message` را تنظیم می‌کند و خط بعد، `name` را به `ValidationError` تغییر می‌دهد. چون `ValidationError` از `Error` ارث می‌برد (`extends`)، شیء هم نمونه‌ی `ValidationError` است و هم نمونه‌ی `Error`؛ پس `true true`. گزینه‌ی ۱ خط `this.name = ...` را نادیده گرفته، گزینه‌ی ۳ ارث‌بری را فراموش کرده و گزینه‌ی ۴ فکر کرده `message` بدون مقداردهی دستی تنظیم نمی‌شود، در حالی که `super(message)` این کار را می‌کند.

---

۶۴۸ **گزینه‌ی ۴** متوسط

`return "data"` مقدار برگشتی را آماده می‌کند، اما قبل از اینکه تابع واقعاً خارج شود، `finally` اجرا می‌شود و `cleanup` را چاپ می‌کند؛ بعد `load()` مقدار `data` را برمی‌گرداند و `console.log` بیرونی آن را چاپ می‌کند. گزینه‌ی ۱ فکر کرده از روی `return` می‌پرد، گزینه‌ی ۲ فکر کرده `finally` بعد از چاپ مقدار برگشتی اجرا می‌شود و گزینه‌ی ۳ فکر کرده `finally` قبل از `try` اجرا می‌شود.

---

۶۴۹ **گزینه‌ی ۳** متوسط

هر مقداری که `throw` شود، عیناً به `catch` می‌رسد؛ اینجا یک رشته پرتاب شده، پس `typeof err` برابر `string` است. رشته ویژگی‌ای به اسم `message` ندارد و خواندن ویژگیِ ناموجود فقط `undefined` می‌دهد (خطا فقط برای خواندن ویژگی از `null` و `undefined` است؛ رد گزینه‌ی ۴). گزینه‌ی ۱ فکر کرده JavaScript رشته را خودکار به شیء `Error` تبدیل می‌کند و گزینه‌ی ۲ فکر کرده هر چیزی که پرتاب شود `message` دارد. برای همین همیشه `new Error(...)` پرتاب کن.

---

۶۵۰ **گزینه‌ی ۱** متوسط

این توصیه یعنی خطایی را بگیر که انتظارش را داری و می‌دانی با آن چه کنی، و بقیه را (مثل باگ‌های برنامه) پنهان نکن؛ بهترین کار این است که آن‌ها را دوباره پرتاب کنی (rethrow). گزینه‌های ۲ و ۴ همه‌ی خطاها، حتی باگ‌ها، را بی‌صدا می‌بلعند و پیدا کردن مشکل را بسیار سخت می‌کنند. گزینه‌ی ۳ هم خلاف اسلاید است که `try...catch` را برای اجرای امن کد پرخطر پیشنهاد می‌کند.

---

۶۵۱ **گزینه‌ی ۴** متوسط

گزینه‌ی ۴ از **optional catch binding** (از ES2019) استفاده می‌کند: وقتی به شیء خطا نیازی نداریم، می‌توان پرانتز `(err)` را حذف کرد؛ خطای `JSON.parse` گرفته می‌شود و `invalid JSON` چاپ می‌شود. گزینه‌ی ۱ فقط `try` دارد که مجاز نیست (`try` باید `catch` یا `finally` داشته باشد)، `finally` پارامتر نمی‌گیرد (گزینه‌ی ۲) و `catch` بدون `try` معنا ندارد (گزینه‌ی ۳)؛ هر سه `SyntaxError` می‌دهند.

---

۶۵۲ **گزینه‌ی ۳** متوسط

`divide(10, 2)` عدد `5` را برمی‌گرداند. `divide(1, 0)` با `throw` خطا پرتاب می‌کند، پس خط بعدیِ `try` (یعنی `divide(9, 3)`) هرگز اجرا نمی‌شود و اجرا به `catch` می‌رود. چون خطا گرفته شده، برنامه بعد از ساختار ادامه پیدا می‌کند و `program continues` چاپ می‌شود. گزینه‌ی ۱ فکر کرده بعد از `catch` به `try` برمی‌گردیم؛ گزینه‌ی ۲ رفتار خودِ JavaScript برای `1 / 0` (یعنی `Infinity`) را در نظر گرفته، اما شرط `b === 0` قبل از تقسیم خطا پرتاب می‌کند؛ گزینه‌ی ۴ فکر کرده خطای گرفته‌شده هم برنامه را متوقف می‌کند.

۶۵۳ **گزینه‌ی ۲** سخت

`try` خطا پرتاب می‌کند و `catch` با `return "from catch"` می‌خواهد خارج شود، اما قبل از خروج `finally` اجرا می‌شود و `return` داخل آن، مقدار برگشتی را **جایگزین** می‌کند؛ پس نتیجه `from finally` است. گزینه‌ی ۱ فکر کرده `finally` نمی‌تواند مقدار برگشتی را عوض کند. گزینه‌ی ۳ فکر کرده هر `return` جداگانه چیزی چاپ می‌کند، در حالی که تابع فقط یک مقدار برمی‌گرداند و فقط یک `console.log` داریم. گزینه‌ی ۴ فکر کرده `return` داخل `finally` مجاز نیست؛ مجاز است، فقط کار خطرناکی است و نباید از آن استفاده کرد.

۶۵۴ **گزینه‌ی ۱** سخت

`"25"` یک JSON معتبر است و `25` برمی‌گردد. `"abc"` JSON نامعتبر است، پس `SyntaxError` رخ می‌دهد و `catch` داخلی که این نوع خطا را می‌شناسد، `0` برمی‌گرداند. `"-3"` معتبر است ولی منفی است، پس `RangeError` پرتاب می‌شود؛ `catch` داخلی نمی‌تواند به آن رسیدگی کند و با `throw err` دوباره پرتابش می‌کند. خطا به `catch` بیرونی می‌رسد و چون بقیه‌ی `try` بیرونی رد می‌شود، `end` چاپ نمی‌شود. گزینه‌ی ۲ بررسی منفی بودن را نادیده گرفته، گزینه‌ی ۳ فکر کرده بعد از `catch` بیرونی، `end` هم چاپ می‌شود و گزینه‌ی ۴ فکر کرده `catch` داخلی برای هر خطایی `0` برمی‌گرداند.

۶۵۵ **گزینه‌ی ۴** سخت

`null.length` خطای `TypeError` می‌دهد. `try` داخلی `catch` ندارد، پس خطا به بیرون می‌رود؛ اما قبل از آن `finally` داخلی اجرا می‌شود (`inner finally`). خط `after inner` رد می‌شود، `catch` بیرونی خطا را می‌گیرد و در آخر `finally` بیرونی اجرا می‌شود. گزینه‌ی ۱ ترتیب را اشتباه گرفته (`finally` داخلی زودتر است)، گزینه‌ی ۲ فکر کرده `finally` خطا را «حل» می‌کند و گزینه‌ی ۳ فکر کرده بدون `catch` داخلی، خطا گرفته نمی‌شود؛ اما `catch` بیرونی آن را می‌گیرد.

۶۵۶ **گزینه‌ی ۲** سخت

`let x = ;` اشتباه دستوری در **خود کد** است. موتور قبل از اجرای اسکریپت کل آن را تجزیه (parse) می‌کند و همین‌جا متوقف می‌شود؛ پس هیچ خطی، حتی `console.log("start")`، اجرا نمی‌شود و `try...catch` هم فرصت گرفتن خطا را پیدا نمی‌کند. گزینه‌های ۱ و ۳ فکر کرده‌اند `catch` این خطا را می‌گیرد و گزینه‌ی ۴ فکر کرده خط‌های قبل از اشتباه اجرا می‌شوند. (برخلاف این مورد، `SyntaxError` `JSON.parse` در زمان اجرا رخ می‌دهد و قابل گرفتن است.)

۶۵۷ **گزینه‌ی ۱** سخت

`setTimeout` فقط تابع را برای بعد ثبت می‌کند و بلافاصله برمی‌گردد؛ پس `try` بدون خطا تمام می‌شود و `timer scheduled` و بعد `after try` چاپ می‌شوند. وقتی تایمر اجرا می‌شود، `try...catch` مدت‌ها تمام شده است؛ پس `TypeError` `user.name` گرفته نمی‌شود و برنامه با خطای Uncaught متوقف می‌شود. گزینه‌های ۲، ۳ و ۴ همه فرض کرده‌اند `catch` بیرونی خطای callback را می‌گیرد. راه درست، گذاشتن `try...catch` داخل خودِ callback است.

۶۵۸ **گزینه‌ی ۴** سخت

فرق در **زمان** رخ دادن خطاست. کدی که `JSON.parse` را صدا می‌زند از نظر دستوری سالم است و خطا وقتی رخ می‌دهد که داده‌ی خراب خوانده می‌شود؛ پس `catch` آن را می‌گیرد (کد اسلاید ۵۰). اما اگر خود اسکریپت اشتباه دستوری داشته باشد، موتور هنگام تجزیه‌ی (parse) کد متوقف می‌شود و هیچ خطی اجرا نمی‌شود؛ پس گزینه‌های ۱ و ۳ غلط‌اند. گزینه‌ی ۲ هم غلط است، چون `JSON.parse` برای متن نامعتبر `SyntaxError` پرتاب می‌کند.

۶۵۹ **گزینه‌ی ۳** سخت

کلاس `AppError` چیزی به `Error` اضافه نکرده است؛ پس `message` را سازنده‌ی `Error` تنظیم می‌کند، اما `name` از `Error` به ارث می‌رسد و همان `Error` می‌ماند. با این حال شیء واقعاً با `new AppError` ساخته شده و `instanceof AppError` برابر `true` است. گزینه‌های ۱ و ۴ فکر کرده‌اند `name` خودکار اسم کلاس می‌شود (برای همین در کلاس‌های خطای سفارشی `this.name` را دستی تنظیم می‌کنیم) و گزینه‌های ۲ و ۴ ارث‌بری را در `instanceof` نادیده گرفته‌اند.

۶۶۰ **گزینه‌ی ۴** سخت

در `c`، `settings` برابر `null` است و `settings.theme` خطای `TypeError` می‌دهد. نه `c` و نه `b` این خطا را نمی‌گیرند، پس خطا در زنجیره‌ی فراخوانی بالا می‌رود تا به `try` تابع `a` برسد؛ در این مسیر `b got ...` و `a done` رد می‌شوند. `catch` تابع `a` خطا را می‌گیرد و بعد برنامه ادامه پیدا می‌کند و `end` چاپ می‌شود. گزینه‌ی ۱ فکر کرده خواندن ویژگی از `null` فقط `undefined` می‌دهد، گزینه‌ی ۲ فکر کرده خطا فقط در همان تابعی که رخ داده قابل گرفتن است و گزینه‌ی ۳ فکر کرده بعد از `catch` به ادامه‌ی `try` برمی‌گردیم.

Promise ۱-۲۷

۶۶۱ **گزینه‌ی ۲** آسان

اسلاید ۵۵ دقیقاً همین را می‌گوید: «A Promise represents a value that may arrive now, later, or never». گزینه‌ی ۱ غلط است؛ Promise تابع نیست و زمان ثابتی ندارد (۱ ثانیه فقط زمانِ تایمرِ مثال اسلاید بود). گزینه‌ی ۳ غلط است؛ JavaScript تک‌رشته‌ای (single-threaded) است و Promise هیچ thread جدیدی نمی‌سازد. گزینه‌ی ۴ ویژگی `const` را با Promise قاطی کرده است.

۶۶۲ **گزینه‌ی ۳** آسان

Promise از `pending` شروع می‌شود و فقط یک بار settled می‌شود: **یا** fulfilled **یا** rejected؛ بعد از آن هرگز تغییر نمی‌کند. گزینه‌ی ۱ برداشتِ غلط از فلش‌های اسلاید ۵۵ است (آن فلش‌ها دنباله نیستند). گزینه‌ی ۲ غلط است، چون Promise ِfulfilled دیگر rejected نمی‌شود (خطای بعدی فقط Promise جدیدی را که `then` می‌سازد rejected می‌کند). گزینه‌ی ۴ هم غلط است؛ Promise به pending برنمی‌گردد و کاری را تکرار نمی‌کند.

۶۶۳ **گزینه‌ی ۴** آسان

اسلاید ۵۵: «`.catch()` for errors, `.then()` for success, Use». گزینه‌ی ۱ جای دو متد را عوض کرده است. `resolve` و `reject` (گزینه‌ی ۲) توابعی هستند که داخل executor برای **اعلام** نتیجه صدا زده می‌شوند، نه برای گرفتنِ آن. `.finally()` (گزینه‌ی ۳) در هر دو حالت اجرا می‌شود و هیچ مقداری دریافت نمی‌کند.

۶۶۴ **گزینه‌ی ۱** آسان

بعد از ۱ ثانیه `resolve("Data received!")` اجرا می‌شود، Promise fulfilled می‌شود و تابعِ داخل `.then` مقدار را چاپ می‌کند. خط `reject` کامنت است و اجرا نمی‌شود، پس `.catch` کاری نمی‌کند (گزینه‌های ۲ و ۳ غلط‌اند؛ تازه اگر `reject` هم بعد از `resolve` اجرا می‌شد، بی‌اثر بود، چون Promise فقط یک بار settled می‌شود). گزینه‌ی ۴ غلط است، چون `result` مقدارِ نهاییِ Promise است، نه خودِ Promise.

۶۶۵ **گزینه‌ی ۳** آسان

این بار فقط `reject("Error occurred!")` اجرا می‌شود؛ Promise rejected می‌شود، تابعِ داخل `.then` کنار گذاشته می‌شود و `.catch` دلیل را چاپ می‌کند: `Error occurred!`. گزینه‌های ۱ و ۲ فرض کرده‌اند `.then` هم اجرا می‌شود. گزینه‌ی ۴ غلط است، چون این rejection با `.catch` مدیریت شده و برنامه crash نمی‌کند.

۶۶۶ **گزینه‌ی ۱** آسان

BEST PRACTICE اسلاید ۵۵: «Always add `.catch()` to handle errors; unhandled rejections can crash apps». گزینه‌ی ۲ برعکسِ هدف Promise است (Promise آمده تا تودرتو شدنِ callbackها را از بین ببرد). گزینه‌ی ۳ بی‌پایه است؛ خودِ اسلاید داخل executor از `setTimeout` استفاده کرده است. گزینه‌ی ۴ هم بی‌معناست، چون فقط اولین `resolve` یا `reject` اثر دارد.

۶۶۷ **گزینه‌ی ۲** آسان

اسلاید ۵۵: «Avoids deep nesting from callbacks»؛ به جای هرمِ تودرتو، مرحله‌ها با `.then()` پشت سر هم زنجیر می‌شوند. Promise کد را چندرشته‌ای نمی‌کند (گزینه‌ی ۱)، برنامه را متوقف یا همگام نمی‌کند (گزینه‌ی ۳) و می‌تواند شکست بخورد (rejected)، پس مدیریت خطا همچنان لازم است (گزینه‌ی ۴).

---

۶۶۸ **گزینه‌ی ۴** آسان

executor همان لحظه‌ی `new Promise(...)` و به صورت همگام اجرا می‌شود؛ فقط callbackهای `then`، `catch` و `finally` ناهمگام‌اند. Promise منتظر صدا زدنِ `.then()` نمی‌ماند (گزینه‌ی ۱)؛ مثلاً در کد اسلاید ۵۵ تایمر همان لحظه‌ی ساختن Promise شروع می‌شود. گزینه‌ی ۲ ویژگی callbackهای `then` را به executor نسبت داده و گزینه‌ی ۳ برعکس است: `resolve` را خودِ executor صدا می‌زند.

---

۶۶۹ **گزینه‌ی ۳** متوسط

`A` همگام چاپ می‌شود. executor هم همان لحظه و همگام اجرا می‌شود و `B` را چاپ می‌کند؛ `resolve("C")` فقط وضعیت را fulfilled می‌کند. callback داخل `then` ناهمگام است، پس اول بقیه‌ی کد همگام (`D`) و بعد `C` چاپ می‌شود. گزینه‌ی ۱ فرض کرده `then` فوراً اجرا می‌شود؛ گزینه‌های ۲ و ۴ فکر کرده‌اند خودِ executor هم ناهمگام است.

---

۶۷۰ **گزینه‌ی ۱** متوسط

`Promise.resolve("X")` یک Promise از قبل fulfilled می‌سازد، اما callback داخل `then` باز هم ناهمگام است و بعد از تمام شدن کدهای همگام اجرا می‌شود؛ پس اول `Y` و بعد `X`. گزینه‌ی ۲ فرض کرده چون Promise آماده است، `then` فوراً اجرا می‌شود؛ گزینه‌ی ۳ فکر کرده `then` روی Promise آماده اصلاً اجرا نمی‌شود؛ گزینه‌ی ۴ خودِ Promise را چاپ‌شده فرض کرده، در حالی که `v` مقدارِ آن است.

---

۶۷۱ **گزینه‌ی ۴** متوسط

`then` اول مقدار `6` را برمی‌گرداند (arrow function بدون آکولاد، خودش return دارد). `then` دوم آکولاد دارد ولی `return` ندارد؛ `x + 1` حساب می‌شود اما برگردانده نمی‌شود، پس ورودیِ مرحله‌ی بعد `undefined` است. گزینه‌ی ۱ وجود return را فرض کرده؛ گزینه‌ی ۲ فکر کرده مقدار قبلی دست‌نخورده رد می‌شود؛ گزینه‌ی ۳ از محاسبه‌ای مثل `undefined + 1` آمده، ولی چنین محاسبه‌ای اینجا چاپ نمی‌شود.

---

۶۷۲ **گزینه‌ی ۳** متوسط

`throw` در `then` اول، Promise آن مرحله را rejected می‌کند؛ `then` دوم (که فقط برای مسیر موفقیت است) کنار گذاشته می‌شود و اجرا به `catch` می‌رسد: `B boom`. `catch` مقدار `10` را برمی‌گرداند، پس زنجیره دوباره در مسیر موفقیت است و `then` آخر `C 10` را چاپ می‌کند. گزینه‌ی ۲ فکر کرده بعد از `catch` زنجیره تمام می‌شود؛ گزینه‌ی ۴ فکر کرده `then` بعد از خطا هم اجرا می‌شود؛ گزینه‌ی ۱ خطا را کلاً نادیده گرفته است.

---

۶۷۳ **گزینه‌ی ۱** متوسط

`finally` در هر دو حالت (موفق و ناموفق) اجرا می‌شود، پس اول `cleanup` چاپ می‌شود. سپس `finally` همان خطا را دست‌نخورده به مرحله‌ی بعد می‌دهد و `catch` آن را می‌گیرد: `caught: fail`. گزینه‌ی ۲ فکر کرده `finally` فقط در موفقیت اجرا می‌شود؛ گزینه‌ی ۳ ترتیب زنجیره را برعکس خوانده (`finally` زودتر نوشته شده)؛ گزینه‌ی ۴ فکر کرده `finally` خطا را «مصرف» می‌کند.

---

۶۷۴ **گزینه‌ی ۴** متوسط

وقتی callback یک `then` خودش یک Promise برمی‌گرداند، زنجیره منتظر آن می‌ماند و `then` بعدی **مقدار نهایی** آن را می‌گیرد: در مرحله‌ی اول `x` برابر `1` است، `wait(100, 2)` برگردانده می‌شود و بعد از حدود ۱۰۰ میلی‌ثانیه‌ی دیگر `got 2` چاپ می‌شود. گزینه‌ی ۱ فکر کرده خودِ Promise به مرحله‌ی بعد می‌رسد؛ گزینه‌ی ۲ مقدار مرحله‌ی اول را دست‌نخورده فرض کرده؛ گزینه‌ی ۳ فکر کرده arrow function چیزی برنمی‌گرداند، در حالی که بدون آکولاد، return خودکار است.

---

۶۷۵ **گزینه‌ی ۲** متوسط

`Promise.all` منتظر می‌ماند تا هر سه fulfilled شوند (حدود ۳۰۰ میلی‌ثانیه، یعنی زمانِ کندترین کار) و آرایه‌ی نتیجه را **به ترتیب آرایه‌ی ورودی** می‌سازد، نه به ترتیب تمام شدن. پس با اینکه B زودتر از همه تمام شده، نتیجه `[ 'A', 'B', 'C' ]` است. گزینه‌ی ۱ ترتیب تمام شدن را در نظر گرفته؛ گزینه‌های ۳ و ۴ رفتار `Promise.race` را به `Promise.all` نسبت داده‌اند.

۶۷۶ **گزینه‌ی ۳** متوسط

`Promise.race` با **اولین** Promiseی که settled شود settled می‌شود. B با ۱۰۰ میلی‌ثانیه زودتر از همه تمام می‌شود، پس `B` چاپ می‌شود. گزینه‌ی ۱ فکر کرده اولین عضوِ آرایه برنده است؛ گزینه‌های ۲ و ۴ رفتار `Promise.all` را فرض کرده‌اند، در حالی که `race` آرایه برنمی‌گرداند.

۶۷۷ **گزینه‌ی ۲** متوسط

`fetchData` یک **شیء Promise** است، نه تابع؛ پس `fetchData()` در خط ۵ خطای `TypeError: fetchData is not a function` می‌دهد. خط ۱ درست است و همان لحظه Promise را می‌سازد؛ خط ۲ هم یک تایمر معمولی است. اگر مثل اسلاید ۵۵ می‌نوشتیم `fetchData.then(...)`، کد بدون خطا `Data received!` را چاپ می‌کرد (پس گزینه‌ی ۴ هم غلط است).

۶۷۸ **گزینه‌ی ۴** متوسط

`.then` همیشه یک Promise **جدید** برمی‌گرداند، پس `p1 === p2` برابر `false` است. `p1` با `1` fulfilled است و `p2` با مقداری که callback برمی‌گرداند، یعنی `2`. گزینه‌های ۱ و ۳ فکر کرده‌اند `then` همان Promise قبلی را برمی‌گرداند؛ گزینه‌ی ۲ فکر کرده `p2` همان مقدارِ `p1` را دارد.

۶۷۹ **گزینه‌ی ۱** متوسط

Promise fulfilled است، پس `catch` (که فقط برای حالت rejected است) کنار گذاشته می‌شود و مقدار `"ok"` دست‌نخورده به `then` می‌رسد. گزینه‌ی ۲ فکر کرده `catch` همیشه اجرا می‌شود؛ گزینه‌ی ۳ فکر کرده رد شدن از کنار `catch` مقدار را از بین می‌برد؛ گزینه‌ی ۴ هم `catch` را بی‌دلیل اجرا کرده است.

۶۸۰ **گزینه‌ی ۱** متوسط

در گزینه‌ی ۱ هر مرحلهِ Promise مرحله‌ی بعد را **برمی‌گرداند**، پس زنجیره منتظر هر کدام می‌ماند و در انتها `price` عدد `100` است (۲ سفارش × ۵۰). در گزینه‌ی ۲ به خاطر آکولاد، `getOrders(user)` برگردانده نمی‌شود؛ مرحله‌ی بعد `undefined` می‌گیرد و `orders.length` داخل تایمر خطای `TypeError` می‌دهد. در گزینه‌ی ۳ هیچ صبری در کار نیست و متغیرها خودِ Promiseها هستند؛ چاپ می‌شود `Total: Promise { <pending> }`. در گزینه‌ی ۴ هم `getPrice(orders)` یک Promise است و بدون `then` یا صبر، همان `Promise { <pending> }` چاپ می‌شود.

۶۸۱ **گزینه‌ی ۴** متوسط

گزینه‌ی ۴ نادرست است: `Promise.all` خودش کاری را «اجرا» نمی‌کند؛ کارها از لحظه‌ی ساخته شدنِ Promiseهایشان شروع شده‌اند و هم‌زمان پیش می‌روند. برای همین زمان کل به اندازه‌ی کندترین کار است، نه مجموع زمان‌ها. سه گزینه‌ی دیگر رفتارهای واقعی `Promise.all` هستند: ترتیبِ نتیجه همان ترتیب ورودی است (گزینه‌ی ۱)، با اولین شکست rejected می‌شود (گزینه‌ی ۲) و مقدارهای غیر Promise را همان‌طور در نتیجه می‌گذارد (گزینه‌ی ۳).

۶۸۲ **گزینه‌ی ۲** سخت

فقط اولین فراخوانی، یعنی `resolve("first")`، اثر دارد؛ `reject("second")` و `resolve("third")` نادیده گرفته می‌شوند، چون Promise قبلاً settled شده است. اما `resolve` اجرای executor را متوقف نمی‌کند، پس `still running` (همگام) چاپ می‌شود و بعد `then: first`. گزینه‌ی ۱ فکر کرده `resolve` مثل `return` عمل می‌کند؛ گزینه‌ی ۳ فکر کرده `reject` بعدی وضعیت را عوض می‌کند؛ گزینه‌ی ۴ فکر کرده آخرین `resolve` برنده است.

۶۸۳ **گزینه‌ی ۲** سخت

callback `finally` هیچ آرگومانی دریافت نمی‌کند، پس `v` در آن `undefined` است. مقدارِ برگردانده‌شده از `finally` هم نادیده گرفته می‌شود و مقدار اصلی (`"data"`) دست‌نخورده به `then` بعدی می‌رسد. گزینه‌های ۱ و ۳ فرض کرده‌اند `finally` مقدار را می‌گیرد؛ گزینه‌های ۱ و ۴ فکر کرده‌اند `return` در `finally` مثل `return` در `then` مقدار را عوض می‌کند.

۶۸۴ **گزینه‌ی ۳** سخت

به `then` باید یک **تابع** داد. اینجا `console.log("done")` همان لحظه (همگام) اجرا می‌شود، `done` را چاپ می‌کند و حاصلش، یعنی `undefined`، به `then` داده می‌شود که آن را نادیده می‌گیرد. بعد `end` چاپ می‌شود و وقتی Promise بعد از ۱۰۰ میلی‌ثانیه fulfilled می‌شود، هیچ تابعی برای چاپ کردن `ok` وجود ندارد. گزینه‌های ۱ و ۴ فکر کرده‌اند `done` بعد از fulfilled شدن چاپ می‌شود؛ گزینه‌ی ۲ فکر کرده مقدار Promise خودبه‌خود چاپ می‌شود. شکل درست: `p.then(() => console.log("done"))`.

۶۸۵ **گزینه‌ی ۴** سخت

اول `1 + 1 = 2`. مرحله‌ی بعد `Promise.reject(2 * 10)` را برمی‌گرداند، یعنی Promiseی rejected با دلیل `20`؛ پس `then` بعدی کنار گذاشته می‌شود و `catch` دلیل را می‌گیرد: `catch 20`. مقدار `21` را برمی‌گرداند و زنجیره در مسیر موفقیت ادامه می‌یابد: `then2 21`. در آخر `finally` اجرا می‌شود. گزینه‌ی ۱ فکر کرده برگرداندنِ rejected Promise مثل برگرداندن یک مقدار معمولی است؛ گزینه‌ی ۲ فکر کرده بعد از `catch` زنجیره تمام می‌شود؛ گزینه‌ی ۳ ضرب در ۱۰ را نادیده گرفته است.

۶۸۶ **گزینه‌ی ۲** سخت

Promise بعد از ۱۰۰ میلی‌ثانیه rejected می‌شود. `.then` فقط برای موفقیت است و Promise جدیدی که برمی‌گرداند هم rejected می‌شود؛ هیچ `.catch()`ی هم وجود ندارد، پس یک **unhandled rejection** رخ می‌دهد و Node.js برنامه را همان لحظه (حدود ۱۰۰ میلی‌ثانیه) متوقف می‌کند و تایمر ۵۰۰ میلی‌ثانیه‌ای هرگز اجرا نمی‌شود. این همان هشدار BEST PRACTICE اسلاید ۵۵ است. گزینه‌ی ۱ فکر کرده rejection بی‌صدا نادیده گرفته می‌شود؛ گزینه‌ی ۳ فکر کرده `then` خطا را چاپ می‌کند؛ گزینه‌ی ۴ زمان crash را اشتباه گرفته است. (در مرورگر صفحه crash نمی‌کند: پیام `Uncaught (in promise)` در کنسول می‌آید و `still alive` هم چاپ می‌شود؛ برای همین محیط را Node.js گفته‌ایم.)

۶۸۷ **گزینه‌ی ۴** سخت

executor فقط خطاهای **همگامِ** خودش را به rejection تبدیل می‌کند. اینجا خطا (`TypeError`، چون از `null` خاصیت می‌خوانیم) بعداً و داخلِ callback `setTimeout` رخ می‌دهد؛ آن لحظه کار executor مدت‌هاست تمام شده و Promise هیچ خبری از این خطا ندارد (و `resolve` هم هرگز صدا زده نمی‌شود). پس خطا مدیریت‌نشده (uncaught) است و Node.js برنامه را در ۱۰۰ میلی‌ثانیه متوقف می‌کند: نه `caught` چاپ می‌شود و نه `after`. گزینه‌های ۱ و ۲ فکر کرده‌اند `catch` این خطا را می‌گیرد؛ گزینه‌ی ۳ فکر کرده خطا بی‌اثر است. اگر بخواهیم خطای داخل تایمر به `catch` برسد، باید خودمان آن را بگیریم و `reject` را صدا بزنیم.

۶۸۸ **گزینه‌ی ۱** سخت

executor همگام اجرا می‌شود: `start` چاپ می‌شود و بعد `throw` رخ می‌دهد. خطای همگامِ داخل executor برنامه را متوقف نمی‌کند؛ JavaScript آن را می‌گیرد و Promise را rejected می‌کند (`resolve("never")` هم دیگر اجرا نمی‌شود). کد همگام ادامه می‌یابد و `end` چاپ می‌شود؛ سپس callback `catch` (که ناهمگام است) اجرا می‌شود: `catch: sync boom`. گزینه‌ی ۲ فکر کرده `throw` در executor برنامه را crash می‌کند؛ گزینه‌ی ۳ فکر کرده `catch` همگام اجرا می‌شود؛ گزینه‌ی ۴ فکر کرده خودِ executor ناهمگام است.

۶۸۹ **گزینه‌ی ۳** سخت

هر سه تایمر هم‌زمان شروع شده‌اند. در ۱۰۰ میلی‌ثانیه B تمام و rejected می‌شود و `Promise.all` **بلافاصله** rejected می‌شود: `error: B failed` (منتظر بقیه نمی‌ماند). اما `Promise.all` کارهای دیگر را لغو نمی‌کند؛ تایمرهای C و A ادامه می‌دهند و در ۲۰۰ و ۳۰۰ میلی‌ثانیه پیامشان را چاپ می‌کنند (نتیجه‌شان فقط نادیده گرفته می‌شود). گزینه‌ی ۱ فکر کرده `Promise.all` تا آخر صبر می‌کند؛ گزینه‌ی ۲ فکر کرده بقیه‌ی کارها متوقف می‌شوند؛ گزینه‌ی ۴ ترتیب آرایه را با ترتیب زمانی قاطی کرده است.

۶۹۰ **گزینه‌ی ۱** سخت

گزینه‌ی ۱ قاعده‌ی «برگرداندن Promise در زنجیره» است: زنجیره صبر می‌کند و مرحله‌ی بعد مقدار نهایی را می‌گیرد، نه خودِ Promise را. گزینه‌ی ۲ غلط است؛ هر `.then()` یک Promise **جدید** می‌سازد و برای همین زنجیره ممکن است. گزینه‌ی ۳ غلط است؛ `finally` هیچ آرگومانی نمی‌گیرد. گزینه‌ی ۴ هم غلط است؛ callbackهای `then` همیشه ناهمگام‌اند و بعد از تمام شدن کدهای همگامِ فعلی اجرا می‌شوند.

پاسخ‌های فصل دوم: JavaScript در HTML (کار با DOM)

کلید سریع فصل دوم

۶۹۱ ۳	۶۹۲ ۴	۶۹۳ ۱	۶۹۴ ۲	۶۹۵ ۳	۶۹۶ ۱	۶۹۷ ۲	۶۹۸ ۳	۶۹۹ ۴	۷۰۰ ۱
۷۰۱ ۲	۷۰۲ ۴	۷۰۳ ۳	۷۰۴ ۱	۷۰۵ ۲	۷۰۶ ۳	۷۰۷ ۴	۷۰۸ ۴	۷۰۹ ۲	۷۱۰ ۳
۷۱۱ ۴	۷۱۲ ۲	۷۱۳ ۱	۷۱۴ ۴	۷۱۵ ۲	۷۱۶ ۳	۷۱۷ ۱	۷۱۸ ۴	۷۱۹ ۳	۷۲۰ ۲
۷۲۱ ۱	۷۲۲ ۲	۷۲۳ ۳	۷۲۴ ۴	۷۲۵ ۱	۷۲۶ ۱	۷۲۷ ۳	۷۲۸ ۱	۷۲۹ ۲	۷۳۰ ۳
۷۳۱ ۴	۷۳۲ ۱	۷۳۳ ۳	۷۳۴ ۲	۷۳۵ ۱	۷۳۶ ۳	۷۳۷ ۲	۷۳۸ ۳	۷۳۹ ۳	۷۴۰ ۴
۷۴۱ ۱	۷۴۲ ۴	۷۴۳ ۱	۷۴۴ ۲	۷۴۵ ۴	۷۴۶ ۴	۷۴۷ ۳	۷۴۸ ۲	۷۴۹ ۴	۷۵۰ ۱
۷۵۱ ۲	۷۵۲ ۴	۷۵۳ ۳	۷۵۴ ۲	۷۵۵ ۳	۷۵۶ ۲	۷۵۷ ۱	۷۵۸ ۳	۷۵۹ ۴	۷۶۰ ۴
۷۶۱ ۱	۷۶۲ ۲	۷۶۳ ۱	۷۶۴ ۳	۷۶۵ ۳	۷۶۶ ۴	۷۶۷ ۱	۷۶۸ ۲	۷۶۹ ۴	۷۷۰ ۳
۷۷۱ ۲	۷۷۲ ۴	۷۷۳ ۲	۷۷۴ ۱	۷۷۵ ۳	۷۷۶ ۴	۷۷۷ ۲	۷۷۸ ۳	۷۷۹ ۱	۷۸۰ ۳
۷۸۱ ۴	۷۸۲ ۲	۷۸۳ ۱	۷۸۴ ۳	۷۸۵ ۱	۷۸۶ ۴	۷۸۷ ۳	۷۸۸ ۲	۷۸۹ ۱	۷۹۰ ۴
۷۹۱ ۴	۷۹۲ ۲	۷۹۳ ۳	۷۹۴ ۲	۷۹۵ ۴	۷۹۶ ۳	۷۹۷ ۲	۷۹۸ ۱	۷۹۹ ۲	۸۰۰ ۴
۸۰۱ ۳	۸۰۲ ۱	۸۰۳ ۱	۸۰۴ ۲	۸۰۵ ۴	۸۰۶ ۳	۸۰۷ ۱	۸۰۸ ۴	۸۰۹ ۱	۸۱۰ ۳
۸۱۱ ۴	۸۱۲ ۱	۸۱۳ ۳	۸۱۴ ۳	۸۱۵ ۲	۸۱۶ ۱	۸۱۷ ۳	۸۱۸ ۲	۸۱۹ ۱	۸۲۰ ۲
۸۲۱ ۳	۸۲۲ ۴	۸۲۳ ۲	۸۲۴ ۴	۸۲۵ ۱	۸۲۶ ۳	۸۲۷ ۴	۸۲۸ ۱	۸۲۹ ۴	۸۳۰ ۱
۸۳۱ ۳	۸۳۲ ۴	۸۳۳ ۲	۸۳۴ ۳	۸۳۵ ۳	۸۳۶ ۲	۸۳۷ ۲	۸۳۸ ۴	۸۳۹ ۱	۸۴۰ ۱
۸۴۱ ۲	۸۴۲ ۳	۸۴۳ ۴	۸۴۴ ۴	۸۴۵ ۲	۸۴۶ ۴	۸۴۷ ۲	۸۴۸ ۱	۸۴۹ ۳	۸۵۰ ۱
۸۵۱ ۱	۸۵۲ ۴	۸۵۳ ۳	۸۵۴ ۳	۸۵۵ ۲	۸۵۶ ۳	۸۵۷ ۴	۸۵۸ ۲	۸۵۹ ۱	۸۶۰ ۲
۸۶۱ ۱	۸۶۲ ۳	۸۶۳ ۴	۸۶۴ ۲	۸۶۵ ۳	۸۶۶ ۱	۸۶۷ ۴	۸۶۸ ۲	۸۶۹ ۳	۸۷۰ ۱
۸۷۱ ۳	۸۷۲ ۴	۸۷۳ ۲	۸۷۴ ۴	۸۷۵ ۱	۸۷۶ ۴	۸۷۷ ۲	۸۷۸ ۱	۸۷۹ ۳	۸۸۰ ۴
۸۸۱ ۲	۸۸۲ ۳	۸۸۳ ۴	۸۸۴ ۲	۸۸۵ ۱	۸۸۶ ۳	۸۸۷ ۲	۸۸۸ ۱	۸۸۹ ۳	۸۹۰ ۴
۸۹۱ ۴	۸۹۲ ۴	۸۹۳ ۱	۸۹۴ ۴	۸۹۵ ۲	۸۹۶ ۳	۸۹۷ ۱	۸۹۸ ۲	۸۹۹ ۳	۹۰۰ ۱
۹۰۱ ۴	۹۰۲ ۲	۹۰۳ ۳	۹۰۴ ۱	۹۰۵ ۴	۹۰۶ ۲	۹۰۷ ۱	۹۰۸ ۳	۹۰۹ ۴	۹۱۰ ۱
۹۱۱ ۲	۹۱۲ ۳	۹۱۳ ۴	۹۱۴ ۲	۹۱۵ ۱	۹۱۶ ۳	۹۱۷ ۴	۹۱۸ ۱	۹۱۹ ۲	۹۲۰ ۳
۹۲۱ ۴	۹۲۲ ۴	۹۲۳ ۱	۹۲۴ ۱	۹۲۵ ۳	۹۲۶ ۲	۹۲۷ ۴	۹۲۸ ۱	۹۲۹ ۲	۹۳۰ ۳
۹۳۱ ۴	۹۳۲ ۱	۹۳۳ ۲	۹۳۴ ۳	۹۳۵ ۴	۹۳۶ ۱	۹۳۷ ۲	۹۳۸ ۳	۹۳۹ ۴	۹۴۰ ۱
۹۴۱ ۲	۹۴۲ ۳	۹۴۳ ۴	۹۴۴ ۱	۹۴۵ ۲	۹۴۶ ۳	۹۴۷ ۲	۹۴۸ ۱	۹۴۹ ۳	۹۵۰ ۴
۹۵۱ ۲	۹۵۲ ۳	۹۵۳ ۱	۹۵۴ ۴	۹۵۵ ۲	۹۵۶ ۴	۹۵۷ ۳	۹۵۸ ۱	۹۵۹ ۲	۹۶۰ ۳
۹۶۱ ۴	۹۶۲ ۱	۹۶۳ ۲	۹۶۴ ۳	۹۶۵ ۴	۹۶۶ ۱	۹۶۷ ۲	۹۶۸ ۳	۹۶۹ ۴	۹۷۰ ۴
۹۷۱ ۱	۹۷۲ ۳	۹۷۳ ۲	۹۷۴ ۲	۹۷۵ ۴	۹۷۶ ۱	۹۷۷ ۳	۹۷۸ ۴	۹۷۹ ۱	۹۸۰ ۲
۹۸۱ ۳	۹۸۲ ۴	۹۸۳ ۱	۹۸۴ ۲	۹۸۵ ۳	۹۸۶ ۴	۹۸۷ ۱	۹۸۸ ۲	۹۸۹ ۴	۹۹۰ ۱
۹۹۱ ۳	۹۹۲ ۳	۹۹۳ ۲							

۲-۱ مرور پیش‌نیازها: سه لایه‌ی وب، شیء، this و async

۶۹۱ گزینه‌ی ۳ آسان

طبق اسلاید ۳، لایه‌ی Behaviour (رفتار) به «How it reacts» پاسخ می‌دهد و فناوری آن JavaScript است. Structure به «محتوا چیست» (HTML) و Style به «چه شکلی است» (CSS) پاسخ می‌دهند. گزینه‌ی ۴ نام لایه را درست گفته ولی CSS زبان ظاهر است و منطق واکنش به کلیک و تایپ را نمی‌سازد.

۶۹۲ گزینه‌ی ۴ آسان

گزینه‌ی ۴ نادرست است: JavaScript (لایه‌ی رفتار) می‌تواند ظاهر را هم تغییر دهد؛ مثلاً در اسلاید ۱۹ با `title.style.color = 'red'` رنگ عنوان پس از کلیک قرمز می‌شود. سه گزینه‌ی دیگر دقیقاً تعریف‌های اسلاید ۳ هستند: محتوا (HTML)، ظاهر (CSS) و واکنش (JavaScript).

۶۹۳ گزینه‌ی ۱ آسان

`name` یک property با مقدار رشته‌ای است، پس `typeof` آن `"string"` است؛ `greet` یک method است و `typeof` هر تابعی `"function"` است (نه `"object"`؛ گزینه‌ی ۲). `typeof` هیچ‌وقت کلمه‌هایی مثل `"property"` یا `"method"` برنمی‌گرداند (گزینه‌ی ۳). چون `greet` به‌صورت `student.greet()` صدا زده شده، `this` همان `student` است و نتیجه `Hi, I am Ali` می‌شود، نه `undefined` (گزینه‌ی ۴).

۶۹۴ گزینه‌ی ۲ متوسط

`JSON.stringify` فقط داده‌ها را به متن JSON تبدیل می‌کند و **متدها را حذف می‌کند**؛ پس `greet` در خروجی نیست (گزینه‌ی ۱ غلط است، چون تابع اجرا نمی‌شود و نتیجه‌اش هم ذخیره نمی‌شود). در JSON کلیدها حتماً داخل `"` هستند (رد گزینه‌ی ۳). خروجی `stringify` رشته است و `JSON.parse` آن را دوباره به شیء تبدیل می‌کند که `age` در آن **عدد** است؛ پس `20 + 1` برابر `21` است، نه `"201"` (گزینه‌ی ۴).

۶۹۵ گزینه‌ی ۳ متوسط

آرایه‌ها شیء هستند و `typeof []` برابر `"object"` است. رشته‌ی `"abc"` یک primitive است و `typeof` آن `"string"` است (برای همین اسلاید می‌گوید «تقریباً» همه‌چیز). `typeof null` به‌خاطر یک باگ تاریخی `"object"` است، نه `"null"`. تابع‌ها هم با اینکه نوع خاصی از شیء هستند، `typeof` آن‌ها `"function"` است.

۶۹۶ گزینه‌ی ۱ سخت

`s` یک primitive است، ولی وقتی روی آن نقطه می‌گذاریم JavaScript یک **wrapper object** موقت (`String`) می‌سازد؛ پس `length` و `toUpperCase()` کار می‌کنند و خطایی رخ نمی‌دهد (رد گزینه‌ی ۴). مقدار `42` روی همان شیء موقت نوشته می‌شود که بلافاصله دور انداخته می‌شود؛ بنابراین `s.extra` برابر `undefined` است (رد گزینه‌های ۲ و ۳). `s` همچنان primitive است و `typeof` آن `"string"` می‌ماند. (در حالت strict، خط `s.extra = 42` خطای `TypeError` می‌داد.)

۶۹۷ گزینه‌ی ۲ متوسط

طبق اسلاید ۵، `this` به **نحوه‌ی فراخوانی** بستگی دارد، نه محل تعریف. `obj.regular()` با `obj` صدا زده شده و `other.regular()` با `other`؛ با اینکه هر دو یک تابع‌اند، نتیجه `obj` و `other` است (رد گزینه‌های ۱ و ۳ که فکر می‌کنند `this` به محل تعریف چسبیده است). `f()` بدون شیء صدا زده شده؛ در حالت sloppy در Node.js، `this` شیء سراسری است که `name` ندارد، پس `undefined` چاپ می‌شود (رد گزینه‌ی ۴).

۶۹۸ گزینه‌ی ۳ سخت

`f()` یک فراخوانی ساده است؛ در اسکریپت معمولی مرورگر (sloppy) `this` برابر `window` می‌شود. `window` از قبل propertyای به نام `name` دارد که مقدار پیش‌فرضش رشته‌ی خالی است؛ پس `f()` مقدار `""` برمی‌گرداند و `JSON.stringify("")` متن `""` را چاپ می‌کند. گزینه‌ی ۲ جواب همین کد در **Node.js** است (آنجا `undefined` می‌شود). گزینه‌ی ۱ فکر کرده تابع شیءاش را به خاطر می‌سپارد، و خطایی هم رخ نمی‌دهد چون `this` در حالت sloppy `undefined` نیست (رد گزینه‌ی ۴).

۶۹۹ **گزینه‌ی ۴** متوسط

`regular` با `counter.regular()` صدا زده شده و `this` همان `counter` است، پس `10`. ولی arrow function `this` خودش را ندارد و آن را از scope بیرونیِ object literal می‌گیرد؛ در یک ماژول CommonJS در Node.js، `this` بیرونی شیء خالی `module.exports` است که `count` ندارد، پس `undefined` (در مرورگر هم `this` بیرونی `window` است و باز `undefined` می‌شود). خطایی رخ نمی‌دهد، چون خواندن property ناموجود از یک شیء فقط `undefined` می‌دهد (رد گزینه‌ی ۳).

---

۷۰۰ **گزینه‌ی ۱** سخت

در `t.report()` مقدار `this` نمونه‌ی `t` است و arrow function `show` همان `this` را به ارث می‌برد، پس `T1` چاپ می‌شود (رد گزینه‌ی ۴). اما `r()` یک فراخوانی ساده است و کدِ داخل کلاس همیشه **strict** است؛ پس `this` برابر `undefined` می‌شود و خواندن `undefined.label` خطای `TypeError` می‌دهد. گزینه‌ی ۲ فکر کرده متد به نمونه چسبیده است و گزینه‌ی ۳ قانون حالت sloppy (شیء سراسری) را اشتباهی به کلاس تعمیم داده است.

---

۷۰۱ **گزینه‌ی ۲** متوسط

در listenerای که با `function` معمولی نوشته شده، مرورگر `this` را همان عنصری قرار می‌دهد که listener روی آن ثبت شده؛ پس `this.textContent` برابر `First` است (رد گزینه‌ی ۳). arrow function `this` خودش را ندارد و `this` بیرونی را برمی‌دارد که در سطح بالای اسکریپت مرورگر `window` است؛ پس `true` چاپ می‌شود (رد گزینه‌ی ۱). گزینه‌ی ۴ فکر کرده arrow function هم `this` را دکمه می‌گیرد، در حالی که اصلاً `textContent` چاپ نمی‌شود.

---

۷۰۲ **گزینه‌ی ۴** آسان

اسلاید ۶ صریحاً می‌گوید: «callbacks → manual, promises → .catch(), async → try...catch». در callbackها معمولاً اولین پارامتر خطاست و باید آن را دستی بررسی کرد؛ Promise خطا را به `.catch()` می‌فرستد؛ و در تابع async، Promise ردشده در خط `await` مثل یک خطای عادی پرتاب می‌شود و `try...catch` آن را می‌گیرد. گزینه‌های ۱ و ۲ جای روش‌ها را جابه‌جا کرده‌اند، و گزینه‌ی ۳ غلط است چون `try...catch` معمولی خطای callbackی را که بعداً اجرا می‌شود و خطای `.then` را نمی‌گیرد.

---

۷۰۳ **گزینه‌ی ۳** سخت

بدنه‌ی تابع async تا اولین `await` همگام اجرا می‌شود؛ پس `load start` بلافاصله بعد از `script start` چاپ می‌شود (رد گزینه‌ی ۲). `await` فقط همان تابع را معلق می‌کند، نه کل برنامه، پس `script end` قبل از `data ready` می‌آید (رد گزینه‌ی ۱). ادامه‌ی تابع بعد از `await` یک microtask است و قبل از (macrotask) `setTimeout` callback اجرا می‌شود، حتی با تأخیر `0` (رد گزینه‌ی ۴).

---

۷۰۴ **گزینه‌ی ۱** متوسط

اولین `await` مقدار `user loaded` را می‌گیرد و چاپ می‌کند (رد گزینه‌ی ۴). دومین Promise رد می‌شود و `await` آن را مثل یک خطا پرتاب می‌کند؛ اجرا مستقیم به `catch` می‌پرد و خط `after` هرگز اجرا نمی‌شود (رد گزینه‌ی ۲). چون خطا با `try...catch` گرفته شده، برنامه crash نمی‌کند (رد گزینه‌ی ۳) و در پایان بلوک `finally` هم `done` را چاپ می‌کند.

---

۷۰۵ **گزینه‌ی ۲** آسان

JavaScript در مرورگر روی یک thread اصلی اجرا می‌شود که پاسخ به کاربر و رسم صفحه هم کار همان thread است؛ اگر کدی همگام منتظر شبکه بماند، صفحه قفل می‌شود. پس کار را شروع می‌کنیم و callback یا Promise را برای «بعداً» ثبت می‌کنیم. گزینه‌ی ۱ برعکس واقعیت است (یک thread اصلی). async کار را سریع‌تر نمی‌کند، فقط منتظر ماندن را غیرمسدودکننده (non-blocking) می‌کند (رد گزینه‌ی ۳). گزینه‌ی ۴ هم ساختگی است؛ کد همگام همیشه مجاز است.

---

۷۰۶ **گزینه‌ی ۳** آسان

callbackهای تو در تو که کد را به شکل هرم درمی‌آورند همان «callback hell» اسلاید ۶ هستند؛ خواندن و مدیریت خطای آن‌ها (بررسی دستی `err` در هر سطح) سخت است. راه‌حل، Promiseهایی است که با `.then()` زنجیر می‌شوند، یا `async/await` که کد را شبیه کد همگام می‌کند. هم‌نام بودن `err`ها مشکل اصلی نیست، و این کد ربطی به TDZ یا DOM ندارد.

---

۷۰۷ **گزینه‌ی ۴** متوسط

callbackهای `.then` و `.catch` همیشه ناهمگام (microtask) هستند، پس اول `sync` چاپ می‌شود (رد گزینه‌های ۱ و ۳). در زنجیره، `5 * 2` برابر `10` است و چون `10 > 5` است خطا پرتاب می‌شود؛ خطا از `.then` بعدی رد می‌شود و مستقیم به `.catch` می‌رسد. پس `value: 10` هرگز چاپ نمی‌شود (رد گزینه‌ی ۲) و خروجی `error: too big` است.

۲-۲ زبان‌های نشانه‌گذاری و درخت DOM

۷۰۸ **گزینه‌ی ۴** آسان

اسلاید ۷ XML، HTML و Markdown را نمونه‌های زبان نشانه‌گذاری می‌آورد؛ این زبان‌ها فقط ساختار سند را با تگ یا نشانه توصیف می‌کنند. JavaScript یک زبان **برنامه‌نویسی** است که شرط، حلقه و منطق دارد و لایه‌ی رفتار صفحه را می‌سازد. Markdown با اینکه به‌جای `<...>` از نشانه‌هایی مثل `#` استفاده می‌کند، باز هم زبان نشانه‌گذاری است.

۷۰۹ **گزینه‌ی ۲** آسان

طبق اسلاید ۹، DOM مخفف **Document Object Model** است: «Document» یعنی صفحه‌ی وب، «Object» یعنی هر بخش صفحه به یک شیء تبدیل می‌شود و «Model» یعنی این اشیاء با هم یک مدل درختی از صفحه می‌سازند. بقیه‌ی گزینه‌ها ساختگی‌اند؛ DOM نه درباره‌ی «Data» است، نه یک زبان نشانه‌گذاری (Markup) و نه یک «Method».

۷۱۰ **گزینه‌ی ۳** آسان

برگ گره‌ای است که **هیچ فرزندی ندارد**. `C` و `D` با اینکه در ردیف دوم هستند، فرزندی ندارند، پس برگ‌اند؛ `G` و `H` هم پایین‌ترین گره‌ها و بدون فرزند هستند. گزینه‌ی ۱ فقط ردیف آخر را دیده و `C` و `D` را جا انداخته است. `E` و `F` هر کدام یک فرزند دارند (رد گزینه‌ی ۲)، و `A` ریشه است و `B` دو فرزند دارد (رد گزینه‌ی ۴).

۷۱۱ **گزینه‌ی ۴** متوسط

هم‌نیاها باید **والد مشترک** داشته باشند. والد `F` گره‌ی `B` است ولی والد `G` گره‌ی `E`؛ پس آن‌ها هم‌نیا نیستند (هم‌نیای `F` فقط `E` است). گزینه‌ی ۱ درست است چون هر سه فرزند `A` اند؛ گزینه‌ی ۲ درست است چون مسیر `A` به `B` به `F` به `H` نشان می‌دهد `A` جدِ `H` است؛ و گزینه‌ی ۳ هم دقیقاً رابطه‌ی نمودار است.

۷۱۲ **گزینه‌ی ۲** آسان

اسلاید ۸ می‌گوید «Attributes become node properties»: attribute به خودِ عنصر چسبیده است و با `a.href` یا `a.getAttribute("href")` خوانده می‌شود. در نمودار W3Schools هم جعبه‌ی `href` کنار `<a>` آمده، نه زیر آن؛ یعنی فرزند نیست (رد گزینه‌ی ۱). attribute عنصر جداگانه هم نیست (رد گزینه‌ی ۳)، و حذف هم نمی‌شود؛ اگر حذف می‌شد لینک‌ها کار نمی‌کردند (رد گزینه‌ی ۴).

۷۱۳ **گزینه‌ی ۱** متوسط

تنها فرزند `<a>` گره‌ی متنی `"Home page"` است، پس `childNodes.length` برابر `1` و `nodeName` آن `#text` است. attributeها (`id` و `href`) فرزند نیستند و در مجموعه‌ی جداگانه‌ی `attributes` قرار دارند که طولش `2` است. گزینه‌ی ۲ و ۳ attributeها را فرزند حساب کرده‌اند، و گزینه‌ی ۴ هم attribute را اولین فرزند فرض کرده و `id` را جا انداخته است.

۷۱۴ **گزینه‌ی ۴** متوسط

`<p>` سه فرزند دارد: گره‌ی متنی `"Hello "`، عنصر `<b>` و گره‌ی متنی `" world"`؛ پس `childNodes.length` برابر `3` است. `children` فقط عنصرها را می‌شمارد که فقط `<b>` است، پس `1`. گزینه‌ی ۱ متن‌ها را گره حساب نکرده، گزینه‌ی ۲ متن قبل و بعد از `<b>` را یکی گرفته، و گزینه‌ی ۳ فکر کرده `children` هم گره‌های متنی را می‌شمارد.

۷۱۵ **گزینه‌ی ۲** متوسط

خط جدید و فاصله‌های بین تگ‌ها گره‌ی متنی می‌سازند؛ پس فرزندان `ul` این‌ها هستند: فاصله، `li`، فاصله، `li`، فاصله، یعنی `5` گره. `children` فقط عنصرها را می‌شمارد: `2`. گزینه‌ی ۱ فاصله‌ها را نادیده گرفته، گزینه‌ی ۳ فقط فاصله‌ی بین دو `li` را دیده، و گزینه‌ی ۴ فکر کرده `children` هم گره‌های متنی را شامل می‌شود.

۷۱۶ **گزینه‌ی ۳** سخت

اولین گره‌ی فرزند `ul` متنِ فاصله (خط جدید و دو فاصله) قبل از اولین `li` است؛ پس `firstChild.nodeName` برابر `#text` است (رد گزینه‌ی ۱). `firstElementChild` گره‌های متنی را رد می‌کند و به `LI` می‌رسد. `lastElementChild` آخرین `li` است که متنش `B` است (رد گزینه‌ی ۲). گزینه‌ی ۴ جای دو property را برعکس گرفته است.

۷۱۷ **گزینه‌ی ۱** متوسط

`nodeType` برای `document` برابر `9`، برای هر عنصر (مثل `body`) برابر `1` و برای گره‌ی متنی برابر `3` است. `p.firstChild` متن `"Hi"` است، نه یک عنصر (رد گزینه‌ی ۴). `document` عنصر نیست، پس `1` نیست (رد گزینه‌ی ۲)، و `2` کد attributeهاست، نه متن (رد گزینه‌ی ۳).

۷۱۸ **گزینه‌ی ۴** متوسط

`nodeName` سند `#document` و گره‌ی متنی `#text` است (با علامت `#`). برای عنصرها `nodeName` نام تگ با حروف بزرگ است و `document.documentElement` همان عنصر ریشه، یعنی `<html>` است؛ پس `HTML`. گزینه‌ی ۲ `documentElement` را با `body` اشتباه گرفته و گزینه‌های ۱ و ۳ قالب `nodeName` را درست نمی‌دانند.

۷۱۹ **گزینه‌ی ۳** آسان

اسلاید ۱۱ صریحاً می‌گوید «Root node is `document`». عنصر `<html>` **عنصر ریشه** (root element) است که فرزند `document` است، نه گره‌ی ریشه (رد گزینه‌ی ۲). `window` اصلاً جزو درخت DOM نیست؛ شیء پنجره است که `document` یکی از propertyهای آن است (رد گزینه‌ی ۱). `<body>` هم فرزند `<html>` است.

۷۲۰ **گزینه‌ی ۲** سخت

`document.documentElement` عنصر ریشه، یعنی `<html>` است و والدش خودِ `document` است؛ پس `HTML true` (رد گزینه‌های ۳ و ۴). `document` دو گره‌ی فرزند دارد: گره‌ی `<!DOCTYPE html>` (که خودش یک گره از نوع DocumentType است) و عنصر `<html>`؛ پس `2`. گزینه‌ی ۱ گره‌ی DOCTYPE را فراموش کرده است.

۷۲۱ **گزینه‌ی ۱** سخت

`div.sampleclass` دو عنصر فرزند دارد (`p` و `h3`) و والدش `body` است (رد گزینه‌ی ۲ که والد را `html` گرفته). عنصر همنیای قبلی `h3` همان `p` است، نه `h1` که بیرون `div` قرار دارد (رد گزینه‌ی ۳). `head` دو عنصر فرزند دارد: `title` و `link` (رد گزینه‌ی ۴). attributeهایی مثل `class` و `id` هم در `children` شمرده نمی‌شوند.

۷۲۲ **گزینه‌ی ۲** سخت

محتوای `<title>` تا رسیدن به یک `</title>` واقعی متن خام حساب می‌شود و تگ‌های داخلش تشخیص داده نمی‌شوند. چون `</title>` ی وجود ندارد، کل باقی فایل (از جمله `<h1>`، `<div>` و `<h3>`) متنِ عنوان می‌شود؛ پس `body` خالی است (`0`)، عنصری با id برابر `2223` وجود ندارد (`true`) و متن `SAMPLE PAGE!` داخل `document.title` است (`true`). parser برای HTML نادرست هیچ‌وقت خطا نمی‌دهد (رد گزینه‌ی ۳)، و گزینه‌های ۱ و ۴ فرض کرده‌اند مرورگر تگ اشتباه را نادیده می‌گیرد و بقیه‌ی صفحه را عادی می‌سازد.

۷۲۳ **گزینه‌ی ۳** متوسط

در نمودار اسلاید ۱۱، `Text` زیر `CharacterData` قرار دارد، نه زیر `Element`؛ داخل تگ بودن متن یعنی «فرزند یک عنصر در درخت DOM بودن»، نه «زیرکلاس Element بودن». این دو را قاطی نکن: درخت DOM نشان می‌دهد کدام گره داخل کدام است، و نمودار کلاس‌ها نشان می‌دهد هر گره از چه نوعی است. سه گزینه‌ی دیگر دقیقاً روابط نمودار هستند: زنجیره‌ی `HTMLParagraphElement` به `Node`، دو زیرکلاس `CharacterData`، و `Attr` زیر `Node`.

۷۲۴ **گزینه‌ی ۴** سخت

`<p>` یک `HTMLParagraphElement` است که از `HTMLElement` ارث می‌برد، پس `true` (رد گزینه‌ی ۳). `p.firstChild` گره‌ی متنی `"Hi"` است: زیرکلاس `CharacterData` هست (`true`) ولی زیرکلاس `Element` نیست (`false`)؛ رد گزینه‌های ۱ و ۲). `document` گره است ولی عنصر نیست (`nodeType` آن `9` است)، پس `instanceof Element` برای آن `false` است.

۷۲۵ **گزینه‌ی ۱** متوسط

طبق اسلاید ۹، DOM یک نمایش درختی از صفحه است که HTML را به JavaScript وصل می‌کند و اجازه‌ی خواندن، تغییر، اضافه و حذف عنصرها را می‌دهد. DOM زبان نیست، بلکه مدلی از اشیاء است (رد گزینه‌ی ۲). مرورگر DOM را از روی فایل در حافظه می‌سازد و تغییر آن فایل را عوض نمی‌کند؛ با refresh همه‌چیز از نو ساخته می‌شود (رد گزینه‌ی ۳). گزینه‌ی ۴ هم با «read, modify, add, or remove» اسلاید ۹ تناقض دارد.

۷۲۶ **گزینه‌ی ۱** آسان

متن داخل هر تگ، گره‌ی متنیِ **فرزندِ همان تگ** می‌شود؛ پس والد `"My title"` عنصر `<title>` است. `<head>` والدِ `<title>` و در نتیجه **جدِ** این متن است، نه والدش (رد گزینه‌ی ۲). `<html>` و `Document` هم اجداد بالاتر هستند.

۷۲۷ **گزینه‌ی ۳** سخت

comment (`<!-- note -->`) هم یک گره در DOM است (از نوع `Comment` با `nodeType` برابر `8`). پس فرزندان `div` سه گره‌اند: comment، عنصر `span` و گره‌ی متنی `"text"`؛ یعنی `childNodes.length` برابر `3` و `children.length` برابر `1`. اولین فرزند همان comment است با `nodeType` برابر `8`. گزینه‌ی ۱ comment را گره ندانسته ولی نوعش را درست گفته، گزینه‌ی ۲ comment را متن (`3`) فرض کرده، و گزینه‌ی ۴ فقط عنصرها را دیده است.

۷۲۸ **گزینه‌ی ۱** متوسط

این همان درخت نمودار Milne در اسلاید ۹ است: `<h1>` و `<a>` هم‌نیا هستند و والد هر دو `body` است (رد گزینه‌ی ۲ که فکر کرده `<a>` فرزند `<h1>` است). عنصر هم‌نیای قبلی `<a>` همان `h1` با id برابر `head` است (رد گزینه‌ی ۳)، و عنصر هم‌نیای بعدی `h1` همان `<a>` است که `href` آن `"#"` است، نه `null` (رد گزینه‌ی ۴).

۳-۲ سلسله‌مراتب DOM در JavaScript و شیء document

۷۲۹ **گزینه‌ی ۲** آسان

اسلاید ۱۲ می‌گوید `window` بالاترین شیء مرورگر و همان global context است و `document` (صفحه‌ی داخل پنجره) یکی از propertyهای آن است. `<html>` عنصر ریشه‌ی درخت DOM است که زیر `document` قرار دارد، و `<body>` هم فرزند `<html>` است؛ هیچ‌کدام بالاترین شیء نیستند.

۷۳۰ **گزینه‌ی ۳** آسان

اسلاید ۱۳: `document` «Represents the entire web page» و «Acts as the entry point to the DOM tree». پنجره، تاریخچه و آدرس مربوط به `window` و فرزندانش (`history` و `location`) هستند (رد گزینه‌ی ۱). `document` شامل `<head>` هم می‌شود، نه فقط `<body>` (رد گزینه‌ی ۲)، و DOM در حافظه است و تغییرش فایل روی دیسک را عوض نمی‌کند (رد گزینه‌ی ۴).

۷۳۱ **گزینه‌ی ۴** متوسط

در نمودار، `forms` (و زیرمجموعه‌اش `elements`) زیر `document` قرار دارد، چون فرم‌ها بخشی از خودِ صفحه هستند. `location` (آدرس صفحه)، `history` (تاریخچه‌ی زبانه) و `navigator` (اطلاعات مرورگر) مستقیماً زیر `window` کشیده شده‌اند؛ این‌ها به پنجره‌ی مرورگر مربوط‌اند، نه به محتوای صفحه.

۷۳۲ **گزینه‌ی ۱** متوسط

در نمودار، `navigator` (اطلاعات خودِ مرورگر) دو فرزند دارد: `plugins` و `mimeTypes` (نوع فایل‌هایی که مرورگر می‌شناسد). `document` هم یک `plugins` دارد ولی `mimeTypes` ندارد (رد گزینه‌ی ۲). `screen` (اطلاعات مانیتور) و `location` (آدرس صفحه) در نمودار فرزندی ندارند.

۷۳۳ **گزینه‌ی ۳** آسان

`document` یکی از propertyهای `window` است، پس `window.document === document` برابر `true` است (رد گزینه‌ی ۱). `document.documentElement` عنصر ریشه، یعنی `<html>` است، نه `<body>` (رد گزینه‌ی ۲) و نه خودِ `document` (رد گزینه‌ی ۴). `document.body` و `document.head` هم مستقیماً به `<body>` و `<head>` اشاره می‌کنند.

۷۳۴ **گزینه‌ی ۲** آسان

با هر کلیک، listener مقدار `textContent` عنوان را **جایگزین** می‌کند (`=` ، نه `+=`)؛ پس کلیک اول متن را عوض می‌کند و کلیک دوم همان متن را دوباره می‌نویسد. نتیجه `Text Changed via document!` است، نه دو بار تکرار آن (رد گزینه‌ی ۳). اسکریپت بعد از `<h1>` آمده، پس `heading` برابر `null` نیست و خطایی رخ نمی‌دهد (رد گزینه‌ی ۴)، و متن هم تغییر کرده است (رد گزینه‌ی ۱).

۷۳۵ **گزینه‌ی ۱** متوسط

همان‌طور که اسلاید ۱۵ نشان می‌دهد، **خودِ عنصر** چاپ می‌شود (`▶ <h1 id="title">` همراه با لینک `doc.html:8:15`)، چون خودِ عنصر را به `console.log` داده‌ایم؛ برای دیدن متن باید `heading.textContent` را چاپ کرد (رد گزینه‌ی ۲). اسکریپت بعد از `<h1>` قرار دارد، پس عنصر از قبل در درخت هست و `null` نمی‌گیریم (رد گزینه‌ی ۳). نبودن `;` هم خطا نیست، چون JavaScript سمی‌کالن را خودکار درج می‌کند (رد گزینه‌ی ۴).

۷۳۶ **گزینه‌ی ۳** سخت

اسکریپت داخل `<head>` است و وقتی اجرا می‌شود، مرورگر هنوز به `<h1>` نرسیده؛ پس `getElementById` چیزی پیدا نمی‌کند و `null` برمی‌گرداند (نه `undefined`؛ رد گزینه‌ی ۲). خط بعد می‌خواهد روی `null` property بنویسد و `TypeError` می‌دهد، پس `done` هرگز چاپ نمی‌شود. گزینه‌ی ۱ فرض کرده عنصر پیدا می‌شود، و `ReferenceError` هم نیست چون متغیر `heading` تعریف شده است، فقط مقدارش `null` است (رد گزینه‌ی ۴).

۷۳۷ **گزینه‌ی ۲** سخت

اسکریپت `<head>` قبل از ساخته شدن `<h1>` اجرا می‌شود، پس `null` می‌بیند (رد گزینه‌ی ۳). اسکریپت انتهای `<body>` هنگام parse اجرا می‌شود، یعنی قبل از اینکه parse کل سند تمام شود؛ پس پیش از `DOMContentLoaded` می‌آید (رد گزینه‌ی ۱). `DOMContentLoaded` به محض آماده شدن درخت DOM رخ می‌دهد، ولی `load` منتظر همه‌ی منابع می‌ماند و همیشه بعد از آن است (رد گزینه‌ی ۴).

۷۳۸ **گزینه‌ی ۳** متوسط

مشکل، زمان اجراست: کد قبل از ساخته شدن `<body>` اجرا می‌شود. `DOMContentLoaded` اجرای کد را تا آماده شدن کامل درخت DOM عقب می‌اندازد، پس عنصر پیدا می‌شود. `defer` فقط روی اسکریپت‌های خارجی (با `src`) اثر دارد و روی اسکریپت درون‌خطی نادیده گرفته می‌شود (رد گزینه‌ی ۱). نوع تعریف متغیر ربطی به وجود عنصر ندارد (رد گزینه‌ی ۲)، و `getElementById` نام id را **بدون** `#` می‌گیرد؛ با `#` حتماً `null` برمی‌گرداند (رد گزینه‌ی ۴).

۷۳۹ **گزینه‌ی ۳** متوسط

`heading` یک شیء از کلاس `HTMLHeadingElement` است؛ پس `typeof` آن `"object"` است (`typeof` هیچ‌وقت `"element"` برنمی‌گرداند؛ رد گزینه‌ی ۴). تبدیل یک شیء DOM به رشته با `String(...)` فقط `[object HTMLHeadingElement]` می‌دهد، نه متن یا کد HTML آن (رد گزینه‌های ۱ و ۲). متن داخل عنصر را باید از `textContent` خواند که `Hello World` است.

۷۴۰ **گزینه‌ی ۴** آسان

طبق اسلاید ۱۲، attributeها و متن propertyهای عنصر هستند: `id` مقدار `title` را بدون `#` می‌دهد و `className` مقدار attribute `class` را بدون نقطه (رد گزینه‌ی ۱). property کلاس در DOM `className` نام دارد و وجود دارد، پس `undefined` نمی‌دهد (رد گزینه‌ی ۲). `textContent` فقط متن را می‌دهد، نه کد HTML عنصر (رد گزینه‌ی ۳)، و `tagName` نام تگ را با حروف بزرگ (`H1`) برمی‌گرداند.

۷۴۱ **گزینه‌ی ۱** متوسط

`document.title` متن عنصر `<title>` (عنوان زبانه‌ی مرورگر) را می‌دهد، نه متن `<h1>`؛ پس اول `Shop` چاپ می‌شود (رد گزینه‌های ۲ و ۴). نوشتن در `document.title` خودِ عنصر `<title>` را به‌روز می‌کند؛ بنابراین `textContent` آن هم `Cart (2)` می‌شود (رد گزینه‌ی ۳). سایت‌ها از همین ترفند برای نشان دادن تعداد پیام یا سبد خرید در عنوان زبانه استفاده می‌کنند.

۷۴۲ **گزینه‌ی ۴** آسان

رابطه برعکس است: طبق اسلاید ۱۲، `window` بالاترین شیء است و `document` یکی از فرزندان (propertyهای) آن؛ یعنی `window.document` درست است، نه `document.window`. سه گزینه‌ی دیگر عیناً بندهای اسلاید ۱۳ هستند: نقطه‌ی ورود به DOM، متدهای پیدا کردن عنصر، و امکان ساختن، حذف و به‌روزرسانی گره‌ها.

۷۴۳ گزینه‌ی ۱ سخت

اسکریپت وسط صفحه وقتی اجرا می‌شود که `<h1>` ساخته شده ولی مرورگر هنوز به `<h2>` نرسیده؛ پس `a` پیدا می‌شود و `b` نه (`true false` ؛ رد گزینه‌های ۲ و ۳). اسکریپت دوم در انتهای `<body>` است و آن موقع `<h2>` هم ساخته شده، پس متنش `B` چاپ می‌شود و خطایی رخ نمی‌دهد (رد گزینه‌ی ۴). هر اسکریپت فقط عنصرهای بالای خودش را می‌بیند.

۷۴۴ گزینه‌ی ۲ متوسط

`DOMContentLoaded` روی `document` به محض کامل شدن درخت DOM رخ می‌دهد و منتظر تصاویر نمی‌ماند؛ `load` روی `window` تا بارگیری همه‌ی منابع صبر می‌کند، پس همیشه دیرتر است (رد گزینه‌های ۱ و ۳). اسکریپت‌های معمولی صفحه هنگام parse اجرا می‌شوند، یعنی **قبل** از `DOMContentLoaded` ، نه بعد از آن (رد گزینه‌ی ۴).

۷۴۵ گزینه‌ی ۴ سخت

دکمه داخل `DOMContentLoaded` جست‌وجو شده و پیدا می‌شود، پس listener ثبت می‌شود و کلیک بی‌اثر نیست (رد گزینه‌ی ۲). اما `heading` **بیرون** از `DOMContentLoaded` و زمانی جست‌وجو شده که `<h1>` هنوز ساخته نشده بود؛ پس مقدارش برای همیشه `null` مانده است. با کلیک، `heading.textContent = ...` روی `null` اجرا می‌شود و `TypeError` می‌دهد، پس `changed` چاپ نمی‌شود (رد گزینه‌های ۱ و ۳).

۷۴۶ گزینه‌ی ۴ متوسط

ویژگی `defer` روی اسکریپت خارجی (دارای `src`) باعث می‌شود فایل هم‌زمان با parse دانلود شود ولی بعد از پایان parse (و قبل از `DOMContentLoaded`) اجرا شود. روی اسکریپت درون‌خطی، `defer` نادیده گرفته می‌شود (رد گزینه‌ی ۱). `<script src>` معمولی parse را متوقف می‌کند و همان لحظه اجرا می‌شود (رد گزینه‌ی ۲)، و ویژگی‌ای به نام `delay` در HTML وجود ندارد (رد گزینه‌ی ۳).

۴-۲ پیدا کردن عناصر (Querying the DOM)

۷۴۷ گزینه‌ی ۳ آسان

طبق اسلاید ۱۶، `getElementById` «finds one element by its ID»؛ چون id باید یکتا باشد، خروجی یک عنصر است (نام متد هم مفرد است: Element). اگر عنصری پیدا نشود `null` برمی‌گرداند. HTMLCollection خروجی `getElementsBy...` است (گزینه‌ی ۱) و NodeList خروجی `querySelectorAll` (گزینه‌ی ۲)؛ هیچ‌کدام از این متدها آرایه‌ی واقعی برنمی‌گردانند (گزینه‌ی ۴).

۷۴۸ گزینه‌ی ۲ آسان

`getElementById` نام خالی id را می‌گیرد و `#` را جزو نام حساب می‌کند؛ عنصری با id برابر `#title` نداریم، پس `null` چاپ می‌شود (رد گزینه‌ی ۱). `querySelector` یک CSS selector می‌گیرد و `#title` یعنی «عنصر با id برابر title»، پس متن `Shop` چاپ می‌شود (رد گزینه‌ی ۳). `getElementById` هرگز به‌خاطر رشته‌ی ورودی‌اش خطای نحوی نمی‌دهد (رد گزینه‌ی ۴).

۷۴۹ گزینه‌ی ۴ آسان

طبق اسلاید ۱۶، `querySelector` اولین تطابق را «using CSS selectors» پیدا می‌کند (و `querySelectorAll` همه‌ی تطابق‌ها را). سه متد دیگر فقط یک **نام** ساده می‌گیرند: نام id، نام کلاس یا نام تگ؛ به همین دلیل `getElementById("#x")` یا `getElementsByClassName(".x")` چیزی پیدا نمی‌کنند.

۷۵۰ گزینه‌ی ۱ متوسط

`div p` (با فاصله) یعنی هر `<p>` ی که **هر جایی داخل** یک `div` باشد: `Inside box` و `Deep` ، پس `2` (پاراگراف `First` بیرون `div` است؛ رد گزینه‌ی ۴). `div > p` فقط **فرزند مستقیم** را می‌گیرد؛ `Deep` داخل `<section>` است و فرزند مستقیم `div` نیست، پس `1` (رد گزینه‌ی ۲). `getElementsByTagName("p")` همه‌ی `<p>` های صفحه را می‌شمارد: `3` . گزینه‌ی ۳ فرق «نواده» و «فرزند مستقیم» را نمی‌داند.

۷۵۱ **گزینه‌ی ۲** سخت

`getElementsByClassName("card big")` عنصرهایی را می‌دهد که **هر دو** کلاس را داشته باشند و ترتیب نوشتن کلاس‌ها مهم نیست؛ پس `div`های ۱ و ۲ (`2`؛ رد گزینه‌ی ۱ که فکر کرده ترتیب مهم است، و گزینه‌ی ۴ که فکر کرده یکی از کلاس‌ها کافی است). `.card.big` (بدون فاصله) هم همین معنا را دارد: `2`. اما `.card .big` (با فاصله) یعنی «عنصر `.big` **داخل** یک `.card`» که اینجا وجود ندارد: `0` (رد گزینه‌ی ۳).

۷۵۲ **گزینه‌ی ۴** متوسط

`getElementsByClassName` یک `HTMLCollection` و `querySelectorAll` یک `NodeList` برمی‌گرداند (رد گزینه‌ی ۱ که جایشان را عوض کرده). هیچ‌کدام آرایه‌ی واقعی نیستند (رد گزینه‌ی ۲). HTMLCollection متد `forEach` ندارد، پس `typeof` آن `undefined` است، ولی NodeList متد `forEach` دارد (رد گزینه‌ی ۳).

۷۵۳ **گزینه‌ی ۳** متوسط

`buttons` یک HTMLCollection است: `length` دارد (پس `3` چاپ می‌شود و گزینه‌ی ۴ غلط است) ولی متد `forEach` ندارد. `buttons.forEach` برابر `undefined` است و صدا زدن آن خطای `TypeError: buttons.forEach is not a function` می‌دهد؛ اجرای اسکریپت همان‌جا متوقف می‌شود و `done` چاپ نمی‌شود (رد گزینه‌های ۱ و ۲). راه‌حل: `Array.from(buttons).forEach(...)` یا `querySelectorAll(".btn").forEach(...)`.

۷۵۴ **گزینه‌ی ۲** متوسط

فقط `items.forEach` خطا می‌دهد، چون HTMLCollection متد `forEach` ندارد (رد گزینه‌ی ۱). `Array.from(items)` و `[...items]` مجموعه را به آرایه‌ی واقعی تبدیل می‌کنند که `forEach` دارد، و HTMLCollection قابل پیمایش (iterable) است؛ پس هم spread و هم `for...of` روی آن کار می‌کنند (رد گزینه‌های ۳ و ۴).

۷۵۵ **گزینه‌ی ۳** سخت

HTMLCollection `getElementsByClassName` **زنده** است و `li` جدید (که کلاس `item` دارد) خودبه‌خود به آن اضافه می‌شود: `3` (رد گزینه‌ی ۲). NodeList `querySelectorAll` **ایستا** است، یعنی عکسی فوری از لحظه‌ی جست‌وجو، و همان `2` عضو را نگه می‌دارد (رد گزینه‌ی ۱). گزینه‌ی ۴ زنده و ایستا را برعکس گرفته است.

۷۵۶ **گزینه‌ی ۲** سخت

`reds` زنده است و هر عنصری که کلاس `red` را از دست بدهد فوراً از آن بیرون می‌رود. وقتی `i = 0` است، پاراگراف ۱ حذف می‌شود و مجموعه `[2, 3, 4]` می‌شود؛ وقتی `i = 1`، پاراگراف ۳ حذف می‌شود و مجموعه `[2, 4]` می‌شود؛ حالا `i = 2` دیگر کمتر از `length` (یعنی `2`) نیست و حلقه تمام می‌شود. پس پاراگراف‌های ۲ و ۴ هنوز `red` هستند: `2` (رد گزینه‌ی ۱). خطایی هم رخ نمی‌دهد، چون همیشه `i < reds.length` بررسی می‌شود (رد گزینه‌ی ۴).

۷۵۷ **گزینه‌ی ۱** متوسط

`querySelector` **اولین** عنصر منطبق را به ترتیب سند برمی‌گرداند. `input` selector همه‌ی inputها را قبول می‌کند و اولینشان چک‌باکس `agree` است (رد گزینه‌ی ۲). `input[type='text']` (attribute selector) فقط inputهایی را قبول می‌کند که `type` آن‌ها `text` است؛ اولینشان `user` است (رد گزینه‌ی ۳). گزینه‌ی ۴ فکر کرده هر دو selector فقط متنی‌ها را می‌گیرند، یا اینکه `querySelector` آخرین مورد را می‌دهد.

۷۵۸ **گزینه‌ی ۳** سخت

selector گروهی `.note, h1` یعنی «هر عنصری که `.note` **یا** `h1` باشد»؛ پس دو پاراگراف و یک `h1`، یعنی `3` عنصر (رد گزینه‌های ۲ و ۴). `querySelectorAll` نتیجه را همیشه به **ترتیب سند** برمی‌گرداند، نه به ترتیب نوشتن selectorها؛ `h1` در صفحه اول آمده، پس `list[0]` همان `H1` است (رد گزینه‌ی ۱).

۷۵۹ **گزینه‌ی ۴** آسان

متدهای «یکی» (`getElementById` و `querySelector`) وقتی چیزی پیدا نکنند `null` برمی‌گردانند، نه `undefined` (رد گزینه‌ی ۲). متدهای «همه» (`querySelectorAll` و `getElementsByClassName`) هیچ‌وقت `null` نمی‌دهند؛ یک مجموعه‌ی خالی برمی‌گردانند که `length` آن `0` است (رد گزینه‌های ۱ و ۳). به همین دلیل `.length` گرفتن از نتیجه‌ی آن‌ها همیشه امن است.

۷۶۰ **گزینه‌ی ۴** سخت

`getElementById` فقط id را مقایسه می‌کند و با رقم اول مشکلی ندارد، پس خط اول درست کار می‌کند (رد گزینه‌ی ۳). اما در CSS، id داخل selector نمی‌تواند با رقم شروع شود؛ `"#2223"` یک selector **نامعتبر** است و `querySelector` به‌جای برگرداندن `null`، خطای `SyntaxError` پرتاب می‌کند (رد گزینه‌ی ۲ که فکر کرده `null` برگشته و `.textContent` خطا داده است). اجرا متوقف می‌شود و `end` چاپ نمی‌شود (رد گزینه‌ی ۱). راه درست: `querySelector('[id="2223"]')` یا `getElementById("2223")`.

۷۶۱ **گزینه‌ی ۱** متوسط

وقتی `querySelector` روی یک عنصر صدا زده شود، فقط **داخل همان عنصر** می‌گردد؛ اولین `<p>` داخل `.box` همان `Inside box` است (رد گزینه‌ی ۲) و داخل آن فقط یک `.note` (یعنی `Deep`) وجود دارد (رد گزینه‌ی ۳). ولی `document.querySelector(".note")` کل صفحه را می‌گردد و اولین `.note` به ترتیب سند `First` است (رد گزینه‌ی ۴).

۷۶۲ **گزینه‌ی ۲** سخت

`querySelector` روی عنصرها هم وجود دارد و داخل `menu` لینک `Home` را پیدا می‌کند، پس خط اول درست است (رد گزینه‌ی ۴). اما `getElementById` فقط متد `document` است و روی عنصرها تعریف نشده؛ `menu.getElementById` برابر `undefined` است و صدا زدنش `TypeError: menu.getElementById is not a function` می‌دهد (رد گزینه‌ی ۱). گزینه‌ی ۳ هم غلط است، چون حتی اگر `null` برمی‌گشت، `.textContent` روی آن باز خطا می‌داد.

۷۶۳ **گزینه‌ی ۱** آسان

`[attr='value']` یک attribute selector است: `input[type='text']` یعنی «تگ `input` که attribute `type` آن دقیقاً `text` باشد»، مثل خط ۴ فعالیت اسلاید ۱۷. کلاس با نقطه (`.text`) و id با `#` (`#text`) انتخاب می‌شوند (رد گزینه‌های ۲ و ۳). گزینه‌ی ۴ هم معنای گروه (`,`) را دارد که اینجا به کار نرفته است.

۷۶۴ **گزینه‌ی ۳** متوسط

`querySelector` فقط **اولین** عنصر منطبق را برمی‌گرداند (یا `null`)، نه یک NodeList؛ برای همه‌ی inputهای متنی باید `querySelectorAll` می‌نوشتیم. پس گزینه‌ی ۳ نادرست است. بقیه درست‌اند: `getElementById` یک عنصر یا `null` می‌دهد، `getElementsByClassName` مجموعه‌ای زنده است که عنصرهای چندکلاسه را هم شامل می‌شود، و `querySelectorAll("a.nav-link")` یک NodeList ایستا از لینک‌های دارای کلاس `nav-link` است.

۷۶۵ **گزینه‌ی ۳** متوسط

id باید یکتا باشد، ولی مرورگر برای HTML نادرست خطا نمی‌دهد. `getElementById` همیشه **اولین** عنصر با آن id را به ترتیب سند برمی‌گرداند، یعنی `first` (رد گزینه‌های ۱ و ۴). `querySelectorAll("#x")` هر عنصری را که با selector جور باشد برمی‌گرداند، پس هر دو را پیدا می‌کند: `2` (رد گزینه‌ی ۲). نتیجه: id تکراری باعث رفتار گیج‌کننده می‌شود؛ برای گروه‌ها از کلاس استفاده کن.

۷۶۶ **گزینه‌ی ۴** متوسط

`#menu` عنصری با id برابر `menu` را انتخاب می‌کند و `>` یعنی «فرزند مستقیم»؛ پس `#menu > li` دقیقاً `li`های سطح اول همان لیست را می‌دهد. `.menu` دنبال **کلاس** `menu` می‌گردد (گزینه‌ی ۱) و `menu` بدون علامت دنبال **تگی** به نام `<menu>` (گزینه‌ی ۲). ویرگول در گزینه‌ی ۳ یعنی گروه: «عنصر `#menu` و همه‌ی `li`های صفحه».

۷۶۷ **گزینه‌ی ۱** سخت

NodeList `querySelectorAll` **ایستا** است: با حذف کلاس، عنصرها از `reds` بیرون نمی‌روند و `forEach` روی هر ۴ پاراگراف اجرا می‌شود؛ پس جست‌وجوی تازه‌ی `.red` چیزی پیدا نمی‌کند: `0` (رد گزینه‌های ۲ و ۴ که دام حلقه روی مجموعه‌ی **زنده** را اینجا اشتباهی به کار برده‌اند). خودِ `reds` هنوز همان ۴ عنصر را نگه داشته است: `4` (رد گزینه‌ی ۳).

۷۶۸ **گزینه‌ی ۲** آسان

HTMLCollection شبه‌آرایه است: `length` برابر `2`، `buttons[0]` اولین دکمه (`A`) و `item(1)` دومین دکمه (`B`) است (رد گزینه‌ی ۳). خواندن اندیسی که وجود ندارد (`buttons[5]`) مثل آرایه `undefined` می‌دهد (رد گزینه‌ی ۱؛ فقط `buttons.item(5)` مقدار `null` برمی‌گرداند). گزینه‌ی ۴ هم تعداد دکمه‌ها را اشتباه شمرده است.

۷۶۹ **گزینه‌ی ۴** سخت

هیچ لینکی هر دو کلاس `nav-link` و `active` را ندارد، پس `querySelector` در خط ۳ مقدار `null` برمی‌گرداند؛ پیدا نکردن عنصر خطا **نیست** (رد گزینه‌ی ۱). خواندن `links[0]` از یک HTMLCollection مثل آرایه کار می‌کند و اولین لینک را می‌دهد (رد گزینه‌ی ۲)، و خط ۵ متن `Home` را چاپ می‌کند (رد گزینه‌ی ۳). اما در خط ۶ برنامه می‌خواهد `textContent` را از `null` بخواند و خطای `TypeError: Cannot read properties of null` می‌گیرد. درس این تست: نتیجه‌ی `querySelector` و `getElementById` را قبل از استفاده بررسی کن (`if (active) ...`).

۲-۵ رویدادها (Events) و addEventListener

۷۷۰ **گزینه‌ی ۳** آسان

اسلاید ۱۸ می‌گوید «addEventListener is the modern way to attach events» و «Multiple handlers can be added to the same event». property و attribute هر کدام فقط یک handler نگه می‌دارند و مقداردهی دوباره قبلی را پاک می‌کند (رد گزینه‌های ۱ و ۲). `document.write` اصلاً ربطی به رویدادها ندارد و فقط متن در سند می‌نویسد.

۷۷۱ **گزینه‌ی ۲** آسان

در `addEventListener` نام رویداد **بدون پیشوند on** نوشته می‌شود: `"click"`. پیشوند `on` فقط در attribute و property به کار می‌رود (`onclick`). `"onclick"` و `"onClick"` رویدادهایی هستند که هرگز رخ نمی‌دهند (نام‌ها به حروف بزرگ و کوچک هم حساس‌اند)، و رویدادی به نام `"clicked"` هم وجود ندارد؛ در هیچ‌کدام خطایی نمی‌گیری، فقط listener هیچ‌وقت اجرا نمی‌شود.

۷۷۲ **گزینه‌ی ۴** متوسط

listener اول برای رویدادی به نام `"onclick"` ثبت شده که هیچ‌وقت رخ نمی‌دهد؛ مرورگر هم خطایی نمی‌دهد و فقط آن listener هرگز اجرا نمی‌شود (رد گزینه‌های ۱ و ۲). listener دوم با نام درست `"click"` ثبت شده و با کلیک اجرا می‌شود، پس خروجی خالی هم نیست (رد گزینه‌ی ۳).

۷۷۳ **گزینه‌ی ۲** سخت

property `onclick` فقط یک handler نگه می‌دارد و مقداردهی دوم، `A` را با `B` جایگزین می‌کند (رد گزینه‌های ۱ و ۳). listenerهای `addEventListener` روی هم جمع می‌شوند و به ترتیب ثبت اجرا می‌شوند. خانه‌ی `onclick` اولین بار قبل از دو listener دیگر پر شده، پس جایگاهش در صف قبل از آن‌هاست و `B` اول اجرا می‌شود (رد گزینه‌ی ۴).

۷۷۴ **گزینه‌ی ۱** سخت

attribute `onclick="..."` و property `btn.onclick` در واقع **یک خانه‌ی مشترک** هستند؛ مقداردهی property همان handler HTML را جایگزین می‌کند، پس `attr` دیگر چاپ نمی‌شود (رد گزینه‌های ۲ و ۳). آن خانه از لحظه‌ی خوانده شدن HTML، یعنی قبل از `addEventListener`، در صف listenerها جا گرفته؛ پس `property` قبل از `listener` اجرا می‌شود (رد گزینه‌ی ۴).

۷۷۵ **گزینه‌ی ۳** متوسط

`sayHi()` با پرانتز یعنی «همین الان اجرا کن»؛ پس `hi` هنگام اجرای اسکریپت و قبل از `ready` چاپ می‌شود. مقدار برگشتی آن `undefined` است که به‌عنوان listener ثبت می‌شود، یعنی هیچ‌چیز؛ در نتیجه کلیک‌ها هیچ خروجی‌ای ندارند (رد گزینه‌های ۱ و ۲). `addEventListener` با listener برابر `undefined` خطا نمی‌دهد و آن را بی‌صدا نادیده می‌گیرد (رد گزینه‌ی ۴). درست آن است که `sayHi` را بدون پرانتز بدهیم.

۷۷۶ **گزینه‌ی ۴** سخت

ثبت دوباره‌ی **همان مرجع تابع** (`sayHi`) برای همان رویداد نادیده گرفته می‌شود، پس `hi` فقط یک بار چاپ می‌شود (رد گزینه‌های ۱ و ۳). اما دو arrow function با متن یکسان دو **شیء تابع متفاوت** هستند و هر دو ثبت می‌شوند، پس `arrow` دو بار چاپ می‌شود (رد گزینه‌ی ۲).

۷۷۷ **گزینه‌ی ۲** متوسط

`removeEventListener` فقط وقتی کار می‌کند که **همان شیء تابع** را بگیرد. در خط سوم یک arrow function **تازه** ساخته شده که با listener ثبت‌شده فقط هم‌متن است، نه یکی؛ پس چیزی حذف نمی‌شود و `anon` چاپ می‌شود (رد گزینه‌های ۱ و ۴). ولی `named` با همان مرجع حذف شده و اجرا نمی‌شود (رد گزینه‌ی ۳).

۷۷۸ **گزینه‌ی ۳** متوسط

گزینه‌ی `{ once: true }` یعنی listener بعد از **اولین** اجرا خودبه‌خود حذف شود؛ پس `once 1` فقط با کلیک اول چاپ می‌شود (رد گزینه‌ی ۲) و اجرا هم می‌شود (رد گزینه‌ی ۴). listener دوم چنین تنظیمی ندارد و با هر سه کلیک اجرا می‌شود (رد گزینه‌ی ۱ که فکر کرده `once` روی همه‌ی listenerهای دکمه اثر دارد).

۷۷۹ **گزینه‌ی ۱** متوسط

`input` با **هر** تغییر مقدار رخ می‌دهد، یعنی یک بار بعد از `A` و یک بار بعد از `Al` (رد گزینه‌ی ۳). در کادر متنی، `change` فقط وقتی رخ می‌دهد که کاربر کارش را تمام کند و از کادر بیرون برود، پس فقط یک بار و در پایان با مقدار نهایی `Al` (رد گزینه‌های ۲ و ۴ که `change` را با هر حرف فرض کرده‌اند).

۷۸۰ **گزینه‌ی ۳** آسان

`keydown` با هر کلید رخ می‌دهد و `e.key` نام کلید را می‌دهد. برای کلید اول `e.key` برابر `"a"` است و شاخه‌ی `else` اجرا می‌شود؛ برای کلید دوم برابر `"Enter"` است و `submit search` چاپ می‌شود. ترتیب خروجی همان ترتیب فشردن کلیدهاست (رد گزینه‌ی ۱)، شرط برای Enter درست است (رد گزینه‌ی ۲)، و کلید `a` هم رویداد دارد (رد گزینه‌ی ۴).

۷۸۱ **گزینه‌ی ۴** آسان

`load` وقتی رخ می‌دهد که صفحه (یا یک منبع مثل تصویر) کاملاً بارگذاری شده باشد؛ رویداد فشردن کلید `keydown` است. سه جفت دیگر درست‌اند و همه از رویدادهای رایج اسلاید ۱۸ هستند: `click` برای کلیک، `input` برای تغییر مقدار کادر ورودی و `submit` برای ارسال فرم.

۷۸۲ **گزینه‌ی ۲** سخت

`querySelector('button')` فقط **اولین** `<button>` صفحه را برمی‌گرداند که اینجا دکمه‌ی Reset است؛ پس listener روی Reset ثبت شده و کلیک روی CLICK ME! هیچ کاری نمی‌کند. متن عنوان همان `Hello World` می‌ماند و رنگ inline آن خالی است (رد گزینه‌های ۱ و ۳). خطایی هم رخ نمی‌دهد، چون عنصرها وجود دارند (رد گزینه‌ی ۴). برای انتخاب دکمه‌ی درست باید از `querySelector('#go')` استفاده می‌کردیم.

۷۸۳ **گزینه‌ی ۱** متوسط

listener با **هر** کلیک اجرا می‌شود، پس `clicks` سه بار زیاد می‌شود و `3` است (رد گزینه‌ی ۲ که فکر کرده listener یک بار مصرف است). `textContent = ...` متن را **جایگزین** می‌کند، نه اینکه به آن اضافه کند؛ پس متن نهایی یک بار `Goodbye World` است (رد گزینه‌ی ۳). گزینه‌ی ۴ فرض کرده کلیک‌ها اثری نداشته‌اند.

۷۸۴ **گزینه‌ی ۳** آسان

اسلاید ۱۸ صریحاً می‌گوید «Multiple handlers can be added to the same event»؛ با `addEventListener` هر تعداد listener می‌توان ثبت کرد و همه به ترتیب ثبت اجرا می‌شوند. محدودیت «فقط یک handler» مربوط به `onclick` (property یا attribute) است. سه گزینه‌ی دیگر عیناً بندهای همان اسلاید هستند.

۷۸۵ **گزینه‌ی ۱** متوسط

مرورگر listener را با مقایسه‌ی **مرجع** پیدا می‌کند: نوع رویداد باید یکی باشد و تابع باید دقیقاً همان شیء باشد (و اگر capture تنظیم شده، همان تنظیم). دو تابع هم‌متن، دو شیء متفاوت‌اند (رد گزینه‌ی ۳)؛ برای همین listener ناشناس قابل حذف نیست. `removeEventListener` بدون تابع چیزی حذف نمی‌کند (رد گزینه‌ی ۲)، و نوع تابع (arrow یا معمولی) هم اهمیتی ندارد (رد گزینه‌ی ۴).

۷۸۶ **گزینه‌ی ۴** آسان

`load` روی `window` آخرین رویداد بارگذاری است و منتظر همه‌ی منابع می‌ماند؛ حتی بعد از `DOMContentLoaded` رخ می‌دهد. اسکریپت‌های صفحه قبل از آن اجرا می‌شوند (رد گزینه‌ی ۱)، با کلیک ربطی ندارد (رد گزینه‌ی ۲)، و قبل از parse هم نمی‌تواند باشد، چون اسکریپتی که listener را ثبت می‌کند خودش بخشی از HTML است (رد گزینه‌ی ۳).

۷۸۷ **گزینه‌ی ۳** سخت

listener اول یک arrow function است که خودش `greet("Sara")` را صدا می‌زند، پس `name` رشته‌ی `"Sara"` است: `string Sara` . در listener دوم، خودِ `greet` ثبت شده و مرورگر هنگام کلیک آن را با **شیء رویداد** به‌عنوان اولین آرگومان صدا می‌زند؛ پس `name` یک شیء است (`typeof` برابر `"object"`) که نمونه‌ای از `Event` است. مرورگر حتماً آرگومان می‌فرستد، پس `undefined` نیست (رد گزینه‌ی ۱)، `"Sara"` هم به خاطر سپرده نمی‌شود (رد گزینه‌ی ۲)، و هیچ عملی در این کد روی شیء رویداد خطا نمی‌دهد (رد گزینه‌ی ۴). اگر `greet` می‌نوشت `"Hello " + name`، برای کلیک دوم چیزی مثل `Hello [object PointerEvent]` چاپ می‌شد.

۷۸۸ **گزینه‌ی ۲** متوسط

با کلیک اول، `handler` اجرا می‌شود، `clicked` را چاپ می‌کند و بعد **خودش** را با همان مرجع حذف می‌کند؛ پس کلیک‌های دوم و سوم هیچ listenerای ندارند (رد گزینه‌ی ۱). حذف بعد از چاپ انجام شده، پس خروجی خالی نیست (رد گزینه‌ی ۳)، و حذف یک listener از داخل خودش کاملاً مجاز است (رد گزینه‌ی ۴). این رفتار معادل `{ once: true }` است.

۷۸۹ **گزینه‌ی ۱** آسان

`onclick` یک property معمولی است و مثل هر property دیگری فقط یک مقدار دارد؛ مقداردهی دوم، اولی را پاک می‌کند (رد گزینه‌ی ۲). این property در JavaScript استفاده می‌شود و نسخه‌ی HTML آن attribute `onclick` است (رد گزینه‌ی ۳). «بدون on» قاعده‌ی `addEventListener` است؛ `btn.click` یک متد دیگر است (کلیک برنامه‌نویسی‌شده روی دکمه) و مقداردهی آن handler ثبت نمی‌کند (رد گزینه‌ی ۴).

۷۹۰ **گزینه‌ی ۴** سخت

مرورگر رشته‌ی attribute را به یک **تابع** تبدیل می‌کند و در property `onclick` می‌گذارد، پس `typeof` آن `"function"` است، در حالی که `getAttribute` همان متن خام را برمی‌گرداند (رد گزینه‌ی ۱). داخل handler attribute، `this` همان دکمه است و `save` چاپ می‌شود (رد گزینه‌ی ۲). این handler از هنگام parse شدن HTML ثبت شده، یعنی قبل از `addEventListener`، پس قبل از `listener` اجرا می‌شود (رد گزینه‌ی ۳).

۷۹۱ **گزینه‌ی ۴** متوسط

دکمه‌ی `save` وجود دارد، پس خط ۲ بدون مشکل listener ثبت می‌کند (رد گزینه‌ی ۱). عنصری با id برابر `cancel` نیست؛ `getElementById` در این حالت خطا نمی‌دهد و فقط `null` برمی‌گرداند (رد گزینه‌ی ۲)، و چاپ `null` هم مشکلی ندارد و `cancel is null` نمایش داده می‌شود (رد گزینه‌ی ۳). در خط ۵ متد `addEventListener` روی `null` صدا زده می‌شود و خطای `TypeError` رخ می‌دهد؛ پس `ready` هرگز چاپ نمی‌شود. (listener `save` که در خط ۲ ثبت شده، همچنان با کلیک کار می‌کند.)

۶-۲ شیء event، انتشار رویداد (Bubbling) و Delegation

۷۹۲ **گزینه‌ی ۲** آسان

مرورگر برای هر رویداد یک شیء event می‌سازد و آن را به‌عنوان اولین آرگومان به listener می‌دهد؛ از طریق آن به `type` (نام رویداد، که فقط یکی از propertyهای شیء است؛ رد گزینه‌ی ۳)، `target`، `currentTarget` و متدهایی مثل `preventDefault()` دسترسی داریم. خودِ عنصر از طریق `event.currentTarget` (یا `this` در `function` معمولی) در دسترس است، ولی آرگومان اول نیست (رد گزینه‌ی ۱).

۷۹۳ **گزینه‌ی ۳** آسان

bubbling یعنی بالا رفتن مثل حباب: رویداد اول روی هدف رخ می‌دهد و بعد به والد، والدِ والد و ... تا `document` و `window` می‌رسد. حرکت از `window` به سمت پایین، مرحله‌ی **capturing** است که قبل از رسیدن به هدف انجام می‌شود (رد گزینه‌ی ۱). رویدادهای رایجی مثل `click` روی خودِ هدف نمی‌مانند (رد گزینه‌ی ۲)، و هیچ مرحله‌ای به سمت فرزندانِ هدف نمی‌رود (رد گزینه‌ی ۴).

۷۹۴ **گزینه‌ی ۲** سخت

listenerها بدون آرگومان سوم در مرحله‌ی bubbling اجرا می‌شوند: اول خودِ دکمه، بعد `mid`، `outer`، `document` و در آخر `window` (رد گزینه‌ی ۱ که ترتیب capturing را فرض کرده، و گزینه‌ی ۴ که bubbling را نادیده گرفته). `e.target` در همه‌ی مراحل همان دکمه (`btn`) است، چون کلیک روی آن رخ داده؛ فقط `currentTarget` در هر مرحله برابر عنصرِ صاحب listener است (رد گزینه‌ی ۳).

۷۹۵ **گزینه‌ی ۴** سخت

اول مرحله‌ی capturing از بالا به پایین اجرا می‌شود، پس listener capture روی `outer` از همه زودتر است (رد گزینه‌های ۱ و ۳). بعد نوبت خودِ هدف است: listenerهای دکمه (اینجا اول capture و بعد bubble). در آخر bubbling به سمت بالا، listener معمولی `outer` را اجرا می‌کند. ترتیب ثبت در کد (`outer bubble` اول نوشته شده) ترتیب مراحل را عوض نمی‌کند (رد گزینه‌ی ۲).

۷۹۶ **گزینه‌ی ۳** متوسط

`e.target` عنصری است که کلیک **واقعاً** روی آن رخ داده: `inner` (رد گزینه‌ی ۱). `e.currentTarget` عنصری است که listener روی آن ثبت شده و رویداد با bubbling به آن رسیده: `outer` (رد گزینه‌ی ۲). در listener با `function` معمولی، `this` همیشه برابر `currentTarget` است، پس `outer` (رد گزینه‌ی ۴).

۷۹۷ **گزینه‌ی ۲** متوسط

listener یک arrow function است و `this` خودش را ندارد؛ `this` بیرونی در سطح بالای اسکریپت مرورگر `window` است، پس `true` (رد گزینه‌های ۱ و ۴). شیء event اما مستقل از نوع تابع کار می‌کند: `currentTarget` عنصر صاحب listener (`outer`) و `target` عنصر کلیک‌شده (`inner`) است (رد گزینه‌ی ۳).

۷۹۸ **گزینه‌ی ۱** متوسط

`stopPropagation()` جلوی رفتن رویداد به عنصرهای **بعدی** (اینجا `outer`) را می‌گیرد، پس `outer` چاپ نمی‌شود (رد گزینه‌های ۳ و ۴). اما listenerهای دیگرِ **همان عنصر** همچنان اجرا می‌شوند، پس `btn 2` هم چاپ می‌شود (رد گزینه‌ی ۲؛ برای متوقف کردن آن‌ها `stopImmediatePropagation()` لازم بود).

۷۹۹ **گزینه‌ی ۲** متوسط

بعد از `e.preventDefault()`، مقدار `e.defaultPrevented` برابر `true` است (رد گزینه‌های ۱ و ۴) و مرورگر رفتار پیش‌فرض submit، یعنی فرستادن فرم و بارگذاری دوباره‌ی صفحه، را انجام نمی‌دهد (رد گزینه‌ی ۳). این همان چیزی است که اسلاید ۳۵ می‌گوید: «Prevent page reload on submit with `event.preventDefault()`».

۸۰۰ **گزینه‌ی ۴** متوسط

`preventDefault()` فقط **رفتار پیش‌فرض** را لغو می‌کند، نه خودِ listener را؛ پس `help clicked` چاپ می‌شود (رد گزینه‌ی ۲)، ولی مرورگر به `#help` نمی‌رود (رد گزینه‌ی ۱). لینک Contact چنین listenerای ندارد، پس پیام چاپ می‌شود و رفتار پیش‌فرض آن، یعنی رفتن به `#contact`، انجام می‌شود (رد گزینه‌ی ۳).

۸۰۱ **گزینه‌ی ۳** سخت

در کلیک روی چک‌باکس، مرورگر تیک را **قبل از** اجرای listenerها عوض می‌کند تا listener مقدار جدید را ببیند؛ پس داخل listener `checked` برابر `true` است (رد گزینه‌های ۱ و ۴). چون `preventDefault()` صدا زده شده، مرورگر بعد از listenerها تغییر را برمی‌گرداند و چک‌باکس دوباره بدون تیک می‌شود: `after: false` (رد گزینه‌ی ۲).

۸۰۲ **گزینه‌ی ۱** متوسط

`preventDefault()` جلوی bubbling را نمی‌گیرد؛ پس رویداد به `div` هم می‌رسد و listener آن اجرا می‌شود (رد گزینه‌ی ۲). listener لینک قبل از listener `box` اجرا شده (bubbling از هدف به بالا)، پس `box` مقدار `defaultPrevented` را `true` می‌بیند (رد گزینه‌ی ۳) و در پایان رفتار پیش‌فرض لغو شده است (رد گزینه‌ی ۴).

۸۰۳ **گزینه‌ی ۱** آسان

این دو متد کارهای کاملاً جدایی دارند: `preventDefault()` فقط کاری را که مرورگر به‌طور پیش‌فرض انجام می‌داد (رفتن به لینک، ارسال فرم، تیک خوردن) لغو می‌کند و رویداد همچنان bubble می‌کند؛ `stopPropagation()` فقط حرکت رویداد به عنصرهای بعدی را متوقف می‌کند و رفتار پیش‌فرض همچنان انجام می‌شود. گزینه‌ی ۲ جای این دو را عوض کرده، گزینه‌ی ۳ تفاوتشان را نادیده گرفته، و حذف listener کار `removeEventListener` است (رد گزینه‌ی ۴).

۸۰۴ **گزینه‌ی ۲** متوسط

این event delegation است: listener روی `<ul>` ثبت شده و کلیک روی هر `<li>`، حتی `<li>`ی که بعداً اضافه شده، با bubbling به آن می‌رسد؛ پس کلیک روی `C` چاپ می‌شود (رد گزینه‌ی ۱). در کلیک روی خودِ `<ul>`، مقدار `e.target` همان `ul` است و `matches("li")` برابر `false`، پس چیزی چاپ نمی‌شود (رد گزینه‌ی ۳). `currentTarget` عنصر صاحب listener یعنی `list` است، نه `li` (رد گزینه‌ی ۴).

۸۰۵ **گزینه‌ی ۴** سخت

`e.target` درونی‌ترین عنصری است که کلیک رویش رخ داده، یعنی `<b>`، نه `<li>` (رد گزینه‌ی ۱) و نه `<ul>` که فقط صاحب listener است (رد گزینه‌ی ۳). `<b>` با selector `li` جور نیست، پس `matches` برابر `false` است (رد گزینه‌ی ۲). `closest("li")` از خودِ `<b>` شروع می‌کند و به اولین جدِ `<li>` می‌رسد که متنش `Milk (2)` است. برای همین در delegation معمولاً `closest` امن‌تر از `matches` است.

۸۰۶ **گزینه‌ی ۳** متوسط

delegation دقیقاً **به** bubbling وابسته است: کلیک روی فرزند باید بالا برود تا به listener والد برسد، و بعد هم مثل همیشه به `document` و `window` ادامه می‌دهد. delegation هیچ‌وقت جلوی bubbling را نمی‌گیرد؛ این کار `stopPropagation()` است. سه گزینه‌ی دیگر مزایای واقعی آن هستند: یک listener به‌جای چند listener، پشتیبانی خودکار از عنصرهای جدید، و کد ساده‌تر با مصرف حافظه‌ی کمتر.

۸۰۷ **گزینه‌ی ۱** آسان

یک تابع می‌تواند listener چند رویداد باشد و با `e.type` بفهمد کدام رویداد رخ داده: اول `input` و بعد `change` (رد گزینه‌ی ۲). `e.type` نام رویداد است، نه نام تابع (رد گزینه‌ی ۳). `e.target` همان کادری است که رویداد رویش رخ داده و id آن `box` است (رد گزینه‌ی ۴).

۸۰۸ **گزینه‌ی ۴** سخت

مرحله‌ی capturing قبل از همه اجرا می‌شود، پس `doc capture` اول چاپ می‌شود (رد گزینه‌های ۱ و ۳). بعد listener خودِ دکمه و سپس با bubbling listener `mid` اجرا می‌شود. `mid` با `stopPropagation()` جلوی ادامه‌ی bubbling را می‌گیرد، پس listener معمولیِ `document` هرگز اجرا نمی‌شود (رد گزینه‌ی ۲).

۸۰۹ **گزینه‌ی ۱** متوسط

رسیدن رویداد به listenerهای والدها نتیجه‌ی **انتشار (propagation)** است، نه رفتار پیش‌فرض؛ آن را فقط `stopPropagation()` متوقف می‌کند. رفتن به آدرس لینک، ارسال فرم همراه با reload صفحه، و تیک خوردن چک‌باکس کارهایی هستند که مرورگر خودش بعد از listenerها انجام می‌دهد و `preventDefault()` دقیقاً همین‌ها را لغو می‌کند.

۸۱۰ **گزینه‌ی ۳** سخت

مقدار برگشتی listenerهای `addEventListener` نادیده گرفته می‌شود؛ پس `return false` در لینک One هیچ اثری ندارد و مرورگر به `#one` می‌رود (رد گزینه‌های ۱ و ۴). اما در handler `onclick` (attribute یا property)، برگرداندن `false` رفتار پیش‌فرض را لغو می‌کند؛ پس کلیک روی Two آدرس را عوض نمی‌کند و hash همان `#one` می‌ماند (رد گزینه‌ی ۲). نتیجه: برای لغو رفتار پیش‌فرض همیشه از `preventDefault()` استفاده کن.

۸۱۱ **گزینه‌ی ۴** متوسط

`<button>` داخل فرم به‌طور پیش‌فرض دکمه‌ی ارسال است؛ رفتار پیش‌فرضِ کلیک روی آن، ایجاد رویداد `submit` روی **فرم** است. پس اول listener `click` دکمه اجرا می‌شود و بعد از آن رویداد `submit` رخ می‌دهد (رد گزینه‌های ۱ و ۲). هدف (target) رویداد `submit` خودِ فرم با id برابر `f` است، نه دکمه (رد گزینه‌ی ۳). به همین دلیل listener ارسال را روی `submit` فرم می‌گذاریم، نه روی `click` دکمه.

۸۱۲ **گزینه‌ی ۱** آسان

`target` عنصری است که کلیک واقعاً روی آن رخ داده (همان `<li>`) و `currentTarget` عنصری است که listener در حال اجرا روی آن ثبت شده (همان `<ul>`). گزینه‌ی ۲ این دو را برعکس گرفته است. این دو فقط وقتی یکی‌اند که کاربر مستقیماً روی خودِ عنصرِ صاحب listener کلیک کند، که اینجا چنین نیست (رد گزینه‌های ۳ و ۴).

۸۱۳ **گزینه‌ی ۳** سخت

در کلیک دوم، `e.target` خودِ `<ul>` است. `closest("li")` از خودِ عنصر شروع می‌کند و به سمت اجداد بالا می‌رود؛ نه `<ul>` و نه اجدادش `<li>` نیستند، پس `null` برمی‌گرداند و این **خطا نیست** (رد گزینه‌ی ۱). خط ۴ فقط `tagName` هدف را چاپ می‌کند (`UL`) و مشکلی ندارد (رد گزینه‌ی ۲). در خط ۵، خواندن `textContent` از `null` خطای `TypeError` می‌دهد (رد گزینه‌ی ۴). در delegation همیشه نتیجه‌ی `closest` را بررسی کن: `if (!li) return;`.

۲-۷ تغییر متن، HTML و استایل عناصر

۸۱۴ **گزینه‌ی ۳** آسان

`innerHTML` رشته را به‌عنوان HTML تفسیر می‌کند و یک عنصر `<b>` واقعی داخل `a` می‌سازد، پس `a` یک فرزندِ عنصری دارد. `textContent` رشته را متن ساده حساب می‌کند و تگ‌ها را عیناً به‌صورت متن نشان می‌دهد؛ پس `b` فقط یک گره‌ی متنی دارد و `children.length` آن `0` است. گزینه‌ی ۱ فکر کرده هر دو property یکسان‌اند، گزینه‌ی ۲ فکر کرده هیچ‌کدام عنصر نمی‌سازد، و گزینه‌ی ۴ جای این دو را عوض کرده است.

۸۱۵ **گزینه‌ی ۲** متوسط

`textContent` متن همه‌ی گره‌های داخل عنصر را، بدون تگ‌ها، پشت هم می‌گذارد: `Hello big world` (متن داخل `<b>` هم حساب می‌شود؛ رد گزینه‌ی ۴). `innerHTML` کد HTML داخل عنصر را همراه با تگ‌ها برمی‌گرداند: `Hello <b>big</b> world` (رد گزینه‌ی ۳). گزینه‌ی ۱ جای این دو property را عوض کرده است.

۸۱۶ **گزینه‌ی ۱** آسان

نوشتن در `textContent` **کل** محتوای قبلی عنصر، از جمله عنصرهای `<h2>` و `<p>`، را پاک می‌کند و فقط یک گره‌ی متنی با مقدار `Empty` باقی می‌گذارد؛ پس هیچ فرزندِ عنصری نمی‌ماند (`0`)، یک گره‌ی فرزند وجود دارد (`1`) و `innerHTML` همان `Empty` است. گزینه‌های ۲ و ۳ فکر کرده‌اند عنصرهای قبلی می‌مانند، و گزینه‌ی ۴ فکر کرده متن جدید به انتهای محتوای قبلی اضافه می‌شود.

۸۱۷ **گزینه‌ی ۳** سخت

`box.innerHTML += ...` یعنی `box.innerHTML = box.innerHTML + ...`: کل HTML داخل `box` به رشته تبدیل و دوباره parse می‌شود؛ دکمه‌ی قدیمی نابود و یک دکمه‌ی **تازه** با همان id ساخته می‌شود. پس `getElementById` دکمه‌ی جدید را برمی‌گرداند که با `btn` قدیمی یکی نیست: `false` (رد گزینه‌های ۱ و ۴). listener روی دکمه‌ی قدیمی ثبت شده بود و دکمه‌ی جدید هیچ listenerای ندارد؛ پس کلیک کاربر چیزی چاپ نمی‌کند (رد گزینه‌ی ۲).

۸۱۸ **گزینه‌ی ۲** سخت

با `textContent` رشته عیناً به متن تبدیل می‌شود، پس خواندن دوباره‌ی `textContent` همان `<b>Hi</b>` را می‌دهد (رد گزینه‌های ۱ و ۴ که فکر کرده‌اند تگ‌ها تفسیر شده‌اند). اما در **کد HTML** این متن، علامت‌های `<` و `>` باید به شکل `<` و `>` نوشته شوند تا تگ حساب نشوند؛ پس `innerHTML` مقدار `<b>Hi</b>` را برمی‌گرداند (رد گزینه‌ی ۳ که فکر کرده یک عنصر `<b>` ساخته شده است).

۸۱۹ **گزینه‌ی ۱** آسان

`innerHTML` هر رشته‌ای را HTML حساب می‌کند؛ اگر کاربرِ مهاجم چیزی مثل `<img src="x" onerror="...">` بفرستد، کد او در صفحه‌ی کاربران دیگر اجرا می‌شود که به آن XSS می‌گویند. `textContent` همان رشته را بی‌خطر و به‌صورت متن نمایش می‌دهد. سه گزینه‌ی دیگر ساختگی‌اند: `innerHTML` حروف را تغییر نمی‌دهد، روی همه‌ی عنصرها کار می‌کند، و نتیجه‌اش در خود صفحه دیده می‌شود.

۸۲۰ **گزینه‌ی ۲** متوسط

`el.style.xxx = ...` استایل **inline** می‌سازد، یعنی attribute `style` را می‌نویسد؛ پس `getAttribute` مقدار `null` نیست (رد گزینه‌ی ۳). در attribute، نام‌ها به شکل CSS (با خط تیره: `background-color`) ذخیره می‌شوند، و فقط در JavaScript به شکل camelCase نوشته می‌شوند (رد گزینه‌ی ۱). مقدار `fontSize` همان رشته‌ی `"20px"` است و واحدش حذف نمی‌شود (رد گزینه‌ی ۴).

۸۲۱ **گزینه‌ی ۳** سخت

برای اندازه‌ها در CSS واحد لازم است. عدد `20` بدون واحد یک مقدار نامعتبر برای `font-size` است و مرورگر آن را **بی‌صدا نادیده می‌گیرد**؛ پس `fontSize` همان رشته‌ی خالی می‌ماند (رد گزینه‌ی ۱). مرورگر خودش `px` اضافه نمی‌کند (رد گزینه‌ی ۲)، و مقدار property‌های `style` همیشه رشته است، نه عدد (رد گزینه‌ی ۴). با `"20px"` مقدار درست ثبت می‌شود.

۸۲۲ **گزینه‌ی ۴** سخت

`p.style.background-color = "red"` از نظر JavaScript یعنی `(p.style.background - color) = "red"`؛ حاصل یک تفریق نمی‌تواند سمت چپ انتساب باشد و این یک **خطای نحوی** است. خطاهای نحوی پیش از اجرای کد، هنگام خواندن کل اسکریپت پیدا می‌شوند؛ پس **هیچ خطی** اجرا نمی‌شود و حتی `start` هم چاپ نمی‌شود (رد گزینه‌های ۱ و ۳). گزینه‌ی ۲ فکر کرده این خط کار می‌کند. شکل درست: `p.style.backgroundColor = "red"` .

۸۲۳ **گزینه‌ی ۲** متوسط

`p.style` فقط استایل‌های **inline** (attribute `style` خودِ عنصر) را نشان می‌دهد؛ رنگ سبز در قاعده‌ی `<style>` تعریف شده، پس `p.style.color` رشته‌ی خالی است (رد گزینه‌های ۱ و ۳). `getComputedStyle` استایل **نهایی** اعمال‌شده را برمی‌گرداند و رنگ‌ها را به شکل `rgb(...)` گزارش می‌کند: `rgb(0, 128, 0)` (رد گزینه‌ی ۴).

۸۲۴ **گزینه‌ی ۴** متوسط

مقداردهی به `className` **کل** attribute `class` را جایگزین می‌کند؛ پس `note` و `big` پاک می‌شوند و فقط `warning` `className` است (رد گزینه‌های ۱ و ۲). `+= " bold"` رشته‌ی `" bold"` را که با فاصله شروع می‌شود به انتهای آن می‌چسباند و نتیجه `warning bold` است، یعنی دو کلاس (رد گزینه‌ی ۳). برای اضافه و حذف تک‌تک کلاس‌ها، `classList` راحت‌تر است.

۸۲۵ **گزینه‌ی ۱** آسان

`class` در JavaScript کلمه‌ی کلیدی تعریف کلاس است؛ برای همین، property متناظر با attribute `class` در DOM `className` نام‌گذاری شده و `el.class` اصلاً وجود ندارد (`undefined`). `className` **همه‌ی** کلاس‌ها را به‌صورت یک رشته برمی‌گرداند (رد گزینه‌ی ۳)، attribute `class` هم در HTML کاملاً رایج است (رد گزینه‌ی ۴)، و در Node.js اصلاً DOM وجود ندارد (رد گزینه‌ی ۲).

۸۲۶ **گزینه‌ی ۳** متوسط

`textContent` متن را به `Goodbye World` تغییر می‌دهد (رد گزینه‌ی ۴)، و `style.color = 'red'` یک استایل inline می‌سازد؛ یعنی attribute `style="color: red;"` روی عنصر نوشته می‌شود (رد گزینه‌ی ۲ که فکر کرده تغییر استایل در HTML دیده نمی‌شود). مرورگر attributeی به نام `color` نمی‌سازد (رد گزینه‌ی ۱). این همان حالت اسلاید ۲۱ است: «Goodbye World» به رنگ قرمز.

۸۲۷ **گزینه‌ی ۴** متوسط

اول عبارت `5 + 5` محاسبه می‌شود (`10`) و بعد در `textContent` نوشته می‌شود؛ هر مقداری که در `textContent` بنویسی به **رشته** تبدیل می‌شود، پس خواندنش `"10"` با نوع `string` است (رد گزینه‌ی ۱). عبارت به‌صورت متن `5+5` نوشته نمی‌شود (رد گزینه‌ی ۳). در خط بعد، `"5" + 5` الحاق رشته است و `"55"` می‌شود (همان coercion فصل اول؛ رد گزینه‌ی ۲).

۸۲۸ **گزینه‌ی ۱** سخت

`textContent` همه‌ی متن‌های درخت DOM را، بدون توجه به CSS، برمی‌گرداند؛ پس متن `span` مخفی هم حساب می‌شود: `"Hi secret"` . `innerText` فقط متنی را برمی‌گرداند که **واقعاً روی صفحه دیده می‌شود**؛ چون `span` با `display: none` مخفی است، نتیجه `"Hi"` است. گزینه‌ی ۲ جای این دو را عوض کرده، و گزینه‌های ۳ و ۴ تفاوت آن‌ها را نادیده گرفته‌اند.

۸۲۹ **گزینه‌ی ۴** آسان

دستکاری DOM فقط درخت **داخل حافظه‌ی مرورگر** را تغییر می‌دهد؛ فایل HTML روی سرور (یا روی دیسک) دست نمی‌خورد و با refresh صفحه همه‌چیز از روی فایل اصلی دوباره ساخته می‌شود. سه گزینه‌ی دیگر عیناً بندهای اسلاید ۲۲ هستند: درختی از اشیاء، تغییر پویای متن و HTML و استایل، و تغییر آسان `id` ، `class` و `src` .

۸۳۰ **گزینه‌ی ۱** سخت

`p.style.fontWeight = ...` فقط **یک** property را به استایل inline اضافه می‌کند و بقیه‌ی استایل‌های inline (اینجا `color: blue`) سر جایشان می‌مانند (رد گزینه‌ی ۲). نام‌ها در attribute به شکل CSS (`font-weight`) نوشته می‌شوند (رد گزینه‌ی ۴). مقداردهی رشته‌ی خالی، آن property را کاملاً از استایل inline حذف می‌کند و چیزی مثل `color: ;` باقی نمی‌گذارد (رد گزینه‌ی ۳).

۸۳۱ **گزینه‌ی ۳** متوسط

propertyهای `id` ، `className` و `src` مستقیماً attributeهای عنصر را عوض می‌کنند، پس `outerHTML` مقدارهای جدید را نشان می‌دهد (رد گزینه‌ی ۱). مقداردهی `className` کلاس قبلی (`thumb`) را جایگزین می‌کند، نه اینکه به آن اضافه کند (رد گزینه‌ی ۲). چون id عنصر عوض شده، دیگر عنصری با id برابر `pic` وجود ندارد و `getElementById` مقدار `null` برمی‌گرداند (رد گزینه‌ی ۴).

۸۳۲ **گزینه‌ی ۴** سخت

`innerHTML` تگ `<script>` را parse می‌کند و یک عنصر `<script>` واقعی داخل `div` می‌سازد (پس `children.length` برابر `1` است؛ رد گزینه‌ی ۳)، ولی طبق استاندارد HTML، اسکریپتی که با `innerHTML` درج شود **اجرا نمی‌شود**؛ پس `script ran` نه قبل و نه بعد چاپ نمی‌شود (رد گزینه‌های ۱ و ۲). با این حال `innerHTML` برای ورودی کاربر امن نیست، چون راه‌های دیگری مثل `onerror` در تگ `<img>` همچنان کد را اجرا می‌کنند. (`<\/script>` فقط برای این است که رشته تگ اسکریپتِ خودِ صفحه را زودتر از موعد نبندد.)

۸۳۳ **گزینه‌ی ۲** آسان

در JavaScript، نام propertyهای CSS که خط تیره دارند camelCase می‌شوند: `background-color` می‌شود `backgroundColor` . نوشتن `box.style.background-color` خطای نحوی می‌دهد، چون خط تیره به معنای تفریق است (رد گزینه‌ی ۱). نام‌ها به حروف بزرگ و کوچک حساس‌اند و حرف اول کوچک است (رد گزینه‌ی ۳)، و `bgColor` property CSS نیست و روی `style` هیچ اثری ندارد (رد گزینه‌ی ۴).

۸۳۴ **گزینه‌ی ۳** سخت

`notes[0]` و `notes[1]` عنصرهای واقعی‌اند، پس خط‌های ۲ و ۴ متن و رنگ را بدون مشکل تغییر می‌دهند (رد گزینه‌ی ۱). خط ۳ یک دام بی‌صداست: NodeList خودش `textContent` ندارد و این انتساب فقط یک property بی‌اثر روی شیء NodeList می‌سازد؛ **خطا نمی‌دهد** ولی متن هیچ پاراگرافی هم عوض نمی‌شود (رد گزینه‌ی ۲). در خط ۵، `notes` یک HTMLCollection است که property `style` ندارد؛ `notes.style` برابر `undefined` است و نوشتن `color` روی آن خطای `TypeError` می‌دهد (رد گزینه‌ی ۴). برای تغییر همه‌ی عنصرها باید روی مجموعه حلقه زد.

۸-۲ ساختن و افزودن عناصر جدید

۸۳۵ **گزینه‌ی ۳** آسان

`createElement` فقط یک عنصر **جدید** در حافظه می‌سازد؛ این عنصر پدر ندارد و تا وقتی با متدی مثل `appendChild` یا `append` به درخت اضافه نشود دیده نمی‌شود. گزینه‌های ۱ و ۴ فرض کرده‌اند افزودن خودکار انجام می‌شود که درست نیست، و گزینه‌ی ۲ کارِ `querySelector("li")` را توصیف می‌کند (پیدا کردن، نه ساختن).

۸۳۶ **گزینه‌ی ۲** آسان

پاراگراف جدید ساخته و متن‌دار شده ولی به صفحه اضافه نشده است؛ پس `isConnected` برابر `false` است و `querySelectorAll("p")` فقط همان یک `<p>` قدیمی صفحه را پیدا می‌کند. گزینه‌های ۱ و ۳ فرض کرده‌اند `createElement` یا مقداردهی `textContent` عنصر را خودکار وارد صفحه می‌کند؛ گزینه‌ی ۴ هم با شمارش درست، `isConnected` را اشتباه گفته است.

۸۳۷ **گزینه‌ی ۲** متوسط

ورودی `createElement` فقط نام تگ است؛ علامت‌های `<` و `>` در نام تگ مجاز نیستند و مرورگر خطای `InvalidCharacterError` می‌دهد. گزینه‌ی ۱ یک `<section>` می‌سازد، گزینه‌ی ۳ دو گره‌ی متنی به انتهای body اضافه می‌کند (`append` رشته قبول می‌کند) و در گزینه‌ی ۴ مرجعِ `null` یعنی افزودن به انتها، که کاملاً مجاز است.

۸۳۸ **گزینه‌ی ۴** آسان

`appendChild` گره را آخرین فرزندِ عنصری می‌کند که روی آن صدا زده شده (این‌جا `<body>`)؛ این همان خط آخرِ مثال اسلاید ۲۴ است. `createElement` روی `document` صدا زده می‌شود و عنصر **می‌سازد**، `querySelector` عنصر **پیدا** می‌کند و `setAttribute` دو آرگومان (نام و مقدار attribute) می‌گیرد و چیزی به صفحه اضافه نمی‌کند.

۸۳۹ **گزینه‌ی ۱** آسان

`append(...items)` هر تعداد گره یا رشته می‌گیرد و رشته‌ها را به text node تبدیل می‌کند؛ `appendChild(node)` فقط یک گره می‌گیرد. گزینه‌ی ۲ برعکس است: `appendChild` با رشته خطای `TypeError` می‌دهد. گزینه‌ی ۳ هم غلط است، چون مقدار `append` برمی‌گرداند (این `appendChild` است که خود گره را برمی‌گرداند). هر دو متد به **انتها** اضافه می‌کنند، نه ابتدا؛ برای ابتدا `prepend` داریم.

۸۴۰ **گزینه‌ی ۱** متوسط

`append` برای **هر** رشته یک text node جداگانه می‌سازد؛ پس `<ul>` یک فرزندِ عنصری (`<li>`) و دو گره‌ی متنی (`"note"` و `"!"`) دارد. `children` فقط عنصرها را می‌شمارد (۱) و `childNodes` همه‌ی گره‌ها را (۳). گزینه‌ی ۲ گره‌های متنی را عنصر حساب کرده، گزینه‌ی ۳ فکر کرده رشته‌ها اضافه نمی‌شوند و گزینه‌ی ۴ دو رشته را در یک گره ادغام کرده است.

۸۴۱ **گزینه‌ی ۲** سخت

`appendChild` خودِ گره را برمی‌گرداند، پس `<li>` همان `first` است و خط اول `One` چاپ می‌شود. اما مقدار `append` برمی‌گرداند؛ پس `second` برابر `undefined` است و `undefined.textContent = "Two"` خطای `TypeError` می‌دهد و اجرا متوقف می‌شود. گزینه‌ی ۱ فکر کرده `append` هم گره را برمی‌گرداند، گزینه‌ی ۳ خطا را از همان ابتدا دانسته و گزینه‌ی ۴ فرض کرده مقداردهی به `undefined` بی‌صدا نادیده گرفته می‌شود.

۸۴۲ **گزینه‌ی ۳** متوسط

خط ۳ مجاز است، چون `append` هم رشته و هم گره می‌گیرد. در خط ۵ هم آرگومان اول یک گره است و مرجعِ `null` یعنی افزودن به انتها، پس مشکلی نیست. اما در خط ۷ به `appendChild` یک **رشته** داده شده و این متد فقط Node قبول می‌کند؛ پس `TypeError` رخ می‌دهد و هرگز چاپ نمی‌شود. اگر به‌جای آن `box.append("Done")` یا `box.appendChild(document.createTextNode("Done"))` می‌نوشتیم، خطایی نبود.

۸۴۳ **گزینه‌ی ۴** آسان

متد درست `document.createTextNode(text)` است و خروجی‌اش یک گره است، پس `appendChild` آن را قبول می‌کند. گزینه‌ی ۱ یک **عنصر** با تگ عجیبِ `<text>` می‌سازد، نه گره‌ی متنی. متدهای گزینه‌های ۲ و ۳ اصلاً وجود ندارند و صدا زدنشان خطای `TypeError` (is not a function) می‌دهد.

۸۴۴ **گزینه‌ی ۴** متوسط

محتوای یک گره‌ی متنی هیچ‌وقت به‌عنوان HTML تفسیر نمی‌شود؛ پس عنصر `<b>` ساخته نمی‌شود (`children.length` برابر ۰) و خودِ نویسه‌های `<b>Hi</b>` متنِ box می‌شوند. گزینه‌های ۱ و ۳ فرض کرده‌اند تگ تفسیر شده است (این رفتار `innerHTML` است) و گزینه‌ی ۲ عنصر را ساخته‌شده فرض کرده ولی متن را خام گزارش کرده که با هم تناقض دارد.

۸۴۵ **گزینه‌ی ۲** آسان

`insertBefore` روی **پدر** صدا زده می‌شود و ترتیب آرگومان‌ها «اول گره‌ی جدید، بعد مرجع» است: `ul.insertBefore(newLi, oldLi)`. گزینه‌ی ۱ آرگومان‌ها را جابه‌جا کرده (و چون `newLi` فرزند `ul` نیست خطای `NotFoundError` می‌دهد). گزینه‌ی ۳ متد را روی خودِ مرجع صدا زده و گزینه‌ی ۴ روی گره‌ی جدید؛ در هر دو، پدرِ درست مشخص نشده است.

۸۴۶ **گزینه‌ی ۴** متوسط

`list.children[1]` همان `<li>C</li>` است و `insertBefore` گره‌ی جدید را درست **قبل** از آن می‌گذارد؛ پس ترتیب A، B، C می‌شود و `textContent` متن همه را پشت سر هم می‌دهد: `ABC`. گزینه‌ی ۱ فکر کرده درج در ابتدای لیست انجام می‌شود، گزینه‌ی ۲ آن را مثل `appendChild` در انتها گذاشته و گزینه‌ی ۳ فراموش کرده که `insertBefore` واقعاً گره را به صفحه اضافه می‌کند.

۸۴۷ **گزینه‌ی ۲** متوسط

گره‌ی جدید قبل از **اولین** فرزند فعلی قرار می‌گیرد، پس خودش اولین فرزند می‌شود: `list.insertBefore(newItem, list.firstChild)` (معادل `list.prepend(newItem)`). اگر لیست خالی باشد `firstChild` برابر `null` است و درج در انتها همان درج در ابتداست، پس باز هم درست کار می‌کند. گزینه‌ی ۱ همیشه به **انتها** اضافه می‌کند، گزینه‌ی ۳ گره را قبل از آخرین فرزند می‌گذارد و گزینه‌ی ۴ خطای `NotFoundError` می‌دهد، چون `list` فرزند خودش نیست.

۸۴۸ **گزینه‌ی ۱** سخت

در خط سوم مرجع `null` است، پس `<em>` به انتهای box اضافه می‌شود و تعداد فرزندان ۲ می‌شود. در خط پنجم مرجع (`s`) فرزند `box` **نیست** (هم‌سطح آن است)، پس مرورگر خطای `NotFoundError` می‌دهد. گزینه‌ی ۲ فرض کرده مرجع می‌تواند هر عنصری باشد، گزینه‌ی ۳ فکر کرده `null` مجاز نیست و گزینه‌ی ۴ اثرِ درجِ اول را نادیده گرفته است.

۸۴۹ **گزینه‌ی ۳** متوسط

`b.before(...)` عنصر A را درست قبل از B می‌گذارد (A، B)، `b.after(...)` عنصر C را بعد از B (A، B، C) و `list.prepend(...)` عنصری با متن `"0"` را **اولین** فرزند لیست می‌کند؛ نتیجه `0ABC` است. گزینه‌ی ۲ فکر کرده `prepend` به انتها اضافه می‌کند، گزینه‌ی ۱ آن را قبل از B گذاشته و گزینه‌ی ۴ معنای `before` و `after` را جابه‌جا فهمیده است.

۸۵۰ **گزینه‌ی ۱** متوسط

`task` از قبل در صفحه (داخل `#todo`) است. append کردن یک گره‌ی موجود آن را از جای قبلی **برمی‌دارد** و به جای جدید **منتقل** می‌کند؛ پس `#todo` خالی می‌شود و `#done` یک فرزند پیدا می‌کند. گزینه‌ی ۲ تصور کرده کپی ساخته می‌شود، گزینه‌ی ۳ فکر کرده چون عنصر جای دیگری است اضافه نمی‌شود و گزینه‌ی ۴ فکر کرده عنصر گم می‌شود.

۸۵۱ **گزینه‌ی ۱** متوسط

هر گره فقط یک جا در درخت دارد؛ افزودن دوباره‌ی یک گره‌ی موجود آن را **منتقل** می‌کند و کپی نمی‌سازد (برای کپی باید `cloneNode` صدا زده شود). پس گزینه‌ی ۱ نادرست و پاسخ است. گزینه‌های ۲، ۳ و ۴ همگی رفتار واقعی این متدها هستند.

۸۵۲ **گزینه‌ی ۴** سخت

`cloneNode` کپی می‌سازد و عنصر اصلی سر جایش می‌ماند؛ پس `#a` هنوز ۱ فرزند دارد و دو کپی به `#b` اضافه می‌شوند (۲). کپی عمیق (`true`) متن داخلی `X` را هم دارد، ولی کپی سطحی فقط خود تگ `<li>` است و متنش خالی است؛ پس `true` چاپ می‌شود. گزینه‌های ۱ و ۳ کپی را با جابه‌جایی اشتباه گرفته‌اند و گزینه‌های ۲ و ۳ فکر کرده‌اند کپی سطحی متن را هم دارد.

۸۵۳ **گزینه‌ی ۳** سخت

ابتدا Z قبل از `items[1]` (یعنی B) درج می‌شود: A، Z، B، C. حالا `items[0]` هنوز A است و `insertBefore(A, null)` آن را به انتها **منتقل** می‌کند (کپی نمی‌سازد): Z، B، C، A. گزینه‌ی ۱ تصور کرده A کپی می‌شود، گزینه‌ی ۲ مرجعِ `null` را بی‌اثر دانسته و گزینه‌ی ۴ فکر کرده Z قبل از A درج شده است.

۸۵۴ **گزینه‌ی ۳** آسان

`document.body.appendChild(div)` یک div تازه را **آخرین فرزند** body می‌کند؛ عنوان دست نمی‌خورد و چون div یک عنصر بلوکی است، در خط جدیدی زیر دکمه نمایش داده می‌شود. گزینه‌ی ۱ رفتار تغییر `textContent` عنوان را توصیف می‌کند، گزینه‌ی ۲ رفتار `prepend` را، و گزینه‌ی ۴ اشتباه است چون نبودن CSS فقط ظاهر خاصی به div نمی‌دهد، ولی متنش طبق معمول دیده می‌شود.

۸۵۵ **گزینه‌ی ۲** متوسط

`createElement` داخل listener است، پس هر کلیک یک div **جدید** با کلاس `box` می‌سازد؛ بعد از سه کلیک سه `.box` داریم. `appendChild` هر div را آخرین فرزند body می‌کند، یعنی بعد از تگ `<script>` ؛ پس آخرین عنصرِ body یک `DIV` است. گزینه‌ی ۱ فکر کرده فقط یک div ساخته می‌شود، گزینه‌ی ۳ تصور کرده div قبل از script قرار می‌گیرد و گزینه‌ی ۴ اثر کلیک‌ها را نادیده گرفته است.

۸۵۶ **گزینه‌ی ۳** سخت

این‌بار `createElement` **بیرون** از listener و فقط یک بار اجرا شده است؛ پس در کل یک div وجود دارد. کلیک اول آن را به body اضافه می‌کند و کلیک‌های بعدی همان div را دوباره append می‌کنند که فقط آن را به انتهای body **جابه‌جا** می‌کند؛ تعداد divها همیشه ۱ می‌ماند. گزینه‌ی ۱ اشتباه رایج است (فکر کردن به کپی شدن)، گزینه‌ی ۲ ترتیب اجرا را غلط فهمیده و گزینه‌ی ۴ تصور کرده append دوباره‌ی یک گره خطاست.

۸۵۷ **گزینه‌ی ۴** سخت

بعد از اولین Add، `Read` اضافه و کادر خالی می‌شود. کلیک دوم به شرط `trim() === ""` می‌خورد و `return` می‌کند، پس آیتم خالی ساخته نمی‌شود. کلیک سوم `Write` را با `prepend` به **ابتدای** لیست می‌برد؛ پس ۲ آیتم داریم و اولی `Write` است. گزینه‌های ۱ و ۲ کلیک با کادر خالی را هم شمرده‌اند و گزینه‌های ۲ و ۳ `prepend` را با افزودن به انتها اشتباه گرفته‌اند.

۸۵۸ **گزینه‌ی ۲** متوسط

`forEach` برای هر عضو یک `<li>` تازه می‌سازد و به انتهای لیست اضافه می‌کند؛ پس ۳ فرزند داریم. در آخرین دور `i` برابر ۲ است و عبارت از چپ به راست حساب می‌شود: `2 + 1` برابر عدد `3` و بعد `"3. JS"` . گزینه‌ی ۱ فکر کرده `+` از همان ابتدا رشته می‌چسباند، گزینه‌ی ۳ تصور کرده هر `appendChild` قبلی‌ها را پاک می‌کند و گزینه‌ی ۴ اندیسِ صفرمبنا را بدون `+ 1` در نظر گرفته است.

۸۵۹ **گزینه‌ی ۱** سخت

`map` یک **آرایه** برمی‌گرداند و `innerHTML` آن را به رشته تبدیل می‌کند؛ تبدیل آرایه به رشته عضوها را با **کاما** به هم می‌چسباند: `"<li>Apple</li>,<li>Banana</li>"` . مرورگر این را تفسیر می‌کند: دو `<li>` واقعی و یک گره‌ی متنیِ `","` بینشان؛ پس ۲ فرزند عنصری داریم و متن کل `Apple,Banana` است. گزینه‌ی ۲ کاما را فراموش کرده (این نتیجه با `join("")` به دست می‌آید)، گزینه‌ی ۳ کل رشته را یک عنصر دانسته و گزینه‌ی ۴ فکر کرده `innerHTML` تگ‌ها را تفسیر نمی‌کند.

۹-۲ حذف و جایگزینی عناصر

۸۶۰ **گزینه‌ی ۲** آسان

`el.remove()` روی خودِ عنصر صدا زده می‌شود و آن را با همه‌ی فرزندانش از DOM برمی‌دارد. `removeChild` و `replaceChild` روی **پدر** صدا زده می‌شوند و آرگومان لازم دارند. `delete` فقط برای حذف property از یک شیء است و روی یک متغیر هیچ اثری روی DOM ندارد.

۸۶۱ **گزینه‌ی ۱** آسان

`removeChild` همان گره‌ای را که حذف کرده **برمی‌گرداند**؛ پس `removed === a` برقرار است. لیست یک فرزند کمتر دارد (۱) و گره‌ی حذف‌شده هنوز متنش (`A`) را دارد، چون حذف از DOM آن را نابود نمی‌کند. گزینه‌ی ۲ فرض کرده مقدار برگشتی `undefined` است، گزینه‌ی ۳ فکر کرده محتوای گره پاک می‌شود و گزینه‌ی ۴ تصور کرده پدر برگردانده می‌شود.

۸۶۲ **گزینه‌ی ۳** آسان

امضای درست `parent.replaceChild(newChild, oldChild)` است: **اول جدید، بعد قدیمی** (درست مثل `insertBefore(جدید, مرجع)`). گزینه‌ی ۱ ترتیب را برعکس کرده که معمولاً به `NotFoundError` می‌رسد، گزینه‌ی ۲ پدر را بی‌دلیل آرگومان کرده و گزینه‌ی ۴ مشخص نکرده کدام فرزند باید عوض شود (این شکلِ تک‌آرگومانی مال `oldChild.replaceWith(newChild)` است).

۸۶۳ **گزینه‌ی ۴** متوسط

حذف `#a` درست است، چون فرزند مستقیم لیست است؛ پس `0` چاپ می‌شود. اما `#note` فرزند `list` نیست (هم‌سطح آن است) و `removeChild` در این حالت خطای `NotFoundError` می‌دهد؛ پس `done` چاپ نمی‌شود. گزینه‌ی ۱ فرض کرده هر عنصری را حذف می‌کند، گزینه‌ی ۲ خطا را به خط اول نسبت داده و گزینه‌ی ۳ نوع خطا را اشتباه گفته است (`note` پیدا شده و `null` نیست).

۸۶۴ **گزینه‌ی ۲** متوسط

`replaceChild(fresh, old)` عنصر B را برمی‌دارد و X را **در همان جای** آن می‌گذارد؛ پس متن لیست `AXC` است. مقدار برگشتی **گره‌ی قدیمی** است، یعنی B. گزینه‌ی ۱ فکر کرده گره‌ی جدید برگردانده می‌شود، گزینه‌ی ۳ جای آرگومان‌ها را برعکس فهمیده و گزینه‌ی ۴ تصور کرده X فقط قبل از B درج می‌شود و B می‌ماند.

۸۶۵ **گزینه‌ی ۳** سخت

آرگومان‌ها جابه‌جا نوشته شده‌اند: `replaceChild(old, fresh)` یعنی «`fresh` را با `old` عوض کن». اما `fresh` اصلاً فرزند `list` نیست، پس مرورگر خطای `NotFoundError` می‌دهد و `console.log` اجرا نمی‌شود. گزینه‌ی ۱ نتیجه‌ی نسخه‌ی درست (`replaceChild(fresh, old)`) است، گزینه‌ی ۲ فرض کرده خطا بی‌صدا نادیده گرفته می‌شود و گزینه‌ی ۴ نوع خطا را اشتباه گفته است (هر دو آرگومان گره‌اند و هیچ‌کدام `null` نیست).

۸۶۶ **گزینه‌ی ۱** سخت

C با A عوض می‌شود. اما A از قبل در صفحه است، پس از جای قبلی‌اش **منتقل** می‌شود (کپی نمی‌شود): ابتدا از اول لیست برداشته می‌شود و بعد جای C می‌نشیند. نتیجه B، A و دو فرزند است. گزینه‌ی ۲ تصور کرده A کپی می‌شود، گزینه‌ی ۳ فکر کرده چیزی تغییر نمی‌کند و گزینه‌ی ۴ ترتیب را اشتباه گفته است.

۸۶۷ **گزینه‌ی ۴** آسان

`replaceWith` روی **عنصر قدیمی** صدا زده می‌شود و آن را با گره‌ی جدید عوض می‌کند. در نسخه‌ی قدیمی باید روی پدرِ `old` صدا بزنیم و اول جدید، بعد قدیمی را بدهیم: `old.parentNode.replaceChild(fresh, old)` . گزینه‌ی ۱ ترتیب آرگومان‌ها را برعکس کرده، گزینه‌ی ۲ هم پدر و هم ترتیب را اشتباه گرفته و گزینه‌ی ۳ اصلاً جایگزین نمی‌کند (و چون `fresh` فرزند `old` نیست، خطا هم می‌دهد).

۸۶۸ **گزینه‌ی ۲** آسان

listener عنصر `#title` (یعنی `<h1>`) را پیدا و حذف می‌کند؛ دکمه دست نمی‌خورد و چون جای عنوان خالی شده، بالای صفحه قرار می‌گیرد. گزینه‌ی ۱ اثر `remove()` را نادیده گرفته، گزینه‌ی ۳ عنصر حذف‌شده را اشتباه گرفته و گزینه‌ی ۴ تصور کرده کل body حذف می‌شود.

۸۶۹ **گزینه‌ی ۳** متوسط

کلیک اول عنوان را حذف می‌کند و خروجی ندارد. در کلیک دوم `getElementById("title")` مقدار `null` برمی‌گرداند و `null.remove()` خطای `TypeError: Cannot read properties of null (reading 'remove')` می‌دهد. این خطا فقط همان اجرای listener را متوقف می‌کند؛ دکمه سر جایش است، پس در پایان ۰ عنوان و ۱ دکمه داریم. گزینه‌های ۱ و ۴ خطای کلیک دوم را ندیده‌اند و گزینه‌های ۲ و ۴ فکر کرده‌اند دکمه هم حذف می‌شود.

۸۷۰ **گزینه‌ی ۱** متوسط

با optional chaining (`?.`)، وقتی `element` برابر `null` است متد صدا زده نمی‌شود و خطایی رخ نمی‌دهد؛ در کلیک اول هم عنوان مثل قبل حذف می‌شود. در گزینه‌ی ۲ آرگومان اضافه هیچ اثری ندارد و `null.remove` باز هم `TypeError` می‌دهد. در گزینه‌ی ۳ عنوان فرزند مستقیم `document` نیست (فرزند body است)، پس همان کلیک اول `NotFoundError` می‌دهد. گزینه‌ی ۴ عنوان را حذف نمی‌کند (فقط خالی‌اش می‌کند) و روی `null` هم خطا می‌دهد.

۸۷۱ **گزینه‌ی ۳** سخت

خط ۳ اولین `<li>` را حذف می‌کند. در خط ۵ عنصر دیگر در صفحه نیست، ولی `remove()` روی عنصرِ جداشده **بی‌صدا هیچ کاری نمی‌کند**. اما در خط ۷، `first` دیگر فرزند `list` نیست و `removeChild` سخت‌گیر است: خطای `NotFoundError` می‌دهد و `C` چاپ نمی‌شود. گزینه‌ی ۲ همین سخت‌گیری را به `remove()` نسبت داده و گزینه‌ی ۴ آن را از `removeChild` هم انتظار نداشته است.

۸۷۲ **گزینه‌ی ۴** متوسط

`remove()` عنصر را از صفحه برمی‌دارد ولی متغیر `msg` هنوز به همان شیء اشاره می‌کند و متنش سالم است؛ پس `0 Saved!` چاپ می‌شود. با `append` همان عنصر دوباره به صفحه برمی‌گردد و `getElementById` دقیقاً همان شیء را پیدا می‌کند (`true`). گزینه‌ی ۱ فکر کرده متغیر بعد از حذف `null` می‌شود، گزینه‌ی ۲ حذف را بی‌اثر دانسته و گزینه‌ی ۳ تصور کرده عنصرِ برگردانده‌شده شیء دیگری است.

۸۷۳ **گزینه‌ی ۲** سخت

حذف یک عنصر یعنی حذف **کل زیردرختش**؛ پس `<h2>` و `<p>` هم دیگر در صفحه نیستند (۰) و `title.isConnected` برابر `false` است. اما در حافظه زیردرخت سالم می‌ماند: `card` هنوز دو فرزندش را دارد. گزینه‌ی ۱ فکر کرده فرزندان در صفحه باقی می‌مانند، گزینه‌ی ۳ تصور کرده فرزندان نابود می‌شوند و گزینه‌ی ۴ `title` را هنوز متصل به صفحه دانسته است.

۸۷۴ **گزینه‌ی ۴** متوسط

`remove()` فقط عنصر را از درخت DOM جدا می‌کند و به متغیرهای ما دست نمی‌زند؛ `el` همچنان به همان شیء اشاره می‌کند و می‌شود آن را دوباره به صفحه اضافه کرد. پس گزینه‌ی ۴ نادرست است. سه جمله‌ی دیگر رفتار واقعی این متدها را توصیف می‌کنند.

۸۷۵ **گزینه‌ی ۱** آسان

فقط `remove()` عنصر را واقعاً از درخت DOM بیرون می‌برد، پس جست‌وجوها دیگر آن را پیدا نمی‌کنند. `display = "none"` و `hidden = true` فقط عنصر را **نامرئی** می‌کنند و `textContent = ""` فقط محتوایش را خالی می‌کند؛ در این سه حالت عنصر هنوز در DOM است و `querySelector` پیدایش می‌کند.

۸۷۶ **گزینه‌ی ۴** آسان

`list.remove()` خودِ لیست را (همراه فرزندانش) از صفحه برمی‌دارد، در حالی که خواسته‌ی ما ماندن `<ul>` بود. سه گزینه‌ی دیگر فقط محتوا را خالی می‌کنند: `innerHTML = ""` همه‌ی فرزندان را دور می‌ریزد، `replaceChildren()` بدون آرگومان یعنی «هیچ فرزندی نداشته باش» و حلقه‌ی `while` آن‌قدر اولین فرزند را حذف می‌کند تا چیزی نماند.

۸۷۷ **گزینه‌ی ۲** سخت

`firstChild` همه‌ی انواع گره را حساب می‌کند، از جمله گره‌های متنیِ فاصله و خط جدید بین تگ‌ها. این لیست ۵ گره‌ی فرزند دارد: متنِ فاصله، `<li>A` ، متنِ فاصله، `<li>B` و متنِ فاصله‌ی آخر. حلقه همه را یکی‌یکی حذف می‌کند؛ پس `count` برابر ۵ و در پایان هیچ گره‌ای نمانده است. گزینه‌ی ۱ فقط عنصرها را شمرده (منطق `children`)، گزینه‌ی ۳ فکر کرده گره‌های متنی حذف نمی‌شوند و گزینه‌ی ۴ فاصله‌ی آخر را جا انداخته است.

۸۷۸ **گزینه‌ی ۱** سخت

`getElementsByClassName` یک مجموعه‌ی **زنده** است. وقتی `i = 0` است، آیتم ۱ حذف می‌شود و مجموعه کوچک می‌شود: حالا `items[0]` آیتم ۲ است، ولی `i` به ۱ می‌رود و آیتم ۳ حذف می‌شود. بعد `i = 2` است و طول مجموعه هم ۲؛ حلقه تمام می‌شود و آیتم‌های ۲ و ۴ می‌مانند. گزینه‌ی ۲ همان چیزی است که برنامه‌نویس انتظار داشت (با `querySelectorAll(...).forEach` به دست می‌آید)، گزینه‌ی ۳ آیتم‌های حذف‌شده را جابه‌جا گفته و گزینه‌ی ۴ فرض کرده فقط اولین آیتم حذف می‌شود.

۸۷۹ **گزینه‌ی ۳** متوسط

کلیک روی دکمه تا `<ul>` بالا می‌رود (bubbling) و listener پدر اجرا می‌شود. `e.target` خودِ دکمه است و `parentElement` آن، `<li>` «Code»؛ پس کل این آیتم همراه دکمه‌اش حذف می‌شود و ۲ آیتم و ۲ دکمه می‌ماند. گزینه‌ی ۱ فکر کرده فقط دکمه حذف می‌شود (این نتیجه‌ی `e.target.remove()` است)، گزینه‌ی ۲ دکمه‌ی آیتم حذف‌شده را هنوز شمرده و گزینه‌ی ۴ تصور کرده خودِ `<ul>` (صاحب listener) حذف می‌شود.

۸۸۰ **گزینه‌ی ۴** متوسط

callback تایمر حتی با تأخیر `0` بعد از تمام شدن کد همگامِ فعلی اجرا می‌شود (event loop در فصل اول). پس اولین `console.log` هنوز اعلان را در صفحه می‌بیند (`1`). بعد تایمر اول اعلان را حذف می‌کند و تایمر دوم (۱۰ میلی‌ثانیه بعد) `0` چاپ می‌کند. گزینه‌ی ۱ فکر کرده `setTimeout(..., 0)` فوراً اجرا می‌شود، گزینه‌ی ۲ حذف را بی‌اثر دانسته و گزینه‌ی ۳ ترتیب را برعکس کرده است.

۱۰-۲ ویژگی‌ها (Attributes) و کلاس‌ها: setAttribute و classList

۸۸۱ **گزینه‌ی ۲** آسان

`getAttribute` برای attributeی که روی عنصر وجود ندارد مقدار `null` برمی‌گرداند (و در غیر این صورت همیشه یک رشته). گزینه‌ی ۱ رفتار خواندن یک property ناموجود از شیء را با `getAttribute` اشتباه گرفته، گزینه‌ی ۳ خروجی `hasAttribute` را حدس زده و گزینه‌ی ۴ اشتباه است چون نبودن attribute خطا نیست.

۸۸۲ **گزینه‌ی ۳** آسان

فقط `removeAttribute` خودِ attribute را حذف می‌کند (بعد از آن `hasAttribute("title")` برابر `false` است). گزینه‌ی ۱ مقدار را رشته‌ی `"null"` می‌کند و گزینه‌ی ۲ آن را خالی می‌کند، ولی در هر دو attribute هنوز وجود دارد. گزینه‌ی ۴ با attributeها کاری ندارد و فقط کلاسی به نام `title` را (اگر باشد) از فهرست کلاس‌ها برمی‌دارد.

۸۸۳ **گزینه‌ی ۴** آسان

`setAttribute` اگر attribute وجود داشته باشد مقدارش را **عوض** می‌کند (`href` از `/home` به `/about`) و اگر نباشد آن را **اضافه** می‌کند (`target`). حالا لینک سه attribute دارد: `id`، `href` و `target`. گزینه‌ی ۱ فکر کرده مقدار قبلی عوض نمی‌شود، گزینه‌ی ۲ `id` را نشمرده و گزینه‌ی ۳ تصور کرده `setAttribute` فقط attributeهای موجود را تغییر می‌دهد.

۸۸۴ **گزینه‌ی ۲** متوسط

حذف attributeی که وجود ندارد **خطا نمی‌دهد** و فقط کاری انجام نمی‌شود. `hasAttribute` همیشه یک مقدار بولی برمی‌گرداند: `title` نیست (`false`) و `id` هست (`true`). گزینه‌ی ۱ رفتار سخت‌گیرانه‌ی `removeChild` را به `removeAttribute` نسبت داده، گزینه‌ی ۳ وجود `title` را فرض کرده و گزینه‌ی ۴ فکر کرده `hasAttribute` مقدار `undefined` برمی‌گرداند.

۸۸۵ **گزینه‌ی ۱** متوسط

`getAttribute` همیشه رشته (یا `null`) برمی‌گرداند، پس `"300"` از نوع `string` است. اما property متناظر، `img.width`، مقدار را با نوع واقعی‌اش یعنی عدد `300` می‌دهد. گزینه‌های ۲ و ۴ فکر کرده‌اند `getAttribute` عدد برمی‌گرداند و گزینه‌ی ۳ property را هم رشته دانسته است.

۸۸۶ **گزینه‌ی ۳** سخت

`disabled` یک attribute بولی است و فقط **حضورش** مهم است؛ مقدار `"false"` هم حضور حساب می‌شود، پس دکمه غیرفعال است و `true` چاپ می‌شود. با `removeAttribute` attribute حذف و دکمه فعال می‌شود (`false`). گزینه‌ی ۱ اشتباه رایج است (خیال کرده رشته‌ی `"false"` یعنی فعال)، گزینه‌ی ۲ کاملاً برعکس است و گزینه‌ی ۴ اثر `removeAttribute` را نادیده گرفته است.

۸۸۷ **گزینه‌ی ۲** آسان

`setAttribute("src", "dambooldor2.png")` فقط مقدار `src` همان `<img>` را عوض می‌کند؛ `width` و `height` دست نخورده‌اند، پس تصویر جدید هم ۳۰۰ در ۳۰۰ نمایش داده می‌شود و دکمه سر جایش است. هیچ عنصری حذف یا اضافه نمی‌شود؛ پس گزینه‌های ۱ و ۴ غلط‌اند، و گزینه‌ی ۳ اشتباه است چون attributeهای اندازه هنوز روی تگ هستند.

۸۸۸ **گزینه‌ی ۱** متوسط

listener مقدار attribute `src` را به `damblooldor2.png` تغییر داده و `getAttribute` همان متن نوشته‌شده را برمی‌گرداند؛ `width` هم دست نخورده و هنوز `"300"` است. گزینه‌ی ۲ تغییر را ندیده، گزینه‌ی ۳ فکر کرده تغییر `src` بقیه‌ی attributeها را پاک می‌کند و گزینه‌ی ۴ تصور کرده `setAttribute` یک attribute دوم کنار قبلی می‌سازد.

۸۸۹ **گزینه‌ی ۳** سخت

`getAttribute` همان متنی را می‌دهد که در attribute نوشته شده (`cat.png`)، ولی property `img.src` آدرس را **کامل و مطلق** می‌کند: آدرس نسبی `cat.png` نسبت به پوشه‌ی صفحه (`/gallery/`) حل می‌شود. گزینه‌ی ۱ فرقی بین این دو قائل نشده، گزینه‌ی ۲ جای آن‌ها را برعکس کرده و گزینه‌ی ۴ آدرس را نسبت به ریشه‌ی سایت حل کرده، در حالی که مبنا پوشه‌ی فعلی صفحه است.

۸۹۰ **گزینه‌ی ۴** متوسط

property `value` مقدار **فعلی** کادر (چیزی که کاربر تایپ کرده) را نگه می‌دارد، ولی attribute `value` همان مقدار **اولیه**‌ی نوشته‌شده در HTML است و با تایپ کاربر عوض نمی‌شود. گزینه‌ی ۱ این دو را همیشه یکی دانسته، گزینه‌ی ۲ تایپ کاربر را نادیده گرفته و گزینه‌ی ۳ نقش آن‌ها را برعکس کرده است.

۸۹۱ **گزینه‌ی ۴** سخت

`pic.src` آدرس کامل (`https://quiz.example/day.png`) را برمی‌گرداند، پس شرط `pic.src === "day.png"` **هیچ‌وقت** برقرار نیست و هر بار شاخه‌ی `else` اجرا می‌شود که `src` را دوباره `day.png` می‌کند. نتیجه این است که تصویر هرگز عوض نمی‌شود. گزینه‌ی ۱ همان رفتاری است که برنامه‌نویس می‌خواست (با `pic.getAttribute("src") === "day.png"` به دست می‌آید)؛ گزینه‌های ۲ و ۳ هم شرط را دست‌کم یک بار درست فرض کرده‌اند.

۸۹۲ **گزینه‌ی ۴** سخت

`data-product-id` در `dataset` با نام camelCase یعنی `productId` دیده می‌شود و مقدارش **رشته‌ی** `"42"` است؛ پس `"42" + 1` به‌صورت رشته‌ای `"421"` می‌شود. مقداردهی `dataset.inStock` هم یک attribute واقعی به نام `data-in-stock` روی تگ می‌سازد. گزینه‌ی ۱ مقدار را عدد فرض کرده، گزینه‌ی ۲ فکر کرده نام خط‌تیره‌دار در `dataset` پیدا نمی‌شود و گزینه‌های ۲ و ۳ تصور کرده‌اند `dataset` روی attributeها اثری ندارد.

۸۹۳ **گزینه‌ی ۱** آسان

`classList.add` کلاس‌های قبلی (`card`) را نگه می‌دارد، می‌تواند چند کلاس را یک‌جا بگیرد و کلاس **تکراری** اضافه نمی‌کند؛ پس فهرست نهایی `card active big` با ۳ کلاس است. گزینه‌های ۲ و ۴ تکرار را مجاز دانسته‌اند و گزینه‌های ۳ و ۴ فکر کرده‌اند `add` کلاس‌های قبلی را پاک می‌کند (کاری که `className =` انجام می‌دهد).

۸۹۴ **گزینه‌ی ۴** متوسط

حذف کلاسی که وجود ندارد (`ghost`) خطا نمی‌دهد و `remove` می‌تواند چند کلاس را یک‌جا حذف کند؛ پس فقط `b` می‌ماند و `contains("b")` برابر `true` است. گزینه‌ی ۱ فکر کرده حذف کلاس ناموجود خطاست، گزینه‌ی ۲ فرض کرده `remove` با چند آرگومان کار نمی‌کند و چیزی حذف نمی‌شود و گزینه‌ی ۳ نتیجه‌ی `contains` را اشتباه گفته است.

۸۹۵ **گزینه‌ی ۲** متوسط

`toggle` **وضعیت جدید** را برمی‌گرداند: بار اول کلاس اضافه می‌شود (`true`)، بار دوم حذف (`false`). با آرگومان دوم `true`، `toggle` فقط **اضافه** می‌کند؛ پس کلاس اضافه و `true` چاپ می‌شود و در پایان `className` برابر `dark` است. گزینه‌ی ۱ فکر کرده `toggle` چیزی برنمی‌گرداند، گزینه‌ی ۳ تصور کرده خروجی، وضعیتِ **قبل** از تغییر است و گزینه‌ی ۴ `toggle` را مثل `add` فرض کرده که هیچ‌وقت حذف نمی‌کند.

۸۹۶ **گزینه‌ی ۳** متوسط

`setAttribute("class", "red")` **کل** کلاس‌ها را جایگزین می‌کند و `note` و `big` از بین می‌روند. بعد `classList.add` کلاس `bold` را اضافه می‌کند و `+= " small"` رشته‌ی کلاس را ادامه می‌دهد: `red bold small`. گزینه‌ی ۱ فکر کرده `setAttribute` کلاس اضافه می‌کند، گزینه‌ی ۲ تصور کرده `+=` کلاس‌های `classList` را پاک می‌کند و گزینه‌ی ۴ `red` را جا انداخته است.

۸۹۷ **گزینه‌ی ۱** سخت

نقطه فقط در **انتخابگرهای CSS** (مثل `querySelector(".active")`) معنای «کلاس» دارد. `classList.add` نام خام کلاس را می‌گیرد؛ پس کلاسی دقیقاً به نام `.active` (با نقطه) ساخته می‌شود و کلاس `active` وجود ندارد. گزینه‌ی ۲ فکر کرده نقطه خودکار حذف می‌شود، گزینه‌ی ۳ دو نام متفاوت را یکی دانسته و گزینه‌ی ۴ تصور کرده نقطه در نام کلاس خطاست، در حالی که فقط فاصله (whitespace) و رشته‌ی خالی ممنوع‌اند.

---

۸۹۸ **گزینه‌ی ۲** سخت

هر آرگومانِ `classList.add` باید **یک** نام کلاس بدون فاصله باشد. رشته‌ی `"btn primary"` فاصله دارد، پس مرورگر خطای `InvalidCharacterError` می‌دهد و `console.log` اجرا نمی‌شود. شکل درست `btn.classList.add("btn", "primary")` است. گزینه‌ی ۱ رفتار `className = "btn primary"` را توصیف می‌کند و گزینه‌های ۳ و ۴ حدس‌هایی هستند که هیچ‌کدام رخ نمی‌دهند.

---

۸۹۹ **گزینه‌ی ۳** متوسط

`toggle` دقیقاً همین کار را می‌کند: اگر کلاس نباشد اضافه و اگر باشد حذف می‌کند، و وضعیت جدید را هم برمی‌گرداند. `add` فقط اضافه می‌کند (کلیک دوم حالت تاریک را خاموش نمی‌کند)، `contains` فقط بررسی می‌کند و چیزی را تغییر نمی‌دهد و `replace` دو آرگومان (کلاس قدیم و جدید) می‌گیرد و یک کلاس را با دیگری عوض می‌کند.

---

۹۰۰ **گزینه‌ی ۱** سخت

`contains` فقط **بررسی** می‌کند و `true` یا `false` برمی‌گرداند؛ هیچ تغییری در کلاس‌ها نمی‌دهد. پس گزینه‌ی ۱ نادرست و پاسخ است. گزینه‌ی ۲ درست است، چون `className` کل رشته‌ی کلاس را جایگزین می‌کند؛ گزینه‌ی ۳ هم درست است و گزینه‌ی ۴ همان رفتار واقعی `toggle` است (وضعیت جدید، یعنی «نیست»، را برمی‌گرداند).

---

۹۰۱ **گزینه‌ی ۴** آسان

با کلاس‌ها، «ظاهر» (CSS) از «رفتار» (JavaScript) جدا می‌ماند؛ همان تفکیک سه‌لایه‌ی اسلاید ۳. برگرداندن وضعیت با یک `remove` یا `toggle` انجام می‌شود و تغییر ظاهر فقط در یک جای CSS لازم است. گزینه‌ی ۱ غلط است چون `el.style` کاملاً کار می‌کند، گزینه‌ی ۲ هم غلط است (با `style.color` هم می‌شود رنگ را عوض کرد) و گزینه‌ی ۳ برعکس است: کلاس بدون تعریف در CSS هیچ اثری ندارد.

۱۱-۲ کار با فرم‌ها (Forms)

---

۹۰۲ **گزینه‌ی ۲** آسان

property `value` در **همه‌ی** کادرهای ورودی، حتی با `type="number"`، رشته برمی‌گرداند؛ پس نوع آن `string` و مقدارش `"5"` است (`console.log` رشته را بدون نقل‌قول چاپ می‌کند). `type="number"` فقط شکل کادر و نوع کاراکترهای مجاز را تعیین می‌کند. گزینه‌های ۱ و ۳ فکر کرده‌اند مقدار عدد است و گزینه‌ی ۴ تصور کرده `value` تعریف نشده است.

---

۹۰۳ **گزینه‌ی ۳** آسان

`a` و `b` رشته‌اند (`"2"` و `"3"`)، پس `+` آن‌ها را به هم می‌چسباند: `"23"`. بعد از تبدیل با `Number()` جمع واقعی انجام می‌شود: `5`. گزینه‌ی ۱ فراموش کرده `.value` رشته است، گزینه‌ی ۲ فکر کرده `Number()` اثری ندارد و گزینه‌ی ۴ جای دو نتیجه را برعکس کرده است.

---

۹۰۴ **گزینه‌ی ۱** سخت

کادر `price` خالی است و `Number("")` برابر `0` می‌شود (نه `NaN`)؛ این دام باعث می‌شود کادر خالی بی‌صدا «صفر» حساب شود. `qty` رشته‌ی `"3"` است: عملگر `*` آن را به عدد تبدیل می‌کند (`6`) ولی `+` با رشته، چسباندن انجام می‌دهد (`"32"`). گزینه‌های ۲ و ۴ فکر کرده‌اند رشته‌ی خالی `NaN` می‌شود و گزینه‌های ۳ و ۴ `qty` را عدد فرض کرده‌اند.

---

۹۰۵ **گزینه‌ی ۴** آسان

`checked` یک **property بولی** است: تیک خورده `true` و نخورده `false`. گزینه‌ی ۱ غلط است چون `.value` یک checkbox به‌طور پیش‌فرض همیشه `"on"` است، گزینه‌ی ۲ مال گزینه‌های `<option>` در `<select>` است و گزینه‌ی ۳ غلط است چون تابع `checked` نیست و صدا زدنش خطای `TypeError` می‌دهد.

---

۹۰۶ **گزینه‌ی ۲** متوسط

وقتی برای checkbox attribute `value` ننوشته باشیم، `.value` آن رشته‌ی پیش‌فرض `"on"` است و ربطی به تیک خوردن ندارد؛ وضعیت واقعی در `.checked` است که این‌جا `false` است. گزینه‌ی ۱ فکر کرده `.value` وضعیت را نشان می‌دهد، گزینه‌ی ۳ از `"on"` نتیجه گرفته که تیک خورده و گزینه‌ی ۴ تصور کرده checkbox مقداری ندارد.

۹۰۷ **گزینه‌ی ۱** سخت

ابتدا گزینه‌ی `free` انتخاب شده است. radioهای هم‌نام یک گروه‌اند و فقط **یکی** از آن‌ها می‌تواند انتخاب باشد؛ پس با تیک خوردن `pro`، تیک `free` خودبه‌خود برداشته می‌شود. حالا گزینه‌ی انتخاب‌شده `pro` است و تعداد انتخاب‌شده‌ها ۱. گزینه‌ی ۲ رفتار checkbox را به radio نسبت داده، گزینه‌ی ۳ مقدار پیش‌فرض `"on"` را در نظر گرفته (ولی این radioها `value` دارند) و گزینه‌ی ۴ تغییر انتخاب را ندیده است.

۹۰۸ **گزینه‌ی ۳** سخت

هیچ radioی تیک نخورده، پس `querySelector` چیزی پیدا نمی‌کند و `null` برمی‌گرداند. خواندن `.value` از `null` خطای `TypeError` می‌دهد؛ برای همین باید اول بررسی کرد: `chosen ? chosen.value : "..."`. گزینه‌های ۱ و ۲ فکر کرده‌اند خواندن property از `null` بی‌خطر است و گزینه‌ی ۴ اولین radio را به‌طور خودکار انتخاب‌شده فرض کرده است.

۹۰۹ **گزینه‌ی ۴** متوسط

`.value` یک `<select>` برابر attribute `value` گزینه‌ی **انتخاب‌شده** است (`isf`)، نه متن نمایش‌داده‌شده‌ی آن (`Isfahan`). `.value` یک `<textarea>` متن داخلش است (`"Hello"` با طول ۵). گزینه‌ی ۱ متن گزینه را با مقدار آن اشتباه گرفته، گزینه‌ی ۲ همیشه گزینه‌ی اول را انتخاب‌شده فرض کرده و گزینه‌ی ۳ تصور کرده `textarea` مثل `input` ویژگی `value` ندارد.

۹۱۰ **گزینه‌ی ۱** آسان

رویداد `input` با هر تغییر مقدار (تایپ، پاک کردن، paste) رخ می‌دهد و برای بازخورد لحظه‌ای مناسب است. `change` در کادر متن فقط وقتی رخ می‌دهد که تغییر نهایی شود (کاربر از کادر خارج شود)، `submit` هنگام ارسال فرم و `load` هنگام بارگذاری کامل صفحه رخ می‌دهد.

۹۱۱ **گزینه‌ی ۲** متوسط

`input` با **هر** حرف یک بار اجرا می‌شود (سه بار)، ولی `change` در کادر متن فقط یک بار و وقتی رخ می‌دهد که کاربر کادر را ترک کند و مقدار نهایی شود. گزینه‌ی ۱ فکر کرده `input` فقط یک بار در پایان تایپ رخ می‌دهد، گزینه‌ی ۳ نقش دو رویداد را برعکس کرده و گزینه‌ی ۴ تصور کرده `change` هم با هر حرف اجرا می‌شود.

۹۱۲ **گزینه‌ی ۳** آسان

رویداد `submit` روی خودِ **فرم** رخ می‌دهد، چه کاربر روی دکمه‌ی ارسال کلیک کند و چه داخل یک کادر Enter بزند؛ پس listener آن را به `<form>` وصل می‌کنیم و `preventDefault()` را همان‌جا صدا می‌زنیم. دکمه فقط ارسال را **شروع** می‌کند (رویداد `click` مال دکمه است)؛ `window` و کادرهای ورودی هم هدف این رویداد نیستند.

۹۱۳ **گزینه‌ی ۴** سخت

رویداد `submit` روی **فرم** رخ می‌دهد و از فرم به سمت بالا (اجداد) حرکت می‌کند، نه به سمت فرزندانی مثل دکمه؛ پس listener دکمه هرگز اجرا نمی‌شود و `preventDefault()` آن هم صدا زده نمی‌شود. فقط `form handler` چاپ می‌شود و صفحه reload خواهد شد. گزینه‌های ۱ و ۲ فکر کرده‌اند listener دکمه اجرا می‌شود و گزینه‌ی ۳ لغو شدن رفتار پیش‌فرض را از listener فرم انتظار داشته که `preventDefault()` ندارد.

۹۱۴ **گزینه‌ی ۲** آسان

رفتار پیش‌فرض submit فرستادن داده و بار کردن صفحه‌ی جدید (reload) است؛ `preventDefault()` فقط همین رفتار پیش‌فرض را لغو می‌کند. خالی کردن کادرها کار `form.reset()` است، اعتبارسنجی را خودمان (یا attributeهایی مثل `required`) انجام می‌دهیم و جلوگیری از اجرای listenerهای دیگر کار `stopImmediatePropagation()` است، نه `preventDefault()`.

۹۱۵ **گزینه‌ی ۱** متوسط

`form.elements.q` کنترلی است که `name="q"` دارد و مقدارش `"js"` است. چون `preventDefault()` صدا زده شده، رفتار پیش‌فرض لغو می‌شود و صفحه reload نمی‌شود. گزینه‌ی ۲ اثر `preventDefault()` را ندیده، گزینه‌ی ۳ فکر کرده listener اجرا نمی‌شود و گزینه‌ی ۴ تصور کرده `form.elements` کنترل‌ها را با `name` پیدا نمی‌کند.

۹۱۶ **گزینه‌ی ۳** سخت

`trim()` فاصله‌های اول و آخر را حذف می‌کند. بار اول سه فاصله به رشته‌ی خالی تبدیل می‌شود: `Username is required` . بار دوم `" al "` به `"al"` با طول ۲ تبدیل می‌شود: `Too short` . بار سوم `"sara"` می‌ماند و پیام خوش‌آمد چاپ می‌شود. گزینه‌ی ۱ طول را قبل از `trim` حساب کرده، گزینه‌ی ۲ فکر کرده `" al "` خالی حساب می‌شود و گزینه‌ی ۴ رشته‌ی فقط-فاصله را «کوتاه» دانسته، نه خالی.

۹۱۷ **گزینه‌ی ۴** سخت

مقدار کادر فقط **یک بار** و هنگام بارگذاری صفحه خوانده و در `city` ذخیره شده است؛ آن موقع کادر خالی بود، پس `city` رشته‌ی خالی `""` است و تایپ بعدی کاربر آن را عوض نمی‌کند. خروجی `"City: "` است (با یک فاصله‌ی نامرئی در انتها). گزینه‌ی ۳ نتیجه‌ی کد درست است، یعنی وقتی `.value` داخل listener خوانده شود؛ گزینه‌های ۱ و ۲ فکر کرده‌اند مقدار کادر خالی `undefined` یا `null` است، در حالی که رشته‌ی خالی است.

۹۱۸ **گزینه‌ی ۱** متوسط

listener با هر تغییر اجرا می‌شود و آخرین بار طول متن ۱۲ است؛ پس `10 - 12` برابر `-2` نوشته می‌شود. چون `remaining < 0` درست است، `toggle` با آرگومان دوم `true` کلاس `over` را **اضافه** می‌کند. گزینه‌ی ۲ فقط اثر اولین تایپ را دیده، گزینه‌ی ۳ نقش `force` در `toggle` را نادیده گرفته و گزینه‌ی ۴ طول متن را به‌جای «باقی‌مانده» نوشته است.

۹۱۹ **گزینه‌ی ۲** متوسط

در checkbox رویداد `change` بلافاصله بعد از هر تیک زدن یا برداشتن رخ می‌دهد. بار اول `checked` برابر `true` است، پس `disabled = !true = false` (دکمه فعال می‌شود)؛ بار دوم `checked` برابر `false` است و دکمه دوباره غیرفعال می‌شود (`true`). گزینه‌ی ۱ علامت `!` را نادیده گرفته، گزینه‌ی ۳ فکر کرده دکمه بعد از فعال شدن فعال می‌ماند و گزینه‌ی ۴ مقدار `value` را با وضعیت اشتباه گرفته است.

۹۲۰ **گزینه‌ی ۳** متوسط

رشته‌ی اول بعد از `trim()` خالی است. رشته‌ی دوم `@` دارد ولی نقطه ندارد؛ چون شرط با `||` نوشته شده، نبودنِ **یکی** از این دو برای «نامعتبر» بودن کافی است. رشته‌ی سوم بعد از حذف فاصله‌های اطراف، هر دو را دارد و `OK` است. گزینه‌ی ۱ `trim` را نادیده گرفته، گزینه‌ی ۲ فکر کرده داشتن `@` کافی است و گزینه‌ی ۴ فاصله‌های اطراف رشته‌ی سوم را مشکل‌ساز دانسته است.

۹۲۱ **گزینه‌ی ۴** سخت

`form.reset()` هر کنترل را به **مقدار اولیه**‌اش برمی‌گرداند و مقدار اولیه همان attribute `value` در HTML (`"guest"`) است؛ پس نتیجه لزوماً کادر خالی نیست. گزینه‌ی ۱ فکر کرده `reset` همه‌چیز را خالی می‌کند، گزینه‌ی ۲ تصور کرده `reset` روی مقدارهای تغییرکرده با JavaScript اثری ندارد و گزینه‌ی ۳ هم بی‌پایه است، چون `.value` همیشه یک رشته است.

۹۲۲ **گزینه‌ی ۴** متوسط

اعتبارسنجی سمت کاربر فقط برای **راحتی کاربر** است؛ هر کسی می‌تواند JavaScript صفحه را خاموش کند یا داده را مستقیم برای سرور بفرستد، پس سرور همیشه باید دوباره بررسی کند. بنابراین گزینه‌ی ۴ نادرست است. سه جمله‌ی دیگر رفتار واقعی مرورگر را توصیف می‌کنند.

۹۲۳ **گزینه‌ی ۱** آسان

`.value` رشته‌ی `"20"` است و `Number("20")` آن را به عدد ۲۰ تبدیل می‌کند؛ بعد `20 + 1` برابر ۲۱ می‌شود. با `String` همان رشته می‌ماند و نتیجه `"201"` است، `Boolean("20")` برابر `true` است و `true + 1` می‌شود ۲، و `typeof` رشته‌ی `"string"` را می‌دهد که با ۱ به `"string1"` تبدیل می‌شود.

۲-۱۲ شیء window: پنجره‌ی مرورگر

۹۲۴ **گزینه‌ی ۱** آسان

`window` بالاترین شیء مرورگر است: پنجره‌ای که صفحه در آن باز شده و در عین حال شیء سراسری JavaScript. گزینه‌ی ۲ تعریف `document` است که فقط یکی از فرزندان `window` است، گزینه‌ی ۳ اشتباه است چون JavaScript صفحه به پنجره‌های سیستم‌عامل دسترسی ندارد و گزینه‌ی ۴ همان `document.body` است.

۹۲۵ **گزینه‌ی ۳** آسان

طبق نمودار اسلاید ۱۲، `window` علاوه بر `document` فرزندانی مثل `event`، `frames`، `location`، `history` و `navigator` دارد؛ `screen`، `localStorage`، `sessionStorage`، `console`، تایمرها و همه‌ی متغیرها و توابع سراسری هم روی آن هستند. پس گزینه‌های ۱ و ۲ ناقص و غلط‌اند و گزینه‌ی ۴ هم غلط است، چون در Node.js اصلاً شیء `window` وجود ندارد.

۹۲۶ **گزینه‌ی ۲** متوسط

در سطح بالای یک اسکریپت مرورگر، `var` و اعلان تابع property شیء `window` می‌شوند؛ پس `window.a` برابر ۱ و `typeof window.d` برابر `"function"` است. `let` و `const` هم سراسری‌اند ولی property `window` نمی‌شوند و خواندنشان از `window` مقدار `undefined` می‌دهد. گزینه‌ی ۱ همه را property دانسته، گزینه‌ی ۳ هیچ‌کدام را (این تقریباً رفتار Node.js است) و گزینه‌ی ۴ بین `let` و `const` فرق گذاشته است.

۹۲۷ **گزینه‌ی ۴** آسان

`alert` و `document` در واقع propertyهای شیء سراسری `window` هستند، پس با یا بدون پیشوند `window.` به همان چیز اشاره می‌کنیم. `globalThis` نام استاندارد شیء سراسری است که در مرورگر همان `window` است. هر گزینه‌ای که یکی از این سه را `false` بداند، اختیاری بودن پیشوند `window.` یا معنای `globalThis` را اشتباه فهمیده است.

۹۲۸ **گزینه‌ی ۱** سخت

`var color` در مرورگر به `window.color` تبدیل می‌شود. `show()` به‌تنهایی و در حالت غیر strict صدا زده شده، پس `this` داخل آن `this.color` است و همان `"red"` است. در سطح بالای اسکریپت هم `this === window` است. گزینه‌ی ۳ دقیقاً خروجی همین کد در Node.js است (آن‌جا `var` property شیء سراسری نمی‌شود و `this` سطح بالا `module.exports` است)؛ گزینه‌های ۲ و ۴ هم هر کدام فقط نیمی از رفتار مرورگر را درست گفته‌اند.

۹۲۹ **گزینه‌ی ۲** متوسط

`globalThis` دقیقاً برای همین ساخته شده که در هر محیطی به شیء سراسری همان محیط اشاره کند: `window` در مرورگر و `global` در Node.js. گزینه‌ی ۱ غلط است چون Node.js مرورگر نیست و `window` ندارد (`ReferenceError`)، گزینه‌ی ۳ غلط است چون فقط `var` و توابع سراسری property `window` می‌شوند و گزینه‌ی ۴ غلط است چون پیشوند `window.` اختیاری است.

۹۳۰ **گزینه‌ی ۳** آسان

`confirm` برای گرفتن تأیید است: OK مقدار `true` و Cancel مقدار `false` برمی‌گرداند. `alert` فقط دکمه‌ی OK دارد و `undefined` برمی‌گرداند، `prompt` کادر متن دارد و رشته یا `null` برمی‌گرداند و `console.log` اصلاً پنجره‌ای نشان نمی‌دهد و فقط در کنسول می‌نویسد.

۹۳۱ **گزینه‌ی ۴** متوسط

`alert` چیزی برنمی‌گرداند (`undefined`)، `confirm` بولی برمی‌گرداند و `prompt` متن تایپ‌شده را همیشه به‌صورت **رشته** (حتی وقتی عدد تایپ شود) و با Cancel مقدار `null` را برمی‌گرداند. گزینه‌ی ۱ خروجی `prompt` را عدد دانسته، گزینه‌ی ۲ خروجی `confirm` را رشته و خروجی Cancel در `prompt` را `""` دانسته و گزینه‌ی ۳ غلط است چون این سه تابع **همگام و مسدودکننده**‌اند و Promise برنمی‌گردانند.

۹۳۲ **گزینه‌ی ۱** متوسط

با Cancel، `confirm` مقدار بولی `false` برمی‌گرداند؛ پس شاخه‌ی `else` اجرا می‌شود و نوع `ok` هم `boolean` است. گزینه‌ی ۲ فرض کرده هر جوابی به معنی تأیید است، گزینه‌ی ۳ خروجی را رشته‌ای مثل `"Cancel"` دانسته و گزینه‌ی ۴ آن را با `alert` اشتباه گرفته که `undefined` برمی‌گرداند.

۹۳۳ **گزینه‌ی ۲** متوسط

`prompt` همیشه **رشته** برمی‌گرداند؛ پس `age` برابر `"20"` است و `"20" + 1` به‌صورت رشته‌ای `"201"` می‌شود. بعد از تبدیل با `Number()` جمع عددی انجام می‌شود: `21` . گزینه‌ی ۱ خروجی `prompt` را عدد دانسته، گزینه‌ی ۳ اثر `Number()` را ندیده و گزینه‌ی ۴ فکر کرده جمع رشته و عدد `NaN` می‌دهد.

۹۳۴ **گزینه‌ی ۳** سخت

با Cancel، `prompt` مقدار `null` برمی‌گرداند. چسباندن `null` به رشته، کلمه‌ی `"null"` را می‌سازد و `Number(null)` برابر `0` است (نه `NaN`)؛ پس اگر خروجی را بدون بررسی `null` به عدد تبدیل کنیم، Cancel مثل «صفر» رفتار می‌کند. گزینه‌ی ۱ خروجی Cancel را رشته‌ی خالی دانسته (این حالتِ OK با کادر خالی است)، گزینه‌ی ۲ آن را `undefined` فرض کرده و گزینه‌ی ۴ تبدیل `null` به عدد را اشتباه گفته است.

۹۳۵ **گزینه‌ی ۴** سخت

`confirm` **همگام و مسدودکننده** است: اجرای کد روی همان خط می‌ایستد تا کاربر جواب بدهد. با اینکه زمان تایمر (۰ میلی‌ثانیه) خیلی زودتر گذشته، callback تایمر فقط **بعد از تمام شدن کد همگام** فرصت اجرا دارد؛ پس اول پیام confirm، بعد `answer: true` و در آخر `timer` . گزینه‌های ۱ و ۲ فکر کرده‌اند تایمر می‌تواند وسط کد همگام یا هنگام باز بودن پنجره اجرا شود و گزینه‌ی ۳ خروجی `confirm` را رشته دانسته است.

۹۳۶ **گزینه‌ی ۱** آسان

`location` شیئی برای **آدرس صفحه** است: بخش‌های URL را می‌دهد و با `location.href = "..."` ، `assign` ، `replace` یا `reload` می‌توان صفحه را عوض یا دوباره بارگذاری کرد. موقعیت جغرافیایی کار `navigator.geolocation` است، مختصات ماوس از شیء رویدادهای ماوس به دست می‌آید و محل پنجره ربطی به `location` ندارد.

۹۳۷ **گزینه‌ی ۲** متوسط

`host` نام میزبان **همراه با پورت** است (`hostname` بدون پورت است)، `pathname` فقط مسیر را با `/` ابتدایی می‌دهد و `search` و `hash` علامت‌های `?` و `#` خود را دارند. گزینه‌ی ۱ `host` را با `hostname` اشتباه گرفته، گزینه‌ی ۳ رشته‌ی پرس‌وجو را جزو `pathname` دانسته و گزینه‌ی ۴ `host` را با `origin` یکی گرفته و علامت‌ها را حذف کرده است.

۹۳۸ **گزینه‌ی ۳** آسان

`history.go(-1)` یک قدم در تاریخچه‌ی تب به **عقب** می‌رود؛ دقیقاً کاری که `history.back()` و دکمه‌ی Back مرورگر انجام می‌دهند. `history.forward()` و `history.go(1)` یک قدم به **جلو** می‌روند و `location.reload()` همان صفحه را دوباره بارگذاری می‌کند.

۹۳۹ **گزینه‌ی ۴** متوسط

`location` آدرس (URL) صفحه است، نه موقعیت جغرافیایی؛ برای موقعیت جغرافیایی باید از `navigator.geolocation` (با اجازه‌ی کاربر) استفاده کرد. پس گزینه‌ی ۴ نادرست و پاسخ است. سه جمله‌ی دیگر رفتار واقعی `navigator` ، `screen` و `history` را توصیف می‌کنند.

۹۴۰ **گزینه‌ی ۱** متوسط

callback تایمر حتی با تأخیر `0` بعد از تمام شدن کد همگام اجرا می‌شود؛ پس `B` آخر چاپ می‌شود. `setTimeout` در مرورگر یک **عدد** به‌عنوان شناسه برمی‌گرداند (برای `clearTimeout`). گزینه‌ی ۲ تأخیر صفر را «فوری» دانسته، گزینه‌ی ۳ فکر کرده `setTimeout` چیزی برنمی‌گرداند و گزینه‌ی ۴ خروجی Node.js است که در آن شناسه یک شیء `Timeout` است، نه عدد.

۹۴۱ گزینه‌ی ۲ متوسط

`clearTimeout(t1)` تایمر اول را قبل از اجرا لغو می‌کند، ولی تایمر دوم دست نمی‌خورد. `scheduled` همگام است و اول چاپ می‌شود و حدود ۲۰ میلی‌ثانیه بعد `second` . گزینه‌ی ۱ اثر `clearTimeout` را ندیده، گزینه‌ی ۳ ترتیب همگام و ناهمگام را برعکس کرده و گزینه‌ی ۴ فکر کرده `clearTimeout` همه‌ی تایمرها را لغو می‌کند.

۹۴۲ گزینه‌ی ۳ سخت

`setInterval` فقط زمان‌بندی می‌کند و بلافاصله برمی‌گردد؛ پس `started` (کد همگام) اول چاپ می‌شود. بعد callback هر ۱۰ میلی‌ثانیه اجرا می‌شود و در اجرای سوم، **بعد از** چاپ `tick 3` ، `clearInterval` تکرار را متوقف می‌کند. گزینه‌ی ۱ ترتیب همگام و ناهمگام را برعکس کرده، گزینه‌ی ۲ فکر کرده اجرای سوم قبل از چاپ متوقف می‌شود و گزینه‌ی ۴ تصور کرده `clearInterval` یک دور دیرتر اثر می‌کند.

۹۴۳ گزینه‌ی ۴ سخت

`hello()` با پرانتز یعنی «همین الان صدا بزن»؛ پس `hello` فوراً چاپ می‌شود و نتیجه‌اش (`undefined`) به `setTimeout` داده می‌شود که بعد از ۵۰ میلی‌ثانیه چیزی برای اجرا ندارد. شکل درست `setTimeout(hello, 50)` است که خروجی گزینه‌ی ۱ را می‌دهد. گزینه‌ی ۲ اجرای فوری را ندیده و گزینه‌ی ۳ فکر کرده تابع دو بار اجرا می‌شود. (در Node.js همین کد بعد از چاپ `hello` خطای `TypeError` می‌دهد، چون آن‌جا آرگومان اول حتماً باید تابع باشد.)

۹۴۴ گزینه‌ی ۱ متوسط

`innerWidth` عرض viewport است و با کوچک و بزرگ کردن پنجره عوض می‌شود؛ برای همین در طراحی واکنش‌گرا به کار می‌رود. عرض مانیتور `screen.width` است، عرض کل پنجره با نوار ابزار `outerWidth` و میزان اسکرول افقی `scrollX` .

۹۴۵ گزینه‌ی ۲ آسان

`scrollY` میزان اسکرول عمودی (به پایین) و `scrollX` میزان اسکرول افقی است؛ بالای صفحه `scrollY` برابر `0` است. `innerHeight` ارتفاع viewport است و ربطی به اسکرول ندارد و `screen.height` ارتفاع مانیتور است.

۹۴۶ گزینه‌ی ۳ سخت

`applyLayout()` یک بار هنگام بارگذاری اجرا می‌شود (عرض ۱۲۰۰، پس کلاس `mobile` نیست) و بعد با هر رویداد `resize` دوباره اجرا می‌شود: در عرض ۵۰۰ کلاس اضافه و در عرض ۸۰۰ دوباره حذف می‌شود. گزینه‌ی ۱ فراموش کرده listener `resize` تابع را دوباره اجرا می‌کند، گزینه‌ی ۲ جهت شرط `<` را برعکس فهمیده و گزینه‌ی ۴ شاخه‌ی `else` (حذف کلاس) را نادیده گرفته است.

۲-۱۳ ذخیره‌سازی در مرورگر: localStorage و sessionStorage

۹۴۷ گزینه‌ی ۲ آسان

فرق اصلی در **عمر و دامنه‌ی** داده است: localStorage ماندگار و بین تب‌های یک origin مشترک است و sessionStorage مخصوص یک تب و تا بسته شدن آن. بقیه‌ی ویژگی‌ها یکسان‌اند: هر دو فقط رشته ذخیره می‌کنند (گزینه‌ی ۱ غلط)، هر دو همگام‌اند (گزینه‌ی ۳ غلط) و هر دو از سیاست same-origin پیروی می‌کنند (گزینه‌ی ۴ غلط).

۹۴۸ گزینه‌ی ۱ آسان

طبق اسلاید ۳۸ داده‌ی localStorage بعد از بستن تب و مرورگر هم می‌ماند (تاریخ انقضا ندارد)، ولی طبق اسلاید ۳۹ داده‌ی sessionStorage با بستن تب یا پنجره پاک می‌شود. پس فقط مقدار localStorage باقی است. گزینه‌ی ۲ نقش دو حافظه را برعکس کرده و گزینه‌های ۳ و ۴ تفاوت عمر آن‌ها را نادیده گرفته‌اند.

۹۴۹ گزینه‌ی ۳ متوسط

عمر sessionStorage به **تب** بسته است، نه به یک بار بارگذاری صفحه؛ تا تب باز است داده با reload هم می‌ماند و فقط بستن تب یا پنجره آن را پاک می‌کند. گزینه‌ی ۱ اشتباه رایج است، گزینه‌ی ۲ بی‌پایه است (دو انبار کاملاً جدا هستند) و گزینه‌ی ۴ غلط است چون `JSON.stringify` فقط شکل رشته را تعیین می‌کند و ربطی به عمر داده ندارد.

۹۵۰ **گزینه‌ی ۴** سخت

localStorage بین **همه‌ی تب‌ها و پنجره‌های** یک origin مشترک است، پس `y` در تب B هم دیده می‌شود. sessionStorage علاوه بر origin به **تب** هم وابسته است و تب B که جداگانه باز شده sessionStorage خالیِ خودش را دارد؛ پس `x` آن‌جا نیست. گزینه‌ی ۱ sessionStorage را مشترک دانسته، گزینه‌ی ۲ localStorage را هم مخصوص تب فرض کرده و گزینه‌ی ۳ نقش دو حافظه را برعکس کرده است.

---

۹۵۱ **گزینه‌ی ۲** آسان

`getItem` مقدار ذخیره‌شده (`"dark"`) را برمی‌گرداند و برای کلیدی که وجود ندارد مقدار `null` می‌دهد. گزینه‌ی ۱ اشتباه رایج است (`undefined` مال خواندن property ناموجود از یک شیء معمولی است)، گزینه‌ی ۳ نام کلید را با مقدار اشتباه گرفته و گزینه‌ی ۴ فکر کرده کلید ناموجود `false` برمی‌گرداند.

---

۹۵۲ **گزینه‌ی ۳** آسان

localStorage و sessionStorage دو انبار **جدا** هستند؛ ذخیره در یکی، در دیگری دیده نمی‌شود. پس sessionStorage مقدار `"2"` را دارد و localStorage برای این کلید `null` برمی‌گرداند. گزینه‌ی ۱ دو حافظه را یکی دانسته، گزینه‌ی ۲ نقششان را برعکس کرده و گزینه‌ی ۴ فراموش کرده کلید ناموجود `null` می‌دهد.

---

۹۵۳ **گزینه‌ی ۱** متوسط

Web Storage **همگام** است: `setItem` کارش را تمام می‌کند و همان خط بعد مقدار `"1"` قابل خواندن است (نیازی به `await` یا callback نیست). `setItem` هم چیزی برنمی‌گرداند، پس `undefined` چاپ می‌شود. گزینه‌ی ۲ فکر کرده ذخیره‌سازی ناهمگام است و هنوز انجام نشده، و گزینه‌های ۳ و ۴ برای `setItem` مقدار برگشتی خیالی فرض کرده‌اند.

---

۹۵۴ **گزینه‌ی ۴** متوسط

بعد از `removeItem("a")` فقط کلید `b` در localStorage می‌ماند؛ پس `length` برابر ۱ و `key(0)` **نام** همان کلید، یعنی `"b"`، است. `localStorage.clear()` فقط localStorage را خالی می‌کند و sessionStorage هنوز یک کلید دارد. گزینه‌ی ۱ فکر کرده `clear` هر دو حافظه را پاک می‌کند، گزینه‌ی ۲ اثر `removeItem` را ندیده و گزینه‌ی ۳ کلید حذف‌شده را هنوز در اندیس ۰ دانسته است.

---

۹۵۵ **گزینه‌ی ۲** متوسط

`removeItem(key)` فقط همان یک کلید را حذف می‌کند. `clear()` **همه‌ی** کلیدهای localStorage را پاک می‌کند (آرگومانش هم نادیده گرفته می‌شود)، متدی به نام `deleteItem` وجود ندارد (خطای `TypeError`) و `setItem` برای ذخیره است و دو آرگومان لازم دارد؛ با یک آرگومان خطا می‌دهد.

---

۹۵۶ **گزینه‌ی ۴** آسان

اسلاید ۳۸ صریحاً می‌گوید «Only strings are stored»: هر مقداری که به `setItem` بدهیم به رشته تبدیل می‌شود (عدد `5` به `"5"` و شیء به `"[object Object]"`). پس گزینه‌های ۱ و ۲ غلط‌اند. گزینه‌ی ۳ هم غلط است؛ JSON فقط **روشی** است که ما برای تبدیل شیء به رشته به کار می‌بریم و خودِ حافظه چیزی جز رشته نمی‌شناسد.

---

۹۵۷ **گزینه‌ی ۳** متوسط

عدد `5` هنگام ذخیره به رشته‌ی `"5"` تبدیل می‌شود و `getItem` همین رشته را برمی‌گرداند؛ پس `"5" + 1` به‌صورت رشته‌ای `"51"` است و نوع آن `string`. برای جمع عددی باید `Number(count) + 1` نوشت. گزینه‌های ۱ و ۴ نوع را عدد دانسته‌اند و گزینه‌ی ۲ با وجود رشته بودن، جمع عددی انجام داده است.

---

۹۵۸ **گزینه‌ی ۱** سخت

`false` هنگام ذخیره به **رشته‌ی** `"false"` تبدیل می‌شود. هر رشته‌ی غیرخالی در شرط truthy (درست) است، پس شاخه‌ی اول اجرا می‌شود. راه درست مقایسه با رشته است: `localStorage.getItem("darkMode") === "true"`. گزینه‌ی ۲ فرض کرده مقدار بولی حفظ می‌شود، گزینه‌ی ۳ تصور کرده هیچ شاخه‌ای اجرا نمی‌شود و گزینه‌ی ۴ ذخیره‌ی مقدار بولی را خطا دانسته است.

---

۹۵۹ **گزینه‌ی ۲** متوسط

شیء بدون `JSON.stringify` با روش پیش‌فرض به رشته تبدیل می‌شود که حاصلش `"[object Object]"` است و داده‌های اصلی از دست می‌رود. `saved` یک رشته است و رشته property به نام `name` ندارد؛ خواندن آن `undefined` می‌دهد، نه خطا. گزینه‌های ۱ و ۴ فکر کرده‌اند شیء حفظ یا خودکار به JSON تبدیل می‌شود و گزینه‌ی ۳ خواندن property ناموجود از یک رشته را خطا دانسته است.

۹۶۰ **گزینه‌ی ۳** سخت

آرایه هنگام ذخیره به رشته‌ی `"1,2,3"` تبدیل می‌شود که طولش ۵ کاراکتر است (سه رقم و دو کاما). `split(",")` آن را به `["1", "2", "3"]` می‌شکند و `map(Number)` همه را عدد می‌کند؛ پس عضو سوم `3` است و `3 + 1` برابر ۴. گزینه‌های ۱ و ۴ فکر کرده‌اند آرایه حفظ می‌شود (طول ۳) و گزینه‌های ۲ و ۴ اثر `map(Number)` را ندیده‌اند.

۹۶۱ **گزینه‌ی ۴** متوسط

`JSON.stringify` آرایه را به متن JSON تبدیل می‌کند (کلیدها با نقل‌قول دوتایی و بدون فاصله) و همین رشته ذخیره و چاپ می‌شود. `JSON.parse` دوباره یک آرایه‌ی واقعی می‌سازد که در آن `qty` **عدد** است؛ پس `2 + 1` برابر ۳. گزینه‌ی ۱ فکر کرده `stringify` اثری ندارد، گزینه‌ی ۲ تصور کرده بعد از `parse` هم عددها رشته‌اند و گزینه‌ی ۳ شکل نمایش Node.js را با متن JSON اشتباه گرفته است.

۹۶۲ **گزینه‌ی ۱** سخت

`getItem` برای کلید ناموجود `null` می‌دهد و `JSON.parse(null)` **خطا نمی‌دهد**: `null` به رشته‌ی `"null"` تبدیل می‌شود که JSON معتبری است و حاصلش `null` است. بعد `|| []` یک آرایه‌ی خالی جایگزین می‌کند، آیتم اضافه و آرایه به‌صورت JSON ذخیره می‌شود. گزینه‌ی ۲ فکر کرده `JSON.parse(null)` خطای نحوی می‌دهد، گزینه‌ی ۳ اثر `JSON.stringify` را نادیده گرفته و گزینه‌ی ۴ تصور کرده `push` روی `null` اجرا می‌شود.

۹۶۳ **گزینه‌ی ۲** سخت

`getItem` مقدار `null` می‌دهد و `Number(null)` برابر `0` است، پس `visits` برابر ۱ می‌شود. هنگام ذخیره به رشته‌ی `"1"` تبدیل می‌شود (نوع `string`). در خط بعد، بدون `Number()`، `"1" + 1` به‌صورت رشته‌ای `"11"` می‌شود. گزینه‌ی ۱ فکر کرده `Number(null)` برابر `NaN` است، گزینه‌ی ۳ تصور کرده نوع عدد حفظ می‌شود و گزینه‌ی ۴ در خط آخر جمع عددی انجام داده است.

۹۶۴ **گزینه‌ی ۳** سخت

نوشتن به شکل property (`localStorage.score = 20`) هم مقدار **همان کلید** را بازنویسی می‌کند (به‌صورت رشته‌ی `"20"`)، پس فقط یک کلید داریم. عملگر `*` رشته را به عدد تبدیل می‌کند: `"20" * 2` برابر ۴۰. گزینه‌ی ۱ فکر کرده property جدا از کلید ذخیره می‌شود، گزینه‌ی ۲ دو کلید جدا شمرده و گزینه‌ی ۴ رفتار `+` را به `*` نسبت داده است.

۹۶۵ **گزینه‌ی ۴** متوسط

origin از سه بخش ساخته می‌شود: **پروتکل + میزبان + پورت**؛ مسیر صفحه در آن نقشی ندارد. در گزینه‌ی ۴ هر سه بخش یکسان‌اند و فقط مسیر فرق دارد، پس انبار مشترک دارند. در گزینه‌ی ۱ پروتکل (`http` در برابر `https`)، در گزینه‌ی ۲ میزبان (زیردامنه‌ی `shop`) و در گزینه‌ی ۳ پورت (۴۴۳ در برابر ۸۰۸۰) فرق دارد؛ پس هر کدام origin و localStorage جداگانه دارند.

۹۶۶ **گزینه‌ی ۱** آسان

اسلاید ۳۸ می‌گوید localStorage برای اطلاعات حساس امن نیست: داده به‌صورت متن ساده ذخیره می‌شود، در DevTools دیده می‌شود و هر اسکریپتی که در صفحه اجرا شود، از جمله اسکریپت تزریق‌شده با XSS، به آن دسترسی دارد. گزینه‌ی ۲ غلط است (فقط رشته ذخیره می‌شود، نه فقط عدد)، گزینه‌ی ۳ غلط است (تاریخ انقضا ندارد) و گزینه‌ی ۴ غلط است چون سیاست same-origin جلوی دسترسی سایت‌های دیگر را می‌گیرد.

۹۶۷ **گزینه‌ی ۲** سخت

localStorage **تاریخ انقضا ندارد**؛ داده تا وقتی کد آن را حذف کند یا کاربر داده‌های مرورگر را پاک کند می‌ماند. پس گزینه‌ی ۲ نادرست و پاسخ است. گزینه‌های ۱ و ۳ عیناً از اسلایدهای ۳۸ و ۳۹ هستند و گزینه‌ی ۴ هم درست است، چون این دو، انبارهای جداگانه‌اند.

۹۶۸ **گزینه‌ی ۳** متوسط

ترجیحی که باید در بازدیدهای بعدی هم بماند (زبان، تم) جایش در localStorage است و وضعیتی که فقط برای همین تب و تا بسته شدنش معنا دارد (مرحله‌ی فرم چندمرحله‌ای) جایش در sessionStorage. گزینه‌ی ۲ کاربردها را برعکس کرده؛ سبد خریدی که باید بماند در sessionStorage با بستن تب از دست می‌رود. گزینه‌های ۱ و ۴ اطلاعات حساس را در Web Storage گذاشته‌اند که طبق اسلایدهای ۳۸ و ۳۹ امن نیست؛ تم تیره‌ی همیشگی هم در sessionStorage دوام نمی‌آورد.

۹۶۹ **گزینه‌ی ۴** متوسط

در ابتدا کلید `theme` وجود ندارد، پس کلاس `dark` اضافه نمی‌شود. هر کلیک کلاس را با `toggle` عوض می‌کند و وضعیت جدید (`true` یا `false`) را برمی‌گرداند؛ کد آن را به رشته‌ی `"dark"` یا `"light"` تبدیل و ذخیره می‌کند: تیره، روشن، تیره. گزینه‌ی ۱ فکر کرده `toggle` همیشه اضافه می‌کند، گزینه‌ی ۲ مقدار بولی را مستقیم ذخیره‌شده فرض کرده (ولی کد رشته‌ی `"dark"` یا `"light"` ذخیره می‌کند) و گزینه‌ی ۳ وضعیت شروع را تیره فرض کرده است.

آزمون جامع فصل دوم

۹۷۰ **گزینه‌ی ۴** سخت

اسکریپت معمولیِ داخل `<head>` قبل از parse شدن `<body>` اجرا می‌شود، پس `getElementById("save")` مقدار `null` می‌دهد و `head: null` چاپ می‌شود. ثبت listener برای `DOMContentLoaded` روی `document` مشکلی ندارد، ولی خط بعد `addEventListener` را روی `null` صدا می‌زند و `TypeError` رخ می‌دهد؛ بقیه‌ی **همین** اسکریپت (`head end`) اجرا نمی‌شود، اما مرورگر parse صفحه را ادامه می‌دهد و اسکریپت انتهای body مستقل اجرا می‌شود (`body script`). بعد از پایان parse رویداد `DOMContentLoaded` رخ می‌دهد و حالا دکمه وجود دارد (`ready: Save`). کلیک کاربر چیزی چاپ نمی‌کند، چون listener کلیک هرگز ثبت نشد.

گزینه‌ی ۱ فکر کرده خطای یک اسکریپت کل صفحه را متوقف می‌کند، گزینه‌ی ۲ ترتیب را برعکس گرفته (`DOMContentLoaded` بعد از اجرای همه‌ی اسکریپت‌های معمولی رخ می‌دهد) و گزینه‌ی ۳ فرض کرده listener کلیک با وجود خطا ثبت شده است.

۹۷۱ **گزینه‌ی ۱** سخت

فاصله و خط جدیدِ بین تگ‌ها هم گره‌ی متنی (text node) می‌سازد؛ پس `<ul>` در ابتدا سه گره‌ی فرزند دارد: متن فاصله، `<li>A</li>` و یک متن فاصله‌ی دیگر. `list.lastChild` همان گره‌ی متنی آخر است و `insertBefore(li, list.lastChild)` عنصر `B` را **قبل از آن متن**، یعنی بعد از `A`، می‌گذارد. در نتیجه `lastChild` هنوز `#text` است، آخرین **عنصر** `B` است، `children` فقط عنصرها را می‌شمارد (۲) و `childNodes` همه‌ی گره‌ها را (۴).

گزینه‌ی ۲ گره‌های متنی را نادیده گرفته (در آن صورت `B` قبل از `A` می‌رفت)، گزینه‌ی ۳ تصور کرده `insertBefore` مثل `appendChild` به انتهای لیست اضافه می‌کند و گزینه‌ی ۴ فکر کرده `B` قبل از `A` قرار گرفته است.

۹۷۲ **گزینه‌ی ۳** سخت

`getElementsByClassName` یک HTMLCollection **زنده** می‌دهد و `querySelectorAll` یک NodeList **ایستا** (عکسِ لحظه‌ای). وقتی کلاس `x` از یک پاراگراف حذف می‌شود، آن پاراگراف فوراً از `live` بیرون می‌رود و بقیه یک خانه جلو می‌آیند: در دور اول (`i = 0`) پاراگراف `1` حذف می‌شود و `live` می‌شود `[2, 3, 4]`؛ در دور دوم (`i = 1`) پاراگراف `3` حذف می‌شود و `live` می‌شود `[2, 4]`؛ حالا `i = 2` است و شرط `2 < 2` نادرست است. پس `live.length` برابر ۲، `fixed.length` همان ۴ و `live[0]` پاراگراف `2` است.

گزینه‌ی ۱ هر دو لیست را ایستا فرض کرده، گزینه‌ی ۲ `querySelectorAll` را هم زنده دانسته و گزینه‌ی ۴ فکر کرده حلقه کلاس همه را حذف می‌کند (در آن صورت `live[0]` برابر `undefined` می‌شد و خواندن `textContent` خطا می‌داد).

۹۷۳ **گزینه‌ی ۲** سخت

`list.innerHTML += "<li>C</li>"` یعنی `list.innerHTML = list.innerHTML + "<li>C</li>"`: محتوای لیست به رشته تبدیل، پاک و **از نو** ساخته می‌شود. پس سه `<li>` کاملاً تازه داریم و `first` (عنصر قدیمی `A`) دیگر در صفحه نیست (`isConnected` برابر `false`). `items` زنده است، پس طولش ۳ می‌شود و `items[0]` عنصر **جدید** است، نه `first`. `A` listener فقط روی عنصر قدیمی ثبت شده بود، پس کلیک روی `A` جدید چیزی چاپ نمی‌کند.

گزینه‌ی ۱ تصور کرده `+=` فقط یک آیتم اضافه می‌کند و بقیه دست‌نخورده می‌مانند، گزینه‌ی ۳ مجموعه‌ی `getElementsByTagName` را ایستا فرض کرده و گزینه‌ی ۴ فکر کرده listenerها هم همراه HTML کپی می‌شوند. برای اضافه کردن آیتم بدون از دست دادن listenerها از `createElement` و `append` استفاده کن.

---

۹۷۴ **گزینه‌ی ۲** سخت

خط ۱ یک NodeList ایستا می‌دهد که متد `forEach` دارد، پس خط ۲ بدون مشکل به هر دو آیتم کلاس `ready` می‌دهد (رد گزینه‌ی ۱). خط ۳ یک HTMLCollection زنده برمی‌گرداند و خط ۴ عدد ۲ را چاپ می‌کند. اما HTMLCollection متد `forEach` **ندارد**؛ `ready.forEach` برابر `undefined` است و صدا زدن آن در خط ۵ خطای `TypeError: ready.forEach is not a function` می‌دهد و اجرا همان‌جا متوقف می‌شود. خط ۶ هم خطا دارد (NodeList property به نام `classList` ندارد)، ولی اجرا هرگز به آن نمی‌رسد (رد گزینه‌ی ۳). گزینه‌ی ۴ هم HTMLCollection را مثل آرایه فرض کرده است. راه درست: `Array.from(ready).forEach(...)` یا استفاده از `querySelectorAll`.

---

۹۷۵ **گزینه‌ی ۴** سخت

handlerها به ترتیب ثبت اجرا می‌شوند. در تابع معمولی (`function`)، `this` همان عنصری است که listener روی آن ثبت شده (`btn`)، پس `this.label` برابر `"button"` است. تابع arrow، `this` مخصوص خودش را ندارد و آن را از محیط بیرونی، یعنی سطح بالای اسکریپت، می‌گیرد که در مرورگر `window` است؛ و چون `label` با `var` در سطح بالا تعریف شده، `window.label` برابر `"global"` است. handler سوم (`onclick`) بعد از آن دو ثبت شده و `this` آن هم `btn` است، پس `true` چاپ می‌کند.

گزینه‌ی ۱ فکر کرده arrow هم `this` را از عنصر می‌گیرد، گزینه‌ی ۲ فراموش کرده `var` سراسری property شیء `window` می‌شود و گزینه‌ی ۳ تصور کرده `onclick` همیشه قبل از listenerهای `addEventListener` اجرا می‌شود.

---

۹۷۶ **گزینه‌ی ۱** سخت

رویداد سه مرحله دارد: capturing (از `document` به سمت پایین)، رسیدن به target و bubbling (به سمت بالا). listenerی که روی `outer` با آرگومان سوم `true` ثبت شده در مرحله‌ی capturing اجرا می‌شود، پس اول `outer capture` چاپ می‌شود؛ بعد خود دکمه (`btn`) و در مسیر بالا رفتن `inner`. چون `inner` متد `stopPropagation()` را صدا می‌زند، رویداد دیگر به listener مرحله‌ی bubbling روی `outer` و به `document` نمی‌رسد.

گزینه‌ی ۲ مرحله‌ی capturing را نادیده گرفته، گزینه‌ی ۳ فکر کرده listener روی `document` چون بالاترین است اول اجرا می‌شود (در حالی که بدون `true` و در مرحله‌ی bubbling ثبت شده) و گزینه‌ی ۴ تصور کرده `stopPropagation` جلوی رسیدن رویداد به `document` را نمی‌گیرد.

---

۹۷۷ **گزینه‌ی ۳** سخت

با کلیک روی Add یک `<li>` تازه ساخته می‌شود. در لحظه‌ی ساختن متن، لیست هنوز فقط یک فرزند دارد، پس متن `B1` است و **بعد از آن** `appendChild` انجام می‌شود. کلیک روی آیتم جدید bubble می‌کند و به listener روی `<ul>` می‌رسد (event delegation؛ آیتم‌هایی که بعداً ساخته می‌شوند به listener جداگانه نیاز ندارند). در این listener که تابع معمولی است، `e.target` همان `<li>` کلیک‌شده است ولی `this` برابر `e.currentTarget`، یعنی `<ul>`، است. دکمه‌ی Add هم داخل `<ul>` نیست، پس کلیک اول listener لیست را اجرا نمی‌کند.

گزینه‌ی ۱ طول لیست را بعد از افزودن حساب کرده، گزینه‌ی ۲ `this` را با `e.target` یکی گرفته و گزینه‌ی ۴ فکر کرده عنصرهایی که بعداً ساخته می‌شوند به listener والد نمی‌رسند.

---

۹۷۸ گزینه‌ی ۴ متوسط

وقتی کاربر روی `<i>` کلیک می‌کند، `e.target` همان `<i>` است. `closest(".del")` از **خودِ عنصر** شروع می‌کند و به سمت اجداد بالا می‌رود؛ پس چه کلیک روی دکمه باشد و چه روی `<i>` داخل آن، دکمه را پیدا می‌کند و برای کلیک روی متن آیتم (`<li>`) مقدار `null` می‌دهد که با `return` نادیده گرفته می‌شود.

گزینه‌ی ۱ از `this` داخل یک تابع arrow استفاده کرده که برابر `window` است و `window.querySelector` وجود ندارد (`TypeError`)؛ تازه اگر هم کار می‌کرد، همیشه اولین دکمه را می‌داد. گزینه‌ی ۲ یک مقدار بولی برمی‌گرداند: کلیک روی `<i>` نادیده گرفته می‌شود و کلیک روی خود دکمه به `true.closest(...)` و `TypeError` می‌رسد. گزینه‌ی ۳ از `currentTarget` یعنی خود `<ul>` شروع کرده که هیچ دکمه‌ای در اجدادش نیست، پس همیشه `null` می‌دهد و چیزی حذف نمی‌شود.

---

۹۷۹ گزینه‌ی ۱ سخت

کلیک روی دکمه‌ی `x` آیتم Bread به listener روی `<ul>` می‌رسد؛ `closest` اول دکمه و بعد `<li>` آن را پیدا می‌کند و متن `<span>` در پیام confirm قرار می‌گیرد. کاربر OK می‌زند (`confirm` مقدار `true` برمی‌گرداند)، پس `li.remove()` آیتم را از صفحه جدا می‌کند و `cart.children` که **زنده** است به ۱ می‌رسد. خودِ شیء `li` هنوز در متغیر هست و خواندن `textContent` آن خطا ندارد؛ `textContent` متن همه‌ی نوادگان را پشت سر هم می‌دهد: `Bread` ، فاصله‌ی بین تگ‌ها و `x` دکمه. کلیک دوم روی `<span>` است، `closest(".del")` مقدار `null` می‌دهد و listener با `return` تمام می‌شود.

گزینه‌ی ۲ متن دکمه‌ی داخل `<li>` را حساب نکرده، گزینه‌ی ۳ فکر کرده عنصر حذف‌شده دیگر قابل استفاده نیست و گزینه‌ی ۴ تصور کرده `children` با حذف عنصر به‌روز نمی‌شود.

---

۹۸۰ گزینه‌ی ۲ سخت

`cloneNode(true)` عنصر را با همه‌ی attributeها و فرزندانش کپی می‌کند. `onclick` در HTML یک attribute معمولی است و کپی می‌شود، پس دکمه‌ی جدید handler درون‌خطی خودش را دارد؛ در این handler، `this` همان دکمه‌ای است که کلیک شده و چون `id` آن قبل از کلیک به `"b"` تغییر کرده، `inline b` چاپ می‌شود. اما listenerهایی که با `addEventListener` ثبت شده‌اند جزو HTML نیستند و **هرگز** با `cloneNode` کپی نمی‌شوند.

گزینه‌ی ۱ فکر کرده clone عمیق listenerها را هم کپی می‌کند، گزینه‌ی ۳ برعکس تصور کرده attribute کپی نمی‌شود ولی listener کپی می‌شود و گزینه‌ی ۴ فرض کرده نسخه‌ی کپی هیچ رفتاری ندارد.

---

۹۸۱ گزینه‌ی ۳ سخت

`prompt` رشته‌ای را که کاربر تایپ کرده برمی‌گرداند. `textContent` همه‌چیز را **متن خام** در نظر می‌گیرد؛ پس `a` هیچ عنصر فرزندی ندارد و متنش دقیقاً `Hi <b>Sara</b>` با ۱۴ نویسه است. `innerHTML` رشته را به‌عنوان HTML تفسیر می‌کند و یک عنصر `<b>` می‌سازد؛ پس `b` یک فرزند دارد و `textContent` آن (که تگ‌ها را حساب نمی‌کند) `Hi Sara` با ۷ نویسه است.

گزینه‌ی ۱ فکر کرده `textContent` هم HTML را تفسیر می‌کند، گزینه‌ی ۲ تصور کرده `innerHTML` هم متن خام می‌گذارد و گزینه‌ی ۴ با اینکه عنصر `<b>` را درست شمرده، تگ‌ها را جزو `textContent` حساب کرده است. همین تفاوت دلیل این است که ورودی کاربر را نباید با `innerHTML` در صفحه گذاشت.

---

۹۸۲ گزینه‌ی ۴ سخت

خط ۳ عنصر `b` را از DOM جدا می‌کند و خط ۴ روی عنصری که دیگر پدر ندارد هیچ کاری نمی‌کند، ولی **خطا هم نمی‌دهد** (رد گزینه‌ی ۱). در خط ۵ جست‌وجو در صفحه `null` برمی‌گرداند، اما متغیر `b` هنوز به شیء عنصر اشاره می‌کند و `textContent` آن `B` است (رد گزینه‌ی ۲).
خط ۶ همان عنصر را دوباره به انتهای `list` اضافه می‌کند که کاملاً مجاز است (رد گزینه‌ی ۳). اما `removeChild` فقط یک فرزند **مستقیم** را حذف می‌کند و پدر `b` حالا `list` است، نه `body` ؛ پس خط ۷ خطای `NotFoundError` می‌دهد. راه امن‌تر: `b.remove()` یا `b.parentNode.removeChild(b)` .

---

۹۸۳ گزینه‌ی ۱ سخت

تیک زدن checkbox فقط **property** `checked` را `true` می‌کند و attribute `checked` به HTML اضافه نمی‌شود (این attribute فقط حالت اولیه را مشخص می‌کند)، پس `agree.hasAttribute("checked")` برابر `false` است. اما `disabled` یک property بازتابی (reflected، یعنی متصل به attribute هم‌نام) است: `send.disabled = false` attribute `disabled` را حذف می‌کند، پس `hasAttribute("disabled")` هم `false` است. `toggle("ready")` کلاس را به کلاس‌های قبلی **اضافه** می‌کند، پس `className` برابر `"btn ready"` می‌شود.

گزینه‌ی ۲ فکر کرده تغییر property `disabled` به attribute نمی‌رسد، گزینه‌ی ۳ تصور کرده تیک زدن attribute `checked` را اضافه می‌کند و گزینه‌ی ۴ فکر کرده `toggle` کل `className` را جایگزین می‌کند.

---

۹۸۴ گزینه‌ی ۲ متوسط

`setInterval` callback را **بلافاصله** اجرا نمی‌کند؛ اولین اجرا حدود ۱۰ میلی‌ثانیه بعد است و تا آن موقع کد همگام تمام شده، پس اول `start card` چاپ می‌شود. سپس هر تیک کلاس `on` را تغییر وضعیت می‌دهد: تیک ۱ اضافه، تیک ۲ حذف و تیک ۳ دوباره اضافه. در تیک سوم `clearInterval` تکرار را متوقف می‌کند و `3 card on` چاپ می‌شود (کلاس `card` دست نمی‌خورد).

گزینه‌ی ۱ تایمر را قبل از کد همگام اجرا کرده، گزینه‌ی ۳ فکر کرده `toggle` کل `className` را جایگزین می‌کند و گزینه‌ی ۴ تصور کرده اولین اجرای `setInterval` همان لحظه‌ی صدا زدن آن رخ می‌دهد.

---

۹۸۵ گزینه‌ی ۳ سخت

هر رویداد `input` اول تایمر **قبلی** را با `clearTimeout` لغو می‌کند و بعد یک تایمر ۵۰۰ میلی‌ثانیه‌ای تازه می‌سازد. چون حرف‌ها با فاصله‌ی کمتر از ۵۰۰ میلی‌ثانیه تایپ شده‌اند، دو تایمر اول قبل از اجرا لغو می‌شوند و فقط تایمر آخر اجرا می‌شود و مقدار نهایی کادر، `dom`، را چاپ می‌کند (به این الگو debounce می‌گویند: کار سنگین فقط بعد از توقف تایپ انجام شود).

گزینه‌ی ۲ همان نتیجه‌ی کد **بدون** `clearTimeout` است: هر سه تایمر بعد از حرف آخر اجرا می‌شوند و همه `q.value` را **هنگام اجرا** می‌خوانند. گزینه‌ی ۱ حتی بدون `clearTimeout` هم غلط است، چون callback مقدار لحظه‌ی ساخته شدن تایمر را نگه نمی‌دارد. گزینه‌ی ۴ فکر کرده `clearTimeout` تایمر تازه را هم لغو می‌کند، در حالی که فقط تایمری لغو می‌شود که شناسه‌اش در آن لحظه در `timer` است.

---

۹۸۶ گزینه‌ی ۴ سخت

بار اول کلید `items` وجود ندارد، `getItem` مقدار `null` می‌دهد و `JSON.parse(null)` هم `null` است؛ پس `|| []` یک آرایه‌ی خالی می‌سازد و `pen` به آن اضافه می‌شود. اما `setItem` مقدار را فقط **به رشته تبدیل** می‌کند (نه به JSON) و آرایه‌ی `["pen"]` با `toString` می‌شود `"pen"`؛ پس خروجی اول `pen` است. در ارسال دوم `JSON.parse("pen")` یک JSON نامعتبر را می‌خواند و `SyntaxError` پرتاب می‌کند؛ `|| []` کمکی نمی‌کند، چون خطا قبل از رسیدن به آن رخ می‌دهد. `preventDefault` فقط جلوی reload صفحه را می‌گیرد و ربطی به این خطا ندارد.

گزینه‌ی ۱ فکر کرده `setItem` خودش JSON می‌سازد، گزینه‌ی ۲ تصور کرده رشته‌ی `"pen"` دوباره به آرایه برمی‌گردد و گزینه‌ی ۳ فکر کرده خطای `JSON.parse` بی‌صدا به `|| []` می‌رسد. راه درست: `localStorage.setItem("items", JSON.stringify(items))`.

---

۹۸۷ گزینه‌ی ۱ متوسط

`e.preventDefault()` رفتار پیش‌فرض رویداد `submit`، یعنی ارسال فرم و بارگذاری دوباره‌ی صفحه، را لغو می‌کند؛ `stopPropagation` فقط جلوی bubbling را می‌گیرد و صفحه باز هم reload می‌شود (رد گزینه‌ی ۲). `localStorage` فقط رشته نگه می‌دارد، پس آرایه باید با `JSON.stringify` به رشته‌ی JSON (مثلاً `'["Read"]'`) تبدیل شود تا `JSON.parse` بعداً همان آرایه را بسازد. `todos.toString()` رشته‌ای مثل `"Read"` می‌سازد که JSON معتبر نیست و `JSON.parse` روی آن `SyntaxError` می‌دهد (رد گزینه‌ی ۳)؛ `JSON.parse(todos)` هم کار برعکس را انجام می‌دهد و همان موقع ذخیره با `SyntaxError` متوقف می‌شود (رد گزینه‌ی ۴).

---

۹۸۸ **گزینه‌ی ۲** متوسط

هر چیزی که در Web Storage ذخیره شود، چه با `setItem` و چه با نوشتن مستقیم property (`sessionStorage.visits = ...`)، به **رشته** تبدیل می‌شود. `Number("1") + 1` برابر عدد ۲ است که به شکل `"2"` در `sessionStorage` ذخیره می‌شود؛ `getItem` رشته‌ی `"2"` را برمی‌گرداند و `"2" + 1` الحاق رشته‌ای است: `"21"`. `typeof sessionStorage.visits` هم `"string"` است و `localStorage` از `sessionStorage` جداست، پس مقدار آن همان `"1"` می‌ماند.

گزینه‌ی ۱ فکر کرده عدد به همان شکل عددی ذخیره می‌شود، گزینه‌ی ۳ دو حافظه را یکی گرفته و گزینه‌ی ۴ با اینکه نوع را رشته دانسته، `+ 1` را جمع عددی حساب کرده است.

---

۹۸۹ **گزینه‌ی ۴** سخت

خط ۱ خطا نمی‌دهد: `setItem` شیء را به رشته‌ی `"[object Object]"` تبدیل و ذخیره می‌کند (رد گزینه‌ی ۱). در خط ۳، `raw` یک رشته است و خواندن property ناموجود روی رشته فقط `undefined` می‌دهد (رد گزینه‌ی ۲). در خط ۴ کلید `cart` وجود ندارد، `getItem` مقدار `null` برمی‌گرداند و `JSON.parse(null)` بدون خطا `null` می‌دهد (رد گزینه‌ی ۳). اما در خط ۶، `JSON.parse("[object Object]")` یک JSON نامعتبر را می‌خواند و `SyntaxError` رخ می‌دهد. درس این تست: شیء و آرایه را همیشه با `JSON.stringify` ذخیره کن و با `JSON.parse` بخوان.

---

۹۹۰ **گزینه‌ی ۱** سخت

`querySelectorAll` یک NodeList **ایستا** می‌دهد که سه عنصر `A`، `B` و `D` را از اول تا آخر حلقه ثابت نگه می‌دارد؛ حذف هر کدام روی خود لیست اثری ندارد و هر سه حذف می‌شوند.

`getElementsByClassName` زنده است: بعد از حذف `A`، عنصر `B` به اندیس ۰ می‌آید و حلقه با `i = 1` از رویش رد می‌شود و `D` را حذف می‌کند؛ پس `B` می‌ماند (رد گزینه‌ی ۲). `querySelector` فقط یک عنصر برمی‌گرداند که property به نام `length` ندارد (`undefined`)؛ شرط `0 < undefined` نادرست است و هیچ چیزی حذف نمی‌شود (رد گزینه‌ی ۳). `getElementsByTagName("li")` هم زنده است و هم آیتم‌های بدون `done` را شامل می‌شود؛ در نتیجه `C` به‌اشتباه حذف می‌شود و `B` و `D` می‌مانند (رد گزینه‌ی ۴).

---

۹۹۱ **گزینه‌ی ۳** متوسط

`remove()` فقط عنصر را از درخت DOM جدا می‌کند؛ خودِ شیء از بین نمی‌رود و اگر متغیری به آن اشاره کند، می‌توان آن را با `appendChild` یا `append` دوباره در صفحه گذاشت (گزینه‌ی ۳ درست است). گزینه‌ی ۱ نادرست است: NodeList حاصل از `querySelectorAll` ایستاست و عنصرهای بعدی را نمی‌بیند (HTMLCollection حاصل از `getElementsByClassName` زنده است).

گزینه‌ی ۲ نادرست است: برای کلید ناموجود `null` برمی‌گردد. گزینه‌ی ۴ نادرست است: `stopPropagation` فقط bubbling را متوقف می‌کند و برای لغو ارسال فرم باید `preventDefault()` صدا زده شود.

---

۹۹۲ **گزینه‌ی ۳** سخت

`setTimeout(fn, 0)` تابع را فقط **زمان‌بندی** می‌کند؛ `fn` وقتی اجرا می‌شود که کد همگامِ در حال اجرا تمام شده باشد، پس همیشه بعد از خط‌های بعدیِ همان اسکریپت اجرا می‌شود و گزینه‌ی ۳ نادرست است. گزینه‌ی ۱ درست است: اسکریپت معمولی در `<head>` قبل از parse شدن `<body>` اجرا می‌شود و دکمه هنوز وجود ندارد. گزینه‌ی ۲ درست است: `var` سراسری property شیء `window` می‌شود ولی `let` و `const` نه. گزینه‌ی ۴ هم درست است: `sessionStorage` تا وقتی تب باز است (حتی بعد از reload) باقی می‌ماند و با بستن تب پاک می‌شود.

---

۹۹۳ **گزینه‌ی ۲** سخت

در listenerای که با `function` معمولی ثبت شده، مرورگر `this` را برابر عنصری قرار می‌دهد که listener روی آن ثبت شده، یعنی `event.currentTarget`؛ در event delegation این عنصر والد است، در حالی که `event.target` فرزندی است که واقعاً کلیک شده (گزینه‌ی ۲ درست است). گزینه‌ی ۱ نادرست است: مقداردهی به `innerHTML` همه‌ی فرزندان را از نو می‌سازد، حتی اگر رشته همان قبلی باشد، و listenerهای فرزندان قدیمی از بین می‌روند. گزینه‌ی ۳ نادرست است: `toggle` مقدار بولی برمی‌گرداند (`true` اگر کلاس اضافه شد و `false` اگر حذف شد). گزینه‌ی ۴ نادرست است: `el.style` فقط استایل درون‌خطی (attribute `style`) را نشان می‌دهد و برای استایل نهاییِ حاصل از CSS باید از `getComputedStyle` استفاده کرد.

پاسخ‌های فصل سوم: Node.js و TypeScript

کلید سریع فصل سوم

۹۹۴ ۳	۹۹۵ ۴	۹۹۶ ۲	۹۹۷ ۱	۹۹۸ ۲	۹۹۹ ۱	۱۰۰۰ ۱	۱۰۰۱ ۳	۱۰۰۲ ۴	۱۰۰۳ ۲
۱۰۰۴ ۳	۱۰۰۵ ۴	۱۰۰۶ ۱	۱۰۰۷ ۳	۱۰۰۸ ۲	۱۰۰۹ ۴	۱۰۱۰ ۳	۱۰۱۱ ۲	۱۰۱۲ ۴	۱۰۱۳ ۱
۱۰۱۴ ۳	۱۰۱۵ ۱	۱۰۱۶ ۴	۱۰۱۷ ۳	۱۰۱۸ ۲	۱۰۱۹ ۳	۱۰۲۰ ۱	۱۰۲۱ ۴	۱۰۲۲ ۲	۱۰۲۳ ۳
۱۰۲۴ ۱	۱۰۲۵ ۴	۱۰۲۶ ۲	۱۰۲۷ ۳	۱۰۲۸ ۴	۱۰۲۹ ۱	۱۰۳۰ ۲	۱۰۳۱ ۲	۱۰۳۲ ۴	۱۰۳۳ ۱
۱۰۳۴ ۳	۱۰۳۵ ۴	۱۰۳۶ ۱	۱۰۳۷ ۱	۱۰۳۸ ۲	۱۰۳۹ ۳	۱۰۴۰ ۴	۱۰۴۱ ۲	۱۰۴۲ ۱	۱۰۴۳ ۳
۱۰۴۴ ۲	۱۰۴۵ ۴	۱۰۴۶ ۳	۱۰۴۷ ۱	۱۰۴۸ ۲	۱۰۴۹ ۱	۱۰۵۰ ۳	۱۰۵۱ ۲	۱۰۵۲ ۴	۱۰۵۳ ۲
۱۰۵۴ ۱	۱۰۵۵ ۳	۱۰۵۶ ۴	۱۰۵۷ ۱	۱۰۵۸ ۳	۱۰۵۹ ۴	۱۰۶۰ ۲	۱۰۶۱ ۱	۱۰۶۲ ۳	۱۰۶۳ ۴
۱۰۶۴ ۲	۱۰۶۵ ۳	۱۰۶۶ ۴	۱۰۶۷ ۲	۱۰۶۸ ۳	۱۰۶۹ ۱	۱۰۷۰ ۴	۱۰۷۱ ۲	۱۰۷۲ ۳	۱۰۷۳ ۱
۱۰۷۴ ۴	۱۰۷۵ ۲	۱۰۷۶ ۳	۱۰۷۷ ۱	۱۰۷۸ ۴	۱۰۷۹ ۲	۱۰۸۰ ۳	۱۰۸۱ ۱	۱۰۸۲ ۴	۱۰۸۳ ۲
۱۰۸۴ ۳	۱۰۸۵ ۱	۱۰۸۶ ۴	۱۰۸۷ ۲	۱۰۸۸ ۲	۱۰۸۹ ۴	۱۰۹۰ ۱	۱۰۹۱ ۳	۱۰۹۲ ۲	۱۰۹۳ ۴
۱۰۹۴ ۱	۱۰۹۵ ۳	۱۰۹۶ ۲	۱۰۹۷ ۱	۱۰۹۸ ۳	۱۰۹۹ ۴	۱۱۰۰ ۲	۱۱۰۱ ۱	۱۱۰۲ ۳	۱۱۰۳ ۱
۱۱۰۴ ۳	۱۱۰۵ ۲	۱۱۰۶ ۴	۱۱۰۷ ۱	۱۱۰۸ ۳	۱۱۰۹ ۲	۱۱۱۰ ۴	۱۱۱۱ ۱	۱۱۱۲ ۳	۱۱۱۳ ۲
۱۱۱۴ ۴	۱۱۱۵ ۱	۱۱۱۶ ۳	۱۱۱۷ ۴	۱۱۱۸ ۲	۱۱۱۹ ۳	۱۱۲۰ ۲	۱۱۲۱ ۴	۱۱۲۲ ۱	۱۱۲۳ ۱
۱۱۲۴ ۳	۱۱۲۵ ۲	۱۱۲۶ ۲	۱۱۲۷ ۳	۱۱۲۸ ۱	۱۱۲۹ ۴	۱۱۳۰ ۴	۱۱۳۱ ۱	۱۱۳۲ ۳	۱۱۳۳ ۲
۱۱۳۴ ۴	۱۱۳۵ ۴	۱۱۳۶ ۱	۱۱۳۷ ۲	۱۱۳۸ ۳	۱۱۳۹ ۳	۱۱۴۰ ۲	۱۱۴۱ ۱	۱۱۴۲ ۳	۱۱۴۳ ۱
۱۱۴۴ ۳	۱۱۴۵ ۴	۱۱۴۶ ۳	۱۱۴۷ ۴	۱۱۴۸ ۲	۱۱۴۹ ۱	۱۱۵۰ ۲	۱۱۵۱ ۳	۱۱۵۲ ۲	۱۱۵۳ ۳
۱۱۵۴ ۳	۱۱۵۵ ۱	۱۱۵۶ ۱	۱۱۵۷ ۲	۱۱۵۸ ۴	۱۱۵۹ ۱	۱۱۶۰ ۴	۱۱۶۱ ۴	۱۱۶۲ ۴	۱۱۶۳ ۱
۱۱۶۴ ۲	۱۱۶۵ ۱	۱۱۶۶ ۳	۱۱۶۷ ۴	۱۱۶۸ ۲	۱۱۶۹ ۴	۱۱۷۰ ۱	۱۱۷۱ ۲	۱۱۷۲ ۴	۱۱۷۳ ۳
۱۱۷۴ ۲	۱۱۷۵ ۱	۱۱۷۶ ۱	۱۱۷۷ ۲	۱۱۷۸ ۱	۱۱۷۹ ۳	۱۱۸۰ ۱	۱۱۸۱ ۴	۱۱۸۲ ۳	۱۱۸۳ ۴
۱۱۸۴ ۳	۱۱۸۵ ۴	۱۱۸۶ ۲	۱۱۸۷ ۲	۱۱۸۸ ۱	۱۱۸۹ ۳	۱۱۹۰ ۳	۱۱۹۱ ۴	۱۱۹۲ ۲	۱۱۹۳ ۳
۱۱۹۴ ۱	۱۱۹۵ ۴	۱۱۹۶ ۳	۱۱۹۷ ۲	۱۱۹۸ ۲	۱۱۹۹ ۳	۱۲۰۰ ۱	۱۲۰۱ ۲	۱۲۰۲ ۴	۱۲۰۳ ۱
۱۲۰۴ ۴	۱۲۰۵ ۲	۱۲۰۶ ۱	۱۲۰۷ ۳	۱۲۰۸ ۴	۱۲۰۹ ۱	۱۲۱۰ ۴	۱۲۱۱ ۳	۱۲۱۲ ۳	۱۲۱۳ ۱
۱۲۱۴ ۴	۱۲۱۵ ۲	۱۲۱۶ ۳	۱۲۱۷ ۱	۱۲۱۸ ۱	۱۲۱۹ ۳	۱۲۲۰ ۴	۱۲۲۱ ۱	۱۲۲۲ ۲	۱۲۲۳ ۴
۱۲۲۴ ۲	۱۲۲۵ ۴	۱۲۲۶ ۳	۱۲۲۷ ۴	۱۲۲۸ ۱	۱۲۲۹ ۲	۱۲۳۰ ۲	۱۲۳۱ ۲	۱۲۳۲ ۴	۱۲۳۳ ۱
۱۲۳۴ ۲	۱۲۳۵ ۳	۱۲۳۶ ۴	۱۲۳۷ ۳	۱۲۳۸ ۱	۱۲۳۹ ۳	۱۲۴۰ ۲	۱۲۴۱ ۴	۱۲۴۲ ۳	۱۲۴۳ ۲
۱۲۴۴ ۴	۱۲۴۵ ۱	۱۲۴۶ ۳	۱۲۴۷ ۳	۱۲۴۸ ۴	۱۲۴۹ ۱	۱۲۵۰ ۴	۱۲۵۱ ۲	۱۲۵۲ ۳	۱۲۵۳ ۲
۱۲۵۴ ۱	۱۲۵۵ ۳	۱۲۵۶ ۴	۱۲۵۷ ۲	۱۲۵۸ ۱	۱۲۵۹ ۳	۱۲۶۰ ۱	۱۲۶۱ ۴	۱۲۶۲ ۲	۱۲۶۳ ۳
۱۲۶۴ ۴	۱۲۶۵ ۳	۱۲۶۶ ۱	۱۲۶۷ ۲	۱۲۶۸ ۳	۱۲۶۹ ۴	۱۲۷۰ ۲	۱۲۷۱ ۱	۱۲۷۲ ۲	۱۲۷۳ ۱
۱۲۷۴ ۴	۱۲۷۵ ۳	۱۲۷۶ ۱	۱۲۷۷ ۴	۱۲۷۸ ۳	۱۲۷۹ ۴	۱۲۸۰ ۲	۱۲۸۱ ۱	۱۲۸۲ ۳	۱۲۸۳ ۲
۱۲۸۴ ۴	۱۲۸۵ ۴	۱۲۸۶ ۳	۱۲۸۷ ۱	۱۲۸۸ ۴	۱۲۸۹ ۲	۱۲۹۰ ۳	۱۲۹۱ ۴	۱۲۹۲ ۳	۱۲۹۳ ۱
۱۲۹۴ ۲	۱۲۹۵ ۱								

۳-۱ محدودیت‌های JavaScript و تولد Node.js

۹۹۴ **گزینه‌ی ۳** آسان

سه کارت اسلاید ۲ این‌ها هستند: «Make console applications with JS» (گزینه‌ی ۱)، «Learn how JS runs on servers» (گزینه‌ی ۲) و «Solve the old JS typing issues» (گزینه‌ی ۴). طراحی ظاهر صفحه با CSS اصلاً موضوع این جلسه نیست؛ این جلسه درباره‌ی بردن JavaScript به بیرون از مرورگر (Node.js) و TypeScript است.

۹۹۵ **گزینه‌ی ۴** آسان

ماجرا برعکس است: JavaScript در اصل **برای مرورگر** و کارهای ساده‌ی آن طراحی شده بود و مشکلش این بود که برای برنامه‌های کامل و منطق backend ساخته نشده بود. گزینه‌های ۱، ۲ و ۳ دقیقاً سه مورد از پنج محدودیت اسلاید ۳ هستند (دو مورد دیگر: محدود بودن به کارهای ساده‌ی مرورگر، و ابزار محدود و مقیاس‌پذیری ضعیف).

۹۹۶ **گزینه‌ی ۲** آسان

اسلاید ۴ می‌گوید «Engine improvements like Google's V8 made JavaScript fast enough for server workloads» و اسلاید ۵ می‌گوید «Node.js Powered by Google's fast V8 engine» است. بقیه‌ی گزینه‌ها هم موتورهای واقعی JavaScript هستند (مال Firefox، Safari و Edge قدیمی)، ولی Node.js از هیچ‌کدام استفاده نمی‌کند.

۹۹۷ **گزینه‌ی ۱** متوسط

در non-blocking I/O، تنها thread برنامه (گارسون) کار کُند I/O را به سیستم (آشپزخانه) می‌سپارد، منتظر نمی‌ماند و وقتی نتیجه آماده شد، با یک رویداد (زنگ) callback اجرا می‌شود. گزینه‌ی ۲ همان مدل blocking است. گزینه‌ی ۳ مدل «یک thread برای هر درخواست» است که Node.js از آن استفاده نمی‌کند. در گزینه‌ی ۴ گارسون خودش همه‌ی کار را انجام می‌دهد، پس دوباره همه منتظر یک نفر می‌مانند.

۹۹۸ **گزینه‌ی ۲** متوسط

`fs.readFile` غیرمسدودکننده (non-blocking) است: خواندن فایل را شروع می‌کند و منتظر نمی‌ماند، پس اول `end` چاپ می‌شود. وقتی خواندن فایل تمام شد، callback اجرا می‌شود و `file read` می‌آید. گزینه‌ی ۱ رفتار `readFileSync` (مسدودکننده) است. گزینه‌ی ۴ غلط است چون callback بالاخره اجرا می‌شود و برنامه تا تمام شدن کارهای در جریان بسته نمی‌شود.

۹۹۹ **گزینه‌ی ۱** متوسط

در گزینه‌ی ۱، `start` و `end` همگام چاپ می‌شوند و callback غیرمسدودکننده‌ی `readFile` بعداً `done` را چاپ می‌کند. گزینه‌ی ۲ با `readFileSync` منتظر می‌ماند، پس اول `start`، بعد `done` و در آخر `end` چاپ می‌شود. در گزینه‌ی ۳، `start` داخل callback است و آخر از همه می‌آید (اول `end`، بعد `done`، بعد `start`). در گزینه‌ی ۴ هم `end` داخل callback است، پس `end` بعد از `done` چاپ می‌شود.

۱۰۰۰ **گزینه‌ی ۱** سخت

JavaScript تک‌نخی است. حتی با زمان `0`، callback تایمر فقط وقتی اجرا می‌شود که کد همگام فعلی تمام شده باشد. حلقه‌ی `while` حدود ۱۰۰ میلی‌ثانیه تنها thread را **مسدود** می‌کند، پس `timer` نمی‌تواند وسط حلقه (گزینه‌ی ۳) یا قبل از همه (گزینه‌ی ۲) چاپ شود. گزینه‌ی ۴ هم غلط است، چون تایمر بالاخره اجرا می‌شود. این دقیقاً همان مشکل blocking در اسلاید ۳ است.

۱۰۰۱ **گزینه‌ی ۳** آسان

Node.js یک **runtime** است: موتور V8 به‌اضافه‌ی ابزارهایی مثل `fs` و `http` که اجرای JavaScript را روی سرور و در ترمینال ممکن می‌کند. زبانش همان JavaScript است (رد گزینه‌ی ۱)، کتابخانه‌ی رابط کاربری نیست (React یک کتابخانه‌ی UI است؛ رد گزینه‌ی ۲) و مرورگر هم نیست، چون اصلاً صفحه و DOM ندارد (رد گزینه‌ی ۴).

۱۰۰۲ **گزینه‌ی ۴** متوسط

Node.js شیء `window` ندارد، ولی `typeof` روی متغیر تعریف‌نشده خطا نمی‌دهد و فقط `"undefined"` برمی‌گرداند (رد گزینه‌ی ۲). `globalThis` شیء سراسری استاندارد است که در هر دو محیط وجود دارد (رد گزینه‌ی ۳) و `process` هم مخصوص Node.js است و وجود دارد. گزینه‌ی ۱ جای مرورگر و Node.js را عوض کرده است.

۱۰۰۳ **گزینه‌ی ۲** سخت

خط ۱ فقط `undefined` چاپ می‌کند، چون `typeof` روی متغیر تعریف‌نشده خطا نمی‌دهد. خط‌های ۲ و ۳ هم سالم‌اند، چون `os` یک ماژول داخلی Node.js است. در خط ۴ واقعاً از `window` استفاده شده و Node.js چنین شیئی ندارد، پس `ReferenceError: window is not defined` رخ می‌دهد و برنامه همان‌جا متوقف می‌شود. برای همین خط ۵ هرگز اجرا نمی‌شود و گزینه‌ی «بدون خطا» هم غلط است.

۱۰۰۴ **گزینه‌ی ۳** متوسط

اسلاید ۴: «The rise of AJAX and JSON showed JS could efficiently handle asynchronous operations». با AJAX صفحه در پس‌زمینه و بدون بارگذاری دوباره داده می‌گیرد، یعنی کاری ناهمگام. این ربطی به چندنخی شدن ندارد (رد گزینه‌ی ۱)؛ امروزه بیشتر داده با JSON جابه‌جا می‌شود نه XML (رد گزینه‌ی ۲)؛ و همین موفقیت‌ها الهام‌بخش بردن JS به سرور شد، نه برعکس (رد گزینه‌ی ۴).

۱۰۰۵ **گزینه‌ی ۴** متوسط

در مدل رویدادمحور، برنامه برای رویدادها (درخواست، پایان I/O، تایمر و...) تابع callback ثبت می‌کند و Node.js وقتی رویداد رخ داد، آن تابع را صدا می‌زند. گزینه‌ی ۱ تعریف مدل blocking است و گزینه‌ی ۲ مدل «یک thread برای هر درخواست». گزینه‌ی ۳ رویداد را فقط به کلیک موس در صفحه محدود کرده، در حالی که Node.js اصلاً صفحه‌ای ندارد و رویدادهایش از جنس شبکه، فایل و تایمرند.

۱۰۰۶ **گزینه‌ی ۱** سخت

Node.js صفحه‌ی وب و DOM ندارد؛ `document` و `window` در آن تعریف نشده‌اند و استفاده از آن‌ها `ReferenceError` می‌دهد (شیء سراسری‌اش `globalThis` یا `global` است). گزینه‌های ۲ و ۳ مستقیماً در اسلاید ۵ آمده‌اند. گزینه‌ی ۴ هم درست است، چون JavaScript تک‌نخی است و کار محاسباتی سنگین همان یک thread را مسدود می‌کند.

۱۰۰۷ **گزینه‌ی ۳** متوسط

اسلاید ۵: «Event-driven, non-blocking I/O model». چون Node.js منتظر تمام شدن I/O نمی‌ماند و با رویداد و callback از نتیجه باخبر می‌شود، یک thread برای درخواست‌های زیاد کافی است. گزینه‌ی ۱ با «یک thread» تناقض دارد و گزینه‌های ۲ و ۴ (همگام و مسدودکننده) دقیقاً همان چیزی هستند که باعث معطل ماندن درخواست‌ها می‌شود.

۱۰۰۸ **گزینه‌ی ۲** سخت

Node.js همچنان **تک‌نخی** است و برای هر درخواست thread نمی‌سازد؛ راه‌حلش برای مشکل blocking، مدل **رویدادمحور و غیرمسدودکننده** است: I/O در پس‌زمینه انجام می‌شود و نتیجه با callback می‌رسد. سه جفت دیگر درست‌اند: TypeScript برای مشکلات نوع، ماژول‌ها و npm برای سازمان‌دهی و استفاده‌ی دوباره از کد، و خود Node.js برای بیرون آمدن از مرورگر.

۱۰۰۹ **گزینه‌ی ۴** آسان

پنج مورد اسلاید ۵ این‌هاست: موتور V8، I/O رویدادمحور و غیرمسدودکننده، یک زبان برای frontend و backend، اکوسیستم عظیم npm، و تبدیل JavaScript به یک پلتفرم کامل. بررسی نوع در زمان کامپایل کار **TypeScript** است، نه Node.js. خود Node.js کد را بدون بررسی نوع اجرا می‌کند.

۱۰۱۰ **گزینه‌ی ۳** سخت

`createServer` و `listen` فقط callbackها را **ثبت** می‌کنند و صبر نمی‌کنند، پس اول `setup done` چاپ می‌شود (رد گزینه‌های ۱ و ۴). وقتی سرور آماده‌ی گوش دادن شد، رویدادِ `listening` چاپ می‌شود؛ بعد سرور بسته می‌شود و `closed` می‌آید. callback `createServer` فقط با رسیدن یک **درخواست** اجرا می‌شود و چون درخواستی نیامده، `request received` هرگز چاپ نمی‌شود (رد گزینه‌ی ۲). این همان event-driven بودن است: کد فقط به رویدادهایی که واقعاً رخ می‌دهند واکنش نشان می‌دهد.

۳-۲ ویژگی‌های Node.js

۱۰۱۱ **گزینه‌ی ۲** آسان

هشت برچسب نمودار این‌ها هستند: Single-Threaded، Event-Driven، No Buffering، Asynchronous. Node.js **Single-Threaded** (تک‌نخی) است، نه Multi-Threaded؛ کد JavaScript روی یک thread اجرا می‌شود. گزینه‌های ۱، ۳ و ۴ عیناً روی نمودار هستند.

۱۰۱۲ **گزینه‌ی ۴** آسان

No Buffering یعنی Node.js به‌جای جمع کردن کل داده در حافظه (buffer کردن)، آن را به‌صورت stream و تکه‌تکه خروجی می‌دهد. این به معنای استفاده نکردن از حافظه نیست (رد گزینه‌ی ۱) و نگه داشتن هر تکه در یک متغیر هم کاملاً ممکن است (رد گزینه‌ی ۳). اتفاقاً همین روش باعث می‌شود Node.js فایل‌های خیلی بزرگ را راحت پردازش کند (رد گزینه‌ی ۲).

۱۰۱۳ **گزینه‌ی ۱** متوسط

خواندن stream ناهمگام است، پس اول کد همگام اجرا می‌شود و `reading...` می‌آید (رد گزینه‌های ۳ و ۴). بعد فایل ۱۰ حرفی در تکه‌های حداکثر ۴ حرفی خوانده می‌شود: `abcd`، `efgh` و `ij`، و برای هر تکه یک رویداد `data` رخ می‌دهد. در پایان رویداد `end` و `done` می‌آید. گزینه‌ی ۲ فرض کرده کل فایل یکجا (buffer شده) خوانده می‌شود، که برخلاف ایده‌ی stream است.

۱۰۱۴ **گزینه‌ی ۳** سخت

`on` فقط شنونده را ثبت می‌کند و چیزی چاپ نمی‌کند، پس اول `Waiting...` می‌آید (رد گزینه‌ی ۲). `emit` شنونده‌ها را **همگام** و همان لحظه اجرا می‌کند، پس `Preparing pizza` قبل از `Done` چاپ می‌شود (رد گزینه‌ی ۱ که `emit` را مثل `setTimeout` فرض کرده). برای رویداد `cancel` هیچ شنونده‌ای ثبت نشده؛ در این حالت `emit` فقط `false` برمی‌گرداند و خطایی رخ نمی‌دهد (رد گزینه‌ی ۴).

۱۰۱۵ **گزینه‌ی ۱** آسان

Cross-Platform یعنی «چندسکویی»: Node.js برای Windows، Linux و macOS وجود دارد و همان کد JavaScript روی همه‌شان اجرا می‌شود (`process.platform` نام سیستم‌عامل فعلی را می‌دهد). گزینه‌ی ۲ «سکو» را با مرورگر و سرور اشتباه گرفته؛ گزینه‌ی ۳ دقیقاً برعکس این ویژگی است؛ و Node.js فقط JavaScript اجرا می‌کند، نه زبان‌های دیگر (رد گزینه‌ی ۴).

۱۰۱۶ **گزینه‌ی ۴** متوسط

`fs/promises` نسخه‌ی Promiseمحور `fs` است. `main()` تا اولین `await` جلو می‌رود؛ چون نوشتن فایل کاری ناهمگام است، کنترل به بیرون برمی‌گردد و `main started` چاپ می‌شود. بعد از کامل شدن نوشتن و خواندن، `Hello file` چاپ می‌شود. گزینه‌ی ۱ فرض کرده `await` کل برنامه را متوقف می‌کند، در حالی که فقط ادامه‌ی همان تابع `async` منتظر می‌ماند. گزینه‌های ۲ و ۳ هم یکی از دو خروجی را جا انداخته‌اند؛ هر دو خط چاپ می‌شوند.

۱۰۱۷ **گزینه‌ی ۳** آسان

برنامه‌ی اجرایی Node.js در ترمینال `node` نام دارد و نام فایل را به آن می‌دهیم: `node app.js`. دستور `npm` مدیر بسته است و `npm app.js` خطای «Unknown command» می‌دهد. `run` و `js` هم دستورهای Node.js نیستند.

۱۰۱۸ **گزینه‌ی ۲** متوسط

`node` بدون آرگومان، **REPL** (Read-Eval-Print Loop) را باز می‌کند؛ مثلاً اگر `1 + 2` بنویسی، `3` نشان می‌دهد و با `.exit` بیرون می‌آیی. چاپ نسخه کار `node -v` است (گزینه‌ی ۱)، ساختن `package.json` کار `npm init` است (گزینه‌ی ۳) و نیاز به نام فایل فقط برای اجرای یک فایل است (گزینه‌ی ۴).

۱۰۱۹ **گزینه‌ی ۳** سخت

در CommonJS نوشتن پسوند `.js` در `require` اختیاری است، پس `./math` و `./math.js` هر دو به همان فایل می‌رسند و خطایی رخ نمی‌دهد (رد گزینه‌ی ۴). Node.js هر ماژول را فقط **بار اول** اجرا و نتیجه را cache می‌کند؛ پس `math.js loaded` فقط یک بار چاپ می‌شود (رد گزینه‌ی ۱). کد بالای فایل `math.js` هنگام اولین `require` واقعاً اجرا می‌شود، پس گزینه‌ی ۲ هم غلط است.

۱۰۲۰ **گزینه‌ی ۱** سخت

چیزی که `require` برمی‌گرداند همیشه `module.exports` است. `exports` در ابتدا فقط یک ارجاع (میان‌بُر) به همان شیء است. در `good.js` یک ویژگی به همان شیء اضافه شده، پس `{ hello: 'hi' }` صادر می‌شود. در `bad.js` متغیر محلی `exports` به یک شیء **جدید** اشاره داده شده و `module.exports` همان شیء خالی اولیه مانده است، پس `{}` برمی‌گردد. خروجی `undefined` (گزینه‌ی ۴) هم غلط است، چون `module.exports` از اول یک شیء خالی است.

۱۰۲۱ **گزینه‌ی ۴** متوسط

`"type": "module"` می‌گوید همه‌ی فایل‌های `.js` این پوشه ES Module هستند. در ESM تابع `require` تعریف نشده است، پس خط اول خطای `ReferenceError: require is not defined in ES module scope` می‌دهد و چیزی چاپ نمی‌شود. خطا از نوع SyntaxError نیست، چون صدا زدن یک تابع از نظر نحوی کاملاً درست است و فقط آن نام وجود ندارد. راه درست: `import fs from "fs";` یا تغییر پسوند فایل به `.cjs` .

۱۰۲۲ **گزینه‌ی ۲** سخت

با `"type": "commonjs"`، فایل `index.js` به‌صورت CommonJS اجرا می‌شود و در CommonJS دستور `import` از نظر **نحوی** مجاز نیست؛ پس قبل از اجرای هر خطی، `SyntaxError: Cannot use import statement outside a module` رخ می‌دهد و هیچ چیزی چاپ نمی‌شود. `ReferenceError` (گزینه‌ی ۳) خطای حالت برعکس است، یعنی `require` داخل ESM. برای درست کردنش یا `"type": "module"` بنویس، یا پسوند را `.mjs` کن، یا به جای `import` از `require` استفاده کن.

۱۰۲۳ **گزینه‌ی ۳** متوسط

پسوند `.mjs` همیشه یعنی ES Module و `"type": "module"` هم فایل‌های `.js` آن پروژه را ESM می‌کند. پسوند `.cjs` و `"type": "commonjs"` دقیقاً برعکس، یعنی CommonJS را تعیین می‌کنند. فیلد `main` فقط فایل ورودی پکیج را مشخص می‌کند و ربطی به نوع ماژول ندارد.

۱۰۲۴ **گزینه‌ی ۱** متوسط

در ES Module تابع `require` وجود ندارد، ولی `typeof` روی نام تعریف‌نشده خطا نمی‌دهد و `"undefined"` برمی‌گرداند (رد گزینه‌ی ۴). مقدار `this` در بالاترین سطح یک ES Module برابر `undefined` است. `setTimeout` هم در Node.js یک تابع سراسری است (رد گزینه‌ی ۳). گزینه‌ی ۲ خروجی همین کد در یک فایل **CommonJS** است، که در آن `require` تابع و `this` همان `{}` (یعنی `module.exports`) است.

۱۰۲۵ **گزینه‌ی ۴** متوسط

این میم به dependency bloat اشاره دارد: npm همه‌ی پکیج‌هایی که برنامه لازم دارد، **به‌اضافه‌ی** پکیج‌هایی که خود آن‌ها لازم دارند، را در `node_modules` می‌ریزد و این پوشه از کد خود برنامه خیلی بزرگ‌تر می‌شود. اعداد میم اغراق طنز است، نه قانون (رد گزینه‌های ۱ و ۳). کد خود ما بیرون از `node_modules` است؛ این پوشه فقط پکیج‌های نصب‌شده را نگه می‌دارد (رد گزینه‌ی ۲).

۱۰۲۶ **گزینه‌ی ۲** متوسط

متن اسلاید ۷: «Supports powerful frameworks like Express, NestJS, and Fastify». هر سه برای ساختن سرور و API با Node.js هستند. React کتابخانه‌ای برای ساختن **رابط کاربری** (UI) در سمت frontend است (اسلاید ۱۳: «library for building interactive UIs with components») و فریم‌ورک سمت سرور نیست.

۱۰۲۷ **گزینه‌ی ۳** سخت

`setTimeout`، `Promise`، `console`، `JSON` و `Math` در **هر دو** محیط وجود دارند؛ اصلاً event loop و Promiseها ستون اصلی ناهمگام بودن Node.js هستند. گزینه‌ی ۱ درست است (`globalThis` در مرورگر همان `window` و در Node.js همان `global` است). گزینه‌ی ۲ درست است، چون صفحه‌ی وب برای امنیت کاربر به فایل‌های دیسک دسترسی آزاد ندارد. گزینه‌ی ۴ هم درست است، چون Node.js شیء `window` ندارد و `typeof` روی نام تعریف‌نشده `"undefined"` می‌دهد.

۱۰۲۸ **گزینه‌ی ۴** سخت

Node.js نسخه‌ی ۲۴ فایل `.ts` را با «پاک کردن نوع‌ها» (type stripping) اجرا می‌کند: `: number` حذف می‌شود و کد به `let count = "ten";` تبدیل می‌شود. پس `ten` چاپ می‌شود. Node.js نوع‌ها را **بررسی نمی‌کند**؛ خطای نوع را فقط کامپایلر TypeScript (`tsc`) گزارش می‌دهد (رد گزینه‌ی ۱). Node.js این نحو را می‌شناسد و پاکش می‌کند، پس SyntaxError هم رخ نمی‌دهد (رد گزینه‌ی ۲). هیچ تبدیل نوعی هم انجام نمی‌شود (رد گزینه‌ی ۳).

۱۰۲۹ **گزینه‌ی ۱** آسان

در CommonJS با `require("./file")` وارد می‌کنیم و با `module.exports` (یا میان‌بُر `exports.name`) صادر می‌کنیم. گزینه‌ی ۲ نحو **ES Modules** است، نه CommonJS. `include` و `module.import` در JavaScript وجود ندارند، `fetch` برای درخواست HTTP است و `window` هم اصلاً در Node.js نیست.

۱۰۳۰ **گزینه‌ی ۲** متوسط

راز مقیاس‌پذیری Node.js همان مدل event-driven و non-blocking است: thread اصلی کارهای I/O را به پس‌زمینه می‌سپارد و فوراً سراغ درخواست بعدی می‌رود. گزینه‌ی ۱ با تک‌نخی بودن تناقض دارد. کد سرور در مرورگر کاربر اجرا نمی‌شود (رد گزینه‌ی ۳). گزینه‌ی ۴ هم برعکس ویژگی No Buffering است.

۳-۳ پکیج‌های خارجی و npm

۱۰۳۱ **گزینه‌ی ۲** آسان

عنوان اسلاید ۹: «NPM (Node Package Manager)»؛ یعنی «مدیر بسته‌ی Node». کار npm مدیریت پکیج‌هاست: نصب، به‌روزرسانی، حذف و انتشار. بقیه‌ی گزینه‌ها اسم‌های ساختگی هستند.

۱۰۳۲ **گزینه‌ی ۴** آسان

اسلاید ۸ دقیقاً برعکسِ گزینه‌ی ۴ را می‌گوید: «Also raised issues of trust, maintenance, and dependency overload». پکیج‌های خارجی مشکلات اعتماد و نگه‌داری را **ایجاد** می‌کنند، نه اینکه از بین ببرند. گزینه‌های ۱، ۲ و ۳ سه مورد از مزیت‌های همین اسلاید هستند.

۱۰۳۳ **گزینه‌ی ۱** آسان

npm کد واقعی پکیج‌ها (و وابستگی‌های آن‌ها) را در پوشه‌ی `node_modules` می‌گذارد. در `package.json` فقط **اسم و بازه‌ی نسخه‌ی** وابستگی‌های مستقیم نوشته می‌شود و `package-lock.json` نسخه‌های دقیق را ثبت می‌کند، ولی هیچ‌کدام کد پکیج را نگه نمی‌دارند. `.npmrc` هم فایل تنظیمات npm است (مثلاً آدرس registry).

۱۰۳۴ **گزینه‌ی ۳** متوسط

`package.json` معمولاً بازه‌ای مثل `^3.0.1` دارد، ولی `package-lock.json` نسخه‌ی دقیق نصب‌شده (مثلاً `3.0.1`)، آدرس `resolved` و `integrity` هر پکیج را ثبت می‌کند. پس هر کسی با `npm install` دقیقاً همان درخت وابستگی را می‌گیرد. اسکریپت‌ها در `package.json` هستند (رد گزینه‌ی ۱)، lock فایل کپی `package.json` نیست (رد گزینه‌ی ۲) و «lock» به معنی قفل کردن نسخه‌هاست، نه ممنوع کردن نصب (رد گزینه‌ی ۴).

۱۰۳۵ **گزینه‌ی ۴** متوسط

`^4.17.20` یعنی «از `4.17.20` به بالا، ولی کمتر از `5.0.0`»؛ پس MINOR و PATCH جدیدتر مجازند و `4.18.1` در بازه است. `5.0.0` یک MAJOR جدید (و احتمالاً ناسازگار) است و `^` اجازه‌اش را نمی‌دهد. `4.16.0` و `4.17.19` هم از حداقل بازه (`4.17.20`) قدیمی‌ترند.

۱۰۳۶ **گزینه‌ی ۱** متوسط

علامت `~` فقط PATCH جدیدتر را مجاز می‌کند: `~2.3.1` یعنی از `2.3.1` تا قبل از `2.4.0`. پس `2.3.9` مجاز است. `2.4.0` یک MINOR جدید است و `3.0.0` یک MAJOR جدید، که هیچ‌کدام در بازه نیستند. `2.3.0` هم از حداقل بازه (`2.3.1`) قدیمی‌تر است.

۱۰۳۷ **گزینه‌ی ۱** سخت

دام اصلی همین‌جاست: برای نسخه‌های `0.x` ، علامت `^` محتاط‌تر است و MINOR را تغییرِ احتمالاً ناسازگار حساب می‌کند. پس `^0.4.2` یعنی از `0.4.2` تا قبل از `0.5.0` و `0.4.9` مجاز است. کسی که قاعده‌ی معمول `^` را به کار ببرد (تا قبل از `1.0.0`)، به اشتباه `0.5.0` را انتخاب می‌کند. `1.0.0` قطعاً بیرون از بازه است و `0.4.1` از حداقل بازه پایین‌تر است.

۱۰۳۸ **گزینه‌ی ۲** متوسط

در قالب MAJOR.MINOR.PATCH، افزایش MAJOR (اولین عدد) یعنی تغییر **ناسازگار**؛ برای همین `^3.4.2` هیچ‌وقت خودکار به `4.0.0` نمی‌رود. رفع باگ فقط PATCH را زیاد می‌کند (مثلاً `3.4.3` ، رد گزینه‌ی ۱) و قابلیت جدید سازگار فقط MINOR را (مثلاً `3.5.0` ، رد گزینه‌ی ۳). در SemVer هر عدد معنای مشخصی دارد (رد گزینه‌ی ۴).

۱۰۳۹ **گزینه‌ی ۳** متوسط

رفع باگ بدون تغییر در نحوه‌ی استفاده یعنی افزایش PATCH (سومین عدد): `2.5.4` . `2.6.0` برای قابلیت جدید سازگار است و `3.0.0` برای تغییر ناسازگار. SemVer فقط سه عدد اصلی دارد و قالبی مثل `2.5.3.1` جزو آن نیست.

۱۰۴۰ **گزینه‌ی ۴** سخت

npm وابستگی‌های یک پکیج را خودکار پیدا و نصب می‌کند؛ با این دستور خروجی «added 2 packages» است و هر دو پکیج در `node_modules` قرار می‌گیرند (رد گزینه‌های ۱ و ۳). ولی در `package.json` ما فقط وابستگی **مستقیم**، یعنی `is-odd` ، نوشته می‌شود (رد گزینه‌ی ۲). جای ثبت `is-number` در `package-lock.json` است.

۱۰۴۱ **گزینه‌ی ۲** متوسط

devDependencies فقط هنگام **توسعه** لازم‌اند. کامپایلر TypeScript کارش را قبل از اجرا تمام می‌کند و برنامه‌ی در حال اجرا (که JavaScript کامپایل‌شده است) به آن نیازی ندارد؛ پس با `npm install -D typescript` نصب می‌شود. `express` ، `axios` و `socket.io` همگی هنگام **اجرای** سرور استفاده می‌شوند و باید در dependencies باشند.

۱۰۴۲ **گزینه‌ی ۱** سخت

`node_modules` حجیم است و چون با `npm install` از روی `package.json` و `package-lock.json` دوباره ساخته می‌شود، در git ثبت **نمی‌شود** (اسمش در `.gitignore` می‌آید). هم‌تیمی‌ها بعد از گرفتن پروژه فقط `npm install` می‌زنند. گزینه‌های ۲، ۳ و ۴ هر سه درست‌اند.

۱۰۴۳ **گزینه‌ی ۳** متوسط

جمله‌ی آخر اسلاید ۹: «Later faced challenges with security, size, and version chaos». گزینه‌ی ۱ برعکس است؛ خود npm مخزن مرکزی را آورد («Introduced a central registry»). V8 و async/await ربطی به چالش‌های npm ندارند و npm مثل Node.js روی همه‌ی سیستم‌عامل‌ها کار می‌کند.

۱۰۴۴ **گزینه‌ی ۲** سخت

npm دقیقاً همان اسمی را نصب می‌کند که تایپ کرده‌ای و آن را اصلاح نمی‌کند (رد گزینه‌ی ۱). مهاجم‌ها با منتشر کردن پکیج‌هایی با اسم‌های شبیه پکیج‌های معروف (typosquatting) از همین اشتباه‌ها سوءاستفاده می‌کنند. `npm install <name>` برای اضافه کردن پکیج **جدید** است و لازم نیست اسمش از قبل در `package.json` باشد (رد گزینه‌ی ۳). کد پکیج‌ها پیش از انتشار بررسی امنیتی کامل نمی‌شود و همین یکی از چالش‌های امنیتی است که اسلاید ۹ می‌گوید (رد گزینه‌ی ۴).

۱۰۴۵ **گزینه‌ی ۴** آسان

اسلاید ۹: «registry: Introduced a **central registry** to manage and distribute packages». همان مخزن آنلاینی است که پکیج‌ها در آن منتشر و از آن دانلود می‌شوند (پیش‌فرض: `https://registry.npmjs.org/`). npm نه کامپایلر است، نه مرورگر؛ bundler هم ابزار جداگانه‌ای (مثل Webpack) برای یکی کردن فایل‌ها است.

۱۰۴۶ **گزینه‌ی ۳** سخت

بازه‌ی `^4.1.0` یعنی «از `4.1.0` تا قبل از `5.0.0`». نسخه‌های `4.1.0`، `4.1.2` و `4.2.0` در این بازه‌اند و npm **جدیدترین نسخه‌ی داخل بازه** را انتخاب می‌کند، یعنی `4.2.0`. نسخه‌ی latest یعنی `5.0.0` یک MAJOR جدید است و بیرون از بازه قرار دارد، پس نصب نمی‌شود. `4.1.0` فقط حداقلِ بازه است و `4.1.2` هم جدیدترینِ داخل بازه نیست.

۱۰۴۷ **گزینه‌ی ۱** آسان

اسلاید ۹ می‌گوید npm ساختار پروژه را با `package.json` استاندارد کرد؛ اسکریپت‌ها و وابستگی‌ها در این فایل تعریف می‌شوند. `package-lock.json` نسخه‌های دقیق نصب‌شده را ثبت می‌کند و اسکریپتی ندارد. `node_modules` پوشه‌ی کد پکیج‌هاست و `.npmrc` تنظیمات خود npm را نگه می‌دارد.

۱۰۴۸ **گزینه‌ی ۲** متوسط

اسلاید ۸ سه دردسر می‌شمارد: maintenance، trust و dependency overload. پکیج رهاشده‌ای که دیگر به‌روز نمی‌شود، مشکل **نگه‌داری** است. trust درباره‌ی اطمینان از سالم و غیرمخرب بودن کد است و dependency overload درباره‌ی زیاد بودن تعداد وابستگی‌ها. rapid development هم اصلاً دردسر نیست، بلکه یکی از مزیت‌های همین اسلاید است.

۴-۳ دستورات npm و تنظیم mirror

۱۰۴۹ **گزینه‌ی ۱** آسان

اسلاید ۱۰: «`npm init`: Create a new project and generate a package.json file». `npm start` اسکریپت `start` را اجرا می‌کند، `npm update` وابستگی‌ها را به‌روز می‌کند و `npm config get registry` فقط آدرس registry را نشان می‌دهد؛ هیچ‌کدام `package.json` نمی‌سازند.

۱۰۵۰ **گزینه‌ی ۳** آسان

`-y` کوتاه‌شده‌ی `--yes` است: یعنی «به همه‌ی سؤال‌ها بله بگو» و جواب‌های پیش‌فرض (اسم پوشه، نسخه‌ی `1.0.0`، `index.js` و...) را بپذیر. نصب وابستگی‌ها کار `npm install` است، `-y` چیزی را پاک نمی‌کند و ربطی به yarn ندارد.

۱۰۵۱ **گزینه‌ی ۲** متوسط

`npm init -y` نسخه را `1.0.0` می‌گذارد. اسم پیش‌فرض، اسم **پوشه** است (`"my-app"`، رد گزینه‌ی ۱)، فایل ورودی پیش‌فرض `index.js` است (رد گزینه‌ی ۳) و مجوز پیش‌فرض `ISC` است، نه `MIT` (رد گزینه‌ی ۴).

۱۰۵۲ **گزینه‌ی ۴** متوسط

`npm install <package>` سه کار انجام می‌دهد: دانلود به `node_modules`، ثبت در `dependencies` با بازه‌ی `^` و ساخت یا به‌روزرسانی `package-lock.json`. گزینه‌ی ۱ ثبت در `package.json` را جا انداخته است. `devDependencies` فقط با `-D` پر می‌شود (رد گزینه‌ی ۲) و نصب سراسری فقط با `-g` انجام می‌شود (رد گزینه‌ی ۳).

۱۰۵۳ **گزینه‌ی ۲** آسان

`i` اسم کوتاه `install` است؛ پس `npm i express` همان `npm install express` است. `npm init` دستور جداگانه‌ای برای ساختن پروژه است، `npm info` اطلاعات یک پکیج را از registry نشان می‌دهد و دستوری به اسم `npm import` اصلاً وجود ندارد.

۱۰۵۴ **گزینه‌ی ۱** متوسط

`npm install jest` بدون `-D` پکیج را در `dependencies` (وابستگی‌های زمان اجرا) می‌نویسد. `-D` (یا `--save-dev`) پکیج را در `devDependencies` ثبت می‌کند. `-g` نصب سراسری روی سیستم است و `package.json` را تغییر نمی‌دهد. `npm update` هم فقط پکیج‌های موجود را به‌روز می‌کند.

۱۰۵۵ **گزینه‌ی ۳** متوسط

`npm install` بدون اسم پکیج، همه‌ی وابستگی‌های فهرست‌شده در `package.json` را با نسخه‌های قفل‌شده در `package-lock.json` نصب و `node_modules` را دوباره می‌سازد. `npm init` برای ساختن `package.json` تازه است، `npm update -g` پکیج‌های **سراسری** را به‌روز می‌کند و `npm run node_modules` دنبال اسکریپتی به این اسم می‌گردد و چون چنین اسکریپتی نیست، خطای `Missing script` می‌دهد.

۱۰۵۶ **گزینه‌ی ۴** سخت

`-g` کوتاه‌شده‌ی `--global` است: پکیج در پوشه‌ی سراسری npm نصب می‌شود (نه در `node_modules` پروژه) و دستورهای خط فرمانش، مثل `tsc`، در همه‌جای سیستم در دسترس قرار می‌گیرند. چون نصب سراسری مال سیستم است، `package.json` پروژه دست نمی‌خورد (رد گزینه‌های ۱ و ۲). `-g` هیچ ربطی به GitHub ندارد (رد گزینه‌ی ۳).

۱۰۵۷ **گزینه‌ی ۱** آسان

اسلاید ۱۰: «`npm uninstall <package>` : Remove a previously installed package». این دستور پکیج را از `node_modules`، `package.json` و `package-lock.json` حذف می‌کند. `npm update` پکیج را به‌روز می‌کند و حذفش نمی‌کند. دستوری به اسم `npm delete` در npm وجود ندارد (خطای `Unknown command` می‌دهد). `npm install lodash@0` هم یک نسخه‌ی قدیمی را **نصب** می‌کند.

۱۰۵۸ **گزینه‌ی ۳** متوسط

`npm update` فقط تا جدیدترین نسخه‌ی **داخل بازه** جلو می‌رود و `^6.0.0` یعنی کمتر از `7.0.0`. چون نسخه‌ی جدیدتری در محدوده‌ی `6.x` نیست، خروجی «up to date» است و نسخه `6.0.0` می‌ماند. در خروجی `npm outdated` هم ستون Wanted مقدار `6.0.0` و ستون Latest مقدار `7.0.0` را نشان می‌دهد. `npm update` نه خود npm را به‌روز می‌کند و نه پکیجی را حذف می‌کند.

۱۰۵۹ **گزینه‌ی ۴** آسان

registry پیش‌فرض و رسمی npm آدرس `https://registry.npmjs.org/` است و `npm config get registry` همین را چاپ می‌کند. `npmjs.com` وب‌سایت npm برای جست‌وجو و خواندن مستندات است، نه آدرس registry. دو آدرس دیگر ساختگی‌اند.

۱۰۶۰ **گزینه‌ی ۲** متوسط

اسلاید ۱۰: «`npm config set registry <url>`». `set` مقدار را تنظیم و ذخیره می‌کند، ولی `get` فقط مقدار فعلی را **می‌خواند** (رد گزینه‌ی ۱). `npm install` برای نصب پکیج است و این دستور سعی می‌کند پکیجی به اسم `registry` نصب کند (رد گزینه‌ی ۳). دستوری به اسم `npm mirror` هم وجود ندارد (رد گزینه‌ی ۴).

۱۰۶۱ **گزینه‌ی ۱** متوسط

اسلاید ۱۰ می‌گوید «or set it in .npmrc file». فایل `.npmrc` فایل تنظیمات npm است و خط `registry=<url>` در آن دقیقاً همان کار `npm config set registry <url>` را می‌کند (این دستور هم در اصل همین خط را در `~/.npmrc` می‌نویسد). بخش `dependencies` فقط اسم و نسخه‌ی پکیج‌ها را نگه می‌دارد، `.gitignore` فهرست فایل‌هایی است که git نادیده می‌گیرد و `package-lock.json` را هم خود npm می‌سازد.

۱۰۶۲ **گزینه‌ی ۳** سخت

`npm config set` تنظیم را **ماندگار** در فایل تنظیمات کاربر (`~/.npmrc`) ذخیره می‌کند و از آن به بعد همه‌ی دستورهای npm از mirror استفاده می‌کنند. خود این دستور چیزی نصب نمی‌کند (رد گزینه‌ی ۱) و به `package.json` هم دست نمی‌زند (رد گزینه‌ی ۲). تنظیم یک‌باره برای یک دستور، کار گزینه‌ی `--registry=<url>` روی همان دستور است (رد گزینه‌ی ۴).

۱۰۶۳ **گزینه‌ی ۴** متوسط

mirror کپی همان مخزن پکیج‌ها روی سرور دیگری است؛ پس فقط **محل دانلود** را عوض می‌کند. کاربردش سرعت بیشتر (سرور نزدیک‌تر) و در دسترس بودن (وقتی registry اصلی کُند یا محدود است) است. mirror ربطی به انتشار پکیج، حذف وابستگی‌ها یا سرعت اجرای اسکریپت‌ها ندارد.

۱۰۶۴ **گزینه‌ی ۲** سخت

تنظیمات npm از چند جا خوانده می‌شوند و جای **خاص‌تر** برنده است: گزینه‌ی خود دستور (`--registry=…`)، بعد `.npmrc` پروژه، بعد `~/.npmrc` کاربر و در آخر مقدار پیش‌فرض. پس داخل پروژه، آدرس `.npmrc` پروژه نشان داده می‌شود. مقدار پیش‌فرض فقط وقتی استفاده می‌شود که هیچ‌جا تنظیمی نباشد، و چند تنظیم هم‌زمان خطا نمی‌دهند.

۱۰۶۵ **گزینه‌ی ۳** متوسط

`npx` برای **اجرای** دستورِ یک پکیج است؛ پکیج را (در صورت نیاز) در cache دانلود می‌کند ولی به `package.json` یا `node_modules` پروژه اضافه نمی‌کند (رد گزینه‌ی ۱). انتشار پکیج کار `npm publish` است (رد گزینه‌ی ۲) و اجرای اسکریپت‌های `package.json` کار `npm run` است (رد گزینه‌ی ۴).

۱۰۶۶ **گزینه‌ی ۴** سخت

`npm install` بدون اسم، **همه‌ی** وابستگی‌های نوشته‌شده در `package.json` را نصب می‌کند و رایج‌ترین دستور بعد از گرفتن یک پروژه است؛ پس گزینه‌ی ۴ نادرست است. گزینه‌ی ۱ درست است چون `i` اسم کوتاه `install` است، گزینه‌ی ۲ درست است چون `-D` همان `--save-dev` است و گزینه‌ی ۳ هم رفتار واقعی `npm uninstall` است.

۱۰۶۷ **گزینه‌ی ۲** سخت

همان نسخه‌ی `4.17.21` نصب می‌شود، ولی npm آن را به‌طور پیش‌فرض با پیشوند `^` ذخیره می‌کند. برای ذخیره‌ی دقیق و بدون `^` (گزینه‌ی ۱) باید `-E` یا `--save-exact` بزنی. npm پیشوند `~` (گزینه‌ی ۳) را به‌طور پیش‌فرض استفاده نمی‌کند و `latest` (گزینه‌ی ۴) هم نوشته نمی‌شود.

۳-۵ اسکریپت‌های npm و فایل package.json

۱۰۶۸ **گزینه‌ی ۳** آسان

اسلاید ۱۱: «Define custom commands in the scripts section of package.json». `package-lock.json` نسخه‌های دقیق پکیج‌ها را ثبت می‌کند، `.npmrc` تنظیمات خود npm (مثل registry) را نگه می‌دارد و `node_modules/.bin` محل فایل‌های اجرایی پکیج‌های نصب‌شده است، نه جای تعریف اسکریپت‌ها.

۱۰۶۹ **گزینه‌ی ۱** آسان

اسکریپت‌های دلخواه با `npm run <script>` اجرا می‌شوند. `npm build` در npm نسخه‌ی ۱۱ خطای `Unknown command: "build"` می‌دهد، چون `build` جزو اسم‌های خاصی نیست که بدون `run` اجرا شوند. `node build` به دنبال فایلی به اسم `build` می‌گردد و `npm install build` پکیجی به اسم `build` را از registry نصب می‌کند.

۱۰۷۰ **گزینه‌ی ۴** متوسط

`test` (مثل `start`) اسم خاصی است که npm آن را بدون `run` هم اجرا می‌کند. `npm dev`، `npm lint` و `npm build` هر سه خطای `Unknown command` می‌دهند (npm حتی پیشنهاد می‌دهد «Did you mean this? npm run dev»)؛ این اسکریپت‌ها حتماً با `npm run` اجرا می‌شوند.

۱۰۷۱ **گزینه‌ی ۲** متوسط

وقتی `run` را نوشته‌ای ولی اسکریپتی با آن اسم نیست، npm خطای `Missing script: "deploy"` می‌دهد و پیشنهاد می‌کند با `npm run` فهرست اسکریپت‌ها را ببینی. پیام `Unknown command` (گزینه‌ی ۱) مال وقتی است که **بدون** `run` بنویسی، مثلاً `npm deploy`. npm نه بی‌صدا تمام می‌شود و نه اسکریپتی برای ما می‌سازد.

۱۰۷۲ **گزینه‌ی ۳** آسان

`npm run` به‌تنهایی فقط اسکریپت‌ها را فهرست می‌کند: اول اسکریپت‌های خاص مثل `start` و `test` («Lifecycle scripts included in …») و بعد بقیه («available via `npm run`»). هیچ اسکریپتی را اجرا نمی‌کند و خطایی هم نمی‌دهد.

۱۰۷۳ **گزینه‌ی ۱** سخت

npm برای هر اسکریپت دنبال `pre<name>` و `post<name>` هم می‌گردد. با `npm run build` اول `prebuild`، بعد `build` و در آخر `postbuild` اجرا می‌شود؛ پس خروجی `A`، `B`، `C` است. ترتیب نوشتن کلیدها در فایل اهمیتی ندارد (رد گزینه‌ی ۲)، و `prebuild` و `postbuild` هم واقعاً خودکار اجرا می‌شوند (رد گزینه‌ی ۳). گزینه‌ی ۴ هم ترتیب pre و build را جابه‌جا کرده است.

۱۰۷۴ **گزینه‌ی ۴** سخت

`ci` اول `npm test` را اجرا می‌کند. اسکریپت `test` پیام را چاپ می‌کند و با `exit 1` شکست می‌خورد. عملگر `&&` دستور دوم را **فقط** در صورت موفقیت دستور اول اجرا می‌کند، پس `npm run build` هرگز اجرا نمی‌شود و `npm run ci` با کد ۱ تمام می‌شود. گزینه‌ی ۱ `&&` را با `;` اشتباه گرفته است. ترتیب اجرا هم از چپ به راست است (رد گزینه‌ی ۲) و اسکریپت `ci` تعریف شده است (رد گزینه‌ی ۳).

۱۰۷۵ **گزینه‌ی ۲** متوسط

npm اسم اسکریپت در حال اجرا را در `npm_lifecycle_event` می‌گذارد. `npm_package_name` اسم **پکیج** (فیلد `name` در `package.json`) است. متغیری به اسم `npm_script_name` ساخته نمی‌شود. `NODE_ENV` هم یک متغیر قراردادی است (مثلاً `production`) که npm خودش برای اسم اسکریپت پر نمی‌کند.

۱۰۷۶ **گزینه‌ی ۳** سخت

`package.json` باید JSON معتبر باشد. گزینه‌ی ۳ درست است: کلیدها و رشته‌ها با کوتیشن دوتایی‌اند و `true` هم یک مقدار مجاز JSON است. گزینه‌ی ۱ بعد از آخرین عضو ویرگول اضافه (trailing comma) دارد، گزینه‌ی ۲ از کوتیشن تکی استفاده کرده و در گزینه‌ی ۴ کلید `name` کوتیشن ندارد. هر سه در JavaScript مجازند، ولی در JSON نه، و npm برایشان خطای `EJSONPARSE` می‌دهد.

۱۰۷۷ **گزینه‌ی ۱** سخت

JSON اصلاً comment ندارد و اگر `//` بنویسی، npm خطای `EJSONPARSE` می‌دهد، همراه با پیام «package.json must be actual JSON, not just JavaScript». پس گزینه‌ی ۱ نادرست است. گزینه‌های ۲ و ۳ قانون‌های اصلی JSONاند و گزینه‌ی ۴ هم رفتار واقعی npm است.

۱۰۷۸ **گزینه‌ی ۴** متوسط

npm به‌طور معمول قبل از خروجی اسکریپت دو خط سرآیند مثل `> shop-api@2.1.0 greet` و خود دستور را چاپ می‌کند. با `--silent` این خط‌ها (و پیام‌های خود npm) حذف می‌شوند، ولی اسکریپت کامل اجرا می‌شود و خروجی خودش را چاپ می‌کند. پس گزینه‌های ۱ و ۲ غلط‌اند و `--silent` ربطی به اجرای پس‌زمینه هم ندارد.

۱۰۷۹ **گزینه‌ی ۲** سخت

هر چیزی که بعد از `--` بیاید، به انتهای دستور اسکریپت اضافه می‌شود؛ پس npm در عمل `node server.js --port=4000` را اجرا می‌کند و `process.argv.slice(2)` برابر `[ '--port=4000' ]` است. خود `--` به اسکریپت نمی‌رسد (رد گزینه‌ی ۳). خروجی `[]` (گزینه‌ی ۱) وقتی است که `--` را ننویسی: آن وقت npm گزینه‌ی `--port` را مال خودش حساب می‌کند. `start` هم جزو اسم‌هایی است که بدون `run` اجرا می‌شوند (رد گزینه‌ی ۴).

۱۰۸۰ **گزینه‌ی ۳** متوسط

پکیج‌هایی که دستور خط فرمان دارند، فایل اجرایی‌شان را در `node_modules/.bin` می‌گذارند و npm هنگام اجرای اسکریپت این پوشه را اول PATH قرار می‌دهد؛ پس اسم `jest` پیدا می‌شود. نصب با `-D` فقط محلی است (رد گزینه‌ی ۱)، `jest` یک پکیج جداست، نه دستور داخلی npm (رد گزینه‌ی ۲) و Node.js هم `jest` را همراه ندارد (رد گزینه‌ی ۴).

۱۰۸۱ **گزینه‌ی ۱** متوسط

`npm start` اسکریپت `start` را اجرا می‌کند؛ اگر تعریف نشده باشد، فقط وقتی فایل `server.js` وجود داشته باشد `node server.js` را اجرا می‌کند. اینجا هیچ‌کدام نیست، پس `Missing script: "start"` می‌گیریم. فیلد `main` فایل ورودی پکیج برای `require` است و `npm start` از آن استفاده نمی‌کند (رد گزینه‌ی ۲). `index.js` هم خودکار اجرا نمی‌شود (رد گزینه‌ی ۳)، و `start` دستور شناخته‌شده‌ی npm است، پس `Unknown command` هم نمی‌گیریم (رد گزینه‌ی ۴).

۱۰۸۲ گزینه‌ی ۴ آسان

`&&` دستور دوم را فقط وقتی اجرا می‌کند که دستور اول با موفقیت (کد خروج ۰) تمام شود؛ اسلاید ۱۱ از همین «chaining commands» حرف می‌زند. `||` برعکس است و دستور دوم را فقط در صورت **شکست** دستور اول اجرا می‌کند. `;` در هر حال دستور دوم را اجرا می‌کند و `|` (pipe) خروجی دستور اول را به ورودی دستور دوم می‌فرستد.

۱۰۸۳ گزینه‌ی ۲ متوسط

وقتی دستورهای پروژه در `package.json` نوشته شده باشند، هر کسی (و هر سیستم خودکاری) با همان `npm test` یا `npm run build` کار را انجام می‌دهد؛ مثلاً لازم نیست بداند build با کدام ابزار و کدام گزینه‌ها انجام می‌شود. گزینه‌ی ۱ دقیقاً برعکس یکسان‌سازی است. اسکریپت‌ها روی همه‌ی سیستم‌عامل‌ها اجرا می‌شوند و ادغام کد هم کار git است، نه npm.

۱۰۸۴ گزینه‌ی ۳ سخت

npm هنگام اجرای اسکریپت این متغیرها را می‌سازد: `npm_package_name` (اسم پکیج: `shop-api`)، `npm_package_version` (نسخه: `2.1.0`) و `npm_lifecycle_event` (اسم اسکریپت در حال اجرا: `info`). پس خروجی `shop-api 2.1.0 info` است. اگر همین دستور را مستقیم و بیرون از npm اجرا می‌کردیم، سه `undefined` می‌گرفتیم (گزینه‌ی ۱). اسکریپت در حال اجرا `info` است، نه `start` (گزینه‌ی ۲)، و گزینه‌ی ۴ ترتیب متغیرها را برعکس کرده است.

۱۰۸۵ گزینه‌ی ۱ آسان

`devDependencies` برای ابزارهای زمان توسعه (تست، build، کامپایلر و...) است و با `npm install -D` پر می‌شود. `dependencies` وابستگی‌های زمان **اجرا** را نگه می‌دارد، `scripts` دستورهای میان‌بُر است و `main` فایل ورودی پکیج را مشخص می‌کند.

۱۰۸۶ گزینه‌ی ۴ متوسط

`type` تعیین می‌کند Node.js فایل‌های `.js` را چطور اجرا کند: `"module"` یعنی ES Module و `"commonjs"` یعنی CommonJS. خصوصی کردن پروژه کار فیلد `"private": true` است (رد گزینه‌ی ۱). `type` ربطی به قابل نصب بودن پروژه ندارد (رد گزینه‌ی ۲) و فایل‌های `.mjs` همیشه ES Module هستند (رد گزینه‌ی ۳).

۱۰۸۷ گزینه‌ی ۲ متوسط

`npm init -y` اسکریپت `"test": "echo \"Error: no test specified\" && exit 1"` را می‌سازد: پیام را چاپ می‌کند و با `exit 1` شکست را اعلام می‌کند. این اسکریپت هیچ تستی پیدا یا اجرا نمی‌کند (رد گزینه‌ی ۱). اسکریپت `test` وجود دارد (رد گزینه‌ی ۳) و `test` از اسم‌هایی است که بدون `run` هم اجرا می‌شوند (رد گزینه‌ی ۴).

۳-۶ Bundling برای مرورگر

۱۰۸۸ گزینه‌ی ۲ آسان

اسلاید ۱۲: «Combines multiple JS files into a single output for browsers». تبدیل TypeScript به JavaScript کار transpiler/کامپایلر (مثل `tsc` یا Babel) است، اجرای JS روی سرور کار Node.js است و دانلود پکیج‌ها کار `npm install`.

۱۰۸۹ گزینه‌ی ۴ آسان

اسلاید ۱۲: «Popular tools: Webpack, Rollup, and Parcel». Express یک فریم‌ورک وب برای ساختن سرور و API در Node.js است و هیچ ربطی به bundling ندارد.

۱۰۹۰ گزینه‌ی ۱ متوسط

اسلاید ۱۲: «Reduces HTTP requests and improves load performance». هر فایل یک درخواست HTTP با سربار و تأخیر خودش است و کم شدن تعداد درخواست‌ها بارگذاری را سریع‌تر می‌کند (minification هم با کوچک کردن حجم کمک می‌کند). bundle کد را چندنخی نمی‌کند، خودش هم باید از سرور دانلود شود و مرورگر مثل هر کد دیگری آن را parse می‌کند.

۱۰۹۱ **گزینه‌ی ۳** متوسط

minification متن کد را کوچک می‌کند: فاصله‌ها، خط‌های خالی و توضیح‌ها حذف و اسم‌های محلی کوتاه می‌شوند، ولی خروجی برنامه همان می‌ماند. گزینه‌ی ۱ تعریف **tree-shaking** است (دام اصلی این تست). فشرده‌سازی تصویر و فونت موضوع دیگری است و تبدیل JS به TS هم معنایی ندارد.

۱۰۹۲ **گزینه‌ی ۲** متوسط

tree-shaking در زمان ساختن bundle، کدی را که export شده ولی هیچ‌جا استفاده نشده حذف می‌کند تا bundle کوچک‌تر شود. به `node_modules` دست نمی‌زند (رد گزینه‌ی ۱) و ربطی به ساختار پوشه‌ها ندارد (رد گزینه‌ی ۳). کوتاه کردن اسم‌ها هم کار minification است (رد گزینه‌ی ۴).

۱۰۹۳ **گزینه‌ی ۴** سخت

`subtract` هیچ‌جا import نشده و `multiply` با اینکه import شده، هیچ‌وقت **صدا زده نشده** است؛ پس هر دو کد مرده‌اند و tree-shaking حذفشان می‌کند. خروجی esbuild فقط تابع `add` و خط `console.log(add(2, 3))` را دارد. گزینه‌ی ۲ فکر کرده «import شدن» کافی است، در حالی که ملاک «استفاده شدن» است. `add` واقعاً استفاده شده، پس حذف نمی‌شود (رد گزینه‌ی ۳).

۱۰۹۴ **گزینه‌ی ۱** سخت

`import` / `export` باید در بالاترین سطح فایل و با اسم‌های ثابت بیایند، پس با خواندن کد (بدون اجرا) معلوم است چه چیزی استفاده شده. `require` می‌تواند مسیرش را هنگام اجرا بسازد (مثلاً `require(cond ? "./a.js" : "./b.js")`) و `exports[name] = …` هم اسم export را پویا می‌کند؛ پس bundler مطمئن نیست و معمولاً همه را نگه می‌دارد. اندازه‌ی فایل ربطی ندارد (رد گزینه‌ی ۲)، CommonJS با `module.exports` export دارد (رد گزینه‌ی ۳) و مرورگر کد CommonJS را «نادیده» نمی‌گیرد؛ فقط `require` را نمی‌شناسد (رد گزینه‌ی ۴).

۱۰۹۵ **گزینه‌ی ۳** متوسط

minification فقط شکل کد را عوض می‌کند و رفتارش باید همان بماند. گزینه‌ی ۳ اسم تابع و پارامترها را **یکجا** کوتاه کرده و `30` چاپ می‌کند. گزینه‌ی ۱ عملگر را عوض کرده (`13` چاپ می‌کند). گزینه‌ی ۲ اسم تابع را فقط در تعریفش عوض کرده و هنگام صدا زدن خطای `ReferenceError: calculateTotal is not defined` می‌دهد. گزینه‌ی ۴ کلمه‌ی `return` را حذف کرده و `undefined` چاپ می‌کند.

۱۰۹۶ **گزینه‌ی ۲** متوسط

Babel (و SWC) کد جدید را به JavaScript قدیمی‌تر یا ساده‌تر **ترجمه** می‌کنند؛ یعنی transpiler هستند. Webpack (و Rollup و Parcel) ماژول‌ها را در یک خروجی **یکی** می‌کنند؛ یعنی bundler هستند. گزینه‌ی ۱ این دو را جابه‌جا کرده است. هیچ‌کدام موتور اجرا (مثل V8)، مدیر بسته یا فریم‌ورک سرور نیستند.

۱۰۹۷ **گزینه‌ی ۱** متوسط

resolution یعنی «پیدا کردن»: bundler برای هر `import "./math.js"` یا `import _ from "lodash"` معلوم می‌کند دقیقاً کدام فایل منظور است (برای اسم پکیج، داخل `node_modules`). bundler پکیجی دانلود نمی‌کند (کار `npm install`)، نوع‌ها را بررسی نمی‌کند (کار `tsc`) و با git هم کاری ندارد.

۱۰۹۸ **گزینه‌ی ۳** سخت

bundler فقط از کدی استفاده می‌کند که از قبل در پروژه و `node_modules` هست؛ نصب پکیج‌ها هنوز کار `npm install` است. پس گزینه‌ی ۳ نادرست است. گزینه‌ی ۱ درست است (مثل `<script src="dist/bundle.js"></script>`)، گزینه‌ی ۲ همان «Often includes optimizations like minification and tree-shaking» اسلاید ۱۲ است و گزینه‌ی ۴ کار اصلی source map است.

۱۰۹۹ **گزینه‌ی ۴** آسان

source map «نقشه‌ی» بین کد خروجی و کد منبع است؛ DevTools با آن خطاها و کد اصلی و خوانا را نشان می‌دهد. فهرست پکیج‌ها در `package.json` و `package-lock.json` است، نقشه‌ی سایت برای موتور جست‌وجو (sitemap) چیز دیگری است و source map ربطی به ترتیب تگ‌ها ندارد.

۱۱۰۰ **گزینه‌ی ۲** متوسط

bundle یک فایل JavaScript معمولی است و با یک تگ `<script>` بارگذاری می‌شود؛ همین یک خط جای همه‌ی تگ‌های قبلی را می‌گیرد. `<link rel="stylesheet">` برای فایل CSS است. نوشتن تگ جدا برای هر فایل `src` یعنی اصلاً از bundle استفاده نکرده‌ایم. `require` هم در HTML و در مرورگر وجود ندارد.

۱۱۰۱ **گزینه‌ی ۱** سخت

`"lodash"` یک اسم خالی (bare specifier) است، نه یک مسیر. Node.js یاد گرفته چنین اسمی را در `node_modules` جست‌وجو کند، ولی مرورگر نه؛ برای همین Chrome خطای `Failed to resolve module specifier "lodash"` می‌دهد. مرورگر نه `node_modules` را می‌گردد (رد گزینه‌ی ۲) و نه چیزی از npm دانلود می‌کند (رد گزینه‌ی ۳). `<script type="module">` از `import` با مسیرهای نسبی مثل `./math.js` پشتیبانی می‌کند (رد گزینه‌ی ۴). یکی از کارهای اصلی bundler حل همین مشکل است.

۱۱۰۲ **گزینه‌ی ۳** آسان

متن اسلاید: «Popular tools: Webpack, Rollup, and Parcel». Babel یک transpiler است، Jest فریم‌ورک تست و Lodash کتابخانه‌ی توابع کمکی؛ هیچ‌کدام bundler نیستند.

۷-۳ پکیج‌های محبوب

۱۱۰۳ **گزینه‌ی ۱** آسان

اسلاید ۱۳: «Express: minimalist web framework for building APIs and servers». گزینه‌ی ۲ توصیف React، گزینه‌ی ۳ توصیف Jest و گزینه‌ی ۴ توصیف Chalk است.

۱۱۰۴ **گزینه‌ی ۳** آسان

اسلاید ۱۴: «Chalk. Jest: testing framework for JavaScript and Node.js» برای رنگی کردن خروجی ترمینال است، Axios کلاینت HTTP است و Lodash توابع کمکی برای آرایه‌ها و شیءها دارد.

۱۱۰۵ **گزینه‌ی ۲** آسان

اسلاید ۱۴: «Chalk: style and colorize terminal output»؛ مثلاً `chalk.green("Success!")` . Three.js برای گرافیک سه‌بعدی در مرورگر است، React برای رابط کاربری صفحه‌ی وب و Electron برای برنامه‌ی دسکتاپ؛ هیچ‌کدام برای رنگ‌آمیزی متن ترمینال نیستند.

۱۱۰۶ **گزینه‌ی ۴** متوسط

اسلاید ۱۳: «Socket.io: real-time bidirectional communication for web apps». «بلادرنگ» یعنی پیام‌ها لحظه‌ای می‌رسند و «دوطرفه» یعنی سرور هم می‌تواند بدون درخواست کلاینت پیام بفرستد. Axios هم با شبکه کار می‌کند، ولی مدلش درخواست و پاسخ است و همیشه کلاینت شروع‌کننده است. Lodash و Day.js اصلاً کار شبکه‌ای نمی‌کنند.

۱۱۰۷ **گزینه‌ی ۱** متوسط

اسلاید ۱۴: «Electron: build cross-platform desktop apps with web technologies». Electron موتور Chromium و Node.js را ترکیب می‌کند (VS Code و Slack با آن ساخته شده‌اند). React فقط رابط کاربری صفحه‌ی وب را می‌سازد و به‌تنهایی برنامه‌ی دسکتاپ تولید نمی‌کند. Three.js برای سه‌بعدی در مرورگر و Express برای سرور است.

۱۱۰۸ **گزینه‌ی ۳** متوسط

Axios کلاینت HTTP است: از طرف برنامه به APIها درخواست می‌فرستد و چون هر درخواست یک Promise برمی‌گرداند، با `async/await` هم کار می‌کند. Express **سرور** را می‌سازد (طرف مقابل درخواست)، Socket.io برای ارتباط بلادرنگ دوطرفه است و Day.js برای کار با تاریخ.

۱۱۰۹ **گزینه‌ی ۲** متوسط

Lodash مجموعه‌ی **توابع کمکی** برای آرایه‌ها، شیءها و... است («utility functions for arrays, objects, and more»). ارتباط دوطرفه‌ی بلادرنگ توصیف Socket.io است. سه جفت دیگر دقیقاً با متن اسلایدهای ۱۳ و ۱۴ می‌خوانند.

۱۱۱۰ **گزینه‌ی ۴** متوسط

Express برای ساختن API و سرور روی Node.js است. React و Three.js در مرورگر (frontend) اجرا می‌شوند. Chalk هرچند در Node.js اجرا می‌شود، فقط خروجی ترمینال را رنگی می‌کند و فریم‌ورک سرور نیست.

۱۱۱۱ **گزینه‌ی ۱** سخت

Jest فقط هنگام **توسعه** (اجرای تست‌ها) لازم است و سرور در حال اجرا به آن نیازی ندارد؛ پس جایش `devDependencies` است. Express، Axios و Socket.io همگی هنگام **اجرای** سرور استفاده می‌شوند و باید در `dependencies` باشند؛ اگر با `-D` نصبشان کنی، با `npm install --omit=dev` روی سرور اصلی نصب نمی‌شوند و برنامه خراب می‌شود.

۱۱۱۲ **گزینه‌ی ۳** سخت

«Lodash «utility functions for arrays, objects, and more است و طبق قرارداد با `_` وارد می‌شود (اسمش هم از «low dash» یعنی همین زیرخط می‌آید). `_.chunk` آرایه را تکه‌تکه می‌کند و `_.uniq` تکراری‌ها را حذف می‌کند. Axios برای درخواست HTTP است، React برای رابط کاربری و Socket.io برای ارتباط بلادرنگ؛ هیچ‌کدام چنین توابعی ندارند.

۱۱۱۳ **گزینه‌ی ۲** متوسط

اسلاید ۱۳: «React: library for building interactive UIs with components». ساختن API کار Express است، React پایگاه داده نیست و bundle کردن هم کار ابزارهایی مثل Rollup، Webpack و Parcel است.

۱۱۱۴ **گزینه‌ی ۴** متوسط

اسلاید ۱۴: «Moment.js / Day.js: date and time manipulation utilities»؛ مثلاً `dayjs("2026-11-17").add(7, "day").format("YYYY-MM-DD")` نتیجه‌ی `"2026-11-24"` را می‌دهد. Chalk فقط متن ترمینال را رنگی می‌کند، Jest برای تست است و Socket.io برای ارتباط بلادرنگ.

۱۱۱۵ **گزینه‌ی ۱** سخت

کلمه‌ی کلیدی «Socket.io «bidirectional یعنی **دوطرفه** است: هم کلاینت و هم سرور هر وقت بخواهند می‌توانند پیام بفرستند (مثلاً سرور با `io.emit` به همه پیام می‌دهد). پس گزینه‌ی ۱ نادرست است. گزینه‌ی ۲ توضیح درست Electron است، گزینه‌ی ۳ همان «promise-based» در توصیف Axios است و گزینه‌ی ۴ توصیف اسلاید ۱۳ برای Lodash است.

۱۱۱۶ **گزینه‌ی ۳** سخت

API روی سرور با Express، تست با Jest و رنگی کردن خروجی ترمینال با Chalk. در گزینه‌ی ۱، React برای رابط کاربری مرورگر است نه API، و Three.js برای سه‌بعدی است. در گزینه‌ی ۲، Axios ابزار تست نیست و Electron هم برای برنامه‌ی دسکتاپ است. در گزینه‌ی ۴ هیچ‌کدام با نیازها جور نیست.

۱۱۱۷ **گزینه‌ی ۴** آسان

اسلاید ۱۴: «Three.js: 3D graphics and animations in the browser». Electron برای برنامه‌ی دسکتاپ است، Chalk برای رنگ خروجی ترمینال و Express برای سرور؛ هیچ‌کدام گرافیک سه‌بعدی نمی‌سازند.

۱۱۱۸ **گزینه‌ی ۲** متوسط

`test(...)` و `expect(...).toBe(...)` API فریم‌ورک تست Jest هستند. Jest را با `npm install -D jest` نصب می‌کنیم، اسکریپت `"test": "jest"` را در `package.json` می‌نویسیم و با `npm test` (که بدون `run` هم کار می‌کند) اجرا می‌کنیم. Chalk، Express و React چنین توابعی ندارند. گزینه‌ی ۴ دستور اجرا را درست گفته ولی پکیج را اشتباه.

۳-۸ مشکلات تایپ در JavaScript و معرفی TypeScript

۱۱۱۹ **گزینه‌ی ۳** آسان

طبق اسلاید ۱۷، TypeScript را Microsoft در سال ۲۰۱۲ معرفی کرد. گزینه‌ی ۲ سال درست را با شرکت نادرست ترکیب کرده است. Google سازنده‌ی موتور V8 است و Mozilla سازنده‌ی Firefox؛ هیچ‌کدام TypeScript را نساخته‌اند.

۱۱۲۰ **گزینه‌ی ۲** آسان

اسلاید ۱۷ می‌گوید: Anders Hejlsberg, lead architect of C# and Delphi. زبان‌های گزینه‌های دیگر را افراد دیگری طراحی کرده‌اند و در اسلاید هم نیامده‌اند.

۱۱۲۱ **گزینه‌ی ۴** آسان

در نوع‌دهی پویا نوع مال **مقدار** است، نه متغیر؛ پس `let x = 5; x = "hi";` در JavaScript کاملاً مجاز است و نوع `x` در زمان اجرا عوض می‌شود. گزینه‌ی ۱ تعریف static typing است. گزینه‌ی ۲ غلط است چون مقدارها نوع دارند (`typeof` همین را نشان می‌دهد). گزینه‌ی ۳ هم غلط است چون با یک انتساب ساده نوع متغیر عوض می‌شود و به تابع تبدیل نیازی نیست.

۱۱۲۲ **گزینه‌ی ۱** آسان

اگر یک طرف `+` رشته باشد، طرف دیگر هم رشته می‌شود و دو رشته به هم می‌چسبند: `"5" + 2` یعنی `"52"` (مثال اسلاید ۱۶). عملگر `-` فقط معنای ریاضی دارد، پس `"5"` به عدد ۵ تبدیل می‌شود و حاصل **عدد** `3` است، نه رشته‌ی `"3"` (گزینه‌ی ۳). گزینه‌های ۲ و ۴ فرض کرده‌اند `+` جمع ریاضی انجام می‌دهد.

۱۱۲۳ **گزینه‌ی ۱** آسان

superset یعنی TypeScript همه‌ی JavaScript را در بر دارد و فقط نوع‌ها را اضافه می‌کند (TypeScript = JavaScript + types). گزینه‌ی ۲ برعکس این است. گزینه‌ی ۳ غلط است چون مرورگر TypeScript را نمی‌فهمد و کد باید اول با tsc به JavaScript تبدیل شود. گزینه‌ی ۴ تعریف subset (زیرمجموعه) است، نه superset.

۱۱۲۴ **گزینه‌ی ۳** آسان

`tsc --init` فایل تنظیمات کامپایلر یعنی `tsconfig.json` را می‌سازد و در آن `"strict": true` را هم قرار می‌دهد. نصب TypeScript با `npm install -D typescript` انجام می‌شود (گزینه‌ی ۱). tsc اصلاً برنامه را اجرا نمی‌کند، فقط بررسی و ترجمه می‌کند (گزینه‌ی ۲). `package.json` را `npm init` می‌سازد (گزینه‌ی ۴).

۱۱۲۵ **گزینه‌ی ۲** متوسط

JavaScript نوع‌دهی پویا دارد: `data` اول عدد است و بعد از انتساب `"ten"` رشته می‌شود، بدون هیچ خطایی. سپس `"ten" + 5` چون یک طرف رشته است، به رشته‌ی `"ten5"` تبدیل می‌شود. گزینه‌ی ۳ رفتار TypeScript را با JavaScript قاطی کرده است (تازه TypeScript هم خطا را در زمان کامپایل می‌دهد، نه با TypeError). گزینه‌ی ۴ فکر کرده `+` جمع عددی انجام می‌دهد.

۱۱۲۶ **گزینه‌ی ۲** متوسط

خط ۲ یک غلط املایی دارد (`nmae`)، اما JavaScript فقط `undefined` چاپ می‌کند و خطایی نمی‌دهد؛ این همان «باگ پنهان» اسلاید ۱۶ است. در خط ۳، `toUppercase` وجود ندارد (درستش `toUpperCase` است)، پس `undefined` صدا زده می‌شود و در زمان اجرا `TypeError` رخ می‌دهد. خط ۴ دیگر اجرا نمی‌شود. در TypeScript هر دو غلط (خطوط ۲ و ۳) قبل از اجرا گزارش می‌شدند.

۱۱۲۷ **گزینه‌ی ۳** سخت

`Math.random()` همیشه عددی بین ۰ و ۱ است، پس شرط `> 2` هیچ‌وقت درست نمی‌شود و `shout` اصلاً اجرا نمی‌شود. چون JavaScript قبل از اجرا کد را بررسی نمی‌کند، غلط املایی `toUppercase` هیچ اثری نمی‌گذارد و فقط `start` و `end` چاپ می‌شوند. این یعنی باگ پنهان می‌ماند تا روزی که آن شاخه اجرا شود (گزینه‌ی ۱ فقط در آن حالت درست بود). اشتباه در نام متد خطای نحوی (SyntaxError) نیست (گزینه‌ی ۲) و گزینه‌ی ۴ نیز هم غلط املایی و هم شرط را نادیده گرفته است.

۱۱۲۸ **گزینه‌ی ۱** متوسط

tsc کل کد را **بدون اجرا** بررسی می‌کند؛ پس مهم نیست `shout` هیچ‌وقت صدا زده نشود. چون نوع `msg` رشته است، tsc می‌داند رشته متدی به نام `toUppercase` ندارد و در خط ۲ خطا می‌دهد (پیامش حتی پیشنهاد می‌کند: `Did you mean 'toUpperCase'?`). خط ۵ یک مقایسه‌ی عددی معمولی است و خط ۶ هم یک رشته را به تابعی می‌دهد که رشته می‌خواهد. گزینه‌ی ۴ طرز فکر JavaScript است که خطا را فقط در زمان اجرا می‌بیند.

۱۱۲۹ **گزینه‌ی ۴** سخت

چسباندن رشته با `+` کار مجاز و رایجی است، پس tsc عبارت `"5" + 2` را قبول می‌کند و نوع `a` را `string` می‌داند؛ یعنی TypeScript جلوی همین مثال اسلاید ۱۶ را **نمی‌گیرد**. اما تفریق رشته (`"5" - 2`) خطای TS2362 دارد، جمع دو boolean خطای TS2365 و استفاده از `null` در محاسبه خطای TS18050؛ هر سه در JavaScript بی‌صدا یک عدد می‌ساختند.

۱۱۳۰ **گزینه‌ی ۴** متوسط

tsc از مقدار اولیه نوع `flag` را `boolean` تشخیص می‌دهد و نسبت دادن رشته‌ی `"yes"` به آن خطای TS2322 دارد. این نمونه‌ی همان نکته است که هر کد JavaScript نحوِ معتبر TypeScript دارد ولی ممکن است خطای نوع داشته باشد. در گزینه‌ی ۱ حاصل `n + 1` عدد است. در گزینه‌ی ۲ حاصل `s + 1` رشته است و در متغیر رشته‌ای قرار می‌گیرد. در گزینه‌ی ۳ هم `const` فقط انتساب دوباره را منع می‌کند و `push` کردن عدد به آرایه‌ی عددی مجاز است.

۱۱۳۱ **گزینه‌ی ۱** متوسط

«تدریجی» دقیقاً یعنی لازم نیست همه‌چیز یک‌جا عوض شود؛ می‌شود فایل‌ها را کم‌کم از `.js` به `.ts` برد و فایل‌های JavaScript و TypeScript مدتی کنار هم باشند. گزینه‌ی ۲ معنای «تدریجی» است، گزینه‌ی ۳ معنای «اختیاری» (به کمک type inference) و گزینه‌ی ۴ همان بولت آخر اسلاید ۱۷.

۱۱۳۲ **گزینه‌ی ۳** متوسط

اسلاید ۱۷ برعکس گزینه‌ی ۳ را می‌گوید: TypeScript برای **پروژه‌های بزرگ** (large-scale projects) ساخته شد و در نرم‌افزارهای سازمانی و وب‌اپلیکیشن‌های پیچیده محبوب است؛ چون در کد بزرگ، پیدا کردن خطاهای نوع پیش از اجرا بیشترین ارزش را دارد. گزینه‌های ۱، ۲ و ۴ عیناً بولت‌های همین اسلایدند.

۱۱۳۳ **گزینه‌ی ۲** متوسط

نوع‌ها در کامپایل پاک می‌شوند (type erasure) و مرورگر همان JavaScript معمولی را اجرا می‌کند؛ پس TypeScript سرعت اجرا را بیشتر نمی‌کند. سه گزینه‌ی دیگر دقیقاً کارهایی‌اند که ویرایشگر به لطف شناختن نوع‌ها انجام می‌دهد: autocomplete، نشان دادن خطا قبل از اجرا و rename امن.

۱۱۳۴ **گزینه‌ی ۴** متوسط

تک‌نخی بودن یک محدودیت اجرایی JavaScript است که در اسلاید ۳ (Limitations of JavaScript) آمده و ربطی به نوع داده ندارد. سه گزینه‌ی دیگر از بولت‌های اسلاید ۱۶اند: نوع‌دهی پویا، type coercion و نبودن compile-time checks. دو مشکل دیگر آن اسلاید هم باگ‌های پنهان در کدهای بزرگ و رفتار ناسازگار در موتورها و نسخه‌های مختلف است.

۱۱۳۵ **گزینه‌ی ۴** متوسط

در گزینه‌ی ۴ **یک کد یکسان** روی موتورهای قدیمی و جدید نتیجه‌ی متفاوت می‌دهد؛ این دقیقاً همان «رفتار ناسازگار در موتورها و نسخه‌ها» است که باگ‌های ظریف و وابسته به سیستم می‌سازد. سه گزینه‌ی دیگر در همه‌ی موتورها یکسان رفتار می‌کنند و مثال بولت‌های دیگر اسلاید ۱۶اند: گزینه‌ی ۱ type coercion، گزینه‌ی ۲ نوع‌دهی پویا و گزینه‌ی ۳ باگ پنهان نوع.

۱۱۳۶ **گزینه‌ی ۱** سخت

tsc به‌طور پیش‌فرض خطا را گزارش می‌کند ولی خروجی JavaScript را **باز هم** می‌سازد؛ چون نوع‌ها فقط پاک می‌شوند و کد باقی‌مانده قابل اجراست. گزینه‌ی ۲ فقط وقتی درست است که `noEmitOnError` روشن باشد. گزینه‌ی ۳ غلط است چون کار اصلی tsc گزارش خطا قبل از اجراست. tsc هم هیچ‌وقت کد را خودش اصلاح نمی‌کند (گزینه‌ی ۴).

۱۱۳۷ **گزینه‌ی ۲** متوسط

در کامپایل، همه‌ی type annotationها پاک می‌شوند (type erasure) و خروجی JavaScript ساده است؛ اگر داده‌ی نادرست از جایی که tsc بررسی نکرده (مثلاً یک مقدار `any` یا داده‌ی یک سرور) برسد، در زمان اجرا چیزی جلویش را نمی‌گیرد. گزینه‌ی ۱ غلط است چون نوع‌ها در خروجی نمی‌مانند. گزینه‌ی ۳ غلط است چون موتور همان JavaScript معمولی را اجرا می‌کند. گزینه‌ی ۴ غلط است چون مرورگر TypeScript را نمی‌فهمد و کد باید اول به JavaScript تبدیل شود.

۱۱۳۸ **گزینه‌ی ۳** سخت

چون `input` از نوع `any` است، tsc اجازه می‌دهد آن را به پارامتر `number` بدهیم. در خروجی JavaScript هم نوع‌ها پاک شده‌اند و هیچ بررسی‌ای در زمان اجرا نیست؛ پس رشته‌ی `"21"` واقعاً وارد تابع می‌شود و `"21" * 2` با تبدیل نوع خودکار `42` می‌شود. TypeScript کد بررسی نوع به خروجی اضافه نمی‌کند، پس TypeError رخ نمی‌دهد (گزینه‌ی ۲). عملگر `*` رشته نمی‌چسباند (گزینه‌ی ۱) و `"21"` یک عدد معتبر است، پس حاصل NaN نیست (گزینه‌ی ۴).

۱۱۳۹ **گزینه‌ی ۳** سخت

در خط ۳ اولویت `*` بیشتر است: اول `qty * price` برابر ۶ می‌شود و بعد `"Total: " + 6` یک رشته می‌سازد. چسباندن رشته در TypeScript مجاز است، پس خط ۳ خطا ندارد و نوع `msg` برابر `string` است. در خط ۴ تقسیم یک رشته بر عدد خطای TS2362 دارد (در JavaScript همین خط بی‌صدا `NaN` می‌ساخت). خط ۵ فقط دو مقدار را چاپ می‌کند. گزینه‌ی «بدون خطا» طرز فکر JavaScript است.

۳-۹ متغیرها و انواع داده در TypeScript

۱۱۴۰ **گزینه‌ی ۲** آسان

دونقطه و نام نوع بعد از نام متغیر، **type annotation** است؛ یعنی خودمان نوع را صریح نوشته‌ایم. type inference برعکس آن است: وقتی نوع را نمی‌نویسیم و TypeScript خودش تشخیص می‌دهد. union type با علامت `|` ساخته می‌شود و type coercion تبدیل نوع خودکار JavaScript در زمان اجراست.

۱۱۴۱ **گزینه‌ی ۱** آسان

`age` با annotation از نوع `number` است، پس رشته‌ی `"26"` (حتی با اینکه شبیه عدد است) قابل نسبت دادن نیست و خطای TS2322 می‌گیرد؛ TypeScript برخلاف JavaScript رشته را بی‌صدا به عدد تبدیل نمی‌کند. گزینه‌ی ۳ مجاز است چون `age + 1` عدد است. نوع `number` عدد اعشاری را هم شامل می‌شود، پس گزینه‌ی ۴ هم بی‌خطاست.

۱۱۴۲ **گزینه‌ی ۳** آسان

`const` یک immutable binding می‌سازد و انتساب دوباره به آن ممنوع است؛ پس خط ۴ خطای TS2588 دارد: `Cannot assign to 'PI' because it is a constant`. خط ۲ مجاز است چون `count` با `let` تعریف شده و مقدار جدید هم عدد است. خط ۳ یک تعریف معمولی است.

۱۱۴۳ **گزینه‌ی ۱** متوسط

`const` فقط جلوی **انتساب دوباره** به خودِ متغیر را می‌گیرد؛ پس تغییر محتوای آرایه با `push` (خط ۲) یا تغییر یک خانه (خط ۳) کاملاً مجاز است. فقط خط ۴ که می‌خواهد یک آرایه‌ی جدید به `list` نسبت دهد خطای TS2588 دارد. کسی که فکر کند `const` آرایه را «یخ‌زده» می‌کند، اشتباهاً خط ۲ را انتخاب می‌کند.

۱۱۴۴ **گزینه‌ی ۳** آسان

union type `number | string` یعنی `id` می‌تواند عدد **یا** رشته باشد؛ پس گزینه‌های ۱، ۲ و ۴ مجازند (عدد منفی هم `number` است). اما `true` از نوع `boolean` است که جزو این union نیست و خطای TS2322 می‌گیرد: `Type 'boolean' is not assignable to type 'string | number'`.

۱۱۴۵ **گزینه‌ی ۴** متوسط

هیچ annotationی نوشته نشده، ولی TypeScript از مقدار اولیه نوع `city` را `string` و نوع `year` را `number` استنتاج می‌کند (type inference). خط ۲ رشته را در رشته می‌گذارد و مجاز است. خط ۴ می‌خواهد یک عدد را در متغیر رشته‌ای بگذارد و خطای TS2322 می‌گیرد. گزینه‌ی «بدون خطا» طرز فکر JavaScript است که نوع متغیر را آزاد می‌داند.

۱۱۴۶ **گزینه‌ی ۳** سخت

متغیری که با `const` و یک مقدار ساده تعریف شود، نوع **literal** می‌گیرد؛ پس نوع `y` دقیقاً `5` است و در متغیری از نوع `5` جا می‌شود. اما `let x = 5` نوع عمومی `number` می‌گیرد و هر `number`ی لزوماً ۵ نیست، پس گزینه‌ی ۱ خطا دارد. گزینه‌ی ۲ به همین دلیل خطا دارد (`"hi"` به `string` نسبت داده نمی‌شود). گزینه‌ی ۴ هم انتساب دوباره به `const` است.

۱۱۴۷ **گزینه‌ی ۴** متوسط

از روی مقدار اولیه، TypeScript نوع `x` را «آرایه‌ای از عددها» یعنی `number[]` تشخیص می‌دهد؛ پس بعداً می‌شود هر عددی را `push` کرد ولی رشته را نه. نوع `[1, 2, 3]` یک tuple با مقدارهای ثابت است و TypeScript برای آرایه‌ی معمولی چنین نوعی نمی‌سازد. `any[]` یعنی بدون بررسی، که اینجا لازم نیست چون همه‌ی خانه‌ها عددند. `number` هم نوع یک عدد تنهاست، نه آرایه.

۱۱۴۸ **گزینه‌ی ۲** آسان

`number[]` و `Array<number>` دو شیوه‌ی نوشتن یک نوع‌اند: آرایه‌ای با هر تعداد عدد. `[number]` یک tuple با دقیقاً **یک** عدد است. `number | []` یعنی «یک عدد یا یک آرایه‌ی خالی». `Array[number]` اصلاً نوشتن درستی نیست و tsc برایش خطا می‌دهد (برای Array باید نوع را داخل `<>` نوشت).

۱۱۴۹ **گزینه‌ی ۱** متوسط

`scores` آرایه‌ای از عددهاست و `push` فقط عدد می‌پذیرد؛ رشته‌ی `"19"` خطای TS2345 می‌گیرد: `Argument of type 'string' is not assignable to parameter of type 'number'`. `19.5` یک `number` است (گزینه‌ی ۲). `Array<string>` همان `string[]` است و `"node"` را می‌پذیرد (گزینه‌ی ۳). نسبت دادن آرایه‌ی خالی به `tags` هم مجاز است، چون با `let` تعریف شده و آرایه‌ی خالی با `string[]` سازگار است (گزینه‌ی ۴).

۱۱۵۰ **گزینه‌ی ۲** سخت

روی مقداری از نوع `number | string` فقط کارهایی مجاز است که برای **هر دو** نوع درست باشد. `toString()` هم روی عدد هست هم روی رشته، پس خط ۲ مجاز است. `toUpperCase()` روی عدد وجود ندارد، پس خط ۳ خطای TS2339 دارد. در خط ۵، شرط `typeof id === "string"` نوع را به `string` باریک کرده (narrowing) و همان متد بدون خطاست.

۱۱۵۱ **گزینه‌ی ۳** متوسط

برای `5` شرط `typeof value === "number"` درست است و `toFixed(1)` عدد را با یک رقم اعشار به رشته‌ی `"5.0"` تبدیل می‌کند. برای `"ok"` شرط نادرست است و `toUpperCase()` حاصل `"OK"` را می‌دهد. گزینه‌ی ۱ اثر `toFixed(1)` را نادیده گرفته و گزینه‌های ۲ و ۴ اثر `toUpperCase()` را.

۱۱۵۲ **گزینه‌ی ۲** متوسط

نوع `role` یک union از دو **literal type** است، یعنی فقط دو مقدار `"admin"` و `"user"` مجازند؛ `"guest"` با اینکه رشته است، جزو این فهرست نیست و خطای TS2322 می‌گیرد. گزینه‌های ۱ و ۴ مقدارهای مجازند. گزینه‌ی ۳ هم مجاز است، چون هر کدام از این دو literal یک `string` هم هست و می‌شود آن را در متغیر `string` گذاشت (برعکسش ممکن نیست).

۱۱۵۳ **گزینه‌ی ۳** متوسط

هر دو هر مقداری را قبول می‌کنند، اما رفتارشان هنگام **استفاده** فرق دارد: `any` بررسی را خاموش می‌کند و `unknown` تا وقتی با شرطی مثل `typeof` نوع را باریک نکنی، اجازه‌ی صدا زدن متد یا نسبت دادن به متغیر نوع‌دار را نمی‌دهد. پس گزینه‌ی ۱ فقط درباره‌ی `any` درست است. `unknown` هر مقداری می‌گیرد، نه فقط `undefined` (گزینه‌ی ۲). گزینه‌ی ۴ برعکس است: `unknown` امن‌تر است.

۱۱۵۴ **گزینه‌ی ۳** متوسط

نسبت دادن هر مقداری به `unknown` مجاز است (خط ۲)، اما استفاده از آن پیش از narrowing ممنوع است؛ پس خط ۴ خطای TS18046 دارد: `'u' is of type 'unknown'`. خط ۳ بدون خطاست چون `any` بررسی را کاملاً خاموش می‌کند. اگر خط ۴ داخل `if (typeof u === "string") { … }` بود، خطا برطرف می‌شد.

۱۱۵۵ **گزینه‌ی ۱** سخت

چون `data` از نوع `any` است، tsc هیچ بررسی‌ای انجام نمی‌دهد و کد کامپایل می‌شود. اما در زمان اجرا مقدار `data` عدد ۴۲ است که متد `toUpperCase` ندارد؛ پس پس از چاپ `before`، خط ۳ با `TypeError` متوقف می‌شود و `after` هرگز چاپ نمی‌شود. گزینه‌ی ۲ فکر کرده چون کامپایل موفق بوده، اجرا هم بی‌خطاست؛ این دقیقاً خطر `any` است. خط ۳ چیزی چاپ نمی‌کند (گزینه‌ی ۳) و خطا هم بعد از اجرای خط ۲ رخ می‌دهد، نه قبل از آن (گزینه‌ی ۴).

۱۱۵۶ **گزینه‌ی ۱** متوسط

در حالت strict، `null` و `undefined` جزو نوع‌های `number`، `string` و `boolean` نیستند؛ پس گزینه‌های ۲ تا ۴ خطای TS2322 دارند. اگر متغیری ممکن است خالی بماند، باید آن را صریحاً در union بیاوری: `number | null` (گزینه‌ی ۱).

۱۱۵۷ **گزینه‌ی ۲** متوسط

`[string, number]` یک tuple با **دقیقاً دو خانه** است: خانه‌ی ۰ رشته و خانه‌ی ۱ عدد. خط‌های ۲ و ۳ در هر خانه مقداری با نوع درست می‌گذارند. اما tuple خانه‌ی شماره‌ی ۲ ندارد، پس خط ۴ خطای TS2493 می‌گیرد: `Tuple type '[string, number]' of length '2' has no element at index '2'`. در JavaScript همین کار فقط `undefined` چاپ می‌کرد.

۱۱۵۸ **گزینه‌ی ۴** آسان

در tuple **ترتیب** نوع‌ها مهم است: خانه‌ی اول باید رشته و خانه‌ی دوم عدد باشد. گزینه‌ی ۴ ترتیب را برعکس کرده و خطای TS2322 می‌گیرد. رشته‌ی خالی و عدد صفر یا منفی هم رشته و عدد معتبرند، پس سه گزینه‌ی دیگر مجازند.

۱۱۵۹ **گزینه‌ی ۱** سخت

`(string | number)[]` یعنی «آرایه‌ای که **هر خانه‌اش** می‌تواند عدد یا رشته باشد»، پس `[1, "x"]` در آن جا می‌شود. `string[] | number[]` (بدون پرانتز) یعنی «یک رشته **یا** یک آرایه‌ی تمام‌عددی». `string | number[]` یعنی «آرایه‌ای که یا همه‌اش رشته است یا همه‌اش عدد»؛ پس آرایه‌ی مخلوط در هیچ‌کدام جا نمی‌شود. `number[]` هم رشته‌ی `"x"` را نمی‌پذیرد.

۱۱۶۰ **گزینه‌ی ۴** سخت

تعریف یک متغیر `let` با نوع ولی بدون مقدار اولیه مجاز است (خط ۱)؛ این کار فقط برای `const` ممنوع است. اما در خط ۴ از `total` پیش از مقداردهی استفاده شده و tsc خطای TS2454 می‌دهد: `Variable 'total' is used before being assigned`. مقداردهی در خط ۵ دیر است. خط ۳ هم یک عملیات عددی عادی روی `count` است.

۱۱۶۱ **گزینه‌ی ۴** سخت

`const` فقط انتساب دوباره به `arr` را ممنوع می‌کند؛ نوع همان `number[]` است و `arr.push(3)` بدون خطا کامپایل و اجرا می‌شود؛ پس گزینه‌ی ۴ نادرست است. سه گزینه‌ی دیگر قاعده‌های درست inferiltype است: `let` نوع عمومی (`number`) می‌گیرد، `const` با مقدار ساده نوع literal (`5`) و آرایه‌ی رشته‌ها نوع `string[]`.

۱۱۶۲ **گزینه‌ی ۴** آسان

`id` یک بار عدد و یک بار رشته می‌گیرد، پس به union type `number | string` نیاز دارد (همان مثال اسلاید ۱۸). با `number` خط سوم و با `string` خط دوم خطا می‌دهد و با `boolean` هر دو خط.

۱۱۶۳ **گزینه‌ی ۱** سخت

`status` با `const` تعریف شده، پس نوعش literal یعنی `"ok"` است و در متغیری از نوع `"ok" | "error"` جا می‌شود (خط ۲ مجاز است). اما `message` با `let` تعریف شده و نوعش `string` است؛ هر رشته‌ای لزوماً `"ok"` یا `"error"` نیست، پس خط ۴ خطای TS2322 دارد. مقدار هر دو متغیر یکی است، ولی **نوعشان** فرق دارد.

۱۱۶۴ **گزینه‌ی ۲** متوسط

union type فقط در زمان کامپایل وجود دارد و در JavaScript خروجی پاک می‌شود. در زمان اجرا `typeof` نوع **مقدار فعلی** را برمی‌گرداند: اول `7` که `number` است و بعد `"seven"` که `string` است. گزینه‌ی ۱ فکر کرده union در زمان اجرا هم هست. گزینه‌ی ۳ فکر کرده نوع با اولین مقدار ثابت می‌ماند و گزینه‌ی ۴ آخرین نوع را برای هر دو خط در نظر گرفته است.

۳-۱۰ Interface در TypeScript

۱۱۶۵ گزینه‌ی ۱ آسان

طبق بولت اول اسلاید ۱۹، interface شکل یک شیء (shape of an object) را تعریف می‌کند: چه propertyهایی با چه typeهایی. interface هیچ شیئی نمی‌سازد و در JavaScript خروجی اصلاً وجود ندارد (گزینه‌ی ۲). تبدیل به JavaScript کار tsc است (گزینه‌ی ۳). بررسی‌های TypeScript هم فقط در زمان کامپایل انجام می‌شوند، نه در زمان اجرا (گزینه‌ی ۴).

۱۱۶۶ گزینه‌ی ۳ آسان

`?` یعنی optional property: شیء `User` می‌تواند `email` نداشته باشد و در آن صورت مقدارش `undefined` است؛ ولی اگر داشته باشد، باید `string` باشد. فقط‌خواندنی بودن با کلمه‌ی `readonly` مشخص می‌شود (گزینه‌ی ۲). `?` ربطی به `null` ندارد (گزینه‌ی ۱) و نوع را هم آزاد نمی‌کند (گزینه‌ی ۴).

۱۱۶۷ گزینه‌ی ۴ آسان

`id` ویژگی اجباری است و در گزینه‌ی ۴ جا افتاده؛ پس خطای TS2741 می‌گیرد: `Property 'id' is missing in type '{ name: string; }' but required in type 'User'`. در گزینه‌ی ۱ فقط `email` نیامده که اختیاری است. گزینه‌ی ۲ همه‌ی ویژگی‌ها را با نوع درست دارد. در گزینه‌ی ۳ هم رشته‌ی خالی یک `string` معتبر است.

۱۱۶۸ گزینه‌ی ۲ متوسط

در خط ۷ یک object literal مستقیماً به نوع `User` داده شده و ویژگی `age` را دارد که در `User` تعریف نشده؛ پس خطای TS2353 می‌گیرد: `Object literal may only specify known properties, and 'age' does not exist in type 'User'`. خط ۶ مجاز است چون `email` اختیاری است و خط ۸ هم همه‌ی ویژگی‌ها را با نوع درست دارد.

۱۱۶۹ گزینه‌ی ۴ سخت

TypeScript نوع‌ها را با شکلشان مقایسه می‌کند (structural typing): `p3` ویژگی‌های `x` و `y` را با نوع عددی دارد، پس فرستادنش به `print` در خط ۹ مجاز است و ویژگی اضافه‌ی `z` مشکلی ایجاد نمی‌کند. اما در خط ۱۰ همان شیء به شکل object literal **مستقیم** فرستاده شده و excess property check خطای TS2353 می‌دهد. خط ۸ هم فقط یک شیء معمولی می‌سازد که هنوز نوع `Point` به آن داده نشده است.

۱۱۷۰ گزینه‌ی ۱ متوسط

`email` اختیاری است، پس نوعش `string | undefined` است. در خط ۹ بدون هیچ بررسی‌ای `length` آن خوانده شده و tsc خطای TS18048 می‌دهد: `'u.email' is possibly 'undefined'`. خط ۸ مجاز است چون `?.` در صورت نبودن `email` به‌جای خطا `undefined` برمی‌گرداند. خط ۷ هم مشکلی ندارد، چون `name` اجباری و از نوع `string` است.

۱۱۷۱ گزینه‌ی ۲ متوسط

شیء `u` فقط دو ویژگی `id` و `name` دارد. خواندن ویژگی‌ای که وجود ندارد در زمان اجرا `undefined` می‌دهد (نه `null`) و مقایسه‌ی آن با `undefined` هم `true` است. `Object.keys(u)` فقط کلیدهای موجود یعنی `["id", "name"]` را برمی‌گرداند، پس طولش ۲ است. گزینه‌های ۱ و ۴ فکر کرده‌اند تعریف `email?` در interface یک کلید واقعی به شیء اضافه می‌کند؛ در حالی که interface در زمان اجرا اصلاً وجود ندارد.

۱۱۷۲ گزینه‌ی ۴ آسان

ویژگی `readonly` فقط هنگام ساختن شیء (خط ۵) مقدار می‌گیرد؛ تغییر بعدی آن در خط ۷ خطای TS2540 دارد: `Cannot assign to 'x' because it is a read-only property`. `y` معمولی است و خط ۶ مجاز است. با اینکه `p` با `const` تعریف شده، تغییر ویژگی‌های غیر readonly آن ممنوع نیست.

۱۱۷۳ **گزینه‌ی ۳** سخت

`readonly` روی **ویژگی** `url` است؛ پس تغییر `c.url` حتی وقتی `c` با `let` تعریف شده خطای TS2540 دارد (گزینه‌ی ۳). در گزینه‌ی ۱ ویژگی غیر readonly تغییر کرده که مجاز است، حتی با `const` . در گزینه‌ی ۲ کل متغیر `let` به یک شیء کاملاً جدید اشاره داده شده؛ این انتساب دوباره‌ی متغیر است، نه تغییر ویژگی readonly. گزینه‌ی ۴ هم فقط مقدار را می‌خواند.

۱۱۷۴ **گزینه‌ی ۲** متوسط

`Admin` با `extends` همه‌ی ویژگی‌های `User` را به‌علاوه‌ی `role` دارد و `role` اجباری است؛ پس گزینه‌ی ۲ خطای TS2741 می‌گیرد: `Property 'role' is missing …` . گزینه‌ی ۱ همه‌ی ویژگی‌های اجباری را دارد. گزینه‌ی ۳ یک `User` معمولی است که به `role` نیازی ندارد. در گزینه‌ی ۴ هم `email` از `User` به ارث رسیده و استفاده از آن مجاز است.

۱۱۷۵ **گزینه‌ی ۱** متوسط

`Person` هم‌زمان دو interface را گسترش داده است، پس ویژگی‌های **هر دو** (`id` و `name`) به‌علاوه‌ی ویژگی خودش (`age`) را دارد و هر سه اجباری‌اند. فقط گزینه‌ی ۱ هر سه را دارد. در گزینه‌ی ۲ `age` ، در گزینه‌ی ۳ `name` (از `HasName`) و در گزینه‌ی ۴ `id` (از `HasId`) جا افتاده و هر کدام خطای TS2741 دارند.

۱۱۷۶ **گزینه‌ی ۱** سخت

`Admin` همه‌ی ویژگی‌های `User` را دارد، پس هر جا `User` لازم باشد می‌شود `Admin` داد و خط ۱۳ مجاز است (خط ۱۴ هم که خودِ `User` را می‌دهد). اما برعکسش ممکن نیست: `user` ویژگی `role` را ندارد، پس نسبت دادنش به متغیری از نوع `Admin` در خط ۱۵ خطای TS2741 دارد. قاعده: نوعِ «کامل‌تر» به‌جای نوعِ «ساده‌تر» می‌نشیند، نه برعکس.

۱۱۷۷ **گزینه‌ی ۲** آسان

طبق اسلاید ۱۹، interface را می‌شود extend کرد (`interface Admin extends User`) و کلاس‌ها interface را implement می‌کنند (`class Customer implements User`). گزینه‌ی ۱ برعکس است؛ اگر کلاسی بخواهد interface را extend کند، tsc می‌گوید: `Cannot extend an interface 'User'. Did you mean 'implements'?` . پس این دو کلمه جایگزین هم نیستند (گزینه‌ی ۳). گزینه‌ی ۴ هم غلط است: `extends` برای ارث‌بری کلاس‌ها در خود JavaScript هم هست، اما `implements` فقط در TypeScript وجود دارد.

۱۱۷۸ **گزینه‌ی ۱** متوسط

کلاسی که `implements User` می‌نویسد باید همه‌ی اعضای **اجباری** `User` را داشته باشد. `Broken` عضو `name` را ندارد، پس tsc روی خط ۱۰ (نام کلاس) خطای TS2420 می‌دهد: `Class 'Broken' incorrectly implements interface 'User'` . `Customer` هر دو عضو اجباری را دارد و نبودن `email` اختیاری مشکلی ندارد (خط ۶). خط ۱۱ هم به‌تنهایی درست است؛ مشکل کمبود عضو است که روی کل کلاس گزارش می‌شود.

۱۱۷۹ **گزینه‌ی ۳** سخت

گزینه‌ی ۳ هر دو عضو `Shape` را با نوع درست دارد (`area` عدد برمی‌گرداند) و عضو اضافه‌ی `side` هم مشکلی ایجاد نمی‌کند، چون کلاس می‌تواند بیشتر از interface داشته باشد. گزینه‌ی ۱ متد `area` را ندارد (TS2420). در گزینه‌ی ۲ متد `area` رشته برمی‌گرداند و در گزینه‌ی ۴ نوع `name` عدد است؛ هر دو با interface ناسازگارند (TS2416).

۱۱۸۰ **گزینه‌ی ۱** متوسط

interface فقط در زمان کامپایل وجود دارد و در JavaScript خروجی پاک می‌شود؛ پس نمی‌شود آن را مثل یک مقدار (مثلاً سمت راست `instanceof`) به کار برد و خط ۷ خطای TS2693 دارد: `'User' only refers to a type, but is being used as a value here` . خط ۶ مجاز است، چون `typeof u` نوعِ زمان اجرای **مقدار** `u` را می‌دهد (یعنی `"object"`). خط ۵ هم یک تعریف درست است.

۱۱۸۱ **گزینه‌ی ۴** متوسط

interfaceها فقط برای کامپایلرند و در خروجی کاملاً پاک می‌شوند (type erasure)؛ خروجی کامپایل فایل اسلاید فقط `"use strict";` است. TypeScript نه کلاسی می‌سازد (گزینه‌ی ۱)، نه شیئی برای نگه داشتن نوع‌ها (گزینه‌ی ۲) و نه کدی برای بررسی در زمان اجرا (گزینه‌ی ۳).

۱۱۸۲ **گزینه‌ی ۳** سخت

نسبت دادن `data` به `User` مجاز است، چون `data` یک متغیر است (نه object literal مستقیم) و ویژگی‌های لازم را دارد. اما نوع‌ها در زمان اجرا وجود ندارند و چیزی را از شیء حذف نمی‌کنند؛ `u` و `data` به یک شیء اشاره می‌کنند که هنوز `age` را دارد. گزینه‌ی ۱ فکر کرده interface ویژگی‌های اضافه را پاک می‌کند. گزینه‌ی ۲ فکر کرده interface مثل کلاس نامی روی شیء می‌گذارد. گزینه‌ی ۴ هم غلط است، چون هیچ بررسی‌ای در زمان اجرا انجام نمی‌شود.

۱۱۸۳ **گزینه‌ی ۴** سخت

متد `greet` طبق interface باید رشته برگرداند، اما در گزینه‌ی ۴ عدد (`n.length`) برمی‌گرداند؛ پس خطای TS2322 دارد. در گزینه‌ی ۱ نوع `n` از روی interface فهمیده می‌شود و لازم نیست نوشته شود. گزینه‌ی ۲ همان امضا را صریح نوشته است. گزینه‌ی ۳ هم مجاز است: تابعی که پارامتر کمتری می‌گیرد (و آن را نادیده می‌گیرد) جای تابعی با پارامتر بیشتر می‌نشیند، چون فقط خروجی رشته‌ای لازم است.

۱۱۸۴ **گزینه‌ی ۳** متوسط

interface فقط برای توصیف شکل شیء (و متدهایش) است و نمی‌شود با آن به `number | string` نام داد؛ برای این کار باید از `type` استفاده کرد. پس گزینه‌ی ۳ نادرست است. گزینه‌ی ۱ درست است (`type Point = { x: number; y: number }` هم شکل شیء است)، گزینه‌ی ۲ مثال درست type alias است و گزینه‌ی ۴ همان بولت آخر اسلاید ۱۹.

۱۱۸۵ **گزینه‌ی ۴** سخت

دو interface هم‌نام با هم **ادغام** می‌شوند (declaration merging)، پس `B` هم `x` دارد و هم `y` و گزینه‌ی ۴ بی‌خطاست. اما type alias را نمی‌شود دو بار تعریف کرد و گزینه‌ی ۱ خطای `Duplicate identifier 'A'` دارد. در گزینه‌ی ۲ بررسی ویژگی اضافه برای type alias هم انجام می‌شود (`y` در `C` نیست). در گزینه‌ی ۳ هم ویژگی اجباری `x` جا افتاده و `y` اضافه است.

۱۱۸۶ **گزینه‌ی ۲** آسان

شیء `p` ویژگی `price` را ندارد، پس `price` باید **اختیاری** باشد: `price?: number;` . با `:` ساده، `price` اجباری است و خطای TS2741 (جا افتادن ویژگی) رخ می‌دهد. `!:` و `=` در interface اصلاً نوشتن مجازی نیستند و خطای نحوی (syntax error) دارند؛ interface فقط نوع را می‌گوید و مقدار پیش‌فرض نمی‌گیرد.

۳-۱۱ توابع در TypeScript

۱۱۸۷ **گزینه‌ی ۲** آسان

پارامتر `a` از نوع `number` است و `"2"` رشته است؛ پس خطای TS2345 می‌گیرد:
`Argument of type 'string' is not assignable to parameter of type 'number'` . در JavaScript همین فراخوانی بی‌صدا `"23"` برمی‌گرداند. عدد اعشاری و منفی (گزینه‌ی ۳) و صفر (گزینه‌ی ۴) همه `number` معتبرند.

۱۱۸۸ **گزینه‌ی ۱** آسان

دو آرگومان اول به `name` و `age` می‌رسند و rest parameter یعنی `...hobbies` **همه‌ی** آرگومان‌های باقی‌مانده را در یک آرایه جمع می‌کند: `["Reading", "Cycling"]` . شیء برگشتی با shorthand property ساخته شده (`name` یعنی `name: name`). گزینه‌های ۲ و ۳ فکر کرده‌اند rest فقط یک آرگومان می‌گیرد. گزینه‌ی ۴ هم خروجی را آرایه گرفته، در حالی که تابع یک شیء برمی‌گرداند.

۱۱۸۹ **گزینه‌ی ۳** متوسط

وقتی هیچ آرگومانی برای rest parameter فرستاده نشود، مقدار آن یک آرایه‌ی **خالی** است، نه `undefined` ؛ پس `[]` و طول `0` چاپ می‌شود. اگر `hobbies` برابر `undefined` بود، خواندن `length` آن TypeError می‌داد (گزینه‌ی ۲)، ولی این‌طور نیست. گزینه‌های ۱ و ۴ هم فکر کرده‌اند یک خانه‌ی خالی یا `undefined` در آرایه قرار می‌گیرد.

۱۱۹۰ **گزینه‌ی ۳** متوسط

نوع rest parameter `string[]` است، پس **هر** آرگومانی که بعد از `age` می‌آید باید رشته باشد. در خط ۱۰ عدد `3` آمده و خطای TS2345 رخ می‌دهد: `Argument of type 'number' is not assignable to parameter of type 'string'` . خط ۸ بدون hobby است که مجاز است (`hobbies` آرایه‌ی خالی می‌شود) و خط ۹ یک hobby رشته‌ای دارد.

۱۱۹۱ **گزینه‌ی ۴** سخت

`name` و `age` اجباری‌اند، پس فراخوانی با یک آرگومان خطای TS2555 دارد: `Expected at least 2 arguments, but got 1` (کلمه‌ی at least به خاطر rest parameter است). گزینه‌ی ۱ هیچ hobbyای ندارد که مجاز است. rest parameter هر تعداد رشته می‌پذیرد (گزینه‌ی ۲). در گزینه‌ی ۳ هم یک آرایه‌ی رشته‌ای با spread (`...`) پخش شده و هر عضوش یک آرگومان رشته‌ای می‌شود.

۱۱۹۲ **گزینه‌ی ۲** آسان

`?` بعد از نام پارامتر یعنی optional parameter: می‌شود تابع را با یک آرگومان صدا زد و آن وقت `greeting` برابر `undefined` است؛ به همین دلیل TypeScript نوعش را `string | undefined` می‌داند و استفاده‌ی بی‌بررسی از آن خطا دارد. `?` نوع را آزاد نمی‌کند (گزینه‌ی ۱)، مقدار پیش‌فرض نمی‌دهد (برای آن باید `= value` نوشت؛ گزینه‌ی ۳) و ربطی به `null` ندارد (گزینه‌ی ۴).

۱۱۹۳ **گزینه‌ی ۳** آسان

پارامتر اجباری نمی‌تواند **بعد از** پارامتر اختیاری بیاید؛ در خط ۳ پارامتر اجباری `y` بعد از `x?` آمده و خطای TS1016 رخ می‌دهد: `A required parameter cannot follow an optional parameter`. خط ۱ ترتیب درست را دارد. در خط ۲ هر دو اختیاری‌اند که مجاز است. در خط ۴ هم پارامتر پیش‌فرض (که خودش اختیاری است) بعد از پارامتر اجباری آمده است.

۱۱۹۴ **گزینه‌ی ۱** سخت

مقدار پیش‌فرض وقتی به کار می‌رود که آرگومان فرستاده نشود **یا** دقیقاً `undefined` باشد؛ پس هم `"Ali"` و هم `"Sara"` با `Hi` خوش‌آمد می‌گیرند. اما رشته‌ی خالی `""` یک مقدار واقعی است و پیش‌فرض را فعال نمی‌کند، پس خط سوم `, Reza` است. گزینه‌های ۲ و ۴ فکر کرده‌اند `undefined` صریح جای پیش‌فرض را می‌گیرد و گزینه‌های ۳ و ۴ فکر کرده‌اند هر مقدار falsy پیش‌فرض را فعال می‌کند.

۱۱۹۵ **گزینه‌ی ۴** متوسط

نوع `times` از روی مقدار پیش‌فرض `2` استنتاج شده و `number` است؛ پس رشته‌ی `"3"` خطای TS2345 می‌گیرد. پارامتر پیش‌فرض اختیاری است و می‌شود آن را نفرستاد (گزینه‌ی ۱) یا `undefined` فرستاد تا پیش‌فرض به کار برود (گزینه‌ی ۳). گزینه‌ی ۲ هم یک عدد معمولی می‌فرستد.

۱۱۹۶ **گزینه‌ی ۳** آسان

rest parameter آرگومان‌های متغیر (variable-length) را در یک آرایه جمع می‌کند و چون «بقیه»ی آرگومان‌ها را می‌گیرد، باید آخرین پارامتر باشد (وگرنه خطای TS1014). پس جایش آزاد نیست (گزینه‌ی ۱) و یک تابع هم فقط یک rest parameter دارد (گزینه‌ی ۴). بدون آرگومان اضافه، مقدارش آرایه‌ی خالی `[]` است، نه `undefined` (گزینه‌ی ۲).

۱۱۹۷ **گزینه‌ی ۲** متوسط

`sum()` بدون آرگومان صدا زده شده، پس `nums` آرایه‌ی خالی است و `reduce` با مقدار اولیه‌ی `0` همان `0` را برمی‌گرداند. `sum(1, 2, 3)` برابر ۶ است. در خط آخر، `...` در فراخوانی یعنی spread: آرایه به دو آرگومان ۱۰ و ۲۰ پخش می‌شود و حاصل ۳۰ است (گزینه‌ی ۴ اشتباه فکر کرده آرایه را نمی‌شود پخش کرد). گزینه‌ی ۱ فکر کرده `nums` بدون آرگومان `undefined` است و گزینه‌ی ۳ فکر کرده جمعِ «هیچ عددی» `NaN` می‌شود؛ در حالی که `reduce` روی آرایه‌ی خالی همان مقدار اولیه یعنی `0` را برمی‌گرداند.

۱۱۹۸ **گزینه‌ی ۲** متوسط

rest parameter باید **آخرین** پارامتر باشد؛ در خط ۲ پارامتر `color` بعد از `...names` آمده و خطای TS1014 رخ می‌دهد: `A rest parameter must be last in a parameter list`. خط ۱ ترتیب درست را دارد. در خط ۳ rest تنها پارامتر است. در خط ۴ هم ترتیب «اجباری، پیش‌فرض، rest» کاملاً مجاز است.

۱۱۹۹ **گزینه‌ی ۳** آسان

نوع خروجی `label` رشته اعلام شده، اما در خط ۸ یک عدد برگردانده می‌شود؛ پس خطای TS2322 دارد: `Type 'number' is not assignable to type 'string'`. تابع `log` با نوع `void` چیزی برنمی‌گرداند که درست است (خط ۲) و `double` هم عدد برمی‌گرداند (خط ۵).

۱۲۰۰ **گزینه‌ی ۱** سخت

اگر `x` صفر باشد، هیچ‌کدام از دو شرط برقرار نیست و تابع بدون `return` تمام می‌شود (یعنی `undefined` برمی‌گرداند)، در حالی که نوع خروجی `string` اعلام شده. tsc این را روی خط ۱ (محل نوع خروجی) گزارش می‌کند: TS2366، `Function lacks ending return statement and return type does not include 'undefined'`. مهم نیست که در خط ۸ با عدد ۵ صدا زده شده؛ tsc همه‌ی مسیرهای ممکن را بررسی می‌کند.

۱۲۰۱ **گزینه‌ی ۲** سخت

در خط ۴ پارامتر `n` نوع ندارد و TypeScript هم راهی برای فهمیدن نوعش ندارد؛ در حالت strict این پارامتر به‌طور ضمنی `any` می‌شود که مجاز نیست: TS7006، `Parameter 'n' implicitly has an 'any' type`. در خط ۷ هم پارامتر arrow function نوع ندارد، اما چون به `map` روی آرایه‌ی عددی داده شده، نوعش از زمینه (contextual typing) `number` استنتاج می‌شود. در خط ۱ هم نبودن نوع خروجی مشکلی ندارد، چون نوع خروجی استنتاج می‌شود.

۱۲۰۲ **گزینه‌ی ۴** متوسط

اول تابع اجرا می‌شود و `LOG: hi` چاپ می‌شود. تابعی که `return` ندارد در زمان اجرا `undefined` برمی‌گرداند؛ فقط یک نوع در زمان کامپایل است و مقداری به نام `void` چاپ نمی‌شود (گزینه‌ی ۱). مقدار برگشتی هم `null` نیست (گزینه‌ی ۲) و ترتیب چاپ‌ها هم برعکس نمی‌شود (گزینه‌ی ۳).

۱۲۰۳ **گزینه‌ی ۱** متوسط

برای arrow function نوع خروجی نوشته نشده، اما TypeScript آن را از `a + b` (جمع دو عدد) `number` استنتاج می‌کند؛ پس نسبت دادن حاصل به متغیر `string` خطای TS2322 دارد. گزینه‌ی ۲ و ۳ با عدد کار می‌کنند. در گزینه‌ی ۴ عدد اول با `String()` صریحاً به رشته تبدیل شده و بعد در متغیر رشته‌ای قرار گرفته است.

۱۲۰۴ **گزینه‌ی ۴** سخت

`op` باید تابعی باشد که دو عدد بگیرد و **عدد** برگرداند. در گزینه‌ی ۴ حاصل `x > y` از نوع `boolean` است و خطای TS2322 دارد. در گزینه‌های ۱ و ۳ نوع پارامترها از روی نوع `op` (contextual typing) عدد استنتاج می‌شود و خروجی هم عدد است. گزینه‌ی ۲ هم مجاز است: تابعی که پارامتر کمتری می‌گیرد (و دومی را نادیده می‌گیرد) جای این نوع می‌نشیند.

۱۲۰۵ **گزینه‌ی ۲** سخت

arrow function با بدنه‌ی `{ … }` فقط وقتی مقدار برمی‌گرداند که `return` صریح داشته باشد. در `triple` عبارت `n * 3` حساب می‌شود ولی برگردانده نمی‌شود، در حالی که نوع خروجی `number` اعلام شده؛ tsc خطای TS2355 را روی خط ۲ (محل نوع خروجی) گزارش می‌کند، نه روی خط ۳. خط ۱ بدنه‌ی بدون آکولاد دارد که خودش برمی‌گرداند و خط ۵ هم `return` صریح دارد.

۱۲۰۶ **گزینه‌ی ۱** آسان

`void` یعنی «هیچ مقداری برنمی‌گردد»؛ مثل `function logIt(msg: string): void`. اگر نوع خروجی را `null` بنویسی، تابع باید واقعاً `null` برگرداند و بدون `return` خطا دارد. `empty` و `none` اصلاً نوع‌های TypeScript نیستند.

۱۲۰۷ **گزینه‌ی ۳** سخت

پارامتر پیش‌فرض `a` **قبل از** پارامتر اجباری `b` آمده، پس تابع همیشه به دو آرگومان نیاز دارد و `f(5)` و `f()` خطای `Expected 2 arguments` می‌گیرند (در `f(5)`، عدد ۵ به `a` می‌رسد، نه `b`). برای استفاده از مقدار پیش‌فرض باید صریحاً `undefined` فرستاد: `f(undefined, 5)` که حاصلش ۶ است. در گزینه‌ی ۴ هم `"5"` رشته است و با نوع `number` پارامتر `b` جور نیست.

۱۲۰۸ **گزینه‌ی ۴** متوسط

`?` و مقدار پیش‌فرض را نمی‌شود با هم روی یک پارامتر گذاشت؛ tsc می‌گوید: `Parameter cannot have question mark and initializer`. لازم هم نیست، چون پارامتر پیش‌فرض خودش اختیاری است (گزینه‌ی ۲). گزینه‌ی ۱ تعریف درست پارامتر اختیاری است و گزینه‌ی ۳ همان return type inference است (نوع خروجی `a * 2` عدد استنتاج می‌شود).

۱۲۰۹ **گزینه‌ی ۱** متوسط

در فراخوانی اول `age` فرستاده نشده، پس `undefined` است و شرط برقرار می‌شود: `Sara: unknown` . در فراخوانی دوم `age` برابر `0` است که یک عدد واقعی است و با `=== undefined` برابر نیست؛ پس `Sara: 0` چاپ می‌شود. گزینه‌ی ۳ صفر را با «نبودن مقدار» یکی گرفته (اگر شرط `if (!age)` بود، همین اتفاق می‌افتاد). گزینه‌های ۲ و ۴ هم شرط `if` را نادیده گرفته‌اند.

۱۲۱۰ **گزینه‌ی ۴** متوسط

نوع پارامتر اختیاری `suffix` برابر `string | undefined` است. داخل `if (suffix)` نوعش به `string` باریک شده و خط ۳ مجاز است. اما در خط ۵ (بیرون از `if`) ممکن است `suffix` برابر `undefined` باشد و خواندن `length` آن خطای TS18048 دارد: `'suffix' is possibly 'undefined'` . خط ۱ هم یک تعریف درست است، چون پارامتر اختیاری بعد از پارامتر اجباری آمده است.

۱۲۱۱ **گزینه‌ی ۳** متوسط

`parseInt("2")` خروجی `number` دارد، پس گزینه‌ی ۳ دو آرگومان عددی می‌فرستد و درست است. در JavaScript فراخوانی با آرگومان کمتر یا بیشتر هم اجرا می‌شد، اما tsc تعداد آرگومان‌ها را هم بررسی می‌کند: گزینه‌ی ۱ خطای `Expected 2 arguments, but got 1` و گزینه‌ی ۲ خطای `Expected 2 arguments, but got 3` دارد. در گزینه‌ی ۴ هم `"1"` رشته است.

۱۲-۳ کلاس‌ها در TypeScript

۱۲۱۲ **گزینه‌ی ۳** آسان

این کد فقط کلاس (نقشه) را تعریف می‌کند؛ هیچ شیئی با `new` ساخته نشده و `greet()` هم صدا زده نشده است، پس **هیچ خروجی‌ای** ندارد. constructor فقط هنگام `new` اجرا می‌شود و `console.log` فقط وقتی `greet` صدا زده شود. گزینه‌های ۱ و ۲ فرض کرده‌اند تعریف کلاس خودبه‌خود متد را اجرا می‌کند و گزینه‌ی ۴ هم دلیلی ندارد، چون هیچ متغیر تعریف‌نشده‌ای استفاده نشده است.

۱۲۱۳ **گزینه‌ی ۱** آسان

`new Person("Alice")` constructor را اجرا می‌کند و `this.name` برابر `"Alice"` می‌شود. بعد `p.greet()` با template literal مقدار `${this.name}` را جایگزین می‌کند و `Hello, Alice` چاپ می‌شود. گزینه‌ی ۲ فراموش کرده که داخل بک‌تیک `${...}` با مقدار جایگزین می‌شود، گزینه‌ی ۳ فکر کرده constructor مقدار را ذخیره نکرده و گزینه‌ی ۴ اسم property را با مقدارش اشتباه گرفته است.

۱۲۱۴ **گزینه‌ی ۴** آسان

در TypeScript سطح دسترسی پیش‌فرض **`public`** است؛ یعنی `name: string` دقیقاً همان `public name: string` است. `private` و `protected` را باید صریحاً بنویسی (گزینه‌های ۱ و ۲). `readonly` هم ربطی به سطح دسترسی ندارد و فقط وقتی اعمال می‌شود که خودت آن را بنویسی (گزینه‌ی ۳).

۱۲۱۵ **گزینه‌ی ۲** آسان

`protected` یعنی «خودِ کلاس و فرزندانش»؛ استفاده از بیرون (مثل `s.age`) خطای TS2445 می‌دهد. گزینه‌ی ۱ غلط است چون `private` حتی در کلاس فرزند هم ممنوع است (TS2341). گزینه‌ی ۳ رفتار `protected` را با `readonly` قاطی کرده؛ از `protected` از بیرون نه خواندنی است نه نوشتنی. گزینه‌ی ۴ هم غلط است چون `public` پیش‌فرض است و نوشتنش اختیاری است.

۱۲۱۶ **گزینه‌ی ۳** آسان

access modifierها (`public` ، `private` ، `protected`) کلیدواژه‌های TypeScript هستند و در JavaScript وجود ندارند (JavaScript فقط فیلدهای `#` را دارد که سازوکار دیگری است). در مقابل، `constructor` ، `extends` و متدها از ES6 بخشی از خودِ JavaScript هستند، پس گزینه‌های ۱، ۲ و ۴ امکان تازه‌ای نیستند.

۱۲۱۷ **گزینه‌ی ۱** آسان

وراثت در TypeScript (و JavaScript) با **`extends`** است؛ `Student` بدون constructor خودش، `Person` constructor را به کار می‌برد و `name` را هم به ارث می‌برد. `implements` فقط «قول رعایت شکل» است و چیزی به ارث نمی‌برد؛ این‌جا `Student` property `name` را ندارد، پس خطای TS2720 می‌دهد. `inherits` اصلاً کلیدواژه‌ی زبان نیست (خطای نحوی) و `super` برای صدا زدن constructor یا متد والد است، نه تعریف وراثت.

۱۲۱۸ **گزینه‌ی ۱** آسان

`implements` یک **قرارداد زمان کامپایل** است: اگر عضوی کم باشد خطای TS2420 و اگر نوعش ناسازگار باشد TS2416 می‌گیری. interface هیچ کدی ندارد که به کلاس اضافه شود (گزینه‌ی ۲)، و چون interface و `implements` در JavaScript خروجی پاک می‌شوند، هیچ بررسی‌ای در زمان اجرا انجام نمی‌شود (گزینه‌ی ۳). `extends` ارث‌بری واقعی از یک کلاس است، نه بررسی شکل (گزینه‌ی ۴).

۱۲۱۹ **گزینه‌ی ۳** متوسط

`id` در کلاس `Order` نه مقدار اولیه دارد و نه constructorی که آن را مقدار بدهد، پس خطای **TS2564** (`Property 'id' has no initializer and is not definitely assigned in the constructor`) می‌گیرد. خط ۲ مشکلی ندارد چون `title` در constructor مقدار می‌گیرد؛ خط ۴ اختیاری (`?`) است و می‌تواند `undefined` بماند؛ خط ۱۱ هم مقدار اولیه‌ی `[]` دارد.

۱۲۲۰ **گزینه‌ی ۴** متوسط

`age` `protected` فقط داخل `Person` و فرزندانش قابل استفاده است؛ پس `this.age` در خط ۱۳ (داخل متد `Student`) مجاز است، ولی `s.age` در خط ۱۹ از بیرون کلاس است و خطای **TS2445** می‌دهد. خط ۱۷ مشکلی ندارد چون `name` `public` است و خط ۱۸ هم یک متد `public` را صدا می‌زند.

۱۲۲۱ **گزینه‌ی ۱** متوسط

`balance` `private` فقط داخل خودِ `Account` قابل استفاده است؛ حتی کلاس فرزند (`Savings`) هم به آن دسترسی ندارد، پس خط ۱۰ خطای **TS2341** می‌دهد. خط ۱۳ مجاز است چون `owner` `protected` در فرزند قابل استفاده است، و خط ۱۷ متد `deposit` `public` را صدا می‌زند که خودش داخل `Account` به `balance` دست می‌زند. کسی که «بدون خطا» را انتخاب کند، فکر کرده `private` هم مثل `protected` به فرزند می‌رسد.

۱۲۲۲ **گزینه‌ی ۲** متوسط

property `readonly` فقط کنار اعلانش یا **داخل constructor** مقدار می‌گیرد؛ پس خط ۵ مجاز است ولی خط ۸ (داخل متد معمولی `setUrl`) خطای **TS2540** (`Cannot assign to 'apiUrl' because it is a read-only property`) می‌دهد. خط ۱۱ فقط شیء می‌سازد و خواندن propertyهای `readonly` در خط ۱۲ هم آزاد است. «متد همان کلاس است پس مجاز است» دام اصلی این تست است.

۱۲۲۳ **گزینه‌ی ۴** متوسط

`Circle` قول داده `Shape` را پیاده کند ولی متد `area()` را ندارد؛ پس روی خط ۱۲ خطای **TS2420** (`Class 'Circle' incorrectly implements interface 'Shape'`) گزارش می‌شود. `Square` هم `name` دارد و هم با `area()` خروجی `number`، پس خطوط ۵ تا ۸ درست‌اند. داشتن property اضافه (`side` یا `r`) هم برای `implements` مشکلی ندارد.

۱۲۲۴ **گزینه‌ی ۲** متوسط

`count` چون `static` است فقط **یک نسخه‌ی مشترک** دارد که مال خودِ کلاس است. با هر `new` یکی زیاد می‌شود و همان لحظه در `id` آن شیء کپی می‌شود: `a.id = 1`، `b.id = 2`، `c.id = 3` و در پایان `Counter.count = 3`. گزینه‌ی ۱ فکر کرده هر شیء شمارنده‌ی جدای خودش را دارد، گزینه‌ی ۳ فکر کرده `id` به شمارنده وصل می‌ماند و بعداً عوض می‌شود، و گزینه‌ی ۴ ترتیب افزایش و کپی را برعکس فرض کرده است.

۱۲۲۵ **گزینه‌ی ۴** متوسط

هر سه پارامتر modifier دارند، پس هر سه **parameter property** هستند و خودکار به property تبدیل می‌شوند؛ `z` هم مقدار پیش‌فرض `0` را می‌گیرد. پس `pt.sum()` برابر `1 + 2 + 0 = 3` است. در JavaScript خروجی `y` یک property معمولی است (`private` فقط در زمان کامپایل معنا دارد)، پس `JSON.stringify` هر سه را نشان می‌دهد. گزینه‌ی ۱ فکر کرده `private` در زمان اجرا پنهان می‌ماند و گزینه‌های ۲ و ۳ فکر کرده‌اند پارامترهای constructor به property تبدیل نمی‌شوند.

۱۲۲۶ **گزینه‌ی ۳** متوسط

فقط در `C` مطمئنیم `name` همیشه مقدار دارد (مقدار اولیه‌ی `""`). `A` اصلاً مقداردهی نمی‌کند، `B` فقط گاهی (داخل `if`) مقدار می‌دهد و `D` مقداردهی را به متدی سپرده که شاید هرگز صدا زده نشود؛ هر سه خطای **TS2564** می‌گیرند. strict mode می‌خواهد property کنار اعلان یا در **همه‌ی** مسیرهای constructor مقدار بگیرد.

۱۲۲۷ **گزینه‌ی ۴** متوسط

مقداردهی دوباره‌ی property `readonly` فقط کنار اعلان (گزینه‌ی ۱) یا در constructor (گزینه‌ی ۲) مجاز است؛ در یک متد معمولی، حتی از همان کلاس، خطای **TS2540** می‌دهد، پس گزینه‌ی ۴ نادرست است. گزینه‌ی ۳ درست است چون `readonly` «سطحی» است: فقط جلوی **عوض کردن خودِ property** (مثلاً `this.items = []`) را می‌گیرد، نه تغییر محتوای آرایه‌ای که به آن اشاره می‌کند.

۱۲۲۸ **گزینه‌ی ۱** متوسط

عضو `static` مال **خودِ کلاس** است و با اسم کلاس صدا زده می‌شود: `Counter.count++`. دسترسی از طریق شیء (`c.count`) خطای **TS2576** می‌دهد و TypeScript خودش پیشنهاد می‌کند `Counter.count` بنویسی. در سطح بالای یک ماژول `this` برابر `undefined` است (خطای TS2532)، و `count` به‌تنهایی هم متغیری تعریف‌نشده است (TS2304).

۱۲۲۹ **گزینه‌ی ۲** متوسط

اگر کلاس فرزند constructor خودش را بنویسد، **باید** داخل آن `super(...)` را صدا بزند تا بخش والد شیء ساخته شود؛ این کار را نکرده و خطای **TS2377** (`Constructors for derived classes must contain a 'super' call`) می‌گیرد. `Cat` اصلاً constructor ندارد، پس constructor خودکار استفاده می‌شود و خطایی ندارد. `Animal` و `Dog` و `Bird` هم `super` را صدا زده‌اند؛ مهم نیست constructor فرزند چه پارامترهایی دارد (`Bird` هیچ پارامتری ندارد و خودش `"bird"` را به والد می‌دهد).

۱۲۳۰ **گزینه‌ی ۲** سخت

در constructor کلاس فرزند، **قبل از** صدا زدن `super(...)` شیء هنوز ساخته نشده و نمی‌توان از `this` استفاده کرد؛ پس خط ۷ خطای **TS17009** (`'super' must be called before accessing 'this'`) می‌دهد. خودِ `super(name)` در خط ۸ مشکلی ندارد، فقط باید جابه‌جا شود. خط ۵ هم خطا ندارد چون `breed` در constructor مقدار گرفته است. در JavaScript خالی همین ترتیب در زمان اجرا `ReferenceError` می‌دهد.

۱۲۳۱ **گزینه‌ی ۲** سخت

فقط پارامتری که **modifier** دارد (`public name`) به property تبدیل می‌شود. `age` یک پارامتر معمولی است که فقط داخل constructor زنده است، پس `this.age` در خط ۴ خطای **TS2339** (`Property 'age' does not exist on type 'User'`) می‌دهد. خط ۲ خودش درست است (نوشتن پارامتر بدون modifier مجاز است)، و خطوط ۷ و ۸ هم با دو آرگومان و property `public` `name` کاملاً درست‌اند.

۱۲۳۲ **گزینه‌ی ۴** سخت

`override` یعنی «این متد، متدی از والد را بازنویسی می‌کند»؛ چون `Animal` متدی به اسم `swim` ندارد، گزینه‌ی ۴ خطای **TS4113** (`This member cannot have an 'override' modifier because it is not declared in the base class 'Animal'`) می‌دهد. گزینه‌ی ۱ بدون `override` هم درست است (در strict نوشتن `override` اجباری نیست)، گزینه‌ی ۲ با `override` درست بازنویسی کرده و گزینه‌ی ۳ هم یک متد تازه اضافه کرده که اصلاً بازنویسی نیست.

۱۲۳۳ **گزینه‌ی ۱** سخت

متد بازنویسی‌شده باید با نسخه‌ی والد **سازگار** باشد. `Fish` نوع خروجی `speak` را از `string` به `number` عوض کرده و خطای **TS2416** (`Property 'speak' in type 'Fish' is not assignable to the same property in base type 'Animal'`) می‌گیرد. در گزینه‌ی ۳ نوع خروجی نوشته نشده ولی از `return "Moo"` نوع `string` استنتاج می‌شود و سازگار است؛ گزینه‌ی ۴ هم با `super.speak()` نسخه‌ی والد را صدا زده که کاملاً مجاز است.

۱۲۳۴ **گزینه‌ی ۲** سخت

این‌که نوع آرایه `Animal[]` است فقط برای بررسی در زمان کامپایل است؛ در زمان اجرا برای هر شیء، متدِ **کلاس واقعی‌اش** اجرا می‌شود (polymorphism). برای `Rex` که یک `Dog` است، `speak` متدِ `Dog` اجرا می‌شود که اول با `super.speak()` متن والد (`Rex makes a sound`) را می‌گیرد و `- Woof` را به آن اضافه می‌کند. گزینه‌ی ۱ فکر کرده نوع آرایه تعیین می‌کند کدام متد اجرا شود، گزینه‌ی ۳ نقش `super.speak()` را نادیده گرفته و `Dog` بدون constructor هم از `Animal` استفاده می‌کند، پس خطایی (گزینه‌ی ۴) رخ نمی‌دهد.

۱۲۳۵ **گزینه‌ی ۳** سخت

`private` فقط در زمان کامپایل بررسی می‌شود و در JavaScript خروجی `pin` یک property معمولی است؛ پس هم در `Object.keys` و هم در `JSON.stringify` دیده می‌شود (گزینه‌ی ۱ غلط است). در مقابل، `#balance` یک فیلد خصوصی **واقعی** JavaScript است و نه جزو کلیدهای شیء حساب می‌شود و نه در JSON می‌آید (گزینه‌های ۲ و ۴ غلط‌اند).

۱۲۳۶ **گزینه‌ی ۴** سخت

`p.greet()` چون شیء قبل از نقطه است، `this` را `p` می‌گیرد و `Hello, Alice` چاپ می‌شود. اما `f()` بدون شیء صدا زده شده و چون بدنه‌ی کلاس همیشه **strict** است، `this` برابر `undefined` می‌شود؛ خواندن `this.name` خطای `TypeError` می‌دهد. گزینه‌ی ۱ فکر کرده متد همیشه به شیء اولش چسبیده است، گزینه‌ی ۲ رفتار حالت غیر strict را فرض کرده و گزینه‌ی ۳ فراموش کرده خط اولِ اجرا (`p.greet()`) قبل از خطا خروجی داده است. TypeScript این اشتباه را نمی‌گیرد.

۱۲۳۷ **گزینه‌ی ۳** سخت

access modifierهای TypeScript بعد از کامپایل پاک می‌شوند؛ property `private` در JavaScript خروجی **باقی می‌ماند** (گزینه‌ی ۱ غلط) و یک property معمولی است، پس `obj["pin"]` در زمان اجرا مقدارش را برمی‌گرداند و هیچ خطایی نمی‌دهد (گزینه‌ی ۴ غلط؛ TypeScript هم دسترسی با براکت را مجاز می‌داند). گزینه‌ی ۲ برعکس است: property `private` در `Object.keys` دیده می‌شود ولی فیلد `#` نه. پس فقط گزینه‌ی ۳ درست است.

۱۳-۳ Generics در TypeScript

۱۲۳۸ **گزینه‌ی ۱** آسان

اسلاید ۲۲ می‌گوید generics کامپوننت‌ها و تابع‌های **reusable** و **type-safe** می‌سازند و جلوی تکرار کد را می‌گیرند «بدون از دست رفتن بررسی نوع». generics فقط در زمان کامپایل وجود دارند و هیچ اثری روی سرعت اجرا یا تبدیل مقدارها ندارند (گزینه‌های ۲ و ۴). گزینه‌ی ۳ دقیقاً برعکس است: `any` بررسی نوع را خاموش می‌کند ولی generic آن را حفظ می‌کند.

۱۲۳۹ **گزینه‌ی ۳** آسان

`T` یک type parameter است: همان‌طور که `value` جای خالی برای مقدار است، `T` جای خالی برای **نوع** است و با `identity<number>(42)` (یا با استنتاج) پر می‌شود. `T` لازم نیست جایی تعریف شده باشد (گزینه‌ی ۱)، با `value` فرق دارد چون یکی نوع است و دیگری مقدار (گزینه‌ی ۲)، و برخلاف `any` نوع را حفظ می‌کند (گزینه‌ی ۴).

۱۲۴۰ **گزینه‌ی ۲** آسان

`identity` همان مقدار را برمی‌گرداند، پس `num` عدد `42` و `str` رشته‌ی `"Hello"` است و `typeof` نوع واقعی زمان اجرا را می‌دهد: `number` و `string` . `T` و `any` فقط در زمان کامپایل وجود دارند و در JavaScript خروجی پاک شده‌اند، پس `typeof` هرگز `T` یا `any` برنمی‌گرداند (گزینه‌های ۱ و ۴). `object` هم فقط برای اشیاء (و `null`) است، نه عدد و رشته (گزینه‌ی ۳).

۱۲۴۱ **گزینه‌ی ۴** آسان

اسلاید ۲۲ صریحاً می‌گوید «generics can be applied to functions, classes, and interfaces»: مثل `function identity<T>`، `class Stack<T>` و `interface Box<T>`. هر سه گزینه‌ی دیگر با کلمه‌ی «فقط» یکی از این سه را کنار گذاشته‌اند؛ حتی در Java هم generic مخصوص کلاس‌ها نیست.

۱۲۴۲ **گزینه‌ی ۳** آسان

در خط ۶ آرگومان نوع صریحاً `number` است، پس پارامتر `value` باید `number` باشد؛ دادن `"Hello"` خطای **TS2345** (`Argument of type 'string' is not assignable to parameter of type 'number'`) می‌دهد. خطوط ۴ و ۵ مقدار درست را با آرگومان نوع درست داده‌اند و در خط ۷ آرگومان نوع نوشته نشده و از روی `true` استنتاج می‌شود، پس هیچ‌کدام خطا ندارند.

۱۲۴۳ **گزینه‌ی ۲** آسان

`Array` یک نوع generic آماده است و آرگومان نوعش داخل `< >` می‌آید: `Array<string>` دقیقاً همان `string[]` است. `Array[string]` شبیه دسترسی به عضو آرایه است و خطای TS2314 (`Generic type 'Array<T>' requires 1 type argument(s)`) می‌دهد، `Array(string)` شبیه فراخوانی تابع است و خطای نحوی دارد، و در `string<Array>` جای نوع و آرگومان عوض شده است (`string` اصلاً generic نیست).

۱۲۴۴ **گزینه‌ی ۴** متوسط

با `let`، نوع `a` گسترده‌ی `number` استنتاج می‌شود (پس خط ۶ مجاز است)، ولی با `const`، نوع `b` دقیقاً `42` است (پس خط ۷ هم مجاز است). در خط ۸ می‌خواهیم یک `number` کلی را در متغیری از نوع `42` بگذاریم و خطای **TS2322** (`Type 'number' is not assignable to type '42'`) می‌گیریم؛ TypeScript نمی‌تواند مطمئن باشد `a` (که قبلاً `100` شده) همیشه `42` است.

۱۲۴۵ **گزینه‌ی ۱** متوسط

آرگومان نوع نوشته نشده، پس TypeScript آن را از روی `"Hello"` استنتاج می‌کند؛ چون `s` با `let` تعریف شده و بعداً می‌تواند عوض شود، نوع **گسترده‌ی** `string` را می‌گیرد. اگر `const s` بود، نوع دقیق `"Hello"` می‌شد (گزینه‌ی ۲). generic اطلاعات نوع را حفظ می‌کند، پس نتیجه `any` نیست (گزینه‌ی ۳)، و `T` فقط داخل تعریف تابع معنا دارد و بعد از فراخوانی با یک نوع واقعی جایگزین می‌شود (گزینه‌ی ۴).

۱۲۴۶ **گزینه‌ی ۳** متوسط

پشته LIFO است: آخرین عضوی که وارد شده (`"c"`) اول بیرون می‌آید و بعد از آن دو عضو باقی می‌ماند، پس `pop` . `c 2` روی آرایه‌ی خالی خطا نمی‌دهد و فقط `undefined` برمی‌گرداند؛ برای همین نوع خروجی `T | undefined` است و خط دوم `undefined 0` چاپ می‌کند. گزینه‌ی ۱ رفتار صف (FIFO) را فرض کرده، گزینه‌ی ۲ فراموش کرده `pop` عضو را حذف می‌کند و گزینه‌ی ۴ فکر کرده پشته‌ی خالی خطا می‌دهد.

۱۲۴۷ **گزینه‌ی ۳** متوسط

با `new Stack<number>()`، `T` برابر `number` است و `push` فقط `number` می‌پذیرد؛ پس `s.push("4")` در خط ۱۳ خطای **TS2345** می‌دهد. `2 * 3` یک عدد است (خط ۱۲ درست)، و خط ۱۴ فقط نتیجه را در `top` (با نوع `number | undefined`) می‌گذارد که مشکلی ندارد. خودِ کلاس هم با یک بار نوشتن برای هر نوعی کار می‌کند.

۱۲۴۸ **گزینه‌ی ۴** متوسط

قید `T extends { length: number }` یعنی هر آرگومانی باید یک `length` عددی داشته باشد. رشته، آرایه و حتی شیء `{ length: 7 }` این شرط را دارند، ولی عدد `42` ندارد؛ پس خط ۸ خطای **TS2345** (`Argument of type 'number' is not assignable to parameter of type '{ length: number; }'`) می‌دهد. خط ۲ هم دقیقاً به لطف همین قید مجاز است.

۱۲۴۹ **گزینه‌ی ۱** متوسط

`length` رشته‌ی `"hello"` برابر ۵ و `length` آرایه‌ی سه‌عضوی ۳ است. چون خروجی تابع از نوع `T` است، `r` همان رشته‌ی `"hello"` است (و TypeScript هم می‌داند `string` است)، پس `toUpperCase()` بدون مشکل `HELLO` می‌دهد. گزینه‌ی ۲ فکر کرده نتیجه رشته نیست، گزینه‌ی ۳ خودِ مقدار را به‌جای `length` چاپ کرده و گزینه‌ی ۴ فکر کرده آرایه `length` ندارد.

۱۲۵۰ **گزینه‌ی ۴** متوسط

هر دو هر ورودی‌ای می‌پذیرند، ولی فرق اصلی در **خروجی** است: `f<T>` می‌گوید خروجی همان نوع ورودی است (مثلاً `string`)، ولی `any` هر اطلاعاتی را دور می‌ریزد؛ پس گزینه‌ی ۱ غلط است. آرگومان نوع معمولاً خودکار استنتاج می‌شود (گزینه‌ی ۲ غلط) و generics در زمان اجرا وجود ندارند که بررسی‌ای انجام دهند (گزینه‌ی ۳ غلط).

۱۲۵۱ **گزینه‌ی ۲** متوسط

در `Pair<string, number>`، `K = string` و `V = number` است؛ پس `value` باید عدد باشد و رشته‌ی `"30"` خطای **TS2322** (`Type 'string' is not assignable to type 'number'`) می‌دهد. در بقیه‌ی گزینه‌ها هر مقدار با پارامتر نوع متناظرش جور است؛ ترتیب آرگومان‌های نوع مهم است (`<number, string>` در گزینه‌ی ۳ یعنی کلید عدد و مقدار رشته) و دو پارامتر نوع می‌توانند یکسان هم باشند (گزینه‌ی ۴).

۱۲۵۲ **گزینه‌ی ۳** متوسط

در `Box<boolean>`، نوع `value` برابر `boolean` است؛ `"true"` (بین کوتیشن) یک **رشته** است نه مقدار بولی، پس خط ۶ خطای **TS2322** می‌دهد. آرگومان نوع می‌تواند هر نوعی باشد، حتی آرایه (`number[]` در خط ۵)، و `0` هم یک `number` معتبر است (خط ۷).

۱۲۵۳ **گزینه‌ی ۲** سخت

در تابع `size` هیچ قیدی روی `T` نیست، پس TypeScript نمی‌داند `x` چیست (ممکن است عدد باشد) و `x.length` خطای **TS2339** (`Property 'length' does not exist on type 'T'`) می‌دهد؛ راه‌حلش `<T extends { length: number }>` است. در `first` پارامتر از نوع `T[]` (آرایه) است و دسترسی `items[0]` همیشه مجاز است؛ خطوط ۷ و ۸ هم به‌ترتیب `number | undefined` و `string | undefined` می‌گیرند.

۱۲۵۴ **گزینه‌ی ۱** سخت

چون خروجی `any` است، TypeScript هیچ بررسی‌ای روی `y` نمی‌کند و `y.toFixed(2)` را قبول می‌کند. ولی `y` در زمان اجرا یک رشته است و رشته‌ها متد `toFixed` ندارند: `y.toFixed` برابر `undefined` است و صدا زدن آن `TypeError: y.toFixed is not a function` می‌دهد و برنامه همان‌جا متوقف می‌شود (پس `done` چاپ نمی‌شود). این دقیقاً همان خطری است که generics جلویش را در زمان کامپایل می‌گیرند.

۱۲۵۵ **گزینه‌ی ۳** سخت

نوع `x` (خروجی generic) رشته است (دقیقاً `"hi"`)، پس `toUpperCase` مجاز است ولی `x.toFixed(2)` در خط ۱۱ خطای **TS2551** (`Property 'toFixed' does not exist on type '"hi"'`) می‌دهد. خط ۱۰ با این‌که در زمان اجرا می‌ترکد، **در کامپایل خطایی ندارد**، چون `y` از نوع `any` است و بررسی نوع برایش خاموش است. این تفاوت generic و `any` است.

۱۲۵۶ **گزینه‌ی ۴** سخت

هر دو پارامتر از **یک** نوع `T` هستند. در گزینه‌ی ۴، TypeScript نوع `T` را از آرگومان اول حدس می‌زند و بعد آرایه‌ی `[1, 2]` با آن جور نیست؛ خطای **TS2345** (`Argument of type 'number[]' is not assignable to parameter of type '"abc"'`) می‌دهد. در گزینه‌های ۱ و ۲ هر دو آرگومان هم‌نوع‌اند، و در گزینه‌ی ۳ آرگومان نوع صریحاً `string | number[]` است که هر دو آرگومان را می‌پذیرد و `length` هم دارد، پس خطایی نیست.

۱۲۵۷ **گزینه‌ی ۲** سخت

generics در TypeScript فقط در زمان کامپایل وجود دارند و در JavaScript خروجی **کاملاً پاک** می‌شوند (کد `identity` بعد از کامپایل هیچ `<T>`ی ندارد)؛ پس در زمان اجرا هیچ راهی برای فهمیدن `T` نیست و گزینه‌ی ۲ نادرست است. گزینه‌ی ۱ شباهت نحوی دو زبان، گزینه‌ی ۳ استنتاج آرگومان نوع (`identity(42)`) و گزینه‌ی ۴ کلاس‌های generic مثل `Stack<T>` را درست بیان می‌کنند.

۱۲۵۸ **گزینه‌ی ۱** سخت

وقتی آرگومان نوع **صریحاً** نوشته شود، استنتاج انجام نمی‌شود و `T` همان چیزی است که نوشته‌ایم: `number | string` (ترتیب اعضای union مهم نیست و TypeScript آن را `string | number` نمایش می‌دهد). پس `g` بعداً می‌تواند رشته هم بگیرد (`g = "five"`). اگر `<...>` را نمی‌نوشتیم، با `let` نوع `number` (گزینه‌ی ۲) و با `const` نوع `5` (گزینه‌ی ۳) استنتاج می‌شد؛ `any` هم هیچ‌وقت نتیجه‌ی generic نیست.

۳-۱۴ ماژول‌ها (Modules)

۱۲۵۹ **گزینه‌ی ۳** آسان

اسلاید ۲۳ می‌گوید «A module is simply a file that has imports or exports»؛ پس فقط فایلی که `import` یا `export` دارد ماژول است. فایل‌های گزینه‌های ۱ و ۲ هیچ import/export‌ی ندارند، پس script حساب می‌شوند (داشتن کلاس یا تابع چیزی را عوض نمی‌کند). `<script>` معمولی هم دقیقاً حالت قدیمی و غیرماژولی است؛ برای ماژول در مرورگر باید `type="module"` نوشت.

۱۲۶۰ **گزینه‌ی ۱** آسان

«پیش‌فرض» یعنی «چیز اصلی فایل»، پس فقط **یکی** می‌تواند باشد؛ دو `export default` در یک فایل هم در TypeScript خطا دارد و هم در Node هنگام بارگذاری `SyntaxError: Duplicate export of 'default'` می‌دهد. ولی داشتنش اجباری نیست: فایلی که فقط export نام‌دار دارد هم ماژول است (گزینه‌ی ۳ غلط). export نام‌دار است که هر تعدادی می‌تواند باشد (گزینه‌ی ۲)، و نوع چیزی که export می‌شود قاعده را عوض نمی‌کند (گزینه‌ی ۴).

۱۲۶۱ **گزینه‌ی ۴** آسان

طبق اسلاید ۲۳، هر فایل scope خصوصی دارد؛ چیزی بیرون دیده می‌شود که **export** شده باشد و فایل دیگر با **import** از آن استفاده می‌کند. فقط import کردن کافی نیست، چون چیزی که export نشده قابل import نیست (خطای TS2459؛ گزینه‌ی ۱). `var` در یک ماژول سراسری نمی‌شود (گزینه‌ی ۲)، و `public` یک access modifier کلاس است و برای تابع‌های سطح فایل معنایی ندارد (گزینه‌ی ۳).

۱۲۶۲ **گزینه‌ی ۲** آسان

اسلاید ۲۳: «TypeScript (and modern JavaScript) uses the ES module system»؛ یعنی ES Modules با `export` / `import`، که از ES2015 استاندارد خودِ زبان است. CommonJS (گزینه‌ی ۱) سیستم قدیمی‌تر خودِ Node.js است، نه استاندارد زبان. گزینه‌های ۳ و ۴ هم همان وضعیت قدیمی بدون ماژول را توصیف می‌کنند که ماژول‌ها برای حل آن آمدند.

۱۲۶۳ **گزینه‌ی ۳** آسان

`PI` و `double` export نام‌دارند و با آکولاد و همان اسم import شده‌اند؛ `log` export پیش‌فرض است و بدون آکولاد import شده؛ پس همه‌چیز درست است و `log(6.28)` متن `[LOG] 6.28` را چاپ می‌کند. گزینه‌ی ۱ فراموش کرده خروجی از تابع `log` (با پیشوند `[LOG]`) می‌گذرد، گزینه‌ی ۲ فکر کرده مقدار `PI` به فایل دیگر نمی‌رسد و گزینه‌ی ۴ فکر کرده ترکیب import نام‌دار و پیش‌فرض خطاست.

۱۲۶۴ **گزینه‌ی ۴** آسان

در Mixed exports، **اول** export پیش‌فرض بدون آکولاد و **بعد** exportهای نام‌دار داخل آکولاد می‌آیند:
`import Circle, { area } from ...` . گزینه‌ی ۱ پیش‌فرض را داخل آکولاد گذاشته و خطای TS2614 می‌گیرد (چون `Circle` export نام‌دار نیست). گزینه‌ی ۲ ترتیب را برعکس کرده و گزینه‌ی ۳ آکولاد export نام‌دار را جا انداخته؛ هر دو خطای نحوی دارند.

۱۲۶۵ **گزینه‌ی ۳** متوسط

`secret` در `math.ts` بدون `export` تعریف شده، پس در scope خصوصی همان فایل می‌ماند. import کردن آن در خط ۳ `main.ts` خطای **TS2459** (`Module '"./math.js"' declares 'secret' locally, but it is not exported`) می‌دهد. تعریف `secret` در `math.ts` (خط ۵) کاملاً مجاز است؛ متغیرِ خصوصی داشتن ایرادی ندارد. خط ۴ هم خطای جداگانه‌ای گزارش نمی‌کند، چون مشکل همان import است.

۱۲۶۶ **گزینه‌ی ۱** متوسط

`log` در `logger.ts` export **پیش‌فرض** است (اسم `log` فقط اسم داخلی تابع است)، پس باید بدون آکولاد import شود. `import { log }` در خط ۱ دنبال یک export نام‌دار به اسم `log` می‌گردد و خطای **TS2614** (`Did you mean to use 'import log from "./logger.js"' instead?`) می‌دهد. خطوط ۲ و ۳ درست‌اند: export پیش‌فرض را می‌توان با **هر** اسمی (`printer` یا `write`) import کرد.

۱۲۶۷ گزینه‌ی ۲ متوسط

`math.ts` فقط export نام‌دار دارد و هیچ `export default` ی ندارد؛ پس `import add from` (بدون آکولاد = import پیش‌فرض) در خط ۲ خطای **TS2613** (`Module has no default export. Did you mean to use 'import { add } ...'`) می‌دهد. خط ۱ همه‌ی exportها را در شیء `M` جمع می‌کند و خط ۳ یک export نام‌دار را با `as` تغییر نام داده است؛ هر دو درست‌اند.

۱۲۶۸ گزینه‌ی ۳ متوسط

`math.ts` اصلاً چیزی به اسم `subtract` ندارد (نه exportشده و نه خصوصی)، پس import آن در خط ۳ خطای **TS2305** (`Module '"./math.js"' has no exported member 'subtract'`) می‌دهد؛ همین خط `PI` را درست import می‌کند ولی یک اسم اشتباه کافی است. خط ۱ اسم `add` را با `as` به `plus` تغییر داده، خط ۲ همه‌ی exportها را در `M` جمع کرده و خط ۴ از هر سه درست استفاده می‌کند.

۱۲۶۹ گزینه‌ی ۴ متوسط

`MathUtils` شیئی است که فقط exportهای `math.mjs` را دارد: `MathUtils.PI` برابر `3.14` است و `sum` همان `add` با اسم جدید است (`9`). `useSecret()` چون داخل همان فایل تعریف شده به `secret` دسترسی دارد و `42` برمی‌گرداند، ولی `secret` export نشده و در `MathUtils` نیست؛ خواندن property ناموجود از یک شیء `undefined` می‌دهد، نه `ReferenceError` (گزینه‌ی ۲). گزینه‌ی ۱ فکر کرده چیزهای exportنشده هم دیده می‌شوند.

۱۲۷۰ گزینه‌ی ۲ متوسط

در Node.js فایل `.mjs` همیشه ESM است و فایل `.js` وقتی ESM است که فیلد `"type": "module"` داشته باشد. `require` / `module.exports` مال **CommonJS** هستند (گزینه‌ی ۱ غلط) و `.cjs` دقیقاً یعنی «این فایل CommonJS است» (گزینه‌ی ۳ غلط). در یک ES Module تابع `require` اصلاً تعریف نشده و صدا زدنش `ReferenceError` می‌دهد (گزینه‌ی ۴ غلط).

۱۲۷۱ گزینه‌ی ۱ متوسط

در CommonJS هر چیزی که در `module.exports` بگذاری صادر می‌شود و `require` همان شیء را برمی‌گرداند. `add(2, 3)` عدد `5` است (هر دو عدد هستند، پس الحاق رشته‌ای رخ نمی‌دهد)، `math.PI` برابر `3.14` است و `secret` چون در `module.exports` نیست، روی شیء وجود ندارد و `undefined` می‌دهد. CommonJS هم مثل ESM به هر فایل scope خصوصی می‌دهد، و در فایل `.cjs` تابع `require` کاملاً تعریف‌شده است (گزینه‌ی ۴ غلط).

۱۲۷۲ گزینه‌ی ۲ متوسط

`main.mjs` یک ES Module است و در ESM تابع `require` (که مال CommonJS است) وجود ندارد؛ پس خط ۲ خطای `ReferenceError: require is not defined in ES module scope` می‌دهد. import خط ۱ کاملاً درست است و خط ۳ اصلاً به اجرا نمی‌رسد. در ESM باید نوشت `import fs from "fs";`.

۱۲۷۳ گزینه‌ی ۱ متوسط

بعد از `export default` باید یک **عبارت** (مقدار، تابع یا کلاس) بیاید، نه تعریف متغیر با `const`؛ پس خط ۱ خطای **TS1109** (`Expression expected`) می‌دهد. راه درست: `const settings = { port: 3000 }; export default settings;` یا مستقیماً `export default { port: 3000 };`. خطوط ۲ و ۳ exportهای نام‌دار کاملاً درستی هستند؛ داشتن default و named با هم (mixed) هم مشکلی ندارد.

۱۲۷۴ گزینه‌ی ۴ سخت

سه قاعده: (۱) importها **قبل از** کدِ خودِ فایل اجرا می‌شوند، پس با این‌که `console.log("main starts")` خط اول است، سوم چاپ می‌شود (گزینه‌ی ۱ غلط). (۲) `counter.mjs` با وجود دو بار import شدن **فقط یک بار** اجرا می‌شود و همه یک نسخه‌ی مشترک از آن را می‌بینند (گزینه‌ی ۲ غلط). (۳) importها **زنده**‌اند: `a.mjs` یک بار و `main.mjs` یک بار `increment` را صدا زده‌اند و `count` در `main.mjs` مقدار به‌روز `2` را نشان می‌دهد، نه یک کپی قدیمی (گزینه‌ی ۳ غلط).

۱۲۷۵ **گزینه‌ی ۳** سخت

`increment()` خودِ متغیر `count` را در `counter.mjs` زیاد می‌کند و چون import زنده است، `main.mjs` مقدار `1` را می‌بیند (گزینه‌های ۱ و ۴ فکر کرده‌اند import یک کپی ثابت است). ولی import از دید فایلِ واردکننده **فقط‌خواندنی** است، مثل یک `const`: `count = 5` خطای `TypeError: Assignment to constant variable.` می‌دهد و خط آخر اجرا نمی‌شود (گزینه‌ی ۲ غلط). در TypeScript همین خط خطای TS2632 دارد.

۱۲۷۶ **گزینه‌ی ۱** سخت

Node قبل از اجرای **هر** خطی، همه‌ی importها را به exportها وصل می‌کند (linking). چون `counter.mjs` چیزی به اسم `decrement` export نکرده، همان مرحله با `SyntaxError: The requested module './counter.mjs' does not provide an export named 'decrement'` شکست می‌خورد و حتی `start` هم چاپ نمی‌شود (گزینه‌های ۲ و ۳ غلط). import اشتباه بی‌صدا نادیده گرفته هم نمی‌شود (گزینه‌ی ۴ غلط).

۱۲۷۷ **گزینه‌ی ۴** سخت

هر فایل حداکثر **یک** export پیش‌فرض دارد. وقتی Node فایل `logger.mjs` را بارگذاری می‌کند، به `export default` دوم (خط ۴) که می‌رسد خطای `SyntaxError: Duplicate export of 'default'` می‌دهد؛ مشکل در خودِ `logger.mjs` است، نه در import `main.mjs` (گزینه‌های ۱ و ۲). خط ۱ `logger.mjs` اولین و تنها export پیش‌فرض مجاز است (گزینه‌ی ۳). چون فایل اصلاً بارگذاری نمی‌شود، `hi` هم چاپ نمی‌شود.

۱۲۷۸ **گزینه‌ی ۳** سخت

طبق اسلاید ۲۳ «ماژول فایلی است که import یا export دارد»؛ این دو فایل هیچ‌کدام را ندارند، پس **script** هستند و متغیرهای سطح بالایشان در یک scope مشترک قرار می‌گیرند. دو `let` با یک اسم در یک scope مجاز نیست و `tsc` روی هر دو فایل خطای **TS2451** (`Cannot redeclare block-scoped variable 'counter'`) می‌دهد. scope خصوصی (گزینه‌ی ۱) فقط وقتی برقرار است که فایل‌ها ماژول باشند، مثلاً با افزودن `export {};`. گزینه‌های ۲ و ۴ هم غلط‌اند چون خطا همان موقع کامپایل گرفته می‌شود و `let` اجازه‌ی تعریف دوباره نمی‌دهد.

۱۲۷۹ **گزینه‌ی ۴** سخت

`import * as M` فقط exportها را در شیء `M` جمع می‌کند. `secret` export نشده، پس `M.secret` در خط ۵ خطای **TS2339** (`Property 'secret' does not exist on type 'typeof import(...)'`) می‌دهد. `M.useSecret()` در خط ۴ مجاز است، چون خودِ تابع export شده و داخل فایل خودش به `secret` دسترسی دارد. تعریف `secret` در `math.ts` هم خطا نیست؛ فقط خصوصی است.

۱۲۸۰ **گزینه‌ی ۲** سخت

اسکریپت‌های معمولی ماژول نیستند و همه یک scope سراسری مشترک دارند. اسکریپت دوم دوباره `let counter` تعریف می‌کند و **کل آن اسکریپت** با `SyntaxError: Identifier 'counter' has already been declared` اجرا نمی‌شود. خطای یک اسکریپت بقیه را متوقف نمی‌کند (گزینه‌ی ۴ غلط)، و اسکریپت سوم `counter` سراسری اسکریپت اول را می‌بیند: `third 1`. گزینه‌های ۱ و ۳ فرض کرده‌اند هر اسکریپت scope خصوصی دارد، که فقط برای ماژول‌ها (مثلاً `<script type="module">`) درست است.

۳-۱۵ بهترین شیوه‌ها (Best Practices) در TypeScript

۱۲۸۱ **گزینه‌ی ۱** آسان

interface فقط «شکل» را تعریف می‌کند: اسم اعضا و نوعشان. هر کلاس (با `implements`) یا شیئی که این شکل را داشته باشد، قرارداد را رعایت کرده است. interface هیچ پیاده‌سازی‌ای ندارد که به ارث برسد (گزینه‌های ۲ و ۴) و چون در JavaScript خروجی کاملاً پاک می‌شود، هیچ بررسی‌ای در زمان اجرا انجام نمی‌دهد (گزینه‌ی ۳).

۱۲۸۲ **گزینه‌ی ۳** آسان

خودرو یک موتور **دارد** ولی خودش موتور **نیست**؛ پس `Car` باید یک شیء `Engine` را در خودش نگه دارد (composition)، نه این‌که از آن ارث ببرد. سه جفت دیگر رابطه‌ی «is-a» دارند (دانشجو یک انسان است، سگ یک حیوان است، مربع یک شکل است) که همان جایی است که وراثت معنا دارد.

۱۲۸۳ **گزینه‌ی ۲** آسان

SRP درباره‌ی **مسئولیت** است: یک کلاس باید یک کار مشخص انجام دهد (مثلاً فقط ذخیره در پایگاه داده)، پس فقط یک دلیل برای تغییر دارد. یک کلاس با یک مسئولیت می‌تواند چند متد داشته باشد (گزینه‌ی ۱). ساختن فقط یک شیء از کلاس (گزینه‌ی ۳) و محدودیت تک‌والدی بودن `extends` (گزینه‌ی ۴) مفاهیم دیگری هستند و ربطی به SRP ندارند.

۱۲۸۴ **گزینه‌ی ۴** آسان

پنهان کردن داده‌ی داخلی با `private` و دادن یک راه کنترل‌شده برای تغییر آن (`deposit`)، تعریفِ **encapsulation** است. `private` اتفاقاً جلوی دسترسی کلاس‌های فرزند را هم می‌گیرد (گزینه‌ی ۱ غلط)، ربطی به نوع داده و generics ندارد (گزینه‌ی ۲) و SRP درباره‌ی تعداد مسئولیت‌هاست، نه تعداد propertyها (گزینه‌ی ۳).

۱۲۸۵ **گزینه‌ی ۴** متوسط

`balance` `private` فقط داخل خودِ کلاس قابل تغییر است؛ پس خط ۹ (داخل `deposit`) مجاز است ولی خط ۱۸ از بیرون کلاس است و خطای **TS2341** می‌دهد. این همان فایده‌ی encapsulation است: تنها راه تغییر موجودی، متد `deposit` با بررسی مبلغ است. خواندن `owner` `readonly` در خط ۱۷ و صدا زدن متد `public` در خط ۱۶ هم کاملاً مجازند.

۱۲۸۶ **گزینه‌ی ۳** متوسط

روی آرایه‌ی `readonly` فقط متدهایی وجود دارند که آرایه را تغییر نمی‌دهند؛ `concat` آرایه‌ی **جدیدی** برمی‌گرداند و `days` دست‌نخورده می‌ماند، پس مجاز است. `push` و `sort` آرایه را درجا تغییر می‌دهند و روی `readonly string[]` اصلاً وجود ندارند (خطای TS2339)، و نوشتن در خانه‌ی آرایه (`days[0] = ...`) هم خطای TS2542 می‌دهد.

۱۲۸۷ **گزینه‌ی ۱** متوسط

`UserManager` چهار مسئولیت دارد و با هر تغییر در ایمیل، پایگاه داده یا ظاهر صفحه باید عوض شود؛ این نقض **SRP** است. راه‌حل، شکستن آن به کلاس‌های کوچکی است که هر کدام یک مسئولیت دارند. `private` کردن متدها (گزینه‌ی ۲) فقط آن‌ها را غیرقابل‌استفاده می‌کند و مسئولیت‌ها را جدا نمی‌کند، وراثت (گزینه‌ی ۳) کلاس را کوچک‌تر نمی‌کند و فقط وابستگی اضافه می‌کند، و `readonly` (گزینه‌ی ۴) ربطی به تعداد مسئولیت‌ها ندارد.

۱۲۸۸ **گزینه‌ی ۴** متوسط

«Leverage generics for reusable, type-safe code»: با `Stack<T>` کد فقط یک بار نوشته می‌شود و بررسی نوع هم حفظ می‌شود (`push("x")` روی `Stack<number>` خطای کامپایل دارد). گزینه‌ی ۱ کد تکراری می‌سازد، گزینه‌ی ۲ با `any` بررسی نوع را از دست می‌دهد، و گزینه‌ی ۳ وراثت بی‌معنی است (پشته‌ی رشته «یک» پشته‌ی عدد نیست) و نوع‌ها را هم با هم ناسازگار می‌کند.

۱۲۸۹ **گزینه‌ی ۲** متوسط

`Car` یک شیء `Engine` را **در خودش دارد** (composition) و `drive` کار روشن کردن را به آن می‌سپارد؛ پس اول `engine started` و بعد `driving` چاپ می‌شود. ولی `Car` از `Engine` ارث نبرده، پس `car instanceof Engine` برابر `false` است (رابطه has-a است، نه is-a). گزینه‌ی ۱ ترکیب را با وراثت اشتباه گرفته، گزینه‌ی ۳ فراموش کرده `this.engine.start()` هم چاپ می‌شود، و چون `engine` کنار اعلانش مقدار گرفته خطایی (گزینه‌ی ۴) رخ نمی‌دهد.

۱۲۹۰ **گزینه‌ی ۳** متوسط

با ترکیب، `Car` فقط متدهایی را دارد که خودش تعریف کرده (`drive`)؛ متدهای `Engine` به `Car` منتقل نمی‌شوند. پس `car.start()` در خط ۱۴ خطای **TS2339** (`Property 'start' does not exist on type 'Car'`) می‌دهد. خط ۹ مجاز است چون `engine` داخل کلاس از شیء خودش استفاده می‌کند. این دقیقاً فایده‌ی ترکیب است: فقط چیزهایی بیرون دیده می‌شوند که لازم است (اگر `Car` از `Engine` ارث می‌برد، `car.start()` هم مجاز می‌شد).

۱۲۹۱ **گزینه‌ی ۴** متوسط

`checkout` هر چیزی را که قرارداد `PaymentMethod` را رعایت کند می‌پذیرد: هم شیء کلاس `CardPayment` (خط ۱۲) و هم یک شیء ساده که متد `pay` با همان نوع دارد (خط ۱۳)، حتی بدون کلاس. اما شیء خط ۱۴ به‌جای متد `pay` دارد `charge` و قرارداد را نقض می‌کند؛ خطای **TS2353** (`'charge' does not exist in type 'PaymentMethod'`) می‌گیرد. خط ۴ هم پیاده‌سازی درستِ قرارداد است.

۱۲۹۲ **گزینه‌ی ۳** سخت

`readonly` **سطحی** است: فقط جلوی مقداردهی دوباره‌ی خودِ property را می‌گیرد؛ پس `this.items = []` در خط ۱۰ (که بیرون از constructor است) خطای **TS2540** می‌دهد. اما خطوط ۴ و ۷ property را عوض نمی‌کنند، بلکه **محتوای** همان آرایه را تغییر می‌دهند، که مجاز است. اگر بخواهیم محتوا هم قفل شود، باید نوع را `readonly string[]` تعریف کنیم.

۱۲۹۳ **گزینه‌ی ۱** سخت

قید `T extends Entity` یعنی آرگومان نوع باید حداقل یک `id` عددی داشته باشد. `string` چنین شکلی ندارد و خطای **TS2344** (`Type 'string' does not satisfy the constraint 'Entity'`) می‌دهد. `Product` (که `Entity` را extend کرده)، خودِ `Entity` و حتی یک نوع ناشناس که `id: number` دارد (گزینه‌ی ۴) همگی قید را رعایت می‌کنند. ترکیب generic + interface + `private` همان چیزی است که اسلاید ۲۴ توصیه می‌کند.

۱۲۹۴ **گزینه‌ی ۲** سخت

اسلاید ۲۴ می‌گوید «Use inheritance sparingly; prefer composition»؛ زنجیره‌ی عمیق وراثت فقط برای استفاده‌ی دوباره از چند متد، کلاس‌ها را محکم به هم می‌چسباند و فهم کد را سخت می‌کند؛ بهتر است آن متدها در یک کلاس کمکی قرار بگیرند و با ترکیب استفاده شوند. گزینه‌ی ۱ همان «interface به‌عنوان قرارداد»، گزینه‌ی ۳ همان «ترجیح `readonly` برای تغییرناپذیری» و گزینه‌ی ۴ همان «generics برای کد reusable» است.

۱۲۹۵ **گزینه‌ی ۱** سخت

پشته باید فقط از بالا عضو بگیرد و پس بدهد؛ ولی با `extends Array` تمام رابط آرایه (مثلاً `splice(0, 1)` یا `unshift`) هم در دسترس است و قانون پشته را می‌شکند. با **ترکیب** (`private items: T[] = []` و فقط متدهای `push` / `pop`) این مشکل پیش نمی‌آید. گزینه‌ی ۲ غلط است چون این کد واقعاً کامپایل می‌شود، گزینه‌ی ۳ غلط است چون `Array<T>` خودش generic است، و گزینه‌ی ۴ برعکس است: `push` و `pop` دقیقاً از `Array` به ارث رسیده‌اند.